

RTI Connex Micro C++ API  
Version 4.2.0

---

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>1 RTI Connex Micro C++ API</b>                               | <b>1</b>  |
| <b>2 Topic Index</b>                                            | <b>1</b>  |
| 2.1 Reference Manual . . . . .                                  | 1         |
| <b>3 Hierarchical Index</b>                                     | <b>7</b>  |
| 3.1 Class Hierarchy . . . . .                                   | 7         |
| <b>4 Class Index</b>                                            | <b>13</b> |
| 4.1 Class List . . . . .                                        | 13        |
| <b>5 File Index</b>                                             | <b>23</b> |
| 5.1 File List . . . . .                                         | 23        |
| <b>6 Topic Documentation</b>                                    | <b>24</b> |
| 6.1 Documentation Guide . . . . .                               | 24        |
| 6.1.1 Unsupported Features . . . . .                            | 24        |
| 6.1.2 API Naming Conventions . . . . .                          | 25        |
| 6.1.3 API Documentation Terms . . . . .                         | 25        |
| 6.1.4 Stereotypes . . . . .                                     | 25        |
| 6.2 DDS API . . . . .                                           | 26        |
| 6.2.1 Detailed Description . . . . .                            | 27        |
| 6.2.2 Overview . . . . .                                        | 27        |
| 6.2.3 Conceptual Model . . . . .                                | 28        |
| 6.2.4 Modules . . . . .                                         | 29        |
| 6.2.5 Content Filter Plugin . . . . .                           | 29        |
| 6.2.6 Domain Module . . . . .                                   | 30        |
| 6.2.7 Topic . . . . .                                           | 46        |
| 6.2.8 Publication Module . . . . .                              | 57        |
| 6.2.9 Subscription Module . . . . .                             | 67        |
| 6.2.10 Infrastructure Module . . . . .                          | 79        |
| 6.2.11 General Utilities and Compliance Configuration . . . . . | 139       |
| 6.3 DPSE Static Discovery API . . . . .                         | 140       |
| 6.3.1 Detailed Description . . . . .                            | 141       |
| 6.3.2 Macro Definition Documentation . . . . .                  | 141       |
| 6.4 DPDE Dynamic Discovery API . . . . .                        | 141       |
| 6.4.1 Detailed Description . . . . .                            | 141       |
| 6.4.2 Macro Definition Documentation . . . . .                  | 142       |
| 6.5 CDR Stream API . . . . .                                    | 142       |
| 6.5.1 Detailed Description . . . . .                            | 142       |
| 6.5.2 Function Documentation . . . . .                          | 142       |

|                                                 |     |
|-------------------------------------------------|-----|
| 6.6 RT Run-Time API . . . . .                   | 145 |
| 6.6.1 Detailed Description . . . . .            | 145 |
| 6.7 RHSM Reader History API . . . . .           | 145 |
| 6.7.1 Detailed Description . . . . .            | 145 |
| 6.8 WHSM Writer History API . . . . .           | 145 |
| 6.8.1 Detailed Description . . . . .            | 145 |
| 6.9 NETIO API . . . . .                         | 146 |
| 6.9.1 Detailed Description . . . . .            | 146 |
| 6.9.2 NETIO Packet API . . . . .                | 146 |
| 6.9.3 NETIO Interface API . . . . .             | 147 |
| 6.9.4 NETIO Address API . . . . .               | 149 |
| 6.10 UDP Transport API . . . . .                | 151 |
| 6.10.1 Detailed Description . . . . .           | 152 |
| 6.10.2 Macro Definition Documentation . . . . . | 152 |
| 6.10.3 Function Documentation . . . . .         | 152 |
| 6.11 UDP Transform API . . . . .                | 154 |
| 6.12 RTPS Transport API . . . . .               | 154 |
| 6.12.1 Detailed Description . . . . .           | 155 |
| 6.12.2 Typedef Documentation . . . . .          | 155 |
| 6.12.3 Enumeration Type Documentation . . . . . | 156 |
| 6.12.4 Variable Documentation . . . . .         | 157 |
| 6.13 Shared Memory Transport API . . . . .      | 158 |
| 6.13.1 Detailed Description . . . . .           | 159 |
| 6.13.2 NETIO Shared Memory API . . . . .        | 159 |
| 6.14 NETIO DGRAM API . . . . .                  | 178 |
| 6.14.1 Detailed Description . . . . .           | 179 |
| 6.14.2 Macro Definition Documentation . . . . . | 180 |
| 6.14.3 Typedef Documentation . . . . .          | 182 |
| 6.14.4 Function Documentation . . . . .         | 183 |
| 6.14.5 Variable Documentation . . . . .         | 184 |
| 6.15 Zero Copy v2 API . . . . .                 | 185 |
| 6.15.1 Detailed Description . . . . .           | 186 |
| 6.15.2 Macro Definition Documentation . . . . . | 186 |
| 6.15.3 Typedef Documentation . . . . .          | 186 |
| 6.15.4 Notification Interface . . . . .         | 188 |
| 6.16 OS API . . . . .                           | 189 |
| 6.16.1 Detailed Description . . . . .           | 190 |
| 6.16.2 OSAPI Heap . . . . .                     | 190 |
| 6.16.3 OSAPI Mutex . . . . .                    | 193 |

|                                                     |     |
|-----------------------------------------------------|-----|
| 6.16.4 OSAPI Process                                | 195 |
| 6.16.5 OSAPI Semaphore                              | 196 |
| 6.16.6 OSAPI Memory API                             | 198 |
| 6.16.7 OSAPI String API                             | 198 |
| 6.16.8 OSAPI System API                             | 198 |
| 6.16.9 OSAPI Thread API                             | 202 |
| 6.16.10 OSAPI Time                                  | 205 |
| 6.16.11 OSAPI Timer                                 | 205 |
| 6.16.12 OSAPI Types                                 | 206 |
| 6.16.13 OSAPI_SharedMemoryMonitor                   | 209 |
| 6.16.14 OSAPI_SharedMemorySegment                   | 212 |
| 6.17 Log API                                        | 219 |
| 6.17.1 Detailed Description                         | 220 |
| 6.17.2 Typedef Documentation                        | 220 |
| 6.17.3 Enumeration Type Documentation               | 221 |
| 6.17.4 Function Documentation                       | 221 |
| 6.18 Application Generation API                     | 224 |
| 6.18.1 Detailed Description                         | 224 |
| 6.18.2 Function Documentation                       | 225 |
| 6.19 Lightweight Security API                       | 226 |
| 6.19.1 Detailed Description                         | 226 |
| 6.19.2 Macro Definition Documentation               | 226 |
| 6.19.3 Variable Documentation                       | 227 |
| 6.20 Resource Limits                                | 227 |
| 6.20.1 Detailed Description                         | 227 |
| 6.20.2 Introduction                                 | 227 |
| 6.20.3 Dynamic Memory Allocation                    | 227 |
| 6.20.4 DomainParticipantFactoryQos                  | 228 |
| 6.20.5 DomainParticipantQos                         | 228 |
| 6.20.6 DataReaderQos                                | 234 |
| 6.20.7 DataWriterQos                                | 239 |
| 6.20.8 OSAPI                                        | 241 |
| 6.20.9 UDP Transport                                | 241 |
| 6.20.10 Dynamic Participant Static Endpoint (DPSE)  | 242 |
| 6.20.11 Dynamic Participant Dynamic Endpoint (DPDE) | 243 |
| 6.21 Log Codes                                      | 245 |
| 6.21.1 Detailed Description                         | 245 |
| 6.21.2 OSAPI                                        | 245 |
| 6.21.3 REDA                                         | 264 |

|                                                           |            |
|-----------------------------------------------------------|------------|
| 6.21.4 DB                                                 | 270        |
| 6.21.5 RT                                                 | 276        |
| 6.21.6 CDR                                                | 278        |
| 6.21.7 NETIO                                              | 279        |
| 6.21.8 RTPS                                               | 306        |
| 6.21.9 DDS_C                                              | 336        |
| 6.21.10 WHSQ                                              | 448        |
| 6.21.11 RH                                                | 450        |
| 6.21.12 WH                                                | 455        |
| 6.21.13 DPSE                                              | 457        |
| 6.21.14 DPDE                                              | 470        |
| 6.21.15 APPGEN                                            | 482        |
| 6.21.16 DDS_FILTER                                        | 485        |
| <b>7 Class Documentation</b>                              | <b>495</b> |
| 7.1 DDS_BuiltinTopicKey_t Struct Reference                | 495        |
| 7.1.1 Detailed Description                                | 496        |
| 7.1.2 Member Data Documentation                           | 496        |
| 7.2 DDS_ChecksumProperty Struct Reference                 | 496        |
| 7.2.1 Detailed Description                                | 497        |
| 7.2.2 Member Data Documentation                           | 497        |
| 7.3 DDS_ChecksumProperty_t Struct Reference               | 497        |
| 7.3.1 Detailed Description                                | 497        |
| 7.3.2 Member Data Documentation                           | 498        |
| 7.4 DDS_ContentFilterProperty Struct Reference            | 498        |
| 7.4.1 Detailed Description                                | 498        |
| 7.4.2 Member Data Documentation                           | 499        |
| 7.5 DDS_ContentFilterQosPolicy Struct Reference           | 500        |
| 7.5.1 Detailed Description                                | 500        |
| 7.5.2 Usage                                               | 501        |
| 7.5.3 Member Data Documentation                           | 501        |
| 7.6 DDS_DataReaderInstanceReplacedStatus Struct Reference | 502        |
| 7.6.1 Detailed Description                                | 503        |
| 7.6.2 Member Data Documentation                           | 503        |
| 7.7 DDS_DataReaderProtocolQosPolicy Struct Reference      | 504        |
| 7.7.1 Detailed Description                                | 504        |
| 7.7.2 Member Data Documentation                           | 504        |
| 7.8 DDS_DataReaderQos Struct Reference                    | 505        |
| 7.8.1 Detailed Description                                | 506        |

---

|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| 7.8.2 Member Data Documentation . . . . .                                  | 506 |
| 7.9 DDS_DataReaderResourceLimitsQosPolicy Struct Reference . . . . .       | 508 |
| 7.9.1 Detailed Description . . . . .                                       | 509 |
| 7.9.2 Member Data Documentation . . . . .                                  | 509 |
| 7.10 DDS_DataRepresentationQosPolicy Struct Reference . . . . .            | 512 |
| 7.10.1 Detailed Description . . . . .                                      | 512 |
| 7.11 DDS_DataWriterProtocolQosPolicy Struct Reference . . . . .            | 513 |
| 7.11.1 Detailed Description . . . . .                                      | 513 |
| 7.11.2 Member Data Documentation . . . . .                                 | 513 |
| 7.12 DDS_DataWriterQos Struct Reference . . . . .                          | 514 |
| 7.12.1 Detailed Description . . . . .                                      | 515 |
| 7.12.2 Member Data Documentation . . . . .                                 | 515 |
| 7.13 DDS_DataWriterResourceLimitsQosPolicy Struct Reference . . . . .      | 518 |
| 7.13.1 Detailed Description . . . . .                                      | 518 |
| 7.13.2 Member Data Documentation . . . . .                                 | 519 |
| 7.14 DDS_DataWriterShmemRefTransferModeSettings Struct Reference . . . . . | 521 |
| 7.14.1 Detailed Description . . . . .                                      | 521 |
| 7.14.2 Member Data Documentation . . . . .                                 | 521 |
| 7.15 DDS_DataWriterTransferModeQosPolicy Struct Reference . . . . .        | 521 |
| 7.15.1 Detailed Description . . . . .                                      | 522 |
| 7.15.2 Member Data Documentation . . . . .                                 | 522 |
| 7.16 DDS_DeadlineQosPolicy Struct Reference . . . . .                      | 522 |
| 7.16.1 Detailed Description . . . . .                                      | 523 |
| 7.16.2 Usage . . . . .                                                     | 523 |
| 7.16.3 Compatibility . . . . .                                             | 523 |
| 7.16.4 Member Data Documentation . . . . .                                 | 523 |
| 7.17 DDS_DestinationOrderQosPolicy Struct Reference . . . . .              | 524 |
| 7.17.1 Detailed Description . . . . .                                      | 524 |
| 7.17.2 Member Data Documentation . . . . .                                 | 524 |
| 7.18 DDS_DiscoveryComponent Struct Reference . . . . .                     | 525 |
| 7.18.1 Detailed Description . . . . .                                      | 525 |
| 7.18.2 Member Data Documentation . . . . .                                 | 525 |
| 7.19 DDS_DiscoveryQosPolicy Struct Reference . . . . .                     | 525 |
| 7.19.1 Detailed Description . . . . .                                      | 526 |
| 7.19.2 Usage . . . . .                                                     | 526 |
| 7.19.3 Member Data Documentation . . . . .                                 | 526 |
| 7.20 DDS_DomainParticipantFactoryQos Struct Reference . . . . .            | 528 |
| 7.20.1 Detailed Description . . . . .                                      | 528 |
| 7.20.2 Member Data Documentation . . . . .                                 | 529 |

|                                                                    |     |
|--------------------------------------------------------------------|-----|
| 7.21 DDS_DomainParticipantQos Struct Reference                     | 529 |
| 7.21.1 Detailed Description                                        | 530 |
| 7.21.2 Member Data Documentation                                   | 530 |
| 7.22 DDS_DomainParticipantResourceLimitsQosPolicy Struct Reference | 532 |
| 7.22.1 Detailed Description                                        | 533 |
| 7.22.2 Member Data Documentation                                   | 534 |
| 7.23 DDS_DurabilityQosPolicy Struct Reference                      | 540 |
| 7.23.1 Detailed Description                                        | 540 |
| 7.23.2 Member Data Documentation                                   | 541 |
| 7.24 DDS_Duration_t Struct Reference                               | 541 |
| 7.24.1 Detailed Description                                        | 541 |
| 7.24.2 Member Data Documentation                                   | 541 |
| 7.25 DDS_EntityFactoryQosPolicy Struct Reference                   | 542 |
| 7.25.1 Detailed Description                                        | 542 |
| 7.25.2 Usage                                                       | 542 |
| 7.25.3 Member Data Documentation                                   | 543 |
| 7.26 DDS_EntityNameQosPolicy Struct Reference                      | 543 |
| 7.26.1 Detailed Description                                        | 543 |
| 7.26.2 Usage                                                       | 544 |
| 7.26.3 Member Data Documentation                                   | 544 |
| 7.27 DDS_FilterQosPolicy Struct Reference                          | 544 |
| 7.27.1 Detailed Description                                        | 545 |
| 7.27.2 Usage                                                       | 545 |
| 7.27.3 Member Data Documentation                                   | 545 |
| 7.28 DDS_FilterResourceLimits Struct Reference                     | 546 |
| 7.28.1 Detailed Description                                        | 547 |
| 7.28.2 Member Data Documentation                                   | 547 |
| 7.29 DDS_FlowControllerProperty_t Struct Reference                 | 549 |
| 7.29.1 Detailed Description                                        | 549 |
| 7.29.2 Member Data Documentation                                   | 550 |
| 7.30 DDS_FlowControllerTokenBucketProperty_t Struct Reference      | 550 |
| 7.30.1 Detailed Description                                        | 551 |
| 7.30.2 Member Data Documentation                                   | 551 |
| 7.31 DDS_GroupDataQosPolicy Struct Reference                       | 553 |
| 7.31.1 Detailed Description                                        | 553 |
| 7.31.2 Usage                                                       | 554 |
| 7.31.3 Member Data Documentation                                   | 554 |
| 7.32 DDS_GUID_t Struct Reference                                   | 554 |
| 7.32.1 Detailed Description                                        | 554 |

---

|                                                                  |     |
|------------------------------------------------------------------|-----|
| 7.32.2 Member Data Documentation . . . . .                       | 555 |
| 7.33 DDS_HistoryQosPolicy Struct Reference . . . . .             | 555 |
| 7.33.1 Detailed Description . . . . .                            | 555 |
| 7.33.2 Usage . . . . .                                           | 556 |
| 7.33.3 Consistency . . . . .                                     | 556 |
| 7.33.4 Member Data Documentation . . . . .                       | 556 |
| 7.34 DDS_InconsistentTopicStatus Struct Reference . . . . .      | 557 |
| 7.34.1 Detailed Description . . . . .                            | 557 |
| 7.35 DDS_InstanceHandleSeq Struct Reference . . . . .            | 557 |
| 7.35.1 Detailed Description . . . . .                            | 557 |
| 7.36 DDS_LatencyBudgetQosPolicy Struct Reference . . . . .       | 557 |
| 7.36.1 Detailed Description . . . . .                            | 558 |
| 7.36.2 Usage . . . . .                                           | 558 |
| 7.36.3 Compatibility . . . . .                                   | 558 |
| 7.36.4 Member Data Documentation . . . . .                       | 559 |
| 7.37 DDS_LivelinessChangedStatus Struct Reference . . . . .      | 559 |
| 7.37.1 Detailed Description . . . . .                            | 559 |
| 7.37.2 Member Data Documentation . . . . .                       | 559 |
| 7.38 DDS_LivelinessLostStatus Struct Reference . . . . .         | 560 |
| 7.38.1 Detailed Description . . . . .                            | 561 |
| 7.38.2 Member Data Documentation . . . . .                       | 561 |
| 7.39 DDS_LivelinessQosPolicy Struct Reference . . . . .          | 561 |
| 7.39.1 Detailed Description . . . . .                            | 562 |
| 7.39.2 Usage . . . . .                                           | 563 |
| 7.39.3 Compatibility . . . . .                                   | 563 |
| 7.39.4 Member Data Documentation . . . . .                       | 564 |
| 7.40 DDS_Locator_t Struct Reference . . . . .                    | 564 |
| 7.40.1 Detailed Description . . . . .                            | 564 |
| 7.40.2 Member Data Documentation . . . . .                       | 565 |
| 7.41 DDS_LocatorSeq Struct Reference . . . . .                   | 565 |
| 7.41.1 Detailed Description . . . . .                            | 565 |
| 7.42 DDS_OfferedDeadlineMissedStatus Struct Reference . . . . .  | 565 |
| 7.42.1 Detailed Description . . . . .                            | 566 |
| 7.42.2 Member Data Documentation . . . . .                       | 566 |
| 7.43 DDS_OfferedIncompatibleQosStatus Struct Reference . . . . . | 567 |
| 7.43.1 Detailed Description . . . . .                            | 567 |
| 7.43.2 Member Data Documentation . . . . .                       | 568 |
| 7.44 DDS_OwnershipQosPolicy Struct Reference . . . . .           | 568 |
| 7.44.1 Detailed Description . . . . .                            | 569 |



|                                                       |     |
|-------------------------------------------------------|-----|
| 7.44.2 Usage                                          | 569 |
| 7.44.3 Compatibility                                  | 570 |
| 7.44.4 Member Data Documentation                      | 572 |
| 7.45 DDS_OwnershipStrengthQosPolicy Struct Reference  | 573 |
| 7.45.1 Detailed Description                           | 573 |
| 7.45.2 Member Data Documentation                      | 573 |
| 7.46 DDS_ParticipantBuiltinTopicData Struct Reference | 574 |
| 7.46.1 Detailed Description                           | 574 |
| 7.46.2 Member Data Documentation                      | 575 |
| 7.47 DDS_PartitionQosPolicy Struct Reference          | 577 |
| 7.47.1 Detailed Description                           | 577 |
| 7.47.2 Member Data Documentation                      | 579 |
| 7.48 DDS_PresentationQosPolicy Struct Reference       | 579 |
| 7.48.1 Detailed Description                           | 580 |
| 7.48.2 Compatibility                                  | 582 |
| 7.48.3 Member Data Documentation                      | 582 |
| 7.49 DDS_ProductVersion Struct Reference              | 583 |
| 7.49.1 Detailed Description                           | 583 |
| 7.49.2 Member Data Documentation                      | 583 |
| 7.50 DDS_ProductVersion_t Struct Reference            | 584 |
| 7.50.1 Detailed Description                           | 584 |
| 7.50.2 Member Data Documentation                      | 585 |
| 7.51 DDS_Property_t Struct Reference                  | 585 |
| 7.51.1 Detailed Description                           | 586 |
| 7.51.2 Member Data Documentation                      | 586 |
| 7.52 DDS_PropertyQosPolicy Struct Reference           | 586 |
| 7.52.1 Detailed Description                           | 587 |
| 7.52.2 Member Data Documentation                      | 587 |
| 7.53 DDS_PropertySeq Struct Reference                 | 587 |
| 7.53.1 Detailed Description                           | 587 |
| 7.54 DDS_ProtocolVersion Struct Reference             | 588 |
| 7.54.1 Detailed Description                           | 588 |
| 7.54.2 Member Data Documentation                      | 588 |
| 7.55 DDS_PskServiceFactoryProperty Struct Reference   | 588 |
| 7.55.1 Detailed Description                           | 589 |
| 7.55.2 Member Data Documentation                      | 589 |
| 7.56 DDS_PublicationBuiltinTopicData Struct Reference | 589 |
| 7.56.1 Detailed Description                           | 590 |
| 7.56.2 Member Data Documentation                      | 591 |

---

|                                                               |     |
|---------------------------------------------------------------|-----|
| 7.57 DDS_PublicationMatchedStatus Struct Reference            | 594 |
| 7.57.1 Detailed Description                                   | 594 |
| 7.57.2 Member Data Documentation                              | 594 |
| 7.58 DDS_PublisherQos Struct Reference                        | 595 |
| 7.58.1 Detailed Description                                   | 595 |
| 7.58.2 Member Data Documentation                              | 596 |
| 7.59 DDS_PublishModeQosPolicy Struct Reference                | 596 |
| 7.59.1 Detailed Description                                   | 597 |
| 7.59.2 Usage                                                  | 597 |
| 7.59.3 Member Data Documentation                              | 598 |
| 7.60 DDS_ReliabilityQosPolicy Struct Reference                | 599 |
| 7.60.1 Detailed Description                                   | 599 |
| 7.60.2 Usage                                                  | 599 |
| 7.60.3 Compatibility                                          | 600 |
| 7.60.4 Member Data Documentation                              | 600 |
| 7.61 DDS_ReliableReaderActivityChangedStatus Struct Reference | 601 |
| 7.61.1 Detailed Description                                   | 601 |
| 7.61.2 Member Data Documentation                              | 601 |
| 7.62 DDS_ReliableSampleUnacknowledgedStatus Struct Reference  | 602 |
| 7.62.1 Detailed Description                                   | 603 |
| 7.62.2 Member Data Documentation                              | 603 |
| 7.63 DDS_RequestedDeadlineMissedStatus Struct Reference       | 603 |
| 7.63.1 Detailed Description                                   | 604 |
| 7.63.2 Member Data Documentation                              | 604 |
| 7.64 DDS_RequestedIncompatibleQosStatus Struct Reference      | 604 |
| 7.64.1 Detailed Description                                   | 605 |
| 7.64.2 Member Data Documentation                              | 605 |
| 7.65 DDS_ResourceLimitsQosPolicy Struct Reference             | 605 |
| 7.65.1 Detailed Description                                   | 606 |
| 7.65.2 Usage                                                  | 606 |
| 7.65.3 Consistency                                            | 607 |
| 7.65.4 Member Data Documentation                              | 607 |
| 7.66 DDS_RtpsReliableReaderProtocol_t Struct Reference        | 608 |
| 7.66.1 Detailed Description                                   | 608 |
| 7.66.2 Member Data Documentation                              | 608 |
| 7.67 DDS_RtpsReliableWriterProtocol_t Struct Reference        | 609 |
| 7.67.1 Detailed Description                                   | 609 |
| 7.67.2 Member Data Documentation                              | 609 |
| 7.68 DDS_RtpsWellKnownPorts_t Struct Reference                | 611 |

|                                                                   |     |
|-------------------------------------------------------------------|-----|
| 7.68.1 Detailed Description . . . . .                             | 611 |
| 7.68.2 Member Data Documentation . . . . .                        | 612 |
| 7.69 DDS_SampleIdentity_t Struct Reference . . . . .              | 615 |
| 7.69.1 Detailed Description . . . . .                             | 615 |
| 7.69.2 Member Data Documentation . . . . .                        | 615 |
| 7.70 DDS_SampleInfo Struct Reference . . . . .                    | 616 |
| 7.70.1 Detailed Description . . . . .                             | 616 |
| 7.70.2 Interpretation of the SampleInfo . . . . .                 | 616 |
| 7.70.3 Member Data Documentation . . . . .                        | 617 |
| 7.71 DDS_SampleInfoSeq Struct Reference . . . . .                 | 619 |
| 7.71.1 Detailed Description . . . . .                             | 619 |
| 7.72 DDS_SampleLostStatus Struct Reference . . . . .              | 619 |
| 7.72.1 Detailed Description . . . . .                             | 619 |
| 7.72.2 Member Data Documentation . . . . .                        | 620 |
| 7.73 DDS_SampleRejectedStatus Struct Reference . . . . .          | 620 |
| 7.73.1 Detailed Description . . . . .                             | 621 |
| 7.73.2 Member Data Documentation . . . . .                        | 621 |
| 7.74 DDS_SecTransform_Configuration Struct Reference . . . . .    | 621 |
| 7.74.1 Detailed Description . . . . .                             | 622 |
| 7.74.2 Member Data Documentation . . . . .                        | 622 |
| 7.75 DDS_SequenceNumber_t Struct Reference . . . . .              | 622 |
| 7.75.1 Detailed Description . . . . .                             | 623 |
| 7.75.2 Member Data Documentation . . . . .                        | 623 |
| 7.76 DDS_SubscriberQos Struct Reference . . . . .                 | 623 |
| 7.76.1 Detailed Description . . . . .                             | 623 |
| 7.76.2 Member Data Documentation . . . . .                        | 624 |
| 7.77 DDS_SubscriptionBuiltinTopicData Struct Reference . . . . .  | 624 |
| 7.77.1 Detailed Description . . . . .                             | 626 |
| 7.77.2 Member Data Documentation . . . . .                        | 626 |
| 7.78 DDS_SubscriptionMatchedStatus Struct Reference . . . . .     | 629 |
| 7.78.1 Detailed Description . . . . .                             | 630 |
| 7.78.2 Member Data Documentation . . . . .                        | 630 |
| 7.79 DDS_SystemResourceLimitsQosPolicy Struct Reference . . . . . | 631 |
| 7.79.1 Detailed Description . . . . .                             | 631 |
| 7.79.2 Member Data Documentation . . . . .                        | 631 |
| 7.80 DDS_Time_t Struct Reference . . . . .                        | 632 |
| 7.80.1 Detailed Description . . . . .                             | 632 |
| 7.80.2 Member Data Documentation . . . . .                        | 633 |
| 7.81 DDS_TopicDataQosPolicy Struct Reference . . . . .            | 633 |

---

|                                                 |     |
|-------------------------------------------------|-----|
| 7.81.1 Detailed Description                     | 633 |
| 7.81.2 Usage                                    | 634 |
| 7.81.3 Member Data Documentation                | 634 |
| 7.82 DDS_TopicQos Struct Reference              | 634 |
| 7.82.1 Detailed Description                     | 634 |
| 7.82.2 Member Data Documentation                | 634 |
| 7.83 DDS_TransportQosPolicy Struct Reference    | 635 |
| 7.83.1 Detailed Description                     | 635 |
| 7.83.2 Member Data Documentation                | 635 |
| 7.84 DDS_TypePluginI Struct Reference           | 636 |
| 7.84.1 Detailed Description                     | 636 |
| 7.85 DDS_UserDataQosPolicy Struct Reference     | 636 |
| 7.85.1 Detailed Description                     | 636 |
| 7.85.2 Usage                                    | 636 |
| 7.85.3 Member Data Documentation                | 637 |
| 7.86 DDS_UserTrafficQosPolicy Struct Reference  | 637 |
| 7.86.1 Detailed Description                     | 637 |
| 7.86.2 Member Data Documentation                | 637 |
| 7.87 DDS_VendorId Struct Reference              | 638 |
| 7.87.1 Detailed Description                     | 638 |
| 7.87.2 Member Data Documentation                | 638 |
| 7.88 DDS_VendorId_t Struct Reference            | 638 |
| 7.88.1 Detailed Description                     | 638 |
| 7.88.2 Member Data Documentation                | 639 |
| 7.89 DDS_WireProtocolQosPolicy Struct Reference | 639 |
| 7.89.1 Detailed Description                     | 639 |
| 7.89.2 Usage                                    | 640 |
| 7.89.3 Member Data Documentation                | 640 |
| 7.90 DDS_WriteParams_t Struct Reference         | 643 |
| 7.90.1 Detailed Description                     | 644 |
| 7.90.2 Member Data Documentation                | 644 |
| 7.91 DDSCondition Class Reference               | 644 |
| 7.91.1 Detailed Description                     | 644 |
| 7.92 DDSConditionSeq Class Reference            | 645 |
| 7.92.1 Detailed Description                     | 645 |
| 7.93 DDSDataReader Class Reference              | 645 |
| 7.93.1 Detailed Description                     | 647 |
| 7.93.2 Member Function Documentation            | 648 |
| 7.93.3 SEQUENCES USAGE IN TAKE AND READ         | 652 |

|                                                              |     |
|--------------------------------------------------------------|-----|
| 7.94 DDSDataReaderListener Class Reference . . . . .         | 662 |
| 7.94.1 Detailed Description . . . . .                        | 663 |
| 7.94.2 Member Function Documentation . . . . .               | 664 |
| 7.95 DDSDataWriter Class Reference . . . . .                 | 666 |
| 7.95.1 Detailed Description . . . . .                        | 668 |
| 7.95.2 Member Function Documentation . . . . .               | 669 |
| 7.96 DDSDataWriterListener Class Reference . . . . .         | 681 |
| 7.96.1 Detailed Description . . . . .                        | 681 |
| 7.96.2 Member Function Documentation . . . . .               | 682 |
| 7.97 DDSDomainEntity Class Reference . . . . .               | 685 |
| 7.97.1 Detailed Description . . . . .                        | 685 |
| 7.98 DDSDomainParticipant Class Reference . . . . .          | 685 |
| 7.98.1 Detailed Description . . . . .                        | 688 |
| 7.98.2 Member Function Documentation . . . . .               | 688 |
| 7.99 DDSDomainParticipantFactory Class Reference . . . . .   | 710 |
| 7.99.1 Detailed Description . . . . .                        | 711 |
| 7.99.2 Member Function Documentation . . . . .               | 711 |
| 7.100 DDSDomainParticipantListener Class Reference . . . . . | 718 |
| 7.100.1 Detailed Description . . . . .                       | 719 |
| 7.100.2 Member Function Documentation . . . . .              | 719 |
| 7.101 DDSEntity Class Reference . . . . .                    | 725 |
| 7.101.1 Detailed Description . . . . .                       | 725 |
| 7.101.2 Abstract operations . . . . .                        | 725 |
| 7.101.3 Member Function Documentation . . . . .              | 727 |
| 7.102 DDSFlowController Class Reference . . . . .            | 729 |
| 7.102.1 Detailed Description . . . . .                       | 729 |
| 7.102.2 Member Function Documentation . . . . .              | 729 |
| 7.103 DDSGuardCondition Class Reference . . . . .            | 731 |
| 7.103.1 Detailed Description . . . . .                       | 731 |
| 7.103.2 Member Function Documentation . . . . .              | 732 |
| 7.104 DDSListener Class Reference . . . . .                  | 732 |
| 7.104.1 Detailed Description . . . . .                       | 732 |
| 7.104.2 Access to Plain Communication Status . . . . .       | 733 |
| 7.104.3 Access to Read Communication Status . . . . .        | 733 |
| 7.105 DDSPropertyQosPolicyHelper Class Reference . . . . .   | 734 |
| 7.105.1 Detailed Description . . . . .                       | 735 |
| 7.105.2 Member Function Documentation . . . . .              | 735 |
| 7.106 DDSPublisher Class Reference . . . . .                 | 737 |
| 7.106.1 Detailed Description . . . . .                       | 738 |

|                                                               |     |
|---------------------------------------------------------------|-----|
| 7.106.2 Member Function Documentation . . . . .               | 739 |
| 7.107 DDSPublisherListener Class Reference . . . . .          | 745 |
| 7.107.1 Detailed Description . . . . .                        | 746 |
| 7.108 DDSStatusCondition Class Reference . . . . .            | 746 |
| 7.108.1 Detailed Description . . . . .                        | 747 |
| 7.108.2 Member Function Documentation . . . . .               | 747 |
| 7.109 DDSSubscriber Class Reference . . . . .                 | 748 |
| 7.109.1 Detailed Description . . . . .                        | 749 |
| 7.109.2 Member Function Documentation . . . . .               | 750 |
| 7.110 DDSSubscriberListener Class Reference . . . . .         | 757 |
| 7.110.1 Detailed Description . . . . .                        | 758 |
| 7.110.2 Member Function Documentation . . . . .               | 758 |
| 7.111 DDSTopic Class Reference . . . . .                      | 759 |
| 7.111.1 Detailed Description . . . . .                        | 760 |
| 7.111.2 Member Function Documentation . . . . .               | 760 |
| 7.112 DDSTopicDescription Class Reference . . . . .           | 764 |
| 7.112.1 Detailed Description . . . . .                        | 764 |
| 7.112.2 Member Function Documentation . . . . .               | 765 |
| 7.113 DDSTopicListener Class Reference . . . . .              | 766 |
| 7.113.1 Detailed Description . . . . .                        | 766 |
| 7.113.2 Member Function Documentation . . . . .               | 767 |
| 7.114 DDSWaitSet Class Reference . . . . .                    | 767 |
| 7.114.1 Detailed Description . . . . .                        | 767 |
| 7.114.2 Usage . . . . .                                       | 768 |
| 7.114.3 Trigger State of a ::DDSStatusCondition . . . . .     | 768 |
| 7.114.4 Trigger State of a ::DDSGuardCondition . . . . .      | 769 |
| 7.114.5 Member Function Documentation . . . . .               | 769 |
| 7.115 DPDE_DiscoveryPluginProperty Struct Reference . . . . . | 771 |
| 7.115.1 Detailed Description . . . . .                        | 772 |
| 7.115.2 Member Data Documentation . . . . .                   | 772 |
| 7.116 DPDEDiscoveryFactory Class Reference . . . . .          | 775 |
| 7.116.1 Detailed Description . . . . .                        | 776 |
| 7.116.2 Member Function Documentation . . . . .               | 776 |
| 7.117 DPSE_DiscoveryPluginProperty Struct Reference . . . . . | 776 |
| 7.117.1 Detailed Description . . . . .                        | 777 |
| 7.117.2 Member Data Documentation . . . . .                   | 777 |
| 7.118 DPSEDiscoveryFactory Class Reference . . . . .          | 779 |
| 7.118.1 Detailed Description . . . . .                        | 779 |
| 7.118.2 Member Function Documentation . . . . .               | 779 |

|                                                                      |     |
|----------------------------------------------------------------------|-----|
| 7.119 DPSEDiscoveryPlugin Class Reference . . . . .                  | 779 |
| 7.119.1 Detailed Description . . . . .                               | 780 |
| 7.119.2 Member Function Documentation . . . . .                      | 780 |
| 7.120 FooDataReader Class Reference . . . . .                        | 781 |
| 7.120.1 Detailed Description . . . . .                               | 782 |
| 7.120.2 Member Function Documentation . . . . .                      | 782 |
| 7.120.3 SEQUENCES USAGE IN TAKE AND READ . . . . .                   | 787 |
| 7.121 FooDataWriter Class Reference . . . . .                        | 792 |
| 7.121.1 Detailed Description . . . . .                               | 793 |
| 7.121.2 Member Function Documentation . . . . .                      | 794 |
| 7.122 FooSeq Struct Reference . . . . .                              | 805 |
| 7.122.1 Detailed Description . . . . .                               | 806 |
| 7.122.2 Constructor & Destructor Documentation . . . . .             | 807 |
| 7.122.3 Member Function Documentation . . . . .                      | 808 |
| 7.122.4 Member Data Documentation . . . . .                          | 816 |
| 7.123 FooTypeSupport Class Reference . . . . .                       | 818 |
| 7.123.1 Detailed Description . . . . .                               | 818 |
| 7.123.2 Member Function Documentation . . . . .                      | 818 |
| 7.124 NDDS_Discovery_Property Struct Reference . . . . .             | 824 |
| 7.124.1 Detailed Description . . . . .                               | 824 |
| 7.125 NDDS_Type_PluginVersion Struct Reference . . . . .             | 824 |
| 7.125.1 Detailed Description . . . . .                               | 824 |
| 7.125.2 Member Data Documentation . . . . .                          | 824 |
| 7.126 NETIO_Address Struct Reference . . . . .                       | 825 |
| 7.126.1 Detailed Description . . . . .                               | 825 |
| 7.126.2 Member Data Documentation . . . . .                          | 825 |
| 7.127 NETIO_AddressSeq Struct Reference . . . . .                    | 826 |
| 7.127.1 Detailed Description . . . . .                               | 826 |
| 7.128 NETIO_AddressUInt32 Struct Reference . . . . .                 | 826 |
| 7.128.1 Detailed Description . . . . .                               | 826 |
| 7.128.2 Member Data Documentation . . . . .                          | 826 |
| 7.129 NETIO_AddressValue Union Reference . . . . .                   | 826 |
| 7.129.1 Detailed Description . . . . .                               | 827 |
| 7.129.2 Member Data Documentation . . . . .                          | 827 |
| 7.130 NETIO_DGRAM_InterfaceI Struct Reference . . . . .              | 827 |
| 7.130.1 Detailed Description . . . . .                               | 828 |
| 7.130.2 Member Data Documentation . . . . .                          | 828 |
| 7.131 NETIO_DGRAM_InterfaceMultiCastGroup Struct Reference . . . . . | 831 |
| 7.131.1 Detailed Description . . . . .                               | 831 |

|                                                            |     |
|------------------------------------------------------------|-----|
| 7.131.2 Member Data Documentation                          | 831 |
| 7.132 NETIO_DGRAM_InterfaceRouteEntry Struct Reference     | 832 |
| 7.132.1 Detailed Description                               | 832 |
| 7.132.2 Member Data Documentation                          | 832 |
| 7.133 NETIO_DGRAM_InterfaceTableEntry Struct Reference     | 833 |
| 7.133.1 Detailed Description                               | 833 |
| 7.133.2 Member Data Documentation                          | 833 |
| 7.134 NETIO_DGRAM_InterfaceTableEntrySeq Struct Reference  | 834 |
| 7.134.1 Detailed Description                               | 835 |
| 7.135 NETIO_Interface Struct Reference                     | 835 |
| 7.135.1 Detailed Description                               | 835 |
| 7.136 NETIO_InterfaceI Struct Reference                    | 835 |
| 7.136.1 Detailed Description                               | 835 |
| 7.137 NETIO_Netmask Struct Reference                       | 835 |
| 7.137.1 Detailed Description                               | 836 |
| 7.137.2 Member Data Documentation                          | 836 |
| 7.138 NETIO_NetmaskSeq Struct Reference                    | 836 |
| 7.138.1 Detailed Description                               | 836 |
| 7.139 NETIO_Packet Struct Reference                        | 836 |
| 7.139.1 Detailed Description                               | 836 |
| 7.140 NETIO_SharedMemorySegmentHeader Struct Reference     | 837 |
| 7.140.1 Detailed Description                               | 837 |
| 7.141 NETIO_SHMEMInterfaceFactoryProperty Struct Reference | 837 |
| 7.141.1 Detailed Description                               | 837 |
| 7.141.2 Member Data Documentation                          | 837 |
| 7.142 OSAPI_LogProperty Struct Reference                   | 838 |
| 7.142.1 Detailed Description                               | 838 |
| 7.142.2 Member Data Documentation                          | 839 |
| 7.143 OSAPI_SharedMemorySegmentHandle Struct Reference     | 839 |
| 7.143.1 Detailed Description                               | 839 |
| 7.143.2 Member Data Documentation                          | 839 |
| 7.144 OSAPI_SharedMemorySegmentHeader Struct Reference     | 840 |
| 7.144.1 Detailed Description                               | 840 |
| 7.144.2 Member Data Documentation                          | 840 |
| 7.145 OSAPI_SystemI Struct Reference                       | 841 |
| 7.145.1 Detailed Description                               | 841 |
| 7.145.2 Member Data Documentation                          | 841 |
| 7.146 OSAPI_SystemProperty Struct Reference                | 842 |
| 7.146.1 Detailed Description                               | 843 |



|                                                                |     |
|----------------------------------------------------------------|-----|
| 7.146.2 Member Data Documentation . . . . .                    | 843 |
| 7.147 OSAPI_SystemTime Struct Reference . . . . .              | 844 |
| 7.147.1 Detailed Description . . . . .                         | 844 |
| 7.147.2 Member Data Documentation . . . . .                    | 845 |
| 7.148 OSAPI_ThreadProperty Struct Reference . . . . .          | 845 |
| 7.148.1 Detailed Description . . . . .                         | 845 |
| 7.148.2 Member Data Documentation . . . . .                    | 845 |
| 7.149 OSAPI_TimerProperty Struct Reference . . . . .           | 846 |
| 7.149.1 Detailed Description . . . . .                         | 846 |
| 7.149.2 Member Data Documentation . . . . .                    | 846 |
| 7.150 PSK_ErrListener Struct Reference . . . . .               | 846 |
| 7.150.1 Detailed Description . . . . .                         | 847 |
| 7.150.2 Member Data Documentation . . . . .                    | 847 |
| 7.151 RHSMHistoryFactory Class Reference . . . . .             | 847 |
| 7.151.1 Detailed Description . . . . .                         | 847 |
| 7.151.2 Member Function Documentation . . . . .                | 848 |
| 7.152 RTPS_Checksum Union Reference . . . . .                  | 848 |
| 7.152.1 Detailed Description . . . . .                         | 848 |
| 7.153 RTPS_ChecksumClass Struct Reference . . . . .            | 848 |
| 7.153.1 Detailed Description . . . . .                         | 849 |
| 7.154 RTPS_ChecksumProperty Struct Reference . . . . .         | 849 |
| 7.154.1 Detailed Description . . . . .                         | 849 |
| 7.155 RTPS_InterfaceFactoryProperty Struct Reference . . . . . | 849 |
| 7.155.1 Detailed Description . . . . .                         | 850 |
| 7.156 RTRegistry Class Reference . . . . .                     | 850 |
| 7.156.1 Detailed Description . . . . .                         | 850 |
| 7.156.2 Member Function Documentation . . . . .                | 850 |
| 7.157 SHMEMInterfaceFactory Class Reference . . . . .          | 851 |
| 7.157.1 Detailed Description . . . . .                         | 852 |
| 7.157.2 Member Function Documentation . . . . .                | 852 |
| 7.158 UDP_InterfaceFactoryProperty Struct Reference . . . . .  | 852 |
| 7.158.1 Detailed Description . . . . .                         | 853 |
| 7.158.2 Member Data Documentation . . . . .                    | 853 |
| 7.159 UDP_InterfaceTableEntry Struct Reference . . . . .       | 857 |
| 7.159.1 Detailed Description . . . . .                         | 857 |
| 7.159.2 Member Data Documentation . . . . .                    | 857 |
| 7.160 UDP_InterfaceTableEntrySeq Struct Reference . . . . .    | 858 |
| 7.160.1 Detailed Description . . . . .                         | 858 |
| 7.161 UDP_NatEntry Struct Reference . . . . .                  | 858 |

|                                                                      |            |
|----------------------------------------------------------------------|------------|
| 7.161.1 Detailed Description . . . . .                               | 858        |
| 7.162 UDP_NatEntrySeq Struct Reference . . . . .                     | 859        |
| 7.162.1 Detailed Description . . . . .                               | 859        |
| 7.163 UDPInterfaceFactory Class Reference . . . . .                  | 859        |
| 7.163.1 Detailed Description . . . . .                               | 859        |
| 7.163.2 Member Function Documentation . . . . .                      | 859        |
| 7.164 UDPInterfaceTable Class Reference . . . . .                    | 860        |
| 7.164.1 Detailed Description . . . . .                               | 860        |
| 7.165 WHSMHistoryFactory Class Reference . . . . .                   | 860        |
| 7.165.1 Detailed Description . . . . .                               | 860        |
| 7.165.2 Member Function Documentation . . . . .                      | 860        |
| 7.166 ZCOPY_NotifInterfaceFactoryProperty Struct Reference . . . . . | 861        |
| 7.166.1 Detailed Description . . . . .                               | 861        |
| 7.166.2 Member Data Documentation . . . . .                          | 861        |
| 7.167 ZCOPY_NotifMechanismProperty Struct Reference . . . . .        | 862        |
| 7.167.1 Detailed Description . . . . .                               | 862        |
| 7.167.2 Member Data Documentation . . . . .                          | 862        |
| 7.168 ZCOPY_NotifUserInterfaceI Struct Reference . . . . .           | 863        |
| 7.168.1 Detailed Description . . . . .                               | 863        |
| 7.168.2 Member Data Documentation . . . . .                          | 863        |
| <b>8 File Documentation</b>                                          | <b>868</b> |
| 8.1 app_gen_log.h File Reference . . . . .                           | 868        |
| 8.2 cdr_log.h File Reference . . . . .                               | 869        |
| 8.3 db_log.h File Reference . . . . .                                | 869        |
| 8.3.1 Detailed Description . . . . .                                 | 870        |
| 8.4 dds_c_config.h File Reference . . . . .                          | 871        |
| 8.4.1 Detailed Description . . . . .                                 | 871        |
| 8.5 dds_c_flowcontroller.h File Reference . . . . .                  | 871        |
| 8.5.1 Detailed Description . . . . .                                 | 872        |
| 8.6 dds_c_log.h File Reference . . . . .                             | 872        |
| 8.6.1 Detailed Description . . . . .                                 | 898        |
| 8.7 dds_filter_log.h File Reference . . . . .                        | 898        |
| 8.7.1 Detailed Description . . . . .                                 | 900        |
| 8.7.2 Macro Definition Documentation . . . . .                       | 901        |
| 8.8 dds_psk_log.h File Reference . . . . .                           | 901        |
| 8.8.1 Detailed Description . . . . .                                 | 902        |
| 8.8.2 Macro Definition Documentation . . . . .                       | 902        |
| 8.9 disc_dpde_log.h File Reference . . . . .                         | 902        |

|        |                                              |     |
|--------|----------------------------------------------|-----|
| 8.10   | disc_dpse_log.h File Reference               | 905 |
| 8.10.1 | Macro Definition Documentation               | 909 |
| 8.11   | netio_log.h File Reference                   | 911 |
| 8.11.1 | Detailed Description                         | 916 |
| 8.11.2 | Macro Definition Documentation               | 916 |
| 8.12   | netio_shmem_log.h File Reference             | 918 |
| 8.12.1 | Detailed Description                         | 920 |
| 8.13   | netio_zcopy_log.h File Reference             | 920 |
| 8.13.1 | Detailed Description                         | 920 |
| 8.14   | netio_zcopy_notif_interface.h File Reference | 920 |
| 8.14.1 | Function Documentation                       | 921 |
| 8.15   | netio_zcopy_whsq_log.h File Reference        | 922 |
| 8.15.1 | Detailed Description                         | 923 |
| 8.15.2 | Macro Definition Documentation               | 923 |
| 8.16   | osapi_log.h File Reference                   | 923 |
| 8.16.1 | Detailed Description                         | 929 |
| 8.16.2 | Macro Definition Documentation               | 930 |
| 8.17   | osapi_process.h File Reference               | 933 |
| 8.17.1 | Detailed Description                         | 933 |
| 8.18   | osapi_semaphore.h File Reference             | 933 |
| 8.18.1 | Detailed Description                         | 934 |
| 8.18.2 | Function Documentation                       | 934 |
| 8.19   | osapi_stdio.h File Reference                 | 935 |
| 8.19.1 | Detailed Description                         | 936 |
| 8.20   | osapi_task.h File Reference                  | 936 |
| 8.20.1 | Detailed Description                         | 936 |
| 8.21   | osapi_time.h File Reference                  | 936 |
| 8.21.1 | Detailed Description                         | 936 |
| 8.22   | reda_log.h File Reference                    | 937 |
| 8.22.1 | Detailed Description                         | 938 |
| 8.23   | rh_sm_log.h File Reference                   | 938 |
| 8.23.1 | Detailed Description                         | 940 |
| 8.23.2 | Macro Definition Documentation               | 940 |
| 8.24   | rt_log.h File Reference                      | 942 |
| 8.25   | rtps_config.h File Reference                 | 943 |
| 8.25.1 | Detailed Description                         | 943 |
| 8.26   | rtps_dll.h File Reference                    | 943 |
| 8.26.1 | Detailed Description                         | 943 |
| 8.27   | rtps_log.h File Reference                    | 943 |

|                                       |            |
|---------------------------------------|------------|
| 8.27.1 Detailed Description           | 950        |
| 8.27.2 Macro Definition Documentation | 951        |
| 8.28 rtps_rtps_impl.h File Reference  | 951        |
| 8.28.1 Detailed Description           | 951        |
| 8.29 wh_sm_log.h File Reference       | 951        |
| 8.29.1 Detailed Description           | 952        |
| 8.29.2 Macro Definition Documentation | 952        |
| <b>Index</b>                          | <b>955</b> |

## 1 RTI Connex Micro C++ API

Real-Time Innovations, Inc.

RTI Connex DDS Micro provides a small-footprint, modular messaging solution for resource-limited devices that have limited memory and CPU power, and may not even be running an operating system. It provides the communications services that developers need to distribute time-critical data. Additionally, RTI Connex DDS Micro is designed as a certifiable component in high-assurance systems.

RTI Connex DDS Micro benefits include:

- Accommodations for resource-constrained environments.
- Modular and user extensible architecture.
- Designed to be a certifiable component for safety-critical systems.
- Seamless interoperability with Connex DDS and Connex Messaging.

## 2 Topic Index

### 2.1 Reference Manual

Here is a list of all topics with brief descriptions:

|                                 |           |
|---------------------------------|-----------|
| <b>Documentation Guide</b>      | <b>24</b> |
| <b>DDS API</b>                  | <b>26</b> |
| <b>Content Filter Plugin</b>    | <b>29</b> |
| <b>Domain Module</b>            | <b>30</b> |
| <b>DomainParticipantFactory</b> | <b>31</b> |
| <b>DomainParticipant</b>        | <b>32</b> |

|                                       |     |
|---------------------------------------|-----|
| Discovery                             | 34  |
| Participant Built-in Topic            | 39  |
| Publication Built-in Topic            | 42  |
| Subscription Built-in Topic           | 44  |
| Topic                                 | 46  |
| DDS-Specific Primitive Types          | 46  |
| Topic                                 | 52  |
| FlatData Topic-Types                  | 53  |
| Zero Copy Transfer Over Shared Memory | 53  |
| User Data Type Support                | 54  |
| Publication Module                    | 57  |
| Flow Controllers                      | 58  |
| Publisher                             | 63  |
| DataWriter                            | 65  |
| Subscription Module                   | 67  |
| Subscriber                            | 68  |
| DataReader                            | 70  |
| Data Sample                           | 73  |
| Sample States                         | 73  |
| View States                           | 75  |
| Instance States                       | 76  |
| Infrastructure Module                 | 79  |
| Time Support                          | 81  |
| GUID Support                          | 85  |
| Sequence Number Support               | 86  |
| Return Codes                          | 86  |
| Status Kinds                          | 88  |
| QoS Policies                          | 97  |
| FILTER                                | 102 |
| CONTENT_FILTER                        | 104 |

|                                    |     |
|------------------------------------|-----|
| DISCOVERY                          | 104 |
| USER_TRAFFIC                       | 104 |
| DEADLINE                           | 105 |
| LATENCY_BUDGET                     | 105 |
| OWNERSHIP                          | 105 |
| OWNERSHIP_STRENGTH                 | 106 |
| LIVELINESS                         | 107 |
| RELIABILITY                        | 108 |
| HISTORY                            | 109 |
| DURABILITY                         | 110 |
| DATA_REPRESENTATION                | 111 |
| RESOURCE_LIMITS                    | 114 |
| PRESENTATION                       | 115 |
| DESTINATION_ORDER                  | 117 |
| ENTITY_FACTORY                     | 118 |
| Extended Qos Support               | 119 |
| DataReader Resource Limits         | 119 |
| DataWriter Resource Limits         | 120 |
| SYSTEM_RESOURCE_LIMITS             | 120 |
| WIRE_PROTOCOL                      | 121 |
| DATA_READER_PROTOCOL               | 125 |
| DATA_WRITER_PROTOCOL               | 125 |
| TRANSPORT                          | 125 |
| DOMAIN_PARTICIPANT_RESOURCE_LIMITS | 126 |
| ENTITY_NAME                        | 126 |
| PUBLISH_MODE                       | 127 |
| PARTITION                          | 129 |
| PROPERTY                           | 129 |
| USER_DATA                          | 130 |
| GROUP_DATA                         | 130 |

---

|                                                       |            |
|-------------------------------------------------------|------------|
| <b>TOPIC_DATA</b>                                     | <b>131</b> |
| <b>DATA_WRITER_TRANSFER_MODE</b>                      | <b>131</b> |
| <b>Conditions and WaitSets</b>                        | <b>132</b> |
| <b>Entity Support</b>                                 | <b>133</b> |
| <b>Sequence Support</b>                               | <b>133</b> |
| <b>Exception codes</b>                                | <b>133</b> |
| <b>Type Support</b>                                   | <b>134</b> |
| <b>String Support</b>                                 | <b>135</b> |
| <b>General Utilities and Compliance Configuration</b> | <b>139</b> |
| <b>Compliance Configuration</b>                       | <b>139</b> |
| <b>Property Reference</b>                             | <b>139</b> |
| <b>DPSE Static Discovery API</b>                      | <b>140</b> |
| <b>DPDE Dynamic Discovery API</b>                     | <b>141</b> |
| <b>CDR Stream API</b>                                 | <b>142</b> |
| <b>RT Run-Time API</b>                                | <b>145</b> |
| <b>RHSM Reader History API</b>                        | <b>145</b> |
| <b>WHSM Writer History API</b>                        | <b>145</b> |
| <b>NETIO API</b>                                      | <b>146</b> |
| <b>NETIO Packet API</b>                               | <b>146</b> |
| <b>NETIO Interface API</b>                            | <b>147</b> |
| <b>NETIO Address API</b>                              | <b>149</b> |
| <b>UDP Transport API</b>                              | <b>151</b> |
| <b>UDP Transform API</b>                              | <b>154</b> |
| <b>RTPS Transport API</b>                             | <b>154</b> |
| <b>Shared Memory Transport API</b>                    | <b>158</b> |
| <b>NETIO Shared Memory API</b>                        | <b>159</b> |
| <b>Shared Memory Mutex API</b>                        | <b>160</b> |
| <b>Shared Memory Segment API</b>                      | <b>166</b> |
| <b>Shared Memory Signaling Semaphore</b>              | <b>172</b> |
| <b>NETIO DGRAM API</b>                                | <b>178</b> |

|                             |     |
|-----------------------------|-----|
| Zero Copy v2 API            | 185 |
| Notification Interface      | 188 |
| OS API                      | 189 |
| OSAPI Heap                  | 190 |
| OSAPI Mutex                 | 193 |
| OSAPI Process               | 195 |
| OSAPI Semaphore             | 196 |
| OSAPI Memory API            | 198 |
| OSAPI String API            | 198 |
| OSAPI System API            | 198 |
| OSAPI Thread API            | 202 |
| OSAPI Time                  | 205 |
| OSAPI Timer                 | 205 |
| OSAPI Types                 | 206 |
| OSAPI_SharedMemoryMonitor   | 209 |
| OSAPI_SharedMemorySegment   | 212 |
| Log API                     | 219 |
| Application Generation API  | 224 |
| Lightweight Security API    | 226 |
| Resource Limits             | 227 |
| DomainParticipantFactoryQos | 228 |
| max_components              | 228 |
| max_participants            | 228 |
| DomainParticipantQos        | 228 |
| local_topic_allocation      | 229 |
| local_type_allocation       | 229 |
| local_publisher_allocation  | 229 |
| local_subscriber_allocation | 230 |
| local_reader_allocation     | 230 |
| local_writer_allocation     | 230 |



|                                            |     |
|--------------------------------------------|-----|
| matching_writer_reader_pair_allocation     | 230 |
| remote_participant_allocation              | 231 |
| remote_writer_allocation                   | 232 |
| remote_reader_allocation                   | 232 |
| max_destination_ports                      | 232 |
| max_receive_ports                          | 233 |
| DataReaderQos                              | 234 |
| max_instances                              | 235 |
| max_remote_writers_per_instance            | 235 |
| max_samples                                | 236 |
| max_samples_per_instance                   | 236 |
| max_remote_writers                         | 237 |
| max_samples_per_remote_writer              | 237 |
| max_routes_per_writer                      | 237 |
| max_outstanding_reads                      | 238 |
| max_fragmented_samples                     | 238 |
| max_fragmented_samples_per_remote_writer   | 238 |
| DataWriterQos                              | 239 |
| max_samples                                | 239 |
| max_samples_per_instance                   | 240 |
| max_instances                              | 240 |
| max_routes_per_reader                      | 240 |
| max_remote_readers                         | 241 |
| OSAPI                                      | 241 |
| max_buffer_size                            | 241 |
| UDP Transport                              | 241 |
| max_message_size                           | 241 |
| Dynamic Participant Static Endpoint (DPSE) | 242 |
| max_participant_locators                   | 242 |
| max_locators_per_discovered_participant    | 243 |

|                                             |     |
|---------------------------------------------|-----|
| Dynamic Participant Dynamic Endpoint (DPDE) | 243 |
| max_participant_locators                    | 243 |
| max_locators_per_discovered_participant     | 244 |
| max_samples_per_builtin_endpoint_reader     | 244 |
| Log Codes                                   | 245 |
| OSAPI                                       | 245 |
| REDA                                        | 264 |
| DB                                          | 270 |
| RT                                          | 276 |
| CDR                                         | 278 |
| NETIO                                       | 279 |
| RTPS                                        | 306 |
| DDS_C                                       | 336 |
| WHSQ                                        | 448 |
| RH                                          | 450 |
| WH                                          | 455 |
| DPSE                                        | 457 |
| DPDE                                        | 470 |
| APPGEN                                      | 482 |
| DDS_FILTER                                  | 485 |

## 3 Hierarchical Index

### 3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                            |     |
|----------------------------|-----|
| DDS_BuiltinTopicKey_t      | 495 |
| DDS_ChecksumProperty       | 496 |
| DDS_ChecksumProperty_t     | 497 |
| DDS_ContentFilterProperty  | 498 |
| DDS_ContentFilterQosPolicy | 500 |

|                                              |     |
|----------------------------------------------|-----|
| DDS_DataReaderInstanceReplacedStatus         | 502 |
| DDS_DataReaderProtocolQosPolicy              | 504 |
| DDS_DataReaderQos                            | 505 |
| DDS_DataReaderResourceLimitsQosPolicy        | 508 |
| DDS_DataRepresentationQosPolicy              | 512 |
| DDS_DataWriterProtocolQosPolicy              | 513 |
| DDS_DataWriterQos                            | 514 |
| DDS_DataWriterResourceLimitsQosPolicy        | 518 |
| DDS_DataWriterShmemRefTransferModeSettings   | 521 |
| DDS_DataWriterTransferModeQosPolicy          | 521 |
| DDS_DeadlineQosPolicy                        | 522 |
| DDS_DestinationOrderQosPolicy                | 524 |
| DDS_DiscoveryComponent                       | 525 |
| DDS_DiscoveryQosPolicy                       | 525 |
| DDS_DomainParticipantFactoryQos              | 528 |
| DDS_DomainParticipantQos                     | 529 |
| DDS_DomainParticipantResourceLimitsQosPolicy | 532 |
| DDS_DurabilityQosPolicy                      | 540 |
| DDS_Duration_t                               | 541 |
| DDS_EntityFactoryQosPolicy                   | 542 |
| DDS_EntityNameQosPolicy                      | 543 |
| DDS_FilterQosPolicy                          | 544 |
| DDS_FilterResourceLimits                     | 546 |
| DDS_FlowControllerProperty_t                 | 549 |
| DDS_FlowControllerTokenBucketProperty_t      | 550 |
| DDS_GroupDataQosPolicy                       | 553 |
| DDS_GUID_t                                   | 554 |
| DDS_HistoryQosPolicy                         | 555 |
| DDS_InconsistentTopicStatus                  | 557 |
| DDS_InstanceHandleSeq                        | 557 |

|                                         |     |
|-----------------------------------------|-----|
| DDS_LatencyBudgetQosPolicy              | 557 |
| DDS_LivelinessChangedStatus             | 559 |
| DDS_LivelinessLostStatus                | 560 |
| DDS_LivelinessQosPolicy                 | 561 |
| DDS_Locator_t                           | 564 |
| DDS_LocatorSeq                          | 565 |
| DDS_OfferedDeadlineMissedStatus         | 565 |
| DDS_OfferedIncompatibleQosStatus        | 567 |
| DDS_OwnershipQosPolicy                  | 568 |
| DDS_OwnershipStrengthQosPolicy          | 573 |
| DDS_ParticipantBuiltinTopicData         | 574 |
| DDS_PartitionQosPolicy                  | 577 |
| DDS_PresentationQosPolicy               | 579 |
| DDS_ProductVersion                      | 583 |
| DDS_ProductVersion_t                    | 584 |
| DDS_Property_t                          | 585 |
| DDS_PropertyQosPolicy                   | 586 |
| DDS_PropertySeq                         | 587 |
| DDS_ProtocolVersion                     | 588 |
| DDS_PskServiceFactoryProperty           | 588 |
| DDS_PublicationBuiltinTopicData         | 589 |
| DDS_PublicationMatchedStatus            | 594 |
| DDS_PublisherQos                        | 595 |
| DDS_PublishModeQosPolicy                | 596 |
| DDS_ReliabilityQosPolicy                | 599 |
| DDS_ReliableReaderActivityChangedStatus | 601 |
| DDS_ReliableSampleUnacknowledgedStatus  | 602 |
| DDS_RequestedDeadlineMissedStatus       | 603 |
| DDS_RequestedIncompatibleQosStatus      | 604 |
| DDS_ResourceLimitsQosPolicy             | 605 |

|                                   |     |
|-----------------------------------|-----|
| DDS_RtpsReliableReaderProtocol_t  | 608 |
| DDS_RtpsReliableWriterProtocol_t  | 609 |
| DDS_RtpsWellKnownPorts_t          | 611 |
| DDS_SampleIdentity_t              | 615 |
| DDS_SampleInfo                    | 616 |
| DDS_SampleInfoSeq                 | 619 |
| DDS_SampleLostStatus              | 619 |
| DDS_SampleRejectedStatus          | 620 |
| DDS_SecTransform_Configuration    | 621 |
| DDS_SequenceNumber_t              | 622 |
| DDS_SubscriberQos                 | 623 |
| DDS_SubscriptionBuiltinTopicData  | 624 |
| DDS_SubscriptionMatchedStatus     | 629 |
| DDS_SystemResourceLimitsQosPolicy | 631 |
| DDS_Time_t                        | 632 |
| DDS_TopicDataQosPolicy            | 633 |
| DDS_TopicQos                      | 634 |
| DDS_TransportQosPolicy            | 635 |
| DDS_TypePluginI                   | 636 |
| DDS_UserDataQosPolicy             | 636 |
| DDS_UserTrafficQosPolicy          | 637 |
| DDS_VendorId                      | 638 |
| DDS_VendorId_t                    | 638 |
| DDS_WireProtocolQosPolicy         | 639 |
| DDS_WriteParams_t                 | 643 |
| DDSCondition                      | 644 |
| DDSGuardCondition                 | 731 |
| DDSStatusCondition                | 746 |
| DDSConditionSeq                   | 645 |
| DDSDomainParticipantFactory       | 710 |

|                              |     |
|------------------------------|-----|
| DDSDomainParticipantListener | 718 |
| DDSEntity                    | 725 |
| DDSDataReader                | 645 |
| DDSDataWriter                | 666 |
| DDSDomainEntity              | 685 |
| DDSDomainParticipant         | 685 |
| DDSPublisher                 | 737 |
| DDSSubscriber                | 748 |
| DDSTopic                     | 759 |
| DDSFlowController            | 729 |
| DDSListener                  | 732 |
| DDSDataReaderListener        | 662 |
| DDSSubscriberListener        | 757 |
| DDSDataWriterListener        | 681 |
| DDSPublisherListener         | 745 |
| DDSTopicListener             | 766 |
| DDSPROPERTYQOSPolicyHelper   | 734 |
| DDSTopicDescription          | 764 |
| DDSTopic                     | 759 |
| DDSWaitSet                   | 767 |
| DPDE_DiscoveryPluginProperty | 771 |
| DPDEDiscoveryFactory         | 775 |
| DPSE_DiscoveryPluginProperty | 776 |
| DPSEDiscoveryFactory         | 779 |
| DPSEDiscoveryPlugin          | 779 |
| FooSeq                       | 805 |
| FooTypeSupport               | 818 |
| NDDS_Discovery_Property      | 824 |
| NDDS_Type_PluginVersion      | 824 |
| NETIO_Address                | 825 |

|                                     |     |
|-------------------------------------|-----|
| NETIO_AddressSeq                    | 826 |
| NETIO_AddressUInt32                 | 826 |
| NETIO_AddressValue                  | 826 |
| NETIO_DGRAM_InterfaceI              | 827 |
| NETIO_DGRAM_InterfaceMultiCastGroup | 831 |
| NETIO_DGRAM_InterfaceRouteEntry     | 832 |
| NETIO_DGRAM_InterfaceTableEntry     | 833 |
| NETIO_DGRAM_InterfaceTableEntrySeq  | 834 |
| NETIO_Interface                     | 835 |
| NETIO_InterfaceI                    | 835 |
| NETIO_Netmask                       | 835 |
| NETIO_NetmaskSeq                    | 836 |
| NETIO_Packet                        | 836 |
| NETIO_SharedMemorySegmentHeader     | 837 |
| NETIO_SHMEMInterfaceFactoryProperty | 837 |
| OSAPI_LogProperty                   | 838 |
| OSAPI_SharedMemorySegmentHandle     | 839 |
| OSAPI_SharedMemorySegmentHeader     | 840 |
| OSAPI_SystemI                       | 841 |
| OSAPI_SystemProperty                | 842 |
| OSAPI_SystemTime                    | 844 |
| OSAPI_ThreadProperty                | 845 |
| OSAPI_TimerProperty                 | 846 |
| PSK_ErrListener                     | 846 |
| RHSMHistoryFactory                  | 847 |
| RTPS_Checksum                       | 848 |
| RTPS_ChecksumClass                  | 848 |
| RTPS_ChecksumProperty               | 849 |
| RTPS_InterfaceFactoryProperty       | 849 |
| RTRegistry                          | 850 |

|                                     |     |
|-------------------------------------|-----|
| SHMEMInterfaceFactory               | 851 |
| UDP_InterfaceFactoryProperty        | 852 |
| UDP_InterfaceTableEntry             | 857 |
| UDP_InterfaceTableEntrySeq          | 858 |
| UDP_NatEntry                        | 858 |
| UDP_NatEntrySeq                     | 859 |
| UDPInterfaceFactory                 | 859 |
| UDPInterfaceTable                   | 860 |
| WHSMHistoryFactory                  | 860 |
| ZCOPY_NotifInterfaceFactoryProperty | 861 |
| ZCOPY_NotifMechanismProperty        | 862 |
| ZCOPY_NotifUserInterfaceI           | 863 |

## 4 Class Index

### 4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                                                                                                                                                            |     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>DDS_BuiltinTopicKey_t</b><br>The key type of the built-in topic types                                                                                                   | 495 |
| <b>DDS_ChecksumProperty</b><br><<eXtension>> Type to define the checksum properties of the participant RTI Connex DDS Micro                                                | 496 |
| <b>DDS_ChecksumProperty_t</b><br><<eXtension>> Type to define the checksum properties of the participant RTI Connex DDS Micro                                              | 497 |
| <b>DDS_ContentFilterProperty</b><br>All of the information necessary for a <b>DDSDataWriter</b> to perform writer-side filtering for a <b>DDSDataReader</b>                | 498 |
| <b>DDS_ContentFilterQosPolicy</b><br>Filter that allows a <b>DDSDataReader</b> to specify that it is interested only in (potentially) a subset of the samples on its topic | 500 |
| <b>DDS_DataReaderInstanceReplacedStatus</b><br><<eXtension>> <b>DDS_INSTANCE_REPLACED_STATUS</b>                                                                           | 502 |
| <b>DDS_DataReaderProtocolQosPolicy</b><br><<eXtension>> Protocol that applies only to <b>DDSDataReader</b> instances                                                       | 504 |



|                                                                                                                                                                            |     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>DDS_DataReaderQos</b>                                                                                                                                                   |     |
| QoS policies supported by a <b>DDSDataReader</b> entity                                                                                                                    | 505 |
| <b>DDS_DataReaderResourceLimitsQosPolicy</b>                                                                                                                               |     |
| Resource limits that apply only to <b>DDSDataReader</b> instances                                                                                                          | 508 |
| <b>DDS_DataRepresentationQosPolicy</b>                                                                                                                                     |     |
| This QoS policy contains a list of representation identifiers used by <b>DDSDataWriter</b> and <b>DDSDataReader</b> entities to negotiate which data representation to use | 512 |
| <b>DDS_DataWriterProtocolQosPolicy</b>                                                                                                                                     |     |
| << <i>eXtension</i> >> Protocol that applies only to <b>DDSDataWriter</b> instances                                                                                        | 513 |
| <b>DDS_DataWriterQos</b>                                                                                                                                                   |     |
| QoS policies supported by a <b>DDSDataWriter</b> entity                                                                                                                    | 514 |
| <b>DDS_DataWriterResourceLimitsQosPolicy</b>                                                                                                                               |     |
| Resource limits that apply only to <b>DDSDataWriter</b> instances                                                                                                          | 518 |
| <b>DDS_DataWriterShmemRefTransferModeSettings</b>                                                                                                                          |     |
| Settings related to transferring data using shared memory references                                                                                                       | 521 |
| <b>DDS_DataWriterTransferModeQosPolicy</b>                                                                                                                                 |     |
| << <i>eXtension</i> >> Qos related to transferring data                                                                                                                    | 521 |
| <b>DDS_DeadlineQosPolicy</b>                                                                                                                                               |     |
| Expresses the maximum duration (deadline) within which an instance is expected to be updated                                                                               | 522 |
| <b>DDS_DestinationOrderQosPolicy</b>                                                                                                                                       |     |
| Destination order policies that affect the ordering of subscribed data                                                                                                     | 524 |
| <b>DDS_DiscoveryComponent</b>                                                                                                                                              |     |
| Specifies the discovery plugin the <b>DDSDomainParticipant</b> should use                                                                                                  | 525 |
| <b>DDS_DiscoveryQosPolicy</b>                                                                                                                                              |     |
| Specifies the attributes required to discover participants in the domain                                                                                                   | 525 |
| <b>DDS_DomainParticipantFactoryQos</b>                                                                                                                                     |     |
| QoS policies supported by a <b>DDSDomainParticipantFactory</b>                                                                                                             | 528 |
| <b>DDS_DomainParticipantQos</b>                                                                                                                                            |     |
| QoS policies supported by a <b>DDSDomainParticipant</b> entity                                                                                                             | 529 |
| <b>DDS_DomainParticipantResourceLimitsQosPolicy</b>                                                                                                                        |     |
| Resource limits that apply only to <b>DDSDomainParticipant</b> instances                                                                                                   | 532 |
| <b>DDS_DurabilityQosPolicy</b>                                                                                                                                             |     |
| Durability properties that affect the life-cycle of published data                                                                                                         | 540 |
| <b>DDS_Duration_t</b>                                                                                                                                                      |     |
| Type for <i>duration</i> representation                                                                                                                                    | 541 |
| <b>DDS_EntityFactoryQosPolicy</b>                                                                                                                                          |     |
| A QoS policy for all <b>DDSEntity</b> types that can act as factories for one or more other <b>DDSEntity</b> types                                                         | 542 |
| <b>DDS_EntityNameQosPolicy</b>                                                                                                                                             |     |
| Policy used to name an entity                                                                                                                                              | 543 |

|                                                |                                                                                                                                                                                                    |     |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>DDS_FilterQosPolicy</b>                     | Configures the content filter capabilities and resource limits for a <b>DDSDomainParticipant</b> and all of its child entities                                                                     | 544 |
| <b>DDS_FilterResourceLimits</b>                | Resource limits for content filtering on a <b>DDSDomainParticipant</b> and all of its child entities                                                                                               | 546 |
| <b>DDS_FlowControllerProperty_t</b>            | Determines the flow control characteristics of the <b>DDSFlowController</b>                                                                                                                        | 549 |
| <b>DDS_FlowControllerTokenBucketProperty_t</b> | <b>DDSFlowController</b> uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties                 | 550 |
| <b>DDS_GroupDataQosPolicy</b>                  | Attaches a buffer of opaque data to a <b>DDSEntity</b> which is distributed to other applications as part of the discovery process                                                                 | 553 |
| <b>DDS_GUID_t</b>                              | Type for <i>GUID</i> (Global Unique Identifier) representation                                                                                                                                     | 554 |
| <b>DDS_HistoryQosPolicy</b>                    | Specifies the behavior of RTI Connexx DDS Micro in the case where the value of a sample changes (one or more times) before it can be successfully communicated to one or more existing subscribers | 555 |
| <b>DDS_InconsistentTopicStatus</b>             | <b>DDS_INCONSISTENT_TOPIC_STATUS</b>                                                                                                                                                               | 557 |
| <b>DDS_InstanceHandleSeq</b>                   | Instantiates <b>FooSeq</b> < <b>DDS_InstanceHandle_t</b> >                                                                                                                                         | 557 |
| <b>DDS_LatencyBudgetQosPolicy</b>              | Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications                                                    | 557 |
| <b>DDS_LivelinessChangedStatus</b>             | <b>DDS_LIVELINESS_CHANGED_STATUS</b>                                                                                                                                                               | 559 |
| <b>DDS_LivelinessLostStatus</b>                | <b>DDS_LIVELINESS_LOST_STATUS</b>                                                                                                                                                                  | 560 |
| <b>DDS_LivelinessQosPolicy</b>                 | Determines the mechanism and parameters used by the application to determine whether a <b>DDSEntity</b> is alive                                                                                   | 561 |
| <b>DDS_Locator_t</b>                           | << <i>eXtension</i> >> Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports                                        | 564 |
| <b>DDS_LocatorSeq</b>                          | Declares IDL sequence < <b>DDS_Locator_t</b> >                                                                                                                                                     | 565 |
| <b>DDS_OfferedDeadlineMissedStatus</b>         | <b>DDS_OFFERED_DEADLINE_MISSED_STATUS</b>                                                                                                                                                          | 565 |

---

|                                                                                                                                                                                                                                             |     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>DDS_OfferedIncompatibleQosStatus</b><br><b>DDS_OFFERED_INCOMPATIBLE_QOS_STATUS</b>                                                                                                                                                       | 567 |
| <b>DDS_OwnershipQosPolicy</b><br>Specifies whether it is allowed for multiple <b>DDSDataWriter</b> (s) to write the same instance of the data and if so, how these modifications should be arbitrated                                       | 568 |
| <b>DDS_OwnershipStrengthQosPolicy</b><br>Specifies the value of the strength used to arbitrate among multiple <b>DDSDataWriter</b> objects that attempt to modify the same instance of a data type (identified by <b>DDSTopic</b> + key)    | 573 |
| <b>DDS_ParticipantBuiltinTopicData</b><br>Object representing a remote DomainParticipant                                                                                                                                                    | 574 |
| <b>DDS_PartitionQosPolicy</b><br>A set of strings that introduces logical PARTITIONS                                                                                                                                                        | 577 |
| <b>DDS_PresentationQosPolicy</b><br>Specifies how the samples representing changes to data instances are presented to a subscribing application                                                                                             | 579 |
| <b>DDS_ProductVersion</b><br><< <i>eXtension</i> >> Type used to represent the current version of RTI Connex DDS Micro                                                                                                                      | 583 |
| <b>DDS_ProductVersion_t</b><br><< <i>eXtension</i> >> Type used to represent the current version of RTI Connex DDS Micro                                                                                                                    | 584 |
| <b>DDS_Property_t</b><br>Properties are name/value pairs objects                                                                                                                                                                            | 585 |
| <b>DDS_PropertyQosPolicy</b><br>Stores name/value(string) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies                                                   | 586 |
| <b>DDS_PropertySeq</b><br>Declares IDL sequence < <b>DDS_Property_t</b> >                                                                                                                                                                   | 587 |
| <b>DDS_ProtocolVersion</b><br><< <i>eXtension</i> >> Type used to represent the version of the RTPS protocol                                                                                                                                | 588 |
| <b>DDS_PskServiceFactoryProperty</b><br>Property used to register the Lightweight Security Plugin                                                                                                                                           | 588 |
| <b>DDS_PublicationBuiltinTopicData</b><br>Object describing a remote Publication                                                                                                                                                            | 589 |
| <b>DDS_PublicationMatchedException</b><br><b>DDS_PUBLICATION_MATCHED_STATUS</b>                                                                                                                                                             | 594 |
| <b>DDS_PublisherQos</b><br>QoS policies supported by a <b>DDSPublisher</b> entity                                                                                                                                                           | 595 |
| <b>DDS_PublishModeQosPolicy</b><br>Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its <i>own</i> thread to send data, instead of the user thread | 596 |

|                                                         |                                                                                                                                                                                    |     |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <a href="#">DDS_ReliabilityQosPolicy</a>                | Indicates the level of reliability offered/requested by RTI Connex DDS Micro                                                                                                       | 599 |
| <a href="#">DDS_ReliableReaderActivityChangedStatus</a> | << <i>eXtension</i> >> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer                                             | 601 |
| <a href="#">DDS_ReliableSampleUnacknowledgedStatus</a>  | Status about an unacknowledged sample that is removed from the <a href="#">DDSDataWriter</a> cache before being acknowledged by all matched <a href="#">DDSDataReader</a> entities | 602 |
| <a href="#">DDS_RequestedDeadlineMissedStatus</a>       | <a href="#">DDS_REQUESTED_DEADLINE_MISSED_STATUS</a>                                                                                                                               | 603 |
| <a href="#">DDS_RequestedIncompatibleQosStatus</a>      | <a href="#">DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS</a>                                                                                                                              | 604 |
| <a href="#">DDS_ResourceLimitsQosPolicy</a>             | Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS                                                                                   | 605 |
| <a href="#">DDS_RtpsReliableReaderProtocol_t</a>        | Qos related to reliable reader protocol defined in RTPS                                                                                                                            | 608 |
| <a href="#">DDS_RtpsReliableWriterProtocol_t</a>        | QoS related to the reliable writer protocol defined in RTPS                                                                                                                        | 609 |
| <a href="#">DDS_RtpsWellKnownPorts_t</a>                | RTPS well-known port mapping configuration                                                                                                                                         | 611 |
| <a href="#">DDS_SampleIdentity_t</a>                    | Data structure used to provide explicit identity information about a published data sample                                                                                         | 615 |
| <a href="#">DDS_SampleInfo</a>                          | Information that accompanies each sample that is read or taken                                                                                                                     | 616 |
| <a href="#">DDS_SampleInfoSeq</a>                       | Declares IDL sequence < <a href="#">DDS_SampleInfo</a> >                                                                                                                           | 619 |
| <a href="#">DDS_SampleLostStatus</a>                    | <a href="#">DDS_SAMPLE_LOST_STATUS</a>                                                                                                                                             | 619 |
| <a href="#">DDS_SampleRejectedStatus</a>                | <a href="#">DDS_SAMPLE_REJECTED_STATUS</a>                                                                                                                                         | 620 |
| <a href="#">DDS_SecTransform_Configuration</a>          | A structure for configuration this Transform PSL                                                                                                                                   | 621 |
| <a href="#">DDS_SequenceNumber_t</a>                    | Type for <i>sequence</i> number representation                                                                                                                                     | 622 |
| <a href="#">DDS_SubscriberQos</a>                       | QoS policies supported by a <a href="#">DDSSubscriber</a> entity                                                                                                                   | 623 |
| <a href="#">DDS_SubscriptionBuiltinTopicData</a>        | Entry created when a <a href="#">DDSDataReader</a> is discovered in association with its Subscriber                                                                                | 624 |

|                                                                                                                                                                                                                       |     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>DDS_SubscriptionMatchedStatus</b><br><b>DDS_SUBSCRIPTION_MATCHED_STATUS</b>                                                                                                                                        | 629 |
| <b>DDS_SystemResourceLimitsQosPolicy</b><br>Resource limits that apply only to <b>DDSDomainParticipantFactory</b>                                                                                                     | 631 |
| <b>DDS_Time_t</b><br>Type for <i>time</i> representation                                                                                                                                                              | 632 |
| <b>DDS_TopicDataQosPolicy</b><br>Attaches a buffer of opaque data to a <b>DDSTopic</b> which is distributed to other applications as part of the discovery process                                                    | 633 |
| <b>DDS_TopicQos</b><br>QoS policies supported by a <b>DDSTopic</b> entity                                                                                                                                             | 634 |
| <b>DDS_TransportQosPolicy</b><br><< <i>eXtension</i> >> Settings for available transports                                                                                                                             | 635 |
| <b>DDS_TypePluginI</b><br><< <i>interface</i> >> Type plugin interface                                                                                                                                                | 636 |
| <b>DDS_UserDataQosPolicy</b><br>Attaches a buffer of opaque data to a <b>DDSEntity</b> which is distributed to other applications as part of the discovery process                                                    | 636 |
| <b>DDS_UserTrafficQosPolicy</b><br><< <i>eXtension</i> >> Settings for available transports                                                                                                                           | 637 |
| <b>DDS_VendorId</b><br><< <i>eXtension</i> >> Type used to represent the vendor of the service implementing the RTPS protocol                                                                                         | 638 |
| <b>DDS_VendorId_t</b><br><< <i>eXtension</i> >> Type used to represent the vendor of the service implementing the RTPS protocol                                                                                       | 638 |
| <b>DDS_WireProtocolQosPolicy</b><br>Specifies the wire-protocol-related attributes for the <b>DDSDomainParticipant</b>                                                                                                | 639 |
| <b>DDS_WriteParams_t</b><br>Data structure used to provide custom parameters to a DataWriter's write operation                                                                                                        | 643 |
| <b>DDSDCondition</b><br><< <i>interface</i> >> Root class for all the conditions that may be attached to a <b>DDSWaitSet</b>                                                                                          | 644 |
| <b>DDSDConditionSeq</b><br>A sequence object containing references to one or more <b>DDSDCondition</b> objects                                                                                                        | 645 |
| <b>DDSDDataReader</b><br><< <i>interface</i> >> Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached <b>DDSSubscriber</b> | 645 |
| <b>DDSDDataReaderListener</b><br><< <i>interface</i> >> <b>DDSListener</b> for reader status                                                                                                                          | 662 |
| <b>DDSDDataWriter</b><br><< <i>interface</i> >> Allows an application to set the value of the data to be published under a given <b>DDSTopic</b>                                                                      | 666 |

|                                                                                                                                                                                                                               |     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>DDSDataWriterListener</b><br><<interface>> <b>DDSLISTENER</b> for writer status                                                                                                                                            | 681 |
| <b>DDSDomainEntity</b><br><<interface>> Abstract base class for all DDS entities except for the <b>DDSDomainParticipant</b>                                                                                                   | 685 |
| <b>DDSDomainParticipant</b><br><<interface>> Container for all <b>DDSDomainEntity</b> objects                                                                                                                                 | 685 |
| <b>DDSDomainParticipantFactory</b><br><<singleton>> <<interface>> Allows creation and destruction of <b>DDSDomainParticipant</b> objects                                                                                      | 710 |
| <b>DDSDomainParticipantListener</b><br><<interface>> Listener for participant status                                                                                                                                          | 718 |
| <b>DDSEntity</b><br><<interface>> Abstract base class for all the DDS objects that support QoS policies, and a listener                                                                                                       | 725 |
| <b>DDSTransportController</b><br><<interface>> A flow controller is the object responsible for shaping the network traffic by determining when attached asynchronous <b>DDSDataWriter</b> instances are allowed to write data | 729 |
| <b>DDSGuardCondition</b><br><<interface>> A specific <b>DDSCONDITION</b> whose <code>trigger_value</code> is completely under the control of the application                                                                  | 731 |
| <b>DDSLISTENER</b><br><<interface>> Abstract base class for all Listener interfaces                                                                                                                                           | 732 |
| <b>DDSPROPERTYQOSPOLICYHELPER</b><br>Policy helpers that facilitate management of the properties in the input policy                                                                                                          | 734 |
| <b>DDSPublisher</b><br><<interface>> A publisher is the object responsible for the actual dissemination of publications                                                                                                       | 737 |
| <b>DDSPublisherListener</b><br><<interface>> <b>DDSLISTENER</b> for <b>DDSPublisher</b> status                                                                                                                                | 745 |
| <b>DDSStatusCondition</b><br><<interface>> A specific <b>DDSCONDITION</b> that is associated with each <b>DDSEntity</b>                                                                                                       | 746 |
| <b>DDSSubscriber</b><br><<interface>> A subscriber is the object responsible for actually receiving data from a subscription                                                                                                  | 748 |
| <b>DDSSubscriberListener</b><br><<interface>> <b>DDSLISTENER</b> for status about a subscriber                                                                                                                                | 757 |
| <b>DDSTopic</b><br><<interface>> The most basic description of the data to be published and subscribed                                                                                                                        | 759 |
| <b>DDSTopicDescription</b><br><<interface>> Base class for <b>DDSTopic</b>                                                                                                                                                    | 764 |

|                                                                                                                                                                                               |     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <b>DDSTopicListener</b>                                                                                                                                                                       |     |
| <<interface>> <b>DDSTopicListener</b> for <b>DDSTopic</b> entities                                                                                                                            | 766 |
| <b>DDSWaitSet</b>                                                                                                                                                                             |     |
| <<interface>> Allows an application to wait until one or more of the attached <b>DDSTopic</b> objects has a <b>trigger_value</b> of <b>DDS_BOOLEAN_TRUE</b> or else until the timeout expires | 767 |
| <b>DPDE_DiscoveryPluginProperty</b>                                                                                                                                                           |     |
| <<extension>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery                        | 771 |
| <b>DPDEDiscoveryFactory</b>                                                                                                                                                                   |     |
| <<extension>> Discovery plug-in used for asserting remote entities                                                                                                                            | 775 |
| <b>DPSE_DiscoveryPluginProperty</b>                                                                                                                                                           |     |
| <<extension>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. This includes all discovery timing properties for participant discovery                         | 776 |
| <b>DPSEDiscoveryFactory</b>                                                                                                                                                                   |     |
| <<extension>> A discovery factory that will be used by the <b>DDSDomainParticipantFactory</b> to create a discovery plugin                                                                    | 779 |
| <b>DPSEDiscoveryPlugin</b>                                                                                                                                                                    |     |
| <<extension>> Discovery plug-in used for asserting remote entities                                                                                                                            | 779 |
| <b>FooDataReader</b>                                                                                                                                                                          |     |
| Declares the interface required to support a user data type-specific data reader                                                                                                              | 781 |
| <b>FooDataWriter</b>                                                                                                                                                                          |     |
| Declares the interface required to support a user data type-specific data writer                                                                                                              | 792 |
| <b>FooSeq</b>                                                                                                                                                                                 |     |
| <<interface>> <<generic>> A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo"                                               | 805 |
| <b>FooTypeSupport</b>                                                                                                                                                                         |     |
| <<interface>> <<generic>> User data type specific interface                                                                                                                                   | 818 |
| <b>NDDS_Discovery_Property</b>                                                                                                                                                                |     |
| Specifies internal information used to store discovery information about a <b>DDSDomainParticipant</b> in the registry                                                                        | 824 |
| <b>NDDS_Type_PluginVersion</b>                                                                                                                                                                |     |
| Data type used to keep track of versioning information about a <b>DDS_TypePluginI</b>                                                                                                         | 824 |
| <b>NETIO_Address</b>                                                                                                                                                                          | 825 |
| <b>NETIO_AddressSeq</b>                                                                                                                                                                       |     |
| <<extension>> The <b>NETIO_AddressSeq</b> sequence type                                                                                                                                       | 826 |
| <b>NETIO_AddressUInt32</b>                                                                                                                                                                    |     |
| <<extension>> <b>NETIO_AddressUInt32</b>                                                                                                                                                      | 826 |
| <b>NETIO_AddressValue</b>                                                                                                                                                                     |     |
| <<extension>> <b>NETIO_AddresValue</b>                                                                                                                                                        | 826 |

|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| <a href="#">NETIO_DGRAM_Interface</a>                                      |     |
| << <i>eXtension</i> >> Definition of the NETIO_DGRAM Interface             | 827 |
| <a href="#">NETIO_DGRAM_InterfaceMultiCastGroup</a>                        |     |
| << <i>eXtension</i> >> Definition of a valid multicast group address range | 831 |
| <a href="#">NETIO_DGRAM_InterfaceRouteEntry</a>                            |     |
| << <i>eXtension</i> >> Definition of a route entry                         | 832 |
| <a href="#">NETIO_DGRAM_InterfaceTableEntry</a>                            |     |
| << <i>eXtension</i> >> Definition of an interface                          | 833 |
| <a href="#">NETIO_DGRAM_InterfaceTableEntrySeq</a>                         |     |
| A sequence of NETIO_DGRAMInterfaceTableEntry elements                      | 834 |
| <a href="#">NETIO_Interface</a>                                            |     |
| Base-class definition for all NETIO interfaces                             | 835 |
| <a href="#">NETIO_Interface</a>                                            | 835 |
| <a href="#">NETIO_Netmask</a>                                              |     |
| << <i>eXtension</i> >> NETIO_Netmask                                       | 835 |
| <a href="#">NETIO_NetmaskSeq</a>                                           |     |
| << <i>eXtension</i> >> The NETIO_NetmaskSeq sequence type                  | 836 |
| <a href="#">NETIO_Packet</a>                                               |     |
| Implementation of the abstract NETIO_Packet type                           | 836 |
| <a href="#">NETIO_SharedMemorySegmentHeader</a>                            | 837 |
| <a href="#">NETIO_SHMEMInterfaceFactoryProperty</a>                        |     |
| << <i>eXtension</i> >> Properties for the SHMEM Transport                  | 837 |
| <a href="#">OSAPI_LogProperty</a>                                          |     |
| Configuration of logging functionality                                     | 838 |
| <a href="#">OSAPI_SharedMemorySegmentHandle</a>                            | 839 |
| <a href="#">OSAPI_SharedMemorySegmentHeader</a>                            | 840 |
| <a href="#">OSAPI_SystemI</a>                                              |     |
| System API                                                                 | 841 |
| <a href="#">OSAPI_SystemProperty</a>                                       |     |
| The System Properties                                                      | 842 |
| <a href="#">OSAPI_SystemTime</a>                                           |     |
| System time                                                                | 844 |
| <a href="#">OSAPI_ThreadProperty</a>                                       |     |
| Properties for thread creation                                             | 845 |
| <a href="#">OSAPI_TimerProperty</a>                                        |     |
| Timer properies                                                            | 846 |



|                                                                                                                                                                                            |  |     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-----|
| <b>PSK_ErrListener</b>                                                                                                                                                                     |  |     |
| A structure for holding callback pointers in case of an error                                                                                                                              |  | 846 |
| <b>RHSMHistoryFactory</b>                                                                                                                                                                  |  |     |
| <<eXtension>> A reader history factory that will be used by the DataReader to create a history cache                                                                                       |  | 847 |
| <b>RTPS_Checksum</b>                                                                                                                                                                       |  |     |
| <<eXtension>> RTPS checksum type                                                                                                                                                           |  | 848 |
| <b>RTPS_ChecksumClass</b>                                                                                                                                                                  |  |     |
| <<eXtension>> Definition of a checksum function                                                                                                                                            |  | 848 |
| <b>RTPS_ChecksumProperty</b>                                                                                                                                                               |  |     |
| <<eXtension>> Checksum properties for the RTPS plugin                                                                                                                                      |  | 849 |
| <b>RTPS_InterfaceFactoryProperty</b>                                                                                                                                                       |  |     |
| <<eXtension>> Properties for the RTPS Transport                                                                                                                                            |  | 849 |
| <b>RTRegistry</b>                                                                                                                                                                          |  |     |
| <<eXtension>> A run-time component factory                                                                                                                                                 |  | 850 |
| <b>SHMEMInterfaceFactory</b>                                                                                                                                                               |  |     |
| <<eXtension>> A SHMEM transport factory that will be used by a DomainParticipant to create a SHMEM interface. Only one SHMEM interface can be registered with the DomainParticipantFactory |  | 851 |
| <b>UDP_InterfaceFactoryProperty</b>                                                                                                                                                        |  |     |
| <<eXtension>> Properties for the UDP Transport                                                                                                                                             |  | 852 |
| <b>UDP_InterfaceTableEntry</b>                                                                                                                                                             |  |     |
| Generic structure to describe a network interface                                                                                                                                          |  | 857 |
| <b>UDP_InterfaceTableEntrySeq</b>                                                                                                                                                          |  |     |
| <<eXtension>> Declares IDL sequence<UDP_InterfaceTableEntry>                                                                                                                               |  | 858 |
| <b>UDP_NatEntry</b>                                                                                                                                                                        |  |     |
| <<eXtension>> Properties for a Network Address Translation entry                                                                                                                           |  | 858 |
| <b>UDP_NatEntrySeq</b>                                                                                                                                                                     |  |     |
| <<eXtension>> Declares IDL sequence<UDP_NatEntry>                                                                                                                                          |  | 859 |
| <b>UDPInterfaceFactory</b>                                                                                                                                                                 |  |     |
| <<eXtension>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory       |  | 859 |
| <b>UDPInterfaceTable</b>                                                                                                                                                                   |  | 860 |
| <b>WHSMHistoryFactory</b>                                                                                                                                                                  |  |     |
| <<eXtension>> A writer history factory that will be used by the DataWriter to create a history cache                                                                                       |  | 860 |
| <b>ZCOPY_NotifInterfaceFactoryProperty</b>                                                                                                                                                 |  |     |
| Notification interface property                                                                                                                                                            |  | 861 |
| <b>ZCOPY_NotifMechanismProperty</b>                                                                                                                                                        |  |     |
| Notification mechanism property for the default implementation                                                                                                                             |  | 862 |

[ZCOPY\\_NotifUserInterface](#)

Notification mechanism user interface

863

## 5 File Index

### 5.1 File List

Here is a list of all documented files with brief descriptions:

|                                               |     |
|-----------------------------------------------|-----|
| <a href="#">app_gen_log.h</a>                 | 868 |
| <a href="#">cdr_log.h</a>                     | 869 |
| <a href="#">db_log.h</a>                      |     |
| DB module log codes                           | 869 |
| <a href="#">dds_c_config.h</a>                |     |
| DDS C configuration file                      | 871 |
| <a href="#">dds_c_flowcontroller.h</a>        |     |
| DDS Flow Controller                           | 871 |
| <a href="#">dds_c_log.h</a>                   |     |
| DDS_C module log codes                        | 872 |
| <a href="#">dds_filter_log.h</a>              |     |
| DDS Filter module log codes                   | 898 |
| <a href="#">dds_psk_log.h</a>                 |     |
| DDS_PSK module log codes                      | 901 |
| <a href="#">disc_dpde_log.h</a>               | 902 |
| <a href="#">disc_dpse_log.h</a>               | 905 |
| <a href="#">netio_log.h</a>                   |     |
| NETIO module log codes                        | 911 |
| <a href="#">netio_shmem_log.h</a>             |     |
| NETIO Zero Copy Shared Data module log codes  | 918 |
| <a href="#">netio_zcopy_log.h</a>             |     |
| ZeroCopy module log codes                     | 920 |
| <a href="#">netio_zcopy_notif_interface.h</a> | 920 |
| <a href="#">netio_zcopy_whsq_log.h</a>        |     |
| WHSQ module log codes                         | 922 |
| <a href="#">osapi_log.h</a>                   |     |
| OSAPI Log API                                 | 923 |
| <a href="#">osapi_process.h</a>               |     |
| Process related utility                       | 933 |

|                                                      |     |
|------------------------------------------------------|-----|
| <a href="#">osapi_semaphore.h</a>                    |     |
| Semaphore interface definition                       | 933 |
| <a href="#">osapi_stdio.h</a>                        |     |
| Standard I/O interface definition                    | 935 |
| <a href="#">osapi_task.h</a>                         |     |
| Task interface definition                            | 936 |
| <a href="#">osapi_time.h</a>                         |     |
| Time API definition                                  | 936 |
| <a href="#">reda_log.h</a>                           |     |
| REDA module log codes                                | 937 |
| <a href="#">rh_sm_log.h</a>                          |     |
| RH module log codes                                  | 938 |
| <a href="#">rt_log.h</a>                             | 942 |
| <a href="#">rtps_config.h</a>                        |     |
| RTPS Configuration                                   | 943 |
| <a href="#">rtps_dll.h</a>                           |     |
| RTPS Configuration                                   | 943 |
| <a href="#">rtps_log.h</a>                           |     |
| RTPS module log codes                                | 943 |
| <a href="#">rtps_rtps_impl.h</a>                     |     |
| Implementation of RTPS interface functions and types | 951 |
| <a href="#">wh_sm_log.h</a>                          |     |
| WH module log codes                                  | 951 |

## 6 Topic Documentation

### 6.1 Documentation Guide

This section describes the conventions used in the API documentation.

#### 6.1.1 Unsupported Features

[**Not supported.**] This note means that the feature from the DDS specification is not supported in the current release.

## 6.1.2 API Naming Conventions

### 6.1.2.1 Structure & Class Names

RTI Connex DDS Micro makes a distinction between *value types* and *interface types*. Value types are types such as primitives, enumerations, strings, and structures whose identity and equality are determined solely by explicit state. Interface types are those abstract opaque data types that conceptually have an identity apart from their explicit state. Examples include all of the [DDSEntity](#) subtypes. Instances of value types are frequently transitory and are declared on the stack. Instances of interface types typically have longer lifecycles, are accessible by pointer only, and may be managed by a factory object.

Value type structures are made more explicit through the use of C's structure tag syntax. For example, a [DDS\\_Duration\\_t](#) object must be declared as being of type "struct DDS\_Duration\_t," not simply "DDS\_Duration\_t." Interface types, by contrast, are always of typedef'd types; their underlying representations are opaque.

### 6.1.3 API Documentation Terms

In the API documentation, the term module refers to a logical grouping of documentation and elements in the API.

At this time, typedefs that occur in the API, such as [DDS\\_ReturnCode\\_t](#) do not show up in the compound list or indices. This is a known limitation in the generated HTML.

## 6.1.4 Stereotypes

Commonly used stereotypes in the API documentation include the following.

### 6.1.4.1 Extensions

- [`<<eXtension>>`](#)
  - An RTI Connex DDS Micro product extension to the DDS standard specification.
  - The extension APIs complement the standard APIs specified by the OMG DDS specification. They are provided to improve product usability and enable access to product-specific features.

### 6.1.4.2 Experimental

- [`<<experimental>>`](#)
  - This software may contain experimental features. These are used to evaluate potential new features and obtain customer feedback.
  - Experimental APIs are marked as [`<<experimental>>`](#) in this documentation.
  - Experimental features may be only available in a subset of the supported languages and for a subset of the supported platforms.
  - Experimental features may change in the future.
  - Experimental features may or may not appear in future product releases.
  - Experimental features should not be used in production.
  - Please submit your comments and suggestions about experimental features to [support@rti.com](mailto:support@rti.com) or via the RTI Customer Portal ( <https://support.rti.com/> ).

### 6.1.4.3 Types

- `<<interface>>`
  - Pure interface type with *no state*.
- `<<generic>>`
  - A *generic* type is a *skeleton* class written in terms of generic parameters. Type-specific instantiations of such types are conventionally referred to in this documentation in terms of the hypothetical type "Foo"; for example: [FooSeq](#), [FooDataType](#), [FooDataWriter](#), and [FooDataReader](#).
- `<<singleton>>`
  - Singleton class. There is a single instance of the class.
  - Generally accessed via a `get_instance()` static method.

### 6.1.4.4 Method Parameters

- `<<in>>`
  - An *input* parameter.
- `<<out>>`
  - An *output* parameter.
- `<<inout>>`
  - An *input* and *output* parameter.

### 6.1.4.5 Cert API

- – Supported in safety certified version of RTI Connex DDS Micro. Note, specific fields of data structures may not be supported and will be described in documentation as unsupported.

## 6.2 DDS API

RTI Connex DDS Micro modules following the DDS module definitions.

## Topics

- [Content Filter Plugin](#)  
*<<experimental>> Content Filter Plugin*
- [Domain Module](#)  
*Contains the [DDSDomainParticipant](#) class that acts as an endpoint of RTI Connext DDS Micro and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up RTI Connext DDS Micro. Testing RTI Connext DDS Micro command.*
- [Topic](#)  
*Contains the [DDSTopic](#), the [DDSTopicListener](#) interface, and more generally, all that is needed by an application to define [DDSTopic](#) objects and the data types that are associated with [DDSTopic](#) objects.*
- [Publication Module](#)  
*Contains the [DDSPublisher](#), and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.*
- [Subscription Module](#)  
*Contains the [DDSSubscriber](#), and [DDSDataReader](#) classes, and more generally, all that is needed on the subscription side.*
- [Infrastructure Module](#)  
*Defines the DDS infrastructure package.*
- [General Utilities and Compliance Configuration](#)

### 6.2.1 Detailed Description

RTI Connext DDS Micro modules following the DDS module definitions.

### 6.2.2 Overview

Information flows with the aid of the following constructs: [DDSPublisher](#) and [DDSDataWriter](#) on the sending side, [DDSSubscriber](#) and [DDSDataReader](#) on the receiving side.

- A [DDSPublisher](#) is an object responsible for data distribution. It may publish data of different data types. A [DDSDataWriter](#) acts as an accessor to a publisher - each [DDSDataWriter](#) object is dedicated to one application data type. A [DDSDataWriter](#) is the object the application must use to communicate to a publisher the existence and value of data-objects of a given type. When data-object values have been communicated to the publisher through the appropriate data-writer, it is the publisher's responsibility to perform the distribution (the publisher will do this according to its own QoS, or the QoS attached to the corresponding data-writer). A *publication* is defined by the association of a data-writer to a publisher. This association expresses the intent of the application to publish the data described by the data-writer in the context provided by the publisher.
- A [DDSSubscriber](#) is an object responsible for receiving published data and making it available (according to the Subscriber's QoS) to the receiving application. It may receive and dispatch data of different specified types. To access the received data, the application must use a [DDSDataReader](#) attached to the subscriber. Thus, a *subscription* is defined by the association of a data-reader with a subscriber. This association expresses the intent of the application to subscribe to the data described by the data-reader in the context provided by the subscriber.

**DDSTopic** objects conceptually fit between publications and subscriptions. Publications must be known in such a way that subscriptions can refer to them unambiguously. A **DDSTopic** is meant to fulfill that purpose: it associates a name (unique in the domain i.e. the set of applications that are communicating with each other), a data type, and QoS related to the data itself. In addition to the topic QoS, the QoS of the **DDSDataWriter** associated with that Topic and the QoS of the **DDSPublisher** associated to the **DDSDataWriter** control the behavior on the publisher's side, while the corresponding **DDSTopic**, **DDSDataReader** and **DDSSubscriber** QoS control the behavior on the subscriber's side.

When an application wishes to publish data of a given type, it must create a **DDSPublisher** (or reuse an already created one) and a **DDSDataWriter** with all the characteristics of the desired publication. Similarly, when an application wishes to receive data, it must create a **DDSSubscriber** (or reuse an already created one) and a **DDSDataReader** to define the subscription.

### 6.2.3 Conceptual Model

The overall conceptual model is shown below.

Notice that all the main communication objects (the specializations of Entity) follow unified patterns of:

- Supporting QoS (made up of several QoSPolicy); QoS provides a generic mechanism for the application to control the behavior of the Service and tailor it to its needs. Each **DDSEntity** supports its own specialized kind of QoS policies (see **QoS Policies**).
- Accepting a **DDSListener**; listeners provide a generic mechanism for the middleware to notify the application of relevant asynchronous events, such as arrival of data corresponding to a subscription, violation of a QoS setting, etc. Each **DDSEntity** supports its own specialized kind of listener. Listeners are related to changes in status conditions (see **Status Kinds**).

Note that only one Listener per entity is allowed (instead of a list of them). The reason for that choice is that this allows a much simpler (and, thus, more efficient) implementation as far as the middleware is concerned. Moreover, if it were required, the application could easily implement a listener that, when triggered, triggers in return attached 'sub-listeners'.

All DCPS entities are attached to a **DDSDomainParticipant**. A domain participant represents the local membership of the application in a domain. A *domain* is a distributed concept that links all the applications able to communicate with each other. It represents a communication plane: only the publishers and the subscribers attached to the same domain may interact.

**DDSDomainEntity** is an intermediate object whose only purpose is to state that a DomainParticipant cannot contain other domain participants.

At the DCPS level, data types represent information that is sent atomically. For performance reasons, only plain data structures are handled by this level.

By definition, a **DDSTopic** corresponds to a single data type. However, several topics may refer to the same data type. Therefore, a **DDSTopic** identifies data of a single type, ranging from one single instance to a whole collection of instances of that given type. This is shown below for the hypothetical data type **Foo**.

In case a set of instances is gathered under the same topic, different instances must be distinguishable. This is achieved by means of the values of some data fields that form the **key** to that data set. The *key description* (i.e., the list of data fields whose value forms the key) has to be indicated to the middleware. The rule is simple: *different data samples with the same key value represent successive values for the same instance, while different data samples with different key*

*values represent different instances.* If no key is provided, the data set associated with the [DDSTopic](#) is restricted to a *single instance*.

Topics need to be known by the middleware and potentially propagated. Topic objects are created using the create operations provided by [DDSDomainParticipant](#).

The interaction style is straightforward on the publisher's side: when the application decides that it wants to make data available for publication, it calls the appropriate operation on the related [DDSDataWriter](#) (this, in turn, will trigger its [DDSPublisher](#)).

On the subscriber's side however, there are more choices: relevant information may arrive when the application is busy doing something else or when the application is just waiting for that information. Therefore, depending on the way the application is designed, asynchronous notifications or synchronous access may be more appropriate. Both interaction modes are allowed, a [DDSListener](#) is used to provide a callback for asynchronous access, and polling allows for synchronous access.

#### 6.2.4 Modules

DCPS consists of five modules:

- [Infrastructure module](#) defines the abstract classes and the interfaces that are refined by the other modules. It also provides support for the notification-based interaction style with the middleware.
- [Domain module](#) contains the [DDSDomainParticipant](#) class that acts as an entrypoint of the Service and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up the Service.
- [Topic module](#) contains the [DDSTopic](#) class, and more generally, all that is needed by the application to define [DDSTopic](#) objects and attach QoS policies to them.
- [Publication module](#) contains the [DDSPublisher](#) and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.
- [Subscription module](#) contains the [DDSSubscriber](#), and [DDSDataReader](#), and more generally, all that is needed on the subscription side.

#### 6.2.5 Content Filter Plugin

<<experimental>> Content Filter Plugin

##### Macros

- `#define DDS_SQL_CONTENT_FILTER_CLASS "DDSSQL"`  
*The name of the SQL content filter class.*

##### 6.2.5.1 Detailed Description

<<experimental>> Content Filter Plugin

This module provides the content filter plugin to enable content filtering functionality in RTI Connex DDS Micro.



### 6.2.5.2 Macro Definition Documentation

#### DDS\_SQL\_CONTENT\_FILTER\_CLASS

```
#define DDS_SQL_CONTENT_FILTER_CLASS "DDSSQL"
```

The name of the SQL content filter class.

### 6.2.6 Domain Module

Contains the [DDSDomainParticipant](#) class that acts as an entrypoint of RTI Connex DDS Micro and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up RTI Connex DDS Micro. Testing RTI Connex DDS Micro command.

#### Topics

- [DomainParticipantFactory](#)  
*DDSDomainParticipantFactory entity and associated elements*
- [DomainParticipant](#)  
*DDSDomainParticipant entity and associated elements*
- [Discovery](#)  
*Descriptions of remote entities used for discovery.*

#### Macros

- #define [DDSTheParticipantFactory](#) [DDSDomainParticipantFactory::get\\_instance\(\)](#)  
*Can be used as an alias for the singleton factory returned by the operation [DDSDomainParticipantFactory::get\\_instance\(\)](#).*

#### Typedefs

- typedef DDS\_DOMAINID\_TYPE\_NATIVE [DDS\\_DomainId\\_t](#)  
*An integer that indicates in which domain a [DDSDomainParticipant](#) communicates.*

#### 6.2.6.1 Detailed Description

Contains the [DDSDomainParticipant](#) class that acts as an entrypoint of RTI Connex DDS Micro and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up RTI Connex DDS Micro. Testing RTI Connex DDS Micro command.

Defines the DDS domain package.

This image describes the DDS Domain Package, as described by the DDS standard. Note that not all functions and objects shown in this diagram are supported by RTI Connex DDS Micro. Please read the information on the individual types to determine whether a function is supported.

### 6.2.6.2 Macro Definition Documentation

#### DDSTheParticipantFactory

```
#define DDSTheParticipantFactory DDSDomainParticipantFactory::get_instance()
```

Can be used as an alias for the singleton factory returned by the operation [DDSDomainParticipantFactory::get\\_instance\(\)](#).

See also

[DDSDomainParticipantFactory::get\\_instance](#)

### 6.2.6.3 Typedef Documentation

#### DDS\_DomainId\_t

```
typedef DDS_DOMAINID_TYPE_NATIVE DDS_DomainId_t
```

An integer that indicates in which domain a [DDSDomainParticipant](#) communicates.

Participants with the same [DDS\\_DomainId\\_t](#) are said to be in the same domain, and can thus communicate with one another.

The lower limit for a domain ID is 0. The upper limit for a domain ID is determined by the guidelines of the RTPS specification of the well-known ports on which a Participant will listen for discovery metatraffic from other Participants.

### 6.2.6.4 DomainParticipantFactory

[DDSDomainParticipantFactory](#) entity and associated elements

#### Classes

- struct [DDS\\_DomainParticipantFactoryQos](#)  
*QoS policies supported by a [DDSDomainParticipantFactory](#).*
- class [DDSDomainParticipantFactory](#)  
*<<singleton>> <<interface>> Allows creation and destruction of [DDSDomainParticipant](#) objects.*

#### Variables

- const struct [DDS\\_DomainParticipantFactoryQos](#) [DDS\\_PARTICIPANT\\_FACTORY\\_QOS\\_DEFAULT](#)  
*Special value for creating domain participant with default QoS.*
- const struct [DDS\\_DomainParticipantQos](#) [DDS\\_PARTICIPANT\\_QOS\\_DEFAULT](#)  
*Special value for creating domain participant with default QoS.*

#### 6.2.6.4.1 Detailed Description

[DDSDomainParticipantFactory](#) entity and associated elements

#### 6.2.6.4.2 Variable Documentation

##### DDS\_PARTICIPANT\_FACTORY\_QOS\_DEFAULT

```
const struct DDS_DomainParticipantFactoryQos DDS_PARTICIPANT_FACTORY_QOS_DEFAULT [extern]
```

Special value for creating domain participant with default QoS.

When used in [DDSDomainParticipantFactory::create\\_participant](#), this special value is used to indicate that the [DDSDomainParticipant](#) should be created with the default [DDSDomainParticipant](#) QoS.

See also

[DDSDomainParticipantFactory::create\\_participant\(\)](#)

##### DDS\_PARTICIPANT\_QOS\_DEFAULT

```
const struct DDS_DomainParticipantQos DDS_PARTICIPANT_QOS_DEFAULT [extern]
```

Special value for creating domain participant with default QoS.

When used in [DDSDomainParticipantFactory::create\\_participant](#), this special value is used to indicate that the [DDSDomainParticipant](#) should be created with the default [DDSDomainParticipant](#) QoS.

See also

[DDSDomainParticipantFactory::create\\_participant\(\)](#)

#### 6.2.6.5 DomainParticipant

[DDSDomainParticipant](#) entity and associated elements

##### Classes

- struct [DDS\\_DomainParticipantQos](#)  
*QoS policies supported by a [DDSDomainParticipant](#) entity.*
- class [DDSDomainParticipantListener](#)  
*<<interface>> Listener for participant status.*
- class [DDSDomainParticipant](#)  
*<<interface>> Container for all [DDSDomainEntity](#) objects.*

## Variables

- const struct [DDS\\_TopicQos](#) [DDS\\_TOPIC\\_QOS\\_DEFAULT](#)  
*Special value for creating a [DDSTopic](#) with default QoS.*
- const struct [DDS\\_PublisherQos](#) [DDS\\_PUBLISHER\\_QOS\\_DEFAULT](#)  
*Special value for creating a [DDSPublisher](#) with default QoS.*
- const struct [DDS\\_SubscriberQos](#) [DDS\\_SUBSCRIBER\\_QOS\\_DEFAULT](#)  
*Special value for creating a [DDSSubscriber](#) with default QoS.*
- const struct [DDS\\_FlowControllerProperty\\_t](#) [DDS\\_FLOW\\_CONTROLLER\\_PROPERTY\\_DEFAULT](#)  
*<<eXtension>> Special value for creating a [DDSFlowController](#) with default property.*

### 6.2.6.5.1 Detailed Description

[DDSDomainParticipant](#) entity and associated elements

### 6.2.6.5.2 Variable Documentation

#### DDS\_TOPIC\_QOS\_DEFAULT

```
const struct DDS\_TopicQos DDS_TOPIC_QOS_DEFAULT [extern]
```

Special value for creating a [DDSTopic](#) with default QoS.

When used in [DDSDomainParticipant::create\\_topic](#), this special value indicates that the [DDSTopic](#) should be created with the default [DDSTopic](#) QoS.

See also

[DDSDomainParticipant::create\\_topic](#)

#### DDS\_PUBLISHER\_QOS\_DEFAULT

```
const struct DDS\_PublisherQos DDS_PUBLISHER_QOS_DEFAULT [extern]
```

Special value for creating a [DDSPublisher](#) with default QoS.

When used in [DDSDomainParticipant::create\\_publisher](#), this special value is used to indicate that the [DDSPublisher](#) should be created with the default [DDSPublisher](#) QoS.

See also

[DDSDomainParticipant::create\\_publisher](#)

## DDS\_SUBSCRIBER\_QOS\_DEFAULT

```
const struct DDS_SubscriberQos DDS_SUBSCRIBER_QOS_DEFAULT [extern]
```

Special value for creating a [DDSSubscriber](#) with default QoS.

When used in [DDSDomainParticipant::create\\_subscriber](#), this special value is used to indicate that the [DDSSubscriber](#) should be created with the default [DDSSubscriber](#) QoS.

See also

[DDSDomainParticipant::create\\_subscriber](#)

## DDS\_FLOW\_CONTROLLER\_PROPERTY\_DEFAULT

```
const struct DDS_FlowControllerProperty_t DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT [extern]
```

<<*eXtension*>> Special value for creating a [DDSFlowController](#) with default property.

When used in [DDSDomainParticipant::create\\_flowcontroller](#), this special value is used to indicate that the [DDSFlowController](#) should be created with the default [DDSFlowController](#) property by means of the operation `get_↔default_flowcontroller_property` and using the resulting QoS to create the [DDS\\_FlowControllerProperty\\_t](#).

Note: You cannot use this value to *get* the default properties for a FlowController; for this purpose, use [DDSDomainParticipant::get\\_default\\_flowcontroller\\_property](#).

See also

[DDSDomainParticipant::create\\_flowcontroller](#)

[DDSDomainParticipant::set\\_default\\_flowcontroller\\_property](#)

[DDSFlowController::set\\_property](#)

### 6.2.6.6 Discovery

Descriptions of remote entities used for discovery.

#### Topics

- [Participant Built-in Topic](#)

*Builtin topic for accessing information about the DomainParticipants discovered by RTI Connex DDS Micro.*

- [Publication Built-in Topic](#)

*Builtin topic for configuring information about the Publications to be discovered by RTI Connex DDS Micro.*

- [Subscription Built-in Topic](#)

*Builtin topic for configuring information about the Subscriptions to be discovered by RTI Connex DDS Micro.*

## Classes

- struct [DDS\\_ContentFilterProperty](#)  
All of the information necessary for a [DDSDataWriter](#) to perform writer-side filtering for a [DDSDataReader](#).
- struct [DDS\\_Locator\\_t](#)  
<<eXtension>> Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports.
- struct [DDS\\_LocatorSeq](#)  
Declares IDL sequence < [DDS\\_Locator\\_t](#) >
- struct [DDS\\_ProtocolVersion](#)  
<<eXtension>> Type used to represent the version of the RTPS protocol.
- struct [DDS\\_VendorId](#)  
<<eXtension>> Type used to represent the vendor of the service implementing the RTPS protocol.
- struct [DDS\\_VendorId\\_t](#)  
<<eXtension>> Type used to represent the vendor of the service implementing the RTPS protocol.
- struct [DDS\\_ProductVersion](#)  
<<eXtension>> Type used to represent the current version of RTI Connex DDS Micro.
- struct [DDS\\_ProductVersion\\_t](#)  
<<eXtension>> Type used to represent the current version of RTI Connex DDS Micro.
- struct [DDS\\_BuiltinTopicKey\\_t](#)  
The key type of the built-in topic types.

## Macros

- #define [DDS\\_LOCATOR\\_INVALID](#) RTPS\_LOCATOR\_INVALID  
An invalid locator.
- #define [DDS\\_LOCATOR\\_KIND\\_INVALID](#)  
Locator of this kind is invalid.
- #define [DDS\\_LOCATOR\\_PORT\\_INVALID](#)  
An invalid port.
- #define [DDS\\_LOCATOR\\_ADDRESS\\_INVALID](#) RTPS\_LOCATOR\_ADDRESS\_INVALID  
An invalid address.
- #define [DDS\\_LOCATOR\\_KIND\\_UDPv4](#)  
A locator for a UDPv4 address.
- #define [DDS\\_LOCATOR\\_KIND\\_UDPv6](#)  
A locator for a UDPv6 address.
- #define [DDS\\_LOCATOR\\_KIND\\_RESERVED](#)  
Locator of this kind is reserved.
- #define [DDS\\_LOCATOR\\_KIND\\_SHMEM](#)  
A locator for an address accessed via shared memory.
- #define [DDS\\_PROTOCOLVERSION\\_1\\_0](#) { 1, 0 }  
The protocol version 1.0.
- #define [DDS\\_PROTOCOLVERSION\\_1\\_1](#) { 1, 1 }  
The protocol version 1.1.
- #define [DDS\\_PROTOCOLVERSION\\_1\\_2](#) { 1, 2 }  
The protocol version 1.2.
- #define [DDS\\_PROTOCOLVERSION\\_2\\_0](#) { 2, 0 }

- *The most recent protocol version. Currently 1.2.*  
• #define `DDS_PROTOCOLVERSION_2_1` { 2, 1 }
- *The protocol version 2.1.*  
• #define `DDS_PROTOCOLVERSION` { 2, 5 }
- *The most recent protocol version. Currently 1.2.*  
• #define `DDS_VENDOR_ID_LENGTH_MAX` 2
- *Length of vendor id.*  
• #define `DDS_PRODUCTVERSION_UNKNOWN` { 0, 0, '0', 0 }
- *The value used when the product version is unknown.*

## Typedefs

- typedef struct `DDS_ProtocolVersion` `DDS_ProtocolVersion_t`  
    <<eXtension>> Type used to represent the version of the RTPS protocol.
- typedef struct `DDS_BuiltinTopicKey_t` `DDS_BuiltinTopicKey_t`  
    The key type of the built-in topic types.

### 6.2.6.6.1 Detailed Description

Descriptions of remote entities used for discovery.

Given a DDS entity, a remote entity is any other entity that can be discovered and matched. A built-in object is a description of a remote entity, including all the QoS which are important for discovery and matching of remote readers and writers with local readers and writers.

RTI Connex DDS Micro must discover and keep track of the remote entities, such as new participants in the domain. Depending on the type of discovery used, this information about remote entities is gathered in three ways:

1. No data is sent over the network (static discovery)
2. A subset is sent over the network (dynamic participant/static endpoint discovery), while the rest is statically asserted.
3. All data about DomainParticipants DataReaders and DataWriters is over the network (simple discovery)

In the first case, all information about remote DomainParticipants DataReaders and DataWriters is statically configured by the user before starting the middleware. To do this static configuration, the user must fill in the built-in structures for all remote entities it wishes to discover, and must assert them to the middleware. This type of discovery is not supported in this version of RTI Connex DDS Micro.

In the second case, the participant data is sent over the network, but the endpoint data is statically configured. So, the local application fills in builtin structures representing the QoS and object IDs all the remote readers and writers it wishes to discover. For this type of discovery the user does not configure builtin data for remote participants. This type of discovery is faster than Simple Discovery, and does not require reliable communication. This is supported by RTI Connex DDS Micro and is known as Dynamic Participant, Static Endpoint (DPSE) discovery.

In the last case, the user does not need to fill in any data about remote entities, because it is automatically sent over the network by the middleware. The user can access the data through special listeners for discovery traffic. This is supported by RTI Connex DDS Micro and is known as Dynamic Participant, Dynamic Endpoint (DPDE) discovery.

Refer to `DDS_ParticipantBuiltinTopicData`, `DDS_SubscriptionBuiltinTopicData` and `DDS_PublicationBuiltinTopicData` for a description of all the built-in topics and their contents.

<<eXtension>> Dynamic Participant, Static Endpoint Discovery plugin mechanism for RTI Connex DDS Micro.

### 6.2.6.6.2 Macro Definition Documentation

#### DDS\_LOCATOR\_INVALID

```
#define DDS_LOCATOR_INVALID RTPS_LOCATOR_INVALID
```

An invalid locator.

#### DDS\_LOCATOR\_KIND\_INVALID

```
#define DDS_LOCATOR_KIND_INVALID
```

Locator of this kind is invalid.

#### DDS\_LOCATOR\_PORT\_INVALID

```
#define DDS_LOCATOR_PORT_INVALID
```

An invalid port.

#### DDS\_LOCATOR\_ADDRESS\_INVALID

```
#define DDS_LOCATOR_ADDRESS_INVALID RTPS_LOCATOR_ADDRESS_INVALID
```

An invalid address.

#### DDS\_LOCATOR\_KIND\_UDPv4

```
#define DDS_LOCATOR_KIND_UDPv4
```

A locator for a UDPv4 address.

#### DDS\_LOCATOR\_KIND\_UDPv6

```
#define DDS_LOCATOR_KIND_UDPv6
```

A locator for a UDPv6 address.

#### DDS\_LOCATOR\_KIND\_RESERVED

```
#define DDS_LOCATOR_KIND_RESERVED
```

Locator of this kind is reserved.



**DDS\_LOCATOR\_KIND\_SHMEM**

```
#define DDS_LOCATOR_KIND_SHMEM
```

A locator for an address accessed via shared memory.

**DDS\_PROTOCOLVERSION\_1\_0**

```
#define DDS_PROTOCOLVERSION_1_0 { 1, 0 }
```

The protocol version 1.0.

**DDS\_PROTOCOLVERSION\_1\_1**

```
#define DDS_PROTOCOLVERSION_1_1 { 1, 1 }
```

The protocol version 1.1.

**DDS\_PROTOCOLVERSION\_1\_2**

```
#define DDS_PROTOCOLVERSION_1_2 { 1, 2 }
```

The protocol version 1.2.

**DDS\_PROTOCOLVERSION\_2\_0**

```
#define DDS_PROTOCOLVERSION_2_0 { 2, 0 }
```

The most recent protocol version. Currently 1.2.

**DDS\_PROTOCOLVERSION\_2\_1**

```
#define DDS_PROTOCOLVERSION_2_1 { 2, 1 }
```

The protocol version 2.1.

**DDS\_PROTOCOLVERSION**

```
#define DDS_PROTOCOLVERSION { 2, 5 }
```

The most recent protocol version. Currently 1.2.

**DDS\_VENDOR\_ID\_LENGTH\_MAX**

```
#define DDS_VENDOR_ID_LENGTH_MAX 2
```

Length of vendor id.

**DDS\_PRODUCTVERSION\_UNKNOWN**

```
#define DDS_PRODUCTVERSION_UNKNOWN { 0, 0, '0', 0 }
```

The value used when the product version is unknown.

**6.2.6.6.3 Typedef Documentation****DDS\_ProtocolVersion\_t**

```
typedef struct DDS_ProtocolVersion DDS_ProtocolVersion_t
```

<<*eXtension*>> Type used to represent the version of the RTPS protocol.

**DDS\_BuiltinTopicKey\_t**

```
typedef struct DDS_BuiltinTopicKey_t DDS_BuiltinTopicKey_t
```

The key type of the built-in topic types.

Each remote [DDSEntity](#) to be discovered can be uniquely identified by this key. This is the key of all the built-in topic data types.

The value of element `DDS_BUILTIN_TOPIC_KEY_OBJECT_ID` is the object ID of the remote entity. In Dynamic Participant/Static Endpoint discovery, this must be set correctly for each remote [DDSDataReader](#) and [DDSDataWriter](#) that an application intends to discover.

For example, to configure a remote [DDSDataWriter](#) with object ID 100 by setting the key in the [DDS\\_PublicationBuiltinTopicData](#)↔:

```
remote_pub_data.key.value[DDS_BUILTIN_TOPIC_KEY_OBJECT_ID] = 100;
```

See also

[DDS\\_ParticipantBuiltinTopicData](#)

[DDS\\_PublicationBuiltinTopicData](#)

[DDS\\_SubscriptionBuiltinTopicData](#)

**6.2.6.6.4 Participant Built-in Topic**

Builtin topic for accessing information about the DomainParticipants discovered by RTI Connex DDS Micro.

## Classes

- struct [DDS\\_ParticipantBuiltinTopicData](#)  
*Object representing a remote DomainParticipant.*

## Macros

- #define [DDS\\_PARTICIPANT\\_TOPIC\\_NAME](#) DDS\_PARTICIPANT\_BUILTIN\_TOPIC\_NAME  
*Participant topic name.*

## Functions

- [DDS\\_ParticipantBuiltinTopicData](#) (const [DDS\\_DomainParticipantQos](#) &dp\_qos)  
*A Constructor that uses the domain participants resource limits to initialize the object memory.*
- [DDS\\_ParticipantBuiltinTopicData](#) ()  
*Default constructor.*
- [DDS\\_Boolean](#) [DDS\\_ParticipantBuiltinTopicData\\_finalize](#) (struct [DDS\\_ParticipantBuiltinTopicData](#) \*self)  
*Finalize the contents of this ParticipantBuiltinTopicData.*
- [DDS\\_Boolean](#) [DDS\\_ParticipantBuiltinTopicData\\_is\\_equal](#) (const struct [DDS\\_ParticipantBuiltinTopicData](#) \*left, const struct [DDS\\_ParticipantBuiltinTopicData](#) \*right)  
*Compare two ParticipantBuiltinTopicData structures for equality.*

## Detailed Description

Builtin topic for accessing information about the DomainParticipants discovered by RTI Connex DDS Micro.

## Macro Definition Documentation

### DDS\_PARTICIPANT\_TOPIC\_NAME

```
#define DDS_PARTICIPANT_TOPIC_NAME DDS_PARTICIPANT_BUILTIN_TOPIC_NAME
```

Participant topic name.

Topic name of participant builtin topic data datareader.

## See also

[DDS\\_ParticipantBuiltinTopicData](#)

## Function Documentation

### DDS\_ParticipantBuiltinTopicData() [1/2]

```
DDS_ParticipantBuiltinTopicData (  
    const DDS\_DomainParticipantQos & dp_qos)
```

A Constructor that uses the domain participants resource limits to initialize the object memory.

## Parameters

|           |               |                                               |
|-----------|---------------|-----------------------------------------------|
| <i>in</i> | <i>dp_qos</i> | The resource limits to initialize the object. |
|-----------|---------------|-----------------------------------------------|

**DDS\_ParticipantBuiltinTopicData()** [2/2]

```
DDS_ParticipantBuiltinTopicData ()
```

Default constructor.

**DDS\_ParticipantBuiltinTopicData\_finalize()**

```
DDS_Boolean DDS_ParticipantBuiltinTopicData_finalize (
    struct DDS_ParticipantBuiltinTopicData * self)
```

Finalize the contents of this ParticipantBuiltinTopicData.

Frees all memory owned by the ParticipantBuiltinTopicData which was allocated through the standard DDS APIs, such as `initialize_from_qos`, `copy`, or `sequence set_maximum`. If you have allocated memory for any attribute in this object using non-standard APIs, then the attribute must be returned to its initialized state before calling `finalize`. Any memory allocated using non-standard APIs is the responsibility of the user to manage.

## Returns

One of the [Standard Return Codes](#)

**DDS\_ParticipantBuiltinTopicData\_is\_equal()**

```
DDS_Boolean DDS_ParticipantBuiltinTopicData_is_equal (
    const struct DDS_ParticipantBuiltinTopicData * left,
    const struct DDS_ParticipantBuiltinTopicData * right)
```

Compare two ParticipantBuiltinTopicData structures for equality.

This method compares two ParticipantBuiltinTopicData structures for equality. The comparison is performed field by field.

## Parameters

|              |                                                    |
|--------------|----------------------------------------------------|
| <i>left</i>  | << <i>in</i> >>. The left side of the comparison.  |
| <i>right</i> | << <i>in</i> >>. The right side of the comparison. |

## Returns

DDS\_BOOLEAN\_TRUE if the two structures are equal, DDS\_BOOLEAN\_FALSE otherwise.

References [RTI\\_UINT32](#).

#### 6.2.6.6.5 Publication Built-in Topic

Builtin topic for configuring information about the Publications to be discovered by RTI Connex DDS Micro.

##### Classes

- struct [DDS\\_PublicationBuiltinTopicData](#)  
*Object describing a remote Publication.*

##### Macros

- #define [DDS\\_PUBLICATION\\_TOPIC\\_NAME](#) DDS\_PUBLICATION\_BUILTIN\_TOPIC\_NAME  
*Publication topic name.*

##### Functions

- [DDS\\_PublicationBuiltinTopicData](#) (const [DDS\\_DomainParticipantQos](#) &dp\_qos)  
*A Constructor that uses the domain participants resource limits to initialize the object memory.*
- [DDS\\_PublicationBuiltinTopicData](#) ()  
*Default constructor.*
- [DDS\\_Boolean DDS\\_PublicationBuiltinTopicData\\_finalize](#) (struct [DDS\\_PublicationBuiltinTopicData](#) \*self)  
*Finalize the contents of this PublicationBuiltinTopicData.*
- [DDS\\_Boolean DDS\\_PublicationBuiltinTopicData\\_is\\_equal](#) (const struct [DDS\\_PublicationBuiltinTopicData](#) \*left, const struct [DDS\\_PublicationBuiltinTopicData](#) \*right)  
*Compare two PublicationBuiltinTopicData structures for equality.*

##### Detailed Description

Builtin topic for configuring information about the Publications to be discovered by RTI Connex DDS Micro.

##### Macro Definition Documentation

###### DDS\_PUBLICATION\_TOPIC\_NAME

```
#define DDS_PUBLICATION_TOPIC_NAME DDS_PUBLICATION_BUILTIN_TOPIC_NAME
```

Publication topic name.

Topic name of publication builtin topic data DataReader.

See also

[DDS\\_PublicationBuiltinTopicData](#)

##### Function Documentation

###### DDS\_PublicationBuiltinTopicData() [1/2]

```
DDS_PublicationBuiltinTopicData (  
    const DDS\_DomainParticipantQos & dp_qos)
```

A Constructor that uses the domain participants resource limits to initialize the object memory.

## Parameters

|           |               |                                               |
|-----------|---------------|-----------------------------------------------|
| <i>in</i> | <i>dp_qos</i> | The resource limits to initialize the object. |
|-----------|---------------|-----------------------------------------------|

**DDS\_PublicationBuiltinTopicData()** [2/2]

```
DDS_PublicationBuiltinTopicData ()
```

Default constructor.

**DDS\_PublicationBuiltinTopicData\_finalize()**

```
DDS_Boolean DDS_PublicationBuiltinTopicData_finalize (
    struct DDS_PublicationBuiltinTopicData * self)
```

Finalize the contents of this PublicationBuiltinTopicData.

Frees all memory owned by the PublicationBuiltinTopicData which was allocated through the standard DDS APIs, such as `initialize_from_qos`, `copy`, or `sequence set_maximum`. If you have allocated memory for any attribute in this object using non-standard APIs, then the attribute must be returned to its initialized state before calling `finalize`. Any memory allocated using non-standard APIs is the responsibility of the user to manage.

## Returns

One of the [Standard Return Codes](#)

**DDS\_PublicationBuiltinTopicData\_is\_equal()**

```
DDS_Boolean DDS_PublicationBuiltinTopicData_is_equal (
    const struct DDS_PublicationBuiltinTopicData * left,
    const struct DDS_PublicationBuiltinTopicData * right)
```

Compare two PublicationBuiltinTopicData structures for equality.

This method compares two PublicationBuiltinTopicData structures for equality. The comparison is performed field by field.

## Parameters

|              |                                                    |
|--------------|----------------------------------------------------|
| <i>left</i>  | << <i>in</i> >>. The left side of the comparison.  |
| <i>right</i> | << <i>in</i> >>. The right side of the comparison. |

## Returns

DDS\_BOOLEAN\_TRUE if the two structures are equal, DDS\_BOOLEAN\_FALSE otherwise.

#### 6.2.6.6.6 Subscription Built-in Topic

Builtin topic for configuring information about the Subscriptions to be discovered by RTI Connex DDS Micro.

#### Classes

- struct [DDS\\_SubscriptionBuiltinTopicData](#)  
*Entry created when a [DDSDDataReader](#) is discovered in association with its Subscriber.*

#### Macros

- #define [DDS\\_SUBSCRIPTION\\_TOPIC\\_NAME](#) DDS\_SUBSCRIPTION\_BUILTIN\_TOPIC\_NAME  
*Subscription topic name.*

#### Functions

- [DDS\\_SubscriptionBuiltinTopicData](#) (const [DDS\\_DomainParticipantQos](#) &dp\_qos)  
*A Constructor that uses the domain participants resource limits to initialize the object memory.*
- [DDS\\_SubscriptionBuiltinTopicData](#) ()  
*Default constructor.*
- [DDS\\_Boolean DDS\\_SubscriptionBuiltinTopicData\\_finalize](#) (struct [DDS\\_SubscriptionBuiltinTopicData](#) \*self)  
*Finalize the contents of this SubscriptionBuiltinTopicData.*
- [DDS\\_Boolean DDS\\_SubscriptionBuiltinTopicData\\_is\\_equal](#) (const struct [DDS\\_SubscriptionBuiltinTopicData](#) \*left, const struct [DDS\\_SubscriptionBuiltinTopicData](#) \*right)  
*Compare two SubscriptionBuiltinTopicData structures for equality.*

#### Detailed Description

Builtin topic for configuring information about the Subscriptions to be discovered by RTI Connex DDS Micro.

#### Macro Definition Documentation

##### DDS\_SUBSCRIPTION\_TOPIC\_NAME

```
#define DDS_SUBSCRIPTION_TOPIC_NAME DDS_SUBSCRIPTION_BUILTIN_TOPIC_NAME
```

Subscription topic name.

Topic name of subscription builtin topic data DataReader.

See also

[DDS\\_SubscriptionBuiltinTopicData](#)

#### Function Documentation

##### DDS\_SubscriptionBuiltinTopicData() [1/2]

```
DDS_SubscriptionBuiltinTopicData (  
    const DDS\_DomainParticipantQos & dp_qos)
```

A Constructor that uses the domain participants resource limits to initialize the object memory.

## Parameters

|           |               |                                               |
|-----------|---------------|-----------------------------------------------|
| <i>in</i> | <i>dp_qos</i> | The resource limits to initialize the object. |
|-----------|---------------|-----------------------------------------------|

**DDS\_SubscriptionBuiltinTopicData()** [2/2]

```
DDS_SubscriptionBuiltinTopicData ()
```

Default constructor.

**DDS\_SubscriptionBuiltinTopicData\_finalize()**

```
DDS_Boolean DDS_SubscriptionBuiltinTopicData_finalize (
    struct DDS_SubscriptionBuiltinTopicData * self)
```

Finalize the contents of this SubscriptionBuiltinTopicData.

Frees all memory owned by the SubscriptionBuiltinTopicData which was allocated through the standard DDS APIs, such as `initialize_from_qos`, `copy`, or `sequence set_maximum`. If you have allocated memory for any attribute in this object using non-standard APIs, then the attribute must be returned to its initialized state before calling `finalize`. Any memory allocated using non-standard APIs is the responsibility of the user to manage.

## Returns

One of the [Standard Return Codes](#)

**DDS\_SubscriptionBuiltinTopicData\_is\_equal()**

```
DDS_Boolean DDS_SubscriptionBuiltinTopicData_is_equal (
    const struct DDS_SubscriptionBuiltinTopicData * left,
    const struct DDS_SubscriptionBuiltinTopicData * right)
```

Compare two SubscriptionBuiltinTopicData structures for equality.

This method compares two SubscriptionBuiltinTopicData structures for equality. The comparison is performed field by field.

## Parameters

|              |                                                    |
|--------------|----------------------------------------------------|
| <i>left</i>  | << <i>in</i> >>. The left side of the comparison.  |
| <i>right</i> | << <i>in</i> >>. The right side of the comparison. |

## Returns

DDS\_BOOLEAN\_TRUE if the two structures are equal, DDS\_BOOLEAN\_FALSE otherwise.

References [RTI\\_UINT32](#).



### 6.2.7 Topic

Contains the [DDSTopic](#), the [DDSTopicListener](#) interface, and more generally, all that is needed by an application to define [DDSTopic](#) objects and the data types that are associated with [DDSTopic](#) objects.

#### Topics

- [DDS-Specific Primitive Types](#)  
*Basic DDS value types for use in user data types.*
- [Topic](#)  
*DDSTopic entity and associated elements*
- [FlatData Topic-Types](#)  
*<<eXtension>> FlatData Language Binding for IDL topic-types*
- [Zero Copy Transfer Over Shared Memory](#)  
*<<eXtension>> Zero Copy transfer over shared memory*
- [User Data Type Support](#)  
*Defines generic classes and macros to support user data types.*

#### 6.2.7.1 Detailed Description

Contains the [DDSTopic](#), the [DDSTopicListener](#) interface, and more generally, all that is needed by an application to define [DDSTopic](#) objects and the data types that are associated with [DDSTopic](#) objects.

#### 6.2.7.2 DDS-Specific Primitive Types

Basic DDS value types for use in user data types.

#### Macros

- `#define DDS\_BOOLEAN\_TRUE`  
*Defines TRUE value of IDL boolean data type.*
- `#define DDS\_BOOLEAN\_FALSE`  
*Defines FALSE value of IDL boolean data type.*

## Typedefs

- typedef CDR\_Char [DDS\\_Char](#)  
*Defines IDL char data type.*
- typedef RTI\_UINT16 [DDS\\_Wchar](#)  
*Defines IDL wchar data type.*
- typedef CDR\_Octet [DDS\\_Octet](#)  
*Defines IDL octet data type.*
- typedef CDR\_Short [DDS\\_Short](#)  
*Defines IDL short data type.*
- typedef CDR\_UnsignedShort [DDS\\_UnsignedShort](#)  
*Defines IDL unsigned short data type.*
- typedef CDR\_Long [DDS\\_Long](#)  
*Defines IDL long data type.*
- typedef CDR\_UnsignedLong [DDS\\_UnsignedLong](#)  
*Defines IDL unsigned long data type.*
- typedef CDR\_LongLong [DDS\\_LongLong](#)  
*Defines IDL long long data type.*
- typedef CDR\_UnsignedLongLong [DDS\\_UnsignedLongLong](#)  
*Defines IDL unsigned long long data type.*
- typedef CDR\_Float [DDS\\_Float](#)  
*Defines IDL float data type.*
- typedef CDR\_Double [DDS\\_Double](#)  
*Defines IDL double data type.*
- typedef CDR\_LongDouble [DDS\\_LongDouble](#)  
*Defines IDL long double data type.*
- typedef CDR\_Boolean [DDS\\_Boolean](#)  
*Defines IDL boolean data type.*
- typedef CDR\_Enum [DDS\\_Enum](#)  
*Defines IDL enum data type.*
- typedef CDR\_String [DDS\\_String](#)  
*IDL string type representation.*
- typedef [DDS\\_Wchar](#) \* [DDS\\_Wstring](#)  
*Defines IDL wstring data type.*

### 6.2.7.2.1 Detailed Description

Basic DDS value types for use in user data types.

As part of the finalization of the DDS standard, a number of DDS-specific primitive types will be introduced. By using these types, you will ensure that your data is serialized consistently across platforms even if the built-in types have different sizes on those platforms.

In this version of RTI Connex Micro DDS, the DDS primitive types are defined using the OMG's Common Data Representation (CDR) standard.

#### 6.2.7.2.2 Macro Definition Documentation

##### DDS\_BOOLEAN\_TRUE

```
#define DDS_BOOLEAN_TRUE
```

Defines TRUE value of IDL boolean data type.

##### DDS\_BOOLEAN\_FALSE

```
#define DDS_BOOLEAN_FALSE
```

Defines FALSE value of IDL boolean data type.

#### 6.2.7.2.3 Typedef Documentation

##### DDS\_Char

```
typedef CDR_Char DDS_Char
```

Defines IDL char data type.

An 8-bit quantity that encodes a single byte character from any byte-oriented code set.

##### See also

For sequence support, see [FooSeq](#).

##### DDS\_Wchar

```
typedef RTI_UINT16 DDS_Wchar
```

Defines IDL wchar data type.

An 16-bit quantity that encodes a wide character from any character set.

##### See also

For sequence support, see [FooSeq](#).

### DDS\_Octet

```
typedef CDR_Octet DDS_Octet
```

Defines IDL octet data type.

An 8-bit quantity that is guaranteed not to undergo any conversion when transmitted by the middleware.

#### See also

For sequence support, see [FooSeq](#).

### DDS\_Short

```
typedef CDR_Short DDS_Short
```

Defines IDL short data type.

A 16-bit signed short integer value.

#### See also

For sequence support, see [FooSeq](#).

### DDS\_UnsignedShort

```
typedef CDR_UnsignedShort DDS_UnsignedShort
```

Defines IDL unsigned short data type.

A 16-bit unsigned short integer value.

#### See also

For sequence support, see [FooSeq](#).

### DDS\_Long

```
typedef CDR_Long DDS_Long
```

Defines IDL long data type.

A 32-bit signed long integer value.

#### See also

For sequence support, see [FooSeq](#).

## DDS\_UnsignedLong

```
typedef CDR_UnsignedLong DDS_UnsignedLong
```

Defines IDL unsigned long data type.

A 32-bit unsigned long integer value.

### See also

For sequence support, see [FooSeq](#).

## DDS\_LongLong

```
typedef CDR_LongLong DDS_LongLong
```

Defines IDL long long data type.

A 64-bit signed long long integer value.

Since some architectures do not support long long (8 bytes long), RTI has defined character arrays that match the expected size of these types.

### See also

For sequence support, see [FooSeq](#).

## DDS\_UnsignedLongLong

```
typedef CDR_UnsignedLongLong DDS_UnsignedLongLong
```

Defines IDL unsigned long long data type.

An 64-bit unsigned long long integer value.

Since some architectures do not support (unsigned) long long (8 byte long), RTI has defined character arrays that match the expected size of these types.

### See also

For sequence support, see [FooSeq](#).

### DDS\_Float

```
typedef CDR_Float DDS_Float
```

Defines IDL float data type.

A 32-bit floating point value.

#### See also

For sequence support, see [FooSeq](#).

### DDS\_Double

```
typedef CDR_Double DDS_Double
```

Defines IDL double data type.

A 64-bit floating point value.

#### See also

For sequence support, see [FooSeq](#).

### DDS\_LongDouble

```
typedef CDR_LongDouble DDS_LongDouble
```

Defines IDL long double data type.

A 16-byte floating point value.

Since some architectures do not support long double, RTI has defined character arrays that match the expected size of this type. On systems that do have native long double, you have to define `RTI_CDR_SIZEOF_LONG_DOUBLE` as 16 to map them to native types.

#### See also

For sequence support, see [FooSeq](#).

### DDS\_Boolean

```
typedef CDR_Boolean DDS_Boolean
```

Defines IDL boolean data type.

An 8-bit boolean value that is used to denote a data item that can only take one of the values [DDS\\_BOOLEAN\\_TRUE](#) (1) or [DDS\\_BOOLEAN\\_FALSE](#) (0).

#### See also

For sequence support, see [FooSeq](#).

## DDS\_Enum

```
typedef CDR_Enum DDS_Enum
```

Defines IDL enum data type.

Encoded as unsigned long value. The first enum identifier has the numeric value zero (0). Successive enum identifiers take ascending numeric values, in order of declaration from left to right.

### See also

For sequence support, see [FooSeq](#).

## DDS\_String

```
typedef CDR_String DDS_String
```

IDL `string` type representation.

Has a range of ISO Latin-1 (except ASCII NULL) and a variable length.

### See also

[String Support](#)

For sequence support, see [FooSeq](#).

## DDS\_Wstring

```
typedef DDS_Wchar* DDS_Wstring
```

Defines IDL wstring data type.

Each character composing the string is a 16-bit quantity that encodes a wide character from any character set.

### See also

For sequence support, see [FooSeq](#).

## 6.2.7.3 Topic

[DDSTopic](#) entity and associated elements

## Classes

- struct `DDS_InconsistentTopicStatus`  
`DDS_INCONSISTENT_TOPIC_STATUS`
- struct `DDS_TopicQos`  
*QoS policies supported by a `DDSTopic` entity.*
- class `DDSTopicDescription`  
`<<interface>>` *Base class for `DDSTopic`.*
- class `DDSTopic`  
`<<interface>>` *The most basic description of the data to be published and subscribed.*
- class `DDSTopicListener`  
`<<interface>>` *`DDSListener` for `DDSTopic` entities.*

### 6.2.7.3.1 Detailed Description

`DDSTopic` entity and associated elements

### 6.2.7.4 FlatData Topic-Types

`<<eXtension>>` FlatData Language Binding for IDL topic-types

`<<eXtension>>` FlatData Language Binding for IDL topic-types

#### Warning

The FlatData language binding is only available in the Traditional C++ API. RTI Connex DDS Micro does not support Modern C++. It is not available in the C API.

FlatData is a language binding for IDL types in which the in-memory representation of a sample matches the wire representation. Therefore, the cost of serialization/deserialization is zero.

#### Note

For a complete description of the FlatData language binding and its benefits, and a tutorial, see the "Sending Large Data" chapter in the **RTI Connex DDS Micro User's Manual**.

For buildable **code examples**, see <https://community.rti.com/kb/flatdata-and-zero-copy-examples>.

### 6.2.7.5 Zero Copy Transfer Over Shared Memory

`<<eXtension>>` Zero Copy transfer over shared memory

`<<eXtension>>` Zero Copy transfer over shared memory

#### Note

For a description of Zero Copy transfer over shared memory and its benefits, and a tutorial, see the "Sending Large Data" chapter in the **RTI Connex DDS Micro User's Manual**.

For buildable **code examples**, see <https://community.rti.com/kb/flatdata-and-zero-copy-examples>.

Zero Copy transfer over shared memory is available in the C API, in the Traditional C++ API, and in the Modern C++ API.



### 6.2.7.6 User Data Type Support

Defines generic classes and macros to support user data types.

#### Classes

- struct `DDS_InstanceHandleSeq`  
*Instantiates `FooSeq` < `DDS_InstanceHandle_t` > .*
- class `FooTypeSupport`  
*<<interface>> <<generic>> User data type specific interface.*

#### Macros

- #define `DDS_DATAWRITER_CPP`(TDataWriter, TData)  
*Declares the interface required to support a user data type specific data writer.*
- #define `DDS_DATAREADER_CPP`(TDataReader, TData)  
*Declares the interface required to support a user data type-specific data reader.*
- #define `DDS_TYPESUPPORT_CPP`(TTypeSupport, TData)  
*Declares the interface required to support a user data type.*

#### Typedefs

- typedef `DDS_HANDLE_TYPE_NATIVE DDS_InstanceHandle_t`  
*Type definition for an instance handle.*

#### Functions

- `DDS_Boolean DDS_InstanceHandle_equals` (const `DDS_InstanceHandle_t` \*self, const `DDS_InstanceHandle_t` \*other)  
*Compares this instance handle with another handle for equality.*
- `DDS_Boolean DDS_InstanceHandle_is_nil` (const `DDS_InstanceHandle_t` \*self)  
*Compare this handle to `DDS_HANDLE_NIL`.*

#### Variables

- const `DDS_InstanceHandle_t DDS_HANDLE_NIL`  
*The NIL instance handle.*

### 6.2.7.6.1 Detailed Description

Defines generic classes and macros to support user data types.

Defines the DDS user data type support.

DDS specifies strongly typed interfaces to read and write user data. For each data class defined by the application, there is a number of specialised classes that are required to facilitate the type-safe interaction of the application with RTI Connex DDS Micro.

RTI Connex DDS Micro provides an automatic means to generate all these type-specific classes with the RTI IDL Compiler User Manual utility. The complete set of automatic classes created for a hypothetical user data type named `Foo` are shown below.

The macros defined here declare the strongly typed APIs needed to support an arbitrary user defined data of type `Foo`.

### 6.2.7.6.2 Macro Definition Documentation

#### DDS\_DATAWRITER\_CPP

```
#define DDS_DATAWRITER_CPP (
    TDataWriter,
    TData)
```

Declares the interface required to support a user data type specific data writer.

Uses:

[FooTypeSupport](#) user data type, `Foo`

Defines:

[FooDataWriter](#) `DDSDataWriter` of type `Foo`, i.e. [FooDataWriter](#)

#### DDS\_DATAREADER\_CPP

```
#define DDS_DATAREADER_CPP (
    TDataReader,
    TData)
```

Declares the interface required to support a user data type-specific data reader.

Uses:

[FooTypeSupport](#) user data type, `Foo` [FooSeq](#) sequence of user data type, `sequence<Foo>`

Defines:

[FooDataReader](#) `DDSDataReader` of type `Foo`, i.e. [FooDataReader](#)

See also

[FooSeq](#)

## DDS\_TYPESUPPORT\_CPP

```
#define DDS_TYPESUPPORT_CPP(
    TTypeSupport,
    TData)
```

Declares the interface required to support a user data type.

Defines:

[FooTypeSupport](#) TypeSupport of type `Foo`, i.e. [FooTypeSupport](#)

### 6.2.7.6.3 Typedef Documentation

#### DDS\_InstanceHandle\_t

```
typedef DDS_HANDLE_TYPE_NATIVE DDS_InstanceHandle_t
```

Type definition for an instance handle.

Handle to identify different instances of the same [DDSTopic](#) of a certain type.

See also

[FooDataWriter::register\\_instance](#)  
[DDS\\_SampleInfo::instance\\_handle](#)

### 6.2.7.6.4 Function Documentation

#### DDS\_InstanceHandle\_equals()

```
DDS_Boolean DDS_InstanceHandle_equals (
    const DDS_InstanceHandle_t * self,
    const DDS_InstanceHandle_t * other)
```

Compares this instance handle with another handle for equality.

Parameters

|              |                                                                          |
|--------------|--------------------------------------------------------------------------|
| <i>self</i>  | <<in>> This handle. Cannot be NULL.                                      |
| <i>other</i> | <<in>> The other handle to be compared with this handle. Cannot be NULL. |

Returns

[DDS\\_BOOLEAN\\_TRUE](#) if the two handles have equal values, or [DDS\\_BOOLEAN\\_FALSE](#) otherwise.

See also

[DDS\\_InstanceHandle\\_is\\_nil](#)

### DDS\_InstanceHandle\_is\_nil()

```
DDS_Boolean DDS_InstanceHandle_is_nil (  
    const DDS_InstanceHandle_t * self)
```

Compare this handle to [DDS\\_HANDLE\\_NIL](#).

#### Returns

[DDS\\_BOOLEAN\\_TRUE](#) if the given instance handle is equal to [DDS\\_HANDLE\\_NIL](#) or [DDS\\_BOOLEAN\\_FALSE](#) otherwise.

#### See also

[DDS\\_InstanceHandle\\_equals](#)

#### 6.2.7.6.5 Variable Documentation

### DDS\_HANDLE\_NIL

```
const DDS_InstanceHandle_t DDS_HANDLE_NIL [extern]
```

The NIL instance handle.

Special [DDS\\_InstanceHandle\\_t](#) value

#### See also

[DDS\\_InstanceHandle\\_is\\_nil](#)

#### 6.2.8 Publication Module

Contains the [DDSPublisher](#), and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.

#### Topics

- [Flow Controllers](#)  
*<<eXtension>> [DDSFlowController](#) and associated elements*
- [Publisher](#)  
*[DDSPublisher](#) entity and associated elements*
- [DataWriter](#)  
*[DDSDataWriter](#) entity and associated elements*

### 6.2.8.1 Detailed Description

Contains the [DDSPublisher](#), and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.

Defines the DDS publication package.

This diagram defines the elements that are defined in the DDS specification, and their relationships to each other. Not all entities and functions shown in the diagram are supported by RTI Connex DDS Micro. Please read the information on the individual types to determine whether a function is supported.

### 6.2.8.2 Flow Controllers

<<eXtension>> [DDSFlowController](#) and associated elements

#### Classes

- struct [DDS\\_FlowControllerTokenBucketProperty\\_t](#)  
*[DDSFlowController](#) uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties.*
- struct [DDS\\_FlowControllerProperty\\_t](#)  
*Determines the flow control characteristics of the [DDSFlowController](#).*
- class [DDSFlowController](#)  
<<interface>> *A flow controller is the object responsible for shaping the network traffic by determining when attached asynchronous [DDSDataWriter](#) instances are allowed to write data.*

#### Enumerations

- enum [DDS\\_FlowControllerSchedulingPolicy](#) { [DDS\\_RR\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#) , [DDS\\_EDF\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#) , [DDS\\_HPF\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#) }  
*Kinds of flow controller scheduling policy.*

#### Variables

- const char \*const [DDS\\_DEFAULT\\_FLOW\\_CONTROLLER\\_NAME](#)  
*[default] Special value of [DDS\\_PublishModeQosPolicy::flow\\_controller\\_name](#) that refers to the built-in default flow controller.*
- const char \*const [DDS\\_FIXED\\_RATE\\_FLOW\\_CONTROLLER\\_NAME](#)  
*Special value of [DDS\\_PublishModeQosPolicy::flow\\_controller\\_name](#) that refers to the built-in fixed-rate flow controller.*
- const char \*const [DDS\\_ON\\_DEMAND\\_FLOW\\_CONTROLLER\\_NAME](#)  
*Special value of [DDS\\_PublishModeQosPolicy::flow\\_controller\\_name](#) that refers to the built-in on-demand flow controller.*

### 6.2.8.2.1 Detailed Description

<<*eXtension*>> [DDSFlowController](#) and associated elements

[DDSFlowController](#) provides the network traffic shaping capability to asynchronous [DDSDataWriter](#) instances. For use cases and advantages of publishing asynchronously, please refer to [DDS\\_PublishModeQosPolicy](#) of [DDS\\_DataWriterQos](#).

See also

[DDS\\_PublishModeQosPolicy](#)

[DDS\\_DataWriterQos::publish\\_mode](#)

### 6.2.8.2.2 Enumeration Type Documentation

#### DDS\_FlowControllerSchedulingPolicy

enum [DDS\\_FlowControllerSchedulingPolicy](#)

Kinds of flow controller scheduling policy.

QoS:

[DDS\\_FlowControllerProperty\\_t](#)

Samples written by an asynchronous [DDSDataWriter](#) are not sent in the context of the [FooDataWriter::write](#) call. Instead, the middleware puts the samples in a queue for future processing. The [DDSFlowController](#) associated with each asynchronous DataWriter instance determines when the samples are actually sent.

Each [DDSFlowController](#) maintains a separate FIFO queue for each unique destination (remote application). Samples written by asynchronous [DDSDataWriter](#) instances associated with the flow controller, are placed in the queues that correspond to the intended destinations of the sample.

When tokens become available, a flow controller must decide which queue(s) to grant tokens first. This is determined by the flow controller's scheduling policy. Once a queue has been granted tokens, it is serviced by the asynchronous publishing thread. The queued up samples will be coalesced and sent to the corresponding destination. The number of samples sent depends on the data size and the number of tokens granted.

Enumerator

|                                     |                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_RR_FLOW_CONTROLLER_SCHED_POLICY | Indicates to flow control in a round-robin fashion. Whenever tokens become available, the flow controller distributes the tokens uniformly across all of its (non-empty) destination queues. No destinations are prioritized. Instead, all destinations are treated equally and are serviced in a round-robin fashion. |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Enumerator

|                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_EDF_FLOW_CONTROLLER_SCHED_POLICY | <p>Indicates to flow control in an earliest-deadline-first fashion. A sample's deadline is determined by the time it was written plus the latency budget of the <code>DataWriter</code> at the time of the write call (as specified in the <a href="#">DDS_LatencyBudgetQosPolicy</a>). The relative priority of a flow controller's destination queue is determined by the earliest deadline across all samples it contains. When tokens become available, the <a href="#">DDSFlowController</a> distributes tokens to the destination queues in order of their deadline priority. In other words, the queue containing the sample with the earliest deadline is serviced first. The number of tokens granted equals the number of tokens required to send the first sample in the queue. Note that the priority of a queue may change as samples are sent (i.e. removed from the queue). If a sample must be sent to multiple destinations or two samples have an equal deadline value, the corresponding destination queues are serviced in a round-robin fashion.</p> <p>Hence, under the default <a href="#">DDS_LatencyBudgetQosPolicy::duration</a> setting, an <code>EDF_FLOW_CONTROLLER_SCHED_POLICY</code> <a href="#">DDSFlowController</a> preserves the order in which the user calls <a href="#">FooDataWriter::write</a> across the <code>DataWriters</code> associated with the flow controller.</p> <p>In other words, the priority of a destination queue is always determined by the earliest deadline among all samples contained in the queue. This priority inheritance approach is required in order to both honor the updated <a href="#">DDS_LatencyBudgetQosPolicy::duration</a> and adhere to the <a href="#">DDSDataWriter</a> in-order data delivery guarantee.</p> <p><b>[default]</b> for <a href="#">DDSDataWriter</a></p> |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Enumerator

|                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_HPF_FLOW_CONTROLLER_SCHED_POLICY | <p>Indicates to flow control in a highest-priority-first fashion. Determines the next destination queue to service as determined by the publication priority of the <a href="#">DDSDataWriter</a>, channel of multi-channel DataWriter, or individual sample.</p> <p>The relative priority of a flow controller's destination queue is determined by the highest publication priority of all samples it contains.</p> <p>When tokens become available, the <a href="#">DDSFlowController</a> distributes tokens to the destination queues in order of their publication priority. In other words, the queue containing the sample with the highest publication priority is serviced first. The number of tokens granted equals the number of tokens required to send the first sample in the queue. Note that the priority of a queue may change as samples are sent (i.e. removed from the queue). If a sample must be sent to multiple destinations or two samples have an equal publication priority, the corresponding destination queues are serviced in a round-robin fashion.</p> <p>This priority inheritance approach is required in order to both honor the designated publication priority and adhere to the <a href="#">DDSDataWriter</a> in-order data delivery guarantee.</p> |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 6.2.8.2.3 Variable Documentation

## DDS\_DEFAULT\_FLOW\_CONTROLLER\_NAME

```
const char* const DDS_DEFAULT_FLOW_CONTROLLER_NAME [extern]
```

**[default]** Special value of [DDS\\_PublishModeQosPolicy::flow\\_controller\\_name](#) that refers to the built-in default flow controller.

RTI Connex DDS Micro provides several built-in [DDSFlowController](#) for use with an asynchronous [DDSDataWriter](#). The user can choose to use the built-in flow controllers and optionally modify their properties or can create a custom flow controller.

By default, flow control is disabled. That is, the built-in [DDS\\_DEFAULT\\_FLOW\\_CONTROLLER\\_NAME](#) flow controller does not apply any flow control. Instead, it allows data to be sent asynchronously as soon as it is written by the [DDSDataWriter](#).

Essentially, this is equivalent to a user-created [DDSFlowController](#) with the following [DDS\\_FlowControllerProperty\\_t](#) settings:

- [DDS\\_FlowControllerProperty\\_t::scheduling\\_policy](#) = [DDS\\_EDF\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `max_tokens` = [DDS\\_LENGTH\\_UNLIMITED](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `tokens_added_per_period` = [DDS\\_LENGTH\\_UNLIMITED](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `tokens_leaked_per_period` = 0
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `period` = 1 second
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `bytes_per_token` = [DDS\\_LENGTH\\_UNLIMITED](#)



See also

[DDSDomainParticipant::lookup\\_flowcontroller](#)  
[DDSFlowController::set\\_property](#)  
[DDS\\_PublishModeQosPolicy](#)

## DDS\_FIXED\_RATE\_FLOW\_CONTROLLER\_NAME

```
const char* const DDS_FIXED_RATE_FLOW_CONTROLLER_NAME [extern]
```

Special value of [DDS\\_PublishModeQosPolicy::flow\\_controller\\_name](#) that refers to the built-in fixed-rate flow controller.

RTI Connex DDS Micro provides several builtin [DDSFlowController](#) for use with an asynchronous [DDSDataWriter](#). The user can choose to use the built-in flow controllers and optionally modify their properties or can create a custom flow controller.

The built-in [DDS\\_FIXED\\_RATE\\_FLOW\\_CONTROLLER\\_NAME](#) flow controller shapes the network traffic by allowing data to be sent only once every second. Any accumulated samples destined for the same destination are coalesced into as few network packets as possible.

Essentially, this is equivalent to a user-created [DDSFlowController](#) with the following [DDS\\_FlowControllerProperty\\_t](#) settings:

- [DDS\\_FlowControllerProperty\\_t::scheduling\\_policy](#) = [DDS\\_EDF\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `max_tokens` = [DDS\\_LENGTH\\_UNLIMITED](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `tokens_added_per_period` = [DDS\\_LENGTH\\_UNLIMITED](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `tokens_leaked_per_period` = [DDS\\_LENGTH\\_UNLIMITED](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `period` = 1 second
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `bytes_per_token` = [DDS\\_LENGTH\\_UNLIMITED](#)

See also

[DDSDomainParticipant::lookup\\_flowcontroller](#)  
[DDSFlowController::set\\_property](#)  
[DDS\\_PublishModeQosPolicy](#)

**DDS\_ON\_DEMAND\_FLOW\_CONTROLLER\_NAME**

```
const char* const DDS_ON_DEMAND_FLOW_CONTROLLER_NAME [extern]
```

Special value of [DDS\\_PublishModeQosPolicy::flow\\_controller\\_name](#) that refers to the built-in on-demand flow controller.

RTI Connex DDS Micro provides several builtin [DDSFlowController](#) for use with an asynchronous [DDSDataWriter](#). The user can choose to use the built-in flow controllers and optionally modify their properties or can create a custom flow controller.

The built-in [DDS\\_ON\\_DEMAND\\_FLOW\\_CONTROLLER\\_NAME](#) allows data to be sent only when the user calls [DDSFlowController::trigger\\_flow](#). With each trigger, all accumulated data since the previous trigger is sent (across all [DDSPublisher](#) or [DDSDataWriter](#) instances). In other words, the network traffic shape is fully controlled by the user. Any accumulated samples destined for the same destination are coalesced into as few network packets as possible.

Essentially, this is equivalent to a user-created [DDSFlowController](#) with the following [DDS\\_FlowControllerProperty\\_t](#) settings:

- [DDS\\_FlowControllerProperty\\_t::scheduling\\_policy](#) = [DDS\\_EDF\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `max_tokens` = [DDS\\_LENGTH\\_UNLIMITED](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `tokens_added_per_period` = [DDS\\_LENGTH\\_UNLIMITED](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `tokens_leaked_per_period` = [DDS\\_LENGTH\\_UNLIMITED](#) - [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `period` = [DDS\\_DURATION\\_INFINITE](#)
- [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#) `bytes_per_token` = [DDS\\_LENGTH\\_UNLIMITED](#)

See also

```
DDSDomainParticipant::lookup_flowcontroller
DDSFlowController::set_property
DDS_PublishModeQosPolicy;
```

**6.2.8.3 Publisher**

[DDSPublisher](#) entity and associated elements

**Classes**

- struct [DDS\\_PublisherQos](#)  
*QoS policies supported by a [DDSPublisher](#) entity.*
- class [DDSPublisherListener](#)  
*<<interface>> [DDSListener](#) for [DDSPublisher](#) status.*
- class [DDSPublisher](#)  
*<<interface>> A publisher is the object responsible for the actual dissemination of publications.*

## Typedefs

- typedef struct DDS\_PublisherImpl [DDS\\_Publisher](#)  
*<<interface>> A publisher is the object responsible for the actual dissemination of publications.*

## Variables

- const struct [DDS\\_DataWriterQos](#) [DDS\\_DATAWRITER\\_QOS\\_DEFAULT](#)  
*Special value for creating [DDSDDataWriter](#) with default QoS*

### 6.2.8.3.1 Detailed Description

[DDSPublisher](#) entity and associated elements

### 6.2.8.3.2 Typedef Documentation

#### DDS\_Publisher

```
typedef struct DDS_PublisherImpl DDS\_Publisher
```

*<<interface>> A publisher is the object responsible for the actual dissemination of publications.*

QoS:

[DDS\\_PublisherQos](#)

Listener:

[DDSPublisherListener](#)

A publisher acts on the behalf of one or several [DDSDDataWriter](#) objects that belong to it. When it is informed of a change to the data associated with one of its [DDSDDataWriter](#) objects, it decides when it is appropriate to actually send the data-update message. In making this decision, it considers any extra information that goes with the data (timestamp, writer, etc.) as well as the QoS of the [DDSPublisher](#) and the [DDSDDataWriter](#).

The following operations may be called even if the [DDSPublisher](#) is not enabled. Other operations will fail with the value [DDS\\_RETCODE\\_NOT\\_ENABLED](#) if called on a disabled [DDSPublisher](#):

- The base-class operations [DDSEntity::enable](#)
- [DDSPublisher::create\\_datawriter](#), [DDSPublisher::delete\\_datawriter](#), [DDSPublisher::delete\\_contained\\_entities](#),

### 6.2.8.3.3 Variable Documentation

#### DDS\_DATAWRITER\_QOS\_DEFAULT

```
const struct DDS_DataWriterQos DDS_DATAWRITER_QOS_DEFAULT [extern]
```

Special value for creating [DDSDataWriter](#) with default QoS

When used in [DDSPublisher::create\\_datawriter](#), this special value is used to indicate that the [DDSDataWriter](#) should be created with the default [DDSDataWriter](#) QoS.

See also

[DDSPublisher::create\\_datawriter](#)

### 6.2.8.4 DataWriter

[DDSDataWriter](#) entity and associated elements

#### Classes

- struct [DDS\\_OfferedDeadlineMissedStatus](#)  
[DDS\\_OFFERED\\_DEADLINE\\_MISSED\\_STATUS](#)
- struct [DDS\\_LivelinessLostStatus](#)  
[DDS\\_LIVELINESS\\_LOST\\_STATUS](#)
- struct [DDS\\_OfferedIncompatibleQosStatus](#)  
[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)
- struct [DDS\\_PublicationMatchedStatus](#)  
[DDS\\_PUBLICATION\\_MATCHED\\_STATUS](#)
- struct [DDS\\_ReliableReaderActivityChangedStatus](#)  
*<<eXtension>> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer.*
- struct [DDS\\_ReliableSampleUnacknowledgedStatus](#)  
*Status about an unacknowledged sample that is removed from the [DDSDataWriter](#) cache before being acknowledged by all matched [DDSDataReader](#) entities.*
- struct [DDS\\_DataWriterQos](#)  
*QoS policies supported by a [DDSDataWriter](#) entity.*
- class [FooDataWriter](#)  
*Declares the interface required to support a user data type-specific data writer.*
- class [DDSDataWriterListener](#)  
*<<interface>> [DDSListener](#) for writer status.*
- class [DDSDataWriter](#)  
*<<interface>> Allows an application to set the value of the data to be published under a given [DDSTopic](#).*

#### Macros

- #define [DDS\\_ReliableReaderActivityChangedStatus\\_INITIALIZER](#)  
*Initializer for new status instances.*

## Typedefs

- typedef void(\* [DDS\\_DataWriterListener\\_ReliableReaderActivityChangedCallback](#)) (void \*listener\_data, DDS\_DataWriter \*writer, const struct [DDS\\_ReliableReaderActivityChangedStatus](#) \*status)  
*<<eXtension>> A matched reliable reader has become active or become inactive.*
- typedef void(\* [DDS\\_DataWriterListener\\_ReliableSampleUnacknowledgedCallback](#)) (void \*listener\_data, DDS\_DataWriter \*writer, const struct [DDS\\_ReliableSampleUnacknowledgedStatus](#) \*status)  
*<<eXtension>> Unacknowledged samples are removed from the writer cache.*
- typedef void(\* [DDS\\_DataWriterListener\\_SampleRemovedCallback](#)) (void \*listener\_data, DDS\_DataWriter \*writer, const struct DDS\_Cookie\_t \*cookie)  
*Prototype of a [DDSDDataWriterListener](#) on\_sample\_removed function.*

### 6.2.8.4.1 Detailed Description

[DDSDDataWriter](#) entity and associated elements

### 6.2.8.4.2 Macro Definition Documentation

#### DDS\_ReliableReaderActivityChangedStatus\_INITIALIZER

```
#define DDS_ReliableReaderActivityChangedStatus_INITIALIZER
```

Initializer for new status instances.

New [DDS\\_ReliableReaderActivityChangedStatus](#) instances stored on the stack should be initialized with this value before they are passed to any method. This step ensures that those fields that use dynamic memory are properly initialized. This does not allocate memory.

The simplest way to create a status structure is to initialize it on the stack at the time of its creation.

### 6.2.8.4.3 Typedef Documentation

#### DDS\_DataWriterListener\_ReliableReaderActivityChangedCallback

```
typedef void(* DDS_DataWriterListener_ReliableReaderActivityChangedCallback) (void *listener_data, DDS_DataWriter *writer, const struct DDS\_ReliableReaderActivityChangedStatus *status)
```

*<<eXtension>> A matched reliable reader has become active or become inactive.*

#### Parameters

|                      |                                                                                                                                  |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>listener_data</i> | <i>&lt;&lt;out&gt;&gt;</i> Data associated with the listener when the listener is set                                            |
| <i>writer</i>        | <i>&lt;&lt;out&gt;&gt;</i> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback                    |
| <i>status</i>        | <i>&lt;&lt;out&gt;&gt;</i> Current reliable reader activity changed status of the locally created <a href="#">DDSDDataWriter</a> |

#### DDS\_DataWriterListener\_ReliableSampleUnacknowledgedCallback

```
typedef void(* DDS_DataWriterListener_ReliableSampleUnacknowledgedCallback) (void *listener_data, DDS_DataWriter *writer, const struct DDS\_ReliableSampleUnacknowledgedStatus *status)
```

*<<eXtension>> Unacknowledged samples are removed from the writer cache.*

## Parameters

|                      |                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------|
| <i>listener_data</i> | <<out>> Data associated with the listener when the listener is set                        |
| <i>writer</i>        | <<out>> Locally created <a href="#">DDSDataWriter</a> that triggers the listener callback |
| <i>status</i>        | <<out>> Unacknowledged sample status for <a href="#">DDSDataWriter</a>                    |

**DDS\_DataWriterListener\_SampleRemovedCallback**

```
typedef void(* DDS_DataWriterListener_SampleRemovedCallback) (void *listener_data, DDS_DataWriter
*writer, const struct DDS_Cookie_t *cookie)
```

Prototype of a [DDSDataWriterListener](#) on `_sample_removed` function.

## Parameters

|                      |                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------|
| <i>listener_data</i> | <<out>> Data associated with the listener when the listener is set                        |
| <i>writer</i>        | <<out>> Locally created <a href="#">DDSDataWriter</a> that triggers the listener callback |
| <i>cookie</i>        | <<out>> Cookie uniquely identifying the sample that was removed                           |

**6.2.9 Subscription Module**

Contains the [DDSSubscriber](#), and [DDSDataReader](#) classes, and more generally, all that is needed on the subscription side.

**Topics**

- [Subscriber](#)  
*DDSSubscriber* entity and associated elements
- [DataReader](#)  
*DDSDataReader* entity and associated elements
- [Data Sample](#)  
*DDS\_SampleInfo*, *DDS\_SampleStateKind*, *DDS\_ViewStateKind*, *DDS\_InstanceStateKind* and associated elements

**6.2.9.1 Detailed Description**

Contains the [DDSSubscriber](#), and [DDSDataReader](#) classes, and more generally, all that is needed on the subscription side.

Defines the DDS subscription package.

This diagram defines the elements that are defined in the DDS specification, and their relationships to each other. Not all entities and functions shown in the diagram are supported by RTI Connex DDS Micro. Please read the information on the individual types to determine whether a function is supported.

### 6.2.9.2 Access to data samples

Data is made available to the application by the following operations on [DDSDataReader](#) objects: [FooDataReader::read\\_next\\_sample](#), [FooDataReader::take\\_next\\_sample](#)

The general semantics of the `read()` operation is that the application only gets access to the corresponding data (i.e. a precise instance value); the data remains the responsibility of RTI Connex DDS Micro and can be read again.

The semantics of the `take()` operations is that the application takes full responsibility for the data; that data will no longer be available locally to RTI Connex DDS Micro. Consequently, it is possible to access the same information multiple times only if all previous accesses were `read()` operations, not `take()`.

Each of these operations returns a collection of `Data` values and associated [DDS\\_SampleInfo](#) objects. Each data value represents an atom of data information (i.e., a value for one instance). This collection may contain samples related to the same or different instances (identified by the key). Multiple samples can refer to the same instance if the settings of the [HISTORY](#) QoS allow for it.

**To return the memory back to the middleware, every `read()` or `take()` that retrieves a sequence of samples must be followed with a call to [FooDataReader::return\\_loan](#).**

See also

[Interpretation of the SampleInfo](#)

#### 6.2.9.2.1 Data access patterns

The application accesses data by means of the operations `read` or `take` on the [DDSDataReader](#). These operations return an ordered collection of `DataSamples` consisting of a [DDS\\_SampleInfo](#) part and a `Data` part.

The way RTI Connex DDS Micro builds the collection depends on QoS policies set on the [DDSDataReader](#) and [DDSSubscriber](#), as well as the `source_timestamp` of the samples, and the parameters passed to the `read()` / `take()` operations, namely:

- the desired sample states (any combination of [DDS\\_SampleStateKind](#))
- the desired view states (any combination of [DDS\\_ViewStateKind](#))
- the desired instance states (any combination of [DDS\\_InstanceStateKind](#))

The `read()` and `take()` operations are non-blocking and just deliver what is currently available that matches the specified states.

Once the data samples are available to the data readers, they can be read or taken by the application. The basic rule is that the application may do this in any order it wishes. This approach is very flexible and allows the application ultimate control.

### 6.2.9.3 Subscriber

[DDSSubscriber](#) entity and associated elements

## Classes

- struct [DDS\\_SubscriberQos](#)  
*QoS policies supported by a [DDSSubscriber](#) entity.*
- class [DDSSubscriberListener](#)  
*<<interface>> [DDSListener](#) for status about a subscriber.*
- class [DDSSubscriber](#)  
*<<interface>> A subscriber is the object responsible for actually receiving data from a subscription.*

## Variables

- const struct [DDS\\_DataReaderQos](#) [DDS\\_DATAREADER\\_QOS\\_DEFAULT](#)  
*Special value for creating data reader with default QoS.*
- const struct [DDS\\_DataReaderQos](#) [DDS\\_DATAREADER\\_QOS\\_USE\\_TOPIC\\_QOS](#)  
*Special value for creating [DDSDataReader](#) with a combination of the default [DDS\\_DataReaderQos](#) and the [DDS\\_TopicQos](#).*

### 6.2.9.3.1 Detailed Description

[DDSSubscriber](#) entity and associated elements

### 6.2.9.3.2 Variable Documentation

#### DDS\_DATAREADER\_QOS\_DEFAULT

```
const struct DDS\_DataReaderQos DDS_DATAREADER_QOS_DEFAULT [extern]
```

Special value for creating data reader with default QoS.

When used in [DDSSubscriber::create\\_datareader](#), this special value is used to indicate that the [DDSDataReader](#) should be created with the default [DDSDataReader](#) QoS.

See also

[DDSSubscriber::create\\_datareader](#)

#### DDS\_DATAREADER\_QOS\_USE\_TOPIC\_QOS

```
const struct DDS\_DataReaderQos DDS_DATAREADER_QOS_USE_TOPIC_QOS [extern]
```

Special value for creating [DDSDataReader](#) with a combination of the default [DDS\\_DataReaderQos](#) and the [DDS\\_TopicQos](#).

The use of this value is equivalent to the application obtaining the default [DDS\\_DataReaderQos](#) and the [DDS\\_TopicQos](#) (by means of the operation [DDSTopic::get\\_qos](#)) and then combining these two QoS so that any policy that is set on the [DDS\\_TopicQos](#) "overrides" the corresponding policy on the default QoS. The resulting QoS is then applied to the creation of the [DDSDataReader](#).

This value should only be used in [DDSSubscriber::create\\_datareader](#).

See also

[DDSSubscriber::create\\_datareader](#)  
[DDSSubscriber::get\\_default\\_datareader\\_qos](#)  
[DDSTopic::get\\_qos](#)



#### 6.2.9.4 DataReader

[DDSDataReader](#) entity and associated elements

##### Classes

- struct [DDS\\_RequestedDeadlineMissedStatus](#)  
[DDS\\_REQUESTED\\_DEADLINE\\_MISSED\\_STATUS](#)
- struct [DDS\\_LivelinessChangedStatus](#)  
[DDS\\_LIVELINESS\\_CHANGED\\_STATUS](#)
- struct [DDS\\_RequestedIncompatibleQosStatus](#)  
[DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)
- struct [DDS\\_SampleRejectedStatus](#)  
[DDS\\_SAMPLE\\_REJECTED\\_STATUS](#)
- struct [DDS\\_DataReaderInstanceReplacedStatus](#)  
[<<eXtension>> DDS\\_INSTANCE\\_REPLACED\\_STATUS.](#)
- struct [DDS\\_SubscriptionMatchedStatus](#)  
[DDS\\_SUBSCRIPTION\\_MATCHED\\_STATUS](#)
- struct [DDS\\_SampleLostStatus](#)  
[DDS\\_SAMPLE\\_LOST\\_STATUS](#)
- struct [DDS\\_DataReaderQos](#)  
*QoS policies supported by a [DDSDataReader](#) entity.*
- class [FooDataReader](#)  
*Declares the interface required to support a user data type-specific data reader.*
- class [DDSDataReaderListener](#)  
[<<interface>> DDSListener](#) for reader status.
- class [DDSDataReader](#)  
[<<interface>>](#) Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached [DDSSubscriber](#).

##### Enumerations

- enum [DDS\\_SampleRejectedStatusKind](#) {  
[DDS\\_NOT\\_REJECTED](#) , [DDS\\_REJECTED\\_BY\\_INSTANCES\\_LIMIT](#) , [DDS\\_REJECTED\\_BY\\_SAMPLES\\_LIMIT](#)  
 , [DDS\\_REJECTED\\_BY\\_SAMPLES\\_PER\\_INSTANCE\\_LIMIT](#) ,  
[DDS\\_REJECTED\\_BY\\_REMOTE\\_WRITERS\\_LIMIT](#) , [DDS\\_REJECTED\\_BY\\_SAMPLES\\_PER\\_REMOTE\\_WRITER\\_LIMIT](#)  
 }  
*Kinds of reasons for rejecting a sample.*
- enum [DDS\\_SampleLostStatusKind](#) {  
[DDS\\_SAMPLE\\_LOST\\_NOT\\_LOST](#) , [DDS\\_SAMPLE\\_LOST\\_BY\\_DATAWRITER](#) , [DDS\\_SAMPLE\\_LOST\\_BY\\_MAX\\_SAMPLES\\_LIM](#)  
 , [DDS\\_SAMPLE\\_LOST\\_BY\\_HISTORY\\_DEPTH\\_LIMIT](#) ,  
[DDS\\_SAMPLE\\_LOST\\_BY\\_META\\_SAMPLE\\_LIMIT](#) , [DDS\\_SAMPLE\\_LOST\\_BY\\_NOT\\_READ\\_ON\\_CACHE\\_DELETION](#)  
 , [DDS\\_SAMPLE\\_LOST\\_BY\\_MANAGEDPOOL\\_OVERWRITE](#) }  
*Kinds of reasons a sample was lost.*

### 6.2.9.4.1 Detailed Description

[DDSDataReader](#) entity and associated elements

### 6.2.9.4.2 Enumeration Type Documentation

#### DDS\_SampleRejectedStatusKind

enum [DDS\\_SampleRejectedStatusKind](#)

Kinds of reasons for rejecting a sample.

Enumerator

|                                                     |                                                                                                                                                                                                                                                              |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_NOT_REJECTED                                    | Samples are never rejected.<br><br>See also<br><a href="#">DDS_ResourceLimitsQosPolicy</a>                                                                                                                                                                   |
| DDS_REJECTED_BY_INSTANCES_LIMIT                     | A resource limit on the number of instances was reached.<br><br>See also<br><a href="#">DDS_ResourceLimitsQosPolicy</a>                                                                                                                                      |
| DDS_REJECTED_BY_SAMPLES_LIMIT                       | A resource limit on the number of samples was reached.<br><br>See also<br><a href="#">DDS_ResourceLimitsQosPolicy</a>                                                                                                                                        |
| DDS_REJECTED_BY_SAMPLES_PER_INSTANCE↔<br>_LIMIT     | A resource limit on the number of samples per instance was reached.<br><br>See also<br><a href="#">DDS_ResourceLimitsQosPolicy</a>                                                                                                                           |
| DDS_REJECTED_BY_REMOTE_WRITERS_LIMIT                | A resource limit on the number of remote writers from which a <a href="#">DDSDataReader</a> may read was reached. This constant is an extension to the DDS standard.<br><br>See also<br><a href="#">DDS_DataReaderResourceLimitsQosPolicy</a>                |
| DDS_REJECTED_BY_SAMPLES_PER_REMOTE↔<br>WRITER_LIMIT | A resource limit on the number of samples from a given remote writer that a <a href="#">DDSDataReader</a> may store was reached. This constant is an extension to the DDS standard.<br><br>See also<br><a href="#">DDS_DataReaderResourceLimitsQosPolicy</a> |

**DDS\_SampleLostStatusKind**

enum [DDS\\_SampleLostStatusKind](#)

Kinds of reasons a sample was lost.

**Enumerator**

|                                                |                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_SAMPLE_LOST_NOT_LOST                       | <p>Samples was not lost, normal removal.</p> <p>See also</p> <p><a href="#">DDS_ResourceLimitsQosPolicy</a></p>                                                                                                                                                                                                                                |
| DDS_SAMPLE_LOST_BY_DATAWRITER                  | <p>Sample was lost was by the DataWriter, it was never received by the DataReader.</p> <p>See also</p> <p><a href="#">DDS_ResourceLimitsQosPolicy</a></p>                                                                                                                                                                                      |
| DDS_SAMPLE_LOST_BY_MAX_SAMPLES_LIMIT           | <p>A sample was received by the DataReader but lost because the max_samples resource was exceeded and not samples could be freed to receive it.</p> <p>See also</p> <p><a href="#">DDS_ResourceLimitsQosPolicy</a></p>                                                                                                                         |
| DDS_SAMPLE_LOST_BY_HISTORY_DEPTH_LIMIT         | <p>A sample was received by the DataReader but lost because the max depth was reached for the instance and no samples could be removed from the history. For example, if all the samples in the DataReader cache are loaned by the application, they cannot be removed.</p> <p>See also</p> <p><a href="#">DDS_ResourceLimitsQosPolicy</a></p> |
| DDS_SAMPLE_LOST_BY_META_SAMPLE_LIMIT           | <p>A meta sample was received by the DataReader but lost because the meta sample for the instance was loaned by the application and could not be reused.</p> <p>See also</p> <p><a href="#">DDS_ResourceLimitsQosPolicy</a></p>                                                                                                                |
| DDS_SAMPLE_LOST_BY_NOT_READ_ON_CACHE_DELETION↔ | <p>The DataReader has samples in the cache that were not seen by the application and were lost when the DataReader was deleted.</p> <p>See also</p> <p><a href="#">DDS_ResourceLimitsQosPolicy</a></p>                                                                                                                                         |
| DDS_SAMPLE_LOST_BY_MANAGEDPOOL_OVERWRITE↔      | <p>The DataReader has samples that may have been overwritten.</p> <p>See also</p> <p><a href="#">DDS_ResourceLimitsQosPolicy</a></p>                                                                                                                                                                                                           |

### 6.2.9.5 Data Sample

[DDS\\_SampleInfo](#), [DDS\\_SampleStateKind](#), [DDS\\_ViewStateKind](#), [DDS\\_InstanceStateKind](#) and associated elements

#### Topics

- [Sample States](#)  
*[DDS\\_SampleStateKind](#) and associated elements*
- [View States](#)  
*[DDS\\_ViewStateKind](#) and associated elements*
- [Instance States](#)  
*[DDS\\_InstanceStateKind](#) and associated elements*

#### Classes

- struct [DDS\\_SampleInfo](#)  
*Information that accompanies each sample that is read or taken.*
- struct [DDS\\_SampleInfoSeq](#)  
*Declares IDL sequence `< DDS\_SampleInfo >` .*

#### 6.2.9.5.1 Detailed Description

[DDS\\_SampleInfo](#), [DDS\\_SampleStateKind](#), [DDS\\_ViewStateKind](#), [DDS\\_InstanceStateKind](#) and associated elements

#### 6.2.9.5.2 Sample States

[DDS\\_SampleStateKind](#) and associated elements

#### Typedefs

- typedef [DDS\\_UnsignedLong](#) [DDS\\_SampleStateMask](#)  
*A bit-mask (list) of sample states, i.e. [DDS\\_SampleStateKind](#).*

#### Enumerations

- enum [DDS\\_SampleStateKind](#) { [DDS\\_READ\\_SAMPLE\\_STATE](#) = 0x0001 << 0 , [DDS\\_NOT\\_READ\\_SAMPLE\\_STATE](#) = 0x0001 << 1 }
- Indicates whether or not a sample has ever been read.*

#### Variables

- const [DDS\\_SampleStateMask](#) [DDS\\_ANY\\_SAMPLE\\_STATE](#)  
*Any sample state [DDS\\_READ\\_SAMPLE\\_STATE](#) | [DDS\\_NOT\\_READ\\_SAMPLE\\_STATE](#).*

## Detailed Description

[DDS\\_SampleStateKind](#) and associated elements

## Typedef Documentation

### DDS\_SampleStateMask

```
typedef DDS\_UnsignedLong DDS_SampleStateMask
```

A bit-mask (list) of sample states, i.e. [DDS\\_SampleStateKind](#).

## Enumeration Type Documentation

### DDS\_SampleStateKind

```
enum DDS\_SampleStateKind
```

Indicates whether or not a sample has ever been read.

For each sample received, the middleware internally maintains a sample\_state relative to each [DDSDataReader](#). The sample state can be either:

- [DDS\\_READ\\_SAMPLE\\_STATE](#) indicates that the [DDSDataReader](#) has already accessed that sample by means of a read or take operation.
- [DDS\\_NOT\\_READ\\_SAMPLE\\_STATE](#) indicates that the [DDSDataReader](#) has not accessed that sample before.

The sample state will, in general, be different for each sample in the collection returned by read or take.

#### Enumerator

|                                           |                           |
|-------------------------------------------|---------------------------|
| <a href="#">DDS_READ_SAMPLE_STATE</a>     | Sample has been read.     |
| <a href="#">DDS_NOT_READ_SAMPLE_STATE</a> | Sample has not been read. |

## Variable Documentation

### DDS\_ANY\_SAMPLE\_STATE

```
const DDS\_SampleStateMask DDS_ANY_SAMPLE_STATE [extern]
```

Any sample state [DDS\\_READ\\_SAMPLE\\_STATE](#) | [DDS\\_NOT\\_READ\\_SAMPLE\\_STATE](#).

### 6.2.9.5.3 View States

[DDS\\_ViewStateKind](#) and associated elements

#### Typedefs

- typedef [DDS\\_UnsignedLong](#) [DDS\\_ViewStateMask](#)  
*A bit-mask (list) of view states, i.e. [DDS\\_ViewStateKind](#).*

#### Enumerations

- enum [DDS\\_ViewStateKind](#) { [DDS\\_NEW\\_VIEW\\_STATE](#) = 0x0001 << 0 , [DDS\\_NOT\\_NEW\\_VIEW\\_STATE](#) = 0x0001 << 1 }
- Indicates whether or not an instance is new.*

#### Variables

- const [DDS\\_ViewStateMask](#) [DDS\\_ANY\\_VIEW\\_STATE](#)  
*Any view state [DDS\\_NEW\\_VIEW\\_STATE](#) | [DDS\\_NOT\\_NEW\\_VIEW\\_STATE](#).*

#### Detailed Description

[DDS\\_ViewStateKind](#) and associated elements

#### Typedef Documentation

##### **DDS\_ViewStateMask**

```
typedef DDS\_UnsignedLong DDS\_ViewStateMask
```

A bit-mask (list) of view states, i.e. [DDS\\_ViewStateKind](#).

## Enumeration Type Documentation

### DDS\_ViewStateKind

enum [DDS\\_ViewStateKind](#)

Indicates whether or not an instance is new.

For each instance (identified by the key), the middleware internally maintains a view state relative to each [DDSDataReader](#). The view state can be either:

- [DDS\\_NEW\\_VIEW\\_STATE](#) indicates that either this is the first time that the [DDSDataReader](#) has ever accessed samples of that instance, or else that the [DDSDataReader](#) has accessed previous samples of the instance, but the instance has since been reborn (i.e. become not-alive and then alive again).
- [DDS\\_NOT\\_NEW\\_VIEW\\_STATE](#) indicates that the [DDSDataReader](#) has already accessed samples of the same instance and that the instance has not been reborn since.

The `view_state` available in the [DDS\\_SampleInfo](#) is a snapshot of the view state of the instance relative to the [DDSDataReader](#) used to access the samples at the time the collection was obtained (i.e. at the time `read` or `take` was called). The `view_state` is therefore the same for all samples in the returned collection that refer to the same instance.

Once an instance has been detected as not having any "live" writers and all the samples associated with the instance are "taken" from the [DDSDataReader](#), the middleware can reclaim all local resources regarding the instance. Future samples will be treated as "never seen."

#### Enumerator

|                                        |                                                                                          |
|----------------------------------------|------------------------------------------------------------------------------------------|
| <a href="#">DDS_NEW_VIEW_STATE</a>     | New instance. This latest generation of the instance has not previously been accessed.   |
| <a href="#">DDS_NOT_NEW_VIEW_STATE</a> | Not a new instance. This latest generation of the instance has previously been accessed. |

## Variable Documentation

### DDS\_ANY\_VIEW\_STATE

const [DDS\\_ViewStateMask](#) [DDS\\_ANY\\_VIEW\\_STATE](#) [extern]

Any view state [DDS\\_NEW\\_VIEW\\_STATE](#) | [DDS\\_NOT\\_NEW\\_VIEW\\_STATE](#).

#### 6.2.9.5.4 Instance States

[DDS\\_InstanceStateKind](#) and associated elements

## Typedefs

- typedef [DDS\\_UnsignedLong](#) [DDS\\_InstanceStateMask](#)  
A bit-mask (list) of instance states, i.e. [DDS\\_InstanceStateKind](#).

## Enumerations

- enum [DDS\\_InstanceStateKind](#) { [DDS\\_ALIVE\\_INSTANCE\\_STATE](#) = 0x0001 << 0 , [DDS\\_NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) = 0x0001 << 1 , [DDS\\_NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#) = 0x0001 << 2 }
- Indicates if the samples are from a live [DDSDataWriter](#) or not.

## Variables

- const [DDS\\_InstanceStateMask](#) [DDS\\_ANY\\_INSTANCE\\_STATE](#)  
Any instance state [ALIVE\\_INSTANCE\\_STATE](#) | [NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) | [NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#).
- const [DDS\\_InstanceStateMask](#) [DDS\\_NOT\\_ALIVE\\_INSTANCE\\_STATE](#)  
Not alive instance state [NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) | [NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#).

## Detailed Description

[DDS\\_InstanceStateKind](#) and associated elements

## Typedef Documentation

### DDS\_InstanceStateMask

```
typedef DDS\_UnsignedLong DDS\_InstanceStateMask
```

A bit-mask (list) of instance states, i.e. [DDS\\_InstanceStateKind](#).

## Enumeration Type Documentation

### DDS\_InstanceStateKind

```
enum DDS\_InstanceStateKind
```

Indicates if the samples are from a live [DDSDataWriter](#) or not.

For each instance, the middleware internally maintains an instance state. The instance state can be:

- [DDS\\_ALIVE\\_INSTANCE\\_STATE](#) indicates that (a) samples have been received for the instance, (b) there are live [DDSDataWriter](#) entities writing the instance, and (c) the instance has not been explicitly disposed (or else more samples have been received after it was disposed).



- [DDS\\_NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) indicates the instance was explicitly disposed by a [DDSDDataWriter](#) by means of the dispose operation.
- [DDS\\_NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#) indicates the instance has been declared as not-alive by the [DDSDDataReader](#) because it detected that there are no live [DDSDDataWriter](#) entities writing that instance.

The precise behavior events that cause the instance state to change depends on the setting of the OWNERSHIP QoS:

- If [OWNERSHIP](#) is set to [DDS\\_EXCLUSIVE\\_OWNERSHIP\\_QOS](#), then the instance state becomes [DDS\\_NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) only if the [DDSDDataWriter](#) that "owns" the instance explicitly disposes it. The instance state becomes [DDS\\_ALIVE\\_INSTANCE\\_STATE](#) again only if the [DDSDDataWriter](#) that owns the instance writes it.
- If [OWNERSHIP](#) is set to [DDS\\_SHARED\\_OWNERSHIP\\_QOS](#), then the instance state becomes [DDS\\_NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) if any [DDSDDataWriter](#) explicitly disposes the instance. The instance state becomes [DDS\\_ALIVE\\_INSTANCE\\_STATE](#) as soon as any [DDSDDataWriter](#) writes the instance again.

The instance state available in the [DDS\\_SampleInfo](#) is a snapshot of the instance state of the instance at the time the collection was obtained (i.e. at the time read or take was called). The instance state is therefore the same for all samples in the returned collection that refer to the same instance.

#### Enumerator

|                                                         |                                                                                                                                                                                    |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">DDS_ALIVE_INSTANCE_STATE</a>                | Instance is currently in existence.                                                                                                                                                |
| <a href="#">DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE</a>   | Not alive disposed instance. The instance has been disposed by a DataWriter.                                                                                                       |
| <a href="#">DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE</a> | Not alive no writers for instance. None of the <a href="#">DDSDDataWriter</a> objects are currently alive (according to the <a href="#">LIVELINESS</a> ) are writing the instance. |

#### Variable Documentation

##### DDS\_ANY\_INSTANCE\_STATE

```
const DDS\_InstanceStateMask DDS_ANY_INSTANCE_STATE [extern]
```

Any instance state [ALIVE\\_INSTANCE\\_STATE](#) | [NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) | [NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#).

##### DDS\_NOT\_ALIVE\_INSTANCE\_STATE

```
const DDS\_InstanceStateMask DDS_NOT_ALIVE_INSTANCE_STATE [extern]
```

Not alive instance state [NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) | [NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#).

### 6.2.10 Infrastructure Module

Defines the DDS infrastructure package.

#### Topics

- [Time Support](#)  
*Time and duration types and defines.*
- [GUID Support](#)  
*GUID type and defines.*
- [Sequence Number Support](#)  
*Sequence number type and defines.*
- [Return Codes](#)  
*Types of return codes.*
- [Status Kinds](#)  
*Kinds of communication status.*
- [QoS Policies](#)  
*Quality of Service (QoS) policies.*
- [Conditions and WaitSets](#)  
*[DDSCondition](#) and [DDSWaitSet](#) and related items.*
- [Entity Support](#)  
*[DDSEntity](#), [DDSListener](#) and related items.*
- [Sequence Support](#)  
*Defines sequence interface and primitive data types sequences.*
- [Exception codes](#)  
*Exception codes.*
- [Type Support](#)  
*Type Plugin.*
- [String Support](#)  
*String creation, cloning, assignment, and deletion.*

#### Classes

- struct [DDS\\_SampleIdentity\\_t](#)  
*Data structure used to provide explicit identity information about a published data sample.*
- struct [DDS\\_WriteParams\\_t](#)  
*Data structure used to provide custom parameters to a [DataWriter](#)'s write operation.*

#### Macros

- `#define` [DDS\\_WRITEPARAMS\\_DEFAULT](#)  
*Default values used to initialize instances of [DDS\\_WriteParams\\_t](#).*

## Enumerations

- enum `DDS_EntityKind_t` {  
`DDS_UNKNOWN_ENTITY_KIND = 0` , `DDS_PARTICIPANT_ENTITY_KIND = 1` , `DDS_PUBLISHER_ENTITY_KIND = 2` , `DDS_SUBSCRIBER_ENTITY_KIND = 3` ,  
`DDS_TOPIC_ENTITY_KIND = 4` , `DDS_DATAREADER_ENTITY_KIND = 5` , `DDS_DATAWRITER_ENTITY_KIND = 6` }

*Type of a DDS entity.*

### 6.2.10.1 Detailed Description

Defines the DDS infrastructure package.

Defines the abstract classes and the interfaces that are refined by the other modules. Contains common definitions such as return codes, status values, and QoS policies.

This diagram defines the elements that are defined in the DDS specification, and their relationships to each other. Not all entities and functions shown in the diagram are supported by RTI Connex DDS Micro. Please read the information on the individual types to determine whether a function is supported.

### 6.2.10.2 Macro Definition Documentation

#### DDS\_WRITEPARAMS\_DEFAULT

```
#define DDS_WRITEPARAMS_DEFAULT
```

#### Value:

```
{ \
    DDS_SAMPLE_IDENTITY_UNKNOWN, \
    DDS_SAMPLE_IDENTITY_UNKNOWN, \
    DDS_TIME_ZERO, \
    DDS_HANDLE_NIL_NATIVE \
}
```

Default values used to initialize instances of `DDS_WriteParams_t`.

### 6.2.10.3 Enumeration Type Documentation

#### DDS\_EntityKind\_t

```
enum DDS_EntityKind_t
```

Type of a DDS entity.

#### Enumerator

|                                          |                                                    |
|------------------------------------------|----------------------------------------------------|
| <code>DDS_UNKNOWN_ENTITY_KIND</code>     | Entity is of an unknown type.                      |
| <code>DDS_PARTICIPANT_ENTITY_KIND</code> | Entity is a <a href="#">DDSDomainParticipant</a> . |
| <code>DDS_PUBLISHER_ENTITY_KIND</code>   | Entity is a <a href="#">DDSPublisher</a> .         |
| <code>DDS_SUBSCRIBER_ENTITY_KIND</code>  | Entity is a <a href="#">DDSSubscriber</a> .        |
| <code>DDS_TOPIC_ENTITY_KIND</code>       | Entity is a <a href="#">DDSTopic</a> .             |
| <code>DDS_DATAREADER_ENTITY_KIND</code>  | Entity is a <a href="#">DDSDataReader</a> .        |
| <code>DDS_DATAWRITER_ENTITY_KIND</code>  | Entity is a <a href="#">DDSDataWriter</a> .        |

### 6.2.10.4 Time Support

Time and duration types and defines.

#### Classes

- struct [DDS\\_Time\\_t](#)  
*Type for time representation.*
- struct [DDS\\_Duration\\_t](#)  
*Type for duration representation.*

#### Macros

- #define [DDS\\_TIME\\_ZERO](#)  
*The default instant in time: zero seconds and zero nanoseconds.*

#### Functions

- [DDS\\_Boolean DDS\\_Time\\_greater\\_than](#) (const struct [DDS\\_Time\\_t](#) \*l, const struct [DDS\\_Time\\_t](#) \*r)
- [DDS\\_Boolean DDS\\_Time\\_is\\_zero](#) (const struct [DDS\\_Time\\_t](#) \*time)  
*Check if time is zero.*
- [DDS\\_Boolean DDS\\_Duration\\_is\\_infinite](#) (const struct [DDS\\_Duration\\_t](#) \*duration)
- [DDS\\_Boolean DDS\\_Duration\\_equal](#) (const struct [DDS\\_Duration\\_t](#) \*self, const struct [DDS\\_Duration\\_t](#) \*other)
- [DDS\\_Boolean DDS\\_Duration\\_is\\_zero](#) (const struct [DDS\\_Duration\\_t](#) \*duration)

#### Variables

- const [DDS\\_Long DDS\\_TIME\\_INVALID\\_SEC](#)  
*A sentinel indicating an invalid second of time.*
- const [DDS\\_UnsignedLong DDS\\_TIME\\_INVALID\\_NSEC](#)  
*A sentinel indicating an invalid nano-second of time.*
- const struct [DDS\\_Time\\_t DDS\\_TIME\\_INVALID](#)  
*A sentinel indicating an invalid time.*
- const [DDS\\_Long DDS\\_DURATION\\_INFINITE\\_SEC](#)  
*An infinite second period of time.*
- const [DDS\\_UnsignedLong DDS\\_DURATION\\_INFINITE\\_NSEC](#)  
*An infinite nano-second period of time.*
- const struct [DDS\\_Duration\\_t DDS\\_DURATION\\_INFINITE](#)  
*An infinite period of time.*
- const [DDS\\_Long DDS\\_DURATION\\_ZERO\\_SEC](#)  
*A zero-length second period of time.*
- const [DDS\\_UnsignedLong DDS\\_DURATION\\_ZERO\\_NSEC](#)  
*A zero-length nano-second period of time.*
- const struct [DDS\\_Duration\\_t DDS\\_DURATION\\_ZERO](#)  
*A zero-length period of time.*

#### 6.2.10.4.1 Detailed Description

Time and duration types and defines.

#### 6.2.10.4.2 Macro Definition Documentation

##### DDS\_TIME\_ZERO

```
#define DDS_TIME_ZERO
```

The default instant in time: zero seconds and zero nanoseconds.

#### 6.2.10.4.3 Function Documentation

##### DDS\_Time\_greater\_than()

```
DDS_Boolean DDS_Time_greater_than (  
    const struct DDS_Time_t * l,  
    const struct DDS_Time_t * r)
```

Returns

[DDS\\_BOOLEAN\\_TRUE](#) if the first time is greater than the second.

##### DDS\_Time\_is\_zero()

```
DDS_Boolean DDS_Time_is_zero (  
    const struct DDS_Time_t * time)
```

Check if time is zero.

Returns

[DDS\\_BOOLEAN\\_TRUE](#) if the given time is equal to [DDS\\_TIME\\_ZERO](#) or [DDS\\_BOOLEAN\\_FALSE](#) otherwise.

References [DDS\\_TIME\\_INVALID](#), [DDS\\_TIME\\_INVALID\\_NSEC](#), and [DDS\\_TIME\\_INVALID\\_SEC](#).

##### DDS\_Duration\_is\_infinite()

```
DDS_Boolean DDS_Duration_is_infinite (  
    const struct DDS_Duration_t * duration)
```

Returns

[DDS\\_BOOLEAN\\_TRUE](#) if the given duration is of infinite length.

References [RTI\\_INT32](#), and [RTI\\_UINT32](#).

##### DDS\_Duration\_equal()

```
DDS_Boolean DDS_Duration_equal (  
    const struct DDS_Duration_t * self,  
    const struct DDS_Duration_t * other)
```

## Parameters

|              |                                                                      |
|--------------|----------------------------------------------------------------------|
| <i>self</i>  | << <i>in</i> >> Duration to compare. Cannot be NULL.                 |
| <i>other</i> | << <i>in</i> >> Duration to be compared to this one. Cannot be NULL. |

## Returns

DDS\_BOOLEAN\_TRUE if the two objects contain the same value, DDS\_BOOLEAN\_FALSE otherwise.

References [DDS\\_DURATION\\_ZERO](#), [DDS\\_DURATION\\_ZERO\\_NSEC](#), and [DDS\\_DURATION\\_ZERO\\_SEC](#).

**DDS\_Duration\_is\_zero()**

```
DDS_Boolean DDS_Duration_is_zero (
    const struct DDS_Duration_t * duration)
```

## Returns

[DDS\\_BOOLEAN\\_TRUE](#) if the given duration is of zero length.

**6.2.10.4.4 Variable Documentation****DDS\_TIME\_INVALID\_SEC**

```
const DDS_Long DDS_TIME_INVALID_SEC [extern]
```

A sentinel indicating an invalid second of time.

Referenced by [DDS\\_Time\\_is\\_zero\(\)](#).

**DDS\_TIME\_INVALID\_NSEC**

```
const DDS_UnsignedLong DDS_TIME_INVALID_NSEC [extern]
```

A sentinel indicating an invalid nano-second of time.

Referenced by [DDS\\_Time\\_is\\_zero\(\)](#).

**DDS\_TIME\_INVALID**

```
const struct DDS_Time_t DDS_TIME_INVALID [extern]
```

A sentinel indicating an invalid time.

Referenced by [DDS\\_Time\\_is\\_zero\(\)](#).

**DDS\_DURATION\_INFINITE\_SEC**

```
const DDS\_Long DDS_DURATION_INFINITE_SEC [extern]
```

An infinite second period of time.

**DDS\_DURATION\_INFINITE\_NSEC**

```
const DDS\_UnsignedLong DDS_DURATION_INFINITE_NSEC [extern]
```

An infinite nano-second period of time.

**DDS\_DURATION\_INFINITE**

```
const struct DDS\_Duration\_t DDS_DURATION_INFINITE [extern]
```

An infinite period of time.

**DDS\_DURATION\_ZERO\_SEC**

```
const DDS\_Long DDS_DURATION_ZERO_SEC [extern]
```

A zero-length second period of time.

Referenced by [DDS\\_Duration\\_equal\(\)](#).

**DDS\_DURATION\_ZERO\_NSEC**

```
const DDS\_UnsignedLong DDS_DURATION_ZERO_NSEC [extern]
```

A zero-length nano-second period of time.

Referenced by [DDS\\_Duration\\_equal\(\)](#).

**DDS\_DURATION\_ZERO**

```
const struct DDS\_Duration\_t DDS_DURATION_ZERO [extern]
```

A zero-length period of time.

Referenced by [DDS\\_Duration\\_equal\(\)](#).

### 6.2.10.5 GUID Support

GUID type and defines.

#### Classes

- struct [DDS\\_GUID\\_t](#)  
*Type for GUID (Global Unique Identifier) representation.*

#### Typedefs

- typedef struct DDS\_GUID\_t [DDS\\_GUID\\_t](#)  
*Type for GUID (Global Unique Identifier) representation.*

#### Variables

- const struct [DDS\\_GUID\\_t](#) [DDS\\_GUID\\_UNKNOWN](#)  
*Unknown GUID.*

#### 6.2.10.5.1 Detailed Description

GUID type and defines.

#### 6.2.10.5.2 Typedef Documentation

##### DDS\_GUID\_t

```
typedef struct DDS_GUID_t DDS_GUID_t
```

Type for *GUID* (Global Unique Identifier) representation.

Represents a 128 bit GUID.

#### 6.2.10.5.3 Variable Documentation

##### DDS\_GUID\_UNKNOWN

```
const struct DDS\_GUID\_t DDS_GUID_UNKNOWN [extern]
```

Unknown GUID.



### 6.2.10.6 Sequence Number Support

Sequence number type and defines.

#### Classes

- struct [DDS\\_SequenceNumber\\_t](#)  
*Type for sequence number representation.*

#### Functions

- int [DDS\\_SequenceNumber\\_compare](#) (const struct [DDS\\_SequenceNumber\\_t](#) \*sn1, const struct [DDS\\_SequenceNumber\\_t](#) \*sn2)  
*Compares two sequence numbers.*

#### 6.2.10.6.1 Detailed Description

Sequence number type and defines.

#### 6.2.10.6.2 Function Documentation

##### DDS\_SequenceNumber\_compare()

```
int DDS_SequenceNumber_compare (  
    const struct DDS\_SequenceNumber\_t * sn1,  
    const struct DDS\_SequenceNumber\_t * sn2)
```

Compares two sequence numbers.

#### Parameters

|            |                                                             |
|------------|-------------------------------------------------------------|
| <i>sn1</i> | << <i>in</i> >> Sequence number to compare. Cannot be NULL. |
| <i>sn2</i> | << <i>in</i> >> Sequence number to compare. Cannot be NULL. |

#### Returns

If the two sequence numbers are equal, the function returns 0. If sn1 is greater than sn2 the function returns a positive number; otherwise, it returns a negative number.

### 6.2.10.7 Return Codes

Types of return codes.

## Enumerations

- enum `DDS_ReturnCode_t` {  
`DDS_RETCODE_OK`, `DDS_RETCODE_ERROR`, `DDS_RETCODE_UNSUPPORTED`, `DDS_RETCODE_BAD_PARAMETER`  
`,`  
`DDS_RETCODE_PRECONDITION_NOT_MET`, `DDS_RETCODE_OUT_OF_RESOURCES`, `DDS_RETCODE_NOT_ENABLED`  
`,`  
`DDS_RETCODE_IMMUTABLE_POLICY`,  
`DDS_RETCODE_INCONSISTENT_POLICY`, `DDS_RETCODE_ALREADY_DELETED`, `DDS_RETCODE_TIMEOUT`  
`,`  
`DDS_RETCODE_NO_DATA`,  
`DDS_RETCODE_ILLEGAL_OPERATION`, `DDS_RETCODE_NOT_ALLOWED_BY_SEC` }

*Type for return codes.*

### 6.2.10.7.1 Detailed Description

Types of return codes.

### 6.2.10.7.2 Standard Return Codes

Any operation with return type `DDS_ReturnCode_t` may return `DDS_RETCODE_OK`, `DDS_RETCODE_ERROR` or `DDS_RETCODE_ILLEGAL_OPERATION`. Any operation that takes one or more input parameters may additionally return `DDS_RETCODE_BAD_PARAMETER`. Any operation on an object created from any of the factories may additionally return `DDS_RETCODE_ALREADY_DELETED`. Any operation that is stated as optional may additionally return `DDS_RETCODE_UNSUPPORTED`.

Thus, the standard return codes are:

- `DDS_RETCODE_ERROR`
- `DDS_RETCODE_ILLEGAL_OPERATION`
- `DDS_RETCODE_ALREADY_DELETED`
- `DDS_RETCODE_BAD_PARAMETER`
- `DDS_RETCODE_UNSUPPORTED`

Operations that may return any of the additional return codes will state so explicitly.

### 6.2.10.7.3 Enumeration Type Documentation

#### `DDS_ReturnCode_t`

enum `DDS_ReturnCode_t`

Type for return codes.

Errors are modeled as operation return codes of this type.

## Enumerator

|                                  |                                                                                                                                                                                                                                                                                                                     |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_RETCODE_OK                   | Successful return.                                                                                                                                                                                                                                                                                                  |
| DDS_RETCODE_ERROR                | Generic, unspecified error.                                                                                                                                                                                                                                                                                         |
| DDS_RETCODE_UNSUPPORTED          | Unsupported operation. Can only returned by operations that are unsupported.                                                                                                                                                                                                                                        |
| DDS_RETCODE_BAD_PARAMETER        | Illegal parameter value. The value of the parameter that is passed in has an illegal value. Things that falls into this category include NULL parameters and parameter values that are out of range.                                                                                                                |
| DDS_RETCODE_PRECONDITION_NOT_MET | A pre-condition for the operation was not met. The system is not in the expected state when the function is called, or the parameter itself is not in the expected state when the function is called.                                                                                                               |
| DDS_RETCODE_OUT_OF_RESOURCES     | RTI Connex DDS Micro ran out of the resources needed to complete the operation.                                                                                                                                                                                                                                     |
| DDS_RETCODE_NOT_ENABLED          | Operation invoked on a <a href="#">DDSEntity</a> that is not yet enabled.                                                                                                                                                                                                                                           |
| DDS_RETCODE_IMMUTABLE_POLICY     | Application attempted to modify an immutable QoS policy.                                                                                                                                                                                                                                                            |
| DDS_RETCODE_INCONSISTENT_POLICY  | Application specified a set of QoS policies that are not consistent with each other.                                                                                                                                                                                                                                |
| DDS_RETCODE_ALREADY_DELETED      | The object target of this operation has already been deleted.                                                                                                                                                                                                                                                       |
| DDS_RETCODE_TIMEOUT              | The operation timed out.                                                                                                                                                                                                                                                                                            |
| DDS_RETCODE_NO_DATA              | Indicates a transient situation where the operation did not return any data but there is no inherent error.                                                                                                                                                                                                         |
| DDS_RETCODE_ILLEGAL_OPERATION    | The operation was called under improper circumstances. An operation was invoked on an inappropriate object or at an inappropriate time. This return code is similar to <a href="#">DDS_RETCODE_PRECONDITION_NOT_MET</a> , except that there is no precondition that could be changed to make the operation succeed. |
| DDS_RETCODE_NOT_ALLOWED_BY_SEC   | An operation on the DDS API that fails because the security plugins do not allow it. An operation on the DDS API that fails because the security plugins do not allow it.                                                                                                                                           |

## 6.2.10.8 Status Kinds

Kinds of communication status.

## Macros

- `#define DDS_STATUS_MASK_NONE`  
*No bits are set.*
- `#define DDS_STATUS_MASK_ALL`  
*All bits are set.*

## Typedefs

- typedef [DDS\\_UnsignedLong](#) [DDS\\_StatusMask](#)  
A bit-mask (list) of concrete status types, i.e. [DDS\\_StatusKind](#)[].

## Enumerations

- enum [DDS\\_StatusKind](#) {  
[DDS\\_INCONSISTENT\\_TOPIC\\_STATUS](#) , [DDS\\_OFFERED\\_DEADLINE\\_MISSED\\_STATUS](#) , [DDS\\_REQUESTED\\_DEADLINE\\_MISSED\\_STATUS](#) ,  
[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) ,  
[DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) , [DDS\\_SAMPLE\\_LOST\\_STATUS](#) , [DDS\\_SAMPLE\\_REJECTED\\_STATUS](#) ,  
[DDS\\_DATA\\_ON\\_READERS\\_STATUS](#) ,  
[DDS\\_DATA\\_AVAILABLE\\_STATUS](#) , [DDS\\_LIVELINESS\\_LOST\\_STATUS](#) , [DDS\\_LIVELINESS\\_CHANGED\\_STATUS](#) ,  
[DDS\\_PUBLICATION\\_MATCHED\\_STATUS](#) ,  
[DDS\\_SUBSCRIPTION\\_MATCHED\\_STATUS](#) , [DDS\\_INSTANCE\\_REPLACED\\_STATUS](#) , [DDS\\_RELIABLE\\_READER\\_ACTIVITY\\_CHANGED\\_STATUS](#) ,  
[DDS\\_DATA\\_WRITER\\_SAMPLE\\_REMOVED\\_STATUS](#) }  
Type for status kinds.

### 6.2.10.8.1 Detailed Description

Kinds of communication status.

Entity:

[DDSEntity](#)

QoS:

[QoS Policies](#)

Listener:

[DDSListener](#)

Each concrete [DDSEntity](#) is associated with a set of Status objects whose value represents the communication status of that entity. Each status value can be accessed with a corresponding method on the [DDSEntity](#).

RTI Connex DDS Micro supports a subset of these statuses.

When these status values change, the proper [DDSListener](#) objects are invoked to asynchronously inform the application.

An application is notified of communication status by means of the [DDSListener](#) mechanism.

The statuses may be classified into:

- read communication statuses*: i.e., those that are related to arrival of data, namely [DDS\\_DATA\\_ON\\_READERS\\_STATUS](#) and [DDS\\_DATA\\_AVAILABLE\\_STATUS](#).
- plain communication statuses*: i.e., all the others.

Read communication statuses are treated slightly differently than the others because they don't change independently. In other words, at least two changes will appear at the same time ( [DDS\\_DATA\\_ON\\_READERS\\_STATUS](#) and [DDS\\_DATA\\_AVAILABLE\\_STATUS](#) ) and even several of the last kind may be part of the set. This 'grouping' has to be communicated to the application.

For each plain communication status, there is a corresponding structure to hold the status value. These values contain the information related to the change of status, as well as information related to the statuses themselves (e.g., contains cumulative counts).

### 6.2.10.8.2 Changes in Status

Associated with each one of an [DDSEntity](#)'s communication status is a logical `StatusChangedFlag`. This flag indicates whether that particular communication status has changed since the last time the status was read by the application. The way the status changes is slightly different for the Plain Communication Status and the Read Communication status.

#### Changes in plain communication status

For the plain communication status, the `StatusChangedFlag` flag is initially set to `FALSE`. It becomes `TRUE` whenever the plain communication status changes and it is reset to `DDS_BOOLEAN_FALSE` each time the application accesses the plain communication status via the proper `get_<plain communication status>()` operation on the [DDSEntity](#).

The communication status is also reset to `FALSE` whenever the associated listener operation is called as the listener implicitly accesses the status which is passed as a parameter to the operation. The fact that the status is reset prior to calling the listener means that if the application calls the `get_<plain communication status>` from inside the listener it will see the status already reset.

An exception to this rule is when the associated listener is the 'nil' listener. The 'nil' listener is treated as a NOOP and the act of calling the 'nil' listener does not reset the communication status.

#### Changes in read communication status

For the read communication status, the `StatusChangedFlag` flag is initially set to `FALSE`. The `StatusChangedFlag` becomes `TRUE` when either a data-sample arrives or else the [DDS\\_ViewStateKind](#), [DDS\\_SampleStateKind](#), or [DDS\\_InstanceStateKind](#) of any existing sample changes for any reason other than a call to [FooDataReader::read\\_next\\_sample](#), or [FooDataReader::take\\_next\\_sample](#). Specifically any of the following events will cause the `StatusChangedFlag` to become `TRUE`:

- The arrival of new data.
- A change in the [DDS\\_InstanceStateKind](#) of a contained instance. This can be caused by either:
  - The arrival of the notification that an instance has been disposed by:
    - \* the [DDSDDataWriter](#) that owns it if `OWNERSHIP` QoS kind= `DDS_EXCLUSIVE_OWNERSHIP_QOS`
    - \* or by any [DDSDDataWriter](#) if `OWNERSHIP` QoS kind= `DDS_SHARED_OWNERSHIP_QOS`
  - The loss of liveliness of the [DDSDDataWriter](#) of an instance for which there is no other [DDSDDataWriter](#).
  - The arrival of the notification that an instance has been unregistered by the only [DDSDDataWriter](#) that is known to be writing the instance.

Depending on the kind of `StatusChangedFlag`, the flag transitions to `FALSE` again as follows:

- The [DDS\\_DATA\\_AVAILABLE\\_STATUS](#) `StatusChangedFlag` becomes `FALSE` when either the corresponding listener operation (`on_data_available`) is called or the `read_next_sample` or `take_next_sample` operation is called on the associated [DDSDDataReader](#).

- The `DDS_DATA_ON_READERS_STATUS` `StatusChangedFlag` becomes FALSE when any of the following events occurs:
  - The corresponding listener operation (`on_data_on_readers`) is called.
  - The `on_data_available` listener operation is called on any `DDSDataReader` belonging to the `DDSSubscriber`.
  - The read or take operation (or their variants) is called on any `DDSDataReader` belonging to the `DDSSubscriber`.

"Changes in `StatusChangedFlag` for read communication status"

See also

[DDSListener](#)

### 6.2.10.8.3 Macro Definition Documentation

#### DDS\_STATUS\_MASK\_NONE

```
#define DDS_STATUS_MASK_NONE
```

No bits are set.

#### DDS\_STATUS\_MASK\_ALL

```
#define DDS_STATUS_MASK_ALL
```

All bits are set.

### 6.2.10.8.4 Typedef Documentation

#### DDS\_StatusMask

```
typedef DDS_UnsignedLong DDS_StatusMask
```

A bit-mask (list) of concrete status types, i.e. `DDS_StatusKind`[].

The bit-mask is an efficient and compact representation of a fixed-length list of `DDS_StatusKind` values.

Bits in the mask correspond to different statuses. You can choose which changes in status will trigger a callback by setting the corresponding status bits in this bit-mask and installing callbacks for each of those statuses.

The bits that are true indicate that the listener will be called back for changes in the corresponding status.

For example:

```
DDS_StatusMask mask = DDS_REQUESTED_DEADLINE_MISSED_STATUS |
                     DDS_DATA_AVAILABLE_STATUS;
datareader->set_listener(listener, mask);
```

or

```
DDS_StatusMask mask = DDS_REQUESTED_DEADLINE_MISSED_STATUS |
                     DDS_DATA_AVAILABLE_STATUS;
datareader = subscriber->create_datareader(topic,
                                           DDS_DATAREADER_QOS_DEFAULT,
                                           listener, mask);
```

### 6.2.10.8.5 Enumeration Type Documentation

#### DDS\_StatusKind

enum [DDS\\_StatusKind](#)

Type for *status* kinds.

Each concrete [DDSEntity](#) is associated with a set of `*Status` objects whose values represent the communication status of that [DDSEntity](#).

The communication statuses whose changes can be communicated to the application depend on the [DDSEntity](#).

Each status value can be accessed with a corresponding method on the [DDSEntity](#). The changes on these status values cause activation of the corresponding trigger invocation of the proper [DDSListener](#) objects to asynchronously inform the application.

See also

[DDSEntity](#), [DDSListener](#)

#### Enumerator

|                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_INCONSISTENT_TOPIC_STATUS                                                                  | <p>Another topic exists with the same name but different characteristics.</p> <p>Entity:</p> <p><a href="#">DDSTopic</a></p> <p>Status:</p> <p><a href="#">DDS_InconsistentTopicStatus</a></p> <p>Listener:</p> <p><a href="#">DDSTopicListener</a></p>                                                                                                                                                      |
| DDS_OFFERED_DEADLINE_MISSED_STATUS                                                             | <p>The deadline that the <a href="#">DDSDDataWriter</a> has committed through its <a href="#">DDS_DeadlineQosPolicy</a> was not respected for a specific instance.</p> <p>Entity:</p> <p><a href="#">DDSDDataWriter</a></p> <p>QoS:</p> <p><a href="#">DEADLINE</a></p> <p>Status:</p> <p><a href="#">DDS_OfferedDeadlineMissedStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDDataWriterListener</a></p> |
| RTI Connext Micro C++ API Version 4.2.0 Copyright © Thu Sep 11 2025 Real-Time Innovations, Inc |                                                                                                                                                                                                                                                                                                                                                                                                              |

## Enumerator

|                                       |                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_REQUESTED_DEADLINE_MISSED_STATUS  | <p>The deadline that the <a href="#">DDSDataReader</a> was expecting through its <a href="#">DDS_DeadlineQosPolicy</a> was not respected for a specific instance.</p> <p>Entity:</p> <p><a href="#">DDSDataReader</a></p> <p>QoS:</p> <p><a href="#">DEADLINE</a></p> <p>Status:</p> <p><a href="#">DDS_RequestedDeadlineMissedStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDataReaderListener</a></p> |
| DDS_OFFERED_INCOMPATIBLE_QOS_STATUS   | <p>A QosPolicy value was incompatible with what was requested.</p> <p>Entity:</p> <p><a href="#">DDSDataWriter</a></p> <p>Status:</p> <p><a href="#">DDS_OfferedIncompatibleQosStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDataWriterListener</a></p>                                                                                                                                                 |
| DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS | <p>A QosPolicy value was incompatible with what is offered.</p> <p>Entity:</p> <p><a href="#">DDSDataReader</a></p> <p>Status:</p> <p><a href="#">DDS_RequestedIncompatibleQosStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDataReaderListener</a></p>                                                                                                                                                  |



## Enumerator

|                            |                                                                                                                                                                                                                                                                                   |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_SAMPLE_LOST_STATUS     | <p>A sample has been lost (i.e. was never received).</p> <p>Entity:</p> <p><a href="#">DDSSubscriber</a></p> <p>Status:</p> <p><a href="#">DDS_SampleLostStatus</a></p> <p>Listener:</p> <p><a href="#">DDSSubscriberListener</a></p>                                             |
| DDS_SAMPLE_REJECTED_STATUS | <p>A (received) sample has been rejected.</p> <p>Entity:</p> <p><a href="#">DDSDataReader</a></p> <p>QoS:</p> <p><a href="#">RESOURCE_LIMITS</a></p> <p>Status:</p> <p><a href="#">DDS_SampleRejectedStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDataReaderListener</a></p> |
| DDS_DATA_ON_READERS_STATUS | <p>New data is available.</p> <p>Entity:</p> <p><a href="#">DDSSubscriber</a></p> <p>Listener:</p> <p><a href="#">DDSSubscriberListener</a></p>                                                                                                                                   |
| DDS_DATA_AVAILABLE_STATUS  | <p>One or more new data samples have been received.</p> <p>Entity:</p> <p><a href="#">DDSDataReader</a></p> <p>Listener:</p> <p><a href="#">DDSDataReaderListener</a></p>                                                                                                         |

## Enumerator

|                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_LIVELINESS_LOST_STATUS    | <p>The liveliness that the <a href="#">DDSDataWriter</a> has committed to through its <a href="#">DDS_LivelinessQosPolicy</a> was not respected, thus <a href="#">DDSDataReader</a> entities will consider the <a href="#">DDSDataWriter</a> as no longer alive.</p> <p><b>Entity:</b></p> <p><a href="#">DDSDataWriter</a></p> <p><b>QoS:</b></p> <p><a href="#">LIVELINESS</a></p> <p><b>Status:</b></p> <p><a href="#">DDS_LivelinessLostStatus</a></p> <p><b>Listener:</b></p> <p><a href="#">DDSDataWriterListener</a></p> |
| DDS_LIVELINESS_CHANGED_STATUS | <p>The liveliness of one or more <a href="#">DDSDataWriter</a> that were writing instances read through the <a href="#">DDSDataReader</a> has changed. Some <a href="#">DDSDataWriter</a> have become alive or not_alive.</p> <p><b>Entity:</b></p> <p><a href="#">DDSDataReader</a></p> <p><b>QoS:</b></p> <p><a href="#">LIVELINESS</a></p> <p><b>Status:</b></p> <p><a href="#">DDS_LivelinessChangedStatus</a></p> <p><b>Listener:</b></p> <p><a href="#">DDSDataReaderListener</a></p>                                     |

## Enumerator

|                                 |                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_PUBLICATION_MATCHED_STATUS  | <p>The <a href="#">DDSDataWriter</a> has found <a href="#">DDSDataReader</a> that matches the <a href="#">DDSTopic</a> and has compatible QoS.</p> <p>Entity:</p> <p><a href="#">DDSDataWriter</a></p> <p>Status:</p> <p><a href="#">DDS_PublicationMatchedStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDataWriterListener</a></p>  |
| DDS_SUBSCRIPTION_MATCHED_STATUS | <p>The <a href="#">DDSDataReader</a> has found <a href="#">DDSDataWriter</a> that matches the <a href="#">DDSTopic</a> and has compatible QoS.</p> <p>Entity:</p> <p><a href="#">DDSDataReader</a></p> <p>Status:</p> <p><a href="#">DDS_SubscriptionMatchedStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDataReaderListener</a></p> |
| DDS_INSTANCE_REPLACED_STATUS    | <p>The <a href="#">DDSDataReader</a> has replaced an instance.</p> <p>Entity:</p> <p><a href="#">DDSDataReader</a></p> <p>Status:</p> <p><a href="#">DDS_DataReaderInstanceReplacedStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDataReaderListener</a></p>                                                                          |

## Enumerator

|                                             |                                                                                                                                                                                                                                                                                                                         |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_RELIABLE_READER_ACTIVITY_CHANGED_STATUS | <p>The <a href="#">DDSDataWriter</a> has detected a change of activity in a reliable <a href="#">DDSDataReader</a>.</p> <p>Entity:</p> <p><a href="#">DDSDataWriter</a></p> <p>Status:</p> <p><a href="#">DDS_ReliableReaderActivityChangedStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDataWriterListener</a></p> |
| DDS_DATA_WRITER_SAMPLE_REMOVED_STATUS       | <p>The <a href="#">DDSDataWriter</a> has detected a change of activity in a reliable <a href="#">DDSDataReader</a>.</p> <p>Entity:</p> <p><a href="#">DDSDataWriter</a></p> <p>Status:</p> <p><a href="#">DDS_ReliableReaderActivityChangedStatus</a></p> <p>Listener:</p> <p><a href="#">DDSDataWriterListener</a></p> |

## 6.2.10.9 QoS Policies

Quality of Service (QoS) policies.

## Topics

- [FILTER](#)  
*<<experimental>> The FILTER QoS policy allows you to configure the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.*
- [CONTENT\\_FILTER](#)  
*<<experimental>> <<eXtension>> Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.*
- [DISCOVERY](#)  
*<<eXtension>> Specifies the attributes required to discover participants in the domain.*
- [USER\\_TRAFFIC](#)  
*<<eXtension>> Specify user-traffic transport settings*
- [DEADLINE](#)  
*Expresses the maximum duration (deadline) within which an instance is expected to be updated.*

- **LATENCY\_BUDGET**

*Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.*

- **OWNERSHIP**

*Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.*

- **OWNERSHIP\_STRENGTH**

*Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).*

- **LIVELINESS**

*Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.*

- **RELIABILITY**

*Indicates the level of reliability offered/requested by RTI Connex DDS Micro.*

- **HISTORY**

*Specifies the behavior of RTI Connex DDS Micro in the case where the value of an instance changes (one or more times) before it can be successfully communicated to one or more existing subscribers.*

- **DURABILITY**

*<<eXtension>> Specifies the Durability properties used by a [DDSDataWriter](#) and/or [DDSDataReader](#).*

- **DATA\_REPRESENTATION**

*A list of data representations supported by a [DDSDataWriter](#) or [DDSDataReader](#).*

- **RESOURCE\_LIMITS**

*Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.*

- **PRESENTATION**

*Specifies how the samples representing changes to data instances are presented to a subscribing application.*

- **DESTINATION\_ORDER**

*<<eXtension>> Specifies the DestinationOrder policy.*

- **ENTITY\_FACTORY**

*A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.*

- **Extended QoS Support**

*<<eXtension>> Types and defines used in extended QoS policies.*

- **SYSTEM\_RESOURCE\_LIMITS**

*<<eXtension>> Specifies the system-specific resources that RTI Connex DDS Micro can use.*

- **WIRE\_PROTOCOL**

*<<eXtension>> Specifies the wire protocol related attributes for the [DDSDomainParticipant](#).*

- **DATA\_READER\_PROTOCOL**

*<<eXtension>> Specifies the DataReader-specific protocol QoS.*

- **DATA\_WRITER\_PROTOCOL**

*<<eXtension>> Specifies the DataWriter-specific protocol QoS.*

- **TRANSPORT**

*<<eXtension>> Specify transport settings*

- **DOMAIN\_PARTICIPANT\_RESOURCE\_LIMITS**

*<<eXtension>> Specifies the DomainParticipant-specific resources that RTI Connex DDS Micro can use.*

- **ENTITY\_NAME**

*<<eXtension>> Name of the entity.*

- **PUBLISH\_MODE**

*<<eXtension>> Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its own thread to send data, instead of the user thread.*

- **PARTITION**

The *PARTITION* QoS policy provides a method to prevent Entities that have otherwise compatible QoS policies from matching with each other and thus communicating with each other.

- **PROPERTY**

*<<eXtension>>* Stores (name, value) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.

- **USER\_DATA**

The *USER\_DATA* QoS policy allows the user to attach a buffer of opaque data to a *DDSDomainParticipant*, *DDSDataWriter*, or *DDSDataReader*. How this information is used will be up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

- **GROUP\_DATA**

The *GROUP\_DATA* QoS policy allows you to attach a buffer of opaque data to a *DDSPublisher* or *DDSSubscriber*. How this information is used will be up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

- **TOPIC\_DATA**

The *TOPIC\_DATA* QoS policy allows the user to attach a buffer of opaque data to a *DDSTopic*. How this information is used will be up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

- **DATA\_WRITER\_TRANSFER\_MODE**

*<<eXtension>>* Specifies the *DataWriter* transfer mode QoS.

## Classes

- struct **DDS\_ChecksumProperty**

*<<eXtension>>* Type to define the checksum properties of the participant RTI Connex DDS Micro.

- struct **DDS\_ChecksumProperty\_t**

*<<eXtension>>* Type to define the checksum properties of the participant RTI Connex DDS Micro.

## Enumerations

- enum **DDS\_QosPolicyId\_t** {  
*DDS\_INVALID\_QOS\_POLICY\_ID* , *DDS\_USERDATA\_QOS\_POLICY\_ID* = 1 , *DDS\_DURABILITY\_QOS\_POLICY\_ID* = 2 , *DDS\_PRESENTATION\_QOS\_POLICY\_ID* = 3 ,  
*DDS\_DEADLINE\_QOS\_POLICY\_ID* , *DDS\_LATENCYBUDGET\_QOS\_POLICY\_ID* = 5 , *DDS\_OWNERSHIP\_QOS\_POLICY\_ID* ,  
*DDS\_OWNERSHIPSTRENGTH\_QOS\_POLICY\_ID* ,  
*DDS\_LIVELINESS\_QOS\_POLICY\_ID* , *DDS\_TIMEBASEDFILTER\_QOS\_POLICY\_ID* = 9 , *DDS\_PARTITION\_QOS\_POLICY\_ID* = 10 , *DDS\_RELIABILITY\_QOS\_POLICY\_ID* ,  
*DDS\_DESTINATIONORDER\_QOS\_POLICY\_ID* = 12 , *DDS\_HISTORY\_QOS\_POLICY\_ID* , *DDS\_RESOURCELIMITS\_QOS\_POLICY\_ID* = 14 , *DDS\_ENTITYFACTORY\_QOS\_POLICY\_ID* ,  
*DDS\_WRITERDATALIFECYCLE\_QOS\_POLICY\_ID* = 16 , *DDS\_READERDATALIFECYCLE\_QOS\_POLICY\_ID* = 17 , *DDS\_TOPICDATA\_QOS\_POLICY\_ID* = 18 , *DDS\_GROUPDATA\_QOS\_POLICY\_ID* = 19 ,  
*DDS\_TRANSPORTPRIORITY\_QOS\_POLICY\_ID* = 20 , *DDS\_LIFESPAN\_QOS\_POLICY\_ID* = 21 , *DDS\_DURABILITYSERVICE\_QOS\_POLICY\_ID* = 22 , *DDS\_DATA\_REPRESENTATION\_QOS\_POLICY\_ID* = 23 ,  
*DDS\_PROPERTY\_QOS\_POLICY\_ID* = 24 }

Type to identify QoS Policies.

#### 6.2.10.9.1 Detailed Description

Quality of Service (QoS) policies.

Data Distribution Service (DDS) relies on the use of QoS. A QoS is a set of characteristics that controls some aspect of the behavior of DDS. A QoS is comprised of individual QoS policies (objects conceptually deriving from an *abstract QosPolicy* class).

RTI Connex DDS Micro supports a subset of these QoS.

The QosPolicy provides the basic mechanism for an application to specify quality of service parameters. A QosPolicy may have additional properties associated with it:

- **RxOproperty**

The QosPolicy objects that need to be set in a compatible manner between the **publisher** and **subscriber** end are indicated by the setting of the RxO property:

- RxO = **YES** indicates that the policy can be set both at the publishing and subscribing ends and the values must be set in a compatible manner. In this case the compatible values are explicitly defined.
- RxO = **NO** indicates that the policy can be set both at the publishing and subscribing ends but the two settings are independent. That is, all combinations of values are compatible.
- RxO = **N/A** indicates that the policy can only be specified at either the publishing or the subscribing end, but not at both ends. So compatibility does not apply.

- **Changeableproperty**

Determines whether a QosPolicy can be changed.

**NO** -- policy can only be specified at *DDSEntity* creation time.

**UNTIL ENABLE** -- policy can only be changed before the *DDSEntity* is enabled.

**YES** -- policy can be changed at any time.

QosPolicy implementation is comprised of a name, an ID, and a type. The type of a *QosPolicy* value may be atomic, such as an integer or float, or compound (a structure). Compound types are used whenever multiple parameters must be set coherently to define a consistent value for a QosPolicy.

QoS (i.e., a list of *QosPolicy* objects) may be associated with all *DDSEntity* objects in the system such as *DDSTopic*, *DDSDataWriter*, *DDSDataReader*, *DDSPublisher*, *DDSSubscriber*, and *DDSDomainParticipant*.

#### 6.2.10.9.2 Specifying QoS on entities

QoS Policies can be set programmatically when an *DDSEntity* is created, or modified with the *DDSEntity*'s *set\_qos (abstract)* method.

To customize a *DDSEntity*'s QoS before creating the entity, the correct pattern is:

- First, initialize a QoS object with the appropriate *INITIALIZER* constructor.
- Call the relevant *get\_<entity>\_default\_qos()* method.
- Modify the QoS values as desired.

- Finally, create the entity.

Each `QosPolicy` is treated independently from the others. This approach has the advantage of being very extensible. However, there may be cases where several policies are in conflict. Consistency checking is performed each time the policies are modified via the `set_qos (abstract)` operation, or when the `DDSEntity` is created.

When a policy is changed after being set to a given value, it is not required that the new value be applied instantaneously; RTI Connex DDS Micro is allowed to apply it after a transition phase. In addition, some `QosPolicy` have immutable semantics, meaning that they can only be specified either at `DDSEntity` creation time or else prior to calling the `DDSEntity::enable` operation on the entity.

Each `DDSEntity` can be configured with a list of `QosPolicy` objects. However, not all `QosPolicies` are supported by each `DDSEntity`. For instance, a `DDSDomainParticipant` supports a different set of `QosPolicies` than a `DDSTopic` or a `DDSPublisher`.

### 6.2.10.9.3 QoS compatibility

In several cases, for communications to occur properly (or efficiently), a `QosPolicy` on the publisher side must be compatible with a corresponding policy on the subscriber side. For example, if a `DDSSubscriber` requests to receive data reliably while the corresponding `DDSPublisher` defines a best-effort policy, communication will not happen as requested.

To address this issue and maintain the desirable decoupling of publication and subscription as much as possible, the `QosPolicy` specification follows the **subscriber-requested, publisher-offered pattern**.

In this pattern, the subscriber side can specify a "requested" value for a particular `QosPolicy`. The publisher side specifies an "offered" value for that `QosPolicy`. RTI Connex DDS Micro will then determine whether the value requested by the subscriber side is compatible with what is offered by the publisher side. If the two policies are compatible, then communication will be established. If the two policies are not compatible, RTI Connex DDS Micro will not establish communications between the two `DDSEntity` objects and will record this fact by means of the `DDS_OFFERED_INCOMPATIBLE_QOS_STATUS` on the publisher end and `DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS` on the subscriber end. The application can detect this fact by means of a `DDSListener`.

### 6.2.10.9.4 Enumeration Type Documentation

#### DDS\_QosPolicyId\_t

```
enum DDS_QosPolicyId_t
```

Type to identify `QosPolicies`.

Note that the value of these constants disagree with the values of the corresponding parameter IDs in the RTPS protocol. This conflict is unavoidable since these values are given in the DDS specification, which is not tied to RTPS.

#### Enumerator

|                                                  |                                                              |
|--------------------------------------------------|--------------------------------------------------------------|
| <code>DDS_INVALID_QOS_POLICY_ID</code>           | Identifier for an invalid QoS policy.                        |
| <code>DDS_DEADLINE_QOS_POLICY_ID</code>          | Identifier for <code>DDS_DeadlineQosPolicy</code> .          |
| <code>DDS_OWNERSHIP_QOS_POLICY_ID</code>         | Identifier for <code>DDS_OwnershipQosPolicy</code> .         |
| <code>DDS_OWNERSHIPSTRENGTH_QOS_POLICY_ID</code> | Identifier for <code>DDS_OwnershipStrengthQosPolicy</code> . |
| <code>DDS_LIVELINESS_QOS_POLICY_ID</code>        | Identifier for <code>DDS_LivelinessQosPolicy</code> .        |
| <code>DDS_RELIABILITY_QOS_POLICY_ID</code>       | Identifier for <code>DDS_ReliabilityQosPolicy</code> .       |
| <code>DDS_HISTORY_QOS_POLICY_ID</code>           | Identifier for <code>DDS_HistoryQosPolicy</code> .           |
| <code>DDS_ENTITYFACTORY_QOS_POLICY_ID</code>     | Identifier for <code>DDS_EntityFactoryQosPolicy</code> .     |



### 6.2.10.9.5 FILTER

<<*experimental*>> The FILTER QoS policy allows you to configure the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.

#### Classes

- struct [DDS\\_FilterResourceLimits](#)  
*Resource limits for content filtering on a [DDSDomainParticipant](#) and all of its child entities.*
- struct [DDS\\_FilterQosPolicy](#)  
*Configures the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.*

#### Macros

- #define [DDS\\_CONTENT\\_FILTER\\_NAME\\_MAX\\_LENGTH](#) 15  
*The absolute maximum length of the content filter name.*
- #define [DDS\\_CONTENT\\_FILTER\\_CLASS\\_NAME\\_MAX\\_LENGTH](#) 15  
*The absolute maximum length of the content filter class name.*
- #define [DDS\\_CONTENT\\_FILTER\\_EXPRESSION\\_MAX\\_LENGTH](#) 255  
*The absolute maximum length of the content filter expression.*
- #define [DDS\\_CONTENT\\_FILTER\\_MAX\\_PARAMETERS](#) 16  
*The absolute maximum number of parameters in a content filter expression.*
- #define [DDS\\_FILTER\\_RESOURCE\\_LIMITS\\_DEFAULT](#)  
*Default resource limits for content filtering.*
- #define [DDS\\_FILTER\\_PLUGIN\\_FACTORY\\_DEFAULT\\_NAME](#) "filter"  
*Default name for the filter plugin factory.*

#### Detailed Description

<<*experimental*>> The FILTER QoS policy allows you to configure the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.

#### Macro Definition Documentation

##### DDS\_CONTENT\_FILTER\_NAME\_MAX\_LENGTH

```
#define DDS_CONTENT_FILTER_NAME_MAX_LENGTH 15
```

The absolute maximum length of the content filter name.

A lower maximum length can be configured in [DDS\\_FilterResourceLimits::filter\\_class\\_max\\_length](#) to reduce memory usage, but this value is the absolute maximum.

**DDS\_CONTENT\_FILTER\_CLASS\_NAME\_MAX\_LENGTH**

```
#define DDS_CONTENT_FILTER_CLASS_NAME_MAX_LENGTH 15
```

The absolute maximum length of the content filter class name.

A lower maximum length can be configured in [DDS\\_FilterResourceLimits::filter\\_class\\_max\\_length](#) to reduce memory usage, but this value is the absolute maximum.

**DDS\_CONTENT\_FILTER\_EXPRESSION\_MAX\_LENGTH**

```
#define DDS_CONTENT_FILTER_EXPRESSION_MAX_LENGTH 255
```

The absolute maximum length of the content filter expression.

A lower maximum length can be configured in [DDS\\_FilterResourceLimits::filter\\_expression\\_max\\_length](#) to reduce memory usage, but this value is the absolute maximum.

**DDS\_CONTENT\_FILTER\_MAX\_PARAMETERS**

```
#define DDS_CONTENT_FILTER_MAX_PARAMETERS 16
```

The absolute maximum number of parameters in a content filter expression.

A lower maximum can be configured in [DDS\\_FilterResourceLimits::filter\\_parameter\\_max\\_count\\_per\\_expression](#) to reduce memory usage, but this value is the absolute maximum.

**DDS\_FILTER\_RESOURCE\_LIMITS\_DEFAULT**

```
#define DDS_FILTER_RESOURCE_LIMITS_DEFAULT
```

**Value:**

```
{\
    7,          /* filter_class_max_length */      \
    1,          /* filter_class_max_count */    \
    63,         /* filter_expression_max_length */ \
    DDS_LENGTH_AUTO, /* filter_expression_max_count */ \
    4,          /* filter_parameter_max_count_per_expression */ \
    15,         /* filter_parameter_max_length */ \
    4           /* sql_predicate_max_count_per_expression */ \
}
```

Default resource limits for content filtering.

**DDS\_FILTER\_PLUGIN\_FACTORY\_DEFAULT\_NAME**

```
#define DDS_FILTER_PLUGIN_FACTORY_DEFAULT_NAME "filter"
```

Default name for the filter plugin factory.

#### 6.2.10.9.6 CONTENT\_FILTER

`<<experimental>>` `<<eXtension>>` Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.

##### Classes

- struct [DDS\\_ContentFilterQosPolicy](#)

*Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.*

##### Detailed Description

`<<experimental>>` `<<eXtension>>` Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.

#### 6.2.10.9.7 DISCOVERY

`<<eXtension>>` Specifies the attributes required to discover participants in the domain.

##### Classes

- struct [DDS\\_DiscoveryQosPolicy](#)

*Specifies the attributes required to discover participants in the domain.*

##### Detailed Description

`<<eXtension>>` Specifies the attributes required to discover participants in the domain.

#### 6.2.10.9.8 USER\_TRAFFIC

`<<eXtension>>` Specify user-traffic transport settings

##### Classes

- struct [DDS\\_UserTrafficQosPolicy](#)

`<<eXtension>>` *Settings for available transports*

##### Detailed Description

`<<eXtension>>` Specify user-traffic transport settings

#### 6.2.10.9.9 DEADLINE

Expresses the maximum duration (deadline) within which an instance is expected to be updated.

##### Classes

- struct [DDS\\_DeadlineQosPolicy](#)

*Expresses the maximum duration (deadline) within which an instance is expected to be updated.*

##### Detailed Description

Expresses the maximum duration (deadline) within which an instance is expected to be updated.

#### 6.2.10.9.10 LATENCY\_BUDGET

Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

##### Classes

- struct [DDS\\_LatencyBudgetQosPolicy](#)

*Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.*

##### Detailed Description

Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

#### 6.2.10.9.11 OWNERSHIP

Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.

##### Classes

- struct [DDS\\_OwnershipQosPolicy](#)

*Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.*

## Enumerations

- enum [DDS\\_OwnershipQosPolicyKind](#) { [DDS\\_SHARED\\_OWNERSHIP\\_QOS](#) , [DDS\\_EXCLUSIVE\\_OWNERSHIP\\_QOS](#) }

*Kinds of ownership.*

## Detailed Description

Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.

## Enumeration Type Documentation

### DDS\_OwnershipQosPolicyKind

enum [DDS\\_OwnershipQosPolicyKind](#)

Kinds of ownership.

QoS:

[DDS\\_OwnershipQosPolicy](#)

### Enumerator

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_SHARED_OWNERSHIP_QOS    | <b>[default]</b> Indicates shared ownership for each instance. Multiple writers are allowed to update the same instance and all the updates are made available to the readers. In other words there is no concept of an owner for the instances.<br>This is the <b>default</b> behavior if the <a href="#">OWNERSHIP</a> policy is not specified or supported.                                                              |
| DDS_EXCLUSIVE_OWNERSHIP_QOS | Indicates each instance can only be owned by one <a href="#">DDSDataWriter</a> , but the owner of an instance can change dynamically. The selection of the owner is controlled by the setting of the <a href="#">OWNERSHIP_STRENGTH</a> policy. The owner is always set to be the highest-strength <a href="#">DDSDataWriter</a> object among the ones currently active (as determined by the <a href="#">LIVELINESS</a> ). |

### 6.2.10.9.12 OWNERSHIP\_STRENGTH

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

## Classes

- struct [DDS\\_OwnershipStrengthQosPolicy](#)

*Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).*

## Detailed Description

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

### 6.2.10.9.13 LIVELINESS

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

## Classes

- struct [DDS\\_LivelinessQosPolicy](#)

*Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.*

## Enumerations

- enum [DDS\\_LivelinessQosPolicyKind](#) { [DDS\\_AUTOMATIC\\_LIVELINESS\\_QOS](#) , [DDS\\_MANUAL\\_BY\\_PARTICIPANT\\_LIVELINESS\\_QOS](#) , [DDS\\_MANUAL\\_BY\\_TOPIC\\_LIVELINESS\\_QOS](#) }

*Kinds of liveliness.*

## Detailed Description

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

## Enumeration Type Documentation

### DDS\_LivelinessQosPolicyKind

```
enum DDS\_LivelinessQosPolicyKind
```

Kinds of liveliness.

QoS:

[DDS\\_LivelinessQosPolicy](#)

## Enumerator

|                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_AUTOMATIC_LIVELINESS_QOS             | <b>[default]</b> The infrastructure will automatically signal liveliness for the <a href="#">DDSDDataWriter</a> (s) at least as often as required by the <a href="#">DDSDDataWriter</a> (S) <code>lease_duration</code> . A <a href="#">DDSDDataWriter</a> with this setting does not need to take any specific action in order to be considered 'alive.' The <a href="#">DDSDDataWriter</a> is only 'not alive' when the participant to which it belongs terminates (gracefully or not), or when there is a network problem that prevents the current participant from contacting that remote participant.                                                                                                                                 |
| DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS | RTI Connex DDS Micro will assume that as long as at least one <a href="#">DDSDDataWriter</a> belonging to the <a href="#">DDSDDomainParticipant</a> (or the <a href="#">DDSDDomainParticipant</a> itself) has asserted its liveliness, then the other <a href="#">DataWriters</a> belonging to that same <a href="#">DDSDDomainParticipant</a> are also alive. The user application takes responsibility to signal liveliness to RTI Connex DDS Micro either by calling <a href="#">DDSDDomainParticipant::assert_liveliness</a> , or by calling <a href="#">DDSDDataWriter::assert_liveliness</a> , or <a href="#">FooDataWriter::write</a> on any <a href="#">DDSDDataWriter</a> belonging to the <a href="#">DDSDDomainParticipant</a> . |
| DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS       | RTI Connex DDS Micro will only assume liveliness of the <a href="#">DDSDDataWriter</a> if the application has asserted liveliness of that <a href="#">DDSDDataWriter</a> itself. The user application takes responsibility to signal liveliness to RTI Connex DDS Micro using the <a href="#">DDSDDataWriter::assert_liveliness</a> method, or by writing some data.                                                                                                                                                                                                                                                                                                                                                                        |

## 6.2.10.9.14 RELIABILITY

Indicates the level of reliability offered/requested by RTI Connex DDS Micro.

## Classes

- struct [DDS\\_ReliabilityQosPolicy](#)

*Indicates the level of reliability offered/requested by RTI Connex DDS Micro.*

## Enumerations

- enum [DDS\\_ReliabilityQosPolicyKind](#) { [DDS\\_BEST\\_EFFORT\\_RELIABILITY\\_QOS](#) , [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#) }

*Kinds of reliability.*

## Detailed Description

Indicates the level of reliability offered/requested by RTI Connex DDS Micro.

## Enumeration Type Documentation

### DDS\_ReliabilityQosPolicyKind

enum [DDS\\_ReliabilityQosPolicyKind](#)

Kinds of reliability.

QoS:

[DDS\\_ReliabilityQosPolicy](#)

### Enumerator

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_BEST_EFFORT_RELIABILITY_QOS | Indicates that it is acceptable to not retry propagation of any samples. Presumably new values for the samples are generated often enough that it is not necessary to re-send or acknowledge any samples.<br><br><b>[default]</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| DDS_RELIABLE_RELIABILITY_QOS    | Specifies RTI Connex DDS Micro will attempt to deliver all samples in its history. Missed samples may be retried. In steady-state (no modifications communicated via the <a href="#">DDSDDataWriter</a> ) the middleware guarantees that all samples in the <a href="#">DDSDDataWriter</a> history will eventually be delivered to all the <a href="#">DDSDDataReader</a> objects (subject to timeouts that indicate loss of communication with a particular <a href="#">DDSSubscriber</a> ). Outside steady state the <a href="#">HISTORY</a> and <a href="#">RESOURCE_LIMITS</a> policies will determine how samples become part of the history and whether samples can be discarded from it.<br>NOTE: The RTPS spec defines reliability as 0x03 to comply with RTPS spec. However, RTI Connex Core uses 0x3, as well as others. |

### 6.2.10.9.15 HISTORY

Specifies the behavior of RTI Connex DDS Micro in the case where the value of an instance changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

## Classes

- struct [DDS\\_HistoryQosPolicy](#)

*Specifies the behavior of RTI Connex DDS Micro in the case where the value of a sample changes (one or more times) before it can be successfully communicated to one or more existing subscribers.*



## Enumerations

- enum [DDS\\_HistoryQosPolicyKind](#) { [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#) , [DDS\\_KEEP\\_ALL\\_HISTORY\\_QOS](#) }  
*Kinds of history.*

## Detailed Description

Specifies the behavior of RTI Connex DDS Micro in the case where the value of an instance changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

## Enumeration Type Documentation

### DDS\_HistoryQosPolicyKind

enum [DDS\\_HistoryQosPolicyKind](#)

Kinds of history.

QoS:

[DDS\\_HistoryQosPolicy](#)

### Enumerator

|                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_KEEP_LAST_HISTORY_QOS | <p><b>[default]</b> Keep the last <code>depth</code> samples. On the publishing side, RTI Connex DDS Micro will only attempt to keep the most recent <code>depth</code> samples of each instance of data (identified by its key) managed by the <a href="#">DDSDDataWriter</a>.</p> <p>On the subscribing side, the <a href="#">DDSDDataReader</a> will only attempt to keep the most recent <code>depth</code> samples received for each instance (identified by its key) until the application takes them via the <a href="#">DDSDDataReader</a> 's <b>take()</b> operation.</p>                          |
| DDS_KEEP_ALL_HISTORY_QOS  | <p>Keep <i>all</i> the samples. On the publishing side, RTI Connex DDS Micro will attempt to keep all samples (representing each value written) of each instance of data (identified by its key) managed by the <a href="#">DDSDDataWriter</a> until they can be delivered to all subscribers.</p> <p>On the subscribing side, RTI Connex DDS Micro will attempt to keep all samples of each instance of data (identified by its key) managed by the <a href="#">DDSDDataReader</a>. These samples are kept until the application takes them from RTI Connex DDS Micro via the <b>take()</b> operation.</p> |

## 6.2.10.9.16 DURABILITY

<<*eXtension*>> Specifies the Durability properties used by a [DDSDDataWriter](#) and/or [DDSDDataReader](#).

## Classes

- struct [DDS\\_DurabilityQosPolicy](#)  
*Durability properties that affect the life-cycle of published data.*

## Enumerations

- enum [DDS\\_DurabilityQosPolicyKind](#) { [DDS\\_VOLATILE\\_DURABILITY\\_QOS](#) , [DDS\\_TRANSIENT\\_LOCAL\\_DURABILITY\\_QOS](#) , [DDS\\_TRANSIENT\\_DURABILITY\\_QOS](#) , [DDS\\_PERSISTENT\\_DURABILITY\\_QOS](#) }  
*Enumerated values used to describe the durability of published data.*

## Detailed Description

<<*eXtension*>> Specifies the Durability properties used by a [DDSDataWriter](#) and/or [DDSDataReader](#).

## Enumeration Type Documentation

### DDS\_DurabilityQosPolicyKind

enum [DDS\\_DurabilityQosPolicyKind](#)

Enumerated values used to describe the durability of published data.

#### Enumerator

|                                                    |                             |
|----------------------------------------------------|-----------------------------|
| <a href="#">DDS_VOLATILE_DURABILITY_QOS</a>        | Volatile durability.        |
| <a href="#">DDS_TRANSIENT_LOCAL_DURABILITY_QOS</a> | Transient local durability. |

### 6.2.10.9.17 DATA\_REPRESENTATION

A list of data representations supported by a [DDSDataWriter](#) or [DDSDataReader](#).

## Classes

- struct [DDS\\_DataRepresentationQosPolicy](#)  
*This QoS policy contains a list of representation identifiers used by [DDSDataWriter](#) and [DDSDataReader](#) entities to negotiate which data representation to use.*

## Macros

- `#define DDS_XCDR_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)0)`  
*Corresponds to the Extended Common Data Representation encoding version 1.*
- `#define DDS_XML_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)1)`  
*Corresponds to the XML Data Representation (unsupported).*
- `#define DDS_XCDR2_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)2)`  
*Corresponds to the Extended Common Data Representation encoding version 2.*
- `#define DDS_AUTO_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)-1)`  
*Representation is automatically chosen based on the type.*
- `#define T DDS_InstanceHandle_t`  
*A two-byte signed integer that identifies a data representation.*
- `#define T struct DDS_QosPolicyCount`  
*A two-byte signed integer that identifies a data representation.*
- `#define T DDS_EncapsulationId_t`  
*A two-byte signed integer that identifies a data representation.*
- `#define T struct DDS_TransportEncapsulationSettings_t`  
*A two-byte signed integer that identifies a data representation.*
- `#define T DDS_DataRepresentationId_t`  
*A two-byte signed integer that identifies a data representation.*
- `#define T struct RTPS_Locator`  
*A two-byte signed integer that identifies a data representation.*
- `#define T struct DDS_ConditionImpl*`  
*A two-byte signed integer that identifies a data representation.*

## Typedefs

- `typedef DDS_Short DDS_DataRepresentationId_t`  
*A two-byte signed integer that identifies a data representation.*

## Detailed Description

A list of data representations supported by a [DDSDDataWriter](#) or [DDSDDataReader](#).

## Macro Definition Documentation

### DDS\_XCDR\_DATA\_REPRESENTATION

```
#define DDS_XCDR_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)0)
```

Corresponds to the Extended Common Data Representation encoding version 1.

**DDS\_XML\_DATA\_REPRESENTATION**

```
#define DDS_XML_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)1)
```

Corresponds to the XML Data Representation (unsupported).

**Note**

This value is currently not supported.

**DDS\_XCDR2\_DATA\_REPRESENTATION**

```
#define DDS_XCDR2_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)2)
```

Corresponds to the Extended Common Data Representation encoding version 2.

**DDS\_AUTO\_DATA\_REPRESENTATION**

```
#define DDS_AUTO_DATA_REPRESENTATION ((DDS_DataRepresentationId_t)-1)
```

Representation is automatically chosen based on the type.

For plain language binding, if the `allow_data_representation` annotation is not specified or if it contains the value XCDR, RTI Connex DDS Micro translates this field to [DDS\\_XCDR\\_DATA\\_REPRESENTATION](#). Otherwise, it translates this field to [DDS\\_XCDR2\\_DATA\\_REPRESENTATION](#).

For the [FlatData Topic-Types](#) "FlatData language binding", RTI Connex DDS Micro translates this field to [DDS\\_XCDR2\\_DATA\\_REPRESENTATION](#).

**T [1/7]**

```
#define T DDS_InstanceHandle_t
```

A two-byte signed integer that identifies a data representation.

**T [2/7]**

```
#define T struct DDS_QosPolicyCount
```

A two-byte signed integer that identifies a data representation.

**T [3/7]**

```
#define T DDS_EncapsulationId_t
```

A two-byte signed integer that identifies a data representation.

**T [4/7]**

```
#define T struct DDS_TransportEncapsulationSettings_t
```

A two-byte signed integer that identifies a data representation.

**T [5/7]**

```
#define T DDS_DataRepresentationId_t
```

A two-byte signed integer that identifies a data representation.

**T [6/7]**

```
#define T struct RTPS_Locator
```

A two-byte signed integer that identifies a data representation.

**T [7/7]**

```
#define T struct DDS_ConditionImpl*
```

A two-byte signed integer that identifies a data representation.

**Typedef Documentation****DDS\_DataRepresentationId\_t**

```
typedef DDS_Short DDS_DataRepresentationId_t
```

A two-byte signed integer that identifies a data representation.

**6.2.10.9.18 RESOURCE\_LIMITS**

Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

**Classes**

- struct [DDS\\_ResourceLimitsQosPolicy](#)

*Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.*

## Variables

- const `DDS_Long DDS_LENGTH_UNLIMITED`  
*A special value indicating an unlimited quantity.*
- const `DDS_Long DDS_LENGTH_AUTO`  
*A special value indicating the resource-limit is determined based on other settings.*
- const `DDS_Long DDS_SIZE_AUTO`  
*A special value indicating the resource-limit is determined based on other settings.*

## Detailed Description

Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

## Variable Documentation

### DDS\_LENGTH\_UNLIMITED

```
const DDS_Long DDS_LENGTH_UNLIMITED [extern]
```

A special value indicating an unlimited quantity.

### DDS\_LENGTH\_AUTO

```
const DDS_Long DDS_LENGTH_AUTO [extern]
```

A special value indicating the resource-limit is determined based on other settings.

Some resource-limits can be derived from other limits using rules. However, the rules are usually conservative and typically the resulting limit is larger than needed. Thus, while this is a useful value when starting development, it should be adjusted properly when the exact limits are known.

### DDS\_SIZE\_AUTO

```
const DDS_Long DDS_SIZE_AUTO [extern]
```

A special value indicating the resource-limit is determined based on other settings.

Some resource-limits can be derived from other limits. However, the rules for this are conservative, and the resulting limit is usually larger than needed. This is a useful value when starting development, but it should be adjusted properly when the exact limits are known.

## 6.2.10.9.19 PRESENTATION

Specifies how the samples representing changes to data instances are presented to a subscribing application.

## Classes

- struct [DDS\\_PresentationQosPolicy](#)

*Specifies how the samples representing changes to data instances are presented to a subscribing application.*

## Enumerations

- enum [DDS\\_PresentationQosPolicyAccessScopeKind](#) { [DDS\\_INSTANCE\\_PRESENTATION\\_QOS](#) , [DDS\\_TOPIC\\_PRESENTATION\\_QOS](#) , [DDS\\_GROUP\\_PRESENTATION\\_QOS](#) , [DDS\\_HIGHEST\\_OFFERED\\_PRESENTATION\\_QOS](#) }

*Kinds of presentation "access scope".*

## Detailed Description

Specifies how the samples representing changes to data instances are presented to a subscribing application.

## Enumeration Type Documentation

### DDS\_PresentationQosPolicyAccessScopeKind

enum [DDS\\_PresentationQosPolicyAccessScopeKind](#)

Kinds of presentation "access scope".

Access scope determines the largest scope spanning the entities for which the order and coherency of changes can be preserved.

QoS:

[DDS\\_PresentationQosPolicy](#)

## Enumerator

|                               |                                                                                                                                                                                                                                                                                                |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_INSTANCE_PRESENTATION_QOS | <b>[default]</b> for <a href="#">DDSDDataReader</a> . Scope spans only a single instance. Indicates that changes to one instance need not be coherent nor ordered with respect to changes to any other instance. In other words, order and coherent changes apply to each instance separately. |
| DDS_TOPIC_PRESENTATION_QOS    | <b>[default]</b> for <a href="#">DDSDDataWriter</a> . Scope spans all instances within the same <a href="#">DDSDDataWriter</a> (or <a href="#">DDSDDataReader</a> ), but not across instances in different <a href="#">DDSDDataWriter</a> (or <a href="#">DDSDDataReader</a> ) entities.       |
| DDS_GROUP_PRESENTATION_QOS    | Scope spans all instances belonging to <a href="#">DDSDDataWriter</a> (or <a href="#">DDSDDataReader</a> ) entities within the same <a href="#">DDSPublisher</a> (or <a href="#">DDSSubscriber</a> ).                                                                                          |

#### 6.2.10.9.20 DESTINATION\_ORDER

<<*eXtension*>> Specifies the DestinationOrder policy.

#### Classes

- struct `DDS_DestinationOrderQosPolicy`  
*Destination order policies that affect the ordering of subscribed data.*

#### Enumerations

- enum `DDS_DestinationOrderQosPolicyKind` { `DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS`, `DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS` }  
*Enumerated values used to describe the destination order of published data.*

#### Detailed Description

<<*eXtension*>> Specifies the DestinationOrder policy.

#### Enumeration Type Documentation

##### DDS\_DestinationOrderQosPolicyKind

```
enum DDS_DestinationOrderQosPolicyKind
```

Enumerated values used to describe the destination order of published data.



## Enumerator

|                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_BY_RECEPTION_TIMESTAMP_↔<br>DESTINATIONORDER_QOS | destination ordering by reception timestamp.<br><br><b>Entity:</b><br><br><a href="#">DDSDataReader</a> , <a href="#">DDSDataWriter</a><br><br><b>Properties:</b><br><br><a href="#">RxO</a> = YES;<br><a href="#">Changeable</a> = NO<br><br><b>[default]</b> Indicates that data is ordered based on the reception time at each <a href="#">DDSDataReader</a> . Since each subscriber may receive the data at different times there is no guarantee that the changes will be seen in the same order. Consequently, it is possible for each subscriber to end up with a different final value for the data. |
| DDS_BY_SOURCE_TIMESTAMP_↔<br>DESTINATIONORDER_QOS    | destination ordering by source timestamp.<br><br><b>Entity:</b><br><br><a href="#">DDSDataWriter</a><br><br><b>Properties:</b><br><br><a href="#">RxO</a> = YES;<br><a href="#">Changeable</a> = NO<br><br>Indicates that data is ordered based on the source time provided by the source application. A consistent final value is guaranteed for the data in all subscribers.                                                                                                                                                                                                                               |

## 6.2.10.9.21 ENTITY\_FACTORY

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

## Classes

- struct [DDS\\_EntityFactoryQosPolicy](#)

*A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.*

## Detailed Description

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

### 6.2.10.9.22 Extended Qos Support

<<eXtension>> Types and defines used in extended QoS policies.

#### Topics

- [DataReader Resource Limits](#)  
<<eXtension>> Specifies the DataReader-specific resources that RTI Connex DDS Micro can use.
- [DataWriter Resource Limits](#)  
<<eXtension>> Specifies the DataWriter-specific resources that RTI Connex DDS Micro can use.

#### Classes

- struct [DDS\\_RtpsReliableReaderProtocol\\_t](#)  
Qos related to reliable reader protocol defined in RTPS.
- struct [DDS\\_RtpsReliableWriterProtocol\\_t](#)  
QoS related to the reliable writer protocol defined in RTPS.

#### Detailed Description

<<eXtension>> Types and defines used in extended QoS policies.

#### DataReader Resource Limits

<<eXtension>> Specifies the DataReader-specific resources that RTI Connex DDS Micro can use.

#### Classes

- struct [DDS\\_DataReaderResourceLimitsQosPolicy](#)  
Resource limits that apply only to [DDSDataReader](#) instances.

#### Enumerations

- enum [DDS\\_DataReaderResourceLimitsInstanceReplacementKind](#) { [DDS\\_NO\\_INSTANCE\\_REPLACEMENT\\_QOS](#), [DDS\\_REPLACE\\_OLDEST\\_INSTANCE\\_REPLACEMENT\\_QOS](#) }  
Kinds of instance replacement.

#### Detailed Description

<<eXtension>> Specifies the DataReader-specific resources that RTI Connex DDS Micro can use.

#### Enumeration Type Documentation

##### DDS\_DataReaderResourceLimitsInstanceReplacementKind

```
enum DDS_DataReaderResourceLimitsInstanceReplacementKind
```

Kinds of instance replacement.

QoS:

[DDS\\_DataReaderResourceLimitsQosPolicy](#)

## Enumerator

|                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_NO_INSTANCE_REPLACEMENT_QOS                  | <b>[default]</b> No instances will be replaced when a new instance is detected and resources are exhausted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| DDS_REPLACE_OLDEST_INSTANCE_↔<br>REPLACEMENT_QOS | Replace the oldest instance when out of instance resources. When a new instance is detected and resources are exhausted the oldest instance will be removed even if samples with a not read state exists for the instance. Thus, this is a forceful removal of samples and the instance. The freed instance resources will be re-used for the newly detected instance. If there are outstanding loans for the replaced instance the samples can not be freed until returned. However, when the loan is returned the samples are returned back to the free sample pool. Note that the instance will not be rejected. |

## DataWriter Resource Limits

<<eXtension>> Specifies the DataWriter-specific resources that RTI Connex DDS Micro can use.

## Classes

- struct [DDS\\_DataWriterResourceLimitsQosPolicy](#)  
Resource limits that apply only to [DDSDataWriter](#) instances.

## Detailed Description

<<eXtension>> Specifies the DataWriter-specific resources that RTI Connex DDS Micro can use.

## 6.2.10.9.23 SYSTEM\_RESOURCE\_LIMITS

<<eXtension>> Specifies the system-specific resources that RTI Connex DDS Micro can use.

## Classes

- struct [DDS\\_SystemResourceLimitsQosPolicy](#)  
Resource limits that apply only to [DDSDomainParticipantFactory](#)

## Detailed Description

<<eXtension>> Specifies the system-specific resources that RTI Connex DDS Micro can use.

#### 6.2.10.9.24 WIRE\_PROTOCOL

<<eXtension>> Specifies the wire protocol related attributes for the [DDSDomainParticipant](#).

#### Classes

- struct [DDS\\_RtpsWellKnownPorts\\_t](#)  
*RTPS well-known port mapping configuration.*
- struct [DDS\\_WireProtocolQosPolicy](#)  
*Specifies the wire-protocol-related attributes for the [DDSDomainParticipant](#).*

#### Macros

- #define [DDS\\_CHECKSUM\\_NONE](#) (0)  
*Bitmap indicating no checksum selection.*
- #define [DDS\\_CHECKSUM\\_AUTO](#) (0xffffU)  
*Bitmap indicating automatic checksum selection.*
- #define [DDS\\_CHECKSUM\\_BUILTIN32](#) (0x1U)  
*Bitmap for selecting the builtin 32bit checksum.*
- #define [DDS\\_CHECKSUM\\_BUILTIN64](#) (0x2U)  
*Bitmap for selecting the builtin 64bit checksum.*
- #define [DDS\\_CHECKSUM\\_BUILTIN128](#) (0x4U)  
*Bitmap for selecting the builtin 128bit checksum.*

#### Typedefs

- typedef [DDS\\_UnsignedShort DDS\\_ChecksumKind\\_t](#)  
*Datatype for checksum kind.*
- typedef [DDS\\_UnsignedShort DDS\\_ChecksumKindMask\\_t](#)  
*Datatype for an OR'ing for [DDS\\_ChecksumKind\\_t](#).*

#### Variables

- struct [DDS\\_RtpsWellKnownPorts\\_t DDS\\_RTI\\_BACKWARDS\\_COMPATIBLE RTPS\\_WELL\\_KNOWN\\_PORTS](#)  
*Assign to use well-known port mappings which are compatible with previous versions of the RTI Connext DDS Micro middleware.*
- struct [DDS\\_RtpsWellKnownPorts\\_t DDS\\_INTEROPERABLE RTPS\\_WELL\\_KNOWN\\_PORTS](#)  
*Assign to use well-known port mappings which are compliant with OMG's DDS Interoperability Wire Protocol.*

#### Detailed Description

<<eXtension>> Specifies the wire protocol related attributes for the [DDSDomainParticipant](#).

## Macro Definition Documentation

### DDS\_CHECKSUM\_NONE

```
#define DDS_CHECKSUM_NONE (0)
```

Bitmap indicating no checksum selection.

QoS:

[DDS\\_WireProtocolQosPolicy](#)

### DDS\_CHECKSUM\_AUTO

```
#define DDS_CHECKSUM_AUTO (0xffffU)
```

Bitmap indicating automatic checksum selection.

QoS:

[DDS\\_WireProtocolQosPolicy](#)

### DDS\_CHECKSUM\_BUILTIN32

```
#define DDS_CHECKSUM_BUILTIN32 (0x1U)
```

Bitmap for selecting the builtin 32bit checksum.

QoS:

[DDS\\_WireProtocolQosPolicy](#)

### DDS\_CHECKSUM\_BUILTIN64

```
#define DDS_CHECKSUM_BUILTIN64 (0x2U)
```

Bitmap for selecting the builtin 64bit checksum.

QoS:

[DDS\\_WireProtocolQosPolicy](#)

**DDS\_CHECKSUM\_BUILTIN128**

```
#define DDS_CHECKSUM_BUILTIN128 (0x4U)
```

Bitmap for selecting the builtin 128bit checksum.

QoS:

[DDS\\_WireProtocolQosPolicy](#)

**Typedef Documentation****DDS\_ChecksumKind\_t**

```
typedef DDS_UnsignedShort DDS_ChecksumKind_t
```

Datatype for checksum kind.

QoS:

[DDS\\_WireProtocolQosPolicy](#)

**DDS\_ChecksumKindMask\_t**

```
typedef DDS_UnsignedShort DDS_ChecksumKindMask_t
```

Datatype for an OR'ing for [DDS\\_ChecksumKind\\_t](#).

QoS:

[DDS\\_WireProtocolQosPolicy](#)

## Variable Documentation

### DDS\_RTI\_BACKWARDS\_COMPATIBLE\_RTPS\_WELL\_KNOWN\_PORTS

```
struct DDS_RtpsWellKnownPorts_t DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS [extern]
```

Assign to use well-known port mappings which are compatible with previous versions of the RTI Connex DDS Micro middleware.

Assign [DDS\\_WireProtocolQosPolicy::rtps\\_well\\_known\\_ports](#) to this value to remain compatible with previous versions of the RTI Connex DDS Micro middleware that used fixed port mappings.

The following are the `rtps_well_known_ports` values for [DDS\\_RTI\\_BACKWARDS\\_COMPATIBLE\\_RTPS\\_WELL\\_KNOWN\\_PORTS](#) :

```
port_base = 7400
domain_id_gain = 10
participant_id_gain = 1000
builtin_multicast_port_offset = 2
builtin_unicast_port_offset = 0
user_multicast_port_offset = 1
user_unicast_port_offset = 3
```

These settings are *not* compliant with OMG's DDS Interoperability Wire Protocol. To comply with the specification, please use [DDS\\_INTEROPERABLE\\_RTPS\\_WELL\\_KNOWN\\_PORTS](#).

See also

[DDS\\_WireProtocolQosPolicy::rtps\\_well\\_known\\_ports](#)  
[DDS\\_INTEROPERABLE\\_RTPS\\_WELL\\_KNOWN\\_PORTS](#)

### DDS\_INTEROPERABLE\_RTPS\_WELL\_KNOWN\_PORTS

```
struct DDS_RtpsWellKnownPorts_t DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS [extern]
```

Assign to use well-known port mappings which are compliant with OMG's DDS Interoperability Wire Protocol.

Assign [DDS\\_WireProtocolQosPolicy::rtps\\_well\\_known\\_ports](#) to this value to use well-known port mappings which are compliant with OMG's DDS Interoperability Wire Protocol.

The following are the `rtps_well_known_ports` values for [DDS\\_INTEROPERABLE\\_RTPS\\_WELL\\_KNOWN\\_PORTS](#)↔ :

```
port_base = 7400
domain_id_gain = 250
participant_id_gain = 2
builtin_multicast_port_offset = 0
builtin_unicast_port_offset = 10
user_multicast_port_offset = 1
user_unicast_port_offset = 11
```

Assuming a maximum port number of 65535 (UDPv4), the above settings enable the use of about 230 domains with up to 120 Participants per node per domain.

These settings are *not* backwards compatible with previous versions of the RTI Connex DDS Micro middleware that used fixed port mappings. For backwards compatibility, please use [DDS\\_RTI\\_BACKWARDS\\_COMPATIBLE\\_RTPS\\_WELL\\_KNOWN\\_PORTS](#).

See also

[DDS\\_WireProtocolQosPolicy::rtps\\_well\\_known\\_ports](#)  
[DDS\\_RTI\\_BACKWARDS\\_COMPATIBLE\\_RTPS\\_WELL\\_KNOWN\\_PORTS](#)

#### 6.2.10.9.25 DATA\_READER\_PROTOCOL

<<eXtension>> Specifies the DataReader-specific protocol QoS.

##### Classes

- struct [DDS\\_DataReaderProtocolQosPolicy](#)  
<<eXtension>> Protocol that applies only to [DDSDataReader](#) instances.

##### Detailed Description

<<eXtension>> Specifies the DataReader-specific protocol QoS.

#### 6.2.10.9.26 DATA\_WRITER\_PROTOCOL

<<eXtension>> Specifies the DataWriter-specific protocol QoS.

##### Classes

- struct [DDS\\_DataWriterProtocolQosPolicy](#)  
<<eXtension>> Protocol that applies only to [DDSDataWriter](#) instances.

##### Detailed Description

<<eXtension>> Specifies the DataWriter-specific protocol QoS.

#### 6.2.10.9.27 TRANSPORT

<<eXtension>> Specify transport settings

##### Classes

- struct [DDS\\_TransportQosPolicy](#)  
<<eXtension>> Settings for available transports

##### Detailed Description

<<eXtension>> Specify transport settings



#### 6.2.10.9.28 DOMAIN\_PARTICIPANT\_RESOURCE\_LIMITS

<<*eXtension*>> Specifies the DomainParticipant-specific resources that RTI Connex DDS Micro can use.

##### Classes

- struct [DDS\\_DomainParticipantResourceLimitsQosPolicy](#)  
*Resource limits that apply only to [DDSDomainParticipant](#) instances.*

##### Detailed Description

<<*eXtension*>> Specifies the DomainParticipant-specific resources that RTI Connex DDS Micro can use.

#### 6.2.10.9.29 ENTITY\_NAME

<<*eXtension*>> Name of the entity.

##### Classes

- struct [DDS\\_EntityNameQosPolicy](#)  
*Policy used to name an entity.*

##### Macros

- #define [DDS\\_ENTITYNAME\\_QOS\\_NAME\\_MAX](#) 255  
*Constant determining the maximum length of the name string contained in a [DDS\\_EntityNameQosPolicy](#) instance.*

##### Functions

- bool [DDS\\_EntityNameQosPolicy::set\\_name](#) (const char \*const [name](#))  
*Set the EntityName policy.*

##### Detailed Description

<<*eXtension*>> Name of the entity.

##### Macro Definition Documentation

#### DDS\_ENTITYNAME\_QOS\_NAME\_MAX

```
#define DDS_ENTITYNAME_QOS_NAME_MAX 255
```

Constant determining the maximum length of the name string contained in a [DDS\\_EntityNameQosPolicy](#) instance.

## Function Documentation

### set\_name()

```
bool DDS_EntityNameQosPolicy::set_name (
    const char *const name)
```

Set the EntityName policy.

Set the entity name Qos policy. The maximum allowed string length is [DDS\\_ENTITYNAME\\_QOS\\_NAME\\_MAX](#) (excluding the NUL character).

#### Parameters

|             |                                                         |
|-------------|---------------------------------------------------------|
| <i>name</i> | << <i>in</i> >> The entity name to assign to the policy |
|-------------|---------------------------------------------------------|

#### Returns

DDS\_BOOLEAN\_TRUE on success, DDS\_BOOLEAN\_FALSE on failure

References [name](#).

### 6.2.10.9.30 PUBLISH\_MODE

<<*eXtension*>> Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its *own* thread to send data, instead of the user thread.

## Classes

- struct [DDS\\_PublishModeQosPolicy](#)

*Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its own thread to send data, instead of the user thread.*

## Macros

- #define [DDS\\_PUBLICATION\\_PRIORITY\\_UNDEFINED](#)  
*Initializer value for [DDS\\_PublishModeQosPolicy::priority](#).*
- #define [DDS\\_PUBLICATION\\_PRIORITY\\_AUTOMATIC](#)  
*Constant value for [DDS\\_PublishModeQosPolicy::priority](#).*

## Enumerations

- enum [DDS\\_PublishModeQosPolicyKind](#) { [DDS\\_SYNCHRONOUS\\_PUBLISH\\_MODE\\_QOS](#), [DDS\\_ASYNCHRONOUS\\_PUBLISH\\_MODE\\_QOS](#), [DDS\\_AUTOMATIC\\_PUBLISH\\_MODE\\_QOS](#) }
- Kinds of publishing mode.*

## Detailed Description

<<[eXtension](#)>> Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its *own* thread to send data, instead of the user thread.

## Macro Definition Documentation

### DDS\_PUBLICATION\_PRIORITY\_UNDEFINED

```
#define DDS_PUBLICATION_PRIORITY_UNDEFINED
```

Initializer value for [DDS\\_PublishModeQosPolicy::priority](#).

When assigned this value, the publication priority of the data writer will be set to the lowest possible value.

### DDS\_PUBLICATION\_PRIORITY\_AUTOMATIC

```
#define DDS_PUBLICATION_PRIORITY_AUTOMATIC
```

Constant value for [DDS\\_PublishModeQosPolicy::priority](#).

When assigned this value the publication priority of the data writer will be set to the largest priority value of any sample currently queued for publication by the data writer or data writer channel.

## Enumeration Type Documentation

### DDS\_PublishModeQosPolicyKind

```
enum DDS_PublishModeQosPolicyKind
```

Kinds of publishing mode.

QoS:

[DDS\\_PublishModeQosPolicy](#)

#### Enumerator

|                                  |                                                                                                                               |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| DDS_SYNCHRONOUS_PUBLISH_MODE_QOS | Indicates to send data synchronously. Data is sent immediately in the context of the user thread, no flow control is applied. |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------|

## Enumerator

|                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS | <p>Indicates to send data asynchronously. Configures the <a href="#">DDSDataWriter</a> to delegate the task of data transmission to a separate thread. The <a href="#">FooDataWriter::write</a> call does not send the data, but instead schedules the data to be sent later.</p> <p>See also</p> <p><a href="#">DDSFlowController</a></p> <p><a href="#">DDS_HistoryQosPolicy</a></p>                                                                                                                                                                                                                                                  |
| DDS_AUTOMATIC_PUBLISH_MODE_QOS    | <p>Automatically configures the publishing mode. Configures the <a href="#">DDSDataWriter</a> to automatically determine how data transmission is performed. If the minimum data size required for the serialized type requires fragmentation, then data is handled in the <a href="#">DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS</a> mode. If the minimum data size does not require fragmentation, then data is sent in the <a href="#">DDS_SYNCHRONOUS_PUBLISH_MODE_QOS</a> mode.</p> <p>See also</p> <p><a href="#">DDSFlowController</a></p> <p><a href="#">DDS_HistoryQosPolicy</a> <b>[default]</b> for <a href="#">DDSDataWriter</a></p> |

## 6.2.10.9.31 PARTITION

The PARTITION QoS policy provides a method to prevent Entities that have otherwise compatible Qos policies from matching with each other and thus communicating with each other.

## Classes

- struct [DDS\\_PartitionQosPolicy](#)  
*A set of strings that introduces logical PARTITIONS.*

## Detailed Description

The PARTITION QoS policy provides a method to prevent Entities that have otherwise compatible Qos policies from matching with each other and thus communicating with each other.

## 6.2.10.9.32 PROPERTY

[<<eXtension>>](#) Stores (name, value) pairs that can be used to configure certain parameters of RTI Connext DDS Micro that are not exposed through formal QoS policies.

## Classes

- struct [DDS\\_Property\\_t](#)  
*Properties are name/value pairs objects.*
- struct [DDS\\_PropertySeq](#)  
*Declares IDL sequence < [DDS\\_Property\\_t](#) >*
- struct [DDS\\_PropertyQosPolicy](#)  
*Stores name/value(string) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.*
- class [DDSPROPERTYQOSPolicyHelper](#)  
*Policy helpers that facilitate management of the properties in the input policy.*

## Detailed Description

<<*eXtension*>> Stores (name, value) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.

### 6.2.10.9.33 USER\_DATA

The USER\_DATA QoS policy allows the user to attach a buffer of opaque data to a [DDSDomainParticipant](#), [DDSDataWriter](#), or [DDSDataReader](#). How this information is used will be up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

## Classes

- struct [DDS\\_UserDataQosPolicy](#)  
*Attaches a buffer of opaque data to a [DDSEntity](#) which is distributed to other applications as part of the discovery process.*

## Detailed Description

The USER\_DATA QoS policy allows the user to attach a buffer of opaque data to a [DDSDomainParticipant](#), [DDSDataWriter](#), or [DDSDataReader](#). How this information is used will be up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

### 6.2.10.9.34 GROUP\_DATA

The GROUP\_DATA QoS policy allows you to attach a buffer of opaque data to a [DDSPublisher](#) or [DDSSubscriber](#). How this information is used will be up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

## Classes

- struct [DDS\\_GroupDataQosPolicy](#)

*Attaches a buffer of opaque data to a [DDSEntity](#) which is distributed to other applications as part of the discovery process.*

## Detailed Description

The GROUP\_DATA QoS policy allows you to attach a buffer of opaque data to a [DDSPublisher](#) or [DDSSubscriber](#). How this information is used will be up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

### 6.2.10.9.35 TOPIC\_DATA

The TOPIC\_DATA QoS policy allows the user to attach a buffer of opaque data to a [DDSTopic](#). How this information is used will be up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

## Classes

- struct [DDS\\_TopicDataQosPolicy](#)

*Attaches a buffer of opaque data to a [DDSTopic](#) which is distributed to other applications as part of the discovery process.*

## Detailed Description

The TOPIC\_DATA QoS policy allows the user to attach a buffer of opaque data to a [DDSTopic](#). How this information is used will be up to user code; RTI Connex DDS Micro distributes this information to other applications as part of the discovery process, but RTI Connex DDS Micro does not interpret the information.

### 6.2.10.9.36 DATA\_WRITER\_TRANSFER\_MODE

[<<eXtension>>](#) Specifies the DataWriter transfer mode QoS.

## Classes

- struct [DDS\\_DataWriterShmemRefTransferModeSettings](#)  
*Settings related to transferring data using shared memory references.*
- struct [DDS\\_DataWriterTransferModeQosPolicy](#)  
[<<eXtension>>](#) *Qos related to transferring data*

## Variables

- struct [DDS\\_DataWriterTransferModeQosPolicy](#) [DDS\\_DataWriterQos::transfer\\_mode](#)  
[<<eXtension>>](#) *Qos related to transferring data*

## Detailed Description

<<*eXtension*>> Specifies the `DataWriter` transfer mode QoS.

## Variable Documentation

### transfer\_mode

```
struct DDS_DataWriterTransferModeQosPolicy DDS_DataWriterQos::transfer_mode
```

<<*eXtension*>> Qos related to transferring data

It contains qualitative settings related to the actions a `DDSDataWriter` performs while transferring its data.

Entity:

`DDSDataWriter`

Properties:

`RxO` = N/A

`Changeable` = NO

## 6.2.10.10 Conditions and WaitSets

`DDSCondition` and `DDSWaitSet` and related items.

## Classes

- class `DDSCondition`  
<<*interface*>> Root class for all the conditions that may be attached to a `DDSWaitSet`.
- class `DDSGuardCondition`  
<<*interface*>> A specific `DDSCondition` whose `trigger_value` is completely under the control of the application.
- class `DDSStatusCondition`  
<<*interface*>> A specific `DDSCondition` that is associated with each `DDSEntity`.
- class `DDSWaitSet`  
<<*interface*>> Allows an application to wait until one or more of the attached `DDSCondition` objects has a `trigger_value` of `DDS_BOOLEAN_TRUE` or else until the timeout expires.

### 6.2.10.10.1 Detailed Description

`DDSCondition` and `DDSWaitSet` and related items.

### 6.2.10.11 Entity Support

[DDSEntity](#), [DDSListener](#) and related items.

#### Classes

- class [DDSListener](#)  
*<<interface>> Abstract base class for all Listener interfaces.*
- class [DDSDomainEntity](#)  
*<<interface>> Abstract base class for all DDS entities except for the [DDSDomainParticipant](#).*
- class [DDSEntity](#)  
*<<interface>> Abstract base class for all the DDS objects that support QoS policies, and a listener.*

#### 6.2.10.11.1 Detailed Description

[DDSEntity](#), [DDSListener](#) and related items.

[DDSEntity](#) subtypes are created and destroyed by factory objects. With the exception of [DDSDomainParticipant](#), whose factory is [DDSDomainParticipantFactory](#), all [DDSEntity](#) factory objects are themselves [DDSEntity](#) subtypes as well.

*Important:* all [DDSEntity](#) delete operations are inherently thread-unsafe. The user must take extreme care that a given [DDSEntity](#) is not destroyed in one thread while being used concurrently (including being deleted concurrently) in another thread. An operation's effect in the presence of the concurrent deletion of the operation's target [DDSEntity](#) is undefined.

### 6.2.10.12 Sequence Support

Defines sequence interface and primitive data types sequences.

#### Classes

- struct [FooSeq](#)  
*<<interface>> <<generic>> A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "FOO".*

#### 6.2.10.12.1 Detailed Description

Defines sequence interface and primitive data types sequences.

### 6.2.10.13 Exception codes

Exception codes.



### 6.2.10.13.1 Detailed Description

Exception codes.

### 6.2.10.14 Type Support

Type Plugin.

#### Classes

- struct [NDDS\\_Type\\_PluginVersion](#)  
*Data type used to keep track of versioning information about a [DDS\\_TypePluginI](#).*
- struct [DDS\\_TypePluginI](#)  
*<<interface>> Type plugin interface.*

#### Typedefs

- typedef struct [DDS\\_BuiltinTopicKey\\_t](#) [DDS\\_InstanceId\\_t](#)  
*The data structure by which RTI Connex DDS Micro identifies keys in user data types.*
- typedef struct [NDDS\\_Type\\_PluginVersion](#) [NDDS\\_Type\\_PluginVersion](#)  
*Data type used to keep track of versioning information about a [DDS\\_TypePluginI](#).*
- typedef struct [DDS\\_TypePluginI](#) [DDS\\_TypePluginI](#)  
*<<interface>> Type plugin interface.*

#### Enumerations

- enum [NDDS\\_TypePluginKeyKind](#) { [NDDS\\_TYPEPLUGIN\\_NO\\_KEY](#) , [NDDS\\_TYPEPLUGIN\\_GUID\\_KEY](#) , [NDDS\\_TYPEPLUGIN\\_USER\\_KEY](#) }  
*Type key kind: either keyed or unkeyed.*

### 6.2.10.14.1 Detailed Description

Type Plugin.

### 6.2.10.14.2 Typedef Documentation

#### DDS\_InstanceId\_t

```
typedef struct DDS_BuiltinTopicKey_t DDS_InstanceId_t
```

The data structure by which RTI Connex DDS Micro identifies keys in user data types.

RTI Connex DDS Micro propagates and differentiates between instances of keyed types using [DDS\\_InstanceId\\_t](#) . Before a sample of a keyed type can be written, its key must be translated into one of these.

Users who do not edit the code generated by `rtiddsgen` do not need to deal with this type.

This type is structurally identical to the key type of the built-in topic types. A one-to-one mapping must exist from the key fields of a user data sample to the values of this type's fields.

See also

[DDS\\_BuiltinTopicKey\\_t](#)

**NDDS\_Type\_PluginVersion**

```
typedef struct NDDS_Type_PluginVersion NDDS_Type_PluginVersion
```

Data type used to keep track of versioning information about a [DDS\\_TypePluginI](#).

**DDS\_TypePluginI**

```
typedef struct DDS_TypePluginI DDS_TypePluginI
```

<<[interface](#)>> Type plugin interface.

**6.2.10.14.3 Enumeration Type Documentation****NDDS\_TypePluginKeyKind**

```
enum NDDS_TypePluginKeyKind
```

Type key kind: either keyed or unkeyed.

This type defines whether a user data type is keyed or unkeyed.

**Enumerator**

|                          |                    |
|--------------------------|--------------------|
| NDDS_TYPEPLUGIN_NO_KEY   | Data is not keyed. |
| NDDS_TYPEPLUGIN_USER_KEY | Data is keyed.     |

**6.2.10.15 String Support**

String creation, cloning, assignment, and deletion.

**Functions**

- char \* [DDS\\_String\\_alloc](#) (DDS\_UnsignedLong length)  
*Create a new empty string that can hold up to `length` characters.*
- char \* [DDS\\_String\\_dup](#) (const char \*str)  
*Clone a string. Creates a new string that duplicates the value of `string`.*
- void [DDS\\_String\\_free](#) (char \*str)  
*Delete a string.*
- RTI\_INT32 [DDS\\_String\\_cmp](#) (const char \*s1, const char \*s2)  
*Compare two strings.*
- RTI\_INT32 [DDS\\_String\\_ncmp](#) (const char \*s1, const char \*s2, DDS\_UnsignedLong num)  
*Compare two strings.*
- RTI\_INT32 [DDS\\_String\\_length](#) (const char \*string)  
*Return the length of a string.*

### 6.2.10.15.1 Detailed Description

String creation, cloning, assignment, and deletion.

The methods in this class ensure consistent cross-platform implementations for string creation ([DDS\\_String\\_alloc\(\)](#)), deletion ([DDS\\_String\\_free\(\)](#)), and cloning ([DDS\\_String\\_dup\(\)](#)) that preserve the mutable value type semantics. These are to be viewed as methods that define a `string` class whose data is represented by a `'char*'`.

### 6.2.10.15.2 Conventions

The following conventions govern the memory management of strings in RTI Connex DDS Micro.

- The DDS implementation ensures that when value types containing strings are passed back and forth to the DDS APIs, the strings are created/deleted/assigned/cloned using the `string` class methods.
  - Value types containing strings have ownership of the contained string. Thus, when a value type is deleted, the contained string field is also deleted.
- The user must ensure that when value types containing strings are passed back and forth to the DDS APIs, the strings are created/deleted/assigned/cloned using the `String` class methods.

The representation of a string in C/C++ unfortunately does not allow programs to detect how much memory has been allocated for a string. RTI Connex DDS Micro must therefore make some assumptions when a user requests that a string be copied into. The following rules apply when RTI Connex DDS Micro is copying into a string or string sequence:

- If the `'char*'` is NULL, RTI Connex DDS Micro will log a warning and allocate a new string on behalf of the user. *To avoid leaking memory, you must ensure that the string will be freed (see [Usage](#) below).*
- If the `'char*'` is not NULL, RTI Connex DDS Micro will assume that you are managing the string's memory yourself and have allocated enough memory to store the string to be copied. *RTI Connex DDS Micro will copy into your memory; to avoid memory corruption, be sure to allocate enough of it. Also, do not pass structures containing junk pointers into RTI Connex DDS Micro; you are likely to crash.*

### 6.2.10.15.3 Usage

This requirement can generally be assured by adhering to the following *idiom* for manipulating strings.

```
Always use
    DDS_String_alloc() to create,
    DDS_String_dup() to clone,
    DDS_String_free() to delete
a string 'char*' that is passed back and forth between
user code and the DDS C/C++ APIs.
```

Not adhering to this idiom can result in bad pointers, and incorrect memory being freed.

In addition, the user code should be vigilant to avoid memory leaks. It is good practice to:

- Balance occurrences of [DDS\\_String\\_alloc\(\)](#), [DDS\\_String\\_dup\(\)](#), with matching occurrences of [DDS\\_String\\_free\(\)](#) in the code.
- Finalize value types containing strings. In C++ the destructor accomplishes this automatically. In C, explicit "destructor" functions are provided; these functions are typically called "finalize."

#### 6.2.10.15.4 Function Documentation

##### DDS\_String\_alloc()

```
char * DDS_String_alloc (  
    DDS_UnsignedLong length)
```

Create a new empty string that can hold up to `length` characters.

A string created by this method must be deleted using [DDS\\_String\\_free\(\)](#).

This function will allocate enough memory to hold a string of `length` characters, **plus** one additional byte to hold the NULL terminating character.

##### Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>length</i> | << <i>in</i> >> Capacity of the string. |
|---------------|-----------------------------------------|

##### Returns

A newly created non-NULL string upon success or NULL upon failure.

##### DDS\_String\_dup()

```
char * DDS_String_dup (  
    const char * str)
```

Clone a string. Creates a new string that duplicates the value of `string`.

A string created by this method must be deleted using [DDS\\_String\\_free\(\)](#)

##### Parameters

|            |                                          |
|------------|------------------------------------------|
| <i>str</i> | << <i>in</i> >> The string to duplicate. |
|------------|------------------------------------------|

##### Returns

If `string == NULL`, this method always returns NULL. Otherwise, upon success it returns a newly created string whose value is `string`; upon failure it returns NULL.

##### DDS\_String\_free()

```
void DDS_String_free (  
    char * str)
```

Delete a string.

##### Precondition

`string` must be either NULL, or must have been created using [DDS\\_String\\_alloc\(\)](#), [DDS\\_String\\_dup\(\)](#)

**Parameters**

|            |                                       |
|------------|---------------------------------------|
| <i>str</i> | << <i>in</i> >> The string to delete. |
|------------|---------------------------------------|

References [RTI\\_INT32](#).

**DDS\_String\_cmp()**

```
RTI_INT32 DDS_String_cmp (  
    const char * s1,  
    const char * s2)
```

Compare two strings.

**Precondition**

*s1* and *s2* can be NULL or non-NULL.

**Parameters**

|           |                                      |
|-----------|--------------------------------------|
| <i>s1</i> | << <i>in</i> >> String 1 to compare. |
| <i>s2</i> | << <i>in</i> >> String 2 to compare. |

**Returns**

0 if *s1* equals *s2*, positive integer if *s1* is greater than *s2*, negative integer if *s1* is less than *s2*

References [RTI\\_INT32](#).

**DDS\_String\_ncmp()**

```
RTI_INT32 DDS_String_ncmp (  
    const char * s1,  
    const char * s2,  
    DDS_UnsignedLong num)
```

Compare two strings.

This function compares two strings for lexicographic equivalence, but only up to *num* bytes.

**Parameters**

|    |            |                                          |
|----|------------|------------------------------------------|
| in | <i>s1</i>  | << <i>in</i> >> Left side of comparison  |
| in | <i>s2</i>  | << <i>in</i> >> Right side of comparison |
| in | <i>num</i> | Maximum number of bytes to compare       |

**Returns**

0 if *s1* equals *s2*, positive integer if *s1* is greater than *s2*, negative integer if *s1* is less than *s2*

References [RTI\\_INT32](#).

**DDS\_String\_length()**

```
RTI_INT32 DDS_String_length (
    const char * string)
```

Return the length of a string.

**Precondition**

`string` must be non-NULL

**Parameters**

|               |                                            |
|---------------|--------------------------------------------|
| <i>string</i> | << <i>in</i> >> String to return length of |
|---------------|--------------------------------------------|

**Returns**

length of string. Does not include the NUL-termination character. If a NULL value is passed as argument, RTI\_↔  
SIZE\_INVALID will be returned.

References [RTI\\_INT32](#), and [RTI\\_UINT32](#).

**6.2.11 General Utilities and Compliance Configuration****Topics**

- [Compliance Configuration](#)  
*APIs to configure compliance with certain standard specifications.*
- [Property Reference](#)  
*This is the property reference for RTI Connex DDS Micro.*

**6.2.11.1 Detailed Description****6.2.11.2 Compliance Configuration**

APIs to configure compliance with certain standard specifications.

APIs to configure compliance with certain standard specifications.

**6.2.11.3 Property Reference**

This is the property reference for RTI Connex DDS Micro.

## Variables

- `const char * DDS\_XTYPES\_COMPLIANCE\_MASK\_PROPERTY`  
*Set the `::NDDS_Config_XTypesComplianceMask`.*

### 6.2.11.3.1 Detailed Description

This is the property reference for RTI Connex DDS Micro.

The goal of this Property Reference Guide is to gather all RTI Connex DDS Micro properties in one place and provide the main characteristics of each.

All the properties that RTI Connex DDS Micro provides (which begin with `dds.`, `com.rti.`, or `rti.`) are validated when the applicable entity is created. Please refer to [PROPERTY](#) for information about properties and the Property API.

### 6.2.11.3.2 Variable Documentation

#### DDS\_XTYPES\_COMPLIANCE\_MASK\_PROPERTY

```
const char* DDS_XTYPES_COMPLIANCE_MASK_PROPERTY [extern]
```

Set the `::NDDS_Config_XTypesComplianceMask`.

This property sets the `::NDDS_Config_XTypesComplianceMask` on a [DDSDomainParticipant](#) or [DDSDataWriter](#). If it is set on the [DDSDomainParticipant](#) and [DDSDataWriter](#), the value set on the [DDSDataWriter](#) takes precedence.

## 6.3 DPSE Static Discovery API

### Classes

- struct [DPSE\\_DiscoveryPluginProperty](#)  
*<<eXtension>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. This includes all discovery timing properties for participant discovery.*
- class [DPSEDiscoveryFactory](#)  
*<<eXtension>> A discovery factory that will be used by the [DDSDomainParticipantFactory](#) to create a discovery plugin.*
- class [DPSEDiscoveryPlugin](#)  
*<<eXtension>> Discovery plug-in used for asserting remote entities.*

### Macros

- `#define DPSE\_DiscoveryPluginProperty\_INITIALIZER`  
*Initializer for the [DPSE\\_DiscoveryPluginProperty](#).*

### 6.3.1 Detailed Description

<<eXtension>> The RTI Connex DDS Micro support for DPSE discovery of remote entities. Please refer to the Discovery chapter in the User's Manual, and the Getting Started Guide, for an example of how to register and use the DPSE plugin.

### 6.3.2 Macro Definition Documentation

#### DPSE\_DiscoveryPluginProperty\_INITIALIZER

```
#define DPSE_DiscoveryPluginProperty_INITIALIZER
```

#### Value:

```
{ \
    RT_ComponentFactoryProperty_INITIALIZER, \
    {30,0}, \
    {100,0}, \
    5, \
    {1,0}, \
    DDS_LENGTH_AUTO, \
    4 \
    DDS_PARTICIPANT_MESSAGE_READER_RELIABILITY_KIND_INITIALIZER \
}
```

Initializer for the [DPSE\\_DiscoveryPluginProperty](#).

## 6.4 DPDE Dynamic Discovery API

### Classes

- struct [DPDE\\_DiscoveryPluginProperty](#)  
 <<eXtension>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery.
- class [DPDEDiscoveryFactory](#)  
 <<eXtension>> Discovery plug-in used for asserting remote entities.

### Macros

- #define [DPDE\\_DiscoveryPluginProperty\\_INITIALIZER](#)  
 Initializer for the [DPDE\\_DiscoveryPluginProperty](#).

### 6.4.1 Detailed Description

<<eXtension>> The RTI Connex DDS Micro support for DPDE discovery of remote entities. Please refer to Discovery chapter in the User's Manual, and the Getting Started Guide, for an example of how to register and use the DPDE plugin.



## 6.4.2 Macro Definition Documentation

### DPDE\_DiscoveryPluginProperty\_INITIALIZER

```
#define DPDE_DiscoveryPluginProperty_INITIALIZER
```

#### Value:

```
{\
  RT_ComponentFactoryProperty_INITIALIZER,\
  {30,0},\
  {100,0},\
  5,\
  {1,0},\
  DDS_BOOLEAN_FALSE,\
  DDS_LENGTH_AUTO,\
  4,\
  8,\
  DDS_MAX_UNLIMITED,\
  DDS_LENGTH_UNLIMITED,\
  {0,100000000},\
  DDS_LENGTH_UNLIMITED,\
  DDS_RTPS_NACK_PERIOD_DEFAULT \
  DDS_PARTICIPANT_MESSAGE_READER_RELIABILITY_KIND_INITIALIZER \
}
```

Initializer for the [DPDE\\_DiscoveryPluginProperty](#).

## 6.5 CDR Stream API

### Functions

- [RTI\\_UINT32 CDR\\_Stream\\_get\\_current\\_position\\_offset](#) (struct CDR\_Stream\_t \*cdrs)  
*Return current offset of stream pointer relative to buffer*
- [RTI\\_BOOL CDR\\_Stream\\_set\\_current\\_position\\_offset](#) (struct CDR\_Stream\_t \*cdrs, [RTI\\_UINT32](#) num)  
*Set current offset of stream pointer relative to buffer*
- [RTI\\_BOOL CDR\\_Stream\\_increment\\_current\\_position](#) (struct CDR\_Stream\_t \*me, [RTI\\_INT32](#) amount)  
*Increase stream pointer by specified number of bytes*
- [RTI\\_BOOL CDR\\_Stream\\_check\\_size](#) (struct CDR\_Stream\_t \*me, [RTI\\_UINT32](#) size)  
*Verifies whether remaining space in stream buffer is larger than specified size*
- [char \\* CDR\\_Stream\\_get\\_current\\_position\\_ptr](#) (struct CDR\_Stream\_t \*me)  
*Returns a pointer to the current offset in the stream*
- [RTI\\_BOOL CDR\\_Stream\\_is\\_byte\\_swapped](#) (struct CDR\_Stream\_t \*me)  
*Whether a stream is byte-swapped*

### 6.5.1 Detailed Description

<<*eXtension*>> CDR Stream

### 6.5.2 Function Documentation

#### CDR\_Stream\_get\_current\_position\_offset()

```
RTI\_UINT32 CDR_Stream_get_current_position_offset (
    struct CDR_Stream_t * cdrs)
```

Return current offset of stream pointer relative to buffer

## Parameters

|    |             |      |
|----|-------------|------|
| in | <i>cdrs</i> | Self |
|----|-------------|------|

## Returns

The number of bytes between the stream buffer and the current stream pointer.

**CDR\_Stream\_set\_current\_position\_offset()**

```
RTI_BOOL CDR_Stream_set_current_position_offset (
    struct CDR_Stream_t * cdrs,
    RTI_UINT32 num)
```

Set current offset of stream pointer relative to buffer

## Parameters

|    |             |                                                       |
|----|-------------|-------------------------------------------------------|
| in | <i>cdrs</i> | Self                                                  |
| in | <i>num</i>  | Offset in bytes to set stream pointer ahead of buffer |

## Returns

RTI\_TRUE on success with stream pointer updated. Otherwise, RTI\_FALSE on failure with stream pointer unchanged.

References [RTI\\_UINT32](#).

**CDR\_Stream\_increment\_current\_position()**

```
RTI_BOOL CDR_Stream_increment_current_position (
    struct CDR_Stream_t * me,
    RTI_INT32 amount)
```

Increase stream pointer by specified number of bytes

## Parameters

|    |               |                                            |
|----|---------------|--------------------------------------------|
| in | <i>me</i>     | Self                                       |
| in | <i>amount</i> | Number of bytes to increase stream pointer |

## Returns

RTI\_TRUE on success with stream pointer incremented. RTI\_FALSE on failure with stream pointer unchanged.

References [RTI\\_INT32](#), and [RTI\\_UINT32](#).

**CDR\_Stream\_check\_size()**

```
RTI_BOOL CDR_Stream_check_size (
    struct CDR_Stream_t * me,
    RTI_UINT32 size)
```

Verifies whether remaining space in stream buffer is larger than specified size

**Parameters**

|           |             |                                                 |
|-----------|-------------|-------------------------------------------------|
| <i>in</i> | <i>me</i>   | Self                                            |
| <i>in</i> | <i>size</i> | Number of bytes to check whether fits in stream |

**Returns**

RTI\_TRUE if size fits within stream, otherwise RTI\_FALSE.

References [RTI\\_UINT32](#).

**CDR\_Stream\_get\_current\_position\_ptr()**

```
char * CDR_Stream_get_current_position_ptr (  
    struct CDR_Stream_t * me)
```

Returns a pointer to the current offset in the stream

**Parameters**

|           |           |      |
|-----------|-----------|------|
| <i>in</i> | <i>me</i> | Self |
|-----------|-----------|------|

**Returns**

Pointer to current offset in stream on success, NULL on failure.

References [RTI\\_UINT32](#).

**CDR\_Stream\_is\_byte\_swapped()**

```
RTI_BOOL CDR_Stream_is_byte_swapped (  
    struct CDR_Stream_t * me)
```

Whether a stream is byte-swapped

This function verifies whether a stream is set to serialize or deserialize in byte-swapped order, relative to its host endianness.

**Parameters**

|           |           |      |
|-----------|-----------|------|
| <i>in</i> | <i>me</i> | Self |
|-----------|-----------|------|

**Returns**

RTI\_TRUE if stream is byte-swapped

References [RTI\\_UINT32](#), and [RTI\\_UINT8](#).

## 6.6 RT Run-Time API

### Classes

- class [RTRegistry](#)  
*<<eXtension>> A run-time component factory.*

#### 6.6.1 Detailed Description

The run-time module (RT) manages RTI Connex Micro modules. It provides a common interface to add, remove, and lookup components.

## 6.7 RHSM Reader History API

### Classes

- class [RHSMHistoryFactory](#)  
*<<eXtension>> A reader history factory that will be used by the DataReader to create a history cache.*

#### 6.7.1 Detailed Description

The RHSM plugin is an implementation of the RTI Connex DDS Micro ReaderHistory plugin interface and implements the DataReader reader cache as supported by RTI Connex DDS Micro. RTI Connex DDS Micro requires that a ReaderHistory plugin is registered with the name "rh" in the DomainParticipantFactory before a DataReader can be created. Each instance of a DataReader creates its own instance of the ReaderHistory plugin. Please refer to User Manual for an example on how to register the writer history factory.

## 6.8 WHSM Writer History API

### Classes

- class [WHSMHistoryFactory](#)  
*<<eXtension>> A writer history factory that will be used by the DataWriter to create a history cache.*

#### 6.8.1 Detailed Description

The WHSM plugin is an implementation of the RTI Connex DDS Micro WriterHistory plugin interface and implements the DataWriter writer cache as supported by RTI Connex DDS Micro. RTI Connex DDS Micro requires that a WriterHistory plugin is registered with the name "wh" in the DomainParticipantFactory before a DataWriter can be created. Each instance of a DataWriter creates its own instance of the WriterHistory plugin. Please refer to User Manual for an example on how to register the writer history factory.

## 6.9 NETIO API

<<*eXtension*>> The NETIO API.

### Topics

- [NETIO Packet API](#)  
*NETIO\_Packet implementation.*
- [NETIO Interface API](#)
- [NETIO Address API](#)

### 6.9.1 Detailed Description

<<*eXtension*>> The NETIO API.

### 6.9.2 NETIO Packet API

[NETIO\\_Packet](#) implementation.

### Classes

- struct [NETIO\\_Packet](#)  
*Implementation of the abstract [NETIO\\_Packet](#) type.*

### Typedefs

- typedef struct [NETIO\\_Packet](#) [NETIO\\_Packet\\_T](#)  
*A NETIO Packet.*

### 6.9.2.1 Detailed Description

[NETIO\\_Packet](#) implementation.

### 6.9.2.2 Typedef Documentation

#### NETIO\_Packet\_T

```
typedef struct NETIO\_Packet NETIO\_Packet\_T
```

A NETIO Packet.

The NETIO layers passed [NETIO\\_Packet](#) structures upstream/downstream.

### 6.9.3 NETIO Interface API

#### Classes

- struct [NETIO\\_Interface](#)  
*Base-class definition for all NETIO interfaces.*
- struct [NETIO\\_InterfaceI](#)

#### Typedefs

- typedef struct [NETIO\\_Interface](#) [NETIO\\_Interface\\_T](#)
- typedef [RTI\\_BOOL](#)(\* [NETIO\\_Interface\\_receiveFunc](#)) ([NETIO\\_Interface\\_T](#) \*netio\_intf, struct [NETIO\\_Address](#) \*src\_addr, struct [NETIO\\_Address](#) \*dst\_addr, [NETIO\\_Packet\\_T](#) \*packet)  
*Definition of the ::DDSNETIO\_InterfaceI receive method.*
- typedef [RTI\\_BOOL](#)(\* [NETIO\\_Interface\\_reserve\\_addressFunc](#)) ([NETIO\\_Interface\\_T](#) \*self, struct [NETIO\\_AddressSeq](#) \*req\_address, struct [NETIO\\_AddressSeq](#) \*rsvd\_address, struct [NETIOBindProperty](#) \*property)  
*<<eXtension>> Definition of the ::DDSNETIO\_InterfaceI reserve\_address method*

#### 6.9.3.1 Detailed Description

#### 6.9.3.2 Typedef Documentation

##### NETIO\_Interface\_T

```
typedef struct NETIO\_Interface NETIO\_Interface\_T
```

<<eXtension>> Abstract NETIO Interface type.

##### NETIO\_Interface\_receiveFunc

```
typedef RTI\_BOOL(* NETIO\_Interface\_receiveFunc) (NETIO\_Interface\_T *netio_intf, struct NETIO\_Address *src_addr, struct NETIO\_Address *dst_addr, NETIO\_Packet\_T *packet)
```

Definition of the ::DDSNETIO\_InterfaceI receive method.

<<eXtension>> [NETIO\\_Interface](#) send function.

##### Parameters

|    |                   |                                        |
|----|-------------------|----------------------------------------|
| in | <i>netio_intf</i> | NETIO interface to receive in          |
| in | <i>src_addr</i>   | The source address of the packet       |
| in | <i>dst_addr</i>   | The destination address for the packet |
| in | <i>packet</i>     | The forwarded packet from downstream   |

##### Returns

RTI\_TRUE on success, RTI\_FALSE on failure

See Also [::DDSNETIO\\_Interface::receive](#), [::DDSNETIO\\_InterfaceI](#)

**NETIO\_Interface\_reserve\_addressFunc**

```
typedef RTI_BOOL(* NETIO_Interface_reserve_addressFunc) (NETIO_Interface_T *self, struct NETIO_AddressSeq  
*req_address, struct NETIO_AddressSeq *rsvd_address, struct NETIOBindProperty *property)
```

<<*eXtension*>> Definition of the ::DDSNETIO\_Interface::reserve\_address method

## Parameters

|         |                     |                                                       |
|---------|---------------------|-------------------------------------------------------|
| in      | <i>self</i>         | NETIO interface to reserve addresses on               |
| in      | <i>req_address</i>  | List of requested addresses                           |
| in, out | <i>rsvd_address</i> | The downstream interface                              |
| in      | <i>property</i>     | Properties to use to listen on the reserved addresses |

## Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

## 6.9.4 NETIO Address API

## Classes

- struct [NETIO\\_AddressUInt32](#)  
 <<eXtension>> [NETIO\\_AddressUInt32](#)
- union [NETIO\\_AddressValue](#)  
 <<eXtension>> [NETIO\\_AddresValue](#)
- struct [NETIO\\_Address](#)
- struct [NETIO\\_Netmask](#)  
 <<eXtension>> [NETIO\\_Netmask](#).
- struct [NETIO\\_AddressSeq](#)  
 <<eXtension>> The [NETIO\\_AddressSeq](#) sequence type.
- struct [NETIO\\_NetmaskSeq](#)  
 <<eXtension>> The [NETIO\\_NetmaskSeq](#) sequence type.

## Macros

- #define [NETIO\\_ADDRESS\\_KIND\\_UDPv4](#) 1  
 <<eXtension>> The UDPv4 NETIO Address.
- #define [NETIO\\_ADDRESS\\_KIND\\_UDPv6](#) 2  
 <<eXtension>> The UDPv6 NETIO Address.
- #define [NETIO\\_ADDRESS\\_RTPS\\_RESERVED\\_LOW](#) 0x02000000  
 <<eXtension>> Addresses in the range [[NETIO\\_ADDRESS\\_RTPS\\_RESERVED\\_LOW](#), [NETIO\\_ADDRESS\\_RTPS\\_RESERVED\\_HIGH](#)] are reserved by the RTPS standard.
- #define [NETIO\\_ADDRESS\\_RTPS\\_RESERVED\\_HIGH](#) 0x02ffffff  
 <<eXtension>> Addresses in the range [[NETIO\\_ADDRESS\\_RTPS\\_RESERVED\\_LOW](#), [NETIO\\_ADDRESS\\_RTPS\\_RESERVED\\_HIGH](#)] are reserved by the RTPS standard.
- #define [NETIO\\_Netmask\\_INITIALIZER](#)  
 <<eXtension>> Initialize a [NETIO\\_Netmask](#).
- #define [NETIO\\_Address\\_INITIALIZER](#)  
 <<eXtension>> Initialize a [NETIO\\_Address](#).



#### 6.9.4.1 Detailed Description

#### 6.9.4.2 Macro Definition Documentation

##### NETIO\_ADDRESS\_KIND\_UDPv4

```
#define NETIO_ADDRESS_KIND_UDPv4 1
```

<<*eXtension*>> The UDPv4 NETIO Address.

##### NETIO\_ADDRESS\_KIND\_UDPv6

```
#define NETIO_ADDRESS_KIND_UDPv6 2
```

<<*eXtension*>> The UDPv6 NETIO Address.

##### NETIO\_ADDRESS\_RTPS\_RESERVED\_LOW

```
#define NETIO_ADDRESS_RTPS_RESERVED_LOW 0x02000000
```

<<*eXtension*>> Addresses in the range [NETIO\_ADDRESS\_RTPS\_RESERVED\_LOW, NETIO\_ADDRESS\_RTPS\_RESERVED\_HIGH] are reserved by the RTPS standard.

##### NETIO\_ADDRESS\_RTPS\_RESERVED\_HIGH

```
#define NETIO_ADDRESS_RTPS_RESERVED_HIGH 0x02ffffff
```

<<*eXtension*>> Addresses in the range [NETIO\_ADDRESS\_RTPS\_RESERVED\_LOW, NETIO\_ADDRESS\_RTPS\_RESERVED\_HIGH] are reserved by the RTPS standard.

##### NETIO\_Netmask\_INITIALIZER

```
#define NETIO_Netmask_INITIALIZER
```

##### Value:

```
{ \
    0, \
    {0, 0, 0, 0} \
}
```

<<*eXtension*>> Initialize a NETIO\_Netmask.

**NETIO\_Address\_INITIALIZER**

```
#define NETIO_Address_INITIALIZER
```

**Value:**

```
{\
    NETIO_ADDRESS_KIND_RESERVED, /* kind */ \
    0, /* port */ \
    {{0,0,0,0}}, /* value */ \
    {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0}\
}
```

<<eXtension>> Initialize a [NETIO\\_Address](#).

**6.10 UDP Transport API****Classes**

- struct [UDP\\_NatEntry](#)  
     <<eXtension>> Properties for a Network Address Translation entry
- struct [UDP\\_NatEntrySeq](#)  
     <<eXtension>> Declares IDL sequence<[UDP\\_NatEntry](#)>
- struct [UDP\\_InterfaceTableEntry](#)  
     Generic structure to describe a network interface.
- struct [UDP\\_InterfaceTableEntrySeq](#)  
     <<eXtension>> Declares IDL sequence<[UDP\\_InterfaceTableEntry](#)>
- struct [UDP\\_InterfaceFactoryProperty](#)  
     <<eXtension>> Properties for the UDP Transport.
- class [UDPInterfaceFactory](#)  
     <<eXtension>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory.
- class [UDPInterfaceTable](#)

**Macros**

- #define [UDP\\_INTERFACE\\_MAX\\_IFNAME](#) 64  
     Maximum length of a UDP network interface.
- #define [UDP\\_INTERFACE\\_INTERFACE\\_UP\\_FLAG](#) 0x1U  
     Generic flag to indicate if a network stack is up.
- #define [UDP\\_INTERFACE\\_INTERFACE\\_MULTICAST\\_FLAG](#) 0x2U  
     Generic flag to indicate if a network stack supports multicast.
- #define [UDP\\_INTERFACE\\_MAX\\_NETMASK\\_BITS](#) (32U)  
     The maximum number of bits in a UDP netmask.

## Functions

- NETIOPSLDIIExport [RTI\\_BOOL](#) [UDP\\_InterfaceFactoryProperty\\_initialize](#) (struct [UDP\\_InterfaceFactoryProperty](#) \*self)  
*Initialize a UDP factory property.*
- NETIOPSLDIIExport [RTI\\_BOOL](#) [UDP\\_InterfaceFactoryProperty\\_finalize](#) (struct [UDP\\_InterfaceFactoryProperty](#) \*self)  
*Finalize a UDP factory property.*
- static bool [UDPInterfaceTable::add\\_entry](#) (struct [UDP\\_InterfaceTableEntrySeq](#) \*seq, [RTI\\_UINT32](#) address, [RTI\\_UINT32](#) netmask, const char \*ifname, [RTI\\_UINT32](#) flags)  
*Add an entry to the list of available interfaces.*

### 6.10.1 Detailed Description

<<[eXtension](#)>> The UDP transport implements the RTI Connex DDS Micro NETIO interface and supports UDPv4. Please refer to the User Manual for information about how to use the UDP transport.

### 6.10.2 Macro Definition Documentation

#### UDP\_INTERFACE\_MAX\_IFNAME

```
#define UDP_INTERFACE_MAX_IFNAME 64
```

Maximum length of a UDP network interface.

#### UDP\_INTERFACE\_INTERFACE\_UP\_FLAG

```
#define UDP_INTERFACE_INTERFACE_UP_FLAG 0x1U
```

Generic flag to indicate if a network stack is up.

#### UDP\_INTERFACE\_INTERFACE\_MULTICAST\_FLAG

```
#define UDP_INTERFACE_INTERFACE_MULTICAST_FLAG 0x2U
```

Generic flag to indicate if a network stack supports multicast.

#### UDP\_INTERFACE\_MAX\_NETMASK\_BITS

```
#define UDP_INTERFACE_MAX_NETMASK_BITS (32U)
```

The maximum number of bits in a UDP netmask.

### 6.10.3 Function Documentation

#### UDP\_InterfaceFactoryProperty\_initialize()

```
NETIOPSLD11Export RTI\_BOOL UDP_InterfaceFactoryProperty_initialize (  
    struct UDP\_InterfaceFactoryProperty * self)
```

Initialize a UDP factory property.

## Parameters

|    |      |                        |
|----|------|------------------------|
| in | self | Property to initialize |
|----|------|------------------------|

## Returns

RTI\_TRUE on success, RTI\_FALSE on failure

**UDP\_InterfaceFactoryProperty\_finalize()**

```
NETIOPSLDllExport RTI_BOOL UDP_InterfaceFactoryProperty_finalize (
    struct UDP_InterfaceFactoryProperty * self)
```

Finalize a UDP factory property.

## Parameters

|    |      |                      |
|----|------|----------------------|
| in | self | Property to finalize |
|----|------|----------------------|

## Returns

RTI\_TRUE on success, RTI\_FALSE on failure

**add\_entry()**

```
static bool UDPInterfaceTable::add_entry (
    struct UDP_InterfaceTableEntrySeq * seq,
    RTI_UINT32 address,
    RTI_UINT32 netmask,
    const char * ifname,
    RTI_UINT32 flags) [static]
```

Add an entry to the list of available interfaces.

This function can be used to add an interface to the if\_table in the [UDP\\_InterfaceFactoryProperty](#).

## Parameters

|    |         |                                                                                                                         |
|----|---------|-------------------------------------------------------------------------------------------------------------------------|
| in | seq     | The sequence to add the entry to                                                                                        |
| in | address | The UDPv4 interface address to add in host order                                                                        |
| in | netmask | The interface netmask, maximum of <a href="#">UDP_INTERFACE_MAX_NETMASK_BITS</a> bits.                                  |
| in | ifname  | The interface name. Cannot exceed <a href="#">UDP_INTERFACE_MAX_IFNAME</a> including NUL.                               |
| in | flags   | Bitmap of <a href="#">UDP_INTERFACE_INTERFACE_UP_FLAG</a> and/or <a href="#">UDP_INTERFACE_INTERFACE_MULTICAST_FLAG</a> |

## Returns

RTI\_TRUE on success, RTI\_FALSE on failure

References [RTI\\_UINT32](#).

## 6.11 UDP Transform API

<<eXtension>> The UDP transform API enables custom transformation of UDP incoming and outgoing UDP datagrams. Please refer to User Manual for the User Manual and how to use the UDP transforms.

## 6.12 RTPS Transport API

### Classes

- union [RTPS\\_Checksum](#)  
<<eXtension>> RTPS checksum type
- struct [RTPS\\_ChecksumClass](#)  
<<eXtension>> Definition of a checksum function
- struct [RTPS\\_ChecksumProperty](#)  
<<eXtension>> Checksum properties for the RTPS plugin
- struct [RTPS\\_InterfaceFactoryProperty](#)  
<<eXtension>> Properties for the RTPS Transport.

### Typedefs

- typedef [RTI\\_INT16 RTPS\\_ChecksumClassId\\_T](#)  
<<eXtension>> A checksum class ID
- typedef union [RTPS\\_Checksum RTPS\\_Checksum\\_T](#)  
<<eXtension>> RTPS checksum type
- typedef [RTI\\_BOOL\(\\* RTPS\\_ChecksumCalculate\\_T\)](#) (void \*context, const struct REDA\_Buffer \*buf, [RTI\\_UINT32](#) buf\_length, [RTPS\\_Checksum\\_T](#) \*checksum)  
<<eXtension>> The checksum calculation function
- typedef struct [RTPS\\_ChecksumClass RTPS\\_ChecksumClass\\_T](#)  
<<eXtension>> Definition of a checksum function
- typedef struct [RTPS\\_InterfaceFactoryProperty RTPS\\_InterfaceFactoryProperty\\_T](#)  
<<eXtension>> Properties for the RTPS Transport.

### Enumerations

- enum [RTPS\\_ChecksumTxMode\\_T](#) { [RTPS\\_CHECKSUM\\_TXMODE\\_RTICRC32](#) , [RTPS\\_CHECKSUM\\_TXMODE\\_OMG](#) }  
<<eXtension>> Transmit mode for checksums

## Variables

- `RTI_UINT32 RTPS_Checksum::checksum32`  
*A 32-bit unsigned checksum in host endianness order.*
- `RTI_UINT64 RTPS_Checksum::checksum64`  
*A 64-bit unsigned integer checksum in host endianness order.*
- `RTI_UINT8 RTPS_Checksum::checksum128 [16]`  
*A 128-bit checksum as a 16-byte array.*
- `RTPS_ChecksumClassId_T RTPS_ChecksumClass::class_id`  
*The checksum class ID.*
- `void * RTPS_ChecksumClass::context`  
*User defined context.*
- `RTPS_ChecksumCalculate_T RTPS_ChecksumClass::checksum_calculate`  
*User defined function to calculate a checksum.*
- `RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum32_class`  
*The builtin 32-bit checksum class.*
- `RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum64_class`  
*The builtin 64-bit checksum class.*
- `RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum128_class`  
*The builtin 128-bit checksum class.*
- `RTPS_ChecksumTxMode_T RTPS_ChecksumProperty::checksum_tx_mode`  
*The transmit mode for checksums.*
- `RTI_BOOL RTPS_ChecksumProperty::allow_builtin_override`  
*If TRUE, allow overriding the builtin checksum functions.*
- `struct RTPS_ChecksumProperty RTPS_InterfaceFactoryProperty::checksum`  
*The checksum properties for RTPS.*
- `const struct RTPS_InterfaceFactoryProperty RTPS_INTERFACE_FACTORY_DEFAULT`  
*<<eXtension>> Default properties for the RTPS Transport.*

### 6.12.1 Detailed Description

<<eXtension>> The RTPS Plugin implements the RTI Connex DDS Micro RTPS protocol as an RTI Connex DDS Micro NETIO interface.

### 6.12.2 Typedef Documentation

#### RTPS\_ChecksumClassId\_T

```
typedef RTI_INT16 RTPS_ChecksumClassId_T
```

<<eXtension>> A checksum class ID

#### RTPS\_Checksum\_T

```
typedef union RTPS_Checksum RTPS_Checksum_T
```

<<eXtension>> RTPS checksum type

## RTPS\_ChecksumCalculate\_T

```
typedef RTI_BOOL(* RTPS_ChecksumCalculate_T) (void *context, const struct REDA_Buffer *buf, RTI_UINT32
buf_length, RTPS_Checksum_T *checksum)
```

<<*eXtension*>> The checksum calculation function

### Parameters

|     |                   |                                                  |
|-----|-------------------|--------------------------------------------------|
| in  | <i>context</i>    | Context passed as property                       |
| in  | <i>buf</i>        | Vector of buffers to calculate the checksum over |
| in  | <i>buf_length</i> | Number of elements in buf                        |
| out | <i>checksum</i>   | Calculated checksum                              |

### Returns

RTI\_TRUE on success, RTI\_FALSE on failure

## RTPS\_ChecksumClass\_T

```
typedef struct RTPS_ChecksumClass RTPS_ChecksumClass_T
```

<<*eXtension*>> Definition of a checksum function

## RTPS\_InterfaceFactoryProperty\_T

```
typedef struct RTPS_InterfaceFactoryProperty RTPS_InterfaceFactoryProperty_T
```

<<*eXtension*>> Properties for the RTPS Transport.

## 6.12.3 Enumeration Type Documentation

### RTPS\_ChecksumTxMode\_T

```
enum RTPS_ChecksumTxMode_T
```

<<*eXtension*>> Transmit mode for checksums

### Enumerator

|                               |                                              |
|-------------------------------|----------------------------------------------|
| RTPS_CHECKSUM_TXMODE_RTICRC32 | Always use CRC32, fail if not possible.      |
| RTPS_CHECKSUM_TXMODE_OMG      | Always use OMG's header extension [Default]. |

#### 6.12.4 Variable Documentation

##### checksum32

`RTI_UINT32` RTPS\_Checksum::checksum32

A 32-bit unsigned checksum in host endianness order.

##### checksum64

`RTI_UINT64` RTPS\_Checksum::checksum64

A 64-bit unsigned integer checksum in host endianness order.

##### checksum128

`RTI_UINT8` RTPS\_Checksum::checksum128[16]

A 128-bit checksum as a 16-byte array.

##### class\_id

`RTPS_ChecksumClassId_T` RTPS\_ChecksumClass::class\_id

The checksum class ID.

##### context

`void*` RTPS\_ChecksumClass::context

User defined context.

##### checksum\_calculate

`RTPS_ChecksumCalculate_T` RTPS\_ChecksumClass::checksum\_calculate

User defined function to calculate a checksum.

##### builtin\_checksum32\_class

`RTPS_ChecksumClass_T` RTPS\_ChecksumProperty::builtin\_checksum32\_class

The builtin 32-bit checksum class.



**builtin\_checksum64\_class**

```
RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum64_class
```

The builtin 64-bit checksum class.

**builtin\_checksum128\_class**

```
RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum128_class
```

The builtin 128-bit checksum class.

**checksum\_tx\_mode**

```
RTPS_ChecksumTxMode_T RTPS_ChecksumProperty::checksum_tx_mode
```

The transmit mode for checksums.

**allow\_builtin\_override**

```
RTI_BOOL RTPS_ChecksumProperty::allow_builtin_override
```

If TRUE, allow overriding the builtin checksum functions.

**checksum**

```
struct RTPS_ChecksumProperty RTPS_InterfaceFactoryProperty::checksum
```

The checksum properties for RTPS.

**RTPS\_INTERFACE\_FACTORY\_DEFAULT**

```
const struct RTPS_InterfaceFactoryProperty RTPS_INTERFACE_FACTORY_DEFAULT [extern]
```

<<*eXtension*>> Default properties for the RTPS Transport.

## 6.13 Shared Memory Transport API

**Topics**

- [NETIO Shared Memory API](#)

*OS independent way to setup and access shared-memory segments, shared-memory mutexes and shared memory binary semaphores.*

## Classes

- struct [NETIO\\_SHMEMInterfaceFactoryProperty](#)  
*<<eXtension>> Properties for the SHMEM Transport.*
- class [SHMEMInterfaceFactory](#)  
*<<eXtension>> A SHMEM transport factory that will be used by a DomainParticipant to create a SHMEM interface. Only one SHMEM interface can be registered with the DomainParticipantFactory.*

### 6.13.1 Detailed Description

*<<eXtension>>* The shared memory (SHMEM) transport implements the RTI Connex DDS Micro NETIO interface and uses a message queue in shared memory for communication between DomainParticipants. Please refer to the User's Manual for information about how to use the SHMEM transport.

### 6.13.2 NETIO Shared Memory API

OS independent way to setup and access shared-memory segments, shared-memory mutexes and shared memory binary semaphores.

## Topics

- [Shared Memory Mutex API](#)
- [Shared Memory Segment API](#)  
*Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.*
- [Shared Memory Signaling Semaphore](#)

#### 6.13.2.1 Detailed Description

OS independent way to setup and access shared-memory segments, shared-memory mutexes and shared memory binary semaphores.

This module consist of three constructs: SharedMemorySegment, SharedMemoryBinarySemaphore, SharedMemory↵Mutex.

These objects, represent operating-system resources that are accessible from different processes running on the same computer. These three objects are manipulated in very similar ways and have also very similar lifecycles.

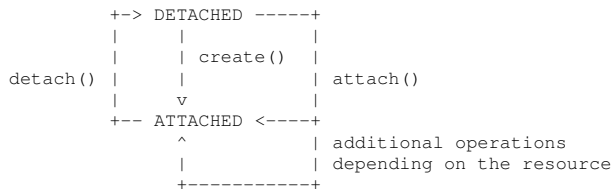
The model is as follows. The underlying shared memory resource ( segment, semaphore, and mutex) is identified using an integer "key". Processes wishing to create or interact with the resource objects must do it via the appropriate handles. To be portable across operating systems the application should consider keys to be unique even across different kinds of shared memory resources, that is if a key is specified for a shared-memory segment it cannot be re-used to specify a shared-memory semaphore or mutex. Different keys must be used for different resources even if they are of different kinds.

The thing to keep in mind when looking at the lifecycles is that there are potentially many shared-memory handles referencing the same underlying shared-memory resource. That is in fact the whole point of using shared memory resource.

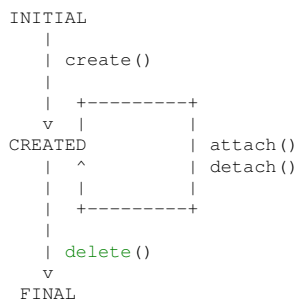
A shared-memory object can be in one of the following states: CREATED, OPEN, CLOSED.

A shared-memory handle can be in one of the following states: DETACHED, ATTACHED.

The state transitions of the shared-memory object are surrogate to those of corresponding handles because processes can only manipulate shared memory using the handles. The following charts describe this:



State of the shared-memory resource:



No control is performed on the permission of the delete() function. Once a resource is deleted, it is permanently destroyed and any subsequent call to attach() should fail.

The shared-memory objects used in this module represent named operating-system resources. Many OSs provide utilities to view and administer these operating system resources from an independent API. For example, in Unix/Linux the command-line utility "ipcs" can be used to see a listing of the current shared memory segments, semaphores and mutexes. Similarly "ipcrm" can be used to remove shared-memory resources.

### 6.13.2.2 Shared Memory Mutex API

#### Functions

- RTIBool [NETIO\\_SharedMemoryMutex\\_create](#) (struct NETIO\_SharedMemoryMutexHandle \*handle, [RTI\\_INT32](#) \*status\_out, [RTI\\_INT32](#) key)  
*Create a shared memory mutex.*
- RTIBool [NETIO\\_SharedMemoryMutex\\_attach](#) (struct NETIO\_SharedMemoryMutexHandle \*handle, [RTI\\_INT32](#) \*status\_out, [RTI\\_INT32](#) key)  
*Attach to an existing shared memory binary semaphore with the provided key and sets up the handle to access it.*
- RTIBool [NETIO\\_SharedMemoryMutex\\_create\\_or\\_attach](#) (struct NETIO\_SharedMemoryMutexHandle \*handle, [RTI\\_INT32](#) \*status\_out, [RTI\\_INT32](#) key)  
*Attempts to create a new binary semaphore with the provided key and if the semaphore already exist, attach to it.*
- RTIBool [NETIO\\_SharedMemoryMutex\\_unlock](#) (struct NETIO\_SharedMemoryMutexHandle \*handle, [RTI\\_INT32](#) \*status\_out)

*Give the semaphore.*

- RTIBool [NETIO\\_SharedMemoryMutex\\_lock](#) (struct NETIO\_SharedMemoryMutexHandle \*handle, [RTI\\_INT32](#) \*status\_out)

*Take the semaphore.*

- RTIBool [NETIO\\_SharedMemoryMutex\\_detach](#) (struct NETIO\_SharedMemoryMutexHandle \*handle)

*Detach from the shared memory binary semaphore.*

- RTIBool [NETIO\\_SharedMemoryMutex\\_delete](#) (struct NETIO\_SharedMemoryMutexHandle \*handle)

*Detach from the shared memory semaphore and deletes it.*

### 6.13.2.2.1 Detailed Description

### 6.13.2.2.2 Function Documentation

#### NETIO\_SharedMemoryMutex\_create()

```
RTIBool NETIO_SharedMemoryMutex_create (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Create a shared memory mutex.

#### Parameters

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>     | out Pointer to handle on successful return                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <i>status_out</i> | <b>out.</b> Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION</b> = precondition check failed.</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b>= this error should never happen and is caused by a failed create() followed by a failed attach().</li> <li>• <b>OSAPI_SHARED_MEMORY_CREATED</b> = the semaphore has been successfully created.</li> </ul> |
| <i>key</i>        | <b>in.</b> Key to identify the shared memory semaphore across processes.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

#### Returns

RTI\_TRUE on success. RTI\_FALSE on error.

References [RTI\\_INT32](#).

## NETIO\_SharedMemoryMutex\_attach()

```
RTIBool NETIO_SharedMemoryMutex_attach (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attach to an existing shared memory binary semaphore with the provided key and sets up the handle to access it.

### Precondition

Handle in the DETACHED state. Signaling semaphore in the CREATED state. Some other process has called [NETIO\\_SharedMemoryMutex\\_create](#) using the same key in the shared memory semaphore

### Postcondition

handle in the ATTACHED state, Semaphore exists and is ready to be used.

### Parameters

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>     | <b>out.</b> The shared memory semaphore handle to initialize.                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>status_out</i> | <b>out.</b> Optional. If not NULL, will be set to: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION</b> = precondition check failed.</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b> = no such a semaphore.</li> <li>• <b>OSAPI_SHARED_MEMORY_ATTACHED</b> = semaphore attached successfully.</li> </ul> |
| <i>key</i>        | <b>in.</b> Key to identify the shared memory semaphore across processes.                                                                                                                                                                                                                                                                                                                                                                                                                                               |

### Returns

RTI\_TRUE on success. RTI\_FALSE on error.

### Note

On some platforms this method makes sure that the given key represent a shared memory semaphore object (to avoid situations like the same key is used to create a mutex and then used to attach it as a semaphore). If such a situation is detected, the function will fail and set status\_out to: OSAPI\_SHARED\_MEMORYFAIL\_REASON\_↵ NO\_ENTRY. Not all the platforms support this feature.

### See also

[NETIO\\_SharedMemoryMutex\\_detach](#)

References [RTI\\_INT32](#).

**NETIO\_SharedMemoryMutex\_create\_or\_attach()**

```
RTIBool NETIO_SharedMemoryMutex_create_or_attach (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attempts to create a new binary semaphore with the provided key and if the semaphore already exist, attach to it.

**Precondition**

Handle in the DETACHED state. Signaling semaphore in the INITIAL, that is, Nobody has called create() before with that same key.

**Postcondition**

handle in the ATTACHED state, Semaphore is in the CREATED state and ready to be used.

**Parameters**

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>     | <b>inOut.</b> The shared memory semaphore handle to initialize.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>status_out</i> | <b>out.</b> Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION</b> = precondition check failed.</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b>= this error should never happen and is caused by a failed create() followed by a failed attach().</li> <li>• <b>OSAPI_SHARED_MEMORY_CREATED</b> = the semaphore has been successfully created.</li> </ul> |
| <i>key</i>        | <b>in.</b> Key to identify the shared memory binary semaphore across processes.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

**Returns**

RTI\_TRUE on success. RTI\_FALSE on error.

References [RTI\\_INT32](#).

**NETIO\_SharedMemoryMutex\_unlock()**

```
RTIBool NETIO_SharedMemoryMutex_unlock (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out)
```

Give the semaphore.

**Precondition**

Handle in the ATTACHED state.

**Postcondition**

Semaphore count is set to 1 and any thread blocked on this semaphore is awoken.

## Parameters

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>     | <b>in.</b> Handle to the semaphore. Must match the one returned by <a href="#">NETIO_SharedMemoryMutex_create</a>                                                                                                                                                                                                                                                                                                                                                                                   |
| <i>status_out</i> | <b>out.</b> Optional: if not NULL, will be set to: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b> = the creator of the mutex has shutdown and removed this mutex. This is expected to happen routinely.</li> <li>• <b>OSAPI_SHARED_MEMORY_SUCCESS</b> = Generic no error condition.</li> </ul> |

## Returns

RTI\_TRUE if the semaphore is given. RTI\_FALSE if an error occurs. If an error occurs the handle should not be used anymore. In particular detach() should not be called either. An error may occur in certain OSs if the handle that created the semaphore calls [NETIO\\_SharedMemoryMutex\\_detach](#).

## See also

[NETIO\\_SharedMemorySignalingSemaphore\\_wait](#)

References [RTI\\_INT32](#).

**NETIO\_SharedMemoryMutex\_lock()**

```
RTIBool NETIO_SharedMemoryMutex_lock (
    struct NETIO_SharedMemoryMutexHandle * handle,
    RTI_INT32 * status_out)
```

Take the semaphore.

## Precondition

Handle in the ATTACHED state.

## Postcondition

Semaphore count is decremented.

## Parameters

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>     | <b>in.</b> Handle to the semaphore. Must match the one returned by <a href="#">NETIO_SharedMemoryMutex_create</a>                                                                                                                                                                                                                                                                                                                                                                                   |
| <i>status_out</i> | <b>out.</b> Optional: if not NULL, will be set to: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b> = the creator of the mutex has shutdown and removed this mutex. This is expected to happen routinely.</li> <li>• <b>OSAPI_SHARED_MEMORY_SUCCESS</b> = Generic no error condition.</li> </ul> |

**Returns**

RTI\_TRUE if the semaphore is taken. RTI\_FALSE if an error occurs. If an error occurs the handle should not be used anymore. In particular [NETIO\\_SharedMemoryMutex\\_detach](#) should not be called either.

**See also**

[NETIO\\_SharedMemorySignalingSemaphore\\_signal](#)

References [RTI\\_INT32](#).

**NETIO\_SharedMemoryMutex\_detach()**

```
RTIBool NETIO_SharedMemoryMutex_detach (
    struct NETIO_SharedMemoryMutexHandle * handle)
```

Detach from the shared memory binary semaphore.

**Precondition**

Handle in the ATTACHED state.

**Postcondition**

handle in the DETACHED state.

**Parameters**

|               |                                                                                                               |
|---------------|---------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | in handle. Handle to the shared memory semaphore obtained from <a href="#">NETIO_SharedMemoryMutex_attach</a> |
|---------------|---------------------------------------------------------------------------------------------------------------|

**See also**

[NETIO\\_SharedMemoryMutex\\_attach](#)

**NETIO\_SharedMemoryMutex\_delete()**

```
RTIBool NETIO_SharedMemoryMutex_delete (
    struct NETIO_SharedMemoryMutexHandle * handle)
```

Detach from the shared memory semaphore and deletes it.

The exact behavior of this may be OS dependent. In some OSs calling this will immediately cause the semaphore to be removed, and other threads waiting on it to be woken up. This is the typical behavior on Unix. In other OSs (windows) this call just requests the semaphore to be removed but delay the removal until all handles are detached.

The only way to ensure the same behavior across multiple OSs is to only call this method on the handle that created the semaphore after all the other handles have been closed. In this scenario no handles will get an error. Alternatively the code could be written to always expect the [NETIO\\_SharedMemorySignalingSemaphore\\_wait](#) or [NETIO\\_SharedMemorySignalingSemaphore\\_signal](#) call to fail and when that occurs close the handle.

**Precondition**

Handle in the ATTACHED state.

**Postcondition**

handle in the DETACHED state. The numHandles to the semaphore is decremented. When the last handle is detached, the semaphore is removed by the OS.



## Parameters

|               |                                                                                                                      |
|---------------|----------------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | <b>in</b> handle. Handle to the shared memory semaphore obtained from <a href="#">NETIO_SharedMemoryMutex_create</a> |
|---------------|----------------------------------------------------------------------------------------------------------------------|

## See also

[NETIO\\_SharedMemoryMutex\\_create](#)

References [RTI\\_INT32](#).

### 6.13.2.3 Shared Memory Segment API

Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.

## Classes

- struct [NETIO\\_SharedMemorySegmentHeader](#)

## Functions

- RTIBool [NETIO\\_SharedMemorySegment\\_create\\_or\\_attach](#) (struct NETIO\_SharedMemorySegmentHandle \*handle, [RTI\\_INT32](#) \*status\_out, [RTI\\_INT32](#) key, [RTI\\_UINT32](#) size, OSAPI\_ProcessId pid\_in)  
*Attempts to create a new shared memory segment. If the segment already exists and the creator process is dead, attach and recycle it.*
- RTIBool [NETIO\\_SharedMemorySegment\\_create](#) (struct NETIO\_SharedMemorySegmentHandle \*handle, [RTI\\_INT32](#) \*status\_out, [RTI\\_INT32](#) key, [RTI\\_UINT32](#) size, OSAPI\_ProcessId pid\_in)  
*Creates a new shared memory segment with the provided key and returns the handle to access it.*
- RTIBool [NETIO\\_SharedMemorySegment\\_delete](#) (struct NETIO\_SharedMemorySegmentHandle \*handle)  
*Deletes the shared memory segment previously create with [NETIO\\_SharedMemorySegment\\_create](#). If the segment is still attached, it is detached first.*
- RTIBool [NETIO\\_SharedMemorySegment\\_attach](#) (struct NETIO\_SharedMemorySegmentHandle \*handle, [RTI\\_INT32](#) \*status\_out, [RTI\\_INT32](#) key)  
*Attach to the shared memory previously initialized with [NETIO\\_SharedMemorySegment\\_create\\_or\\_attach](#) and returns a handle to it.*
- RTIBool [NETIO\\_SharedMemorySegment\\_detach](#) (struct NETIO\_SharedMemorySegmentHandle \*handle)  
*Detach from the shared memory previously created with [NETIO\\_SharedMemorySegment\\_attach](#).*
- char \* [NETIO\\_SharedMemorySegment\\_get\\_address](#) (struct NETIO\_SharedMemorySegmentHandle \*handle)  
*Get the address of the shared memory attached by the handle.*
- NETIOPSLDIExport [RTI\\_INT32 NETIO\\_SharedMemorySegment\\_get\\_size](#) (struct NETIO\_SharedMemorySegmentHandle \*handle)  
*Get the size (available for user data) of the shared memory segment.*
- NETIOPSLDIExport [RTI\\_UINT32 NETIO\\_SharedMemorySegment\\_get\\_max\\_size](#) (void)  
*Maximum allowable size of a shared memory segment.*

### 6.13.2.3.1 Detailed Description

Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.

### 6.13.2.3.2 Function Documentation

#### NETIO\_SharedMemorySegment\_create\_or\_attach()

```
RTIBool NETIO_SharedMemorySegment_create_or_attach (
    struct NETIO_SharedMemorySegmentHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key,
    RTI_UINT32 size,
    OSAPI_ProcessId pid_in)
```

Attempts to create a new shared memory segment. If the segment already exists and the creator process is dead, attach and recycle it.

#### Parameters

|         |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in, out | <i>handle</i>     | The shared memory segment handle to initialize.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| out     | <i>status_out</i> | Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular, <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION</b> = precondition check failed.</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_ALREADY_CLAIMED</b> = someone is already sitting on this address.</li> <li>• <b>OSAPI_SHARED_MEMORY_CREATED</b> = the segment has been created new.</li> <li>• <b>OSAPI_SHARED_MEMORY_ATTACHED</b> = the segment was reused from an existing one.</li> </ul> |
| in      | <i>key</i>        | Key to identify the shared memory segment Across processes. Other processes should call <a href="#">NETIO_SharedMemorySegment_attach</a> with the same key to attach to this segment.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| in      | <i>size</i>       | Size of memory to be initialized.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| in      | <i>pid_in</i>     | PID of the process that calls this method.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

#### Returns

RTI\_TRUE if success, or RTI\_FALSE if something failed.

References [RTI\\_INT32](#), and [RTI\\_UINT32](#).

## NETIO\_SharedMemorySegment\_create()

```
RTIBool NETIO_SharedMemorySegment_create (
    struct NETIO_SharedMemorySegmentHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key,
    RTI_UINT32 size,
    OSAPI_ProcessId pid_in)
```

Creates a new shared memory segment with the provided key and returns the handle to access it.

### Parameters

|         |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in, out | <i>handle</i>     | The shared memory segment handle to initialize.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| out     | <i>status_out</i> | Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular, <ul style="list-style-type: none"> <li><b>OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION</b> = precondition check failed.</li> <li><b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li><b>OSAPI_SHARED_MEMORY_FAIL_REASON_ALREADY_CLAIMED</b> = segment already exist (no checks is performed whether the creator is still alive or not).</li> <li><b>OSAPI_SHARED_MEMORY_CREATED</b> = the segment has been successfully created.</li> </ul> |
| in      | <i>key</i>        | <b>Key</b> to identify the shared memory segment across processes. Other processes should call <a href="#">NETIO_SharedMemorySegment_attach</a> with the same key to attach to this segment.                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| in      | <i>size</i>       | <b>Size</b> of memory to be initialized.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| in      | <i>pid_in</i>     | <b>PID</b> of the process that calls this method.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

### Returns

RTI\_TRUE if success, or RTI\_FALSE if something failed.

### Note

Depending on the underlying OS, the key may need to be transformed to the actual representation needed by the operating system

References [RTI\\_INT32](#), and [RTI\\_UINT32](#).

## NETIO\_SharedMemorySegment\_delete()

```
RTIBool NETIO_SharedMemorySegment_delete (
    struct NETIO_SharedMemorySegmentHandle * handle)
```

deletes the shared memory segment previously create with [NETIO\\_SharedMemorySegment\\_create](#). If the segment is still attached, it is detached first.

## Parameters

|    |               |                                                                    |
|----|---------------|--------------------------------------------------------------------|
| in | <i>handle</i> | The shared memory segment handle identifying the segment to detach |
|----|---------------|--------------------------------------------------------------------|

## Returns

RTI\_TRUE on success. RTI\_FALSE if either the precondition was not met or the shared memory segment couldn't be removed (i.e. the user running this app is not the owner or creator).

## Note

This function does not perform any control on the ownership of the segment. If you attach to a segment, then when you call this function the segment will be destroyed (if you have enough permission to complete the operation)

When you create a segment with [NETIO\\_SharedMemorySegment\\_create](#) or with [NETIO\\_SharedMemorySegment\\_create\\_or\\_attach](#) you can delete it by calling directly [NETIO\\_SharedMemorySegment\\_delete](#) without calling [NETIO\\_SharedMemorySegment\\_detach](#).

Once called this function (even if it returns RTI\_FALSE) you cannot use anymore this shared memory segment.

**NETIO\_SharedMemorySegment\_attach()**

```
RTIBool NETIO_SharedMemorySegment_attach (
    struct NETIO_SharedMemorySegmentHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attach to the shared memory previously initialized with [NETIO\\_SharedMemorySegment\\_create\\_or\\_attach](#) and returns a handle to it.

## Parameters

|         |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in, out | <i>handle</i>     | The shared memory segment handle to initialize.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| out     | <i>status_out</i> | Optional. If not NULL, will be set to: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION</b> = precondition check failed.</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b> = no such a segment.</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_OUT_OF_MEMORY</b> = out of memory allocating the handle structure.</li> <li>• <b>OSAPI_SHARED_MEMORY_ATTACHED</b> = segment attached successfully.</li> </ul> |
| in      | <i>key</i>        | Key to identify the shared memory segment across processes. Must match that used by other processes calling <a href="#">NETIO_SharedMemorySegment_create_or_attach</a> or <a href="#">NETIO_SharedMemorySegment_attach()</a> .                                                                                                                                                                                                                                                                                                                                                                                          |

## Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

References [RTI\\_INT32](#).

## NETIO\_SharedMemorySegment\_detach()

```
RTIBool NETIO_SharedMemorySegment_detach (  
    struct NETIO_SharedMemorySegmentHandle * handle)
```

Detach from the shared memory previously created with [NETIO\\_SharedMemorySegment\\_attach](#).

### Parameters

|    |               |                                                                    |
|----|---------------|--------------------------------------------------------------------|
| in | <i>handle</i> | The shared memory segment handle identifying the segment to detach |
|----|---------------|--------------------------------------------------------------------|

### Returns

RTI\_TRUE on success. RTI\_FALSE if the precondition was not met.

The shared memory segment is always detached. Once the precondition is met, no errors will be generated.

Once called this function (even if it returns RTI\_FALSE) you cannot use anymore this shared memory segment, except for destroying it.

## NETIO\_SharedMemorySegment\_get\_address()

```
char * NETIO_SharedMemorySegment_get_address (  
    struct NETIO_SharedMemorySegmentHandle * handle)
```

Get the address of the shared memory attached by the handle.

### Precondition

Handle in the ATTACHED state.

### Postcondition

none

### Parameters

|    |               |                                                                                                                                                                       |
|----|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>handle</i> | Handle to the shared memory segment obtained from an <a href="#">NETIO_SharedMemorySegment_attach</a> or a <a href="#">NETIO_SharedMemorySegment_create_or_attach</a> |
|----|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Returns

the address that can be used to read and write the memory. For performance this function never checks pre-conditions. It is the caller's responsibility to ensure the handle is NON null and ATTACHED. A NULL handle will cause a seg-fault. An un-attached handle may give invalid values for the [NETIO\\_SharedMemorySegment\\_get\\_size](#)

References [RTI\\_INT32](#).

**NETIO\_SharedMemorySegment\_get\_size()**

```
NETIOPSLD11Export RTI_INT32 NETIO_SharedMemorySegment_get_size (  
    struct NETIO_SharedMemorySegmentHandle * handle)
```

Get the size (available for user data) of the shared memory segment.

This size will match the one passed to the [NETIO\\_SharedMemorySegment\\_create\\_or\\_attach](#) call

**Precondition**

Handle in the ATTACHED state.

**Postcondition**

none

**Parameters**

|    |               |                                                                                                                                                                             |
|----|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>handle</i> | Handle to the shared memory segment obtained from an <a href="#">NETIO_SharedMemorySegment_attach</a> or a <a href="#">NETIO_SharedMemorySegment_create_or_attach</a> call. |
|----|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Returns**

the number of bytes that can be read/written starting at the address provided by [NETIO\\_SharedMemorySegment\\_get\\_address](#). For performance this function never checks pre-conditions. It is the caller's responsibility to ensure the handle is NON null and ATTACHED. A NULL handle will cause a seg-fault. An un-attached handle may give invalid values for the size.

References [RTI\\_UINT32](#).

**NETIO\_SharedMemorySegment\_get\_max\_size()**

```
NETIOPSLD11Export RTI_UINT32 NETIO_SharedMemorySegment_get_max_size (  
    void )
```

Maximum allowable size of a shared memory segment.

References [RTI\\_INT32](#).

### 6.13.2.4 Shared Memory Signaling Semaphore

#### Functions

- RTIBool [NETIO\\_SharedMemorySignalingSemaphore\\_create](#) (struct NETIO\_SharedMemorySignalingSemaphoreHandle \*handle, RTI\_INT32 \*status\_out, RTI\_INT32 key)  
*Initializes a shared memory binary semaphore with the provided key and sets up the handle to access it.*
- RTIBool [NETIO\\_SharedMemorySignalingSemaphore\\_attach](#) (struct NETIO\_SharedMemorySignalingSemaphoreHandle \*handle, RTI\_INT32 \*status\_out, RTI\_INT32 key)  
*Attach to an existing shared memory binary semaphore with the provided key and sets up the handle to access it.*
- RTIBool [NETIO\\_SharedMemorySignalingSemaphore\\_create\\_or\\_attach](#) (struct NETIO\_SharedMemorySignalingSemaphoreHandle \*handle, RTI\_INT32 \*status\_out, RTI\_INT32 key)  
*Attempts to create a new binary semaphore with the provided key and if the semaphore already exist, attach to it.*
- RTIBool [NETIO\\_SharedMemorySignalingSemaphore\\_signal](#) (struct NETIO\_SharedMemorySignalingSemaphoreHandle \*handle, RTI\_INT32 \*status\_out)  
*Give the semaphore.*
- RTIBool [NETIO\\_SharedMemorySignalingSemaphore\\_wait](#) (struct NETIO\_SharedMemorySignalingSemaphoreHandle \*handle, RTI\_INT32 \*status\_out)  
*Take the semaphore.*
- RTIBool [NETIO\\_SharedMemorySignalingSemaphore\\_detach](#) (struct NETIO\_SharedMemorySignalingSemaphoreHandle \*handle)  
*Detach from the shared memory binary semaphore.*
- RTIBool [NETIO\\_SharedMemorySignalingSemaphore\\_delete](#) (struct NETIO\_SharedMemorySignalingSemaphoreHandle \*handle)  
*Detach from the shared memory semaphore and deletes it.*

#### 6.13.2.4.1 Detailed Description

#### 6.13.2.4.2 Function Documentation

##### NETIO\_SharedMemorySignalingSemaphore\_create()

```
RTIBool NETIO_SharedMemorySignalingSemaphore_create (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Initializes a shared memory binary semaphore with the provided key and sets up the handle to access it.

##### Precondition

Handle in the DETACHED state. Signaling semaphore in the INITIAL, that is, Nobody has called create() before with that same key

##### Postcondition

handle in the ATTACHED state, Semaphore exists and ready to be used.

## Parameters

|         |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in, out | <i>handle</i>     | The shared memory semaphore handle to initialize.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| out     | <i>status_out</i> | Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION</b> = precondition check failed.</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_ALREADY_CLAIMED</b> = semaphore already exist (no checks is performed whether the creator is still alive or not).</li> <li>• <b>OSAPI_SHARED_MEMORY_CREATED</b> = the semaphore has been successfully created.</li> </ul> |
| in      | <i>key</i>        | Key to identify the shared memory semaphore across processes. Other processes should call <a href="#">NETIO_SharedMemorySignalingSemaphore_attach</a> with the same key to attach to this semaphore.                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

## Returns

RTI\_TRUE on success. RTI\_FALSE on error.

References [RTI\\_INT32](#).

**NETIO\_SharedMemorySignalingSemaphore\_attach()**

```
RTIBool NETIO_SharedMemorySignalingSemaphore_attach (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attach to an existing shared memory binary semaphore with the provided key and sets up the handle to access it.

## Precondition

Handle in the DETACHED state. Signaling semaphore in the CREATED state. Some other process has called [NETIO\\_SharedMemorySignalingSemaphore\\_create](#) using the same key in the shared memory semaphore

## Postcondition

handle in the ATTACHED state, Semaphore exists and is ready to be used.

## Parameters

|     |               |                                                   |
|-----|---------------|---------------------------------------------------|
| out | <i>handle</i> | The shared memory semaphore handle to initialize. |
|-----|---------------|---------------------------------------------------|



## Parameters

|     |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| out | <i>status_out</i> | Optional. If not NULL, will be set to: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION</b> = precondition check failed.</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b> = no such a semaphore.</li> <li>• <b>OSAPI_SHARED_MEMORY_ATTACHED</b> = semaphore attached successfully.</li> </ul> |
| in  | <i>key</i>        | Key to identify the shared memory semaphore across processes.                                                                                                                                                                                                                                                                                                                                                                                                                                              |

## Returns

RTI\_TRUE on success. RTI\_FALSE on error.

## Note

On some platforms this method makes sure that the given key represent a shared memory semaphore object (to avoid situations like the same key is used to create a mutex and then used to attach it as a semaphore). If such a situation is detected, the function will fail and set status\_out to: OSAPI\_SHARED\_MEMORYFAIL\_REASON\_NO\_ENTRY. Not all the platforms support this feature.

## See also

[NETIO\\_SharedMemorySignalingSemaphore\\_detach](#)

References [RTI\\_INT32](#).

**NETIO\_SharedMemorySignalingSemaphore\_create\_or\_attach()**

```
RTIBool NETIO_SharedMemorySignalingSemaphore_create_or_attach (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out,
    RTI_INT32 key)
```

Attempts to create a new binary semaphore with the provided key and if the semaphore already exist, attach to it.

## Precondition

Handle in the DETACHED state. Signaling semaphore in the INITIAL, that is, Nobody has called create() before with that same key.

## Postcondition

handle in the ATTACHED state, Semaphore is in the CREATED state and ready to be used.

## Parameters

|         |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in, out | <i>handle</i>     | The shared memory semaphore handle to initialize.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| out     | <i>status_out</i> | Optional: if not NULL is set to the reason why the call has succeeded or failed. In particular: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_PRECONDITION</b> = precondition check failed.</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b> = this error should never happen and is caused by a failed create() followed by a failed attach().</li> <li>• <b>OSAPI_SHARED_MEMORY_CREATED</b> = the semaphore has been successfully created.</li> </ul> |
| in      | <i>key</i>        | Key to identify the shared memory binary semaphore across processes.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

## Returns

RTI\_TRUE on success. RTI\_FALSE on error.

References [RTI\\_INT32](#).

**NETIO\_SharedMemorySignalingSemaphore\_signal()**

```
RTIBool NETIO_SharedMemorySignalingSemaphore_signal (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out)
```

Give the semaphore.

## Precondition

Handle in the ATTACHED state.

## Postcondition

Semaphore count is set to 1 and any thread blocked on this semaphore is awoken.

## Parameters

|     |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>handle</i>     | Handle to the semaphore. Must match the one returned by <a href="#">NETIO_SharedMemorySignalingSemaphore_create</a>                                                                                                                                                                                                                                                                                                                                                                     |
| out | <i>status_out</i> | Optional: if not NULL, will be set to: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b> = the creator of the mutex has shutdown and removed this mutex. This is expected to happen routinely.</li> <li>• <b>OSAPI_SHARED_MEMORY_SUCCESS</b> = Generic no error condition.</li> </ul> |

**Returns**

RTI\_TRUE if the semaphore is given. RTI\_FALSE if an error occurs. If an error occurs the handle should not be used anymore. In particular detach() should not be called either. An error may occur in certain OSs if the handle that created the semaphore calls [NETIO\\_SharedMemorySignalingSemaphore\\_detach](#).

**See also**

[NETIO\\_SharedMemorySignalingSemaphore\\_wait](#)

References [RTI\\_INT32](#).

**NETIO\_SharedMemorySignalingSemaphore\_wait()**

```
RTIBool NETIO_SharedMemorySignalingSemaphore_wait (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle,
    RTI_INT32 * status_out)
```

Take the semaphore.

**Precondition**

Handle in the ATTACHED state.

**Postcondition**

Semaphore count is decremented.

**Parameters**

|     |                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>handle</i>     | Handle to the semaphore. Must match the one returned by <a href="#">NETIO_SharedMemorySignalingSemaphore_create</a>                                                                                                                                                                                                                                                                                                                                                                     |
| out | <i>status_out</i> | Optional: if not NULL, will be set to: <ul style="list-style-type: none"> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_UNKNOWN</b> = an error occurred in the os-specific shared memory implementation (see error log for details).</li> <li>• <b>OSAPI_SHARED_MEMORY_FAIL_REASON_NO_ENTRY</b> = the creator of the mutex has shutdown and removed this mutex. This is expected to happen routinely.</li> <li>• <b>OSAPI_SHARED_MEMORY_SUCCESS</b> = Generic no error condition.</li> </ul> |

**Returns**

RTI\_TRUE if the semaphore is taken. RTI\_FALSE if an error occurs. If an error occurs the handle should not be used anymore. In particular [NETIO\\_SharedMemorySignalingSemaphore\\_detach](#) should not be called either.

**See also**

[NETIO\\_SharedMemorySignalingSemaphore\\_signal](#)

References [RTI\\_INT32](#).

**NETIO\_SharedMemorySignalingSemaphore\_detach()**

```
RTIBool NETIO_SharedMemorySignalingSemaphore_detach (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle)
```

Detach from the shared memory binary semaphore.

**Precondition**

Handle in the ATTACHED state.

**Postcondition**

handle in the DETACHED state.

**Parameters**

|    |               |                                                                                                                 |
|----|---------------|-----------------------------------------------------------------------------------------------------------------|
| in | <i>handle</i> | Handle to the shared memory semaphore obtained from <a href="#">NETIO_SharedMemorySignalingSemaphore_attach</a> |
|----|---------------|-----------------------------------------------------------------------------------------------------------------|

**See also**

[NETIO\\_SharedMemorySignalingSemaphore\\_attach](#)

**NETIO\_SharedMemorySignalingSemaphore\_delete()**

```
RTIBool NETIO_SharedMemorySignalingSemaphore_delete (
    struct NETIO_SharedMemorySignalingSemaphoreHandle * handle)
```

Detach from the shared memory semaphore and deletes it.

The exact behavior of this may be OS dependent. In some OSs calling this will immediately cause the semaphore to be removed, and other threads waiting on it to be woken up. This is the typical behavior on Unix. In other OSs (windows) this call just requests the semaphore to be removed but delay the removal until all handles are detached.

The only way to ensure the same behavior across multiple OSs is to only call this method on the handle that created the semaphore after all the other handles have been closed. In this scenario no handles will get an error. Alternatively the code could be written to always expect the [NETIO\\_SharedMemorySignalingSemaphore\\_wait](#) or [NETIO\\_SharedMemorySignalingSemaphore\\_signal](#) call to fail and when that occurs close the handle.

**Precondition**

Handle in the ATTACHED state.

**Postcondition**

handle in the DETACHED state. The numHandles to the semaphore is decremented. When the last handle is detached, the semaphore is removed by the OS.

## Parameters

|    |        |                                                                                                                 |
|----|--------|-----------------------------------------------------------------------------------------------------------------|
| in | handle | Handle to the shared memory semaphore obtained from <a href="#">NETIO_SharedMemorySignalingSemaphore_create</a> |
|----|--------|-----------------------------------------------------------------------------------------------------------------|

## See also

[NETIO\\_SharedMemorySignalingSemaphore\\_create](#)

## 6.14 NETIO DGRAM API

NETIO DGRAM Interface.

## Classes

- struct [NETIO\\_DGRAM\\_InterfaceMultiCastGroup](#)  
*<<eXtension>> Definition of a valid multicast group address range.*
- struct [NETIO\\_DGRAM\\_InterfaceTableEntry](#)  
*<<eXtension>> Definition of an interface.*
- struct [NETIO\\_DGRAM\\_InterfaceRouteEntry](#)  
*<<eXtension>> Definition of a route entry.*
- struct [NETIO\\_DGRAM\\_InterfaceTableEntrySeq](#)  
*A sequence of NETIO\_DGRAMInterfaceTableEntry elements.*
- struct [NETIO\\_DGRAM\\_Interfaceel](#)  
*<<eXtension>> Definition of the NETIO\_DGRAM Interface.*

## Macros

- #define [NETIO\\_DGRAM\\_INTERFACE\\_SHARED\\_PORT\\_FLAG](#) (0x1U)  
*Flag that indicates that an interface uses shared ports.*
- #define [NETIO\\_DGRAM\\_INTERFACE\\_MAX\\_INTERFACES](#) (8)  
*The maximum number of network interfaces a NETIO\_DGRAM Transport instance can manage.*
- #define [NETIO\\_DGRAM\\_INTERFACE\\_MAX\\_ADDRESSES](#) (8)  
*The maximum number of addresses which can be returned in an address reservation request.*
- #define [NETIO\\_DGRAM\\_InterfaceMultiCastGroup\\_INITIALIZER](#)  
*Constant to initialize a NETIO\_DGRAM\_InterfaceMultiCastGroup attribute as empty.*
- #define [NETIO\\_DGRAM\\_InterfaceMultiCastGroup\\_Invalid](#)  
*Constant to initialize a NETIO\_DGRAM\_InterfaceMultiCastGroup attribute as invalid.*
- #define [NETIO\\_DGRAM\\_InterfaceMultiCastGroup\\_UDPv4](#)  
*Constant to initialize a NETIO\_DGRAM\_InterfaceMultiCastGroup attribute as a UDPv4.*
- #define [NETIO\\_DGRAM\\_InterfaceMultiCastGroup\\_UDPv6](#)  
*Constant to initialize a NETIO\_DGRAM\_InterfaceMultiCastGroup attribute as a UDPv6 multicast group.*
- #define [NETIO\\_DGRAM\\_InterfaceTableEntry\\_INITIALIZER](#)  
*Constant to initialize an NETIO\_DGRAM\_InterfaceTableEntry attribute as an empty entry.*
- #define [NETIO\\_DGRAM\\_InterfaceTableEntrySeq\\_INITIALIZER](#) REDA\_DEFINE\_SEQUENCE\_INITIALIZER(struct [NETIO\\_DGRAM\\_InterfaceTableEntry](#))  
*Constant to initialize a NETIO\_DGRAM\_InterfaceTableEntry as empty.*

## Typedefs

- typedef `NETIO_Interface_T` `*(``NETIO_DGRAM_Interface_createFunc``)` (`NETIO_Interface_T` `*upstream`, void `*property`)  
*The NETIO\_DGRAM create interface function required by NETIO\_DGRAM\_InterfaceI.*
- typedef `RTI_BOOL`(`* NETIO_DGRAMUserInterface_get_interface_listFunc`) (`NETIO_Interface_T` `*user_intf`, struct `NETIO_DGRAM_InterfaceTableEntrySeq` `*if_table`)  
*NETIO\_DGRAM get interface list function required by NETIO\_DGRAM\_InterfaceI.*
- typedef `RTI_BOOL`(`* NETIO_DGRAMUserInterface_resolve_addressFunc`) (`NETIO_Interface_T` `*netio_intf`, const struct `NETIO_DGRAM_InterfaceTableEntry` `*if_entry`, const char `*address_string`, struct `NETIO_Address` `*address_value`, `RTI_BOOL` `*is_invalid`)  
*NETIO\_DGRAM address resolve function.*
- typedef `RTI_BOOL`(`* NETIO_DGRAMUserInterface_bind_addressFunc`) (`NETIO_Interface_T` `*user_intf`, struct `NETIO_Address` `*source`)  
*NETIO\_DGRAM bind address function. by NETIO\_DGRAM\_InterfaceI.*
- typedef void(`* NETIO_DGRAMUserInterface_deleteFunc`) (`NETIO_Interface_T` `*user_intf`)  
*The NETIO\_DGRAM delete interface function. NOT in CERT.*
- typedef struct `NETIO_DGRAM_InterfaceI` `NETIO_DGRAM_InterfaceI`  
*<<eXtension>> Definition of the NETIO\_DGRAM Interface.*

## Functions

- `RTI_BOOL` `NETIO_DGRAM_InterfaceFactory_register` (`RT_Registry_T` `*registry`, const char `*name`, `NETIO_DGRAM_InterfaceI` `*user_intf`, void `*user_property`)  
*Register the DGRAM interface.*

## Variables

- const struct `NETIO_DGRAM_InterfaceRouteEntry` `NETIO_ADDRESS_UDPv4_ANY_UNICAST_ROUTE`  
*Constant to initialize a UDPv4 unicast route.*
- const struct `NETIO_DGRAM_InterfaceRouteEntry` `NETIO_ADDRESS_UDPv4_ANY_MULTICAST_ROUTE`  
*Constant to initialize UDPv4 multicast route.*
- const struct `NETIO_DGRAM_InterfaceRouteEntry` `NETIO_ADDRESS_UDPv6_ANY_UNICAST_ROUTE`  
*Constant to initialize a UDPv6 unicast route.*
- const struct `NETIO_DGRAM_InterfaceRouteEntry` `NETIO_ADDRESS_UDPv6_ANY_MULTICAST_ROUTE`  
*Constant to initialize UDPv6 multicast route.*

### 6.14.1 Detailed Description

NETIO DGRAM Interface.

<<eXtension>> The NETIO DGRAM implements the RTI Connex DDS Micro NETIO interface and exposes a subset of the interface enabling users to write an integration with concrete network interface.

## 6.14.2 Macro Definition Documentation

### NETIO\_DGRAM\_INTERFACE\_SHARED\_PORT\_FLAG

```
#define NETIO_DGRAM_INTERFACE_SHARED_PORT_FLAG (0x1U)
```

Flag that indicates that an interface uses shared ports.

RTI Connex DDS Micro uses Locators to specify transport addresses to send and receive data. A Locator consists of a kind, port, and a transport address. The kind indicates the type of transport, such as UDPv4, the port is used to reach a DDS DomainParticipant, and the address is used to reach the destination transport. RTI Connex DDS Micro can work with two different types of transports, one that uses shared ports and one that does not.

When a transport uses shared ports, it does not matter which transport address a message was received on; only the port matters. For example, if a computer has two network interfaces A and B and is listening for messages on port P, it does not matter if the message is received on A or B. That is, as long as the message is received on any network interface capable of receiving on port P, the message is accepted.

When a transport does not use shared ports it means it does matter which transport address a message was received on. For example, if a computer has two network interfaces A and B and is listening for messages on port P, but has only specified that the A should receive on port P, then messages received on interface B and port P are ignored.

RTI Connex DDS Micro support this flag on per RTI Connex DDS Micro transport basis.

It is important to note that when a message is accepted, it is routed to all relevant DDS datareaders and datawriters. Thus, this feature cannot be used to control that some DDS topics should only be accepted when received on a specific transport interface. However, this feature could be useful to allow different DDS DomainParticipants to use the same port, but with different network interfaces.

### NETIO\_DGRAM\_INTERFACE\_MAX\_INTERFACES

```
#define NETIO_DGRAM_INTERFACE_MAX_INTERFACES (8)
```

The maximum number of network interfaces a NETIO\_DGRAM Transport instance can manage.

Each NETIO\_DGRAM Transport instance is associated with network interface as returned by the [NETIO\\_DGRAM\\_InterfaceTableEntrySeq](#). The maximum number of interfaces that can be managed by NETIO\_DGRAM is NETIO\_DGRAMINTERFACE\_MAX↵\_INTERFACES.

### NETIO\_DGRAM\_INTERFACE\_MAX\_ADDRESSES

```
#define NETIO_DGRAM_INTERFACE_MAX_ADDRESSES (8)
```

The maximum number of addresses which can be returned in an address reservation request.

Each NETIO\_DGRAM Transport instance receives address reservation requests to reserve addresses to receive discovery and user data based on. The maximum number of addresses that can be reserved in a single request is NETIO\_DGRAM\_INTERFACE\_MAX\_ADDRESSES.

**NETIO\_DGRAM\_InterfaceMultiCastGroup\_INITIALIZER**

```
#define NETIO_DGRAM_InterfaceMultiCastGroup_INITIALIZER
```

**Value:**

```
{\
    NETIO_Address_INITIALIZER,\
    NETIO_Address_INITIALIZER,\
    NETIO_Netmask_INITIALIZER\
}
```

Constant to initialize a [NETIO\\_DGRAM\\_InterfaceMultiCastGroup](#) attribute as empty.

**NETIO\_DGRAM\_InterfaceMultiCastGroup\_Invalid**

```
#define NETIO_DGRAM_InterfaceMultiCastGroup_Invalid
```

**Value:**

```
{\
    {-1,0,{0,0,0,0}}, {0}},\
    {-1,0,{0,0,0,0}}, {0}},\
    {0,{0,0,0,0}} \
}
```

Constant to initialize a [NETIO\\_DGRAM\\_InterfaceMultiCastGroup](#) attribute as invalid.

**NETIO\_DGRAM\_InterfaceMultiCastGroup\_UDPv4**

```
#define NETIO_DGRAM_InterfaceMultiCastGroup_UDPv4
```

**Value:**

```
{\
    {NETIO_ADDRESS_KIND_UDPv4,0,{NETIO_htonl(0xe0000000),0,0,0}}, {0}},\
    {NETIO_ADDRESS_KIND_UDPv4,0,{NETIO_htonl(0xffffffff),0,0,0}}, {0}},\
    {4,{0,0,0,0}} \
}
```

Constant to initialize a [NETIO\\_DGRAM\\_InterfaceMultiCastGroup](#) attribute as a UDPv4.

**NETIO\_DGRAM\_InterfaceMultiCastGroup\_UDPv6**

```
#define NETIO_DGRAM_InterfaceMultiCastGroup_UDPv6
```

**Value:**

```
{\
    {NETIO_ADDRESS_KIND_UDPv6,0,{NETIO_htonl(0xff000000),0,0,0}}, {0}},\
    {NETIO_ADDRESS_KIND_UDPv6,0,{0xffffffff,0xffffffff,0xffffffff,0xffffffff}}, {0}},\
    {8,{0,0,0,0}} \
}
```

Constant to initialize a [NETIO\\_DGRAM\\_InterfaceMultiCastGroup](#) attribute as a UDPv6 multicast group.



### NETIO\_DGRAM\_InterfaceTableEntry\_INITIALIZER

```
#define NETIO_DGRAM_InterfaceTableEntry_INITIALIZER
```

#### Value:

```
{ \
    OU, \
    OU, \
    OU, \
    NETIO_Address_INITIALIZER, \
    NETIO_DGRAM_InterfaceMultiCastGroup_INITIALIZER, \
    NULL \
}
```

Constant to initialize an [NETIO\\_DGRAM\\_InterfaceTableEntry](#) attribute as an empty entry.

### NETIO\_DGRAM\_InterfaceTableEntrySeq\_INITIALIZER

```
#define NETIO_DGRAM_InterfaceTableEntrySeq_INITIALIZER REDA_DEFINE_SEQUENCE_INITIALIZER(struct \
NETIO_DGRAM_InterfaceTableEntry)
```

Constant to initialize a [NETIO\\_DGRAM\\_InterfaceTableEntry](#) as empty.

## 6.14.3 Typedef Documentation

### NETIO\_DGRAM\_Interface\_createFunc

```
typedef NETIO_Interface_T *(* NETIO_DGRAM_Interface_createFunc) (NETIO_Interface_T *upstream, void \
*property)
```

The NETIO\_DGRAM create interface function required by [NETIO\\_DGRAM\\_Interface](#).

### NETIO\_DGRAMUserInterface\_get\_interface\_listFunc

```
typedef RTI_BOOL(* NETIO_DGRAMUserInterface_get_interface_listFunc) (NETIO_Interface_T *user_intf, \
struct NETIO_DGRAM_InterfaceTableEntrySeq *if_table)
```

NETIO\_DGRAM get interface list function required by [NETIO\\_DGRAM\\_Interface](#).

### NETIO\_DGRAMUserInterface\_resolve\_addressFunc

```
typedef RTI_BOOL(* NETIO_DGRAMUserInterface_resolve_addressFunc) (NETIO_Interface_T *netio_↵ \
intf, const struct NETIO_DGRAM_InterfaceTableEntry *if_entry, const char *address_string, struct \
NETIO_Address *address_value, RTI_BOOL *is_invalid)
```

NETIO\_DGRAM address resolve function.

**NETIO\_DGRAMUserInterface\_bind\_addressFunc**

```
typedef RTI_BOOL(* NETIO_DGRAMUserInterface_bind_addressFunc) (NETIO_Interface_T *user_intf, struct
NETIO_Address *source)
```

NETIO\_DGRAM bind address function. by [NETIO\\_DGRAM\\_InterfaceI](#).

**NETIO\_DGRAMUserInterface\_deleteFunc**

```
typedef void(* NETIO_DGRAMUserInterface_deleteFunc) (NETIO_Interface_T *user_intf)
```

The NETIO\_DGRAM delete interface function. NOT in CERT.

**NETIO\_DGRAM\_InterfaceI**

```
typedef struct NETIO_DGRAM_InterfaceI NETIO_DGRAM_InterfaceI
```

<<*eXtension*>> Definition of the NETIO\_DGRAM Interface.

**6.14.4 Function Documentation****NETIO\_DGRAM\_InterfaceFactory\_register()**

```
RTI_BOOL NETIO_DGRAM_InterfaceFactory_register (
    RT_Registry_T * registry,
    const char * name,
    NETIO_DGRAM_InterfaceI * user_intf,
    void * user_property)
```

Register the DGRAM interface.

Registers the DGRAM interface component factory, sets the interface name, sets the instance's properties, and the user interface operations.

The interface operations are the supporting network layer functions that the DGRAM uses for it's routing, binding, and communication with peer NETIO.

**Parameters**

|    |                      |                             |
|----|----------------------|-----------------------------|
| in | <i>registry</i>      | Registry instance.          |
| in | <i>name</i>          | Interface name to register. |
| in | <i>user_intf</i>     | User interface.             |
| in | <i>user_property</i> | User defined property.      |

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure.

**MT Safety:**

This operation is not thread safe.

### 6.14.5 Variable Documentation

#### NETIO\_ADDRESS\_UDPv4\_ANY\_UNICAST\_ROUTE

```
const struct NETIO_DGRAM_InterfaceRouteEntry NETIO_ADDRESS_UDPv4_ANY_UNICAST_ROUTE [extern]
```

Constant to initialize a UDPv4 unicast route.

RTI Connex DDS Micro uses an internal route table to determine which RTI Connex DDS Micro transport to use to send outgoing messages on (refer to `NETIO_TransportI.get_route_table` for details). The `NETIO_ADDRESS_UDPv4_ANY_UNICAST_ROUTE` can be used to configure a transport as being able to send messages to any UDPv4 address.

#### NETIO\_ADDRESS\_UDPv4\_ANY\_MULTICAST\_ROUTE

```
const struct NETIO_DGRAM_InterfaceRouteEntry NETIO_ADDRESS_UDPv4_ANY_MULTICAST_ROUTE [extern]
```

Constant to initialize UDPv4 multicast route.

RTI Connex DDS Micro uses an internal route table to determine which RTI Connex DDS Micro transport to use to send outgoing messages on (refer to `NETIO_TransportI.get_route_table` for details). The `NETIO_ADDRESS_UDPv4_ANY_MULTICAST_ROUTE` can be used to configure a transport as being able to send messages to any UDPv4 multicast address.

#### NETIO\_ADDRESS\_UDPv6\_ANY\_UNICAST\_ROUTE

```
const struct NETIO_DGRAM_InterfaceRouteEntry NETIO_ADDRESS_UDPv6_ANY_UNICAST_ROUTE [extern]
```

Constant to initialize a UDPv6 unicast route.

RTI Connex DDS Micro uses an internal route table to determine which RTI Connex DDS Micro transport to use to send outgoing messages on (refer to `NETIO_TransportI.get_route_table` for details). The `NETIO_ADDRESS_UDPv6_ANY_UNICAST_ROUTE` can be used to configure a transport as being able to send messages to any UDPv6 address.

#### NETIO\_ADDRESS\_UDPv6\_ANY\_MULTICAST\_ROUTE

```
const struct NETIO_DGRAM_InterfaceRouteEntry NETIO_ADDRESS_UDPv6_ANY_MULTICAST_ROUTE [extern]
```

Constant to initialize UDPv6 multicast route.

RTI Connex DDS Micro uses an internal route table to determine which RTI Connex DDS Micro transport to use to send outgoing messages on (refer to `NETIO_TransportI.get_route_table` for details). The `NETIO_ADDRESS_UDPv6_ANY_MULTICAST_ROUTE` can be used to configure a transport as being able to send messages to any UDPv6 multicast address.

## 6.15 Zero Copy v2 API

NETIO Zero Copy v2 interface.

### Topics

- [Notification Interface](#)  
*ZCOPY Notification Interface.*

### Classes

- struct [ZCOPY\\_NotifMechanismProperty](#)  
*Notification mechanism property for the default implementation.*
- struct [ZCOPY\\_NotifInterfaceFactoryProperty](#)  
*Notification interface property.*
- struct [ZCOPY\\_NotifUserInterface](#)  
*Notification mechanism user interface.*

### Macros

- #define [ZCOPY\\_NotifInterfaceFactoryProperty\\_INITIALIZER](#)  
*Constant to initialize a [ZCOPY\\_NotifInterfaceFactoryProperty](#).*

### Typedefs

- typedef [NETIO\\_Interface\\_T](#) \*(\* [ZCOPY\\_NotifUserInterface\\_createFunc](#)) ([NETIO\\_Interface\\_T](#) \*upstream, void \*user\_property)  
*Create an instance of a user interface with upstream as the owner.*
- typedef void(\* [ZCOPY\\_NotifUserInterface\\_deleteFunc](#)) ([NETIO\\_Interface\\_T](#) \*user\_intf)  
*Delete an instance of a user interface.*
- typedef [RTI\\_BOOL](#)(\* [ZCOPY\\_NotifUserInterface\\_reserve\\_addressFunc](#)) ([NETIO\\_Interface\\_T](#) \*user\_intf, struct [NETIO\\_Address](#) \*src\_addr, void \*\*port\_entry\_out)  
*Reserve an address on a user interface.*
- typedef [RTI\\_BOOL](#)(\* [ZCOPY\\_NotifUserInterface\\_release\\_addressFunc](#)) ([NETIO\\_Interface\\_T](#) \*user\_intf, struct [NETIO\\_Address](#) \*src\_addr, void \*port\_entry)  
*Release an address on a user interface.*
- typedef [RTI\\_BOOL](#)(\* [ZCOPY\\_NotifUserInterface\\_add\\_routeFunc](#)) ([NETIO\\_Interface\\_T](#) \*user\_intf, struct [NETIO\\_Address](#) \*source, struct [NETIO\\_Address](#) \*destination, void \*\*route\_entry\_out)  
*Add a route on a user interface.*
- typedef [RTI\\_BOOL](#)(\* [ZCOPY\\_NotifUserInterface\\_delete\\_routeFunc](#)) ([NETIO\\_Interface\\_T](#) \*user\_intf, struct [NETIO\\_Address](#) \*source, struct [NETIO\\_Address](#) \*destination, void \*route\_entry)  
*Delete a route on a user interface.*
- typedef [RTI\\_BOOL](#)(\* [ZCOPY\\_NotifUserInterface\\_sendFunc](#)) ([NETIO\\_Interface\\_T](#) \*self, struct [NETIO\\_Address](#) \*source, struct [NETIO\\_Address](#) \*destination, void \*route\_entry)  
*Send a notification on a user interface.*

- typedef [RTI\\_BOOL](#)(\* [ZCOPY\\_NotifUserInterface\\_bindFunc](#)) ([NETIO\\_Interface\\_T](#) \*user\_intf, struct [NETIO\\_Address](#) \*src\_addr, struct [NETIO\\_Address](#) \*dst\_addr, void \*port\_entry, void \*\*bind\_entry\_out)  
*Bind an address on a user interface.*
- typedef [RTI\\_BOOL](#)(\* [ZCOPY\\_NotifUserInterface\\_unbindFunc](#)) ([NETIO\\_Interface\\_T](#) \*user\_intf, struct [NETIO\\_Address](#) \*src\_addr, struct [NETIO\\_Address](#) \*dst\_addr, void \*port\_entry, void \*bind\_entry)  
*Unbind an address on a user interface.*
- typedef [RTI\\_BOOL](#)(\* [ZCOPY\\_NotifUserInterface\\_notify\\_portFunc](#)) ([NETIO\\_Interface\\_T](#) \*user\_intf, void \*port\_entry)  
*Notify a specific receive port.*
- typedef struct [ZCOPY\\_NotifUserInterfaceI](#) [ZCOPY\\_NotifUserInterfaceI](#)  
*Notification mechanism user interface.*

### 6.15.1 Detailed Description

NETIO Zero Copy v2 interface.

<<[eXtension](#)>> The NETIO Zero Copy v2 module implements the RTI Connex DDS Micro Zero Copy v2 interface.

### 6.15.2 Macro Definition Documentation

#### [ZCOPY\\_NotifInterfaceFactoryProperty\\_INITIALIZER](#)

```
#define ZCOPY_NotifInterfaceFactoryProperty_INITIALIZER
```

Value:

```
{
    NETIO\_InterfaceFactoryProperty\_INITIALIZER, /* _parent */
    1, /* max_samples_per_notif */
    NULL, /* user_intf */
    NULL, /* user_property */
}
```

Constant to initialize a [ZCOPY\\_NotifInterfaceFactoryProperty](#).

### 6.15.3 Typedef Documentation

#### [ZCOPY\\_NotifUserInterface\\_createFunc](#)

```
typedef NETIO\_Interface\_T *(* ZCOPY\_NotifUserInterface\_createFunc) (NETIO\_Interface\_T *upstream,
void *user_property)
```

Create an instance of a user interface with upstream as the owner.

#### [ZCOPY\\_NotifUserInterface\\_deleteFunc](#)

```
typedef void(* ZCOPY\_NotifUserInterface\_deleteFunc) (NETIO\_Interface\_T *user_intf)
```

Delete an instance of a user interface.

**ZCOPY\_NotifUserInterface\_reserve\_addressFunc**

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_reserve_addressFunc) (NETIO_Interface_T *user_intf,  
struct NETIO_Address *src_addr, void **port_entry_out)
```

Reserve an address on a user interface.

Instruct the notification mechanism interface specified by user\_intf to setup resources for listening to messages on the address src\_addr and return a port\_entry\_out. The port\_entry\_out will be provided to a bind call.

**ZCOPY\_NotifUserInterface\_release\_addressFunc**

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_release_addressFunc) (NETIO_Interface_T *user_intf,  
struct NETIO_Address *src_addr, void *port_entry)
```

Release an address on a user interface.

Instruct the notification mechanism interface specified by user\_intf to release resources for listening to messages on the address src\_addr.

**ZCOPY\_NotifUserInterface\_add\_routeFunc**

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_add_routeFunc) (NETIO_Interface_T *user_intf, struct  
NETIO_Address *source, struct NETIO_Address *destination, void **route_entry_out)
```

Add a route on a user interface.

Instruct the notification mechanism interface specified by user\_intf to add a route from the source to the destination. The route\_entry\_out will be provided to the user later when calling send.

**ZCOPY\_NotifUserInterface\_delete\_routeFunc**

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_delete_routeFunc) (NETIO_Interface_T *user_intf, struct  
NETIO_Address *source, struct NETIO_Address *destination, void *route_entry)
```

Delete a route on a user interface.

Not available in CERT.

Instruct the notification mechanism interface specified by user\_intf to remove a route from the source to the destination.

**ZCOPY\_NotifUserInterface\_sendFunc**

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_sendFunc) (NETIO_Interface_T *self, struct NETIO_Address  
*source, struct NETIO_Address *destination, void *route_entry)
```

Send a notification on a user interface.

Instruct the notification mechanism interface specified by user\_intf to send a notification to the destination using the route\_entry.

### ZCOPY\_NotifUserInterface\_bindFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_bindFunc) (NETIO_Interface_T *user_intf, struct NETIO_Address *src_addr, struct NETIO_Address *dst_addr, void *port_entry, void **bind_entry_out)
```

Bind an address on a user interface.

Instruct the notification mechanism interface specified by user\_intf to start listening for messages on the address specified by src\_addr on the given port\_entry.

### ZCOPY\_NotifUserInterface\_unbindFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_unbindFunc) (NETIO_Interface_T *user_intf, struct NETIO_Address *src_addr, struct NETIO_Address *dst_addr, void *port_entry, void *bind_entry)
```

Unbind an address on a user interface.

Instruct the notification mechanism interface specified by user\_intf to stop listening for messages on the address specified by src\_addr.

### ZCOPY\_NotifUserInterface\_notify\_portFunc

```
typedef RTI_BOOL(* ZCOPY_NotifUserInterface_notify_portFunc) (NETIO_Interface_T *user_intf, void *port_entry)
```

Notify a specific receive port.

Instruct the notification mechanism interface specified by user\_intf to notify the receive port specified by port\_entry.

### ZCOPY\_NotifUserInterfaceI

```
typedef struct ZCOPY_NotifUserInterfaceI ZCOPY_NotifUserInterfaceI
```

Notification mechanism user interface.

## 6.15.4 Notification Interface

ZCOPY Notification Interface.

### Variables

- NETIO\_ZCOPYDIExport const char \*const NETIO\_DEFAULT\_NOTIF\_NAME  
*Default name for notification interface.*

#### 6.15.4.1 Detailed Description

ZCOPY Notification Interface.

The Notification Interface is implemented as a NETIO interface and NETIO interface factory.

#### 6.15.4.2 Variable Documentation

##### NETIO\_DEFAULT\_NOTIF\_NAME

```
NETIO_ZCOPYDllExport const char* const NETIO_DEFAULT_NOTIF_NAME [extern]
```

Default name for notification interface.

## 6.16 OS API

### Topics

- [OSAPI Heap](#)
- [OSAPI Mutex](#)  
*Mutex API.*
- [OSAPI Process](#)  
*Abstract Process API.*
- [OSAPI Semaphore](#)  
*Abstract Semaphore API.*
- [OSAPI Memory API](#)
- [OSAPI String API](#)
- [OSAPI System API](#)  
*Abstract System API.*
- [OSAPI Thread API](#)  
*OSAPI Thread API.*
- [OSAPI Time](#)
- [OSAPI Timer](#)
- [OSAPI Types](#)  
*OSAPI Types.*
- [OSAPI\\_SharedMemoryMonitor](#)  
*A shared memory monitor to provide mutual exclusion between processes and inter-process signaling.*
- [OSAPI\\_SharedMemorySegment](#)  
*Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.*



### 6.16.1 Detailed Description

### 6.16.2 OSAPI Heap

#### Macros

- `#define OSAPI_ALIGNMENT_DEFAULT (-1)`  
*Certain methods allow a default alignment: this should be an alignment that follows the "malloc" alignment of the architecture (aligned sufficiently to store any C-structure efficiently).*
- `#define OSAPI_Heap_is_address_aligned(location, alignment)`  
*Check if a particular address is aligned to an alignment.*
- `#define OSAPI_Heap_align_size_up(size, alignment)`  
*Aligns a particular address to an alignment alignment.*

#### Functions

- `void * OSAPI_Heap_allocate (RTI_SIZE_T count, RTI_SIZE_T size)`  
*Allocates zero-initialized memory from the heap.*
- `OSPSLDIExport void * OSAPI_Heap_realloc (void *ptr, RTI_SIZE_T size)`  
*Reallocate memory from the heap.*
- `void OSAPI_Heap_free (void *ptr)`  
*Frees memory allocated from the heap.*
- `OSPSLDIExport void OSAPI_Heap_allocate_buffer (char **buffer, RTI_SIZE_T size, OSAPI_Alignment_T alignment)`  
*Allocate a block of memory of the provided size from the heap.*
- `OSPSLDIExport void OSAPI_Heap_free_buffer (void *buffer)`  
*A static method to return a block (that was previously allocated with `OSAPI_Heap_allocate_buffer`) to the heap.*

#### 6.16.2.1 Detailed Description

#### 6.16.2.2 Macro Definition Documentation

##### OSAPI\_ALIGNMENT\_DEFAULT

```
#define OSAPI_ALIGNMENT_DEFAULT (-1)
```

Certain methods allow a default alignment: this should be an alignment that follows the "malloc" alignment of the architecture (aligned sufficiently to store any C-structure efficiently).

##### OSAPI\_Heap\_is\_address\_aligned

```
#define OSAPI_Heap_is_address_aligned(  
    location,  
    alignment)
```

##### Value:

```
((RTI_UINT32)((char*)(location) - (char*)OSAPI_CC_NullPtr) % (alignment)) == 0)
```

Check if a particular address is aligned to an alignment.

**OSAPI\_Heap\_align\_size\_up**

```
#define OSAPI_Heap_align_size_up(
    size,
    alignment)
```

**Value:**

```
((size) + ((alignment) - 1U)) & (~((alignment) - 1U))
```

Aligns a particular address to an alignment alignment.

**6.16.2.3 Function Documentation****OSAPI\_Heap\_allocate()**

```
void * OSAPI_Heap_allocate (
    RTI_SIZE_T count,
    RTI_SIZE_T size)
```

Allocates zero-initialized memory from the heap.

The function allocates `count` elements of size `size` and sets the memory region to 0.

NOTE: This operation assumes the specified size to be > 0; this operation is never used directly but it is only accessed by using one of the other allocate operations of OSAPI\_Heap (e.g. `allocate_struct`, `allocate_array`, `allocate_string...`); these operations are implemented as macros and accept a "type" argument which is always converted to a size > 0 using the `sizeof` operator.

**Parameters**

|    |              |                                                                       |
|----|--------------|-----------------------------------------------------------------------|
| in | <i>count</i> | The number of elements to allocate.                                   |
| in | <i>size</i>  | The size of the element; size is assumed to be always greater than 0. |

**Returns**

. A pointer to a contiguous memory area guaranteed to be large enough store `count` elements of size `size`.

See Also [OSAPI\\_Heap\\_free](#)

**OSAPI\_Heap\_realloc()**

```
OSPSLDllExport void * OSAPI_Heap_realloc (
    void * ptr,
    RTI_SIZE_T size)
```

Reallocate memory from the heap.

**Parameters**

|    |             |                            |
|----|-------------|----------------------------|
| in | <i>ptr</i>  | Currently allocated buffer |
| in | <i>size</i> | The new desired size       |

**Returns**

. A pointer to a contiguous memory area guaranteed to be large enough store size bytes, NULL if the reallocation failed.

See Also [OSAPI\\_Heap\\_free](#)

**OSAPI\_Heap\_free()**

```
void OSAPI_Heap_free (
    void * ptr)
```

Frees memory allocated from the heap.

The function frees memory previously allocated with [OSAPI\\_Heap\\_allocate](#) back to the heap.

**Parameters**

|    |            |                                                                               |
|----|------------|-------------------------------------------------------------------------------|
| in | <i>ptr</i> | Pointer to region previous allocated with <a href="#">OSAPI_Heap_allocate</a> |
|----|------------|-------------------------------------------------------------------------------|

See Also [OSAPI\\_Heap\\_allocate](#)

References [RTI\\_UINT32](#).

**OSAPI\_Heap\_allocate\_buffer()**

```
OSPSLDllExport void OSAPI_Heap_allocate_buffer (
    char ** buffer,
    RTI_SIZE_T size,
    OSAPI_Alignment_T alignment)
```

Allocate a block of memory of the provided size from the heap.

**Parameters**

|     |                  |                                               |
|-----|------------------|-----------------------------------------------|
| out | <i>buffer</i>    | Cannot be NULL.                               |
| in  | <i>size</i>      | Must be > 0.                                  |
| in  | <i>alignment</i> | Power of 2 and >0 or OSAPI_ALIGNMENT_DEFAULT. |

If the alignment is OSAPI\_ALIGNMENT\_DEFAULT, the returned pointer must have "default alignment" (meaning that any C-structure can start at the beginning of the buffer).

**OSAPI\_Heap\_free\_buffer()**

```
OSPSLDllExport void OSAPI_Heap_free_buffer (
    void * buffer)
```

A static method to return a block (that was previously allocated with [OSAPI\\_Heap\\_allocate\\_buffer](#)) to the heap.

## Parameters

|                 |                     |                 |
|-----------------|---------------------|-----------------|
| <code>in</code> | <code>buffer</code> | If NULL, no op. |
|-----------------|---------------------|-----------------|

## 6.16.3 OSAPI Mutex

Mutex API.

## Typedefs

- typedef struct OSAPI\_Mutex [OSAPI\\_Mutex\\_T](#)

## Functions

- OSPSLDllExport [RTI\\_BOOL OSAPI\\_Mutex\\_delete](#) ([OSAPI\\_Mutex\\_T](#) \*mutex)  
*Delete a mutex.*
- OSPSLDllExport [OSAPI\\_Mutex\\_T](#) \* [OSAPI\\_Mutex\\_new](#) (void)  
*Create a mutex.*
- OSPSLDllExport [RTI\\_BOOL OSAPI\\_Mutex\\_take](#) ([OSAPI\\_Mutex\\_T](#) \*self)  
*Take a mutex.*
- OSPSLDllExport [RTI\\_BOOL OSAPI\\_Mutex\\_give](#) ([OSAPI\\_Mutex\\_T](#) \*self)  
*Give a mutex.*

## 6.16.3.1 Detailed Description

Mutex API.

## 6.16.3.2 Typedef Documentation

**OSAPI\_Mutex\_T**

```
typedef struct OSAPI_Mutex OSAPI\_Mutex\_T
```

Abstract Mutex type

## 6.16.3.3 Function Documentation

**OSAPI\_Mutex\_delete()**

```
OSPSLDllExport RTI\_BOOL OSAPI\_Mutex\_delete (  
    OSAPI\_Mutex\_T * mutex)
```

Delete a mutex.

**Parameters**

|           |              |                                                               |
|-----------|--------------|---------------------------------------------------------------|
| <i>in</i> | <i>mutex</i> | Delete a mutex created with <a href="#">OSAPI_Mutex_new</a> . |
|-----------|--------------|---------------------------------------------------------------|

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure

**OSAPI\_Mutex\_new()**

```
OSPSLDllExport OSAPI_Mutex_T * OSAPI_Mutex_new (  
    void )
```

Create a mutex.

**Returns**

Pointer to a mutex in a not taken condition. NULL on failure.

See Also [OSAPI\\_Mutex\\_delete](#)

**OSAPI\_Mutex\_take()**

```
OSPSLDllExport RTI_BOOL OSAPI_Mutex_take (  
    OSAPI_Mutex_T * self)
```

Take a mutex.

A mutex can only be taken if it is not currently already taken by another thread; it CAN be taken if is it already taken by the same thread. In order to release a mutex, it must be given as many times a it has been taken. Note that `take` will block indefinitely.

Note: the mutex is not required to support priority inversion; there is no protection against deadlocks or starvation.

**Parameters**

|           |             |                                                                        |
|-----------|-------------|------------------------------------------------------------------------|
| <i>in</i> | <i>self</i> | Take a mutex previously created with <a href="#">OSAPI_Mutex_new</a> . |
|-----------|-------------|------------------------------------------------------------------------|

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure

See Also [OSAPI\\_Mutex\\_give](#), [OSAPI\\_Mutex\\_new](#)

**OSAPI\_Mutex\_give()**

```
OSPSLDllExport RTI_BOOL OSAPI_Mutex_give (  
    OSAPI_Mutex_T * self)
```

Give a mutex.

A mutex can only be given if it is owned by the calling thread. Furthermore, it must be given as many times as it has been taken.

**Parameters**

|                 |                   |                                                                        |
|-----------------|-------------------|------------------------------------------------------------------------|
| <code>in</code> | <code>self</code> | Take a mutex previously created with <a href="#">OSAPI_Mutex_new</a> . |
|-----------------|-------------------|------------------------------------------------------------------------|

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure

See Also [OSAPI\\_Mutex\\_take](#), [OSAPI\\_Mutex\\_new](#)

References [RTI\\_UINT32](#).

**6.16.4 OSAPI Process**

Abstract Process API.

**Functions**

- OSPLDIIExport [RTI\\_BOOL OSAPI\\_Process\\_is\\_alive](#) (OSAPI\_ProcessId pid)  
*Returns true if a process is alive.*
- OSPLDIIExport OSAPI\_ProcessId [OSAPI\\_Process\\_getpid](#) (void)  
*Return the process ID, which is unique for the application.*

**6.16.4.1 Detailed Description**

Abstract Process API.

**6.16.4.2 Function Documentation****OSAPI\_Process\_is\_alive()**

```
OSPLDIIExport RTI\_BOOL OSAPI_Process_is_alive (
    OSAPI_ProcessId pid)
```

Returns true if a process is alive.

When passed in PID 0, the function will always return RTI\_TRUE

**Returns**

[RTI\\_BOOL](#)

## OSAPI\_Process\_getpid()

```
OSPSLDllExport OSAPI_ProcessId OSAPI_Process_getpid (
    void )
```

Return the process ID, which is unique for the application.

Return the process ID.

### Returns

Process id

## 6.16.5 OSAPI Semaphore

Abstract Semaphore API.

### Macros

- #define [OSAPI\\_SEMAPHORE\\_TIMEOUT\\_INFINITE](#) -1  
*If [OSAPI\\_Semaphore\\_take](#) is called with [OSAPI\\_SEMAPHORE\\_TIMEOUT\\_INFINITE](#) as timeout, [OSAPI\\_Semaphore\\_take](#) will not return until the semaphore is signaled.*
- #define [OSAPI\\_SEMAPHORE\\_RESULT\\_OK](#) 0  
*If [OSAPI\\_Semaphore\\_take](#) succeeds and the semaphore is signaled within the timeout period, [OSAPI\\_Semaphore\\_take](#) returns TRUE and sets the failure reason to [OSAPI\\_SEMAPHORE\\_RESULT\\_OK](#).*
- #define [OSAPI\\_SEMAPHORE\\_RESULT\\_TIMEOUT](#) 1  
*If [OSAPI\\_Semaphore\\_take](#) was called with a timeout value different from [OSAPI\\_SEMAPHORE\\_TIMEOUT\\_INFINITE](#), and the semaphore was not signaled before the timeout expired, waiting for the semaphore to unblock timed out, [OSAPI\\_Semaphore\\_take](#) returns FALSE and sets the failure reason to [OSAPI\\_SEMAPHORE\\_RESULT\\_TIMEOUT](#).*
- #define [OSAPI\\_SEMAPHORE\\_RESULT\\_ERROR](#) 2  
*If [OSAPI\\_Semaphore\\_take](#) fails for an unknown reason, [OSAPI\\_Semaphore\\_take](#) returns FALSE and sets the failure reason to [OSAPI\\_SEMAPHORE\\_RESULT\\_ERROR](#).*

### Typedefs

- typedef struct OSAPI\_Semaphore [OSAPI\\_Semaphore\\_T](#)  
*Abstract Semaphore type.*

### Functions

- OSPSLDIExport [RTI\\_BOOL OSAPI\\_Semaphore\\_delete](#) ([OSAPI\\_Semaphore\\_T](#) \*self)  
*Delete a semaphore.*

#### 6.16.5.1 Detailed Description

Abstract Semaphore API.

### 6.16.5.2 Macro Definition Documentation

#### OSAPI\_SEMAPHORE\_TIMEOUT\_INFINITE

```
#define OSAPI_SEMAPHORE_TIMEOUT_INFINITE -1
```

If [OSAPI\\_Semaphore\\_take](#) is called with OSAPI\_SEMAPHORE\_TIMEOUT\_INFINITE as timeout, [OSAPI\\_Semaphore\\_take](#) will not return until the semaphore is signaled.

#### OSAPI\_SEMAPHORE\_RESULT\_OK

```
#define OSAPI_SEMAPHORE_RESULT_OK 0
```

If [OSAPI\\_Semaphore\\_take](#) succeeds and the semaphore is signaled within the timeout period, [OSAPI\\_Semaphore\\_take](#) returns TRUE and sets the failure reason to OSAPI\_SEMAPHORE\_RESULT\_OK.

#### OSAPI\_SEMAPHORE\_RESULT\_TIMEOUT

```
#define OSAPI_SEMAPHORE_RESULT_TIMEOUT 1
```

If [OSAPI\\_Semaphore\\_take](#) was called with a timeout value different from OSAPI\_SEMAPHORE\_TIMEOUT\_INFINITE, and the semaphore was not signaled before the timeout expired, waiting for the semaphore to unblock timed out, [OSAPI\\_Semaphore\\_take](#) returns FALSE and sets the failure reason to OSAPI\_SEMAPHORE\_RESULT\_TIMEOUT.

#### OSAPI\_SEMAPHORE\_RESULT\_ERROR

```
#define OSAPI_SEMAPHORE_RESULT_ERROR 2
```

If [OSAPI\\_Semaphore\\_take](#) fails for an unknown reason, [OSAPI\\_Semaphore\\_take](#) returns FALSE and sets the failure reason to OSAPI\_SEMAPHORE\_RESULT\_ERROR.

### 6.16.5.3 Typedef Documentation

#### OSAPI\_Semaphore\_T

```
typedef struct OSAPI_Semaphore OSAPI_Semaphore_T
```

Abstract Semaphore type.

### 6.16.5.4 Function Documentation

#### OSAPI\_Semaphore\_delete()

```
OSPSLDllExport RTI_BOOL OSAPI_Semaphore_delete (
    OSAPI_Semaphore_T * self)
```

Delete a semaphore.



#### Parameters

|                 |                   |                                                              |
|-----------------|-------------------|--------------------------------------------------------------|
| <code>in</code> | <code>self</code> | Semaphore created with <a href="#">OSAPI_Semaphore_new</a> . |
|-----------------|-------------------|--------------------------------------------------------------|

#### Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

See Also [OSAPI\\_Semaphore\\_new](#)

### 6.16.6 OSAPI Memory API

### 6.16.7 OSAPI String API

### 6.16.8 OSAPI System API

Abstract System API.

#### Classes

- struct [OSAPI\\_SystemI](#)  
*System API.*
- struct [OSAPI\\_SystemProperty](#)  
*The System Properties.*

#### Macros

- `#define` [OSAPI\\_SYSTEM\\_MAX\\_HOSTNAME](#) (64)  
*Maximum hostname length.*

#### Typedefs

- typedef [RTI\\_BOOL](#)(\* [OSAPI\\_System\\_start\\_timer\\_T](#)) ([OSAPI\\_Timer\\_T](#) self, [OSAPI\\_TimerTickHandlerFunction](#) tick\_handler)  
*Start a OSAPI timer.*
- typedef [RTI\\_INT32](#)(\* [OSAPI\\_System\\_get\\_timer\\_resolution\\_T](#)) (void)  
*Get the resolution of the clock driving the timer in nano seconds.*
- typedef [RTI\\_BOOL](#)(\* [OSAPI\\_System\\_get\\_time\\_T](#)) ([OSAPI\\_SystemTime](#) \*now)  
*Get the current system time.*
- typedef [RTI\\_BOOL](#)(\* [OSAPI\\_System\\_initialize\\_T](#)) (void)  
*Initialize the system.*
- typedef [RTI\\_BOOL](#)(\* [OSAPI\\_System\\_get\\_hostname\\_T](#)) (char \*const hostname)  
*Get the hostname.*
- typedef [RTI\\_BOOL](#)(\* [OSAPI\\_System\\_generate\\_uuid\\_T](#)) (struct [OSAPI\\_SystemUUID](#) \*uuid\_out)  
*Generate a unique universal identifier (UUID).*
- typedef [RTI\\_BOOL](#)(\* [OSAPI\\_System\\_get\\_ticktime\\_T](#)) ([RTI\\_INT32](#) \*sec, [RTI\\_UINT32](#) \*nanosec)  
*Get current tick time.*
- typedef struct [OSAPI\\_SystemI](#) [OSAPI\\_SystemI](#)  
*System API.*

### 6.16.8.1 Detailed Description

Abstract System API.

The System is defined as the physical or virtual execution environment and implements functions that are not necessarily provided by the operating system. For example, a custom embedded board may include a special real-time clock which is synchronized via GPS, but is not available via a regular OS system call.

The following functions are not thread-safe:

- OSAPI\_System\_set\_interface
- OSAPI\_System\_get\_property
- OSAPI\_System\_set\_property
- OSAPI\_System\_get\_native\_interface
- OSAPI\_System\_clock\_tick
- OSAPI\_System\_clock\_tick\_from\_time

### 6.16.8.2 Macro Definition Documentation

#### OSAPI\_SYSTEM\_MAX\_HOSTNAME

```
#define OSAPI_SYSTEM_MAX_HOSTNAME (64)
```

Maximum hostname length.

### 6.16.8.3 Typedef Documentation

#### OSAPI\_System\_start\_timer\_T

```
typedef RTI_BOOL(* OSAPI_System_start_timer_T) (OSAPI_Timer_T self, OSAPI_TimerTickHandlerFunction tick_handler)
```

Start a OSAPI timer.

#### Parameters

|    |                     |               |
|----|---------------------|---------------|
| in | <i>self</i>         | Timer object. |
| in | <i>tick_handler</i> | Timer handle. |

#### Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

### OSAPI\_System\_get\_timer\_resolution\_T

```
typedef RTI_INT32(* OSAPI_System_get_timer_resolution_T) (void)
```

Get the resolution of the clock driving the timer in nano seconds.

This function must return the frequency of the system timer used to implement [OSAPI\\_SystemI::start\\_timer](#) and [OSAPI\\_SystemI::stop\\_timer](#) API.

#### Returns

timer resolution in nanoseconds.

### OSAPI\_System\_get\_time\_T

```
typedef RTI_BOOL(* OSAPI_System_get_time_T) (OSAPI_SystemTime *now)
```

Get the current system time.

In general, the system time is used by components to correlate both internal and external events, such as data reception and ordering. Thus, it is recommended that this function returns the real time. However, it is not strictly required.

Notes: - It is assumed that the time returned by `get_time` is monotonically increasing. It is up to the implementation of this function to ensure this holds true.

- It is ok to return the same time as the last call.

- The clock used to report real-time can be different than the clock used to support `start_timer` and `stop_timer`.

#### Parameters

|     |     |       |
|-----|-----|-------|
| out | now | Time. |
|-----|-----|-------|

#### Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

### OSAPI\_System\_initialize\_T

```
typedef RTI_BOOL(* OSAPI_System_initialize_T) (void)
```

Initialize the system.

This function initializes the system and calls the port specific initialize method first. The port specific initialization method must return RTI\_TRUE on success and RTI\_FALSE on failure. A system can only be initialized once.

#### Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

### OSAPI\_System\_get\_hostname\_T

```
typedef RTI_BOOL(* OSAPI_System_get_hostname_T) (char *const hostname)
```

Get the hostname.

**Parameters**

|     |                 |                                                                                                                                                                                                                      |
|-----|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| out | <i>hostname</i> | The buffer to store the hostname. Must be at least OSAPI_SYSTEM_MAX_HOSTNAME bytes. If the actual hostname is longer than OSAPI_SYSTEM_MAX_HOSTNAME bytes (including the NUL termination) the hostname is truncated. |
|-----|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure.

**OSAPI\_System\_generate\_uuid\_T**

```
typedef RTI_BOOL(* OSAPI_System_generate_uuid_T) (struct OSAPI_SystemUUID *uuid_out)
```

Generate a unique universal identifier (UUID).

**Parameters**

|     |                 |                          |
|-----|-----------------|--------------------------|
| out | <i>uuid_out</i> | The generated UUID value |
|-----|-----------------|--------------------------|

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure

**OSAPI\_System\_get\_ticktime\_T**

```
typedef RTI_BOOL(* OSAPI_System_get_ticktime_T) (RTI_INT32 *sec, RTI_UINT32 *nanosec)
```

Get current tick time.

The ticktime is a time measurement used by Micro to determine how much time has elapsed in a period. It does not have to be an absolute time, but it *must* be monotonically increasing. For this reason it is important to choose the time source with care. For example, the system time may mbe adjusted backward or forward and this could affect the measure time lapse. The resolution of the tick time is expected to be no less than the system timer, although it is not a requirement.

**Parameters**

|     |                |                                                |
|-----|----------------|------------------------------------------------|
| out | <i>sec</i>     | The current ticktime in seconds                |
| out | <i>nanosec</i> | Additional nanoseconds in the current ticktime |

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure.

## OSAPI\_SystemI

```
typedef struct OSAPI_SystemI OSAPI_SystemI
```

System API.

The methods in this interface are called by the corresponding OSAPI\_System function. For example, OSAPI\_System↔\_get\_time() calls system->get\_time().

### 6.16.9 OSAPI Thread API

OSAPI Thread API.

#### Classes

- struct [OSAPI\\_ThreadProperty](#)  
*Properties for thread creation.*

#### Macros

- #define [OSAPI\\_THREAD\\_PRIORITY\\_LOW](#) -1  
*Low priority.*
- #define [OSAPI\\_THREAD\\_PRIORITY\\_BELOW\\_NORMAL](#) -2  
*Below normal priority.*
- #define [OSAPI\\_THREAD\\_PRIORITY\\_NORMAL](#) -3  
*Normal priority.*
- #define [OSAPI\\_THREAD\\_PRIORITY\\_ABOVE\\_NORMAL](#) -4  
*Above normal priority.*
- #define [OSAPI\\_THREAD\\_PRIORITY\\_HIGH](#) -5  
*High priority.*
- #define [OSAPI\\_THREAD\\_PRIORITY\\_INHERIT](#) -6  
*Use the same priority as the spawning thread. Only applies to POSIX.*
- #define [OSAPI\\_THREAD\\_USE\\_OSDEFAULT\\_STACKSIZE](#) 0  
*Use the default stack size when creating a thread.*
- #define [OSAPI\\_THREAD\\_DEFAULT\\_OPTIONS](#) 0x00  
*Use only the default options the OS gives you.*
- #define [OSAPI\\_THREAD\\_FLOATING\\_POINT](#) 0x01  
*Hint to the OS to create a thread that supports floating point.*
- #define [OSAPI\\_THREAD\\_STDIO](#) 0x02  
*Hint to the OS to create a thread that supports standard I/O.*
- #define [OSAPI\\_THREAD\\_REALTIME\\_PRIORITY](#) 0x08  
*Hint to the OS to create a thread that runs in real-time priority mode.*
- #define [OSAPI\\_THREAD\\_PROPERTY\\_DEFAULT](#)  
*Default thread properties.*
- #define [OSAPI\\_ThreadProperty\\_INITIALIZER](#) [OSAPI\\_THREAD\\_PROPERTY\\_DEFAULT](#)  
*Initializer for [OSAPI\\_ThreadProperty](#).*

## Typedefs

- typedef [RTI\\_UINT32 OSAPI\\_ThreadOptions](#)  
*Thread options.*

### 6.16.9.1 Detailed Description

OSAPI Thread API.

### 6.16.9.2 Macro Definition Documentation

#### OSAPI\_THREAD\_PRIORITY\_LOW

```
#define OSAPI_THREAD_PRIORITY_LOW -1
```

Low priority.

#### OSAPI\_THREAD\_PRIORITY\_BELOW\_NORMAL

```
#define OSAPI_THREAD_PRIORITY_BELOW_NORMAL -2
```

Below normal priority.

#### OSAPI\_THREAD\_PRIORITY\_NORMAL

```
#define OSAPI_THREAD_PRIORITY_NORMAL -3
```

Normal priority.

#### OSAPI\_THREAD\_PRIORITY\_ABOVE\_NORMAL

```
#define OSAPI_THREAD_PRIORITY_ABOVE_NORMAL -4
```

Above normal priority.

#### OSAPI\_THREAD\_PRIORITY\_HIGH

```
#define OSAPI_THREAD_PRIORITY_HIGH -5
```

High priority.

**OSAPI\_THREAD\_PRIORITY\_INHERIT**

```
#define OSAPI_THREAD_PRIORITY_INHERIT -6
```

Use the same priority as the spawning thread. Only applies to POSIX.

**OSAPI\_THREAD\_USE\_OSDEFAULT\_STACKSIZE**

```
#define OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE 0
```

Use the default stack size when creating a thread.

**OSAPI\_THREAD\_DEFAULT\_OPTIONS**

```
#define OSAPI_THREAD_DEFAULT_OPTIONS 0x00
```

Use only the default options the OS gives you.

**OSAPI\_THREAD\_FLOATING\_POINT**

```
#define OSAPI_THREAD_FLOATING_POINT 0x01
```

Hint to the OS to create a thread that supports floating point.

**OSAPI\_THREAD\_STDIO**

```
#define OSAPI_THREAD_STDIO 0x02
```

Hint to the OS to create a thread that supports standard I/O.

**OSAPI\_THREAD\_REALTIME\_PRIORITY**

```
#define OSAPI_THREAD_REALTIME_PRIORITY 0x08
```

Hint to the OS to create a thread that runs in real-time priority mode.

**OSAPI\_THREAD\_PROPERTY\_DEFAULT**

```
#define OSAPI_THREAD_PROPERTY_DEFAULT
```

**Value:**

```
{ \
    OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE, \
    OSAPI_THREAD_PRIORITY_DEFAULT, \
    OSAPI_THREAD_DEFAULT_OPTIONS \
}
```

Default thread properties.

## OSAPI\_ThreadProperty\_INITIALIZER

```
#define OSAPI_ThreadProperty_INITIALIZER OSAPI_THREAD_PROPERTY_DEFAULT
```

Initializer for [OSAPI\\_ThreadProperty](#).

### 6.16.9.3 Typedef Documentation

## OSAPI\_ThreadOptions

```
typedef RTI_UINT32 OSAPI_ThreadOptions
```

Thread options.

## 6.16.10 OSAPI Time

### Classes

- struct [OSAPI\\_SystemTime](#)  
*System time.*

### Typedefs

- typedef struct OSAPI\_SystemTime [OSAPI\\_SystemTime](#)  
*System time.*

### 6.16.10.1 Detailed Description

### 6.16.10.2 Typedef Documentation

## OSAPI\_SystemTime

```
typedef struct OSAPI_SystemTime OSAPI_SystemTime
```

System time.

Expresses time in seconds and nanoseconds. Assumes that Nanoseconds is less than 1000000000.

## 6.16.11 OSAPI Timer

### Classes

- struct [OSAPI\\_TimerProperty](#)  
*Timer properties.*



### 6.16.11.1 Detailed Description

### 6.16.12 OSAPI Types

OSAPI Types.

#### Macros

- `#define RTI_INT8`  
*A 2's complement 8-bit signed integer.*
- `#define RTI_UINT8`  
*A 8-bit unsigned integer.*
- `#define RTI_INT16`  
*A 2's complement 16-bit signed integer.*
- `#define RTI_UINT16`  
*A 2's complement 16-bit unsigned integer.*
- `#define RTI_INT32`  
*A 2's complement 32-bit signed integer.*
- `#define RTI_UINT32`  
*A 32-bit unsigned integer.*
- `#define RTI_INT64`  
*A 64-bit signed integer.*
- `#define RTI_UINT64`  
*A 64-bit unsigned integer.*
- `#define RTI_FALSE ((RTI_BOOL) 0)`
- `#define RTI_TRUE ((RTI_BOOL) 1)`

#### Typedefs

- `typedef RTI_INT32 RTI_BOOL`
- `typedef RTI_UINT32 RTI_SIZE_T`

### 6.16.12.1 Detailed Description

OSAPI Types.

### 6.16.12.2 Macro Definition Documentation

#### RTI\_INT8

```
#define RTI_INT8
```

A 2's complement 8-bit signed integer.

**RTI\_UINT8**

```
#define RTI_UINT8
```

A 8-bit unsigned integer.

Referenced by [CDR\\_Stream\\_is\\_byte\\_swapped\(\)](#).

**RTI\_INT16**

```
#define RTI_INT16
```

A 2's complement 16-bit signed integer.

**RTI\_UINT16**

```
#define RTI_UINT16
```

A 2's complement 16-bit unsigned integer.

**RTI\_INT32**

```
#define RTI_INT32
```

A 2's complement 32-bit signed integer.

Referenced by [CDR\\_Stream\\_increment\\_current\\_position\(\)](#), [DDS\\_Duration\\_is\\_infinite\(\)](#), [DDS\\_String\\_cmp\(\)](#), [DDS\\_String\\_free\(\)](#), [DDS\\_String\\_length\(\)](#), [DDS\\_String\\_ncmp\(\)](#), [NETIO\\_SharedMemoryMutex\\_attach\(\)](#), [NETIO\\_SharedMemoryMutex\\_create\(\)](#), [NETIO\\_SharedMemoryMutex\\_create\\_or\\_attach\(\)](#), [NETIO\\_SharedMemoryMutex\\_delete\(\)](#), [NETIO\\_SharedMemoryMutex\\_lock\(\)](#), [NETIO\\_SharedMemoryMutex\\_unlock\(\)](#), [NETIO\\_SharedMemorySegment\\_attach\(\)](#), [NETIO\\_SharedMemorySegment\\_create\(\)](#), [NETIO\\_SharedMemorySegment\\_create\\_or\\_attach\(\)](#), [NETIO\\_SharedMemorySegment\\_get\\_address\(\)](#), [NETIO\\_SharedMemorySegment\\_get\\_size\(\)](#), [NETIO\\_SharedMemorySignalingSemaphore\\_attach\(\)](#), [NETIO\\_SharedMemorySignalingSemaphore\\_create\(\)](#), [NETIO\\_SharedMemorySignalingSemaphore\\_signal\(\)](#), [NETIO\\_SharedMemorySignalingSemaphore\\_wait\(\)](#), [OSAPI\\_Log\\_get\\_last\\_error\\_code\(\)](#), [OSAPI\\_Log\\_set\\_property\(\)](#), [OSAPI\\_Semaphore\\_give\(\)](#), and [OSAPI\\_Semaphore\\_take\(\)](#).

**RTI\_UINT32**

```
#define RTI_UINT32
```

A 32-bit unsigned integer.

Referenced by [UDPInterfaceTable::add\\_entry\(\)](#), [CDR\\_Stream\\_check\\_size\(\)](#), [CDR\\_Stream\\_get\\_current\\_position\\_ptr\(\)](#), [CDR\\_Stream\\_increment\\_current\\_position\(\)](#), [CDR\\_Stream\\_is\\_byte\\_swapped\(\)](#), [CDR\\_Stream\\_set\\_current\\_position\\_offset\(\)](#), [DDS\\_Duration\\_is\\_infinite\(\)](#), [DDS\\_ParticipantBuiltinTopicData\\_is\\_equal\(\)](#), [DDS\\_String\\_length\(\)](#), [DDS\\_SubscriptionBuiltinTopicData\\_is\\_equal\(\)](#), [NETIO\\_SharedMemorySegment\\_create\(\)](#), [NETIO\\_SharedMemorySegment\\_create\\_or\\_attach\(\)](#), [NETIO\\_SharedMemorySegment\\_get\\_size\(\)](#), [OSAPI\\_Heap\\_free\(\)](#), [OSAPI\\_Log\\_get\\_last\\_error\\_code\(\)](#), [OSAPI\\_Log\\_set\\_property\(\)](#), [OSAPI\\_Log\\_set\\_verbosity\(\)](#), [OSAPI\\_Mutex\\_give\(\)](#), and [OSAPI\\_Semaphore\\_give\(\)](#).

**RTI\_INT64**

```
#define RTI_INT64
```

A 64-bit signed integer.

Referenced by [OSAPI\\_Log\\_get\\_last\\_error\\_code\(\)](#).

**RTI\_UINT64**

```
#define RTI_UINT64
```

A 64-bit unsigned integer.

Referenced by [OSAPI\\_SharedMemorySegment\\_create\\_impl\(\)](#).

**RTI\_FALSE**

```
#define RTI_FALSE ((RTI_BOOL) 0)
```

Functions that return [RTI\\_BOOL](#) return RTI\_FALSE as logical false.

**RTI\_TRUE**

```
#define RTI_TRUE ((RTI_BOOL) 1)
```

Functions that return [RTI\\_BOOL](#) return RTI\_TRUE as logical true.

**6.16.12.3 Typedef Documentation****RTI\_BOOL**

```
typedef RTI_INT32 RTI_BOOL
```

The type [RTI\\_BOOL](#) uses [RTI\\_TRUE](#) as logical true and [RTI\\_FALSE](#) as logical false. In general, a zero value is considered false, and a non-zero value is considered true.

**RTI\_SIZE\_T**

```
typedef RTI_UINT32 RTI_SIZE_T
```

### 6.16.13 OSAPI\_SharedMemoryMonitor

A shared memory monitor to provide mutual exclusion between processes and inter-process signaling.

#### Functions

- NETIOPSLDIExport [RTI\\_SIZE\\_T OSAPI\\_SharedMemoryMonitor\\_get\\_size](#) (void)  
*Get the amount of memory required to store OSAPI\_SharedMemoryMonitor on the current platform.*
- NETIOPSLDIExport [RTI\\_BOOL OSAPI\\_SharedMemoryMonitor\\_initialize](#) ([RTI\\_SIZE\\_T](#) mem\_len, void \*mem, struct OSAPI\_SharedMemoryMonitor \*\*self\_out)  
*Initialize a shared memory monitor in a provided memory segment.*
- NETIOPSLDIExport [RTI\\_BOOL OSAPI\\_SharedMemoryMonitor\\_finalize](#) (struct OSAPI\_SharedMemoryMonitor \*self)  
*Finalize a shared memory monitor.*
- NETIOPSLDIExport [RTI\\_BOOL OSAPI\\_SharedMemoryMonitor\\_acquire](#) (struct OSAPI\_SharedMemoryMonitor \*self, [OSAPI\\_SHMEM\\_STATUS](#) \*status\_out)  
*Acquire the lock protecting the monitor.*
- NETIOPSLDIExport [RTI\\_BOOL OSAPI\\_SharedMemoryMonitor\\_release](#) (struct OSAPI\_SharedMemoryMonitor \*self)  
*Release the lock protecting the monitor.*
- NETIOPSLDIExport [RTI\\_BOOL OSAPI\\_SharedMemoryMonitor\\_wait](#) (struct OSAPI\_SharedMemoryMonitor \*self, [OSAPI\\_SHMEM\\_STATUS](#) \*status\_out)  
*Wait for the monitor to be signaled.*
- NETIOPSLDIExport [RTI\\_BOOL OSAPI\\_SharedMemoryMonitor\\_signal](#) (struct OSAPI\_SharedMemoryMonitor \*self)  
*Signal a thread waiting on a monitor to wake up.*
- NETIOPSLDIExport [RTI\\_BOOL OSAPI\\_SharedMemoryMonitor\\_mark\\_consistent](#) (struct OSAPI\_SharedMemoryMonitor \*self)  
*Mark the shared state protected by the monitor as consistent after a process died while holding the lock on the monitor.*

#### 6.16.13.1 Detailed Description

A shared memory monitor to provide mutual exclusion between processes and inter-process signaling.

#### 6.16.13.2 Function Documentation

##### OSAPI\_SharedMemoryMonitor\_get\_size()

```
NETIOPSLDIExport RTI\_SIZE\_T OSAPI_SharedMemoryMonitor_get_size (
    void )
```

Get the amount of memory required to store OSAPI\_SharedMemoryMonitor on the current platform.

#### Returns

The number of bytes required to store a shared memory monitor.

### OSAPI\_SharedMemoryMonitor\_initialize()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_initialize (
    RTI_SIZE_T mem_len,
    void * mem,
    struct OSAPI_SharedMemoryMonitor ** self_out)
```

Initialize a shared memory monitor in a provided memory segment.

#### Parameters

|     |                 |                                 |
|-----|-----------------|---------------------------------|
| in  | <i>mem_len</i>  | Size of provided memory.        |
| in  | <i>mem</i>      | Memory segment.                 |
| out | <i>self_out</i> | Pointer to initialized monitor. |

#### Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

### OSAPI\_SharedMemoryMonitor\_finalize()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_finalize (
    struct OSAPI_SharedMemoryMonitor * self)
```

Finalize a shared memory monitor.

#### Parameters

|    |             |                      |
|----|-------------|----------------------|
| in | <i>self</i> | Monitor to finalize. |
|----|-------------|----------------------|

#### Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

Once this function is called (even if it returns RTI\_FALSE) you can no longer use this monitor.

### OSAPI\_SharedMemoryMonitor\_acquire()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_acquire (
    struct OSAPI_SharedMemoryMonitor * self,
    OSAPI_SHMEM_STATUS * status_out)
```

Acquire the lock protecting the monitor.

If this function returns RTI\_TRUE, the calling thread is guaranteed mutual exclusion by the monitor. No other thread can acquire the lock until the lock is returned with [OSAPI\\_SharedMemoryMonitor\\_release](#). This function shall not be called by a thread which currently holds the lock.

If this function returns status\_out=OSAPI\_SHMEM\_STATUS\_OWNER\_DEAD, then any shared data protected by the monitor may be in an inconsistent state. If the shared data can be restored to a consistent state, then it can be marked as consistent with [OSAPI\\_SharedMemoryMonitor\\_mark\\_consistent](#). Otherwise, any subsequent attempts to acquire or wait on the monitor will fail.

**Parameters**

|     |                   |                                                   |
|-----|-------------------|---------------------------------------------------|
| in  | <i>self</i>       | Monitor to acquire.                               |
| out | <i>status_out</i> | If successful, the status of the protected state. |

**Returns**

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

**OSAPI\_SharedMemoryMonitor\_release()**

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_release (
    struct OSAPI_SharedMemoryMonitor * self)
```

Release the lock protecting the monitor.

**Parameters**

|    |             |                     |
|----|-------------|---------------------|
| in | <i>self</i> | Monitor to release. |
|----|-------------|---------------------|

**Returns**

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

This function can only be called on a monitor which is currently locked by the calling thread.

**OSAPI\_SharedMemoryMonitor\_wait()**

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_wait (
    struct OSAPI_SharedMemoryMonitor * self,
    OSAPI_SHMEM_STATUS * status_out)
```

Wait for the monitor to be signaled.

Only one thread can wait on a monitor at any one time. This function can only be called on a monitor which is currently locked by the calling thread. The function will return successfully only when another thread has called [OSAPI\\_SharedMemoryMonitor\\_signal](#) and when the calling thread has acquired the lock.

If this function returns status\_out=OSAPI\_SHMEM\_STATUS\_OWNER\_DEAD, then any shared data protected by the monitor may be in an inconsistent state. If the shared data can be restored to a consistent state, then it can be marked as consistent with [OSAPI\\_SharedMemoryMonitor\\_mark\\_consistent](#). Otherwise, any subsequent attempts to acquire or wait on the monitor will fail.

**Parameters**

|     |                   |                                                   |
|-----|-------------------|---------------------------------------------------|
| in  | <i>self</i>       | Monitor to wait on.                               |
| out | <i>status_out</i> | If successful, the status of the protected state. |

**Returns**

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

### OSAPI\_SharedMemoryMonitor\_signal()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_signal (  
    struct OSAPI_SharedMemoryMonitor * self)
```

Signal a thread waiting on a monitor to wake up.

#### Parameters

|    |      |                    |
|----|------|--------------------|
| in | self | Monitor to signal. |
|----|------|--------------------|

#### Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

This function can only be called on a monitor which is currently locked by the calling thread.

### OSAPI\_SharedMemoryMonitor\_mark\_consistent()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemoryMonitor_mark_consistent (  
    struct OSAPI_SharedMemoryMonitor * self)
```

Mark the shared state protected by the monitor as consistent after a process died while holding the lock on the monitor.

This function can only be called on a monitor which is currently locked by the calling thread and currently inconsistent. The function shall return the monitor to a consistent state and allow other functions to be called on it. This function may not be supported on all platforms.

#### Parameters

|    |      |                             |
|----|------|-----------------------------|
| in | self | Monitor to mark consistent. |
|----|------|-----------------------------|

#### Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

### 6.16.14 OSAPI\_SharedMemorySegment

Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.

#### Classes

- struct [OSAPI\\_SharedMemorySegmentHeader](#)
- struct [OSAPI\\_SharedMemorySegmentHandle](#)

## Enumerations

- enum `OSAPI_SHMEM_MODE` {  
`OSAPI_SHMEM_MODE_UNKNOWN` , `OSAPI_SHMEM_MODE_READ` , `OSAPI_SHMEM_MODE_WRITE` ,  
`OSAPI_SHMEM_MODE_LOCKABLE` ,  
`OSAPI_SHMEM_MODE_ROBUST` }
- enum `OSAPI_SHMEM_STATUS` { `OSAPI_SHMEM_STATUS_OK` , `OSAPI_SHMEM_STATUS_OWNER_DEAD` }
- enum `OSAPI_SHMEM_ATTACH_STATUS` { `OSAPI_SHMEM_ATTACH_STATUS_OK` , `OSAPI_SHMEM_ATTACH_STATUS_SEGMENT_DEAD` }

## Functions

- `RTI_UINT64 OSAPI_SharedMemorySegment_get_max_size` (void)  
*Get maximum size of a shared memory segment on the current platform.*
- `RTI_BOOL OSAPI_SharedMemorySegment_is_owner_alive` (struct `OSAPI_SharedMemorySegmentHandle` \*handle, `RTI_BOOL` \*alive)  
*Check if the process which created the shared memory segment is still alive.*
- `NETIOPSLDIIExport RTI_SIZE_T OSAPI_SharedMemorySegmentHandle_get_size` (`OSAPI_SHMEM_MODE` mode)  
*Get the amount of memory required to store a `OSAPI_SharedMemorySegmentHandle` on the current platform for a given mode.*
- `NETIOPSLDIIExport RTI_SIZE_T OSAPI_SharedMemorySegmentHeader_get_size` (`OSAPI_SHMEM_MODE` mode)  
*Get the amount of memory, aligned to the maximum alignment, required to store a `OSAPI_SharedMemorySegmentHeader` on the current platform for a given mode.*
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_create_impl` (struct `OSAPI_SharedMemorySegmentHandle` \*handle, const char \*name, `RTI_UINT64` size, `OSAPI_SHMEM_MODE` mode, `RTI_BOOL` \*exists)  
*Creates a new shared memory segment with the provided key and initialize a handle to it.*
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_delete_impl` (struct `OSAPI_SharedMemorySegmentHandle` \*handle)  
*Delete a shared memory segment.*
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_attach_impl` (struct `OSAPI_SharedMemorySegmentHandle` \*handle, const char \*name, `OSAPI_SHMEM_MODE` mode, `OSAPI_SHMEM_ATTACH_STATUS` \*status\_out)  
*Attach to the shared memory and initialize a handle to it.*
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_detach_impl` (struct `OSAPI_SharedMemorySegmentHandle` \*handle)  
*detach the shared memory segment.*
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_lock_impl` (struct `OSAPI_SharedMemorySegmentHandle` \*handle, `OSAPI_SHMEM_STATUS` \*status\_out)  
*Lock a shared memory segment.*
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_unlock_impl` (struct `OSAPI_SharedMemorySegmentHandle` \*handle)  
*Unlock a shared memory segment previously locked with `OSAPI_SharedMemorySegment_lock_impl`.*
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemorySegment_mark_consistent_impl` (struct `OSAPI_SharedMemorySegmentHandle` \*handle)  
*Mark a shared memory segment as consistent. This function can only be called if a previous call to `OSAPI_SharedMemorySegment_lock_impl` returned `OSAPI_SHMEM_FAIL_REASON_OWNER_DEAD`.*
- `NETIOPSLDIIExport RTI_BOOL OSAPI_SharedMemory_is_robust_mutex_supported` (void)  
*Checks whether robust mutex is supported or not.*



### 6.16.14.1 Detailed Description

Allow access to a shared memory segment (memory block) so that multiple processes can read-and write a common memory area.

### 6.16.14.2 Enumeration Type Documentation

#### OSAPI\_SHMEM\_MODE

enum [OSAPI\\_SHMEM\\_MODE](#)

Mode used to create or attach to a shared memory segment. Each mode defines specific capabilities of the segment.

##### Enumerator

|                           |                                                                              |
|---------------------------|------------------------------------------------------------------------------|
| OSAPI_SHMEM_MODE_UNKNOWN  | Invalid mode                                                                 |
| OSAPI_SHMEM_MODE_READ     | Read permission                                                              |
| OSAPI_SHMEM_MODE_WRITE    | Write permission, implies <a href="#">OSAPI_SHMEM_MODE_READ</a>              |
| OSAPI_SHMEM_MODE_LOCKABLE | Locking capability, implies <a href="#">OSAPI_SHMEM_MODE_WRITE</a>           |
| OSAPI_SHMEM_MODE_ROBUST   | Robust locking capability, implies <a href="#">OSAPI_SHMEM_MODE_LOCKABLE</a> |

#### OSAPI\_SHMEM\_STATUS

enum [OSAPI\\_SHMEM\\_STATUS](#)

Status used to convey the state of a shared memory object.

##### Enumerator

|                               |                                              |
|-------------------------------|----------------------------------------------|
| OSAPI_SHMEM_STATUS_OK         | Success                                      |
| OSAPI_SHMEM_STATUS_OWNER_DEAD | Previous owner of lock died while holding it |

#### OSAPI\_SHMEM\_ATTACH\_STATUS

enum [OSAPI\\_SHMEM\\_ATTACH\\_STATUS](#)

Status used to convey the state of a shared memory object's attachment to a shared memory segment.

##### Enumerator

|                                             |                                             |
|---------------------------------------------|---------------------------------------------|
| OSAPI_SHMEM_ATTACH_STATUS_OK                | Success                                     |
| OSAPI_SHMEM_ATTACH_STATUS_SEGMENT_NOT_FOUND | Segment was not found when trying to attach |

### 6.16.14.3 Function Documentation

#### OSAPI\_SharedMemorySegment\_get\_max\_size()

```
RTI_UINT64 OSAPI_SharedMemorySegment_get_max_size (
    void )
```

Get maximum size of a shared memory segment on the current platform.

##### Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

#### OSAPI\_SharedMemorySegment\_is\_owner\_alive()

```
RTI_BOOL OSAPI_SharedMemorySegment_is_owner_alive (
    struct OSAPI_SharedMemorySegmentHandle * handle,
    RTI_BOOL * alive)
```

Check if the process which created the shared memory segment is still alive.

##### Parameters

|     |               |                                                           |
|-----|---------------|-----------------------------------------------------------|
| in  | <i>handle</i> | The shared memory segment handle identifying the segment. |
| out | <i>alive</i>  | If the process is alive.                                  |

##### Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

The behavior of this function is undefined if the segment is not currently attached.

#### OSAPI\_SharedMemorySegmentHandle\_get\_size()

```
NETIOPSLDllExport RTI_SIZE_T OSAPI_SharedMemorySegmentHandle_get_size (
    OSAPI_SHMEM_MODE mode)
```

Get the amount of memory required to store a [OSAPI\\_SharedMemorySegmentHandle](#) on the current platform for a given mode.

##### Parameters

|    |             |                                                |
|----|-------------|------------------------------------------------|
| in | <i>mode</i> | The creation or attachment mode of the handle. |
|----|-------------|------------------------------------------------|

##### Returns

Number of bytes required to store a handle.

#### OSAPI\_SharedMemorySegmentHeader\_get\_size()

```
NETIOPSLDllExport RTI_SIZE_T OSAPI_SharedMemorySegmentHeader_get_size (
    OSAPI_SHMEM_MODE mode)
```

Get the amount of memory, aligned to the maximum alignment, required to store a [OSAPI\\_SharedMemorySegmentHeader](#) on the current platform for a given mode.

**Parameters**

|    |             |                                                         |
|----|-------------|---------------------------------------------------------|
| in | <i>mode</i> | The creation mode of the segment containing the header. |
|----|-------------|---------------------------------------------------------|

**Returns**

Number of bytes required to store a header.

The number of bytes returned by this function shall be aligned to the maximum alignment of the platform.

**OSAPI\_SharedMemorySegment\_create\_impl()**

```
NETIOPSLD11Export RTI_BOOL OSAPI_SharedMemorySegment_create_impl (  
    struct OSAPI_SharedMemorySegmentHandle * handle,  
    const char * name,  
    RTI_UINT64 size,  
    OSAPI_SHMEM_MODE mode,  
    RTI_BOOL * exists)
```

Creates a new shared memory segment with the provided key and initialize a handle to it.

**Parameters**

|         |               |                                                                                                                                                     |
|---------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| in, out | <i>handle</i> | Handle to be initialized.                                                                                                                           |
| in      | <i>name</i>   | Name to identify the shared memory segment.                                                                                                         |
| in      | <i>size</i>   | Size of memory to be allocated.                                                                                                                     |
| in      | <i>mode</i>   | Mode used to create the shared memory segment. The mode determines the capabilities of the segment. The mode must be $\geq$ OSAPI_SHMEM_MODE_WRITE. |
| out     | <i>exists</i> | If the segment already exists                                                                                                                       |

**Returns**

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

The new shared memory segment shall be initialized with null bytes ('\0').

References [RTI\\_UINT64](#).

**OSAPI\_SharedMemorySegment\_delete\_impl()**

```
NETIOPSLD11Export RTI_BOOL OSAPI_SharedMemorySegment_delete_impl (  
    struct OSAPI_SharedMemorySegmentHandle * handle)
```

Delete a shared memory segment.

**Parameters**

|    |               |                                                                     |
|----|---------------|---------------------------------------------------------------------|
| in | <i>handle</i> | The shared memory segment handle identifying the segment to delete. |
|----|---------------|---------------------------------------------------------------------|

**Returns**

RTI\_TRUE on success. RTI\_FALSE if either the precondition was not met or the shared memory segment could not be deleted.

Once this function is called (even if it returns RTI\_FALSE) you can no longer use this shared memory segment.

**OSAPI\_SharedMemorySegment\_attach\_impl()**

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_attach_impl (
    struct OSAPI_SharedMemorySegmentHandle * handle,
    const char * name,
    OSAPI_SHMEM_MODE mode,
    OSAPI_SHMEM_ATTACH_STATUS * status_out)
```

Attach to the shared memory and initialize a handle to it.

**Parameters**

|         |                   |                                                                    |
|---------|-------------------|--------------------------------------------------------------------|
| in, out | <i>handle</i>     | Handle to be initialized.                                          |
| in      | <i>name</i>       | Name to identify the shared memory segment.                        |
| in      | <i>mode</i>       | Mode used to attach the shared memory segment.                     |
| out     | <i>status_out</i> | If successful, the attachment status of the shared memory segment. |

**Returns**

RTI\_TRUE on success. Note that the attachment status may still be different from OK, so checking the value of status\_out is mandatory. RTI\_FALSE if an error occurred.

**OSAPI\_SharedMemorySegment\_detach\_impl()**

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_detach_impl (
    struct OSAPI_SharedMemorySegmentHandle * handle)
```

detach the shared memory segment.

**Parameters**

|    |               |                                                                     |
|----|---------------|---------------------------------------------------------------------|
| in | <i>handle</i> | The shared memory segment handle identifying the segment to detach. |
|----|---------------|---------------------------------------------------------------------|

**Returns**

RTI\_TRUE on success. RTI\_FALSE if either the precondition was not met or the shared memory segment could not be detached.

Once this function is called (even if it returns RTI\_FALSE) you can no longer use this shared memory segment handle.

### OSAPI\_SharedMemorySegment\_lock\_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_lock_impl (  
    struct OSAPI_SharedMemorySegmentHandle * handle,  
    OSAPI_SHMEM_STATUS * status_out)
```

Lock a shared memory segment.

#### Parameters

|     |                   |                                                                   |
|-----|-------------------|-------------------------------------------------------------------|
| in  | <i>handle</i>     | The shared memory segment handle identifying the segment to lock. |
| out | <i>status_out</i> | Success or failure reason of acquiring the lock.                  |

#### Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

### OSAPI\_SharedMemorySegment\_unlock\_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_unlock_impl (  
    struct OSAPI_SharedMemorySegmentHandle * handle)
```

Unlock a shared memory segment previously locked with [OSAPI\\_SharedMemorySegment\\_lock\\_impl](#).

#### Parameters

|    |               |                                                                     |
|----|---------------|---------------------------------------------------------------------|
| in | <i>handle</i> | The shared memory segment handle identifying the segment to unlock. |
|----|---------------|---------------------------------------------------------------------|

#### Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

The behavior of this method is undefined if the calling process has not previously acquired the lock with [OSAPI\\_SharedMemorySegment\\_lock\\_impl](#).

### OSAPI\_SharedMemorySegment\_mark\_consistent\_impl()

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemorySegment_mark_consistent_impl (  
    struct OSAPI_SharedMemorySegmentHandle * handle)
```

Mark a shared memory segment as consistent. This function can only be called if a previous call to [OSAPI\\_SharedMemorySegment\\_lock\\_impl](#) returned OSAPI\_SHMEM\_FAIL\_REASON\_OWNER\_DEAD.

## Parameters

|    |        |                                                                              |
|----|--------|------------------------------------------------------------------------------|
| in | handle | The shared memory segment handle identifying the segment to mark consistent. |
|----|--------|------------------------------------------------------------------------------|

## Returns

RTI\_TRUE on success. RTI\_FALSE if an error occurred.

**OSAPI\_SharedMemory\_is\_robust\_mutex\_supported()**

```
NETIOPSLDllExport RTI_BOOL OSAPI_SharedMemory_is_robust_mutex_supported (
    void )
```

Checks whether robust mutex is supported or not .

## Returns

RTI\_TRUE is supported. RTI\_FALSE if not supported.

**6.17 Log API****Classes**

- struct [OSAPI\\_LogProperty](#)  
*Configuration of logging functionality.*

**Typedefs**

- typedef void(\* [OSAPI\\_Log\\_write\\_buffer\\_T](#)) (const char \*buffer, [RTI\\_SIZE\\_T](#) length)  
*Optional user-defined function to output a log/trace buffer.*
- typedef void(\* [OSAPI\\_LogHandler\\_T](#)) (void \*param, [OSAPI\\_LogEntry\\_T](#) \*entry)  
*Optional user-defined function for processing new log messages.*

**Enumerations**

- enum [OSAPI\\_LogVerbosity\\_T](#) { [OSAPI\\_LOG\\_VERBOSITY\\_SILENT](#) = 0 , [OSAPI\\_LOG\\_VERBOSITY\\_ERROR](#) = 1 , [OSAPI\\_LOG\\_VERBOSITY\\_WARNING](#) = 2 , [OSAPI\\_LOG\\_VERBOSITY\\_DEBUG](#) = 3 }  
*Logging verbosity.*

## Functions

- [RTI\\_BOOL OSAPI\\_Log\\_set\\_log\\_handler](#) ([OSAPI\\_LogHandler\\_T](#) handler, void \*param)  
*Install a log handler.*
- [RTI\\_BOOL OSAPI\\_Log\\_get\\_log\\_handler](#) ([OSAPI\\_LogHandler\\_T](#) \*handler, void \*\*param)  
*Return the current log handler.*
- [RTI\\_BOOL OSAPI\\_Log\\_set\\_display\\_handler](#) ([OSAPI\\_LogDisplay\\_T](#) handler, void \*param)  
*Install a display handler.*
- [RTI\\_BOOL OSAPI\\_Log\\_get\\_display\\_handler](#) ([OSAPI\\_LogDisplay\\_T](#) \*handler, void \*\*param)  
*Return the current display function.*
- void [OSAPI\\_Log\\_set\\_verbosity](#) ([OSAPI\\_LogVerbosity\\_T](#) verbosity)  
*Set the log verbosity.*
- [RTI\\_BOOL OSAPI\\_Log\\_clear](#) (void)  
*Clear the log buffer.*
- void [OSAPI\\_Log\\_get\\_property](#) (struct [OSAPI\\_LogProperty](#) \*property)  
*Get the current log properties.*
- [RTI\\_BOOL OSAPI\\_Log\\_set\\_property](#) (struct [OSAPI\\_LogProperty](#) \*property)  
*Set the log properties.*
- [OSPSLDIExport RTI\\_INT32 OSAPI\\_Log\\_get\\_last\\_error\\_code](#) (void)  
*Returns the error code for a function that failed.*

### 6.17.1 Detailed Description

### 6.17.2 Typedef Documentation

#### OSAPI\_Log\_write\_buffer\_T

```
typedef void(* OSAPI_Log_write_buffer_T) (const char *buffer, RTI\_SIZE\_T length)
```

Optional user-defined function to output a log/trace buffer.

The handler is set by [OSAPI\\_Log\\_set\\_property\(\)](#).

#### Parameters

|    |               |                                     |
|----|---------------|-------------------------------------|
| in | <i>buffer</i> | Pointer to buffer with log to write |
| in | <i>length</i> | Length of the buffer to write       |

#### OSAPI\_LogHandler\_T

```
typedef void(* OSAPI_LogHandler_T) (void *param, OSAPI\_LogEntry\_T *entry)
```

Optional user-defined function for processing new log messages.

Definition of a function that can be installed with the logger and that will be called for each new log event.

The handler is set by [OSAPI\\_Log\\_set\\_log\\_handler\(\)](#).

When called, provides parameters containing the raw log entry and additional optional information.

## Parameters

|    |              |                        |
|----|--------------|------------------------|
| in | <i>param</i> | User-defined parameter |
| in | <i>entry</i> | Log entry              |

## 6.17.3 Enumeration Type Documentation

**OSAPI\_LogVerbosity\_T**

```
enum OSAPI_LogVerbosity_T
```

Logging verbosity.

## Enumerator

|                             |                                     |
|-----------------------------|-------------------------------------|
| OSAPI_LOG_VERBOSITY_SILENT  | Logs are not written.               |
| OSAPI_LOG_VERBOSITY_ERROR   | Only error logs are written.        |
| OSAPI_LOG_VERBOSITY_WARNING | Error and warning logs are written. |
| OSAPI_LOG_VERBOSITY_DEBUG   | All logs are written.               |

## 6.17.4 Function Documentation

**OSAPI\_Log\_set\_log\_handler()**

```
RTI_BOOL OSAPI_Log_set_log_handler (
    OSAPI_LogHandler_T handler,
    void * param)
```

Install a log handler.

The log functionality allows the user to specify a log handler. The log handler is a function which is called for every logged event. It is up to the user to decide what to do with the log message. The handler is a global function pointer.

## Parameters

|    |                |                                                                                                       |
|----|----------------|-------------------------------------------------------------------------------------------------------|
| in | <i>handler</i> | Pointer to log handler function                                                                       |
| in | <i>param</i>   | Parameter passed to the log handler function. This parameter is transparent to the log functionality. |

## Returns

RTI\_TRUE on success, RTI\_FALSE on failure

**OSAPI\_Log\_get\_log\_handler()**

```
RTI_BOOL OSAPI_Log_get_log_handler (
    OSAPI_LogHandler_T * handler,
    void ** param)
```

Return the current log handler.

Return the current log handler.



**Parameters**

|    |                |                                                     |
|----|----------------|-----------------------------------------------------|
| in | <i>handler</i> | Pointer to store log handler function               |
| in | <i>param</i>   | Pointer to store the current log handler parameter. |

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure

**OSAPI\_Log\_set\_display\_handler()**

```
RTI_BOOL OSAPI_Log_set_display_handler (  
    OSAPI_LogDisplay_T handler,  
    void * param)
```

Install a display handler.

The display handler is responsible for outputting log messages to a console.

**Parameters**

|    |                |                                                                                                           |
|----|----------------|-----------------------------------------------------------------------------------------------------------|
| in | <i>handler</i> | Pointer to display function                                                                               |
| in | <i>param</i>   | Parameter passed to the display handler function. This parameter is transparent to the log functionality. |

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure

**OSAPI\_Log\_get\_display\_handler()**

```
RTI_BOOL OSAPI_Log_get_display_handler (  
    OSAPI_LogDisplay_T * handler,  
    void ** param)
```

Return the current display function.

Return the current log handler.

**Parameters**

|    |                |                                                          |
|----|----------------|----------------------------------------------------------|
| in | <i>handler</i> | Pointer to store display handler function                |
| in | <i>param</i>   | Pointer to store the current display handler parameters. |

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure

**OSAPI\_Log\_set\_verbosity()**

```
void OSAPI_Log_set_verbosity (
    OSAPI_LogVerbosity_T verbosity)
```

Set the log verbosity.

Change the log verbosity. The new setting takes immediate effect.

**Parameters**

|    |                  |                   |
|----|------------------|-------------------|
| in | <i>verbosity</i> | New log verbosity |
|----|------------------|-------------------|

References [RTI\\_UINT32](#).

**OSAPI\_Log\_clear()**

```
RTI_BOOL OSAPI_Log_clear (
    void )
```

Clear the log buffer.

Clear the log buffer, all the current entries are lost.

NOTE: This function must only be called inside a log-handler as it is not thread-safe.

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure

**OSAPI\_Log\_get\_property()**

```
void OSAPI_Log_get_property (
    struct OSAPI_LogProperty * property)
```

Get the current log properties.

Return the current log properties

**Parameters**

|     |                 |                          |
|-----|-----------------|--------------------------|
| out | <i>property</i> | - Current log properties |
|-----|-----------------|--------------------------|

See Also [OSAPI\\_Log\\_set\\_property](#)

**OSAPI\_Log\_set\_property()**

```
RTI_BOOL OSAPI_Log_set_property (
    struct OSAPI_LogProperty * property)
```

Set the log properties.

Set the current log properties. It is not possible to set new properties after [OSAPI\\_Log\\_initialize\(\)](#) has been called/

**Parameters**

|    |                 |                    |
|----|-----------------|--------------------|
| in | <i>property</i> | New log properties |
|----|-----------------|--------------------|

**Returns**

RTI\_TRUE if new properties can be set, RTI\_FALSE otherwise See Also OSAPI\_Log\_get\_property

References [RTI\\_INT32](#), and [RTI\\_UINT32](#).

**OSAPI\_Log\_get\_last\_error\_code()**

```
OSPSLDllExport RTI_INT32 OSAPI_Log_get_last_error_code (
    void )
```

Returns the error code for a function that failed.

Many functions returns RTI\_FALSE or NULL on failure. In order to provide additional information about reason for the failure functions may set an additional error code. This function returns the last error-code recorded for the calling thread.

**Returns**

Last error-code recorded for this thread

References [RTI\\_INT32](#), [RTI\\_INT64](#), and [RTI\\_UINT32](#).

**6.18 Application Generation API****Functions**

- APPGENDllExport struct RT\_ComponentFactoryI \* [APPGEN\\_Factory\\_get\\_interface](#) (void)  
*Gets the singleton instance of the Application Generation factory.*
- APPGENDllExport [RTI\\_BOOL APPGEN\\_Factory\\_register](#) (RT\_Registry\_T \*registry, struct APPGEN\_Factory↵  
Property \*property)  
*Registers the Micro Application Generation factory.*
- APPGENDllExport [RTI\\_BOOL APPGEN\\_Factory\\_unregister](#) (RT\_Registry\_T \*registry, struct APPGEN\_↵  
FactoryProperty \*\*property)  
*Unregisters the Micro Application Generation factory.*

**6.18.1 Detailed Description**

The AppGen plugin is an implementation of the RTI Connex DDS Micro DDS\_AppGen plugin interface and implements the application generation supported by RTI Connex DDS Micro.

RTI Connex DDS Micro requires that a DDS\_AppGen plugin is registered with the name "\_mag" in the Domain↵ ParticipantFactory before a participant can be created.

Each call to [DDSDomainParticipantFactory::create\\_participant\\_from\\_config](#) creates its own instance of the DDS\_↵ AppGen plugin. Please refer to the Getting Started Guide for an example of how to register the application generation factory.

### 6.18.2 Function Documentation

#### APPGEN\_Factory\_get\_interface()

```
APPGENDllExport struct RT_ComponentFactoryI * APPGEN_Factory_get_interface (
    void )
```

Gets the singleton instance of the Application Generation factory.

This function gets the singleton instance of the Application Generation plugin factory that is used by the middleware to create an AppGen plugin. The singleton instance of the factory must be registered with the name "\_mag" in the DomainParticipantFactory.

#### Returns

Pointer to Application Generation plugin factory

#### APPGEN\_Factory\_register()

```
APPGENDllExport RTI_BOOL APPGEN_Factory_register (
    RT_Registry_T * registry,
    struct APPGEN_FactoryProperty * property)
```

Registers the Micro Application Generation factory.

#### Parameters

|                 |                                                                        |
|-----------------|------------------------------------------------------------------------|
| <i>registry</i> | Registry with which to register the component factory. Cannot be NULL. |
| <i>property</i> | Pointer to factory properties                                          |

#### Returns

BOOLEAN\_TRUE if factory is successfully registered BOOLEAN\_FALSE in case of error.

#### APPGEN\_Factory\_unregister()

```
APPGENDllExport RTI_BOOL APPGEN_Factory_unregister (
    RT_Registry_T * registry,
    struct APPGEN_FactoryProperty ** property)
```

Unregisters the Micro Application Generation factory.

#### Parameters

|                 |                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------|
| <i>registry</i> | Registry with which to register the component factory. Cannot be NULL.                                           |
| <i>property</i> | Pointer where to copy the pointer to the factory properties passed in call to function AppGenFactory_register(). |

#### Returns

BOOLEAN\_TRUE if factory is successfully unregistered. BOOLEAN\_FALSE in case of error.

## 6.19 Lightweight Security API

Lightweight Security Plugin API.

### Classes

- struct [DDS\\_PskServiceFactoryProperty](#)  
*Property used to register the Lightweight Security Plugin.*

### Macros

- #define [DDS\\_PskServiceFactoryProperty\\_INITIALIZER](#)  
*Constant to initialize a [DDS\\_PskServiceFactoryProperty](#).*

### Variables

- DDSPSKDllExport const char \*const [DDS\\_PSK\\_DEFAULT\\_PLUGIN\\_NAME](#)  
*Default name used for the Lightweight Security Plugin.*

### 6.19.1 Detailed Description

Lightweight Security Plugin API.

<<*eXtension*>> This module provides the security solution defined by the DDS Security Specification PSK (Pre-shared Key) Builtin Plugin.

### 6.19.2 Macro Definition Documentation

#### DDS\_PskServiceFactoryProperty\_INITIALIZER

```
#define DDS_PskServiceFactoryProperty_INITIALIZER
```

#### Value:

```
{ \
    RT_ComponentFactoryProperty_INITIALIZER, \
    NULL, \
    NULL, \
    NULL \
}
```

Constant to initialize a [DDS\\_PskServiceFactoryProperty](#).

### 6.19.3 Variable Documentation

#### DDS\_PSK\_DEFAULT\_PLUGIN\_NAME

```
DDSPSKDllExport const char* const DDS_PSK_DEFAULT_PLUGIN_NAME [extern]
```

Default name used for the Lightweight Security Plugin.

## 6.20 Resource Limits

### Topics

- [DomainParticipantFactoryQos](#)
- [DomainParticipantQos](#)
- [DataReaderQos](#)
- [DataWriterQos](#)
- [OSAPI](#)
- [UDP Transport](#)
- [Dynamic Participant Static Endpoint \(DPSE\)](#)
- [Dynamic Participant Dynamic Endpoint \(DPDE\)](#)

### 6.20.1 Detailed Description

### 6.20.2 Introduction

RTI Connex DDS Micro is designed for use in real-time systems and uses a predictable and deterministic memory manager to ensure that memory growth is not unbounded, OS memory fragmentation is eliminated and memory usage can be determined a-priori. The advantage with this design is that proper operation is ensured as soon as steady state has been reached. However, it also places an additional burden on the system designer to properly configure each resource-limit. This section describes all the resource limits in RTI Connex DDS Micro and how they impact RTI Connex DDS Micro's behavior and memory usage.

### 6.20.3 Dynamic Memory Allocation

RTI Connex DDS Micro allocates heap memory to create internal data-structures. It is important to know that RTI Connex DDS Micro manages memory allocated from the system heap using its own internal memory management, and only returns memory allocated from the system back to the system when something is deleted. That is, if an application never deletes anything, no memory is returned to the system.

As a rule of thumb, in RTI Connex DDS Micro only APIs that end in `_create()` or `_new()` allocate heap memory and APIs that end in `_delete()` or `_free()` frees memory. An exception to this rule is dynamic discovery which allocates memory at run-time for the topic and type names.

RTI Connex DDS Micro does not support dynamically allocating resources beyond the initial configuration. That is, all resource limits must be finite. This restriction may be removed in a future version.

## 6.20.4 DomainParticipantFactoryQos

### Topics

- [max\\_components](#)
- [max\\_participants](#)

#### 6.20.4.1 Detailed Description

#### 6.20.4.2 max\_components

##### NAME

[DDS\\_SystemResourceLimitsQosPolicy::max\\_components](#)

##### DESCRIPTION

RTI Connex DDS Micro consists of a number of plugins and components that perform various tasks. For example, the UDP transport and the DPDE discovery plugins are RTI Connex DDS Micro components. This resource-limit limits the number of components that can be registered with RTI Connex DDS Micro. This value should only be changed if plugins and components are developed.

#### 6.20.4.3 max\_participants

##### NAME

[DDS\\_SystemResourceLimitsQosPolicy::max\\_participants](#)

##### DESCRIPTION

[DDS\\_SystemResourceLimitsQosPolicy::max\\_participants](#) limits the number of local DomainParticipants that can be created with the [DDSDomainParticipantFactory::create\\_participant\(\)](#) API. For the maximum number of participants that can be discovered, please refer to [remote\\_participant\\_allocation](#).

## 6.20.5 DomainParticipantQos

### Topics

- [local\\_topic\\_allocation](#)
- [local\\_type\\_allocation](#)
- [local\\_publisher\\_allocation](#)
- [local\\_subscriber\\_allocation](#)
- [local\\_reader\\_allocation](#)
- [local\\_writer\\_allocation](#)
- [matching\\_writer\\_reader\\_pair\\_allocation](#)
- [remote\\_participant\\_allocation](#)
- [remote\\_writer\\_allocation](#)
- [remote\\_reader\\_allocation](#)
- [max\\_destination\\_ports](#)
- [max\\_receive\\_ports](#)

### 6.20.5.1 Detailed Description

The `DomainParticipantQos` controls resources that are applicable to the entire `DomainParticipant`. All the resources specified in the `DomainParticipantQos` are allocated when the `DomainParticipant` is created with the `DDSDomainParticipantFactory::create_participant()` call.

### 6.20.5.2 `local_topic_allocation`

#### NAME

`DDS_DomainParticipantResourceLimitsQosPolicy::local_topic_allocation`

#### DESCRIPTION

`DDS_DomainParticipantResourceLimitsQosPolicy::local_topic_allocation` limits the maximum number of unique `DDSTopic`'s that can be created within a single `DDSDomainParticipant` with the `DDSDomainParticipant::create_topic()` API. When a `DDSTopic` is created a string equal to the length of the topic name in bytes plus one is allocated.

### 6.20.5.3 `local_type_allocation`

#### NAME

`DDS_DomainParticipantResourceLimitsQosPolicy::local_type_allocation`

#### DESCRIPTION

`DDS_DomainParticipantResourceLimitsQosPolicy::local_type_allocation` limits the maximum number of unique `DDS_TypePluginI` plugins that can be registered locally within a single `DDSDomainParticipant` with the `FooTypeSupport_register_type()` API. When a `DDS_TypePluginI` is registered a string equal to the length of the type name in bytes plus one is allocated.

### 6.20.5.4 `local_publisher_allocation`

#### NAME

`DDS_DomainParticipantResourceLimitsQosPolicy::local_publisher_allocation`

#### DESCRIPTION

`DDS_DomainParticipantResourceLimitsQosPolicy::local_publisher_allocation` limits the maximum number of `DDSPublisher` entities that can be created within a `DomainParticipant` with the `DDSDomainParticipant::create_publisher()` API.



#### 6.20.5.5 local\_subscriber\_allocation

##### NAME

[DDS\\_DomainParticipantResourceLimitsQosPolicy::local\\_subscriber\\_allocation](#)

##### DESCRIPTION

[DDS\\_DomainParticipantResourceLimitsQosPolicy::local\\_subscriber\\_allocation](#) limits the maximum number of [DDSSubscriber](#) entities that can be created within a [DomainParticipant](#) with the [DDSDomainParticipant::create\\_subscriber\(\)](#) API.

#### 6.20.5.6 local\_reader\_allocation

##### NAME

[DDS\\_DomainParticipantResourceLimitsQosPolicy::local\\_reader\\_allocation](#)

##### DESCRIPTION

[DDS\\_DomainParticipantResourceLimitsQosPolicy::local\\_reader\\_allocation](#) limits the maximum number of [DDSDataReader](#) entities that can be created with the [DDSSubscriber::create\\_datareader\(\)](#) API within a single [DDSDomainParticipant](#) across all DDS Subscriber entities.

#### 6.20.5.7 local\_writer\_allocation

##### NAME

[DDS\\_DomainParticipantResourceLimitsQosPolicy::local\\_writer\\_allocation](#)

##### DESCRIPTION

[DDS\\_DomainParticipantResourceLimitsQosPolicy::local\\_writer\\_allocation](#) limits the maximum number of [DDSDataWriter](#) entities that can be created with the [DDSPublisher::create\\_datawriter\(\)](#) API within a single [DDSDomainParticipant](#) across all [DDSPublisher](#) entities.

#### 6.20.5.8 matching\_writer\_reader\_pair\_allocation

##### NAME

[DDS\\_DomainParticipantResourceLimitsQosPolicy::matching\\_writer\\_reader\\_pair\\_allocation](#)

##### DESCRIPTION

When RTI Connex Micro DDS discovers DDS entities using a DDS discovery protocol, such as DPSE (Dynamic Participant Static Endpoint) or DPDE (Dynamic Participant Dynamic Endpoint), it matches the discovered entities with the locally created DataReaders and DataWriters. For each match, either a local DataWriter matching a remote DataReader or a local DataReader matching a remote DataWriter, internal state must be managed.

A match, in this context, is defined as being able to receive packets sent from a [DDSDDataReader](#) to a [DDSDDataWriter](#), or from a [DDSDDataWriter](#) to a [DDSDDataReader](#):

- A data-writer needs one resource for each DataReader it matches.
- A data-reader needs one resource for each DataWriter it matches.

[DDS\\_DomainParticipantResourceLimitsQosPolicy::matching\\_writer\\_reader\\_pair\\_allocation](#) sets the limit for the maximum number of matches that can be managed. When this resource-limit is reached no further matching can occur, even if new [DDSDDataReader](#)'s and [DDSDDataWriter](#)'s are discovered.

A simple rule of thumb is to set this resource-limit to:

$(\text{local\_reader\_allocation} + \text{local\_writer\_allocation}) * \text{max\_remote\_participants}$ .

This assumes that each DataReader and DataWriter matches one DataWriter and DataReader respectively for each discovered participant. However, this may over-allocate resources.

#### NOTES

[DDS\\_DomainParticipantResourceLimitsQosPolicy::matching\\_reader\\_writer\\_pair\\_allocation](#) is not used and will be deprecated in future versions.

#### 6.20.5.9 remote\_participant\_allocation

##### NAME

[DDS\\_DomainParticipantResourceLimitsQosPolicy::remote\\_participant\\_allocation](#)

##### DESCRIPTION

[DDS\\_DomainParticipantResourceLimitsQosPolicy::remote\\_participant\\_allocation](#) limits the maximum number of [DDSDomainParticipant](#)'s that can be discovered by a local [DDSDomainParticipant](#).

The DPDE discovery plugin is a first discovered first served basis and the only way to limit discovery is careful use of peer addresses.

The DPSE discovery plugin limits the discovery to those that have been asserted based on the DomainParticipant name using [DPSEDiscoveryPlugin::RemoteParticipant\\_assert](#)

#### 6.20.5.10 remote\_writer\_allocation

##### NAME

[DDS\\_DomainParticipantResourceLimitsQosPolicy::remote\\_writer\\_allocation](#)

##### DESCRIPTION

[DDS\\_DomainParticipantResourceLimitsQosPolicy::remote\\_writer\\_allocation](#) limits the maximum number of remote [DDSDataWriter](#) entities that can be discovered by a local [DDSDomainParticipant](#). Note that a remote [DDSDataWriter](#) cannot be discovered until its parent [DDSDomainParticipant](#) has been discovered.

#### 6.20.5.11 remote\_reader\_allocation

##### NAME

[DDS\\_DomainParticipantResourceLimitsQosPolicy::remote\\_reader\\_allocation](#)

##### DESCRIPTION

[DDS\\_DomainParticipantResourceLimitsQosPolicy::remote\\_reader\\_allocation](#) limits the maximum number of remote [DDSDataReader](#) entities that can be discovered by a local [DomainParticipant](#). Note that a remote [DDSDataReader](#) cannot be discovered until its parent [DDSDomainParticipant](#) has been discovered.

#### 6.20.5.12 max\_destination\_ports

##### NAME

[DDS\\_DomainParticipantResourceLimitsQosPolicy::max\\_destination\\_ports](#)

##### DESCRIPTION

RTI Connex DDS Micro maintains a table of destination addresses which it can send to based on information queried from the registered transports. The [DDS\\_DomainParticipantResourceLimitsQosPolicy::max\\_destination\\_ports](#) resource-limit is unfortunately difficult to give a precise rule for how to determine since it is transport dependent. However, as a rule of thumb the following is true:

- Each network that can be reached directly by the transport requires 1 resource.
- If multicast is supported, it counts as 1 resource.
- If the transport is the default transport (by default UDP is), it counts as 1 resource.

##### EXAMPLES

Consider a typical Linux computer with 2 network interfaces, eth0 and lo, with support for multicast.

- 1 for the eth0 network.
- 1 for the lo network.
- 1 for multicast.
- 1 for being the default transport.

This requires 4 resources.

Consider a typical embedded computer with 1 network interface, eth0, with no support for multicast:

- 1 for the eth0 network.
- 1 for being the default transport.

This requires 2 resources.

## NOTES

The default value has been set to cover most cases. This resource-limit should only be changed if a lot of interfaces are supported (increase it) or if memory is extremely limited (decrease it).

### 6.20.5.13 max\_receive\_ports

#### NAME

[DDS\\_DomainParticipantResourceLimitsQosPolicy::max\\_receive\\_ports](#)

#### DESCRIPTION

RTI Connext DDS Micro listens for discovery data and user data on different transports as specified by the user. The [DDS\\_DomainParticipantResourceLimitsQosPolicy::max\\_receive\\_ports](#) resource-limit sets the maximum number of addresses that can be listened on. As a rule of thumb, this resource limit can be determined as follows:

- A user-traffic unicast port counts as 1.
- A meta-traffic unicast port counts as 1.
- A user-traffic multicast address counts as 1.
- A meta-traffic multicast address counts as 1.

## EXAMPLES

Consider the typical case with the UDP transport, using unicast and multicast for discovery data and unicast for user data:

- Discovery data needs two ports.
- User data needs one port.

Thus, this requires 3 resources.

Consider a case with the UDP transport, using unicast and multicast for discovery data and user data:

- Discovery data needs two ports.
- User data needs two ports.

Thus, this requires 4 resources.

Consider a case with the UDP transport, using unicast and 2 multicast addresses each for discovery data and user data:

- Discovery data needs three ports.
- User data needs three ports.

Thus, this requires 6 resources.

## NOTES

The default value has been set to cover most cases. This resource-limit should only be changed if a lot of interfaces are supported (increase it) or if memory is extremely limited (decrease it).

### 6.20.6 DataReaderQos

#### Topics

- [max\\_instances](#)
- [max\\_remote\\_writers\\_per\\_instance](#)
- [max\\_samples](#)
- [max\\_samples\\_per\\_instance](#)
- [max\\_remote\\_writers](#)
- [max\\_samples\\_per\\_remote\\_writer](#)
- [max\\_routes\\_per\\_writer](#)
- [max\\_outstanding\\_reads](#)
- [max\\_fragmented\\_samples](#)
- [max\\_fragmented\\_samples\\_per\\_remote\\_writer](#)

### 6.20.6.1 Detailed Description

The [DDS\\_DataReaderQos](#) controls the resources used by the [DDSDataReader](#). Each [DDSDataReader](#) allocates its own resources, even [DDSDataReader](#)'s of the same [DDSTopic](#). For this reason is it advised to limit the number of [DDSDataReader](#)'s per [DDSTopic](#) to one.

### 6.20.6.2 max\_instances

#### NAME

[DDS\\_ResourceLimitsQosPolicy::max\\_instances](#)

#### DESCRIPTION

[DDS\\_ResourceLimitsQosPolicy::max\\_instances](#) limits the maximum number of instances a [DDSDataReader](#) can manage. Instance state is one of the most expensive resources to maintain as well as often being the highest one.

The resources allocated by an instance is normally only released when an instance is disposed *and* all known owners have unregistered the instance, please refer to [FooDataWriter::unregister\\_instance](#) and [FooDataWriter::dispose](#).

The [DDS\\_DataReaderResourceLimitsQosPolicy::instance\\_replacement](#) policy is useful to maintain state for only the last N instances and when it is not necessary to keep samples in the [DDSDataReader](#) history cache. When the [DDS\\_DataReaderResourceLimitsQosPolicy::instance\\_replacement](#) policy is set to replace the oldest instance ([DDS\\_REPLACE\\_OLDEST\\_INSTANCE\\_REPLACEMENT\\_QOS](#)), information about the oldest instance is automatically removed and the newly discovered instance is received. The disadvantage is that [DDS\\_NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#) state cannot be maintained. The [DDS\\_NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) is maintained for an instance that is accepted (either there is space or another instance is replaced), regardless of whether any samples have previously been received.

#### SEE ALSO

[DDS\\_DataReaderResourceLimitsInstanceReplacementKind](#) instance\_replacement

### 6.20.6.3 max\_remote\_writers\_per\_instance

#### NAME

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_remote\\_writers\\_per\\_instance](#)

#### DESCRIPTION

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_remote\\_writers\\_per\\_instance](#) limits the maximum number of remote [DDSDataWriter](#) entities a [DDSDataReader](#) maintains state for per instance. If this resource-limit is lower than the actual number of remote datawriters updating an instance only state for up to [DDS\\_DataReaderResourceLimitsQosPolicy::max\\_remote\\_writers\\_per\\_instance](#) is maintained. It is not guaranteed to be the last [DDS\\_DataReaderResourceLimitsQosPolicy::max\\_remote\\_writers\\_per\\_instance](#) though. However, the last remote [DDSDataWriter](#) to update an instance is *always* saved. When an instance is no longer updated by a known, to the DataReader, DataWriter, [DDS\\_NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#) state is signaled on the [DDSDataReader](#). Thus, if this resource-limit is lower than the actual number of remote datawriters updating an instance, [DDS\\_NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#) may be signaled even though there may be one or more datawriters still updating an instance. In this case the instance will go back to the [DDS\\_ALIVE\\_INSTANCE\\_STATE](#).

#### 6.20.6.4 max\_samples

##### NAME

[DDS\\_ResourceLimitsQosPolicy::max\\_samples](#)

##### DESCRIPTION

[DDS\\_ResourceLimitsQosPolicy::max\\_samples](#) limits the maximum number of samples a [DDSDataReader](#) can allocate and cache. Samples are cached before they are made available to the application, for example when samples are received out-of-order. Thus, this resource-limit may impact throughput and latency.

What this value should be set to is determined by a number of factors:

- For best-effort it can be set to 1 since samples are not cached. However, if data is polled or samples are loaned longer than the update frequency this resource-limit should be increased to prevent samples from being reused before the application has a chance to access the data (along with [DDS\\_HistoryQosPolicy::depth](#)).
- For reliable data the value should take into account the data-rate, the heartbeat rate of the [DDSDataWriter](#) and the number of piggy-backed heartbeats.

##### SEE ALSO

[DDS\\_HistoryQosPolicy::depth](#), [DDS\\_DataWriterQos](#)

#### 6.20.6.5 max\_samples\_per\_instance

##### NAME

[DDS\\_ResourceLimitsQosPolicy::max\\_samples\\_per\\_instance](#)

##### DESCRIPTION

[DDS\\_ResourceLimitsQosPolicy::max\\_samples\\_per\\_instance](#) limits the maximum number of samples that can be cached for a single instance and must be less than or equal to `max_samples` and greater or equal to the history depth. When [DDS\\_ResourceLimitsQosPolicy::max\\_samples\\_per\\_instance](#) is higher than depth it makes it possible to loan samples and receive up to depth samples before returning loaned samples.

When this resource-limit or [DDS\\_HistoryQosPolicy::depth](#) is exceeded and the [DDS\\_HistoryQosPolicy::kind](#) is [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#), the oldest sample is removed from the cache.

#### 6.20.6.6 max\_remote\_writers

##### NAME

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_remote\\_writers](#)

##### DESCRIPTION

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_remote\\_writers](#) limits the maximum number of remote datawriters the [DDSDataReader](#) can receive data from regardless of which instance it is.

##### NOTES

Resources controlled by this limit can only be reused when a remote DataWriter is either deleted or loses liveliness.

#### 6.20.6.7 max\_samples\_per\_remote\_writer

##### NAME

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_samples\\_per\\_remote\\_writer](#)

##### DESCRIPTION

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_samples\\_per\\_remote\\_writer](#) limits the maximum number of samples that can be cached per remote [DDSDataWriter](#) the [DDSDataReader](#) is receiving data from. This resource-limit is useful in preventing one [DDSDataWriter](#) from starving other [DDSDataWriter](#). NOTE↵: The samples cached from a [DDSDataWriter](#) is allocated from the same sample pool controlled by [DDS\\_ResourceLimitsQosPolicy::max\\_samples](#).

#### 6.20.6.8 max\_routes\_per\_writer

##### NAME

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_routes\\_per\\_writer](#)

##### DESCRIPTION

Each [DDSDataReader](#) maintains information about the state of its peer [DDSDataWriter](#) (those it has matched with). Part of this state is which locators (or destination addresses) it should use to send data to a particular [DDSDataWriter](#). [DDS\\_DataReaderResourceLimitsQosPolicy::max\\_routes\\_per\\_writer](#) limits the maximum number of routes that can be saved per [DDSDataWriter](#).

##### NOTES

This resource-limit is shared across all matched [DDSDataWriter](#) per [DDSDataReader](#). Thus, if [DDS\\_DataReaderResourceLimitsQosPolicy::max\\_routes\\_per\\_writer](#) is 2 and [DDS\\_DataReaderResourceLimitsQosPolicy::max\\_routes\\_per\\_writer](#) is 4, a total of 8 routes can be saved for both datawriters. One [DDSDataWriter](#) may have 6 routes and the other 2.



#### 6.20.6.9 max\_outstanding\_reads

##### NAME

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_outstanding\\_reads](#)

##### DESCRIPTION

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_outstanding\\_reads](#) limits the maximum number of simultaneous loans an application can make at any given time using the [FooDataReader::read\(\)](#) and [FooDataReader::take\(\)](#) APIs.

Typically a value of 1 is sufficient. It is only necessary to increase this value if an application thread will access the same cache before another thread has returned the loan.

#### 6.20.6.10 max\_fragmented\_samples

##### NAME

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_fragmented\\_samples](#)

##### DESCRIPTION

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_fragmented\\_samples](#) limits the maximum number of fragmented samples a [DDSDataReader](#) can cache while waiting for all fragments. A fragmented sample must be fully received before it can be deserialized. If this resource limit is reached, new fragmented samples will be rejected.

A default value equal to [DDS\\_ResourceLimitsQosPolicy::max\\_samples](#) will give the best results. However, if the data type is variable-sized where some samples may be fragmented and others not, this value can be set lower.

#### 6.20.6.11 max\_fragmented\_samples\_per\_remote\_writer

##### NAME

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_fragmented\\_samples\\_per\\_remote\\_writer](#)

##### DESCRIPTION

[DDS\\_DataReaderResourceLimitsQosPolicy::max\\_fragmented\\_samples\\_per\\_remote\\_writer](#) limits the maximum number of fragmented samples per matched [DDSDataWriter](#). This is useful to prevent one [DDSDataWriter](#) from using all the samples allocated based on [DDS\\_DataReaderResourceLimitsQosPolicy::max\\_fragmented\\_samples](#).

This resource limit does not allocate any resources. The resources used by a matched [DDSDataWriter](#) are allocated from the [DDS\\_DataReaderResourceLimitsQosPolicy::max\\_fragmented\\_samples](#) resources.

### 6.20.7 DataWriterQos

#### Topics

- [max\\_samples](#)
- [max\\_samples\\_per\\_instance](#)
- [max\\_instances](#)
- [max\\_routes\\_per\\_reader](#)
- [max\\_remote\\_readers](#)

#### 6.20.7.1 Detailed Description

The [DDS\\_DataWriterQos](#) controls the resources allocated by a [DDSDataWriter](#). Each [DDSDataWriter](#) allocates its own resources, even datawriters of the same [DDSTopic](#). For this reason is it advised to limit the number of datawriters per topic to one.

#### 6.20.7.2 max\_samples

##### NAME

[DDS\\_ResourceLimitsQosPolicy::max\\_samples](#)

##### DESCRIPTION

[DDS\\_ResourceLimitsQosPolicy::max\\_samples](#) limits the maximum number of samples a [DDSDataWriter](#) can allocate and maintain state for at any given time. Every time a write operation is called, a sample is allocated from the cache and filled in with user-data.

A sample allocated from the pool controlled by this resource-limit can be reused based on the following rules:

- If the [DDSDataWriter](#) is using [DDS\\_BEST\\_EFFORT\\_RELIABILITY\\_QOS](#) effort the sample is returned for reuse after the transport returns from the write call.
- If the DataWriter is using [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#) and [DDS\\_VOLATILE\\_DURABILITY\\_QOS](#), the sample is returned when all known DataReaders have acknowledged the sample or the DataWriter has given up on delivering the sample (controlled by [DDS\\_RtpsReliableWriterProtocol\\_t::max\\_heartbeat\\_retries](#)), whichever happens first.
- If the DataWriter is using [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#) and not using [DDS\\_VOLATILE\\_DURABILITY\\_QOS](#), the sample is returned to the pool for reuse only if the history depth is exceeded and the [DDS\\_HistoryQosPolicy::kind](#) is set to [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#)

##### NOTES

For best-effort communication this resource-limit can be set to 1.

For reliable communication the value of this resource-limit determines the maximum number of samples a [DDSDataWriter](#) can keep in unacknowledged state before reusing potentially unacknowledged samples (for [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#)). For reliable data the value should take into account the data-rate, the [DDS\\_RtpsReliableWriterProtocol\\_t::heartbeat\\_period](#) and [DDS\\_RtpsReliableWriterProtocol\\_t::heartbeats\\_per\\_max\\_samples](#).

### 6.20.7.3 max\_samples\_per\_instance

#### NAME

[DDS\\_ResourceLimitsQosPolicy::max\\_samples\\_per\\_instance](#)

#### DESCRIPTION

[DDS\\_ResourceLimitsQosPolicy::max\\_samples\\_per\\_instance](#) limits the maximum number of samples that can be cached for a single instance and must be less than or equal to [max\\_samples](#) and greater or equal to the [DDS\\_HistoryQosPolicy::depth](#).

[DDS\\_HistoryQosPolicy::depth](#) determines how many samples are kept in case of [DDS\\_TRANSIENT\\_LOCAL\\_DURABILITY\\_QOS](#) durability and made available to late joining [DDSDataReader](#), while [DDS\\_ResourceLimitsQosPolicy::max\\_samples\\_per\\_instance](#) is the maximum number of *unacknowledged* samples the [DDSDataWriter](#) can keep per instance. Thus, if  $\text{max\_samples\_per\_instance} > \text{depth}$  it is possible to keep unacknowledged samples while limiting the number of samples made available to late joining DataReaders.

### 6.20.7.4 max\_instances

#### NAME

[DDS\\_ResourceLimitsQosPolicy::max\\_instances](#)

#### DESCRIPTION

[DDS\\_ResourceLimitsQosPolicy::max\\_instances](#) limits the maximum number of instances a [DDSDataWriter](#) can manage. An instance resource can only be reused when it is unregistered by the [DDSDataWriter](#).

### 6.20.7.5 max\_routes\_per\_reader

#### NAME

[DDS\\_DataWriterResourceLimitsQosPolicy::max\\_routes\\_per\\_reader](#)

#### DESCRIPTION

Each [DDSDataWriter](#) maintains information about the state of its peer DataReaders (those it has matched with). Part of this state is which locators (or destination addresses) it should use to send data to a particular [DDSDataReader](#). [DDS\\_DataWriterResourceLimitsQosPolicy::max\\_routes\\_per\\_reader](#) limits the number of routes that can be saved per [DDSDataReader](#).

#### NOTES

This resource-limit is shared across all matched DataReaders per [DDSDataWriter](#). Thus, if [DDS\\_DataWriterResourceLimitsQosPolicy::max\\_routes\\_per\\_reader](#) is 2 and [DDS\\_DataWriterResourceLimitsQosPolicy::max\\_routes\\_per\\_reader](#) is 4, a total of 8 routes can be saved for both DataReaders. One [DDSDataReader](#) may have 6 routes and the other 2.

### 6.20.7.6 max\_remote\_readers

#### NAME

[DDS\\_DataWriterResourceLimitsQosPolicy::max\\_remote\\_readers](#)

#### DESCRIPTION

[DDS\\_DataWriterResourceLimitsQosPolicy::max\\_remote\\_readers](#) limits the maximum number of DataReaders the [DDSDataWriter](#) can maintain state for.

## 6.20.8 OSAPI

### Topics

- [max\\_buffer\\_size](#)

### 6.20.8.1 Detailed Description

### 6.20.8.2 max\_buffer\_size

#### NAME

[OSAPI\\_LogProperty::max\\_buffer\\_size](#)

#### DESCRIPTION

This resource-limit limits the amount logging information that can be saved in bytes. When the log-buffer is full no more logging information is saved.

## 6.20.9 UDP Transport

### Topics

- [max\\_message\\_size](#)

### 6.20.9.1 Detailed Description

### 6.20.9.2 max\_message\_size

#### NAME

[UDP\\_InterfaceFactoryProperty::max\\_message\\_size](#)

#### DESCRIPTION

[UDP\\_InterfaceFactoryProperty::max\\_message\\_size](#) limits the size of the maximum message that can be received.

#### NOTES

RTI Connex Micro C++ API Version 4.2.0 supports RTPS fragmentation. Fragmentation will be applied when sending samples larger than the `max_message_size`.

The UDP transport allocates one buffer for each resource in [DDS\\_DomainParticipantResourceLimitsQosPolicy::max\\_receive\\_ports](#). Thus, the total amount of memory is  $(\text{max\_message\_size} * \text{max\_receive\_ports})$

## 6.20.10 Dynamic Participant Static Endpoint (DPSE)

### Topics

- [max\\_participant\\_locators](#)
- [max\\_locators\\_per\\_discovered\\_participant](#)

### 6.20.10.1 Detailed Description

The Dynamic Participant Static Endpoint (DPSE) discovery plugin creates one `DataWriter` and one `DataReader` for the `ParticipantBuiltinTopicData`. The memory for the plugin includes the memory allocated by the `DataReader` and `DataWriter`. The memory allocated by the properties must be added for total memory usage.

### 6.20.10.2 max\_participant\_locators

#### NAME

[DPSE\\_DiscoveryPluginProperty::max\\_participant\\_locators](#)

#### DESCRIPTION

The DPSE builtin participant topic `DataWriter` allocates `max_participant_locators` to send participant announcements to. The default value `DDS_LENGTH_AUTO` calculates `max_participant_locators` as `remote_participant_allocation + (DPSE_MAX_ANON_PARTICIPANT * (peer_length + 1))` where the constant `DPSE_MAX_ANON_PARTICIPANT = 6`.

A participant locator is defined as a unique `index@address` to send to where `index` is an integer  $\geq 0$  and only applies to unicast addresses. For multicast address the `index` is 0. This `index` is the highest `participant_id` that will receive participant announcements at the address.

The maximum number of addresses to send to is defined by the peer list (either specified in [DDS\\_DiscoveryQosPolicy::initial\\_peers](#) or by calling [DDSDomainParticipant::add\\_peer](#)). The maximum number of addresses needed is determined by adding all indices for all unicast addresses and adding one for each multicast address.

If memory is limited it is advised to set this value as low as possible.

**6.20.10.3 max\_locators\_per\_discovered\_participant****NAME**

[DPSE\\_DiscoveryPluginProperty::max\\_locators\\_per\\_discovered\\_participant](#)

**DESCRIPTION**

When a participant discovers another participant it must keep track of which addresses to respond to for each participant. This resource-limit sets the maximum number of addresses which can be saved.

**NOTES**

If two participants respond with the same addresses, for example multicast, only one address is saved.

**6.20.11 Dynamic Participant Dynamic Endpoint (DPDE)****Topics**

- [max\\_participant\\_locators](#)
- [max\\_locators\\_per\\_discovered\\_participant](#)
- [max\\_samples\\_per\\_builtin\\_endpoint\\_reader](#)

**6.20.11.1 Detailed Description**

The Dynamic Participant Dynamic Endpoint (DPSE) discovery plugin creates one DataWriter and one DataReader for each of the ParticipantBuiltinTopicData, PublicationBuiltinTopicData, and SubscriptionBuiltinTopicData. The memory for the plugin includes the memory allocated by these DataReaders and datawriters. The memory allocated by the properties must be added for total memory usage.

**6.20.11.2 max\_participant\_locators****NAME**

[DPDE\\_DiscoveryPluginProperty::max\\_participant\\_locators](#)

**DESCRIPTION**

The DPDE builtin participant topic DataWriter allocates max\_participant\_locators to send participant announcements to. The default value DDS\_LENGTH\_AUTO calculates max\_participant\_locators as remote\_participant\_allocation + (DPDE\_MAX\_ANON\_PARTICIPANT \* (peer\_length + 1)) where the constant DPDE\_MAX\_ANON\_PARTICIPANT = 6.

A participant locator is defined as a unique index@address to send to where index is an integer  $\geq 0$  and only applies to unicast addresses. For multicast address the index is 0. This index is the highest participant\_id that will receive participant announcements at the address.

The maximum number of addresses to send to is defined by the peer list (either specified in [DDS\\_DiscoveryQosPolicy::initial\\_peers](#) or by calling [DDSDomainParticipant::add\\_peer](#). The maximum number of addresses needed is determined by adding all indices for all unicast addresses and adding one for each multicast address.

If memory is limited it is advised to set this value as low as possible.

### 6.20.11.3 max\_locators\_per\_discovered\_participant

#### NAME

[DPDE\\_DiscoveryPluginProperty::max\\_locators\\_per\\_discovered\\_participant](#)

#### DESCRIPTION

When a participant discovers another participant it must keep track of which addresses to respond to for each participant. This resource-limit sets the maximum number of addresses which can be saved.

#### NOTES

If two participants respond with the same addresses, for example multicast, only one address is saved.

#### NAME

[DPDE\\_DiscoveryPluginProperty::max\\_locators\\_per\\_discovered\\_participant](#)

#### DESCRIPTION

When a participant discovers another participant it must keep track of which addresses to respond to for each participant. This resource-limit sets the maximum number of addresses which can be saved.

#### NOTES

If two participants respond with the same addresses, for example multicast, only one address is saved.

### 6.20.11.4 max\_samples\_per\_builtin\_endpoint\_reader

#### NAME

[DPDE\\_DiscoveryPluginProperty::max\\_samples\\_per\\_builtin\\_endpoint\\_reader](#)

#### DESCRIPTION

The builtin endpoint DataReaders are reliable and may cache samples received out-of-order to improve discovery performance. This resource-limit determines the maximum number of outstanding samples at any given time.

## 6.21 Log Codes

### Topics

- [OSAPI](#)  
*OSAPI. ModuleID = 0.*
- [REDA](#)  
*Real-time Efficient Data Structures and Algorithms. ModuleID = 1.*
- [DB](#)  
*Database. ModuleID = 2.*
- [RT](#)  
*RT. ModuleID = 3.*
- [CDR](#)  
*CDR. ModuleID = 5.*
- [NETIO](#)  
*Network I/O. ModuleID = 4.*
- [RTPS](#)  
*Real-Time Publish-Subscribe. ModuleID = 6.*
- [DDS\\_C](#)  
*DDS C. ModuleID = 7.*
- [WHSQ](#)  
*Writer History Shared Queue. ModuleID = 22.*
- [RH](#)  
*Reader History. ModuleID = 8.*
- [WH](#)  
*Writer History. ModuleID = 9.*
- [DPSE](#)  
*DPSE. ModuleID = 10.*
- [DPDE](#)  
*DPDE. ModuleID = 11.*
- [APPGEN](#)  
*APPGEN. ModuleID = 13.*
- [DDS\\_FILTER](#)  
*DDS Filter. ModuleID = 16.*

### 6.21.1 Detailed Description

This page contains references to all log codes reported by RTI Connext DDS Micro. Please refer to the [Debugging](#) section in the [User's Manual](#) for how to interpret log messages and [Log API](#) for the Log API.

### 6.21.2 OSAPI

OSAPI. ModuleID = 0.



## Macros

- `#define OSAPI_LOG_GET_NEXT_OBJECT_ID_EC (OSAPI_LOG_BASE + 1)`  
*Retrieving the next error code failed.*
- `#define OSAPI_LOG_SYSTEM_SET_PROPERTY_EC (OSAPI_LOG_BASE + 2)`  
*An error occurred while setting the system properties.*
- `#define OSAPI_LOG_SYSTEM_TIMER_START_EC (OSAPI_LOG_BASE + 4)`  
*An error occurred when starting the system timer.*
- `#define OSAPI_LOG_SYSTEM_TIMER_STOP_EC (OSAPI_LOG_BASE + 5)`  
*An error occurred when stopping the system timer.*
- `#define OSAPI_LOG_THREAD_NEW_EC (OSAPI_LOG_BASE + 6)`  
*An error when allocating the a thread object.*
- `#define OSAPI_LOG_THREAD_CREATE_EC (OSAPI_LOG_BASE + 7)`  
*An error when creating the a thread object.*
- `#define OSAPI_LOG_THREAD_SEM_EC (OSAPI_LOG_BASE + 8)`  
*An error when creating thread sync semaphore.*
- `#define OSAPI_LOG_THREAD_EXEC_CREATE_EC (OSAPI_LOG_BASE + 9)`  
*Failed to signal that a thread has been created.*
- `#define OSAPI_LOG_THREAD_EXEC_START_EC (OSAPI_LOG_BASE + 10)`  
*Failed to signal the start a created thread.*
- `#define OSAPI_LOG_THREAD_START_EC (OSAPI_LOG_BASE + 11)`  
*Failed to start a thread.*
- `#define OSAPI_LOG_THREAD_DESTROY_EC (OSAPI_LOG_BASE + 12)`  
*Failed to destroy a thread.*
- `#define OSAPI_LOG_THREAD_DESTROY_NO_START_EC (OSAPI_LOG_BASE + 13)`  
*Failed to start an unstarted thread being destroyed.*
- `#define OSAPI_LOG_THREAD_DESTROY_NO_WAKEUP_EC (OSAPI_LOG_BASE + 14)`  
*Failed wakeup of a thread being destroyed.*
- `#define OSAPI_LOG_THREAD_INIT_EC (OSAPI_LOG_BASE + 15)`  
*Failed initializing a thread.*
- `#define OSAPI_LOG_THREAD_SCHEDPARAM_EC (OSAPI_LOG_BASE + 16)`  
*Failed to set scheduling policy of a thread.*
- `#define OSAPI_LOG_THREAD_GET_POLICY_EC (OSAPI_LOG_BASE + 17)`  
*Failed to get the scheduling policy of a thread.*
- `#define OSAPI_LOG_THREAD_POLICY_DIFFER_EC (OSAPI_LOG_BASE + 18)`  
*Mismatch of scheduling policy of a created thread and the application thread.*
- `#define OSAPI_LOG_THREAD_PRIORITY_MAP_EC (OSAPI_LOG_BASE + 19)`  
*Failed to map to native thread priority values.*
- `#define OSAPI_LOG_TIMER_DELETE_EC (OSAPI_LOG_BASE + 20)`  
*Failed to delete the Timer object.*
- `#define OSAPI_LOG_TIMER_TICK_MUTEX_EC (OSAPI_LOG_BASE + 22)`  
*Failed taking or giving the Timer mutex.*
- `#define OSAPI_LOG_TIMER_GET_USER_DATA_EPOCH_EC (OSAPI_LOG_BASE + 27)`  
*Failed to return user data for a timeout due to mismatched epochs.*
- `#define OSAPI_LOG_TIMER_NEW_EC (OSAPI_LOG_BASE + 28)`  
*Failed to allocate memory for a new Timer.*
- `#define OSAPI_LOG_TIMER_NEW_ENTRY_EC (OSAPI_LOG_BASE + 29)`

- Failed to allocate memory for a new Timer entry.*

  - #define `OSAPI_LOG_TIMER_NEW_WHEEL_EC` (`OSAPI_LOG_BASE` + 30)
- Failed to allocate memory for a new Timer wheel.*

  - #define `OSAPI_LOG_TIMER_NEW_MUTEX_EC` (`OSAPI_LOG_BASE` + 31)
- Failed to create a new Timer mutex.*

  - #define `OSAPI_LOG_TIMER_NEW_START_TIMER_EC` (`OSAPI_LOG_BASE` + 32)
- Failed to start a new Timer being created.*

  - #define `OSAPI_LOG_TIMER_DELETE_STOP_TIMER_EC` (`OSAPI_LOG_BASE` + 33)
- Failed to stop a Timer being deleted.*

  - #define `OSAPI_LOG_TIMER_DELETE_MUTEX_EC` (`OSAPI_LOG_BASE` + 34)
- Failed to delete the Timer mutex.*

  - #define `OSAPI_LOG_TIMER_MUTEX_EC` (`OSAPI_LOG_BASE` + 35)
- Failed to take or give a Timer mutex.*

  - #define `OSAPI_LOG_SEMAPHORE_DELETE_EC` (`OSAPI_LOG_BASE` + 36)
- Failed to delete a semaphore.*

  - #define `OSAPI_LOG_SEMAPHORE_NEW_EC` (`OSAPI_LOG_BASE` + 37)
- Failed to create a semaphore.*

  - #define `OSAPI_LOG_SEMAPHORE_NEW_INIT_EC` (`OSAPI_LOG_BASE` + 38)
- Failed to initialize a new semaphore.*

  - #define `OSAPI_LOG_SEMAPHORE_GIVE_EC` (`OSAPI_LOG_BASE` + 39)
- Failed to give a semaphore.*

  - #define `OSAPI_LOG_SEMAPHORE_TAKE_EC` (`OSAPI_LOG_BASE` + 40)
- Failed to take a semaphore.*

  - #define `OSAPI_LOG_MUTEX_DELETE_EC` (`OSAPI_LOG_BASE` + 41)
- Failed to delete a mutex.*

  - #define `OSAPI_LOG_MUTEX_NEW_EC` (`OSAPI_LOG_BASE` + 42)
- Failed to create a mutex.*

  - #define `OSAPI_LOG_MUTEX_TAKE_EC` (`OSAPI_LOG_BASE` + 43)
- Failed to take a mutex.*

  - #define `OSAPI_LOG_MUTEX_GIVE_EC` (`OSAPI_LOG_BASE` + 44)
- Failed to give a mutex.*

  - #define `OSAPI_LOG_MUTEX_INIT_EC` (`OSAPI_LOG_BASE` + 45)
- Failed to initialize a mutex.*

  - #define `OSAPI_LOG_HEAP_INTERNAL_ALLOCATE_EC` (`OSAPI_LOG_BASE` + 46)
- Failed to allocate a buffer from the heap.*

  - #define `OSAPI_LOG_SYSTEM_GET_TIME_EC` (`OSAPI_LOG_BASE` + 48)
- Failed to get current system time.*

  - #define `OSAPI_LOG_LAST_RECORDED_ERROR_EC` (`OSAPI_LOG_BASE` + 49)
- Return the last recorded error-code for the calling thread.*

  - #define `OSAPI_LOG_SET_THREAD_NAME_EC` (`OSAPI_LOG_BASE` + 50)
- Failed to set the thread name in the OS.*

  - #define `OSAPI_LOG_SEM_OPEN_EC` (`OSAPI_LOG_BASE` + 52)
- Failure during call VxWorks semOpen(...) or Posix sem\_open(...) when managing shared memory segments, semaphores, or mutexes.*

  - #define `OSAPI_LOG_SEM_INFO_GET_EC` (`OSAPI_LOG_BASE` + 53)
- Failure during VxWorks call semInfoGet when managing shared memory segments, semaphores, or mutexes.*

  - #define `OSAPI_LOG_SD_UNMAP_EC` (`OSAPI_LOG_BASE` + 54)

- Failure during VxWorks call sdUnmap when managing shared memory segments, semaphores, or mutexes.*

  - #define `OSAPI_LOG_SD_OPEN_EC` (`OSAPI_LOG_BASE` + 55)
- Failure during VxWorks call sdOpen when managing shared memory segments, semaphores, or mutexes.*

  - #define `OSAPI_LOG_SEM_GIVE_EC` (`OSAPI_LOG_BASE` + 56)
- Failure during VxWorks call semGive when managing shared memory segments, semaphores, or mutexes.*

  - #define `OSAPI_LOG_SEM_TAKE_EC` (`OSAPI_LOG_BASE` + 57)
- Failure during VxWorks call semTake when managing shared memory segments, semaphores, or mutexes.*

  - #define `OSAPI_LOG_UNKNOWN_SEMAPHORE_TYPE_EC` (`OSAPI_LOG_BASE` + 58)
- Trying to perform an operation an unknown semmutex type.*

  - #define `OSAPI_LOG_SEGMENT_KEY_ALREADY_EXISTS_EC` (`OSAPI_LOG_BASE` + 59)
- Failed to create segment due to a segment already existing.*

  - #define `OSAPI_LOG_SEGMENT_KEY_DOESNT_EXIST_EC` (`OSAPI_LOG_BASE` + 60)
- Segment trying to be attached to does not exist.*

  - #define `OSAPI_LOG_SEGMENT_DETACH_FAILED_EC` (`OSAPI_LOG_BASE` + 61)
- Failed to detach from shared memory segment.*

  - #define `OSAPI_LOG_SHMEM_ATTACH_FAILED_EC` (`OSAPI_LOG_BASE` + 62)
- Attaching to shared memory segment/mutex/signaling semaphore failed.*

  - #define `OSAPI_LOG_SHMEM_GIVE_FAILED_EC` (`OSAPI_LOG_BASE` + 63)
- Failed to give shared memory mutex/signaling semaphore.*

  - #define `OSAPI_LOG_SHMEM_TAKE_FAILED_EC` (`OSAPI_LOG_BASE` + 64)
- Failed to take shared memory mutex/signaling semaphore.*

  - #define `OSAPI_LOG_SHMEM_SEM_DELETE_FAILED_EC` (`OSAPI_LOG_BASE` + 65)
- Failed to delete shared memory semaphore or mutex.*

  - #define `OSAPI_LOG_SHMEM_SEM_MUTEX_CREATED_EC` (`OSAPI_LOG_BASE` + 66)
- Error during shared memory semaphore creation/deletion.*

  - #define `OSAPI_LOG_SHMEM_SEGMENT_FILE_MAP_FAILED_EC` (`OSAPI_LOG_BASE` + 67)
- Windows specific API MapViewOfFile failed.*

  - #define `OSAPI_LOG_SHMEM_SET_DESCRIPTOR_EC` (`OSAPI_LOG_BASE` + 68)
- Windows specific error during setting security descriptor.*

  - #define `OSAPI_LOG_SHMEM_INIT_DESCRIPTOR_EC` (`OSAPI_LOG_BASE` + 69)
- Windows specific error during initialization of security descriptor.*

  - #define `OSAPI_LOG_SHMEM_FTRUNCATE_EC` (`OSAPI_LOG_BASE` + 70)
- Posix specific error during ftruncate(...) call.*

  - #define `OSAPI_LOG_SHMEM_MMAP_EC` (`OSAPI_LOG_BASE` + 71)
- Posix specific error during mmap(...) call.*

  - #define `OSAPI_LOG_SHMEM_SHM_OPEN_EC` (`OSAPI_LOG_BASE` + 72)
- Posix specific error during shm\_open(...) call.*

  - #define `OSAPI_LOG_SHMEM_MUNMAP_EC` (`OSAPI_LOG_BASE` + 73)
- Posix specific error during munmap(...) call.*

  - #define `OSAPI_LOG_CLOSE_EC` (`OSAPI_LOG_BASE` + 74)
- Posix specific error during close(...) call.*

  - #define `OSAPI_LOG_SHMEM_UNLINK_EC` (`OSAPI_LOG_BASE` + 76)
- Posix specific error during unlink(...) call.*

  - #define `OSAPI_LOG_EXHAUSTED_POSIX_SEM_LIMIT_EC` (`OSAPI_LOG_BASE` + 77)
- System ran out of space to create semaphores. Consider increasing Posix sem limit.*

  - #define `OSAPI_LOG_SHMEM_SEM_WAIT_EC` (`OSAPI_LOG_BASE` + 78)
- Posix specific error during unlink(...) call.*

- #define `OSAPI_LOG_SHMEM_SEM_UNLINK_EC` (`OSAPI_LOG_BASE` + 79)  
*Posix specific error during unlink(...) call.*
- #define `OSAPI_LOG_SHMEM_POSIX_GIVE_EC` (`OSAPI_LOG_BASE` + 80)  
*Posix specific error during os specific implementation of a semaphore mutex give.*
- #define `OSAPI_LOG_THREAD_SEMPOOL_EXCEEDED_EC` (`OSAPI_LOG_BASE` + 90)  
*Failed to add a semaphore to the thread semaphore pool.*
- #define `OSAPI_LOG_THREAD_SEMPOOL_INVALID_SEM_EC` (`OSAPI_LOG_BASE` + 91)  
*Failed to delete a semaphore from the thread semaphore pool.*
- #define `OSAPI_LOG_SHMEM_SIZE_EXCEEDED_EC` (`OSAPI_LOG_BASE` + 92)  
*Trying to create a shared memory segment that is greater than  $(2^{31}) - 1$ . The middleware cannot support shared memory segments created than ~ 2 GB even though the operating system may.*
- #define `OSAPI_LOG_THREAD_JOIN_EC` (`OSAPI_LOG_BASE` + 95)  
*Failed to join a thread.*
- #define `OSAPI_LOG_THREAD_GET_MAX_PRIORITY_EC` (`OSAPI_LOG_BASE` + 96)  
*Failed to get the maximum priority value for a scheduling policy.*
- #define `OSAPI_LOG_SHMEM_FSTAT_EC` (`OSAPI_LOG_BASE` + 97)  
*POSIX specific error during fstat(...) call.*
- #define `OSAPI_LOG_SHMEM_MUTEX_INIT_TIMEOUT_EC` (`OSAPI_LOG_BASE` + 97)  
*Timed out waiting for a shared memory mutex to be initialized.*
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_INIT_EC` (`OSAPI_LOG_BASE` + 200)  
*Failed to create a pthread mutex attribute.*
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPSHARED_EC` (`OSAPI_LOG_BASE` + 201)  
*Failed to set the pthread mutex attribute to be process shared.*
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETROBUST_EC` (`OSAPI_LOG_BASE` + 202)  
*Failed to set the pthread mutex attribute to be robust.*
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_DESTROY_EC` (`OSAPI_LOG_BASE` + 203)  
*Failed to destroy the pthread mutex attribute.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_INIT_EC` (`OSAPI_LOG_BASE` + 204)  
*Failed to initialize a pthread mutex.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_DESTROY_EC` (`OSAPI_LOG_BASE` + 205)  
*Failed to destroy a pthread mutex.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_LOCK_EC` (`OSAPI_LOG_BASE` + 206)  
*Failed to lock a pthread mutex.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_UNLOCK_EC` (`OSAPI_LOG_BASE` + 207)  
*Failed to unlock a pthread mutex.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_CONSISTENT_EC` (`OSAPI_LOG_BASE` + 208)  
*Failed to set the pthread mutex to be consistent.*
- #define `OSAPI_LOG_PTHREAD_CONDATTR_INIT_EC` (`OSAPI_LOG_BASE` + 209)  
*Failed to create a pthread condition variable attribute.*
- #define `OSAPI_LOG_PTHREAD_CONDATTR_SETPSHARED_EC` (`OSAPI_LOG_BASE` + 210)  
*Failed to set the pthread condition variable attribute to be process shared.*
- #define `OSAPI_LOG_PTHREAD_CONDATTR_DESTROY_EC` (`OSAPI_LOG_BASE` + 211)  
*Failed to destroy the pthread condition variable attribute.*
- #define `OSAPI_LOG_PTHREAD_COND_INIT_EC` (`OSAPI_LOG_BASE` + 212)  
*Failed to initialize a pthread condition variable.*
- #define `OSAPI_LOG_PTHREAD_COND_DESTROY_EC` (`OSAPI_LOG_BASE` + 213)  
*Failed to destroy a pthread condition variable.*

- `#define OSAPI_LOG_PTHREAD_COND_WAIT_EC (OSAPI_LOG_BASE + 214)`  
*Failed to wait on a pthread condition variable.*
- `#define OSAPI_LOG_PTHREAD_COND_SIGNAL_EC (OSAPI_LOG_BASE + 215)`  
*Failed to signal a pthread condition variable.*
- `#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETTYPE_EC (OSAPI_LOG_BASE + 216)`  
*Failed to set the type of a pthread mutex attribute.*
- `#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETPROTOCOL_EC (OSAPI_LOG_BASE + 217)`  
*Failed to set the protocol of a pthread mutex attribute.*
- `#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETPRIOCEILING_EC (OSAPI_LOG_BASE + 218)`  
*Failed to set the priority ceiling of a pthread mutex attribute.*

#### 6.21.2.1 Detailed Description

OSAPI. ModuleID = 0.

#### 6.21.2.2 Macro Definition Documentation

##### OSAPI\_LOG\_GET\_NEXT\_OBJECT\_ID\_EC

```
#define OSAPI_LOG_GET_NEXT_OBJECT_ID_EC (OSAPI_LOG_BASE + 1)
```

Retrieving the next error code failed.

##### OSAPI\_LOG\_SYSTEM\_SET\_PROPERTY\_EC

```
#define OSAPI_LOG_SYSTEM_SET_PROPERTY_EC (OSAPI_LOG_BASE + 2)
```

An error occurred while setting the system properties.

##### OSAPI\_LOG\_SYSTEM\_TIMER\_START\_EC

```
#define OSAPI_LOG_SYSTEM_TIMER_START_EC (OSAPI_LOG_BASE + 4)
```

An error occurred when starting the system timer.

##### OSAPI\_LOG\_SYSTEM\_TIMER\_STOP\_EC

```
#define OSAPI_LOG_SYSTEM_TIMER_STOP_EC (OSAPI_LOG_BASE + 5)
```

An error occurred when stopping the system timer.

**OSAPI\_LOG\_THREAD\_NEW\_EC**

```
#define OSAPI_LOG_THREAD_NEW_EC (OSAPI_LOG_BASE + 6)
```

An error when allocating the a thread object.

**OSAPI\_LOG\_THREAD\_CREATE\_EC**

```
#define OSAPI_LOG_THREAD_CREATE_EC (OSAPI_LOG_BASE + 7)
```

An error when creating the a thread object.

**OSAPI\_LOG\_THREAD\_SEM\_EC**

```
#define OSAPI_LOG_THREAD_SEM_EC (OSAPI_LOG_BASE + 8)
```

An error when creating thread sync semaphore.

**OSAPI\_LOG\_THREAD\_EXEC\_CREATE\_EC**

```
#define OSAPI_LOG_THREAD_EXEC_CREATE_EC (OSAPI_LOG_BASE + 9)
```

Failed to signal that a thread has been created.

**OSAPI\_LOG\_THREAD\_EXEC\_START\_EC**

```
#define OSAPI_LOG_THREAD_EXEC_START_EC (OSAPI_LOG_BASE + 10)
```

Failed to signal the start a created thread.

**OSAPI\_LOG\_THREAD\_START\_EC**

```
#define OSAPI_LOG_THREAD_START_EC (OSAPI_LOG_BASE + 11)
```

Failed to start a thread.

**OSAPI\_LOG\_THREAD\_DESTROY\_EC**

```
#define OSAPI_LOG_THREAD_DESTROY_EC (OSAPI_LOG_BASE + 12)
```

Failed to destroy a thread.

**OSAPI\_LOG\_THREAD\_DESTROY\_NO\_START\_EC**

```
#define OSAPI_LOG_THREAD_DESTROY_NO_START_EC (OSAPI_LOG_BASE + 13)
```

Failed to start an unstarted thread being destroyed.

**OSAPI\_LOG\_THREAD\_DESTROY\_NO\_WAKEUP\_EC**

```
#define OSAPI_LOG_THREAD_DESTROY_NO_WAKEUP_EC (OSAPI_LOG_BASE + 14)
```

Failed wakeup of a thread being destroyed.

**OSAPI\_LOG\_THREAD\_INIT\_EC**

```
#define OSAPI_LOG_THREAD_INIT_EC (OSAPI_LOG_BASE + 15)
```

Failed initializing a thread.

**OSAPI\_LOG\_THREAD\_SCHEDPARAM\_EC**

```
#define OSAPI_LOG_THREAD_SCHEDPARAM_EC (OSAPI_LOG_BASE + 16)
```

Failed to set scheduling policy of a thread.

**OSAPI\_LOG\_THREAD\_GET\_POLICY\_EC**

```
#define OSAPI_LOG_THREAD_GET_POLICY_EC (OSAPI_LOG_BASE + 17)
```

Failed to get the scheduling policy of a thread.

**OSAPI\_LOG\_THREAD\_POLICY\_DIFFER\_EC**

```
#define OSAPI_LOG_THREAD_POLICY_DIFFER_EC (OSAPI_LOG_BASE + 18)
```

Mismatch of scheduling policy of a created thread and the application thread.

**OSAPI\_LOG\_THREAD\_PRIORITY\_MAP\_EC**

```
#define OSAPI_LOG_THREAD_PRIORITY_MAP_EC (OSAPI_LOG_BASE + 19)
```

Failed to map to native thread priority values.

**OSAPI\_LOG\_TIMER\_DELETE\_EC**

```
#define OSAPI_LOG_TIMER_DELETE_EC (OSAPI_LOG_BASE + 20)
```

Failed to delete the Timer object.

**OSAPI\_LOG\_TIMER\_TICK\_MUTEX\_EC**

```
#define OSAPI_LOG_TIMER_TICK_MUTEX_EC (OSAPI_LOG_BASE + 22)
```

Failed taking or giving the Timer mutex.

**OSAPI\_LOG\_TIMER\_GET\_USER\_DATA\_EPOCH\_EC**

```
#define OSAPI_LOG_TIMER_GET_USER_DATA_EPOCH_EC (OSAPI_LOG_BASE + 27)
```

Failed to return user data for a timeout due to mismatched epochs.

**OSAPI\_LOG\_TIMER\_NEW\_EC**

```
#define OSAPI_LOG_TIMER_NEW_EC (OSAPI_LOG_BASE + 28)
```

Failed to allocate memory for a new Timer.

**OSAPI\_LOG\_TIMER\_NEW\_ENTRY\_EC**

```
#define OSAPI_LOG_TIMER_NEW_ENTRY_EC (OSAPI_LOG_BASE + 29)
```

Failed to allocate memory for a new Timer entry.

**OSAPI\_LOG\_TIMER\_NEW\_WHEEL\_EC**

```
#define OSAPI_LOG_TIMER_NEW_WHEEL_EC (OSAPI_LOG_BASE + 30)
```

Failed to allocate memory for a new Timer wheel.

**OSAPI\_LOG\_TIMER\_NEW\_MUTEX\_EC**

```
#define OSAPI_LOG_TIMER_NEW_MUTEX_EC (OSAPI_LOG_BASE + 31)
```

Failed to create a new Timer mutex.



**OSAPI\_LOG\_TIMER\_NEW\_START\_TIMER\_EC**

```
#define OSAPI_LOG_TIMER_NEW_START_TIMER_EC (OSAPI_LOG_BASE + 32)
```

Failed to start a new Timer being created.

**OSAPI\_LOG\_TIMER\_DELETE\_STOP\_TIMER\_EC**

```
#define OSAPI_LOG_TIMER_DELETE_STOP_TIMER_EC (OSAPI_LOG_BASE + 33)
```

Failed to stop a Timer being deleted.

**OSAPI\_LOG\_TIMER\_DELETE\_MUTEX\_EC**

```
#define OSAPI_LOG_TIMER_DELETE_MUTEX_EC (OSAPI_LOG_BASE + 34)
```

Failed to delete the Timer mutex.

**OSAPI\_LOG\_TIMER\_MUTEX\_EC**

```
#define OSAPI_LOG_TIMER_MUTEX_EC (OSAPI_LOG_BASE + 35)
```

Failed to take or give a Timer mutex.

**OSAPI\_LOG\_SEMAPHORE\_DELETE\_EC**

```
#define OSAPI_LOG_SEMAPHORE_DELETE_EC (OSAPI_LOG_BASE + 36)
```

Failed to delete a semaphore.

**OSAPI\_LOG\_SEMAPHORE\_NEW\_EC**

```
#define OSAPI_LOG_SEMAPHORE_NEW_EC (OSAPI_LOG_BASE + 37)
```

Failed to create a semaphore.

**OSAPI\_LOG\_SEMAPHORE\_NEW\_INIT\_EC**

```
#define OSAPI_LOG_SEMAPHORE_NEW_INIT_EC (OSAPI_LOG_BASE + 38)
```

Failed to initialize a new semaphore.

**OSAPI\_LOG\_SEMAPHORE\_GIVE\_EC**

```
#define OSAPI_LOG_SEMAPHORE_GIVE_EC (OSAPI_LOG_BASE + 39)
```

Failed to give a semaphore.

**OSAPI\_LOG\_SEMAPHORE\_TAKE\_EC**

```
#define OSAPI_LOG_SEMAPHORE_TAKE_EC (OSAPI_LOG_BASE + 40)
```

Failed to take a semaphore.

**OSAPI\_LOG\_MUTEX\_DELETE\_EC**

```
#define OSAPI_LOG_MUTEX_DELETE_EC (OSAPI_LOG_BASE + 41)
```

Failed to delete a mutex.

**OSAPI\_LOG\_MUTEX\_NEW\_EC**

```
#define OSAPI_LOG_MUTEX_NEW_EC (OSAPI_LOG_BASE + 42)
```

Failed to create a mutex.

**OSAPI\_LOG\_MUTEX\_TAKE\_EC**

```
#define OSAPI_LOG_MUTEX_TAKE_EC (OSAPI_LOG_BASE + 43)
```

Failed to take a mutex.

**OSAPI\_LOG\_MUTEX\_GIVE\_EC**

```
#define OSAPI_LOG_MUTEX_GIVE_EC (OSAPI_LOG_BASE + 44)
```

Failed to give a mutex.

**OSAPI\_LOG\_MUTEX\_INIT\_EC**

```
#define OSAPI_LOG_MUTEX_INIT_EC (OSAPI_LOG_BASE + 45)
```

Failed to initialize a mutex.

**OSAPI\_LOG\_HEAP\_INTERNAL\_ALLOCATE\_EC**

```
#define OSAPI_LOG_HEAP_INTERNAL_ALLOCATE_EC (OSAPI_LOG_BASE + 46)
```

Failed to allocate a buffer from the heap.

**OSAPI\_LOG\_SYSTEM\_GET\_TIME\_EC**

```
#define OSAPI_LOG_SYSTEM_GET_TIME_EC (OSAPI_LOG_BASE + 48)
```

Failed to get current system time.

**OSAPI\_LOG\_LAST\_RECORDED\_ERROR\_EC**

```
#define OSAPI_LOG_LAST_RECORDED_ERROR_EC (OSAPI_LOG_BASE + 49)
```

Return the last recorded error-code for the calling thread.

This log-messages retrieves the last recorded error-code for the calling thread. It is used a function calls another function that fails.

**OSAPI\_LOG\_SET\_THREAD\_NAME\_EC**

```
#define OSAPI_LOG_SET_THREAD_NAME_EC (OSAPI_LOG_BASE + 50)
```

Failed to set the thread name in the OS.

When OSAPI starts a thread it also calls the OS to set the thread name. If this fails this warning message indicates why.

**OSAPI\_LOG\_SEM\_OPEN\_EC**

```
#define OSAPI_LOG_SEM_OPEN_EC (OSAPI_LOG_BASE + 52)
```

Failure during call VxWorks semOpen(...) or Posix sem\_open(...) when managing shared memory segments, semaphores, or mutexes.

**OSAPI\_LOG\_SEM\_INFO\_GET\_EC**

```
#define OSAPI_LOG_SEM_INFO_GET_EC (OSAPI_LOG_BASE + 53)
```

Failure during VxWorks call semInfoGet when managing shared memory segments, semaphores, or mutexes.

**OSAPI\_LOG\_SD\_UNMAP\_EC**

```
#define OSAPI_LOG_SD_UNMAP_EC (OSAPI_LOG_BASE + 54)
```

Failure during VxWorks call sdUnmap when managing shared memory segments, semaphores, or mutexes.

**OSAPI\_LOG\_SD\_OPEN\_EC**

```
#define OSAPI_LOG_SD_OPEN_EC (OSAPI_LOG_BASE + 55)
```

Failure during VxWorks call sdOpen when managing shared memory segments, semaphores, or mutexes.

**OSAPI\_LOG\_SEM\_GIVE\_EC**

```
#define OSAPI_LOG_SEM_GIVE_EC (OSAPI_LOG_BASE + 56)
```

Failure during VxWorks call semGive when managing shared memory segments, semaphores, or mutexes.

**OSAPI\_LOG\_SEM\_TAKE\_EC**

```
#define OSAPI_LOG_SEM_TAKE_EC (OSAPI_LOG_BASE + 57)
```

Failure during VxWorks call semTake when managing shared memory segments, semaphores, or mutexes.

**OSAPI\_LOG\_UNKNOWN\_SEMAPHORE\_TYPE\_EC**

```
#define OSAPI_LOG_UNKNOWN_SEMAPHORE_TYPE_EC (OSAPI_LOG_BASE + 58)
```

Trying to perform an operation an unknown semmutex type.

**OSAPI\_LOG\_SEGMENT\_KEY\_ALREADY\_EXISTS\_EC**

```
#define OSAPI_LOG_SEGMENT_KEY_ALREADY_EXISTS_EC (OSAPI_LOG_BASE + 59)
```

Failed to create segment due to a segment already existing.

**OSAPI\_LOG\_SEGMENT\_KEY\_DOESNT\_EXIST\_EC**

```
#define OSAPI_LOG_SEGMENT_KEY_DOESNT_EXIST_EC (OSAPI_LOG_BASE + 60)
```

Segment trying to be attached to does not exist.

**OSAPI\_LOG\_SEGMENT\_DETACH\_FAILED\_EC**

```
#define OSAPI_LOG_SEGMENT_DETACH_FAILED_EC (OSAPI_LOG_BASE + 61)
```

Failed to detach from shared memory segment.

**OSAPI\_LOG\_SHMEM\_ATTACH\_FAILED\_EC**

```
#define OSAPI_LOG_SHMEM_ATTACH_FAILED_EC (OSAPI_LOG_BASE + 62)
```

Attaching to shared memory segment/mutex/signaling semaphore failed.

**OSAPI\_LOG\_SHMEM\_GIVE\_FAILED\_EC**

```
#define OSAPI_LOG_SHMEM_GIVE_FAILED_EC (OSAPI_LOG_BASE + 63)
```

Failed to give shared memory mutex/signaling semaphore.

**OSAPI\_LOG\_SHMEM\_TAKE\_FAILED\_EC**

```
#define OSAPI_LOG_SHMEM_TAKE_FAILED_EC (OSAPI_LOG_BASE + 64)
```

Failed to take shared memory mutex/signaling semaphore.

**OSAPI\_LOG\_SHMEM\_SEM\_DELETE\_FAILED\_EC**

```
#define OSAPI_LOG_SHMEM_SEM_DELETE_FAILED_EC (OSAPI_LOG_BASE + 65)
```

Failed to delete shared memory semaphore or mutex.

**OSAPI\_LOG\_SHMEM\_SEM\_MUTEX\_CREATED\_EC**

```
#define OSAPI_LOG_SHMEM_SEM_MUTEX_CREATED_EC (OSAPI_LOG_BASE + 66)
```

Error during shared memory semaphore creation/deletion.

**OSAPI\_LOG\_SHMEM\_SEGMENT\_FILE\_MAP\_FAILED\_EC**

```
#define OSAPI_LOG_SHMEM_SEGMENT_FILE_MAP_FAILED_EC (OSAPI_LOG_BASE + 67)
```

Windows specific API MapViewOfFile failed.

**OSAPI\_LOG\_SHMEM\_SET\_DESCRIPTOR\_EC**

```
#define OSAPI_LOG_SHMEM_SET_DESCRIPTOR_EC (OSAPI_LOG_BASE + 68)
```

Windows specific error during setting security descriptor.

**OSAPI\_LOG\_SHMEM\_INIT\_DESCRIPTOR\_EC**

```
#define OSAPI_LOG_SHMEM_INIT_DESCRIPTOR_EC (OSAPI_LOG_BASE + 69)
```

Windows specific error during initialization of security descriptor.

**OSAPI\_LOG\_SHMEM\_FTRUNCATE\_EC**

```
#define OSAPI_LOG_SHMEM_FTRUNCATE_EC (OSAPI_LOG_BASE + 70)
```

Posix specific error during ftruncate(...) call.

**OSAPI\_LOG\_SHMEM\_MMAP\_EC**

```
#define OSAPI_LOG_SHMEM_MMAP_EC (OSAPI_LOG_BASE + 71)
```

Posix specific error during mmap(...) call.

**OSAPI\_LOG\_SHMEM\_SHM\_OPEN\_EC**

```
#define OSAPI_LOG_SHMEM_SHM_OPEN_EC (OSAPI_LOG_BASE + 72)
```

Posix specific error during shm\_open(...) call.

**OSAPI\_LOG\_SHMEM\_MUNMAP\_EC**

```
#define OSAPI_LOG_SHMEM_MUNMAP_EC (OSAPI_LOG_BASE + 73)
```

Posix specific error during munmap(...) call.

**OSAPI\_LOG\_CLOSE\_EC**

```
#define OSAPI_LOG_CLOSE_EC (OSAPI_LOG_BASE + 74)
```

Posix specific error during close(...) call.

**OSAPI\_LOG\_SHMEM\_UNLINK\_EC**

```
#define OSAPI_LOG_SHMEM_UNLINK_EC (OSAPI_LOG_BASE + 76)
```

Posix specific error during unlink(...) call.

**OSAPI\_LOG\_EXHAUSTED\_POSIX\_SEM\_LIMIT\_EC**

```
#define OSAPI_LOG_EXHAUSTED_POSIX_SEM_LIMIT_EC (OSAPI_LOG_BASE + 77)
```

System ran out of space to create semaphores. Consider increasing Posix sem limit.

**OSAPI\_LOG\_SHMEM\_SEM\_WAIT\_EC**

```
#define OSAPI_LOG_SHMEM_SEM_WAIT_EC (OSAPI_LOG_BASE + 78)
```

Posix specific error during unlink(...) call.

**OSAPI\_LOG\_SHMEM\_SEM\_UNLINK\_EC**

```
#define OSAPI_LOG_SHMEM_SEM_UNLINK_EC (OSAPI_LOG_BASE + 79)
```

Posix specific error during unlink(...) call.

**OSAPI\_LOG\_SHMEM\_POSIX\_GIVE\_EC**

```
#define OSAPI_LOG_SHMEM_POSIX_GIVE_EC (OSAPI_LOG_BASE + 80)
```

Posix specific error during os specific implementation of a semaphore mutex give.

**OSAPI\_LOG\_THREAD\_SEMPOOL\_EXCEEDED\_EC**

```
#define OSAPI_LOG_THREAD_SEMPOOL_EXCEEDED_EC (OSAPI_LOG_BASE + 90)
```

Failed to add a semaphore to the thread semaphore pool.

This maximum number of semaphores is configured with ::max\_user\_blocking\_threads. One semaphore is needed per receive thread created.

**OSAPI\_LOG\_THREAD\_SEMPOOL\_INVALID\_SEM\_EC**

```
#define OSAPI_LOG_THREAD_SEMPOOL_INVALID_SEM_EC (OSAPI_LOG_BASE + 91)
```

Failed to delete a semaphore from the thread semaphore pool.

This error indicates an imbalanced number of add/delete calls to the thread semaphore pool.

**OSAPI\_LOG\_SHMEM\_SIZE\_EXCEEDED\_EC**

```
#define OSAPI_LOG_SHMEM_SIZE_EXCEEDED_EC (OSAPI_LOG_BASE + 92)
```

Trying to create a shared memory segment that is greater than  $(2^{31}) - 1$ . The middleware cannot support shared memory segments created than  $\sim 2$  GB even though the operating system may.

**OSAPI\_LOG\_THREAD\_JOIN\_EC**

```
#define OSAPI_LOG_THREAD_JOIN_EC (OSAPI_LOG_BASE + 95)
```

Failed to join a thread.

**OSAPI\_LOG\_THREAD\_GET\_MAX\_PRIORITY\_EC**

```
#define OSAPI_LOG_THREAD_GET_MAX_PRIORITY_EC (OSAPI_LOG_BASE + 96)
```

Failed to get the maximum priority value for a scheduling policy.

**OSAPI\_LOG\_SHMEM\_FSTAT\_EC**

```
#define OSAPI_LOG_SHMEM_FSTAT_EC (OSAPI_LOG_BASE + 97)
```

POSIX specific error during fstat(...) call.

**OSAPI\_LOG\_SHMEM\_MUTEX\_INIT\_TIMEOUT\_EC**

```
#define OSAPI_LOG_SHMEM_MUTEX_INIT_TIMEOUT_EC (OSAPI_LOG_BASE + 97)
```

Timed out waiting for a shared memory mutex to be initialized.

This condition could occur if a process dies while initializing a shared memory mutex segment. The segment containing the mutex must be manually removed before it can be reinitialized.

**OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_INIT\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_INIT_EC (OSAPI_LOG_BASE + 200)
```

Failed to create a pthread mutex attribute.

**OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETPSHARED\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETPSHARED_EC (OSAPI_LOG_BASE + 201)
```

Failed to set the pthread mutex attribute to be process shared.



**OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETROBUST\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETROBUST_EC (OSAPI_LOG_BASE + 202)
```

Failed to set the pthread mutex attribute to be robust.

**OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_DESTROY\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_DESTROY_EC (OSAPI_LOG_BASE + 203)
```

Failed to destroy the pthread mutex attribute.

**OSAPI\_LOG\_PTHREAD\_MUTEX\_INIT\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEX_INIT_EC (OSAPI_LOG_BASE + 204)
```

Failed to initialize a pthread mutex.

**OSAPI\_LOG\_PTHREAD\_MUTEX\_DESTROY\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEX_DESTROY_EC (OSAPI_LOG_BASE + 205)
```

Failed to destroy a pthread mutex.

**OSAPI\_LOG\_PTHREAD\_MUTEX\_LOCK\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEX_LOCK_EC (OSAPI_LOG_BASE + 206)
```

Failed to lock a pthread mutex.

**OSAPI\_LOG\_PTHREAD\_MUTEX\_UNLOCK\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEX_UNLOCK_EC (OSAPI_LOG_BASE + 207)
```

Failed to unlock a pthread mutex.

**OSAPI\_LOG\_PTHREAD\_MUTEX\_CONSISTENT\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEX_CONSISTENT_EC (OSAPI_LOG_BASE + 208)
```

Failed to set the pthread mutex to be consistent.

**OSAPI\_LOG\_PTHREAD\_CONDATTR\_INIT\_EC**

```
#define OSAPI_LOG_PTHREAD_CONDATTR_INIT_EC (OSAPI_LOG_BASE + 209)
```

Failed to create a pthread condition variable attribute.

**OSAPI\_LOG\_PTHREAD\_CONDATTR\_SETPSHARED\_EC**

```
#define OSAPI_LOG_PTHREAD_CONDATTR_SETPSHARED_EC (OSAPI_LOG_BASE + 210)
```

Failed to set the pthread condition variable attribute to be process shared.

**OSAPI\_LOG\_PTHREAD\_CONDATTR\_DESTROY\_EC**

```
#define OSAPI_LOG_PTHREAD_CONDATTR_DESTROY_EC (OSAPI_LOG_BASE + 211)
```

Failed to destroy the pthread condition variable attribute.

**OSAPI\_LOG\_PTHREAD\_COND\_INIT\_EC**

```
#define OSAPI_LOG_PTHREAD_COND_INIT_EC (OSAPI_LOG_BASE + 212)
```

Failed to initialize a pthread condition variable.

**OSAPI\_LOG\_PTHREAD\_COND\_DESTROY\_EC**

```
#define OSAPI_LOG_PTHREAD_COND_DESTROY_EC (OSAPI_LOG_BASE + 213)
```

Failed to destroy a pthread condition variable.

**OSAPI\_LOG\_PTHREAD\_COND\_WAIT\_EC**

```
#define OSAPI_LOG_PTHREAD_COND_WAIT_EC (OSAPI_LOG_BASE + 214)
```

Failed to wait on a pthread condition variable.

**OSAPI\_LOG\_PTHREAD\_COND\_SIGNAL\_EC**

```
#define OSAPI_LOG_PTHREAD_COND_SIGNAL_EC (OSAPI_LOG_BASE + 215)
```

Failed to signal a pthread condition variable.

**OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETTYPE\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETTYPE_EC (OSAPI_LOG_BASE + 216)
```

Failed to set the type of a pthread mutex attribute.

**OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETPROTOCOL\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETPROTOCOL_EC (OSAPI_LOG_BASE + 217)
```

Failed to set the protocol of a pthread mutex attribute.

**OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETPRIOCEILING\_EC**

```
#define OSAPI_LOG_PTHREAD_MUTEXATTR_SETPRIOCEILING_EC (OSAPI_LOG_BASE + 218)
```

Failed to set the priority ceiling of a pthread mutex attribute.

**6.21.3 REDA**

Real-time Efficient Data Structures and Algorithms. ModuleID = 1.

**Macros**

- #define REDA\_LOG\_HEAP\_MGR\_SAMPLE\_BUFFERPOOL 1  
*Sample buffer pool object.*
- #define REDA\_LOG\_HEAP\_MGR\_LISTNODE BUFFERPOOL 2  
*Listnode buffer pool object.*
- #define REDA\_LOG\_HEAP\_MGR\_OBJECT\_MEMORY 3  
*Heap manager object.*
- #define REDA\_LOG\_HEAP\_MGR\_INDEXER 4  
*REDA indexer object.*
- #define REDA\_LOG\_HEAP\_MGR\_MUTEX 5  
*REDA heap manager object.*
- #define REDA\_LOG\_BUFFERPOOL\_OUT\_OF\_RESOURCES\_EC (REDA\_LOG\_BASE + 1)  
*Not sufficient memory to allocate buffer-pool.*
- #define REDA\_LOG\_BUFFERPOOL\_BUFFER\_INITIALIZATION\_FAILED\_EC (REDA\_LOG\_BASE + 2)  
*The buffer initialization method failed.*
- #define REDA\_LOG\_BUFFERPOOL\_NOT\_EMPTY\_EC (REDA\_LOG\_BASE + 3)  
*Cannot delete buffer-pool due to buffer(s) not returned to pool.*
- #define REDA\_LOG\_BUFFERPOOL\_DOUBLE\_FREE\_EC (REDA\_LOG\_BASE + 4)  
*A buffer was freed more than once.*
- #define REDA\_LOG\_MEMPOOL\_INCONSISTENT\_PROPERTIES\_EC (REDA\_LOG\_BASE + 5)  
*Inconsistent properties specified for a REDA\_MemPool.*

- #define REDA\_LOG\_MEMPOOL\_OUT\_OF\_RESOURCES\_EC (REDA\_LOG\_BASE + 6)  
*Insufficient memory to allocate a new REDA\_MemPool.*
- #define REDA\_LOG\_MEMPOOL\_POOL\_ALLOCATION\_EC (REDA\_LOG\_BASE + 7)  
*Error during allocation of the memory pool used by a REDA\_MemPool.*
- #define REDA\_LOG\_BUFFERPOOL\_NULL\_POINTER\_EC (REDA\_LOG\_BASE + 8)  
*A pointer in a pool's linked list was NULL.*
- #define REDA\_LOG\_SEQUENCE\_COPY\_FAILED\_EC (REDA\_LOG\_BASE + 100)  
*An error occurred while copying a sequence.*
- #define REDA\_LOG\_SEQUENCE\_ALLOC\_FAILED\_EC (REDA\_LOG\_BASE + 101)  
*An error occurred while allocating space for a sequence.*
- #define REDA\_LOG\_SEQUENCE\_SET\_MAX\_FAILED\_EC (REDA\_LOG\_BASE + 102)  
*An error occurred while setting the maximum sequence size.*
- #define REDA\_LOG\_SEQUENCE\_REALLOCATION\_EC (REDA\_LOG\_BASE + 103)  
*An attempt was made to resize an already allocated sequence.*
- #define REDA\_LOG\_SEQUENCE\_INVALID\_OPERATION\_EC (REDA\_LOG\_BASE + 104)  
*An error occurred while operating on two sequences (copy/compare)*
- #define REDA\_LOG\_SEQUENCE\_INVALID\_LENGTH\_EC (REDA\_LOG\_BASE + 105)  
*An attempt was made to set an illegal length (length < 0 or length > max\_length)*
- #define REDA\_LOG\_SEQUENCE\_INDEX\_OUT\_OF\_BOUNDS\_EC (REDA\_LOG\_BASE + 106)  
*An illegal sequence index was specified (index < 0 or index > length)*
- #define REDA\_LOG\_STRING\_ALLOC\_FAILED\_EC (REDA\_LOG\_BASE + 200)  
*An error occurred while allocating memory for a string.*
- #define REDA\_LOG\_INDEX\_FULL\_EC (REDA\_LOG\_BASE + 300)  
*An attempt was made to add an element to a full index.*
- #define REDA\_LOG\_INDEX\_ENTRY\_EXISTS\_EC (REDA\_LOG\_BASE + 301)  
*An attempt was made to add an existing element to an index.*
- #define REDA\_LOG\_INVALID\_LIST\_NODE\_EC (REDA\_LOG\_BASE + 302)  
*An attempt was made to add an existing element to an index.*
- #define REDA\_LOG\_BUFFER\_OUT\_OF\_MEMORY\_EC (REDA\_LOG\_BASE + 307)  
*Failed to allocate buffer of a specified kind.*
- #define REDA\_LOG\_HEAP\_MGR\_ALLOC\_EC (REDA\_LOG\_BASE + 308)  
*Allocation error in REDA Heap Manager.*
- #define REDA\_LOG\_HEAP\_MGR\_GET\_BUFFER\_EC (REDA\_LOG\_BASE + 309)  
*Error when there are no more available loanable buffers.*
- #define REDA\_LOG\_HEAP\_MGR\_MUTEX\_FAILURE\_EC (REDA\_LOG\_BASE + 310)  
*Error during locking or unlocking of a mutex.*
- #define NETIO\_SHMEM\_LOG\_CQ\_INT\_REPRESENTATION\_EC (SHMEM\_LOG\_BASE + 200)  
*Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*
- #define NETIO\_SHMEM\_LOG\_CQ\_UNLINKED\_EC (SHMEM\_LOG\_BASE + 201)  
*Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*
- #define NETIO\_SHMEM\_LOG\_CQ\_INVALID\_SIGNATURE\_EC (SHMEM\_LOG\_BASE + 202)  
*Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*
- #define NETIO\_SHMEM\_LOG\_CQ\_INCOMPATIBLE\_VERSION\_EC (SHMEM\_LOG\_BASE + 203)  
*Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*
- #define NETIO\_SHMEM\_LOG\_CQ\_INCONSISTENT\_STATE\_EC (SHMEM\_LOG\_BASE + 204)  
*Found an inconsistent concurrent queue state while performing a a read or write. The queue memory may have been corrupted or otherwise tampered with.*

### 6.21.3.1 Detailed Description

Real-time Efficient Data Structures and Algorithms. ModuleID = 1.

### 6.21.3.2 Macro Definition Documentation

#### **REDA\_LOG\_HEAP\_MGR\_SAMPLE\_BUFFERPOOL**

```
#define REDA_LOG_HEAP_MGR_SAMPLE_BUFFERPOOL 1
```

Sample buffer pool object.

#### **REDA\_LOG\_HEAP\_MGR\_LISTNODE**

```
#define REDA_LOG_HEAP_MGR_LISTNODE BUFFERPOOL 2
```

Listnode buffer pool object.

#### **REDA\_LOG\_HEAP\_MGR\_OBJECT\_MEMORY**

```
#define REDA_LOG_HEAP_MGR_OBJECT_MEMORY 3
```

Heap manager object.

#### **REDA\_LOG\_HEAP\_MGR\_INDEXER**

```
#define REDA_LOG_HEAP_MGR_INDEXER 4
```

REDA indexer object.

#### **REDA\_LOG\_HEAP\_MGR\_MUTEX**

```
#define REDA_LOG_HEAP_MGR_MUTEX 5
```

REDA heap manager object.

#### **REDA\_LOG\_BUFFERPOOL\_OUT\_OF\_RESOURCES\_EC**

```
#define REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 1)
```

Not sufficient memory to allocate buffer-pool.

**REDA\_LOG\_BUFFERPOOL\_BUFFER\_INITIALIZATION\_FAILED\_EC**

```
#define REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC (REDA_LOG_BASE + 2)
```

The buffer initialization method failed.

**REDA\_LOG\_BUFFERPOOL\_NOT\_EMPTY\_EC**

```
#define REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC (REDA_LOG_BASE + 3)
```

Cannot delete buffer-pool due to buffer(s) not returned to pool.

**REDA\_LOG\_BUFFERPOOL\_DOUBLE\_FREE\_EC**

```
#define REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC (REDA_LOG_BASE + 4)
```

A buffer was freed more than once.

**REDA\_LOG\_MEMPOOL\_INCONSISTENT\_PROPERTIES\_EC**

```
#define REDA_LOG_MEMPOOL_INCONSISTENT_PROPERTIES_EC (REDA_LOG_BASE + 5)
```

Inconsistent properties specified for a REDA\_MemPool.

**REDA\_LOG\_MEMPOOL\_OUT\_OF\_RESOURCES\_EC**

```
#define REDA_LOG_MEMPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 6)
```

Insufficient memory to allocate a new REDA\_MemPool.

**REDA\_LOG\_MEMPOOL\_POOL\_ALLOCATION\_EC**

```
#define REDA_LOG_MEMPOOL_POOL_ALLOCATION_EC (REDA_LOG_BASE + 7)
```

Error during allocation of the memory pool used by a REDA\_MemPool.

**REDA\_LOG\_BUFFERPOOL\_NULL\_POINTER\_EC**

```
#define REDA_LOG_BUFFERPOOL_NULL_POINTER_EC (REDA_LOG_BASE + 8)
```

A pointer in a pool's linked list was NULL.

**REDA\_LOG\_SEQUENCE\_COPY\_FAILED\_EC**

```
#define REDA_LOG_SEQUENCE_COPY_FAILED_EC (REDA_LOG_BASE + 100)
```

An error occurred while copying a sequence.

**REDA\_LOG\_SEQUENCE\_ALLOC\_FAILED\_EC**

```
#define REDA_LOG_SEQUENCE_ALLOC_FAILED_EC (REDA_LOG_BASE + 101)
```

An error occurred while allocating space for a sequence.

**REDA\_LOG\_SEQUENCE\_SET\_MAX\_FAILED\_EC**

```
#define REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC (REDA_LOG_BASE + 102)
```

An error occurred while setting the maximum sequence size.

**REDA\_LOG\_SEQUENCE\_REALLOCATION\_EC**

```
#define REDA_LOG_SEQUENCE_REALLOCATION_EC (REDA_LOG_BASE + 103)
```

An attempt was made to resize an already allocated sequence.

**REDA\_LOG\_SEQUENCE\_INVALID\_OPERATION\_EC**

```
#define REDA_LOG_SEQUENCE_INVALID_OPERATION_EC (REDA_LOG_BASE + 104)
```

An error occurred while operating on two sequences (copy/compare)

**REDA\_LOG\_SEQUENCE\_INVALID\_LENGTH\_EC**

```
#define REDA_LOG_SEQUENCE_INVALID_LENGTH_EC (REDA_LOG_BASE + 105)
```

An attempt was made to set an illegal length (length < 0 or length > max\_length)

**REDA\_LOG\_SEQUENCE\_INDEX\_OUT\_OF\_BOUNDS\_EC**

```
#define REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC (REDA_LOG_BASE + 106)
```

An illegal sequence index was specified (index < 0 or index > length)

**REDA\_LOG\_STRING\_ALLOC\_FAILED\_EC**

```
#define REDA_LOG_STRING_ALLOC_FAILED_EC (REDA_LOG_BASE + 200)
```

An error occurred while allocating memory for a string.

**REDA\_LOG\_INDEX\_FULL\_EC**

```
#define REDA_LOG_INDEX_FULL_EC (REDA_LOG_BASE + 300)
```

An attempt was made to add an element to a full index.

**REDA\_LOG\_INDEX\_ENTRY\_EXISTS\_EC**

```
#define REDA_LOG_INDEX_ENTRY_EXISTS_EC (REDA_LOG_BASE + 301)
```

An attempt was made to add an existing element to an index.

**REDA\_LOG\_INVALID\_LIST\_NODE\_EC**

```
#define REDA_LOG_INVALID_LIST_NODE_EC (REDA_LOG_BASE + 302)
```

An attempt was made to add an existing element to an index.

**REDA\_LOG\_BUFFER\_OUT\_OF\_MEMORY\_EC**

```
#define REDA_LOG_BUFFER_OUT_OF_MEMORY_EC (REDA_LOG_BASE + 307)
```

Failed to allocate buffer of a specified kind.

**REDA\_LOG\_HEAP\_MGR\_ALLOC\_EC**

```
#define REDA_LOG_HEAP_MGR_ALLOC_EC (REDA_LOG_BASE + 308)
```

Allocation error in REDA Heap Manager.

**REDA\_LOG\_HEAP\_MGR\_GET\_BUFFER\_EC**

```
#define REDA_LOG_HEAP_MGR_GET_BUFFER_EC (REDA_LOG_BASE + 309)
```

Error when there are no more available loanable buffers.



**REDA\_LOG\_HEAP\_MGR\_MUTEX\_FAILURE\_EC**

```
#define REDA_LOG_HEAP_MGR_MUTEX_FAILURE_EC (REDA_LOG_BASE + 310)
```

Error during locking or unlocking of a mutex.

**NETIO\_SHMEM\_LOG\_CQ\_INT\_REPRESENTATION\_EC**

```
#define NETIO_SHMEM_LOG_CQ_INT_REPRESENTATION_EC (SHMEM_LOG_BASE + 200)
```

Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.

**NETIO\_SHMEM\_LOG\_CQ\_UNLINKED\_EC**

```
#define NETIO_SHMEM_LOG_CQ_UNLINKED_EC (SHMEM_LOG_BASE + 201)
```

Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.

**NETIO\_SHMEM\_LOG\_CQ\_INVALID\_SIGNATURE\_EC**

```
#define NETIO_SHMEM_LOG_CQ_INVALID_SIGNATURE_EC (SHMEM_LOG_BASE + 202)
```

Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.

**NETIO\_SHMEM\_LOG\_CQ\_INCOMPATIBLE\_VERSION\_EC**

```
#define NETIO_SHMEM_LOG_CQ_INCOMPATIBLE_VERSION_EC (SHMEM_LOG_BASE + 203)
```

Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.

**NETIO\_SHMEM\_LOG\_CQ\_INCONSISTENT\_STATE\_EC**

```
#define NETIO_SHMEM_LOG_CQ_INCONSISTENT_STATE_EC (SHMEM_LOG_BASE + 204)
```

Found an inconsistent concurrent queue state while performing a read or write. The queue memory may have been corrupted or otherwise tampered with.

**6.21.4 DB**

Database. ModuleID = 2.

**Macros**

- #define `DB_LOG_SORTED_ALLOC_EC` (`DB_LOG_BASE` + 1)  
*Not sufficient memory to allocate sorted list for an index.*
- #define `DB_LOG_NAME_TOO_LONG_EC` (`DB_LOG_BASE` + 2)  
*The specified database name is too long.*
- #define `DB_LOG_ILLEGAL_TABLE_SIZE_EC` (`DB_LOG_BASE` + 3)  
*Illegal table size specified.*
- #define `DB_LOG_ILLEGAL_LOCK_MODE_EC` (`DB_LOG_BASE` + 4)  
*Illegal combination of lock mode and mutex given.*
- #define `DB_LOG_MUTEX_ALLOC_EC` (`DB_LOG_BASE` + 5)  
*Failed to allocate database mutex.*
- #define `DB_LOG_ALLOC_TABLE_POOL_EC` (`DB_LOG_BASE` + 6)  
*Failed to allocate buffer pool.*
- #define `DB_LOG_TABLES_INUSE_EC` (`DB_LOG_BASE` + 7)  
*Table is in use.*
- #define `DB_LOG_TABLE_NAME_TOO_LONG_EC` (`DB_LOG_BASE` + 8)  
*Specified table name too long.*
- #define `DB_LOG_ILLEGAL_RECORD_COUNT_EC` (`DB_LOG_BASE` + 9)  
*Illegal record count specified.*
- #define `DB_LOG_TABLE_EXISTS_EC` (`DB_LOG_BASE` + 10)  
*Specified table already exists.*
- #define `DB_LOG_OUT_OF_TABLES_EC` (`DB_LOG_BASE` + 11)  
*Table resources exceeded.*
- #define `DB_LOG_OUT_OF_RECORDS_EC` (`DB_LOG_BASE` + 12)  
*Table record resources exceeded.*
- #define `DB_LOG_OUT_OF_INDICES_EC` (`DB_LOG_BASE` + 13)  
*Table index resources exceeded.*
- #define `DB_LOG_OUT_OF_CURSORS_EC` (`DB_LOG_BASE` + 14)  
*Table cursor resources exceeded.*
- #define `DB_LOG_RECORDS_INUSE_EC` (`DB_LOG_BASE` + 15)  
*Cannot delete table, records are still in table.*
- #define `DB_LOG_CURSORS_INUSE_EC` (`DB_LOG_BASE` + 16)  
*Cannot delete table, cursors are still in use.*
- #define `DB_LOG_INDEX_INUSE_EC` (`DB_LOG_BASE` + 17)  
*Cannot delete table, indices are still in use.*
- #define `DB_LOG_ALLOC_CURSOR_POOL_EC` (`DB_LOG_BASE` + 18)  
*Failed to allocate cursor buffer pool.*
- #define `DB_LOG_ALLOC_INDEX_POOL_EC` (`DB_LOG_BASE` + 19)  
*Failed to allocate index buffer pool.*
- #define `DB_LOG_ALLOC_RECORD_POOL_EC` (`DB_LOG_BASE` + 20)  
*Failed to allocate record buffer pool.*
- #define `DB_LOG_ALLOC_DATABASE_EC` (`DB_LOG_BASE` + 21)  
*Failed to allocate database.*
- #define `DB_LOG_TABLE_NOT_INUSE_EC` (`DB_LOG_BASE` + 22)  
*An attempt was made to delete a table not in use.*
- #define `DB_LOG_RECORD_ALREADY_EXISTS_EC` (`DB_LOG_BASE` + 23)

*An attempt was made to insert an already existing record.*

- #define `DB_LOG_RECORD_DOES_NOT_EXIST_EC` (`DB_LOG_BASE` + 24)

*An attempt was made to delete a record that does not exist.*

- #define `DB_LOG_CURSOR_INVALIDATED_EC` (`DB_LOG_BASE` + 25)

*An attempt was made to use a cursor that has been invalidated.*

- #define `DB_LOG_LOCK_FAILURE_EC` (`DB_LOG_BASE` + 26)

*Failed to lock the database.*

- #define `DB_LOG_UNLOCK_FAILURE_EC` (`DB_LOG_BASE` + 27)

*Failed to unlock the database.*

- #define `DB_LOG_OUT_OF_CTORDTOR_EC` (`DB_LOG_BASE` + 28)

*Constructor/Destructor memory could not be allocated.*

#### 6.21.4.1 Detailed Description

Database. ModuleID = 2.

#### 6.21.4.2 Macro Definition Documentation

##### **DB\_LOG\_SORTED\_ALLOC\_EC**

```
#define DB_LOG_SORTED_ALLOC_EC (DB_LOG_BASE + 1)
```

Not sufficient memory to allocate sorted list for an index.

##### **DB\_LOG\_NAME\_TOO\_LONG\_EC**

```
#define DB_LOG_NAME_TOO_LONG_EC (DB_LOG_BASE + 2)
```

The specified database name is too long.

##### **DB\_LOG\_ILLEGAL\_TABLE\_SIZE\_EC**

```
#define DB_LOG_ILLEGAL_TABLE_SIZE_EC (DB_LOG_BASE + 3)
```

Illegal table size specified.

##### **DB\_LOG\_ILLEGAL\_LOCK\_MODE\_EC**

```
#define DB_LOG_ILLEGAL_LOCK_MODE_EC (DB_LOG_BASE + 4)
```

Illegal combination of lock mode and mutex given.

**DB\_LOG\_MUTEX\_ALLOC\_EC**

```
#define DB_LOG_MUTEX_ALLOC_EC (DB_LOG_BASE + 5)
```

Failed to allocate database mutex.

**DB\_LOG\_ALLOC\_TABLE\_POOL\_EC**

```
#define DB_LOG_ALLOC_TABLE_POOL_EC (DB_LOG_BASE + 6)
```

Failed to allocate buffer pool.

**DB\_LOG\_TABLES\_INUSE\_EC**

```
#define DB_LOG_TABLES_INUSE_EC (DB_LOG_BASE + 7)
```

Table is in use.

**DB\_LOG\_TABLE\_NAME\_TOO\_LONG\_EC**

```
#define DB_LOG_TABLE_NAME_TOO_LONG_EC (DB_LOG_BASE + 8)
```

Specified table name too long.

**DB\_LOG\_ILLEGAL\_RECORD\_COUNT\_EC**

```
#define DB_LOG_ILLEGAL_RECORD_COUNT_EC (DB_LOG_BASE + 9)
```

Illegal record count specified.

**DB\_LOG\_TABLE\_EXISTS\_EC**

```
#define DB_LOG_TABLE_EXISTS_EC (DB_LOG_BASE + 10)
```

Specified table already exists.

**DB\_LOG\_OUT\_OF\_TABLES\_EC**

```
#define DB_LOG_OUT_OF_TABLES_EC (DB_LOG_BASE + 11)
```

Table resources exceeded.

**DB\_LOG\_OUT\_OF\_RECORDS\_EC**

```
#define DB_LOG_OUT_OF_RECORDS_EC (DB_LOG_BASE + 12)
```

Table record resources exceeded.

**DB\_LOG\_OUT\_OF\_INDICES\_EC**

```
#define DB_LOG_OUT_OF_INDICES_EC (DB_LOG_BASE + 13)
```

Table index resources exceeded.

**DB\_LOG\_OUT\_OF\_CURSORS\_EC**

```
#define DB_LOG_OUT_OF_CURSORS_EC (DB_LOG_BASE + 14)
```

Table cursor resources exceeded.

**DB\_LOG\_RECORDS\_INUSE\_EC**

```
#define DB_LOG_RECORDS_INUSE_EC (DB_LOG_BASE + 15)
```

Cannot delete table, records are still in table.

**DB\_LOG\_CURSORS\_INUSE\_EC**

```
#define DB_LOG_CURSORS_INUSE_EC (DB_LOG_BASE + 16)
```

Cannot delete table, cursors are still in use.

**DB\_LOG\_INDEX\_INUSE\_EC**

```
#define DB_LOG_INDEX_INUSE_EC (DB_LOG_BASE + 17)
```

Cannot delete table, indices are still in use.

**DB\_LOG\_ALLOC\_CURSOR\_POOL\_EC**

```
#define DB_LOG_ALLOC_CURSOR_POOL_EC (DB_LOG_BASE + 18)
```

Failed to allocate cursor buffer pool.

**DB\_LOG\_ALLOC\_INDEX\_POOL\_EC**

```
#define DB_LOG_ALLOC_INDEX_POOL_EC (DB_LOG_BASE + 19)
```

Failed to allocate index buffer pool.

**DB\_LOG\_ALLOC\_RECORD\_POOL\_EC**

```
#define DB_LOG_ALLOC_RECORD_POOL_EC (DB_LOG_BASE + 20)
```

Failed to allocate record buffer pool.

**DB\_LOG\_ALLOC\_DATABASE\_EC**

```
#define DB_LOG_ALLOC_DATABASE_EC (DB_LOG_BASE + 21)
```

Failed to allocate database.

**DB\_LOG\_TABLE\_NOT\_INUSE\_EC**

```
#define DB_LOG_TABLE_NOT_INUSE_EC (DB_LOG_BASE + 22)
```

An attempt was made to delete a table not in use.

**DB\_LOG\_RECORD\_ALREADY\_EXISTS\_EC**

```
#define DB_LOG_RECORD_ALREADY_EXISTS_EC (DB_LOG_BASE + 23)
```

An attempt was made to insert an already existing record.

**DB\_LOG\_RECORD\_DOES\_NOT\_EXIST\_EC**

```
#define DB_LOG_RECORD_DOES_NOT_EXIST_EC (DB_LOG_BASE + 24)
```

An attempt was made to delete a record that does not exist.

**DB\_LOG\_CURSOR\_INVALIDATED\_EC**

```
#define DB_LOG_CURSOR_INVALIDATED_EC (DB_LOG_BASE + 25)
```

An attempt was made to use a cursor that has been invalidated.

**DB\_LOG\_LOCK\_FAILURE\_EC**

```
#define DB_LOG_LOCK_FAILURE_EC (DB_LOG_BASE + 26)
```

Failed to lock the database.

**DB\_LOG\_UNLOCK\_FAILURE\_EC**

```
#define DB_LOG_UNLOCK_FAILURE_EC (DB_LOG_BASE + 27)
```

Failed to unlock the database.

**DB\_LOG\_OUT\_OF\_CTORDTOR\_EC**

```
#define DB_LOG_OUT_OF_CTORDTOR_EC (DB_LOG_BASE + 28)
```

Constructor/Destructor memory could not be allocated.

**6.21.5 RT**

RT. ModuleID = 3.

**Macros**

- #define **RT\_LOG\_SET\_IMMUTABLE\_PROPERTY\_EC** (RT\_LOG\_BASE + 1)  
*Failed to set registry property due to registry already being enabled.*
- #define **RT\_LOG\_REGISTRY\_INIT\_FAILURE\_EC** (RT\_LOG\_BASE + 2)  
*Failed to initialize registry due to failed table creation.*
- #define **RT\_LOG\_REGISTRY\_FINALIZE\_EC** (RT\_LOG\_BASE + 3)  
*Failed to finalize registry due to failed table deletion.*
- #define **RT\_LOG\_REGISTRY\_EXISTS\_EC** (RT\_LOG\_BASE + 4)  
*An attempt was made to register a factory that already existed.*
- #define **RT\_LOG\_REGISTRY\_REGISTER\_EC** (RT\_LOG\_BASE + 5)  
*Error registering a component factory. May have exceeded RT\_RegistryProperty.max\_factories.*
- #define **RT\_LOG\_REGISTRY\_INIT\_FACTORY\_EC** (RT\_LOG\_BASE + 7)  
*A registered factory failed to initialize.*
- #define **RT\_LOG\_REGISTRY\_NAME\_TOO\_LONG\_EC** (RT\_LOG\_BASE + 8)  
*Factory name longer than maximum length of 8.*
- #define **RT\_LOG\_REGISTRY\_INCONSISTENT\_CID\_EC** (RT\_LOG\_BASE + 9)  
*Factory name exists, but the class ID is not of the requested type.*
- #define **RT\_LOG\_REGISTRY\_NOT\_INITIALIZED\_EC** (RT\_LOG\_BASE + 10)  
*An attempt was made to operate on an uninitialized registry.*

### 6.21.5.1 Detailed Description

RT.ModuleID = 3.

### 6.21.5.2 Macro Definition Documentation

#### RT\_LOG\_SET\_IMMUTABLE\_PROPERTY\_EC

```
#define RT_LOG_SET_IMMUTABLE_PROPERTY_EC (RT_LOG_BASE + 1)
```

Failed to set registry property due to registry already being enabled.

#### RT\_LOG\_REGISTRY\_INIT\_FAILURE\_EC

```
#define RT_LOG_REGISTRY_INIT_FAILURE_EC (RT_LOG_BASE + 2)
```

Failed to initialize registry due to failed table creation.

#### RT\_LOG\_REGISTRY\_FINALIZE\_EC

```
#define RT_LOG_REGISTRY_FINALIZE_EC (RT_LOG_BASE + 3)
```

Failed to finalize registry due to failed table deletion.

#### RT\_LOG\_REGISTRY\_EXISTS\_EC

```
#define RT_LOG_REGISTRY_EXISTS_EC (RT_LOG_BASE + 4)
```

An attempt was made to register a factory that already existed.

#### RT\_LOG\_REGISTRY\_REGISTER\_EC

```
#define RT_LOG_REGISTRY_REGISTER_EC (RT_LOG_BASE + 5)
```

Error registering a component factory. May have exceeded RT\_RegistryProperty.max\_factories.

#### RT\_LOG\_REGISTRY\_INIT\_FACTORY\_EC

```
#define RT_LOG_REGISTRY_INIT_FACTORY_EC (RT_LOG_BASE + 7)
```

A registered factory failed to initialize.



**RT\_LOG\_REGISTRY\_NAME\_TOO\_LONG\_EC**

```
#define RT_LOG_REGISTRY_NAME_TOO_LONG_EC (RT_LOG_BASE + 8)
```

Factory name longer than maximum length of 8.

**RT\_LOG\_REGISTRY\_INCONSISTENT\_CID\_EC**

```
#define RT_LOG_REGISTRY_INCONSISTENT_CID_EC (RT_LOG_BASE + 9)
```

Factory name exists, but the class ID is not of the requested type.

**RT\_LOG\_REGISTRY\_NOT\_INITIALIZED\_EC**

```
#define RT_LOG_REGISTRY_NOT_INITIALIZED_EC (RT_LOG_BASE + 10)
```

An attempt was made to operate on an uninitialized registry.

**6.21.6 CDR**

CDR. ModuleID = 5.

**Macros**

- `#define CDR_LOG_STREAM_ALLOC_EC (CDR_LOG_BASE + 1)`  
*Failed to allocate stream or stream buffer.*
- `#define CDR_LOG_SET_OFFSET_EC (CDR_LOG_BASE + 2)`  
*Failed to set stream's current offset.*
- `#define CDR_LOG_INCR_OFFSET_EC (CDR_LOG_BASE + 3)`  
*Failed to increment stream's current offset.*
- `#define CDR_LOG_UNSUPPORTED_OPTIONS_EC (CDR_LOG_BASE + 4)`  
*Unsupported CDR encapsulations were received. The sample is dropped.*

**6.21.6.1 Detailed Description**

CDR. ModuleID = 5.

**6.21.6.2 Macro Definition Documentation****CDR\_LOG\_STREAM\_ALLOC\_EC**

```
#define CDR_LOG_STREAM_ALLOC_EC (CDR_LOG_BASE + 1)
```

Failed to allocate stream or stream buffer.

**CDR\_LOG\_SET\_OFFSET\_EC**

```
#define CDR_LOG_SET_OFFSET_EC (CDR_LOG_BASE + 2)
```

Failed to set stream's current offset.

**CDR\_LOG\_INCR\_OFFSET\_EC**

```
#define CDR_LOG_INCR_OFFSET_EC (CDR_LOG_BASE + 3)
```

Failed to increment stream's current offset.

**CDR\_LOG\_UNSUPPORTED\_OPTIONS\_EC**

```
#define CDR_LOG_UNSUPPORTED_OPTIONS_EC (CDR_LOG_BASE + 4)
```

Unsupported CDR encapsulations were received. The sample is dropped.

**6.21.7 NETIO**

Network I/O. ModuleID = 4.

**Macros**

- #define **NETIO\_LOG\_GETHOST\_BYNAME\_EC** (NETIO\_LOG\_BASE + 1)  
*Failed to get host address from a string representation.*
- #define **NETIO\_LOG\_BIND\_NEW\_EC** (NETIO\_LOG\_BASE + 2)  
*Failed to allocate memory for a bind-resolver.*
- #define **NETIO\_LOG\_BIND\_NEW\_RTABLE\_EC** (NETIO\_LOG\_BASE + 3)  
*Failed to create a database table for routes of a bind-resolver.*
- #define **NETIO\_LOG\_BIND\_CREATE\_RTABLE\_EC** (NETIO\_LOG\_BASE + 4)  
*Failed to create route record.*
- #define **NETIO\_LOG\_BIND\_DELETE\_RTABLE\_EC** (NETIO\_LOG\_BASE + 5)  
*Failed to delete a database table for routes of a bind\_resolver.*
- #define **NETIO\_LOG\_BIND\_SET\_LENGTH\_EC** (NETIO\_LOG\_BASE + 6)  
*Failed to set the length for a sequence of resolved addresses.*
- #define **NETIO\_LOG\_BIND\_SET\_MAX\_EC** (NETIO\_LOG\_BASE + 7)  
*Failed to set the maximum length of an internal sequence of addresses.*
- #define **NETIO\_LOG\_BIND\_EXTERNAL\_FAILED\_EC** (NETIO\_LOG\_BASE + 8)  
*An interface failed on bind\_external.*
- #define **NETIO\_LOG\_UNBIND\_EXTERNAL\_FAILED\_EC** (NETIO\_LOG\_BASE + 10)  
*An interface failed on unbind\_external.*
- #define **NETIO\_LOG\_ROUTE\_RTABLE\_ALLOC\_EC** (NETIO\_LOG\_BASE + 11)  
*Failed to allocate memory for a route resolver.*

- #define `NETIO_LOG_ROUTE_RTABLE_CREATE_EC` (`NETIO_LOG_BASE` + 12)  
*Failed to create a database table for routes of a route-resolver.*
- #define `NETIO_LOG_ROUTE_RTABLE_DELETE_EC` (`NETIO_LOG_BASE` + 13)  
*Failed to delete a database table for routes of a route-resolver.*
- #define `NETIO_LOG_ROUTE_RTABLE_ADD_EC` (`NETIO_LOG_BASE` + 14)  
*Failed to create route resolver record.*
- #define `NETIO_LOG_ROUTE_RTABLE_UPDATE_EC` (`NETIO_LOG_BASE` + 15)  
*Failed to find routes to update for a route-resolver.*
- #define `NETIO_LOG_AR_ALLOC_FAILED_EC` (`NETIO_LOG_BASE` + 16)  
*Failed to allocate memory for address resolver.*
- #define `NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC` (`NETIO_LOG_BASE` + 17)  
*An attempt was made to add an existing interface with a different port resolver to the address resolver.*
- #define `NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC` (`NETIO_LOG_BASE` + 18)  
*An interface failed to resolve an address.*
- #define `NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC` (`NETIO_LOG_BASE` + 19)  
*An interface failed to resolve a port.*
- #define `NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC` (`NETIO_LOG_BASE` + 21)  
*NETIO\_AddressResolver\_resolve/get\_next was called with an uninitialized context.*
- #define `NETIO_LOG_AR_INVALID_TOKEN_EC` (`NETIO_LOG_BASE` + 22)  
*An invalid token was encountered in the address string.*
- #define `NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC` (`NETIO_LOG_BASE` + 23)  
*The length of the address exceeded the address buffer passed in.*
- #define `NETIO_LOG_INVALID_ADDRESS_INDEX_EC` (`NETIO_LOG_BASE` + 24)  
*An address index was not parsed correctly.*
- #define `NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC` (`NETIO_LOG_BASE` + 25)  
*Illegal route operation attempted.*
- #define `NETIO_LOG_SET_NAME_EC` (`UDP_LOG_BASE` + 26)  
*Failed to set the name of a runtime component interface.*
- #define `NETIO_LOG_PACKET_SET_HEAD_EC` (`UDP_LOG_BASE` + 27)  
*Failed to set packet's head.*
- #define `NETIO_LOG_PACKET_SET_TAIL_EC` (`UDP_LOG_BASE` + 28)  
*Failed to set packet's tail.*
- #define `NETIO_LOG_PACKET_INITIALIZE_EC` (`UDP_LOG_BASE` + 29)  
*Failed to initialize packet.*
- #define `NETIO_LOG_LOOP_DELETE_TABLE_EC` (`NETIO_LOG_BASE` + 100)  
*Failed to delete a database table for a NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_CREATE_TABLE_EC` (`NETIO_LOG_BASE` + 101)  
*Failed to create a database table for a NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_SELECT_TABLE_EC` (`NETIO_LOG_BASE` + 102)  
*Failed to select a valid record when sending with the NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_FWD_EC` (`NETIO_LOG_BASE` + 103)  
*Failed to send forward with the NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_BINDX_DB_EC` (`NETIO_LOG_BASE` + 104)  
*Failed to bind an external interface with the NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_UNBINDX_DB_EC` (`NETIO_LOG_BASE` + 105)  
*Failed to unbind an external interface with the NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_SET_LENGTH_EC` (`NETIO_LOG_BASE` + 106)

- Failed to set the length of an address sequence.*

  - #define `NETIO_LOG_LOOP_INVALID_PROPERTY_EC` (`NETIO_LOG_BASE` + 107)

*Invalid property passed in when creating a loopback interface.*
- #define `NETIO_LOG_LOOP_INITIALIZE_FAILED_EC` (`NETIO_LOG_BASE` + 108)

*Failed to initialize the loopback interface.*
- #define `NETIO_LOG_LOOP_CURSOR_ERROR_EC` (`NETIO_LOG_BASE` + 109)

*An error occurred with a cursor while iterating over a table.*
- #define `NETIO_LOG_LOOP_INVALID_FACTORY_EC` (`NETIO_LOG_BASE` + 110)

*An invalid factory was passed in to create a loopback interface.*
- #define `NETIO_LOG_LOOP_INVALID_COMPONENT_EC` (`NETIO_LOG_BASE` + 111)

*An invalid component was passed in to delete a loopback interface.*
- #define `UDP_LOG_SOCKET_INIT_EC` (`UDP_LOG_BASE` + 200)

*Failed to initialize use of sockets.*
- #define `UDP_LOG_GETHOSTNAME_EC` (`UDP_LOG_BASE` + 201)

*Failed to get the host address given the host name.*
- #define `UDP_LOG_SOCKET_SET_MCAST_EC` (`UDP_LOG_BASE` + 202)

*Failed to set a socket for multicast loopback.*
- #define `UDP_LOG_SOCKET_SET_MCASTIF_EC` (`UDP_LOG_BASE` + 203)

*Failed to set a socket for multicast bound to a specific interface.*
- #define `UDP_LOG_SOCKET_CREATE_EC` (`UDP_LOG_BASE` + 204)

*Failed to create a new socket, the sysrc code is the OS return code and reason for failure.*
- #define `UDP_LOG_SOCKET_SETFD_EC` (`UDP_LOG_BASE` + 205)

*Failed to set the close-on-exec flag for a socket.*
- #define `UDP_LOG_SOCKET_SNDBUF_EC` (`UDP_LOG_BASE` + 206)

*Failed to set the socket send buffer size.*
- #define `UDP_LOG_SOCKET_TTL_EC` (`UDP_LOG_BASE` + 207)

*Failed to set multicast TTL for a socket.*
- #define `UDP_LOG_PACKET_INIT_EC` (`UDP_LOG_BASE` + 208)

*Failed to initialize a receive packet.*
- #define `UDP_LOG_PACKET_HEAD_EC` (`UDP_LOG_BASE` + 209)

*Failed to set the head of a receive packet.*
- #define `UDP_LOG_PACKET_FWD_EC` (`UDP_LOG_BASE` + 210)

*Failed reception upstream for a received packet.*
- #define `UDP_LOG_NAT_DELETE_EC` (`UDP_LOG_BASE` + 211)

*Failed to delete a database record or index from an internal NAT.*
- #define `UDP_LOG_FINALIZE_EC` (`UDP_LOG_BASE` + 212)

*Failed to finalize the parent NETIO interface.*
- #define `UDP_LOG_SOCKET_REUSE_PORT_EC` (`UDP_LOG_BASE` + 213)

*Failed to set socket to reuse a port or address.*
- #define `UDP_LOG_SOCKET_BIND_EC` (`UDP_LOG_BASE` + 214)

*Failed a socket bind.*
- #define `UDP_LOG_SOCKET_ADDGRP_EC` (`UDP_LOG_BASE` + 215)

*Failed to add membership to a multicast group for a socket.*
- #define `UDP_LOG_PORT_ENTRY_EC` (`UDP_LOG_BASE` + 216)

*Failed to create a bind entry for a receive port.*
- #define `UDP_LOG_PORT_ALLOC_EC` (`UDP_LOG_BASE` + 217)

*Failed to allocate memory for a receive buffer of a specific port.*

- `#define UDP_LOG_SOCKET_RXBUF_EC (UDP_LOG_BASE + 218)`  
*Failed to set the receive buffer size of a socket.*
- `#define UDP_LOG_PORT_ADD_EC (UDP_LOG_BASE + 219)`  
*Failed to add a port entry record to a database table.*
- `#define UDP_LOG_PORT_THREAD_EC (UDP_LOG_BASE + 220)`  
*Failed to create a receive thread for a receive port.*
- `#define UDP_LOG_SOCKET_IFLIST_EC (UDP_LOG_BASE + 221)`  
*Failed to get a list of interface addresses.*
- `#define UDP_LOG_SOCKET_IFFLAGS_EC (UDP_LOG_BASE + 222)`  
*Failed to get the network interface flags or mask of a socket.*
- `#define UDP_LOG_SOCKET_CLOSE_EC (UDP_LOG_BASE + 223)`  
*Failed to close a socket.*
- `#define UDP_LOG_LOADLIB_EC (UDP_LOG_BASE + 224)`  
*Failed to load a dynamic library.*
- `#define UDP_LOG_GET_BUF_SIZE_EC (UDP_LOG_BASE + 225)`  
*Failed to get interface buffer size.*
- `#define UDP_LOG_GET_BUF_ALLOC_EC (UDP_LOG_BASE + 226)`  
*Out of memory to allocate interface buffer.*
- `#define UDP_LOG_GET_IFLIST_EC (UDP_LOG_BASE + 227)`  
*Failed to get an interface list.*
- `#define UDP_LOG_GET_NAMEINFO_EC (UDP_LOG_BASE + 228)`  
*Failed getnameinfo()*
- `#define UDP_LOG_MULTICAST_ENABLE_EC (UDP_LOG_BASE + 229)`  
*Failed to enable multicast loopback.*
- `#define UDP_LOG_UDP_CREATE_TABLE_EC (UDP_LOG_BASE + 230)`  
*Failed to create a database table for a UDP interface.*
- `#define UDP_LOG_UDP_CREATE_INDEX_EC (UDP_LOG_BASE + 231)`  
*Failed to create an index for a UDP interface.*
- `#define UDP_LOG_RECORD_EC (UDP_LOG_BASE + 232)`  
*Failed to create or insert a database record for a UDP interface.*
- `#define UDP_LOG_PROPERTY_FINALIZE_EC (UDP_LOG_BASE + 233)`  
*Failed to finalize UDP interface factory property.*
- `#define UDP_LOG_ALLOC_EC (UDP_LOG_BASE + 234)`  
*Failed to allocate memory for a UDP interface.*
- `#define UDP_LOG_SEND_PING_EC (UDP_LOG_BASE + 235)`  
*Failed sending a ping message, upon adding a route or waking up a receive thread.*
- `#define UDP_LOG_DROPGRP_EC (UDP_LOG_BASE + 236)`  
*Failed to drop membership from a multicast group.*
- `#define UDP_LOG_GET_LENGTH_EC (UDP_LOG_BASE + 237)`  
*Length of address sequence exceeds its maximum.*
- `#define UDP_LOG_SET_LENGTH_EC (UDP_LOG_BASE + 240)`  
*Failed to set the length of an address sequence.*
- `#define UDP_LOG_WSA_STARTUP_EC (UDP_LOG_BASE + 241)`  
*Failed WSStartup()*
- `#define UDP_LOG_WINSOCK_INCOMPATIBLE_EC (UDP_LOG_BASE + 242)`  
*Incompatible version of Winsock, must not be older than 2.0.*
- `#define UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC (UDP_LOG_BASE + 243)`

- Inconsistent max\_message\_size.*

  - #define `UDP_LOG_INITIALIZE_FAILED_EC` (`UDP_LOG_BASE` + 244)
- Failed to initialize the loopback interface.*

  - #define `UDP_LOG_PORT_NOT_FOUND_EC` (`UDP_LOG_BASE` + 245)
- Did not find the specified thread port.*

  - #define `UDP_LOG_CURSOR_ERROR_EC` (`UDP_LOG_BASE` + 246)
- An error occurred while iterating over a cursor.*

  - #define `UDP_LOG_SEND_ERROR_EC` (`UDP_LOG_BASE` + 247)
- Failed to send message.*

  - #define `UDP_LOG_MULTICAST_NOT_ENABLED_EC` (`UDP_LOG_BASE` + 248)
- Multicast address ignored because multicast supported is not compiled in.*

  - #define `UDP_LOG_TRANSPORT_NO_INTERFACES_EC` (`UDP_LOG_BASE` + 249)
- Transport does not have any interfaces.*

  - #define `UDP_LOG_RECV_ERROR_EC` (`UDP_LOG_BASE` + 250)
- Failed to receive message.*

  - #define `UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC` (`UDP_LOG_BASE` + 300)
- The returned factory reference is NULL.*

  - #define `UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC` (`UDP_LOG_BASE` + 301)
- Could not find the factory used as part of a transform rule.*

  - #define `UDP_TRANSFORM_LOG_ALLOC_EC` (`UDP_LOG_BASE` + 302)
- Failed to allocate entry of the specified kind.*

  - #define `UDP_TRANSFORM_LOG_CREATE_EC` (`UDP_LOG_BASE` + 303)
- Failed to create a transformation of the specified kind.*

  - #define `UDP_TRANSFORM_LOG_ADD_EC` (`UDP_LOG_BASE` + 304)
- Failed to add object of specified kind.*

  - #define `UDP_TRANSFORM_LOG_FIND_EC` (`UDP_LOG_BASE` + 305)
- Failed to find a transformation.*

  - #define `UDP_TRANSFORM_LOG_DUPLICATE_EC` (`UDP_LOG_BASE` + 306)
- Duplicate object.*

  - #define `UDP_TRANSFORM_LOG_TRANSFORM_EC` (`UDP_LOG_BASE` + 307)
- Source or destination transformation function failed.*

  - #define `UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC` (`UDP_LOG_BASE` + 308)
- Invalid UDP mode specified.*

  - #define `UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC` (`UDP_LOG_BASE` + 309)
- Invalid transformation UDP locator kind specified.*

  - #define `NETIO_SHMEM_LOG_DELETE_TABLE_EC` (`SHMEM_LOG_BASE` + 1)
- Failed to delete a database table for a NETIO shared memory interface.*

  - #define `NETIO_SHMEM_LOG_CREATE_TABLE_EC` (`SHMEM_LOG_BASE` + 2)
- Failed to create a database table for a NETIO shared memory interface.*

  - #define `NETIO_SHMEM_LOG_SELECT_TABLE_EC` (`SHMEM_LOG_BASE` + 3)
- Failed to select a valid record when sending with the NETIO shared memory.*

  - #define `NETIO_SHMEM_LOG_FWD_EC` (`SHMEM_LOG_BASE` + 4)
- Failed to send forward with the NETIO shared memory interface.*

  - #define `NETIO_SHMEM_LOG_BINDX_DB_EC` (`SHMEM_LOG_BASE` + 5)
- Failed to bind an external interface with the NETIO shared memory interface.*

  - #define `NETIO_SHMEM_LOG_UNBINDX_DB_EC` (`SHMEM_LOG_BASE` + 6)
- Failed to unbind an external interface with the NETIO shared memory interface.*

- `#define NETIO_SHMEM_LOG_SET_LENGTH_EC (SHMEM_LOG_BASE + 7)`  
*Failed to set the length of an address sequence.*
- `#define NETIO_SHMEM_LOG_INVALID_PROPERTY_EC (SHMEM_LOG_BASE + 8)`  
*Invalid property passed in when creating a shared memory interface.*
- `#define NETIO_SHMEM_LOG_INITIALIZE_FAILED_EC (SHMEM_LOG_BASE + 9)`  
*Failed to initialize the shared memory interface.*
- `#define NETIO_SHMEM_LOG_CURSOR_ERROR_EC (SHMEM_LOG_BASE + 10)`  
*An error occurred with a cursor while iterating over a table.*
- `#define NETIO_SHMEM_LOG_INVALID_FACTORY_EC (SHMEM_LOG_BASE + 11)`  
*An invalid factory was passed in to create a shared memory interface.*
- `#define NETIO_SHMEM_LOG_INVALID_COMPONENT_EC (SHMEM_LOG_BASE + 12)`  
*An invalid component was passed in to delete a shared memory interface.*
- `#define NETIO_SHMEM_LOG_INCOMPATIBLE_COOKIE_EC (SHMEM_LOG_BASE + 13)`  
*Found incompatible cookie in shared memory header.*
- `#define NETIO_SHMEM_LOG_INCOMPATIBLE_VERSION_EC (SHMEM_LOG_BASE + 14)`  
*Found incompatible major version in shared memory header.*
- `#define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_SIZE_EC (SHMEM_LOG_BASE + 15)`  
*Found incompatible header size in shared memory header.*
- `#define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_EC (SHMEM_LOG_BASE + 16)`  
*Attempted to attach to shared memory header transport concurrent queue that is incompatible.*
- `#define NETIO_SHMEM_LOG_ADDRESS_IN_USE_EC (SHMEM_LOG_BASE + 17)`  
*Failed to create or attach to a shared memory segment.*
- `#define NETIO_SHMEM_LOG_CHECK_SYSTEM_SETTINGS_EC (SHMEM_LOG_BASE + 18)`  
*Possible failure due to misconfigured system settings on Darwin operating systems.*
- `#define NETIO_SHMEM_LOG_SHMEM_MUTEX_LOCK_FAILED_EC (SHMEM_LOG_BASE + 19)`  
*Failed to lock shared memory mutex protecting the shared memory segment.*
- `#define NETIO_SHMEM_LOG_SHMEM_MUTEX_UNLOCK_FAILED_EC (SHMEM_LOG_BASE + 20)`  
*Failed to unlock shared memory mutex protecting the shared memory segment.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_ALLOCATE_STRUCT_EC (SHMEM_LOG_BASE + 21)`  
*Failed to allocate memory needed by shared memory transport.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_INITIALIZE_EC (SHMEM_LOG_BASE + 22)`  
*Failed to initialize shared memory transport.*
- `#define NETIO_SHMEM_LOG_CREATE_ATTACH_INFINITE_EC (SHMEM_LOG_BASE + 23)`  
*Failed to create or attach. Possible shared memory key conflict.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_INIT_MUTEX_EC (SHMEM_LOG_BASE + 24)`  
*Failed to create a new shared memory mutex or failed to attach to an existing one.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_INIT_SIG_SEM_EC (SHMEM_LOG_BASE + 25)`  
*Failed to create signaling semaphore. A Domain Participant's shared memory transport blocks on a signaling semaphore when waiting for data.*
- `#define NETIO_SHMEM_LOG_FAILED_TAKE_SIGN_SEM_EC (SHMEM_LOG_BASE + 27)`  
*Failed to take a shared memory signaling semaphore.*
- `#define NETIO_SHMEM_LOG_SIG_SEM_SIGNAL_FAILED_EC (SHMEM_LOG_BASE + 28)`  
*Failed to give a shared memory signaling semaphore.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_INIT_SEGMENT_ADDR_EC (SHMEM_LOG_BASE + 29)`  
*Failed to attach to a shared memory segment because it doesn't exist.*
- `#define NETIO_SHMEM_LOG_PACKET_INIT_EC (SHMEM_LOG_BASE + 30)`  
*Failed to initialize a receive packet.*

- #define `NETIO_SHMEM_LOG_FAILED_TO_INIT_CONCURRENT_Q_EC` (`SHMEM_LOG_BASE` + 31)  
*Failed to create a concurrent queue.*
- #define `NETIO_SHMEM_LOG_FAILED_TO_ATTACH_CONCURRENT_Q_EC` (`SHMEM_LOG_BASE` + 32)  
*Failed to attach to a concurrent queue.*
- #define `NETIO_SHMEM_LOG_INCOMPATIBLE_CONCURRENT_Q_EC` (`SHMEM_LOG_BASE` + 33)  
*Attempting to attach to an incompatible concurrent queue with incompatible message\_size\_max.*
- #define `NETIO_SHMEM_LOG_FAILED_TO_GET_TIMESTAMP_EC` (`SHMEM_LOG_BASE` + 33)  
*Failed to get timestamp.*
- #define `NETIO_SHMEM_LOG_NONFATAL_SIG_SEM_RC_EC` (`SHMEM_LOG_BASE` + 34)  
*Failed to unlock shared memory mutex.*
- #define `NETIO_SHMEM_LOG_INCONSISTENT_MESSAGE_SIZE_EC` (`SHMEM_LOG_BASE` + 35)  
*Failed to unlock shared memory mutex.*
- #define `NETIO_SHMEM_LOG_RECORD_EC` (`SHMEM_LOG_BASE` + 36)  
*Database operation on record failed.*
- #define `NETIO_SHMEM_LOG_PACKET_HEAD_EC` (`SHMEM_LOG_BASE` + 37)  
*Failed to set packet head.*
- #define `NETIO_SHMEM_LOG_PACKET_FWD_EC` (`SHMEM_LOG_BASE` + 38)  
*Failed reception upstream for a received packet.*
- #define `NETIO_SHMEM_LOG_QUEUE_FULL_EC` (`SHMEM_LOG_BASE` + 39)  
*Failed reception upstream for a received packet.*
- #define `NETIO_SHMEM_LOG_ROUTE_LOOKUP_FAILED_EC` (`SHMEM_LOG_BASE` + 40)  
*Failed to evict find an entry in the route table.*
- #define `NETIO_SHMEM_LOG_ROUTE_DELETE_FAILED_EC` (`SHMEM_LOG_BASE` + 41)  
*Failed to delete an entry from the route table.*
- #define `NETIO_SHMEM_LOG_IGNORE_TRANSPORT_COMPATIBILITY_EC` (`SHMEM_LOG_BASE` + 42)  
*The shared memory transport is ignoring the requested transport minimum compatibility version because it cannot be backward compatible.*
- #define `NETIO_SHMEM_LOG_UNSUPPORTED_TRANSPORT_COMPATIBILITY_EC` (`SHMEM_LOG_BASE` + 43)  
*Unsupported minimum compatibility version because it requires a version of the shared memory transport which is not supported.*

### 6.21.7.1 Detailed Description

Network I/O. ModuleID = 4.

### 6.21.7.2 Macro Definition Documentation

#### NETIO\_LOG\_GETHOST\_BYNAME\_EC

```
#define NETIO_LOG_GETHOST_BYNAME_EC (NETIO_LOG_BASE + 1)
```

Failed to get host address from a string representation.



**NETIO\_LOG\_BIND\_NEW\_EC**

```
#define NETIO_LOG_BIND_NEW_EC (NETIO_LOG_BASE + 2)
```

Failed to allocate memory for a bind-resolver.

**NETIO\_LOG\_BIND\_NEW\_RTABLE\_EC**

```
#define NETIO_LOG_BIND_NEW_RTABLE_EC (NETIO_LOG_BASE + 3)
```

Failed to create a database table for routes of a bind-resolver.

**NETIO\_LOG\_BIND\_CREATE\_RTABLE\_EC**

```
#define NETIO_LOG_BIND_CREATE_RTABLE_EC (NETIO_LOG_BASE + 4)
```

Failed to create route record.

May have exceeded DomainParticipantQos.resource\_limits.max\_receive\_ports or DomainParticipantQos.resource\_limits.local\_writer\_allocation

**NETIO\_LOG\_BIND\_DELETE\_RTABLE\_EC**

```
#define NETIO_LOG_BIND_DELETE_RTABLE_EC (NETIO_LOG_BASE + 5)
```

Failed to delete a database table for routes of a bind\_resolver.

**NETIO\_LOG\_BIND\_SET\_LENGTH\_EC**

```
#define NETIO_LOG_BIND_SET_LENGTH_EC (NETIO_LOG_BASE + 6)
```

Failed to set the length for a sequence of resolved addresses.

**NETIO\_LOG\_BIND\_SET\_MAX\_EC**

```
#define NETIO_LOG_BIND_SET_MAX_EC (NETIO_LOG_BASE + 7)
```

Failed to set the maximum length of an internal sequence of addresses.

**NETIO\_LOG\_BIND\_EXTERNAL\_FAILED\_EC**

```
#define NETIO_LOG_BIND_EXTERNAL_FAILED_EC (NETIO_LOG_BASE + 8)
```

An interface failed on bind\_external.

**NETIO\_LOG\_UNBIND\_EXTERNAL\_FAILED\_EC**

```
#define NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC (NETIO_LOG_BASE + 10)
```

An interface failed on unbind\_external.

**NETIO\_LOG\_ROUTE\_RTABLE\_ALLOC\_EC**

```
#define NETIO_LOG_ROUTE_RTABLE_ALLOC_EC (NETIO_LOG_BASE + 11)
```

Failed to allocate memory for a route resolver.

**NETIO\_LOG\_ROUTE\_RTABLE\_CREATE\_EC**

```
#define NETIO_LOG_ROUTE_RTABLE_CREATE_EC (NETIO_LOG_BASE + 12)
```

Failed to create a database table for routes of a route-resolver.

**NETIO\_LOG\_ROUTE\_RTABLE\_DELETE\_EC**

```
#define NETIO_LOG_ROUTE_RTABLE_DELETE_EC (NETIO_LOG_BASE + 13)
```

Failed to delete a database table for routes of a route-resolver.

**NETIO\_LOG\_ROUTE\_RTABLE\_ADD\_EC**

```
#define NETIO_LOG_ROUTE_RTABLE_ADD_EC (NETIO_LOG_BASE + 14)
```

Failed to create route resolver record.

May have exceeded DomainParticipantQos.resource\_limits.max\_destination\_ports

**NETIO\_LOG\_ROUTE\_RTABLE\_UPDATE\_EC**

```
#define NETIO_LOG_ROUTE_RTABLE_UPDATE_EC (NETIO_LOG_BASE + 15)
```

Failed to find routes to update for a route-resolver.

**NETIO\_LOG\_AR\_ALLOC\_FAILED\_EC**

```
#define NETIO_LOG_AR_ALLOC_FAILED_EC (NETIO_LOG_BASE + 16)
```

Failed to allocate memory for address resolver.

**NETIO\_LOG\_AR\_ADD\_INVALID\_INTERFACE\_EC**

```
#define NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC (NETIO_LOG_BASE + 17)
```

An attempt was made to add an existing interface with a different port resolver to the address resolver.

**NETIO\_LOG\_AR\_ADDRESS\_RESOLVE\_FAILED\_EC**

```
#define NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC (NETIO_LOG_BASE + 18)
```

An interface failed to resolve an address.

**NETIO\_LOG\_AR\_PORT\_RESOLVE\_FAILED\_EC**

```
#define NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC (NETIO_LOG_BASE + 19)
```

An interface failed to resolve a port.

**NETIO\_LOG\_AR\_CONTEXT\_UNINITIALIZED\_EC**

```
#define NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC (NETIO_LOG_BASE + 21)
```

NETIO\_AddressResolver\_resolve/get\_next was called with an uninitialized context.

**NETIO\_LOG\_AR\_INVALID\_TOKEN\_EC**

```
#define NETIO_LOG_AR_INVALID_TOKEN_EC (NETIO_LOG_BASE + 22)
```

An invalid token was encountered in the address string.

**NETIO\_LOG\_AR\_ADDRESS\_STRING\_EXCEEDED\_EC**

```
#define NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC (NETIO_LOG_BASE + 23)
```

The length of the address exceeded the address buffer passed in.

**NETIO\_LOG\_INVALID\_ADDRESS\_INDEX\_EC**

```
#define NETIO_LOG_INVALID_ADDRESS_INDEX_EC (NETIO_LOG_BASE + 24)
```

An address index was not parsed correctly.

When parsing an address string the index was not specified correctly. Legal formats are: N@ [,N]@ [N]@ [M,N]@

Where N and M are integers  $\geq 0$

**NETIO\_LOG\_ILLEGAL\_ROUTE\_OPERATION\_EC**

```
#define NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC (NETIO_LOG_BASE + 25)
```

Illegal route operation attempted.

**NETIO\_LOG\_SET\_NAME\_EC**

```
#define NETIO_LOG_SET_NAME_EC (UDP_LOG_BASE + 26)
```

Failed to set the name of a runtime component interface.

**NETIO\_LOG\_PACKET\_SET\_HEAD\_EC**

```
#define NETIO_LOG_PACKET_SET_HEAD_EC (UDP_LOG_BASE + 27)
```

Failed to set packet's head.

NETIO\_Packet\_set\_head() failed, due to an invalid delta that would have moved the head position out-of-bounds of the packet.

**NETIO\_LOG\_PACKET\_SET\_TAIL\_EC**

```
#define NETIO_LOG_PACKET_SET_TAIL_EC (UDP_LOG_BASE + 28)
```

Failed to set packet's tail.

NETIO\_Packet\_set\_tail() failed, due to an invalid delta that would have moved the tail position out-of-bounds of the packet.

**NETIO\_LOG\_PACKET\_INITIALIZE\_EC**

```
#define NETIO_LOG_PACKET_INITIALIZE_EC (UDP_LOG_BASE + 29)
```

Failed to initialize packet.

NETIO\_Packet\_initialize() failed due to invalid arguments

**NETIO\_LOG\_LOOP\_DELETE\_TABLE\_EC**

```
#define NETIO_LOG_LOOP_DELETE_TABLE_EC (NETIO_LOG_BASE + 100)
```

Failed to delete a database table for a NETIO loopback interface.

**NETIO\_LOG\_LOOP\_CREATE\_TABLE\_EC**

```
#define NETIO_LOG_LOOP_CREATE_TABLE_EC (NETIO_LOG_BASE + 101)
```

Failed to create a database table for a NETIO loopback interface.

**NETIO\_LOG\_LOOP\_SELECT\_TABLE\_EC**

```
#define NETIO_LOG_LOOP_SELECT_TABLE_EC (NETIO_LOG_BASE + 102)
```

Failed to select a valid record when sending with the NETIO loopback interface.

**NETIO\_LOG\_LOOP\_FWD\_EC**

```
#define NETIO_LOG_LOOP_FWD_EC (NETIO_LOG_BASE + 103)
```

Failed to send forward with the NETIO loopback interface.

**NETIO\_LOG\_LOOP\_BINDX\_DB\_EC**

```
#define NETIO_LOG_LOOP_BINDX_DB_EC (NETIO_LOG_BASE + 104)
```

Failed to bind an external interface with the NETIO loopback interface.

**NETIO\_LOG\_LOOP\_UNBINDX\_DB\_EC**

```
#define NETIO_LOG_LOOP_UNBINDX_DB_EC (NETIO_LOG_BASE + 105)
```

Failed to unbind an external interface with the NETIO loopback interface.

**NETIO\_LOG\_LOOP\_SET\_LENGTH\_EC**

```
#define NETIO_LOG_LOOP_SET_LENGTH_EC (NETIO_LOG_BASE + 106)
```

Failed to set the length of an address sequence.

**NETIO\_LOG\_LOOP\_INVALID\_PROPERTY\_EC**

```
#define NETIO_LOG_LOOP_INVALID_PROPERTY_EC (NETIO_LOG_BASE + 107)
```

Invalid property passed in when creating a loopback interface.

**NETIO\_LOG\_LOOP\_INITIALIZE\_FAILED\_EC**

```
#define NETIO_LOG_LOOP_INITIALIZE_FAILED_EC (NETIO_LOG_BASE + 108)
```

Failed to initialize the loopback interface.

**NETIO\_LOG\_LOOP\_CURSOR\_ERROR\_EC**

```
#define NETIO_LOG_LOOP_CURSOR_ERROR_EC (NETIO_LOG_BASE + 109)
```

An error occurred with a cursor while iterating over a table.

**NETIO\_LOG\_LOOP\_INVALID\_FACTORY\_EC**

```
#define NETIO_LOG_LOOP_INVALID_FACTORY_EC (NETIO_LOG_BASE + 110)
```

An invalid factory was passed in to create a loopback interface.

**NETIO\_LOG\_LOOP\_INVALID\_COMPONENT\_EC**

```
#define NETIO_LOG_LOOP_INVALID_COMPONENT_EC (NETIO_LOG_BASE + 111)
```

An invalid component was passed in to delete a loopback interface.

**UDP\_LOG\_SOCKET\_INIT\_EC**

```
#define UDP_LOG_SOCKET_INIT_EC (UDP_LOG_BASE + 200)
```

Failed to initialize use of sockets.

**UDP\_LOG\_GETHOSTNAME\_EC**

```
#define UDP_LOG_GETHOSTNAME_EC (UDP_LOG_BASE + 201)
```

Failed to get the host address given the host name.

**UDP\_LOG\_SOCKET\_SET\_MCAST\_EC**

```
#define UDP_LOG_SOCKET_SET_MCAST_EC (UDP_LOG_BASE + 202)
```

Failed to set a socket for multicast loopback.

**UDP\_LOG\_SOCKET\_SET\_MCASTIF\_EC**

```
#define UDP_LOG_SOCKET_SET_MCASTIF_EC (UDP_LOG_BASE + 203)
```

Failed to set a socket for multicast bound to a specific interface.

**UDP\_LOG\_SOCKET\_CREATE\_EC**

```
#define UDP_LOG_SOCKET_CREATE_EC (UDP_LOG_BASE + 204)
```

Failed to create a new socket, the sysrc code is the OS return code and reason for failure.

**UDP\_LOG\_SOCKET\_SETFD\_EC**

```
#define UDP_LOG_SOCKET_SETFD_EC (UDP_LOG_BASE + 205)
```

Failed to set the close-on-exec flag for a socket.

**UDP\_LOG\_SOCKET\_SNDBUF\_EC**

```
#define UDP_LOG_SOCKET_SNDBUF_EC (UDP_LOG_BASE + 206)
```

Failed to set the socket send buffer size.

**UDP\_LOG\_SOCKET\_TTL\_EC**

```
#define UDP_LOG_SOCKET_TTL_EC (UDP_LOG_BASE + 207)
```

Failed to set multicast TTL for a socket.

**UDP\_LOG\_PACKET\_INIT\_EC**

```
#define UDP_LOG_PACKET_INIT_EC (UDP_LOG_BASE + 208)
```

Failed to initialize a receive packet.

**UDP\_LOG\_PACKET\_HEAD\_EC**

```
#define UDP_LOG_PACKET_HEAD_EC (UDP_LOG_BASE + 209)
```

Failed to set the head of a receive packet.

**UDP\_LOG\_PACKET\_FWD\_EC**

```
#define UDP_LOG_PACKET_FWD_EC (UDP_LOG_BASE + 210)
```

Failed reception upstream for a received packet.

**UDP\_LOG\_NAT\_DELETE\_EC**

```
#define UDP_LOG_NAT_DELETE_EC (UDP_LOG_BASE + 211)
```

Failed to delete a database record or index from an internal NAT.

**UDP\_LOG\_FINALIZE\_EC**

```
#define UDP_LOG_FINALIZE_EC (UDP_LOG_BASE + 212)
```

Failed to finalize the parent NETIO interface.

**UDP\_LOG\_SOCKET\_REUSE\_PORT\_EC**

```
#define UDP_LOG_SOCKET_REUSE_PORT_EC (UDP_LOG_BASE + 213)
```

Failed to set socket to reuse a port or address.

**UDP\_LOG\_SOCKET\_BIND\_EC**

```
#define UDP_LOG_SOCKET_BIND_EC (UDP_LOG_BASE + 214)
```

Failed a socket bind.

**UDP\_LOG\_SOCKET\_ADDGRP\_EC**

```
#define UDP_LOG_SOCKET_ADDGRP_EC (UDP_LOG_BASE + 215)
```

Failed to add membership to a multicast group for a socket.

**UDP\_LOG\_PORT\_ENTRY\_EC**

```
#define UDP_LOG_PORT_ENTRY_EC (UDP_LOG_BASE + 216)
```

Failed to create a bind entry for a receive port.



**UDP\_LOG\_PORT\_ALLOC\_EC**

```
#define UDP_LOG_PORT_ALLOC_EC (UDP_LOG_BASE + 217)
```

Failed to allocate memory for a receive buffer of a specific port.

**UDP\_LOG\_SOCKET\_RXBUF\_EC**

```
#define UDP_LOG_SOCKET_RXBUF_EC (UDP_LOG_BASE + 218)
```

Failed to set the receive buffer size of a socket.

**UDP\_LOG\_PORT\_ADD\_EC**

```
#define UDP_LOG_PORT_ADD_EC (UDP_LOG_BASE + 219)
```

Failed to add a port entry record to a database table.

**UDP\_LOG\_PORT\_THREAD\_EC**

```
#define UDP_LOG_PORT_THREAD_EC (UDP_LOG_BASE + 220)
```

Failed to create a receive thread for a receive port.

**UDP\_LOG\_SOCKET\_IFLIST\_EC**

```
#define UDP_LOG_SOCKET_IFLIST_EC (UDP_LOG_BASE + 221)
```

Failed to get a list of interface addresses.

**UDP\_LOG\_SOCKET\_IFFLAGS\_EC**

```
#define UDP_LOG_SOCKET_IFFLAGS_EC (UDP_LOG_BASE + 222)
```

Failed to get the network interface flags or mask of a socket.

**UDP\_LOG\_SOCKET\_CLOSE\_EC**

```
#define UDP_LOG_SOCKET_CLOSE_EC (UDP_LOG_BASE + 223)
```

Failed to close a socket.

**UDP\_LOG\_LOADLIB\_EC**

```
#define UDP_LOG_LOADLIB_EC (UDP_LOG_BASE + 224)
```

Failed to load a dynamic library.

**UDP\_LOG\_GET\_BUF\_SIZE\_EC**

```
#define UDP_LOG_GET_BUF_SIZE_EC (UDP_LOG_BASE + 225)
```

Failed to get interface buffer size.

**UDP\_LOG\_GET\_BUF\_ALLOC\_EC**

```
#define UDP_LOG_GET_BUF_ALLOC_EC (UDP_LOG_BASE + 226)
```

Out of memory to allocate interface buffer.

**UDP\_LOG\_GET\_IFLIST\_EC**

```
#define UDP_LOG_GET_IFLIST_EC (UDP_LOG_BASE + 227)
```

Failed to get an interface list.

**UDP\_LOG\_GET\_NAMEINFO\_EC**

```
#define UDP_LOG_GET_NAMEINFO_EC (UDP_LOG_BASE + 228)
```

Failed getnameinfo()

**UDP\_LOG\_MULTICAST\_ENABLE\_EC**

```
#define UDP_LOG_MULTICAST_ENABLE_EC (UDP_LOG_BASE + 229)
```

Failed to enable multicast loopback.

**UDP\_LOG\_UDP\_CREATE\_TABLE\_EC**

```
#define UDP_LOG_UDP_CREATE_TABLE_EC (UDP_LOG_BASE + 230)
```

Failed to create a database table for a UDP interface.

**UDP\_LOG\_UDP\_CREATE\_INDEX\_EC**

```
#define UDP_LOG_UDP_CREATE_INDEX_EC (UDP_LOG_BASE + 231)
```

Failed to create an index for a UDP interface.

**UDP\_LOG\_RECORD\_EC**

```
#define UDP_LOG_RECORD_EC (UDP_LOG_BASE + 232)
```

Failed to create or insert a database record for a UDP interface.

**UDP\_LOG\_PROPERTY\_FINALIZE\_EC**

```
#define UDP_LOG_PROPERTY_FINALIZE_EC (UDP_LOG_BASE + 233)
```

Failed to finalize UDP interface factory property.

**UDP\_LOG\_ALLOC\_EC**

```
#define UDP_LOG_ALLOC_EC (UDP_LOG_BASE + 234)
```

Failed to allocate memory for a UDP interface.

**UDP\_LOG\_SEND\_PING\_EC**

```
#define UDP_LOG_SEND_PING_EC (UDP_LOG_BASE + 235)
```

Failed sending a ping message, upon adding a route or waking up a receive thread.

**UDP\_LOG\_DROPGRP\_EC**

```
#define UDP_LOG_DROPGRP_EC (UDP_LOG_BASE + 236)
```

Failed to drop membership from a multicast group.

**UDP\_LOG\_GET\_LENGTH\_EC**

```
#define UDP_LOG_GET_LENGTH_EC (UDP_LOG_BASE + 237)
```

Length of address sequence exceeds its maximum.

**UDP\_LOG\_SET\_LENGTH\_EC**

```
#define UDP_LOG_SET_LENGTH_EC (UDP_LOG_BASE + 240)
```

Failed to set the length of an address sequence.

**UDP\_LOG\_WSA\_STARTUP\_EC**

```
#define UDP_LOG_WSA_STARTUP_EC (UDP_LOG_BASE + 241)
```

Failed WSStartup()

**UDP\_LOG\_WINSOCK\_INCOMPATIBLE\_EC**

```
#define UDP_LOG_WINSOCK_INCOMPATIBLE_EC (UDP_LOG_BASE + 242)
```

Incompatible version of Winsock, must not be older than 2.0.

**UDP\_LOG\_INCONSISTENT\_MAX\_MESSAGE\_SIZE\_EC**

```
#define UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC (UDP_LOG_BASE + 243)
```

Inconsistent max\_message\_size.

UDP\_InterfaceProperty.max\_message\_size is set to an inconsistent value, larger than UDP\_InterfaceProperty.max\_send\_buffer\_size and/or larger than UDP\_InterfaceProperty.max\_receive\_buffer\_size .

**UDP\_LOG\_INITIALIZE\_FAILED\_EC**

```
#define UDP_LOG_INITIALIZE_FAILED_EC (UDP_LOG_BASE + 244)
```

Failed to initialize the loopback interface.

**UDP\_LOG\_PORT\_NOT\_FOUND\_EC**

```
#define UDP_LOG_PORT_NOT_FOUND_EC (UDP_LOG_BASE + 245)
```

Did not find the specified thread port.

**UDP\_LOG\_CURSOR\_ERROR\_EC**

```
#define UDP_LOG_CURSOR_ERROR_EC (UDP_LOG_BASE + 246)
```

An error occurred while iterating over a cursor.

**UDP\_LOG\_SEND\_ERROR\_EC**

```
#define UDP_LOG_SEND_ERROR_EC (UDP_LOG_BASE + 247)
```

Failed to send message.

A message could not be sent due to system error

**UDP\_LOG\_MULTICAST\_NOT\_ENABLED\_EC**

```
#define UDP_LOG_MULTICAST_NOT_ENABLED_EC (UDP_LOG_BASE + 248)
```

Multicast address ignored because multicast supported is not compiled in.

A multicast address was specified as an address, but multicast support has not been enabled with `NETIO_CONFIG_ENABLE_MULTICAST` in `netio_config.h`

**UDP\_LOG\_TRANSPORT\_NO\_INTERFACES\_EC**

```
#define UDP_LOG_TRANSPORT_NO_INTERFACES_EC (UDP_LOG_BASE + 249)
```

Transport does not have any interfaces.

UDP configuration might be wrong.

**UDP\_LOG\_RECV\_ERROR\_EC**

```
#define UDP_LOG_RECV_ERROR_EC (UDP_LOG_BASE + 250)
```

Failed to receive message.

A message could not be received due to system error

**UDP\_TRANSFORM\_LOG\_INVALID\_FACTORY\_REFERENCE\_EC**

```
#define UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC (UDP_LOG_BASE + 300)
```

The returned factory reference is NULL.

This error indicates that the system has run out of memory

**UDP\_TRANSFORM\_LOG\_FACTORY\_NOT\_FOUND\_EC**

```
#define UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC (UDP_LOG_BASE + 301)
```

Could not find the factory used as part of a transform rule.

This error indicates that a factory for the specified rule has not been registered before the UDP transport is registered.

**UDP\_TRANSFORM\_LOG\_ALLOC\_EC**

```
#define UDP_TRANSFORM_LOG_ALLOC_EC (UDP_LOG_BASE + 302)
```

Failed to allocate entry of the specified kind.

**UDP\_TRANSFORM\_LOG\_CREATE\_EC**

```
#define UDP_TRANSFORM_LOG_CREATE_EC (UDP_LOG_BASE + 303)
```

Failed to create a transformation of the specified kind.

**UDP\_TRANSFORM\_LOG\_ADD\_EC**

```
#define UDP_TRANSFORM_LOG_ADD_EC (UDP_LOG_BASE + 304)
```

Failed to add object of specified kind.

**UDP\_TRANSFORM\_LOG\_FIND\_EC**

```
#define UDP_TRANSFORM_LOG_FIND_EC (UDP_LOG_BASE + 305)
```

Failed to find a transformation.

**UDP\_TRANSFORM\_LOG\_DUPLICATE\_EC**

```
#define UDP_TRANSFORM_LOG_DUPLICATE_EC (UDP_LOG_BASE + 306)
```

Duplicate object.

**UDP\_TRANSFORM\_LOG\_TRANSFORM\_EC**

```
#define UDP_TRANSFORM_LOG_TRANSFORM_EC (UDP_LOG_BASE + 307)
```

Source or destination transformation function failed.

**UDP\_TRANSFORM\_LOG\_INVALID\_UDP\_MODE\_EC**

```
#define UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC (UDP_LOG_BASE + 308)
```

Invalid UDP mode specified.

**UDP\_TRANSFORM\_LOG\_INVALID\_TUDP\_LOCATOR\_KIND\_EC**

```
#define UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC (UDP_LOG_BASE + 309)
```

Invalid transformation UDP locator kind specified.

[UDP\\_InterfaceFactoryProperty::transform\\_locator\\_kind](#) must be greater or equal than `NETIO_ADDRESS_KIND_USER` and not equal to `NETIO_ADDRESS_KIND_SHMEM` or `NETIO_ADDRESS_KIND_INTRA`.

**NETIO\_SHMEM\_LOG\_DELETE\_TABLE\_EC**

```
#define NETIO_SHMEM_LOG_DELETE_TABLE_EC (SHMEM_LOG_BASE + 1)
```

Failed to delete a database table for a NETIO shared memory interface.

**NETIO\_SHMEM\_LOG\_CREATE\_TABLE\_EC**

```
#define NETIO_SHMEM_LOG_CREATE_TABLE_EC (SHMEM_LOG_BASE + 2)
```

Failed to create a database table for a NETIO shared memory interface.

**NETIO\_SHMEM\_LOG\_SELECT\_TABLE\_EC**

```
#define NETIO_SHMEM_LOG_SELECT_TABLE_EC (SHMEM_LOG_BASE + 3)
```

Failed to select a valid record when sending with the NETIO shared memory.

**NETIO\_SHMEM\_LOG\_FWD\_EC**

```
#define NETIO_SHMEM_LOG_FWD_EC (SHMEM_LOG_BASE + 4)
```

Failed to send forward with the NETIO shared memory interface.

**NETIO\_SHMEM\_LOG\_BINDX\_DB\_EC**

```
#define NETIO_SHMEM_LOG_BINDX_DB_EC (SHMEM_LOG_BASE + 5)
```

Failed to bind an external interface with the NETIO shared memory interface.

**NETIO\_SHMEM\_LOG\_UNBINDX\_DB\_EC**

```
#define NETIO_SHMEM_LOG_UNBINDX_DB_EC (SHMEM_LOG_BASE + 6)
```

Failed to unbind an external interface with the NETIO shared memory interface.

**NETIO\_SHMEM\_LOG\_SET\_LENGTH\_EC**

```
#define NETIO_SHMEM_LOG_SET_LENGTH_EC (SHMEM_LOG_BASE + 7)
```

Failed to set the length of an address sequence.

**NETIO\_SHMEM\_LOG\_INVALID\_PROPERTY\_EC**

```
#define NETIO_SHMEM_LOG_INVALID_PROPERTY_EC (SHMEM_LOG_BASE + 8)
```

Invalid property passed in when creating a shared memory interface.

**NETIO\_SHMEM\_LOG\_INITIALIZE\_FAILED\_EC**

```
#define NETIO_SHMEM_LOG_INITIALIZE_FAILED_EC (SHMEM_LOG_BASE + 9)
```

Failed to initialize the shared memory interface.

**NETIO\_SHMEM\_LOG\_CURSOR\_ERROR\_EC**

```
#define NETIO_SHMEM_LOG_CURSOR_ERROR_EC (SHMEM_LOG_BASE + 10)
```

An error occurred with a cursor while iterating over a table.

**NETIO\_SHMEM\_LOG\_INVALID\_FACTORY\_EC**

```
#define NETIO_SHMEM_LOG_INVALID_FACTORY_EC (SHMEM_LOG_BASE + 11)
```

An invalid factory was passed in to create a shared memory interface.

**NETIO\_SHMEM\_LOG\_INVALID\_COMPONENT\_EC**

```
#define NETIO_SHMEM_LOG_INVALID_COMPONENT_EC (SHMEM_LOG_BASE + 12)
```

An invalid component was passed in to delete a shared memory interface.

**NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_COOKIE\_EC**

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_COOKIE_EC (SHMEM_LOG_BASE + 13)
```

Found incompatible cookie in shared memory header.



**NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_VERSION\_EC**

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_VERSION_EC (SHMEM_LOG_BASE + 14)
```

Found incompatible major version in shared memory header.

**NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_HEADER\_SIZE\_EC**

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_SIZE_EC (SHMEM_LOG_BASE + 15)
```

Found incompatible header size in shared memory header.

**NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_HEADER\_EC**

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_HEADER_EC (SHMEM_LOG_BASE + 16)
```

Attempted to attach to shared memory header transport concurrent queue that is incompatible.

**NETIO\_SHMEM\_LOG\_ADDRESS\_IN\_USE\_EC**

```
#define NETIO_SHMEM_LOG_ADDRESS_IN_USE_EC (SHMEM_LOG_BASE + 17)
```

Failed to create or attach to a shared memory segment.

**NETIO\_SHMEM\_LOG\_CHECK\_SYSTEM\_SETTINGS\_EC**

```
#define NETIO_SHMEM_LOG_CHECK_SYSTEM_SETTINGS_EC (SHMEM_LOG_BASE + 18)
```

Possible failure due to misconfigured system settings on Darwin operating systems.

**NETIO\_SHMEM\_LOG\_SHMEM\_MUTEX\_LOCK\_FAILED\_EC**

```
#define NETIO_SHMEM_LOG_SHMEM_MUTEX_LOCK_FAILED_EC (SHMEM_LOG_BASE + 19)
```

Failed to lock shared memory mutex protecting the shared memory segment.

**NETIO\_SHMEM\_LOG\_SHMEM\_MUTEX\_UNLOCK\_FAILED\_EC**

```
#define NETIO_SHMEM_LOG_SHMEM_MUTEX_UNLOCK_FAILED_EC (SHMEM_LOG_BASE + 20)
```

Failed to unlock shared memory mutex protecting the shared memory segment.

**NETIO\_SHMEM\_LOG\_FAILED\_TO\_ALLOCATE\_STRUCT\_EC**

```
#define NETIO_SHMEM_LOG_FAILED_TO_ALLOCATE_STRUCT_EC (SHMEM_LOG_BASE + 21)
```

Failed to allocate memory needed by shared memory transport.

**NETIO\_SHMEM\_LOG\_FAILED\_TO\_INITIALIZE\_EC**

```
#define NETIO_SHMEM_LOG_FAILED_TO_INITIALIZE_EC (SHMEM_LOG_BASE + 22)
```

Failed to initialize shared memory transport.

**NETIO\_SHMEM\_LOG\_CREATE\_ATTACH\_INFINITE\_EC**

```
#define NETIO_SHMEM_LOG_CREATE_ATTACH_INFINITE_EC (SHMEM_LOG_BASE + 23)
```

Failed to create or attach. Possible shared memory key conflict.

**NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_MUTEX\_EC**

```
#define NETIO_SHMEM_LOG_FAILED_TO_INIT_MUTEX_EC (SHMEM_LOG_BASE + 24)
```

Failed to create a new shared memory mutex or failed to attach to an existing one.

**NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_SIG\_SEM\_EC**

```
#define NETIO_SHMEM_LOG_FAILED_TO_INIT_SIG_SEM_EC (SHMEM_LOG_BASE + 25)
```

Failed to create signaling semaphore. A Domain Participant's shared memory transport blocks on a signaling semaphore when waiting for data.

**NETIO\_SHMEM\_LOG\_FAILED\_TAKE\_SIGN\_SEM\_EC**

```
#define NETIO_SHMEM_LOG_FAILED_TAKE_SIGN_SEM_EC (SHMEM_LOG_BASE + 27)
```

Failed to take a shared memory signaling semaphore.

**NETIO\_SHMEM\_LOG\_SIG\_SEM\_SIGNAL\_FAILED\_EC**

```
#define NETIO_SHMEM_LOG_SIG_SEM_SIGNAL_FAILED_EC (SHMEM_LOG_BASE + 28)
```

Failed to give a shared memory signaling semaphore.

**NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_SEGMENT\_ADDR\_EC**

```
#define NETIO_SHMEM_LOG_FAILED_TO_INIT_SEGMENT_ADDR_EC (SHMEM_LOG_BASE + 29)
```

Failed to attach to a shared memory segment because it doesn't exist.

**NETIO\_SHMEM\_LOG\_PACKET\_INIT\_EC**

```
#define NETIO_SHMEM_LOG_PACKET_INIT_EC (SHMEM_LOG_BASE + 30)
```

Failed to initialize a receive packet.

**NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_CONCURRENT\_Q\_EC**

```
#define NETIO_SHMEM_LOG_FAILED_TO_INIT_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 31)
```

Failed to create a concurrent queue.

**NETIO\_SHMEM\_LOG\_FAILED\_TO\_ATTACH\_CONCURRENT\_Q\_EC**

```
#define NETIO_SHMEM_LOG_FAILED_TO_ATTACH_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 32)
```

Failed to attach to a concurrent queue.

**NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_CONCURRENT\_Q\_EC**

```
#define NETIO_SHMEM_LOG_INCOMPATIBLE_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 33)
```

Attempting to attach to an incompatible concurrent queue with incompatible message\_size\_max.

**NETIO\_SHMEM\_LOG\_FAILED\_TO\_GET\_TIMESTAMP\_EC**

```
#define NETIO_SHMEM_LOG_FAILED_TO_GET_TIMESTAMP_EC (SHMEM_LOG_BASE + 33)
```

Failed to get timestamp.

**NETIO\_SHMEM\_LOG\_NONFATAL\_SIG\_SEM\_RC\_EC**

```
#define NETIO_SHMEM_LOG_NONFATAL_SIG_SEM_RC_EC (SHMEM_LOG_BASE + 34)
```

Failed to unlock shared memory mutex.

**NETIO\_SHMEM\_LOG\_INCONSISTENT\_MESSAGE\_SIZE\_EC**

```
#define NETIO_SHMEM_LOG_INCONSISTENT_MESSAGE_SIZE_EC (SHMEM_LOG_BASE + 35)
```

Failed to unlock shared memory mutex.

**NETIO\_SHMEM\_LOG\_RECORD\_EC**

```
#define NETIO_SHMEM_LOG_RECORD_EC (SHMEM_LOG_BASE + 36)
```

Database operation on record failed.

**NETIO\_SHMEM\_LOG\_PACKET\_HEAD\_EC**

```
#define NETIO_SHMEM_LOG_PACKET_HEAD_EC (SHMEM_LOG_BASE + 37)
```

Failed to set packet head.

**NETIO\_SHMEM\_LOG\_PACKET\_FWD\_EC**

```
#define NETIO_SHMEM_LOG_PACKET_FWD_EC (SHMEM_LOG_BASE + 38)
```

Failed reception upstream for a received packet.

**NETIO\_SHMEM\_LOG\_QUEUE\_FULL\_EC**

```
#define NETIO_SHMEM_LOG_QUEUE_FULL_EC (SHMEM_LOG_BASE + 39)
```

Failed reception upstream for a received packet.

**NETIO\_SHMEM\_LOG\_ROUTE\_LOOKUP\_FAILED\_EC**

```
#define NETIO_SHMEM_LOG_ROUTE_LOOKUP_FAILED_EC (SHMEM_LOG_BASE + 40)
```

Failed to evict find an entry in the route table.

**NETIO\_SHMEM\_LOG\_ROUTE\_DELETE\_FAILED\_EC**

```
#define NETIO_SHMEM_LOG_ROUTE_DELETE_FAILED_EC (SHMEM_LOG_BASE + 41)
```

Failed to delete an entry from the route table.

## NETIO\_SHMEM\_LOG\_IGNORE\_TRANSPORT\_COMPATIBILITY\_EC

```
#define NETIO_SHMEM_LOG_IGNORE_TRANSPORT_COMPATIBILITY_EC (SHMEM_LOG_BASE + 42)
```

The shared memory transport is ignoring the requested transport minimum compatibility version because it cannot be backward compatible.

## NETIO\_SHMEM\_LOG\_UNSUPPORTED\_TRANSPORT\_COMPATIBILITY\_EC

```
#define NETIO_SHMEM_LOG_UNSUPPORTED_TRANSPORT_COMPATIBILITY_EC (SHMEM_LOG_BASE + 43)
```

Unsupported minimum compatibility version because it requires a version of the shared memory transport which is not supported.

## 6.21.8 RTPS

Real-Time Publish-Subscribe. ModuleID = 6.

### Macros

- `#define RTPS_LOG_INITIALIZE_INTERFACE_EC (RTPS_LOG_BASE + 1)`  
*Failed to initialize an RTPS interface.*
- `#define RTPS_LOG_ALLOCATE_EC (RTPS_LOG_BASE + 2)`  
*Failed to allocate heap memory for internal resources.*
- `#define RTPS_LOG_CREATE_ROUTE_TABLE_EC (RTPS_LOG_BASE + 3)`  
*Failed to create a database table for storing RTPS route info.*
- `#define RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC (RTPS_LOG_BASE + 4)`  
*Failed to create database index for route-peer info.*
- `#define RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC (RTPS_LOG_BASE + 5)`  
*Failed to create database index for route-transport info.*
- `#define RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC (RTPS_LOG_BASE + 6)`  
*Failed to create database index for bind-peer info.*
- `#define RTPS_LOG_INDEX_ADD_ENTRY_EC (RTPS_LOG_BASE + 8)`  
*Failed to add an entry to a database index.*
- `#define RTPS_LOG_DB_SELECT_ALL_EC (RTPS_LOG_BASE + 9)`  
*Failed to select all entries of a database table.*
- `#define RTPS_LOG_DB_SELECT_MATCH_EC (RTPS_LOG_BASE + 10)`  
*Matching entry not found in a database table.*
- `#define RTPS_LOG_DB_SELECT_RANGE_EC (RTPS_LOG_BASE + 11)`  
*No matching entries found in a database table.*
- `#define RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 12)`  
*Failed to create database entry for route.*
- `#define RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 13)`  
*Failed to insert entry into route table.*
- `#define RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 15)`

- Failed to delete entry from route table.*

  - #define RTPS\_LOG\_DB\_CREATE\_BIND\_ENTRY\_EC (RTPS\_LOG\_BASE + 16)
- Failed to create database entry for bind.*

  - #define RTPS\_LOG\_DB\_INSERT\_BIND\_ENTRY\_EC (RTPS\_LOG\_BASE + 17)
- Failed to insert an entry into the bind table.*

  - #define RTPS\_LOG\_DB\_REMOVE\_BIND\_ENTRY\_EC (RTPS\_LOG\_BASE + 18)
- Failed to remove an entry from the bind table.*

  - #define RTPS\_LOG\_DB\_REMOVE\_EXTERNAL\_BIND\_ENTRY\_EC (RTPS\_LOG\_BASE + 19)
- Failed to remove an entry from the external bind table.*

  - #define RTPS\_LOG\_DB\_DELETE\_EXTERNAL\_BIND\_ENTRY\_EC (RTPS\_LOG\_BASE + 20)
- Failed to delete entry from external bind table.*

  - #define RTPS\_LOG\_SEND\_EC (RTPS\_LOG\_BASE + 21)
- Failed to send a packet due to a lower module failing to send the packet.*

  - #define RTPS\_LOG\_RECEIVE\_EC (RTPS\_LOG\_BASE + 22)
- Failed to receive a packet due to a higher module failing to receive the packet.*

  - #define RTPS\_LOG\_ROUTE\_PACKET\_EC (RTPS\_LOG\_BASE + 23)
- Failed to send a packet, when routing to a lower module or peer.*

  - #define RTPS\_LOG\_FORWARD\_UPSTREAM\_EC (RTPS\_LOG\_BASE + 24)
- Failed to receive a packet, when forwarding to a higher upstream module.*

  - #define RTPS\_LOG\_BAD\_PARAMETER\_EC (RTPS\_LOG\_BASE + 25)
- Bad parameter to an RTPS interface function.*

  - #define RTPS\_LOG\_NOT\_ENABLED\_EC (RTPS\_LOG\_BASE + 26)
- Failed because interface is not enabled.*

  - #define RTPS\_LOG\_READER\_UNSUPPORTED\_EC (RTPS\_LOG\_BASE + 27)
- RTPS reader does not support send.*

  - #define RTPS\_LOG\_INTERFACE\_MISMATCH\_EC (RTPS\_LOG\_BASE + 28)
- Upstream interface does not match source or destination of packet being sent or received.*

  - #define RTPS\_LOG\_FULL\_SEND\_WINDOW\_EC (RTPS\_LOG\_BASE + 29)
- Failed to send due to reaching maximum send window size.*

  - #define RTPS\_LOG\_NETIO\_PACKET\_SET\_TAIL\_EC (RTPS\_LOG\_BASE + 31)
- Failed to set tail of packet to send.*

  - #define RTPS\_LOG\_FUNC\_UNSUPPORTED\_EC (RTPS\_LOG\_BASE + 32)
- RTPS interface does not support this function.*

  - #define RTPS\_LOG\_EXCEEDED\_LIMIT\_TRANSPORTS\_EC (RTPS\_LOG\_BASE + 33)
- Out of transport entries.*

  - #define RTPS\_LOG\_EXCEEDED\_LIMIT\_PEERS\_EC (RTPS\_LOG\_BASE + 34)
- Out of peer entries.*

  - #define RTPS\_LOG\_ASSERT\_PEER\_EC (RTPS\_LOG\_BASE + 35)
- Failed to assert a remote writer or reader.*

  - #define RTPS\_LOG\_ASSERT\_TRANSPORT\_EC (RTPS\_LOG\_BASE + 36)
- Failed to assert a downstream transport.*

  - #define RTPS\_LOG\_FIND\_EXTERNAL\_INTERFACE\_EC (RTPS\_LOG\_BASE + 37)
- Failed to lookup an external interface.*

  - #define RTPS\_LOG\_NONEXISTENT\_ROUTE\_EC (RTPS\_LOG\_BASE + 38)
- Failed to delete a nonexistent route.*

  - #define RTPS\_LOG\_NONEXISTENT\_BIND\_EC (RTPS\_LOG\_BASE + 39)
- Failed to remove a nonexistent bind entry.*

- `#define RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC (RTPS_LOG_BASE + 40)`  
*Failed to remove a nonexistent external bind entry.*
- `#define RTPS_LOG_STALE_ACK_EPOCH_EC (RTPS_LOG_BASE + 41)`  
*Dropped an ACKNACK submessage with an old epoch.*
- `#define RTPS_LOG_STALE_HB_EPOCH_EC (RTPS_LOG_BASE + 42)`  
*Dropped a HEARTBEAT submessage with an old epoch.*
- `#define RTPS_LOG_ACK_EC (RTPS_LOG_BASE + 43)`  
*Failed an acknack() upstream.*
- `#define RTPS_LOG_REQUEST_EC (RTPS_LOG_BASE + 45)`  
*Failed a request() upstream.*
- `#define RTPS_LOG_RETURN_LOAN_EC (RTPS_LOG_BASE + 46)`  
*Failed a return\_loan() upstream.*
- `#define RTPS_LOG_NETIO_PACKET_SET_HEAD_EC (RTPS_LOG_BASE + 47)`  
*Failed to set the head of a packet.*
- `#define RTPS_LOG_SEND_HEARTBEAT_EC (RTPS_LOG_BASE + 48)`  
*Failed to send a HEARTBEAT submessage.*
- `#define RTPS_LOG_SHIFT_BITMAP_EC (RTPS_LOG_BASE + 49)`  
*Failed to shift a bitmap.*
- `#define RTPS_LOG_FULLY_ACKED_READER_EC (RTPS_LOG_BASE + 50)`  
*A Reader is fully acknowledged and does not need to respond to a final HEARTBEAT.*
- `#define RTPS_LOG_DATA_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 51)`  
*Dropping a DATA submessage whose sequence number is outside of a Reader's receive window.*
- `#define RTPS_LOG_DATA_ALREADY_RECEIVED_EC (RTPS_LOG_BASE + 52)`  
*Dropping a DATA submessage whose sequence number was previously received.*
- `#define RTPS_LOG_INTERFACE_NOT_ENABLED_EC (RTPS_LOG_BASE + 53)`  
*Failed to receive a message because interface is not enabled.*
- `#define RTPS_LOG_INVALID_PACKET_EC (RTPS_LOG_BASE + 54)`  
*Dropped a message with an invalid packet header.*
- `#define RTPS_LOG_UNSUPPORTED_INTERFACE_EC (RTPS_LOG_BASE + 55)`  
*Received a message on an unsupported interface.*
- `#define RTPS_LOG_UNKNOWN_SUBMESSAGE_EC (RTPS_LOG_BASE + 56)`  
*Received a submessage with an unknown ID.*
- `#define RTPS_LOG_PROCESS_ACKNACK_EC (RTPS_LOG_BASE + 57)`  
*Failed to process received ACKNACK submessage.*
- `#define RTPS_LOG_PROCESS_DATA_EC (RTPS_LOG_BASE + 58)`  
*Failed to process received DATA submessage.*
- `#define RTPS_LOG_PROCESS_GAP_EC (RTPS_LOG_BASE + 59)`  
*Failed to process received GAP submessage.*
- `#define RTPS_LOG_PROCESS_HEARTBEAT_EC (RTPS_LOG_BASE + 60)`  
*Failed to process received HEARTBEAT submessage.*
- `#define RTPS_LOG_CREATE_HB_EVENT_EC (RTPS_LOG_BASE + 61)`  
*Failed to create periodic HEARTBEAT event.*
- `#define RTPS_LOG_CREATE_EVENT_TIMER_EC (RTPS_LOG_BASE + 62)`  
*Failed to create periodic HEARTBEAT event timer.*
- `#define RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 63)`  
*Failed to set bitmap bit due to out-of-range sequence number.*
- `#define RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC (RTPS_LOG_BASE + 65)`

- Invalid range of sequence numbers to fill in bitmap.*

  - #define RTPS\_LOG\_BITCOUNT\_OUT\_OF\_BOUNDS\_EC (RTPS\_LOG\_BASE + 66)

*Deserialized an invalid out-of-bounds bitcount for a bitmap.*
- #define RTPS\_LOG\_SHIFT\_SN\_OUT\_OF\_RANGE\_EC (RTPS\_LOG\_BASE + 67)

*Failed to shift bitmap due to invalid starting sequence number.*
- #define RTPS\_LOG\_SERIALIZE\_HOST\_ID\_EC (RTPS\_LOG\_BASE + 68)

*Failed to serialize host ID of GUID.*
- #define RTPS\_LOG\_SERIALIZE\_APP\_ID\_EC (RTPS\_LOG\_BASE + 69)

*Failed to serialize app ID of GUID.*
- #define RTPS\_LOG\_SERIALIZE\_INSTANCE\_ID\_EC (RTPS\_LOG\_BASE + 70)

*Failed to serialized instance ID of GUID.*
- #define RTPS\_LOG\_SERIALIZE\_OBJECT\_ID\_EC (RTPS\_LOG\_BASE + 71)

*Failed to serialize object ID of GUID.*
- #define RTPS\_LOG\_DESERIALIZE\_HOST\_ID\_EC (RTPS\_LOG\_BASE + 72)

*Failed to deserialize host ID of GUID.*
- #define RTPS\_LOG\_DESERIALIZE\_APP\_ID\_EC (RTPS\_LOG\_BASE + 73)

*Failed to deserialize app ID of GUID.*
- #define RTPS\_LOG\_DESERIALIZE\_INSTANCE\_ID\_EC (RTPS\_LOG\_BASE + 74)

*Failed to deserialize instance ID of GUID.*
- #define RTPS\_LOG\_DESERIALIZE\_OBJECT\_ID\_EC (RTPS\_LOG\_BASE + 75)

*Failed to deserialize object ID of GUID.*
- #define RTPS\_LOG\_DB\_CREATE\_EXT\_BIND\_ENTRY\_EC (RTPS\_LOG\_BASE + 76)

*Failed to create database entry for external bind table.*
- #define RTPS\_LOG\_BITMAP\_SET\_BIT\_EC (RTPS\_LOG\_BASE + 77)

*Failed to set bit in bitmap.*
- #define RTPS\_LOG\_UNSUPPORTED\_INFO\_REPLY\_EC (RTPS\_LOG\_BASE + 78)

*Received and dropped currently unsupported INFO\_REPLY submessage.*
- #define RTPS\_LOG\_UNSUPPORTED\_INFO\_REPLY\_IP4\_EC (RTPS\_LOG\_BASE + 79)

*Received and dropped currently unsupported INFO\_REPLY\_IP4 submessage.*
- #define RTPS\_LOG\_UNSUPPORTED\_INFO\_SRC\_EC (RTPS\_LOG\_BASE + 80)

*Received and dropped currently unsupported INFO\_SRC submessage.*
- #define RTPS\_LOG\_DELETE\_INDEX\_EC (RTPS\_LOG\_BASE + 81)

*Failed to delete index of a database table.*
- #define RTPS\_LOG\_DELETE\_TABLE\_EC (RTPS\_LOG\_BASE + 82)

*Failed to delete a database table.*
- #define RTPS\_LOG\_DB\_LOCK\_EC (RTPS\_LOG\_BASE + 83)

*Failed to take database lock.*
- #define RTPS\_LOG\_DB\_UNLOCK\_EC (RTPS\_LOG\_BASE + 84)

*Failed to give database lock.*
- #define RTPS\_LOG\_CREATE\_BIND\_TABLE\_EC (RTPS\_LOG\_BASE + 85)

*Failed to create a database table for storing RTPS bind info.*
- #define RTPS\_LOG\_STATUS\_CHANGE\_EC (RTPS\_LOG\_BASE + 86)

*Failed to update status for reliable reader activity changed.*
- #define RTPS\_LOG\_SET\_GAP\_EC (RTPS\_LOG\_BASE + 87)

*Failed to update status for reliable reader activity changed.*
- #define RTPS\_LOG\_WINDOW\_POOL\_ALLOC\_EC (RTPS\_LOG\_BASE + 88)

*Out of memory to allocate send window pool.*



- #define RTPS\_LOG\_GET\_WINDOW\_BUFFER\_EC (RTPS\_LOG\_BASE + 89)  
*Failed to get buffer from send window pool.*
- #define RTPS\_LOG\_INSERT\_WINDOW\_FULL\_EC (RTPS\_LOG\_BASE + 90)  
*Cannot insert sample into full window.*
- #define RTPS\_LOG\_WINDOW\_INSERT\_EC (RTPS\_LOG\_BASE + 91)  
*Failed to insert sample into send window.*
- #define RTPS\_LOG\_TIMER\_DELETE\_TIMEOUT\_EC (RTPS\_LOG\_BASE + 92)  
*Failed to delete a timeout event.*
- #define RTPS\_LOG\_BUFFERPOOL\_DELETE\_EC (RTPS\_LOG\_BASE + 93)  
*Failed to delete a buffer pool.*
- #define RTPS\_LOG\_DIRECT\_SEND\_EC (RTPS\_LOG\_BASE + 94)  
*Failed to send a message to a specific peer.*
- #define RTPS\_LOG\_PACKET\_INITIALIZE\_EC (RTPS\_LOG\_BASE + 95)  
*RTPS writer failed to initialize a packet.*
- #define RTPS\_LOG\_WRITER\_REQUEST\_SENT\_HISTORY\_EC (RTPS\_LOG\_BASE + 96)  
*RTPS reliable writer failed to request and resend history.*
- #define RTPS\_LOG\_BITMAP\_SHIFT\_EC (RTPS\_LOG\_BASE + 97)  
*Failed to shift bitmap to new lead sequence number.*
- #define RTPS\_LOG\_WINDOW\_ADVANCE\_EC (RTPS\_LOG\_BASE + 98)  
*Send window of a remote reader failed to advance.*
- #define RTPS\_LOG\_WINDOW\_ADVANCE\_UNACKED\_EC (RTPS\_LOG\_BASE + 99)  
*Failed when advancing window ahead of unacknowledged sequence number.*
- #define RTPS\_LOG\_LOOKUP\_PEER\_EC (RTPS\_LOG\_BASE + 100)  
*Failed to find peer that should exist.*
- #define RTPS\_LOG\_READER\_SEND\_ACKNACK\_EC (RTPS\_LOG\_BASE + 101)  
*RTPS reader failed to send an ACKNACK message.*
- #define RTPS\_LOG\_FINALIZE\_INTERFACE\_EC (RTPS\_LOG\_BASE + 102)  
*Failed to finalize an RTPS interface.*
- #define RTPS\_LOG\_INDEXER\_REMOVE\_ENTRY\_EC (RTPS\_LOG\_BASE + 103)  
*Failed to remove index entry for external interface upon deleting RTPS interface.*
- #define RTPS\_LOG\_LOOKUP\_EXT\_BIND\_ENTRY\_EC (RTPS\_LOG\_BASE + 104)  
*Failed when looking up external bind entry from database.*
- #define RTPS\_LOG\_SEQ\_INIT\_EC (RTPS\_LOG\_BASE + 105)  
*Failed to initialize a local sequence.*
- #define RTPS\_LOG\_SEQ\_MAX\_EC (RTPS\_LOG\_BASE + 106)  
*Failed to set maximum of a local sequence.*
- #define RTPS\_LOG\_SEQ\_LEN\_EC (RTPS\_LOG\_BASE + 107)  
*Failed to set length of a local sequence.*
- #define RTPS\_LOG\_INTF\_MODE\_UNDEF\_EC (RTPS\_LOG\_BASE + 108)  
*Initializing an RTPS interface with undefined mode.*
- #define RTPS\_LOG\_DELETE\_BUFFER\_POOL\_EC (RTPS\_LOG\_BASE + 109)  
*Failed to delete buffer pool.*
- #define RTPS\_LOG\_CREATE\_BUFFER\_POOL\_EC (RTPS\_LOG\_BASE + 110)  
*Failed to create buffer pool.*
- #define RTPS\_LOG\_CREATE\_PEERS\_INDEX\_EC (RTPS\_LOG\_BASE + 111)  
*Failed to create index of peers.*
- #define RTPS\_LOG\_DELETE\_INDEXER\_EC (RTPS\_LOG\_BASE + 112)

- Failed to delete indexer.*

  - #define RTPS\_LOG\_GET\_PEER\_BUFFER\_EC (RTPS\_LOG\_BASE + 113)
- Failed to get buffer from peer buffer pool.*

  - #define RTPS\_LOG\_DB\_CREATE\_DIRECT\_INDEX\_EC (RTPS\_LOG\_BASE + 114)
- Failed to create database index for direct sends.*

  - #define RTPS\_LOG\_DB\_CREATE\_GROUP\_INDEX\_EC (RTPS\_LOG\_BASE + 115)
- Failed to create database index for group sends.*

  - #define RTPS\_LOG\_NETIO\_PACKET\_INSUFFICIENT\_BUFFER\_EC (RTPS\_LOG\_BASE + 116)
- Insufficient buffer in packet.*

  - #define RTPS\_LOG\_TRANSFORM\_RTPS\_EC (RTPS\_LOG\_BASE + 117)
- Failed to transform rtps message.*

  - #define RTPS\_LOG\_RESTORE\_TRUST\_LOAN\_BUFFER\_EC (RTPS\_LOG\_BASE + 118)
- Failed to restore trust loan buffer.*

  - #define RTPS\_LOG\_PROCESS\_DATA\_BATCH\_EC (RTPS\_LOG\_BASE + 136)
- Failed to process received DATA BATCH submessage.*

  - #define RTPS\_LOG\_PROCESS\_HEARTBEAT\_BATCH\_EC (RTPS\_LOG\_BASE + 137)
- Failed to process received HEARTBEAT\_BATCH submessage.*

  - #define RTPS\_LOG\_NEGATIVE\_RESERVATION\_COUNT\_EC (RTPS\_LOG\_BASE + 138)
- The reservation count is zero.*

  - #define RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_LOOKUP\_FAILED\_EC (RTPS\_LOG\_BASE + 139)
- Failed to lookup a remote participant's interceptor handle.*

  - #define RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_CREATE\_FAILED\_EC (RTPS\_LOG\_BASE + 140)
- Failed to create a record to store a remote participant's interceptor handle.*

  - #define RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_INSERT\_FAILED\_EC (RTPS\_LOG\_BASE + 141)
- Failed to insert a record in the table storing interceptor handles of matched remote participants.*

  - #define RTPS\_LOG\_TRUST\_INTERCEPTOR\_HANDLE\_LIST\_FULL\_EC (RTPS\_LOG\_BASE + 142)
- Interceptor handle list out of resources.*

  - #define RTPS\_LOG\_TRUST\_INCONSISTENT\_INTERCEPTOR\_HANDLE\_EC (RTPS\_LOG\_BASE + 143)
- Inconsistent handles found for remote peer.*

  - #define RTPS\_LOG\_TRUST\_BIND\_FAILED\_EC (RTPS\_LOG\_BASE + 144)
- Failed to bind resources for trust behavior.*

  - #define RTPS\_LOG\_TRUST\_SUBMSG\_CONVERSION\_FAILED\_EC (RTPS\_LOG\_BASE + 145)
- Failed to bind resources for trust behavior.*

  - #define RTPS\_LOG\_TRUST\_INVALID\_SUBMSG\_EC (RTPS\_LOG\_BASE + 146)
- A malformed trust message was received.*

  - #define RTPS\_LOG\_TRUST\_UNEXPECTED\_READER\_STATE\_EC (RTPS\_LOG\_BASE + 147)
- The RTPS reader was in an unexpected state.*

  - #define RTPS\_LOG\_DB\_CREATE\_SELECTED\_GROUP\_INDEX\_EC (RTPS\_LOG\_BASE + 148)
- Failed to create database index for the selected route group.*

  - #define RTPS\_LOG\_LOCK\_EC (RTPS\_LOG\_BASE + 149)
- Failed to take a lock.*

  - #define RTPS\_LOG\_UNLOCK\_EC (RTPS\_LOG\_BASE + 150)
- Failed to give a lock.*

  - #define RTPS\_LOG\_SCHEDULE\_PACKET\_EC (RTPS\_LOG\_BASE + 151)

- Failed to schedule a packet.*

  - #define `RTPS_LOG_SEQ_FINALIZE_EC` (`RTPS_LOG_BASE` + 152)
- Failed to finalize a sequence.*

  - #define `RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC` (`RTPS_LOG_BASE` + 200)

*The property tries to override the BUILTIN128 checksum without setting allow\_builtin\_override to TRUE.*

  - #define `RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC` (`RTPS_LOG_BASE` + 201)

*The property tries to override the BUILTIN32 checksum without setting allow\_builtin\_override to TRUE.*

  - #define `RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC` (`RTPS_LOG_BASE` + 202)

*The property tries to override the BUILTIN64 checksum without setting allow\_builtin\_override to TRUE.*

  - #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC` (`RTPS_LOG_BASE` + 203)

*The property override of the BUILTIN128 checksum is invalid. It is not legal to reuse any of the built-in functions.*

  - #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC` (`RTPS_LOG_BASE` + 204)

*The property override of the BUILTIN32 checksum is invalid. It is not legal to reuse any of the built-in functions functions.*

  - #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC` (`RTPS_LOG_BASE` + 205)

*The property override of the BUILTIN64 checksum is invalid. It is not legal to reuse any of the built-in functions functions.*

  - #define `RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC` (`RTPS_LOG_BASE` + 210)

*RTPS\_Interface\_calculate\_checksum() failed to calculate the checksum.*

  - #define `RTPS_LOG_UNSUPPORTED_MANDATORY_HDR_EXT_PID_EC` (`RTPS_LOG_BASE` + 211)

*The HeaderExtension contained a mandatory PID that was not understood.*

  - #define `RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC` (`RTPS_LOG_BASE` + 212)

*An invalid HeaderExtension PID was detected.*

  - #define `RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC` (`RTPS_LOG_BASE` + 213)

*Failed to decode the HeaderExtension PID list.*

  - #define `RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC` (`RTPS_LOG_BASE` + 214)

*The HeaderExtension length is inconsistent with the decoded encoded HeaderExtension length.*

  - #define `RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC` (`RTPS_LOG_BASE` + 215)

*A RTPS message which was longer than the indicated message length in the RTPS header was received. This causes the rest of the RTPS message to be invalidated and the remaining submessages to be ignored.*

  - #define `RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC` (`RTPS_LOG_BASE` + 216)

*The CRC32 submsg flags are invalid.*

  - #define `RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC` (`RTPS_LOG_BASE` + 217)

*A header extension was received, but the checksum length is not present.*

  - #define `RTPS_LOG_CHECKSUM_UNSUPPORTED_EC` (`RTPS_LOG_BASE` + 218)

*Unsupported checksum bits received, message dropped.*

  - #define `RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC` (`RTPS_LOG_BASE` + 219)

*Checksum length could not be deserialized.*

  - #define `RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC` (`RTPS_LOG_BASE` + 220)

*The payload length is less than the deserialized checksum length.*

  - #define `RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC` (`RTPS_LOG_BASE` + 221)

*Checksum length could not be deserialized.*

  - #define `RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC` (`RTPS_LOG_BASE` + 222)

*The checksum is invalid.*

  - #define `RTPS_LOG_CHECKSUM_TOO_MANY_SG_BUFFERS_EC` (`RTPS_LOG_BASE` + 223)

*Found too many scatter-gather buffers when calculating checksum.*

  - #define `RTPS_LOG_SEND_LIVELINESS_EC` (`RTPS_LOG_BASE` + 224)

*Failure to send Liveliness packets.*

  - #define `RTPS_LOG_DELETE_RECORD_EC` (`RTPS_LOG_BASE` + 225)

*Failure to delete a record.*

- `#define RTPS_LOG_APPLY_CONTENT_FILTER_EC (RTPS_LOG_BASE + 300)`

*Failed to apply a content filter to a message.*

- `#define RTPS_LOG_UPDATE_RELIABLE_PEERS_FILTER_EC (RTPS_LOG_BASE + 301)`

*Failed to update the state of reliable peers while applying filtering.*

- `#define RTPS_LOG_ADD_FILTERED_ROUTE_EC (RTPS_LOG_BASE + 302)`

*Failed to add a filtered route.*

- `#define RTPS_LOG_DELETE_FILTERED_ROUTE_EC (RTPS_LOG_BASE + 303)`

*Failed to delete a filtered route.*

### 6.21.8.1 Detailed Description

Real-Time Publish-Subscribe. ModuleID = 6.

### 6.21.8.2 Macro Definition Documentation

#### RTPS\_LOG\_INITIALIZE\_INTERFACE\_EC

```
#define RTPS_LOG_INITIALIZE_INTERFACE_EC (RTPS_LOG_BASE + 1)
```

Failed to initialize an RTPS interface.

#### RTPS\_LOG\_ALLOCATE\_EC

```
#define RTPS_LOG_ALLOCATE_EC (RTPS_LOG_BASE + 2)
```

Failed to allocate heap memory for internal resources.

#### RTPS\_LOG\_CREATE\_ROUTE\_TABLE\_EC

```
#define RTPS_LOG_CREATE_ROUTE_TABLE_EC (RTPS_LOG_BASE + 3)
```

Failed to create a database table for storing RTPS route info.

#### RTPS\_LOG\_DB\_CREATE\_ROUTE\_PEER\_INDEX\_EC

```
#define RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC (RTPS_LOG_BASE + 4)
```

Failed to create database index for route-peer info.

**RTPS\_LOG\_DB\_CREATE\_ROUTE\_XPORT\_INDEX\_EC**

```
#define RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC (RTPS_LOG_BASE + 5)
```

Failed to create database index for route-transport info.

**RTPS\_LOG\_DB\_CREATE\_BIND\_PEER\_INDEX\_EC**

```
#define RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC (RTPS_LOG_BASE + 6)
```

Failed to create database index for bind-peer info.

**RTPS\_LOG\_INDEX\_ADD\_ENTRY\_EC**

```
#define RTPS_LOG_INDEX_ADD_ENTRY_EC (RTPS_LOG_BASE + 8)
```

Failed to add an entry to a database index.

**RTPS\_LOG\_DB\_SELECT\_ALL\_EC**

```
#define RTPS_LOG_DB_SELECT_ALL_EC (RTPS_LOG_BASE + 9)
```

Failed to select all entries of a database table.

Failure reason given by database return code (dbrc)

**RTPS\_LOG\_DB\_SELECT\_MATCH\_EC**

```
#define RTPS_LOG_DB_SELECT_MATCH_EC (RTPS_LOG_BASE + 10)
```

Matching entry not found in a database table.

Failure reason given by database return code (dbrc)

**RTPS\_LOG\_DB\_SELECT\_RANGE\_EC**

```
#define RTPS_LOG_DB_SELECT_RANGE_EC (RTPS_LOG_BASE + 11)
```

No matching entries found in a database table.

**RTPS\_LOG\_DB\_CREATE\_ROUTE\_ENTRY\_EC**

```
#define RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 12)
```

Failed to create database entry for route.

May have exceeded DomainParticipantQos.resource\_limits.matching\_writer\_reader\_pair\_allocation, DataWriterQos.writer\_resource\_limits.max\_remote\_readers, or DataReaderQos.reader\_resource\_limits.max\_remote\_writers

**RTPS\_LOG\_DB\_INSERT\_ROUTE\_ENTRY\_EC**

```
#define RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 13)
```

Failed to insert entry into route table.

Failure reason given by database return code (dbrc)

**RTPS\_LOG\_DB\_DELETE\_ROUTE\_ENTRY\_EC**

```
#define RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 15)
```

Failed to delete entry from route table.

Failure reason given by database return code (dbrc)

**RTPS\_LOG\_DB\_CREATE\_BIND\_ENTRY\_EC**

```
#define RTPS_LOG_DB_CREATE_BIND_ENTRY_EC (RTPS_LOG_BASE + 16)
```

Failed to create database entry for bind.

If DomainParticipant, may have exceeded DomainParticipantQos.resource\_limits.matching\_reader\_writer\_pair\_allocation. Failure reason given by database return code (dbrc).

**RTPS\_LOG\_DB\_INSERT\_BIND\_ENTRY\_EC**

```
#define RTPS_LOG_DB_INSERT_BIND_ENTRY_EC (RTPS_LOG_BASE + 17)
```

Failed to insert an entry into the bind table.

Failure reason given by database return code (dbrc)

**RTPS\_LOG\_DB\_REMOVE\_BIND\_ENTRY\_EC**

```
#define RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC (RTPS_LOG_BASE + 18)
```

Failed to remove an entry from the bind table.

Failure reason given by database return code (dbrc)

**RTPS\_LOG\_DB\_REMOVE\_EXTERNAL\_BIND\_ENTRY\_EC**

```
#define RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC (RTPS_LOG_BASE + 19)
```

Failed to remove an entry from the external bind table.

Failure reason given by database return code (dbrc)

**RTPS\_LOG\_DB\_DELETE\_EXTERNAL\_BIND\_ENTRY\_EC**

```
#define RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC (RTPS_LOG_BASE + 20)
```

Failed to delete entry from external bind table.

Failure reason given by database return code (dbrc)

**RTPS\_LOG\_SEND\_EC**

```
#define RTPS_LOG_SEND_EC (RTPS_LOG_BASE + 21)
```

Failed to send a packet due to a lower module failing to send the packet.

**RTPS\_LOG\_RECEIVE\_EC**

```
#define RTPS_LOG_RECEIVE_EC (RTPS_LOG_BASE + 22)
```

Failed to receive a packet due to a higher module failing to receive the packet.

**RTPS\_LOG\_ROUTE\_PACKET\_EC**

```
#define RTPS_LOG_ROUTE_PACKET_EC (RTPS_LOG_BASE + 23)
```

Failed to send a packet, when routing to a lower module or peer.

**RTPS\_LOG\_FORWARD\_UPSTREAM\_EC**

```
#define RTPS_LOG_FORWARD_UPSTREAM_EC (RTPS_LOG_BASE + 24)
```

Failed to receive a packet, when forwarding to a higher upstream module.

**RTPS\_LOG\_BAD\_PARAMETER\_EC**

```
#define RTPS_LOG_BAD_PARAMETER_EC (RTPS_LOG_BASE + 25)
```

Bad parameter to an RTPS interface function.

**RTPS\_LOG\_NOT\_ENABLED\_EC**

```
#define RTPS_LOG_NOT_ENABLED_EC (RTPS_LOG_BASE + 26)
```

Failed because interface is not enabled.

**RTPS\_LOG\_READER\_UNSUPPORTED\_EC**

```
#define RTPS_LOG_READER_UNSUPPORTED_EC (RTPS_LOG_BASE + 27)
```

RTPS reader does not support send.

**RTPS\_LOG\_INTERFACE\_MISMATCH\_EC**

```
#define RTPS_LOG_INTERFACE_MISMATCH_EC (RTPS_LOG_BASE + 28)
```

Upstream interface does not match source or destination of packet being sent or received.

**RTPS\_LOG\_FULL\_SEND\_WINDOW\_EC**

```
#define RTPS_LOG_FULL_SEND_WINDOW_EC (RTPS_LOG_BASE + 29)
```

Failed to send due to reaching maximum send window size.

**RTPS\_LOG\_NETIO\_PACKET\_SET\_TAIL\_EC**

```
#define RTPS_LOG_NETIO_PACKET_SET_TAIL_EC (RTPS_LOG_BASE + 31)
```

Failed to set tail of packet to send.



**RTPS\_LOG\_FUNC\_UNSUPPORTED\_EC**

```
#define RTPS_LOG_FUNC_UNSUPPORTED_EC (RTPS_LOG_BASE + 32)
```

RTPS interface does not support this function.

**RTPS\_LOG\_EXCEEDED\_LIMIT\_TRANSPORTS\_EC**

```
#define RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC (RTPS_LOG_BASE + 33)
```

Out of transport entries.

Exceeded either `DataWriterQos.writer_resource_limits.max_remote_readers` or `DataReaderQos.reader_resource_↵limits.max_remote_writers`

**RTPS\_LOG\_EXCEEDED\_LIMIT\_PEERS\_EC**

```
#define RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC (RTPS_LOG_BASE + 34)
```

Out of peer entries.

Exceeded either `DataWriterQos.writer_resource_limits.max_remote_readers` or `DataReaderQos.reader_resource_↵limits.max_remote_writers`

**RTPS\_LOG\_ASSERT\_PEER\_EC**

```
#define RTPS_LOG_ASSERT_PEER_EC (RTPS_LOG_BASE + 35)
```

Failed to assert a remote writer or reader.

**RTPS\_LOG\_ASSERT\_TRANSPORT\_EC**

```
#define RTPS_LOG_ASSERT_TRANSPORT_EC (RTPS_LOG_BASE + 36)
```

Failed to assert a downstream transport.

**RTPS\_LOG\_FIND\_EXTERNAL\_INTERFACE\_EC**

```
#define RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC (RTPS_LOG_BASE + 37)
```

Failed to lookup an external interface.

**RTPS\_LOG\_NONEXISTENT\_ROUTE\_EC**

```
#define RTPS_LOG_NONEXISTENT_ROUTE_EC (RTPS_LOG_BASE + 38)
```

Failed to delete a nonexistent route.

**RTPS\_LOG\_NONEXISTENT\_BIND\_EC**

```
#define RTPS_LOG_NONEXISTENT_BIND_EC (RTPS_LOG_BASE + 39)
```

Failed to remove a nonexistent bind entry.

**RTPS\_LOG\_NONEXISTENT\_EXTERNAL\_BIND\_EC**

```
#define RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC (RTPS_LOG_BASE + 40)
```

Failed to remove a nonexistent external bind entry.

**RTPS\_LOG\_STALE\_ACK\_EPOCH\_EC**

```
#define RTPS_LOG_STALE_ACK_EPOCH_EC (RTPS_LOG_BASE + 41)
```

Dropped an ACKNACK submessage with an old epoch.

**RTPS\_LOG\_STALE\_HB\_EPOCH\_EC**

```
#define RTPS_LOG_STALE_HB_EPOCH_EC (RTPS_LOG_BASE + 42)
```

Dropped a HEARTBEAT submessage with an old epoch.

**RTPS\_LOG\_ACK\_EC**

```
#define RTPS_LOG_ACK_EC (RTPS_LOG_BASE + 43)
```

Failed an acknack() upstream.

**RTPS\_LOG\_REQUEST\_EC**

```
#define RTPS_LOG_REQUEST_EC (RTPS_LOG_BASE + 45)
```

Failed a request() upstream.

**RTPS\_LOG\_RETURN\_LOAN\_EC**

```
#define RTPS_LOG_RETURN_LOAN_EC (RTPS_LOG_BASE + 46)
```

Failed a return\_loan() upstream.

**RTPS\_LOG\_NETIO\_PACKET\_SET\_HEAD\_EC**

```
#define RTPS_LOG_NETIO_PACKET_SET_HEAD_EC (RTPS_LOG_BASE + 47)
```

Failed to set the head of a packet.

**RTPS\_LOG\_SEND\_HEARTBEAT\_EC**

```
#define RTPS_LOG_SEND_HEARTBEAT_EC (RTPS_LOG_BASE + 48)
```

Failed to send a HEARTBEAT submessage.

**RTPS\_LOG\_SHIFT\_BITMAP\_EC**

```
#define RTPS_LOG_SHIFT_BITMAP_EC (RTPS_LOG_BASE + 49)
```

Failed to shift a bitmap.

**RTPS\_LOG\_FULLY\_ACKED\_READER\_EC**

```
#define RTPS_LOG_FULLY_ACKED_READER_EC (RTPS_LOG_BASE + 50)
```

A Reader is fully acknowledged and does not need to respond to a final HEARTBEAT.

**RTPS\_LOG\_DATA\_OUT\_OF\_RANGE\_EC**

```
#define RTPS_LOG_DATA_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 51)
```

Dropping a DATA submessage whose sequence number is outside of a Reader's receive window.

**RTPS\_LOG\_DATA\_ALREADY\_RECEIVED\_EC**

```
#define RTPS_LOG_DATA_ALREADY_RECEIVED_EC (RTPS_LOG_BASE + 52)
```

Dropping a DATA submessage whose sequence number was previously received.

**RTPS\_LOG\_INTERFACE\_NOT\_ENABLED\_EC**

```
#define RTPS_LOG_INTERFACE_NOT_ENABLED_EC (RTPS_LOG_BASE + 53)
```

Failed to receive a message because interface is not enabled.

**RTPS\_LOG\_INVALID\_PACKET\_EC**

```
#define RTPS_LOG_INVALID_PACKET_EC (RTPS_LOG_BASE + 54)
```

Dropped a message with an invalid packet header.

**RTPS\_LOG\_UNSUPPORTED\_INTERFACE\_EC**

```
#define RTPS_LOG_UNSUPPORTED_INTERFACE_EC (RTPS_LOG_BASE + 55)
```

Received a message on an unsupported interface.

**RTPS\_LOG\_UNKNOWN\_SUBMESSAGE\_EC**

```
#define RTPS_LOG_UNKNOWN_SUBMESSAGE_EC (RTPS_LOG_BASE + 56)
```

Received a submessage with an unknown ID.

**RTPS\_LOG\_PROCESS\_ACKNACK\_EC**

```
#define RTPS_LOG_PROCESS_ACKNACK_EC (RTPS_LOG_BASE + 57)
```

Failed to process received ACKNACK submessage.

**RTPS\_LOG\_PROCESS\_DATA\_EC**

```
#define RTPS_LOG_PROCESS_DATA_EC (RTPS_LOG_BASE + 58)
```

Failed to process received DATA submessage.

**RTPS\_LOG\_PROCESS\_GAP\_EC**

```
#define RTPS_LOG_PROCESS_GAP_EC (RTPS_LOG_BASE + 59)
```

Failed to process received GAP submessage.

**RTPS\_LOG\_PROCESS\_HEARTBEAT\_EC**

```
#define RTPS_LOG_PROCESS_HEARTBEAT_EC (RTPS_LOG_BASE + 60)
```

Failed to process received HEARTBEAT submessage.

**RTPS\_LOG\_CREATE\_HB\_EVENT\_EC**

```
#define RTPS_LOG_CREATE_HB_EVENT_EC (RTPS_LOG_BASE + 61)
```

Failed to create periodic HEARTBEAT event.

**RTPS\_LOG\_CREATE\_EVENT\_TIMER\_EC**

```
#define RTPS_LOG_CREATE_EVENT_TIMER_EC (RTPS_LOG_BASE + 62)
```

Failed to create periodic HEARTBEAT event timer.

**RTPS\_LOG\_SEQ\_NUM\_OUT\_OF\_RANGE\_EC**

```
#define RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 63)
```

Failed to set bitmap bit due to out-of-range sequence number.

**RTPS\_LOG\_FIRST\_SN\_GREATER\_LAST\_SN\_EC**

```
#define RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC (RTPS_LOG_BASE + 65)
```

Invalid range of sequence numbers to fill in bitmap.

**RTPS\_LOG\_BITCOUNT\_OUT\_OF\_BOUNDS\_EC**

```
#define RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC (RTPS_LOG_BASE + 66)
```

Deserialized an invalid out-of-bounds bitcount for a bitmap.

**RTPS\_LOG\_SHIFT\_SN\_OUT\_OF\_RANGE\_EC**

```
#define RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 67)
```

Failed to shift bitmap due to invalid starting sequence number.

**RTPS\_LOG\_SERIALIZE\_HOST\_ID\_EC**

```
#define RTPS_LOG_SERIALIZE_HOST_ID_EC (RTPS_LOG_BASE + 68)
```

Failed to serialize host ID of GUID.

**RTPS\_LOG\_SERIALIZE\_APP\_ID\_EC**

```
#define RTPS_LOG_SERIALIZE_APP_ID_EC (RTPS_LOG_BASE + 69)
```

Failed to serialize app ID of GUID.

**RTPS\_LOG\_SERIALIZE\_INSTANCE\_ID\_EC**

```
#define RTPS_LOG_SERIALIZE_INSTANCE_ID_EC (RTPS_LOG_BASE + 70)
```

Failed to serialized instance ID of GUID.

**RTPS\_LOG\_SERIALIZE\_OBJECT\_ID\_EC**

```
#define RTPS_LOG_SERIALIZE_OBJECT_ID_EC (RTPS_LOG_BASE + 71)
```

Failed to serialize object ID of GUID.

**RTPS\_LOG\_DESERIALIZE\_HOST\_ID\_EC**

```
#define RTPS_LOG_DESERIALIZE_HOST_ID_EC (RTPS_LOG_BASE + 72)
```

Failed to deserialize host ID of GUID.

**RTPS\_LOG\_DESERIALIZE\_APP\_ID\_EC**

```
#define RTPS_LOG_DESERIALIZE_APP_ID_EC (RTPS_LOG_BASE + 73)
```

Failed to deserialize app ID of GUID.

**RTPS\_LOG\_DESERIALIZE\_INSTANCE\_ID\_EC**

```
#define RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC (RTPS_LOG_BASE + 74)
```

Failed to deserialize instance ID of GUID.

**RTPS\_LOG\_DESERIALIZE\_OBJECT\_ID\_EC**

```
#define RTPS_LOG_DESERIALIZE_OBJECT_ID_EC (RTPS_LOG_BASE + 75)
```

Failed to deserialize object ID of GUID.

**RTPS\_LOG\_DB\_CREATE\_EXT\_BIND\_ENTRY\_EC**

```
#define RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC (RTPS_LOG_BASE + 76)
```

Failed to create database entry for external bind table.

If DomainParticipant, may have exceeded DomainParticipantQos.resource\_limits.matching\_reader\_writer\_pair\_↔ allocation.

**RTPS\_LOG\_BITMAP\_SET\_BIT\_EC**

```
#define RTPS_LOG_BITMAP_SET_BIT_EC (RTPS_LOG_BASE + 77)
```

Failed to set bit in bitmap.

**RTPS\_LOG\_UNSUPPORTED\_INFO\_REPLY\_EC**

```
#define RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC (RTPS_LOG_BASE + 78)
```

Received and dropped currently unsupported INFO\_REPLY submessage.

**RTPS\_LOG\_UNSUPPORTED\_INFO\_REPLY\_IP4\_EC**

```
#define RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC (RTPS_LOG_BASE + 79)
```

Received and dropped currently unsupported INFO\_REPLY\_IP4 submessage.

**RTPS\_LOG\_UNSUPPORTED\_INFO\_SRC\_EC**

```
#define RTPS_LOG_UNSUPPORTED_INFO_SRC_EC (RTPS_LOG_BASE + 80)
```

Received and dropped currently unsupported INFO\_SRC submessage.

**RTPS\_LOG\_DELETE\_INDEX\_EC**

```
#define RTPS_LOG_DELETE_INDEX_EC (RTPS_LOG_BASE + 81)
```

Failed to delete index of a database table.

**RTPS\_LOG\_DELETE\_TABLE\_EC**

```
#define RTPS_LOG_DELETE_TABLE_EC (RTPS_LOG_BASE + 82)
```

Failed to delete a database table.

**RTPS\_LOG\_DB\_LOCK\_EC**

```
#define RTPS_LOG_DB_LOCK_EC (RTPS_LOG_BASE + 83)
```

Failed to take database lock.

**RTPS\_LOG\_DB\_UNLOCK\_EC**

```
#define RTPS_LOG_DB_UNLOCK_EC (RTPS_LOG_BASE + 84)
```

Failed to give database lock.

**RTPS\_LOG\_CREATE\_BIND\_TABLE\_EC**

```
#define RTPS_LOG_CREATE_BIND_TABLE_EC (RTPS_LOG_BASE + 85)
```

Failed to create a database table for storing RTPS bind info.

**RTPS\_LOG\_STATUS\_CHANGE\_EC**

```
#define RTPS_LOG_STATUS_CHANGE_EC (RTPS_LOG_BASE + 86)
```

Failed to update status for reliable reader activity changed.

**RTPS\_LOG\_SET\_GAP\_EC**

```
#define RTPS_LOG_SET_GAP_EC (RTPS_LOG_BASE + 87)
```

Failed to update status for reliable reader activity changed.

**RTPS\_LOG\_WINDOW\_POOL\_ALLOC\_EC**

```
#define RTPS_LOG_WINDOW_POOL_ALLOC_EC (RTPS_LOG_BASE + 88)
```

Out of memory to allocate send window pool.



**RTPS\_LOG\_GET\_WINDOW\_BUFFER\_EC**

```
#define RTPS_LOG_GET_WINDOW_BUFFER_EC (RTPS_LOG_BASE + 89)
```

Failed to get buffer from send window pool.

**RTPS\_LOG\_INSERT\_WINDOW\_FULL\_EC**

```
#define RTPS_LOG_INSERT_WINDOW_FULL_EC (RTPS_LOG_BASE + 90)
```

Cannot insert sample into full window.

**RTPS\_LOG\_WINDOW\_INSERT\_EC**

```
#define RTPS_LOG_WINDOW_INSERT_EC (RTPS_LOG_BASE + 91)
```

Failed to insert sample into send window.

**RTPS\_LOG\_TIMER\_DELETE\_TIMEOUT\_EC**

```
#define RTPS_LOG_TIMER_DELETE_TIMEOUT_EC (RTPS_LOG_BASE + 92)
```

Failed to delete a timeout event.

**RTPS\_LOG\_BUFFERPOOL\_DELETE\_EC**

```
#define RTPS_LOG_BUFFERPOOL_DELETE_EC (RTPS_LOG_BASE + 93)
```

Failed to delete a buffer pool.

**RTPS\_LOG\_DIRECT\_SEND\_EC**

```
#define RTPS_LOG_DIRECT_SEND_EC (RTPS_LOG_BASE + 94)
```

Failed to send a message to a specific peer.

**RTPS\_LOG\_PACKET\_INITIALIZE\_EC**

```
#define RTPS_LOG_PACKET_INITIALIZE_EC (RTPS_LOG_BASE + 95)
```

RTPS writer failed to initialize a packet.

**RTPS\_LOG\_WRITER\_REQUEST\_SENT\_HISTORY\_EC**

```
#define RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC (RTPS_LOG_BASE + 96)
```

RTPS reliable writer failed to request and resend history.

**RTPS\_LOG\_BITMAP\_SHIFT\_EC**

```
#define RTPS_LOG_BITMAP_SHIFT_EC (RTPS_LOG_BASE + 97)
```

Failed to shift bitmap to new lead sequence number.

**RTPS\_LOG\_WINDOW\_ADVANCE\_EC**

```
#define RTPS_LOG_WINDOW_ADVANCE_EC (RTPS_LOG_BASE + 98)
```

Send window of a remote reader failed to advance.

**RTPS\_LOG\_WINDOW\_ADVANCE\_UNACKED\_EC**

```
#define RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC (RTPS_LOG_BASE + 99)
```

Failed when advancing window ahead of unacknowledged sequence number.

**RTPS\_LOG\_LOOKUP\_PEER\_EC**

```
#define RTPS_LOG_LOOKUP_PEER_EC (RTPS_LOG_BASE + 100)
```

Failed to find peer that should exist.

**RTPS\_LOG\_READER\_SEND\_ACKNACK\_EC**

```
#define RTPS_LOG_READER_SEND_ACKNACK_EC (RTPS_LOG_BASE + 101)
```

RTPS reader failed to send an ACKNACK message.

**RTPS\_LOG\_FINALIZE\_INTERFACE\_EC**

```
#define RTPS_LOG_FINALIZE_INTERFACE_EC (RTPS_LOG_BASE + 102)
```

Failed to finalize an RTPS interface.

**RTPS\_LOG\_INDEXER\_REMOVE\_ENTRY\_EC**

```
#define RTPS_LOG_INDEXER_REMOVE_ENTRY_EC (RTPS_LOG_BASE + 103)
```

Failed to remove index entry for external interface upon deleting RTPS interface.

**RTPS\_LOG\_LOOKUP\_EXT\_BIND\_ENTRY\_EC**

```
#define RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC (RTPS_LOG_BASE + 104)
```

Failed when looking up external bind entry from database.

**RTPS\_LOG\_SEQ\_INIT\_EC**

```
#define RTPS_LOG_SEQ_INIT_EC (RTPS_LOG_BASE + 105)
```

Failed to initialize a local sequence.

**RTPS\_LOG\_SEQ\_MAX\_EC**

```
#define RTPS_LOG_SEQ_MAX_EC (RTPS_LOG_BASE + 106)
```

Failed to set maximum of a local sequence.

**RTPS\_LOG\_SEQ\_LEN\_EC**

```
#define RTPS_LOG_SEQ_LEN_EC (RTPS_LOG_BASE + 107)
```

Failed to set length of a local sequence.

**RTPS\_LOG\_INTF\_MODE\_UNDEF\_EC**

```
#define RTPS_LOG_INTF_MODE_UNDEF_EC (RTPS_LOG_BASE + 108)
```

Initializing an RTPS interface with undefined mode.

**RTPS\_LOG\_DELETE\_BUFFER\_POOL\_EC**

```
#define RTPS_LOG_DELETE_BUFFER_POOL_EC (RTPS_LOG_BASE + 109)
```

Failed to delete buffer pool.

May still have buffers in use, thus cannot delete pool.

**RTPS\_LOG\_CREATE\_BUFFER\_POOL\_EC**

```
#define RTPS_LOG_CREATE_BUFFER_POOL_EC (RTPS_LOG_BASE + 110)
```

Failed to create buffer pool.

May have insufficient memory to allocate pool.

**RTPS\_LOG\_CREATE\_PEERS\_INDEX\_EC**

```
#define RTPS_LOG_CREATE_PEERS_INDEX_EC (RTPS_LOG_BASE + 111)
```

Failed to create index of peers.

System may have insufficient memory to allocate new index.

**RTPS\_LOG\_DELETE\_INDEXER\_EC**

```
#define RTPS_LOG_DELETE_INDEXER_EC (RTPS_LOG_BASE + 112)
```

Failed to delete indexer.

**RTPS\_LOG\_GET\_PEER\_BUFFER\_EC**

```
#define RTPS_LOG_GET_PEER_BUFFER_EC (RTPS_LOG_BASE + 113)
```

Failed to get buffer from peer buffer pool.

**RTPS\_LOG\_DB\_CREATE\_DIRECT\_INDEX\_EC**

```
#define RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC (RTPS_LOG_BASE + 114)
```

Failed to create database index for direct sends.

**RTPS\_LOG\_DB\_CREATE\_GROUP\_INDEX\_EC**

```
#define RTPS_LOG_DB_CREATE_GROUP_INDEX_EC (RTPS_LOG_BASE + 115)
```

Failed to create database index for group sends.

**RTPS\_LOG\_NETIO\_PACKET\_INSUFFICIENT\_BUFFER\_EC**

```
#define RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC (RTPS_LOG_BASE + 116)
```

Insufficient buffer in packet.

Receive buffer does not have enough space for a valid RTPS message

**RTPS\_LOG\_TRANSFORM\_RTPS\_EC**

```
#define RTPS_LOG_TRANSFORM_RTPS_EC (RTPS_LOG_BASE + 117)
```

Failed to transform rtps message.

**RTPS\_LOG\_RESTORE\_TRUST\_LOAN\_BUFFER\_EC**

```
#define RTPS_LOG_RESTORE_TRUST_LOAN_BUFFER_EC (RTPS_LOG_BASE + 118)
```

Failed to restore trust loan buffer.

**RTPS\_LOG\_PROCESS\_DATA\_BATCH\_EC**

```
#define RTPS_LOG_PROCESS_DATA_BATCH_EC (RTPS_LOG_BASE + 136)
```

Failed to process received DATA\_BATCH submessage.

This indicates an invalid DATA\_BATCH submessage

**RTPS\_LOG\_PROCESS\_HEARTBEAT\_BATCH\_EC**

```
#define RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC (RTPS_LOG_BASE + 137)
```

Failed to process received HEARTBEAT\_BATCH submessage.

This indicates an invalid HEARTBEAT\_BATCH submessage

**RTPS\_LOG\_NEGATIVE\_RESERVATION\_COUNT\_EC**

```
#define RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC (RTPS_LOG_BASE + 138)
```

The reservation count is zero.

The reservation count should never be less than 0, this is an internal error.

**RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_LOOKUP\_FAILED\_EC**

```
#define RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_LOOKUP_FAILED_EC (RTPS_LOG_BASE + 139)
```

Failed to lookup a remote participant's interceptor handle.

An interceptor handle must be always supplied for a remote participant, if any encoding and/or decoding are to be performed. This is an internal error.

**RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_CREATE\_FAILED\_EC**

```
#define RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_CREATE_FAILED_EC (RTPS_LOG_BASE + 140)
```

Failed to create a record to store a remote participant's interceptor handle.

Interceptor handles for remote participants are stored in an indexed table for quicker access. This is an internal error.

**RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_INSERT\_FAILED\_EC**

```
#define RTPS_LOG_TRUST_REMOTE_PARTICIPANT_HANDLE_INSERT_FAILED_EC (RTPS_LOG_BASE + 141)
```

Failed to insert a record in the table storing interceptor handles of matched remote participants.

**RTPS\_LOG\_TRUST\_INTERCEPTOR\_HANDLE\_LIST\_FULL\_EC**

```
#define RTPS_LOG_TRUST_INTERCEPTOR_HANDLE_LIST_FULL_EC (RTPS_LOG_BASE + 142)
```

Interceptor handle list out of resources.

**RTPS\_LOG\_TRUST\_INCONSISTENT\_INTERCEPTOR\_HANDLE\_EC**

```
#define RTPS_LOG_TRUST_INCONSISTENT_INTERCEPTOR_HANDLE_EC (RTPS_LOG_BASE + 143)
```

Inconsistent handles found for remote peer.

**RTPS\_LOG\_TRUST\_BIND\_FAILED\_EC**

```
#define RTPS_LOG_TRUST_BIND_FAILED_EC (RTPS_LOG_BASE + 144)
```

Failed to bind resources for trust behavior.

**RTPS\_LOG\_TRUST\_SUBMSG\_CONVERSION\_FAILED\_EC**

```
#define RTPS_LOG_TRUST_SUBMSG_CONVERSION_FAILED_EC (RTPS_LOG_BASE + 145)
```

Failed to bind resources for trust behavior.

**RTPS\_LOG\_TRUST\_INVALID\_SUBMSG\_EC**

```
#define RTPS_LOG_TRUST_INVALID_SUBMSG_EC (RTPS_LOG_BASE + 146)
```

A malformed trust message was received.

**RTPS\_LOG\_TRUST\_UNEXPECTED\_READER\_STATE\_EC**

```
#define RTPS_LOG_TRUST_UNEXPECTED_READER_STATE_EC (RTPS_LOG_BASE + 147)
```

The RTPS reader was in an unexpected state.

**RTPS\_LOG\_DB\_CREATE\_SELECTED\_GROUP\_INDEX\_EC**

```
#define RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC (RTPS_LOG_BASE + 148)
```

Failed to create database index for the selected route group.

**RTPS\_LOG\_LOCK\_EC**

```
#define RTPS_LOG_LOCK_EC (RTPS_LOG_BASE + 149)
```

Failed to take a lock.

**RTPS\_LOG\_UNLOCK\_EC**

```
#define RTPS_LOG_UNLOCK_EC (RTPS_LOG_BASE + 150)
```

Failed to give a lock.

**RTPS\_LOG\_SCHEDULE\_PACKET\_EC**

```
#define RTPS_LOG_SCHEDULE_PACKET_EC (RTPS_LOG_BASE + 151)
```

Failed to schedule a packet.

**RTPS\_LOG\_SEQ\_FINALIZE\_EC**

```
#define RTPS_LOG_SEQ_FINALIZE_EC (RTPS_LOG_BASE + 152)
```

Failed to finalize a sequence.

**RTPS\_LOG\_CHECKSUM\_ILLEGAL\_OVERRIDE\_CHECKSUM128\_EC**

```
#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC (RTPS_LOG_BASE + 200)
```

The property tries to override the BUILTIN128 checksum without setting allow\_builtin\_override to TRUE.

**RTPS\_LOG\_CHECKSUM\_ILLEGAL\_OVERRIDE\_CHECKSUM32\_EC**

```
#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC (RTPS_LOG_BASE + 201)
```

The property tries to override the BUILTIN32 checksum without setting allow\_builtin\_override to TRUE.

**RTPS\_LOG\_CHECKSUM\_ILLEGAL\_OVERRIDE\_CHECKSUM64\_EC**

```
#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC (RTPS_LOG_BASE + 202)
```

The property tries to override the BUILTIN64 checksum without setting allow\_builtin\_override to TRUE.

**RTPS\_LOG\_CHECKSUM\_INVALID\_OVERRIDE\_BUILTIN128\_EC**

```
#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC (RTPS_LOG_BASE + 203)
```

The property override of the BUILTIN128 checksum is invalid. It is not legal to reuse any of the built-in functions.

**RTPS\_LOG\_CHECKSUM\_INVALID\_OVERRIDE\_BUILTIN32\_EC**

```
#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC (RTPS_LOG_BASE + 204)
```

The property override of the BUILTIN32 checksum is invalid. It is not legal to reuse any of the built-in functions functions.

**RTPS\_LOG\_CHECKSUM\_INVALID\_OVERRIDE\_BUILTIN64\_EC**

```
#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC (RTPS_LOG_BASE + 205)
```

The property override of the BUILTIN64 checksum is invalid. It is not legal to reuse any of the built-in functions functions.



**RTPS\_LOG\_CHECKSUM\_CHECKSUM\_CALCULATION\_FAILED\_EC**

```
#define RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC (RTPS_LOG_BASE + 210)
```

RTPS\_Interface\_calculate\_checksum() failed to calculate the checksum.

**RTPS\_LOG\_UNSUPPORTED\_MANDTORY\_HDR\_EXT\_PID\_EC**

```
#define RTPS_LOG_UNSUPPORTED_MANDTORY_HDR_EXT_PID_EC (RTPS_LOG_BASE + 211)
```

The HeaderExtension contained a mandatory PID that was not understood.

**RTPS\_LOG\_INVALID\_HDR\_EXT\_PID\_LIST\_EC**

```
#define RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC (RTPS_LOG_BASE + 212)
```

An invalid HeaderExtension PID was detected.

**RTPS\_LOG\_INVALID\_HDR\_EXT\_PID\_ERROR\_EC**

```
#define RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC (RTPS_LOG_BASE + 213)
```

Failed to decode the HeaderExtension PID list.

**RTPS\_LOG\_INCONSISTENT\_HDR\_EXT\_LENGTH\_ERROR\_EC**

```
#define RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC (RTPS_LOG_BASE + 214)
```

The HeaderExtension length is inconsistent with the decoded encoded HeaderExtension length.

**RTPS\_LOG\_MESSAGE\_EXCEED\_LENGTH\_ERROR\_EC**

```
#define RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC (RTPS_LOG_BASE + 215)
```

A RTPS message which was longer than the indicated message length in the RTPS header was received. This causes the rest of the RTPS message to be invalidated and the remaining submessages to be ignored.

**RTPS\_LOG\_CHECKSUM\_INVALID\_CRC32\_FLAGS\_EC**

```
#define RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC (RTPS_LOG_BASE + 216)
```

The CRC32 submsg flags are invalid.

**RTPS\_LOG\_CHECKSUM\_MISSING\_CHECKSUM\_LENGTH\_EC**

```
#define RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC (RTPS_LOG_BASE + 217)
```

A header extension was received, but the checksum length is not present.

**RTPS\_LOG\_CHECKSUM\_UNSUPPORTED\_EC**

```
#define RTPS_LOG_CHECKSUM_UNSUPPORTED_EC (RTPS_LOG_BASE + 218)
```

Unsupported checksum bits received, message dropped.

**RTPS\_LOG\_CHECKSUM\_LENGTH\_DESERIALIZE\_ERROR\_EC**

```
#define RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 219)
```

Checksum length could not be deserialized.

**RTPS\_LOG\_CHECKSUM\_INCONSISTENT\_LENGTH\_EC**

```
#define RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC (RTPS_LOG_BASE + 220)
```

The payload length is less than the deserialized checksum length.

**RTPS\_LOG\_CHECKSUM\_CHECKSUM\_DESERIALIZE\_ERROR\_EC**

```
#define RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 221)
```

Checksum length could not be deserialized.

**RTPS\_LOG\_CHECKSUM\_CHECKSUM\_ERROR\_EC**

```
#define RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC (RTPS_LOG_BASE + 222)
```

The checksum is invalid.

**RTPS\_LOG\_CHECKSUM\_TOO\_MANY\_SG\_BUFFERS\_EC**

```
#define RTPS_LOG_CHECKSUM_TOO_MANY_SG_BUFFERS_EC (RTPS_LOG_BASE + 223)
```

Found too many scatter-gather buffers when calculating checksum.

**RTPS\_LOG\_SEND\_LIVELINESS\_EC**

```
#define RTPS_LOG_SEND_LIVELINESS_EC (RTPS_LOG_BASE + 224)
```

Failure to send Liveliness packets.

**RTPS\_LOG\_DELETE\_RECORD\_EC**

```
#define RTPS_LOG_DELETE_RECORD_EC (RTPS_LOG_BASE + 225)
```

Failure to delete a record.

**RTPS\_LOG\_APPLY\_CONTENT\_FILTER\_EC**

```
#define RTPS_LOG_APPLY_CONTENT_FILTER_EC (RTPS_LOG_BASE + 300)
```

Failed to apply a content filter to a message.

**RTPS\_LOG\_UPDATE\_RELIABLE\_PEERS\_FILTER\_EC**

```
#define RTPS_LOG_UPDATE_RELIABLE_PEERS_FILTER_EC (RTPS_LOG_BASE + 301)
```

Failed to update the state of reliable peers while applying filtering.

**RTPS\_LOG\_ADD\_FILTERED\_ROUTE\_EC**

```
#define RTPS_LOG_ADD_FILTERED_ROUTE_EC (RTPS_LOG_BASE + 302)
```

Failed to add a filtered route.

**RTPS\_LOG\_DELETE\_FILTERED\_ROUTE\_EC**

```
#define RTPS_LOG_DELETE_FILTERED_ROUTE_EC (RTPS_LOG_BASE + 303)
```

Failed to delete a filtered route.

**6.21.9 DDS\_C**

DDS C. ModuleID = 7.

## Macros

- #define `DDSC_LOG_INVALID_DURATION_EC` (`DDSC_LOG_BASE` + 1)  
*The specified duration is not valid.*
- #define `DDSC_LOG_INVALID_PARTICIPANT_NAME_EC` (`DDSC_LOG_BASE` + 2)  
*An invalid participant name was specified with an unknown GUID.*
- #define `DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC` (`DDSC_LOG_BASE` + 3)  
*An invalid participant GUID prefix was specified, typically the GUID prefix does not match an already detected participant.*
- #define `DDSC_LOG_SYS_GETTIME_EC` (`DDSC_LOG_BASE` + 4)  
*A call to `OSAPI_System_get_time` failed.*
- #define `DDSC_LOG_GET_NEXT_OBJECT_ID_EC` (`DDSC_LOG_BASE` + 5)  
*Failed to get the next automatically generated object ID for an entity's GUID. This typically means the object id pool has been exhausted.*
- #define `DDSC_LOG_SET_ENTITY_NAME_EC` (`DDSC_LOG_BASE` + 6)  
*Failed to set name string for a DDS entity in the `DomainParticipantQos` entity\_name policy.*
- #define `DDSC_LOG_SYS_GET_HOSTNAME_EC` (`DDSC_LOG_BASE` + 7)  
*A call to `OSAPI_System_get_hostname` failed.*
- #define `DDSC_LOG_IO_SNPRINTF_FAILED_EC` (`DDSC_LOG_BASE` + 8)  
*A call to `OSAPI_Stdio_snprintf` failed. Typically this means the destination buffer was too small.*
- #define `DDSC_LOG_TOPIC_FIND_EC` (`DDSC_LOG_BASE` + 9)  
*Failed to find a topic created by a `DomainParticipant`.*
- #define `DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC` (`DDSC_LOG_BASE` + 10)  
*Endpoint discovery failed because the name of the remote participant parent for an endpoint was not found.*
- #define `DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC` (`DDSC_LOG_BASE` + 11)  
*Endpoint discovery failed because the key of the remote participant parent for an endpoint was not found. Note that the key is logged as 4 integers in host endianness format.*
- #define `DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC` (`DDSC_LOG_BASE` + 12)  
*Failed endpoint discovery when key does not match the remote participant.*
- #define `DDSC_LOG_INVALID_ENDPOINT_GUID_EC` (`DDSC_LOG_BASE` + 13)  
*Failed endpoint discovery due to an invalid or unknown endpoint GUID.*
- #define `DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 14)  
*Failed endpoint discovery when an endpoint is determined to belong to a different remote participant.*
- #define `DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC` (`DDSC_LOG_BASE` + 15)  
*Failed participant discovery because a remote participant that should have been already asserted locally was not found.*
- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 16)  
*Did not find a remote participant when asserting participant liveliness for it. Note that the key is logged as 4 integers in host endianness format.*
- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC` (`DDSC_LOG_BASE` + 17)  
*Failed to assert participant liveliness to a remote participant.*
- #define `DDSC_LOG_PARTICIPANT_LOOKUP_EC` (`DDSC_LOG_BASE` + 18)  
*Failed to find a remote participant as previously discovered.*
- #define `DDSC_LOG_REMOVE_PUBLICATION_EC` (`DDSC_LOG_BASE` + 19)  
*Failed to remove resources for a remote publication.*
- #define `DDSC_LOG_REMOVE_SUBSCRIPTION_EC` (`DDSC_LOG_BASE` + 20)  
*Failed to remove resources for a remote subscription.*
- #define `DDSC_LOG_FIND_PUBLICATION_PARENT_EC` (`DDSC_LOG_BASE` + 21)  
*Cannot determine the participant of a remote publication.*
- #define `DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC` (`DDSC_LOG_BASE` + 22)

- Cannot determine the participant of a remote subscription.*

  - #define `DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC` (`DDSC_LOG_BASE` + 23)  
*Failed to create DomainParticipant due to running out of participant IDs. This is typically caused by all UDP ports being used.*
  - #define `DDSC_LOG_RESERVE_LOCATORS_EC` (`DDSC_LOG_BASE` + 24)  
*Failed to reserve endpoint locators.*
  - #define `DDSC_LOG_TIMER_CREATE_TIMEOUT_EC` (`DDSC_LOG_BASE` + 25)  
*Failed to create a timeout.*
  - #define `DDSC_LOG_ILLEGAL_OBJECTID_EC` (`DDSC_LOG_BASE` + 26)  
*Illegal object id specified.*
  - #define `DDSC_LOG_TOPIC_NARROW_EC` (`DDSC_LOG_BASE` + 27)  
*Failed to narrow a TopicDescription to the named Topic.*
  - #define `DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC` (`DDSC_LOG_BASE` + 28)  
*Failed to update state of StatusCondition.*
  - #define `DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC` (`DDSC_LOG_BASE` + 29)  
*Failed to remove a condition reference from a waitset.*
  - #define `DDSC_LOG_WS_ADD_COND_REFERENCE_EC` (`DDSC_LOG_BASE` + 30)  
*Failed to add a condition reference to a waitset.*
  - #define `DDSC_LOG_RELEASE_META_MC_EC` (`DDSC_LOG_BASE` + 31)  
*Failed to release resources for multicast discovery locators.*
  - #define `DDSC_LOG_RELEASE_META_UC_EC` (`DDSC_LOG_BASE` + 32)  
*Failed to release resources for unicast discovery locators.*
  - #define `DDSC_LOG_RELEASE_USER_MC_EC` (`DDSC_LOG_BASE` + 33)  
*Failed to release resources for multicast user locators.*
  - #define `DDSC_LOG_RELEASE_USER_UC_EC` (`DDSC_LOG_BASE` + 34)  
*Failed to release resources for unicast user locators.*
  - #define `DDSC_LOG_INVALID_DOMAINID_EC` (`DDSC_LOG_BASE` + 35)  
*The domain ID specified exceeds what is allowed based on the parameters specified in `dp_qos.protocol.rtps_well_known_ports`.*
  - #define `DDSC_LOG_ON_TYPE_REGISTERED_FAILURE_EC` (`DDSC_LOG_BASE` + 36)  
*Error occurred during the `on_type_registered` call back.*
  - #define `DDSC_LOG_ON_TYPE_UNREGISTERED_FAILURE_EC` (`DDSC_LOG_BASE` + 37)  
*Error occurred during the `on_type_unregistered` call back.*
  - #define `DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC` (`DDSC_LOG_BASE` + 38)  
*Error occurred during the `on_type_unregistered` call back.*
  - #define `DDSC_LOG_MIN_DISCOVERY_MTU_EC` (`DDSC_LOG_BASE` + 39)  
*Minimum MTU across all Discovery transports must be greater or equal to a packet containing serialized Participant↔BuiltinTopicData.*
  - #define `DDSC_LOG_MIN_MTU_SIZE_EC` (`DDSC_LOG_BASE` + 40)  
*MTU for a transport set lower or equal to Protocol Overhead of 448 bytes.*
  - #define `DDSC_LOG_GET_MTU_EC` (`DDSC_LOG_BASE` + 40)  
*Error getting the mtu from the route resolver.*
  - #define `DDSC_LOG_GET_MTU_NO_ROUTES_EC` (`DDSC_LOG_BASE` + 41)  
*MTU could not be found because of no routes.*
  - #define `DDSC_LOG_RESOLVE_LOCATORS_PRIORITY_EC` (`DDSC_LOG_BASE` + 42)  
*Failed to resolve locators based on priority.*
  - #define `DDSC_LOG_REMOTE_PUBLICATION_DATA_CHANGED_EC` (`DDSC_LOG_BASE` + 43)  
*The publication builtin topic data for a remote publication changed after the remote publication was asserted.*

- #define DDSC\_LOG\_REMOTE\_SUBSCRIPTION\_DATA\_CHANGED\_EC (DDSC\_LOG\_BASE + 44)  
*Fields in subscription builtin topic data for a remote subscription which are not supported to dynamically change were updated after the remote subscription was asserted.*
- #define DDSC\_LOG\_DATABASE\_CREATE\_EC (DDSC\_LOG\_BASE + 100)  
*Failed to create database.*
- #define DDSC\_LOG\_DATABASE\_DELETE\_EC (DDSC\_LOG\_BASE + 101)  
*Failed to delete database.*
- #define DDSC\_LOG\_TABLE\_CREATE\_EC (DDSC\_LOG\_BASE + 102)  
*Failed to create database table of the specified name.*
- #define DDSC\_LOG\_TABLE\_INUSE\_EC (DDSC\_LOG\_BASE + 103)  
*Failed to delete a database table because it is not empty.*
- #define DDSC\_LOG\_TABLE\_DELETE\_EC (DDSC\_LOG\_BASE + 104)  
*Failed to delete a database table.*
- #define DDSC\_LOG\_TABLE\_SELECT\_EC (DDSC\_LOG\_BASE + 105)  
*A selection operation failed on the specified database table.*
- #define DDSC\_LOG\_CREATE\_INDEX\_EC (DDSC\_LOG\_BASE + 106)  
*Failed to create an index on a database table.*
- #define DDSC\_LOG\_DELETE\_INDEX\_EC (DDSC\_LOG\_BASE + 107)  
*Failed to delete an index on a database table.*
- #define DDSC\_LOG\_DB\_CURSOR\_INVALIDATED\_EC (DDSC\_LOG\_BASE + 108)  
*A database table cursor was invalidated while in use.*
- #define DDSC\_LOG\_SUBSCRIPTION\_RECORD 1  
*Database record for a Subscription.*
- #define DDSC\_LOG\_PUBLICATION\_RECORD 2  
*Database record for a Publication.*
- #define DDSC\_LOG\_PARTICIPANT\_RECORD 3  
*Database record for a Remote Participant.*
- #define DDSC\_LOG\_TOPIC\_RECORD 4  
*Database record for a Topic.*
- #define DDSC\_LOG\_DATAWRITER\_RECORD 5  
*Database record for a DataWriter.*
- #define DDSC\_LOG\_DATAREADER\_RECORD 6  
*Database record for a DataReader.*
- #define DDSC\_LOG\_PUBLISHER\_RECORD 7  
*Database record for a Publisher.*
- #define DDSC\_LOG\_SUBSCRIBER\_RECORD 8  
*Database record for a Subscriber.*
- #define DDSC\_LOG\_ROUTE\_RECORD 9  
*Database record for a route record.*
- #define DDSC\_LOG\_BIND\_RECORD 10  
*Database record for a bind record.*
- #define DDSC\_LOG\_TYPE\_RECORD 11  
*Database record for a type plugin.*
- #define DDSC\_LOG\_REMOTE\_ENDPOINT\_TRUST\_STATE\_RECORD 12  
*Database record for a remote trust endpoint.*
- #define DDSC\_LOG\_MATCHED\_DW\_RECORD 13  
*Database record for a matched remote data writer.*

- #define `DDSC_LOG_STRING_RECORD` 14  
*Database record for a string property.*
- #define `DDSC_LOG_RECORD_CREATE_EC` (`DDSC_LOG_BASE` + 109)  
*Failed to create a database record of the specified kind.*
- #define `DDSC_LOG_RECORD_DELETE_EC` (`DDSC_LOG_BASE` + 110)  
*Failed to delete a database record of the specified kind.*
- #define `DDSC_LOG_RECORD_INSERT_EC` (`DDSC_LOG_BASE` + 111)  
*Failed to insert a database record of the specified kind.*
- #define `DDSC_LOG_RECORD_ERROR_EC` (`DDSC_LOG_BASE` + 112)  
*Unknown error for database record of the specified kind.*
- #define `DDSC_LOG_RECORD_EXISTS_EC` (`DDSC_LOG_BASE` + 113)  
*A database record of the specified kind already exists.*
- #define `DDSC_LOG_RECORD_LOOKUP_EC` (`DDSC_LOG_BASE` + 114)  
*A lookup of a database record of the specified kind failed.*
- #define `DDSC_LOG_RECORD_NOT_EXISTS_EC` (`DDSC_LOG_BASE` + 115)  
*A database record of the specified kind does not exist.*
- #define `DDSC_LOG_RECORD_SELECT_EC` (`DDSC_LOG_BASE` + 116)  
*A database select on the specified record kind failed.*
- #define `DDSC_LOG_RECORD_REMOVE_EC` (`DDSC_LOG_BASE` + 117)  
*Removal a database record of the specified kind failed.*
- #define `DDSC_LOG_RECORD_INITIALIZE_EC` (`DDSC_LOG_BASE` + 118)  
*A database record of the specified kind could not be initialized.*
- #define `DDSC_LOG_RECORD_FINALIZE_EC` (`DDSC_LOG_BASE` + 119)  
*A database record of the specified kind could not be finalized.*
- #define `DDSC_LOG_RECORD_RESET_EC` (`DDSC_LOG_BASE` + 120)  
*A database record of the specified kind could not be reset.*
- #define `DDSC_LOG_PUBLICATIONDATA_OBJECT` 1  
*DDS PublicationBuiltinTopicData object.*
- #define `DDSC_LOG_SUBSCRIPTIONDATA_OBJECT` 2  
*DDS SubscriptionBuiltinTopicData object.*
- #define `DDSC_LOG_PARTICIPANTDATA_OBJECT` 3  
*DDS ParticipantBuiltinTopicData object.*
- #define `DDSC_LOG_DATAREADERQOS_OBJECT` 4  
*DDS DataReaderQos object.*
- #define `DDSC_LOG_DATAWRITERQOS_OBJECT` 5  
*DDS DataWriterQos object.*
- #define `DDSC_LOG_TOPICQOS_OBJECT` 6  
*DDS TopicQos object.*
- #define `DDSC_LOG_PUBLISHERQOS_OBJECT` 7  
*DDS PublisherQos object.*
- #define `DDSC_LOG_SUBSCRIBERQOS_OBJECT` 8  
*DDS SubscriberQos object.*
- #define `DDSC_LOG_PARTICIPANTQOS_OBJECT` 9  
*DDS DomainParticipantQos object.*
- #define `DDSC_LOG_ENTITY_OBJECT` 10  
*DDS Entity object.*
- #define `DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT` 11

- *DDS DomainParticipantFactoryQos object.*
- #define [DDSC\\_LOG\\_TYPE\\_OBJECT](#) 12
  - *A DDS Type object.*
- #define [DDSC\\_LOG\\_BINDRESOLVER\\_OBJECT](#) 13
  - *A NETIO BindResolver object.*
- #define [DDSC\\_LOG\\_ROUTERESOLVER\\_OBJECT](#) 14
  - *A NETIO RouteResolver object.*
- #define [DDSC\\_LOG\\_ADDRESSRESOLVER\\_OBJECT](#) 15
  - *A NETIO AddressResolver object.*
- #define [DDSC\\_LOG\\_DATAWRITERIO\\_OBJECT](#) 16
  - *A NETIO DataWriterInterface object.*
- #define [DDSC\\_LOG\\_DATAREADERIO\\_OBJECT](#) 17
  - *A NETIO DataReaderInterface object.*
- #define [DDSC\\_LOG\\_LOG\\_OBJECT](#) 18
  - *A OSAPI Log object.*
- #define [DDSC\\_LOG\\_SYSTEM\\_OBJECT](#) 19
  - *A OSAPI system object.*
- #define [DDSC\\_LOG\\_MUTEX\\_OBJECT](#) 20
  - *A OSAPI mutex object.*
- #define [DDSC\\_LOG\\_CONDREF\\_OBJECT](#) 21
  - *A DDS Waitset condition reference object.*
- #define [DDSC\\_LOG\\_WSREF\\_OBJECT](#) 22
  - *A DDS Condition waitset reference object.*
- #define [DDSC\\_LOG\\_MD5STREAM\\_OBJECT](#) 23
  - *A MD5 stream object.*
- #define [DDSC\\_LOG\\_CONDITION\\_OBJECT](#) 24
  - *A DDS Condition object.*
- #define [DDSC\\_LOG\\_RT\\_OBJECT](#) 25
  - *A RT Condition object.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_POOL\\_OBJECT](#) 26
  - *A Participant pool object.*
- #define [DDSC\\_LOG\\_TIMER\\_OBJECT](#) 27
  - *A OSAPI\_Timer object.*
- #define [DDSC\\_LOG\\_TIMEROUT\\_OBJECT](#) 28
  - *A OSAPI\_Timer timeout object.*
- #define [DDSC\\_LOG\\_TOPIC\\_OBJECT](#) 29
  - *A DDS Topic object.*
- #define [DDSC\\_LOG\\_WAITSET\\_OBJECT](#) 30
  - *A DDS WaitSet object.*
- #define [DDSC\\_LOG\\_UUID\\_OBJECT](#) 31
  - *A UUID object.*
- #define [DDSC\\_LOG\\_MEMPOOL\\_OBJECT](#) 32
  - *A REDA\_MemPool object.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_PROPERTIES](#) 33
  - *A DomainParticipant's string properties sequence.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_BINARY\\_PROPERTIES](#) 34
  - *A DomainParticipant's binary properties sequence.*



- #define [DDSC\\_LOG\\_PARTICIPANT\\_BUILTIN\\_PUBLISHER](#) 35  
*A DomainParticipant's built-in Publisher.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_BUILTIN\\_SUBSCRIBER](#) 36  
*A DomainParticipant's built-in Subscriber.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_GENERIC\\_MESSAGE\\_OBJECT](#) 37  
*A built-in Inter-Participant Channel data type object.*
- #define [DDSC\\_LOG\\_DATA HOLDER\\_SEQ\\_OBJECT](#) 38  
*A Trust data holder sequence object.*
- #define [DDSC\\_LOG\\_PROPERTYQOSPOLICY\\_OBJECT](#) 39  
*A Property QoS policy object.*
- #define [DDSC\\_LOG\\_STRING\\_OBJECT](#) 40  
*A String object.*
- #define [DDSC\\_LOG\\_AUTHPOOL\\_OBJECT](#) 41  
*An authentication pool object.*
- #define [DDSC\\_LOG\\_BUFFER\\_OBJECT](#) 42  
*A Log buffer object.*
- #define [DDSC\\_LOG\\_STRINGMANAGER\\_OBJECT](#) 43  
*A String Manager object.*
- #define [DDSC\\_LOG\\_OBJECT\\_INITIALIZE\\_EC](#) (DDSC\_LOG\_BASE + 200)  
*Out of resources to initialize object of the specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_ALLOCATE\\_EC](#) (DDSC\_LOG\_BASE + 201)  
*Out of resources to allocate an object of the specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_FINALIZE\\_EC](#) (DDSC\_LOG\_BASE + 202)  
*Failed to finalize object of specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_DELETE\\_EC](#) (DDSC\_LOG\_BASE + 203)  
*Failed to delete object of specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_COPY\\_EC](#) (DDSC\_LOG\_BASE + 204)  
*Failed to copy object of specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_REFCOUNT\\_EC](#) (DDSC\_LOG\_BASE + 205)  
*Failed to delete/finalize an object because other objects are referencing it.*
- #define [DDSC\\_LOG\\_OBJECT\\_GET\\_PROPERTY\\_EC](#) (DDSC\_LOG\_BASE + 206)  
*Failed to get the object properties.*
- #define [DDSC\\_LOG\\_OBJECT\\_SET\\_PROPERTY\\_EC](#) (DDSC\_LOG\_BASE + 207)  
*Failed to set the object properties.*
- #define [DDSC\\_LOG\\_OBJECT\\_EMPTY\\_EC](#) (DDSC\_LOG\_BASE + 208)  
*An object is empty, typically applies only to buffer-pool objects.*
- #define [DDSC\\_LOG\\_OBJECT\\_INUSECOUNT\\_EC](#) (DDSC\_LOG\_BASE + 209)  
*Failed to delete/finalize an object because other objects are using it.*
- #define [DDSC\\_LOG\\_OBJECT\\_CREATE\\_EC](#) (DDSC\_LOG\_BASE + 210)  
*Failed to create an object.*
- #define [DDSC\\_LOG\\_NETMASK\\_SEQUENCE](#) 1  
*A NETIO Netmask sequence.*
- #define [DDSC\\_LOG\\_ROUTE\\_SEQUENCE](#) 2  
*A NETIO Route sequence.*
- #define [DDSC\\_LOG\\_RESERVED\\_SEQUENCE](#) 3  
*A NETIO Reserved address sequence.*
- #define [DDSC\\_LOG\\_ENABLED\\_USER\\_TRANSPORT\\_SEQUENCE](#) 4

- *A DDS sequence of enabled user NETIO transports.*
- #define `DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE` 5
- *A DDS sequence of enabled transports.*
- #define `DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE` 6
- *A DDS sequence of enabled discovery transports.*
- #define `DDSC_LOG_INITIAL_PEER_SEQUENCE` 7
- *A DDS sequence of initial peers.*
- #define `DDSC_LOG_DESTINATION_SEQUENCE` 8
- *A NETIO sequence of destinations to send to.*
- #define `DDSC_LOG_METAUNICAST_SEQUENCE` 9
- *A DDS sequence of meta-traffic unicast locators.*
- #define `DDSC_LOG_METAMULTICAST_SEQUENCE` 10
- *A DDS sequence of meta-traffic multicast locators.*
- #define `DDSC_LOG_USERUNICAST_SEQUENCE` 11
- *A DDS sequence of user-traffic unicast locators.*
- #define `DDSC_LOG_USERMULTICAST_SEQUENCE` 12
- *A DDS sequence of user-traffic multicast locators.*
- #define `DDSC_LOG_READTAKE_SEQUENCE` 13
- *A DDS sequence used in a read/take call.*
- #define `DDSC_LOG_DATAHOLDER_SEQUENCE` 14
- *A DDS sequence of DataHolder objects.*
- #define `DDSC_LOG_COOKIE_PAYLOAD_SEQUENCE` 15
- *A cookie octet sequence.*
- #define `DDSC_LOG_REPRESENTATION_ID_SEQUENCE` 16
- *a sequence for data representation IDs*
- #define `DDSC_LOG_SEQUENCE_SETMAX_EC (DDSC_LOG_BASE + 300)`
- *Failed to set the maximum length of a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_SETLENGTH_EC (DDSC_LOG_BASE + 301)`
- *Failed to set the length of a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_GETREF_EC (DDSC_LOG_BASE + 302)`
- *Failed to get a reference at the specified index for a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_INITIALIZE_EC (DDSC_LOG_BASE + 303)`
- *Failed to initialize a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_FINALIZE_EC (DDSC_LOG_BASE + 304)`
- *Failed to finalize a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_COPY_EC (DDSC_LOG_BASE + 305)`
- *Failed to copy a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_INVALID_EC (DDSC_LOG_BASE + 306)`
- *The sequence of the specified kind was invalid in the context it is used.*
- #define `DDSC_LOG_DISCOVERY_COMPONENT` 1
- *A component of discovery kind.*
- #define `DDSC_LOG_RTPS_COMPONENT` 2
- *A component of RTPS kind.*
- #define `DDSC_LOG_READERHISTORY_COMPONENT` 3
- *A component of reader-history kind.*
- #define `DDSC_LOG_WRITERHISTORY_COMPONENT` 4
- *A component of writer-history kind.*

- #define `DDSC_LOG_DATAREADERIO_COMPONENT` 5  
*A component of DataReaderInterface kind.*
- #define `DDSC_LOG_DATAWRITERIO_COMPONENT` 6  
*A component of DataWriterInterface kind.*
- #define `DDSC_LOG_NETIO_COMPONENT` 7  
*A component of NETIO kind.*
- #define `DDSC_LOG_TRANSPORT_COMPONENT` 8  
*A component of transport kind.*
- #define `DDSC_LOG_APPGEN_COMPONENT` 9  
*A component of application generation kind.*
- #define `DDSC_LOG_COMPONENT_LOOKUP_EC` (`DDSC_LOG_BASE` + 400)  
*Did not find a component factory with the given name in the registry.*
- #define `DDSC_LOG_COMPONENT_CREATE_EC` (`DDSC_LOG_BASE` + 401)  
*Could not create a component of the specified kind using the specified factory.*
- #define `DDSC_LOG_COMPONENT_DELETE_EC` (`DDSC_LOG_BASE` + 402)  
*Could not delete a component of the specified kind using the specified factory.*
- #define `DDSC_LOG_HISTORY_RESOURCE` 1  
*Exceeded resource limits for writer history queue.*
- #define `DDSC_LOG_SAMPLE_RESOURCES` 2  
*Failed to get resource for new sample due to resource limit.*
- #define `DDSC_LOG_PARTICIPANT_RESOURCES` 3  
*Failed to allocate a new participant.*
- #define `DDSC_LOG_RESOURCE_EXCEEDED_EC` (`DDSC_LOG_BASE` + 500)  
*Could not allocate a resource of the specified kind.*
- #define `DDSC_LOG_TOPIC_QOS` 1  
*The TopicQos kind.*
- #define `DDSC_LOG_DATAREADER_QOS` 2  
*The DataReaderQos kind.*
- #define `DDSC_LOG_DATAWRITER_QOS` 3  
*The DataWriterQos kind.*
- #define `DDSC_LOG_SUBSCRIBER_QOS` 4  
*The SubscriberQos kind.*
- #define `DDSC_LOG_PUBLISHER_QOS` 5  
*The PublisherQos kind.*
- #define `DDSC_LOG_PARTICIPANT_QOS` 6  
*The DomainParticipantQos kind.*
- #define `DDSC_LOG_PARTICIPANTFACTORY_QOS` 7  
*The DomainParticipantFactoryQos kind.*
- #define `DDSC_LOG_DEFAULTTOPIC_QOS` 8  
*The default TopicQos kind.*
- #define `DDSC_LOG_DEFAULTDATAREADER_QOS` 9  
*The default DataReaderQos kind.*
- #define `DDSC_LOG_DEFAULTDATAWRITER_QOS` 10  
*The default DataWriterQos kind.*
- #define `DDSC_LOG_DEFAULTSUBSCRIBER_QOS` 11  
*The default SubscriberQos kind.*
- #define `DDSC_LOG_DEFAULTPUBLISHER_QOS` 12

- *The default PublisherQos kind.*  
• #define DDSC\_LOG\_DEFAULTPARTICIPANT\_QOS 13
- *The default PublisherQos kind.*  
• #define DDSC\_LOG\_DEADLINE\_QOS\_POLICY 14
- *The deadline qos policy kind.*  
• #define DDSC\_LOG\_LIVELINESS\_QOS\_POLICY 15
- *The liveliness qos policy kind.*  
• #define DDSC\_LOG\_HISTORY\_QOS\_POLICY 16
- *The history qos policy kind.*  
• #define DDSC\_LOG\_RESOURCE\_LIMIT\_QOS\_POLICY 17
- *The resource limits qos policy kind.*  
• #define DDSC\_LOG\_PROTOCOL\_QOS\_POLICY 18
- *The protocol qos policy kind.*  
• #define DDSC\_LOG\_TYPE\_SUPPORT\_QOS\_POLICY 19
- *The type-support qos policy kind.*  
• #define DDSC\_LOG\_RELIABILITY\_QOS\_POLICY 20
- *The reliability qos policy kind.*  
• #define DDSC\_LOG\_DURABILITY\_QOS\_POLICY 21
- *The durability qos policy kind.*  
• #define DDSC\_LOG\_OWNERSHIP\_QOS\_POLICY 22
- *The ownership qos policy kind.*  
• #define DDSC\_LOG\_OWNERSHIP\_STRENGTH\_QOS\_POLICY 23
- *The ownership-strength qos policy kind.*  
• #define DDSC\_LOG\_TRANSPORT\_QOS\_POLICY 24
- *The transport qos policy kind.*  
• #define DDSC\_LOG\_PARTICIPANT\_ID\_QOS\_POLICY 25
- *The participant id qos policy kind.*  
• #define DDSC\_LOG\_HEARTBEATS\_QOS\_POLICY 26
- *The heartbeat qos policy kind.*  
• #define DDSC\_LOG\_DATAWRITER\_RESOURCE\_QOS\_POLICY 27
- *The DataWriterQos writer\_resource\_limits policy.*  
• #define DDSC\_LOG\_DATAREADER\_RESOURCE\_QOS\_POLICY 28
- *The DataReaderQos reader\_resource\_limits policy.*  
• #define DDSC\_LOG\_DESTINATION\_ORDER\_POLICY 29
- *The DestinationOrderQos destination\_order policy.*  
• #define DDSC\_LOG\_PROPERTY\_QOS\_POLICY 30
- *The PropertyQos policy.*  
• #define DDSC\_LOG\_PUBLISH\_MODE\_QOS\_POLICY 31
- *The PublishModeQos policy.*  
• #define DDSC\_LOG\_DATA\_REPRESENTATION\_QOS\_POLICY 33
- *The DataRepresentationQos policy.*  
• #define DDSC\_LOG\_LATENCY\_BUDGET\_QOS\_POLICY 34
- *The LatencyBudgetQos policy.*  
• #define DDSC\_LOG\_CONTENT\_FILTER\_QOS\_POLICY 35
- *The ContentFilterQos policy.*  
• #define DDSC\_LOG\_QOS\_INCONSISTENT\_POLICY\_EC (DDSC\_LOG\_BASE + 501)
- *An inconsistent Qos policy for the specified Qos kind was found.*

- #define `DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC` (`DDSC_LOG_BASE` + 502)  
*Inconsistency between two Qos policies for the specified Qos kind was found.*
- #define `DDSC_LOG_QOS_INCONSISTENT_EC` (`DDSC_LOG_BASE` + 503)  
*Failed to create an entity or set a qos due to inconsistent policy.*
- #define `DDSC_LOG_QOS_COPY_EC` (`DDSC_LOG_BASE` + 504)  
*Failed to copy a Qos of the specified kind.*
- #define `DDSC_LOG_QOS_INITIALIZE_EC` (`DDSC_LOG_BASE` + 505)  
*Failed to initialize a Qos of the specified kind.*
- #define `DDSC_LOG_QOS_FINALIZE_EC` (`DDSC_LOG_BASE` + 506)  
*Failed to finalize a Qos of the specified kind.*
- #define `DDSC_LOG_QOS_SET_EC` (`DDSC_LOG_BASE` + 507)  
*Failed to set a Qos of the specified kind.*
- #define `DDSC_LOG_QOS_SET_ON_ENABLED_EC` (`DDSC_LOG_BASE` + 508)  
*Failed to set a Qos of the specified kind because the entity is already enabled.*
- #define `DDSC_LOG_QOS_IMMUTABLE_EC` (`DDSC_LOG_BASE` + 509)  
*Failed to set a Qos of the specified kind the immutable Qos policies have been changed.*
- #define `DDSC_LOG_QOS_CHANGED_EC` (`DDSC_LOG_BASE` + 510)  
*A discovered Qos changed (the entity already existed)*
- #define `DDSC_LOG_QOS_GET_EC` (`DDSC_LOG_BASE` + 511)  
*Failed to get a Qos of the specified kind.*
- #define `DDSC_LOG_TOPIC_LISTENER` 1  
*The topic listener kind.*
- #define `DDSC_LOG_DATAREADER_LISTENER` 2  
*The datareader listener kind.*
- #define `DDSC_LOG_DATAWRITER_LISTENER` 3  
*The datawriter listener kind.*
- #define `DDSC_LOG_SUBSCRIBER_LISTENER` 4  
*The subscriber listener kind.*
- #define `DDSC_LOG_PUBLISHER_LISTENER` 5  
*The publisher listener kind.*
- #define `DDSC_LOG_PARTICIPANT_LISTENER` 6  
*The participant listener kind.*
- #define `DDSC_LOG_PARTICIPANTFACTORY_LISTENER` 7  
*The participant-factory listener kind.*
- #define `DDSC_LOG_LISTENER_INCONSISTENT_EC` (`DDSC_LOG_BASE` + 512)  
*Failed to create an entity due to inconsistent listener and status mask.*
- #define `DDSC_LOG_LISTENER_SET_EC` (`DDSC_LOG_BASE` + 513)  
*Failed to set the listener of the specified kind.*
- #define `DDSC_LOG_LISTENER_GET_EC` (`DDSC_LOG_BASE` + 514)  
*Failed to get the listener of the specified kind.*
- #define `DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC` (`DDSC_LOG_BASE` + 515)  
*Illegal combination of NULL listener and non-NONE status mask when setting a listener for an Entity.*
- #define `DDSC_LOG_TOPIC_ENTITY` 1  
*The topic entity kind.*
- #define `DDSC_LOG_DATAWRITER_ENTITY` 2  
*The datawriter entity kind.*
- #define `DDSC_LOG_DATAREADER_ENTITY` 3

- *The datareader entity kind.*  
• #define DDSC\_LOG\_PUBLISHER\_ENTITY 4
- *The publisher entity kind.*  
• #define DDSC\_LOG\_SUBSCRIBER\_ENTITY 5
- *The subscriber entity kind.*  
• #define DDSC\_LOG\_PARTICIPANT\_ENTITY 6
- *The participant entity kind.*  
• #define DDSC\_LOG\_PUBLICATION\_ENTITY 7
- *The publication entity kind.*  
• #define DDSC\_LOG\_SUBSCRIPTION\_ENTITY 8
- *The subscription entity kind.*  
• #define DDSC\_LOG\_ENTITY\_ENABLE\_EC (DDSC\_LOG\_BASE + 600)  
*Failed to enable an entity of the specified kind.*
- #define DDSC\_LOG\_ENTITY\_NOT\_EMPTY\_EC (DDSC\_LOG\_BASE + 601)  
*Failed to delete/finalize an entity of the specified kind because it is not empty.*
- #define DDSC\_LOG\_ENTITY\_FINALIZE\_EC (DDSC\_LOG\_BASE + 602)  
*Failed to finalize an entity of the specified kind.*
- #define DDSC\_LOG\_ENTITY\_INITIALIZE\_EC (DDSC\_LOG\_BASE + 603)  
*Failed to initialize an entity of the specified kind.*
- #define DDSC\_LOG\_ENTITY\_NOT\_ENABLED\_EC (DDSC\_LOG\_BASE + 604)  
*An operation was attempted on an entity that is not enabled.*
- #define DDSC\_LOG\_ENTITY\_DIFFERENT\_FACTORY\_EC (DDSC\_LOG\_BASE + 605)  
*Entities are in different factories.*
- #define DDSC\_LOG\_ENTITY\_INVALID\_PROPERTY\_EC (DDSC\_LOG\_BASE + 606)  
*An invalid property was specified for the entity.*
- #define DDSC\_LOG\_DATAWRITER\_CDR 1  
*The datawriter CDR kind.*
- #define DDSC\_LOG\_DATAREADER\_CDR 2  
*The datareader CDR kind.*
- #define DDSC\_LOG\_DATAWRITER\_INLINE 3  
*The datawriter inline kind.*
- #define DDSC\_LOG\_CDR\_POOL\_ALLOC\_EC (DDSC\_LOG\_BASE + 700)  
*Failed to allocate a pool of the specified kind.*
- #define DDSC\_LOG\_CDR\_BUFFER\_SET\_EC (DDSC\_LOG\_BASE + 701)  
*Failed to set the CDR buffer for a packet.*
- #define DDSC\_LOG\_CDR\_POOL\_DELETE\_EC (DDSC\_LOG\_BASE + 702)  
*Failed to delete the CDR pool.*
- #define DDSC\_LOG\_CDR\_SERIALIZE\_PID\_EC (DDSC\_LOG\_BASE + 703)  
*Failed to serialize a parameter ID.*
- #define DDSC\_LOG\_CDR\_SERIALIZE\_PID\_LENGTH\_EC (DDSC\_LOG\_BASE + 704)  
*Failed to serialize a parameter length.*
- #define DDSC\_LOG\_CDR\_SERIALIZE\_KEYHASH\_EC (DDSC\_LOG\_BASE + 705)  
*Failed to serialize a key-hash.*
- #define DDSC\_LOG\_CDR\_SERIALIZE\_DATA\_EC (DDSC\_LOG\_BASE + 706)  
*Failed to serialize payload data.*
- #define DDSC\_LOG\_DESERIALIZE\_BAD\_PID\_LENGTH\_EC (DDSC\_LOG\_BASE + 707)  
*Deserialized an invalid parameter length for a specific parameter ID.*

- #define `DDSC_LOG_CDR_DESERIALIZE_PID_EC` (`DDSC_LOG_BASE` + 708)  
*Failed to deserialize the ID of an inline parameter.*
- #define `DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC` (`DDSC_LOG_BASE` + 709)  
*Failed to deserialize the length of an inline parameter.*
- #define `DDSC_LOG_CDR_INCREMENT_POS_EC` (`DDSC_LOG_BASE` + 710)  
*Failed to increment to the position of the next inline parameter.*
- #define `DDSC_LOG_CDR_SET_POS_EC` (`DDSC_LOG_BASE` + 711)  
*Failed to set the reception stream position.*
- #define `DDSC_LOG_CDR_DESERIALIZE_HEADER_EC` (`DDSC_LOG_BASE` + 712)  
*Failed to deserialize the encapsulation header.*
- #define `DDSC_LOG_CDR_DESERIALIZE_DATA_EC` (`DDSC_LOG_BASE` + 713)  
*Failed to deserialize CDR payload data.*
- #define `DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC` (`DDSC_LOG_BASE` + 714)  
*Failed to initialize CDR sample.*
- #define `DDSC_LOG_CDR_FINALIZE_SAMPLE_EC` (`DDSC_LOG_BASE` + 715)  
*Failed to finalize sample.*
- #define `DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC` (`DDSC_LOG_BASE` + 716)  
*Failed to serialize the status info parameter.*
- #define `DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC` (`DDSC_LOG_BASE` + 717)  
*Failed to deserialize a key-hash.*
- #define `DDSC_LOG_CDR_DESERIALIZE_KEY_EC` (`DDSC_LOG_BASE` + 718)  
*Failed to deserialize CDR payload key.*
- #define `DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 800)  
*Failed to add a route to an anonymous participant discovery datawriter.*
- #define `DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 801)  
*Failed to add a route to a topic from a datawriter.*
- #define `DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 802)  
*Failed to delete a route to a topic.*
- #define `DDSC_LOG_NETIO_FORWARD_TOPIC_EC` (`DDSC_LOG_BASE` + 803)  
*Failed to forward a topic.*
- #define `DDSC_LOG_NETIO_NETIO_KIND` 1  
*Any NETIO interface kind, typically UDP.*
- #define `DDSC_LOG_INTRA_NETIO_KIND` 2  
*The intra interface kind.*
- #define `DDSC_LOG_RTPS_NETIO_KIND` 3  
*The RTPS interface kind.*
- #define `DDSC_LOG_DATAREADER_NETIO_KIND` 4  
*The DataReader interface kind.*
- #define `DDSC_LOG_DATAWRITER_NETIO_KIND` 5  
*The DataWriter interface kind.*
- #define `DDSC_LOG_NETIO_BIND_EXTERNAL_EC` (`DDSC_LOG_BASE` + 804)  
*Failed to bind two external interface of the specified kind.*
- #define `DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC` (`DDSC_LOG_BASE` + 805)  
*Failed to unbind two external interfaces of the specified kind.*
- #define `DDSC_LOG_NETIO_BIND_EC` (`DDSC_LOG_BASE` + 806)  
*Failed to bind an interface to a peer interface.*
- #define `DDSC_LOG_NETIO_UNBIND_EC` (`DDSC_LOG_BASE` + 807)

- Failed to unbind an interface from a peer interface.*

  - #define `DDSC_LOG_NETIO_ADD_ROUTE_EC` (`DDSC_LOG_BASE` + 808)
- Failed to add a route from an interface to a peer interface.*

  - #define `DDSC_LOG_NETIO_DELETE_ROUTE_EC` (`DDSC_LOG_BASE` + 809)
- Failed to delete a route from an interface to a peer interface.*

  - #define `DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC` (`DDSC_LOG_BASE` + 810)
- Failed to get an external interface for the specified interface kind.*

  - #define `DDSC_LOG_NETIO_NO_ROUTE_EC` (`DDSC_LOG_BASE` + 811)
- A DataReader failed a bind due to no existing route.*

  - #define `DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC` (`DDSC_LOG_BASE` + 812)
- Lookup a route to a destination failed.*

  - #define `DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC` (`DDSC_LOG_BASE` + 813)
- Failed to get the route table for an interface.*

  - #define `DDSC_LOG_NETIO_SEND_FAILED_EC` (`DDSC_LOG_BASE` + 814)
- Failure when sending on an interface.*

  - #define `DDSC_LOG_NETIO_SET_STATE_EC` (`DDSC_LOG_BASE` + 815)
- Failed to set an interface state.*

  - #define `DDSC_LOG_NETIO_PEER_LOOKUP_EC` (`DDSC_LOG_BASE` + 816)
- Datawriter did not find a peer.*

  - #define `DDSC_LOG_NETIO_FORCED_REMOVE_EC` (`DDSC_LOG_BASE` + 817)
- Forced removal of sample downstream failed.*

  - #define `DDSC_LOG_PACKET_INIT_EC` (`DDSC_LOG_BASE` + 818)
- Failed to initialize a packet.*

  - #define `DDSC_LOG_PACKET_SET_HEAD_EC` (`DDSC_LOG_BASE` + 819)
- Failed to set the head of a packet.*

  - #define `DDSC_LOG_PACKET_SET_TAIL_EC` (`DDSC_LOG_BASE` + 820)
- Failed to set the tail of a packet.*

  - #define `DDSC_LOG_NETIO_DISCOVERY_ENABLED_TRANSPORT_EC` (`DDSC_LOG_BASE` + 821)
- Configured discovery enabled transport is not valid.*

  - #define `DDSC_LOG_DW_ACKNACK_FAILED_EC` (`DDSC_LOG_BASE` + 900)
- Failed to ACKNACK sample in the writer history.*

  - #define `DDSC_LOG_DW_COMMIT_EC` (`DDSC_LOG_BASE` + 901)
- Failed to commit a sample to the writer queue.*

  - #define `DDSC_LOG_DW_KEYHASH_CREATE_EC` (`DDSC_LOG_BASE` + 902)
- Failed to create keyhash of instance handle.*

  - #define `DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC` (`DDSC_LOG_BASE` + 903)
- Failed a write due to an invalid key kind for the type being written.*

  - #define `DDSC_LOG_DW_CREATE_TYPED_WRITER_EC` (`DDSC_LOG_BASE` + 904)
- Failed to create a typed writer.*

  - #define `DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC` (`DDSC_LOG_BASE` + 905)
- Failed to register the key of an instance.*

  - #define `DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC` (`DDSC_LOG_BASE` + 906)
- Failed to create a flow-controller.*

  - #define `DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC` (`DDSC_LOG_BASE` + 907)
- The data-sample is larger then supported by the transport, but the flow-controller has been compiled out.*

  - #define `DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC` (`DDSC_LOG_BASE` + 908)



*Failed to reserve an instance to publish discovery data for a user created data writer. This typically means Domain↔ ParticipantQos.resource\_limits.remote\_writer\_allocation is too small.*

- #define DDSC\_LOG\_DW\_INCOMPATIBLE\_PADDING\_EC (DDSC\_LOG\_BASE + 909)  
*A DataReader incompatible with padding bits in encapsulation header was discovered.*
- #define DDSC\_LOG\_DR\_CREATE\_TYPED\_READER\_EC (DDSC\_LOG\_BASE + 1000)  
*Failed to create a typed datareader.*
- #define DDSC\_LOG\_DR\_COPY\_DATA\_SAMPLE\_EC (DDSC\_LOG\_BASE + 1001)  
*Failed to copy a sample upon reception, read, or take.*
- #define DDSC\_LOG\_DR\_COMMIT\_SAMPLE\_EC (DDSC\_LOG\_BASE + 1002)  
*Failed to commit a sample to be made available to be read or taken.*
- #define DDSC\_LOG\_DR\_FILTER\_ERROR\_EC (DDSC\_LOG\_BASE + 1003)  
*A datareader filter function failed.*
- #define DDSC\_LOG\_DR\_DESERIALIZE\_KEYHASH\_EC (DDSC\_LOG\_BASE + 1004)  
*Failed to deserialize a key-hash parameter.*
- #define DDSC\_LOG\_DR\_GET\_ENTRY\_FAILED\_EC (DDSC\_LOG\_BASE + 1005)  
*Failed to get a Reader History entry for a received sample.*
- #define DDSC\_LOG\_DR\_COMMIT\_ENTRY\_EC (DDSC\_LOG\_BASE + 1006)  
*Failed to commit a receive sample to Reader History to be read or taken.*
- #define DDSC\_LOG\_DR\_UNREGISTER\_KEY\_EC (DDSC\_LOG\_BASE + 1007)  
*A DataReader failed to unregister an instance.*
- #define DDSC\_LOG\_DR\_DISPOSE\_KEY\_EC (DDSC\_LOG\_BASE + 1008)  
*A DataReader failed to dispose an instance.*
- #define DDSC\_LOG\_DR\_READ\_TAKE\_FAILURE\_EC (DDSC\_LOG\_BASE + 1009)  
*A call to a reader/take function failed.*
- #define DDSC\_LOG\_DR\_INSTANCE\_TO\_KEYHASH\_EC (DDSC\_LOG\_BASE + 1010)  
*Failed to create keyhash of instance handle.*
- #define DDSC\_LOG\_DR\_INSTANCE\_MAPPING\_EXHAUSTED\_EC (DDSC\_LOG\_BASE + 1011)  
*Failed to create keyhash of instance handle.*
- #define DDSC\_LOG\_DR\_INSTANCE\_ASSERTION\_FAILED\_EC (DDSC\_LOG\_BASE + 1012)  
*Failed to reserve an instance to publish discovery data for a user created data reader. This typically means Domain↔ ParticipantQos.resource\_limits.remote\_reader\_allocation is too small.*
- #define DDSC\_LOG\_DR\_ON\_SAMPLE\_REMOVED\_EC (DDSC\_LOG\_BASE + 1013)  
*Failed to remove a sample when it was being pushed out from the Reader History.*
- #define DDSC\_LOG\_TYPE\_NAME\_CMP\_EC (DDSC\_LOG\_BASE + 1100)  
*Two type names are incompatible.*
- #define DDSC\_LOG\_TOPIC\_NAME\_CMP\_EC (DDSC\_LOG\_BASE + 1101)  
*Two topic names are incompatible.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_GET\_SERIALIZED\_SAMPLE\_MAX\_SIZE 1  
*The get\_serialized\_sample\_max\_size function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_SERIALIZE\_DATA 2  
*The serialize\_data function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_DESERIALIZE\_DATA 3  
*The deserialize\_data function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_CREATE\_SAMPLE 4  
*The create\_sample function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_COPY\_SAMPLE 5  
*The copy\_sample function kind.*

- `#define DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE 6`  
*The delete\_sample function kind.*
- `#define DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND 7`  
*The get\_key\_kind function kind.*
- `#define DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH 8`  
*The instance\_to\_keyhash function kind.*
- `#define DDSC_LOG_TYPE_FUNCTION_NULL_EC (DDSC_LOG_BASE + 1102)`  
*Invalid type plugin, The specified function pointer is NULL.*
- `#define DDSC_LOG_TOPIC_TOO_LONG_EC (DDSC_LOG_BASE + 1103)`  
*Failed to create a topic because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)*
- `#define DDSC_LOG_TYPE_TOO_LONG_EC (DDSC_LOG_BASE + 1104)`  
*Failed to create a type because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)*
- `#define DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC (DDSC_LOG_BASE + 1105)`  
*A type-plugin for the given type could not be found.*
- `#define DDSC_LOG_TYPE_KEY_TYPE_EC (DDSC_LOG_BASE + 1106)`  
*Two types have incompatible keys.*
- `#define DDSC_LOG_TYPE_PLUGIN_GET_BUFFER_EC (DDSC_LOG_BASE + 1107)`  
*Type-plugin cannot return a buffer.*
- `#define DDSC_LOG_TYPE_PLUGIN_INCONSISTENT_DR_EC (DDSC_LOG_BASE + 1108)`  
*Inconsisten type-plugin DataReader.*
- `#define DDSC_LOG_DUPLICATE_GUID_PREFIX_EC (DDSC_LOG_BASE + 1109)`  
*A DomainParticipant with the specified GUID already exists in the DomainParticipant factory.*
- `#define DDSC_LOG_TYPE_MANAGED_POOL_FAILURE_EC (DDSC_LOG_BASE + 1110)`  
*Managed pool function failed.*
- `#define DDSC_LOG_TYPE_GET_SAMPLE_ID_EC (DDSC_LOG_BASE + 1111)`  
*Managed pool failed to get sample id.*
- `#define DDSC_LOG_PARTICIPANT_NAME_EMPTY_EC (DDSC_LOG_BASE + 1112)`  
*A DomainParticipant with enable\_participant\_discovery\_by\_name set to to TRUE but an empty name is created.*
- `#define DDSC_LOG_DUPLICATE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 1113)`  
*A DomainParticipant with enable\_participant\_discovery\_by\_name set to to TRUE, but a local participant with the same DomainParticipantQos.participant\_name.name already exists.*
- `#define DDSC_LOG_DUPLICATE_REMOTE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 1114)`  
*A remote participant is discovered with the same name as the a local participant name in the same domain ID.*
- `#define DDSC_LOG_REMOTE_PARTICIPANT_NAME_EMPTY_EC (DDSC_LOG_BASE + 1115)`  
*A remote participant without a name is discovered.*
- `#define DDSC_LOG_REMOTE_PARTICIPANT_RESTART_EC (DDSC_LOG_BASE + 1116)`  
*Failed to reset a remote participant.*
- `#define DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC (DDSC_LOG_BASE + 1200)`  
*Discovery plugin failed its update after a local DomainParticipant was enabled.*
- `#define DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1201)`  
*Discovery plugin failed its update before a local DomainParticipant was created.*
- `#define DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1202)`  
*Discovery plugin failed its update after a local DomainParticipant was created.*
- `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC (DDSC_LOG_BASE + 1203)`  
*Discovery plugin failed its update after a local DataReader was enabled.*
- `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC (DDSC_LOG_BASE + 1204)`  
*Discovery plugin failed its update after a local DataReader was deleted.*

- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC` (`DDSC_LOG_BASE` + 1205)  
*Discovery plugin failed its update after a local DataWriter was enabled.*
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC` (`DDSC_LOG_BASE` + 1206)  
*Discovery plugin failed its update after a local DataWriter was deleted.*
- #define `DDSC_LOG_DISC_BEFORE_REMOTE_PARTICIPANT_DELETED_EC` (`DDSC_LOG_BASE` + 1207)  
*Discovery plugin failed its update after being notified that a remote DomainParticipant was about to be deleted.*
- #define `DDSC_LOG_DISC_ADD_PEER_EC` (`DDSC_LOG_BASE` + 1208)  
*Failed to add a peer with discovery plugin.*
- #define `DDSC_LOG_DESERIALIZE_UNKNOWN_PID_EC` (`DDSC_LOG_BASE` + 1209)  
*Deserialized a parameter of unknown ID.*
- #define `DDSC_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (`DDSC_LOG_BASE` + 1210)  
*Failed to serialize Builtin Endpoint Mask parameter.*
- #define `DDSC_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (`DDSC_LOG_BASE` + 1211)  
*Failed to deserialize Builtin Endpoint Mask parameter.*
- #define `DDSC_LOG_SERIALIZE_TOPIC_NAME_EC` (`DDSC_LOG_BASE` + 1212)  
*Failed to serialize Topic Name parameter.*
- #define `DDSC_LOG_CDR_SET_OFFSET_EC` (`DDSC_LOG_BASE` + 1213)  
*Failed to set current offset of stream.*
- #define `DDSC_LOG_SERIALIZE_ENTITY_NAME_EC` (`DDSC_LOG_BASE` + 1214)  
*Failed to serialize Entity Name parameter.*
- #define `DDSC_LOG_SERIALIZE_TYPE_NAME_EC` (`DDSC_LOG_BASE` + 1215)  
*Failed to serialize Type Name parameter.*
- #define `DDSC_LOG_SERIALIZE_GUID_EC` (`DDSC_LOG_BASE` + 1216)  
*Failed to serialize GUID key.*
- #define `DDSC_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (`DDSC_LOG_BASE` + 1217)  
*Failed to serialize Default Unicast Locator parameter.*
- #define `DDSC_LOG_DESERIALIZE_LOCATOR_EC` (`DDSC_LOG_BASE` + 1218)  
*Failed to deserialize a locator of the specified kind.*
- #define `DDSC_LOG_LOCATORS_FULL_EC` (`DDSC_LOG_BASE` + 1219)  
*A locator sequence is full, the remaining locators of the specified kind are dropped.*
- #define `DDSC_LOG_UNSUPPORTED_LOCATOR_EC` (`DDSC_LOG_BASE` + 1220)  
*The locator kind is not supported by any transport registered with the domain participant and is dropped.*
- #define `DDSC_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (`DDSC_LOG_BASE` + 1221)  
*Failed to serialize Protocol Version parameter.*
- #define `DDSC_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (`DDSC_LOG_BASE` + 1222)  
*Failed to deserialize Protocol Version parameter.*
- #define `DDSC_LOG_DESERIALIZE_VENDOR_ID_EC` (`DDSC_LOG_BASE` + 1223)  
*Failed to deserialize Vendor ID parameter.*
- #define `DDSC_LOG_SERIALIZE_VENDOR_ID_EC` (`DDSC_LOG_BASE` + 1224)  
*Failed to serialize Vendor ID parameter.*
- #define `DDSC_LOG_SERIALIZE_PRODUCT_VERSION_EC` (`DDSC_LOG_BASE` + 1225)  
*Failed to serialize Product Version parameter.*
- #define `DDSC_LOG_SERIALIZE_LEASE_DURATION_EC` (`DDSC_LOG_BASE` + 1226)  
*Failed to serialize Lease Duration parameter.*
- #define `DDSC_LOG_DATAWRITER_ADD_PEER_FAILED_EC` (`DDSC_LOG_BASE` + 1227)  
*Failed to add a remote peer to a local DataWriter.*
- #define `DDSC_LOG_DATAREADER_ADD_PEER_FAILED_EC` (`DDSC_LOG_BASE` + 1228)

- Failed to add a remote peer to a local DataReader.*

  - #define `DDSC_LOG_DISC_REMOTE_PARTICIPANT_REMOVE_EC` (`DDSC_LOG_BASE` + 1229)
- Failed to remote/reset remote participant after liveliness expired.*

  - #define `DDSC_LOG_DESERIALIZE_PARTITION_SEQ_EC` (`DDSC_LOG_BASE` + 1230)
- Failed to deserialize partition seq.*

  - #define `DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC` (`DDSC_LOG_BASE` + 1231)
- Failed to initialize discovery queue.*

  - #define `DDSC_LOG_QUEUE_PUBLICATION_DISCOVERY_EC` (`DDSC_LOG_BASE` + 1232)
- Failed to queue a publication discovery message.*

  - #define `DDSC_LOG_QUEUE_SUBSCRIPTION_DISCOVERY_EC` (`DDSC_LOG_BASE` + 1233)
- Failed to queue a subscription discovery message.*

  - #define `DDSC_LOG_QUEUE_FINALIZE_EC` (`DDSC_LOG_BASE` + 1234)
- Failed finalize discovery queue.*

  - #define `DDSC_LOG_IPC_INIT_FAILED_EC` (`DDSC_LOG_BASE` + 1300)
- Failed to initialize Builtin IPC entities.*

  - #define `DDSC_LOG_IPC_ALLOC_FAILED_EC` (`DDSC_LOG_BASE` + 1301)
- Failed to allocate memory for one of the Builtin IPC entities.*

  - #define `DDSC_LOG_IPC_FINALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1302)
- Failed to finalize all resources A potential memory leak might have been caused by unexpected errors during finalization of acquired resources.*

  - #define `DDSC_LOG_IPC_CHANNEL_TYPE_REGISTER_EC` (`DDSC_LOG_BASE` + 1303)
- Failed to register type for a Builtin IPC channel.*

  - #define `DDSC_LOG_IPC_CHANNEL_TOPIC_CREATE_EC` (`DDSC_LOG_BASE` + 1304)
- Failed to create topic for a Builtin IPC channel.*

  - #define `DDSC_LOG_IPC_CHANNEL_READER_CREATE_EC` (`DDSC_LOG_BASE` + 1305)
- Failed to create reader for a Builtin IPC channel.*

  - #define `DDSC_LOG_IPC_CHANNEL_WRITER_CREATE_EC` (`DDSC_LOG_BASE` + 1306)
- Failed to create writer for a Builtin IPC channel.*

  - #define `DDSC_LOG_IPC_CHANNEL_TYPE_PLUGIN_EC` (`DDSC_LOG_BASE` + 1307)
- Failed to find type plugin for a Builtin IPC channel.*

  - #define `DDSC_LOG_IPC_CHANNEL_UNKNOWN_EC` (`DDSC_LOG_BASE` + 1308)
- Unknown builtin IPC channel requested.*

  - #define `DDSC_LOG_IPC_UPDATE_QOS_FAILED_EC` (`DDSC_LOG_BASE` + 1309)
- Failed to update DomainParticipantQos to account for built-in IPC endpoints.*

  - #define `DDSC_LOG_IPC_CHANNEL_ADD_ANON_ROUTE_EC` (`DDSC_LOG_BASE` + 1310)
- Failed to add an anonymous route to a built-in DataWriter or DataReader.*

  - #define `DDSC_LOG_IPC_CHANNEL_DELETE_ANON_ROUTE_EC` (`DDSC_LOG_BASE` + 1311)
- Failed to delete an anonymous route to a built-in DataWriter or DataReader.*

  - #define `DDSC_LOG_IPC_CHANNEL_ADD_PEER_EC` (`DDSC_LOG_BASE` + 1312)
- Failed to add an inter-participant channel peer.*

  - #define `DDSC_LOG_IPC_CHANNEL_REMOVE_PEER_EC` (`DDSC_LOG_BASE` + 1313)
- Failed to remove an inter-participant channel peer.*

  - #define `DDSC_LOG_IPC_CHANNEL_RETURN_LOAN_EC` (`DDSC_LOG_BASE` + 1314)
- Failed to return a loaned inter-participant sample.*

  - #define `DDSC_LOG_IPC_CHANNEL_PROCESS_SAMPLE_EC` (`DDSC_LOG_BASE` + 1315)
- Failed to process inter-participant message.*

  - #define `DDSC_LOG_IPC_ASSERT_ROUTES_EC` (`DDSC_LOG_BASE` + 1316)

- Failed to assert inter-participant routes.*

  - #define `DDSC_LOG_IPC_CHANNEL_WRITE_SAMPLE_EC` (`DDSC_LOG_BASE` + 1317)
- Failed to write inter-participant message.*

  - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_HANDSHAKE_FAILED_EC` (`DDSC_LOG_BASE` + 1318)
- Failed to process handshake authentication.*

  - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1319)
- Failed to process participant interceptor tokens.*

  - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1320)
- Failed to process endpoint interceptor tokens.*

  - #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_REQUEST_FAILED_EC` (`DDSC_LOG_BASE` + 1321)
- Failed to process authentication request.*

  - #define `DDSC_LOG_IPC_CHANNEL_CONFIG_INITIALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1322)
- Failed to initialize inter-participant channel.*

  - #define `DDSC_LOG_IPC_CHANNEL_CONFIG_FINALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1323)
  - #define `DDSC_LOG_IPC_CHANNEL_CONFIG_SET_FAILED_EC` (`DDSC_LOG_BASE` + 1324)
- Failed to get inter-participant channel configuration.*

  - #define `DDSC_LOG_IPC_CHANNEL_SET_LISTENERS_FAILED_EC` (`DDSC_LOG_BASE` + 1325)
- Failed to set inter-participant channel listeners.*

  - #define `DDSC_LOG_IPC_CHANNEL_ENABLE_FAILED_EC` (`DDSC_LOG_BASE` + 1326)
- Failed to enable inter-participant channel entities.*

  - #define `DDSC_LOG_IPC_CHANNEL_CREATE_FAILED_EC` (`DDSC_LOG_BASE` + 1327)
- Failed to create inter-participant channel.*

  - #define `DDSC_LOG_TRUST_UNKNOWN_GMCLASSID_EC` (`DDSC_LOG_BASE` + 1401)
- Unknown generic message class id.*

  - #define `DDSC_LOG_TRUST_UNKNOWN_SERVICEID_EC` (`DDSC_LOG_BASE` + 1402)
- Unknown builtin service id.*

  - #define `DDSC_LOG_TRUST_IGNORED_REMOTE_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 1403)
- Ignored remote participant.*

  - #define `DDSC_LOG_TRUST_IGNORED_HANDSHAKE_MESSAGE_EC` (`DDSC_LOG_BASE` + 1404)
- Ignored handshake message from remote participant.*

  - #define `DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC` (`DDSC_LOG_BASE` + 1405)
- Unexpected validation result.*

  - #define `DDSC_LOG_TRUST_UNAUTHORIZED_REMOTE_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 1406)
- A remote participant failed the authorization process.*

  - #define `DDSC_LOG_TRUST_FAILED_VALIDATE_LOCAL_IDENTITY_EC` (`DDSC_LOG_BASE` + 1407)
- Failed to validate local identity.*

  - #define `DDSC_LOG_TRUST_INVALID_IDENTITY_HANDLE_EC` (`DDSC_LOG_BASE` + 1408)
- Invalid local trust identity.*

  - #define `DDSC_LOG_TRUST_GET_IDENTITY_TOKEN_FAILED_EC` (`DDSC_LOG_BASE` + 1409)
- Failed to get identity token.*

  - #define `DDSC_LOG_TRUST_INVALID_PERMISSIONS_HANDLE_EC` (`DDSC_LOG_BASE` + 1410)
- Invalid local trust permissions.*

  - #define `DDSC_LOG_TRUST_INVALID_PARTICIPANT_GUID_EC` (`DDSC_LOG_BASE` + 1411)
- Invalid DomainParticipant GUID configuration.*

- #define DDSC\_LOG\_TRUST\_GET\_PARTICIPANT\_TRUST\_ATTRIBUTES\_FAILED\_EC (DDSC\_LOG\_BASE + 1412)  
*Failed to get DomainParticipant trust attributes.*
- #define DDSC\_LOG\_TRUST\_CHECK\_CREATE\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1413)  
*Failed to create access control DomainParticipant.*
- #define DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1414)  
*Failed to get permissions credential token.*
- #define DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1415)  
*Failed to get permissions token.*
- #define DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_HANDLE\_EC (DDSC\_LOG\_BASE + 1416)  
*Invalid interceptor handle.*
- #define DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1417)  
*Failed to register local DomainParticipant.*
- #define DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1418)  
*Failed to return permissions credential token.*
- #define DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1419)  
*Failed to return permissions token.*
- #define DDSC\_LOG\_TRUST\_RETURN\_IDENTITY\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1420)  
*Failed to return identity token.*
- #define DDSC\_LOG\_TRUST\_UNREGISTER\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1421)  
*Failed to unregister local participant.*
- #define DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_HANDLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1422)  
*Failed to return permissions handle.*
- #define DDSC\_LOG\_TRUST\_RETURN\_IDENTITY\_HANDLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1423)  
*Failed to return identity handle.*
- #define DDSC\_LOG\_TRUST\_RETURN\_SHAREDSECRET\_HANDLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1424)  
*Failed to return shared secret handle.*
- #define DDSC\_LOG\_TRUST\_GET\_AUTHENTICATED\_PEER\_CREDENTIAL\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1425)  
*Failed to get authenticated peer credential token.*
- #define DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1426)  
*Failed to check access control remote DomainParticipant.*
- #define DDSC\_LOG\_TRUST\_GET\_SHARED\_SECRET\_FAILED\_EC (DDSC\_LOG\_BASE + 1427)  
*Failed to get shared secret.*
- #define DDSC\_LOG\_TRUST\_PREPARE\_LOCAL\_PARTICIPANT\_STATE\_FAILED\_EC (DDSC\_LOG\_BASE + 1428)  
*Failed to validate local DomainParticipant trust.*
- #define DDSC\_LOG\_TRUST\_DELETE\_TRUST\_PLUGINS\_EC (DDSC\_LOG\_BASE + 1429)  
*Failed to delete trust plugins.*
- #define DDSC\_LOG\_TRUST\_SET\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1430)  
*Failed to set permissions credential and token.*
- #define DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1431)  
*Failed to register matched remote DomainParticipant.*

- #define DDSC\_LOG\_TRUST\_GET\_TOPIC\_TRUST\_ATTRIBUTES\_FAILED\_EC (DDSC\_LOG\_BASE + 1432)  
*Failed to get topic trust attributes.*
- #define DDSC\_LOG\_TRUST\_CHECK\_CREATE\_DATAWRITER\_FAILED\_EC (DDSC\_LOG\_BASE + 1433)  
*Failed to create trust check DataWriter.*
- #define DDSC\_LOG\_TRUST\_RETURN\_DATAWRITER\_TRUST\_ATTRIBUTES\_FAILED\_EC (DDSC\_LOG\_BASE + 1434)  
*Failed to return DataWriter trust attributes.*
- #define DDSC\_LOG\_TRUST\_GET\_DATAWRITER\_TRUST\_ATTRIBUTES\_FAILED\_EC (DDSC\_LOG\_BASE + 1435)  
*Failed to get DataWriter trust attributes.*
- #define DDSC\_LOG\_TRUST\_GET\_DATAREADER\_TRUST\_ATTRIBUTES\_FAILED\_EC (DDSC\_LOG\_BASE + 1436)  
*Failed to get DataReader trust attributes.*
- #define DDSC\_LOG\_TRUST\_CHECK\_CREATE\_DATAREADER\_FAILED\_EC (DDSC\_LOG\_BASE + 1437)  
*Failed to create trust check DataReader.*
- #define DDSC\_LOG\_TRUST\_BEGIN\_HANDSHAKE\_REPLY\_EC (DDSC\_LOG\_BASE + 1438)  
*Handshake begin reply failed.*
- #define DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_PARTICIPANT\_INTERCEPTOR\_TOKENS\_EC (DDSC\_LOG\_BASE + 1439)  
*Failed to create local DomainParticipant interceptor tokens.*
- #define DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DW\_TOKEN\_EC (DDSC\_LOG\_BASE + 1440)  
*Failed to create local DataWriter interceptor token.*
- #define DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DR\_TOKEN\_EC (DDSC\_LOG\_BASE + 1441)  
*Failed to create local DataReader interceptor token.*
- #define DDSC\_LOG\_TRUST\_RETURN\_HANDSHAKE\_HANDLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1442)  
*Failed to return handshake handle.*
- #define DDSC\_LOG\_TRUST\_SET\_REMOTE\_PARTICIPANT\_INTERCEPTOR\_TOKENS\_FAILED\_EC (DDSC\_LOG\_BASE + 1443)  
*Failed to set remote DomainParticipant interceptor tokens.*
- #define DDSC\_LOG\_TRUST\_SET\_REMOTE\_DATAREADER\_INTERCEPTOR\_TOKENS\_FAILED\_EC (DDSC\_LOG\_BASE + 1444)  
*Failed to set remote DataReader interceptor tokens.*
- #define DDSC\_LOG\_TRUST\_SET\_REMOTE\_DATAWRITER\_INTERCEPTOR\_TOKENS\_FAILED\_EC (DDSC\_LOG\_BASE + 1445)  
*Failed to set remote DataWriter interceptor tokens.*
- #define DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_DATAREADER\_FAILED\_EC (DDSC\_LOG\_BASE + 1446)  
*Failed to register local DataReader.*
- #define DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_DATAWRITER\_FAILED\_EC (DDSC\_LOG\_BASE + 1447)  
*Failed to register local DataWriter.*
- #define DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_DATAWRITER\_FAILED\_EC (DDSC\_LOG\_BASE + 1448)  
*Failed to check remote DataWriter.*
- #define DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_DATAREADER\_FAILED\_EC (DDSC\_LOG\_BASE + 1449)  
*Failed to check remote DataReader.*
- #define DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_DATAWRITER\_FAILED\_EC (DDSC\_LOG\_BASE + 1450)  
*Failed to register matched remote DataWriter.*
- #define DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_DATAREADER\_FAILED\_EC (DDSC\_LOG\_BASE + 1451)



- Failed to register matched remote DataReader.*

  - #define `DDSC_LOG_TRUST_UNREGISTER_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1452)
- Failed to unregister DataReader.*

  - #define `DDSC_LOG_TRUST_UNREGISTER_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1453)
- Failed to unregister DataWriter.*

  - #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_IDENTITY_EC` (`DDSC_LOG_BASE` + 1454)
- Failed to validate remote identity.*

  - #define `DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REQUEST_EC` (`DDSC_LOG_BASE` + 1455)
- Handshake begin request failed.*

  - #define `DDSC_LOG_TRUST_PROCESS_HANDSHAKE_EC` (`DDSC_LOG_BASE` + 1456)
- Handshake process failed.*

  - #define `DDSC_LOG_TRUST_CREATE_TRUST_PLUGINS_FAILED_EC` (`DDSC_LOG_BASE` + 1457)
- Failed to create trust plugins.*

  - #define `DDSC_LOG_TRUST_PARSE_PROPERTY_FAILED_EC` (`DDSC_LOG_BASE` + 1458)
- Failed to parse trust property.*

  - #define `DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_READY_FAILED_EC` (`DDSC_LOG_BASE` + 1459)
- Error after remote trust DomainParticipant is ready.*

  - #define `DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 1460)
- Failed to remove remote DomainParticipant.*

  - #define `DDS_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC` (`DDSC_LOG_BASE` + 1461)
- Failed to assert DomainParticipant generic message.*

  - #define `DDSC_LOG_TRUST_SEND_HANDSHAKE_EC` (`DDSC_LOG_BASE` + 1462)
- Failed to send handshake message.*

  - #define `DDSC_LOG_VALIDATE_REMOTE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE` + 1463)
- Failed to validate trust remote DomainParticipant.*

  - #define `DDSC_LOG_TRUST_BIND_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1464)
- Failed to get remote DataReader bind properties.*

  - #define `DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1465)
- Failed to register remote DataReader.*

  - #define `DDSC_LOG_TRUST_BIND_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1466)
- Failed to get remote DataWriter bind properties.*

  - #define `DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1467)
- Failed to register remote DataWriter.*

  - #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAWRITER_TRUST_FAILED_EC` (`DDSC_LOG_BASE` + 1468)
- Failed to validate remote DataWriter.*

  - #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAREADER_TRUST_FAILED_EC` (`DDSC_LOG_BASE` + 1469)
- Failed to validate remote DataReader.*

  - #define `DDSC_LOG_TRUST_VALIDATE_LOCAL_DATAWRITER_TRUST_FAILED_EC` (`DDSC_LOG_BASE` + 1470)
- Failed to validate local DataWriter.*

  - #define `DDSC_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC` (`DDSC_LOG_BASE` + 1471)
- Invalid DomainParticipant trust generic message.*

  - #define `DDSC_LOG_TRUST_PREPARE_AUTH_REQUEST_EC` (`DDSC_LOG_BASE` + 1472)
- Failed to prepare authentication request.*



- #define DDSC\_LOG\_TRUST\_PREPARE\_AUTH\_HANDSHAKE\_MESSAGE\_EC (DDSC\_LOG\_BASE + 1473)  
*Failed to prepare authentication handshake message.*
- #define DDSC\_LOG\_TRUST\_PREPARE\_ENDPOINT\_INTERCEPTOR\_MESSAGE\_EC (DDSC\_LOG\_BASE + 1474)  
*Failed to prepare endpoint interceptor message.*
- #define DDSC\_LOG\_TRUST\_INVALID\_GENERIC\_MESSAGE\_REPLY\_EC (DDSC\_LOG\_BASE + 1475)  
*Invalid trust generic message reply.*
- #define DDSC\_LOG\_FINALIZE\_TRUST\_FAILED\_EC (DDSC\_LOG\_BASE + 1476)  
*Failed to finalize trust.*
- #define DDSC\_LOG\_TRUST\_DATA HOLDER\_COPY\_FAILED\_EC (DDSC\_LOG\_BASE + 1477)  
*Failed to copy trust data holder.*
- #define DDSC\_LOG\_TRUST\_INVALID\_SAMPLE\_MAX\_SERIALIZED\_SIZE\_EC (DDSC\_LOG\_BASE + 1478)  
*Invalid sample serialized size.*
- #define DDSC\_LOG\_TRUST\_ALLOC\_FAILED\_EC (DDSC\_LOG\_BASE + 1479)  
*Failed to allocate memory.*
- #define DDSC\_LOG\_TRUST\_SERIALIZE\_DATA\_FAILED\_EC (DDSC\_LOG\_BASE + 1480)  
*Failed to serialize trust data.*
- #define DDSC\_LOG\_TRUST\_INITIALIZE\_PARTICIPANT\_TRUST\_FAILED\_EC (DDSC\_LOG\_BASE + 1481)  
*Failed to initialize trust.*
- #define DDSC\_LOG\_TRUST\_INITIALIZE\_PARTICIPANT\_BUILTIN\_DATA\_FAILED\_EC (DDSC\_LOG\_BASE + 1482)  
*Failed to initialize DomainParticipant trust data.*
- #define DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_GENERIC\_MESSAGE\_EC (DDSC\_LOG\_BASE + 1483)  
*Invalid DomainParticipant trust generic message.*
- #define DDSC\_LOG\_TRUST\_ADVANCE\_SN\_FAILED\_EC (DDSC\_LOG\_BASE + 1484)  
*Failed to advance trust sequence number.*
- #define DDSC\_LOG\_TRUST\_WRITE\_SAMPLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1485)  
*Failed to write sample.*
- #define DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_IN\_PROGRESS\_EC (DDSC\_LOG\_BASE + 1486)  
*Failed to resend handshake because it is already in progress.*
- #define DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_STOP\_FAILED\_EC (DDSC\_LOG\_BASE + 1487)  
*Failed to stop periodic handshake.*
- #define DDSC\_LOG\_TRUST\_RELEASE\_PARTICIPANT\_GENERIC\_MESSAGE\_FAILED\_EC (DDSC\_LOG\_BASE + 1488)  
*Failed to release DomainParticipant trust generic message.*
- #define DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_MESSAGE\_START\_FAILED\_EC (DDSC\_LOG\_BASE + 1489)  
*Failed to start periodic handshake.*
- #define DDSC\_LOG\_TRUST\_UNEXPECTED\_REMOTE\_PARTICIPANT\_STATUS\_EC (DDSC\_LOG\_BASE + 1490)  
*Invalid remote DomainParticipant trust status.*
- #define DDSC\_LOG\_TRUST\_SEND\_PARTICIPANT\_INTERCEPTOR\_STATE\_FAILED\_EC (DDSC\_LOG\_BASE + 1491)  
*Failed to send DomainParticipant interceptor state.*
- #define DDSC\_LOG\_TRUST\_PARTICIPANT\_AUTHENTICATION\_FAILED\_EC (DDSC\_LOG\_BASE + 1492)  
*DomainParticipant authentication failed.*

- #define `DDSC_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC` (`DDSC_LOG_BASE + 1493`)  
*Failed to assert DomainParticipant generic message.*
- #define `DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC` (`DDSC_LOG_BASE + 1494`)  
*Failed to send handshake message.*
- #define `DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_EC` (`DDSC_LOG_BASE + 1495`)  
*Invalid interceptor tokens.*
- #define `DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_DESTINATION_EC` (`DDSC_LOG_BASE + 1496`)  
*Invalid interceptor token destination.*
- #define `DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC` (`DDSC_LOG_BASE + 1497`)  
*Failed to copy endpoint trust attributes.*
- #define `DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE + 1498`)  
*Failed to send tokens to remote DomainParticipant.*
- #define `DDSC_LOG_TRUST_CREATE_LOCAL_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE + 1499`)  
*Failed to create tokens for the remote DataReader.*
- #define `DDSC_LOG_TRUST_CREATE_LOCAL_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE + 1500`)  
*Failed to create tokens for the remote DataWriter.*
- #define `DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE + 1501`)  
*Failed to return interceptor tokens.*
- #define `DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC` (`DDSC_LOG_BASE + 1502`)  
*Failed to return authenticated peer credential token.*
- #define `DDSC_LOG_TRUST_PROCESS_REMOTE_INTERCEPTOR_TOKENS_FAILED_EC` (`DDSC_LOG_BASE + 1503`)  
*Failed to process interceptor tokens.*
- #define `DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE + 1504`)  
*Failed to unregister remote DataWriter.*
- #define `DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE + 1505`)  
*Failed to unregister remote DataReader.*
- #define `DDSC_LOG_TRUST_INVALID_SHARED_SECRET_EC` (`DDSC_LOG_BASE + 1506`)  
*Invalid shared secret.*
- #define `DDSC_LOG_TRUST_PARTICIPANT_UNEXPECTED_AUTHENTICATION_ERROR_EC` (`DDSC_LOG_BASE + 1507`)  
*Unexpected DomainParticipant authentication error.*
- #define `DDSC_LOG_TRUST_PREPARE_LOCAL_DP_TOKENS_EC` (`DDSC_LOG_BASE + 1508`)  
*Failed to prepare local DomainParticipant interceptor tokens.*
- #define `DDSC_LOG_TRUST_ASSERT_PROPERTY_FAILED_EC` (`DDSC_LOG_BASE + 1509`)  
*Failed to assert trust property.*
- #define `DDSC_LOG_TRUST_PARTICIPANT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC` (`DDSC_LOG_BASE + 1510`)  
*Failed to copy DomainParticipant trust attributes.*

- #define DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_INTERNAL\_ATTRIBUTES\_EC (DDSC\_LOG\_BASE + 1511)  
*Invalid DomainParticipant trust attributes.*
- #define DDSC\_LOG\_TRUST\_TRANSFORM\_OUTGOING\_SERIALIZED\_PAYLOAD\_FAILED\_EC (DDSC\_LOG\_BASE + 1512)  
*Failed to encode outgoing serialized payload.*
- #define DDSC\_LOG\_TRUST\_TRANSFORM\_INCOMING\_SERIALIZED\_PAYLOAD\_FAILED\_EC (DDSC\_LOG\_BASE + 1513)  
*Failed to decode incoming serialized payload.*
- #define DDSC\_LOG\_TRUST\_FINALIZE\_AC\_PLUGIN\_PROPERTIES\_FAILED\_EC (DDSC\_LOG\_BASE + 1514)  
*Failed to finalize trust properties.*
- #define DDSC\_LOG\_TRUST\_AFTER\_REMOTE\_PARTICIPANT\_AUTHENTICATED\_FAILED\_EC (DDSC\_LOG\_BASE + 1515)  
*Error after remote trust DomainParticipant is authenticated.*
- #define DDSC\_LOG\_TRUST\_CACHE\_REMOTE\_PARTICIPANT\_SAMPLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1516)  
*Failed to copy remote DomainParticipant sample.*
- #define DDSC\_LOG\_TRUST\_FINALIZE\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC (DDSC\_LOG\_BASE + 1517)  
*Failed to finalize authentication handshake.*
- #define DDSC\_LOG\_TRUST\_FINALIZE\_REMOTE\_PARTICIPANT\_TRUST\_FAILED\_EC (DDSC\_LOG\_BASE + 1518)  
*Failed to finalize remote trust DomainParticipant record.*
- #define DDSC\_LOG\_TRUST\_RESET\_REMOTE\_PARTICIPANT\_TRUST\_FAILED\_EC (DDSC\_LOG\_BASE + 1519)  
*Failed to finalize remote trust DomainParticipant.*
- #define DDSC\_LOG\_TRUST\_ASSERT\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC (DDSC\_LOG\_BASE + 1520)  
*Failed to assert authentication handshake state.*
- #define DDSC\_LOG\_TRUST\_INVALID\_AUTH\_HANDSHAKE\_STATE\_EC (DDSC\_LOG\_BASE + 1521)  
*Invalid authentication handshake state.*
- #define DDSC\_LOG\_TRUST\_SEND\_AUTH\_REQUEST\_FAILED\_EC (DDSC\_LOG\_BASE + 1522)  
*Failed to send authentication request.*
- #define DDSC\_LOG\_TRUST\_GET\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC (DDSC\_LOG\_BASE + 1523)  
*Failed to get authentication handshake state.*
- #define DDSC\_LOG\_TRUST\_SET\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC (DDSC\_LOG\_BASE + 1524)  
*Failed to set authentication handshake state.*
- #define DDSC\_LOG\_TRUST\_REAUTH\_REMOTE\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1525)  
*Failed to reauthenticate a remote DomainParticipant.*
- #define DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_MESSAGE\_FAILED\_EC (DDSC\_LOG\_BASE + 1526)  
*Failed to resend handshake message.*
- #define DDSC\_LOG\_TRUST\_PBUFS\_MAX\_EXCEEDED\_EC (DDSC\_LOG\_BASE + 1527)  
*Maximum number of buffers exceeded.*
- #define DDSC\_LOG\_TRUST\_CREATE\_TRUST\_PLUGIN\_EC (DDSC\_LOG\_BASE + 1528)  
*Failed to create trust plugin.*
- #define DDSC\_LOG\_TRUST\_LOOKUP\_FACTORY\_FAILED\_EC (DDSC\_LOG\_BASE + 1529)  
*Failed to lookup trust plugin factory.*

- #define `DDSC_LOG_NAME_LOOKUP_EC` (`DDSC_LOG_BASE` + 1700)  
*Fully qualified name does not contain substring "::~".*
- #define `DDSC_LOG_ENTITY_NAME_TOO_LONG_EC` (`DDSC_LOG_BASE` + 1701)  
*Failed to get an entity because the specified name exceeds the maximum length of 255 octets (excluding the terminating NUL)*
- #define `DDSC_LOG_MESSAGE_DATA_UNKNOWN_LIVENESS_KIND_EC` (`DDSC_LOG_BASE` + 1800)  
*Unknown inter-participant message liveliness kind received.*
- #define `DDSC_LOG_MESSAGE_DATA_WRONG_LIVENESS_KIND_EC` (`DDSC_LOG_BASE` + 1801)  
*Wrong inter-participant message liveliness kind received.*
- #define `DDSC_LOG_UPDATE_LEASE_DURATION_TIMER_EC` (`DDSC_LOG_BASE` + 1802)  
*Failed to update DataWriter lease duration.*
- #define `DDSC_LOG_PARTMESSAGE_WRITE_SAMPLE_FAILED_EC` (`DDSC_LOG_BASE` + 1803)  
*Failed to write inter-participant liveliness message.*
- #define `DDSC_LOG_SEND_LIVENESS_FAILED_EC` (`DDSC_LOG_BASE` + 1804)  
*Failed to send inter-participant liveliness message.*
- #define `DDSC_LOG_UPDATE_LIVENESS_FAILED_EC` (`DDSC_LOG_BASE` + 1805)  
*Failed to update DataWriter liveliness.*
- #define `DDSC_LOG_REFRESH_LIVENESS_FAILED_EC` (`DDSC_LOG_BASE` + 1806)  
*Failed to refresh endpoint liveliness.*
- #define `DDSC_LOG_TRUST_INCOMPATIBLE_ENDPOINT_TRUST_ATTRIBUTES_EC` (`DDSC_LOG_BASE` + 1807)  
*Two security endpoints are incompatible.*
- #define `DDSC_LOG_FLOW_CONTROLLER_CREATE_EC` (`DDSC_LOG_BASE` + 1900)  
*Failed to create flow controller.*
- #define `DDSC_LOG_FLOW_CONTROLLER_DELETE_EC` (`DDSC_LOG_BASE` + 1901)  
*Failed to delete flow controller.*
- #define `DDSC_LOG_XTYPES_LOANED_SAMPLE_STATE_ERROR_EC` (`DDSC_LOG_BASE` + 2000)  
*Extensible types sample state error.*
- #define `DDSC_LOG_INTERPRETER_SUPPORT_UNEXPECTED_ERROR_EC` (`DDSC_LOG_BASE` + 2001)  
*Interpreter unexpected error.*
- #define `DDSC_LOG_MEMORY_MANAGER_OUT_OF_RESOURCES_EC` (`DDSC_LOG_BASE` + 2002)  
*Memory manager out of resources.*
- #define `DDSC_LOG_MEMORY_MANAGER_NOT_OWNER_EC` (`DDSC_LOG_BASE` + 2003)  
*Memory manager is not the owner of the data passed in.*
- #define `DDSC_LOG_MEMORY_MANAGER_DELETE_EC` (`DDSC_LOG_BASE` + 2004)  
*The wire manager was not able to delete a resource.*
- #define `DDSC_LOG_BUFFER_MAX_EXCEEDED_EC` (`DDSC_LOG_BASE` + 2101)  
*Maximum size of buffer exceeded.*
- #define `DDSC_LOG_INVALID_BUFFER_LENGTH_EC` (`DDSC_LOG_BASE` + 2102)  
*Invalid buffer length.*
- #define `DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC` (`DDSC_LOG_BASE` + 2201)  
*Matching error because Presentation QoS is not compatible.*
- #define `DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC` (`DDSC_LOG_BASE` + 2202)  
*Two endpoints did not match because of incompatible partition QoS.*
- #define `DDSC_LOG_CHECKSUM_REQUIRED_EC` (`DDSC_LOG_BASE` + 2301)  
*A participant requires checksum, but the discovered participant does not send a checksum.*
- #define `DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC` (`DDSC_LOG_BASE` + 2302)

- A participant does not support checksum's sent by a discovered participant.*

  - #define `DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC` (`DDSC_LOG_BASE` + 2303)
- The checksum class id for a custom checksum does match.*

  - #define `DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC` (`DDSC_LOG_BASE` + 2304)
- The computed\_crc\_kind is inconsistent with what is allowed or supported.*

  - #define `DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC` (`DDSC_LOG_BASE` + 2305)
- The allowed\_crc\_mask is inconsistent with what is supported.*

  - #define `DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROP_EC` (`DDSC_LOG_BASE` + 2306)
- The token bucket properties for the flow controller are inconsistent.*

  - #define `DDSC_LOG_LOCK_EC` (`DDSC_LOG_BASE` + 2401)
- Failed to take a lock.*

  - #define `DDSC_LOG_UNLOCK_EC` (`DDSC_LOG_BASE` + 2402)
- Failed to give a lock.*

  - #define `DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC` (`DDSC_LOG_BASE` + 2501)
- Failed to find the filter plugin factory specified in a DomainParticipant's QoS.*

  - #define `DDSC_LOG_ENABLE_WRITER_FILTERING_EC` (`DDSC_LOG_BASE` + 2502)
- A local writer failed to enable writer filtering for a reader.*

  - #define `DDSC_LOG_CREATE_WRITER_FILTER_EC` (`DDSC_LOG_BASE` + 2503)
- A local writer failed to create its writer filter.*

  - #define `DDSC_LOG_EVALUATE_WRITER_FILTER_EC` (`DDSC_LOG_BASE` + 2504)
- A local writer failed to evaluate a sample against the filters of its matched readers.*

  - #define `DDSC_LOG_APPLY_READER_FILTER_EC` (`DDSC_LOG_BASE` + 2505)
- A local writer failed to check if a reader had filtered a sample.*

  - #define `DDSC_LOG_COMPILE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2506)
- Failed to compile the content filter configured in a DataReader's QoS.*

  - #define `DDSC_LOG_CONTENT_FILTERING_NOT_ENABLED_EC` (`DDSC_LOG_BASE` + 2507)
- Reader configured a content filter but content filtering is not enabled.*

  - #define `DDSC_LOG_EVALUATE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2508)
- Failed to evaluate a sample against a reader's content filter.*

  - #define `DDSC_LOG_DESERIALIZE_CONTENT_FILTER_INFO_EC` (`DDSC_LOG_BASE` + 2509)
- Failed to deserialize the content filter info from the in-line QoS of a sample.*

  - #define `DDSC_LOG_CHANGE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2510)
- Failed to change the configured content filter of a DataReader.*

  - #define `DDSC_LOG_UPDATE_REMOTE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2511)
- Failed to update the content filter of a remote subscription.*

  - #define `DDSC_LOG_CREATE_FILTER_PLUGIN_EC` (`DDSC_LOG_BASE` + 2512)
- DomainParticipant failed to create its filter plugin.*

  - #define `DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC` (`DDSC_LOG_BASE` + 2513)
- DomainParticipant failed to finalize its filter plugin.*

  - #define `DDSC_LOG_DESERIALIZE_REMOTE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2514)
- Failed to deserialize the content filter property in the subscription built-in topic data for a remote subscription.*

### 6.21.9.1 Detailed Description

DDS C. ModuleID = 7.

### 6.21.9.2 Macro Definition Documentation

#### DDSC\_LOG\_INVALID\_DURATION\_EC

```
#define DDSC_LOG_INVALID_DURATION_EC (DDSC_LOG_BASE + 1)
```

The specified duration is not valid.

#### DDSC\_LOG\_INVALID\_PARTICIPANT\_NAME\_EC

```
#define DDSC_LOG_INVALID_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 2)
```

An invalid participant name was specified with an unknown GUID.

#### DDSC\_LOG\_INVALID\_PARTICIPANT\_GUID\_PREFIX\_EC

```
#define DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC (DDSC_LOG_BASE + 3)
```

An invalid participant GUID prefix was specified, typically the GUID prefix does not match an already detected participant.

#### DDSC\_LOG\_SYS\_GETTIME\_EC

```
#define DDSC_LOG_SYS_GETTIME_EC (DDSC_LOG_BASE + 4)
```

A call to `OSAPI_System_get_time` failed.

#### DDSC\_LOG\_GET\_NEXT\_OBJECT\_ID\_EC

```
#define DDSC_LOG_GET_NEXT_OBJECT_ID_EC (DDSC_LOG_BASE + 5)
```

Failed to get the next automatically generated object ID for an entity's GUID. This typically means the object id pool has been exhausted.

#### DDSC\_LOG\_SET\_ENTITY\_NAME\_EC

```
#define DDSC_LOG_SET_ENTITY_NAME_EC (DDSC_LOG_BASE + 6)
```

Failed to set name string for a DDS entity in the `DomainParticipantQos` `entity_name` policy.

**DDSC\_LOG\_SYS\_GET\_HOSTNAME\_EC**

```
#define DDSC_LOG_SYS_GET_HOSTNAME_EC (DDSC_LOG_BASE + 7)
```

A call to OSAPI\_System\_get\_hostname failed.

**DDSC\_LOG\_IO\_SNPRINTF\_FAILED\_EC**

```
#define DDSC_LOG_IO_SNPRINTF_FAILED_EC (DDSC_LOG_BASE + 8)
```

A call to OSAPI\_stdio\_snprintf failed. Typically this means the destination buffer was too small.

**DDSC\_LOG\_TOPIC\_FIND\_EC**

```
#define DDSC_LOG_TOPIC_FIND_EC (DDSC_LOG_BASE + 9)
```

Failed to find a topic created by a DomainParticipant.

**DDSC\_LOG\_UNKNOWN\_REMOTE\_PARTICIPANT\_NAME\_EC**

```
#define DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 10)
```

Endpoint discovery failed because the name of the remote participant parent for an endpoint was not found.

**DDSC\_LOG\_UNKNOWN\_REMOTE\_PARTICIPANT\_KEY\_EC**

```
#define DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC (DDSC_LOG_BASE + 11)
```

Endpoint discovery failed because the key of the remote participant parent for an endpoint was not found. Note that the key is logged as 4 integers in host endianness format.

**DDSC\_LOG\_REMOTE\_PARTICIPANT\_KEY\_NOT\_EQUAL\_EC**

```
#define DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC (DDSC_LOG_BASE + 12)
```

Failed endpoint discovery when key does not match the remote participant.

**DDSC\_LOG\_INVALID\_ENDPOINT\_GUID\_EC**

```
#define DDSC_LOG_INVALID_ENDPOINT_GUID_EC (DDSC_LOG_BASE + 13)
```

Failed endpoint discovery due to an invalid or unknown endpoint GUID.

**DDSC\_LOG\_ENDPOINT\_NOT\_CHILD\_OF\_PARTICIPANT\_EC**

```
#define DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC (DDSC_LOG_BASE + 14)
```

Failed endpoint discovery when an endpoint is determined to belong to a different remote participant.

**DDSC\_LOG\_PARTICIPANT\_DOES\_NOT\_EXIST\_EC**

```
#define DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC (DDSC_LOG_BASE + 15)
```

Failed participant discovery because a remote participant that should have been already asserted locally was not found.

**DDSC\_LOG\_REFRESH\_REM\_PARTICIPANT\_EC**

```
#define DDSC_LOG_REFRESH_REM_PARTICIPANT_EC (DDSC_LOG_BASE + 16)
```

Did not find a remote participant when asserting participant liveliness for it. Note that the key is logged as 4 integers in host endianness format.

**DDSC\_LOG\_REFRESH\_REM\_PARTICIPANT\_TIMEOUT\_EC**

```
#define DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC (DDSC_LOG_BASE + 17)
```

Failed to assert participant liveliness to a remote participant.

**DDSC\_LOG\_PARTICIPANT\_LOOKUP\_EC**

```
#define DDSC_LOG_PARTICIPANT_LOOKUP_EC (DDSC_LOG_BASE + 18)
```

Failed to find a remote participant as previously discovered.

**DDSC\_LOG\_REMOVE\_PUBLICATION\_EC**

```
#define DDSC_LOG_REMOVE_PUBLICATION_EC (DDSC_LOG_BASE + 19)
```

Failed to remove resources for a remote publication.

**DDSC\_LOG\_REMOVE\_SUBSCRIPTION\_EC**

```
#define DDSC_LOG_REMOVE_SUBSCRIPTION_EC (DDSC_LOG_BASE + 20)
```

Failed to remove resources for a remote subscription.



**DDSC\_LOG\_FIND\_PUBLICATION\_PARENT\_EC**

```
#define DDSC_LOG_FIND_PUBLICATION_PARENT_EC (DDSC_LOG_BASE + 21)
```

Cannot determine the participant of a remote publication.

**DDSC\_LOG\_FIND\_SUBSCRIPTION\_PARENT\_EC**

```
#define DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC (DDSC_LOG_BASE + 22)
```

Cannot determine the participant of a remote subscription.

**DDSC\_LOG\_MAX\_PARTICIPANT\_ID\_REACHED\_EC**

```
#define DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC (DDSC_LOG_BASE + 23)
```

Failed to create DomainParticipant due to running out of participant IDs. This is typically caused by all UDP ports being used.

**DDSC\_LOG\_RESERVE\_LOCATORS\_EC**

```
#define DDSC_LOG_RESERVE_LOCATORS_EC (DDSC_LOG_BASE + 24)
```

Failed to reserve endpoint locators.

**DDSC\_LOG\_TIMER\_CREATE\_TIMEOUT\_EC**

```
#define DDSC_LOG_TIMER_CREATE_TIMEOUT_EC (DDSC_LOG_BASE + 25)
```

Failed to create a timeout.

**DDSC\_LOG\_ILLEGAL\_OBJECTID\_EC**

```
#define DDSC_LOG_ILLEGAL_OBJECTID_EC (DDSC_LOG_BASE + 26)
```

Illegal object id specified.

Manually specified object id must be in the range [0,0xfffff]

**DDSC\_LOG\_TOPIC\_NARROW\_EC**

```
#define DDSC_LOG_TOPIC_NARROW_EC (DDSC_LOG_BASE + 27)
```

Failed to narrow a TopicDescription to the named Topic.

**DDSC\_LOG\_FAILED\_UPDATE\_STATUS\_CONDITION\_EC**

```
#define DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC (DDSC_LOG_BASE + 28)
```

Failed to update state of StatusCondition.

An error occurred when trying to update the state (i.e. the trigger value) of a StatusCondition.

**DDSC\_LOG\_WS\_REMOVE\_COND\_REFERENCE\_EC**

```
#define DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC (DDSC_LOG_BASE + 29)
```

Failed to remove a condition reference from a waitset.

**DDSC\_LOG\_WS\_ADD\_COND\_REFERENCE\_EC**

```
#define DDSC_LOG_WS_ADD_COND_REFERENCE_EC (DDSC_LOG_BASE + 30)
```

Failed to add a condition reference to a waitset.

**DDSC\_LOG\_RELEASE\_META\_MC\_EC**

```
#define DDSC_LOG_RELEASE_META_MC_EC (DDSC_LOG_BASE + 31)
```

Failed to release resources for multicast discovery locators.

**DDSC\_LOG\_RELEASE\_META\_UC\_EC**

```
#define DDSC_LOG_RELEASE_META_UC_EC (DDSC_LOG_BASE + 32)
```

Failed to release resources for unicast discovery locators.

**DDSC\_LOG\_RELEASE\_USER\_MC\_EC**

```
#define DDSC_LOG_RELEASE_USER_MC_EC (DDSC_LOG_BASE + 33)
```

Failed to release resources for multicast user locators.

**DDSC\_LOG\_RELEASE\_USER\_UC\_EC**

```
#define DDSC_LOG_RELEASE_USER_UC_EC (DDSC_LOG_BASE + 34)
```

Failed to release resources for unicast user locators.

**DDSC\_LOG\_INVALID\_DOMAINID\_EC**

```
#define DDSC_LOG_INVALID_DOMAINID_EC (DDSC_LOG_BASE + 35)
```

The domain ID specified exceeds what is allowed based on the parameters specified in `dp_qos.protocol.rtps_well_known_ports`.

**DDSC\_LOG\_ON\_TYPE\_REGISTERED\_FAILURE\_EC**

```
#define DDSC_LOG_ON_TYPE_REGISTERED_FAILURE_EC (DDSC_LOG_BASE + 36)
```

Error occurred during the `on_type_registered` call back.

**DDSC\_LOG\_ON\_TYPE\_UNREGISTERED\_FAILURE\_EC**

```
#define DDSC_LOG_ON_TYPE_UNREGISTERED_FAILURE_EC (DDSC_LOG_BASE + 37)
```

Error occurred during the `on_type_unregistered` call back.

**DDSC\_LOG\_GET\_SERIALIZED\_KEY\_SIZE\_EC**

```
#define DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC (DDSC_LOG_BASE + 38)
```

Error occurred during the `on_type_unregistered` call back.

**DDSC\_LOG\_MIN\_DISCOVERY\_MTU\_EC**

```
#define DDSC_LOG_MIN_DISCOVERY_MTU_EC (DDSC_LOG_BASE + 39)
```

Minimum MTU across all Discovery transports must be greater or equal to a packet containing serialized ParticipantBuiltinTopicData.

**DDSC\_LOG\_MIN\_MTU\_SIZE\_EC**

```
#define DDSC_LOG_MIN_MTU_SIZE_EC (DDSC_LOG_BASE + 40)
```

MTU for a transport set lower or equal to Protocol Overhead of 448 bytes.

**DDSC\_LOG\_GET\_MTU\_EC**

```
#define DDSC_LOG_GET_MTU_EC (DDSC_LOG_BASE + 40)
```

Error getting the mtu from the route resolver.

**DDSC\_LOG\_GET\_MTU\_NO\_ROUTES\_EC**

```
#define DDSC_LOG_GET_MTU_NO_ROUTES_EC (DDSC_LOG_BASE + 41)
```

MTU could not be found because of no routes.

**DDSC\_LOG\_RESOLVE\_LOCATORS\_PRIORITY\_EC**

```
#define DDSC_LOG_RESOLVE_LOCATORS_PRIORITY_EC (DDSC_LOG_BASE + 42)
```

Failed to resolve locators based on priority.

**DDSC\_LOG\_REMOTE\_PUBLICATION\_DATA\_CHANGED\_EC**

```
#define DDSC_LOG_REMOTE_PUBLICATION_DATA_CHANGED_EC (DDSC_LOG_BASE + 43)
```

The publication builtin topic data for a remote publication changed after the remote publication was asserted.

No dynamic changes are supported to the publication builtin topic data after the remote publication is asserted. The previous data will continue to be used.

**DDSC\_LOG\_REMOTE\_SUBSCRIPTION\_DATA\_CHANGED\_EC**

```
#define DDSC_LOG_REMOTE_SUBSCRIPTION_DATA_CHANGED_EC (DDSC_LOG_BASE + 44)
```

Fields in subscription builtin topic data for a remote subscription which are not supported to dynamically change were updated after the remote subscription was asserted.

The only supported dynamic changes to the subscription builtin topic data are the configured content filter on the remote subscription. Any other changes are not supported and will be ignored.

**DDSC\_LOG\_DATABASE\_CREATE\_EC**

```
#define DDSC_LOG_DATABASE_CREATE_EC (DDSC_LOG_BASE + 100)
```

Failed to create database.

**DDSC\_LOG\_DATABASE\_DELETE\_EC**

```
#define DDSC_LOG_DATABASE_DELETE_EC (DDSC_LOG_BASE + 101)
```

Failed to delete database.

**DDSC\_LOG\_TABLE\_CREATE\_EC**

```
#define DDSC_LOG_TABLE_CREATE_EC (DDSC_LOG_BASE + 102)
```

Failed to create database table of the specified name.

**DDSC\_LOG\_TABLE\_INUSE\_EC**

```
#define DDSC_LOG_TABLE_INUSE_EC (DDSC_LOG_BASE + 103)
```

Failed to delete a database table because it is not empty.

**DDSC\_LOG\_TABLE\_DELETE\_EC**

```
#define DDSC_LOG_TABLE_DELETE_EC (DDSC_LOG_BASE + 104)
```

Failed to delete a database table.

**DDSC\_LOG\_TABLE\_SELECT\_EC**

```
#define DDSC_LOG_TABLE_SELECT_EC (DDSC_LOG_BASE + 105)
```

A selection operation failed on the specified database table.

**DDSC\_LOG\_CREATE\_INDEX\_EC**

```
#define DDSC_LOG_CREATE_INDEX_EC (DDSC_LOG_BASE + 106)
```

Failed to create an index on a database table.

**DDSC\_LOG\_DELETE\_INDEX\_EC**

```
#define DDSC_LOG_DELETE_INDEX_EC (DDSC_LOG_BASE + 107)
```

Failed to delete an index on a database table.

**DDSC\_LOG\_DB\_CURSOR\_INVALIDATED\_EC**

```
#define DDSC_LOG_DB_CURSOR_INVALIDATED_EC (DDSC_LOG_BASE + 108)
```

A database table cursor was invalidated while in use.

**DDSC\_LOG\_SUBSCRIPTION\_RECORD**

```
#define DDSC_LOG_SUBSCRIPTION_RECORD 1
```

Database record for a Subscription.

**DDSC\_LOG\_PUBLICATION\_RECORD**

```
#define DDSC_LOG_PUBLICATION_RECORD 2
```

Database record for a Publication.

**DDSC\_LOG\_PARTICIPANT\_RECORD**

```
#define DDSC_LOG_PARTICIPANT_RECORD 3
```

Database record for a Remote Participant.

**DDSC\_LOG\_TOPIC\_RECORD**

```
#define DDSC_LOG_TOPIC_RECORD 4
```

Database record for a Topic.

**DDSC\_LOG\_DATAWRITER\_RECORD**

```
#define DDSC_LOG_DATAWRITER_RECORD 5
```

Database record for a DataWriter.

**DDSC\_LOG\_DATAREADER\_RECORD**

```
#define DDSC_LOG_DATAREADER_RECORD 6
```

Database record for a DataReader.

**DDSC\_LOG\_PUBLISHER\_RECORD**

```
#define DDSC_LOG_PUBLISHER_RECORD 7
```

Database record for a Publisher.

**DDSC\_LOG\_SUBSCRIBER\_RECORD**

```
#define DDSC_LOG_SUBSCRIBER_RECORD 8
```

Database record for a Subscriber.

**DDSC\_LOG\_ROUTE\_RECORD**

```
#define DDSC_LOG_ROUTE_RECORD 9
```

Database record for a route record.

**DDSC\_LOG\_BIND\_RECORD**

```
#define DDSC_LOG_BIND_RECORD 10
```

Database record for a bind record.

**DDSC\_LOG\_TYPE\_RECORD**

```
#define DDSC_LOG_TYPE_RECORD 11
```

Database record for a type plugin.

**DDSC\_LOG\_REMOTE\_ENDPOINT\_TRUST\_STATE\_RECORD**

```
#define DDSC_LOG_REMOTE_ENDPOINT_TRUST_STATE_RECORD 12
```

Database record for a remote trust endpoint.

**DDSC\_LOG\_MATCHED\_DW\_RECORD**

```
#define DDSC_LOG_MATCHED_DW_RECORD 13
```

Database record for a matched remote data writer.

**DDSC\_LOG\_STRING\_RECORD**

```
#define DDSC_LOG_STRING_RECORD 14
```

Database record for a string property.

**DDSC\_LOG\_RECORD\_CREATE\_EC**

```
#define DDSC_LOG_RECORD_CREATE_EC (DDSC_LOG_BASE + 109)
```

Failed to create a database record of the specified kind.

The creation of an internal database record failed. The failure may have been caused by insufficient resources based on the record kind. The following resource-limits may apply:

DomainParticipantQos.resource\_limit.local\_publisher\_allocation limits the number of DDS Publishers.

DomainParticipantQos.resource\_limit.local\_subscriber\_allocation limits the number of DDS Subscribers.

DomainParticipantQos.resource\_limit.local\_topic\_allocation limits the number of DDS Topics.

DomainParticipantQos.resource\_limits.local\_reader\_allocation limits the number of DDS DataReader records.

DomainParticipantQos.resource\_limits.local\_writer\_allocation limits the number of DDS DataWriter records.

DomainParticipantQos.resource\_limits.remote\_writer\_allocation limits the number of remote publication records.

DomainParticipantQos.resource\_limits.remote\_reader\_allocation limits the number of remote subscription records.

DomainParticipantQos.resource\_limits.remote\_participant\_allocation limits the number of remote participant records.

DataWriterQos.writer\_resource\_limits.max\_remote\_readers limits the number of DDS DataReaders a DDS DataWriter can communicate with.

DataReaderQos.reader\_resource\_limits.max\_remote\_writers limits the number of DDS DataWriters a DDS DataReader can communicate with.

**DDSC\_LOG\_RECORD\_DELETE\_EC**

```
#define DDSC_LOG_RECORD_DELETE_EC (DDSC_LOG_BASE + 110)
```

Failed to delete a database record of the specified kind.

**DDSC\_LOG\_RECORD\_INSERT\_EC**

```
#define DDSC_LOG_RECORD_INSERT_EC (DDSC_LOG_BASE + 111)
```

Failed to insert a database record of the specified kind.

**DDSC\_LOG\_RECORD\_ERROR\_EC**

```
#define DDSC_LOG_RECORD_ERROR_EC (DDSC_LOG_BASE + 112)
```

Unknown error for database record of the specified kind.



**DDSC\_LOG\_RECORD\_EXISTS\_EC**

```
#define DDSC_LOG_RECORD_EXISTS_EC (DDSC_LOG_BASE + 113)
```

A database record of the specified kind already exists.

**DDSC\_LOG\_RECORD\_LOOKUP\_EC**

```
#define DDSC_LOG_RECORD_LOOKUP_EC (DDSC_LOG_BASE + 114)
```

A lookup of a database record of the specified kind failed.

**DDSC\_LOG\_RECORD\_NOT\_EXISTS\_EC**

```
#define DDSC_LOG_RECORD_NOT_EXISTS_EC (DDSC_LOG_BASE + 115)
```

A database record of the specified kind does not exist.

**DDSC\_LOG\_RECORD\_SELECT\_EC**

```
#define DDSC_LOG_RECORD_SELECT_EC (DDSC_LOG_BASE + 116)
```

A database select on the specified record kind failed.

**DDSC\_LOG\_RECORD\_REMOVE\_EC**

```
#define DDSC_LOG_RECORD_REMOVE_EC (DDSC_LOG_BASE + 117)
```

Removal a database record of the specified kind failed.

**DDSC\_LOG\_RECORD\_INITIALIZE\_EC**

```
#define DDSC_LOG_RECORD_INITIALIZE_EC (DDSC_LOG_BASE + 118)
```

A database record of the specified kind could not be initialized.

**DDSC\_LOG\_RECORD\_FINALIZE\_EC**

```
#define DDSC_LOG_RECORD_FINALIZE_EC (DDSC_LOG_BASE + 119)
```

A database record of the specified kind could not be finalized.

**DDSC\_LOG\_RECORD\_RESET\_EC**

```
#define DDSC_LOG_RECORD_RESET_EC (DDSC_LOG_BASE + 120)
```

A database record of the specified kind could not be reset.

A database record is "reset" when all its attributes are reverted to a previous state (typically the one it had at creation/insertion), without removing it from the database.

The actions performed by the "reset" depend on the type of record.

**DDSC\_LOG\_PUBLICATIONDATA\_OBJECT**

```
#define DDSC_LOG_PUBLICATIONDATA_OBJECT 1
```

DDS PublicationBuiltinTopicData object.

**DDSC\_LOG\_SUBSCRIPTIONDATA\_OBJECT**

```
#define DDSC_LOG_SUBSCRIPTIONDATA_OBJECT 2
```

DDS SubscriptionBuiltinTopicData object.

**DDSC\_LOG\_PARTICIPANTDATA\_OBJECT**

```
#define DDSC_LOG_PARTICIPANTDATA_OBJECT 3
```

DDS ParticipantBuiltinTopicData object.

**DDSC\_LOG\_DATAREADERQOS\_OBJECT**

```
#define DDSC_LOG_DATAREADERQOS_OBJECT 4
```

DDS DataReaderQos object.

**DDSC\_LOG\_DATAWRITERQOS\_OBJECT**

```
#define DDSC_LOG_DATAWRITERQOS_OBJECT 5
```

DDS DataWriterQos object.

**DDSC\_LOG\_TOPICQOS\_OBJECT**

```
#define DDSC_LOG_TOPICQOS_OBJECT 6
```

DDS TopicQos object.

**DDSC\_LOG\_PUBLISHERQOS\_OBJECT**

```
#define DDSC_LOG_PUBLISHERQOS_OBJECT 7
```

DDS PublisherQos object.

**DDSC\_LOG\_SUBSCRIBERQOS\_OBJECT**

```
#define DDSC_LOG_SUBSCRIBERQOS_OBJECT 8
```

DDS SubscriberQos object.

**DDSC\_LOG\_PARTICIPANTQOS\_OBJECT**

```
#define DDSC_LOG_PARTICIPANTQOS_OBJECT 9
```

DDS DomainParticipantQos object.

**DDSC\_LOG\_ENTITY\_OBJECT**

```
#define DDSC_LOG_ENTITY_OBJECT 10
```

DDS Entity object.

**DDSC\_LOG\_PARTICIPANTFACTORYQOS\_OBJECT**

```
#define DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT 11
```

DDS DomainParticipantFactoryQos object.

**DDSC\_LOG\_TYPE\_OBJECT**

```
#define DDSC_LOG_TYPE_OBJECT 12
```

A DDS Type object.

**DDSC\_LOG\_BINDRESOLVER\_OBJECT**

```
#define DDSC_LOG_BINDRESOLVER_OBJECT 13
```

A NETIO BindResolver object.

**DDSC\_LOG\_ROUTERESOLVER\_OBJECT**

```
#define DDSC_LOG_ROUTERESOLVER_OBJECT 14
```

A NETIO RouteResolver object.

**DDSC\_LOG\_ADDRESSRESOLVER\_OBJECT**

```
#define DDSC_LOG_ADDRESSRESOLVER_OBJECT 15
```

A NETIO AddressResolver object.

**DDSC\_LOG\_DATAWRITERIO\_OBJECT**

```
#define DDSC_LOG_DATAWRITERIO_OBJECT 16
```

A NETIO DataWriterInterface object.

**DDSC\_LOG\_DATAREADERIO\_OBJECT**

```
#define DDSC_LOG_DATAREADERIO_OBJECT 17
```

A NETIO DataReaderInterface object.

**DDSC\_LOG\_LOG\_OBJECT**

```
#define DDSC_LOG_LOG_OBJECT 18
```

A OSAPI Log object.

**DDSC\_LOG\_SYSTEM\_OBJECT**

```
#define DDSC_LOG_SYSTEM_OBJECT 19
```

A OSAPI system object.

**DDSC\_LOG\_MUTEX\_OBJECT**

```
#define DDSC_LOG_MUTEX_OBJECT 20
```

A OSAPI mutex object.

**DDSC\_LOG\_CONDREF\_OBJECT**

```
#define DDSC_LOG_CONDREF_OBJECT 21
```

A DDS Waitset condition reference object.

**DDSC\_LOG\_WSREF\_OBJECT**

```
#define DDSC_LOG_WSREF_OBJECT 22
```

A DDS Condition waitset reference object.

**DDSC\_LOG\_MD5STREAM\_OBJECT**

```
#define DDSC_LOG_MD5STREAM_OBJECT 23
```

A MD5 stream object.

**DDSC\_LOG\_CONDITION\_OBJECT**

```
#define DDSC_LOG_CONDITION_OBJECT 24
```

A DDS Condition object.

**DDSC\_LOG\_RT\_OBJECT**

```
#define DDSC_LOG_RT_OBJECT 25
```

A RT Condition object.

**DDSC\_LOG\_PARTICIPANT\_POOL\_OBJECT**

```
#define DDSC_LOG_PARTICIPANT_POOL_OBJECT 26
```

A Participant pool object.

**DDSC\_LOG\_TIMER\_OBJECT**

```
#define DDSC_LOG_TIMER_OBJECT 27
```

A OSAPI\_Timer object.

**DDSC\_LOG\_TIMEROUT\_OBJECT**

```
#define DDSC_LOG_TIMEROUT_OBJECT 28
```

A OSAPI\_Timer timeout object.

**DDSC\_LOG\_TOPIC\_OBJECT**

```
#define DDSC_LOG_TOPIC_OBJECT 29
```

A DDS Topic object.

**DDSC\_LOG\_WAITSET\_OBJECT**

```
#define DDSC_LOG_WAITSET_OBJECT 30
```

A DDS WaitSet object.

**DDSC\_LOG\_UUID\_OBJECT**

```
#define DDSC_LOG_UUID_OBJECT 31
```

A UUID object.

**DDSC\_LOG\_MEMPOOL\_OBJECT**

```
#define DDSC_LOG_MEMPOOL_OBJECT 32
```

A REDA\_MemPool object.

**DDSC\_LOG\_PARTICIPANT\_PROPERTIES**

```
#define DDSC_LOG_PARTICIPANT_PROPERTIES 33
```

A DomainParticipant's string properties sequence.

**DDSC\_LOG\_PARTICIPANT\_BINARY\_PROPERTIES**

```
#define DDSC_LOG_PARTICIPANT_BINARY_PROPERTIES 34
```

A DomainParticipant's binary properties sequence.

**DDSC\_LOG\_PARTICIPANT\_BUILTIN\_PUBLISHER**

```
#define DDSC_LOG_PARTICIPANT_BUILTIN_PUBLISHER 35
```

A DomainParticipant's built-in Publisher.

**DDSC\_LOG\_PARTICIPANT\_BUILTIN\_SUBSCRIBER**

```
#define DDSC_LOG_PARTICIPANT_BUILTIN_SUBSCRIBER 36
```

A DomainParticipant's built-in Subscriber.

**DDSC\_LOG\_PARTICIPANT\_GENERIC\_MESSAGE\_OBJECT**

```
#define DDSC_LOG_PARTICIPANT_GENERIC_MESSAGE_OBJECT 37
```

A built-in Inter-Participant Channel data type object.

**DDSC\_LOG\_DATA\_HOLDER\_SEQ\_OBJECT**

```
#define DDSC_LOG_DATA_HOLDER_SEQ_OBJECT 38
```

A Trust data holder sequence object.

**DDSC\_LOG\_PROPERTYQOSPOLICY\_OBJECT**

```
#define DDSC_LOG_PROPERTYQOSPOLICY_OBJECT 39
```

A Property QoS policy object.

**DDSC\_LOG\_STRING\_OBJECT**

```
#define DDSC_LOG_STRING_OBJECT 40
```

A String object.

**DDSC\_LOG\_AUTHPOOL\_OBJECT**

```
#define DDSC_LOG_AUTHPOOL_OBJECT 41
```

An authentication pool object.

**DDSC\_LOG\_BUFFER\_OBJECT**

```
#define DDSC_LOG_BUFFER_OBJECT 42
```

A Log buffer object.

**DDSC\_LOG\_STRINGMANAGER\_OBJECT**

```
#define DDSC_LOG_STRINGMANAGER_OBJECT 43
```

A String Manager object.

**DDSC\_LOG\_OBJECT\_INITIALIZE\_EC**

```
#define DDSC_LOG_OBJECT_INITIALIZE_EC (DDSC_LOG_BASE + 200)
```

Out of resources to initialize object of the specified kind.

**DDSC\_LOG\_OBJECT\_ALLOCATE\_EC**

```
#define DDSC_LOG_OBJECT_ALLOCATE_EC (DDSC_LOG_BASE + 201)
```

Out of resources to allocate an object of the specified kind.

**DDSC\_LOG\_OBJECT\_FINALIZE\_EC**

```
#define DDSC_LOG_OBJECT_FINALIZE_EC (DDSC_LOG_BASE + 202)
```

Failed to finalize object of specified kind.

**DDSC\_LOG\_OBJECT\_DELETE\_EC**

```
#define DDSC_LOG_OBJECT_DELETE_EC (DDSC_LOG_BASE + 203)
```

Failed to delete object of specified kind.



**DDSC\_LOG\_OBJECT\_COPY\_EC**

```
#define DDSC_LOG_OBJECT_COPY_EC (DDSC_LOG_BASE + 204)
```

Failed to copy object of specified kind.

**DDSC\_LOG\_OBJECT\_REFCOUNT\_EC**

```
#define DDSC_LOG_OBJECT_REFCOUNT_EC (DDSC_LOG_BASE + 205)
```

Failed to delete/finalize an object because other objects are referencing it.

**DDSC\_LOG\_OBJECT\_GET\_PROPERTY\_EC**

```
#define DDSC_LOG_OBJECT_GET_PROPERTY_EC (DDSC_LOG_BASE + 206)
```

Failed to get the object properties.

**DDSC\_LOG\_OBJECT\_SET\_PROPERTY\_EC**

```
#define DDSC_LOG_OBJECT_SET_PROPERTY_EC (DDSC_LOG_BASE + 207)
```

Failed to set the object properties.

**DDSC\_LOG\_OBJECT\_EMPTY\_EC**

```
#define DDSC_LOG_OBJECT_EMPTY_EC (DDSC_LOG_BASE + 208)
```

An object is empty, typically applies only to buffer-pool objects.

**DDSC\_LOG\_OBJECT\_INUSECOUNT\_EC**

```
#define DDSC_LOG_OBJECT_INUSECOUNT_EC (DDSC_LOG_BASE + 209)
```

Failed to delete/finalize an object because other objects are using it.

**DDSC\_LOG\_OBJECT\_CREATE\_EC**

```
#define DDSC_LOG_OBJECT_CREATE_EC (DDSC_LOG_BASE + 210)
```

Failed to create an object.

**DDSC\_LOG\_NETMASK\_SEQUENCE**

```
#define DDSC_LOG_NETMASK_SEQUENCE 1
```

A NETIO Netmask sequence.

**DDSC\_LOG\_ROUTE\_SEQUENCE**

```
#define DDSC_LOG_ROUTE_SEQUENCE 2
```

A NETIO Route sequence.

**DDSC\_LOG\_RESERVED\_SEQUENCE**

```
#define DDSC_LOG_RESERVED_SEQUENCE 3
```

A NETIO Reserved address sequence.

**DDSC\_LOG\_ENABLED\_USER\_TRANSPORT\_SEQUENCE**

```
#define DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE 4
```

A DDS sequence of enabled user NETIO transports.

**DDSC\_LOG\_ENABLED\_TRANSPORT\_SEQUENCE**

```
#define DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE 5
```

A DDS sequence of enabled transports.

**DDSC\_LOG\_ENABLED\_DISCOVERY\_TRANSPORT\_SEQUENCE**

```
#define DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE 6
```

A DDS sequence of enabled discovery transports.

**DDSC\_LOG\_INITIAL\_PEER\_SEQUENCE**

```
#define DDSC_LOG_INITIAL_PEER_SEQUENCE 7
```

A DDS sequence of initial peers.

**DDSC\_LOG\_DESTINATION\_SEQUENCE**

```
#define DDSC_LOG_DESTINATION_SEQUENCE 8
```

A NETIO sequence of destinations to send to.

**DDSC\_LOG\_METAUNICAST\_SEQUENCE**

```
#define DDSC_LOG_METAUNICAST_SEQUENCE 9
```

A DDS sequence of meta-traffic unicast locators.

**DDSC\_LOG\_METAMULTICAST\_SEQUENCE**

```
#define DDSC_LOG_METAMULTICAST_SEQUENCE 10
```

A DDS sequence of meta-traffic multicast locators.

**DDSC\_LOG\_USERUNICAST\_SEQUENCE**

```
#define DDSC_LOG_USERUNICAST_SEQUENCE 11
```

A DDS sequence of user-traffic unicast locators.

**DDSC\_LOG\_USERMULTICAST\_SEQUENCE**

```
#define DDSC_LOG_USERMULTICAST_SEQUENCE 12
```

A DDS sequence of user-traffic multicast locators.

**DDSC\_LOG\_READTAKE\_SEQUENCE**

```
#define DDSC_LOG_READTAKE_SEQUENCE 13
```

A DDS sequence used in a read/take call.

**DDSC\_LOG\_DATAHOLDER\_SEQUENCE**

```
#define DDSC_LOG_DATAHOLDER_SEQUENCE 14
```

A DDS sequence of DataHolder objects.

**DDSC\_LOG\_COOKIE\_PAYLOAD\_SEQUENCE**

```
#define DDSC_LOG_COOKIE_PAYLOAD_SEQUENCE 15
```

A cookie octet sequence.

**DDSC\_LOG\_REPRESENTATION\_ID\_SEQUENCE**

```
#define DDSC_LOG_REPRESENTATION_ID_SEQUENCE 16
```

a sequence for data representation IDs

**DDSC\_LOG\_SEQUENCE\_SETMAX\_EC**

```
#define DDSC_LOG_SEQUENCE_SETMAX_EC (DDSC_LOG_BASE + 300)
```

Failed to set the maximum length of a sequence of the specified kind.

**DDSC\_LOG\_SEQUENCE\_SETLENGTH\_EC**

```
#define DDSC_LOG_SEQUENCE_SETLENGTH_EC (DDSC_LOG_BASE + 301)
```

Failed to set the length of a sequence of the specified kind.

**DDSC\_LOG\_SEQUENCE\_GETREF\_EC**

```
#define DDSC_LOG_SEQUENCE_GETREF_EC (DDSC_LOG_BASE + 302)
```

Failed to get a reference at the specified index for a sequence of the specified kind.

**DDSC\_LOG\_SEQUENCE\_INITIALIZE\_EC**

```
#define DDSC_LOG_SEQUENCE_INITIALIZE_EC (DDSC_LOG_BASE + 303)
```

Failed to initialize a sequence of the specified kind.

**DDSC\_LOG\_SEQUENCE\_FINALIZE\_EC**

```
#define DDSC_LOG_SEQUENCE_FINALIZE_EC (DDSC_LOG_BASE + 304)
```

Failed to finalize a sequence of the specified kind.

**DDSC\_LOG\_SEQUENCE\_COPY\_EC**

```
#define DDSC_LOG_SEQUENCE_COPY_EC (DDSC_LOG_BASE + 305)
```

Failed to copy a sequence of the specified kind.

**DDSC\_LOG\_SEQUENCE\_INVALID\_EC**

```
#define DDSC_LOG_SEQUENCE_INVALID_EC (DDSC_LOG_BASE + 306)
```

The sequence of the specified kind was invalid in the context it is used.

**DDSC\_LOG\_DISCOVERY\_COMPONENT**

```
#define DDSC_LOG_DISCOVERY_COMPONENT 1
```

A component of discovery kind.

**DDSC\_LOG\_RTPS\_COMPONENT**

```
#define DDSC_LOG_RTPS_COMPONENT 2
```

A component of RTPS kind.

**DDSC\_LOG\_READERHISTORY\_COMPONENT**

```
#define DDSC_LOG_READERHISTORY_COMPONENT 3
```

A component of reader-history kind.

**DDSC\_LOG\_WRITERHISTORY\_COMPONENT**

```
#define DDSC_LOG_WRITERHISTORY_COMPONENT 4
```

A component of writer-history kind.

**DDSC\_LOG\_DATAREADERIO\_COMPONENT**

```
#define DDSC_LOG_DATAREADERIO_COMPONENT 5
```

A component of DataReaderInterface kind.

**DDSC\_LOG\_DATAWRITERIO\_COMPONENT**

```
#define DDSC_LOG_DATAWRITERIO_COMPONENT 6
```

A component of DataWriterInterface kind.

**DDSC\_LOG\_NETIO\_COMPONENT**

```
#define DDSC_LOG_NETIO_COMPONENT 7
```

A component of NETIO kind.

**DDSC\_LOG\_TRANSPORT\_COMPONENT**

```
#define DDSC_LOG_TRANSPORT_COMPONENT 8
```

A component of transport kind.

**DDSC\_LOG\_APPGEN\_COMPONENT**

```
#define DDSC_LOG_APPGEN_COMPONENT 9
```

A component of application generation kind.

**DDSC\_LOG\_COMPONENT\_LOOKUP\_EC**

```
#define DDSC_LOG_COMPONENT_LOOKUP_EC (DDSC_LOG_BASE + 400)
```

Did not find a component factory with the given name in the registry.

**DDSC\_LOG\_COMPONENT\_CREATE\_EC**

```
#define DDSC_LOG_COMPONENT_CREATE_EC (DDSC_LOG_BASE + 401)
```

Could not create a component of the specified kind using the specified factory.

**DDSC\_LOG\_COMPONENT\_DELETE\_EC**

```
#define DDSC_LOG_COMPONENT_DELETE_EC (DDSC_LOG_BASE + 402)
```

Could not delete a component of the specified kind using the specified factory.

## DDSC\_LOG\_HISTORY\_RESOURCE

```
#define DDSC_LOG_HISTORY_RESOURCE 1
```

Exceeded resource limits for writer history queue.

A DataWriter failed to get a queue entry for a new sample it is attempting to send, due to exceeding a limit:

- When the sample is for new instance: DataWriterQos.resource\_limits.max\_instances may have been exceeded
- When the sample is for an existing instance, and History kind is KEEP\_ALL: DataWriterQos.resource\_limits.max\_samples\_per\_instance may have been exceeded.
- The limit on the total number of samples in the queue: DataWriterQos.resource\_limits.max\_samples, may have been exceeded.

## DDSC\_LOG\_SAMPLE\_RESOURCES

```
#define DDSC_LOG_SAMPLE_RESOURCES 2
```

Failed to get resource for new sample due to resource limit.

A DataWriter failed to get a buffer for a new sample being written because the limit DataWriterQos.resource\_limits.max\_samples was exceeded.

A DataReader failed in getting a buffer for a newly received sample because DataReaderQos.resource\_limits.max\_samples was exceeded.

## DDSC\_LOG\_PARTICIPANT\_RESOURCES

```
#define DDSC_LOG_PARTICIPANT_RESOURCES 3
```

Failed to allocate a new participant.

The DomainParticipantFactory failed to get a buffer for a new DomainParticipant because DomainParticipantFactoryQos.system\_resource.max\_participants because was exceeded.

## DDSC\_LOG\_RESOURCE\_EXCEEDED\_EC

```
#define DDSC_LOG_RESOURCE_EXCEEDED_EC (DDSC_LOG_BASE + 500)
```

Could not allocate a resource of the specified kind.

## DDSC\_LOG\_TOPIC\_QOS

```
#define DDSC_LOG_TOPIC_QOS 1
```

The TopicQos kind.

**DDSC\_LOG\_DATAREADER\_QOS**

```
#define DDSC_LOG_DATAREADER_QOS 2
```

The DataReaderQos kind.

**DDSC\_LOG\_DATAWRITER\_QOS**

```
#define DDSC_LOG_DATAWRITER_QOS 3
```

The DataWriterQos kind.

**DDSC\_LOG\_SUBSCRIBER\_QOS**

```
#define DDSC_LOG_SUBSCRIBER_QOS 4
```

The SubscriberQos kind.

**DDSC\_LOG\_PUBLISHER\_QOS**

```
#define DDSC_LOG_PUBLISHER_QOS 5
```

The PublisherQos kind.

**DDSC\_LOG\_PARTICIPANT\_QOS**

```
#define DDSC_LOG_PARTICIPANT_QOS 6
```

The DomainParticipantQos kind.

**DDSC\_LOG\_PARTICIPANTFACTORY\_QOS**

```
#define DDSC_LOG_PARTICIPANTFACTORY_QOS 7
```

The DomainParticipantFactoryQos kind.

**DDSC\_LOG\_DEFAULTTOPIC\_QOS**

```
#define DDSC_LOG_DEFAULTTOPIC_QOS 8
```

The default TopicQos kind.



**DDSC\_LOG\_DEFAULTDATAREADER\_QOS**

```
#define DDSC_LOG_DEFAULTDATAREADER_QOS 9
```

The default DataReaderQos kind.

**DDSC\_LOG\_DEFAULTDATAWRITER\_QOS**

```
#define DDSC_LOG_DEFAULTDATAWRITER_QOS 10
```

The default DataWriterQos kind.

**DDSC\_LOG\_DEFAULTSUBSCRIBER\_QOS**

```
#define DDSC_LOG_DEFAULTSUBSCRIBER_QOS 11
```

The default SubscriberQos kind.

**DDSC\_LOG\_DEFAULTPUBLISHER\_QOS**

```
#define DDSC_LOG_DEFAULTPUBLISHER_QOS 12
```

The default PublisherQos kind.

**DDSC\_LOG\_DEFAULTPARTICIPANT\_QOS**

```
#define DDSC_LOG_DEFAULTPARTICIPANT_QOS 13
```

The default PublisherQos kind.

**DDSC\_LOG\_DEADLINE\_QOS\_POLICY**

```
#define DDSC_LOG_DEADLINE_QOS_POLICY 14
```

The deadline qos policy kind.

**DDSC\_LOG\_LIVELINESS\_QOS\_POLICY**

```
#define DDSC_LOG_LIVELINESS_QOS_POLICY 15
```

The liveliness qos policy kind.

**DDSC\_LOG\_HISTORY\_QOS\_POLICY**

```
#define DDSC_LOG_HISTORY_QOS_POLICY 16
```

The history qos policy kind.

**DDSC\_LOG\_RESOURCE\_LIMIT\_QOS\_POLICY**

```
#define DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY 17
```

The resource limits qos policy kind.

**DDSC\_LOG\_PROTOCOL\_QOS\_POLICY**

```
#define DDSC_LOG_PROTOCOL_QOS_POLICY 18
```

The protocol qos policy kind.

**DDSC\_LOG\_TYPE\_SUPPORT\_QOS\_POLICY**

```
#define DDSC_LOG_TYPE_SUPPORT_QOS_POLICY 19
```

The type-support qos policy kind.

**DDSC\_LOG\_RELIABILITY\_QOS\_POLICY**

```
#define DDSC_LOG_RELIABILITY_QOS_POLICY 20
```

The reliability qos policy kind.

**DDSC\_LOG\_DURABILITY\_QOS\_POLICY**

```
#define DDSC_LOG_DURABILITY_QOS_POLICY 21
```

The durability qos policy kind.

**DDSC\_LOG\_OWNERSHIP\_QOS\_POLICY**

```
#define DDSC_LOG_OWNERSHIP_QOS_POLICY 22
```

The ownership qos policy kind.

**DDSC\_LOG\_OWNERSHIP\_STRENGTH\_QOS\_POLICY**

```
#define DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY 23
```

The ownership-strength qos policy kind.

**DDSC\_LOG\_TRANSPORT\_QOS\_POLICY**

```
#define DDSC_LOG_TRANSPORT_QOS_POLICY 24
```

The transport qos policy kind.

**DDSC\_LOG\_PARTICIPANT\_ID\_QOS\_POLICY**

```
#define DDSC_LOG_PARTICIPANT_ID_QOS_POLICY 25
```

The participant id qos policy kind.

**DDSC\_LOG\_HEARTBEATS\_QOS\_POLICY**

```
#define DDSC_LOG_HEARTBEATS_QOS_POLICY 26
```

The heartbeat qos policy kind.

When configured for reliable communication, `heartbeats_per_max_samples` must be fit within `max_samples`

**DDSC\_LOG\_DATAWRITER\_RESOURCE\_QOS\_POLICY**

```
#define DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY 27
```

The DataWriterQos `writer_resource_limits` policy.

An invalid value has been set for a limit of `DataWriterQos.writer_resource_limits`. Each value must be positive and finite. May be logged by Discovery writers with no `initial_peers` set.

**DDSC\_LOG\_DATAREADER\_RESOURCE\_QOS\_POLICY**

```
#define DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY 28
```

The DataReaderQos `reader_resource_limits` policy.

**DDSC\_LOG\_DESTINATION\_ORDER\_POLICY**

```
#define DDSC_LOG_DESTINATION_ORDER_POLICY 29
```

The DestinationOrderQos destination\_order policy.

**DDSC\_LOG\_PROPERTY\_QOS\_POLICY**

```
#define DDSC_LOG_PROPERTY_QOS_POLICY 30
```

The PropertyQos policy.

**DDSC\_LOG\_PUBLISH\_MODE\_QOS\_POLICY**

```
#define DDSC_LOG_PUBLISH_MODE_QOS_POLICY 31
```

The PublishModeQos policy.

**DDSC\_LOG\_DATA\_REPRESENTATION\_QOS\_POLICY**

```
#define DDSC_LOG_DATA_REPRESENTATION_QOS_POLICY 33
```

The DataRepresentationQos policy.

**DDSC\_LOG\_LATENCY\_BUDGET\_QOS\_POLICY**

```
#define DDSC_LOG_LATENCY_BUDGET_QOS_POLICY 34
```

The LatencyBudgetQos policy.

**DDSC\_LOG\_CONTENT\_FILTER\_QOS\_POLICY**

```
#define DDSC_LOG_CONTENT_FILTER_QOS_POLICY 35
```

The ContentFilterQos policy.

**DDSC\_LOG\_QOS\_INCONSISTENT\_POLICY\_EC**

```
#define DDSC_LOG_QOS_INCONSISTENT_POLICY_EC (DDSC_LOG_BASE + 501)
```

An inconsistent Qos policy for the specified Qos kind was found.

**DDSC\_LOG\_QOS\_INCONSISTENT\_POLICIES\_EC**

```
#define DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC (DDSC_LOG_BASE + 502)
```

Inconsistency between two Qos policies for the specified Qos kind was found.

**DDSC\_LOG\_QOS\_INCONSISTENT\_EC**

```
#define DDSC_LOG_QOS_INCONSISTENT_EC (DDSC_LOG_BASE + 503)
```

Failed to create an entity or set a qos due to inconsistent policy.

**DDSC\_LOG\_QOS\_COPY\_EC**

```
#define DDSC_LOG_QOS_COPY_EC (DDSC_LOG_BASE + 504)
```

Failed to copy a Qos of the specified kind.

**DDSC\_LOG\_QOS\_INITIALIZE\_EC**

```
#define DDSC_LOG_QOS_INITIALIZE_EC (DDSC_LOG_BASE + 505)
```

Failed to initialize a Qos of the specified kind.

**DDSC\_LOG\_QOS\_FINALIZE\_EC**

```
#define DDSC_LOG_QOS_FINALIZE_EC (DDSC_LOG_BASE + 506)
```

Failed to finalize a Qos of the specified kind.

**DDSC\_LOG\_QOS\_SET\_EC**

```
#define DDSC_LOG_QOS_SET_EC (DDSC_LOG_BASE + 507)
```

Failed to set a Qos of the specified kind.

**DDSC\_LOG\_QOS\_SET\_ON\_ENABLED\_EC**

```
#define DDSC_LOG_QOS_SET_ON_ENABLED_EC (DDSC_LOG_BASE + 508)
```

Failed to set a Qos of the specified kind because the entity is already enabled.

**DDSC\_LOG\_QOS\_IMMUTABLE\_EC**

```
#define DDSC_LOG_QOS_IMMUTABLE_EC (DDSC_LOG_BASE + 509)
```

Failed to set a Qos of the specified kind the immutable Qos policies have been changed.

**DDSC\_LOG\_QOS\_CHANGED\_EC**

```
#define DDSC_LOG_QOS_CHANGED_EC (DDSC_LOG_BASE + 510)
```

A discovered Qos changed (the entity already existed)

**DDSC\_LOG\_QOS\_GET\_EC**

```
#define DDSC_LOG_QOS_GET_EC (DDSC_LOG_BASE + 511)
```

Failed to get a Qos of the specified kind.

**DDSC\_LOG\_TOPIC\_LISTENER**

```
#define DDSC_LOG_TOPIC_LISTENER 1
```

The topic listener kind.

**DDSC\_LOG\_DATAREADER\_LISTENER**

```
#define DDSC_LOG_DATAREADER_LISTENER 2
```

The datareader listener kind.

**DDSC\_LOG\_DATAWRITER\_LISTENER**

```
#define DDSC_LOG_DATAWRITER_LISTENER 3
```

The datawriter listener kind.

**DDSC\_LOG\_SUBSCRIBER\_LISTENER**

```
#define DDSC_LOG_SUBSCRIBER_LISTENER 4
```

The subscriber listener kind.

**DDSC\_LOG\_PUBLISHER\_LISTENER**

```
#define DDSC_LOG_PUBLISHER_LISTENER 5
```

The publisher listener kind.

**DDSC\_LOG\_PARTICIPANT\_LISTENER**

```
#define DDSC_LOG_PARTICIPANT_LISTENER 6
```

The participant listener kind.

**DDSC\_LOG\_PARTICIPANTFACTORY\_LISTENER**

```
#define DDSC_LOG_PARTICIPANTFACTORY_LISTENER 7
```

The participant-factory listener kind.

**DDSC\_LOG\_LISTENER\_INCONSISTENT\_EC**

```
#define DDSC_LOG_LISTENER_INCONSISTENT_EC (DDSC_LOG_BASE + 512)
```

Failed to create an entity due to inconsistent listener and status mask.

**DDSC\_LOG\_LISTENER\_SET\_EC**

```
#define DDSC_LOG_LISTENER_SET_EC (DDSC_LOG_BASE + 513)
```

Failed to set the listener of the specified kind.

**DDSC\_LOG\_LISTENER\_GET\_EC**

```
#define DDSC_LOG_LISTENER_GET_EC (DDSC_LOG_BASE + 514)
```

Failed to get the listener of the specified kind.

**DDSC\_LOG\_LISTENER\_SET\_ILLEGAL\_NULL\_EC**

```
#define DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC (DDSC_LOG_BASE + 515)
```

Illegal combination of NULL listener and non-NONE status mask when setting a listener for an Entity.

**DDSC\_LOG\_TOPIC\_ENTITY**

```
#define DDSC_LOG_TOPIC_ENTITY 1
```

The topic entity kind.

**DDSC\_LOG\_DATAWRITER\_ENTITY**

```
#define DDSC_LOG_DATAWRITER_ENTITY 2
```

The datawriter entity kind.

**DDSC\_LOG\_DATAREADER\_ENTITY**

```
#define DDSC_LOG_DATAREADER_ENTITY 3
```

The datareader entity kind.

**DDSC\_LOG\_PUBLISHER\_ENTITY**

```
#define DDSC_LOG_PUBLISHER_ENTITY 4
```

The publisher entity kind.

**DDSC\_LOG\_SUBSCRIBER\_ENTITY**

```
#define DDSC_LOG_SUBSCRIBER_ENTITY 5
```

The subscriber entity kind.

**DDSC\_LOG\_PARTICIPANT\_ENTITY**

```
#define DDSC_LOG_PARTICIPANT_ENTITY 6
```

The participant entity kind.

**DDSC\_LOG\_PUBLICATION\_ENTITY**

```
#define DDSC_LOG_PUBLICATION_ENTITY 7
```

The publication entity kind.



**DDSC\_LOG\_SUBSCRIPTION\_ENTITY**

```
#define DDSC_LOG_SUBSCRIPTION_ENTITY 8
```

The subscription entity kind.

**DDSC\_LOG\_ENTITY\_ENABLE\_EC**

```
#define DDSC_LOG_ENTITY_ENABLE_EC (DDSC_LOG_BASE + 600)
```

Failed to enable an entity of the specified kind.

**DDSC\_LOG\_ENTITY\_NOT\_EMPTY\_EC**

```
#define DDSC_LOG_ENTITY_NOT_EMPTY_EC (DDSC_LOG_BASE + 601)
```

Failed to an delete/finalize an entity of the specified kind because it is not empty.

**DDSC\_LOG\_ENTITY\_FINALIZE\_EC**

```
#define DDSC_LOG_ENTITY_FINALIZE_EC (DDSC_LOG_BASE + 602)
```

Failed to finalize an entity of the specified kind.

**DDSC\_LOG\_ENTITY\_INITIALIZE\_EC**

```
#define DDSC_LOG_ENTITY_INITIALIZE_EC (DDSC_LOG_BASE + 603)
```

Failed to initialize an entity of the specified kind.

**DDSC\_LOG\_ENTITY\_NOT\_ENABLED\_EC**

```
#define DDSC_LOG_ENTITY_NOT_ENABLED_EC (DDSC_LOG_BASE + 604)
```

An operation was attempted on an entity that is not enabled.

**DDSC\_LOG\_ENTITY\_DIFFERENT\_FACTORY\_EC**

```
#define DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC (DDSC_LOG_BASE + 605)
```

Entities are in different factories.

A entity tried to use an entity created by a different factory. For example, it is illegal to create a datawriter using a topic from a different participant.

**DDSC\_LOG\_ENTITY\_INVALID\_PROPERTY\_EC**

```
#define DDSC_LOG_ENTITY_INVALID_PROPERTY_EC (DDSC_LOG_BASE + 606)
```

An invalid property was specified for the entity.

**DDSC\_LOG\_DATAWRITER\_CDR**

```
#define DDSC_LOG_DATAWRITER_CDR 1
```

The datawriter CDR kind.

**DDSC\_LOG\_DATAREADER\_CDR**

```
#define DDSC_LOG_DATAREADER_CDR 2
```

The datareader CDR kind.

**DDSC\_LOG\_DATAWRITER\_INLINE**

```
#define DDSC_LOG_DATAWRITER_INLINE 3
```

The datawriter inline kind.

**DDSC\_LOG\_CDR\_POOL\_ALLOC\_EC**

```
#define DDSC_LOG_CDR_POOL_ALLOC_EC (DDSC_LOG_BASE + 700)
```

Failed to allocate a pool of the specified kind.

**DDSC\_LOG\_CDR\_BUFFER\_SET\_EC**

```
#define DDSC_LOG_CDR_BUFFER_SET_EC (DDSC_LOG_BASE + 701)
```

Failed to set the CDR buffer for a packet.

**DDSC\_LOG\_CDR\_POOL\_DELETE\_EC**

```
#define DDSC_LOG_CDR_POOL_DELETE_EC (DDSC_LOG_BASE + 702)
```

Failed to delete the CDR pool.

**DDSC\_LOG\_CDR\_SERIALIZE\_PID\_EC**

```
#define DDSC_LOG_CDR_SERIALIZE_PID_EC (DDSC_LOG_BASE + 703)
```

Failed to serialize a parameter ID.

**DDSC\_LOG\_CDR\_SERIALIZE\_PID\_LENGTH\_EC**

```
#define DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC (DDSC_LOG_BASE + 704)
```

Failed to serialize a parameter length.

**DDSC\_LOG\_CDR\_SERIALIZE\_KEYHASH\_EC**

```
#define DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 705)
```

Failed to serialize a key-hash.

**DDSC\_LOG\_CDR\_SERIALIZE\_DATA\_EC**

```
#define DDSC_LOG_CDR_SERIALIZE_DATA_EC (DDSC_LOG_BASE + 706)
```

Failed to serialize payload data.

**DDSC\_LOG\_DESERIALIZE\_BAD\_PID\_LENGTH\_EC**

```
#define DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC (DDSC_LOG_BASE + 707)
```

Deserialized an invalid parameter length for a specific parameter ID.

**DDSC\_LOG\_CDR\_DESERIALIZE\_PID\_EC**

```
#define DDSC_LOG_CDR_DESERIALIZE_PID_EC (DDSC_LOG_BASE + 708)
```

Failed to deserialize the ID of an inline parameter.

**DDSC\_LOG\_CDR\_DESERIALIZE\_PID\_LENGTH\_EC**

```
#define DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC (DDSC_LOG_BASE + 709)
```

Failed to deserialize the length of an inline parameter.

**DDSC\_LOG\_CDR\_INCREMENT\_POS\_EC**

```
#define DDSC_LOG_CDR_INCREMENT_POS_EC (DDSC_LOG_BASE + 710)
```

Failed to increment to the position of the next inline parameter.

**DDSC\_LOG\_CDR\_SET\_POS\_EC**

```
#define DDSC_LOG_CDR_SET_POS_EC (DDSC_LOG_BASE + 711)
```

Failed to set the reception stream position.

**DDSC\_LOG\_CDR\_DESERIALIZE\_HEADER\_EC**

```
#define DDSC_LOG_CDR_DESERIALIZE_HEADER_EC (DDSC_LOG_BASE + 712)
```

Failed to deserialize the encapsulation header.

**DDSC\_LOG\_CDR\_DESERIALIZE\_DATA\_EC**

```
#define DDSC_LOG_CDR_DESERIALIZE_DATA_EC (DDSC_LOG_BASE + 713)
```

Failed to deserialize CDR payload data.

**DDSC\_LOG\_CDR\_INITIALIZE\_SAMPLE\_EC**

```
#define DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC (DDSC_LOG_BASE + 714)
```

Failed to initialize CDR sample.

**DDSC\_LOG\_CDR\_FINALIZE\_SAMPLE\_EC**

```
#define DDSC_LOG_CDR_FINALIZE_SAMPLE_EC (DDSC_LOG_BASE + 715)
```

Failed to finalize sample.

**DDSC\_LOG\_CDR\_SERIALIZE\_STATUS\_INFO\_EC**

```
#define DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC (DDSC_LOG_BASE + 716)
```

Failed to serialize the status info parameter.

**DDSC\_LOG\_CDR\_DESERIALIZE\_KEYHASH\_EC**

```
#define DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 717)
```

Failed to deserialize a key-hash.

**DDSC\_LOG\_CDR\_DESERIALIZE\_KEY\_EC**

```
#define DDSC_LOG_CDR_DESERIALIZE_KEY_EC (DDSC_LOG_BASE + 718)
```

Failed to deserialize CDR payload key.

**DDSC\_LOG\_NETIO\_ADD\_ANON\_TOPIC\_ROUTE\_EC**

```
#define DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC (DDSC_LOG_BASE + 800)
```

Failed to add a route to an anonymous participant discovery datawriter.

**DDSC\_LOG\_NETIO\_ADD\_TOPIC\_ROUTE\_EC**

```
#define DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC (DDSC_LOG_BASE + 801)
```

Failed to add a route to a topic from a datawriter.

**DDSC\_LOG\_NETIO\_DELETE\_TOPIC\_ROUTE\_EC**

```
#define DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC (DDSC_LOG_BASE + 802)
```

Failed to delete a route to a topic.

**DDSC\_LOG\_NETIO\_FORWARD\_TOPIC\_EC**

```
#define DDSC_LOG_NETIO_FORWARD_TOPIC_EC (DDSC_LOG_BASE + 803)
```

Failed to forward a topic.

**DDSC\_LOG\_NETIO\_NETIO\_KIND**

```
#define DDSC_LOG_NETIO_NETIO_KIND 1
```

Any NETIO interface kind, typically UDP.

**DDSC\_LOG\_INTRA\_NETIO\_KIND**

```
#define DDSC_LOG_INTRA_NETIO_KIND 2
```

The intra interface kind.

**DDSC\_LOG\_RTPS\_NETIO\_KIND**

```
#define DDSC_LOG_RTPS_NETIO_KIND 3
```

The RTPS interface kind.

**DDSC\_LOG\_DATAREADER\_NETIO\_KIND**

```
#define DDSC_LOG_DATAREADER_NETIO_KIND 4
```

The DataReader interface kind.

**DDSC\_LOG\_DATAWRITER\_NETIO\_KIND**

```
#define DDSC_LOG_DATAWRITER_NETIO_KIND 5
```

The DataWriter interface kind.

**DDSC\_LOG\_NETIO\_BIND\_EXTERNAL\_EC**

```
#define DDSC_LOG_NETIO_BIND_EXTERNAL_EC (DDSC_LOG_BASE + 804)
```

Failed to bind two external interface of the specified kind.

**DDSC\_LOG\_NETIO\_UNBIND\_EXTERNAL\_EC**

```
#define DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC (DDSC_LOG_BASE + 805)
```

Failed to unbind two external interfaces of the specified kind.

**DDSC\_LOG\_NETIO\_BIND\_EC**

```
#define DDSC_LOG_NETIO_BIND_EC (DDSC_LOG_BASE + 806)
```

Failed to bind an interface to a peer interface.

**DDSC\_LOG\_NETIO\_UNBIND\_EC**

```
#define DDSC_LOG_NETIO_UNBIND_EC (DDSC_LOG_BASE + 807)
```

Failed to unbind an interface from a peer interface.

**DDSC\_LOG\_NETIO\_ADD\_ROUTE\_EC**

```
#define DDSC_LOG_NETIO_ADD_ROUTE_EC (DDSC_LOG_BASE + 808)
```

Failed to add a route from an interface to a peer interface.

**DDSC\_LOG\_NETIO\_DELETE\_ROUTE\_EC**

```
#define DDSC_LOG_NETIO_DELETE_ROUTE_EC (DDSC_LOG_BASE + 809)
```

Failed to delete a route from an interface to a peer interface.

**DDSC\_LOG\_NETIO\_GET\_EXTERNAL\_INTF\_EC**

```
#define DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC (DDSC_LOG_BASE + 810)
```

Failed to get an external interface for the specified interface kind.

**DDSC\_LOG\_NETIO\_NO\_ROUTE\_EC**

```
#define DDSC_LOG_NETIO_NO_ROUTE_EC (DDSC_LOG_BASE + 811)
```

A DataReader failed a bind due to no existing route.

**DDSC\_LOG\_NETIO\_ROUTE\_LOOKUP\_FAILED\_EC**

```
#define DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC (DDSC_LOG_BASE + 812)
```

Lookup a route to a destination failed.

A failure was encountered when trying to lookup a route to a destination, this log message is preceded by a more specific message. Failure to lookup a route does not mean the route does not exist, it means it failed to determine if a route did exist.

**DDSC\_LOG\_NETIO\_GET\_ROUTE\_TABLE\_FAILED\_EC**

```
#define DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC (DDSC_LOG_BASE + 813)
```

Failed to get the route table for an interface.

**DDSC\_LOG\_NETIO\_SEND\_FAILED\_EC**

```
#define DDSC_LOG_NETIO_SEND_FAILED_EC (DDSC_LOG_BASE + 814)
```

Failure when sending on an interface.

**DDSC\_LOG\_NETIO\_SET\_STATE\_EC**

```
#define DDSC_LOG_NETIO_SET_STATE_EC (DDSC_LOG_BASE + 815)
```

Failed to set an interface state.

**DDSC\_LOG\_NETIO\_PEER\_LOOKUP\_EC**

```
#define DDSC_LOG_NETIO_PEER_LOOKUP_EC (DDSC_LOG_BASE + 816)
```

Datawriter did not find a peer.

**DDSC\_LOG\_NETIO\_FORCED\_REMOVE\_EC**

```
#define DDSC_LOG_NETIO_FORCED_REMOVE_EC (DDSC_LOG_BASE + 817)
```

Forced removal of sample downstream failed.

The datawriter failed to force a sample removal downstream

**DDSC\_LOG\_PACKET\_INIT\_EC**

```
#define DDSC_LOG_PACKET_INIT_EC (DDSC_LOG_BASE + 818)
```

Failed to initialize a packet.

**DDSC\_LOG\_PACKET\_SET\_HEAD\_EC**

```
#define DDSC_LOG_PACKET_SET_HEAD_EC (DDSC_LOG_BASE + 819)
```

Failed to set the head of a packet.

**DDSC\_LOG\_PACKET\_SET\_TAIL\_EC**

```
#define DDSC_LOG_PACKET_SET_TAIL_EC (DDSC_LOG_BASE + 820)
```

Failed to set the tail of a packet.



**DDSC\_LOG\_NETIO\_DISCOVERY\_ENABLED\_TRANSPORT\_EC**

```
#define DDSC_LOG_NETIO_DISCOVERY_ENABLED_TRANSPORT_EC (DDSC_LOG_BASE + 821)
```

Configured discovery enabled transport is not valid.

**DDSC\_LOG\_DW\_ACKNACK\_FAILED\_EC**

```
#define DDSC_LOG_DW_ACKNACK_FAILED_EC (DDSC_LOG_BASE + 900)
```

Failed to ACKNACK sample in the writer history.

**DDSC\_LOG\_DW\_COMMIT\_EC**

```
#define DDSC_LOG_DW_COMMIT_EC (DDSC_LOG_BASE + 901)
```

Failed to commit a sample to the writer queue.

**DDSC\_LOG\_DW\_KEYHASH\_CREATE\_EC**

```
#define DDSC_LOG_DW_KEYHASH_CREATE_EC (DDSC_LOG_BASE + 902)
```

Failed to create keyhash of instance handle.

**DDSC\_LOG\_DW\_ILLEGAL\_KEY\_KIND\_EC**

```
#define DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC (DDSC_LOG_BASE + 903)
```

Failed a write due to an invalid key kind for the type being written.

**DDSC\_LOG\_DW\_CREATE\_TYPED\_WRITER\_EC**

```
#define DDSC_LOG_DW_CREATE_TYPED_WRITER_EC (DDSC_LOG_BASE + 904)
```

Failed to create a typed writer.

**DDSC\_LOG\_DW\_HISTORY\_REGISTER\_KEY\_EC**

```
#define DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC (DDSC_LOG_BASE + 905)
```

Failed to register the key of an instance.

A DataWriter failed to register the key of a new instance because DataWriterQos.resource\_limits.max\_instances was exceeded.

**DDSC\_LOG\_DW\_UNKNOWN\_FLOW\_CONTROLLER\_EC**

```
#define DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC (DDSC_LOG_BASE + 906)
```

Failed to create a flow-controller.

A DataWriter failed to create the flow-controller specified in the Qos policy.

**DDSC\_LOG\_DW\_FLOW\_CONTROLLER\_REQUIRED\_EC**

```
#define DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC (DDSC_LOG_BASE + 907)
```

The data-sample is larger than supported by the transport, but the flow-controller has been compiled out.

**DDSC\_LOG\_DW\_INSTANCE\_ASSERTION\_FAILED\_EC**

```
#define DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC (DDSC_LOG_BASE + 908)
```

Failed to reserve an instance to publish discovery data for a user created data writer. This typically means DomainParticipantQos.resource\_limits.remote\_writer\_allocation is too small.

**DDSC\_LOG\_DW\_INCOMPATIBLE\_PADDING\_EC**

```
#define DDSC_LOG_DW_INCOMPATIBLE_PADDING_EC (DDSC_LOG_BASE + 909)
```

A DataReader incompatible with padding bits in encapsulation header was discovered.

Discovered an incompatible Micro DataReader that cannot parse the padding bits set in the encapsulation options of a sample payload by the Micro DataWriter. Resolve by configuring the Micro DataWriter to omit padding bits or upgrade the Micro DataReader to a version that can interpret them.

Disable padding bits in the Micro DataWriter by setting the property [DDS\\_XTYPES\\_COMPLIANCE\\_MASK\\_PROPERTY](#) to a value that removes the encapsulation option padding bit. See the `::NDDS_Config_XTypesComplianceMask` section in RTI Connext Micro documentation for more information.

The version number of RTI Connext DDS Micro is printed as 4 hexadecimal digits:

- byte 3: Major
- byte 2: Minor
- byte 1: Release
- byte 0: Revision

**DDSC\_LOG\_DR\_CREATE\_TYPED\_READER\_EC**

```
#define DDSC_LOG_DR_CREATE_TYPED_READER_EC (DDSC_LOG_BASE + 1000)
```

Failed to create a typed datareader.

**DDSC\_LOG\_DR\_COPY\_DATA\_SAMPLE\_EC**

```
#define DDSC_LOG_DR_COPY_DATA_SAMPLE_EC (DDSC_LOG_BASE + 1001)
```

Failed to copy a sample upon reception, read, or take.

**DDSC\_LOG\_DR\_COMMIT\_SAMPLE\_EC**

```
#define DDSC_LOG_DR_COMMIT_SAMPLE_EC (DDSC_LOG_BASE + 1002)
```

Failed to commit a sample to be made available to be read or taken.

**DDSC\_LOG\_DR\_FILTER\_ERROR\_EC**

```
#define DDSC_LOG_DR_FILTER_ERROR_EC (DDSC_LOG_BASE + 1003)
```

A datareader filter function failed.

**DDSC\_LOG\_DR\_DESERIALIZE\_KEYHASH\_EC**

```
#define DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 1004)
```

Failed to deserialize a key-hash parameter.

**DDSC\_LOG\_DR\_GET\_ENTRY\_FAILED\_EC**

```
#define DDSC_LOG_DR_GET_ENTRY_FAILED_EC (DDSC_LOG_BASE + 1005)
```

Failed to get a Reader History entry for a received sample.

**DDSC\_LOG\_DR\_COMMIT\_ENTRY\_EC**

```
#define DDSC_LOG_DR_COMMIT_ENTRY_EC (DDSC_LOG_BASE + 1006)
```

Failed to commit a receive sample to Reader History to be read or taken.

**DDSC\_LOG\_DR\_UNREGISTER\_KEY\_EC**

```
#define DDSC_LOG_DR_UNREGISTER_KEY_EC (DDSC_LOG_BASE + 1007)
```

A DataReader failed to unregister an instance.

**DDSC\_LOG\_DR\_DISPOSE\_KEY\_EC**

```
#define DDSC_LOG_DR_DISPOSE_KEY_EC (DDSC_LOG_BASE + 1008)
```

A DataReader failed to dispose an instance.

**DDSC\_LOG\_DR\_READ\_TAKE\_FAILURE\_EC**

```
#define DDSC_LOG_DR_READ_TAKE_FAILURE_EC (DDSC_LOG_BASE + 1009)
```

A call to a reader/take function failed.

**DDSC\_LOG\_DR\_INSTANCE\_TO\_KEYHASH\_EC**

```
#define DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC (DDSC_LOG_BASE + 1010)
```

Failed to create keyhash of instance handle.

**DDSC\_LOG\_DR\_INSTANCE\_MAPPING\_EXHAUSTED\_EC**

```
#define DDSC_LOG_DR_INSTANCE_MAPPING_EXHAUSTED_EC (DDSC_LOG_BASE + 1011)
```

Failed to create keyhash of instance handle.

**DDSC\_LOG\_DR\_INSTANCE\_ASSERTION\_FAILED\_EC**

```
#define DDSC_LOG_DR_INSTANCE_ASSERTION_FAILED_EC (DDSC_LOG_BASE + 1012)
```

Failed to reserve an instance to publish discovery data for a user created data reader. This typically means Domain↔ ParticipantQos.resource\_limits.remote\_reader\_allocation is too small.

**DDSC\_LOG\_DR\_ON\_SAMPLE\_REMOVED\_EC**

```
#define DDSC_LOG_DR_ON_SAMPLE_REMOVED_EC (DDSC_LOG_BASE + 1013)
```

Failed to remove a sample when it was being pushed out from the Reader History.

**DDSC\_LOG\_TYPE\_NAME\_CMP\_EC**

```
#define DDSC_LOG_TYPE_NAME_CMP_EC (DDSC_LOG_BASE + 1100)
```

Two type names are incompatible.

**DDSC\_LOG\_TOPIC\_NAME\_CMP\_EC**

```
#define DDSC_LOG_TOPIC_NAME_CMP_EC (DDSC_LOG_BASE + 1101)
```

Two topic names are incompatible.

**DDSC\_LOG\_TYPE\_FUNCTION\_GET\_SERIALIZED\_SAMPLE\_MAX\_SIZE**

```
#define DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE 1
```

The `get_serialized_sample_max_size` function kind.

**DDSC\_LOG\_TYPE\_FUNCTION\_SERIALIZE\_DATA**

```
#define DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA 2
```

The `serialize_data` function kind.

**DDSC\_LOG\_TYPE\_FUNCTION\_DESERIALIZE\_DATA**

```
#define DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA 3
```

The `deserialize_data` function kind.

**DDSC\_LOG\_TYPE\_FUNCTION\_CREATE\_SAMPLE**

```
#define DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE 4
```

The `create_sample` function kind.

**DDSC\_LOG\_TYPE\_FUNCTION\_COPY\_SAMPLE**

```
#define DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE 5
```

The `copy_sample` function kind.

**DDSC\_LOG\_TYPE\_FUNCTION\_DELETE\_SAMPLE**

```
#define DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE 6
```

The delete\_sample function kind.

**DDSC\_LOG\_TYPE\_FUNCTION\_GET\_KEY\_KIND**

```
#define DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND 7
```

The get\_key\_kind function kind.

**DDSC\_LOG\_TYPE\_FUNCTION\_INSTANCE\_TO\_KEYHASH**

```
#define DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH 8
```

The instance\_to\_keyhash function kind.

**DDSC\_LOG\_TYPE\_FUNCTION\_NULL\_EC**

```
#define DDSC_LOG_TYPE_FUNCTION_NULL_EC (DDSC_LOG_BASE + 1102)
```

Invalid type plugin, The specified function pointer is NULL.

**DDSC\_LOG\_TOPIC\_TOO\_LONG\_EC**

```
#define DDSC_LOG_TOPIC_TOO_LONG_EC (DDSC_LOG_BASE + 1103)
```

Failed to create a topic because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)

**DDSC\_LOG\_TYPE\_TOO\_LONG\_EC**

```
#define DDSC_LOG_TYPE_TOO_LONG_EC (DDSC_LOG_BASE + 1104)
```

Failed to create a type because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)

**DDSC\_LOG\_LOOKUP\_TYPE\_PLUGIN\_EC**

```
#define DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC (DDSC_LOG_BASE + 1105)
```

A type-plugin for the given type could not be found.

**DDSC\_LOG\_TYPE\_KEY\_TYPE\_EC**

```
#define DDSC_LOG_TYPE_KEY_TYPE_EC (DDSC_LOG_BASE + 1106)
```

Two types have incompatible keys.

**DDSC\_LOG\_TYPE\_PLUGIN\_GET\_BUFFER\_EC**

```
#define DDSC_LOG_TYPE_PLUGIN_GET_BUFFER_EC (DDSC_LOG_BASE + 1107)
```

Type-plugin cannot return a buffer.

**DDSC\_LOG\_TYPE\_PLUGIN\_INCONSISTENT\_DR\_EC**

```
#define DDSC_LOG_TYPE_PLUGIN_INCONSISTENT_DR_EC (DDSC_LOG_BASE + 1108)
```

Inconsisten type-plugin DataReader.

**DDSC\_LOG\_DUPLICATE\_GUID\_PREFIX\_EC**

```
#define DDSC_LOG_DUPLICATE_GUID_PREFIX_EC (DDSC_LOG_BASE + 1109)
```

A DomainParticipant with the specified GUID already exists in the DomainParticipant factory.

A DomainParticipant with the specified GUID already exists in the domain in the DomainParticipant factory. The GUID is either generated (default) or specified manually in the DomainParticipantQos.protocol policy. This error typically indicates that the DomainParticipantQos.protocol is manually specified and is a duplicate.

**DDSC\_LOG\_TYPE\_MANAGED\_POOL\_FAILURE\_EC**

```
#define DDSC_LOG_TYPE_MANAGED_POOL_FAILURE_EC (DDSC_LOG_BASE + 1110)
```

Managed pool function failed.

**DDSC\_LOG\_TYPE\_GET\_SAMPLE\_ID\_EC**

```
#define DDSC_LOG_TYPE_GET_SAMPLE_ID_EC (DDSC_LOG_BASE + 1111)
```

Managed pool failed to get sample id.

**DDSC\_LOG\_PARTICIPANT\_NAME\_EMPTY\_EC**

```
#define DDSC_LOG_PARTICIPANT_NAME_EMPTY_EC (DDSC_LOG_BASE + 1112)
```

A DomainParticipant with enable\_participant\_discovery\_by\_name set to TRUE but an empty name is created.

If DomainParticipantQos.discovery.enable\_participant\_discovery\_by\_name is set to TRUE, but DomainParticipantQos.participant\_name.name is empty. This is not allowed.

If DomainParticipantQos.discovery.enable\_participant\_discovery\_by\_name is set to TRUE, the requirement is that all DomainParticipant's in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connext DDS Micro to verify that all DomainParticipant's are uniquely named. This is the responsibility of the applications.

**DDSC\_LOG\_DUPLICATE\_PARTICIPANT\_NAME\_EC**

```
#define DDSC_LOG_DUPLICATE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 1113)
```

A DomainParticipant with enable\_participant\_discovery\_by\_name set to TRUE, but a local participant with the same DomainParticipantQos.participant\_name.name already exists.

If DomainParticipantQos.discovery.enable\_participant\_discovery\_by\_name is set to TRUE, but an local DomainParticipant in the same DomainParticipantQos.participant\_name.name already exists in the same process in the same domain. This is not allowed.

If DomainParticipantQos.discovery.enable\_participant\_discovery\_by\_name is set to TRUE, the requirement is that all DomainParticipant's in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connext DDS Micro to verify that all DomainParticipant's are uniquely named. This is the responsibility of the applications.

**DDSC\_LOG\_DUPLICATE\_REMOTE\_PARTICIPANT\_NAME\_EC**

```
#define DDSC_LOG_DUPLICATE_REMOTE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 1114)
```

A remote participant is discovered with the same name as the a local participant name in the same domain ID.

If DomainParticipantQos.discovery.enable\_participant\_discovery\_by\_name is set to TRUE, but a remote participant is discovered with the same name as a local participant name in the same domain ID.

If DomainParticipantQos.discovery.enable\_participant\_discovery\_by\_name is set to TRUE, the requirement is that all DomainParticipant's in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connext DDS Micro to verify that all DomainParticipant's are uniquely named. This is the responsibility of the applications.



**DDSC\_LOG\_REMOTE\_PARTICIPANT\_NAME\_EMPTY\_EC**

```
#define DDSC_LOG_REMOTE_PARTICIPANT_NAME_EMPTY_EC (DDSC_LOG_BASE + 1115)
```

A remote participant without a name is discovered.

If `DomainParticipantQos.discovery.enable_participant_discovery_by_name` is set to `TRUE`, but a discovered participant's name is empty. This is not allowed.

If `DomainParticipantQos.discovery.enable_participant_discovery_by_name` is set to `TRUE`, the requirement is that all `DomainParticipant`'s in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connex DDS Micro to verify that all `DomainParticipant`'s are uniquely named. This is the responsibility of the applications.

**DDSC\_LOG\_REMOTE\_PARTICIPANT\_RESTART\_EC**

```
#define DDSC_LOG_REMOTE_PARTICIPANT_RESTART_EC (DDSC_LOG_BASE + 1116)
```

Failed to reset a remote participant.

If `DomainParticipantQos.discovery.enable_participant_discovery_by_name` is set to `TRUE`, but a discovered participant's name is empty. This is not allowed.

If `DomainParticipantQos.discovery.enable_participant_discovery_by_name` is set to `TRUE`, the requirement is that all `DomainParticipant`'s in the domain are uniquely named. If this is not true the system may experience undefined behavior.

Note that it is not possible For RTI Connex DDS Micro to verify that all `DomainParticipant`'s are uniquely named. This is the responsibility of the applications.

**DDSC\_LOG\_DISC\_LOCAL\_PARTICIPANT\_ENABLED\_EC**

```
#define DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC (DDSC_LOG_BASE + 1200)
```

Discovery plugin failed its update after a local `DomainParticipant` was enabled.

**DDSC\_LOG\_DISC\_BEFORE\_LOCAL\_PARTICIPANT\_CREATED\_EC**

```
#define DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1201)
```

Discovery plugin failed its update before a local `DomainParticipant` was created.

**DDSC\_LOG\_DISC\_AFTER\_LOCAL\_PARTICIPANT\_CREATED\_EC**

```
#define DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1202)
```

Discovery plugin failed its update after a local `DomainParticipant` was created.

**DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAREADER\_ENABLED\_EC**

```
#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC (DDSC_LOG_BASE + 1203)
```

Discovery plugin failed its update after a local DataReader was enabled.

**DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAREADER\_DELETED\_EC**

```
#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC (DDSC_LOG_BASE + 1204)
```

Discovery plugin failed its update after a local DataReader was deleted.

**DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAWRITER\_ENABLED\_EC**

```
#define DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC (DDSC_LOG_BASE + 1205)
```

Discovery plugin failed its update after a local DataWriter was enabled.

**DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAWRITER\_DELETED\_EC**

```
#define DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC (DDSC_LOG_BASE + 1206)
```

Discovery plugin failed its update after a local DataWriter was deleted.

**DDSC\_LOG\_DISC\_BEFORE\_REMOTE\_PARTICIPANT\_DELETED\_EC**

```
#define DDSC_LOG_DISC_BEFORE_REMOTE_PARTICIPANT_DELETED_EC (DDSC_LOG_BASE + 1207)
```

Discovery plugin failed its update after being notified that a remote DomainParticipant was about to be deleted.

**DDSC\_LOG\_DISC\_ADD\_PEER\_EC**

```
#define DDSC_LOG_DISC_ADD_PEER_EC (DDSC_LOG_BASE + 1208)
```

Failed to add a peer with discovery plugin.

**DDSC\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC**

```
#define DDSC_LOG_DESERIALIZE_UNKNOWN_PID_EC (DDSC_LOG_BASE + 1209)
```

Deserialized a parameter of unknown ID.

**DDSC\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC**

```
#define DDSC_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC (DDSC_LOG_BASE + 1210)
```

Failed to serialize Builtin Endpoint Mask parameter.

**DDSC\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC**

```
#define DDSC_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DDSC_LOG_BASE + 1211)
```

Failed to deserialize Builtin Endpoint Mask parameter.

**DDSC\_LOG\_SERIALIZE\_TOPIC\_NAME\_EC**

```
#define DDSC_LOG_SERIALIZE_TOPIC_NAME_EC (DDSC_LOG_BASE + 1212)
```

Failed to serialize Topic Name parameter.

**DDSC\_LOG\_CDR\_SET\_OFFSET\_EC**

```
#define DDSC_LOG_CDR_SET_OFFSET_EC (DDSC_LOG_BASE + 1213)
```

Failed to set current offset of stream.

**DDSC\_LOG\_SERIALIZE\_ENTITY\_NAME\_EC**

```
#define DDSC_LOG_SERIALIZE_ENTITY_NAME_EC (DDSC_LOG_BASE + 1214)
```

Failed to serialize Entity Name parameter.

**DDSC\_LOG\_SERIALIZE\_TYPE\_NAME\_EC**

```
#define DDSC_LOG_SERIALIZE_TYPE_NAME_EC (DDSC_LOG_BASE + 1215)
```

Failed to serialize Type Name parameter.

**DDSC\_LOG\_SERIALIZE\_GUID\_EC**

```
#define DDSC_LOG_SERIALIZE_GUID_EC (DDSC_LOG_BASE + 1216)
```

Failed to serialize GUID key.

**DDSC\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC**

```
#define DDSC_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DDSC_LOG_BASE + 1217)
```

Failed to serialize Default Unicast Locator parameter.

**DDSC\_LOG\_DESERIALIZE\_LOCATOR\_EC**

```
#define DDSC_LOG_DESERIALIZE_LOCATOR_EC (DDSC_LOG_BASE + 1218)
```

Failed to deserialize a locator of the specified kind.

**DDSC\_LOG\_LOCATORS\_FULL\_EC**

```
#define DDSC_LOG_LOCATORS_FULL_EC (DDSC_LOG_BASE + 1219)
```

A locator sequence is full, the remaining locators of the specified kind are dropped.

**DDSC\_LOG\_UNSUPPORTED\_LOCATOR\_EC**

```
#define DDSC_LOG_UNSUPPORTED_LOCATOR_EC (DDSC_LOG_BASE + 1220)
```

The locator kind is not supported by any transport registered with the domain participant and is dropped.

**DDSC\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC**

```
#define DDSC_LOG_SERIALIZE_PROTOCOL_VERSION_EC (DDSC_LOG_BASE + 1221)
```

Failed to serialize Protocol Version parameter.

**DDSC\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC**

```
#define DDSC_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DDSC_LOG_BASE + 1222)
```

Failed to deserialize Protocol Version parameter.

**DDSC\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC**

```
#define DDSC_LOG_DESERIALIZE_VENDOR_ID_EC (DDSC_LOG_BASE + 1223)
```

Failed to deserialize Vendor ID parameter.

**DDSC\_LOG\_SERIALIZE\_VENDOR\_ID\_EC**

```
#define DDSC_LOG_SERIALIZE_VENDOR_ID_EC (DDSC_LOG_BASE + 1224)
```

Failed to serialize Vendor ID parameter.

**DDSC\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC**

```
#define DDSC_LOG_SERIALIZE_PRODUCT_VERSION_EC (DDSC_LOG_BASE + 1225)
```

Failed to serialize Product Version parameter.

**DDSC\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC**

```
#define DDSC_LOG_SERIALIZE_LEASE_DURATION_EC (DDSC_LOG_BASE + 1226)
```

Failed to serialize Lease Duration parameter.

**DDSC\_LOG\_DATAWRITER\_ADD\_PEER\_FAILED\_EC**

```
#define DDSC_LOG_DATAWRITER_ADD_PEER_FAILED_EC (DDSC_LOG_BASE + 1227)
```

Failed to add a remote peer to a local DataWriter.

**DDSC\_LOG\_DATAREADER\_ADD\_PEER\_FAILED\_EC**

```
#define DDSC_LOG_DATAREADER_ADD_PEER_FAILED_EC (DDSC_LOG_BASE + 1228)
```

Failed to add a remote peer to a local DataReader.

**DDSC\_LOG\_DISC\_REMOTE\_PARTICIPANT\_REMOVE\_EC**

```
#define DDSC_LOG_DISC_REMOTE_PARTICIPANT_REMOVE_EC (DDSC_LOG_BASE + 1229)
```

Failed to remote/reset remote participant after liveliness expired.

**DDSC\_LOG\_DESERIALIZE\_PARTITION\_SEQ\_EC**

```
#define DDSC_LOG_DESERIALIZE_PARTITION_SEQ_EC (DDSC_LOG_BASE + 1230)
```

Failed to deserialize partition seq.

**DDSC\_LOG\_INITIALIZE\_DISCOVERY\_QUEUE\_EC**

```
#define DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC (DDSC_LOG_BASE + 1231)
```

Failed to initialize discovery queue.

**DDSC\_LOG\_QUEUE\_PUBLICATION\_DISCOVERY\_EC**

```
#define DDSC_LOG_QUEUE_PUBLICATION_DISCOVERY_EC (DDSC_LOG_BASE + 1232)
```

Failed to queue a publication discovery message.

**DDSC\_LOG\_QUEUE\_SUBSCRIPTION\_DISCOVERY\_EC**

```
#define DDSC_LOG_QUEUE_SUBSCRIPTION_DISCOVERY_EC (DDSC_LOG_BASE + 1233)
```

Failed to queue a subscription discovery message.

**DDSC\_LOG\_QUEUE\_FINALIZE\_EC**

```
#define DDSC_LOG_QUEUE_FINALIZE_EC (DDSC_LOG_BASE + 1234)
```

Failed finalize discovery queue.

**DDSC\_LOG\_IPC\_INIT\_FAILED\_EC**

```
#define DDSC_LOG_IPC_INIT_FAILED_EC (DDSC_LOG_BASE + 1300)
```

Failed to initialize Builtin IPC entities.

**DDSC\_LOG\_IPC\_ALLOC\_FAILED\_EC**

```
#define DDSC_LOG_IPC_ALLOC_FAILED_EC (DDSC_LOG_BASE + 1301)
```

Failed to allocate memory for one of the Builtin IPC entities.

**DDSC\_LOG\_IPC\_FINALIZE\_FAILED\_EC**

```
#define DDSC_LOG_IPC_FINALIZE_FAILED_EC (DDSC_LOG_BASE + 1302)
```

Failed to finalize all resources A potential memory leak might have been caused by unexpected errors during finalization of acquired resources.

**DDSC\_LOG\_IPC\_CHANNEL\_TYPE\_REGISTER\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_TYPE_REGISTER_EC (DDSC_LOG_BASE + 1303)
```

Failed to register type for a Builtin IPC channel.

**DDSC\_LOG\_IPC\_CHANNEL\_TOPIC\_CREATE\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_TOPIC_CREATE_EC (DDSC_LOG_BASE + 1304)
```

Failed to create topic for a Builtin IPC channel.

**DDSC\_LOG\_IPC\_CHANNEL\_READER\_CREATE\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_READER_CREATE_EC (DDSC_LOG_BASE + 1305)
```

Failed to create reader for a Builtin IPC channel.

**DDSC\_LOG\_IPC\_CHANNEL\_WRITER\_CREATE\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_WRITER_CREATE_EC (DDSC_LOG_BASE + 1306)
```

Failed to create writer for a Builtin IPC channel.

**DDSC\_LOG\_IPC\_CHANNEL\_TYPE\_PLUGIN\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_TYPE_PLUGIN_EC (DDSC_LOG_BASE + 1307)
```

Failed to find type plugin for a Builtin IPC channel.

**DDSC\_LOG\_IPC\_CHANNEL\_UNKNOWN\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_UNKNOWN_EC (DDSC_LOG_BASE + 1308)
```

Unknown builtin IPC channel requested.

**DDSC\_LOG\_IPC\_UPDATE\_QOS\_FAILED\_EC**

```
#define DDSC_LOG_IPC_UPDATE_QOS_FAILED_EC (DDSC_LOG_BASE + 1309)
```

Failed to update DomainParticipantQos to account for built-in IPC endpoints.

**DDSC\_LOG\_IPC\_CHANNEL\_ADD\_ANON\_ROUTE\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_ADD_ANON_ROUTE_EC (DDSC_LOG_BASE + 1310)
```

Failed to add an anonymous route to a built-in DataWriter or DataReader.

**DDSC\_LOG\_IPC\_CHANNEL\_DELETE\_ANON\_ROUTE\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_DELETE_ANON_ROUTE_EC (DDSC_LOG_BASE + 1311)
```

Failed to delete an anonymous route to a built-in DataWriter or DataReader.

**DDSC\_LOG\_IPC\_CHANNEL\_ADD\_PEER\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_ADD_PEER_EC (DDSC_LOG_BASE + 1312)
```

Failed to add an inter-participant channel peer.

**DDSC\_LOG\_IPC\_CHANNEL\_REMOVE\_PEER\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_REMOVE_PEER_EC (DDSC_LOG_BASE + 1313)
```

Failed to remove an inter-participant channel peer.

**DDSC\_LOG\_IPC\_CHANNEL\_RETURN\_LOAN\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_RETURN_LOAN_EC (DDSC_LOG_BASE + 1314)
```

Failed to return a loaned inter-participant sample.

**DDSC\_LOG\_IPC\_CHANNEL\_PROCESS\_SAMPLE\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_PROCESS_SAMPLE_EC (DDSC_LOG_BASE + 1315)
```

Failed to process inter-participant message.

**DDS\_LOG\_IPC\_ASSERT\_ROUTES\_EC**

```
#define DDS_LOG_IPC_ASSERT_ROUTES_EC (DDSC_LOG_BASE + 1316)
```

Failed to assert inter-participant routes.



**DDSC\_LOG\_IPC\_CHANNEL\_WRITE\_SAMPLE\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_WRITE_SAMPLE_EC (DDSC_LOG_BASE + 1317)
```

Failed to write inter-participant message.

**DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_AUTH\_HANDSHAKE\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_HANDSHAKE_FAILED_EC (DDSC_LOG_BASE + 1318)
```

Failed to process handshake authentication.

**DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_PARTICIPANT\_INTERCEPTOR\_STATE\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1319)
```

Failed to process participant interceptor tokens.

**DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_ENDPOINT\_INTERCEPTOR\_STATE\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1320)
```

Failed to process endpoint interceptor tokens.

**DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_AUTH\_REQUEST\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_REQUEST_FAILED_EC (DDSC_LOG_BASE + 1321)
```

Failed to process authentication request.

**DDSC\_LOG\_IPC\_CHANNEL\_CONFIG\_INITIALIZE\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_CONFIG_INITIALIZE_FAILED_EC (DDSC_LOG_BASE + 1322)
```

Failed to initialize inter-participant channel.

**DDSC\_LOG\_IPC\_CHANNEL\_CONFIG\_FINALIZE\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_CONFIG_FINALIZE_FAILED_EC (DDSC_LOG_BASE + 1323)
```

Failed to finalize inter-participant channel

**DDSC\_LOG\_IPC\_CHANNEL\_CONFIG\_SET\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_CONFIG_SET_FAILED_EC (DDSC_LOG_BASE + 1324)
```

Failed to get inter-participant channel configuration.

**DDSC\_LOG\_IPC\_CHANNEL\_SET\_LISTENERS\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_SET_LISTENERS_FAILED_EC (DDSC_LOG_BASE + 1325)
```

Failed to set inter-participant channel listeners.

**DDSC\_LOG\_IPC\_CHANNEL\_ENABLE\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_ENABLE_FAILED_EC (DDSC_LOG_BASE + 1326)
```

Failed to enable inter-participant channel entities.

**DDSC\_LOG\_IPC\_CHANNEL\_CREATE\_FAILED\_EC**

```
#define DDSC_LOG_IPC_CHANNEL_CREATE_FAILED_EC (DDSC_LOG_BASE + 1327)
```

Failed to create inter-participant channel.

**DDSC\_LOG\_TRUST\_UNKNOWN\_GMCLASSID\_EC**

```
#define DDSC_LOG_TRUST_UNKNOWN_GMCLASSID_EC (DDSC_LOG_BASE + 1401)
```

Unknown generic message class id.

**DDSC\_LOG\_TRUST\_UNKNOWN\_SERVICEID\_EC**

```
#define DDSC_LOG_TRUST_UNKNOWN_SERVICEID_EC (DDSC_LOG_BASE + 1402)
```

Unknown builtin service id.

**DDSC\_LOG\_TRUST\_IGNORED\_REMOTE\_PARTICIPANT\_EC**

```
#define DDSC_LOG_TRUST_IGNORED_REMOTE_PARTICIPANT_EC (DDSC_LOG_BASE + 1403)
```

Ignored remote participant.

**DDSC\_LOG\_TRUST\_IGNORED\_HANDSHAKE\_MESSAGE\_EC**

```
#define DDSC_LOG_TRUST_IGNORED_HANDSHAKE_MESSAGE_EC (DDSC_LOG_BASE + 1404)
```

Ignored handshake message from remote participant.

**DDSC\_LOG\_TRUST\_UNEXPECTED\_VALIDATION\_RESULT\_EC**

```
#define DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC (DDSC_LOG_BASE + 1405)
```

Unexpected validation result.

**DDSC\_LOG\_TRUST\_UNAUTHORIZED\_REMOTE\_PARTICIPANT\_EC**

```
#define DDSC_LOG_TRUST_UNAUTHORIZED_REMOTE_PARTICIPANT_EC (DDSC_LOG_BASE + 1406)
```

A remote participant failed the authorization process.

**DDSC\_LOG\_TRUST\_FAILED\_VALIDATE\_LOCAL\_IDENTITY\_EC**

```
#define DDSC_LOG_TRUST_FAILED_VALIDATE_LOCAL_IDENTITY_EC (DDSC_LOG_BASE + 1407)
```

Failed to validate local identity.

**DDSC\_LOG\_TRUST\_INVALID\_IDENTITY\_HANDLE\_EC**

```
#define DDSC_LOG_TRUST_INVALID_IDENTITY_HANDLE_EC (DDSC_LOG_BASE + 1408)
```

Invalid local trust identity.

**DDSC\_LOG\_TRUST\_GET\_IDENTITY\_TOKEN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_IDENTITY_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1409)
```

Failed to get identity token.

**DDSC\_LOG\_TRUST\_INVALID\_PERMISSIONS\_HANDLE\_EC**

```
#define DDSC_LOG_TRUST_INVALID_PERMISSIONS_HANDLE_EC (DDSC_LOG_BASE + 1410)
```

Invalid local trust permissions.

**DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_GUID\_EC**

```
#define DDSC_LOG_TRUST_INVALID_PARTICIPANT_GUID_EC (DDSC_LOG_BASE + 1411)
```

Invalid DomainParticipant GUID configuration.

**DDSC\_LOG\_TRUST\_GET\_PARTICIPANT\_TRUST\_ATTRIBUTES\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_PARTICIPANT_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1412)
```

Failed to get DomainParticipant trust attributes.

**DDSC\_LOG\_TRUST\_CHECK\_CREATE\_PARTICIPANT\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CHECK_CREATE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1413)
```

Failed to create access control DomainParticipant.

**DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1414)
```

Failed to get permissions credential token.

**DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_TOKEN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_PERMISSIONS_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1415)
```

Failed to get permissions token.

**DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_HANDLE\_EC**

```
#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_HANDLE_EC (DDSC_LOG_BASE + 1416)
```

Invalid interceptor handle.

**DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_PARTICIPANT\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_REGISTER_LOCAL_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1417)
```

Failed to register local DomainParticipant.

**DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1418)
```

Failed to return permissions credential token.

**DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_TOKEN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_PERMISSIONS_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1419)
```

Failed to return permissions token.

**DDSC\_LOG\_TRUST\_RETURN\_IDENTITY\_TOKEN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_IDENTITY_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1420)
```

Failed to return identity token.

**DDSC\_LOG\_TRUST\_UNREGISTER\_PARTICIPANT\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_UNREGISTER_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1421)
```

Failed to unregister local participant.

**DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_HANDLE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_PERMISSIONS_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1422)
```

Failed to return permissions handle.

**DDSC\_LOG\_TRUST\_RETURN\_IDENTITY\_HANDLE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_IDENTITY_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1423)
```

Failed to return identity handle.

**DDSC\_LOG\_TRUST\_RETURN\_SHAREDSECRET\_HANDLE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_SHAREDSECRET_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1424)
```

Failed to return shared secret handle.

**DDSC\_LOG\_TRUST\_GET\_AUTHENTICATED\_PEER\_CREDENTIAL\_TOKEN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1425)
```

Failed to get authenticated peer credential token.

**DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_PARTICIPANT\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CHECK_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1426)
```

Failed to check access control remote DomainParticipant.

**DDSC\_LOG\_TRUST\_GET\_SHARED\_SECRET\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_SHARED_SECRET_FAILED_EC (DDSC_LOG_BASE + 1427)
```

Failed to get shared secret.

**DDSC\_LOG\_TRUST\_PREPARE\_LOCAL\_PARTICIPANT\_STATE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_PREPARE_LOCAL_PARTICIPANT_STATE_FAILED_EC (DDSC_LOG_BASE + 1428)
```

Failed to validate local DomainParticipant trust.

**DDSC\_LOG\_TRUST\_DELETE\_TRUST\_PLUGINS\_EC**

```
#define DDSC_LOG_TRUST_DELETE_TRUST_PLUGINS_EC (DDSC_LOG_BASE + 1429)
```

Failed to delete trust plugins.

**DDSC\_LOG\_TRUST\_SET\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1430)
```

Failed to set permissions credential and token.

**DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_PARTICIPANT\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1431)
```

Failed to register matched remote DomainParticipant.

**DDSC\_LOG\_TRUST\_GET\_TOPIC\_TRUST\_ATTRIBUTES\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_TOPIC_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1432)
```

Failed to get topic trust attributes.

**DDSC\_LOG\_TRUST\_CHECK\_CREATE\_DATAWRITER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CHECK_CREATE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1433)
```

Failed to create trust check DataWriter.

**DDSC\_LOG\_TRUST\_RETURN\_DATAWRITER\_TRUST\_ATTRIBUTES\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1434)
```

Failed to return DataWriter trust attributes.

**DDSC\_LOG\_TRUST\_GET\_DATAWRITER\_TRUST\_ATTRIBUTES\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1435)
```

Failed to get DataWriter trust attributes.

**DDSC\_LOG\_TRUST\_GET\_DATAREADER\_TRUST\_ATTRIBUTES\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_DATAREADER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1436)
```

Failed to get DataReader trust attributes.

**DDSC\_LOG\_TRUST\_CHECK\_CREATE\_DATAREADER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CHECK_CREATE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1437)
```

Failed to create trust check DataReader.

**DDSC\_LOG\_TRUST\_BEGIN\_HANDSHAKE\_REPLY\_EC**

```
#define DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REPLY_EC (DDSC_LOG_BASE + 1438)
```

Handshake begin reply failed.

**DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_PARTICIPANT\_INTERCEPTOR\_TOKENS\_EC**

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_PARTICIPANT_INTERCEPTOR_TOKENS_EC (DDSC_LOG_BASE + 1439)
```

Failed to create local DomainParticipant interceptor tokens.

**DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DW\_TOKEN\_EC**

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_DW_TOKEN_EC (DDSC_LOG_BASE + 1440)
```

Failed to create local DataWriter interceptor token.

**DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DR\_TOKEN\_EC**

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_DR_TOKEN_EC (DDSC_LOG_BASE + 1441)
```

Failed to create local DataReader interceptor token.

**DDSC\_LOG\_TRUST\_RETURN\_HANDSHAKE\_HANDLE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_HANDSHAKE_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1442)
```

Failed to return handshake handle.

**DDSC\_LOG\_TRUST\_SET\_REMOTE\_PARTICIPANT\_INTERCEPTOR\_TOKENS\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SET_REMOTE_PARTICIPANT_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1443)
```

Failed to set remote DomainParticipant interceptor tokens.

**DDSC\_LOG\_TRUST\_SET\_REMOTE\_DATAREADER\_INTERCEPTOR\_TOKENS\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SET_REMOTE_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1444)
```

Failed to set remote DataReader interceptor tokens.

**DDSC\_LOG\_TRUST\_SET\_REMOTE\_DATAWRITER\_INTERCEPTOR\_TOKENS\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SET_REMOTE_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1445)
```

Failed to set remote DataWriter interceptor tokens.



**DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_DATAREADER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_REGISTER_LOCAL_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1446)
```

Failed to register local DataReader.

**DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_DATAWRITER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_REGISTER_LOCAL_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1447)
```

Failed to register local DataWriter.

**DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_DATAWRITER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CHECK_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1448)
```

Failed to check remote DataWriter.

**DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_DATAREADER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CHECK_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1449)
```

Failed to check remote DataReader.

**DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_DATAWRITER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1450)
```

Failed to register matched remote DataWriter.

**DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_DATAREADER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1451)
```

Failed to register matched remote DataReader.

**DDSC\_LOG\_TRUST\_UNREGISTER\_DATAREADER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_UNREGISTER_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1452)
```

Failed to unregister DataReader.

**DDSC\_LOG\_TRUST\_UNREGISTER\_DATAWRITER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_UNREGISTER_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1453)
```

Failed to unregister DataWriter.

**DDSC\_LOG\_TRUST\_VALIDATE\_REMOTE\_IDENTITY\_EC**

```
#define DDSC_LOG_TRUST_VALIDATE_REMOTE_IDENTITY_EC (DDSC_LOG_BASE + 1454)
```

Failed to validate remote identity.

**DDSC\_LOG\_TRUST\_BEGIN\_HANDSHAKE\_REQUEST\_EC**

```
#define DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REQUEST_EC (DDSC_LOG_BASE + 1455)
```

Handshake begin request failed.

**DDSC\_LOG\_TRUST\_PROCESS\_HANDSHAKE\_EC**

```
#define DDSC_LOG_TRUST_PROCESS_HANDSHAKE_EC (DDSC_LOG_BASE + 1456)
```

Handshake process failed.

**DDSC\_LOG\_TRUST\_CREATE\_TRUST\_PLUGINS\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CREATE_TRUST_PLUGINS_FAILED_EC (DDSC_LOG_BASE + 1457)
```

Failed to create trust plugins.

**DDSC\_LOG\_TRUST\_PARSE\_PROPERTY\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_PARSE_PROPERTY_FAILED_EC (DDSC_LOG_BASE + 1458)
```

Failed to parse trust property.

**DDSC\_LOG\_TRUST\_AFTER\_REMOTE\_PARTICIPANT\_READY\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_READY_FAILED_EC (DDSC_LOG_BASE + 1459)
```

Error after remote trust DomainParticipant is ready.

**DDSC\_LOG\_TRUST\_REMOVE\_REMOTE\_PARTICIPANT\_EC**

```
#define DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_EC (DDSC_LOG_BASE + 1460)
```

Failed to remove remote DomainParticipant.

**DDS\_LOG\_TRUST\_ASSERT\_PARTICIPANT\_GENERIC\_MESSAGE\_FAILED\_EC**

```
#define DDS_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1461)
```

Failed to assert DomainParticipant generic message.

**DDSC\_LOG\_TRUST\_SEND\_HANDSHAKE\_EC**

```
#define DDSC_LOG_TRUST_SEND_HANDSHAKE_EC (DDSC_LOG_BASE + 1462)
```

Failed to send handshake message.

**DDSC\_LOG\_VALIDATE\_REMOTE\_PARTICIPANT\_TRUST\_FAILED\_EC**

```
#define DDSC_LOG_VALIDATE_REMOTE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1463)
```

Failed to validate trust remote DomainParticipant.

**DDSC\_LOG\_TRUST\_BIND\_REMOTE\_DATAREADER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_BIND_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1464)
```

Failed to get remote DataReader bind properties.

**DDSC\_LOG\_TRUST\_REGISTER\_REMOTE\_DATAREADER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1465)
```

Failed to register remote DataReader.

**DDSC\_LOG\_TRUST\_BIND\_REMOTE\_DATAWRITER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_BIND_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1466)
```

Failed to get remote DataWriter bind properties.

**DDSC\_LOG\_TRUST\_REGISTER\_REMOTE\_DATAWRITER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1467)
```

Failed to register remote DataWriter.

**DDSC\_LOG\_TRUST\_VALIDATE\_REMOTE\_DATAWRITER\_TRUST\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAWRITER_TRUST_FAILED_EC (DDSC_LOG_BASE + 1468)
```

Failed to validate remote DataWriter.

**DDSC\_LOG\_TRUST\_VALIDATE\_REMOTE\_DATAREADER\_TRUST\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAREADER_TRUST_FAILED_EC (DDSC_LOG_BASE + 1469)
```

Failed to validate remote DataReader.

**DDSC\_LOG\_TRUST\_VALIDATE\_LOCAL\_DATAWRITER\_TRUST\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_VALIDATE_LOCAL_DATAWRITER_TRUST_FAILED_EC (DDSC_LOG_BASE + 1470)
```

Failed to validate local DataWriter.

**DDSC\_TRUST\_INVALID\_PARTICIPANT\_GENERIC\_MESSAGE\_EC**

```
#define DDSC_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC (DDSC_LOG_BASE + 1471)
```

Invalid DomainParticipant trust generic message.

**DDSC\_LOG\_TRUST\_PREPARE\_AUTH\_REQUEST\_EC**

```
#define DDSC_LOG_TRUST_PREPARE_AUTH_REQUEST_EC (DDSC_LOG_BASE + 1472)
```

Failed to prepare authentication request.

**DDSC\_LOG\_TRUST\_PREPARE\_AUTH\_HANDSHAKE\_MESSAGE\_EC**

```
#define DDSC_LOG_TRUST_PREPARE_AUTH_HANDSHAKE_MESSAGE_EC (DDSC_LOG_BASE + 1473)
```

Failed to prepare authentication handshake message.

**DDSC\_LOG\_TRUST\_PREPARE\_ENDPOINT\_INTERCEPTOR\_MESSAGE\_EC**

```
#define DDSC_LOG_TRUST_PREPARE_ENDPOINT_INTERCEPTOR_MESSAGE_EC (DDSC_LOG_BASE + 1474)
```

Failed to prepare endpoint interceptor message.

**DDSC\_LOG\_TRUST\_INVALID\_GENERIC\_MESSAGE\_REPLY\_EC**

```
#define DDSC_LOG_TRUST_INVALID_GENERIC_MESSAGE_REPLY_EC (DDSC_LOG_BASE + 1475)
```

Invalid trust generic message reply.

**DDSC\_LOG\_FINALIZE\_TRUST\_FAILED\_EC**

```
#define DDSC_LOG_FINALIZE_TRUST_FAILED_EC (DDSC_LOG_BASE + 1476)
```

Failed to finalize trust.

**DDSC\_LOG\_TRUST\_DATA HOLDER\_COPY\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_DATA_HOLDER_COPY_FAILED_EC (DDSC_LOG_BASE + 1477)
```

Failed to copy trust data holder.

**DDSC\_LOG\_TRUST\_INVALID\_SAMPLE\_MAX\_SERIALIZED\_SIZE\_EC**

```
#define DDSC_LOG_TRUST_INVALID_SAMPLE_MAX_SERIALIZED_SIZE_EC (DDSC_LOG_BASE + 1478)
```

Invalid sample serialized size.

**DDSC\_LOG\_TRUST\_ALLOC\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_ALLOC_FAILED_EC (DDSC_LOG_BASE + 1479)
```

Failed to allocate memory.

**DDSC\_LOG\_TRUST\_SERIALIZE\_DATA\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SERIALIZE_DATA_FAILED_EC (DDSC_LOG_BASE + 1480)
```

Failed to serialize trust data.

**DDSC\_LOG\_TRUST\_INITIALIZE\_PARTICIPANT\_TRUST\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1481)
```

Failed to initialize trust.

**DDSC\_LOG\_TRUST\_INITIALIZE\_PARTICIPANT\_BUILTIN\_DATA\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_BUILTIN_DATA_FAILED_EC (DDSC_LOG_BASE + 1482)
```

Failed to initialize DomainParticipant trust data.

**DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_GENERIC\_MESSAGE\_EC**

```
#define DDSC_LOG_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC (DDSC_LOG_BASE + 1483)
```

Invalid DomainParticipant trust generic message.

**DDSC\_LOG\_TRUST\_ADVANCE\_SN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_ADVANCE_SN_FAILED_EC (DDSC_LOG_BASE + 1484)
```

Failed to advance trust sequence number.

**DDSC\_LOG\_TRUST\_WRITE\_SAMPLE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_WRITE_SAMPLE_FAILED_EC (DDSC_LOG_BASE + 1485)
```

Failed to write sample.

**DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_IN\_PROGRESS\_EC**

```
#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_IN_PROGRESS_EC (DDSC_LOG_BASE + 1486)
```

Failed to resend handshake because it is already in progress.

**DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_STOP\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_STOP_FAILED_EC (DDSC_LOG_BASE + 1487)
```

Failed to stop periodic handshake.

**DDSC\_LOG\_TRUST\_RELEASE\_PARTICIPANT\_GENERIC\_MESSAGE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RELEASE_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1488)
```

Failed to release DomainParticipant trust generic message.

**DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_MESSAGE\_START\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_START_FAILED_EC (DDSC_LOG_BASE + 1489)
```

Failed to start periodic handshake.

**DDSC\_LOG\_TRUST\_UNEXPECTED\_REMOTE\_PARTICIPANT\_STATUS\_EC**

```
#define DDSC_LOG_TRUST_UNEXPECTED_REMOTE_PARTICIPANT_STATUS_EC (DDSC_LOG_BASE + 1490)
```

Invalid remote DomainParticipant trust status.

**DDSC\_LOG\_TRUST\_SEND\_PARTICIPANT\_INTERCEPTOR\_STATE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SEND_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1491)
```

Fail to send DomainParticipant interceptor state.

**DDSC\_LOG\_TRUST\_PARTICIPANT\_AUTHENTICATION\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_PARTICIPANT_AUTHENTICATION_FAILED_EC (DDSC_LOG_BASE + 1492)
```

DomainParticipant authentication failed.

**DDSC\_LOG\_TRUST\_ASSERT\_PARTICIPANT\_GENERIC\_MESSAGE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1493)
```

Failed to assert DomainParticipant generic message.

**DDSC\_LOG\_TRUST\_SEND\_HANDSHAKE\_MESSAGE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1494)
```

Failed to send handshake message.

**DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_TOKENS\_EC**

```
#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_EC (DDSC_LOG_BASE + 1495)
```

Invalid interceptor tokens.

**DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_TOKENS\_DESTINATION\_EC**

```
#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_DESTINATION_EC (DDSC_LOG_BASE + 1496)
```

Invalid interceptor token destination.

**DDSC\_LOG\_TRUST\_ENDPOINT\_INTERNAL\_ATTRIBUTES\_CREATE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC (DDSC_LOG_BASE + 1497)
```

Failed to copy endpoint trust attributes.

**DDSC\_LOG\_TRUST\_SEND\_ENDPOINT\_INTERCEPTOR\_STATE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1498)
```

Failed to send tokens to remote DomainParticipant.

**DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DATAWRITER\_INTERCEPTOR\_TOKENS\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1499)
```

Failed to create tokens for the remote DataReader.

**DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DATAREADER\_INTERCEPTOR\_TOKENS\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CREATE_LOCAL_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1500)
```

Failed to create tokens for the remote DataWriter.

**DDSC\_LOG\_TRUST\_RETURN\_INTERCEPTOR\_TOKENS\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1501)
```

Failed to return interceptor tokens.



**DDSC\_LOG\_TRUST\_RETURN\_AUTHENTICATED\_PEER\_CREDENTIAL\_TOKEN\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1502)
```

Failed to reutrn authenticated peer credential token.

**DDSC\_LOG\_TRUST\_PROCESS\_REMOTE\_INTERCEPTOR\_TOKENS\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_PROCESS_REMOTE_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1503)
```

Failed to process interceptor tokens.

**DDSC\_LOG\_TRUST\_UNREGISTER\_REMOTE\_DATAWRITER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1504)
```

Failed to unregister remote DataWriter.

**DDSC\_LOG\_TRUST\_UNREGISTER\_REMOTE\_DATAREADER\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1505)
```

Failed to unregister remote DataReader.

**DDSC\_LOG\_TRUST\_INVALID\_SHARED\_SECRET\_EC**

```
#define DDSC_LOG_TRUST_INVALID_SHARED_SECRET_EC (DDSC_LOG_BASE + 1506)
```

Invalid shared secret.

**DDSC\_LOG\_TRUST\_PARTICIPANT\_UNEXPECTED\_AUTHENTICATION\_ERROR\_EC**

```
#define DDSC_LOG_TRUST_PARTICIPANT_UNEXPECTED_AUTHENTICATION_ERROR_EC (DDSC_LOG_BASE + 1507)
```

Unexpected DomainParticipant authentication error.

**DDSC\_LOG\_TRUST\_PREPARE\_LOCAL\_DP\_TOKENS\_EC**

```
#define DDSC_LOG_TRUST_PREPARE_LOCAL_DP_TOKENS_EC (DDSC_LOG_BASE + 1508)
```

Failed to prepare local DomainParticipant interceptor tokens.

**DDSC\_LOG\_TRUST\_ASSERT\_PROPERTY\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_ASSERT_PROPERTY_FAILED_EC (DDSC_LOG_BASE + 1509)
```

Failed to assert trust property.

**DDSC\_LOG\_TRUST\_PARTICIPANT\_INTERNAL\_ATTRIBUTES\_CREATE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_PARTICIPANT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC (DDSC_LOG_BASE + 1510)
```

Failed to copy DomainParticipant trust attributes.

**DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_INTERNAL\_ATTRIBUTES\_EC**

```
#define DDSC_LOG_TRUST_INVALID_PARTICIPANT_INTERNAL_ATTRIBUTES_EC (DDSC_LOG_BASE + 1511)
```

Invalid DomainParticipant trust attributes.

**DDSC\_LOG\_TRUST\_TRANSFORM\_OUTGOING\_SERIALIZED\_PAYLOAD\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_TRANSFORM_OUTGOING_SERIALIZED_PAYLOAD_FAILED_EC (DDSC_LOG_BASE + 1512)
```

Failed to encode outgoing serialized payload.

**DDSC\_LOG\_TRUST\_TRANSFORM\_INCOMING\_SERIALIZED\_PAYLOAD\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_TRANSFORM_INCOMING_SERIALIZED_PAYLOAD_FAILED_EC (DDSC_LOG_BASE + 1513)
```

Failed to decode incoming serialized payload.

**DDSC\_LOG\_TRUST\_FINALIZE\_AC\_PLUGIN\_PROPERTIES\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_FINALIZE_AC_PLUGIN_PROPERTIES_FAILED_EC (DDSC_LOG_BASE + 1514)
```

Failed to finalize trust properties.

**DDSC\_LOG\_TRUST\_AFTER\_REMOTE\_PARTICIPANT\_AUTHENTICATED\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_AUTHENTICATED_FAILED_EC (DDSC_LOG_BASE + 1515)
```

Error after remote trust DomainParticipant is authenticated.

**DDSC\_LOG\_TRUST\_CACHE\_REMOTE\_PARTICIPANT\_SAMPLE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_CACHE_REMOTE_PARTICIPANT_SAMPLE_FAILED_EC (DDSC_LOG_BASE + 1516)
```

Failed to copy remote DomainParticipant sample.

**DDSC\_LOG\_TRUST\_FINALIZE\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_FINALIZE_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1517)
```

Failed to finalize authentication handshake.

**DDSC\_LOG\_TRUST\_FINALIZE\_REMOTE\_PARTICIPANT\_TRUST\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_FINALIZE_REMOTE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1518)
```

Failed to finalize remote trust DomainParticipant record.

**DDSC\_LOG\_TRUST\_RESET\_REMOTE\_PARTICIPANT\_TRUST\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RESET_REMOTE_PARTICIPANT_TRUST_FAILED_EC (DDSC_LOG_BASE + 1519)
```

Failed to finalize remote trust DomainParticipant.

**DDSC\_LOG\_TRUST\_ASSERT\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_ASSERT_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1520)
```

Failed to assert authentication handshake state.

**DDSC\_LOG\_TRUST\_INVALID\_AUTH\_HANDSHAKE\_STATE\_EC**

```
#define DDSC_LOG_TRUST_INVALID_AUTH_HANDSHAKE_STATE_EC (DDSC_LOG_BASE + 1521)
```

Invalid authentication handshake state.

**DDSC\_LOG\_TRUST\_SEND\_AUTH\_REQUEST\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SEND_AUTH_REQUEST_FAILED_EC (DDSC_LOG_BASE + 1522)
```

Failed to send authentication request.

**DDSC\_LOG\_TRUST\_GET\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_GET_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1523)
```

Failed to get authentication handshake state.

**DDSC\_LOG\_TRUST\_SET\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_SET_AUTH_HANDSHAKE_STATE_FAILED_EC (DDSC_LOG_BASE + 1524)
```

Failed to set authentication handshake state.

**DDSC\_LOG\_TRUST\_REAUTH\_REMOTE\_PARTICIPANT\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_REAUTH_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1525)
```

Failed to reauthenticate a remote DomainParticipant.

**DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_MESSAGE\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1526)
```

Failed to resend handshake message.

**DDSC\_LOG\_TRUST\_PBUFS\_MAX\_EXCEEDED\_EC**

```
#define DDSC_LOG_TRUST_PBUFS_MAX_EXCEEDED_EC (DDSC_LOG_BASE + 1527)
```

Maximum number of buffers exceeded.

**DDSC\_LOG\_TRUST\_CREATE\_TRUST\_PLUGIN\_EC**

```
#define DDSC_LOG_TRUST_CREATE_TRUST_PLUGIN_EC (DDSC_LOG_BASE + 1528)
```

Failed to create trust plugin.

**DDSC\_LOG\_TRUST\_LOOKUP\_FACTORY\_FAILED\_EC**

```
#define DDSC_LOG_TRUST_LOOKUP_FACTORY_FAILED_EC (DDSC_LOG_BASE + 1529)
```

Failed to lookup trust plugin factory.

**DDSC\_LOG\_NAME\_LOOKUP\_EC**

```
#define DDSC_LOG_NAME_LOOKUP_EC (DDSC_LOG_BASE + 1700)
```

Fully qualified name does not contain substring "::".

**DDSC\_LOG\_ENTITY\_NAME\_TOO\_LONG\_EC**

```
#define DDSC_LOG_ENTITY_NAME_TOO_LONG_EC (DDSC_LOG_BASE + 1701)
```

Failed to get an entity because the specified name exceeds the maximum length of 255 octets (excluding the terminating NUL)

**DDSC\_LOG\_MESSAGE\_DATA\_UNKNOWN\_LIVELINESS\_KIND\_EC**

```
#define DDSC_LOG_MESSAGE_DATA_UNKNOWN_LIVELINESS_KIND_EC (DDSC_LOG_BASE + 1800)
```

Unknown inter-participant message liveliness kind received.

**DDSC\_LOG\_MESSAGE\_DATA\_WRONG\_LIVELINESS\_KIND\_EC**

```
#define DDSC_LOG_MESSAGE_DATA_WRONG_LIVELINESS_KIND_EC (DDSC_LOG_BASE + 1801)
```

Wrong inter-participant message liveliness kind received.

**DDSC\_LOG\_UPDATE\_LEASE\_DURATION\_TIMER\_EC**

```
#define DDSC_LOG_UPDATE_LEASE_DURATION_TIMER_EC (DDSC_LOG_BASE + 1802)
```

Failed to update DataWriter lease duration.

**DDSC\_LOG\_PARTMESSAGE\_WRITE\_SAMPLE\_FAILED\_EC**

```
#define DDSC_LOG_PARTMESSAGE_WRITE_SAMPLE_FAILED_EC (DDSC_LOG_BASE + 1803)
```

Failed to write inter-participant liveliness message.

**DDSC\_LOG\_SEND\_LIVELINESS\_FAILED\_EC**

```
#define DDSC_LOG_SEND_LIVELINESS_FAILED_EC (DDSC_LOG_BASE + 1804)
```

Failed to send inter-participant liveliness message.

**DDSC\_LOG\_UPDATE\_LIVELINESS\_FAILED\_EC**

```
#define DDSC_LOG_UPDATE_LIVELINESS_FAILED_EC (DDSC_LOG_BASE + 1805)
```

Failed to update DataWriter liveliness.

**DDSC\_LOG\_REFRESH\_LIVELINESS\_FAILED\_EC**

```
#define DDSC_LOG_REFRESH_LIVELINESS_FAILED_EC (DDSC_LOG_BASE + 1806)
```

Failed to refresh endpoint liveliness.

**DDSC\_LOG\_TRUST\_INCOMPATIBLE\_ENDPOINT\_TRUST\_ATTRIBUTES\_EC**

```
#define DDSC_LOG_TRUST_INCOMPATIBLE_ENDPOINT_TRUST_ATTRIBUTES_EC (DDSC_LOG_BASE + 1807)
```

Two security endpoints are incompatible.

**DDSC\_LOG\_FLOW\_CONTROLLER\_CREATE\_EC**

```
#define DDSC_LOG_FLOW_CONTROLLER_CREATE_EC (DDSC_LOG_BASE + 1900)
```

Failed to create flow controller.

**DDSC\_LOG\_FLOW\_CONTROLLER\_DELETE\_EC**

```
#define DDSC_LOG_FLOW_CONTROLLER_DELETE_EC (DDSC_LOG_BASE + 1901)
```

Failed to delete flow controller.

**DDSC\_LOG\_XTYPES\_LOANED\_SAMPLE\_STATE\_ERROR\_EC**

```
#define DDSC_LOG_XTYPES_LOANED_SAMPLE_STATE_ERROR_EC (DDSC_LOG_BASE + 2000)
```

Extensible types sample state error.

**DDSC\_LOG\_INTERPRETER\_SUPPORT\_UNEXPECTED\_ERROR\_EC**

```
#define DDSC_LOG_INTERPRETER_SUPPORT_UNEXPECTED_ERROR_EC (DDSC_LOG_BASE + 2001)
```

Interpreter unexpected error.

**DDSC\_LOG\_MEMORY\_MANAGER\_OUT\_OF\_RESOURCES\_EC**

```
#define DDSC_LOG_MEMORY_MANAGER_OUT_OF_RESOURCES_EC (DDSC_LOG_BASE + 2002)
```

Memory manager out of resources.

**DDSC\_LOG\_MEMORY\_MANAGER\_NOT\_OWNER\_EC**

```
#define DDSC_LOG_MEMORY_MANAGER_NOT_OWNER_EC (DDSC_LOG_BASE + 2003)
```

Memory manager is not the owner of the data passed in.

**DDSC\_LOG\_MEMORY\_MANAGER\_DELETE\_EC**

```
#define DDSC_LOG_MEMORY_MANAGER_DELETE_EC (DDSC_LOG_BASE + 2004)
```

The wire manager was not able to delete a resource.

**DDSC\_LOG\_BUFFER\_MAX\_EXCEEDED\_EC**

```
#define DDSC_LOG_BUFFER_MAX_EXCEEDED_EC (DDSC_LOG_BASE + 2101)
```

Mamimum size of buffer excedeed.

**DDSC\_LOG\_INVALID\_BUFFER\_LENGTH\_EC**

```
#define DDSC_LOG_INVALID_BUFFER_LENGTH_EC (DDSC_LOG_BASE + 2102)
```

Invalid buffer length.

**DDSC\_LOG\_INCOMPATIBLE\_PRESENTATION\_QOS\_EC**

```
#define DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC (DDSC_LOG_BASE + 2201)
```

Matching error because Presentation QoS is not compatible.

**DDSC\_LOG\_INCOMPATIBLE\_PARTITION\_QOS\_EC**

```
#define DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC (DDSC_LOG_BASE + 2202)
```

Two endpoints did not match because of incompatible partition QoS.

**DDSC\_LOG\_CHECKSUM\_REQUIRED\_EC**

```
#define DDSC_LOG_CHECKSUM_REQUIRED_EC (DDSC_LOG_BASE + 2301)
```

A participant requires checksum, but the discovered participant does not send a checksum.

**DDSC\_LOG\_CHECKSUM\_INCOMPATIBLE\_EC**

```
#define DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC (DDSC_LOG_BASE + 2302)
```

A participant does not support checksum's sent by a discovered participant.

**DDSC\_LOG\_CHECKSUM\_INCONSISTENT\_CLASS\_EC**

```
#define DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC (DDSC_LOG_BASE + 2303)
```

The checksum class id for a custom checksum does match.

**DDSC\_LOG\_CHECKSUM\_INCONSISTENT\_COMPUTE\_EC**

```
#define DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC (DDSC_LOG_BASE + 2304)
```

The computed\_crc\_kind is inconsistent with what is allowed or supported.

**DDSC\_LOG\_CHECKSUM\_INCONSISTENT\_ALLOW\_EC**

```
#define DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC (DDSC_LOG_BASE + 2305)
```

The allowed\_crc\_mask is inconsistent with what is supported.

**DDSC\_LOG\_FLOWCONTROLLER\_INCONSISTENT\_PROP\_EC**

```
#define DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROP_EC (DDSC_LOG_BASE + 2306)
```

The token bucket properties for the flow controller are inconsistent.

**DDSC\_LOG\_LOCK\_EC**

```
#define DDSC_LOG_LOCK_EC (DDSC_LOG_BASE + 2401)
```

Failed to take a lock.



**DDSC\_LOG\_UNLOCK\_EC**

```
#define DDSC_LOG_UNLOCK_EC (DDSC_LOG_BASE + 2402)
```

Failed to give a lock.

**DDSC\_LOG\_FILTER\_PLUGIN\_FACTORY\_NOT\_FOUND\_EC**

```
#define DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC (DDSC_LOG_BASE + 2501)
```

Failed to find the filter plugin factory specified in a DomainParticipant's QoS.

**DDSC\_LOG\_ENABLE\_WRITER\_FILTERING\_EC**

```
#define DDSC_LOG_ENABLE_WRITER_FILTERING_EC (DDSC_LOG_BASE + 2502)
```

A local writer failed to enable writer filtering for a reader.

**DDSC\_LOG\_CREATE\_WRITER\_FILTER\_EC**

```
#define DDSC_LOG_CREATE_WRITER_FILTER_EC (DDSC_LOG_BASE + 2503)
```

A local writer failed to create its writer filter.

**DDSC\_LOG\_EVALUATE\_WRITER\_FILTER\_EC**

```
#define DDSC_LOG_EVALUATE_WRITER_FILTER_EC (DDSC_LOG_BASE + 2504)
```

A local writer failed to evaluate a sample against the filters of its matched readers.

**DDSC\_LOG\_APPLY\_READER\_FILTER\_EC**

```
#define DDSC_LOG_APPLY_READER_FILTER_EC (DDSC_LOG_BASE + 2505)
```

A local writer failed to check if a reader had filtered a sample.

**DDSC\_LOG\_COMPILE\_CONTENT\_FILTER\_EC**

```
#define DDSC_LOG_COMPILE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2506)
```

Failed to compile the content filter configured in a DataReader's QoS.

**DDSC\_LOG\_CONTENT\_FILTERING\_NOT\_ENABLED\_EC**

```
#define DDSC_LOG_CONTENT_FILTERING_NOT_ENABLED_EC (DDSC_LOG_BASE + 2507)
```

Reader configured a content filter but content filtering is not enabled.

**DDSC\_LOG\_EVALUATE\_CONTENT\_FILTER\_EC**

```
#define DDSC_LOG_EVALUATE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2508)
```

Failed to evaluate a sample against a reader's content filter.

**DDSC\_LOG\_DESERIALIZE\_CONTENT\_FILTER\_INFO\_EC**

```
#define DDSC_LOG_DESERIALIZE_CONTENT_FILTER_INFO_EC (DDSC_LOG_BASE + 2509)
```

Failed to deserialize the content filter info from the in-line QoS of a sample.

**DDSC\_LOG\_CHANGE\_CONTENT\_FILTER\_EC**

```
#define DDSC_LOG_CHANGE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2510)
```

Failed to change the configured content filter of a DataReader.

**DDSC\_LOG\_UPDATE\_REMOTE\_CONTENT\_FILTER\_EC**

```
#define DDSC_LOG_UPDATE_REMOTE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2511)
```

Failed to update the content filter of a remote subscription.

**DDSC\_LOG\_CREATE\_FILTER\_PLUGIN\_EC**

```
#define DDSC_LOG_CREATE_FILTER_PLUGIN_EC (DDSC_LOG_BASE + 2512)
```

DomainParticipant failed to create its filter plugin.

**DDSC\_LOG\_FINALIZE\_FILTER\_PLUGIN\_EC**

```
#define DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC (DDSC_LOG_BASE + 2513)
```

DomainParticipant failed to finalize its filter plugin.

## DDSC\_LOG\_DESERIALIZE\_REMOTE\_CONTENT\_FILTER\_EC

```
#define DDSC_LOG_DESERIALIZE_REMOTE_CONTENT_FILTER_EC (DDSC_LOG_BASE + 2514)
```

Failed to deserialize the content filter property in the subscription built-in topic data for a remote subscription.

This warning is expected if the remote subscription has configured a content filter which exceeds the resource limits of the local participant. No writers on this participant will be able perform writer filtering for this remote subscription. The resource limits can be increased to allow the filter to be stored locally if writer filtering is desired.

### 6.21.10 WHSQ

Writer History Shared Queue. ModuleID = 22.

#### Macros

- #define [WHSQ\\_LOG\\_KEEP\\_ALL\\_HISTORY\\_NOT\\_SUPPORTED\\_EC](#) (WHSQ\_LOG\_BASE + 1)  
*KEEP\_ALL History kind is not supported.*
- #define [WHSQ\\_LOG\\_UNLIMITED\\_HISTORY\\_NOT\\_SUPPORTED\\_EC](#) (WHSQ\_LOG\_BASE + 2)  
*Unlimited length resource limits unsupported.*
- #define [WHSQ\\_LOG\\_MAX\\_SAMPLES\\_TOO\\_SMALL\\_EC](#) (WHSQ\_LOG\_BASE + 3)  
*DataWriterQos.resource\_limits.max\_samples set too small.*
- #define [WHSQ\\_LOG\\_OBJECT\\_ALLOCATE\\_EC](#) (WHSQ\_LOG\_BASE + 4)  
*Failed to allocate object of the specified kind.*
- #define [WHSQ\\_LOG\\_PURGE\\_FAILED\\_EC](#) (WHSQ\_LOG\_BASE + 5)  
*Failed to purge a sample from the shared queue.*
- #define [WHSQ\\_LOG\\_WH\\_CREATE\\_FAILED\\_EC](#) (WHSQ\_LOG\_BASE + 6)  
*Failed to create the underlying writer history.*
- #define [WHSQ\\_LOG\\_OBJECT\\_DELETE\\_EC](#) (WHSQ\_LOG\_BASE + 7)  
*Failed to delete object of the specified kind.*
- #define [WHSQ\\_LOG\\_CONFIGURATION\\_EC](#) (WHSQ\_LOG\_BASE + 8)  
*Listener or Property not configured when creating the component.*
- #define [WHSQ\\_LOG\\_NO\\_DDS\\_FACTORY\\_EC](#) (WHSQ\_LOG\_BASE + 9)  
*No factory for forwarding when creating a writer history instance.*
- #define [WHSQ\\_LOG\\_COMMIT\\_FAILED\\_EC](#) (WHSQ\_LOG\_BASE + 10)  
*Committing a Sample failed.*

#### 6.21.10.1 Detailed Description

Writer History Shared Queue. ModuleID = 22.

### 6.21.10.2 Macro Definition Documentation

#### WHSQ\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC

```
#define WHSQ_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC (WHSQ_LOG_BASE + 1)
```

KEEP\_ALL History kind is not supported.

#### WHSQ\_LOG\_UNLIMITED\_HISTORY\_NOT\_SUPPORTED\_EC

```
#define WHSQ_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC (WHSQ_LOG_BASE + 2)
```

Unlimited length resource limits unsupported.

Ensure that max\_samples, max\_samples\_per\_instance, and max\_instances resource limits are all finite values

#### WHSQ\_LOG\_MAX\_SAMPLES\_TOO\_SMALL\_EC

```
#define WHSQ_LOG_MAX_SAMPLES_TOO_SMALL_EC (WHSQ_LOG_BASE + 3)
```

DataWriterQos.resource\_limits.max\_samples set too small.

DataWriterQos.resource\_limits.max\_samples must be no less than max\_instances \* max\_samples\_per\_instance

#### WHSQ\_LOG\_OBJECT\_ALLOCATE\_EC

```
#define WHSQ_LOG_OBJECT_ALLOCATE_EC (WHSQ_LOG_BASE + 4)
```

Failed to allocate object of the specified kind.

#### WHSQ\_LOG\_PURGE\_FAILED\_EC

```
#define WHSQ_LOG_PURGE_FAILED_EC (WHSQ_LOG_BASE + 5)
```

Failed to purge a sample from the shared queue.

#### WHSQ\_LOG\_WH\_CREATE\_FAILED\_EC

```
#define WHSQ_LOG_WH_CREATE_FAILED_EC (WHSQ_LOG_BASE + 6)
```

Failed to create the underlying writer history.

**WHSQ\_LOG\_OBJECT\_DELETE\_EC**

```
#define WHSQ_LOG_OBJECT_DELETE_EC (WHSQ_LOG_BASE + 7)
```

Failed to delete object of the specified kind.

**WHSQ\_LOG\_CONFIGURATION\_EC**

```
#define WHSQ_LOG_CONFIGURATION_EC (WHSQ_LOG_BASE + 8)
```

Listener or Property not configured when creating the component.

**WHSQ\_LOG\_NO\_DDS\_FACTORY\_EC**

```
#define WHSQ_LOG_NO_DDS_FACTORY_EC (WHSQ_LOG_BASE + 9)
```

No factory for forwarding when creating a writer history instance.

**WHSQ\_LOG\_COMMIT\_FAILED\_EC**

```
#define WHSQ_LOG_COMMIT_FAILED_EC (WHSQ_LOG_BASE + 10)
```

Committing a Sample failed.

**6.21.11 RH**

Reader History. ModuleID = 8.

**Macros**

- `#define RHSM_LOG_OUTSTANDING_SAMPLE_EC (RHSM_LOG_BASE + 1)`  
*Failed to delete Reader History due to outstanding, unremoved sample.*
- `#define RHSM_LOG_NO_PROPERTY_QOS_EC (RHSM_LOG_BASE + 2)`  
*Failed to create Reader History due to unspecified property Qos.*
- `#define RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 3)`  
*KEEP\_ALL History kind is not supported.*
- `#define RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC (RHSM_LOG_BASE + 4)`  
*DataReaderQos.resource\_limits.max\_samples set too small.*
- `#define RHSM_LOG_TBF_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 5)`  
*Time-based filtering is not supported.*
- `#define RHSM_LOG_SAMPLE_INFO_POOL_EC (RHSM_LOG_BASE + 6)`  
*Failed to allocate a buffer pool for sample info.*
- `#define RHSM_LOG_SAMPLE_POOL_EC (RHSM_LOG_BASE + 7)`

- Failed to allocate a buffer pool for sample pointers.*

  - #define `RHSM_LOG_SAMPLE_PTR_ARRAY_EC` (`RHSM_LOG_BASE` + 8)
- Failed to get sample buffer.*

  - #define `RHSM_LOG_INFO_ARRAY_EC` (`RHSM_LOG_BASE` + 9)
- Failed to get sample info buffer.*

  - #define `RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC` (`RHSM_LOG_BASE` + 10)
- Failed to get time for reception timestamp.*

  - #define `RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC` (`RHSM_LOG_BASE` + 11)
- An invalid instance replacement policy was specified.*

  - #define `RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC` (`RHSM_LOG_BASE` + 12)
- Failed to remove the oldest sample.*

  - #define `RHSM_LOG_SAMPLE_POOL_EMPTY_EC` (`RHSM_LOG_BASE` + 13)
- Sample pool empty, may have exceeded.*

  - #define `RHSM_LOG_RW_PRUNE_FAILED_EC` (`RHSM_LOG_BASE` + 14)
- Failed to prune remote writer entry.*

  - #define `RHSM_LOG_ENTRY_RESERVATION_FAILED_EC` (`RHSM_LOG_BASE` + 15)
- Failed to reserve an entry.*

  - #define `RHSM_LOG_RETURN_SAMPLE_EC` (`RHSM_LOG_BASE` + 16)
- Failed to return a sample.*

  - #define `RHSM_LOG_HISTORY_UPDATE_EC` (`RHSM_LOG_BASE` + 17)
- Failed to update the history queue.*

  - #define `RHSM_LOG_GET_TIME_EC` (`RHSM_LOG_BASE` + 18)
- Failed to get the system time.*

  - #define `RHSM_LOG_OBJECT_ALLOCATE_EC` (`RHSM_LOG_BASE` + 19)
- Failed to allocate object of the specified kind.*

  - #define `RHSM_LOG_OBJECT_DELETE_EC` (`RHSM_LOG_BASE` + 20)
- Failed to delete object of the specified kind.*

  - #define `RHSM_LOG_OBJECT_INDEX_EC` (`RHSM_LOG_BASE` + 21)
- Failed to index an object of the specified kind.*

  - #define `RHSM_LOG_OBJECT_INVALID_EC` (`RHSM_LOG_BASE` + 22)
- Invalid object specified.*

  - #define `RHSM_LOG_NO_PROPERTY_EC` (`RHSM_LOG_BASE` + 23)
- No property when creating a reader history instance.*

  - #define `RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC` (`RHSM_LOG_BASE` + 24)
- Destination ordering by `SOURCE_TIMESTAMP` ordering is not supported.*

  - #define `RHSM_LOG_RECOMMIT_FAILURE_EC` (`RHSM_LOG_BASE` + 25)
- Failed to recommit sample.*

### 6.21.11.1 Detailed Description

Reader History. ModuleID = 8.

### 6.21.11.2 Macro Definition Documentation

#### **RHSM\_LOG\_OUTSTANDING\_SAMPLE\_EC**

```
#define RHSM_LOG_OUTSTANDING_SAMPLE_EC (RHSM_LOG_BASE + 1)
```

Failed to delete Reader History due to outstanding, unremoved sample.

**RHSM\_LOG\_NO\_PROPERTY\_QOS\_EC**

```
#define RHSM_LOG_NO_PROPERTY_QOS_EC (RHSM_LOG_BASE + 2)
```

Failed to create Reader History due to unspecified property Qos.

**RHSM\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC**

```
#define RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 3)
```

KEEP\_ALL History kind is not supported.

**RHSM\_LOG\_MAX\_SAMPLE\_TOO\_SMALL\_EC**

```
#define RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC (RHSM_LOG_BASE + 4)
```

DataReaderQos.resource\_limits.max\_samples set too small.

DataReaderQos.resource\_limits.max\_samples too small, must be at least DataReaderQos.resource\_limits.max\_instances \* DataReaderQos.resource\_limits.max\_samples\_per\_instance

**RHSM\_LOG\_TBF\_NOT\_SUPPORTED\_EC**

```
#define RHSM_LOG_TBF_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 5)
```

Time-based filtering is not supported.

**RHSM\_LOG\_SAMPLE\_INFO\_POOL\_EC**

```
#define RHSM_LOG_SAMPLE_INFO_POOL_EC (RHSM_LOG_BASE + 6)
```

Failed to allocate a buffer pool for sample info.

**RHSM\_LOG\_SAMPLE\_POOL\_EC**

```
#define RHSM_LOG_SAMPLE_POOL_EC (RHSM_LOG_BASE + 7)
```

Failed to allocate a buffer pool for sample pointers.

**RHSM\_LOG\_SAMPLE\_PTR\_ARRAY\_EC**

```
#define RHSM_LOG_SAMPLE_PTR_ARRAY_EC (RHSM_LOG_BASE + 8)
```

Failed to get sample buffer.

May have exceeded DataReaderQos.reader\_resource\_limits.max\_outstanding\_reads

**RHSM\_LOG\_INFO\_ARRAY\_EC**

```
#define RHSM_LOG_INFO_ARRAY_EC (RHSM_LOG_BASE + 9)
```

Failed to get sample info buffer.

May have exceeded DataReaderQos.reader\_resource\_limits.max\_outstanding\_reads

**RHSM\_LOG\_GET\_RECEPTION\_TIMESTAMP\_EC**

```
#define RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC (RHSM_LOG_BASE + 10)
```

Failed to get time for reception timestamp.

**RHSM\_LOG\_INVALID\_INSTANCE\_REPLACEMENT\_EC**

```
#define RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC (RHSM_LOG_BASE + 11)
```

An invalid instance replacement policy was specified.

**RHSM\_LOG\_FAILED\_TO\_REMOVE\_OLDEST\_EC**

```
#define RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC (RHSM_LOG_BASE + 12)
```

Failed to remove the oldest sample.

**RHSM\_LOG\_SAMPLE\_POOL\_EMPTY\_EC**

```
#define RHSM_LOG_SAMPLE_POOL_EMPTY_EC (RHSM_LOG_BASE + 13)
```

Sample pool empty, may have exceeded.

The sample pool is empty, may have exceeded DataReaderQos.resource\_limits.max\_samples

**RHSM\_LOG\_RW\_PRUNE\_FAILED\_EC**

```
#define RHSM_LOG_RW_PRUNE_FAILED_EC (RHSM_LOG_BASE + 14)
```

Failed to prune remote writer entry.



**RHSM\_LOG\_ENTRY\_RESERVATION\_FAILED\_EC**

```
#define RHSM_LOG_ENTRY_RESERVATION_FAILED_EC (RHSM_LOG_BASE + 15)
```

Failed to reserve an entry.

An entry reservation failed beyond normal resource settings

**RHSM\_LOG\_RETURN\_SAMPLE\_EC**

```
#define RHSM_LOG_RETURN_SAMPLE_EC (RHSM_LOG_BASE + 16)
```

Failed to return a sample.

**RHSM\_LOG\_HISTORY\_UPDATE\_EC**

```
#define RHSM_LOG_HISTORY_UPDATE_EC (RHSM_LOG_BASE + 17)
```

Failed to update the history queue.

**RHSM\_LOG\_GET\_TIME\_EC**

```
#define RHSM_LOG_GET_TIME_EC (RHSM_LOG_BASE + 18)
```

Failed to get the system time.

**RHSM\_LOG\_OBJECT\_ALLOCATE\_EC**

```
#define RHSM_LOG_OBJECT_ALLOCATE_EC (RHSM_LOG_BASE + 19)
```

Failed to allocate object of the specified kind.

**RHSM\_LOG\_OBJECT\_DELETE\_EC**

```
#define RHSM_LOG_OBJECT_DELETE_EC (RHSM_LOG_BASE + 20)
```

Failed to delete object of the specified kind.

**RHSM\_LOG\_OBJECT\_INDEX\_EC**

```
#define RHSM_LOG_OBJECT_INDEX_EC (RHSM_LOG_BASE + 21)
```

Failed to index an object of the specified kind.

**RHSM\_LOG\_OBJECT\_INVALID\_EC**

```
#define RHSM_LOG_OBJECT_INVALID_EC (RHSM_LOG_BASE + 22)
```

Invalid object specified.

**RHSM\_LOG\_NO\_PROPERTY\_EC**

```
#define RHSM_LOG_NO_PROPERTY_EC (RHSM_LOG_BASE + 23)
```

No property when creating a reader history instance.

**RHSM\_LOG\_DESTINATION\_BY\_SOURCE\_NOT\_SUPPORTED\_EC**

```
#define RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 24)
```

Destination ordering by SOURCE\_TIMESTAMP ordering is not supported.

**RHSM\_LOG\_RECOMMIT\_FAILURE\_EC**

```
#define RHSM_LOG_RECOMMIT_FAILURE_EC (RHSM_LOG_BASE + 25)
```

Failed to recommit sample.

**6.21.12 WH**

Writer History. ModuleID = 9.

**Macros**

- #define WHSM\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC (WHSM\_LOG\_BASE + 1)  
*KEEP\_ALL History kind is not supported.*
- #define WHSM\_LOG\_UNLIMITED\_HISTORY\_NOT\_SUPPORTED\_EC (WHSM\_LOG\_BASE + 2)  
*Unlimited length resource limits unsupported.*
- #define WHSM\_LOG\_MAX\_SAMPLES\_TOO\_SMALL\_EC (WHSM\_LOG\_BASE + 3)  
*DataWriterQos.resource\_limits.max\_samples set too small.*
- #define WHSM\_LOG\_OBJECT\_ALLOCATE\_EC (WHSM\_LOG\_BASE + 4)  
*Failed to allocate object of the specified kind.*
- #define WHSM\_LOG\_OBJECT\_DELETE\_EC (WHSM\_LOG\_BASE + 5)  
*Failed to delete object of the specified kind.*
- #define WHSM\_LOG\_OBJECT\_INDEX\_EC (WHSM\_LOG\_BASE + 6)  
*Failed to index an object of the specified kind.*
- #define WHSM\_LOG\_NO\_PROPERTY\_EC (WHSM\_LOG\_BASE + 7)  
*No property when creating a writer history instance.*
- #define WHSM\_LOG\_INVALID\_INDEX\_OBJECT\_EC (WHSM\_LOG\_BASE + 8)  
*The sample removed from an index did not match sample's key.*

### 6.21.12.1 Detailed Description

Writer History. ModuleID = 9.

### 6.21.12.2 Macro Definition Documentation

#### WHSM\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC

```
#define WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC (WHSM_LOG_BASE + 1)
```

KEEP\_ALL History kind is not supported.

#### WHSM\_LOG\_UNLIMITED\_HISTORY\_NOT\_SUPPORTED\_EC

```
#define WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC (WHSM_LOG_BASE + 2)
```

Unlimited length resource limits unsupported.

Ensure that max\_samples, max\_samples\_per\_instance, and max\_instances resource limits are all finite values

#### WHSM\_LOG\_MAX\_SAMPLES\_TOO\_SMALL\_EC

```
#define WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC (WHSM_LOG_BASE + 3)
```

DataWriterQos.resource\_limits.max\_samples set too small.

DataWriterQos.resource\_limits.max\_samples must be no less than max\_instances \* max\_samples\_per\_instance

#### WHSM\_LOG\_OBJECT\_ALLOCATE\_EC

```
#define WHSM_LOG_OBJECT_ALLOCATE_EC (WHSM_LOG_BASE + 4)
```

Failed to allocate object of the specified kind.

#### WHSM\_LOG\_OBJECT\_DELETE\_EC

```
#define WHSM_LOG_OBJECT_DELETE_EC (WHSM_LOG_BASE + 5)
```

Failed to delete object of the specified kind.

**WHSM\_LOG\_OBJECT\_INDEX\_EC**

```
#define WHSM_LOG_OBJECT_INDEX_EC (WHSM_LOG_BASE + 6)
```

Failed to index an object of the specified kind.

**WHSM\_LOG\_NO\_PROPERTY\_EC**

```
#define WHSM_LOG_NO_PROPERTY_EC (WHSM_LOG_BASE + 7)
```

No property when creating a writer history instance.

**WHSM\_LOG\_INVALID\_INDEX\_OBJECT\_EC**

```
#define WHSM_LOG_INVALID_INDEX_OBJECT_EC (WHSM_LOG_BASE + 8)
```

The sample removed from an index did not match sample's key.

**6.21.13 DPSE**

DPSE. ModuleID = 10.

**Macros**

- #define **DPSE\_LOG\_INVALID\_PEER\_ADDRESS\_EC** (DPSE\_LOG\_BASE + 1)  
*A peer address string specifies an invalid address.*
- #define **DPSE\_LOG\_CREATE\_DISCOVERY\_PUBLISHER\_EC** (DPSE\_LOG\_BASE + 2)  
*Failed to create the publisher for a participant discovery writer.*
- #define **DPSE\_LOG\_CREATE\_DISCOVERY\_SUBSCRIBER\_EC** (DPSE\_LOG\_BASE + 3)  
*Failed to create the subscriber for a participant discovery datareader.*
- #define **DPSE\_LOG\_REGISTER\_TYPE\_EC** (DPSE\_LOG\_BASE + 4)  
*Failed to register a built-in participant discovery type.*
- #define **DPSE\_LOG\_CREATE\_TOPIC\_EC** (DPSE\_LOG\_BASE + 5)  
*Failed to create a topic for the built-in participant discovery topic.*
- #define **DPSE\_LOG\_CREATE\_WRITER\_EC** (DPSE\_LOG\_BASE + 6)  
*Failed to create a DataWriter for participant discovery.*
- #define **DPSE\_LOG\_CREATE\_READER\_EC** (DPSE\_LOG\_BASE + 7)  
*Failed to create a DataReader for participant discovery.*
- #define **DPSE\_LOG\_DISPOSE\_EC** (DPSE\_LOG\_BASE + 8)  
*Failed to dispose a participant discovery instance.*
- #define **DPSE\_LOG\_ANNOUNCE\_LOCAL\_PARTICIPANT\_DELETION\_EC** (DPSE\_LOG\_BASE + 9)  
*Failed to dispose a participant discovery instance upon deletion.*
- #define **DPSE\_LOG\_DELETE\_WRITER\_EC** (DPSE\_LOG\_BASE + 10)  
*Failed to delete a participant discovery DataWriter.*

- #define DPSE\_LOG\_DELETE\_PUBLISHER\_EC (DPSE\_LOG\_BASE + 11)  
*Failed to delete a participant discovery Publisher.*
- #define DPSE\_LOG\_DELETE\_READER\_EC (DPSE\_LOG\_BASE + 12)  
*Failed to delete a participant discovery DataReader.*
- #define DPSE\_LOG\_DELETE\_TOPIC\_EC (DPSE\_LOG\_BASE + 13)  
*Failed to delete a participant discovery Topic.*
- #define DPSE\_LOG\_DELETE\_SUBSCRIBER\_EC (DPSE\_LOG\_BASE + 14)  
*Failed to delete a participant discovery Subscriber.*
- #define DPSE\_LOG\_OBJECT\_ALLOCATE\_EC (DPSE\_LOG\_BASE + 15)  
*Failed to allocate an object of the specified kind.*
- #define DPSE\_LOG\_OBJECT\_INITIALIZE\_EC (DPSE\_LOG\_BASE + 16)  
*Failed to initialize an object of the specified kind.*
- #define DPSE\_LOG\_OBJECT\_FINALIZE\_EC (DPSE\_LOG\_BASE + 17)  
*Failed to finalize an object of the specified kind.*
- #define DPSE\_LOG\_OBJECT\_DELETE\_EC (DPSE\_LOG\_BASE + 18)  
*Failed to delete an object of the specified kind.*
- #define DPSE\_LOG\_OBJECT\_INVALID\_EC (DPSE\_LOG\_BASE + 19)  
*The object was invalid in the context it was used.*
- #define DPSE\_LOG\_CDR\_SET\_POSITION\_EC (DPSE\_LOG\_BASE + 20)  
*Failed to set the CDR stream position.*
- #define DPSE\_LOG\_CDR\_SERIALIZE\_EC (DPSE\_LOG\_BASE + 21)  
*Failed to serialize the specified kind.*
- #define DPSE\_LOG\_CDR\_DESERIALIZE\_EC (DPSE\_LOG\_BASE + 22)  
*Failed to deserialize the specified kind.*
- #define DPSE\_LOG\_SERIALIZE\_GUID\_EC (DPSE\_LOG\_BASE + 23)  
*Failed to serialize a GUID key parameter.*
- #define DPSE\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC (DPSE\_LOG\_BASE + 24)  
*Failed to serialize a Builtin Endpoint Mask parameter.*
- #define DPSE\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC (DPSE\_LOG\_BASE + 25)  
*Failed to serialize a Protocol Version parameter.*
- #define DPSE\_LOG\_SERIALIZE\_VENDOR\_ID\_EC (DPSE\_LOG\_BASE + 26)  
*Failed to serialize a Vendor ID parameter.*
- #define DPSE\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC (DPSE\_LOG\_BASE + 27)  
*Failed to serialize a Default Unicast Locator parameter.*
- #define DPSE\_LOG\_SERIALIZE\_META\_UNICAST\_EC (DPSE\_LOG\_BASE + 28)  
*Failed to serialize a Meta Unicast Locator parameter.*
- #define DPSE\_LOG\_SERIALIZE\_META\_MULTICAST\_EC (DPSE\_LOG\_BASE + 29)  
*Failed to serialize a Meta Multicast Locator parameter.*
- #define DPSE\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC (DPSE\_LOG\_BASE + 30)  
*Failed to serialize a Lease Duration parameter.*
- #define DPSE\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC (DPSE\_LOG\_BASE + 31)  
*Failed to serialize a Product Version parameter.*
- #define DPSE\_LOG\_DESERIALIZE\_PRODUCT\_VERSION\_EC (DPSE\_LOG\_BASE + 32)  
*Failed to serialize a Product Version parameter.*
- #define DPSE\_LOG\_SERIALIZE\_PARTICIPANT\_NAME\_EC (DPSE\_LOG\_BASE + 33)  
*Failed to serialize a Participant Name parameter.*
- #define DPSE\_LOG\_DESERIALIZE\_GUID\_EC (DPSE\_LOG\_BASE + 34)

- Failed to deserialize a GUID key parameter.*

  - #define `DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (`DPSE_LOG_BASE` + 35)
- Failed to deserialize a Builtin Endpoint Mask parameter.*

  - #define `DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (`DPSE_LOG_BASE` + 36)
- Failed to deserialize a Protocol Version parameter.*

  - #define `DPSE_LOG_DESERIALIZE_VENDOR_ID_EC` (`DPSE_LOG_BASE` + 37)
- Failed to deserialize a Vendor ID parameter.*

  - #define `DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC` (`DPSE_LOG_BASE` + 38)
- Cannot deserialize another locator parameter, having reached maximum of 4 unicast or 4 multicast locators.*

  - #define `DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC` (`DPSE_LOG_BASE` + 39)
- Failed to deserialize a Participant Name parameter.*

  - #define `DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC` (`DPSE_LOG_BASE` + 40)
- Failed to deserialize a parameter with an unknown ID.*

  - #define `DPSE_LOG_ANNOUNCEMENT_EC` (`DPSE_LOG_BASE` + 41)
- Failed to send a participant discovery announcement.*

  - #define `DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC` (`DPSE_LOG_BASE` + 42)
- Failed to schedule an event to send the next participant discovery announcement.*

  - #define `DPSE_LOG_ADVANCE_SN_EC` (`DPSE_LOG_BASE` + 43)
- Failed to advance the sequence number of a participant discovery announcement.*

  - #define `DPSE_LOG_ANNOUNCE_WRITE_EC` (`DPSE_LOG_BASE` + 44)
- Failed to write a participant discovery announcement message.*

  - #define `DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC` (`DPSE_LOG_BASE` + 45)
- Failed to schedule an event to send the next participant discovery announcement.*

  - #define `DPSE_LOG_PARTICIPANT_TAKE_EC` (`DPSE_LOG_BASE` + 46)
- Failed to take a sample from a participant discovery DataReader.*

  - #define `DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 47)
- Failed to assert remote participant.*

  - #define `DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 48)
- Failed to assert and complete discovery of a remote participant.*

  - #define `DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC` (`DPSE_LOG_BASE` + 49)
- Failed to add an anonymous route.*

  - #define `DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC` (`DPSE_LOG_BASE` + 50)
- Failed to delete an anonymous route.*

  - #define `DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 51)
- Exceeded resource limit, remote\_participant\_allocation.*

  - #define `DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 52)
- Failed to refresh liveliness for a remote participant.*

  - #define `DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 53)
- Failed to reset liveliness for a remote participant.*

  - #define `DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC` (`DPSE_LOG_BASE` + 54)
- Received a participant discovery announcement with invalid state.*

  - #define `DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC` (`DPSE_LOG_BASE` + 55)
- Failed to return a loan on a participant discovery announcement.*

  - #define `DPSE_LOG_SERIALIZE_MULTICAST_EC` (`DPSE_LOG_BASE` + 56)
- Failed to serialize a Multicast Locator parameter.*

  - #define `DPSE_LOG_GET_DDS_PROPERTIES_EC` (`DPSE_LOG_BASE` + 57)
- Failed to serialize a Multicast Locator parameter.*

- `#define DPSE_LOG_ENTITY_ENABLE_EC (DPSE_LOG_BASE + 58)`  
*Failed to enable the specified entity kind.*
- `#define DPSE_LOG_SEQUENCE_SETMAX_EC (DPSE_LOG_BASE + 59)`  
*Failed to set the maximum length of a sequence of the specified kind.*
- `#define DPSE_LOG_SEQUENCE_SETLENGTH_EC (DPSE_LOG_BASE + 60)`  
*Failed to set the length of a sequence of the specified kind.*
- `#define DPSE_LOG_SEQUENCE_GETREF_EC (DPSE_LOG_BASE + 61)`  
*Failed to get a reference at the specified index for a sequence of the specified kind.*
- `#define DPSE_LOG_SEQUENCE_INITIALIZE_EC (DPSE_LOG_BASE + 62)`  
*Failed to initialize a sequence of the specified kind.*
- `#define DPSE_LOG_SEQUENCE_FINALIZE_EC (DPSE_LOG_BASE + 63)`  
*Failed to finalize a sequence of the specified kind.*
- `#define DPSE_LOG_SEQUENCE_COPY_EC (DPSE_LOG_BASE + 64)`  
*Failed to copy a sequence of the specified kind.*
- `#define DPSE_LOG_GET_TIMER_EC (DPSE_LOG_BASE + 65)`  
*Failed to get timer.*
- `#define DPSE_LOG_UNSUPPORTED_LOCATOR_EC (DPSE_LOG_BASE + 66)`  
*The locator kind is not supported by any transport registered with the domain participant and is dropped.*
- `#define DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPSE_LOG_BASE + 67)`  
*A participant with incompatible checksum was discovered and ignored.*
- `#define DPSE_LOG_SERIALIZE_PROP_EC (DPSE_LOG_BASE + 68)`  
*A Participant failed to serialize its properties.*

### 6.21.13.1 Detailed Description

DPSE. ModuleID = 10.

### 6.21.13.2 Macro Definition Documentation

#### **DPSE\_LOG\_INVALID\_PEER\_ADDRESS\_EC**

```
#define DPSE_LOG_INVALID_PEER_ADDRESS_EC (DPSE_LOG_BASE + 1)
```

A peer address string specifies an invalid address.

#### **DPSE\_LOG\_CREATE\_DISCOVERY\_PUBLISHER\_EC**

```
#define DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC (DPSE_LOG_BASE + 2)
```

Failed to create the publisher for a participant discovery writer.

#### **DPSE\_LOG\_CREATE\_DISCOVERY\_SUBSCRIBER\_EC**

```
#define DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC (DPSE_LOG_BASE + 3)
```

Failed to create the subscriber for a participant discovery datareader.

**DPSE\_LOG\_REGISTER\_TYPE\_EC**

```
#define DPSE_LOG_REGISTER_TYPE_EC (DPSE_LOG_BASE + 4)
```

Failed to register a built-in participant discovery type.

**DPSE\_LOG\_CREATE\_TOPIC\_EC**

```
#define DPSE_LOG_CREATE_TOPIC_EC (DPSE_LOG_BASE + 5)
```

Failed to create a topic for the built-in participant discovery topic.

**DPSE\_LOG\_CREATE\_WRITER\_EC**

```
#define DPSE_LOG_CREATE_WRITER_EC (DPSE_LOG_BASE + 6)
```

Failed to create a DataWriter for participant discovery.

**DPSE\_LOG\_CREATE\_READER\_EC**

```
#define DPSE_LOG_CREATE_READER_EC (DPSE_LOG_BASE + 7)
```

Failed to create a DataReader for participant discovery.

**DPSE\_LOG\_DISPOSE\_EC**

```
#define DPSE_LOG_DISPOSE_EC (DPSE_LOG_BASE + 8)
```

Failed to dispose a participant discovery instance.

**DPSE\_LOG\_ANNOUNCE\_LOCAL\_PARTICIPANT\_DELETION\_EC**

```
#define DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC (DPSE_LOG_BASE + 9)
```

Failed to dispose a participant discovery instance upon deletion.

**DPSE\_LOG\_DELETE\_WRITER\_EC**

```
#define DPSE_LOG_DELETE_WRITER_EC (DPSE_LOG_BASE + 10)
```

Failed to delete a participant discovery DataWriter.



**DPSE\_LOG\_DELETE\_PUBLISHER\_EC**

```
#define DPSE_LOG_DELETE_PUBLISHER_EC (DPSE_LOG_BASE + 11)
```

Failed to delete a participant discovery Publisher.

**DPSE\_LOG\_DELETE\_READER\_EC**

```
#define DPSE_LOG_DELETE_READER_EC (DPSE_LOG_BASE + 12)
```

Failed to delete a participant discovery DataReader.

**DPSE\_LOG\_DELETE\_TOPIC\_EC**

```
#define DPSE_LOG_DELETE_TOPIC_EC (DPSE_LOG_BASE + 13)
```

Failed to delete a participant discovery Topic.

**DPSE\_LOG\_DELETE\_SUBSCRIBER\_EC**

```
#define DPSE_LOG_DELETE_SUBSCRIBER_EC (DPSE_LOG_BASE + 14)
```

Failed to delete a participant discovery Subscriber.

**DPSE\_LOG\_OBJECT\_ALLOCATE\_EC**

```
#define DPSE_LOG_OBJECT_ALLOCATE_EC (DPSE_LOG_BASE + 15)
```

Failed to allocate an object of the specified kind.

**DPSE\_LOG\_OBJECT\_INITIALIZE\_EC**

```
#define DPSE_LOG_OBJECT_INITIALIZE_EC (DPSE_LOG_BASE + 16)
```

Failed to initialize an object of the specified kind.

**DPSE\_LOG\_OBJECT\_FINALIZE\_EC**

```
#define DPSE_LOG_OBJECT_FINALIZE_EC (DPSE_LOG_BASE + 17)
```

Failed to finalize an object of the specified kind.

**DPSE\_LOG\_OBJECT\_DELETE\_EC**

```
#define DPSE_LOG_OBJECT_DELETE_EC (DPSE_LOG_BASE + 18)
```

Failed to delete an object of the specified kind.

**DPSE\_LOG\_OBJECT\_INVALID\_EC**

```
#define DPSE_LOG_OBJECT_INVALID_EC (DPSE_LOG_BASE + 19)
```

The object was invalid in the context it was used.

**DPSE\_LOG\_CDR\_SET\_POSITION\_EC**

```
#define DPSE_LOG_CDR_SET_POSITION_EC (DPSE_LOG_BASE + 20)
```

Failed to set the CDR stream position.

**DPSE\_LOG\_CDR\_SERIALIZE\_EC**

```
#define DPSE_LOG_CDR_SERIALIZE_EC (DPSE_LOG_BASE + 21)
```

Failed to serialize the specified kind.

**DPSE\_LOG\_CDR\_DESERIALIZE\_EC**

```
#define DPSE_LOG_CDR_DESERIALIZE_EC (DPSE_LOG_BASE + 22)
```

Failed to deserialize the specified kind.

**DPSE\_LOG\_SERIALIZE\_GUID\_EC**

```
#define DPSE_LOG_SERIALIZE_GUID_EC (DPSE_LOG_BASE + 23)
```

Failed to serialize a GUID key parameter.

**DPSE\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC**

```
#define DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC (DPSE_LOG_BASE + 24)
```

Failed to serialize a Builtin Endpoint Mask parameter.

**DPSE\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC**

```
#define DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC (DPSE_LOG_BASE + 25)
```

Failed to serialize a Protocol Version parameter.

**DPSE\_LOG\_SERIALIZE\_VENDOR\_ID\_EC**

```
#define DPSE_LOG_SERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 26)
```

Failed to serialize a Vendor ID parameter.

**DPSE\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC**

```
#define DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DPSE_LOG_BASE + 27)
```

Failed to serialize a Default Unicast Locator parameter.

**DPSE\_LOG\_SERIALIZE\_META\_UNICAST\_EC**

```
#define DPSE_LOG_SERIALIZE_META_UNICAST_EC (DPSE_LOG_BASE + 28)
```

Failed to serialize a Meta Unicast Locator parameter.

**DPSE\_LOG\_SERIALIZE\_META\_MULTICAST\_EC**

```
#define DPSE_LOG_SERIALIZE_META_MULTICAST_EC (DPSE_LOG_BASE + 29)
```

Failed to serialize a Meta Multicast Locator parameter.

**DPSE\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC**

```
#define DPSE_LOG_SERIALIZE_LEASE_DURATION_EC (DPSE_LOG_BASE + 30)
```

Failed to serialize a Lease Duration parameter.

**DPSE\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC**

```
#define DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 31)
```

Failed to serialize a Product Version parameter.

**DPSE\_LOG\_DESERIALIZE\_PRODUCT\_VERSION\_EC**

```
#define DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 32)
```

Failed to serialize a Product Version parameter.

**DPSE\_LOG\_SERIALIZE\_PARTICIPANT\_NAME\_EC**

```
#define DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 33)
```

Failed to serialize a Participant Name parameter.

**DPSE\_LOG\_DESERIALIZE\_GUID\_EC**

```
#define DPSE_LOG_DESERIALIZE_GUID_EC (DPSE_LOG_BASE + 34)
```

Failed to deserialize a GUID key parameter.

**DPSE\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC**

```
#define DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DPSE_LOG_BASE + 35)
```

Failed to deserialize a Builtin Endpoint Mask parameter.

**DPSE\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC**

```
#define DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DPSE_LOG_BASE + 36)
```

Failed to deserialize a Protocol Version parameter.

**DPSE\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC**

```
#define DPSE_LOG_DESERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 37)
```

Failed to deserialize a Vendor ID parameter.

**DPSE\_LOG\_DESERIALIZE\_TOO\_MANY\_LOCATORS\_EC**

```
#define DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC (DPSE_LOG_BASE + 38)
```

Cannot deserialize another locator parameter, having reached maximum of 4 unicast or 4 multicast locators.

**DPSE\_LOG\_DESERIALIZE\_PARTICIPANT\_NAME\_EC**

```
#define DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 39)
```

Failed to deserialize a Participant Name parameter.

**DPSE\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC**

```
#define DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC (DPSE_LOG_BASE + 40)
```

Failed to deserialize a parameter with an unknown ID.

**DPSE\_LOG\_ANNOUNCEMENT\_EC**

```
#define DPSE_LOG_ANNOUNCEMENT_EC (DPSE_LOG_BASE + 41)
```

Failed to send a participant discovery announcement.

**DPSE\_LOG\_UPDATE\_PARTICIPANT\_ASSERT\_PERIOD\_EC**

```
#define DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC (DPSE_LOG_BASE + 42)
```

Failed to schedule an event to send the next participant discovery announcement.

**DPSE\_LOG\_ADVANCE\_SN\_EC**

```
#define DPSE_LOG_ADVANCE_SN_EC (DPSE_LOG_BASE + 43)
```

Failed to advance the sequence number of a participant discovery announcement.

**DPSE\_LOG\_ANNOUNCE\_WRITE\_EC**

```
#define DPSE_LOG_ANNOUNCE_WRITE_EC (DPSE_LOG_BASE + 44)
```

Failed to write a participant discovery announcement message.

**DPSE\_LOG\_SCHEDULE\_FAST\_ASSERTION\_EC**

```
#define DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC (DPSE_LOG_BASE + 45)
```

Failed to schedule an event to send the next participant discovery announcement.

**DPSE\_LOG\_PARTICIPANT\_TAKE\_EC**

```
#define DPSE_LOG_PARTICIPANT_TAKE_EC (DPSE_LOG_BASE + 46)
```

Failed to take a sample from a participant discovery DataReader.

**DPSE\_LOG\_ASSERT\_REMOTE\_PARTICIPANT\_EC**

```
#define DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 47)
```

Failed to assert remote participant.

**DPSE\_LOG\_ON\_ASSERT\_REMOTE\_PARTICIPANT\_EC**

```
#define DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 48)
```

Failed to assert and complete discovery of a remote participant.

**DPSE\_LOG\_ADD\_ANONYMOUS\_ROUTE\_EC**

```
#define DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 49)
```

Failed to add an anonymous route.

Results in a DomainParticipant not having a destination to which it should have sent participant discovery announcements

**DPSE\_LOG\_DELETE\_ANONYMOUS\_ROUTE\_EC**

```
#define DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 50)
```

Failed to delete an anonymous route.

**DPSE\_LOG\_MAX\_REMOTE\_PARTICIPANT\_EC**

```
#define DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 51)
```

Exceeded resource limit, remote\_participant\_allocation.

**DPSE\_LOG\_REFRESH\_REMOTE\_PARTICIPANT\_EC**

```
#define DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 52)
```

Failed to refresh liveliness for a remote participant.

**DPSE\_LOG\_RESET\_REMOTE\_PARTICIPANT\_EC**

```
#define DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 53)
```

Failed to reset liveliness for a remote participant.

**DPSE\_LOG\_INVALID\_DISCOVERY\_SAMPLE\_EC**

```
#define DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC (DPSE_LOG_BASE + 54)
```

Received a participant discovery announcement with invalid state.

**DPSE\_LOG\_RETURN\_DISCOVERY\_SAMPLE\_EC**

```
#define DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC (DPSE_LOG_BASE + 55)
```

Failed to return a loan on a participant discovery announcement.

**DPSE\_LOG\_SERIALIZE\_MULTICAST\_EC**

```
#define DPSE_LOG_SERIALIZE_MULTICAST_EC (DPSE_LOG_BASE + 56)
```

Failed to serialize a Multicast Locator parameter.

**DPSE\_LOG\_GET\_DDS\_PROPERTIES\_EC**

```
#define DPSE_LOG_GET_DDS_PROPERTIES_EC (DPSE_LOG_BASE + 57)
```

Failed to serialize a Multicast Locator parameter.

**DPSE\_LOG\_ENTITY\_ENABLE\_EC**

```
#define DPSE_LOG_ENTITY_ENABLE_EC (DPSE_LOG_BASE + 58)
```

Failed to enable the specified entity kind.

**DPSE\_LOG\_SEQUENCE\_SETMAX\_EC**

```
#define DPSE_LOG_SEQUENCE_SETMAX_EC (DPSE_LOG_BASE + 59)
```

Failed to set the maximum length of a sequence of the specified kind.

**DPSE\_LOG\_SEQUENCE\_SETLENGTH\_EC**

```
#define DPSE_LOG_SEQUENCE_SETLENGTH_EC (DPSE_LOG_BASE + 60)
```

Failed to set the length of a sequence of the specified kind.

**DPSE\_LOG\_SEQUENCE\_GETREF\_EC**

```
#define DPSE_LOG_SEQUENCE_GETREF_EC (DPSE_LOG_BASE + 61)
```

Failed to get a reference at the specified index for a sequence of the specified kind.

**DPSE\_LOG\_SEQUENCE\_INITIALIZE\_EC**

```
#define DPSE_LOG_SEQUENCE_INITIALIZE_EC (DPSE_LOG_BASE + 62)
```

Failed to initialize a sequence of the specified kind.

**DPSE\_LOG\_SEQUENCE\_FINALIZE\_EC**

```
#define DPSE_LOG_SEQUENCE_FINALIZE_EC (DPSE_LOG_BASE + 63)
```

Failed to finalize a sequence of the specified kind.

**DPSE\_LOG\_SEQUENCE\_COPY\_EC**

```
#define DPSE_LOG_SEQUENCE_COPY_EC (DPSE_LOG_BASE + 64)
```

Failed to copy a sequence of the specified kind.

**DPSE\_LOG\_GET\_TIMER\_EC**

```
#define DPSE_LOG_GET_TIMER_EC (DPSE_LOG_BASE + 65)
```

Failed to get timer.

**DPSE\_LOG\_UNSUPPORTED\_LOCATOR\_EC**

```
#define DPSE_LOG_UNSUPPORTED_LOCATOR_EC (DPSE_LOG_BASE + 66)
```

The locator kind is not supported by any transport registered with the domain participant and is dropped.



**DPSE\_LOG\_CHECKSUM\_INCOMPATIBLE\_PARTICIPANT\_IGNORED\_EC**

```
#define DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPDE_LOG_BASE + 67)
```

A participant with incompatible checksum was discovered and ignored.

**DPSE\_LOG\_SERIALIZE\_PROP\_EC**

```
#define DPSE_LOG_SERIALIZE_PROP_EC (DPDE_LOG_BASE + 68)
```

A Participant failed to serialize its properties.

**6.21.14 DPDE**

DPDE. ModuleID = 11.

**Macros**

- #define DPDE\_LOG\_CDR\_SET\_OFFSET\_EC (DPDE\_LOG\_BASE + 1)  
*Failed to set current offset of stream.*
- #define DPDE\_LOG\_SERIALIZE\_ENTITY\_NAME\_EC (DPDE\_LOG\_BASE + 2)  
*Failed to serialize Entity Name parameter.*
- #define DPDE\_LOG\_SERIALIZE\_TOPIC\_NAME\_EC (DPDE\_LOG\_BASE + 3)  
*Failed to serialize Topic Name parameter.*
- #define DPDE\_LOG\_SERIALIZE\_TYPE\_NAME\_EC (DPDE\_LOG\_BASE + 4)  
*Failed to serialize Type Name parameter.*
- #define DPDE\_LOG\_SERIALIZE\_GUID\_EC (DPDE\_LOG\_BASE + 5)  
*Failed to serialize GUID key.*
- #define DPDE\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC (DPDE\_LOG\_BASE + 6)  
*Failed to serialize Default Unicast Locator parameter.*
- #define DPDE\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC (DPDE\_LOG\_BASE + 7)  
*Failed to serialize Protocol Version parameter.*
- #define DPDE\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC (DPDE\_LOG\_BASE + 8)  
*Failed to deserialize Protocol Version parameter.*
- #define DPDE\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC (DPDE\_LOG\_BASE + 9)  
*Failed to deserialize Vendor ID parameter.*
- #define DPDE\_LOG\_SERIALIZE\_VENDOR\_ID\_EC (DPDE\_LOG\_BASE + 10)  
*Failed to serialize Vendor ID parameter.*
- #define DPDE\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC (DPDE\_LOG\_BASE + 11)  
*Failed to serialize Product Version parameter.*
- #define DPDE\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC (DPDE\_LOG\_BASE + 12)  
*Failed to serialize Lease Duration parameter.*
- #define DPDE\_LOG\_CREATE\_PUBLISHER\_EC (DPDE\_LOG\_BASE + 13)  
*Failed to create Publisher for built-in endpoint DataWriters.*
- #define DPDE\_LOG\_CREATE\_SUBSCRIBER\_EC (DPDE\_LOG\_BASE + 14)

- Failed to create Subscriber for built-in endpoint DataReaders.*

  - #define `DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC` (`DPDE_LOG_BASE` + 15)
- Failed to create built-in endpoint for participant discovery after creating local user DomainParticipant.*

  - #define `DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC` (`DPDE_LOG_BASE` + 16)
- Failed to create built-in publication endpoint after creating local user DomainParticipant.*

  - #define `DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC` (`DPDE_LOG_BASE` + 17)
- Failed to create built-in subscription endpoint after creating local DomainParticipant.*

  - #define `DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC` (`DPDE_LOG_BASE` + 18)
- Failed to delete subscription built-in topic.*

  - #define `DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC` (`DPDE_LOG_BASE` + 19)
- Failed to delete publication built-in topic.*

  - #define `DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC` (`DPDE_LOG_BASE` + 20)
- Failed to delete participant built-in topic.*

  - #define `DPDE_LOG_DISPOSE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 21)
- Failed to dispose built-in participant when deleting DomainParticipant.*

  - #define `DPDE_LOG_ALLOCATE_DPDE_EC` (`DPDE_LOG_BASE` + 22)
- Out of memory to allocate discovery plugin.*

  - #define `DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (`DPDE_LOG_BASE` + 23)
- Failed to serialize Builtin Endpoint Mask parameter.*

  - #define `DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (`DPDE_LOG_BASE` + 24)
- Failed to deserialize Builtin Endpoint Mask parameter.*

  - #define `DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC` (`DPDE_LOG_BASE` + 25)
- Failed to deserialize a parameter of unknown ID.*

  - #define `DPDE_LOG_ANNOUNCEMENT_EC` (`DPDE_LOG_BASE` + 26)
- Failed to write a dynamic participant discovery message.*

  - #define `DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC` (`DPDE_LOG_BASE` + 27)
- Failed to schedule an event to assert the next participant discovery announcement.*

  - #define `DPDE_LOG_ADVANCE_SN_EC` (`DPDE_LOG_BASE` + 28)
- Failed to advance the sequence number of a participant discovery announcement.*

  - #define `DPDE_LOG_ANNOUNCE_WRITE_EC` (`DPDE_LOG_BASE` + 29)
- Failed to write initial participant discovery announcement.*

  - #define `DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC` (`DPDE_LOG_BASE` + 30)
- Failed to schedule event for asserting participant discovery announcements.*

  - #define `DPDE_LOG_REGISTER_TYPE_EC` (`DPDE_LOG_BASE` + 32)
- Failed to register a built-in type for discovery.*

  - #define `DPDE_LOG_CREATE_TOPIC_EC` (`DPDE_LOG_BASE` + 33)
- Failed to create a topic for discovery.*

  - #define `DPDE_LOG_CREATE_WRITER_EC` (`DPDE_LOG_BASE` + 34)
- Failed to create a built-in DataWriter for discovery.*

  - #define `DPDE_LOG_CREATE_READER_EC` (`DPDE_LOG_BASE` + 35)
- Failed to create a built-in DataReader for discovery.*

  - #define `DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 36)
- Failed to assert and complete discovery of a remote participant.*

  - #define `DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 37)
- Failed to enable and complete discovery of a remote participant.*

  - #define `DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 38)
- Failed to refresh liveness for a discovered remote participant.*

- #define `DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 39)  
*Failed to take a sample from a participant discovery DataReader.*
- #define `DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 40)  
*Received a participant discovery announcement with invalid state.*
- #define `DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 41)  
*Failed to return a loan on a participant discovery sample.*
- #define `DPDE_LOG_DISPOSE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 42)  
*Failed to dispose an instance for a publication.*
- #define `DPDE_LOG_DISPOSE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 43)  
*Failed to dispose an instance of a subscription.*
- #define `DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 44)  
*Failed to assert and complete discovery of a remote publication.*
- #define `DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 45)  
*Failed to assert and complete discovery of a remote subscription.*
- #define `DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 46)  
*Failed to take a sample from a publication discovery DataReader.*
- #define `DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 47)  
*Failed to dispose or unregister a remote publication instance.*
- #define `DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 48)  
*Received a publication discovery announcement with invalid state.*
- #define `DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 49)  
*Failed to return a loan on a publication discovery sample.*
- #define `DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC` (`DPDE_LOG_BASE` + 51)  
*Failed to take a sample from a subscription discovery DataReader.*
- #define `DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC` (`DPDE_LOG_BASE` + 52)  
*Received a subscription discovery sample with invalid state.*
- #define `DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC` (`DPDE_LOG_BASE` + 53)  
*Failed to return a loan on a subscription discovery sample.*
- #define `DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 54)  
*Failed to dispose or unregister a remote subscription instance.*
- #define `DPDE_LOG_LOCATORS_FULL_EC` (`DPDE_LOG_BASE` + 55)  
*A locator sequence is full, the remaining locators of the specified kind are dropped.*
- #define `DPDE_LOG_DESERIALIZE_LOCATOR_EC` (`DPDE_LOG_BASE` + 56)  
*Failed to deserialize a locator of the specified kind.*
- #define `DPDE_LOG_UNSUPPORTED_LOCATOR_EC` (`DPDE_LOG_BASE` + 57)  
*The locator kind is not supported by any transport registered with the domain participant and is dropped.*
- #define `DPDE_LOG_ADD_PEER_EC` (`DPDE_LOG_BASE` + 58)  
*Failed to add the specified entity as a peer.*
- #define `DPDE_LOG_REMOVE_PEER_EC` (`DPDE_LOG_BASE` + 59)  
*Failed to add the specified entity as a peer.*
- #define `DPDE_LOG_SERIALIZE_PRESENTATION_EC` (`DPDE_LOG_BASE` + 60)  
*Failed to serialize Presentation parameter.*
- #define `DPDE_LOG_UNREGISTER_TYPE_EC` (`DPDE_LOG_BASE` + 61)  
*Failed to unregister a built-in type for discovery.*
- #define `DPDE_LOG_ADD_REMOTE_PARTICIPANT_ROUTES_FAILED_EC` (`DPDE_LOG_BASE` + 62)  
*Failed to add routes to a remote participant.*
- #define `DPDE_LOG_DELETE_REMOTE_PARTICIPANT_ROUTES_FAILED_EC` (`DPDE_LOG_BASE` + 63)

*Failed to delete routes to a remote participant.*

- #define DPDE\_LOG\_REMOVE\_REMOTE\_PARTICIPANT\_EC (DPDE\_LOG\_BASE + 64)

*Failed remove a remote participant.*

- #define DPDE\_LOG\_CHECKSUM\_INCOMPATIBLE\_PARTICIPANT\_IGNORED\_EC (DPDE\_LOG\_BASE + 65)

*A participant with incompatible checksum was discovered and ignored.*

#### 6.21.14.1 Detailed Description

DPDE. ModuleID = 11.

#### 6.21.14.2 Macro Definition Documentation

##### DPDE\_LOG\_CDR\_SET\_OFFSET\_EC

```
#define DPDE_LOG_CDR_SET_OFFSET_EC (DPDE_LOG_BASE + 1)
```

Failed to set current offset of stream.

##### DPDE\_LOG\_SERIALIZE\_ENTITY\_NAME\_EC

```
#define DPDE_LOG_SERIALIZE_ENTITY_NAME_EC (DPDE_LOG_BASE + 2)
```

Failed to serialize Entity Name parameter.

##### DPDE\_LOG\_SERIALIZE\_TOPIC\_NAME\_EC

```
#define DPDE_LOG_SERIALIZE_TOPIC_NAME_EC (DPDE_LOG_BASE + 3)
```

Failed to serialize Topic Name parameter.

##### DPDE\_LOG\_SERIALIZE\_TYPE\_NAME\_EC

```
#define DPDE_LOG_SERIALIZE_TYPE_NAME_EC (DPDE_LOG_BASE + 4)
```

Failed to serialize Type Name parameter.

##### DPDE\_LOG\_SERIALIZE\_GUID\_EC

```
#define DPDE_LOG_SERIALIZE_GUID_EC (DPDE_LOG_BASE + 5)
```

Failed to serialize GUID key.

**DPDE\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC**

```
#define DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DPDE_LOG_BASE + 6)
```

Failed to serialize Default Unicast Locator parameter.

**DPDE\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC**

```
#define DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC (DPDE_LOG_BASE + 7)
```

Failed to serialize Protocol Version parameter.

**DPDE\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC**

```
#define DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DPDE_LOG_BASE + 8)
```

Failed to deserialize Protocol Version parameter.

**DPDE\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC**

```
#define DPDE_LOG_DESERIALIZE_VENDOR_ID_EC (DPDE_LOG_BASE + 9)
```

Failed to deserialize Vendor ID parameter.

**DPDE\_LOG\_SERIALIZE\_VENDOR\_ID\_EC**

```
#define DPDE_LOG_SERIALIZE_VENDOR_ID_EC (DPDE_LOG_BASE + 10)
```

Failed to serialize Vendor ID parameter.

**DPDE\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC**

```
#define DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC (DPDE_LOG_BASE + 11)
```

Failed to serialize Product Version parameter.

**DPDE\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC**

```
#define DPDE_LOG_SERIALIZE_LEASE_DURATION_EC (DPDE_LOG_BASE + 12)
```

Failed to serialize Lease Duration parameter.

**DPDE\_LOG\_CREATE\_PUBLISHER\_EC**

```
#define DPDE_LOG_CREATE_PUBLISHER_EC (DPDE_LOG_BASE + 13)
```

Failed to create Publisher for built-in endpoint DataWriters.

**DPDE\_LOG\_CREATE\_SUBSCRIBER\_EC**

```
#define DPDE_LOG_CREATE_SUBSCRIBER_EC (DPDE_LOG_BASE + 14)
```

Failed to create Subscriber for built-in endpoint DataReaders.

**DPDE\_LOG\_CREATE\_PARTICIPANT\_DISCOVERY\_EC**

```
#define DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC (DPDE_LOG_BASE + 15)
```

Failed to create built-in endpoint for participant discovery after creating local user DomainParticipant.

**DPDE\_LOG\_CREATE\_PUBLICATION\_DISCOVERY\_EC**

```
#define DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC (DPDE_LOG_BASE + 16)
```

Failed to create built-in publication endpoint after creating local user DomainParticipant.

**DPDE\_LOG\_CREATE\_SUBSCRIPTION\_DISCOVERY\_EC**

```
#define DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC (DPDE_LOG_BASE + 17)
```

Failed to create built-in subscription endpoint after creating local DomainParticipant.

**DPDE\_LOG\_DELETE\_SUBSCRIPTION\_TOPIC\_EC**

```
#define DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC (DPDE_LOG_BASE + 18)
```

Failed to delete subscription built-in topic.

**DPDE\_LOG\_DELETE\_PUBLICATION\_TOPIC\_EC**

```
#define DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC (DPDE_LOG_BASE + 19)
```

Failed to delete publication built-in topic.

**DPDE\_LOG\_DELETE\_PARTICIPANT\_TOPIC\_EC**

```
#define DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC (DPDE_LOG_BASE + 20)
```

Failed to delete participant built-in topic.

**DPDE\_LOG\_DISPOSE\_PARTICIPANT\_EC**

```
#define DPDE_LOG_DISPOSE_PARTICIPANT_EC (DPDE_LOG_BASE + 21)
```

Failed to dispose built-in participant when deleting DomainParticipant.

**DPDE\_LOG\_ALLOCATE\_DPDE\_EC**

```
#define DPDE_LOG_ALLOCATE_DPDE_EC (DPDE_LOG_BASE + 22)
```

Out of memory to allocate discovery plugin.

**DPDE\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC**

```
#define DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC (DPDE_LOG_BASE + 23)
```

Failed to serialize Builtin Endpoint Mask parameter.

**DPDE\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC**

```
#define DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DPDE_LOG_BASE + 24)
```

Failed to deserialize Builtin Endpoint Mask parameter.

**DPDE\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC**

```
#define DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC (DPDE_LOG_BASE + 25)
```

Failed to deserialize a parameter of unknown ID.

**DPDE\_LOG\_ANNOUNCEMENT\_EC**

```
#define DPDE_LOG_ANNOUNCEMENT_EC (DPDE_LOG_BASE + 26)
```

Failed to write a dynamic participant discovery message.

**DPDE\_LOG\_UPDATE\_PARTICIPANT\_ASSERT\_PERIOD\_EC**

```
#define DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC (DPDE_LOG_BASE + 27)
```

Failed to schedule an event to assert the next participant discovery announcement.

**DPDE\_LOG\_ADVANCE\_SN\_EC**

```
#define DPDE_LOG_ADVANCE_SN_EC (DPDE_LOG_BASE + 28)
```

Failed to advance the sequence number of a participant discovery announcement.

**DPDE\_LOG\_ANNOUNCE\_WRITE\_EC**

```
#define DPDE_LOG_ANNOUNCE_WRITE_EC (DPDE_LOG_BASE + 29)
```

Failed to write initial participant discovery announcement.

**DPDE\_LOG\_SCHEDULE\_FAST\_ASSERTION\_EC**

```
#define DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC (DPDE_LOG_BASE + 30)
```

Failed to schedule event for asserting participant discovery announcements.

**DPDE\_LOG\_REGISTER\_TYPE\_EC**

```
#define DPDE_LOG_REGISTER_TYPE_EC (DPDE_LOG_BASE + 32)
```

Failed to register a built-in type for discovery.

**DPDE\_LOG\_CREATE\_TOPIC\_EC**

```
#define DPDE_LOG_CREATE_TOPIC_EC (DPDE_LOG_BASE + 33)
```

Failed to create a topic for discovery.

**DPDE\_LOG\_CREATE\_WRITER\_EC**

```
#define DPDE_LOG_CREATE_WRITER_EC (DPDE_LOG_BASE + 34)
```

Failed to create a built-in DataWriter for discovery.



**DPDE\_LOG\_CREATE\_READER\_EC**

```
#define DPDE_LOG_CREATE_READER_EC (DPDE_LOG_BASE + 35)
```

Failed to create a built-in DataReader for discovery.

**DPDE\_LOG\_ASSERT\_REMOTE\_PARTICIPANT\_EC**

```
#define DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 36)
```

Failed to assert and complete discovery of a remote participant.

**DPDE\_LOG\_ENABLE\_REMOTE\_PARTICIPANT\_EC**

```
#define DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 37)
```

Failed to enable and complete discovery of a remote participant.

**DPDE\_LOG\_REFRESH\_REMOTE\_PARTICIPANT\_EC**

```
#define DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 38)
```

Failed to refresh liveliness for a discovered remote participant.

**DPDE\_LOG\_TAKE\_PARTICIPANT\_SAMPLE\_EC**

```
#define DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC (DPDE_LOG_BASE + 39)
```

Failed to take a sample from a participant discovery DataReader.

**DPDE\_LOG\_INVALID\_PARTICIPANT\_SAMPLE\_EC**

```
#define DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC (DPDE_LOG_BASE + 40)
```

Received a participant discovery announcement with invalid state.

**DPDE\_LOG\_RETURN\_PARTICIPANT\_SAMPLE\_EC**

```
#define DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC (DPDE_LOG_BASE + 41)
```

Failed to return a loan on a participant discovery sample.

**DPDE\_LOG\_DISPOSE\_PUBLICATION\_EC**

```
#define DPDE_LOG_DISPOSE_PUBLICATION_EC (DPDE_LOG_BASE + 42)
```

Failed to dispose an instance for a publication.

**DPDE\_LOG\_DISPOSE\_SUBSCRIPTION\_EC**

```
#define DPDE_LOG_DISPOSE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 43)
```

Failed to dispose an instance of a subscription.

**DPDE\_LOG\_ASSERT\_REMOTE\_PUBLICATION\_EC**

```
#define DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC (DPDE_LOG_BASE + 44)
```

Failed to assert and complete discovery of a remote publication.

**DPDE\_LOG\_ASSERT\_REMOTE\_SUBSCRIPTION\_EC**

```
#define DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 45)
```

Failed to assert and complete discovery of a remote subscription.

**DPDE\_LOG\_TAKE\_PUBLICATION\_SAMPLE\_EC**

```
#define DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 46)
```

Failed to take a sample from a publication discovery DataReader.

**DPDE\_LOG\_REMOVE\_REMOTE\_PUBLICATION\_EC**

```
#define DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC (DPDE_LOG_BASE + 47)
```

Failed to dispose or unregister a remote publication instance.

**DPDE\_LOG\_INVALID\_PUBLICATION\_SAMPLE\_EC**

```
#define DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 48)
```

Received a publication discovery announcement with invalid state.

**DPDE\_LOG\_RETURN\_PUBLICATION\_SAMPLE\_EC**

```
#define DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 49)
```

Failed to return a loan on a publication discovery sample.

**DPDE\_LOG\_TAKE\_SUBSCRIPTION\_SAMPLE\_EC**

```
#define DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 51)
```

Failed to take a sample from a subscription discovery DataReader.

**DPDE\_LOG\_INVALID\_SUBSCRIPTION\_SAMPLE\_EC**

```
#define DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 52)
```

Received a subscription discovery sample with invalid state.

**DPDE\_LOG\_RETURN\_SUBSCRIPTION\_SAMPLE\_EC**

```
#define DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 53)
```

Failed to return a loan on a subscription discovery sample.

**DPDE\_LOG\_REMOVE\_REMOTE\_SUBSCRIPTION\_EC**

```
#define DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 54)
```

Failed to dispose or unregister a remote subscription instance.

**DPDE\_LOG\_LOCATORS\_FULL\_EC**

```
#define DPDE_LOG_LOCATORS_FULL_EC (DPDE_LOG_BASE + 55)
```

A locator sequence is full, the remaining locators of the specified kind are dropped.

**DPDE\_LOG\_DESERIALIZE\_LOCATOR\_EC**

```
#define DPDE_LOG_DESERIALIZE_LOCATOR_EC (DPDE_LOG_BASE + 56)
```

Failed to deserialize a locator of the specified kind.

**DPDE\_LOG\_UNSUPPORTED\_LOCATOR\_EC**

```
#define DPDE_LOG_UNSUPPORTED_LOCATOR_EC (DPDE_LOG_BASE + 57)
```

The locator kind is not supported by any transport registered with the domain participant and is dropped.

**DPDE\_LOG\_ADD\_PEER\_EC**

```
#define DPDE_LOG_ADD_PEER_EC (DPDE_LOG_BASE + 58)
```

Failed to add the specified entity as a peer.

**DPDE\_LOG\_REMOVE\_PEER\_EC**

```
#define DPDE_LOG_REMOVE_PEER_EC (DPDE_LOG_BASE + 59)
```

Failed to add the specified entity as a peer.

**DPDE\_LOG\_SERIALIZE\_PRESENTATION\_EC**

```
#define DPDE_LOG_SERIALIZE_PRESENTATION_EC (DPDE_LOG_BASE + 60)
```

Failed to serialize Presentation parameter.

**DPDE\_LOG\_UNREGISTER\_TYPE\_EC**

```
#define DPDE_LOG_UNREGISTER_TYPE_EC (DPDE_LOG_BASE + 61)
```

Failed to unregister a built-in type for discovery.

**DPDE\_LOG\_ADD\_REMOTE\_PARTICIPANT\_ROUTES\_FAILED\_EC**

```
#define DPDE_LOG_ADD_REMOTE_PARTICIPANT_ROUTES_FAILED_EC (DPDE_LOG_BASE + 62)
```

Failed to add routes to a remote participant.

**DPDE\_LOG\_DELETE\_REMOTE\_PARTICIPANT\_ROUTES\_FAILED\_EC**

```
#define DPDE_LOG_DELETE_REMOTE_PARTICIPANT_ROUTES_FAILED_EC (DPDE_LOG_BASE + 63)
```

Failed to delete routes to a remote participant.

**DPDE\_LOG\_REMOVE\_REMOTE\_PARTICIPANT\_EC**

```
#define DPDE_LOG_REMOVE_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 64)
```

Failed remove a remote participant.

**DPDE\_LOG\_CHECKSUM\_INCOMPATIBLE\_PARTICIPANT\_IGNORED\_EC**

```
#define DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPDE_LOG_BASE + 65)
```

A participant with incompatible checksum was discovered and ignored.

**6.21.15 APPGEN**

APPGEN. ModuleID = 13.

**Macros**

- #define APPGEN\_LOG\_ALLOCATE\_EC (APPGEN\_LOG\_BASE + 1)  
*Failed to allocate memory for an Application Generator interface.*
- #define APPGEN\_LOG\_COPY\_ENTITY\_NAME\_EC (APPGEN\_LOG\_BASE + 2)  
*Cannot copy a name into an entity name. Name too long.*
- #define APPGEN\_LOG\_CREATE\_ENTITY\_NAME\_EC (APPGEN\_LOG\_BASE + 3)  
*Cannot generate an entity name. Name too long.*
- #define APPGEN\_MULTIPLICITY\_EC (APPGEN\_LOG\_BASE + 4)  
*Cannot create entity name because multiplicity is wrong (0)*
- #define APPGEN\_FACTORY\_REGISTER\_EC (APPGEN\_LOG\_BASE + 5)  
*Cannot register a factory.*
- #define APPGEN\_FACTORY\_UNREGISTER\_EC (APPGEN\_LOG\_BASE + 5)  
*Cannot unregister a factory.*
- #define APPGEN\_LOG\_FACTORY\_TOPIC\_NOT\_FOUND\_EC (APPGEN\_LOG\_BASE + 6)  
*Topic not found. Wrong topic name in writer configuration.*
- #define APPGEN\_LOG\_DDS\_ENTITY\_EC (APPGEN\_LOG\_BASE + 7)  
*Failed to create/delete/lookup DDS entity.*
- #define APPGEN\_LOG\_API\_ERROR\_EC (APPGEN\_LOG\_BASE + 8)  
*A call to a DDS api returned an error.*
- #define APPGEN\_LOG\_APP\_NAME\_ERROR\_EC (APPGEN\_LOG\_BASE + 9)  
*Wrong application or participant name. Not a qualified name.*
- #define APPGEN\_LOG\_LIB\_NOT\_FOUND\_EC (APPGEN\_LOG\_BASE + 10)  
*Participant cannot be created because the library name is not found in the configuration.*
- #define APPGEN\_APP\_PARTICIPANT\_NOT\_FOUND\_EC (APPGEN\_LOG\_BASE + 11)  
*Participant cannot be created because the participant name is not found in the configuration.*
- #define APPGEN\_LOG\_APP\_CONFIG\_NOT\_CORRECT\_EC (APPGEN\_LOG\_BASE + 12)  
*Application generation configuration is not correct or not consistent.*
- #define APPGEN\_LOG\_APP\_CONFIG\_POINTER\_NOT\_CORRECT\_EC (APPGEN\_LOG\_BASE + 13)  
*Application generation configuration is not correct or not consistent.*

### 6.21.15.1 Detailed Description

APPGEN. ModuleID = 13.

### 6.21.15.2 Macro Definition Documentation

#### APPGEN\_LOG\_ALLOCATE\_EC

```
#define APPGEN_LOG_ALLOCATE_EC (APPGEN_LOG_BASE + 1)
```

Failed to allocate memory for an Application Generator interface.

#### APPGEN\_LOG\_COPY\_ENTITY\_NAME\_EC

```
#define APPGEN_LOG_COPY_ENTITY_NAME_EC (APPGEN_LOG_BASE + 2)
```

Cannot copy a name into an entity name. Name too long.

#### APPGEN\_LOG\_CREATE\_ENTITY\_NAME\_EC

```
#define APPGEN_LOG_CREATE_ENTITY_NAME_EC (APPGEN_LOG_BASE + 3)
```

Cannot generate an entity name. Name too long.

#### APPGEN\_MULTIPLICITY\_EC

```
#define APPGEN_MULTIPLICITY_EC (APPGEN_LOG_BASE + 4)
```

Cannot create entity name because multiplicity is wrong (0)

#### APPGEN\_FACTORY\_REGISTER\_EC

```
#define APPGEN_FACTORY_REGISTER_EC (APPGEN_LOG_BASE + 5)
```

Cannot register a factory.

#### APPGEN\_FACTORY\_UNREGISTER\_EC

```
#define APPGEN_FACTORY_UNREGISTER_EC (APPGEN_LOG_BASE + 5)
```

Cannot unregister a factory.

**APPGEN\_LOG\_FACTORY\_TOPIC\_NOT\_FOUND\_EC**

```
#define APPGEN_LOG_FACTORY_TOPIC_NOT_FOUND_EC (APPGEN_LOG_BASE + 6)
```

Topic not found. Wrong topic name in writer configuration.

**APPGEN\_LOG\_DDS\_ENTITY\_EC**

```
#define APPGEN_LOG_DDS_ENTITY_EC (APPGEN_LOG_BASE + 7)
```

Failed to create/delete/lookup DDS entity.

**APPGEN\_LOG\_API\_ERROR\_EC**

```
#define APPGEN_LOG_API_ERROR_EC (APPGEN_LOG_BASE + 8)
```

A call to a DDS api returned an error.

**APPGEN\_LOG\_APP\_NAME\_ERROR\_EC**

```
#define APPGEN_LOG_APP_NAME_ERROR_EC (APPGEN_LOG_BASE + 9)
```

Wrong application or participant name. Not a qualified name.

**APPGEN\_LOG\_LIB\_NOT\_FOUND\_EC**

```
#define APPGEN_LOG_LIB_NOT_FOUND_EC (APPGEN_LOG_BASE + 10)
```

Participant cannot be created because the library name is not found in the configuration.

**APPGEN\_APP\_PARTICIPANT\_NOT\_FOUND\_EC**

```
#define APPGEN_APP_PARTICIPANT_NOT_FOUND_EC (APPGEN_LOG_BASE + 11)
```

Participant cannot be created because the participant name is not found in the configuration.

**APPGEN\_LOG\_APP\_CONFIG\_NOT\_CORRECT\_EC**

```
#define APPGEN_LOG_APP_CONFIG_NOT_CORRECT_EC (APPGEN_LOG_BASE + 12)
```

Application generation configuration is not correct or not consistent.

**APPGEN\_LOG\_APP\_CONFIG\_POINTER\_NOT\_CORRECT\_EC**

```
#define APPGEN_LOG_APP_CONFIG_POINTER_NOT_CORRECT_EC (APPGEN_LOG_BASE + 13)
```

Application generation configuration is not correct or not consistent.

**6.21.16 DDS\_FILTER**

DDS Filter. ModuleID = 16.

**Macros**

- #define **DDS\_FILTER\_LOG\_FILTER\_CLASS\_RECORD** 1  
*Database record for a Filter class.*
- #define **DDS\_FILTER\_LOG\_COMPILED\_FILTER\_RECORD** 2  
*Database record for a compiled filter.*
- #define **DDS\_FILTER\_LOG\_READER\_RECORD** 3  
*Database record for a reader entry.*
- #define **DDS\_FILTER\_LOG\_ROUTE\_RECORD** 4  
*Database record for a route entry.*
- #define **DDS\_FILTER\_LOG\_RECORD\_SELECT\_EC** (DDS\_FILTER\_LOG\_BASE + 1)  
*A database select on the specified record kind failed.*
- #define **DDS\_FILTER\_LOG\_RECORD\_CREATE\_EC** (DDS\_FILTER\_LOG\_BASE + 2)  
*Failed to create a database record of the specified kind.*
- #define **DDS\_FILTER\_LOG\_RECORD\_DELETE\_EC** (DDS\_FILTER\_LOG\_BASE + 3)  
*Failed to delete a database record of the specified kind.*
- #define **DDS\_FILTER\_LOG\_RECORD\_INSERT\_EC** (DDS\_FILTER\_LOG\_BASE + 4)  
*Failed to insert a database record of the specified kind.*
- #define **DDS\_FILTER\_LOG\_RECORD\_REMOVE\_EC** (DDS\_FILTER\_LOG\_BASE + 5)  
*Failed to remove a database record of the specified kind.*
- #define **DDS\_FILTER\_LOG\_TABLE\_CREATE\_EC** (DDS\_FILTER\_LOG\_BASE + 6)  
*Failed to create a database table of the specified kind.*
- #define **DDS\_FILTER\_LOG\_TABLE\_DELETE\_EC** (DDS\_FILTER\_LOG\_BASE + 7)  
*Failed to delete a database table of the specified kind.*
- #define **DDS\_FILTER\_LOG\_FILTER\_CREATE\_INSTANCE\_EC** (DDS\_FILTER\_LOG\_BASE + 10)  
*Failed to create an instance of the specified filter class.*
- #define **DDS\_FILTER\_LOG\_FILTER\_DELETE\_INSTANCE\_EC** (DDS\_FILTER\_LOG\_BASE + 11)  
*Failed to delete an instance of the specified filter class.*
- #define **DDS\_FILTER\_LOG\_FILTER\_COMPILE\_EC** (DDS\_FILTER\_LOG\_BASE + 12)  
*Failed to compile a filter of the specified class.*
- #define **DDS\_FILTER\_LOG\_FILTER\_EVALUATE\_EC** (DDS\_FILTER\_LOG\_BASE + 13)  
*Failed to evaluate a filter of the specified class.*
- #define **DDS\_FILTER\_LOG\_WRITER\_DETACH\_EC** (DDS\_FILTER\_LOG\_BASE + 14)  
*Failed to detach a writer because it still has remote readers attached.*
- #define **DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_MAX\_COUNT\_EC** (DDS\_FILTER\_LOG\_BASE + 15)  
*No more filters can be compiled because the configured maximum number of filter expressions has been reached.*



- #define `DDS_FILTER_LOG_FILTER_CLASS_REGISTER_EC` (`DDS_FILTER_LOG_BASE` + 20)  
*Failed to register a filter class because one with the same name has already been registered.*
- #define `DDS_FILTER_LOG_FILTER_CLASS_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 21)  
*Failed to find a filter class registered with the specified name.*
- #define `DDS_FILTER_LOG_FILTER_CLASS_UNREGISTER_EC` (`DDS_FILTER_LOG_BASE` + 22)  
*Failed to unregister a filter class because it is still in use.*
- #define `DDS_FILTER_LOG_CONTENT_FILTERED_TOPIC_NAME_STRING` 1  
*Content filter topic name string.*
- #define `DDS_FILTER_LOG_RELATED_TOPIC_NAME_STRING` 2  
*Content filter related topic name string.*
- #define `DDS_FILTER_LOG_FILTER_CLASS_NAME_STRING` 3  
*Filter class name string.*
- #define `DDS_FILTER_LOG_FILTER_EXPRESSION_STRING` 4  
*Filter expression string.*
- #define `DDS_FILTER_LOG_FILTER_EXPRESSION_PARAMETER_STRING` 5  
*Filter expression parameter string.*
- #define `DDS_FILTER_LOG_CONTENT_FILTER_NAME_STRING` 6  
*Content filter QoS expression name string.*
- #define `DDS_FILTER_LOG_STRING_ASSERT_EC` (`DDS_FILTER_LOG_BASE` + 30)  
*Failed to assert a string in the string manager of a specified kind.*
- #define `DDS_FILTER_LOG_STRING_DELETE_EC` (`DDS_FILTER_LOG_BASE` + 31)  
*Failed to delete a string in the string manager of a specified kind.*
- #define `DDS_FILTER_LOG_CONTENT_FILTER_NAME_TOO_LONG_EC` (`DDS_FILTER_LOG_BASE` + 32)  
*Failed to construct a content filter topic name from the combination of the related topic name and the filter name because it exceeded the maximum topic name length.*
- #define `DDS_FILTER_LOG_STRING_MANAGER_CREATE_EC` (`DDS_FILTER_LOG_BASE` + 33)  
*Failed to create a string manager for a specified string kind, max\_string\_size, and memory\_allocation.*
- #define `DDS_FILTER_LOG_FILTER_PARAMETER_COUNT_EC` (`DDS_FILTER_LOG_BASE` + 34)  
*Failed to deserialize a content filter property because it contained too many expression parameters.*
- #define `DDS_FILTER_LOG_HEAP_ALLOC_EC` (`DDS_FILTER_LOG_BASE` + 50)  
*Failed to allocate heap memory for a buffer of a given type.*
- #define `DDS_FILTER_LOG_FILTER_PLUGIN_INIT_EC` (`DDS_FILTER_LOG_BASE` + 51)  
*Failed to initialize a filter plugin.*
- #define `DDS_FILTER_LOG_FILTER_PLUGIN_FINALIZE_EC` (`DDS_FILTER_LOG_BASE` + 52)  
*Failed to finalize a filter plugin.*
- #define `DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_IN_USE_EC` (`DDS_FILTER_LOG_BASE` + 53)  
*Failed to finalize a filter plugin factory because it is still in use.*
- #define `DDS_FILTER_LOG_WRITER_FILTER_COMPILE_EC` (`DDS_FILTER_LOG_BASE` + 61)  
*A writer filter failed to compile a filter for a discovered reader.*
- #define `DDS_FILTER_LOG_WRITER_FILTER_REMOVE_READER_EC` (`DDS_FILTER_LOG_BASE` + 62)  
*Failed to remove a reader from a writer filter because it still has active routes.*
- #define `DDS_FILTER_LOG_WRITER_FILTER_SEND_GAP_EC` (`DDS_FILTER_LOG_BASE` + 63)  
*Writer failed to send a reader a GAP to indicate filtered data.*
- #define `DDS_FILTER_LOG_WRITER_SERIALIZE_INLINE_QOS_EC` (`DDS_FILTER_LOG_BASE` + 64)  
*Writer failed to serialize the content filter info into the in-line QoS.*
- #define `DDS_FILTER_LOG_WRITER_FILTER_EVALUATE_OUT_OF_ORDER_EC` (`DDS_FILTER_LOG_BASE` + 65)

- Writer filter was evaluated out of order which requires invalidating a previously results.*

  - #define `DDS_FILTER_LOG_WRITER_FILTER_STORE_RESULT_EC` (`DDS_FILTER_LOG_BASE` + 66)
- Writer filter unexpectedly failed to store the result of a filter evaluation.*

  - #define `DDS_FILTER_LOG_SQL_UNSUPPORTED_TYPE_CODE_EC` (`DDS_FILTER_LOG_BASE` + 100)
- Unsupported type code for SQL filtering.*

  - #define `DDS_FILTER_LOG_SQL_UNEXPECTED_TOKEN_EC` (`DDS_FILTER_LOG_BASE` + 101)
- Found an unexpected token while parsing an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_MAX_PREDICATES_EXCEEDED_EC` (`DDS_FILTER_LOG_BASE` + 102)
- Failed to parse an SQL filter expression because the number of predicates in the expression exceeded the configured maximum.*

  - #define `DDS_FILTER_LOG_SQL_INVALID_TOKEN_EC` (`DDS_FILTER_LOG_BASE` + 103)
- Found an invalid token while parsing an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_PARSE_PARAMETER_EC` (`DDS_FILTER_LOG_BASE` + 104)
- Failed to parse a parameter in an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_FIELD_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 105)
- Failed to parse an SQL filter expression because a given field name did not match any field in the type code of the data being filtered.*

  - #define `DDS_FILTER_LOG_SQL_UNSUPPORTED_FIELD_TYPE_EC` (`DDS_FILTER_LOG_BASE` + 106)
- A field specified in an SQL filter expression is not a supported type for use in an SQL filter.*

  - #define `DDS_FILTER_LOG_SQL_ENUM_LABEL_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 107)
- Failed to find a given enum label in an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_INCOMPATIBLE_TYPES_EC` (`DDS_FILTER_LOG_BASE` + 108)
- Comparison between two incompatible types in an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_INTEGER_OUT_OF_RANGE_EC` (`DDS_FILTER_LOG_BASE` + 109)
- Integer constant in an SQL expression is out of range for the type.*

### 6.21.16.1 Detailed Description

DDS Filter. ModuleID = 16.

### 6.21.16.2 Macro Definition Documentation

#### DDS\_FILTER\_LOG\_FILTER\_CLASS\_RECORD

```
#define DDS_FILTER_LOG_FILTER_CLASS_RECORD 1
```

Database record for a Filter class.

#### DDS\_FILTER\_LOG\_COMPILED\_FILTER\_RECORD

```
#define DDS_FILTER_LOG_COMPILED_FILTER_RECORD 2
```

Database record for a compiled filter.

**DDS\_FILTER\_LOG\_READER\_RECORD**

```
#define DDS_FILTER_LOG_READER_RECORD 3
```

Database record for a reader entry.

**DDS\_FILTER\_LOG\_ROUTE\_RECORD**

```
#define DDS_FILTER_LOG_ROUTE_RECORD 4
```

Database record for a route entry.

**DDS\_FILTER\_LOG\_RECORD\_SELECT\_EC**

```
#define DDS_FILTER_LOG_RECORD_SELECT_EC (DDS_FILTER_LOG_BASE + 1)
```

A database select on the specified record kind failed.

**DDS\_FILTER\_LOG\_RECORD\_CREATE\_EC**

```
#define DDS_FILTER_LOG_RECORD_CREATE_EC (DDS_FILTER_LOG_BASE + 2)
```

Failed to create a database record of the specified kind.

**DDS\_FILTER\_LOG\_RECORD\_DELETE\_EC**

```
#define DDS_FILTER_LOG_RECORD_DELETE_EC (DDS_FILTER_LOG_BASE + 3)
```

Failed to delete a database record of the specified kind.

**DDS\_FILTER\_LOG\_RECORD\_INSERT\_EC**

```
#define DDS_FILTER_LOG_RECORD_INSERT_EC (DDS_FILTER_LOG_BASE + 4)
```

Failed to insert a database record of the specified kind.

**DDS\_FILTER\_LOG\_RECORD\_REMOVE\_EC**

```
#define DDS_FILTER_LOG_RECORD_REMOVE_EC (DDS_FILTER_LOG_BASE + 5)
```

Failed to remove a database record of the specified kind.

**DDS\_FILTER\_LOG\_TABLE\_CREATE\_EC**

```
#define DDS_FILTER_LOG_TABLE_CREATE_EC (DDS_FILTER_LOG_BASE + 6)
```

Failed to create a database table of the specified kind.

**DDS\_FILTER\_LOG\_TABLE\_DELETE\_EC**

```
#define DDS_FILTER_LOG_TABLE_DELETE_EC (DDS_FILTER_LOG_BASE + 7)
```

Failed to delete a database table of the specified kind.

**DDS\_FILTER\_LOG\_FILTER\_CREATE\_INSTANCE\_EC**

```
#define DDS_FILTER_LOG_FILTER_CREATE_INSTANCE_EC (DDS_FILTER_LOG_BASE + 10)
```

Failed to create an instance of the specified filter class.

**DDS\_FILTER\_LOG\_FILTER\_DELETE\_INSTANCE\_EC**

```
#define DDS_FILTER_LOG_FILTER_DELETE_INSTANCE_EC (DDS_FILTER_LOG_BASE + 11)
```

Failed to delete an instance of the specified filter class.

**DDS\_FILTER\_LOG\_FILTER\_COMPILE\_EC**

```
#define DDS_FILTER_LOG_FILTER_COMPILE_EC (DDS_FILTER_LOG_BASE + 12)
```

Failed to compile a filter of the specified class.

For a reader, it is an error to not be able to compile a filter which was explicitly configured for it. For a writer performing writer-side filtering, it is only a warning that it will not be able to filter the data for the reader because it was not able to compile the filter.

**DDS\_FILTER\_LOG\_FILTER\_EVALUATE\_EC**

```
#define DDS_FILTER_LOG_FILTER_EVALUATE_EC (DDS_FILTER_LOG_BASE + 13)
```

Failed to evaluate a filter of the specified class.

**DDS\_FILTER\_LOG\_WRITER\_DETACH\_EC**

```
#define DDS_FILTER_LOG_WRITER_DETACH_EC (DDS_FILTER_LOG_BASE + 14)
```

Failed to detach a writer because it still has remote readers attached.

**DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_MAX\_COUNT\_EC**

```
#define DDS_FILTER_LOG_FILTER_EXPRESSION_MAX_COUNT_EC (DDS_FILTER_LOG_BASE + 15)
```

No more filters can be compiled because the configured maximum number of filter expressions has been reached.

**DDS\_FILTER\_LOG\_FILTER\_CLASS\_REGISTER\_EC**

```
#define DDS_FILTER_LOG_FILTER_CLASS_REGISTER_EC (DDS_FILTER_LOG_BASE + 20)
```

Failed to register a filter class because one with the same name has already been registered.

**DDS\_FILTER\_LOG\_FILTER\_CLASS\_NOT\_FOUND\_EC**

```
#define DDS_FILTER_LOG_FILTER_CLASS_NOT_FOUND_EC (DDS_FILTER_LOG_BASE + 21)
```

Failed to find a filter class registered with the specified name.

**DDS\_FILTER\_LOG\_FILTER\_CLASS\_UNREGISTER\_EC**

```
#define DDS_FILTER_LOG_FILTER_CLASS_UNREGISTER_EC (DDS_FILTER_LOG_BASE + 22)
```

Failed to unregister a filter class because it is still in use.

**DDS\_FILTER\_LOG\_CONTENT\_FILTERED\_TOPIC\_NAME\_STRING**

```
#define DDS_FILTER_LOG_CONTENT_FILTERED_TOPIC_NAME_STRING 1
```

Content filter topic name string.

**DDS\_FILTER\_LOG\_RELATED\_TOPIC\_NAME\_STRING**

```
#define DDS_FILTER_LOG_RELATED_TOPIC_NAME_STRING 2
```

Content filter related topic name string.

**DDS\_FILTER\_LOG\_FILTER\_CLASS\_NAME\_STRING**

```
#define DDS_FILTER_LOG_FILTER_CLASS_NAME_STRING 3
```

Filter class name string.

**DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_STRING**

```
#define DDS_FILTER_LOG_FILTER_EXPRESSION_STRING 4
```

Filter expression string.

**DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_PARAMETER\_STRING**

```
#define DDS_FILTER_LOG_FILTER_EXPRESSION_PARAMETER_STRING 5
```

Filter expression parameter string.

**DDS\_FILTER\_LOG\_CONTENT\_FILTER\_NAME\_STRING**

```
#define DDS_FILTER_LOG_CONTENT_FILTER_NAME_STRING 6
```

Content filter QoS expression name string.

**DDS\_FILTER\_LOG\_STRING\_ASSERT\_EC**

```
#define DDS_FILTER_LOG_STRING_ASSERT_EC (DDS_FILTER_LOG_BASE + 30)
```

Failed to assert a string in the string manager of a specified kind.

For string that are configured by the user in their QoS, this is an error that indicates that the resource limits configured were not sufficient to hold the string. For content filter's received through discovery, this is a warning that the content filter will be dropped because of inadequate resources. This could either be because the string is too long or because too many strings of that kind have been configured.

**DDS\_FILTER\_LOG\_STRING\_DELETE\_EC**

```
#define DDS_FILTER_LOG_STRING_DELETE_EC (DDS_FILTER_LOG_BASE + 31)
```

Failed to delete a string in the string manager of a specified kind.

**DDS\_FILTER\_LOG\_CONTENT\_FILTER\_NAME\_TOO\_LONG\_EC**

```
#define DDS_FILTER_LOG_CONTENT_FILTER_NAME_TOO_LONG_EC (DDS_FILTER_LOG_BASE + 32)
```

Failed to construct a content filter topic name from the combination of the related topic name and the filter name because it exceeded the maximum topic name length.

To maintain compatibility with the RTPS specification, an internal name is constructed to represent a content filter on the wire. This name is constructed by concatenating the related topic name, two colons, and the filter name. If the resulting name exceeds the maximum topic name length, then either the related topic name or the filter name must be shortened.

**DDS\_FILTER\_LOG\_STRING\_MANAGER\_CREATE\_EC**

```
#define DDS_FILTER_LOG_STRING_MANAGER_CREATE_EC (DDS_FILTER_LOG_BASE + 33)
```

Failed to create a string manager for a specified string kind, max\_string\_size, and memory\_allocation.

**DDS\_FILTER\_LOG\_FILTER\_PARAMETER\_COUNT\_EC**

```
#define DDS_FILTER_LOG_FILTER_PARAMETER_COUNT_EC (DDS_FILTER_LOG_BASE + 34)
```

Failed to deserialize a content filter property because it contained too many expression parameters.

**DDS\_FILTER\_LOG\_HEAP\_ALLOC\_EC**

```
#define DDS_FILTER_LOG_HEAP_ALLOC_EC (DDS_FILTER_LOG_BASE + 50)
```

Failed to allocate heap memory for a buffer of a given type.

**DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_INIT\_EC**

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_INIT_EC (DDS_FILTER_LOG_BASE + 51)
```

Failed to initialize a filter plugin.

**DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_FINALIZE\_EC**

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_FINALIZE_EC (DDS_FILTER_LOG_BASE + 52)
```

Failed to finalize a filter plugin.

**DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_FACTORY\_IN\_USE\_EC**

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_IN_USE_EC (DDS_FILTER_LOG_BASE + 53)
```

Failed to finalize a filter plugin factory because it is still in use.

**DDS\_FILTER\_LOG\_WRITER\_FILTER\_COMPILE\_EC**

```
#define DDS_FILTER_LOG_WRITER_FILTER_COMPILE_EC (DDS_FILTER_LOG_BASE + 61)
```

A writer filter failed to compile a filter for a discovered reader.

This is a warning that the writer will not be able to filter the data for the reader because it was not able to compile the filter.

**DDS\_FILTER\_LOG\_WRITER\_FILTER\_REMOVE\_READER\_EC**

```
#define DDS_FILTER_LOG_WRITER_FILTER_REMOVE_READER_EC (DDS_FILTER_LOG_BASE + 62)
```

Failed to remove a reader from a writer filter because it still has active routes.

**DDS\_FILTER\_LOG\_WRITER\_FILTER\_SEND\_GAP\_EC**

```
#define DDS_FILTER_LOG_WRITER_FILTER_SEND_GAP_EC (DDS_FILTER_LOG_BASE + 63)
```

Writer failed to send a reader a GAP to indicate filtered data.

**DDS\_FILTER\_LOG\_WRITER\_SERIALIZE\_INLINE\_QOS\_EC**

```
#define DDS_FILTER_LOG_WRITER_SERIALIZE_INLINE_QOS_EC (DDS_FILTER_LOG_BASE + 64)
```

Writer failed to serialize the content filter info into the in-line QoS.

**DDS\_FILTER\_LOG\_WRITER\_FILTER\_EVALUATE\_OUT\_OF\_ORDER\_EC**

```
#define DDS_FILTER_LOG_WRITER_FILTER_EVALUATE_OUT_OF_ORDER_EC (DDS_FILTER_LOG_BASE + 65)
```

Writer filter was evaluated out of order which requires invalidating a previously results.

**DDS\_FILTER\_LOG\_WRITER\_FILTER\_STORE\_RESULT\_EC**

```
#define DDS_FILTER_LOG_WRITER_FILTER_STORE_RESULT_EC (DDS_FILTER_LOG_BASE + 66)
```

Writer filter unexpectedly failed to store the result of a filter evaluation.

**DDS\_FILTER\_LOG\_SQL\_UNSUPPORTED\_TYPE\_CODE\_EC**

```
#define DDS_FILTER_LOG_SQL_UNSUPPORTED_TYPE_CODE_EC (DDS_FILTER_LOG_BASE + 100)
```

Unsupported type code for SQL filtering.

The DDS SQL filter only supports filtering on structs with the C or C++ type binding. The flat data language binding is not supported.



**DDS\_FILTER\_LOG\_SQL\_UNEXPECTED\_TOKEN\_EC**

```
#define DDS_FILTER_LOG_SQL_UNEXPECTED_TOKEN_EC (DDS_FILTER_LOG_BASE + 101)
```

Found an unexpected token while parsing an SQL filter expression.

This error indicate that the SQL parser found a token that it did not expect at the given index in the expression. This could occur for various syntax errors in the expression. Check that the expression follows the supported SQL syntax.

**DDS\_FILTER\_LOG\_SQL\_MAX\_PREDICATES\_EXCEEDED\_EC**

```
#define DDS_FILTER_LOG_SQL_MAX_PREDICATES_EXCEEDED_EC (DDS_FILTER_LOG_BASE + 102)
```

Failed to parse an SQL filter expression because the number of predicates in the expression exceeded the configured maximum.

The maximum number of predicates in an SQL filter expression is configured with `max_predicates_per_expression` in the `DDS_SqlFilterProperty` when registering SQL content filter support with `DDS_SqlContentFilter_register()`.

**DDS\_FILTER\_LOG\_SQL\_INVALID\_TOKEN\_EC**

```
#define DDS_FILTER_LOG_SQL_INVALID_TOKEN_EC (DDS_FILTER_LOG_BASE + 103)
```

Found an invalid token while parsing an SQL filter expression.

This error indicates that the SQL parser found a token that it did not recognize at the given index in the expression. This could occur for various syntax errors in the expression. Check that the expression follows the supported SQL syntax.

**DDS\_FILTER\_LOG\_SQL\_PARSE\_PARAMETER\_EC**

```
#define DDS_FILTER_LOG_SQL_PARSE_PARAMETER_EC (DDS_FILTER_LOG_BASE + 104)
```

Failed to parse a parameter in an SQL filter expression.

**DDS\_FILTER\_LOG\_SQL\_FIELD\_NOT\_FOUND\_EC**

```
#define DDS_FILTER_LOG_SQL_FIELD_NOT_FOUND_EC (DDS_FILTER_LOG_BASE + 105)
```

Failed to parse an SQL filter expression because a given field name did not match any field in the type code of the data being filtered.

This error indicates a field name in an SQL filter expression did not correspond to any field in the type code of the data being filtered. This could also occur if the specified field is not a field which can be used in an SQL filter. Only primitive, top-level, non-optional fields can be used in an SQL expression.

**DDS\_FILTER\_LOG\_SQL\_UNSUPPORTED\_FIELD\_TYPE\_EC**

```
#define DDS_FILTER_LOG_SQL_UNSUPPORTED_FIELD_TYPE_EC (DDS_FILTER_LOG_BASE + 106)
```

A field specified in an SQL filter expression is not a supported type for use in an SQL filter.

This error indicates that a field in an SQL filter expression is not a supported type for use in an SQL filter. Only primitive fields can be used in an SQL filter expression and strings are not supported.

**DDS\_FILTER\_LOG\_SQL\_ENUM\_LABEL\_NOT\_FOUND\_EC**

```
#define DDS_FILTER_LOG_SQL_ENUM_LABEL_NOT_FOUND_EC (DDS_FILTER_LOG_BASE + 107)
```

Failed to find a given enum label in an SQL filter expression.

This error indicates that an enum label specified in an SQL filter expression did not correspond to any label in type code of the enum type.

**DDS\_FILTER\_LOG\_SQL\_INCOMPATIBLE\_TYPES\_EC**

```
#define DDS_FILTER_LOG_SQL_INCOMPATIBLE_TYPES_EC (DDS_FILTER_LOG_BASE + 108)
```

Comparison between two incompatible types in an SQL filter expression.

**DDS\_FILTER\_LOG\_SQL\_INTEGER\_OUT\_OF\_RANGE\_EC**

```
#define DDS_FILTER_LOG_SQL_INTEGER_OUT_OF_RANGE_EC (DDS_FILTER_LOG_BASE + 109)
```

Integer constant in an SQL expression is out of range for the type.

The default integer type in an SQL expression is 32-bit signed integer. If additional range is needed, then the constant can be suffixed with "L" to indicate that it is a 64-bit signed integer.

## 7 Class Documentation

### 7.1 DDS\_BuiltinTopicKey\_t Struct Reference

The key type of the built-in topic types.

```
#include <dds_c_infrastructure.h>
```

**Public Attributes**

- DDS\_BUILTIN\_TOPIC\_KEY\_TYPE\_NATIVE [value](#) [DDS\_BUILTIN\_TOPIC\_KEY\_TYPE\_NATIVE\_LENGTH]  
*An array of four integers that uniquely represents a remote [DDSEntity](#).*

### 7.1.1 Detailed Description

The key type of the built-in topic types.

Each remote [DDSEntity](#) to be discovered can be uniquely identified by this key. This is the key of all the built-in topic data types.

The value of element `DDS_BUILTIN_TOPIC_KEY_OBJECT_ID` is the object ID of the remote entity. In Dynamic Participant/Static Endpoint discovery, this must be set correctly for each remote [DDSDataReader](#) and [DDSDataWriter](#) that an application intends to discover.

For example, to configure a remote [DDSDataWriter](#) with object ID 100 by setting the key in the [DDS\\_PublicationBuiltinTopicData](#)↔:

```
remote_pub_data.key.value[DDS_BUILTIN_TOPIC_KEY_OBJECT_ID] = 100;
```

See also

[DDS\\_ParticipantBuiltinTopicData](#)  
[DDS\\_PublicationBuiltinTopicData](#)  
[DDS\\_SubscriptionBuiltinTopicData](#)

### 7.1.2 Member Data Documentation

**value**

```
DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE DDS_BuiltinTopicKey_t::value[DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE↔_LENGTH]
```

An array of four integers that uniquely represents a remote [DDSEntity](#).

## 7.2 DDS\_ChecksumProperty Struct Reference

<<[eXtension](#)>> Type to define the checksum properties of the participant RTI Connex DDS Micro.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_ChecksumKind\\_t](#) `computed_crc_kind`  
*The checksum kind for the computed and sent checksum.*
- [DDS\\_ChecksumKindMask\\_t](#) `allowed_crc_mask`  
*The allowed checksum for checked checksum.*
- [DDS\\_Boolean](#) `require_crc`  
*Whether the discovered participant requires checksum or not.*

### 7.2.1 Detailed Description

`<<eXtension>>` Type to define the checksum properties of the participant RTI Connex DDS Micro.

### 7.2.2 Member Data Documentation

#### computed\_crc\_kind

`DDS_ChecksumKind_t DDS_ChecksumProperty::computed_crc_kind`

The checksum kind for the computed and sent checksum.

#### allowed\_crc\_mask

`DDS_ChecksumKindMask_t DDS_ChecksumProperty::allowed_crc_mask`

The allowed checksum for checked checksum.

#### require\_crc

`DDS_Boolean DDS_ChecksumProperty::require_crc`

Whether the discovered participant requires checksum or not.

## 7.3 DDS\_ChecksumProperty\_t Struct Reference

`<<eXtension>>` Type to define the checksum properties of the participant RTI Connex DDS Micro.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- `DDS_ChecksumKind_t computed_crc_kind`  
*The checksum kind for the computed and sent checksum.*
- `DDS_ChecksumKindMask_t allowed_crc_mask`  
*The allowed checksum for checked checksum.*
- `DDS_Boolean require_crc`  
*Whether the discovered participant requires checksum or not.*

### 7.3.1 Detailed Description

`<<eXtension>>` Type to define the checksum properties of the participant RTI Connex DDS Micro.

### 7.3.2 Member Data Documentation

#### computed\_crc\_kind

[DDS\\_ChecksumKind\\_t](#) [DDS\\_ChecksumProperty\\_t::computed\\_crc\\_kind](#)

The checksum kind for the computed and sent checksum.

#### allowed\_crc\_mask

[DDS\\_ChecksumKindMask\\_t](#) [DDS\\_ChecksumProperty\\_t::allowed\\_crc\\_mask](#)

The allowed checksum for checked checksum.

#### require\_crc

[DDS\\_Boolean](#) [DDS\\_ChecksumProperty\\_t::require\\_crc](#)

Whether the discovered participant requires checksum or not.

## 7.4 DDS\_ContentFilterProperty Struct Reference

All of the information necessary for a [DDSDDataWriter](#) to perform writer-side filtering for a [DDSDDataReader](#).

```
#include <dds_c_content_filter.h>
```

### Public Attributes

- [DDS\\_String content\\_filtered\\_topic\\_name](#)  
*Name of the content-filtered topic.*
- [DDS\\_String related\\_topic\\_name](#)  
*Name of the related (base) topic.*
- [DDS\\_String filter\\_class\\_name](#)  
*Name of the filter class to use.*
- [DDS\\_String filter\\_expression](#)  
*The filter expression string.*
- struct [DDS\\_StringSeq expression\\_parameters](#)  
*Parameters for the filter expression.*

#### 7.4.1 Detailed Description

All of the information necessary for a [DDSDDataWriter](#) to perform writer-side filtering for a [DDSDDataReader](#).

See also

[DDS\\_SubscriptionBuiltinTopicData](#)

### 7.4.2 Member Data Documentation

#### content\_filtered\_topic\_name

`DDS_String DDS_ContentFilterProperty::content_filtered_topic_name`

Name of the content-filtered topic.

For a Connex Professional [DDSDataReader](#), this is the name of the content-filtered topic that the [DDSDataReader](#) is associated with.

For interoperability with Connex Professional and the DDS standard, an RTI Connex DDS Micro [DDSDataReader](#) will include a content-filtered topic name in its discovery information even though RTI Connex DDS Micro does not support content-filtered topics. RTI Connex DDS Micro automatically derives this name for local DataReaders from the [DDS\\_ContentFilterProperty::related\\_topic\\_name](#) and [DDS\\_ContentFilterQosPolicy::filter\\_name](#). Specifically, the name will be formatted as:

```
sprintf(content_filtered_topic_name, "%s::%s", related_topic_name, filter_name);
```

If asserting a remote RTI Connex DDS Micro [DDSDataReader](#) using DPSE, you can specify the content-filtered topic name in that format to achieve writer-side filtering.

#### related\_topic\_name

`DDS_String DDS_ContentFilterProperty::related_topic_name`

Name of the related (base) topic.

Specifies the name of the base topic from which the content-filtered topic is derived, or in RTI Connex DDS Micro simply the topic that the reader is associated with.

#### filter\_class\_name

`DDS_String DDS_ContentFilterProperty::filter_class_name`

Name of the filter class to use.

Specifies which filter class should be used to evaluate the filter expression. See [DDS\\_ContentFilterQosPolicy::filter\\_class\\_name](#) for additional information.

#### filter\_expression

`DDS_String DDS_ContentFilterProperty::filter_expression`

The filter expression string.

Contains the filter expression that determines which samples are accepted by the content filter. See [DDS\\_ContentFilterQosPolicy::filter\\_expression](#) for additional information.

## expression\_parameters

```
struct DDS_StringSeq DDS_ContentFilterProperty::expression_parameters
```

Parameters for the filter expression.

A sequence of string parameters that can be referenced in the filter expression. See [DDS\\_ContentFilterQosPolicy::expression\\_parameters](#) for additional information.

## 7.5 DDS\_ContentFilterQosPolicy Struct Reference

Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.

```
#include <dds_c_content_filter.h>
```

### Public Attributes

- [DDS\\_String filter\\_name](#)  
*Name of the content filter.*
- [DDS\\_String filter\\_class\\_name](#)  
*Name of the filter class.*
- [DDS\\_String filter\\_expression](#)  
*Filter expression.*
- struct DDS\_StringSeq [expression\\_parameters](#)  
*Filter expression parameters.*

### 7.5.1 Detailed Description

Filter that allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic.

Entity:

[DDSDataReader](#)

Properties:

[RxO](#) = NO

[Changeable](#) = YES

### 7.5.2 Usage

The CONTENT\_FILTER QoS policy allows a [DDSDataReader](#) to specify that it is interested only in (potentially) a subset of the samples on its topic. Once a content filter is configured, the [DDSDataReader](#) will only receive samples that pass the filter. If content filtering is supported by the [DDSDataWriter](#), samples that do not pass the filter may not be sent at all, thus reducing the network bandwidth usage.

A [DDSDataReader](#) will fail to be created if it specifies a [DDS\\_ContentFilterQosPolicy](#) and the content filtering feature is not enabled for its parent [DDSDomainParticipant](#). It will also fail to be created if the [DDS\\_ContentFilterQosPolicy](#) specifies a filter that is not compatible with the resource limits specified in the [DDS\\_FilterQosPolicy](#) for its parent [DDSDomainParticipant](#).

Generally, RTI Connex DDS Micro does not support changing the QoS of DDS entities after they are enabled. [DDS\\_ContentFilterQosPolicy](#) is an exception to this rule, since it can be updated at runtime using [DDSDataReader::set\\_qos](#).

For all strings contained in this QoS, be sure to follow the [Conventions](#) for allocating and freeing memory. In particular, if you use [::DDS\\_DataReaderQos::finalize](#) to finalize the [DDS\\_DataReaderQos](#), it will expect that all strings in the [DDS\\_ContentFilterQosPolicy](#) are owned by the policy and will attempt to free them when finalizing the QoS. Therefore, it is recommended to use [DDS\\_String\\_dup](#) to duplicate all strings that you want to use in the [DDS\\_ContentFilterQosPolicy](#).

As an example, a content filter could be configured as follows:

```
dr_qos.content_filter.filter_name = DDS_String_dup("MyFilter");
dr_qos.content_filter.filter_class_name = DDS_String_dup(DDS_SQL_CONTENT_FILTER_CLASS);
dr_qos.content_filter.filter_expression = DDS_String_dup("a < %0");
if (!DDS_StringSeq_set_maximum(&dr_qos.content_filter.expression_parameters, 1))
{
    /* error handling */
}
if (!DDS_StringSeq_set_length(&dr_qos.content_filter.expression_parameters, 1))
{
    /* error handling */
}
*DDS_StringSeq_get_reference(&dr_qos.content_filter.expression_parameters, 0) = DDS_String_dup("10");
```

See also

[::DDSFilterPluginFactory::register](#) for how to enable the content filtering feature  
[DDS\\_FilterQosPolicy](#) for how to configure the content filtering feature

### 7.5.3 Member Data Documentation

#### filter\_name

[DDS\\_String](#) [DDS\\_ContentFilterQosPolicy::filter\\_name](#)

Name of the content filter.

A name to uniquely identify this content filter within a DomainParticipant. For interoperability with Connex Professional, it is equivalent to the *filter\_name* when configuring a content filter with XML application creation.



**filter\_class\_name**

```
DDS_String DDS_ContentFilterQosPolicy::filter_class_name
```

Name of the filter class.

The name that identifies the type of filter expression. RTI Connex DDS Micro currently only supports the “DDSQL” filter class.

**filter\_expression**

```
DDS_String DDS_ContentFilterQosPolicy::filter_expression
```

Filter expression.

A logical expression on the contents of the DataReader's Topic. If the expression evaluates to TRUE, a DDS sample is received; otherwise, it is discarded. The notation for this expression depends on the filter that you are using (specified by the [DDS\\_ContentFilterQosPolicy::filter\\_class\\_name](#)).

**expression\_parameters**

```
struct DDS_StringSeq DDS_ContentFilterQosPolicy::expression_parameters
```

Filter expression parameters.

A string sequence of filter expression parameters. Each parameter corresponds to a positional argument in the filter expression: element 0 corresponds to positional argument 0, element 1 to positional argument 1, and so forth.

## 7.6 DDS\_DataReaderInstanceReplacedStatus Struct Reference

```
<<eXtension>> DDS_INSTANCE_REPLACED_STATUS.
```

```
#include <dds_c_subscription.h>
```

**Public Attributes**

- [DDS\\_Long total\\_count](#)  
*The total number of instances that have been replaced during the life-span of the [DDSDDataReader](#).*
- [DDS\\_Long total\\_count\\_change](#)  
*The incremental number of instances replaced since the last time the listener was called or the status was read.*
- [DDS\\_InstanceHandle\\_t last\\_instance\\_handle](#)  
*The instance that is being replaced.*
- [DDS\\_InstanceHandle\\_t last\\_replacement\\_instance](#)  
*Handle to the instance which last replaced an instance.*
- [DDS\\_InstanceHandle\\_t publication\\_handle](#)  
*Handle to the publisher which last replaced an instance.*
- [DDS\\_Long lost\\_samples](#)  
*The number of samples which were lost due to the instance being replaced. Note that this number is an approximation because the middleware does not keep track of how many samples were removed prior to the instance being replaced (or the state of the samples being replaced).*

### 7.6.1 Detailed Description

<<*eXtension*>> [DDS\\_INSTANCE\\_REPLACED\\_STATUS](#).

A status structure describing the current instance replacement status for the reader.

### 7.6.2 Member Data Documentation

#### **total\_count**

[DDS\\_Long](#) [DDS\\_DataReaderInstanceReplacedStatus::total\\_count](#)

The total number of instances that have been replaced during the life-span of the [DDSDataReader](#).

#### **total\_count\_change**

[DDS\\_Long](#) [DDS\\_DataReaderInstanceReplacedStatus::total\\_count\\_change](#)

The incremental number of instances replaced since the last time the listener was called or the status was read.

#### **last\_instance\_handle**

[DDS\\_InstanceHandle\\_t](#) [DDS\\_DataReaderInstanceReplacedStatus::last\\_instance\\_handle](#)

The instance that is being replaced.

#### **last\_replacement\_instance**

[DDS\\_InstanceHandle\\_t](#) [DDS\\_DataReaderInstanceReplacedStatus::last\\_replacement\\_instance](#)

Handle to the instance which last replaced an instance.

#### **publication\_handle**

[DDS\\_InstanceHandle\\_t](#) [DDS\\_DataReaderInstanceReplacedStatus::publication\\_handle](#)

Handle to the publisher which last replaced an instance.

#### **lost\_samples**

[DDS\\_Long](#) [DDS\\_DataReaderInstanceReplacedStatus::lost\\_samples](#)

The number of samples which were lost due to the instance being replaced. Note that this number is an approximation because the middleware does not keep track of how many samples were removed prior to the instance being replaced (or the state of the samples being replaced).

## 7.7 DDS\_DataReaderProtocolQosPolicy Struct Reference

<<*eXtension*>> Protocol that applies only to [DDSDataReader](#) instances.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_UnsignedLong](#) `rtps_object_id`  
*The RTPS Object ID.*
- struct [DDS\\_RtpsReliableReaderProtocol\\_t](#) `rtps_reliable_reader`  
*RTPS protocol-related configuration settings for the RTPS reliable reader associated to a [DDSDataReader](#). This parameter only has effect if the reader is configured with [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#) [DDS\\_ReliabilityQosPolicyKind](#).*

### 7.7.1 Detailed Description

<<*eXtension*>> Protocol that applies only to [DDSDataReader](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataReader](#)

Properties:

[RxO](#) = N/A  
[Changeable](#) = NO

### 7.7.2 Member Data Documentation

#### `rtps_object_id`

[DDS\\_UnsignedLong](#) `DDS_DataReaderProtocolQosPolicy::rtps_object_id`

The RTPS Object ID.

This value is used to determine the RTPS object ID of a data reader according to the DDS-RTPS Interoperability Wire Protocol.

Only the last 3 bytes are used; the most significant byte is ignored.

[default] [DDS\\_RTPS\\_AUTO\\_ID](#)

[range] [0,0x00ffffff]

**rtps\_reliable\_reader**

```
struct DDS_RtpsReliableReaderProtocol_t DDS_DataReaderProtocolQosPolicy::rtps_reliable_reader
```

RTPS protocol-related configuration settings for the RTPS reliable reader associated to a [DDSDataReader](#). This parameter only has effect if the reader is configured with [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#) [DDS\\_ReliabilityQosPolicyKind](#).

For details, refer to the [DDS\\_RtpsReliableReaderProtocol\\_t](#)

**[default]**

nack\_period 50ms seconds;

**7.8 DDS\_DataReaderQos Struct Reference**

QoS policies supported by a [DDSDataReader](#) entity.

```
#include <dds_c_subscription.h>
```

**Public Attributes**

- struct [DDS\\_DeadlineQosPolicy](#) deadline  
*Deadline policy, [DEADLINE](#).*
- struct [DDS\\_LivelinessQosPolicy](#) liveliness  
*Liveliness policy, [LIVELINESS](#).*
- struct [DDS\\_HistoryQosPolicy](#) history  
*History policy, [HISTORY](#).*
- struct [DDS\\_ResourceLimitsQosPolicy](#) resource\_limits  
*Resource limits policy, [RESOURCE\\_LIMITS](#).*
- struct [DDS\\_OwnershipQosPolicy](#) ownership  
*Ownership policy, [OWNERSHIP](#).*
- struct [DDS\\_ReliabilityQosPolicy](#) reliability  
*Reliability policy, [RELIABILITY](#).*
- struct [DDS\\_DurabilityQosPolicy](#) durability  
*<<eXtension>> Durability policy, [DURABILITY](#).*
- struct [DDS\\_DestinationOrderQosPolicy](#) destination\_order  
*<<eXtension>> Destination Order policy, [DESTINATION\\_ORDER](#).*
- struct [DDS\\_DataRepresentationQosPolicy](#) representation  
*Data representation policy, [DATA\\_REPRESENTATION](#).*
- struct [DDS\\_DataReaderProtocolQosPolicy](#) protocol  
*<<eXtension>> [DDSDataReader](#) protocol policy, [DATA\\_READER\\_PROTOCOL](#).*
- struct [DDS\\_TransportQosPolicy](#) transport  
*<<eXtension>> Transport policy, [TRANSPORT](#).*
- struct [DDS\\_DataReaderResourceLimitsQosPolicy](#) reader\_resource\_limits  
*<<eXtension>> [DDSDataReader](#) resource limits policy, [DataReader Resource Limits](#). This policy is an extension to the DDS standard.*
- struct [DDS\\_UserDataQosPolicy](#) user\_data

User data policy, [USER\\_DATA](#).

- struct [DDS\\_PropertyQosPolicy](#) property  
    <<eXtension>> The [DDSDDataReader](#) properties.
- struct [DDS\\_ContentFilterQosPolicy](#) content\_filter  
    <<experimental>> <<eXtension>> Content filter policy, [CONTENT\\_FILTER](#).
- struct [DDS\\_EntityNameQosPolicy](#) subscription\_name  
    <<eXtension>> The [DDSDDataReader](#) name. [ENTITY\\_NAME](#).

### 7.8.1 Detailed Description

QoS policies supported by a [DDSDDataReader](#) entity.

### 7.8.2 Member Data Documentation

#### deadline

```
struct DDS\_DeadlineQosPolicy DDS_DataReaderQos::deadline
```

Deadline policy, [DEADLINE](#).

#### liveliness

```
struct DDS\_LivelinessQosPolicy DDS_DataReaderQos::liveliness
```

Liveliness policy, [LIVELINESS](#).

#### history

```
struct DDS\_HistoryQosPolicy DDS_DataReaderQos::history
```

History policy, [HISTORY](#).

#### resource\_limits

```
struct DDS\_ResourceLimitsQosPolicy DDS_DataReaderQos::resource_limits
```

Resource limits policy, [RESOURCE\\_LIMITS](#).

#### ownership

```
struct DDS\_OwnershipQosPolicy DDS_DataReaderQos::ownership
```

Ownership policy, [OWNERSHIP](#).

**reliability**

```
struct DDS_ReliabilityQosPolicy DDS_DataReaderQos::reliability
```

Reliability policy, [RELIABILITY](#).

**durability**

```
struct DDS_DurabilityQosPolicy DDS_DataReaderQos::durability
```

<<*eXtension*>> Durability policy, [DURABILITY](#).

**destination\_order**

```
struct DDS_DestinationOrderQosPolicy DDS_DataReaderQos::destination_order
```

<<*eXtension*>> Destination Order policy, [DESTINATION\\_ORDER](#).

**representation**

```
struct DDS_DataRepresentationQosPolicy DDS_DataReaderQos::representation
```

Data representation policy, [DATA\\_REPRESENTATION](#).

**protocol**

```
struct DDS_DataReaderProtocolQosPolicy DDS_DataReaderQos::protocol
```

<<*eXtension*>> [DDSDataReader](#) protocol policy, [DATA\\_READER\\_PROTOCOL](#)

**transport**

```
struct DDS_TransportQosPolicy DDS_DataReaderQos::transport
```

<<*eXtension*>> Transport policy, [TRANSPORT](#).

**reader\_resource\_limits**

```
struct DDS_DataReaderResourceLimitsQosPolicy DDS_DataReaderQos::reader_resource_limits
```

<<*eXtension*>> [DDSDataReader](#) resource limits policy, [DataReader Resource Limits](#). This policy is an extension to the DDS standard.

**user\_data**

```
struct DDS_UserDataQosPolicy DDS_DataReaderQos::user_data
```

User data policy, [USER\\_DATA](#).

**property**

```
struct DDS_PropertyQosPolicy DDS_DataReaderQos::property
```

[\*<<eXtension>>\*](#) The [DDSDataReader](#) properties.

**content\_filter**

```
struct DDS_ContentFilterQosPolicy DDS_DataReaderQos::content_filter
```

[\*<<experimental>>\*](#) [\*<<eXtension>>\*](#) Content filter policy, [CONTENT\\_FILTER](#).

If a content filter is specified, then the content filtering feature must be enabled or the [DDSDataReader](#) will fail to be created.

**See also**

[::DDSFilterPluginFactory::register](#) for how to enable the content filtering feature

**subscription\_name**

```
struct DDS_EntityNameQosPolicy DDS_DataReaderQos::subscription_name
```

[\*<<eXtension>>\*](#) The [DDSDataReader](#) name. [ENTITY\\_NAME](#).

## 7.9 DDS\_DataReaderResourceLimitsQosPolicy Struct Reference

Resource limits that apply only to [DDSDataReader](#) instances.

```
#include <dds_c_infrastructure.h>
```

**Public Attributes**

- [DDS\\_Long max\\_remote\\_writers](#)  
*The maximum number of remote writers for which the reader will maintain state and communication.*
- [DDS\\_Long max\\_remote\\_writers\\_per\\_instance](#)  
*The maximum number of remote writers for a unique instance for which the reader will maintain state and communication.*
- [DDS\\_Long max\\_samples\\_per\\_remote\\_writer](#)  
*The maximum number of untaken samples from a given remote DataWriter that a DataReader may store.*
- [DDS\\_Long max\\_outstanding\\_reads](#)  
*The maximum number of outstanding read/take calls (or one of the variants) on the same DataReader for which the memory has not been returned by calling [FooDataReader::return\\_loan](#).*
- [DDS\\_DataReaderResourceLimitsInstanceReplacementKind instance\\_replacement](#)  
*The instance replacement policy when the [DDS\\_ResourceLimitsQosPolicy::max\\_samples](#) limit has been reached and a new instance is detected.*
- [DDS\\_Long max\\_routes\\_per\\_writer](#)  
*The maximum number of routes the reader will send to per matched writer.*
- [DDS\\_Long max\\_fragmented\\_samples](#)  
*The maximum number of samples for which the built-in topic [DDSDDataReader](#) may store fragments at a given point in time.*
- [DDS\\_Long max\\_fragmented\\_samples\\_per\\_remote\\_writer](#)  
*The maximum number of samples per remote writer for which a [DDSDDataReader](#) may store fragments.*
- [DDS\\_Long shmем\\_ref\\_transfer\\_mode\\_attached\\_segment\\_allocation](#)  
*Number of shared memory segments a Zero Copy transfer over shared memory DataReader can attach to at any given time.*

**7.9.1 Detailed Description**

Resource limits that apply only to [DDSDDataReader](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDDataReader](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

**7.9.2 Member Data Documentation****max\_remote\_writers**

[DDS\\_Long](#) [DDS\\_DataReaderResourceLimitsQosPolicy::max\\_remote\\_writers](#)

The maximum number of remote writers for which the reader will maintain state and communication.

**[default]** 1

**[range]** [1, 1000000], >= max\_remote\_writers\_per\_instance



**max\_remote\_writers\_per\_instance**

`DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance`

The maximum number of remote writers for a unique instance for which the reader will maintain state and communication.

**[default]** 1

**[range]** [1, 2147483647], <= max\_remote\_writers

**max\_samples\_per\_remote\_writer**

`DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_samples_per_remote_writer`

The maximum number of untaken samples from a given remote DataWriter that a DataReader may store.

**[default]** 1

**[range]** [1, 100000000], <= `DDS_ResourceLimitsQosPolicy::max_samples`

**max\_outstanding\_reads**

`DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads`

The maximum number of outstanding read/take calls (or one of the variants) on the same DataReader for which the memory has not been returned by calling `FooDataReader::return_loan`.

**[default]** 1

**[range]** [1, 2147483647]

**instance\_replacement**

`DDS_DataReaderResourceLimitsInstanceReplacementKind DDS_DataReaderResourceLimitsQosPolicy::instance_replacement`

The instance replacement policy when the `DDS_ResourceLimitsQosPolicy::max_samples` limit has been reached and a new instance is detected.

**[default]** `DDS_NO_INSTANCE_REPLACEMENT_QOS`

**max\_routes\_per\_writer**

DDS\_Long DDS\_DataReaderResourceLimitsQosPolicy::max\_routes\_per\_writer

The maximum number of routes the reader will send to per matched writer.

NOTE: If this resource limit is exceeded when matching a [DDSDDataReader](#) with a [DDSDDataWriter](#), the two entities are not matched.

**[default]** 4

**[range]** [1, 2147483647]

**max\_fragmented\_samples**

DDS\_Long DDS\_DataReaderResourceLimitsQosPolicy::max\_fragmented\_samples

The maximum number of samples for which the built-in topic [DDSDDataReader](#) may store fragments at a given point in time.

At any given time, a built-in topic [DDSDDataReader](#) may store fragments for up to `max_fragmented_samples` samples while waiting for the remaining fragments. These samples need not have consecutive sequence numbers and may have been sent by different built-in topic [DDSDDataWriter](#) instances.

Once all fragments of a sample have been received, the sample is treated as a regular sample and becomes subject to standard QoS settings such as [DDS\\_ResourceLimitsQosPolicy::max\\_samples](#).

The middleware will drop fragments if the `max_fragmented_samples` limit has been reached. For best-effort communication, the middleware will accept a fragment for a new sample, but drop the oldest fragmented sample from the same remote writer. For reliable communication, the middleware will drop fragments for any new samples until all fragments for at least one older sample from that writer have been received.

**[default]** `max_samples * (fragments needed per sample)`

**[range]** [1, 1 million]

**max\_fragmented\_samples\_per\_remote\_writer**

DDS\_Long DDS\_DataReaderResourceLimitsQosPolicy::max\_fragmented\_samples\_per\_remote\_writer

The maximum number of samples per remote writer for which a [DDSDDataReader](#) may store fragments.

Logical limit so a single remote writer cannot consume all available resources.

**[default]** 256

**[range]** [1, 1 million],  $\leq$  `max_fragmented_samples`

## shmem\_ref\_transfer\_mode\_attached\_segment\_allocation

`DDS_Long DDS_DataReaderResourceLimitsQosPolicy::shmem_ref_transfer_mode_attached_segment_allocation`

Number of shared memory segments a Zero Copy transfer over shared memory DataReader can attach to at any given time.

The `max_count` does not limit the total number of shared memory segments used by the `DDSDataReader`. When this limit is hit, the `DDSDataReader` will try to detach from a segment that doesn't contain any loaned samples and attach to a new segment. If samples are loaned from all attached segments, then the `DDSDataReader` will fail to attach to the new segment. This scenario will result in the sample being rejected.

**[default]** `DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers` **[range]** [0, 1000000]

## 7.10 DDS\_DataRepresentationQosPolicy Struct Reference

This QoS policy contains a list of representation identifiers used by `DDSDataWriter` and `DDSDataReader` entities to negotiate which data representation to use.

```
#include <dds_c_infrastructure.h>
```

### 7.10.1 Detailed Description

This QoS policy contains a list of representation identifiers used by `DDSDataWriter` and `DDSDataReader` entities to negotiate which data representation to use.

Entity:

`DDSTopic`, `DDSDataReader`, `DDSDataWriter`

Status:

`DDS_OFFERED_INCOMPATIBLE_QOS_STATUS`, `DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS`

Properties:

`RxO` = YES

`Changeable` = UNTIL ENABLE

See also

`DATA_REPRESENTATION`

This policy has request-offer semantics. A `DDSDataWriter` may only offer a single representation. Attempting to put multiple representations in a `DDSDataWriter` will result in `DDS_RETCODE_INCONSISTENT_POLICY`. A `DDSDataWriter` will use its offered policy to communicate with its matched `DDSDataReader` entities. A `DDSDataReader` requests one or more representations. If a `DDSDataWriter` offers a representation that is contained within the sequence of the `DDSDataReader`, the offer satisfies the request and the policies are compatible. Otherwise, they are incompatible.

## 7.11 DDS\_DataWriterProtocolQosPolicy Struct Reference

<<eXtension>> Protocol that applies only to [DDSDataWriter](#) instances.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_UnsignedLong](#) `rtps_object_id`  
*The RTPS Object ID.*
- struct [DDS\\_RtpsReliableWriterProtocol\\_t](#) `rtps_reliable_writer`  
*The reliable protocol defined in RTPS.*
- [DDS\\_Boolean](#) `serialize_on_write`  
*internal, do not use*

### 7.11.1 Detailed Description

<<eXtension>> Protocol that applies only to [DDSDataWriter](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataWriter](#)

Properties:

[RxO](#) = N/A  
[Changeable](#) = NO

### 7.11.2 Member Data Documentation

#### `rtps_object_id`

[DDS\\_UnsignedLong](#) `DDS_DataWriterProtocolQosPolicy::rtps_object_id`

The RTPS Object ID.

This value is used to determine the RTPS object ID of a data writer according to the DDS-RTPS Interoperability Wire Protocol.

Only the last 3 bytes are used; the most significant byte is ignored.

**[default]** [DDS\\_RTPS\\_AUTO\\_ID](#)

**[range]** [0,0x00ffffff]

## rtps\_reliable\_writer

```
struct DDS_RtpsReliableWriterProtocol_t DDS_DataWriterProtocolQosPolicy::rtps_reliable_writer
```

The reliable protocol defined in RTPS.

```
[default] heartbeat_period 3.0 seconds;
heartbeats_per_max_samples: 1;
max_send_window: minimum(256, ::DDS_HistoryQosPolicy::depth * ::DDS_ResourceLimitsQosPolicy::max_instances);
max_heartbeat_retries: DDS_LENGTH_UNLIMITED;
first_write_sequence_number: {0,1};
```

## serialize\_on\_write

```
DDS_Boolean DDS_DataWriterProtocolQosPolicy::serialize_on_write
```

internal, do not use

## 7.12 DDS\_DataWriterQos Struct Reference

QoS policies supported by a [DDSDataWriter](#) entity.

```
#include <dds_c_publication.h>
```

### Public Attributes

- struct [DDS\\_DeadlineQosPolicy](#) deadline  
*Deadline policy, [DEADLINE](#).*
- struct [DDS\\_LivelinessQosPolicy](#) liveliness  
*Liveliness policy, [LIVELINESS](#).*
- struct [DDS\\_HistoryQosPolicy](#) history  
*History policy, [HISTORY](#).*
- struct [DDS\\_ResourceLimitsQosPolicy](#) resource\_limits  
*Resource limits policy, [RESOURCE\\_LIMITS](#).*
- struct [DDS\\_OwnershipQosPolicy](#) ownership  
*Ownership policy, [OWNERSHIP](#).*
- struct [DDS\\_OwnershipStrengthQosPolicy](#) ownership\_strength  
*Ownership strength policy, [OWNERSHIP\\_STRENGTH](#).*
- struct [DDS\\_ReliabilityQosPolicy](#) reliability  
*Reliability policy, [RELIABILITY](#).*
- struct [DDS\\_DurabilityQosPolicy](#) durability  
*<<eXtension>> Durability policy, [DURABILITY](#).*
- struct [DDS\\_DestinationOrderQosPolicy](#) destination\_order  
*<<eXtension>> Destination Order policy, [DESTINATION\\_ORDER](#).*

- struct [DDS\\_DataRepresentationQosPolicy](#) `representation`  
Data representation policy, [DATA\\_REPRESENTATION](#).
- struct [DDS\\_DataWriterProtocolQosPolicy](#) `protocol`  
<<eXtension>> [DDSDDataWriter](#) protocol policy, [DATA\\_WRITER\\_PROTOCOL](#)
- struct [DDS\\_TransportQosPolicy](#) `transport`  
Transport policy, [TRANSPORT](#). Only unicast transports are supported.
- struct [DDS\\_DataWriterResourceLimitsQosPolicy](#) `writer_resource_limits`  
<<eXtension>> Writer resource limits policy, [DataWriter Resource Limits](#).
- struct [DDS\\_PublishModeQosPolicy](#) `publish_mode`  
<<eXtension>> Publish mode policy, [PUBLISH\\_MODE](#).
- struct [DDS\\_UserDataQosPolicy](#) `user_data`  
User data policy, [USER\\_DATA](#).
- struct [DDS\\_PropertyQosPolicy](#) `property`  
<<eXtension>> The [DDSDDataWriter](#) properties. Please refer to the [Property Reference](#) for available properties.
- struct [DDS\\_EntityNameQosPolicy](#) `publication_name`  
<<eXtension>> The [DDSDDataWriter](#) name. [ENTITY\\_NAME](#).
- struct [DDS\\_DataWriterTransferModeQosPolicy](#) `transfer_mode`  
<<eXtension>> Qos related to transferring data

### 7.12.1 Detailed Description

QoS policies supported by a [DDSDDataWriter](#) entity.

You must set certain members in a consistent manner:

- [DDS\\_DataWriterQos::DDS\\_HistoryQosPolicy::depth](#) <= [DDS\\_DataWriterQos::DDS\\_ResourceLimitsQosPolicy::max\\_samples\\_per\\_instance](#)
- [DDS\\_DataWriterQos::DDS\\_ResourceLimitsQosPolicy::max\\_samples\\_per\\_instance](#) <= [DDS\\_DataWriterQos::DDS\\_ResourceLimitsQosPolicy::max\\_samples](#)

If any of the above are not true, [DDSDDataWriter::set\\_qos](#) and [DDSPublisher::set\\_default\\_datawriter\\_qos](#) will fail with [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#) and [DDSPublisher::create\\_datawriter](#) and will return NULL.

Entity:

[DDSDDataWriter](#)

See also

[QoS Policies](#) allowed ranges within each Qos.

### 7.12.2 Member Data Documentation

#### deadline

```
struct DDS\_DeadlineQosPolicy DDS_DataWriterQos::deadline
```

Deadline policy, [DEADLINE](#).

**liveliness**

```
struct DDS_LivelinessQosPolicy DDS_DataWriterQos::liveliness
```

Liveliness policy, [LIVELINESS](#).

**history**

```
struct DDS_HistoryQosPolicy DDS_DataWriterQos::history
```

History policy, [HISTORY](#).

**resource\_limits**

```
struct DDS_ResourceLimitsQosPolicy DDS_DataWriterQos::resource_limits
```

Resource limits policy, [RESOURCE\\_LIMITS](#).

**ownership**

```
struct DDS_OwnershipQosPolicy DDS_DataWriterQos::ownership
```

Ownership policy, [OWNERSHIP](#).

**ownership\_strength**

```
struct DDS_OwnershipStrengthQosPolicy DDS_DataWriterQos::ownership_strength
```

Ownership strength policy, [OWNERSHIP\\_STRENGTH](#).

**reliability**

```
struct DDS_ReliabilityQosPolicy DDS_DataWriterQos::reliability
```

Reliability policy, [RELIABILITY](#).

**durability**

```
struct DDS_DurabilityQosPolicy DDS_DataWriterQos::durability
```

<<*eXtension*>> Durability policy, [DURABILITY](#).

**destination\_order**

```
struct DDS_DestinationOrderQosPolicy DDS_DataWriterQos::destination_order
```

<<*eXtension*>> Destination Order policy, [DESTINATION\\_ORDER](#).

**representation**

```
struct DDS_DataRepresentationQosPolicy DDS_DataWriterQos::representation
```

Data representation policy, [DATA\\_REPRESENTATION](#).

**protocol**

```
struct DDS_DataWriterProtocolQosPolicy DDS_DataWriterQos::protocol
```

<<*eXtension*>> [DDSDataWriter](#) protocol policy, [DATA\\_WRITER\\_PROTOCOL](#)

**transport**

```
struct DDS_TransportQosPolicy DDS_DataWriterQos::transport
```

Transport policy, [TRANSPORT](#). Only unicast transports are supported.

**writer\_resource\_limits**

```
struct DDS_DataWriterResourceLimitsQosPolicy DDS_DataWriterQos::writer_resource_limits
```

<<*eXtension*>> Writer resource limits policy, [DataWriter Resource Limits](#).

**publish\_mode**

```
struct DDS_PublishModeQosPolicy DDS_DataWriterQos::publish_mode
```

<<*eXtension*>> Publish mode policy, [PUBLISH\\_MODE](#).

Determines whether the [DDSDataWriter](#) publishes data synchronously or asynchronously and how.

**user\_data**

```
struct DDS_UserDataQosPolicy DDS_DataWriterQos::user_data
```

User data policy, [USER\\_DATA](#).



**property**

```
struct DDS_PropertyQosPolicy DDS_DataWriterQos::property
```

<<*eXtension*>> The [DDSDataWriter](#) properties. Please refer to the [Property Reference](#) for available properties.

**publication\_name**

```
struct DDS_EntityNameQosPolicy DDS_DataWriterQos::publication_name
```

<<*eXtension*>> The [DDSDataWriter](#) name. [ENTITY\\_NAME](#).

## 7.13 DDS\_DataWriterResourceLimitsQosPolicy Struct Reference

Resource limits that apply only to [DDSDataWriter](#) instances.

```
#include <dds_c_infrastructure.h>
```

**Public Attributes**

- [DDS\\_Long max\\_remote\\_readers](#)  
*The maximum number of remote readers for which the writer will maintain state and communication.*
- [DDS\\_Long max\\_routes\\_per\\_reader](#)  
*The maximum number of routes the writer will send to per matched reader.*
- [DDS\\_Long writer\\_loaned\\_sample\\_allocation](#)  
*Represents the amount of loanable samples managed by a [DDSDataWriter](#).*
- [DDS\\_Boolean initialize\\_writer\\_loaned\\_sample](#)  
*Whether or not to initialize loaned samples returned by a [DDSDataWriter](#).*
- [DDS\\_Long max\\_remote\\_reader\\_filters](#)  
<<*experimental*>> *The maximum number of remote readers for which a [DDSDataWriter](#) will simultaneously perform writer-side filtering.*

### 7.13.1 Detailed Description

Resource limits that apply only to [DDSDataWriter](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataWriter](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

### 7.13.2 Member Data Documentation

#### max\_remote\_readers

`DDS_Long DDS_DataWriterResourceLimitsQosPolicy::max_remote_readers`

The maximum number of remote readers for which the writer will maintain state and communication.

**[default]** 16

**[range]** [1, 100000000]

#### max\_routes\_per\_reader

`DDS_Long DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader`

The maximum number of routes the writer will send to per matched reader.

NOTE: If this resource limit is exceeded when matching a [DDSDDataWriter](#) with a [DDSDDataReader](#), the two entities are not matched.

**[default]** 2

**[range]** [1, 2147483647]

#### writer\_loaned\_sample\_allocation

`DDS_Long DDS_DataWriterResourceLimitsQosPolicy::writer_loaned_sample_allocation`

Represents the amount of loanable samples managed by a [DDSDDataWriter](#).

The number of samples loaned by a [DDSDDataWriter](#) via [FooDataWriter::get\\_loan](#) is limited by [DDS\\_DataWriterResourceLimitsQosPolicy::writer\\_loaned\\_sample\\_allocation](#). [FooDataWriter::get\\_loan](#) returns NULL if and only if [DDS\\_DataWriterResourceLimitsQosPolicy::writer\\_loaned\\_sample\\_allocation](#) samples have been loaned, and none of those samples have been written with [FooDataWriter::write](#) or discarded via [FooDataWriter::discard\\_loan](#). If the default value of [DDS\\_SIZE\\_AUTO](#) is used the [DDS\\_DataWriterResourceLimitsQosPolicy::writer\\_loaned\\_sample\\_allocation](#) is set to [DDS\\_ResourceLimitsQosPolicy::max\\_samples](#) + 1.

**[default]** [DDS\\_SIZE\\_AUTO](#)

**[range]** [1, 1000000]

See also

[FooDataWriter::get\\_loan](#)

[FooDataWriter::discard\\_loan](#)

## initialize\_writer\_loaned\_sample

`DDS_Boolean DDS_DataWriterResourceLimitsQosPolicy::initialize_writer_loaned_sample`

Whether or not to initialize loaned samples returned by a [DDSDataWriter](#).

**[default]** `DDS_BOOLEAN_FALSE`

See also

[FooDataWriter::get\\_loan](#)

## max\_remote\_reader\_filters

`DDS_Long DDS_DataWriterResourceLimitsQosPolicy::max_remote_reader_filters`

*<<experimental>>* The maximum number of remote readers for which a [DDSDataWriter](#) will simultaneously perform writer-side filtering.

This QoS policy has no effect if the content filtering feature is not enabled or if [DDS\\_FilterQosPolicy::disable\\_writer\\_filtering](#) is set to `DDS_BOOLEAN_TRUE`.

This resource limit controls how many remote [DDSDataReader](#) entities a [DDSDataWriter](#) will simultaneously perform writer-side content filtering for. The [DDSDataWriter](#) will store the filter result per sample per filtered [DDSDataReader](#).

The behavior depends on the configured value:

- 0: The [DDSDataWriter](#) will not perform writer-side filtering for any [DDSDataReader](#). All content filtering will be performed on the reader side.
- 1 to  $(2^{31})-1$ : The [DDSDataWriter](#) will perform writer-side filtering for up to the specified number of [DDSDataReader](#) entities.
- `DDS_LENGTH_UNLIMITED`: The [DDSDataWriter](#) will perform writer-side filtering for up to [DDS\\_DataWriterResourceLimitsQosPolicy::max\\_remote\\_reader\\_filters](#) [DDSDataReader](#) entities.

If a [DDSDataWriter](#) is already filtering the maximum number of [DDSDataReader](#) entities specified by this limit and a new filtered [DDSDataReader](#) is discovered, the newly discovered [DDSDataReader](#) will not be filtered by the [DDSDataWriter](#), and the [DDSDataReader](#) will instead apply the content filter itself.

Once a [DDSDataReader](#) is excluded from writer-side filtering due to this resource limit, it will remain excluded even if one of the previously filtered [DDSDataReader](#) entities goes away. However, any subsequently created [DDSDataReader](#) entities will be eligible for writer-side filtering as long as the number of currently filtered [DDSDataReader](#) entities does not exceed this limit.

No matter the value of this QoS policy, a [DDSDataWriter](#) will only ever be able to apply writer-side filtering for up to 4 unique filters on the same locator.

**[default]** `DDS_LENGTH_UNLIMITED`

**[range]** `[0,  $(2^{31})-1$ ]` or `DDS_LENGTH_UNLIMITED`

See also

[::DDSFilterPluginFactory::register](#) for how to enable the content filtering feature

## 7.14 DDS\_DataWriterShmemRefTransferModeSettings Struct Reference

Settings related to transferring data using shared memory references.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Boolean enable\\_data\\_consistency\\_check](#)  
*Controls if samples can be checked for consistency.*

### 7.14.1 Detailed Description

Settings related to transferring data using shared memory references.

Properties:

RxO = N/A  
Changeable = NO

QoS:

[DDS\\_DataWriterTransferModeQosPolicy](#)

### 7.14.2 Member Data Documentation

#### **enable\_data\_consistency\_check**

[DDS\\_Boolean](#) [DDS\\_DataWriterShmemRefTransferModeSettings::enable\\_data\\_consistency\\_check](#)

Controls if samples can be checked for consistency.

When this setting is true, the [DDSDataWriter](#) sends an incrementing sequence number as an inline QoS with every sample. This sequence number allows a [DDSDataReader](#) to use the [FooDataReader::is\\_data\\_consistent](#) API to detect if the [DDSDataWriter](#) overwrote the sample before the [DDSDataReader](#) could complete processing the sample.

[default] [DDS\\_BOOLEAN\\_TRUE](#)

## 7.15 DDS\_DataWriterTransferModeQosPolicy Struct Reference

<<[eXtension](#)>> Qos related to transferring data

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- struct [DDS\\_DataWriterShmemRefTransferModeSettings](#) `shmem_ref_settings`  
*Settings related to transferring data using shared memory references.*

#### 7.15.1 Detailed Description

<<*eXtension*>> Qos related to transferring data

It contains qualitative settings related to the actions a [DDSDDataWriter](#) performs while transferring its data.

Entity:

[DDSDDataWriter](#)

Properties:

[RxO](#) = N/A  
[Changeable](#) = NO

#### 7.15.2 Member Data Documentation

##### `shmem_ref_settings`

```
struct DDS\_DataWriterShmemRefTransferModeSettings DDS_DataWriterTransferModeQosPolicy::shmem_ref←  
_settings
```

Settings related to transferring data using shared memory references.

For details, refer to the [DDS\\_DataWriterShmemRefTransferModeSettings](#)

### 7.16 DDS\_DeadlineQosPolicy Struct Reference

Expresses the maximum duration (deadline) within which an instance is expected to be updated.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- struct [DDS\\_Duration\\_t](#) `period`  
*Duration of the deadline period.*

### 7.16.1 Detailed Description

Expresses the maximum duration (deadline) within which an instance is expected to be updated.

[DDSDataReader](#) expects a new sample updating the value of each instance at least once every `period`. [DDSDataWriter](#) indicates that the application commits to write a new value (using the [DDSDataWriter](#)) for each instance managed by the [DDSDataWriter](#) at least once every `period`.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS\\_OFFERED\\_DEADLINE\\_MISSED\\_STATUS](#), [DDS\\_REQUESTED\\_DEADLINE\\_MISSED\\_STATUS](#), [DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#), [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)

Properties:

[RxO](#) = YES  
[Changeable](#) = NO

### 7.16.2 Usage

This policy is useful for cases where a [DDSTopic](#) is expected to have each instance updated periodically. On the publishing side this setting establishes a contract that the application must meet. On the subscribing side the setting establishes a minimum requirement for the remote publishers that are expected to supply the data values.

When RTI Connex DDS Micro 'matches' a [DDSDataWriter](#) and a [DDSDataReader](#) it checks whether the settings are compatible (i.e., *offered deadline*  $\leq$  *requested deadline*); if they are not, the two entities are informed (via the [DDSListener](#) mechanism) of the incompatibility of the QoS settings and communication will not occur.

Assuming that the reader and writer ends have compatible settings, the fulfilment of this contract is monitored by RTI Connex DDS Micro and the application is informed of any violations by means of the proper [DDSListener](#).

### 7.16.3 Compatibility

The value offered is considered compatible with the value requested if and only if the inequality *offered period*  $\leq$  *requested period* evaluates to 'TRUE'.

### 7.16.4 Member Data Documentation

#### **period**

```
struct DDS_Duration_t DDS_DeadlineQosPolicy::period
```

Duration of the deadline period.

**[default]** [DDS\\_DURATION\\_INFINITE](#)

**[range]** [1 nanosec, 1 year] or [DDS\\_DURATION\\_INFINITE](#)

## 7.17 DDS\_DestinationOrderQosPolicy Struct Reference

Destination order policies that affect the ordering of subscribed data.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_DestinationOrderQosPolicyKind](#) kind  
*The kind of destination ordering that applies to subscribed data.*
- struct [DDS\\_Duration\\_t](#) [source\\_timestamp\\_tolerance](#)  
*<<eXtension>> Allowed tolerance between source timestamps of consecutive samples.*

### 7.17.1 Detailed Description

Destination order policies that affect the ordering of subscribed data.

### 7.17.2 Member Data Documentation

#### kind

```
DDS_DestinationOrderQosPolicyKind DDS_DestinationOrderQosPolicy::kind
```

The kind of destination ordering that applies to subscribed data.

**[default]** [DDS\\_BY\\_RECEPTION\\_TIMESTAMP\\_DESTINATIONORDER\\_QOS](#)

#### source\_timestamp\_tolerance

```
struct DDS_Duration_t DDS_DestinationOrderQosPolicy::source_timestamp_tolerance
```

*<<eXtension>> Allowed tolerance between source timestamps of consecutive samples.*

When a [DDSDataWriter](#) sets [DDS\\_DestinationOrderQosPolicy](#) to [DDS\\_BY\\_SOURCE\\_TIMESTAMP\\_DESTINATIONORDER\\_QOS](#), when writing a sample, its timestamp must not be less than the timestamp of the previously written sample. However, if it is less than the timestamp of the previously written sample but the difference is less than this tolerance, the sample will use the previously written sample's timestamp as its timestamp. Otherwise, if the difference is greater than this tolerance, the write will fail.

[DDS\\_BY\\_SOURCE\\_TIMESTAMP\\_DESTINATIONORDER\\_QOS](#) is not support on the [DDSDataReader](#).

**[default]** 100 milliseconds for [DDSDataWriter](#)

**[range]** [1 nanosec, 1 year]

## 7.18 DDS\_DiscoveryComponent Struct Reference

Specifies the discovery plugin the [DDSDomainParticipant](#) should use.

```
#include <dds_c_domain.h>
```

### Public Attributes

- [RT\\_ComponentFactoryId\\_T](#) [name](#)  
*Specifies the name of a registered discovery plugin the [DDSDomainParticipant](#) should use.*

### 7.18.1 Detailed Description

Specifies the discovery plugin the [DDSDomainParticipant](#) should use.

### 7.18.2 Member Data Documentation

#### name

```
RT_ComponentFactoryId_T DDS_DiscoveryComponent::name
```

Specifies the name of a registered discovery plugin the [DDSDomainParticipant](#) should use.

This attribute specifies the name of the plug-in to be used for DDS Discovery. The maximum length of the name is RT\_MAX\_FACTORY\_NAME excluding the NUL termination.

## 7.19 DDS\_DiscoveryQosPolicy Struct Reference

Specifies the attributes required to discover participants in the domain.

```
#include <dds_c_domain.h>
```

### Public Attributes

- struct [DDS\\_StringSeq](#) [initial\\_peers](#)  
*Determines the initial list of peers that will be contacted by the Discovery mechanism to send announcements about the presence of this participant.*
- struct [DDS\\_StringSeq](#) [enabled\\_transports](#)  
*The transports available for use by the Discovery mechanism.*
- struct [DDS\\_DiscoveryComponent](#) [discovery](#)  
*The name of the discovery plugin that will be used to create a discovery plugin for this [DDSDomainParticipant](#).*
- [DDS\\_Boolean](#) [accept\\_unknown\\_peers](#)  
*Whether to accept a new participant that is not in the initial peer list.*
- [DDS\\_Boolean](#) [enable\\_participant\\_discovery\\_by\\_name](#)  
*Enable Participant discovery by name.*
- [DDS\\_Boolean](#) [enable\\_endpoint\\_discovery\\_queue](#)  
*Enable the endpoint discovery queue and queue endpoint discovery messages if there are no resources to assert the endpoint at the time of discovery.*



### 7.19.1 Detailed Description

Specifies the attributes required to discover participants in the domain.

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

### 7.19.2 Usage

This extended Qos is used to define how participants find each other on a network.

### 7.19.3 Member Data Documentation

#### **initial\_peers**

```
struct DDS_StringSeq DDS_DiscoveryQosPolicy::initial_peers
```

Determines the initial list of peers that will be contacted by the Discovery mechanism to send announcements about the presence of this participant.

If there is a remote peer [DDSDomainParticipant](#) such as is described this list, it will become aware of this participant and will engage in the Discovery protocol to exchange meta-data with this participant.

Each element of this list must be a `peer` descriptor in the proper format.

**[default]** Empty sequence

**[range]** Sequence of arbitrary length.

#### **enabled\_transports**

```
struct DDS_StringSeq DDS_DiscoveryQosPolicy::enabled_transports
```

The transports available for use by the Discovery mechanism.

Only these transports can be used by the discovery mechanism to send meta-traffic via the builtin endpoints (built-in [DDSDataReader](#) and [DDSDataWriter](#)).

Also determines the unicast addresses on which the Discovery mechanism will listen for meta-traffic. These along with the `domain_id` and `participant_id` determine the unicast locators on which the Discovery mechanism can receive meta-data.

The memory for the strings in this sequence is managed according to the conventions described in [Conventions](#). In particular, be careful to avoid a situation in which RTI Connex DDS Micro allocates a string on your behalf and you then reuse that string in such a way that RTI Connex DDS Micro believes it to have more memory allocated to it than it actually does.

**[default]** Empty sequence.

When an empty sequence is passed to [DDSDomainParticipantFactory::create\\_participant](#) in the [DDS\\_DomainParticipantQos](#), the domain participant will automatically add the following locators to listen for discovery traffic on: `"_udp://", "_udp↵://127.0.0.1", and "_udp://239.255.0"`.

**[range]** Sequence of non-null, non-empty strings.

**discovery**

```
struct DDS_DiscoveryComponent DDS_DiscoveryQosPolicy::discovery
```

The name of the discovery plugin that will be used to create a discovery plugin for this [DDSDomainParticipant](#).

The [DDSDomainParticipantFactory](#) will search through a list of registered discovery plugins. If it finds a registered discovery factory with a name matching this factory name, it will use that discovery plugin to create a new instance of the discovery plugin and register the plugin with the [DDSDomainParticipant](#).

If it does not find a registered discovery factory with this name, it will return [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#)

**Precondition**

One or more discovery plugins must have been previously registered with the [DDSDomainParticipantFactory](#), each with an associated name.

**accept\_unknown\_peers**

```
DDS_Boolean DDS_DiscoveryQosPolicy::accept_unknown_peers
```

Whether to accept a new participant that is not in the initial peer list.

If [DDS\\_BOOLEAN\\_FALSE](#), the participant will only communicate with those in the initial peer list

If [DDS\\_BOOLEAN\\_TRUE](#), the participant will also communicate with all discovered remote participants.

**[default]** [DDS\\_BOOLEAN\\_TRUE](#)

**enable\_participant\_discovery\_by\_name**

```
DDS_Boolean DDS_DiscoveryQosPolicy::enable_participant_discovery_by_name
```

Enable Participant discovery by name.

If [DDS\\_BOOLEAN\\_FALSE](#), a [DDSDomainParticipant](#) is only identified by its GUID.

If [DDS\\_BOOLEAN\\_TRUE](#), a [DDSDomainParticipant](#) is identified by its [DDS\\_DomainParticipantQos::participant\\_name](#). Thus, each [DDSDomainParticipant](#) in a domain must be uniquely in the DDS domain. Failure to uniquely name each [DDSDomainParticipant](#) may result in undefined behavior.

Note that [DDS\\_DomainParticipantQos::participant\\_name](#) cannot be empty when set to [DDS\\_BOOLEAN\\_TRUE](#).

**[default]** [DDS\\_BOOLEAN\\_FALSE](#)

## enable\_endpoint\_discovery\_queue

`DDS_Boolean DDS_DiscoveryQosPolicy::enable_endpoint_discovery_queue`

Enable the endpoint discovery queue and queue endpoint discovery messages if there are no resources to assert the endpoint at the time of discovery.

When a remote [DDSDataReader](#) or [DDSDataWriter](#) is discovered, the discovery information is saved, and the discovered entity is matched with local endpoints.

However, if there are insufficient resources to save the information or perform the matching, the discovery message is discarded by default. That is, all information about the discovered endpoint is lost, even if resources are later freed up.

This behavior can be changed by setting this value to [DDS\\_BOOLEAN\\_TRUE](#). If there are insufficient resources to store the discovery information, the message is queued and processed when resources becomes available.

Discovery messages that are queued are queued until the [DDSDomainParticipant](#) that published the discovery messages is no longer available.

[default] [DDS\\_BOOLEAN\\_FALSE](#)

## 7.20 DDS\_DomainParticipantFactoryQos Struct Reference

QoS policies supported by a [DDSDomainParticipantFactory](#).

```
#include <dds_c_domain.h>
```

### Public Attributes

- struct [DDS\\_EntityFactoryQosPolicy](#) `entity_factory`  
*Entity factory policy, [ENTITY\\_FACTORY](#).*
- struct [DDS\\_SystemResourceLimitsQosPolicy](#) `resource_limits`  
*<<eXtension>> System resource limits, [SYSTEM\\_RESOURCE\\_LIMITS](#).*

### 7.20.1 Detailed Description

QoS policies supported by a [DDSDomainParticipantFactory](#).

Entity:

[DDSDomainParticipantFactory](#)

See also

[QoS Policies](#) and allowed ranges within each QoS.

### 7.20.2 Member Data Documentation

#### entity\_factory

```
struct DDS_EntityFactoryQosPolicy DDS_DomainParticipantFactoryQos::entity_factory
```

Entity factory policy, [ENTITY\\_FACTORY](#).

#### resource\_limits

```
struct DDS_SystemResourceLimitsQosPolicy DDS_DomainParticipantFactoryQos::resource_limits
```

<<eXtension>> System resource limits, [SYSTEM\\_RESOURCE\\_LIMITS](#).

## 7.21 DDS\_DomainParticipantQos Struct Reference

QoS policies supported by a [DDSDomainParticipant](#) entity.

```
#include <dds_c_domain.h>
```

#### Public Attributes

- struct [DDS\\_EntityFactoryQosPolicy](#) entity\_factory  
Entity factory policy, [ENTITY\\_FACTORY](#).
- struct [DDS\\_DiscoveryQosPolicy](#) discovery  
<<eXtension>> Discovery policy, [DISCOVERY](#).
- struct [DDS\\_DomainParticipantResourceLimitsQosPolicy](#) resource\_limits  
<<eXtension>> Domain participant resource limits policy, [DOMAIN\\_PARTICIPANT\\_RESOURCE\\_LIMITS](#).
- struct [DDS\\_EntityNameQosPolicy](#) participant\_name  
<<eXtension>> The participant name. [ENTITY\\_NAME](#)
- struct [DDS\\_WireProtocolQosPolicy](#) protocol  
<<eXtension>> Wire Protocol policy, [WIRE\\_PROTOCOL](#).
- struct [DDS\\_TransportQosPolicy](#) transports  
<<eXtension>> Available transports for all communication to and from this DomainParticipant.
- struct [DDS\\_UserTrafficQosPolicy](#) user\_traffic  
<<eXtension>> Transports enabled for user traffic.
- struct [DDS\\_TrustQosPolicy](#) trust  
<<eXtension>> Available transports for all communication to and from this DomainParticipant.
- struct [DDS\\_PropertyQosPolicy](#) property  
<<eXtension>> The [DDSDomainParticipant](#) properties. Please refer to the [Property Reference](#) for available properties.
- struct [DDS\\_UserDataQosPolicy](#) user\_data  
User data policy, [USER\\_DATA](#).
- struct [DDS\\_FilterQosPolicy](#) filter  
<<experimental>> <<eXtension>> Filter policy, [FILTER](#).

### 7.21.1 Detailed Description

QoS policies supported by a [DDSDomainParticipant](#) entity.

Certain members must be set in a consistent manner:

If any of the above are not true, [DDSDomainParticipant::set\\_qos](#) and [DDSDomainParticipantFactory::set\\_default\\_participant\\_qos](#) will fail with [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#), and [DDSDomainParticipantFactory::create\\_participant](#) will fail.

Entity:

[DDSDomainParticipant](#)

See also

[QoS Policies](#) and allowed ranges within each Qos.

### 7.21.2 Member Data Documentation

#### entity\_factory

```
struct DDS\_EntityFactoryQosPolicy DDS_DomainParticipantQos::entity_factory
```

Entity factory policy, [ENTITY\\_FACTORY](#).

#### discovery

```
struct DDS\_DiscoveryQosPolicy DDS_DomainParticipantQos::discovery
```

<<*eXtension*>> Discovery policy, [DISCOVERY](#).

#### resource\_limits

```
struct DDS\_DomainParticipantResourceLimitsQosPolicy DDS_DomainParticipantQos::resource_limits
```

<<*eXtension*>> Domain participant resource limits policy, [DOMAIN\\_PARTICIPANT\\_RESOURCE\\_LIMITS](#).

#### participant\_name

```
struct DDS\_EntityNameQosPolicy DDS_DomainParticipantQos::participant_name
```

<<*eXtension*>> The participant name. [ENTITY\\_NAME](#)

**protocol**

```
struct DDS_WireProtocolQosPolicy DDS_DomainParticipantQos::protocol
```

<<*eXtension*>> Wire Protocol policy, [WIRE\\_PROTOCOL](#).

The wire protocol (RTPS) attributes associated with the participant.

**transports**

```
struct DDS_TransportQosPolicy DDS_DomainParticipantQos::transports
```

<<*eXtension*>> Available transports for all communication to and from this DomainParticipant.

**user\_traffic**

```
struct DDS_UserTrafficQosPolicy DDS_DomainParticipantQos::user_traffic
```

<<*eXtension*>> Transports enabled for user traffic.

**trust**

```
struct DDS_TrustQosPolicy DDS_DomainParticipantQos::trust
```

<<*eXtension*>> Available transports for all communication to and from this DomainParticipant.

**property**

```
struct DDS_PropertyQosPolicy DDS_DomainParticipantQos::property
```

<<*eXtension*>> The [DDSDomainParticipant](#) properties. Please refer to the [Property Reference](#) for available properties.

**user\_data**

```
struct DDS_UserDataQosPolicy DDS_DomainParticipantQos::user_data
```

User data policy, [USER\\_DATA](#).

**filter**

```
struct DDS_FilterQosPolicy DDS_DomainParticipantQos::filter
```

<<*experimental*>> <<*eXtension*>> Filter policy, [FILTER](#).

## 7.22 DDS\_DomainParticipantResourceLimitsQosPolicy Struct Reference

Resource limits that apply only to [DDSDomainParticipant](#) instances.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Long local\\_writer\\_allocation](#)  
*The maximum number of local Writers that can be created.*
- [DDS\\_Long local\\_reader\\_allocation](#)  
*The maximum number of local Readers that can be created.*
- [DDS\\_Long local\\_publisher\\_allocation](#)  
*The maximum number of local Publishers that can be created.*
- [DDS\\_Long local\\_subscriber\\_allocation](#)  
*The maximum number of local Subscribers that can be created.*
- [DDS\\_Long local\\_topic\\_allocation](#)  
*The maximum number of topics that can be created.*
- [DDS\\_Long local\\_type\\_allocation](#)  
*The maximum number of local types that can be registered with this DomainParticipant.*
- [DDS\\_Long remote\\_participant\\_allocation](#)  
*The maximum number of remote participants that can be discovered by this DomainParticipant.*
- [DDS\\_Long remote\\_writer\\_allocation](#)  
*The maximum number of remote writers that can be discovered by this DomainParticipant.*
- [DDS\\_Long remote\\_reader\\_allocation](#)  
*The maximum number of remote readers that can be discovered by this DomainParticipant.*
- [DDS\\_Long matching\\_writer\\_reader\\_pair\\_allocation](#)  
*The maximum number of matching local writer and remote/local reader pairs.*
- [DDS\\_Long matching\\_reader\\_writer\\_pair\\_allocation](#)  
*The maximum number of matching local reader and remote/local writer pairs contained in this DomainParticipant.*
- [DDS\\_Long max\\_receive\\_ports](#)  
*Maximum number of addresses that can be listened on.*
- [DDS\\_Long max\\_destination\\_ports](#)  
*Maximum number of unique destination addresses that a participant can reach.*
- [DDS\\_UnsignedLong shmем\\_ref\\_transfer\\_mode\\_max\\_segments](#)  
*Maximum number of segments created by all DataWriters belonging to a [DDSDomainParticipant](#).*
- [DDS\\_Long participant\\_user\\_data\\_max\\_length](#)  
*Maximum length of user data in [DDS\\_DomainParticipantQos](#) and [DDS\\_ParticipantBuiltinTopicData](#).*
- [DDS\\_Long participant\\_user\\_data\\_max\\_count](#)  
*Maximum number of unique user data sequences allowed across DomainParticipants (the local [DDS\\_DomainParticipantQos](#) and discovered participants).*
- [DDS\\_Long topic\\_data\\_max\\_length](#)  
*Maximum length of topic data in [DDS\\_TopicQos](#), [DDS\\_PublicationBuiltinTopicData](#), and [DDS\\_SubscriptionBuiltinTopicData](#).*
- [DDS\\_Long topic\\_data\\_max\\_count](#)  
*Maximum number of unique topic data sequences allowed in the [DDS\\_TopicQos](#) across all local topics, discovered publications, and discovered subscriptions.*
- [DDS\\_Long publisher\\_group\\_data\\_max\\_length](#)

- Maximum length of group data in [DDS\\_PublisherQos](#) and [DDS\\_PublicationBuiltinTopicData](#).
- [DDS\\_Long publisher\\_group\\_data\\_max\\_count](#)

Maximum number of unique group data sequences allowed in the [DDS\\_PublisherQos](#) across all local publishers and discovered publications.
- [DDS\\_Long subscriber\\_group\\_data\\_max\\_length](#)

Maximum length of group data in [DDS\\_SubscriberQos](#) and [DDS\\_SubscriptionBuiltinTopicData](#).
- [DDS\\_Long subscriber\\_group\\_data\\_max\\_count](#)

Maximum number of unique group data sequences allowed in the [DDS\\_SubscriberQos](#) across all local subscribers and discovered subscriptions.
- [DDS\\_Long writer\\_user\\_data\\_max\\_length](#)

Maximum length of user data in [DDS\\_DataWriterQos](#) and [DDS\\_PublicationBuiltinTopicData](#).
- [DDS\\_Long writer\\_user\\_data\\_max\\_count](#)

Maximum number of unique user data sequences allowed in the [DDS\\_DataWriterQos](#) across all local DataReaders and discovered publications.
- [DDS\\_Long reader\\_user\\_data\\_max\\_length](#)

Maximum length of user data in [DDS\\_DataReaderQos](#) and [DDS\\_SubscriptionBuiltinTopicData](#).
- [DDS\\_Long reader\\_user\\_data\\_max\\_count](#)

Maximum number of unique user data sequences allowed in the [DDS\\_DataReaderQos](#) across all local DataReaders and discovered subscriptions.
- [DDS\\_Long max\\_partitions](#)

The maximum number of elements in the [DDS\\_PartitionQosPolicy](#).
- [DDS\\_Long max\\_partition\\_cumulative\\_characters](#)

The maximum number of combined characters across all [DDS\\_PartitionQosPolicy](#) PARTITION.name sequence elements, including the NUL characters.
- [DDS\\_Long max\\_partition\\_string\\_size](#)

The maximum number of 8-bit characters in a PARTITION.name element, including the NUL termination.
- [DDS\\_Long max\\_partition\\_string\\_allocation](#)

The maximum number of 8-bit characters reserved for PARTITION.name strings to allocate for the PARTITION [DDS\\_PartitionQosPolicy](#).

### 7.22.1 Detailed Description

Resource limits that apply only to [DDSDomainParticipant](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDomainParticipant](#)

Properties:

RxO = N/A

Changeable = NO



## 7.22.2 Member Data Documentation

### local\_writer\_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_writer_allocation`

The maximum number of local Writers that can be created.

**[default]** 1

### local\_reader\_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_reader_allocation`

The maximum number of local Readers that can be created.

**[default]** 1

### local\_publisher\_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_publisher_allocation`

The maximum number of local Publishers that can be created.

**[default]** 1

### local\_subscriber\_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_subscriber_allocation`

The maximum number of local Subscribers that can be created.

**[default]** 1

### local\_topic\_allocation

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_topic_allocation`

The maximum number of topics that can be created.

**[default]** 1

**local\_type\_allocation**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_type_allocation`

The maximum number of local types that can be registered with this DomainParticipant.

**[default]** 1

**remote\_participant\_allocation**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::remote_participant_allocation`

The maximum number of remote participants that can be discovered by this DomainParticipant.

Remote DomainParticipants include all DomainParticipants, both local and remote.

**[default]** 1

**remote\_writer\_allocation**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::remote_writer_allocation`

The maximum number of remote writers that can be discovered by this DomainParticipant.

Remote DataWriters include all DataWriters, both local and remote.

**[default]** 1

**remote\_reader\_allocation**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::remote_reader_allocation`

The maximum number of remote readers that can be discovered by this DomainParticipant.

Remote DataReaders include all DataReaders, both local and remote.

**[default]** 1

**matching\_writer\_reader\_pair\_allocation**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::matching_writer_reader_pair_allocation`

The maximum number of matching local writer and remote/local reader pairs.

A resource is used for each discovery match between a writer of the participant and a reader of this or another participant.

**[default]** 32

**matching\_reader\_writer\_pair\_allocation**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::matching_reader_writer_pair_allocation`

The maximum number of matching local reader and remote/local writer pairs contained in this DomainParticipant.

A resource is used for each discovery match between a reader of the participant and a writer of this or another participant.

**[default]** 32

**max\_receive\_ports**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_receive_ports`

Maximum number of addresses that can be listened on.

Sets the maximum number of ports to receive discovery and user data on different transports. In general, each of the following counts as one resource:

- User-traffic unicast port
- Meta-traffic unicast port
- User-traffic multicast address
- Meta-traffic multicast address

**[default]** 8

**max\_destination\_ports**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_destination_ports`

Maximum number of unique destination addresses that a participant can reach.

**[default]** 8

**shmem\_ref\_transfer\_mode\_max\_segments**

`DDS_UnsignedLong DDS_DomainParticipantResourceLimitsQosPolicy::shmem_ref_transfer_mode_max_↵  
segments`

Maximum number of segments created by all DataWriters belonging to a [DDSDomainParticipant](#).

**[default]** 500

**[range]** [0,100000] but in practice, this value will be limited by the system-wide maximum number of shared memory segments. On a Linux machine, this value is provided by the kernel parameter shmmni.

**participant\_user\_data\_max\_length**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::participant_user_data_max_length`

Maximum length of user data in [DDS\\_DomainParticipantQos](#) and [DDS\\_ParticipantBuiltinTopicData](#).

**[default]** 0

**participant\_user\_data\_max\_count**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::participant_user_data_max_count`

Maximum number of unique user data sequences allowed across DomainParticipants (the local [DDS\\_DomainParticipantQos](#) and discovered participants).

For example, if a discovered participant has the same user data as the local participant, then that unique sequence only counts once against this resource-limit.

**[default]** DDS\_SIZE\_AUTO

**topic\_data\_max\_length**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::topic_data_max_length`

Maximum length of topic data in [DDS\\_TopicQos](#), [DDS\\_PublicationBuiltinTopicData](#), and [DDS\\_SubscriptionBuiltinTopicData](#).

**[default]** 0

**topic\_data\_max\_count**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::topic_data_max_count`

Maximum number of unique topic data sequences allowed in the [DDS\\_TopicQos](#) across all local topics, discovered publications, and discovered subscriptions.

For example, if a local topic has the same topic data as a the topic of a discovered publication or subscription, then that unique sequence only counts once against this resource-limit.

**[default]** DDS\_SIZE\_AUTO

**publisher\_group\_data\_max\_length**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::publisher_group_data_max_length`

Maximum length of group data in [DDS\\_PublisherQos](#) and [DDS\\_PublicationBuiltinTopicData](#).

**[default]** 0

**publisher\_group\_data\_max\_count**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::publisher_group_data_max_count`

Maximum number of unique group data sequences allowed in the [DDS\\_PublisherQos](#) across all local publishers and discovered publications.

For example, if a local publisher has the same group data as a discovered publication, then that unique sequence only counts once against this resource-limit.

**[default]** DDS\_SIZE\_AUTO

**subscriber\_group\_data\_max\_length**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::subscriber_group_data_max_length`

Maximum length of group data in [DDS\\_SubscriberQos](#) and [DDS\\_SubscriptionBuiltinTopicData](#).

**[default]** 0

**subscriber\_group\_data\_max\_count**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::subscriber_group_data_max_count`

Maximum number of unique group data sequences allowed in the [DDS\\_SubscriberQos](#) across all local subscribers and discovered subscriptions.

For example, if a local subscriber has the same group data as a discovered subscription, then that unique sequence only counts once against this resource-limit.

**[default]** DDS\_SIZE\_AUTO

**writer\_user\_data\_max\_length**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::writer_user_data_max_length`

Maximum length of user data in [DDS\\_DataWriterQos](#) and [DDS\\_PublicationBuiltinTopicData](#).

**[default]** 0

**writer\_user\_data\_max\_count**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::writer_user_data_max_count`

Maximum number of unique user data sequences allowed in the [DDS\\_DataWriterQos](#) across all local DataReaders and discovered publications.

For example, if a local DataWriter has the same user data as a discovered publication, then that unique sequence only counts once against this resource-limit.

**[default]** DDS\_SIZE\_AUTO

**reader\_user\_data\_max\_length**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::reader_user_data_max_length`

Maximum length of user data in [DDS\\_DataReaderQos](#) and [DDS\\_SubscriptionBuiltinTopicData](#).

**[default]** 0

**reader\_user\_data\_max\_count**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::reader_user_data_max_count`

Maximum number of unique user data sequences allowed in the [DDS\\_DataReaderQos](#) across all local DataReaders and discovered subscriptions.

For example, if a local DataReader has the same user data as a discovered subscription, then that unique sequence only counts once against this resource-limit.

**[default]** DDS\_SIZE\_AUTO

**max\_partitions**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_partitions`

The maximum number of elements in the [DDS\\_PartitionQosPolicy](#).

This setting is made on a per DomainParticipant basis; it cannot be set individually on a per Publisher/Subscriber basis. However, the limit is enforced and applies per Publisher/Subscriber.

This value cannot exceed 64.

**[default]** 64

**[range]** [0,64]

**max\_partition\_cumulative\_characters**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_partition_cumulative_characters`

The maximum number of combined characters across all [DDS\\_PartitionQosPolicy](#) PARTITION.name sequence elements, including the NUL characters.

The maximum number of combined characters should account for a terminating NULL ('\0') character for each partition name string.

This setting is made on a per DomainParticipant basis; it cannot be set individually on a per Publisher/Subscriber basis. However, the limit is enforced and applies per Publisher/Subscriber.

This value cannot exceed 256.

**[default]** 256

**[range]** [0,256]

**max\_partition\_string\_size**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_partition_string_size`

The maximum number of 8-bit characters in a PARTITION.name element, including the NUL termination.

**[default]** DDS\_LENGTH\_UNLIMITED

**[range]** DDS\_LENGTH\_UNLIMITED or greater than 0

**max\_partition\_string\_allocation**

`DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_partition_string_allocation`

The maximum number of 8-bit characters reserved for PARTITION.name strings to allocate for the PARTITION [DDS\\_PartitionQosPolicy](#).

If max\_partition\_string\_allocation is DDS\_LENGTH\_UNLIMITED then Memory will be allocated and freed during operation.

This setting is made on a per DomainParticipant basis; it cannot be set individually on a per Publisher/Subscriber basis. However, the limit is enforced and applies per Publisher/Subscriber.

**[default]** DDS\_LENGTH\_UNLIMITED

**[range]** DDS\_LENGTH\_UNLIMITED or greater than 1

## 7.23 DDS\_DurabilityQosPolicy Struct Reference

Durability properties that affect the life-cycle of published data.

```
#include <dds_c_infrastructure.h>
```

**Public Attributes**

- [DDS\\_DurabilityQosPolicyKind kind](#)  
*The kind of durability that applies to published data.*

### 7.23.1 Detailed Description

Durability properties that affect the life-cycle of published data.

### 7.23.2 Member Data Documentation

#### kind

`DDS_DurabilityQosPolicyKind DDS_DurabilityQosPolicy::kind`

The kind of durability that applies to published data.

## 7.24 DDS\_Duration\_t Struct Reference

Type for *duration* representation.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- `DDS_Long sec`  
*seconds*
- `DDS_UnsignedLong nanosec`  
*nanoseconds*

### 7.24.1 Detailed Description

Type for *duration* representation.

Represents a time interval.

### 7.24.2 Member Data Documentation

#### sec

`DDS_Long DDS_Duration_t::sec`

seconds

#### nanosec

`DDS_UnsignedLong DDS_Duration_t::nanosec`

nanoseconds



## 7.25 DDS\_EntityFactoryQosPolicy Struct Reference

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Boolean autoenable\\_created\\_entities](#)

*Specifies whether the entity acting as a factory automatically enables the instances it creates.*

### 7.25.1 Detailed Description

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

Entity:

[DDSDomainParticipantFactory](#), [DDSDomainParticipant](#), [DDSPublisher](#), [DDSSubscriber](#)

Properties:

[RxO](#) = NO

[Changeable](#) = YES

### 7.25.2 Usage

This policy controls the behavior of the [DDSEntity](#) as a factory for other entities.

This policy is mutable. A change in the policy affects only the entities created after the change; not the previously created entities.

The setting of `autoenable_created_entities` to [DDS\\_BOOLEAN\\_TRUE](#) indicates that the factory `create_<entity>` operation will automatically invoke the [DDSEntity::enable](#) operation each time a new [DDSEntity](#) is created. Therefore, the [DDSEntity](#) returned by `create_<entity>` will already be enabled. A setting of [DDS\\_BOOLEAN\\_FALSE](#) indicates that the [DDSEntity](#) will not be automatically enabled. The application will need to enable it explicitly by means of [DDSEntity::enable](#) operation.

The default setting of `autoenable_created_entities` = [DDS\\_BOOLEAN\\_TRUE](#) means that, by default, it is not necessary to explicitly call [DDSEntity::enable](#) on newly created entities.

### 7.25.3 Member Data Documentation

#### autoenable\_created\_entities

`DDS_Boolean DDS_EntityFactoryQosPolicy::autoenable_created_entities`

Specifies whether the entity acting as a factory automatically enables the instances it creates.

If `DDS_BOOLEAN_TRUE`, the factory will automatically enable each created entity. Otherwise, it will not.

[default] `DDS_BOOLEAN_TRUE`

## 7.26 DDS\_EntityNameQosPolicy Struct Reference

Policy used to name an entity.

```
#include <dds_c_infrastructure.h>
```

### Public Member Functions

- bool `set_name` (const char \*const `name`)  
*Set the EntityName policy.*

### Public Attributes

- char `name` [`DDS_ENTITYNAME_QOS_NAME_MAX+1`]  
*The actual name string.*

### 7.26.1 Detailed Description

Policy used to name an entity.

Entity:

`DDSDomainParticipant`

Properties:

`RxO` = NO;  
`Changeable` = NO

### 7.26.2 Usage

The purpose of this QoS is to allow the user to attach naming information to created [DDSEntity](#) objects.

It can only be 255 characters in length.

It must be a NULL-terminated string allocated with [DDS\\_String\\_alloc](#) or [DDS\\_String\\_dup](#).

If the user provides a non-NULL pointer when getting the QoS, then it should point to valid memory that can be written to, to avoid ungraceful failures.

### 7.26.3 Member Data Documentation

#### name

```
char DDS_EntityNameQosPolicy::name[DDS_ENTITYNAME_QOS_NAME_MAX+1]
```

The actual name string.

**[default]** "[ENTITY]cccc"

**[range]** Null terminated string with length not exceeding 255. Can be NULL.

Referenced by [set\\_name\(\)](#).

## 7.27 DDS\_FilterQosPolicy Struct Reference

Configures the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.

```
#include <dds_c_content_filter.h>
```

#### Public Attributes

- [RT\\_ComponentFactoryId\\_T name](#)  
*Name of the filter plugin factory to use.*
- struct [DDS\\_FilterResourceLimits resource\\_limits](#)  
*Resource limits for content filtering.*
- [DDS\\_Boolean disable\\_writer\\_filtering](#)  
*Controls whether writer-side filtering is disabled.*
- [DDS\\_Boolean disable\\_builtin\\_sql\\_filter](#)  
*Controls whether the built-in SQL filter is disabled.*

### 7.27.1 Detailed Description

Configures the content filter capabilities and resource limits for a [DDSDomainParticipant](#) and all of its child entities.

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = NO

[Changeable](#) = NO

### 7.27.2 Usage

By default if the content filtering feature is enabled, then every [DDSDomainParticipant](#) and its child entities will support content filtering. This policy allows the feature or parts of the feature to be disabled for a specific [DDSDomainParticipant](#).

It is also used to configure the resource limits for all content filters that are created by the child entities of the [DDSDomainParticipant](#). Reducing the resource limits can be useful to reduce the memory usage of the filter, or the limits can be increased to allow for more complex filters to be used.

A [DDSDataReader](#) will fail to be created if it specifies a [DDS\\_ContentFilterQosPolicy](#) that is not compatible with the resource limits specified in this policy for its parent [DDSDomainParticipant](#).

A [DDSDataWriter](#) will be unable to perform writer-side filtering for any [DDSDataReader](#) that specifies a content filter that is not compatible with the resource limits specified in this policy for its parent [DDSDomainParticipant](#).

See also

`::DDSFilterPluginFactory::register` for how to enable the content filtering feature

### 7.27.3 Member Data Documentation

**name**

`RT_ComponentFactoryId_T DDS_FilterQosPolicy::name`

Name of the filter plugin factory to use.

**[default]** [DDS\\_FILTER\\_PLUGIN\\_FACTORY\\_DEFAULT\\_NAME](#)

If the `FilterPluginFactory` is registered with this name, then the [DDSDomainParticipant](#) will use that filter plugin to support content filtering.

If it is cleared (set to an empty string), then content filtering is disabled for the [DDSDomainParticipant](#) and all of its child entities.

If it is set to a different name than [DDS\\_FILTER\\_PLUGIN\\_FACTORY\\_DEFAULT\\_NAME](#), and that factory cannot be found, then the [DDSDomainParticipant](#) will fail to be created with this policy.

**resource\_limits**

```
struct DDS_FilterResourceLimits DDS_FilterQosPolicy::resource_limits
```

Resource limits for content filtering.

**[default]** `DDS_FILTER_RESOURCE_LIMITS_DEFAULT`

Defines the resource constraints for the content filters created by the child entities of a [DDSDomainParticipant](#).

**disable\_writer\_filtering**

```
DDS_Boolean DDS_FilterQosPolicy::disable_writer_filtering
```

Controls whether writer-side filtering is disabled.

**[default]** `DDS_BOOLEAN_FALSE`

When set to `DDS_BOOLEAN_TRUE`, prevents all child [DDSDataWriter](#) entities of a [DDSDomainParticipant](#) from performing writer-side filtering. This reduces the memory usage of the content filtering feature, but also prevents the [DDSDataWriter](#) from filtering data before it is sent to a [DDSDataReader](#) that has a content filter.

**disable\_builtin\_sql\_filter**

```
DDS_Boolean DDS_FilterQosPolicy::disable_builtin_sql_filter
```

Controls whether the built-in SQL filter is disabled.

**[default]** `DDS_BOOLEAN_FALSE`

By default, a built-in SQL content filter is enabled that allows filtering based on SQL-like expressions.

When set to `DDS_BOOLEAN_TRUE`, this disables the built-in SQL content filter from being automatically enabled.

## 7.28 DDS\_FilterResourceLimits Struct Reference

Resource limits for content filtering on a [DDSDomainParticipant](#) and all of its child entities.

```
#include <dds_c_content_filter.h>
```

**Public Attributes**

- [DDS\\_Long filter\\_class\\_max\\_length](#)  
*Maximum length of a filter class name.*
- [DDS\\_Long filter\\_class\\_max\\_count](#)  
*Maximum number of filter classes supported.*
- [DDS\\_Long filter\\_expression\\_max\\_length](#)  
*Maximum length of a filter expression.*
- [DDS\\_Long filter\\_expression\\_max\\_count](#)  
*Maximum number of unique filter expressions supported.*
- [DDS\\_Long filter\\_parameter\\_max\\_count\\_per\\_expression](#)  
*Maximum number of filter parameters per filter expression.*
- [DDS\\_Long filter\\_parameter\\_max\\_length](#)  
*Maximum length of a filter parameter.*
- [DDS\\_Long sql\\_predicate\\_max\\_count\\_per\\_expression](#)  
*Maximum number of SQL predicates per filter expression.*

**7.28.1 Detailed Description**

Resource limits for content filtering on a [DDSDomainParticipant](#) and all of its child entities.

The resources used for content filtering are all pre-allocated and shared by all the child entities of a [DDSDomainParticipant](#). This includes the filters specified by a [DDSDataReader](#) in its [DDS\\_ContentFilterQosPolicy](#) and the filters discovered by a [DDSDataWriter](#) for remote [DDSDataReader](#) entities.

**7.28.2 Member Data Documentation****filter\_class\_max\_length**

[DDS\\_Long](#) `DDS_FilterResourceLimits::filter_class_max_length`

Maximum length of a filter class name.

**[default]** 7

**[range]** [1, [DDS\\_CONTENT\\_FILTER\\_NAME\\_MAX\\_LENGTH](#)]

**filter\_class\_max\_count**

[DDS\\_Long](#) `DDS_FilterResourceLimits::filter_class_max_count`

Maximum number of filter classes supported.

**[default]** 1

**[range]** [1, 2147483647]

**filter\_expression\_max\_length**

`DDS_Long DDS_FilterResourceLimits::filter_expression_max_length`

Maximum length of a filter expression.

**[default]** 63

**[range]** [1, [DDS\\_CONTENT\\_FILTER\\_EXPRESSION\\_MAX\\_LENGTH](#)]

**filter\_expression\_max\_count**

`DDS_Long DDS_FilterResourceLimits::filter_expression_max_count`

Maximum number of unique filter expressions supported.

**[default]** DDS\_LENGTH\_AUTO

**[range]** [1, 2147483647]

If set to DDS\_LENGTH\_AUTO, then enough resources will be allocated for every local and remote [DDSDataReader](#) to have a unique filter expression.

**filter\_parameter\_max\_count\_per\_expression**

`DDS_Long DDS_FilterResourceLimits::filter_parameter_max_count_per_expression`

Maximum number of filter parameters per filter expression.

**[default]** 4

**[range]** [1, [DDS\\_CONTENT\\_FILTER\\_MAX\\_PARAMETERS](#)]

This is the maximum number of parameters that can be specified in a single filter expression.

**filter\_parameter\_max\_length**

`DDS_Long DDS_FilterResourceLimits::filter_parameter_max_length`

Maximum length of a filter parameter.

**[default]** 15

**[range]** [1, 2147483647]

**sql\_predicate\_max\_count\_per\_expression**

`DDS_Long DDS_FilterResourceLimits::sql_predicate_max_count_per_expression`

Maximum number of SQL predicates per filter expression.

**[default]** 1

**[range]** [1, 2147483647]

If using the built-in SQL content filter, then this is the maximum number of SQL predicates that can be specified in a single filter expression.

If not using the built-in SQL content filter, then this value may or may not be used, depending on the filter implementation.

**7.29 DDS\_FlowControllerProperty\_t Struct Reference**

Determines the flow control characteristics of the [DDSFlowController](#).

```
#include <dds_c_flowcontroller.h>
```

**Public Attributes**

- [DDS\\_FlowControllerSchedulingPolicy scheduling\\_policy](#)  
*Scheduling policy.*
- struct [DDS\\_FlowControllerTokenBucketProperty\\_t token\\_bucket](#)  
*Settings for the token bucket.*

**7.29.1 Detailed Description**

Determines the flow control characteristics of the [DDSFlowController](#).

The flow control characteristics shape the network traffic by determining how often and in what order associated asynchronous [DDSDataWriter](#) instances are serviced and how much data they are allowed to send.

Note that these settings apply directly to the [DDSFlowController](#), and do not depend on the number of [DDSDataWriter](#) instances the [DDSFlowController](#) is servicing. For instance, the specified flow rate does *not* double simply because two [DDSDataWriter](#) instances are waiting to write.

Entity:

[DDSFlowController](#)

Properties:

RxO = N/A

**Changeable** = NO for [DDS\\_FlowControllerProperty\\_t::scheduling\\_policy](#), YES for [DDS\\_FlowControllerProperty\\_t::token\\_bucket](#). However, the special value of [DDS\\_DURATION\\_INFINITE](#) as [DDS\\_FlowControllerTokenBucketProperty\\_t::period](#) is strictly used to create an *on-demand* [DDSFlowController](#). The token period cannot toggle from an infinite to finite value (or vice versa). It can, however, change from one finite value to another.



## 7.29.2 Member Data Documentation

### scheduling\_policy

`DDS_FlowControllerSchedulingPolicy DDS_FlowControllerProperty_t::scheduling_policy`

Scheduling policy.

Determines the scheduling policy for servicing the [DDSDDataWriter](#) instances associated with the [DDSFlowController](#).

[default] [DDS\\_EDF\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#)

### token\_bucket

`struct DDS_FlowControllerTokenBucketProperty_t DDS_FlowControllerProperty_t::token_bucket`

Settings for the token bucket.

## 7.30 DDS\_FlowControllerTokenBucketProperty\_t Struct Reference

[DDSFlowController](#) uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties.

```
#include <dds_c_flowcontroller.h>
```

### Public Attributes

- [DDS\\_Long max\\_tokens](#)  
*Maximum number of tokens than can accumulate in the token bucket.*
- [DDS\\_Long tokens\\_added\\_per\\_period](#)  
*The number of tokens added to the token bucket per specified period.*
- [DDS\\_Long tokens\\_leaked\\_per\\_period](#)  
*The number of tokens removed from the token bucket per specified period.*
- struct [DDS\\_Duration\\_t period](#)  
*Period for adding tokens to and removing tokens from the bucket.*
- [DDS\\_Long bytes\\_per\\_token](#)  
*Maximum number of bytes allowed to send for each token available.*

### 7.30.1 Detailed Description

[DDSFlowController](#) uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties.

Asynchronously published samples are queued up and transmitted based on the token bucket flow control scheme. The token bucket contains tokens, each of which represents a number of bytes. Samples can be sent only when there are sufficient tokens in the bucket. As samples are sent, tokens are consumed. The number of tokens consumed is proportional to the size of the data being sent. Tokens are replenished on a periodic basis.

The rate at which tokens become available and other token bucket properties determine the network traffic flow.

Note that if the same sample must be sent to multiple destinations, separate tokens are required for each destination. Only when multiple samples are destined to the same destination will they be co-alesced and sent using the same token(s). In other words, each token can only contribute to a single network packet.

Entity:

[DDSFlowController](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = YES. However, the special value of [DDS\\_DURATION\\_INFINITE](#) as [DDS\\_FlowControllerTokenBucketProperty\\_t::period](#) is strictly used to create an *on-demand* [DDSFlowController](#). The token period cannot toggle from an infinite to finite value (or vice versa). It can, however, change from one finite value to another.

### 7.30.2 Member Data Documentation

#### **max\_tokens**

[DDS\\_Long](#) [DDS\\_FlowControllerTokenBucketProperty\\_t::max\\_tokens](#)

Maximum number of tokens than can accumulate in the token bucket.

The number of tokens in the bucket will never exceed this value. Any excess tokens are discarded. This property value, combined with [DDS\\_FlowControllerTokenBucketProperty\\_t::bytes\\_per\\_token](#), determines the maximum allowable data burst.

Use [DDS\\_LENGTH\\_UNLIMITED](#) to allow accumulation of an unlimited amount of tokens (and therefore potentially an unlimited burst size).

**[default]** [DDS\\_LENGTH\\_UNLIMITED](#)

**[range]** [1, [DDS\\_LENGTH\\_UNLIMITED](#)]

**tokens\_added\_per\_period**

```
DDS_Long DDS_FlowControllerTokenBucketProperty_t::tokens_added_per_period
```

The number of tokens added to the token bucket per specified period.

[DDSFlowController](#) transmits data only when tokens are available. Tokens are periodically replenished. This field determines the number of tokens added to the token bucket with each periodic replenishment.

Available tokens are distributed to associated [DDSDDataWriter](#) instances based on the [DDS\\_FlowControllerProperty\\_t::scheduling\\_policy](#).

Use `DDS_LENGTH_UNLIMITED` to add the maximum number of tokens allowed by [DDS\\_FlowControllerTokenBucketProperty\\_t::max\\_tokens](#).

**[default]** `DDS_LENGTH_UNLIMITED`

**[range]** `[1, DDS_LENGTH_UNLIMITED]`

**tokens\_leaked\_per\_period**

```
DDS_Long DDS_FlowControllerTokenBucketProperty_t::tokens_leaked_per_period
```

The number of tokens removed from the token bucket per specified period.

[DDSFlowController](#) transmits data only when tokens are available. When tokens are replenished and there are sufficient tokens to send all samples in the queue, this property determines whether any or all of the leftover tokens remain in the bucket.

Use `DDS_LENGTH_UNLIMITED` to remove all excess tokens from the token bucket once all samples have been sent. In other words, no token accumulation is allowed. When new samples are written after tokens were purged, the earliest point in time at which they can be sent is at the next periodic replenishment.

**[default]** `0`

**[range]** `[0, DDS_LENGTH_UNLIMITED]`

**period**

```
struct DDS_Duration_t DDS_FlowControllerTokenBucketProperty_t::period
```

Period for adding tokens to and removing tokens from the bucket.

[DDSFlowController](#) transmits data only when tokens are available. This field determines the period by which tokens are added or removed from the token bucket.

The special value [DDS\\_DURATION\\_INFINITE](#) can be used to create an *on-demand* [DDSFlowController](#), for which tokens are no longer replenished periodically. Instead, tokens must be added explicitly by calling [DDSFlowController::trigger\\_flow](#). This external trigger adds [DDS\\_FlowControllerTokenBucketProperty\\_t::tokens\\_added\\_per\\_period](#) tokens each time it is called (subject to the other property settings).

**[default]** 1 second

**[range]** `[1 nanosec, 1 year]` or [DDS\\_DURATION\\_INFINITE](#)

**bytes\_per\_token**

`DDS_Long DDS_FlowControllerTokenBucketProperty_t::bytes_per_token`

Maximum number of bytes allowed to send for each token available.

`DDSSFlowController` transmits data only when tokens are available. This field determines the number of bytes that can actually be transmitted based on the number of tokens.

Tokens are always consumed in whole by each `DDSDDataWriter`. That is, in cases where `DDS_FlowControllerTokenBucketProperty_t::bytes_per_token` is greater than the sample size, multiple samples may be sent to the same destination using a single token (regardless of `DDS_FlowControllerProperty_t::scheduling_policy`).

Where fragmentation is required, the fragment size will be `DDS_FlowControllerTokenBucketProperty_t::bytes_per_token` or the minimum largest message size across all transports installed with the `DDSDDataWriter`, whichever is less.

Use `DDS_LENGTH_UNLIMITED` to indicate that an unlimited number of bytes can be transmitted per token. In other words, a single token allows the recipient `DDSDDataWriter` to transmit all its queued samples to a single destination. A separate token is required to send to each additional destination.

**[default]** `DDS_LENGTH_UNLIMITED`

**[range]** `[1024, DDS_LENGTH_UNLIMITED]`

**7.31 DDS\_GroupDataQosPolicy Struct Reference**

Attaches a buffer of opaque data to a `DDSEntity` which is distributed to other applications as part of the discovery process.

```
#include <dds_c_user_data_qos.h>
```

**Public Attributes**

- struct `DDS_OctetSeq` [value](#)

*The opaque data to be attached to the `DDSEntity`.*

**7.31.1 Detailed Description**

Attaches a buffer of opaque data to a `DDSEntity` which is distributed to other applications as part of the discovery process.

Entity:

[DDSPublisher](#), [DDSSubscriber](#)

Properties:

[RxO](#) = NO

[Changeable](#) = NO

### 7.31.2 Usage

The GROUP\_DATA QoS policy allows an application to attach additional information to [DDSEntity](#) objects so that when a remote application discovers them, it can access that information and use it for its own purposes. This information is not interpreted by RTI Connex DDS Micro. Use cases are usually application-to-application identification, authentication, authorization, and encryption.

The group data for a remote entity is received through the discovery process. The received topic data can be accessed through the functions for accessing the discovered entity data: [DDSDataWriter::get\\_matched\\_subscription\\_data](#) and [DDSDataReader::get\\_matched\\_publication\\_data](#).

*Important:* RTI Connex DDS Micro stores the data placed in this policy in pre-allocated pools. You must configure RTI Connex DDS Micro with the maximum size and number of the data sequences that will be stored in policies of this type. These settings are configured in the [DDS\\_DomainParticipantResourceLimitsQosPolicy](#).

### 7.31.3 Member Data Documentation

#### value

```
struct DDS_OctetSeq DDS_GroupDataQosPolicy::value
```

The opaque data to be attached to the [DDSEntity](#).

**[default]** empty (zero length)

## 7.32 DDS\_GUID\_t Struct Reference

Type for *GUID* (Global Unique Identifier) representation.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Octet value](#) [DDS\_GUID\_LENGTH]  
*A 16 byte array containing the GUID value.*

### 7.32.1 Detailed Description

Type for *GUID* (Global Unique Identifier) representation.

Represents a 128 bit GUID.

### 7.32.2 Member Data Documentation

#### value

[DDS\\_Octet](#) DDS\_GUID\_t::value[DDS\_GUID\_LENGTH]

A 16 byte array containing the GUID value.

## 7.33 DDS\_HistoryQosPolicy Struct Reference

Specifies the behavior of RTI Connex DDS Micro in the case where the value of a sample changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_HistoryQosPolicyKind](#) kind  
*Specifies the kind of history to be kept.*
- [DDS\\_Long](#) depth  
*Specifies the number of samples to be kept, when the kind is [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#).*

### 7.33.1 Detailed Description

Specifies the behavior of RTI Connex DDS Micro in the case where the value of a sample changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

This `QosPolicy` controls whether RTI Connex DDS Micro should deliver only the most recent value, attempt to deliver all intermediate values, or do something in between.

On the publishing side this policy controls the samples that should be maintained by the [DDSDataWriter](#) on behalf of existing [DDSDataReader](#) entities.

On the subscribing side it controls the samples that should be maintained until the application "takes" them from RTI Connex DDS Micro.

RTI Connex DDS Micro supports [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#) kind.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Properties:

[RxO](#) = NO  
[Changeable](#) = NO

See also

[DDS\\_ReliabilityQosPolicy](#)  
[DDS\\_HistoryQosPolicy](#)

### 7.33.2 Usage

This policy controls the behavior of RTI Connex DDS Micro when the value of an instance changes before it is finally communicated to some of its existing [DDSDataReader](#) entities.

For kind set to [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#), RTI Connex DDS Micro will only attempt to keep the latest values of the instance and discard the older ones. In this case, the value of `depth` regulates the maximum number of values (up to and including the most current one) RTI Connex DDS Micro will maintain and deliver. The default (and most common setting) for `depth` is 1, indicating that only the most recent value should be delivered.

### 7.33.3 Consistency

The setting of [HISTORY](#) `depth` must be consistent with the [RESOURCE\\_LIMITS](#) `max_samples_per_instance`. For these two QoS to be consistent, they must verify that *depth*  $\leq$  *max\_samples\_per\_instance*.

See also

[DDS\\_ResourceLimitsQosPolicy](#)

### 7.33.4 Member Data Documentation

#### kind

[DDS\\_HistoryQosPolicyKind](#) `DDS_HistoryQosPolicy::kind`

Specifies the kind of history to be kept.

**[default]** [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#)

#### depth

[DDS\\_Long](#) `DDS_HistoryQosPolicy::depth`

Specifies the number of samples to be kept, when the `kind` is [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#).

If a value other than 1 (the default) is specified, it should be consistent with the settings of the [RESOURCE\\_LIMITS](#) policy. That is:

`depth`  $\leq$  [DDS\\_ResourceLimitsQosPolicy::max\\_samples\\_per\\_instance](#)

**[default]** 1

**[range]** [1,2147483647]

## 7.34 DDS\_InconsistentTopicStatus Struct Reference

### DDS\_INCONSISTENT\_TOPIC\_STATUS

```
#include <dds_c_topic.h>
```

#### 7.34.1 Detailed Description

### DDS\_INCONSISTENT\_TOPIC\_STATUS

Entity:

[DDSTopic](#)

Listener:

[DDSTopicListener](#)

A remote [DDSTopic](#) will be inconsistent with the locally created [DDSTopic](#) if the type name of the two topics are different.

## 7.35 DDS\_InstanceHandleSeq Struct Reference

Instantiates [FooSeq](#) < [DDS\\_InstanceHandle\\_t](#) > .

```
#include <dds_c_infrastructure.h>
```

#### 7.35.1 Detailed Description

Instantiates [FooSeq](#) < [DDS\\_InstanceHandle\\_t](#) > .

Instantiates:

[<<generic>> FooSeq](#)

See also

[DDS\\_InstanceHandle\\_t](#)

[FooSeq](#)

## 7.36 DDS\_LatencyBudgetQosPolicy Struct Reference

Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

```
#include <dds_c_infrastructure.h>
```



## Public Attributes

- struct [DDS\\_Duration\\_t duration](#)  
*Duration of the maximum acceptable delay.*

### 7.36.1 Detailed Description

Provides a hint as to the maximum acceptable delay from the time the data is written to the time it is received by the subscribing applications.

This policy is a *hint* to a DDS implementation; it can be used to change how it processes and sends data that has low latency requirements. The DDS specification does not mandate whether or how this policy is used.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#), [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)

Properties:

[Rxo](#) = YES  
[Changeable](#) = YES

See also

[DDS\\_PublishModeQosPolicy](#)  
[DDSFlowController](#)

### 7.36.2 Usage

This policy provides a means for the application to indicate to the middleware the urgency of the data communication. By having a non-zero `duration`, RTI Connex DDS Micro can optimize its internal operation.

RTI Connex DDS Micro uses it in conjunction with [DDS\\_ASYNCHRONOUS\\_PUBLISH\\_MODE\\_QOS](#) [DDSDataWriter](#) instances associated with a [DDS\\_EDF\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#) [DDSFlowController](#) only. Together with the time of write, [DDS\\_LatencyBudgetQosPolicy::duration](#) determines the deadline of each individual sample. RTI Connex DDS Micro uses this information to prioritize the sending of asynchronously published data;

### 7.36.3 Compatibility

The value offered is considered compatible with the value requested if and only if the inequality *offered duration* ≤ *requested duration* evaluates to 'TRUE'.

### 7.36.4 Member Data Documentation

#### duration

```
struct DDS_Duration_t DDS_LatencyBudgetQosPolicy::duration
```

Duration of the maximum acceptable delay.

**[default]** 0 (meaning minimize the delay)

## 7.37 DDS\_LivelinessChangedStatus Struct Reference

### DDS\_LIVELINESS\_CHANGED\_STATUS

```
#include <dds_c_subscription.h>
```

#### Public Attributes

- [DDS\\_Long alive\\_count](#)  
The total count of currently alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.
- [DDS\\_Long not\\_alive\\_count](#)  
The total count of currently not\_alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.
- [DDS\\_Long alive\\_count\\_change](#)  
The change in the `alive_count` since the last time the listener was called or the status was read.
- [DDS\\_Long not\\_alive\\_count\\_change](#)  
The change in the `not_alive_count` since the last time the listener was called or the status was read.
- [DDS\\_InstanceHandle\\_t last\\_publication\\_handle](#)  
An instance handle to the last remote writer to change its liveliness.

### 7.37.1 Detailed Description

### DDS\_LIVELINESS\_CHANGED\_STATUS

### 7.37.2 Member Data Documentation

#### alive\_count

```
DDS_Long DDS_LivelinessChangedStatus::alive_count
```

The total count of currently alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.

**not\_alive\_count**

`DDS_Long DDS_LivelinessChangedStatus::not_alive_count`

The total count of currently not\_alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.

**alive\_count\_change**

`DDS_Long DDS_LivelinessChangedStatus::alive_count_change`

The change in the alive\_count since the last time the listener was called or the status was read.

**not\_alive\_count\_change**

`DDS_Long DDS_LivelinessChangedStatus::not_alive_count_change`

The change in the not\_alive\_count since the last time the listener was called or the status was read.

**last\_publication\_handle**

`DDS_InstanceHandle_t DDS_LivelinessChangedStatus::last_publication_handle`

An instance handle to the last remote writer to change its liveliness.

## 7.38 DDS\_LivelinessLostStatus Struct Reference

### [DDS\\_LIVELINESS\\_LOST\\_STATUS](#)

```
#include <dds_c_publication.h>
```

**Public Attributes**

- [DDS\\_Long total\\_count](#)

*Total cumulative number of times that a previously-alive [DDSDDataWriter](#) became not alive due to a failure to to actively signal its liveliness within the offered liveliness period.*

- [DDS\\_Long total\\_count\\_change](#)

*The incremental changees in total\_count since the last time the listener was called or the status was read.*

### 7.38.1 Detailed Description

#### DDS\_LIVELINESS\_LOST\_STATUS

Entity:

[DDSDataWriter](#)

Listener:

[DDSDataWriterListener](#)

The liveliness that the [DDSDataWriter](#) has committed through its [DDS\\_LivelinessQosPolicy](#) was not respected; thus [DDSDataReader](#) entities will consider the [DDSDataWriter](#) as no longer "alive/active".

### 7.38.2 Member Data Documentation

#### total\_count

[DDS\\_Long](#) [DDS\\_LivelinessLostStatus::total\\_count](#)

Total cumulative number of times that a previously-alive [DDSDataWriter](#) became not alive due to a failure to to actively signal its liveliness within the offered liveliness period.

This count does not change when an already not alive [DDSDataWriter](#) simply remains not alive for another liveliness period.

#### total\_count\_change

[DDS\\_Long](#) [DDS\\_LivelinessLostStatus::total\\_count\\_change](#)

The incremental changes in total\_count since the last time the listener was called or the status was read.

## 7.39 DDS\_LivelinessQosPolicy Struct Reference

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_LivelinessQosPolicyKind](#) kind  
*The kind of liveliness desired.*
- struct [DDS\\_Duration\\_t](#) lease\_duration  
*The duration within which a [DDSEntity](#) must be asserted, or else it is assumed to be not alive.*

### 7.39.1 Detailed Description

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

Liveliness must be asserted at least once every `lease_duration` otherwise RTI Connex DDS Micro will assume the corresponding [DDSEntity](#) is no longer alive.

The liveliness status of a [DDSEntity](#) is used to maintain instance ownership in combination with the setting of the [OWNERSHIP](#) policy. The application is also informed via [DDSListener](#) when an [DDSEntity](#) is no longer alive.

A [DDSDataReader](#) requests that liveliness of writers is maintained by the requested means and loss of liveliness is detected with delay not to exceed the `lease_duration`.

A [DDSDataWriter](#) commits to signalling its liveliness using the stated means at intervals not to exceed the `lease_duration`.

Listeners are used to notify a [DDSDataReader](#) of loss of liveliness and [DDSDataWriter](#) of violations to the liveliness contract. The `on_liveliness_lost()` callback is only called *once*, after the first time the `lease_duration` is exceeded (when the [DDSDataWriter](#) first loses liveliness).

This QoS policy can be used during system integration to ensure that applications have been coded to meet design specifications. It can also be used during run time to detect when systems are performing outside of design specifications. Receiving applications can take appropriate actions in response to disconnected DataWriters.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS\\_LIVELINESS\\_LOST\\_STATUS](#), [DDS\\_LivelinessLostStatus](#);  
[DDS\\_LIVELINESS\\_CHANGED\\_STATUS](#), [DDS\\_LivelinessChangedStatus](#);  
[DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#), [DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)

Properties:

[RxO](#) = YES  
[Changeable](#) = NO

### 7.39.2 Usage

This policy controls the mechanism and parameters used by RTI Connex DDS Micro to ensure that particular entities on the network are still alive. The liveliness can also affect the ownership of a particular instance, as determined by the [OWNERSHIP](#) policy.

This policy has several settings to support both data types that are updated periodically as well as those that are changed sporadically. It also allows customisation for different application requirements in terms of the kinds of failures that will be detected by the liveliness mechanism.

The [DDS\\_AUTOMATIC\\_LIVELINESS\\_QOS](#) liveliness setting is most appropriate for applications that only need to detect failures at the process-level, but not application-logic failures within a process. RTI Connex DDS Micro takes responsibility for renewing the leases at the required rates and thus, as long as the local process where a [DDSDomainParticipant](#) is running and the link connecting it to remote participants remains connected, the entities within the [DDSDomainParticipant](#) will be considered alive. This requires the lowest overhead.

The manual settings ([DDS\\_MANUAL\\_BY\\_PARTICIPANT\\_LIVELINESS\\_QOS](#), [DDS\\_MANUAL\\_BY\\_TOPIC\\_LIVELINESS\\_QOS](#)) require the application on the publishing side to periodically assert the liveliness before the lease expires to indicate the corresponding [DDSEntity](#) is still alive. The action can be explicit by calling the [DDSDataWriter::assert\\_liveliness](#) operation or implicit by writing some data.

The two possible manual settings control the granularity at which the application must assert liveliness.

- The setting [DDS\\_MANUAL\\_BY\\_PARTICIPANT\\_LIVELINESS\\_QOS](#) requires only that one [DDSEntity](#) within a participant is asserted to be alive to deduce all other [DDSEntity](#) objects within the same [DDSDomainParticipant](#) are also alive.
- The setting [DDS\\_MANUAL\\_BY\\_TOPIC\\_LIVELINESS\\_QOS](#) requires that at least one instance within the [DDSDataWriter](#) is asserted.

Changes in [LIVELINESS](#) must be detected by the Service with a time-granularity greater or equal to the `lease_↔duration`. This ensures that the value of the [DDS\\_LivelinessChangedStatus](#) is updated at least once during each `lease_duration` and the related Listeners are notified within a `lease_duration` from the time the [LIVELINESS](#) changed.

### 7.39.3 Compatibility

The value offered is considered compatible with the value requested if and only if the following conditions are met:

- the inequality *offered kind*  $\geq$  *requested kind* evaluates to 'TRUE'. For the purposes of this inequality, the values of [DDS\\_LivelinessQosPolicyKind](#) kind are considered ordered such that: [DDS\\_AUTOMATIC\\_LIVELINESS\\_QOS](#) < [DDS\\_MANUAL\\_BY\\_PARTICIPANT\\_LIVELINESS\\_QOS](#) < [DDS\\_MANUAL\\_BY\\_TOPIC\\_LIVELINESS\\_QOS](#).
- the inequality *offered lease\_duration*  $\leq$  *requested lease\_duration* evaluates to [DDS\\_BOOLEAN\\_TRUE](#).

#### 7.39.4 Member Data Documentation

##### kind

`DDS_LivelinessQosPolicyKind DDS_LivelinessQosPolicy::kind`

The kind of liveliness desired.

[default] `DDS_AUTOMATIC_LIVELINESS_QOS`

##### lease\_duration

`struct DDS_Duration_t DDS_LivelinessQosPolicy::lease_duration`

The duration within which a `DDSEntity` must be asserted, or else it is assumed to be not alive.

[default] `DDS_DURATION_INFINITE`

[range] [0,1 year] or `DDS_DURATION_INFINITE`

### 7.40 DDS\_Locator\_t Struct Reference

`<<eXtension>>` Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports.

```
#include <dds_c_infrastructure.h>
```

#### Public Attributes

- `DDS_Long kind`  
*The kind of locator.*
- `DDS_UnsignedLong port`  
*the port number*
- `DDS_Octet address [RTPS_LOCATOR_ADDRESS_LENGTH_MAX]`  
*An octet field (with maximum length of RTPS\_LOCATOR\_ADDRESS\_LENGTH\_MAX to hold the IP address.*

#### 7.40.1 Detailed Description

`<<eXtension>>` Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports.

### 7.40.2 Member Data Documentation

#### kind

`DDS_Long DDS_Locator_t::kind`

The kind of locator.

If the `Locator_t` kind is `DDS_LOCATOR_KIND_UDPv4`, the address contains an IPv4 address. In this case, the leading 12 octets of the `DDS_Locator_t::address` must be zero. The last 4 octets of `DDS_Locator_t::address` are used to store the IPv4 address.

If the `Locator_t` kind is `DDS_LOCATOR_KIND_UDPv6`, the address contains an IPv6 address. IPv6 addresses typically use a shorthand hexadecimal notation that maps one-to-one to the 16 octets in the `DDS_Locator_t::address` field.

#### port

`DDS_UnsignedLong DDS_Locator_t::port`

the port number

#### address

`DDS_Octet DDS_Locator_t::address[RTPS_LOCATOR_ADDRESS_LENGTH_MAX]`

An octet field (with maximum length of `RTPS_LOCATOR_ADDRESS_LENGTH_MAX` to hold the IP address.

## 7.41 DDS\_LocatorSeq Struct Reference

Declares IDL sequence < `DDS_Locator_t` >

```
#include <dds_c_infrastructure.h>
```

### 7.41.1 Detailed Description

Declares IDL sequence < `DDS_Locator_t` >

See also

[DDS\\_Locator\\_t](#)

## 7.42 DDS\_OfferedDeadlineMissedStatus Struct Reference

`DDS_OFFERED_DEADLINE_MISSED_STATUS`

```
#include <dds_c_publication.h>
```



## Public Attributes

- [DDS\\_Long total\\_count](#)  
*Total cumulative count of the number of times the [DDSDDataWriter](#) failed to write within its offered deadline.*
- [DDS\\_Long total\\_count\\_change](#)  
*The incremental changes in total\_count since the last time the listener was called or the status was read.*
- [DDS\\_InstanceHandle\\_t last\\_instance\\_handle](#)  
*Handle to the last instance in the [DDSDDataWriter](#) for which an offered deadline was missed.*

### 7.42.1 Detailed Description

#### [DDS\\_OFFERED\\_DEADLINE\\_MISSED\\_STATUS](#)

Entity:

[DDSDDataWriter](#)

Listener:

[DDSDDataWriterListener](#)

The deadline that the [DDSDDataWriter](#) has committed through its [DDS\\_DeadlineQosPolicy](#) was not respected for a specific instance.

### 7.42.2 Member Data Documentation

#### **total\_count**

[DDS\\_Long](#) [DDS\\_OfferedDeadlineMissedStatus::total\\_count](#)

Total cumulative count of the number of times the [DDSDDataWriter](#) failed to write within its offered deadline.

Missed deadlines accumulate; that is, each deadline period the `total_count` will be incremented by one.

total count

#### **total\_count\_change**

[DDS\\_Long](#) [DDS\\_OfferedDeadlineMissedStatus::total\\_count\\_change](#)

The incremental changes in `total_count` since the last time the listener was called or the status was read.

**last\_instance\_handle**

`DDS_InstanceHandle_t DDS_OfferedDeadlineMissedStatus::last_instance_handle`

Handle to the last instance in the [DDSDDataWriter](#) for which an offered deadline was missed.

**7.43 DDS\_OfferedIncompatibleQoSStatus Struct Reference**[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)

```
#include <dds_c_publication.h>
```

**Public Attributes**

- [DDS\\_Long total\\_count](#)  
*Total cumulative number of times the concerned [DDSDDataWriter](#) discovered a [DDSDDataReader](#) for the same [DDSTopic](#), common partition with a requested QoS that is incompatible with that offered by the [DDSDDataWriter](#).*
- [DDS\\_Long total\\_count\\_change](#)  
*The incremental changes in `total_count` since the last time the listener was called or the status was read.*
- [DDS\\_QosPolicyId\\_t last\\_policy\\_id](#)  
*The [DDS\\_QosPolicyId\\_t](#) of one of the policies that was found to be incompatible the last time an incompatibility was detected.*
- struct [DDS\\_QosPolicyCountSeq policies](#)  
*A list containing for each policy the total number of times that the concerned [DDSDDataWriter](#) discovered a [DDSDDataReader](#) for the same [DDSTopic](#) and common partition with a requested QoS that is incompatible with that offered by the [DDSDDataWriter](#).*

**7.43.1 Detailed Description**[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)

Entity:

[DDSDDataWriter](#)

Listener:

[DDSDDataWriterListener](#)

The qos policy value was incompatible with what was requested.

### 7.43.2 Member Data Documentation

#### **total\_count**

`DDS_Long DDS_OfferedIncompatibleQosStatus::total_count`

Total cumulative number of times the concerned [DDSDataWriter](#) discovered a [DDSDataReader](#) for the same [DDSTopic](#), common partition with a requested QoS that is incompatible with that offered by the [DDSDataWriter](#).

#### **total\_count\_change**

`DDS_Long DDS_OfferedIncompatibleQosStatus::total_count_change`

The incremental changes in `total_count` since the last time the listener was called or the status was read.

#### **last\_policy\_id**

`DDS_QosPolicyId_t DDS_OfferedIncompatibleQosStatus::last_policy_id`

The [DDS\\_QosPolicyId\\_t](#) of one of the policies that was found to be incompatible the last time an incompatibility was detected.

#### **policies**

`struct DDS_QosPolicyCountSeq DDS_OfferedIncompatibleQosStatus::policies`

A list containing for each policy the total number of times that the concerned [DDSDataWriter](#) discovered a [DDSDataReader](#) for the same [DDSTopic](#) and common partition with a requested QoS that is incompatible with that offered by the [DDSDataWriter](#).

## 7.44 DDS\_OwnershipQosPolicy Struct Reference

Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.

```
#include <dds_c_infrastructure.h>
```

#### **Public Attributes**

- [DDS\\_OwnershipQosPolicyKind](#) kind

*The kind of ownership.*

### 7.44.1 Detailed Description

Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#), [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)

Properties:

[RxO](#) = YES  
[Changeable](#) = NO

See also

[OWNERSHIP\\_STRENGTH](#)

### 7.44.2 Usage

This policy controls whether RTI Connex DDS Micro allows multiple [DDSDataWriter](#) objects to update the same instance (identified by [DDSTopic](#) + key) of a data type.

There are two kinds of owner selected by the setting of the `kind`: SHARED and EXCLUSIVE.

#### 7.44.2.1 SHARED ownership

[DDS\\_SHARED\\_OWNERSHIP\\_QOS](#) indicates that RTI Connex DDS Micro does not enforce unique ownership for each instance. In this case, multiple writers can update the same data type instance. The subscriber to the [DDSTopic](#) will be able to access modifications from all [DDSDataWriter](#) objects, subject to the settings of other QoS that may filter particular samples (e.g. the [HISTORY](#) policy). In any case there is no "filtering" of modifications made based on the identity of the [DDSDataWriter](#) that causes the modification.

#### 7.44.2.2 EXCLUSIVE ownership

`DDS_EXCLUSIVE_OWNERSHIP_QOS` indicates that each instance of a data type can only be modified by one `DDSDataWriter`. In other words, at any point in time a single `DDSDataWriter` owns each instance and is the only one whose modifications will be visible to the `DDSDataReader` objects. The owner is determined by selecting the `DDSDataWriter` with the highest value of the `DDS_OwnershipStrengthQosPolicy::value` that is currently alive as defined by the `LIVELINESS` policy and has not violated its `DEADLINE` contract with regards to the data-instance.

Ownership can therefore change as a result of:

- a `DDSDataWriter` in the system with a higher value of the strength that modifies the instance,
- a change in the strength value of the `DDSDataWriter` that owns the instance, and
- a change in the liveliness of the `DDSDataWriter` that owns the instance.
- a deadline with regards to the instance that is missed by the `DDSDataWriter` that owns the instance.

The behavior of the system is as if the determination was made independently by each `DDSDataReader`. Each `DDSDataReader` may detect the change of ownership at a different time. It is not a requirement that at a particular point in time all the `DDSDataReader` objects for that `DDSTopic` have a consistent picture of who owns each instance.

It is also not a requirement that the `DDSDataWriter` objects are aware of whether they own a particular instance. There is no error or notification given to a `DDSDataWriter` that modifies an instance it does not currently own.

The requirements are chosen to (a) preserve the decoupling of publishers and subscribers, and (b) allow the policy to be implemented efficiently.

It is possible that multiple `DDSDataWriter` objects with the same strength modify the same instance. If this occurs RTI Connext DDS Micro will pick one of the `DDSDataWriter` objects as the owner. It is not specified how the owner is selected. However, it is required that the policy used to select the owner is such that all `DDSDataReader` objects will make the same choice of the particular `DDSDataWriter` that is the owner. It is required that the owner remains the same until there is a change in strength, liveliness, the owner misses a deadline on the instance, or a new `DDSDataWriter` with higher same strength, or a new `DDSDataWriter` with same strength that should be deemed the owner according to the policy of the Service, modifies the instance.

Exclusive ownership is on an instance-by-instance basis. That is, a subscriber can receive values written by a lower strength `DDSDataWriter` as long as they affect instances whose values have not been set by the higher-strength `DDSDataWriter`.

#### 7.44.3 Compatibility

The value of the `DDS_OwnershipQosPolicyKind` offered must exactly match the one requested or else they are considered incompatible.

### 7.44.3.1 Ownership Resolution on Redundant Systems

It is expected that users may use DDS to set up redundant systems where multiple [DDSDataWriter](#) entities are "capable" of writing the same instance. In this situation, the [DDSDataWriter](#) entities are configured such that:

- Either both are writing the instance "constantly"
- Or else they use some mechanism to classify each other as "primary" and "secondary", such that the primary is the only one writing, and the secondary monitors the primary and only writes when it detects that the primary "writer" is no longer writing.

Both cases above use the [DDS\\_EXCLUSIVE\\_OWNERSHIP\\_QOS](#) and arbitrate themselves by means of the [DDS\\_OwnershipStrengthQosPolicy](#). Regardless of the scheme, the desired behavior from the [DDSDataReader](#) point of view is that [DDSDataReader](#) normally receives data from the primary unless the "primary" writer stops writing, in which case the [DDSDataReader](#) starts to receive data from the secondary [DDSDataWriter](#).

This approach requires some mechanism to detect that a [DDSDataWriter](#) (the primary) is no longer "writing" the data as it should. There are several reasons why this may happen and all must be detected (but not necessarily distinguished):

- [crash] The writing process is no longer running (e.g. the whole application has crashed)
- [connectivity loss] Connectivity to the writing application has been lost (e.g. network disconnection)
- [application fault] The application logic that was writing the data is faulty and has stopped calling [FooDataWriter::write](#).

Arbitrating from a [DDSDataWriter](#) to one of a higher strength is simple and the decision can be taken autonomously by the [DDSDataReader](#). Switching ownership from a higher strength [DDSDataWriter](#) to one of a lower strength [DDSDataWriter](#) requires that the [DDSDataReader](#) can make a determination that the stronger [DDSDataWriter](#) is "no longer writing the instance".

#### 7.44.3.1.1 Case where the data is periodically updated

This determination is reasonably simple when the data is being written periodically at some rate. The [DDSDataWriter](#) simply states its offered [DDS\\_DeadlineQosPolicy](#) (maximum interval between updates) and the [DDSDataReader](#) automatically monitors that the [DDSDataWriter](#) indeed updates the instance at least once per [DDS\\_DeadlineQosPolicy::period](#). If the deadline is missed, the [DDSDataReader](#) considers the [DDSDataWriter](#) "not alive" and automatically gives ownership to the next highest-strength [DDSDataWriter](#) that *is* alive.

#### 7.44.3.1.2 Case where data is not periodically updated

The case where the `DDSDataWriter` is not writing data periodically is also a very important use-case. Since the instance is not being updated at any fixed period, the "deadline" mechanism cannot be used to determine ownership. The liveliness solves this situation. Ownership is maintained while the `DDSDataWriter` is "alive" and for the `DDSDataWriter` to be alive it must fulfill its `DDS_LivelinessQosPolicy` contract. The different means to renew liveliness (automatic, manual) combined by the implied renewal each time data is written handle the three conditions above [crash], [connectivity loss], and [application fault]. Note that to handle [application fault], `LIVELINESS` must be `DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS`. The `DDSDataWriter` can retain ownership by periodically writing data or else calling `assert_liveliness` if it has no data to write. Alternatively if only protection against [crash] or [connectivity loss] is desired, it is sufficient that some task on the `DDSDataWriter` process periodically writes data. However, this scenario requires that the `DDSDataReader` knows what instances are being "written" by the `DDSDataWriter`. That is the only way that the `DDSDataReader` deduces the ownership of specific instances from the fact that the `DDSDataWriter` is still "alive".

#### 7.44.3.2 Detection of Loss in Topological Connectivity

There are applications that are designed in such a way that their correct operation requires some minimal topological connectivity, that is, the writer needs to have a minimum number of readers or alternatively the reader must have a minimum number of writers.

A common scenario is that the application does not start doing its logic until it knows that some specific writers have the minimum configured readers (e.g. the alarm monitor is up).

A *more* common scenario is that the application logic will wait until some writers appear that can provide some needed source of information (e.g. the raw sensor data that must be processed).

Furthermore, once the application is running it is a requirement that this minimal connectivity (from the source of the data) is monitored and the application informed if it is ever lost. For the case where data is being written periodically, the `DDS_DeadlineQosPolicy` and the `on_deadline_missed` listener provides the notification. The case where data is not periodically updated requires the use of the `DDS_LivelinessQosPolicy` to detect whether the "connectivity" has been lost, and the notification is provided by means of `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE`.

In terms of the required mechanisms, the scenario is very similar to the case of maintaining ownership. In both cases, the reader needs to know whether a writer is still "managing the current value of an instance" even though it is not continually writing it and this knowledge requires the writer to keep its liveliness.

For a `DDSTopic` with `DDS_EXCLUSIVE_OWNERSHIP_QOS`, if the current owner of an instance *disposes* it, the readers accessing the instance will see the `instance_state` as being "DISPOSED" and not see the values being written by the weaker writer (even after the stronger one has disposed the instance). This is because the `DDSDataWriter` that owns the instance is saying that the instance no longer exists (e.g. the master of the database is saying that a record has been deleted) and thus the readers should see it as such.

### 7.44.4 Member Data Documentation

#### kind

```
DDS_OwnershipQosPolicyKind DDS_OwnershipQosPolicy::kind
```

The kind of ownership.

**[default]** `DDS_SHARED_OWNERSHIP_QOS`

## 7.45 DDS\_OwnershipStrengthQosPolicy Struct Reference

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Long](#) value  
*The strength value used to arbitrate among multiple writers.*

### 7.45.1 Detailed Description

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

This policy only applies if the [OWNERSHIP](#) policy is of kind [DDS\\_EXCLUSIVE\\_OWNERSHIP\\_QOS](#).

Entity:

[DDSDataWriter](#)

Properties:

[RxO](#) = N/A  
[Changeable](#) = NO

The value of the [OWNERSHIP\\_STRENGTH](#) is used to determine the ownership of a data-instance (identified by the key). The arbitration is performed by the [DDSDataReader](#).

See also

[EXCLUSIVE ownership](#)

### 7.45.2 Member Data Documentation

**value**

[DDS\\_Long](#) [DDS\\_OwnershipStrengthQosPolicy::value](#)

The strength value used to arbitrate among multiple writers.

**[default]** 0

**[range]** [0, 1 million]



## 7.46 DDS\_ParticipantBuiltinTopicData Struct Reference

Object representing a remote DomainParticipant.

```
#include <dds_c_discovery.h>
```

### Public Attributes

- struct [DDS\\_BuiltinTopicKey\\_t](#) `key`  
*DCPS key to distinguish entries.*
- struct [DDS\\_EntityNameQosPolicy](#) `participant_name`  
*<<eXtension>> The participant name.*
- [DDS\\_UnsignedLong](#) `dds_builtin_endpoints`  
*<<eXtension>> Bitmap of builtin endpoints supported by the participant.*
- [DDS\\_ProtocolVersion\\_t](#) `rtps_protocol_version`  
*<<eXtension>> Version number of the RTPS wire protocol used.*
- struct [DDS\\_VendorId\\_t](#) `rtps_vendor_id`  
*<<eXtension>> ID of vendor implementing the RTPS wire protocol.*
- struct [DDS\\_LocatorSeq](#) `default_unicast_locators`  
*<<eXtension>> Unicast locators used when individual entities do not specify unicast locators.*
- struct [DDS\\_LocatorSeq](#) `default_multicast_locators`  
*<<eXtension>> Multicast locators used when individual entities do not specify multicast locators.*
- struct [DDS\\_LocatorSeq](#) `metatraffic_unicast_locators`  
*<<eXtension>> Unicast locators used for discovery metatraffic.*
- struct [DDS\\_LocatorSeq](#) `metatraffic_multicast_locators`  
*<<eXtension>> Multicast locators used for discovery metatraffic.*
- struct [DDS\\_Duration\\_t](#) `liveliness_lease_duration`  
*<<eXtension>> Liveliness lease duration of participant.*
- struct [DDS\\_ProductVersion\\_t](#) `product_version`  
*<<eXtension>> This is a vendor specific parameter. It gives the current version for RTI Connex DDS Micro.*
- struct [DDS\\_ChecksumProperty](#) `checksum`  
*Checksum properties for the participant.*
- struct [DDS\\_PropertyQosPolicy](#) `property`  
*<<eXtension>> Name value pair properties to be stored with domain participant*
- struct [DDS\\_UserDataQosPolicy](#) `user_data`  
*Policy of the corresponding DomainParticipant.*

### 7.46.1 Detailed Description

Object representing a remote DomainParticipant.

Data associated with the built-in topic `::DDS_PARTICIPANT_TOPIC_NAME`. It contains QoS policies and additional information that apply to the remote [DDSDomainParticipant](#).

### 7.46.2 Member Data Documentation

#### key

```
struct DDS_BuiltinTopicKey_t DDS_ParticipantBuiltinTopicData::key
```

DCPS key to distinguish entries.

#### participant\_name

```
struct DDS_EntityNameQosPolicy DDS_ParticipantBuiltinTopicData::participant_name
```

<<eXtension>> The participant name.

This is the name of the discovered participant.

#### dds\_builtin\_endpoints

```
DDS_UnsignedLong DDS_ParticipantBuiltinTopicData::dds_builtin_endpoints
```

<<eXtension>> Bitmap of builtin endpoints supported by the participant.

Each bit indicates a builtin endpoint that may be available on the participant for use in discovery.

#### rtps\_protocol\_version

```
DDS_ProtocolVersion_t DDS_ParticipantBuiltinTopicData::rtps_protocol_version
```

<<eXtension>> Version number of the RTPS wire protocol used.

#### rtps\_vendor\_id

```
struct DDS_VendorId_t DDS_ParticipantBuiltinTopicData::rtps_vendor_id
```

<<eXtension>> ID of vendor implementing the RTPS wire protocol.

#### default\_unicast\_locators

```
struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::default_unicast_locators
```

<<eXtension>> Unicast locators used when individual entities do not specify unicast locators.

**default\_multicast\_locators**

```
struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::default_multicast_locators
```

<<eXtension>> Multicast locators used when individual entities do not specify multicast locators.

**metatraffic\_unicast\_locators**

```
struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::metatraffic_unicast_locators
```

<<eXtension>> Unicast locators used for discovery metatraffic.

**metatraffic\_multicast\_locators**

```
struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::metatraffic_multicast_locators
```

<<eXtension>> Multicast locators used for discovery metatraffic.

**liveliness\_lease\_duration**

```
struct DDS_Duration_t DDS_ParticipantBuiltinTopicData::liveliness_lease_duration
```

<<eXtension>> Liveliness lease duration of participant.

**product\_version**

```
struct DDS_ProductVersion_t DDS_ParticipantBuiltinTopicData::product_version
```

<<eXtension>> This is a vendor specific parameter. It gives the current version for RTI Connex DDS Micro.

**checksum**

```
struct DDS_ChecksumProperty DDS_ParticipantBuiltinTopicData::checksum
```

Checksum properties for the participant.

**property**

```
struct DDS_PropertyQosPolicy DDS_ParticipantBuiltinTopicData::property
```

<<eXtension>> Name value pair properties to be stored with domain participant

**user\_data**

```
struct DDS_UserDataQosPolicy DDS_ParticipantBuiltinTopicData::user_data
```

Policy of the corresponding DomainParticipant.

## 7.47 DDS\_PartitionQosPolicy Struct Reference

A set of strings that introduces logical PARTITIONS.

```
#include <dds_c_partition_qos.h>
```

### Public Attributes

- struct DDS\_StringSeq [name](#)  
*The PARTITION string Sequence.*

#### 7.47.1 Detailed Description

A set of strings that introduces logical PARTITIONS.

Only entities that belong to the same PARTITION can communicate with each other.

The PARTITION QoS controls which entities will match which other entities. PARTITIONS prevent entities from communicating. Only entities that belong to the same PARTITION can communicate with each other. The PARTITION QoS policy applies directly to Publishers and Subscribers. DataWriters and DataReaders belong to the partitions of the Publishers and Subscribers that created them.

PARTITION names are propagated with the discovery traffic for Publishers and Subscribers.

DataWriters will make their data available on all PARTITIONS that correspond to their Publisher's PARTITION names, and the DataReaders will see the data on all PARTITIONS that correspond to their Subscriber's PARTITION names. For a DataWriter and DataReader to communicate, the DataWriter's Publisher must match the DataReader's Subscribers PARTITION.

DataWriters and DataReaders will only match with each other if their Publisher's QoS and Subscriber's QoS contain a compatible PARTITION.

The PARTITION sequence must be set before enabling the entity. The user must create a Publisher QoS [DDS\\_PublisherQos::partition](#) or a Subscriber QoS [DDS\\_SubscriberQos::partition](#) for PARTITION configuration.

The default value for a PARTITION is the empty sequence. When an entity is created with a [DDS\\_PublisherQos::partition](#) that does not have any concrete strings, meaning it only has regular expressions, then the partition is considered a member of the empty string "".

Publishers and Subscribers without PARTITIONS are considered members of the empty string "".

The regular expression matching rules are as follows: - Two PARTITIONS will match if they are lexicographically identical. - Two PARTITIONS will match if one is a string is not a regular expression and the other is a regular expression that completely matches. - Two regular expressions cannot match.

[DDS\\_PartitionQosPolicy](#) supports the following regular expressions:

\* The '\*' matches any sequence of characters, including the empty string.

? The '?' character matches the preceding character once. Note that the '?' does not conform with the POSIX standard where it is allowed to match zero times.

\ The '\' escape character **shall** cause the next character to be interpreted literally.

[ ] Opening and closing brackets '[' ']' matches one of the characters enclosed in the brackets: A range "[ C0 '-' C1 ]" matches one character between the collating elements C0 and C1, inclusive.

The "^" and "!" characters placed first in opening and closing brackets matches a single character if the character is not enclosed in the brackets.

A range "[ '!' C0 '-' C1 ]" matches a character not in the lexicographically closed interval C0 to C1.

The [DDS\\_PartitionQosPolicy](#) resource limits are made on a per DomainParticipant basis. The resource limits cannot be set individually on a per Publisher/Subscriber basis.

[DDS\\_DomainParticipantResourceLimitsQosPolicy::max\\_partitions](#) controls the maximum number of PARTITIONS that can be assigned per Publisher/Subscriber.

[DDS\\_DomainParticipantResourceLimitsQosPolicy::max\\_partition\\_cumulative\\_characters](#) controls the maximum number of combined characters in all PARTITION names in a [DDS\\_PartitionQosPolicy](#).

The maximum number of combined characters should account for a terminating NUL ('\0') character for each partition name string.

[DDS\\_DomainParticipantResourceLimitsQosPolicy::max\\_partition\\_string\\_allocation](#) The maximum number of 8-bit characters reserved for PARTITION.name strings to allocate for the PARTITION. When this is set to DDS\_LENGTH\_UNLIMITED, no memory is preallocated. Memory is allocated during local and remote entity creation even if the creation occurs during runtime. When max\_partition\_string\_allocation is greater than 0, that amount of memory is preallocated for storing created and discovered PARTITION names. [DDS\\_PartitionQosPolicy](#).

[DDS\\_DomainParticipantResourceLimitsQosPolicy::max\\_partition\\_string\\_size](#) max\_partition\_string\_size configures the PARTITION name element max string length in local and remote Publishers and Subscribers. All applications in the DDS domain must use the same max\_partition\_string\_size in order to communicate.

When creating a Publisher or Subscriber, all PARTITION elements in the Publisher or Subscriber QoS must not exceed max\_partition\_string\_size or the creation will fail. max\_partition\_string\_size also controls the max length for any discovered PARTITION names from remote Publishers and Subscribers. Remote entities that have PARTITION name lengths that exceed max\_partition\_string\_size will not be discovered.

When max\_partition\_string\_allocation is DDS\_LENGTH\_UNLIMITED max\_partition\_string\_size should be set to DDS\_LENGTH\_UNLIMITED. This configuration allows each PARTITION name size to be any length up to max\_partition\_cumulative\_characters. In this configuration, PARTITION strings are allocated and freed during runtime. To prevent allocation during runtime, when max\_partition\_string\_allocation is DDS\_LENGTH\_UNLIMITED, the users can assert all unique PARTITION names during creation. When any additional instances of that name is encountered, during entity creation or discovery, no further memory is used for storing the existing PARTITION names.

Use the following configurations to completely control memory utilization or to avoid any possibility of runtime allocation.

When `max_partition_string_allocation` is not `DDS_LENGTH_UNLIMITED` and `max_partition_string_size` is `DDS_LENGTH_UNLIMITED` then `PARTITION` names can be any length up to `max_partition_cumulative_characters`. Memory for `PARTITION` names will be preallocated and will never be freed or reused. The user must have enough `max_partition_string_allocation` to store every unique instance of each `PARTITION` name to be discovered.

When `max_partition_string_allocation` is not `DDS_LENGTH_UNLIMITED` then `max_partition_string_size` can be set to a value greater than 0. In this configuration, memory for `PARTITION` names are preallocated and will never be freed during operation. `PARTITION` names are stored in memory and consume `max_partition_string_size` sized memory segments up to `max_partition_string_allocation`. Memory is reused for `PARTITION` names that are added to memory, internally deleted, and are no longer needed. As an example, if the application creates a Publisher with a unique `PARTITION` name instance and then deletes that Publisher, if there are no further uses of that partition name, the application will reuse the memory that was storing the unique `PARTITION` name. Reuse occurs in the same way for remote entities. The user must have enough `max_partition_string_allocation` to store every unique instance of each `PARTITION` name to be created and discovered.

### 7.47.2 Member Data Documentation

#### name

```
struct DDS_StringSeq DDS_PartitionQosPolicy::name
```

The `PARTITION` string Sequence.

Sequence of strings that introduce a logical `PARTITION` among the topics visible by the Publisher and Subscriber.

## 7.48 DDS\_PresentationQosPolicy Struct Reference

Specifies how the samples representing changes to data instances are presented to a subscribing application.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_PresentationQosPolicyAccessScopeKind access\\_scope](#)  
*Determines the largest scope spanning the entities for which the order and coherency of changes can be preserved.*
- [DDS\\_Boolean coherent\\_access](#)  
*Specifies support for coherent access. Controls whether coherent access is supported within the scope `access_scope`.*
- [DDS\\_Boolean ordered\\_access](#)  
*Specifies support for ordered access to the samples received at the subscription end. Controls whether ordered access is supported within the scope `access_scope`.*

### 7.48.1 Detailed Description

Specifies how the samples representing changes to data instances are presented to a subscribing application.

**NOTE:** This QoS policy can only be set when using the static discovery API on the [DDS\\_SubscriptionBuiltinTopicData](#).

However, a [DDSDataWriter](#) always announces [DDS\\_TOPIC\\_PRESENTATION\\_QOS](#) via discovery, enabling a [DDSDataReader](#) to match with a [DDSDataReader](#) that requests either [DDS\\_INSTANCE\\_PRESENTATION\\_QOS](#) or [DDS\\_TOPIC\\_PRESENTATION\\_QOS](#), increasing interoperability.

A DataReader supports only [DDS\\_INSTANCE\\_PRESENTATION\\_QOS](#).

This QoS policy controls the extent to which changes to data instances can be made dependent on each other and also the kind of dependencies that can be propagated and maintained by RTI Connex DDS Micro. Specifically, this policy affects the application's ability to:

- specify and receive coherent changes to instances
- specify the relative order in which changes are presented

Entity:

[DDSPublisher](#), [DDSSubscriber](#)

Status:

[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#), [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)

A [DDSDataReader](#) will usually receive data in the order that it was sent by a [DDSDataWriter](#), and the data is presented to the [DDSDataReader](#) as soon as the application receives the next expected value. However, sometimes you may want a set of data for the same [DDSDataWriter](#) to be presented to the [DDSDataReader](#) only after *all* of the elements of the set have been received. Or you may want the data to be presented in a different order than that in which it was received. Specifically for keyed data, you may want the middleware to present the data in keyed – or *instance* – order, such that samples pertaining to the same instance are presented together.

The Presentation QoS policy allows you to specify different *scopes* of presentation: within a [DDSDataWriter](#), across instances of a single [DDSDataWriter](#), and even across multiple DataWriters used by different writers of a publisher. It also controls whether or not a set of changes within the scope is delivered at the same time or can be delivered as soon as each element is received.

- `coherent_access` controls whether RTI Connex DDS Micro will preserve the groupings of changes made by a publishing application by means of the operations `begin_coherent_changes()` and `end_coherent_changes()`. These operations are not supported.
- `ordered_access` controls whether RTI Connex DDS Micro will preserve the order of changes.
- `access_scope` controls the granularity of the other settings. See below:

If `coherent_access` is set, then the `access_scope` controls the maximum extent of coherent changes. The behavior is as follows:

- If `access_scope` is set to `DDS_INSTANCE_PRESENTATION_QOS` (the default), the use of `begin_coherent_changes()` and `end_coherent_changes()` has no effect on how the subscriber can access the data, because with the scope limited to each instance, changes to separate instances are considered independent and thus cannot be grouped into a coherent set.
- If `access_scope` is set to `DDS_TOPIC_PRESENTATION_QOS`, then coherent changes (indicated by their enclosure within calls to `begin_coherent_changes()` and `end_coherent_changes()`) will be made available as such to each remote `DDSDataReader` independently. That is, changes made to instances within each individual `DDSDataWriter` will be available as coherent with respect to other changes to instances in that same `DDSDataWriter`, but will not be grouped with changes made to instances belonging to a different `DDSDataWriter`.
- If `access_scope` is set to `DDS_GROUP_PRESENTATION_QOS`, then coherent changes made to instances through a `DDSDataWriter` attached to a common `DDSPublisher` are made available as a unit to remote subscribers.

If `ordered_access` is set, then the `access_scope` controls the maximum extent for which order will be preserved by RTI Connex DDS Micro.

- If `access_scope` is set to `DDS_INSTANCE_PRESENTATION_QOS` (the lowest level), then changes to each instance are considered unordered relative to changes to any other instance. That means that changes (creations, deletions, modifications) made to two instances are not necessarily seen in the order they occur. This is the case even if it is the same application thread making the changes using the same `DDSDataWriter`.
- If `access_scope` is set to `DDS_TOPIC_PRESENTATION_QOS`, changes (creations, deletions, modifications) made by a single `DDSDataWriter` are made available to subscribers in the same order they occur. Changes made to instances through different `DDSDataWriter` entities are not necessarily seen in the order they occur. This is the case, even if the changes are made by a single application thread using `DDSDataWriter` objects attached to the same `DDSPublisher`.
- Finally, if `access_scope` is set to `DDS_GROUP_PRESENTATION_QOS`, changes made to instances via `DDSDataWriter` entities attached to the same `DDSPublisher` object are made available to subscribers in the same order they occur.

Note that this QoS policy controls the scope at which related changes are made available to the subscriber. This means the subscriber **can** access the changes in a coherent manner and in the proper order; however, it does not necessarily imply that the `DDSSubscriber` **will** indeed access the changes in the correct order. For that to occur, the application at the subscriber end must use the proper logic in reading the `DDSDataReader` objects.



### 7.48.2 Compatibility

The value offered is considered compatible with the value requested if and only if the following conditions are met:

- the inequality *offered access\_scope*  $\geq$  *requested access\_scope* evaluates to 'TRUE'. For the purposes of this inequality, the values of *access\_scope* are considered ordered such that [DDS\\_INSTANCE\\_PRESENTATION\\_QOS](#) < [DDS\\_TOPIC\\_PRESENTATION\\_QOS](#) < [DDS\\_GROUP\\_PRESENTATION\\_QOS](#).
- *requested coherent\_access* is [DDS\\_BOOLEAN\\_FALSE](#), or else both offered and requested *coherent\_access* are [DDS\\_BOOLEAN\\_TRUE](#).
- *requested ordered\_access* is [DDS\\_BOOLEAN\\_FALSE](#), or else both offered and requested *ordered\_access* are [DDS\\_BOOLEAN\\_TRUE](#).

### 7.48.3 Member Data Documentation

#### **access\_scope**

[DDS\\_PresentationQosPolicyAccessScopeKind](#) [DDS\\_PresentationQosPolicy::access\\_scope](#)

Determines the largest scope spanning the entities for which the order and coherency of changes can be preserved.

**NOTE:** This QoS setting can only be set when using the static discovery API on the [DDS\\_SubscriptionBuiltinTopicData](#).

#### **coherent\_access**

[DDS\\_Boolean](#) [DDS\\_PresentationQosPolicy::coherent\\_access](#)

Specifies support for *coherent* access. Controls whether coherent access is supported within the scope *access\_scope*.

That is, the ability to group a set of changes as a unit on the publishing end such that they are received as a unit at the subscribing end.

To use this feature, the DataWriter must be configured for RELIABLE communication (see [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#)).

**NOTE:** This QoS setting can only be set when using the static discovery API on the [DDS\\_SubscriptionBuiltinTopicData](#).

[default] [DDS\\_BOOLEAN\\_FALSE](#)

**ordered\_access**

`DDS_Boolean DDS_PresentationQosPolicy::ordered_access`

Specifies support for *ordered* access to the samples received at the subscription end. Controls whether ordered access is supported within the scope `access_scope`.

That is, the ability of the subscriber to see changes in the same order as they occurred on the publishing end.

**NOTE:** This QoS setting can only be set when using the static discovery API on the [DDS\\_SubscriptionBuiltinTopicData](#).

[default] [DDS\\_BOOLEAN\\_TRUE](#) for DataWriters, [DDS\\_BOOLEAN\\_FALSE](#) for DataReaders

**7.49 DDS\_ProductVersion Struct Reference**

<<*eXtension*>> Type used to represent the current version of RTI Connex DDS Micro.

```
#include <dds_c_infrastructure.h>
```

**Public Attributes**

- [DDS\\_Char major](#)  
*Major product version.*
- [DDS\\_Char minor](#)  
*Minor product version.*
- [DDS\\_Char release](#)  
*Release letter for product version.*
- [DDS\\_Char revision](#)  
*Revision number of product.*

**7.49.1 Detailed Description**

<<*eXtension*>> Type used to represent the current version of RTI Connex DDS Micro.

**7.49.2 Member Data Documentation****major**

`DDS_Char DDS_ProductVersion::major`

Major product version.

**minor**

`DDS_Char DDS_ProductVersion::minor`

Minor product version.

**release**

`DDS_Char DDS_ProductVersion::release`

Release letter for product version.

**revision**

`DDS_Char DDS_ProductVersion::revision`

Revision number of product.

## 7.50 DDS\_ProductVersion\_t Struct Reference

<<*eXtension*>> Type used to represent the current version of RTI Connex DDS Micro.

```
#include <dds_c_infrastructure.h>
```

**Public Attributes**

- `DDS_Char major`  
*Major product version.*
- `DDS_Char minor`  
*Minor product version.*
- `DDS_Char release`  
*Release letter for product version.*
- `DDS_Char revision`  
*Revision number of product.*

### 7.50.1 Detailed Description

<<*eXtension*>> Type used to represent the current version of RTI Connex DDS Micro.

### 7.50.2 Member Data Documentation

#### major

`DDS_Char DDS_ProductVersion_t::major`

Major product version.

#### minor

`DDS_Char DDS_ProductVersion_t::minor`

Minor product version.

#### release

`DDS_Char DDS_ProductVersion_t::release`

Release letter for product version.

#### revision

`DDS_Char DDS_ProductVersion_t::revision`

Revision number of product.

## 7.51 DDS\_Property\_t Struct Reference

Properties are name/value pairs objects.

```
#include <dds_c_property_qos.h>
```

#### Public Attributes

- [DDS\\_String name](#)  
*Property name.*
- [DDS\\_String value](#)  
*Property value.*
- [DDS\\_Boolean propagate](#)  
*Indicates if the property must be propagated on discovery. Must be DDS\_BOOLEAN\_FALSE.*

### 7.51.1 Detailed Description

Properties are name/value pairs objects.

### 7.51.2 Member Data Documentation

#### name

`DDS_String DDS_Property_t::name`

Property name.

It must be a NULL-terminated string allocated with `DDS_String_alloc` or `DDS_String_dup`.

#### value

`DDS_String DDS_Property_t::value`

Property value.

It must be a NULL-terminated string allocated with `DDS_String_alloc` or `DDS_String_dup`.

#### propagate

`DDS_Boolean DDS_Property_t::propagate`

Indicates if the property must be propagated on discovery. Must be `DDS_BOOLEAN_FALSE`.

## 7.52 DDS\_PropertyQosPolicy Struct Reference

Stores name/value(string) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.

```
#include <dds_c_property_qos.h>
```

### Public Attributes

- struct `DDS_PropertySeq` value  
*Sequence of properties.*

### 7.52.1 Detailed Description

Stores name/value(string) pairs that can be used to configure certain parameters of RTI Connex DDS Micro that are not exposed through formal QoS policies.

Entity:

[DDSDomainParticipant](#) [DDSDataReader](#) [DDSDataWriter](#)

Properties:

[RxO](#) = N/A;  
[Changeable](#) = NO

The PROPERTY QoS policy can be used to associate a set of properties in the form of (name, value) pairs with a [DDSDataReader](#), [DDSDataWriter](#), or [DDSDomainParticipant](#).

You can find a complete list of predefined properties in the [Property Reference](#).

There are helper functions to facilitate working with properties, see the [PROPERTY](#) page.

### 7.52.2 Member Data Documentation

**value**

```
struct DDS\_PropertySeq DDS_PropertyQosPolicy::value
```

Sequence of properties.

**[default]** An empty list.

## 7.53 DDS\_PropertySeq Struct Reference

Declares IDL sequence < [DDS\\_Property\\_t](#) >

```
#include <dds_c_property_qos.h>
```

### 7.53.1 Detailed Description

Declares IDL sequence < [DDS\\_Property\\_t](#) >

See also

[DDS\\_Property\\_t](#)

## 7.54 DDS\_ProtocolVersion Struct Reference

<<*eXtension*>> Type used to represent the version of the RTPS protocol.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Octet major](#)  
*Major protocol version number.*
- [DDS\\_Octet minor](#)  
*Minor protocol version number.*

### 7.54.1 Detailed Description

<<*eXtension*>> Type used to represent the version of the RTPS protocol.

### 7.54.2 Member Data Documentation

#### major

[DDS\\_Octet](#) DDS\_ProtocolVersion::major

Major protocol version number.

#### minor

[DDS\\_Octet](#) DDS\_ProtocolVersion::minor

Minor protocol version number.

## 7.55 DDS\_PskServiceFactoryProperty Struct Reference

Property used to register the Lightweight Security Plugin.

```
#include <dds_psk_service.h>
```

### Public Attributes

- const char \* [service\\_name](#)  
*The name the plugin is registered with.*
- DDS\_PskTransform\_get\_interfaceFunc\_T \* [psl\\_get\\_interface\\_func](#)  
*A function pointer to retrieve the Transform PSL implementation interface.*
- struct [DDS\\_SecTransform\\_Configuration](#) \* [psl\\_config](#)  
*Pass-through opaque pointer to the Transform PSL implementation, to provide the implementation with custom configuration settings.*

### 7.55.1 Detailed Description

Property used to register the Lightweight Security Plugin.

### 7.55.2 Member Data Documentation

#### service\_name

```
const char* DDS_PskServiceFactoryProperty::service_name
```

The name the plugin is registered with.

If NULL is provided, the plugin is registered with [DDS\\_PSK\\_DEFAULT\\_PLUGIN\\_NAME](#). **[default]** NULL

#### psl\_get\_interface\_func

```
DDS_PskTransform_get_interfaceFunc_T* DDS_PskServiceFactoryProperty::psl_get_interface_func
```

A function pointer to retrieve the Transform PSL implementation interface.

The Lightweight Security Plugin issues this to retrieve the Transform PSL implementation. When using the RTI-provided OpenSSL-based PSL implementation, this should be set to 'PSK\_OSSL\_get\_interface'.

#### psl\_config

```
struct DDS_SecTransform_Configuration* DDS_PskServiceFactoryProperty::psl_config
```

Pass-through opaque pointer to the Transform PSL implementation, to provide the implementation with custom configuration settings.

When using the RTI-provided OpenSSL-based implementation, this resolves to [DDS\\_SecTransform\\_Configuration](#). **[default]** NULL

## 7.56 DDS\_PublicationBuiltinTopicData Struct Reference

Object describing a remote Publication.

```
#include <dds_c_discovery.h>
```



## Public Attributes

- struct [DDS\\_BuiltinTopicKey\\_t](#) key  
*DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDDataWriter](#).*
- struct [DDS\\_BuiltinTopicKey\\_t](#) participant\_key  
*DCPS key of the participant to which the DataWriter belongs.*
- char \* [topic\\_name](#)  
*Name of the related [DDSTopic](#).*
- char \* [type\\_name](#)  
*Name of the type attached to the [DDSTopic](#).*
- struct [DDS\\_DeadlineQosPolicy](#) deadline  
*Policy of the corresponding DataWriter.*
- struct [DDS\\_OwnershipQosPolicy](#) ownership  
*Policy of the corresponding DataWriter.*
- struct [DDS\\_OwnershipStrengthQosPolicy](#) ownership\_strength  
*Policy of the corresponding DataWriter.*
- struct [DDS\\_LatencyBudgetQosPolicy](#) latency\_budget  
*Latency budget policy, [LATENCY\\_BUDGET](#).*
- struct [DDS\\_ReliabilityQosPolicy](#) reliability  
*Policy of the corresponding DataWriter.*
- struct [DDS\\_LivelinessQosPolicy](#) liveliness  
*Policy of the corresponding DataWriter.*
- struct [DDS\\_DurabilityQosPolicy](#) durability  
*durability policy of the corresponding DataWriter*
- struct [DDS\\_DestinationOrderQosPolicy](#) destination\_order  
*Policy of the corresponding DataWriter.*
- struct [DDS\\_LocatorSeq](#) unicast\_locator  
*<<eXtension>> Custom unicast locators that the endpoint can specify. The default locators will be used if this is not specified.*
- struct [DDS\\_DataRepresentationQosPolicy](#) representation  
*Data representation policy, [DATA\\_REPRESENTATION](#).*
- struct [DDS\\_PartitionQosPolicy](#) partition  
*Policy of the Publisher to which the DataWriter belongs.*
- struct [DDS\\_UserDataQosPolicy](#) user\_data  
*Policy of the corresponding DataWriter.*
- struct [DDS\\_GroupDataQosPolicy](#) group\_data  
*Policy of the Publisher to which the DataWriter belongs.*
- struct [DDS\\_TopicDataQosPolicy](#) topic\_data  
*Policy of the related Topic.*

### 7.56.1 Detailed Description

Object describing a remote Publication.

This object contains all of the information about a remote [DDSDDataWriter](#) and its [DDSPublisher](#) that is necessary for matching the remote [DDSDDataWriter](#) to a local [DDSDDataReader](#) during discovery.

This object is used to pre-configure remote entities for Dynamic Participant/Static Endpoint discovery.

For example: Application A is designed to discover a remote [DDSDataWriter](#) in application B, and that remote [DDSDataWriter](#) has an object ID of 100, and a deadline of three seconds. In Dynamic Participant/Static Endpoint discovery, this discovery data is not sent over the network, so the user must create an instance of a [DDS\\_PublicationBuiltinTopicData](#) structure in application A. The user then must fill in the QoS describing this remote [DDSDataWriter](#), including the writer's object ID and three-second deadline. Then the user must assert this [DDS\\_PublicationBuiltinTopicData](#) describing the remote entity with the discovery plugin. The pre-configured QoS must match the actual QoS of the remote entity, or communication will not occur.

### 7.56.2 Member Data Documentation

#### key

```
struct DDS_BuiltinTopicKey_t DDS_PublicationBuiltinTopicData::key
```

DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDataWriter](#).

#### participant\_key

```
struct DDS_BuiltinTopicKey_t DDS_PublicationBuiltinTopicData::participant_key
```

DCPS key of the participant to which the DataWriter belongs.

#### topic\_name

```
char* DDS_PublicationBuiltinTopicData::topic_name
```

Name of the related [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [Conventions](#).

See also

[Conventions](#)

#### type\_name

```
char* DDS_PublicationBuiltinTopicData::type_name
```

Name of the type attached to the [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [Conventions](#).

See also

[Conventions](#)

**deadline**

```
struct DDS_DeadlineQosPolicy DDS_PublicationBuiltinTopicData::deadline
```

Policy of the corresponding DataWriter.

**ownership**

```
struct DDS_OwnershipQosPolicy DDS_PublicationBuiltinTopicData::ownership
```

Policy of the corresponding DataWriter.

**ownership\_strength**

```
struct DDS_OwnershipStrengthQosPolicy DDS_PublicationBuiltinTopicData::ownership_strength
```

Policy of the corresponding DataWriter.

**latency\_budget**

```
struct DDS_LatencyBudgetQosPolicy DDS_PublicationBuiltinTopicData::latency_budget
```

Latency budget policy, [LATENCY\\_BUDGET](#).

**reliability**

```
struct DDS_ReliabilityQosPolicy DDS_PublicationBuiltinTopicData::reliability
```

Policy of the corresponding DataWriter.

**liveliness**

```
struct DDS_LivelinessQosPolicy DDS_PublicationBuiltinTopicData::liveliness
```

Policy of the corresponding DataWriter.

**durability**

```
struct DDS_DurabilityQosPolicy DDS_PublicationBuiltinTopicData::durability
```

durability policy of the corresponding DataWriter

**destination\_order**

```
struct DDS_DestinationOrderQosPolicy DDS_PublicationBuiltinTopicData::destination_order
```

Policy of the corresponding DataWriter.

**unicast\_locator**

```
struct DDS_LocatorSeq DDS_PublicationBuiltinTopicData::unicast_locator
```

<<eXtension>> Custom unicast locators that the endpoint can specify. The default locators will be used if this is not specified.

**representation**

```
struct DDS_DataRepresentationQosPolicy DDS_PublicationBuiltinTopicData::representation
```

Data representation policy, [DATA\\_REPRESENTATION](#).

**partition**

```
struct DDS_PartitionQosPolicy DDS_PublicationBuiltinTopicData::partition
```

Policy of the Publisher to which the DataWriter belongs.

**user\_data**

```
struct DDS_UserDataQosPolicy DDS_PublicationBuiltinTopicData::user_data
```

Policy of the corresponding DataWriter.

**group\_data**

```
struct DDS_GroupDataQosPolicy DDS_PublicationBuiltinTopicData::group_data
```

Policy of the Publisher to which the DataWriter belongs.

**topic\_data**

```
struct DDS_TopicDataQosPolicy DDS_PublicationBuiltinTopicData::topic_data
```

Policy of the related Topic.

## 7.57 DDS\_PublicationMatchedException Struct Reference

### DDS\_PUBLICATION\_MATCHED\_STATUS

```
#include <dds_c_publication.h>
```

#### Public Attributes

- [DDS\\_Long total\\_count](#)  
The total cumulative number of times the concerned [DDSDataWriter](#) discovered a "match" with a [DDSDataReader](#).
- [DDS\\_Long total\\_count\\_change](#)  
The incremental changes in `total_count` since the last time the listener was called or the status was read.
- [DDS\\_Long current\\_count](#)  
The current number of readers with which the [DDSDataWriter](#) is matched.
- [DDS\\_Long current\\_count\\_change](#)  
The change in `current_count` since the last time the listener was called or the status was read.
- [DDS\\_InstanceHandle\\_t last\\_subscription\\_handle](#)  
A handle to the last [DDSDataReader](#) that caused the the [DDSDataWriter](#)'s status to change.

#### 7.57.1 Detailed Description

### DDS\_PUBLICATION\_MATCHED\_STATUS

A "match" happens when the [DDSDataWriter](#) finds a [DDSDataReader](#) for the same [DDSTopic](#) and common partition with a requested QoS that is compatible with that offered by the [DDSDataWriter](#).

This status is also changed (and the listener, if any, called) when a match is ended. A local [DDSDataWriter](#) will become "unmatched" from a remote [DDSDataReader](#) when that [DDSDataReader](#) goes away for any reason.

#### 7.57.2 Member Data Documentation

##### **total\_count**

```
DDS_Long DDS_PublicationMatchedException::total_count
```

The total cumulative number of times the concerned [DDSDataWriter](#) discovered a "match" with a [DDSDataReader](#).

This number increases whenever a new match is discovered. It does not change when an existing match goes away.

##### **total\_count\_change**

```
DDS_Long DDS_PublicationMatchedException::total_count_change
```

The incremental changes in `total_count` since the last time the listener was called or the status was read.

**current\_count**

`DDS_Long DDS_PublicationMatchedStatus::current_count`

The current number of readers with which the [DDSDataWriter](#) is matched.

This number increases when a new match is discovered and decreases when an existing match goes away.

**current\_count\_change**

`DDS_Long DDS_PublicationMatchedStatus::current_count_change`

The change in `current_count` since the last time the listener was called or the status was read.

**last\_subscription\_handle**

`DDS_InstanceHandle_t DDS_PublicationMatchedStatus::last_subscription_handle`

A handle to the last [DDSDataReader](#) that caused the the [DDSDataWriter](#)'s status to change.

**7.58 DDS\_PublisherQos Struct Reference**

QoS policies supported by a [DDSPublisher](#) entity.

```
#include <dds_c_publication.h>
```

**Public Attributes**

- struct [DDS\\_EntityFactoryQosPolicy](#) `entity_factory`  
Entity factory policy, [ENTITY\\_FACTORY](#).
- struct [DDS\\_PartitionQosPolicy](#) `partition`  
The [DDSPublisher](#) [PARTITION](#) policy.
- struct [DDS\\_GroupDataQosPolicy](#) `group_data`  
Group data policy, [GROUP\\_DATA](#).
- struct [DDS\\_EntityNameQosPolicy](#) `publisher_name`  
<<eXtension>> The [DDSPublisher](#) name. [ENTITY\\_NAME](#).

**7.58.1 Detailed Description**

QoS policies supported by a [DDSPublisher](#) entity.

If the call to [DDSPublisher::set\\_qos](#) attempts to change the [PARTITION](#) QoS then it will return [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#).

## 7.58.2 Member Data Documentation

### entity\_factory

```
struct DDS_EntityFactoryQosPolicy DDS_PublisherQos::entity_factory
```

Entity factory policy, [ENTITY\\_FACTORY](#).

### partition

```
struct DDS_PartitionQosPolicy DDS_PublisherQos::partition
```

The [DDSPublisher](#) PARTITION policy.

A set of strings that introduces logical PARTITIONS.

Only entities that belong to the same PARTITION can communicate to each other. The PARTITION QoS policy applies directly to [DDSPublisher.DDSDataWriter](#) belong to the PARTITIONS of the [DDSPublisher](#) that created them.

The default PARTITION during initialization is the empty string, "".

### PARTITION

### group\_data

```
struct DDS_GroupDataQosPolicy DDS_PublisherQos::group_data
```

Group data policy, [GROUP\\_DATA](#).

### publisher\_name

```
struct DDS_EntityNameQosPolicy DDS_PublisherQos::publisher_name
```

[<<eXtension>>](#) The [DDSPublisher](#) name. [ENTITY\\_NAME](#).

## 7.59 DDS\_PublishModeQosPolicy Struct Reference

Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its *own* thread to send data, instead of the user thread.

```
#include <dds_c_infrastructure.h>
```

## Public Attributes

- [DDS\\_PublishModeQosPolicyKind](#) kind  
*Publishing mode.*
- const char \* [flow\\_controller\\_name](#)  
*Name of the associated flow controller.*
- [DDS\\_Long](#) priority  
*Publication priority.*

### 7.59.1 Detailed Description

Specifies how RTI Connex DDS Micro sends application data on the network. This QoS policy can be used to tell RTI Connex DDS Micro to use its *own* thread to send data, instead of the user thread.

The publishing mode of a [DDSDataWriter](#) determines whether data is written synchronously in the context of the user thread when calling [FooDataWriter::write](#) or asynchronously in the context of a separate thread internal to the middleware.

See also

[DDS\\_HistoryQosPolicy](#)  
[DDSFlowController](#)

Entity:

[DDSDataWriter](#)

Properties:

[RxO](#) = N/A  
[Changeable](#) = NO

### 7.59.2 Usage

The fastest way for RTI Connex DDS Micro to send data is for the user thread to execute the middleware code that actually sends the data itself. However, there are times when user applications may need or want an internal middleware thread to send the data instead. For instance, to send large data reliably, you must use an asynchronous thread.

When data is written asynchronously, a [DDSFlowController](#), identified by `flow_controller_name`, can be used to shape the network traffic. Shaping a data flow usually means limiting the maximum data rates at which the middleware will send data for a [DDSDataWriter](#). The flow controller will buffer any excess data and only send it when the send rate drops below the maximum rate. The flow controller's properties determine when the asynchronous publishing thread is allowed to send data and how much.

Asynchronous publishing may increase latency, but offers the following advantages:

- The [FooDataWriter::write](#) call does not make any network calls and is therefore faster and more deterministic. This becomes important when the user thread is executing time-critical code.
- When data is written in bursts or when sending large data types as multiple fragments, a flow controller can throttle the send rate of the asynchronous publishing thread to avoid flooding the network.

The middleware must queue samples until they can be sent by the publishing thread (as determined by the corresponding [DDSFlowController](#)). The number of samples that will be queued is determined by the [DDS\\_HistoryQosPolicy](#). When using [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#), only the most recent [DDS\\_HistoryQosPolicy::depth](#) samples are kept in the queue. Once unsent samples are removed from the queue, they are no longer available to the asynchronous publishing thread and will therefore never be sent.



### 7.59.3 Member Data Documentation

#### kind

`DDS_PublishModeQosPolicyKind DDS_PublishModeQosPolicy::kind`

Publishing mode.

**[default]** `DDS_SYNCHRONOUS_PUBLISH_MODE_QOS`

#### flow\_controller\_name

`const char* DDS_PublishModeQosPolicy::flow_controller_name`

Name of the associated flow controller.

NULL value or zero-length string refers to `DDS_DEFAULT_FLOW_CONTROLLER_NAME`.

Unless `flow_controller_name` points to a built-in flow controller, finalizing the `DDS_DataWriterQos` will also free the string pointed to by `flow_controller_name`. Therefore, use `DDS_String_dup` before passing the string to `flow_controller_name`, or reset `flow_controller_name` to NULL before finalizing the QoS.

Please refer to [Conventions](#) for more information.

#### See also

`DDSDomainParticipant::create_flowcontroller`  
`DDS_DEFAULT_FLOW_CONTROLLER_NAME`  
`DDS_FIXED_RATE_FLOW_CONTROLLER_NAME`  
`DDS_ON_DEMAND_FLOW_CONTROLLER_NAME`

**[default]** `DDS_DEFAULT_FLOW_CONTROLLER_NAME`

#### priority

`DDS_Long DDS_PublishModeQosPolicy::priority`

Publication priority.

A positive integer value designating the relative priority of the `DDSDataWriter`, used to determine the transmission order of pending writes.

Use of publication priorities requires the asynchronous publisher (`DDS_ASYNCHRONOUS_PUBLISH_MODE_QOS`) with `DDS_FlowControllerProperty_t::scheduling_policy` set to `DDS_HPF_FLOW_CONTROLLER_SCHED_POLICY`.

Larger numbers have higher priority.

**[default]** `DDS_PUBLICATION_PRIORITY_UNDEFINED`

## 7.60 DDS\_ReliabilityQosPolicy Struct Reference

Indicates the level of reliability offered/requested by RTI Connex DDS Micro.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_ReliabilityQosPolicyKind](#) kind  
*Kind of reliability.*
- struct [DDS\\_Duration\\_t](#) max\_blocking\_time  
*For Readers: The maximum time a receive thread may block if the reader is out of resources. For Writers: The maximum time a writer may block on a write call.*

### 7.60.1 Detailed Description

Indicates the level of reliability offered/requested by RTI Connex DDS Micro.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#), [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#)

Properties:

[RxO](#) = YES  
[Changeable](#) = NO

### 7.60.2 Usage

This policy indicates the level of reliability requested by a [DDSDataReader](#) or offered by a [DDSDataWriter](#).

These levels are ordered, [DDS\\_BEST\\_EFFORT\\_RELIABILITY\\_QOS](#) < [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#). A [DDSDataWriter](#) offering a level is implicitly offering all levels below.

If the [DDS\\_ReliabilityQosPolicy::kind](#) is set to [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#), data-samples originating from a single [DDSDataWriter](#) cannot be made available to the [DDSDataReader](#) if there are previous data samples that have not been received yet due to a communication error. In other words, the service will repair the error and re-transmit data-samples as needed in order to reconstruct a correct snapshot of the [DDSDataWriter](#) history before it is accessible by the [DDSDataReader](#).

If the [DDS\\_ReliabilityQosPolicy::kind](#) is set to [DDS\\_BEST\\_EFFORT\\_RELIABILITY\\_QOS](#), the service will not re-transmit missing data-samples. However for data-samples originating from any one DataWriter the service will ensure they are stored in the [DDSDataReader](#) history in the same order they originated in the [DDSDataWriter](#). In other words, the [DDSDataReader](#) may miss some data samples but it will never see the value of a data-object change from a newer value to an older value.

See also

[DDS\\_HistoryQosPolicy](#)  
[DDS\\_ResourceLimitsQosPolicy](#)

### 7.60.3 Compatibility

The value offered is considered compatible with the value requested if and only if the inequality *offered kind*  $\geq$  *requested kind* evaluates to 'TRUE'. For the purposes of this inequality, the values of [DDS\\_ReliabilityQosPolicy::kind](#) are considered ordered such that [DDS\\_BEST\\_EFFORT\\_RELIABILITY\\_QOS](#) < [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#).

### 7.60.4 Member Data Documentation

#### kind

[DDS\\_ReliabilityQosPolicyKind](#) [DDS\\_ReliabilityQosPolicy::kind](#)

Kind of reliability.

**[default]** [DDS\\_BEST\\_EFFORT\\_RELIABILITY\\_QOS](#)

#### max\_blocking\_time

[struct DDS\\_Duration\\_t](#) [DDS\\_ReliabilityQosPolicy::max\\_blocking\\_time](#)

For Readers: The maximum time a receive thread may block if the reader is out of resources. For Writers: The maximum time a writer may block on a write call.

For a [DDSDDataReader](#): A non-zero maximum blocking time enables the receive thread to block up to the maximum blocking time if a [DDSDDataReader](#) is out of resources when new data arrives. This is useful when the [DDS\\_HistoryQosPolicy::depth](#) is equal to `max_samples_per_instance` and the application is using a combination of loans and waitsets or polling. With a value of 0, the receive thread will not block.

For a [DDSDDataWriter](#): This setting applies only to the case where [DDS\\_ReliabilityQosPolicy::kind](#) = [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#). [FooDataWriter::write](#) is allowed to block if the [DDSDDataWriter](#) does not have space to store the value written.

**[default]** 0ms on a [DDSDDataReader](#) **[default]** 100ms on a [DDSDDataWriter](#)

**[range]** [0,1 year] for [DDSDDataWriter](#) and [DDSDDataReader](#)

See also

[DDS\\_ResourceLimitsQosPolicy](#)

## 7.61 DDS\_ReliableReaderActivityChangedStatus Struct Reference

<<*eXtension*>> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer.

```
#include <dds_c_publication.h>
```

### Public Attributes

- [DDS\\_Long active\\_count](#)  
*The current number of reliable readers currently matched with this reliable writer.*
- [DDS\\_Long inactive\\_count](#)  
*The number of reliable readers that have been dropped by this reliable writer because they failed to send acknowledgements in a timely fashion.*
- [DDS\\_Long active\\_count\\_change](#)  
*The most recent change in the number of active remote reliable readers.*
- [DDS\\_Long inactive\\_count\\_change](#)  
*The most recent change in the number of inactive remote reliable readers.*
- [DDS\\_InstanceHandle\\_t last\\_instance\\_handle](#)  
*The instance handle of the last reliable remote reader to be determined inactive.*

### 7.61.1 Detailed Description

<<*eXtension*>> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer.

Entity:

[DDSDataWriter](#)

Listener:

[DDSDataWriterListener](#)

This status is the reciprocal status to the [DDS\\_LivelinessChangedStatus](#) on the reader. It is different than the [DDS\\_LivelinessLostStatus](#) on the writer in that the latter informs the writer about its own liveliness; this status informs the writer about the "liveliness" (activity) of its matched readers.

All counts in this status will remain at zero for best effort writers.

### 7.61.2 Member Data Documentation

#### active\_count

[DDS\\_Long](#) `DDS_ReliableReaderActivityChangedStatus::active_count`

The current number of reliable readers currently matched with this reliable writer.

**inactive\_count**

`DDS_Long DDS_ReliableReaderActivityChangedStatus::inactive_count`

The number of reliable readers that have been dropped by this reliable writer because they failed to send acknowledgements in a timely fashion.

A reader is considered to be inactive after is has been sent heartbeats `DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries` times, each heartbeat having been separated from the previous by the current heartbeat period.

**active\_count\_change**

`DDS_Long DDS_ReliableReaderActivityChangedStatus::active_count_change`

The most recent change in the number of active remote reliable readers.

**inactive\_count\_change**

`DDS_Long DDS_ReliableReaderActivityChangedStatus::inactive_count_change`

The most recent change in the number of inactive remote reliable readers.

**last\_instance\_handle**

`DDS_InstanceHandle_t DDS_ReliableReaderActivityChangedStatus::last_instance_handle`

The instance handle of the last reliable remote reader to be determined inactive.

## 7.62 DDS\_ReliableSampleUnacknowledgedStatus Struct Reference

Status about an unacknowledged sample that is removed from the `DDSDataWriter` cache before being acknowledged by all matched `DDSDataReader` entities.

```
#include <dds_c_publication.h>
```

**Public Attributes**

- struct `DDS_SequenceNumber_t` `sequence_number`  
*The sequence number of the unacknowledged sample.*
- `DDS_Long` `unacknowledged_count`  
*The number of matched `DDSDataReader` entities that did not acknowledge the sample before it was removed.*
- `DDS_InstanceHandle_t` `instance_handle`  
*The instance\_handle for the sample. If the `DDSDataWriter` is not keyed the value is `DDS_HANDLE_NIL`.*

### 7.62.1 Detailed Description

Status about an unacknowledged sample that is removed from the [DDSDataWriter](#) cache before being acknowledged by all matched [DDSDataReader](#) entities.

Entity:

[DDSDataWriter](#)

Listener:

[DDSDataWriterListener](#)

### 7.62.2 Member Data Documentation

#### sequence\_number

```
struct DDS_SequenceNumber_t DDS_ReliableSampleUnacknowledgedStatus::sequence_number
```

The sequence number of the unacknowledged sample.

#### unacknowledged\_count

```
DDS_Long DDS_ReliableSampleUnacknowledgedStatus::unacknowledged_count
```

The number of matched [DDSDataReader](#) entities that did not acknowledge the sample before it was removed.

#### instance\_handle

```
DDS_InstanceHandle_t DDS_ReliableSampleUnacknowledgedStatus::instance_handle
```

The instance\_handle for the sample. If the [DDSDataWriter](#) is not keyed the value is DDS\_HANDLE\_NIL.

## 7.63 DDS\_RequestedDeadlineMissedStatus Struct Reference

### DDS\_REQUESTED\_DEADLINE\_MISSED\_STATUS

```
#include <dds_c_subscription.h>
```

#### Public Attributes

- [DDS\\_Long total\\_count](#)  
*Total cumulative count of the deadlines detected for any instance read by the [DDSDataReader](#).*
- [DDS\\_Long total\\_count\\_change](#)  
*The incremental number of deadlines detected since the last time the listener was called or the status was read.*
- [DDS\\_InstanceHandle\\_t last\\_instance\\_handle](#)  
*Handle to the last instance in the [DDSDataReader](#) for which a deadline was detected.*

### 7.63.1 Detailed Description

#### DDS\_REQUESTED\_DEADLINE\_MISSED\_STATUS

### 7.63.2 Member Data Documentation

#### total\_count

`DDS_Long DDS_RequestedDeadlineMissedStatus::total_count`

Total cumulative count of the deadlines detected for any instance read by the [DDSDataReader](#).

#### total\_count\_change

`DDS_Long DDS_RequestedDeadlineMissedStatus::total_count_change`

The incremental number of deadlines detected since the last time the listener was called or the status was read.

#### last\_instance\_handle

`DDS_InstanceHandle_t DDS_RequestedDeadlineMissedStatus::last_instance_handle`

Handle to the last instance in the [DDSDataReader](#) for which a deadline was detected.

## 7.64 DDS\_RequestedIncompatibleQosStatus Struct Reference

#### DDS\_REQUESTED\_INCOMPATIBLE\_QOS\_STATUS

```
#include <dds_c_subscription.h>
```

#### Public Attributes

- [DDS\\_Long total\\_count](#)  
*Total cumulative count of how many times the concerned [DDSDataReader](#) discovered a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDataReader](#).*
- [DDS\\_Long total\\_count\\_change](#)  
*The change in `total_count` since the last time the listener was called or the status was read.*
- [DDS\\_QosPolicyId\\_t last\\_policy\\_id](#)  
*The `PolicyId_t` of one of the policies that was found to be incompatible the last time an incompatibility was detected.*
- struct `DDS_QosPolicyCountSeq` [policies](#)  
*A list containing, for each policy, the total number of times that the concerned [DDSDataReader](#) discovered a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDataReader](#).*

### 7.64.1 Detailed Description

#### DDS\_REQUESTED\_INCOMPATIBLE\_QOS\_STATUS

See also

[RELIABILITY](#)  
[OWNERSHIP](#)  
[LIVELINESS](#)  
[DEADLINE](#)

### 7.64.2 Member Data Documentation

#### total\_count

`DDS_Long DDS_RequestedIncompatibleQosStatus::total_count`

Total cumulative count of how many times the concerned [DDSDataReader](#) discovered a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDataReader](#).

#### total\_count\_change

`DDS_Long DDS_RequestedIncompatibleQosStatus::total_count_change`

The change in `total_count` since the last time the listener was called or the status was read.

#### last\_policy\_id

`DDS_QosPolicyId_t DDS_RequestedIncompatibleQosStatus::last_policy_id`

The `PolicyId_t` of one of the policies that was found to be incompatible the last time an incompatibility was detected.

#### policies

`struct DDS_QosPolicyCountSeq DDS_RequestedIncompatibleQosStatus::policies`

A list containing, for each policy, the total number of times that the concerned [DDSDataReader](#) discovered a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDataReader](#).

## 7.65 DDS\_ResourceLimitsQosPolicy Struct Reference

Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

```
#include <dds_c_infrastructure.h>
```



## Public Attributes

- [DDS\\_Long max\\_samples](#)  
*Represents the maximum samples the middleware can store for any one [DDSDataWriter](#) (or [DDSDataReader](#)).*
- [DDS\\_Long max\\_instances](#)  
*Represents the maximum number of instances a [DDSDataWriter](#) (or [DDSDataReader](#)) can manage.*
- [DDS\\_Long max\\_samples\\_per\\_instance](#)  
*Represents the maximum number of samples of any one instance a [DDSDataWriter](#) (or [DDSDataReader](#)) can manage.*

### 7.65.1 Detailed Description

Specifies the resources that RTI Connex DDS Micro can consume in order to meet the requested QoS.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS\\_SAMPLE\\_REJECTED\\_STATUS](#), [DDS\\_SampleRejectedStatus](#)

Properties:

[RxO](#) = NO  
[Changeable](#) = NO

### 7.65.2 Usage

This policy controls the resources that RTI Connex DDS Micro can use in order to meet the requirements imposed by the application and other QoS settings.

If [DDSDataWriter](#) objects are communicating samples faster than they are ultimately taken by the [DDSDataReader](#) objects, the middleware will eventually hit against some of the QoS-imposed resource limits. Note that this may occur when just a single [DDSDataReader](#) cannot keep up with its corresponding [DDSDataWriter](#). The behavior in this case depends on the setting for the [RELIABILITY](#). If reliability is [DDS\\_BEST\\_EFFORT\\_RELIABILITY\\_QOS](#) then RTI Connex DDS Micro is allowed to drop samples. If the reliability is [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#), RTI Connex DDS Micro will block the [DDSDataWriter](#) or discard the sample at the [DDSDataReader](#) in order not to lose existing samples.

See also

[DDS\\_ReliabilityQosPolicy](#)  
[DDS\\_HistoryQosPolicy](#)

### 7.65.3 Consistency

All limits must be set to positive finite values. Furthermore, `max_samples` must be large enough to contain all samples for all instances:

```
max_samples >= max_instances * max_samples_per_instance
```

The setting of [RESOURCE\\_LIMITS](#) `max_samples_per_instance` must be consistent with the [HISTORY](#) `depth`. For these two QoS to be consistent, they must verify that *depth* ≤ *max\_samples\_per\_instance*.

See also

[DDS\\_HistoryQosPolicy](#)

### 7.65.4 Member Data Documentation

#### `max_samples`

```
DDS_Long DDS_ResourceLimitsQosPolicy::max_samples
```

Represents the maximum samples the middleware can store for any one [DDSDataWriter](#) (or [DDSDataReader](#)).

Specifies the maximum number of data samples a [DDSDataWriter](#) (or [DDSDataReader](#)) can manage across all the instances associated with it.

For unkeyed types this value has to be equal to `max_samples_per_instance`.

**[default]** 1

**[range]** [1, 100 million] ≥ (max\_samples\_per\_instance \* max\_instances),

#### `max_instances`

```
DDS_Long DDS_ResourceLimitsQosPolicy::max_instances
```

Represents the maximum number of instances a [DDSDataWriter](#) (or [DDSDataReader](#)) can manage.

**[default]** 1

**[range]** [1, 1 million]

#### `max_samples_per_instance`

```
DDS_Long DDS_ResourceLimitsQosPolicy::max_samples_per_instance
```

Represents the maximum number of samples of any one instance a [DDSDataWriter](#) (or [DDSDataReader](#)) can manage.

For unkeyed types this value has to be equal to `max_samples`.

**[default]** 1

**[range]** [1, 100 million], ≤ max\_samples, ≥ [DDS\\_HistoryQosPolicy::depth](#)

## 7.66 DDS\_RtpsReliableReaderProtocol\_t Struct Reference

Qos related to reliable reader protocol defined in RTPS.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- struct [DDS\\_Duration\\_t](#) `nack_period`  
*The period at which to send NACKs.*

#### 7.66.1 Detailed Description

Qos related to reliable reader protocol defined in RTPS.

It is used to configure the reliable reader according to RTPS protocol.

Properties:

[RxO](#) = N/A  
[Changeable](#) = NO

QoS:

[DDS\\_DataReaderProtocolQosPolicy](#)

#### 7.66.2 Member Data Documentation

##### `nack_period`

```
struct DDS\_Duration\_t DDS_RtpsReliableReaderProtocol_t::nack_period
```

The period at which to send NACKs.

A reliable reader will send periodic NACKs at this rate when it first matches with a reliable writer. The reader will stop sending NACKs when it has received all available historical data from the writer. Note that this NACK period only determines the rate at which NACKs are sent until the first HB is received from the writer. When the first HB is received, the writer's HB period determines how fast historical data is sent.

**[default]** 50 ms

**[range]** [1 nanosec, 1 year]

## 7.67 DDS\_RtpsReliableWriterProtocol\_t Struct Reference

QoS related to the reliable writer protocol defined in RTPS.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- struct [DDS\\_Duration\\_t](#) `heartbeat_period`  
*The period at which to send heartbeats.*
- [DDS\\_Long](#) `heartbeats_per_max_samples`  
*The number of heartbeats per send queue.*
- [DDS\\_Long](#) `max_send_window`  
*The maximum size of a reliable send window.*
- [DDS\\_Long](#) `max_heartbeat_retries`  
*The maximum number of heartbeat retries before marking a remote reader as inactive.*

### 7.67.1 Detailed Description

QoS related to the reliable writer protocol defined in RTPS.

It is used to configure a reliable writer according to RTPS protocol.

Properties:

`RxO` = N/A  
`Changeable` = NO

QoS:

[DDS\\_DataWriterProtocolQosPolicy](#)

### 7.67.2 Member Data Documentation

#### `heartbeat_period`

```
struct DDS\_Duration\_t DDS_RtpsReliableWriterProtocol_t::heartbeat_period
```

The period at which to send heartbeats.

A reliable writer will send periodic heartbeat at this rate.

**[default]** 3.0 seconds

**[range]** [1 nanosec, 1 year]

## heartbeats\_per\_max\_samples

`DDS_Long DDS_RtpsReliableWriterProtocol_t::heartbeats_per_max_samples`

The number of heartbeats per send queue.

A piggyback heartbeat will be sent every `[DDS_ResourceLimitsQosPolicy::max_samples/heartbeats_per_max_samples]` number of samples.

If set to zero, no piggyback heartbeat will be sent. If `DDS_ResourceLimitsQosPolicy::max_samples` is `DDS_LENGTH_UNLIMITED`, 100 million is assumed as the maximum value in the calculation.

**[default]** 1

**[range]** [0, 100 million]

- For `DDS_DataWriterProtocolQosPolicy::rtps_reliable_writer`:  
heartbeats\_per\_max\_samples ≤ `DDS_ResourceLimitsQosPolicy::max_samples`

## max\_send\_window

`DDS_Long DDS_RtpsReliableWriterProtocol_t::max_send_window`

The maximum size of a reliable send window.

A reliable DataWriter may maintain a send window of samples in-flight that have not been acknowledged by its DataReaders. The `max_send_window` is the maximum number of unacknowledged samples in-flight. When the window is at `max_send_window`, subsequent writes will pass but not publish any data over the network.

**[default]** `DDS_LENGTH_UNLIMITED`, meaning the window will be as large as resource limits allow

**[range]** [1, 256] of `DDS_LENGTH_UNLIMITED`

## max\_heartbeat\_retries

`DDS_Long DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries`

The maximum number of heartbeat retries before marking a remote reader as inactive.

When a remote reader has not acked all the samples the reliable writer has in its queue, and `max_heartbeat_retries` number of periodic heartbeats has been sent without receiving any ack/nack back, the remote reader will be marked as inactive (not alive) and be ignored until it resumes sending ack/nack.

**[default]** `DDS_LENGTH_UNLIMITED`

**[range]** [1, 1 million] or `DDS_LENGTH_UNLIMITED`

## 7.68 DDS\_RtpsWellKnownPorts\_t Struct Reference

RTPS well-known port mapping configuration.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Long port\\_base](#)  
*The base port offset.*
- [DDS\\_Long domain\\_id\\_gain](#)  
*Tunable domain gain parameter.*
- [DDS\\_Long participant\\_id\\_gain](#)  
*Tunable participant gain parameter.*
- [DDS\\_Long builtin\\_multicast\\_port\\_offset](#)  
*Additional offset for **metatraffic** multicast port.*
- [DDS\\_Long builtin\\_unicast\\_port\\_offset](#)  
*Additional offset for **metatraffic** unicast port.*
- [DDS\\_Long user\\_multicast\\_port\\_offset](#)  
*Additional offset for **usertraffic** multicast port.*
- [DDS\\_Long user\\_unicast\\_port\\_offset](#)  
*Additional offset for **usertraffic** unicast port.*

### 7.68.1 Detailed Description

RTPS well-known port mapping configuration.

RTI Connext DDS Micro uses the RTPS wire protocol. The discovery protocols defined by RTPS rely on well-known ports to initiate discovery. These well-known ports define the multicast and unicast ports on which a Participant will listen for discovery **metatraffic** from other Participants. The discovery metatraffic contains all the information required to establish the presence of remote DDS entities in the network.

The well-known ports are defined by RTPS in terms of port mapping expressions with several tunable parameters, which allow the user to customize what network ports are used by the middleware. These parameters are exposed in [DDS\\_RtpsWellKnownPorts\\_t](#). In order for all Participants in a system to correctly discover each other, it is important that they all use the same port mapping expressions.

The actual port mapping expressions, as defined by the RTPS specification, can be found below. In addition to the parameters listed in [DDS\\_RtpsWellKnownPorts\\_t](#), the port numbers depend on:

- domain\_id, as specified in [DDSDomainParticipantFactory::create\\_participant](#)

- participant\_id, as specified using [DDS\\_WireProtocolQosPolicy::participant\\_id](#)

The domain\_id parameter ensures no port conflicts exist between Participants belonging to different domains. This also means that discovery metatraffic in one domain is not visible to Participants in a different domain. The participant\_id parameter ensures that unique unicast port numbers are assigned to Participants belonging to the same domain on a given host.

The *metatraffic\_unicast\_port* is used to exchange discovery metatraffic using unicast.

```
metatraffic_unicast_port = port_base + (domain_id_gain * domain_id) + (participant_id_gain * participant_id) + bu
```

The *metatraffic\_multicast\_port* is used to exchange discovery metatraffic using multicast.

```
metatraffic_multicast_port = port_base + (domain_id_gain * domain_id) + builtin_multicast_port_offset
```

RTPS also defines the *default* multicast and unicast ports on which DataReaders and DataWriters receive **usertraffic**.

The *usertraffic\_unicast\_port* is used to exchange user data using unicast.

```
usertraffic_unicast_port = port_base + (domain_id_gain * domain_id) + (participant_id_gain * participant_id) + us
```

The *usertraffic\_multicast\_port* is used to exchange user data using multicast. The corresponding multicast group addresses can be configured using

```
usertraffic_multicast_port = port_base + (domain_id_gain * domain_id) + user_multicast_port_offset
```

By default, the port mapping parameters are configured to compliant with OMG's DDS Interoperability Wire Protocol (see also [DDS\\_INTEROPERABLE\\_RTPS\\_WELL\\_KNOWN\\_PORTS](#)).

The OMG's DDS Interoperability Wire Protocol compliant port mapping parameters are *not* backwards compatible with previous versions of the RTI Connex DDS Micro middleware.

When modifying the port mapping parameters, care must be taken to avoid port aliasing. This would result in undefined discovery behavior. The chosen parameter values will also determine the maximum possible number of domains in the system and the maximum number of participants per domain. Additionally, any resulting mapped port number must be within the range imposed by the underlying transport. For example, for UDPv4, this range typically equals [1024 - 65535].

QoS:

[DDS\\_WireProtocolQosPolicy](#)

## 7.68.2 Member Data Documentation

### **port\_base**

```
DDS_Long DDS_RtpsWellKnownPorts_t::port_base
```

The base port offset.

All mapped well-known ports are offset by this value.

**[default]** 7400

**[range]** [ $\geq 1$ ], but resulting ports must be within the range imposed by the underlying transport.

**domain\_id\_gain**

`DDS_Long DDS_RtpsWellKnownPorts_t::domain_id_gain`

Tunable domain gain parameter.

Multiplier of the `domain_id`. Together with `participant_id_gain`, it determines the highest `domain_id` and `participant_id` allowed on this network.

In general, there are two ways to setup `domain_id_gain` and `participant_id_gain` parameters.

If `domain_id_gain > participant_id_gain`, it results in a port mapping layout where all [DDSDomainParticipant](#) instances within a single domain occupy a consecutive range of `domain_id_gain` ports. Precisely, all ports occupied by the domain fall within:

```
(port_base + (domain_id_gain * domain_id))
```

and:

```
(port_base + (domain_id_gain * (domain_id + 1)) - 1)
```

Under such a case, the highest `domain_id` is limited only by the underlying transport's maximum port. The highest `participant_id`, however, must satisfy:

```
max_participant_id < (domain_id_gain / participant_id_gain)
```

On the contrary, if `domain_id_gain <= participant_id_gain`, it results in a port mapping layout where a given domain's [DDSDomainParticipant](#) instances occupy ports spanned across the entire valid port range allowed by the underlying transport. For instance, it results in the following potential mapping:

| Mapped Port        | Domain Id     | Participant ID     |
|--------------------|---------------|--------------------|
| higher port number | Domain Id = 1 | Participant ID = 2 |
|                    | Domain Id = 0 | Participant ID = 2 |
|                    | Domain Id = 1 | Participant ID = 1 |
|                    | Domain Id = 0 | Participant ID = 1 |
|                    | Domain Id = 1 | Participant ID = 0 |
| lower port number  | Domain Id = 0 | Participant ID = 0 |

Under this case, the highest `participant_id` is limited only by the underlying transport's maximum port. The highest `domain_id`, however, must satisfy:

```
max_domain_id < (participant_id_gain / domain_id_gain)
```

Additionally, `domain_id_gain` also determines the range of the port-specific offsets.

```
domain_id_gain > abs(builtin_multicast_port_offset - user_multicast_port_offset)
```

```
domain_id_gain > abs(builtin_unicast_port_offset - user_unicast_port_offset)
```

Violating this may result in port aliasing and undefined discovery behavior.

**[default]** 250

**[range]** [ $> 0$ ], but resulting ports must be within the range imposed by the underlying transport.



### **participant\_id\_gain**

`DDS_Long DDS_RtpsWellKnownPorts_t::participant_id_gain`

Tunable participant gain parameter.

Multiplier of the `participant_id`. See `DDS_RtpsWellKnownPorts_t::domain_id_gain` for its implications on the highest `domain_id` and `participant_id` allowed on this network.

Additionally, `participant_id_gain` also determines the range of `builtin_unicast_port_offset` and `user_unicast_port_offset`.

`participant_id_gain > abs(builtin_unicast_port_offset - user_unicast_port_offset)`

**[default]** 2

**[range]** [ $> 0$ ], but resulting ports must be within the range imposed by the underlying transport.

### **builtin\_multicast\_port\_offset**

`DDS_Long DDS_RtpsWellKnownPorts_t::builtin_multicast_port_offset`

Additional offset for **metatraffic** multicast port.

It must be unique from other port-specific offsets.

**[default]** 0

**[range]** [ $\geq 0$ ], but resulting ports must be within the range imposed by the underlying transport.

### **builtin\_unicast\_port\_offset**

`DDS_Long DDS_RtpsWellKnownPorts_t::builtin_unicast_port_offset`

Additional offset for **metatraffic** unicast port.

It must be unique from other port-specific offsets.

**[default]** 10

**[range]** [ $\geq 0$ ], but resulting ports must be within the range imposed by the underlying transport.

### **user\_multicast\_port\_offset**

`DDS_Long DDS_RtpsWellKnownPorts_t::user_multicast_port_offset`

Additional offset for **usertraffic** multicast port.

It must be unique from other port-specific offsets.

**[default]** 1

**[range]** [ $\geq 0$ ], but resulting ports must be within the range imposed by the underlying transport.

**user\_unicast\_port\_offset**

```
DDS_Long DDS_RtpsWellKnownPorts_t::user_unicast_port_offset
```

Additional offset for **usertraffic** unicast port.

It must be unique from other port-specific offsets.

**[default]** 11

**[range]** [ $\geq 0$ ], but resulting ports must be within the range imposed by the underlying transport.

**7.69 DDS\_SampleIdentity\_t Struct Reference**

Data structure used to provide explicit identity information about a published data sample.

```
#include <dds_c_infrastructure.h>
```

**Public Attributes**

- struct [DDS\\_GUID\\_t](#) **writer\_guid**  
*The GUID of the [DDSDataWriter](#) that published the sample.*
- struct [DDS\\_SequenceNumber\\_t](#) **sequence\_number**  
*Sequence number of the sample.*

**7.69.1 Detailed Description**

Data structure used to provide explicit identity information about a published data sample.

**7.69.2 Member Data Documentation****writer\_guid**

```
struct DDS_GUID_t DDS_SampleIdentity_t::writer_guid
```

The GUID of the [DDSDataWriter](#) that published the sample.

**sequence\_number**

```
struct DDS_SequenceNumber_t DDS_SampleIdentity_t::sequence_number
```

Sequence number of the sample.

## 7.70 DDS\_SampleInfo Struct Reference

Information that accompanies each sample that is `read` or `taken`.

```
#include <dds_c_subscription.h>
```

### Public Attributes

- [DDS\\_SampleStateKind](#) `sample_state`  
*The sample state of the sample.*
- [DDS\\_ViewStateKind](#) `view_state`  
*The view state of the instance.*
- [DDS\\_InstanceStateKind](#) `instance_state`  
*The instance state of the instance.*
- [DDS\\_InstanceHandle\\_t](#) `instance_handle`  
*Identifies locally the corresponding instance.*
- [DDS\\_InstanceHandle\\_t](#) `publication_handle`  
*Identifies locally the DataWriter that modified the instance.*
- [DDS\\_Boolean](#) `valid_data`  
*Indicates whether the `DataSample` contains data or else it is only used to communicate a change in the `instance_state` of the instance.*
- struct [DDS\\_Time\\_t](#) `source_timestamp`  
*The timestamp when the sample was written by a DataWriter.*
- struct [DDS\\_Time\\_t](#) `reception_timestamp`  
*<<eXtension>> The timestamp when the sample was committed by a DataReader.*
- struct [DDS\\_SequenceNumber\\_t](#) `publication_sequence_number`  
*<<eXtension>> The publication sequence number.*
- [DDS\\_EncapsulationId\\_t](#) `encapsulation_id`

### 7.70.1 Detailed Description

Information that accompanies each sample that is `read` or `taken`.

### 7.70.2 Interpretation of the SampleInfo

The [DDS\\_SampleInfo](#) contains information pertaining to the associated `Data` instance sample including:

- the `sample_state` of the `Data` value (i.e., if it has already been read or not)
- the `view_state` of the related instance (i.e., if the instance is new or not)
- the `instance_state` of the related instance (i.e., if the instance is alive or not)
- the `valid_data` flag. This flag indicates whether there is data associated with the sample. Some samples do not contain data indicating only a change on the `instance_state` of the corresponding instance.
- The `source_timestamp` of the sample. This is the timestamp provided by the [DDSDataWriter](#) at the time the sample was produced.

See also

[DDS\\_SampleStateKind](#), [DDS\\_InstanceStateKind](#), [DDS\\_ViewStateKind](#), [DDS\\_SampleInfo::valid\\_data](#)

### 7.70.3 Member Data Documentation

#### sample\_state

`DDS_SampleStateKind DDS_SampleInfo::sample_state`

The sample state of the sample.

Indicates whether or not the corresponding data sample has already been read.

See also

[DDS\\_SampleStateKind](#)

#### view\_state

`DDS_ViewStateKind DDS_SampleInfo::view_state`

The view state of the instance.

Indicates whether the [DDSDataReader](#) has already seen samples for the most-current generation of the related instance.

See also

[DDS\\_ViewStateKind](#)

#### instance\_state

`DDS_InstanceStateKind DDS_SampleInfo::instance_state`

The instance state of the instance.

Indicates whether the instance is currently in existence or, if it has been disposed, the reason why it was disposed.

See also

[DDS\\_InstanceStateKind](#)

#### instance\_handle

`DDS_InstanceHandle_t DDS_SampleInfo::instance_handle`

Identifies locally the corresponding instance.

**publication\_handle**

```
DDS_InstanceHandle_t DDS_SampleInfo::publication_handle
```

Identifies locally the DataWriter that modified the instance.

**valid\_data**

```
DDS_Boolean DDS_SampleInfo::valid_data
```

Indicates whether the DataSample contains data or else it is only used to communicate a change in the instance↵\_state of the instance.

Normally each DataSample contains both a DDS\_SampleInfo and some Data. However there are situations where a DataSample contains only the DDS\_SampleInfo and does not have any associated data. This occurs when the RTI Connex Micro notifies the application of a change of state for an instance that was caused by some internal mechanism (such as a timeout) for which there is no associated data. An example of this situation is when the RTI Connex Micro detects that an instance has no writers and changes the corresponding instance\_state to DDS\_NOT\_ALIVE\_NO\_WRITERS\_INSTANCE\_STATE.

The application can distinguish whether a particular DataSample has data by examining the value of the valid↵\_data flag. If this flag is set to DDS\_BOOLEAN\_TRUE, then the DataSample contains valid Data. If the flag is set to DDS\_BOOLEAN\_FALSE, the DataSample contains no Data.

To ensure correctness and portability, the valid\_data flag must be examined by the application prior to accessing the Data associated with the DataSample and if the flag is set to DDS\_BOOLEAN\_FALSE, the application should not access the Data associated with the DataSample, that is, the application should access only the DDS\_SampleInfo.

**source\_timestamp**

```
struct DDS_Time_t DDS_SampleInfo::source_timestamp
```

The timestamp when the sample was written by a DataWriter.

**reception\_timestamp**

```
struct DDS_Time_t DDS_SampleInfo::reception_timestamp
```

<<eXtension>> The timestamp when the sample was committed by a DataReader.

**publication\_sequence\_number**

```
struct DDS_SequenceNumber_t DDS_SampleInfo::publication_sequence_number
```

<<eXtension>> The publication sequence number.

**encapsulation\_id**

```
DDS_EncapsulationId_t DDS_SampleInfo::encapsulation_id
```

The transport encapsulation the sample was received as

**7.71 DDS\_SampleInfoSeq Struct Reference**

Declares IDL sequence < [DDS\\_SampleInfo](#) > .

```
#include <dds_c_subscription.h>
```

**7.71.1 Detailed Description**

Declares IDL sequence < [DDS\\_SampleInfo](#) > .

See also

[FooSeq](#)

**7.72 DDS\_SampleLostStatus Struct Reference**

[DDS\\_SAMPLE\\_LOST\\_STATUS](#)

```
#include <dds_c_subscription.h>
```

**Public Attributes**

- [DDS\\_Long total\\_count](#)  
*Total cumulative count of all samples lost across all instances of data published under the [DDSTopic](#).*
- [DDS\\_Long total\\_count\\_change](#)  
*The incremental number of samples lost since the last time the listener was called or the status was read.*
- [DDS\\_SampleLostStatusKind reason](#)  
*The reason a sample was lost.*
- struct [DDS\\_SampleInfo sample\\_info](#)  
*Additional information about the sample lost for reasons other than [DDS\\_SAMPLE\\_LOST\\_BY\\_DATAWRITER](#) and [DDS\\_SAMPLE\\_LOST\\_NOT\\_LOST](#).*

**7.72.1 Detailed Description**

[DDS\\_SAMPLE\\_LOST\\_STATUS](#)

### 7.72.2 Member Data Documentation

#### **total\_count**

`DDS_Long DDS_SampleLostStatus::total_count`

Total cumulative count of all samples lost across all instances of data published under the [DDSTopic](#).

#### **total\_count\_change**

`DDS_Long DDS_SampleLostStatus::total_count_change`

The incremental number of samples lost since the last time the listener was called or the status was read.

#### **reason**

`DDS_SampleLostStatusKind DDS_SampleLostStatus::reason`

The reason a sample was lost.

#### **sample\_info**

`struct DDS_SampleInfo DDS_SampleLostStatus::sample_info`

Additional information about the sample lost for reasons other than [DDS\\_SAMPLE\\_LOST\\_BY\\_DATAWRITER](#) and [DDS\\_SAMPLE\\_LOST\\_NOT\\_LOST](#).

## 7.73 DDS\_SampleRejectedStatus Struct Reference

### [DDS\\_SAMPLE\\_REJECTED\\_STATUS](#)

```
#include <dds_c_subscription.h>
```

#### **Public Attributes**

- [DDS\\_Long total\\_count](#)  
*Total cumulative count of samples rejected by the [DDSDataReader](#).*
- [DDS\\_Long total\\_count\\_change](#)  
*The incremental number of samples rejected since the last time the listener was called or the status was read.*
- [DDS\\_SampleRejectedStatusKind last\\_reason](#)  
*Reason for rejecting the last sample rejected.*
- [DDS\\_InstanceHandle\\_t last\\_instance\\_handle](#)  
*Handle to the instance being updated by the last sample that was rejected.*

### 7.73.1 Detailed Description

#### DDS\_SAMPLE\_REJECTED\_STATUS

A status structure describing the samples rejected by the reader.

### 7.73.2 Member Data Documentation

#### total\_count

`DDS_Long DDS_SampleRejectedStatus::total_count`

Total cumulative count of samples rejected by the [DDSDataReader](#).

#### total\_count\_change

`DDS_Long DDS_SampleRejectedStatus::total_count_change`

The incremental number of samples rejected since the last time the listener was called or the status was read.

#### last\_reason

`DDS_SampleRejectedStatusKind DDS_SampleRejectedStatus::last_reason`

Reason for rejecting the last sample rejected.

See also

[DDS\\_SampleRejectedStatusKind](#)

#### last\_instance\_handle

`DDS_InstanceHandle_t DDS_SampleRejectedStatus::last_instance_handle`

Handle to the instance being updated by the last sample that was rejected.

## 7.74 DDS\_SecTransform\_Configuration Struct Reference

A structure for configuration this Transform PSL.

```
#include <psk_oss1_transform.h>
```



## Public Attributes

- `const char * provider_name`  
*A 0-terminated string with the name of the crypto provider to use. or NULL for the default provider.*
- `struct PSK_ErrListener err_listener`  
*Configuration items for an error callback mechanism.*

### 7.74.1 Detailed Description

A structure for configuration this Transform PSL.

The values of the fields in this structure influence how the Transform PSL behaves. The details of what the impact is, is known to the implementation of the PSL only.

The configuration has to be provided at creation time and is immutable.

See Also `PSK_OSSL_get_default_configuration`

### 7.74.2 Member Data Documentation

#### `provider_name`

```
const char* DDS_SecTransform_Configuration::provider_name
```

A 0-terminated string with the name of the crypto provider to use. or NULL for the default provider.

#### `err_listener`

```
struct PSK_ErrListener DDS_SecTransform_Configuration::err_listener
```

Configuration items for an error callback mechanism.

## 7.75 DDS\_SequenceNumber\_t Struct Reference

Type for *sequence* number representation.

```
#include <dds_c_infrastructure.h>
```

## Public Attributes

- `DDS_Long high`  
*The most significant part of the sequence number.*
- `DDS_UnsignedLong low`  
*The least significant part of the sequence number.*

### 7.75.1 Detailed Description

Type for *sequence* number representation.

Represents a 64-bit sequence number.

### 7.75.2 Member Data Documentation

#### high

`DDS_Long DDS_SequenceNumber_t::high`

The most significant part of the sequence number.

#### low

`DDS_UnsignedLong DDS_SequenceNumber_t::low`

The least significant part of the sequence number.

## 7.76 DDS\_SubscriberQos Struct Reference

QoS policies supported by a `DDSSubscriber` entity.

```
#include <dds_c_subscription.h>
```

### Public Attributes

- struct `DDS_EntityFactoryQosPolicy entity_factory`  
*Entity factory policy, `ENTITY_FACTORY`.*
- struct `DDS_PartitionQosPolicy partition`  
*The `DDSSubscriber` `PARTITION` policy.*
- struct `DDS_GroupDataQosPolicy group_data`  
*Group data policy, `GROUP_DATA`.*
- struct `DDS_EntityNameQosPolicy subscriber_name`  
*<<eXtension>> The `DDSSubscriber` name. `ENTITY_NAME`.*

### 7.76.1 Detailed Description

QoS policies supported by a `DDSSubscriber` entity.

If the call to `DDSSubscriber::set_qos` attempts to change the partition QoS then it will return `DDS_RETCODE_INCONSISTENT_POLICY`

### 7.76.2 Member Data Documentation

#### entity\_factory

```
struct DDS_EntityFactoryQosPolicy DDS_SubscriberQos::entity_factory
```

Entity factory policy, [ENTITY\\_FACTORY](#).

#### partition

```
struct DDS_PartitionQosPolicy DDS_SubscriberQos::partition
```

The [DDSSubscriber](#) PARTITION policy.

A set of strings that introduces logical PARTITIONS.

Only entities that belong to the same PARTITION can communicate to each other. The PARTITION QoS policy applies directly to the [DDSSubscriber](#). [DDSDataReader](#) belong to the PARTITIONS of the [DDSSubscriber](#) that created them.

The default PARTITION during initialization is the empty string, "".

#### [PARTITION](#)

#### group\_data

```
struct DDS_GroupDataQosPolicy DDS_SubscriberQos::group_data
```

Group data policy, [GROUP\\_DATA](#).

#### subscriber\_name

```
struct DDS_EntityNameQosPolicy DDS_SubscriberQos::subscriber_name
```

<<*eXtension*>> The [DDSSubscriber](#) name. [ENTITY\\_NAME](#).

## 7.77 DDS\_SubscriptionBuiltinTopicData Struct Reference

Entry created when a [DDSDataReader](#) is discovered in association with its Subscriber.

```
#include <dds_c_discovery.h>
```

**Public Attributes**

- struct [DDS\\_BuiltinTopicKey\\_t](#) `key`  
*DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDataReader](#).*
- struct [DDS\\_BuiltinTopicKey\\_t](#) `participant_key`  
*DCPS key of the participant to which the DataReader belongs.*
- char \* [topic\\_name](#)  
*Name of the related [DDSTopic](#).*
- char \* [type\\_name](#)  
*Name of the type attached to the [DDSTopic](#).*
- struct [DDS\\_DeadlineQosPolicy](#) `deadline`  
*Policy of the corresponding DataReader.*
- struct [DDS\\_OwnershipQosPolicy](#) `ownership`  
*Policy of the corresponding DataReader.*
- struct [DDS\\_LatencyBudgetQosPolicy](#) `latency_budget`  
*Latency budget policy, [LATENCY\\_BUDGET](#).*
- struct [DDS\\_ReliabilityQosPolicy](#) `reliability`  
*Policy of the corresponding DataReader.*
- struct [DDS\\_LivelinessQosPolicy](#) `liveliness`  
*Policy of the corresponding DataReader.*
- struct [DDS\\_DurabilityQosPolicy](#) `durability`  
*Policy of the corresponding DataReader.*
- struct [DDS\\_DestinationOrderQosPolicy](#) `destination_order`  
*Policy of the corresponding DataReader.*
- struct [DDS\\_LocatorSeq](#) `unicast_locator`  
*<<eXtension>> Custom unicast locator that the endpoint can specify. The default locator will be used if this is not specified.*
- struct [DDS\\_LocatorSeq](#) `multicast_locator`  
*<<eXtension>> Custom multicast locator that the endpoint can specify. The default locator will be used if this is not specified.*
- struct [DDS\\_PresentationQosPolicy](#) `presentation`  
*Policy of the Subscriber to which the DataReader belongs.*
- struct [DDS\\_DataRepresentationQosPolicy](#) `representation`  
*Data representation policy, [DATA\\_REPRESENTATION](#).*
- struct [DDS\\_PartitionQosPolicy](#) `partition`  
*Policy of the Subscriber to which the DataReader belongs.*
- struct [DDS\\_UserDataQosPolicy](#) `user_data`  
*Policy of the corresponding DataReader.*
- struct [DDS\\_GroupDataQosPolicy](#) `group_data`  
*Policy of the Subscriber to which the DataReader belongs.*
- struct [DDS\\_TopicDataQosPolicy](#) `topic_data`  
*Policy of the related Topic.*
- struct [DDS\\_ContentFilterProperty](#) `content_filter`  
*<<experimental>> All of the information required for a DataWriter to perform writer-side filtering for a DataReader.*

### 7.77.1 Detailed Description

Entry created when a [DDSDDataReader](#) is discovered in association with its Subscriber.

Data associated with the built-in topic [DDS\\_SUBSCRIPTION\\_TOPIC\\_NAME](#). It contains QoS policies and additional information that apply to the remote [DDSDDataReader](#) the related [DDSSubscriber](#).

See also

[DDS\\_SUBSCRIPTION\\_TOPIC\\_NAME](#)

### 7.77.2 Member Data Documentation

#### **key**

```
struct DDS\_BuiltinTopicKey\_t DDS_SubscriptionBuiltinTopicData::key
```

DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDDataReader](#).

Key GUID must be first

#### **participant\_key**

```
struct DDS\_BuiltinTopicKey\_t DDS_SubscriptionBuiltinTopicData::participant_key
```

DCPS key of the participant to which the DataReader belongs.

#### **topic\_name**

```
char* DDS_SubscriptionBuiltinTopicData::topic_name
```

Name of the related [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [Conventions](#).

See also

[Conventions](#)

**type\_name**

```
char* DDS_SubscriptionBuiltinTopicData::type_name
```

Name of the type attached to the [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [Conventions](#).

See also

[Conventions](#)

**deadline**

```
struct DDS_DeadlineQosPolicy DDS_SubscriptionBuiltinTopicData::deadline
```

Policy of the corresponding DataReader.

**ownership**

```
struct DDS_OwnershipQosPolicy DDS_SubscriptionBuiltinTopicData::ownership
```

Policy of the corresponding DataReader.

**latency\_budget**

```
struct DDS_LatencyBudgetQosPolicy DDS_SubscriptionBuiltinTopicData::latency_budget
```

Latency budget policy, [LATENCY\\_BUDGET](#).

**reliability**

```
struct DDS_ReliabilityQosPolicy DDS_SubscriptionBuiltinTopicData::reliability
```

Policy of the corresponding DataReader.

**liveliness**

```
struct DDS_LivelinessQosPolicy DDS_SubscriptionBuiltinTopicData::liveliness
```

Policy of the corresponding DataReader.

**durability**

```
struct DDS_DurabilityQosPolicy DDS_SubscriptionBuiltinTopicData::durability
```

Policy of the corresponding DataReader.

**destination\_order**

```
struct DDS_DestinationOrderQosPolicy DDS_SubscriptionBuiltinTopicData::destination_order
```

Policy of the corresponding DataReader.

**unicast\_locator**

```
struct DDS_LocatorSeq DDS_SubscriptionBuiltinTopicData::unicast_locator
```

<<*eXtension*>> Custom unicast locator that the endpoint can specify. The default locator will be used if this is not specified.

**multicast\_locator**

```
struct DDS_LocatorSeq DDS_SubscriptionBuiltinTopicData::multicast_locator
```

<<*eXtension*>> Custom multicast locator that the endpoint can specify. The default locator will be used if this is not specified.

**presentation**

```
struct DDS_PresentationQosPolicy DDS_SubscriptionBuiltinTopicData::presentation
```

Policy of the Subscriber to which the DataReader belongs.

**representation**

```
struct DDS_DataRepresentationQosPolicy DDS_SubscriptionBuiltinTopicData::representation
```

Data representation policy, [DATA\\_REPRESENTATION](#).

**partition**

```
struct DDS_PartitionQosPolicy DDS_SubscriptionBuiltinTopicData::partition
```

Policy of the Subscriber to which the DataReader belongs.

**user\_data**

```
struct DDS_UserDataQosPolicy DDS_SubscriptionBuiltinTopicData::user_data
```

Policy of the corresponding DataReader.

**group\_data**

```
struct DDS_GroupDataQosPolicy DDS_SubscriptionBuiltinTopicData::group_data
```

Policy of the Subscriber to which the DataReader belongs.

**topic\_data**

```
struct DDS_TopicDataQosPolicy DDS_SubscriptionBuiltinTopicData::topic_data
```

Policy of the related Topic.

**content\_filter**

```
struct DDS_ContentFilterProperty DDS_SubscriptionBuiltinTopicData::content_filter
```

<<experimental>> All of the information required for a DataWriter to perform writer-side filtering for a DataReader.

If the content filtering feature is not enabled, then this field is unused.

**See also**

`::DDSFilterPluginFactory::register` for how to enable the content filtering feature

**7.78 DDS\_SubscriptionMatchedStatus Struct Reference****DDS\_SUBSCRIPTION\_MATCHED\_STATUS**

```
#include <dds_c_subscription.h>
```

**Public Attributes**

- **DDS\_Long total\_count**  
The total cumulative number of times the concerned *DDSDataReader* discovered a "match" with a *DDSDataWriter*.
- **DDS\_Long total\_count\_change**  
The change in *total\_count* since the last time the listener was called or the status was read.
- **DDS\_Long current\_count**  
The current number of writers with which the *DDSDataReader* is matched.
- **DDS\_Long current\_count\_change**  
The change in *current\_count* since the last time the listener was called or the status was read.
- **DDS\_InstanceHandle\_t last\_publication\_handle**  
A handle to the last *DDSDataWriter* that caused the status to change.



### 7.78.1 Detailed Description

#### DDS\_SUBSCRIPTION\_MATCHED\_STATUS

A "match" happens when the [DDSDataReader](#) finds a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is compatible with that requested by the [DDSDataReader](#).

This status is also changed (and the listener, if any, called) when a match is ended. A local [DDSDataReader](#) will become "unmatched" from a remote [DDSDataWriter](#) when that [DDSDataWriter](#) goes away for any reason.

### 7.78.2 Member Data Documentation

#### total\_count

[DDS\\_Long](#) `DDS_SubscriptionMatchedStatus::total_count`

The total cumulative number of times the concerned [DDSDataReader](#) discovered a "match" with a [DDSDataWriter](#).

This number increases whenever a new match is discovered. It does not change when an existing match goes away.

#### total\_count\_change

[DDS\\_Long](#) `DDS_SubscriptionMatchedStatus::total_count_change`

The change in `total_count` since the last time the listener was called or the status was read.

#### current\_count

[DDS\\_Long](#) `DDS_SubscriptionMatchedStatus::current_count`

The current number of writers with which the [DDSDataReader](#) is matched.

This number increases when a new match is discovered and decreases when an existing match goes away.

#### current\_count\_change

[DDS\\_Long](#) `DDS_SubscriptionMatchedStatus::current_count_change`

The change in `current_count` since the last time the listener was called or the status was read.

#### last\_publication\_handle

[DDS\\_InstanceHandle\\_t](#) `DDS_SubscriptionMatchedStatus::last_publication_handle`

A handle to the last [DDSDataWriter](#) that caused the status to change.

## 7.79 DDS\_SystemResourceLimitsQosPolicy Struct Reference

Resource limits that apply only to [DDSDomainParticipantFactory](#)

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Long max\\_participants](#)  
*The maximum number of DomainParticipants ([DDSDomainParticipant](#)) the [DDSDomainParticipantFactory](#) can manage.*
- [DDS\\_Long max\\_components](#)  
*The maximum number of components that may be registered on the [DDSDomainParticipantFactory](#). Note that changing this value can cause a [DDSDomainParticipantFactory](#) to free and reallocate memory.*

### 7.79.1 Detailed Description

Resource limits that apply only to [DDSDomainParticipantFactory](#)

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDomainParticipantFactory](#)

Properties:

[Rxo](#) = N/A  
[Changeable](#) = NO

### 7.79.2 Member Data Documentation

#### max\_participants

[DDS\\_Long](#) [DDS\\_SystemResourceLimitsQosPolicy::max\\_participants](#)

The maximum number of DomainParticipants ([DDSDomainParticipant](#)) the [DDSDomainParticipantFactory](#) can manage.

The [DDSDomainParticipantFactory](#) can manage up to the number of DomainParticipants specified by this limit. However, the **actual** number of DomainParticipants that can be **created** with [DDSDomainParticipantFactory::create\\_participant](#) is further limited by the following constraints:

- Available memory and system resources such as network sockets and shared memory resources.

- The maximum number of `::max_timers` available. Each [DDSDomainParticipant](#) requires one timer plus one timer if Zero Copy v2 is enabled as a transport in [DDS\\_TransportQosPolicy::enabled\\_transports](#).
- The maximum number of `::max_user_blocking_threads`. Each receive-thread requires one blocking resource.
- DDS uses The [DDS\\_WireProtocolQosPolicy::rtps\\_well\\_known\\_ports](#) to discovery other participant. The number of DomainParticipants that can be created is limited by port aliasing where ports calculated for different DDS domain IDs may overlap. RTI Connex DDS Micro does not determine if port aliasing may occur and the application must ensure that port aliasing does not occur.

Note that this value can only be modified until the first [DDSDomainParticipant](#) is created and may cause memory reallocation.

**[default]** 1

**[range]** [1, INT\_MAX]

### **max\_components**

[DDS\\_Long](#) `DDS_SystemResourceLimitsQosPolicy::max_components`

The maximum number of components that may be registered on the [DDSDomainParticipantFactory](#). Note that changing this value can cause a [DDSDomainParticipantFactory](#) to free and reallocate memory.

**[default]** 16

## **7.80 DDS\_Time\_t Struct Reference**

Type for *time* representation.

```
#include <dds_c_infrastructure.h>
```

### **Public Attributes**

- [DDS\\_LongLong](#) `sec`  
*seconds*
- [DDS\\_UnsignedLong](#) `nanosec`  
*nanoseconds*

### **7.80.1 Detailed Description**

Type for *time* representation.

A [DDS\\_Time\\_t](#) represents a moment in time.

## 7.80.2 Member Data Documentation

### sec

`DDS_LongLong DDS_Time_t::sec`

seconds

### nanosec

`DDS_UnsignedLong DDS_Time_t::nanosec`

nanoseconds

## 7.81 DDS\_TopicDataQosPolicy Struct Reference

Attaches a buffer of opaque data to a [DDSTopic](#) which is distributed to other applications as part of the discovery process.

```
#include <dds_c_user_data_qos.h>
```

### Public Attributes

- struct DDS\_OctetSeq [value](#)  
*The opaque data to be attached to the [DDSEntity](#).*

### 7.81.1 Detailed Description

Attaches a buffer of opaque data to a [DDSTopic](#) which is distributed to other applications as part of the discovery process.

Entity:

[DDSTopic](#)

Properties:

[RxO](#) = NO  
[Changeable](#) = NO

### 7.81.2 Usage

The TOPIC\_DATA QoS policy allows an application to attach additional information to [DDSTopic](#) objects so that when a remote application discovers them, it can access that information and use it for its own purposes. This information is not interpreted by RTI Connex DDS Micro. Use cases are usually application-to-application identification, authentication, authorization, and encryption.

The topic data for a remote entity is received through the discovery process. The received topic data can be accessed through the functions for accessing the discovered entity data: [DDSDataWriter::get\\_matched\\_subscription\\_data](#) and [DDSDataReader::get\\_matched\\_publication\\_data](#).

*Important:* RTI Connex DDS Micro stores the data placed in this policy in pre-allocated pools. You must configure RTI Connex DDS Micro with the maximum size and number of the data sequences that will be stored in policies of this type. These settings are configured in the [DDS\\_DomainParticipantResourceLimitsQosPolicy](#).

### 7.81.3 Member Data Documentation

#### value

```
struct DDS_OctetSeq DDS_TopicDataQosPolicy::value
```

The opaque data to be attached to the [DDSEntity](#).

**[default]** empty (zero length)

## 7.82 DDS\_TopicQos Struct Reference

QoS policies supported by a [DDSTopic](#) entity.

```
#include <dds_c_topic.h>
```

### Public Attributes

- struct [DDS\\_TopicDataQosPolicy](#) [topic\\_data](#)  
*Topic data policy, TOPIC\_DATA.*

### 7.82.1 Detailed Description

QoS policies supported by a [DDSTopic](#) entity.

### 7.82.2 Member Data Documentation

#### topic\_data

```
struct DDS_TopicDataQosPolicy DDS_TopicQos::topic_data
```

Topic data policy, [TOPIC\\_DATA](#).

## 7.83 DDS\_TransportQosPolicy Struct Reference

<<[eXtension](#)>> Settings for available transports

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- struct DDS\_StringSeq [enabled\\_transports](#)  
*The transports available for use.*

#### 7.83.1 Detailed Description

<<[eXtension](#)>> Settings for available transports

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

#### 7.83.2 Member Data Documentation

##### **enabled\_transports**

```
struct DDS_StringSeq DDS_TransportQosPolicy::enabled_transports
```

The transports available for use.

A sequence of transport identifiers that defines the set of transports available for use by the associated DDS entity.

See also

[DDS\\_DomainParticipantQos](#)

**[default]** Empty sequence.

**[range]** Sequence of non-null,non-empty strings.

## 7.84 DDS\_TypePluginI Struct Reference

<<[interface](#)>> Type plugin interface.

```
#include <dds_c_type.h>
```

### 7.84.1 Detailed Description

<<[interface](#)>> Type plugin interface.

## 7.85 DDS\_UserDataQosPolicy Struct Reference

Attaches a buffer of opaque data to a [DDSEntity](#) which is distributed to other applications as part of the discovery process.

```
#include <dds_c_user_data_qos.h>
```

### Public Attributes

- struct DDS\_OctetSeq [value](#)  
*The opaque data to be attached to the [DDSEntity](#).*

### 7.85.1 Detailed Description

Attaches a buffer of opaque data to a [DDSEntity](#) which is distributed to other applications as part of the discovery process.

Entity:

[DDSDomainParticipant](#), [DDSDataWriter](#), [DDSDataReader](#)

Properties:

[RxO](#) = NO  
[Changeable](#) = NO

### 7.85.2 Usage

The USER\_DATA QoS policy allows an application to attach additional information to [DDSEntity](#) objects so that when a remote application discovers them, it can access that information and use it for its own purposes. This information is not interpreted by RTI Connex DDS Micro. Use cases are usually application-to-application identification, authentication, authorization, and encryption.

The user data for a remote entity is received through the discovery process. The received user data can be accessed through the functions for accessing the discovered entity data: [DDSDomainParticipant::get\\_discovered\\_participant\\_data](#), [DDSDataWriter::get\\_matched\\_subscription\\_data](#), and [DDSDataReader::get\\_matched\\_publication\\_data](#) operations.

*Important:* RTI Connex DDS Micro stores the data placed in this policy in pre-allocated pools. You must configure RTI Connex DDS Micro with the maximum size and number of the data sequences that will be stored in policies of this type. These settings are configured in the [DDS\\_DomainParticipantResourceLimitsQosPolicy](#).

### 7.85.3 Member Data Documentation

#### value

```
struct DDS_OctetSeq DDS_UserDataQosPolicy::value
```

The opaque data to be attached to the [DDSEntity](#).

**[default]** empty (zero length)

## 7.86 DDS\_UserTrafficQosPolicy Struct Reference

<<*eXtension*>> Settings for available transports

```
#include <dds_c_domain.h>
```

### Public Attributes

- struct DDS\_StringSeq [enabled\\_transports](#)  
*The transports available for use.*

### 7.86.1 Detailed Description

<<*eXtension*>> Settings for available transports

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A  
[Changeable](#) = NO

### 7.86.2 Member Data Documentation

#### enabled\_transports

```
struct DDS_StringSeq DDS_UserTrafficQosPolicy::enabled_transports
```

The transports available for use.

A sequence of transport identifiers that defines the set of transports available for use by the associated DDS entity.

See also

[DDS\\_DomainParticipantQos](#)

**[default]** Empty sequence.

**[range]** Sequence of non-null, non-empty strings.



## 7.87 DDS\_VendorId Struct Reference

<<*eXtension*>> Type used to represent the vendor of the service implementing the RTPS protocol.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Octet vendorId \[DDS\\_VENDOR\\_ID\\_LENGTH\\_MAX\]](#)  
*The vendor Id.*

#### 7.87.1 Detailed Description

<<*eXtension*>> Type used to represent the vendor of the service implementing the RTPS protocol.

#### 7.87.2 Member Data Documentation

##### vendorId

[DDS\\_Octet](#) DDS\_VendorId::vendorId[[DDS\\_VENDOR\\_ID\\_LENGTH\\_MAX](#)]

The vendor Id.

## 7.88 DDS\_VendorId\_t Struct Reference

<<*eXtension*>> Type used to represent the vendor of the service implementing the RTPS protocol.

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Octet vendorId \[DDS\\_VENDOR\\_ID\\_LENGTH\\_MAX\]](#)  
*The vendor Id.*

#### 7.88.1 Detailed Description

<<*eXtension*>> Type used to represent the vendor of the service implementing the RTPS protocol.

### 7.88.2 Member Data Documentation

#### vendorId

`DDS_Octet DDS_VendorId_t::vendorId[DDS_VENDOR_ID_LENGTH_MAX]`

The vendor Id.

## 7.89 DDS\_WireProtocolQosPolicy Struct Reference

Specifies the wire-protocol-related attributes for the [DDSDomainParticipant](#).

```
#include <dds_c_infrastructure.h>
```

### Public Attributes

- [DDS\\_Long participant\\_id](#)  
*A value used to distinguish between different participants belonging to the same domain on the same host.*
- [DDS\\_UnsignedLong rtps\\_host\\_id](#)  
*The RTPS Host ID of the domain participant.*
- [DDS\\_UnsignedLong rtps\\_app\\_id](#)  
*The RTPS App ID of the domain participant.*
- [DDS\\_UnsignedLong rtps\\_instance\\_id](#)  
*The RTPS Instance ID of the domain participant.*
- struct [DDS\\_RtpsWellKnownPorts\\_t rtps\\_well\\_known\\_ports](#)  
*Configures the RTPS well-known port mappings.*
- [DDS\\_Boolean compute\\_crc](#)  
*Adds RTPS checksum submessage to every message when this field is set to [DDS\\_BOOLEAN\\_TRUE](#).*
- [DDS\\_Boolean check\\_crc](#)  
*Checks if the received RTPS message is valid by comparing the computed checksum with the received RTPS checksum submessage when this field is set to [DDS\\_BOOLEAN\\_TRUE](#).*
- [DDS\\_Boolean require\\_crc](#)  
*Require that discovered participants send one of the checksums enabled in [DDS\\_WireProtocolQosPolicy::allowed\\_crc\\_mask](#).*
- [DDS\\_ChecksumKind\\_t computed\\_crc\\_kind](#)  
*A bitmask indicating which checksum is used for sent messages when `compute_crc` is [DDS\\_BOOLEAN\\_TRUE](#). Only one mask is supported.*
- [DDS\\_ChecksumKindMask\\_t allowed\\_crc\\_mask](#)  
*The checksums which are enabled for validating incoming packets.*

### 7.89.1 Detailed Description

Specifies the wire-protocol-related attributes for the [DDSDomainParticipant](#).

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

### 7.89.2 Usage

The default Qos policies returned by RTI Connex DDS Micro contain the correctly initialized wire protocol attributes.

The defaults are not expected to be modified normally, but are available to the advanced user customizing the implementation behavior.

The default values should not be modified without an understanding of the underlying Real-Time Publish Subscribe (RTPS) wire protocol.

### 7.89.3 Member Data Documentation

#### **participant\_id**

`DDS_Long DDS_WireProtocolQosPolicy::participant_id`

A value used to distinguish between different participants belonging to the same domain on the same host.

Determines the unicast port on which meta-traffic is received. Also defines the *default* unicast port for receiving user-traffic for DataReaders and DataWriters .

For more information on port mapping, please refer to [DDS\\_RtpsWellKnownPorts\\_t](#).

Each [DDSDomainParticipant](#) in the same domain, running on the same host, must have a unique `participant_id`. The participants may be in the same address space or in distinct address spaces.

A negative number (-1) means that RTI Connex DDS Micro will *automatically* resolve the participant ID as follows.

- RTI Connex DDS Micro will pick the *smallest* participant ID, based on the unicast ports available on the transports enabled for discovery.
- RTI Connex DDS Micro will attempt to resolve an automatic port index either when a DomainParticipant is enabled, or when a DataReader or a DataWriter is created. Therefore, all the transports enabled for discovery must have been registered by this time. Otherwise, the discovery transports registered after resolving the automatic port index may produce port conflicts when the DomainParticipant is enabled.

**[default]** -1 [automatic], i.e. RTI Connex DDS Micro will automatically pick the `participant_id`, as described above.

**[range]** [ $\geq 0$ ], or -1, and does not violate guidelines stated in [DDS\\_RtpsWellKnownPorts\\_t](#).

See also

[DDSEntity::enable\(\)](#)

**rtps\_host\_id**

```
DDS_UnsignedLong DDS_WireProtocolQosPolicy::rtps_host_id
```

The RTPS Host ID of the domain participant.

Either all of `DDS_WireProtocolQosPolicy::rtps_host_id`, `DDS_WireProtocolQosPolicy::rtps_app_id`, and `DDS_WireProtocolQosPolicy::rtps_instance_id` for a DomainParticipant are set to `DDS_RTPS_AUTO_ID`, or none of them are set to `DDS_RTPS_AUTO_ID`.

**[default]** `DDS_RTPS_AUTO_ID`.

**[range]** `[0,0xffffffff]`

**rtps\_app\_id**

```
DDS_UnsignedLong DDS_WireProtocolQosPolicy::rtps_app_id
```

The RTPS App ID of the domain participant.

Either all of `DDS_WireProtocolQosPolicy::rtps_host_id`, `DDS_WireProtocolQosPolicy::rtps_app_id`, and `DDS_WireProtocolQosPolicy::rtps_instance_id` for a DomainParticipant are set to `DDS_RTPS_AUTO_ID`, or none of them are set to `DDS_RTPS_AUTO_ID`.

**[default]** `DDS_RTPS_AUTO_ID`.

**[range]** `[0,0xffffffff]`

**rtps\_instance\_id**

```
DDS_UnsignedLong DDS_WireProtocolQosPolicy::rtps_instance_id
```

The RTPS Instance ID of the domain participant.

Either all of `DDS_WireProtocolQosPolicy::rtps_host_id`, `DDS_WireProtocolQosPolicy::rtps_app_id`, and `DDS_WireProtocolQosPolicy::rtps_instance_id` for a DomainParticipant are set to `DDS_RTPS_AUTO_ID`, or none of them are set to `DDS_RTPS_AUTO_ID`.

**[default]** `DDS_RTPS_AUTO_ID`.

**[range]** `[0,0xffffffff]`

**rtps\_well\_known\_ports**

```
struct DDS_RtpsWellKnownPorts_t DDS_WireProtocolQosPolicy::rtps_well_known_ports
```

Configures the RTPS well-known port mappings.

Determines the well-known multicast and unicast port mappings for discovery (meta) traffic and user traffic.

**[default]** `DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS`

**compute\_crc**

`DDS_Boolean DDS_WireProtocolQosPolicy::compute_crc`

Adds RTPS checksum submessage to every message when this field is set to `DDS_BOOLEAN_TRUE`.

The computed checksum covers the entire RTPS message including the RTPS header itself.

**[default]** `DDS_BOOLEAN_FALSE`

**check\_crc**

`DDS_Boolean DDS_WireProtocolQosPolicy::check_crc`

Checks if the received RTPS message is valid by comparing the computed checksum with the received RTPS checksum submessage when this field is set to `DDS_BOOLEAN_TRUE`.

`DDS_WireProtocolQosPolicy::compute_crc` must be enabled at the publishing application for validating the message at the subscribing application.

If a message is received with an invalid checksum, the message is discarded. If no checksum is received or found, the message is accepted but may still be corrupted.

**[default]** `DDS_BOOLEAN_FALSE`

**require\_crc**

`DDS_Boolean DDS_WireProtocolQosPolicy::require_crc`

Require that discovered participants send one of the checksums enabled in `DDS_WireProtocolQosPolicy::allowed_crc_mask`.

If an invalid checksum is received, the message is dropped.

if the value is `DDS_BOOLEAN_TRUE` the participant requires that discovered participants send one of the checksums enabled in `DDS_WireProtocolQosPolicy::allowed_crc_mask`. If this is not the case the participant is ignored and not matching will take place.

This is a stronger check than `idref_WireProtocolQosPolicy_check_crc`. If an invalid checksum is received or not found, the message is dropped.

**[default]** `DDS_BOOLEAN_FALSE`

**computed\_crc\_kind**

DDS\_ChecksumKind\_t DDS\_WireProtocolQosPolicy::computed\_crc\_kind

A bitmask indicating which checksum is used for sent messages when compute\_crc is DDS\_BOOLEAN\_TRUE. Only one mask is supported.

- DDS\_CHECKSUM\_AUTO
- DDS\_CHECKSUM\_BUILTIN32
- DDS\_CHECKSUM\_BUILTIN64
- DDS\_CHECKSUM\_BUILTIN128

[default] DDS\_CHECKSUM\_AUTO.

**allowed\_crc\_mask**

DDS\_ChecksumKindMask\_t DDS\_WireProtocolQosPolicy::allowed\_crc\_mask

The checksums which are enabled for validating incoming packets.

The following bitmask can be OR'ed for all enabled checksums:

- DDS\_CHECKSUM\_AUTO
- DDS\_CHECKSUM\_BUILTIN32
- DDS\_CHECKSUM\_BUILTIN64
- DDS\_CHECKSUM\_BUILTIN128

Note that the selected checksums must be a subset of what is supported. If a bitmask that is not supported is selected participant creation will fail.

[default] DDS\_CHECKSUM\_AUTO.

**7.90 DDS\_WriteParams\_t Struct Reference**

Data structure used to provide custom parameters to a DataWriter's write operation.

```
#include <dds_c_infrastructure.h>
```

**Public Attributes**

- struct DDS\_Time\_t source\_timestamp  
*Source time-stamp of the published sample.*
- DDS\_InstanceHandle\_t handle  
*DDS\_InstanceHandle\_t value identifying the instance the sample is being written to.*

### 7.90.1 Detailed Description

Data structure used to provide custom parameters to a DataWriter's write operation.

### 7.90.2 Member Data Documentation

#### source\_timestamp

```
struct DDS_Time_t DDS_WriteParams_t::source_timestamp
```

Source time-stamp of the published sample.

#### handle

```
DDS_InstanceHandle_t DDS_WriteParams_t::handle
```

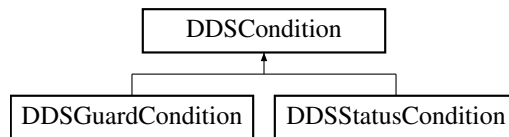
[DDS\\_InstanceHandle\\_t](#) value identifying the instance the sample is being written to.

## 7.91 DDSCondition Class Reference

**<<interface>>** Root class for all the conditions that may be attached to a [DDSWaitSet](#).

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSCondition:



#### Friends

- class [DDSWaitSet](#)

### 7.91.1 Detailed Description

**<<interface>>** Root class for all the conditions that may be attached to a [DDSWaitSet](#).

This basic class is specialised in two classes:

[DDSGuardCondition](#) and [DDSStatusCondition](#).

A [DDSCondition](#) has a `trigger_value` that can be [DDS\\_BOOLEAN\\_TRUE](#) or [DDS\\_BOOLEAN\\_FALSE](#) and is set automatically by RTI Connex Micro DDS.

See also

[DDSWaitSet](#)

## 7.92 DDSEntitySeq Class Reference

A sequence object containing references to one or more [DDSEntity](#) objects.

```
#include <dds_cpp_infrastructure.hxx>
```

### Friends

- class [DDSEntitySeq](#)

### 7.92.1 Detailed Description

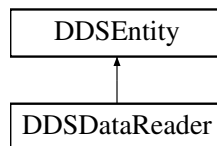
A sequence object containing references to one or more [DDSEntity](#) objects.

## 7.93 DDSDataReader Class Reference

[DDSEntitySeq](#) Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached [DDSEntitySeq](#).

```
#include <dds_cpp_subscription.hxx>
```

Inheritance diagram for DDSDataReader:



### Public Member Functions

- virtual [DDSEntity](#) \* [as\\_entity](#) ()=0  
*Accesses the [DDSDataReader](#)'s supertype instance.*
- virtual [DDSTopicDescription](#) \* [get\\_topicdescription](#) ()=0  
*Returns the [DDSTopicDescription](#) associated with the [DDSDataReader](#).*
- virtual [DDSEntitySeq](#) \* [get\\_subscriber](#) ()=0  
*Returns the [DDSEntitySeq](#) to which the [DDSDataReader](#) belongs.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_qos](#) (const [DDS\\_DataReaderQos](#) &qos)=0  
*Sets the reader QoS.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_qos](#) ([DDS\\_DataReaderQos](#) &qos)=0  
*Gets the reader QoS.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_listener](#) (const [DDSDataReaderListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
*Sets the reader listener.*
- virtual [DDSDataReaderListener](#) \* [get\\_listener](#) ()=0  
*Get the reader listener.*



- virtual `DDS_ReturnCode_t read_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`, `DDS_Long max_samples`, `DDS_SampleStateMask sample_states`, `DDS_ViewStateMask view_states`, `DDS_InstanceStateMask instance_states`)=0  
*Access a collection of data samples from the `DDSDDataReader`.*
- virtual `DDS_ReturnCode_t take_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`, `DDS_Long max_samples`, `DDS_SampleStateMask sample_states`, `DDS_ViewStateMask view_states`, `DDS_InstanceStateMask instance_states`)=0  
*Access a collection of data-samples from the `DDSDDataReader`.*
- virtual `DDS_ReturnCode_t read_instance_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`, `DDS_Long max_samples`, `const DDS_InstanceHandle_t &a_handle`, `DDS_SampleStateMask sample_states`, `DDS_ViewStateMask view_states`, `DDS_InstanceStateMask instance_states`)=0  
*Access a collection of data samples from the `DDSDDataReader`, all of which belong to the same instance .*
- virtual `DDS_ReturnCode_t take_instance_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`, `DDS_Long max_samples`, `const DDS_InstanceHandle_t &a_handle`, `DDS_SampleStateMask sample_states`, `DDS_ViewStateMask view_states`, `DDS_InstanceStateMask instance_states`)=0  
*Access a collection of data samples from the `DDSDDataReader`, all of which belong to the same instance .*
- virtual `DDS_ReturnCode_t read_next_sample_untyped` (`void *received_data`, `DDS_SampleInfo &sample_info`)=0  
*Copies the next not-previously-accessed data value from the `DDSDDataReader`.*
- virtual `DDS_ReturnCode_t take_next_sample_untyped` (`void *received_data`, `DDS_SampleInfo &sample_info`)=0  
*Copies the next not-previously-accessed data value from the `DDSDDataReader`.*
- virtual `DDS_InstanceHandle_t lookup_instance_untyped` (`const void *key_holder`)=0  
*Retrieve the `DDS_InstanceHandle_t` identifying a specific instance.*
- virtual `DDS_ReturnCode_t is_data_consistent_untyped` (`DDS_Boolean &is_data_consistent`, `const void *sample`, `const DDS_SampleInfo &sample_info`)=0  
*Checks if the sample has been overwritten by the `DataWriter`.*
- virtual `DDS_ReturnCode_t return_loan_untyped` (`DDS_UntypedSampleSeq *received_data`, `DDS_SampleInfoSeq &info_seq`)=0  
*Indicates to the `DDSDDataReader` that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the `DDSDDataReader`.*
- virtual `DDS_ReturnCode_t get_subscription_matched_status` (`DDS_SubscriptionMatchedStatus &status`)=0  
*Accesses the `DDS_SUBSCRIPTION_MATCHED_STATUS` communication status.*
- virtual `DDS_ReturnCode_t get_liveliness_changed_status` (`DDS_LivelinessChangedStatus &status`)=0  
*Accesses the `DDS_LIVELINESS_CHANGED_STATUS` communication status.*
- virtual `DDS_ReturnCode_t get_matched_publications` (`DDS_InstanceHandleSeq &publication_handles`)=0  
*Returns a sequence containing `DDS_InstanceHandle_t` for all the remote `DDSDDataWriter` that have been matched by the `DDSDDataReader`.*
- virtual `DDS_ReturnCode_t get_matched_publication_data` (`DDS_PublicationBuiltinTopicData &publication_data`, `const DDS_InstanceHandle_t &publication_handle`)=0  
*Returns information about a remote `DDSDDataWriter` that has been matched by the `DDSDDataReader`.*
- virtual `DDS_ReturnCode_t get_sample_rejected_status` (`DDS_SampleRejectedStatus &status`)=0  
*Accesses the `DDS_SAMPLE_REJECTED_STATUS` communication status.*
- virtual `DDS_ReturnCode_t get_sample_lost_status` (`DDS_SampleLostStatus &status`)=0  
*Accesses the `DDS_SAMPLE_LOST_STATUS` communication status.*
- virtual `DDS_ReturnCode_t get_requested_deadline_missed_status` (`DDS_RequestedDeadlineMissedStatus &status`)=0  
*Accesses the `DDS_REQUESTED_DEADLINE_MISSED_STATUS` communication status.*
- virtual `DDS_ReturnCode_t get_requested_incompatible_qos_status` (`DDS_RequestedIncompatibleQosStatus &status`)=0  
*Accesses the `DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS` communication status.*

- virtual `DDS_ReturnCode_t get_instance_replaced_missed_status (DDS_DataReaderInstanceReplacedStatus &status)=0`  
*Accesses the `DDS_INSTANCE_REPLACED_STATUS` communication status.*

### Public Member Functions inherited from `DDSEntity`

- `DDS_ReturnCode_t enable ()`  
*Enables the `DDSEntity`.*
- `DDSStatusCondition * get_statuscondition ()`  
*Return the `DDSStatusCondition` associated with a particular `DDSEntity`.*
- `DDS_StatusMask get_status_changes ()`  
*Retrieves the list of communication statuses in the `DDSEntity` that are triggered.*
- `DDS_InstanceHandle_t get_instance_handle ()`  
*Allows access to the `DDS_InstanceHandle_t` associated with the `DDSEntity`.*

### 7.93.1 Detailed Description

<<*interface*>> Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached `DDSSubscriber`.

QoS:

`DDS_DataReaderQos`

Status:

`DDS_DATA_AVAILABLE_STATUS;`  
`DDS_LIVELINESS_CHANGED_STATUS, DDS_LivelinessChangedStatus;`  
`DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS, DDS_RequestedIncompatibleQosStatus;`  
`DDS_SUBSCRIPTION_MATCHED_STATUS, DDS_SubscriptionMatchedStatus;`  
`DDS_INSTANCE_REPLACED_STATUS, DDS_DataReaderInstanceReplacedStatus;`

Listener:

`DDSDDataReaderListener`

A `DDSDDataReader` refers to exactly one `DDSTopicDescription` (a `DDSTopic`) that identifies the data to be read.

The subscription has a unique resulting type. The data-reader may give access to several instances of the resulting type, which can be distinguished from each other by their `key`.

The following operations may be called even if the `DDSDDataReader` is not enabled. Other operations will fail with the value `DDS_RETCODE_NOT_ENABLED` if called on a disabled `DDSDDataReader`:

- The base-class operations `DDSDDataReader::set_qos`, `DDSDDataReader::get_qos`, `DDSDDataReader::set_listener`, `DDSDDataReader::get_listener`, and `DDSEntity::enable`

### 7.93.2 Member Function Documentation

#### **as\_entity()**

```
virtual DDSEntity * DDSDataReader::as_entity () [pure virtual]
```

Accesses the [DDSDataReader](#)'s supertype instance.

#### **get\_topicdescription()**

```
virtual DDSTopicDescription * DDSDataReader::get_topicdescription () [pure virtual]
```

Returns the [DDSTopicDescription](#) associated with the [DDSDataReader](#).

Returns that same [DDSTopicDescription](#) that was used to create the [DDSDataReader](#).

#### Returns

[DDSTopicDescription](#) associated with the [DDSDataReader](#).

#### **get\_subscriber()**

```
virtual DDSSubscriber * DDSDataReader::get_subscriber () [pure virtual]
```

Returns the [DDSSubscriber](#) to which the [DDSDataReader](#) belongs.

#### Returns

[DDSSubscriber](#) to which the [DDSDataReader](#) belongs.

#### **set\_qos()**

```
virtual DDS\_ReturnCode\_t DDSDataReader::set_qos (  
    const DDS\_DataReaderQos & qos) [pure virtual]
```

Sets the reader QoS.

This operation modifies the QoS of the [DDSDataReader](#).

The [DDS\\_DataReaderQos::deadline](#)

can be changed. The other policies are immutable.

## Parameters

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> The <a href="#">DDS_DataReaderQos</a> to be set to. Policies must be consistent. Policies cannot be changed after <a href="#">DDSDataReader</a> is enabled. The special value <a href="#">DDS_DATAREADER_QOS_DEFAULT</a> can be used to indicate that the QoS of the <a href="#">DDSDataReader</a> should be changed to match the current default <a href="#">DDS_DataReaderQos</a> set in the <a href="#">DDSSubscriber</a> . |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#).

## See also

[DDS\\_DataReaderQos](#) for rules on consistency among QoS  
[set\\_qos](#) (abstract)  
[DDSDataReader::set\\_qos](#)

**get\_qos()**

```
virtual DDS_ReturnCode_t DDSDataReader::get_qos (
    DDS_DataReaderQos & qos) [pure virtual]
```

Gets the reader QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

## Parameters

|            |                                                                           |
|------------|---------------------------------------------------------------------------|
| <i>qos</i> | << <i>inout</i> >> The <a href="#">DDS_DataReaderQos</a> to be filled up. |
|------------|---------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#)

## See also

[get\\_qos](#) (abstract)

**set\_listener()**

```
virtual DDS_ReturnCode_t DDSDataReader::set_listener (
    const DDSDataReaderListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Sets the reader listener.

## Parameters

|                 |                                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------|
| <i>listener</i> | << <i>in</i> >> <a href="#">DDSDataReaderListener</a> to set to                                            |
| <i>mask</i>     | << <i>in</i> >> <a href="#">DDS_StatusMask</a> associated with the <a href="#">DDSDataReaderListener</a> . |

## Returns

One of the [Standard Return Codes](#)

## See also

[set\\_listener](#) (abstract)

**get\_listener()**

```
virtual DDSDataReaderListener * DDSDataReader::get_listener () [pure virtual]
```

Get the reader listener.

## Returns

[DDSDataReaderListener](#) of the [DDSDataReader](#).

**read\_untyped()**

```
virtual DDS\_ReturnCode\_t DDSDataReader::read_untyped (
    DDSUntypedSampleSeq * received_data,
    DDS\_SampleInfoSeq & info_seq,
    DDS\_Long max_samples,
    DDS\_SampleStateMask sample_states,
    DDS\_ViewStateMask view_states,
    DDS\_InstanceStateMask instance_states) [pure virtual]
```

Access a collection of data samples from the [DDSDataReader](#).

This operation offers the same functionality and API as [DDSDataReader::take\\_untyped](#) except that the samples returned remain in the [DDSDataReader](#) such that they can be retrieved again by means of a read or take operation.

Please refer to the documentation of [DDSDataReader::take\\_untyped\(\)](#) for details on the number of samples returned within the `received_data` and `info_seq` as well as the order in which the samples appear in these sequences.

The act of reading a sample changes its `sample_state` to [DDS\\_READ\\_SAMPLE\\_STATE](#). If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to be [DDS\\_NOT\\_NEW\\_VIEW\\_STATE](#). It will not affect the `instance_state` of the instance.

*Important:* If the samples "returned" by this method are loaned from RTI Connex Micro DDS (see [DDSDataReader::take\\_untyped](#) for more information on memory loaning), it is important that their contents not be changed. Because the memory in which the data is stored belongs to the middleware, any modifications made to the data will be seen the next time the same samples are read or taken; the samples will no longer reflect the state that was received from the network.

## Parameters

|                        |                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i>   | << <i>inout</i> >> User data untyped pointer where the received data samples will be returned. Must be a valid non-NULL pointer. The method will fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> if it is NULL. |
| <i>info_seq</i>        | << <i>inout</i> >> A <a href="#">DDS_SampleInfoSeq</a> object where the received sample info will be returned.                                                                                                      |
| <i>max_samples</i>     | << <i>in</i> >> The maximum number of samples to be returned. If the special value <a href="#">DDS_LENGTH_UNLIMITED</a> is provided, as many samples will be returned as are available.                             |
| <i>sample_states</i>   | << <i>in</i> >> Data samples matching one of these <i>sample_states</i> are returned.                                                                                                                               |
| <i>view_states</i>     | << <i>in</i> >> Data samples matching one of these <i>view_state</i> are returned.                                                                                                                                  |
| <i>instance_states</i> | << <i>in</i> >> Data samples matching ones of these <i>instance_state</i> are returned.                                                                                                                             |

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

## See also

[DDSDataReader::take\\_untyped](#),  
[DDS\\_LENGTH\\_UNLIMITED](#)

**take\_untyped()**

```
virtual DDS\_ReturnCode\_t DDSDataReader::take_untyped (
    DDS\_UntypedSampleSeq * received_data,
    DDS\_SampleInfoSeq & info_seq,
    DDS\_Long max_samples,
    DDS\_SampleStateMask sample_states,
    DDS\_ViewStateMask view_states,
    DDS\_InstanceStateMask instance_states) [pure virtual]
```

Access a collection of data-samples from the [DDSDataReader](#).

The operation will return the list of samples received by the [DDSDataReader](#) since the last [DDSDataReader::take\\_untyped](#) operation that match the specified [DDS\\_SampleStateMask](#), [DDS\\_ViewStateMask](#) and [DDS\\_InstanceStateMask](#).

This operation may fail with [DDS\\_RETCODE\\_ERROR](#) if [DDS\\_DataReaderResourceLimitsQosPolicy::max\\_outstanding\\_reads](#) limit has been exceeded.

The actual number of samples returned depends on the information that has been received by the middleware as well as the [DDS\\_HistoryQosPolicy](#), [DDS\\_ResourceLimitsQosPolicy](#), [DDS\\_DataReaderResourceLimitsQosPolicy](#) and the characteristics of the data-type that is associated with the [DDSDataReader](#):

- In the case where the [DDS\\_HistoryQosPolicy::kind](#) is [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#), the call will return at most [DDS\\_HistoryQosPolicy::depth](#) samples per instance.

- The maximum number of samples returned is limited by `DDS_ResourceLimitsQosPolicy::max_samples`.
- For multiple instances, the number of samples returned is additionally limited by the product (`DDS_ResourceLimitsQosPolicy::max_s`  
\* `DDS_ResourceLimitsQosPolicy::max_instances`)

If the read or take succeeds and the number of samples returned has been limited (by means of a maximum limit, as listed above, or insufficient `DDS_SampleInfo` resources), the call will complete successfully and provide those samples the reader is able to return. The user may need to make additional calls, or return outstanding loaned buffers in the case of insufficient resources, in order to access remaining samples.

Note that in the case where the `DDSTopic` associated with the `DDSDataReader` is bound to a data-type that has no key definition, then there will be at most one instance in the `DDSDataReader`. So the per-sample limits will apply.

The act of *taking* a sample removes it from RTI Connex DDS Micro so it cannot be read or taken again. If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the sample's instance to `DDS_NOT_NEW_VIEW_STATE`. It will not affect the `instance_state` of the sample's instance.

After `DDSDataReader::take_untyped` completes, `received_data` and `info_seq` will be of the same length and contain the received data.

If the sequences are empty (maximum size equal 0) when the `DDSDataReader::take_untyped` is called, the samples returned in the `received_data` and the corresponding `info_seq` are 'loaned' to the application from buffers provided by the `DDSDataReader`. The application can use them as desired and has guaranteed exclusive access to them.

Once the application completes its use of the samples it must 'return the loan' to the `DDSDataReader` by calling the `DDSDataReader::return_loan_untyped` operation.

*Note:* While you must call `DDSDataReader::return_loan_untyped` at some point, you do *not* have to do so before the next `DDSDataReader::take_untyped` call. However, failure to return the loan will eventually deplete the `DDSDataReader` of the buffers it needs to receive new samples and eventually samples will start to be lost. The total number of buffers available to the `DDSDataReader` is specified by the `DDS_ResourceLimitsQosPolicy` and the `DDS_DataReaderResourceLimitsQosPolicy`.

If the sequences are not empty (maximum size not equal 0) when the `DDSDataReader::take_untyped` is called, samples are copied to `received_data` and `info_seq`. The application will not need to call `DDSDataReader::return_loan`.

If the `DDSDataReader` has no samples that meet the constraints, the method will fail with `DDS_RETCODE_NO_DATA`.

In addition to the collection of samples, the read and take operations also use a collection of `DDS_SampleInfo` structures.

### 7.93.3 SEQUENCES USAGE IN TAKE AND READ

The initial (input) properties of the `received_data` and `info_seq` collections will determine the precise behavior of the read or take operation. For the purposes of this description, the collections are modeled as having these properties:

- whether the collection container owns the memory of the elements within (`owns`, see `::FooSeq::has_ownership`)
- the current-length (`len`, see `::FooSeq::length()`)
- the maximum length (`max_len`, see `::FooSeq::maximum()`)

The initial values of the `owns`, `len` and `max_len` properties for the `received_data` and `info_seq` collections govern the behavior of the read and take operations as specified by the following rules:

1. The values of `owns`, `len` and `max_len` properties for the two collections must be identical. Otherwise read/take will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`.
2. On successful output, the values of `owns`, `len` and `max_len` will be the same for both collections.
3. If the initial `max_len==0`, then the `received_data` and `info_seq` collections will be filled with elements that are loaned by the `DDSDataReader`. On output, `owns` will be `FALSE`, `len` will be set to the number of values returned, and `max_len` will be set to a value verifying `max_len >= len`. The use of this variant allows for zero-copy access to the data and the application will need to return the loan to the `DDSDataReader` using `FooDataReader::return_loan`.
4. If the initial `max_len>0` and `owns==FALSE`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential hard-to-detect memory leaks caused by an application forgetting to return the loan.
5. If initial `max_len>0` and `owns==TRUE`, then the read or take operation will copy the `received_data` values and `DDS_SampleInfo` values into the elements already inside the collections. On output, `owns` will be `TRUE`, `len` will be set to the number of values copied and `max_len` will remain unchanged. The use of this variant forces a copy but the application can control where the copy is placed and the application will not need to return the loan. The number of samples copied depends on the relative values of `max_len` and `max_samples`:
  - If `max_samples == LENGTH_UNLIMITED`, then at most `max_len` values will be copied. The use of this variant lets the application limit the number of samples returned to what the sequence can accommodate.
  - If `max_samples <= max_len`, then at most `max_samples` values will be copied. The use of this variant lets the application limit the number of samples returned to fewer than what the sequence can accommodate.
  - If `max_samples > max_len`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential confusion where the application expects to be able to access up to `max_samples`, but that number can never be returned, even if they are available in the `DDSDataReader`, because the output sequence cannot accommodate them.

As described above, upon completion, the `received_data` and `info_seq` collections may contain elements loaned from the `DDSDataReader`. If this is the case, the application will need to use `DDSDataReader::return_loan_untyped` to return the loan once it is no longer using the `received_data` in the collection. When `DDSDataReader::return_loan_untyped` completes, the collection will have `owns=FALSE` and `max_len=0`. The application can determine whether it is necessary to return the loan or not based on how the state of the collections when the read/take operation was called or by accessing the `owns` property. However, in many cases it may be simpler to always call `DDSDataReader::return_loan_untyped`, as this operation is harmless (i.e., it leaves all elements unchanged) if the collection does not have a loan.

To avoid potential memory leaks, the implementation of the `Foo` and `DDS_SampleInfo` collections should disallow changing the length of a collection for which `owns==FALSE`. Furthermore, deleting a collection for which `owns==FALSE` should be considered an error.

On output, the collection of `Foo` values and the collection of `DDS_SampleInfo` structures are of the same length and are in a one-to-one correspondence. Each `DDS_SampleInfo` provides information, such as the `source_timestamp`, the `sample_state`, `view_state`, and `instance_state`, etc., about the corresponding sample.

Some elements in the returned collection may not have valid data. If the `instance_state` in the `DDS_SampleInfo` is `DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE` or `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE`, then the last sample for that instance in the collection (that is, the one whose `DDS_SampleInfo` has `sample_rank==0`) does not contain valid data.



Samples that contain no data do not count towards the limits imposed by the [DDS\\_ResourceLimitsQosPolicy](#). The act of reading/taking a sample sets its `sample_state` to [DDS\\_READ\\_SAMPLE\\_STATE](#).

If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to [DDS\\_NOT\\_NEW\\_VIEW\\_STATE](#). It will not affect the `instance_state` of the instance.

This operation must be provided on the specialized class that is generated for the particular application data-type that is being read . If the [DDSDataReader](#) has no samples that meet the constraints, the operations fails with [DDS\\_RETCODE\\_NO\\_DATA](#).

#### Parameters

|                      |                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------|
| <i>received_data</i> | << <i>inout</i> >> User data untyped pointer where the received data samples will be returned. |
|----------------------|------------------------------------------------------------------------------------------------|

#### Parameters

|                        |                                                                                                                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>info_seq</i>        | << <i>inout</i> >> A <a href="#">DDS_SampleInfoSeq</a> object where the received sample info will be returned.                                                                          |
| <i>max_samples</i>     | << <i>in</i> >> The maximum number of samples to be returned. If the special value <a href="#">DDS_LENGTH_UNLIMITED</a> is provided, as many samples will be returned as are available. |
| <i>sample_states</i>   | << <i>in</i> >> Data samples matching one of these <code>sample_states</code> are returned.                                                                                             |
| <i>view_states</i>     | << <i>in</i> >> Data samples matching one of these <code>view_state</code> are returned.                                                                                                |
| <i>instance_states</i> | << <i>in</i> >> Data samples matching one of these <code>instance_state</code> are returned.                                                                                            |

#### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

#### See also

[DDSDataReader::read\\_untyped](#)

#### **read\_instance\_untyped()**

```
virtual DDS_ReturnCode_t DDSDataReader::read_instance_untyped (
    DDS_UntypedSampleSeq * received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    const DDS_InstanceHandle_t & a_handle,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states) [pure virtual]
```

Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance .

## Parameters

|                        |                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i>   | << <i>inout</i> >> User data untyped pointer where the received data samples will be returned. Must be a valid non-NULL pointer. The method will fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> if it is NULL. |
| <i>info_seq</i>        | << <i>inout</i> >> A <a href="#">DDS_SampleInfoSeq</a> object where the received sample info will be returned.                                                                                                      |
| <i>max_samples</i>     | << <i>in</i> >> The maximum number of samples to be returned. If the special value <a href="#">DDS_LENGTH_UNLIMITED</a> is provided, as many samples will be returned as are available.                             |
| <i>a_handle</i>        | << <i>in</i> >> Instance for which data samples should be read                                                                                                                                                      |
| <i>sample_states</i>   | << <i>in</i> >> Data samples matching one of these <i>sample_states</i> are returned.                                                                                                                               |
| <i>view_states</i>     | << <i>in</i> >> Data samples matching one of these <i>view_state</i> are returned.                                                                                                                                  |
| <i>instance_states</i> | << <i>in</i> >> Data samples matching ones of these <i>instance_state</i> are returned.                                                                                                                             |

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

## See also

[DDSDataReader::read\\_untyped](#),  
[DDS\\_LENGTH\\_UNLIMITED](#)

**take\_instance\_untyped()**

```
virtual DDS_ReturnCode_t DDSDataReader::take_instance_untyped (
    DDS_UntypedSampleSeq * received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    const DDS_InstanceHandle_t & a_handle,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states) [pure virtual]
```

Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance .

## Parameters

|                        |                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i>   | << <i>inout</i> >> User data untyped pointer where the received data samples will be returned. Must be a valid non-NULL pointer. The method will fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> if it is NULL. |
| <i>info_seq</i>        | << <i>inout</i> >> A <a href="#">DDS_SampleInfoSeq</a> object where the received sample info will be returned.                                                                                                      |
| <i>max_samples</i>     | << <i>in</i> >> The maximum number of samples to be returned. If the special value <a href="#">DDS_LENGTH_UNLIMITED</a> is provided, as many samples will be returned as are available.                             |
| <i>a_handle</i>        | << <i>in</i> >> Instance for which data samples should be read                                                                                                                                                      |
| <i>sample_states</i>   | << <i>in</i> >> Data samples matching one of these <i>sample_states</i> are returned.                                                                                                                               |
| <i>view_states</i>     | << <i>in</i> >> Data samples matching one of these <i>view_state</i> are returned.                                                                                                                                  |
| <i>instance_states</i> | << <i>in</i> >> Data samples matching ones of these <i>instance_state</i> are returned.                                                                                                                             |

**Returns**

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

**See also**

[DDSDataReader::take\\_untyped](#),  
[DDS\\_LENGTH\\_UNLIMITED](#)

**read\_next\_sample\_untyped()**

```
virtual DDS_ReturnCode_t DDSDataReader::read_next_sample_untyped (
    void * received_data,
    DDS_SampleInfo & sample_info) [pure virtual]
```

Copies the next not-previously-accessed data value from the [DDSDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDataReader](#). This operation also copies the corresponding [DDS\\_SampleInfo](#). The implied order among the samples stored in the [DDSDataReader](#) is the same as for the [DDSDataReader::read\\_untyped](#) operation.

The [DDSDataReader::read\\_next\\_sample\\_untyped](#) operation is semantically equivalent to the [DDSDataReader::read\\_untyped](#) operation, where the input data sequences has max\_len=1, the sample\_states=NOT\_READ, the view\_states=ANY\_↔VIEW\_STATE, and the instance\_states=ANY\_INSTANCE\_STATE.

The [DDSDataReader::read\\_next\\_sample\\_untyped](#) operation provides a simplified API to 'read' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDataReader](#), the operation will fail with [DDS\\_RETCODE\\_NO\\_DATA](#) and nothing is copied.

**Parameters**

|                      |                                                                                                                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i> | <<inout>> user data type-specific data object where the next received data sample will be returned. The received_data must have been fully allocated. Otherwise, this operation may fail. |
| <i>sample_info</i>   | <<inout>> a <a href="#">DDS_SampleInfo</a> object where the next received sample info will be returned.                                                                                   |

**Returns**

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

**See also**

[DDSDataReader::read\\_untyped](#)

**take\_next\_sample\_untyped()**

```
virtual DDS_ReturnCode_t DDSDataReader::take_next_sample_untyped (
    void * received_data,
    DDS_SampleInfo & sample_info) [pure virtual]
```

Copies the next not-previously-accessed data value from the [DDSDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDataReader](#) and 'removes' it from the [DDSDataReader](#) so that it is no longer accessible. This operation also copies the corresponding [DDS\\_SampleInfo](#). This operation is analogous to the [DDSDataReader::read\\_next\\_sample\\_untyped](#) except for the fact that the sample is removed from the [DDSDataReader](#).

The [DDSDataReader::take\\_next\\_sample\\_untyped](#) operation is semantically equivalent to the [DDSDataReader::take\\_untyped](#) operation, where the input data sequences has max\_len=1, the sample\_states=NOT\_READ, the view\_states=ANY\_↔VIEW\_STATE, and the instance\_states=ANY\_INSTANCE\_STATE.

The [DDSDataReader::take\\_next\\_sample\\_untyped](#) operation provides a simplified API to 'take' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDataReader](#), the operation will fail with [DDS\\_RETCODE\\_NO\\_DATA](#) and nothing is copied.

**Parameters**

|                      |                                                                                                                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i> | <<inout>> user data type-specific Data object where the next received data sample will be returned. The received_data must have been fully allocated. Otherwise, this operation may fail. |
| <i>sample_info</i>   | <<inout>> a <a href="#">DDS_SampleInfo</a> object where the next received sample info will be returned.                                                                                   |

**Returns**

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

**See also**

[DDSDataReader::take\\_untyped](#)

**lookup\_instance\_untyped()**

```
virtual DDS_InstanceHandle_t DDSDataReader::lookup_instance_untyped (
    const void * key_holder) [pure virtual]
```

Retrieve the [DDS\\_InstanceHandle\\_t](#) identifying a specific instance.

**Parameters**

|                   |                                                                        |
|-------------------|------------------------------------------------------------------------|
| <i>key_holder</i> | <<in>> A data sample containing the desired values for key attributes. |
|-------------------|------------------------------------------------------------------------|

**Returns**

The [DDS\\_InstanceHandle\\_t](#) for the selected instance or [DDS\\_HANDLE\\_NIL](#) if the instance was not found or an error occurred.

## is\_data\_consistent\_untyped()

```
virtual DDS_ReturnCode_t DDSDataReader::is_data_consistent_untyped (
    DDS_Boolean & is_data_consistent,
    const void * sample,
    const DDS_SampleInfo & sample_info) [pure virtual]
```

Checks if the sample has been overwritten by the DataWriter.

When a sample is received via Zero Copy transfer over shared memory, the sample can be reused by the DataWriter once it is removed from the DataWriter's send queue. Since there is no synchronization between the DataReader and the DataWriter, the sample could be overwritten by the DataWriter before it is processed by the DataReader. The [FooDataReader::is\\_data\\_consistent](#) operation can be used after processing the sample to check if it was overwritten by the DataWriter.

A precondition for using this operation is to set [DDS\\_DataWriterShmemRefTransferModeSettings::enable\\_data\\_consistency\\_check](#) to true.

### Parameters

|                           |                                                                                                                          |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>is_data_consistent</i> | << <i>inout</i> >> Set to true if the sample is consistent (i.e., the sample has not been overwritten by the DataWriter) |
| <i>sample</i>             | << <i>in</i> >> Sample to be validated                                                                                   |
| <i>sample_info</i>        | << <i>in</i> >> <a href="#">DDS_SampleInfo</a> object received with the sample                                           |

### Returns

One of the [Standard Return Codes](#) or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

## return\_loan\_untyped()

```
virtual DDS_ReturnCode_t DDSDataReader::return_loan_untyped (
    DDS_UntypedSampleSeq * received_data,
    DDS_SampleInfoSeq & info_seq) [pure virtual]
```

Indicates to the [DDSDDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of read or take on the [DDSDDataReader](#).

This operation indicates to the [DDSDDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of read or take on the [DDSDDataReader](#).

The `received_data` and `info_seq` must belong to a single related "pair"; that is, they should correspond to a pair returned from a single call to read or take. The `received_data` and `info_seq` must also have been obtained from the same [DDSDDataReader](#) to which they are returned. If either of these conditions is not met, the operation will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

The operation [DDSDDataReader::return\\_loan\\_untyped](#) allows implementations of the read and take operations to "loan" buffers from the [DDSDDataReader](#) to the application and in this manner provide "zerocopy" access to the data. During the loan, the [DDSDDataReader](#) will guarantee that the data and sample-information are not modified.

It is not necessary for an application to return the loans immediately after the read or take calls. However, as these buffers correspond to internal resources inside the [DDSDataReader](#), the application should not retain them indefinitely.

The use of [DDSDataReader::return\\_loan\\_untyped](#) is only necessary if the read or take calls "loaned" buffers to the application. This only occurs if the `received_data` and `info_seq` collections had `max_len=0` at the time read or take was called.

The application may also examine the "owns" property of the collection to determine where there is an outstanding loan. However, calling [DDSDataReader::return\\_loan\\_untyped](#) on a collection that does not have a loan is safe and has no side effects.

If the collections had a loan, upon completion of [DDSDataReader::return\\_loan\\_untyped](#), the collections will have `max_len=0`.

#### Parameters

|                      |                                                                                                                                                                                          |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i> | <<in>> user data type-specific <a href="#">FooSeq</a> object where the received data samples was obtained from earlier invocation of read or take on the <a href="#">DDSDataReader</a> . |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Parameters

|                 |                                                                                                                                                                              |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>info_seq</i> | <<in>> a <a href="#">DDS_SampleInfoSeq</a> object where the received sample info was obtained from earlier invocation of read or take on the <a href="#">DDSDataReader</a> . |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

### get\_subscription\_matched\_status()

```
virtual DDS_ReturnCode_t DDSDataReader::get_subscription_matched_status (
    DDS_SubscriptionMatchedStatus & status) [pure virtual]
```

Accesses the [DDS\\_SUBSCRIPTION\\_MATCHED\\_STATUS](#) communication status.

#### Parameters

|               |                                                                          |
|---------------|--------------------------------------------------------------------------|
| <i>status</i> | <<inout>> <a href="#">DDS_SubscriptionMatchedStatus</a> to be filled in. |
|---------------|--------------------------------------------------------------------------|

#### Returns

One of the [Standard Return Codes](#)

### get\_liveliness\_changed\_status()

```
virtual DDS_ReturnCode_t DDSDataReader::get_liveliness_changed_status (
    DDS_LivelinessChangedStatus & status) [pure virtual]
```

Accesses the [DDS\\_LIVELINESS\\_CHANGED\\_STATUS](#) communication status.

#### Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>status</i> | <<inout>> <a href="#">DDS_LivelinessChangedStatus</a> to be filled in. |
|---------------|------------------------------------------------------------------------|

#### Returns

One of the [Standard Return Codes](#)

#### **get\_matched\_publications()**

```
virtual DDS_ReturnCode_t DDSDataReader::get_matched_publications (  
    DDS_InstanceHandleSeq & publication_handles) [pure virtual]
```

Returns a sequence containing [DDS\\_InstanceHandle\\_t](#) for all the remote [DDSDDataWriter](#) that have been matched by the [DDSDDataReader](#).

#### Parameters

|                            |                                                                                           |
|----------------------------|-------------------------------------------------------------------------------------------|
| <i>publication_handles</i> | <<out>> sequence of <a href="#">DDS_InstanceHandle_t</a> where the result will be stored. |
|----------------------------|-------------------------------------------------------------------------------------------|

#### Returns

One of the [Standard Return Codes](#)

#### **get\_matched\_publication\_data()**

```
virtual DDS_ReturnCode_t DDSDataReader::get_matched_publication_data (  
    DDS_PublicationBuiltinTopicData & publication_data,  
    const DDS_InstanceHandle_t & publication_handle) [pure virtual]
```

Returns information about a remote [DDSDDataWriter](#) that has been matched by the [DDSDDataReader](#).

#### Parameters

|                           |                                                                                                     |
|---------------------------|-----------------------------------------------------------------------------------------------------|
| <i>publication_data</i>   | <<out>> information about the remote <a href="#">DDSDDataWriter</a> .                               |
| <i>publication_handle</i> | <<in>> <a href="#">DDS_InstanceHandle_t</a> identifying the remote <a href="#">DDSDDataWriter</a> . |

#### Returns

One of the [Standard Return Codes](#)

#### **get\_sample\_rejected\_status()**

```
virtual DDS_ReturnCode_t DDSDataReader::get_sample_rejected_status (  
    DDS_SampleRejectedStatus & status) [pure virtual]
```

Accesses the [DDS\\_SAMPLE\\_REJECTED\\_STATUS](#) communication status.

**Parameters**

|               |                                                                     |
|---------------|---------------------------------------------------------------------|
| <i>status</i> | <<inout>> <a href="#">DDS_SampleRejectedStatus</a> to be filled in. |
|---------------|---------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**get\_sample\_lost\_status()**

```
virtual DDS_ReturnCode_t DDSDataReader::get_sample_lost_status (  
    DDS_SampleLostStatus & status) [pure virtual]
```

Accesses the [DDS\\_SAMPLE\\_LOST\\_STATUS](#) communication status.

**Parameters**

|               |                                                                 |
|---------------|-----------------------------------------------------------------|
| <i>status</i> | <<inout>> <a href="#">DDS_SampleLostStatus</a> to be filled in. |
|---------------|-----------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**get\_requested\_deadline\_missed\_status()**

```
virtual DDS_ReturnCode_t DDSDataReader::get_requested_deadline_missed_status (  
    DDS_RequestedDeadlineMissedStatus & status) [pure virtual]
```

Accesses the [DDS\\_REQUESTED\\_DEADLINE\\_MISSED\\_STATUS](#) communication status.

**Parameters**

|               |                                                                              |
|---------------|------------------------------------------------------------------------------|
| <i>status</i> | <<inout>> <a href="#">DDS_RequestedDeadlineMissedStatus</a> to be filled in. |
|---------------|------------------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**get\_requested\_incompatible\_qos\_status()**

```
virtual DDS_ReturnCode_t DDSDataReader::get_requested_incompatible_qos_status (  
    DDS_RequestedIncompatibleQosStatus & status) [pure virtual]
```

Accesses the [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) communication status.



## Parameters

|               |                                                                                        |
|---------------|----------------------------------------------------------------------------------------|
| <i>status</i> | << <i>inout</i> >> <a href="#">DDS_RequestedIncompatibleQosStatus</a> to be filled in. |
|---------------|----------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#)

**get\_instance\_replaced\_missed\_status()**

```
virtual DDS_ReturnCode_t DDSDataReader::get_instance_replaced_missed_status (
    DDS_DataReaderInstanceReplacedStatus & status) [pure virtual]
```

Accesses the [DDS\\_INSTANCE\\_REPLACED\\_STATUS](#) communication status.

## Parameters

|               |                                                                                          |
|---------------|------------------------------------------------------------------------------------------|
| <i>status</i> | << <i>inout</i> >> <a href="#">DDS_DataReaderInstanceReplacedStatus</a> to be filled in. |
|---------------|------------------------------------------------------------------------------------------|

## Returns

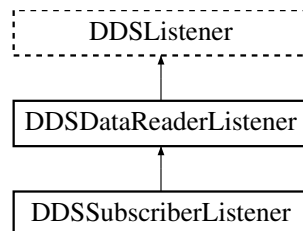
One of the [Standard Return Codes](#)

**7.94 DDSDataReaderListener Class Reference**

<<*interface*>> [DDSListener](#) for reader status.

```
#include <dds_cpp_subscription.hxx>
```

Inheritance diagram for DDSDataReaderListener:



**Public Member Functions**

- virtual void [on\\_data\\_available](#) (DDSDataReader \*)  
*Handle the [DDS\\_DATA\\_AVAILABLE\\_STATUS](#) communication status.*
- virtual void [on\\_requested\\_deadline\\_missed](#) (DDSDataReader \*, const [DDS\\_RequestedDeadlineMissedStatus](#) &)  
*Handles the [DDS\\_REQUESTED\\_DEADLINE\\_MISSED\\_STATUS](#) communication status.*
- virtual void [on\\_liveliness\\_changed](#) (DDSDataReader \*, const [DDS\\_LivelinessChangedStatus](#) &)  
*Handles the [DDS\\_LIVELINESS\\_CHANGED\\_STATUS](#) communication status.*
- virtual void [on\\_requested\\_incompatible\\_qos](#) (DDSDataReader \*, const [DDS\\_RequestedIncompatibleQosStatus](#) &)  
*Handles the [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) communication status.*
- virtual void [on\\_sample\\_rejected](#) (DDSDataReader \*, const [DDS\\_SampleRejectedStatus](#) &)  
*Handles the [DDS\\_SAMPLE\\_REJECTED\\_STATUS](#) communication status.*
- virtual void [on\\_subscription\\_matched](#) (DDSDataReader \*, const [DDS\\_SubscriptionMatchedStatus](#) &)  
*Handles the [DDS\\_SUBSCRIPTION\\_MATCHED\\_STATUS](#) communication status.*
- virtual void [on\\_sample\\_lost](#) (DDSDataReader \*, const [DDS\\_SampleLostStatus](#) &)  
*Handles the [DDS\\_SAMPLE\\_LOST\\_STATUS](#) communication status.*
- virtual void [on\\_instance\\_replaced](#) (DDSDataReader \*, const [DDS\\_DataReaderInstanceReplacedStatus](#) &)  
*Handles the [DDS\\_INSTANCE\\_REPLACED\\_STATUS](#) communication status.*
- virtual [DDS\\_Boolean](#) [on\\_before\\_sample\\_deserialize](#) (DDSDataReader \*, NDDS\_Type\_Plugin \*, CDR\_Stream↔  
\_t \*, [DDS\\_Boolean](#) \*)  
*Callback to filter a received sample based on serialized data*
- virtual [DDS\\_Boolean](#) [on\\_before\\_sample\\_commit](#) (DDSDataReader \*, const void \*const, const struct  
[DDS\\_SampleInfo](#) \*const, [DDS\\_Boolean](#) \*)  
*Callback to filter a received sample based on deserialized data*

**7.94.1 Detailed Description**

<<[interface](#)>> [DDSListener](#) for reader status.

Entity:

[DDSDataReader](#)

Status:

```

DDS\_DATA\_AVAILABLE\_STATUS;
DDS\_LIVELINESS\_CHANGED\_STATUS, DDS\_LivelinessChangedStatus;
DDS\_REQUESTED\_DEADLINE\_MISSED\_STATUS, DDS\_RequestedDeadlineMissedStatus;
DDS\_REQUESTED\_INCOMPATIBLE\_QOS\_STATUS, DDS\_RequestedIncompatibleQosStatus;
DDS\_SAMPLE\_LOST\_STATUS, DDS\_SampleLostStatus;
DDS\_SAMPLE\_REJECTED\_STATUS, DDS\_SampleRejectedStatus;
DDS\_SUBSCRIPTION\_MATCHED\_STATUS, DDS\_SubscriptionMatchedStatus;
DDS\_INSTANCE\_REPLACED\_STATUS, DDS\_DataReaderInstanceReplacedStatus;

```

See also

[Status Kinds](#)

## 7.94.2 Member Function Documentation

### on\_data\_available()

```
virtual void DDSDataReaderListener::on_data_available (
    DDSDataReader * ) [inline], [virtual]
```

Handle the [DDS\\_DATA\\_AVAILABLE\\_STATUS](#) communication status.

### on\_requested\_deadline\_missed()

```
virtual void DDSDataReaderListener::on_requested_deadline_missed (
    DDSDataReader * ,
    const DDS_RequestedDeadlineMissedStatus & ) [inline], [virtual]
```

Handles the [DDS\\_REQUESTED\\_DEADLINE\\_MISSED\\_STATUS](#) communication status.

### on\_liveliness\_changed()

```
virtual void DDSDataReaderListener::on_liveliness_changed (
    DDSDataReader * ,
    const DDS_LivelinessChangedStatus & ) [inline], [virtual]
```

Handles the [DDS\\_LIVELINESS\\_CHANGED\\_STATUS](#) communication status.

### on\_requested\_incompatible\_qos()

```
virtual void DDSDataReaderListener::on_requested_incompatible_qos (
    DDSDataReader * ,
    const DDS_RequestedIncompatibleQosStatus & ) [inline], [virtual]
```

Handles the [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) communication status.

### on\_sample\_rejected()

```
virtual void DDSDataReaderListener::on_sample_rejected (
    DDSDataReader * ,
    const DDS_SampleRejectedStatus & ) [inline], [virtual]
```

Handles the [DDS\\_SAMPLE\\_REJECTED\\_STATUS](#) communication status.

**on\_subscription\_matched()**

```
virtual void DDSDataReaderListener::on_subscription_matched (
    DDSDataReader * ,
    const DDS_SubscriptionMatchedStatus & ) [inline], [virtual]
```

Handles the [DDS\\_SUBSCRIPTION\\_MATCHED\\_STATUS](#) communication status.

**on\_sample\_lost()**

```
virtual void DDSDataReaderListener::on_sample_lost (
    DDSDataReader * ,
    const DDS_SampleLostStatus & ) [inline], [virtual]
```

Handles the [DDS\\_SAMPLE\\_LOST\\_STATUS](#) communication status.

**on\_instance\_replaced()**

```
virtual void DDSDataReaderListener::on_instance_replaced (
    DDSDataReader * ,
    const DDS_DataReaderInstanceReplacedStatus & ) [inline], [virtual]
```

Handles the [DDS\\_INSTANCE\\_REPLACED\\_STATUS](#) communication status.

This listener may only be called when the instance replacement policy is set to [DDS\\_REPLACE\\_OLDEST\\_INSTANCE↵\\_REPLACEMENT\\_QOS](#). When a new instance is detected and resources are exhausted the oldest instance will be removed even if samples with a not read state exists for the instance. Thus, this is a forceful removal of samples and the instance. The freed instance resources will be re-used for the newly detected instance. If there are outstanding loans for the replaced instance the samples can not be freed until returned. However, when the loan is returned the samples are automatically freed. Note that the instance will not be rejected.

**on\_before\_sample\_deserialize()**

```
virtual DDS_Boolean DDSDataReaderListener::on_before_sample_deserialize (
    DDSDataReader * ,
    NDDS_Type_Plugin * ,
    CDR_Stream_t * ,
    DDS_Boolean * ) [inline], [virtual]
```

Callback to filter a received sample based on serialized data

When a DataReader receives a (serialized) sample, it will deserialize it before storing it in its queue. [On\\_before\\_↵sample\\_deserialize\(\)](#) is called before deserialization. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filter out of the DataReader's queue, with the [sample\\_dropped](#) parameter. If user code within the callback determines that the sample should be dropped, the user should set [sample\\_dropped](#) to true; consequently, when the callback returns, the DataReader will drop the sample before it is committed to the queue. Otherwise, returning the callback with [sample\\_dropped](#) as false will allow the sample to be read or taken.

Compared to [on\\_before\\_sample\\_commit\(\)](#), the sample would be dropped with less processing (without deserialization) by the subscribing application, but with the added complexity of filtering on serialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

## on\_before\_sample\_commit()

```
virtual DDS_Boolean DDSDataReaderListener::on_before_sample_commit (
    DDSDataReader * ,
    const void * const ,
    const struct DDS_SampleInfo * const ,
    DDS_Boolean * ) [inline], [virtual]
```

Callback to filter a received sample based on deserialized data

When a DataReader receives a (serialized) sample, it will deserialize it before storing it in its queue. On\_before\_sample\_commit() is called after deserialization, but before the sample is stored into the DataReader's queue. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filter out of the DataReader's queue, with the sample\_dropped parameter. If user code within the callback determines that the sample should be dropped, the user should set sample\_dropped to true; consequently, when the callback returns, the DataReader will drop the sample before it is committed to the queue. Otherwise, returning the callback with sample\_dropped as false will allow the sample to be read or taken.

Compared to on\_before\_sample\_deserialize(), the sample would be dropped with more processing (with deserialization) by the subscribing application, but with the lesser complexity of filtering on deserialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

The following fields are valid in the DDS\_SampleInfo structure:

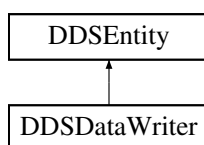
- source\_timestamp
- instance\_handle
- publication\_handle
- valid\_data
- encapsulation\_id

## 7.95 DDSDataWriter Class Reference

<<interface>> Allows an application to set the value of the data to be published under a given DDSTopic.

```
#include <dds_cpp_publication.hxx>
```

Inheritance diagram for DDSDataWriter:



## Public Member Functions

- virtual [DDS\\_ReturnCode\\_t write\\_untyped](#) (const void \*instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle)=0  
*Modifies the value of a data instance.*
- virtual [DDS\\_ReturnCode\\_t assert\\_liveliness](#) ()=0  
*This operation manually asserts the liveliness of this [DDSDDataWriter](#).*
- virtual [DDSTopic](#) \* [get\\_topic](#) ()=0  
*This operation returns the [DDSTopic](#) associated with the [DDSDDataWriter](#).*
- virtual [DDSPublisher](#) \* [get\\_publisher](#) ()=0  
*This operation returns the [DDSPublisher](#) to which the [DDSDDataWriter](#) belongs.*
- virtual [DDS\\_ReturnCode\\_t get\\_qos](#) ([DDS\\_DataWriterQos](#) &qos)=0  
*Gets the writer QoS.*
- virtual [DDS\\_ReturnCode\\_t set\\_qos](#) (const [DDS\\_DataWriterQos](#) &qos)=0  
*Sets the writer QoS.*
- virtual [DDS\\_ReturnCode\\_t set\\_listener](#) (const [DDSDDataWriterListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
*Sets the writer listener.*
- virtual [DDSDDataWriterListener](#) \* [get\\_listener](#) ()=0  
*Get the writer listener.*
- virtual [DDS\\_InstanceHandle\\_t register\\_instance\\_untyped](#) (const void \*instance\_data)=0  
*Informs RTI Connex Micro that the application will be modifying a particular instance.*
- virtual [DDS\\_InstanceHandle\\_t register\\_instance\\_w\\_timestamp\\_untyped](#) (const void \*instance\_data, const [DDS\\_Time\\_t](#) &source\_timestamp)=0  
*Performs the same function as [DDSDDataWriter::register\\_instance\\_untyped](#) except that it also provides the value for the `source_timestamp`.*
- virtual [DDS\\_ReturnCode\\_t unregister\\_instance\\_untyped](#) (const void \*instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle)=0  
*Reverses the action of [DDSDDataWriter::register\\_instance\\_untyped](#).*
- virtual [DDS\\_ReturnCode\\_t unregister\\_instance\\_w\\_timestamp\\_untyped](#) (const void \*instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle, const [DDS\\_Time\\_t](#) &source\_timestamp)=0  
*Performs the same function as [DDSDDataWriter::unregister\\_instance\\_untyped](#) except that it also provides the value for the `source_timestamp`.*
- virtual [DDS\\_ReturnCode\\_t dispose\\_untyped](#) (const void \*instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle)=0  
*Requests the middleware to delete the data.*
- virtual [DDS\\_ReturnCode\\_t dispose\\_w\\_timestamp\\_untyped](#) (const void \*instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle, const [DDS\\_Time\\_t](#) &source\_timestamp)=0  
*Performs the same function as [DDSDDataWriter::dispose\\_untyped](#) except that it also provides the value for the `source_timestamp`.*
- virtual [DDS\\_ReturnCode\\_t write\\_w\\_timestamp\\_untyped](#) (const void \*instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle, const [DDS\\_Time\\_t](#) &source\_timestamp)=0  
*Performs the same function as [DDSDDataWriter::write\\_untyped](#) except that it also provides the value for the `source_timestamp`.*
- virtual [DDS\\_ReturnCode\\_t get\\_publication\\_matched\\_status](#) ([DDS\\_PublicationMatchedStatus](#) &status)=0  
*Accesses the `DDS_PUBLICATION_MATCHED_STATUS` communication status.*
- virtual [DDS\\_ReturnCode\\_t get\\_liveliness\\_lost\\_status](#) ([DDS\\_LivelinessLostStatus](#) &status)=0  
*Accesses the `DDS_LIVELINESS_LOST_STATUS` communication status.*
- virtual [DDS\\_ReturnCode\\_t get\\_matched\\_subscriptions](#) ([DDS\\_InstanceHandleSeq](#) &subscription\_handles)=0  
*Returns a sequence containing [DDS\\_InstanceHandle\\_t](#) for all the remote [DDSDDataReader](#) that have been matched by the [DDSDDataWriter](#).*

- virtual `DDS_ReturnCode_t get_matched_subscription_data (DDS_SubscriptionBuiltinTopicData &subscription_data, const DDS_InstanceHandle_t &subscription_handle)=0`  
Returns information about a remote *DDSDataReader* that has been matched by the *DDSDataWriter*.
- virtual `DDS_ReturnCode_t get_offered_deadline_missed_status (DDS_OfferedDeadlineMissedStatus &status)=0`  
Accesses the *DDS\_OFFERED\_DEADLINE\_MISSED\_STATUS* communication status.
- virtual `DDS_ReturnCode_t get_offered_incompatible_qos_status (DDS_OfferedIncompatibleQosStatus &status)=0`  
Accesses the *DDS\_OFFERED\_INCOMPATIBLE\_QOS\_STATUS* communication status.

### Public Member Functions inherited from *DDSEntity*

- `DDS_ReturnCode_t enable ()`  
Enables the *DDSEntity*.
- `DDSStatusCondition * get_statuscondition ()`  
Return the *DDSStatusCondition* associated with a particular *DDSEntity*.
- `DDS_StatusMask get_status_changes ()`  
Retrieves the list of communication statuses in the *DDSEntity* that are triggered.
- `DDS_InstanceHandle_t get_instance_handle ()`  
Allows access to the *DDS\_InstanceHandle\_t* associated with the *DDSEntity*.

#### 7.95.1 Detailed Description

<<*interface*>> Allows an application to set the value of the data to be published under a given *DDSTopic*.

QoS:

*DDS\_DataWriterQos*

Status:

*DDS\_LIVELINESS\_LOST\_STATUS*, *DDS\_LivelinessLostStatus*;  
*DDS\_OFFERED\_DEADLINE\_MISSED\_STATUS*, *DDS\_OfferedDeadlineMissedStatus*;  
*DDS\_OFFERED\_INCOMPATIBLE\_QOS\_STATUS*, *DDS\_OfferedIncompatibleQosStatus*;  
*DDS\_PUBLICATION\_MATCHED\_STATUS*, *DDS\_PublicationMatchedStatus*;

Listener:

*DDSDataWriterListener*

A *DDSDataWriter* is attached to exactly one *DDSPublisher*, that acts as a factory for it.

A *DDSDataWriter* is bound to exactly one *DDSTopic* and therefore to exactly one data type. The *DDSTopic* must exist prior to the *DDSDataWriter*'s creation.

*DDSDataWriter* is an abstract class. It must be specialized for each particular application data-type. The additional methods or functions that must be defined in the auto-generated class for a hypothetical application type `Foo` are specified in the example type *DDSDataWriter*.

The following operations may be called even if the *DDSDataWriter* is not enabled. Other operations will fail with *DDS\_RETCODE\_NOT\_ENABLED* if called on a disabled *DDSDataWriter*:

- The base-class operations [DDSDataWriter::set\\_qos](#), [DDSDataWriter::get\\_qos](#), [DDSDataWriter::set\\_listener](#), [DDSDataWriter::get\\_listener](#), and [DDSEntity::enable](#)
- [DDSDataWriter::get\\_liveliness\\_lost\\_status](#) [DDSDataWriter::get\\_offered\\_deadline\\_missed\\_status](#) [DDSDataWriter::get\\_offered\\_incomplete\\_status](#) [DDSDataWriter::get\\_publication\\_matched\\_status](#)

Several [DDSDataWriter](#) may operate in different threads. If they share the same [DDSPublisher](#), the middleware guarantees that its operations are thread-safe.

### 7.95.2 Member Function Documentation

#### write\_untyped()

```
virtual DDS_ReturnCode_t DDSDataWriter::write_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle) [pure virtual]
```

Modifies the value of a data instance.

When this operation is used, RTI Connex DDS Micro will automatically supply the value of the `source_timestamp` that is made available to [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS\\_SampleInfo](#). (Refer to [DDS\\_SampleInfo](#) for details).

As a side effect, this operation asserts liveliness on the [DDSDataWriter](#) itself, the [DDSPublisher](#) and the [DDSDomainParticipant](#).

Note that the special value [DDS\\_HANDLE\\_NIL](#) can be used for the parameter `handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the `key`).

If `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#).

RTI Connex DDS Micro will not detect the error when the `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the `key`). RTI Connex DDS Micro will treat as if the `write()` operation is for the instance as indicated by the `handle`.

If there are no instance resources left, this operation may fail with [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#). Calling [DDSDataWriter::unregister\\_instance\\_untyped](#) may help freeing up some resources.

#### Parameters

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i> | <<in>> The data to write.                                                                                                                                                                                                                                                                                                                                                                                                         |
| <i>handle</i>        | <<in>> Either the handle returned by a previous call to <a href="#">DDSDataWriter::register_instance_untyped</a> , or else the special value <a href="#">DDS_HANDLE_NIL</a> . If the data <i>has</i> a key and <code>handle</code> is not <a href="#">DDS_HANDLE_NIL</a> , <code>handle</code> must represent a registered instance of the data. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> . |

#### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

#### See also

[DDSDataReader](#)  
[DDSDataWriter::write\\_w\\_timestamp\\_untyped](#)



**assert\_liveliness()**

```
virtual DDS_ReturnCode_t DDSDataWriter::assert_liveliness () [pure virtual]
```

This operation manually asserts the liveliness of this [DDSDataWriter](#).

This is used in combination with the [LIVELINESS](#) policy to indicate to RTI Connex DDS Micro that the [DDSDataWriter](#) remains active.

You only need to use this operation if the [LIVELINESS](#) setting is [DDS\\_MANUAL\\_BY\\_TOPIC\\_LIVELINESS\\_QOS](#). Otherwise, it has no effect.

**Note:** writing data via the [FooDataWriter::write](#) or [FooDataWriter::write\\_w\\_timestamp](#) operation asserts liveliness on the [DDSDataWriter](#) itself, and its [DDSDomainParticipant](#). Consequently the use of [assert\\_liveliness\(\)](#) is only needed if the application is not writing data regularly.

**Returns**

One of the [Standard Return Codes](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#)

**See also**

[DDS\\_LivelinessQosPolicy](#)

**get\_topic()**

```
virtual DDSTopic * DDSDataWriter::get_topic () [pure virtual]
```

This operation returns the [DDSTopic](#) associated with the [DDSDataWriter](#).

This is the same [DDSTopic](#) that was used to create the [DDSDataWriter](#).

**Returns**

[DDSTopic](#) that was used to create the [DDSDataWriter](#).

**get\_publisher()**

```
virtual DDSPublisher * DDSDataWriter::get_publisher () [pure virtual]
```

This operation returns the [DDSPublisher](#) to which the [DDSDataWriter](#) belongs.

**Returns**

[DDSPublisher](#) to which the [DDSDataWriter](#) belongs.

**get\_qos()**

```
virtual DDS_ReturnCode_t DDSDataWriter::get_qos (
    DDS_DataWriterQos & qos) [pure virtual]
```

Gets the writer QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

## Parameters

|            |                                                                           |
|------------|---------------------------------------------------------------------------|
| <i>qos</i> | << <i>inout</i> >> The <a href="#">DDS_DataWriterQos</a> to be filled up. |
|------------|---------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#)

## See also

[get\\_qos](#) (abstract)

**set\_qos()**

```
virtual DDS_ReturnCode_t DDSDDataWriter::set_qos (
    const DDS_DataWriterQos & qos) [pure virtual]
```

Sets the writer QoS.

This operation modifies the QoS of the [DDSDDataWriter](#).

The [DDS\\_DataWriterQos::deadline](#) and [DDS\\_DataWriterQos::ownership\\_strength](#), can be changed. The other policies are immutable.

## Parameters

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> The <a href="#">DDS_DataWriterQos</a> to be set to. Policies must be consistent. Policies cannot be changed after <a href="#">DDSDDataWriter</a> is enabled. The special value <a href="#">DDS_DATAWRITER_QOS_DEFAULT</a> can be used to indicate that the QoS of the <a href="#">DDSDDataWriter</a> should be changed to match the current default <a href="#">DDS_DataWriterQos</a> set in the <a href="#">DDSPublisher</a> . |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#) or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

## See also

[DDS\\_DataWriterQos](#) for rules on consistency among QoS

[set\\_qos](#) (abstract)

**set\_listener()**

```
virtual DDS_ReturnCode_t DDSDDataWriter::set_listener (
    const DDSDDataWriterListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Sets the writer listener.

## Parameters

|                 |                                                                                                    |
|-----------------|----------------------------------------------------------------------------------------------------|
| <i>listener</i> | <<in>> <a href="#">DDSDDataWriterListener</a> to set to                                            |
| <i>mask</i>     | <<in>> <a href="#">DDS_StatusMask</a> associated with the <a href="#">DDSDDataWriterListener</a> . |

## Returns

One of the [Standard Return Codes](#)

## See also

[set\\_listener](#) (abstract)

**get\_listener()**

```
virtual DDSDDataWriterListener * DDSDataWriter::get_listener () [pure virtual]
```

Get the writer listener.

## Returns

[DDSDDataWriterListener](#) of the [DDSDDataWriter](#).

**register\_instance\_untyped()**

```
virtual DDS\_InstanceHandle\_t DDSDataWriter::register_instance_untyped (
    const void * instance_data) [pure virtual]
```

Informs RTI Connex DDS Micro that the application will be modifying a particular instance.

This operation is only useful for keyed data types. Using it for non-keyed types causes no effect and returns [DDS\\_HANDLE\\_NIL](#). The operation takes as a parameter an instance (of which only the key value is examined) and returns a `handle` that can be used in successive `write()` or `dispose()` operations.

The operation gives RTI Connex DDS Micro an opportunity to pre-configure itself to improve performance.

The use of this operation by an application is optional even for keyed types. If an instance has not been pre-registered, the application can use the special value [DDS\\_HANDLE\\_NIL](#) as the [DDS\\_InstanceHandle\\_t](#) paramater to the write or dispose operation and RTI Connex DDS Micro will auto-register the instance.

For best performance, the operation should be invoked prior to calling any operation that modifies the instance, such as [DDSDDataWriter::write\\_untyped](#), [DDSDDataWriter::write\\_w\\_timestamp\\_untyped](#), [DDSDDataWriter::dispose\\_untyped](#) and the handle used in conjunction with the data for those calls.

When this operation is used, RTI Connex DDS Micro will automatically supply the value of the `source_timestamp` that is used.

This operation may fail and return [DDS\\_HANDLE\\_NIL](#) if [DDS\\_ResourceLimitsQosPolicy::max\\_instances](#) limit has been exceeded.

The operation is **idempotent**. If it is called for an already registered instance, it just returns the already allocated handle. This may be used to lookup and retrieve the handle allocated to a given instance.

This operation can only be called after [DDSDDataWriter](#) has been enabled. Otherwise, [DDS\\_HANDLE\\_NIL](#) will be returned.

## Parameters

|                      |                                                                                                                                         |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i> | <<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the function. . |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|

## Returns

For keyed data type, a handle that can be used in the calls that take a [DDS\\_InstanceHandle\\_t](#), such as `write`, `dispose`, `unregister_instance`, or return `DDS_HANDLE_NIL` on failure. If the `instance_data` is of a data type that has no keys, this function always return `DDS_HANDLE_NIL`.

## See also

[DDSDataWriter::unregister\\_instance\\_untyped](#)

**register\_instance\_w\_timestamp\_untyped()**

```
virtual DDS_InstanceHandle_t DDSDataWriter::register_instance_w_timestamp_untyped (
    const void * instance_data,
    const DDS_Time_t & source_timestamp) [pure virtual]
```

Performs the same function as [DDSDataWriter::register\\_instance\\_untyped](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS\\_SampleInfo](#). (Refer to [DDS\\_SampleInfo](#) for details)

The constraints on the values of the parameters and the corresponding error behavior are the same specified for the [DDSDataWriter::register\\_instance\\_untyped](#) operation.

## Parameters

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i>    | <<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the function. .                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>source_timestamp</i> | <<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the <a href="#">DDSDataReader</a> objects by means of the <code>source_timestamp</code> attribute inside the <a href="#">DDS_SampleInfo</a> . |

## Returns

For keyed data type, a handle that can be used in the calls that take a [DDS\\_InstanceHandle\\_t](#), such as `write`, `dispose`, `unregister_instance`, or return `DDS_HANDLE_NIL` on failure. If the `instance_data` is of a data type that has no keys, this function always return `DDS_HANDLE_NIL`.

## See also

[DDSDataWriter::unregister\\_instance\\_untyped](#)

## unregister\_instance\_untyped()

```
virtual DDS_ReturnCode_t DDSDataWriter::unregister_instance_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle) [pure virtual]
```

Reverses the action of [DDSDataWriter::register\\_instance\\_untyped](#).

This operation is useful only for keyed data types. Using it for non-keyed types causes no effect and reports no error. The operation takes as a parameter an instance (of which only the key value is examined) and a handle.

This operation should only be called on an instance that is currently registered. This includes instances that have been auto-registered by calling operations such as write or dispose as described in [DDSDataWriter::register\\_instance\\_untyped](#). Otherwise, this operation may fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#).

This only need be called just once per instance, regardless of how many times register\_instance was called for that instance.

When this operation is used, RTI Connex DDS Micro will automatically supply the value of the `source_timestamp` that is used.

This operation informs RTI Connex DDS Micro that the [DDSDataWriter](#) is no longer going to provide any information about the instance. This operation also indicates that RTI Connex DDS Micro can locally remove all information regarding that instance. The application should not attempt to use the `handle` previously allocated to that instance after calling [DDSDataWriter::unregister\\_instance\\_untyped\(\)](#).

The special value [DDS\\_HANDLE\\_NIL](#) can be used for the parameter `handle`. This indicates that the identity of the instance should be automatically deduced from the `instance_data` (by means of the `key`).

If `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#).

RTI Connex DDS Micro will not detect the error when the `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the `key`). RTI Connex DDS Micro will treat as if the `unregister_instance()` operation is for the instance as indicated by the `handle`.

If after a [DDSDataWriter::unregister\\_instance\\_untyped](#), the application wants to modify ([DDSDataWriter::write\\_untyped](#) or [DDSDataWriter::dispose\\_untyped](#)) an instance, it has to register it again, or else use the special `handle` value [DDS\\_HANDLE\\_NIL](#).

This operation does not indicate that the instance is deleted (that is the purpose of [DDSDataWriter::dispose\\_untyped](#)). The operation [DDSDataWriter::unregister\\_instance\\_untyped](#) just indicates that the [DDSDataWriter](#) no longer has anything to say about the instance. [DDSDataReader](#) entities that are reading the instance may receive a sample with [DDS\\_NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#) for the instance, unless there are other [DDSDataWriter](#) objects writing that same instance.

This operation can affect the ownership of the data instance (see [OWNERSHIP](#)). If the [DDSDataWriter](#) was the exclusive owner of the instance, then calling `unregister_instance()` will relinquish that ownership.

## Parameters

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i> | <<in>> The instance that should be unregistered. If the data <i>has</i> a key and <i>instance_handle</i> is <a href="#">DDS_HANDLE_NIL</a> , only the fields that represent the key are examined by the function. Otherwise, <i>instance_data</i> is not used. If <i>instance_data</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> .         |
| <i>handle</i>        | <<in>> represents the instance to be unregistered. If the data <i>has</i> a key and <i>handle</i> is <a href="#">DDS_HANDLE_NIL</a> , <i>handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If the data has no key, <i>handle</i> is not used. If <i>handle</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> . |

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#)

## See also

[DDSDataWriter::register\\_instance\\_untyped](#)

**unregister\_instance\_w\_timestamp\_untyped()**

```
virtual DDS\_ReturnCode\_t DDSDataWriter::unregister_instance_w_timestamp_untyped (
    const void * instance_data,
    const DDS\_InstanceHandle\_t & handle,
    const DDS\_Time\_t & source_timestamp) [pure virtual]
```

Performs the same function as [DDSDataWriter::unregister\\_instance\\_untyped](#) except that it also provides the value for the *source\_timestamp*.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the *source\_timestamp* attribute inside the [DDS\\_SampleInfo](#). (Refer to [DDS\\_SampleInfo](#) for details)

The constraints on the values of the parameters and the corresponding error behavior are the same specified for the [DDSDataWriter::unregister\\_instance\\_untyped](#) operation.

## Parameters

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i> | <<in>> The instance that should be unregistered. If the data <i>has</i> a key and <i>instance_handle</i> is <a href="#">DDS_HANDLE_NIL</a> , only the fields that represent the key are examined by the function. Otherwise, <i>instance_data</i> is not used. If <i>instance_data</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> .         |
| <i>handle</i>        | <<in>> represents the instance to be unregistered. If the data <i>has</i> a key and <i>handle</i> is <a href="#">DDS_HANDLE_NIL</a> , <i>handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If the data has no key, <i>handle</i> is not used. If <i>handle</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> . |

## Parameters

|                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>source_timestamp</code> | << <i>in</i> >> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the <a href="#">DDSDDataReader</a> objects by means of the <code>source_timestamp</code> attribute inside the <a href="#">DDS_SampleInfo</a> . |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#)

## See also

[DDSDDataWriter::register\\_instance\\_untyped](#)

**dispose\_untyped()**

```
virtual DDS_ReturnCode_t DDSDataWriter::dispose_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle) [pure virtual]
```

Requests the middleware to delete the data.

This operation is useful only for keyed data types. Using it for non-keyed types has no effect and reports no error.

The actual deletion is postponed until there is no more use for that data in the whole system.

Applications are made aware of the deletion by means of operations on the [DDSDDataReader](#) objects that already knew that instance. [DDSDDataReader](#) objects that didn't know the instance will never see it.

This operation does not modify the value of the instance. The `instance_data` parameter is passed just for the purposes of identifying the instance.

When this operation is used, RTI Connext DDS Micro will automatically supply the value of the `source_timestamp` that is made available to [DDSDDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS\\_SampleInfo](#).

The constraints on the values of the handle parameter and the corresponding error behavior are the same specified for the [DDSDDataWriter::unregister\\_instance\\_untyped](#) operation.

The special value [DDS\\_HANDLE\\_NIL](#) can be used for the parameter `instance_handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the key).

If `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#).

RTI Connext DDS Micro will not detect the error when the `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the key). RTI Connext DDS Micro will treat as if the `dispose()` operation is for the instance as indicated by the `handle`.

This operation may block and time out ([DDS\\_RETCODE\\_TIMEOUT](#)) under the same circumstances described for [DDSDDataWriter::write\\_untyped\(\)](#).

If there are no instance resources left, this operation may fail with [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#). Calling [DDSDDataWriter::unregister\\_instance\\_untyped](#) may help freeing up some resources.

## Parameters

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i> | <<in>> The data to dispose. If the data <i>has</i> a key and <i>instance_handle</i> is <a href="#">DDS_HANDLE_NIL</a> , only the fields that represent the key are examined by the function. Otherwise, <i>instance_data</i> is not used.                                                                                                                                                                                                                                                                                                                                                                          |
| <i>handle</i>        | <<in>> Either the handle returned by a previous call to <a href="#">DDSDataWriter::register_instance_untyped</a> , or else the special value <a href="#">DDS_HANDLE_NIL</a> . If the data <i>has</i> a key and <i>instance_handle</i> is <a href="#">DDS_HANDLE_NIL</a> , <i>instance_handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If the data has no key, <i>instance_handle</i> is not used. If <i>handle</i> is used, it must represent a registered instance of type the matching data type. Otherwise, this method fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> . |

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

**dispose\_w\_timestamp\_untyped()**

```
virtual DDS_ReturnCode_t DDSDataWriter::dispose_w_timestamp_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp) [pure virtual]
```

Performs the same function as [DDSDataWriter::dispose\\_untyped](#) except that it also provides the value for the *source\_timestamp*.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the *source\_timestamp* attribute inside the [DDS\\_SampleInfo](#). (Refer to [DDS\\_SampleInfo](#) for details)

The constraints on the values of the parameters and the corresponding error behavior are the same specified for the [DDSDataWriter::dispose\\_untyped](#) operation.

## Parameters

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i>    | <<in>> The data to dispose. If the data <i>has</i> a key and <i>instance_handle</i> is <a href="#">DDS_HANDLE_NIL</a> , only the fields that represent the key are examined by the function. Otherwise, <i>instance_data</i> is not used.                                                                                                                                                                                                                                                                                                                                                                          |
| <i>handle</i>           | <<in>> Either the handle returned by a previous call to <a href="#">DDSDataWriter::register_instance_untyped</a> , or else the special value <a href="#">DDS_HANDLE_NIL</a> . If the data <i>has</i> a key and <i>instance_handle</i> is <a href="#">DDS_HANDLE_NIL</a> , <i>instance_handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If the data has no key, <i>instance_handle</i> is not used. If <i>handle</i> is used, it must represent a registered instance of type the matching data type. Otherwise, this method fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> . |
| <i>source_timestamp</i> | <<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the <a href="#">DDSDataReader</a> objects by means of the <i>source_timestamp</i> attribute inside the <a href="#">DDS_SampleInfo</a> .                                  |



## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

## write\_w\_timestamp\_untyped()

```
virtual DDS_ReturnCode_t DDSDataWriter::write_w_timestamp_untyped (
    const void * instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp) [pure virtual]
```

Performs the same function as [DDSDataWriter::write\\_untyped](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS\\_SampleInfo](#). (Refer to [DDS\\_SampleInfo](#) for details)

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the [DDSDataWriter::write\\_untyped](#) operation.

If there are no instance resources left, this operation may fail with [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#). Calling [DDSDataWriter::unregister\\_instance\\_untyped](#) may help free up some resources.

This operation may fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#) under the same circumstances described for the write operation.

## Parameters

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i>    | <<in>> The data to write.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <i>handle</i>           | <<in>> Either the handle returned by a previous call to <a href="#">DDSDataWriter::register_instance_untyped</a> , or else the special value <a href="#">DDS_HANDLE_NIL</a> . If the data <i>has</i> a key and <code>handle</code> is not <a href="#">DDS_HANDLE_NIL</a> , <code>handle</code> must represent a registered instance of the matching data type. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> .                                                                                                                                         |
| <i>source_timestamp</i> | <<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the <a href="#">DDSDataReader</a> objects by means of the <code>source_timestamp</code> attribute inside the <a href="#">DDS_SampleInfo</a> . |

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

## See also

[DDSDataWriter::write\\_untyped](#)

[DDSDataReader](#)

**get\_publication\_matched\_status()**

```
virtual DDS_ReturnCode_t DDSDataWriter::get_publication_matched_status (
    DDS_PublicationMatchedStatus & status) [pure virtual]
```

Accesses the [DDS\\_PUBLICATION\\_MATCHED\\_STATUS](#) communication status.

**Parameters**

|               |                                                                         |
|---------------|-------------------------------------------------------------------------|
| <i>status</i> | <<inout>> <a href="#">DDS_PublicationMatchedStatus</a> to be filled in. |
|---------------|-------------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**get\_liveliness\_lost\_status()**

```
virtual DDS_ReturnCode_t DDSDataWriter::get_liveliness_lost_status (
    DDS_LivelinessLostStatus & status) [pure virtual]
```

Accesses the [DDS\\_LIVELINESS\\_LOST\\_STATUS](#) communication status.

**Parameters**

|               |                                                                     |
|---------------|---------------------------------------------------------------------|
| <i>status</i> | <<inout>> <a href="#">DDS_LivelinessLostStatus</a> to be filled in. |
|---------------|---------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**get\_matched\_subscriptions()**

```
virtual DDS_ReturnCode_t DDSDataWriter::get_matched_subscriptions (
    DDS_InstanceHandleSeq & subscription_handles) [pure virtual]
```

Returns a sequence containing [DDS\\_InstanceHandle\\_t](#) for all the remote [DDSDDataReader](#) that have been matched by the [DDSDDataWriter](#).

**Parameters**

|                             |                                                                                           |
|-----------------------------|-------------------------------------------------------------------------------------------|
| <i>subscription_handles</i> | <<out>> sequence of <a href="#">DDS_InstanceHandle_t</a> where the result will be stored. |
|-----------------------------|-------------------------------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**get\_matched\_subscription\_data()**

```
virtual DDS_ReturnCode_t DDSDataWriter::get_matched_subscription_data (
    DDS_SubscriptionBuiltinTopicData & subscription_data,
    const DDS_InstanceHandle_t & subscription_handle) [pure virtual]
```

Returns information about a remote [DDSDDataReader](#) that has been matched by the [DDSDDataWriter](#).

**Parameters**

|                            |                                                                                                    |
|----------------------------|----------------------------------------------------------------------------------------------------|
| <i>subscription_data</i>   | <<out>> information about the remote <a href="#">DDSDataReader</a> .                               |
| <i>subscription_handle</i> | <<in>> <a href="#">DDS_InstanceHandle_t</a> identifying the remote <a href="#">DDSDataReader</a> . |

**Returns**

One of the [Standard Return Codes](#)

**get\_offered\_deadline\_missed\_status()**

```
virtual DDS\_ReturnCode\_t DDSDataWriter::get_offered_deadline_missed_status (  
    DDS\_OfferedDeadlineMissedStatus & status) [pure virtual]
```

Accesses the [DDS\\_OFFERED\\_DEADLINE\\_MISSED\\_STATUS](#) communication status.

**Parameters**

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>status</i> | <<inout>> <a href="#">DDS_OfferedDeadlineMissedStatus</a> to be filled in. |
|---------------|----------------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**get\_offered\_incompatible\_qos\_status()**

```
virtual DDS\_ReturnCode\_t DDSDataWriter::get_offered_incompatible_qos_status (  
    DDS\_OfferedIncompatibleQosStatus & status) [pure virtual]
```

Accesses the [DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) communication status.

**Parameters**

|               |                                                                             |
|---------------|-----------------------------------------------------------------------------|
| <i>status</i> | <<inout>> <a href="#">DDS_OfferedIncompatibleQosStatus</a> to be filled in. |
|---------------|-----------------------------------------------------------------------------|

**Returns**

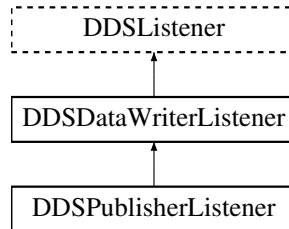
One of the [Standard Return Codes](#)

## 7.96 DDSDataWriterListener Class Reference

<<interface>> [DDSListener](#) for writer status.

```
#include <dds_cpp_publication.hxx>
```

Inheritance diagram for DDSDataWriterListener:



### Public Member Functions

- virtual void [on\\_offered\\_deadline\\_missed](#) (DDSDataWriter \*writer, const [DDS\\_OfferedDeadlineMissedStatus](#) &status)  
*Handles the [DDS\\_OFFERED\\_DEADLINE\\_MISSED\\_STATUS](#) status.*
- virtual void [on\\_liveliness\\_lost](#) (DDSDataWriter \*writer, const [DDS\\_LivelinessLostStatus](#) &status)  
*Handles the [DDS\\_LIVELINESS\\_LOST\\_STATUS](#) status.*
- virtual void [on\\_offered\\_incompatible\\_qos](#) (DDSDataWriter \*writer, const [DDS\\_OfferedIncompatibleQosStatus](#) &status)  
*Handles the [DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) status.*
- virtual void [on\\_publication\\_matched](#) (DDSDataWriter \*writer, const [DDS\\_PublicationMatchedStatus](#) &status)  
*Handles the [DDS\\_PUBLICATION\\_MATCHED\\_STATUS](#) status.*
- virtual void [on\\_reliable\\_reader\\_activity\\_changed](#) (DDSDataWriter \*writer, const [DDS\\_ReliableReaderActivityChangedStatus](#) &status)  
*<<eXtension>> A matched reliable reader has become active or become inactive.*
- virtual void [on\\_reliable\\_sample\\_unacknowledged](#) (DDSDataWriter \*writer, const [DDS\\_ReliableSampleUnacknowledgedStatus](#) &status)  
*<<eXtension>> A matched reliable reader has become active or become inactive.*
- virtual void [on\\_sample\\_removed](#) (DDSDataWriter \*writer, const DDS\_Cookie\_t &cookie)  
*<<eXtension>> Called when a sample is removed from the DataWriter queue.*

### 7.96.1 Detailed Description

<<interface>> [DDSListener](#) for writer status.

Entity:

[DDSDataWriter](#)

Status:

[DDS\\_LIVELINESS\\_LOST\\_STATUS](#), [DDS\\_LivelinessLostStatus](#);  
[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#), [DDS\\_OfferedIncompatibleQosStatus](#);  
[DDS\\_PUBLICATION\\_MATCHED\\_STATUS](#), [DDS\\_PublicationMatchedStatus](#);

See also

[Status Kinds](#)

## 7.96.2 Member Function Documentation

### on\_offered\_deadline\_missed()

```
virtual void DDSDataWriterListener::on_offered_deadline_missed (
    DDSDataWriter * writer,
    const DDS_OfferedDeadlineMissedStatus & status) [inline], [virtual]
```

Handles the [DDS\\_OFFERED\\_DEADLINE\\_MISSED\\_STATUS](#) status.

This callback is called when the deadline that the [DDSDDataWriter](#) has committed through its [DEADLINE](#) qos policy was not respected for a specific instance. This callback is called for each deadline period elapsed during which the [DDSDDataWriter](#) failed to provide data for an instance.

Parameters

|               |                                                                                            |
|---------------|--------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback |
| <i>status</i> | <<out>> Current deadline missed status of locally created <a href="#">DDSDDataWriter</a>   |

### on\_liveliness\_lost()

```
virtual void DDSDataWriterListener::on_liveliness_lost (
    DDSDataWriter * writer,
    const DDS_LivelinessLostStatus & status) [inline], [virtual]
```

Handles the [DDS\\_LIVELINESS\\_LOST\\_STATUS](#) status.

This callback is called when the liveliness that the [DDSDDataWriter](#) has committed through its [LIVELINESS](#) qos policy was not respected; this [DDSDDataReader](#) entities will consider the [DDSDDataWriter](#) as no longer "alive/active". This callback will not be called when an already not alive [DDSDDataWriter](#) simply remains not alive for another liveliness period.

Parameters

|               |                                                                                            |
|---------------|--------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback |
| <i>status</i> | <<out>> Current liveliness lost status of locally created <a href="#">DDSDDataWriter</a>   |

**on\_offered\_incompatible\_qos()**

```
virtual void DDSDataWriterListener::on_offered_incompatible_qos (
    DDSDataWriter * writer,
    const DDS_OfferedIncompatibleQosStatus & status) [inline], [virtual]
```

Handles the [DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) status.

This callback is called when the [DDS\\_DataWriterQos](#) of the [DDSDataWriter](#) was incompatible with what was requested by a [DDSDDataReader](#). This callback is called when a [DDSDDataWriter](#) has discovered a [DDSDDataReader](#) for the same [DDSTopic](#) and common partition, but with a requested QoS that is incompatible with that offered by the [DDSDDataWriter](#).

**Parameters**

|               |                                                                                            |
|---------------|--------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback |
| <i>status</i> | <<out>> Current incompatible qos status of locally created <a href="#">DDSDDataWriter</a>  |

**on\_publication\_matched()**

```
virtual void DDSDataWriterListener::on_publication_matched (
    DDSDataWriter * writer,
    const DDS_PublicationMatchedException & status) [inline], [virtual]
```

Handles the [DDS\\_PUBLICATION\\_MATCHED\\_STATUS](#) status.

This callback is called when the [DDSDDataWriter](#) has found a [DDSDDataReader](#) that matches the [DDSTopic](#), has a common partition and compatible QoS, or has ceased to be matched with a [DDSDDataReader](#) that was previously considered to be matched.

**Parameters**

|               |                                                                                            |
|---------------|--------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback |
| <i>status</i> | <<out>> Current publication match status of locally created <a href="#">DDSDDataWriter</a> |

**on\_reliable\_reader\_activity\_changed()**

```
virtual void DDSDataWriterListener::on_reliable_reader_activity_changed (
    DDSDataWriter * writer,
    const DDS_ReliableReaderActivityChangedStatus & status) [inline], [virtual]
```

<<eXtension>> A matched reliable reader has become active or become inactive.

A reliable writer considers a matched reliable reader to be active when it receives ACKNACK messages from the reader in response to its HEARTBEAT messages. The writer considers an active reader to have become inactive when it has sent a number of consecutive periodic HEARTBEATs equal to [DDS\\_RtpsReliableWriterProtocol\\_t::max\\_heartbeat\\_retries](#) without receiving an ACKNACK from the reader. An inactive reader becomes active once the writer receives an ACKNACK from it.

This callback is called when the transition from active to inactive, and vice versa, has occurred for a matched reliable reader.

## Parameters

|               |                                                                                                           |
|---------------|-----------------------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback                |
| <i>status</i> | <<out>> Current reliable reader activity changed status of locally created <a href="#">DDSDDataWriter</a> |

**on\_reliable\_sample\_unacknowledged()**

```
virtual void DDSDataWriterListener::on_reliable_sample_unacknowledged (
    DDSDataWriter * writer,
    const DDS_ReliableSampleUnacknowledgedStatus & status) [inline], [virtual]
```

<<eXtension>> A matched reliable reader has become active or become inactive.

A reliable writer considers a matched reliable reader to be active when it receives ACKNACK messages from the reader in response to its HEARTBEAT messages. The writer considers an active reader to have become inactive when it has sent a number of consecutive periodic HEARTBEATs equal to [DDS\\_RtpsReliableWriterProtocol\\_t::max\\_heartbeat\\_retries](#) without receiving an ACKNACK from the reader. An inactive reader becomes active once the writer receives an ACKNACK from it.

This callback is called when the transition from active to inactive, and vice versa, has occurred for a matched reliable reader.

## Parameters

|               |                                                                                                           |
|---------------|-----------------------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback                |
| <i>status</i> | <<out>> Current reliable reader activity changed status of locally created <a href="#">DDSDDataWriter</a> |

**on\_sample\_removed()**

```
virtual void DDSDataWriterListener::on_sample_removed (
    DDSDataWriter * writer,
    const DDS_Cookie_t & cookie) [inline], [virtual]
```

<<eXtension>> Called when a sample is removed from the DataWriter queue.

Supported while using [Zero Copy Transfer Over Shared Memory](#) "Zero Copy transfer over shared memory" or [FlatData Topic-Types](#) "FlatData language binding".

## Parameters

|               |                                                                                                                         |
|---------------|-------------------------------------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback                              |
| <i>cookie</i> | <<out>> Contains the absolute address of the sample that is removed. The address of the sample can be obtained by using |

## See also

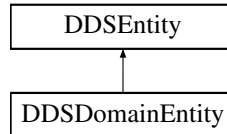
[FooDataWriter::get\\_loan](#)

## 7.97 DDSDomainEntity Class Reference

**<<interface>>** Abstract base class for all DDS entities except for the [DDSDomainParticipant](#).

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSDomainEntity:



### Additional Inherited Members

#### Public Member Functions inherited from [DDSEntity](#)

- [DDS\\_ReturnCode\\_t enable](#) ()  
*Enables the [DDSEntity](#).*
- [DDSStatusCondition \\* get\\_statuscondition](#) ()  
*Return the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).*
- [DDS\\_StatusMask get\\_status\\_changes](#) ()  
*Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.*
- [DDS\\_InstanceHandle\\_t get\\_instance\\_handle](#) ()  
*Allows access to the [DDS\\_InstanceHandle\\_t](#) associated with the [DDSEntity](#).*

### 7.97.1 Detailed Description

**<<interface>>** Abstract base class for all DDS entities except for the [DDSDomainParticipant](#).

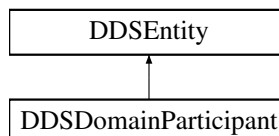
Its sole purpose is to *conceptually* express that [DDSDomainParticipant](#) is a special kind of [DDSEntity](#) that acts as a container of all other [DDSEntity](#) but itself cannot contain other [DDSDomainParticipant](#).

## 7.98 DDSDomainParticipant Class Reference

**<<interface>>** Container for all [DDSDomainEntity](#) objects.

```
#include <dds_cpp_domain.hxx>
```

Inheritance diagram for DDSDomainParticipant:





## Public Member Functions

- virtual [DDSEntity](#) \* [as\\_entity](#) ()=0  
*Access a [DDSDomainParticipant](#)'s supertype instance.*
- virtual [DDSPublisher](#) \* [create\\_publisher](#) (const [DDS\\_PublisherQos](#) &qos, [DDSPublisherListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
*Creates a [DDSPublisher](#) with the desired QoS policies and attaches to it the specified [DDSPublisherListener](#).*
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_publisher](#) ([DDSPublisher](#) \*p)=0  
*Deletes an existing [DDSPublisher](#).*
- virtual [DDSSubscriber](#) \* [create\\_subscriber](#) (const [DDS\\_SubscriberQos](#) &qos, [DDSSubscriberListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
*Creates a [DDSSubscriber](#) with the desired QoS policies and attaches to it the specified [DDSSubscriberListener](#).*
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_subscriber](#) ([DDSSubscriber](#) \*s)=0  
*Deletes an existing [DDSSubscriber](#).*
- virtual [DDSTopic](#) \* [create\\_topic](#) (const char \*topic\_name, const char \*type\_name, const [DDS\\_TopicQos](#) &qos, [DDSTopicListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
*Creates a [DDSTopic](#) with the desired QoS policies*
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_topic](#) ([DDSTopic](#) \*topic)=0  
*Deletes a [DDSTopic](#).*
- virtual [DDSTopic](#) \* [find\\_topic](#) (const char \*topic\_name, const [DDS\\_Duration\\_t](#) &timeout)=0  
*Finds an existing (or ready to exist) [DDSTopic](#), based on its name.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_qos](#) ([DDS\\_DomainParticipantQos](#) &qos)=0  
*Get the participant QoS.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_qos](#) (const [DDS\\_DomainParticipantQos](#) &qos)=0  
*Change the QoS of this domain participant.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_default\\_publisher\\_qos](#) ([DDS\\_PublisherQos](#) &qos)=0  
*Copy the default [DDS\\_PublisherQos](#) values into the provided [DDS\\_PublisherQos](#) instance.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_default\\_publisher\\_qos](#) (const [DDS\\_PublisherQos](#) &qos)=0  
*Set the default [DDS\\_PublisherQos](#) values for this domain participant.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_default\\_subscriber\\_qos](#) ([DDS\\_SubscriberQos](#) &qos)=0  
*Copy the default [DDS\\_SubscriberQos](#) values into the provided [DDS\\_SubscriberQos](#) instance.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_default\\_subscriber\\_qos](#) (const [DDS\\_SubscriberQos](#) &qos)=0  
*Set the default [DDS\\_SubscriberQos](#) values for this domain participant.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_default\\_topic\\_qos](#) ([DDS\\_TopicQos](#) &qos)=0  
*Copies the default [DDS\\_TopicQos](#) values for this domain participant into the given [DDS\\_TopicQos](#) instance.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_default\\_topic\\_qos](#) (const [DDS\\_TopicQos](#) &qos)=0  
*Set the default [DDS\\_TopicQos](#) values for this domain participant.*
- virtual [DDSTopicDescription](#) \* [lookup\\_topicdescription](#) (const char \*topic\_name)=0  
*Lookup an existing locally-created [DDSTopicDescription](#), based on its name.*
- virtual [DDSPublisher](#) \* [lookup\\_publisher\\_by\\_name](#) (const char \*publisher\_name)=0  
*Retrieves a [DDSPublisher](#) by its entity name within this [DDSDomainParticipant](#)*
- virtual [DDSSubscriber](#) \* [lookup\\_subscriber\\_by\\_name](#) (const char \*subscriber\_name)=0  
*Retrieves a [DDSSubscriber](#) by its entity name within this [DDSDomainParticipant](#)*
- virtual [DDSDataWriter](#) \* [lookup\\_datawriter\\_by\\_name](#) (const char \*datawriter\_full\_name)=0  
*Retrieves a [DDSDataWriter](#) by its entity name within this [DDSDomainParticipant](#)*
- virtual [DDSDataReader](#) \* [lookup\\_datareader\\_by\\_name](#) (const char \*datareader\_full\_name)=0  
*Retrieves a [DDSDataReader](#) by its entity name within this [DDSDomainParticipant](#)*
- virtual [DDS\\_ReturnCode\\_t](#) [assert\\_liveliness](#) ()=0

- Manually asserts the liveness of this [DDSDomainParticipant](#).*
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_contained\\_entities](#) ()=0

*Delete all the entities that were created by means of the "create" operations on the [DDSDomainParticipant](#).*
- virtual [DDS\\_ReturnCode\\_t](#) [register\\_type](#) (const char \*type\_name, [DDS\\_TypePluginI](#) \*plugin)=0

*<<eXtension>> Allows an application to communicate to RTI Connex DDS Micro the existence of a data type.*
- virtual [DDS\\_TypePluginI](#) \* [unregister\\_type](#) (const char \*type\_name)=0

*Allows an application to unregister a data type from RTI Connex DDS Micro. After calling [unregister\\_type](#), no further communication using that type is possible.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_listener](#) (const [DDSDomainParticipantListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0

*Sets the participant listener.*
- virtual [DDSDomainParticipantListener](#) \* [get\\_listener](#) ()=0

*Get the participant listener.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_discovered\\_participants](#) ([DDS\\_InstanceHandleSeq](#) &participant\_handles)=0

*Returns a sequence containing [DDS\\_InstanceHandle\\_t](#) for all the remote [DDSDomainParticipant](#) that have been discovered by the [DDSDomainParticipant](#).*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_discovered\\_participant\\_data](#) ([DDS\\_ParticipantBuiltinTopicData](#) &participant\_data, const [DDS\\_InstanceHandle\\_t](#) &participant\_handle)=0

*Returns information about a remote [DDSDomainParticipant](#) that has been discovered by the [DDSDomainParticipant](#).*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_current\\_time](#) ([DDS\\_Time\\_t](#) &current\_time)=0

*Returns the current value of the time.*
- virtual [DDS\\_ReturnCode\\_t](#) [add\\_peer](#) (const char \*peer)=0

*<<eXtension>> Attempt to contact one or more additional peer participants.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_default\\_flowcontroller\\_property](#) ([DDS\\_FlowControllerProperty\\_t](#) &prop)=0

*<<eXtension>> Copies the default [DDS\\_FlowControllerProperty\\_t](#) values for this domain participant into the given [DDS\\_FlowControllerProperty\\_t](#) instance.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_default\\_flowcontroller\\_property](#) (const [DDS\\_FlowControllerProperty\\_t](#) &prop)=0

*<<eXtension>> Set the default [DDS\\_FlowControllerProperty\\_t](#) values for this domain participant.*
- virtual [DDSFlowController](#) \* [create\\_flowcontroller](#) (const char \*name, const [DDS\\_FlowControllerProperty\\_t](#) &prop)=0

*<<eXtension>> Creates a [DDSFlowController](#) with the desired property.*
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_flowcontroller](#) ([DDSFlowController](#) \*fc)=0

*<<eXtension>> Deletes an existing [DDSFlowController](#).*
- virtual [DDSFlowController](#) \* [lookup\\_flowcontroller](#) (const char \*name)=0

*<<eXtension>> Looks up an existing locally-created [DDSFlowController](#), based on its name.*

### Public Member Functions inherited from [DDSEntity](#)

- [DDS\\_ReturnCode\\_t](#) [enable](#) ()

*Enables the [DDSEntity](#).*
- [DDSStatusCondition](#) \* [get\\_statuscondition](#) ()

*Return the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).*
- [DDS\\_StatusMask](#) [get\\_status\\_changes](#) ()

*Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.*
- [DDS\\_InstanceHandle\\_t](#) [get\\_instance\\_handle](#) ()

*Allows access to the [DDS\\_InstanceHandle\\_t](#) associated with the [DDSEntity](#).*

### 7.98.1 Detailed Description

<<*interface*>> Container for all [DDSDomainEntity](#) objects.

The DomainParticipant object plays several roles:

- It acts as a container for all other [DDSEntity](#) objects.
- It acts as *factory* for the [DDSPublisher](#), [DDSSubscriber](#), [DDSTopic](#) and [DDSEntity](#) objects.
- It represents the participation of the application on a communication plane that isolates applications running on the same set of physical computers from each other. A domain establishes a virtual network linking all applications that share the same `domainId` and isolating them from applications running on different domains. In this way, several independent distributed applications can coexist in the same physical network without interfering, or even being aware of each other.

The following operations may be called even if the [DDSDomainParticipant](#) is not enabled. Other operations will fail with the value [DDS\\_RETCODE\\_NOT\\_ENABLED](#) if called on a disabled DomainParticipant:

- Operations defined at the base-class level namely, [set\\_qos\(\)](#), [get\\_qos\(\)](#), [set\\_listener\(\)](#), [get\\_listener\(\)](#), and [enable\(\)](#).
- Factory operations: [create\\_topic\(\)](#), [create\\_publisher\(\)](#), [create\\_subscriber\(\)](#), [delete\\_topic\(\)](#), [delete\\_publisher\(\)](#), [delete\\_subscriber\(\)](#), [delete\\_contained\\_entities\(\)](#).

QoS:

[DDS\\_DomainParticipantQos](#)

Status:

[Status Kinds](#)

Listener:

[DDSDomainParticipantListener](#)

### 7.98.2 Member Function Documentation

#### **as\_entity()**

```
virtual DDSEntity * DDSDomainParticipant::as_entity () [pure virtual]
```

Access a [DDSDomainParticipant](#)'s supertype instance.

Returns

[DDSDomainParticipant](#)'s supertype [DDSEntity](#) instance

**create\_publisher()**

```
virtual DDSPublisher * DDSDomainParticipant::create_publisher (  
    const DDS_PublisherQos & qos,  
    DDSPublisherListener * listener,  
    DDS_StatusMask mask) [pure virtual]
```

Creates a [DDSPublisher](#) with the desired QoS policies and attaches to it the specified [DDSPublisherListener](#).

**Precondition**

The specified QoS policies must be consistent, or the operation will fail and no [DDSPublisher](#) will be created.

Parameters

|                 |                                                                                                                                                                                                                                                                                                                                    |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i>      | << <i>in</i> >> QoS to be used for creating the new <a href="#">DDSPublisher</a> . The special value <a href="#">DDS_PUBLISHER_QOS_DEFAULT</a> can be used to indicate that the <a href="#">DDSPublisher</a> should be created with the default <a href="#">DDS_PublisherQos</a> set in the <a href="#">DDSDomainParticipant</a> . |
| <i>listener</i> | << <i>in</i> >>. Listener to be attached to the newly created <a href="#">DDSPublisher</a> .                                                                                                                                                                                                                                       |
| <i>mask</i>     | << <i>in</i> >>. Changes of communication status to be invoked on the listener.                                                                                                                                                                                                                                                    |

Returns

newly created publisher object or NULL on failure.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation  
[DDS\\_PublisherQos](#) for rules on consistency among QoS  
[DDS\\_PUBLISHER\\_QOS\\_DEFAULT](#)

**delete\_publisher()**

```
virtual DDS\_ReturnCode\_t DDSDomainParticipant::delete_publisher (  
    DDSPublisher * p) [pure virtual]
```

Deletes an existing [DDSPublisher](#).

Precondition

The [DDSPublisher](#) must not have any attached [DDSDataWriter](#) objects. If there are existing [DDSDataWriter](#) objects, it will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).  
[DDSPublisher](#) must have been created by this [DDSDomainParticipant](#), or else it will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NO](#)

Postcondition

Listener installed on the [DDSPublisher](#) will not be called after this method completes successfully.

Parameters

|          |                                                             |
|----------|-------------------------------------------------------------|
| <i>p</i> | << <i>in</i> >> <a href="#">DDSPublisher</a> to be deleted. |
|----------|-------------------------------------------------------------|

Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**create\_subscriber()**

```
virtual DDSSubscriber * DDSDomainParticipant::create_subscriber (
    const DDS_SubscriberQos & qos,
    DDSSubscriberListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Creates a [DDSSubscriber](#) with the desired QoS policies and attaches to it the specified [DDSSubscriberListener](#).

**Precondition**

The specified QoS policies must be consistent, or the operation will fail and no [DDSSubscriber](#) will be created.

**MT Safety:**

UNSAFE. If [DDS\\_SUBSCRIBER\\_QOS\\_DEFAULT](#) is used for `qos`, it is not safe to create the publisher while another thread may be simultaneously calling [DDSDomainParticipant::set\\_default\\_subscriber\\_qos](#).

**Parameters**

|                 |                                                                                                                                                                                                                                                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i>      | <<in>> QoS to be used for creating the new <a href="#">DDSSubscriber</a> . The special value <a href="#">DDS_SUBSCRIBER_QOS_DEFAULT</a> can be used to indicate that the <a href="#">DDSSubscriber</a> should be created with the default <a href="#">DDS_SubscriberQos</a> set in the <a href="#">DDSDomainParticipant</a> . |
| <i>listener</i> | <<in>>. Listener to be attached to the newly created <a href="#">DDSSubscriber</a> .                                                                                                                                                                                                                                          |
| <i>mask</i>     | <<in>>. Changes of communication status to be invoked on the listener.                                                                                                                                                                                                                                                        |

**Returns**

newly created subscriber object or NULL on failure.

**See also**

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS\\_SubscriberQos](#) for rules on consistency among QoS

[DDS\\_SUBSCRIBER\\_QOS\\_DEFAULT](#)

**delete\_subscriber()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::delete_subscriber (
    DDSSubscriber * s) [pure virtual]
```

Deletes an existing [DDSSubscriber](#).

**Precondition**

The [DDSSubscriber](#) must not have any attached [DDSDataReader](#) objects. If there are existing [DDSDataReader](#) objects, it will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#)

The [DDSSubscriber](#) must have been created by this [DDSDomainParticipant](#), or else it will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**Postcondition**

A Listener installed on the [DDSSubscriber](#) will not be called after this method completes successfully.

## Parameters

|          |                                                              |
|----------|--------------------------------------------------------------|
| <i>s</i> | << <i>in</i> >> <a href="#">DDSSubscriber</a> to be deleted. |
|----------|--------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**create\_topic()**

```
virtual DDSTopic * DDSDomainParticipant::create_topic (
    const char * topic_name,
    const char * type_name,
    const DDS\_TopicQos & qos,
    DDSTopicListener * listener,
    DDS\_StatusMask mask) [pure virtual]
```

Creates a [DDSTopic](#) with the desired QoS policies

## Precondition

The application is not allowed to create two [DDSTopicDescription](#) objects with the same *topic\_name* attached to the same [DDSDomainParticipant](#). If the application attempts this, this method will fail and return a NULL topic.

The specified QoS policies must be consistent, or the operation will fail and no [DDSTopic](#) will be created.

Prior to creating a [DDSTopic](#), the type must have been registered with RTI Connex DDS Micro. This is done using the [DDSDomainParticipant::register\\_type](#) operation on a derived class of the [DDS\\_TypePluginI](#) interface.

## MT Safety:

UNSAFE. It is not safe to create a topic while another thread is trying to lookup that topic description with [DDSDomainParticipant::lookup\\_topicdescription](#).

## MT Safety:

UNSAFE. If [DDS\\_TOPIC\\_QOS\\_DEFAULT](#) is used for *qos*, it is not safe create the publisher while another thread may be simultaneously calling [DDSDomainParticipant::set\\_default\\_topic\\_qos](#).

## Parameters

|                   |                                                                                                                                                                                                                                                                                                                    |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>topic_name</i> | << <i>in</i> >> Name for the new topic, must not exceed 255 characters. Cannot be NULL.                                                                                                                                                                                                                            |
| <i>type_name</i>  | << <i>in</i> >> The type to which the new <a href="#">DDSTopic</a> will be bound. Cannot be NULL.                                                                                                                                                                                                                  |
| <i>qos</i>        | << <i>in</i> >> QoS to be used for creating the new <a href="#">DDSTopic</a> . The special value <a href="#">DDS_TOPIC_QOS_DEFAULT</a> can be used to indicate that the <a href="#">DDSTopic</a> should be created with the default <a href="#">DDS_TopicQos</a> set in the <a href="#">DDSDomainParticipant</a> . |
| <i>listener</i>   | << <i>in</i> >>. Listener to be attached to the newly created <a href="#">DDSTopic</a> .                                                                                                                                                                                                                           |
| <i>mask</i>       | << <i>in</i> >>. Changes of communication status to be invoked on the listener.                                                                                                                                                                                                                                    |

**Returns**

newly created topic, or NULL on failure

**See also**

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS\\_TopicQos](#) for rules on consistency among QoS

[DDS\\_TOPIC\\_QOS\\_DEFAULT](#)

**delete\_topic()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::delete_topic (
    DDSTopic * topic) [pure virtual]
```

Deletes a [DDSTopic](#).

**Precondition**

If the [DDSTopic](#) does not belong to the application's [DDSDomainParticipant](#), this operation fails with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

Make sure no objects are using the topic. More specifically, there must be no existing [DDSDataReader](#), or [DDSDataWriter](#), objects belonging to the same [DDSDomainParticipant](#) that are using the [DDSTopic](#). If `delete_topic` is called on a [DDSTopic](#) with any of these existing objects attached to it, it will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**Postcondition**

Listener installed on the [DDSTopic](#) will not be called after this method completes successfully.

**Parameters**

|              |                                                         |
|--------------|---------------------------------------------------------|
| <i>topic</i> | << <i>in</i> >> <a href="#">DDSTopic</a> to be deleted. |
|--------------|---------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#)

**find\_topic()**

```
virtual DDSTopic * DDSDomainParticipant::find_topic (
    const char * topic_name,
    const DDS_Duration_t & timeout) [pure virtual]
```

Finds an existing (or ready to exist) [DDSTopic](#), based on its name.

If a [DDSTopic](#) of the same name already exists, it gives access to it, otherwise it waits until either another mechanism creates it, or else completes when the specified `timeout` occurs.

[DDSTopic](#) obtained by [DDSDomainParticipant::find\\_topic](#) must also be deleted by means of [DDSDomainParticipant::delete\\_topic](#). If [DDSTopic](#) is obtained multiple times by means of [DDSDomainParticipant::find\\_topic](#) or [DDSDomainParticipant::create\\_topic](#), it must also be deleted that same number of times using [DDSDomainParticipant::delete\\_topic](#)



**Parameters**

|                   |                                                                                         |
|-------------------|-----------------------------------------------------------------------------------------|
| <i>topic_name</i> | << <i>in</i> >> Name of the <a href="#">DDSTopic</a> to search for. Cannot be NULL.     |
| <i>timeout</i>    | << <i>in</i> >> The time to wait if the <a href="#">DDSTopic</a> doesn't exist already. |

**Returns**

the topic, if it exists, or NULL

**get\_qos()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_qos (  
    DDS_DomainParticipantQos & qos) [pure virtual]
```

Get the participant QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

**Parameters**

|            |                                         |
|------------|-----------------------------------------|
| <i>qos</i> | << <i>inout</i> >> QoS to be filled up. |
|------------|-----------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**See also**

[get\\_qos \(abstract\)](#)

**set\_qos()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_qos (  
    const DDS_DomainParticipantQos & qos) [pure virtual]
```

Change the QoS of this domain participant.

The [DDS\\_DomainParticipantQos::entity\\_factory](#) can be changed. The other policies are immutable.

## Parameters

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | <a href="#">&lt;&lt;in&gt;&gt;</a> Set of policies to be applied to <a href="#">DDSDomainParticipant</a> . Policies must be consistent. Immutable policies cannot be changed after <a href="#">DDSDomainParticipant</a> is enabled. The special value <a href="#">DDS_PARTICIPANT_QOS_DEFAULT</a> can be used to indicate that the QoS of the <a href="#">DDSDomainParticipant</a> should be changed to match the current default <a href="#">DDS_DomainParticipantQos</a> set in the <a href="#">DDSDomainParticipantFactory</a> . |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#) if immutable policy is changed, or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#) if policies are inconsistent

## See also

[DDS\\_DomainParticipantQos](#) for rules on consistency among QoS  
[set\\_qos](#) (abstract)

**get\_default\_publisher\_qos()**

```
virtual DDS\_ReturnCode\_t DDSDomainParticipant::get_default_publisher_qos (
    DDS\_PublisherQos & qos) [pure virtual]
```

Copy the default [DDS\\_PublisherQos](#) values into the provided [DDS\\_PublisherQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSDomainParticipant::set\\_default\\_publisher\\_qos](#), or else, if the call was never made, the default values listed in [DDS\\_PublisherQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

If [DDS\\_PUBLISHER\\_QOS\\_DEFAULT](#) is specified as the `qos` parameter when [DDSDomainParticipant::create\\_topic](#) is called, the default value of the QoS set in the factory, equivalent to the value obtained by calling [DDSDomainParticipant::get\\_default\\_publisher\\_qos](#), will be used to create the [DDSPublisher](#).

## MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::set\\_default\\_publisher\\_qos](#)

## Parameters

|            |                                                            |
|------------|------------------------------------------------------------|
| <i>qos</i> | <a href="#">&lt;&lt;inout&gt;&gt;</a> Qos to be filled up. |
|------------|------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#)

## See also

[DDS\\_PUBLISHER\\_QOS\\_DEFAULT](#)  
[DDSDomainParticipant::create\\_publisher](#)

## set\_default\_publisher\_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_publisher_qos (
    const DDS_PublisherQos & qos) [pure virtual]
```

Set the default [DDS\\_PublisherQos](#) values for this domain participant.

This default value will be used for newly created [DDSPublisher](#) if [DDS\\_PUBLISHER\\_QOS\\_DEFAULT](#) is specified as the `qos` parameter when [DDSDomainParticipant::create\\_publisher](#) is called.

### Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

### MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::get\\_default\\_publisher\\_qos](#) or calling [DDSDomainParticipant::create\\_publisher](#) with [DDS\\_PUBLISHER\\_QOS\\_DEFAULT](#) as the `qos` parameter.

### Parameters

|            |                                                                                                                                                                                                                                                                                                                                  |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> Default qos to be set. The special value <a href="#">DDS_PUBLISHER_QOS_DEFAULT</a> may be passed as <code>qos</code> to indicate that the default QoS should be reset back to the initial values the factory would use if <a href="#">DDSDomainParticipant::set_default_publisher_qos</a> had never been called. |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

### See also

[DDS\\_PUBLISHER\\_QOS\\_DEFAULT](#)  
[DDSDomainParticipant::create\\_publisher](#)

## get\_default\_subscriber\_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_subscriber_qos (
    DDS_SubscriberQos & qos) [pure virtual]
```

Copy the default [DDS\\_SubscriberQos](#) values into the provided [DDS\\_SubscriberQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSDomainParticipant::set\\_default\\_subscriber\\_qos](#), or else, if the call was never made, the default values listed in [DDS\\_SubscriberQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

If [DDS\\_SUBSCRIBER\\_QOS\\_DEFAULT](#) is specified as the `qos` parameter when [DDSDomainParticipant::create\\_subscriber](#) is called, the default value of the QoS set in the factory, equivalent to the value obtained by calling [DDSDomainParticipant::get\\_default\\_subscriber\\_qos](#) will be used to create the [DDSSubscriber](#).

### MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::set\\_default\\_subscriber\\_qos](#)

## Parameters

|            |                                         |
|------------|-----------------------------------------|
| <i>qos</i> | << <i>inout</i> >> Qos to be filled up. |
|------------|-----------------------------------------|

## Returns

One of the [Standard Return Codes](#)

## See also

[DDS\\_SUBSCRIBER\\_QOS\\_DEFAULT](#)

[DDSDomainParticipant::create\\_subscriber](#)

**set\_default\_subscriber\_qos()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_subscriber_qos (
    const DDS_SubscriberQos & qos) [pure virtual]
```

Set the default [DDS\\_SubscriberQos](#) values for this domain participant.

This default value will be used for newly created [DDSSubscriber](#) if [DDS\\_SUBSCRIBER\\_QOS\\_DEFAULT](#) is specified as the *qos* parameter when [DDSDomainParticipant::create\\_subscriber](#) is called.

## Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

## MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::get\\_default\\_subscriber\\_qos](#) or calling [DDSDomainParticipant::create\\_subscriber](#) with [DDS\\_SUBSCRIBER\\_QOS\\_DEFAULT](#) as the *qos* parameter.

## Parameters

|            |                                                                                                                                                                                                                                                                                                                              |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> Default qos to be set. The special value <a href="#">DDS_SUBSCRIBER_QOS_DEFAULT</a> may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would use if <a href="#">DDSDomainParticipant::set_default_subscriber_qos</a> had never been called. |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

### get\_default\_topic\_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_topic_qos (  
    DDS_TopicQos & qos) [pure virtual]
```

Copies the default [DDS\\_TopicQos](#) values for this domain participant into the given [DDS\\_TopicQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSDomainParticipant::set\\_default\\_topic\\_qos](#), or else, if the call was never made, the default values listed in [DDS\\_TopicQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

#### MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::get\\_default\\_topic\\_qos](#) or calling [DDSDomainParticipant::create\\_topic](#) with [DDS\\_TOPIC\\_QOS\\_DEFAULT](#) as the `qos` parameter.

#### Parameters

|            |                                              |
|------------|----------------------------------------------|
| <i>qos</i> | << <i>in</i> >> Default qos to be retrieved. |
|------------|----------------------------------------------|

#### Returns

One of the [Standard Return Codes](#)

#### See also

[DDS\\_TOPIC\\_QOS\\_DEFAULT](#)  
[DDSDomainParticipant::create\\_topic](#)

### set\_default\_topic\_qos()

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_topic_qos (  
    const DDS_TopicQos & qos) [pure virtual]
```

Set the default [DDS\\_TopicQos](#) values for this domain participant.

This default value will be used for newly created [DDSTopic](#) if [DDS\\_TOPIC\\_QOS\\_DEFAULT](#) is specified as the `qos` parameter when [DDSDomainParticipant::create\\_topic](#) is called.

#### Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

#### MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::get\\_default\\_topic\\_qos](#) or calling [DDSDomainParticipant::create\\_topic](#) with [DDS\\_TOPIC\\_QOS\\_DEFAULT](#) as the `qos` parameter.

## Parameters

|            |                                                                                                                                                                                                                                                                                                                          |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> Default qos to be set. The special value <a href="#">DDS_TOPIC_QOS_DEFAULT</a> may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would have used if <a href="#">DDSDomainParticipant::set_default_topic_qos</a> had never been called. |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

## See also

[DDS\\_TOPIC\\_QOS\\_DEFAULT](#)  
[DDSDomainParticipant::create\\_topic](#)

**lookup\_topicdescription()**

```
virtual DDSTopicDescription * DDSDomainParticipant::lookup_topicdescription (
    const char * topic_name) [pure virtual]
```

Lookup an existing locally-created [DDSTopicDescription](#), based on its name.

The returned topic can either be enabled or disabled.

The operation [DDSDomainParticipant::lookup\\_topicdescription](#) searches among the locally created topics and returns a pointer to a [DDSTopicDescription](#). It is possible to delete the [DDSTopicDescription](#) returned by [DDSDomainParticipant::lookup\\_topicdescription](#), provided it has no readers or writers, but then it is really deleted and subsequent lookups will fail.

## MT Safety:

UNSAFE. It is not safe to lookup a topic description while another thread is creating that topic.

## Parameters

|                   |                                                                                                                                             |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <i>topic_name</i> | << <i>in</i> >> Name of <a href="#">DDSTopicDescription</a> to search for. This string must be no more than 255 characters. Cannot be NULL. |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

the topic description if it has already been created locally, or NULL otherwise.

**lookup\_publisher\_by\_name()**

```
virtual DDSPublisher * DDSDomainParticipant::lookup_publisher_by_name (
    const char * publisher_name) [pure virtual]
```

Retrieves a [DDSPublisher](#) by its entity name within this [DDSDomainParticipant](#)

This returned [DDSPublisher](#) is either enabled or disabled.

Every [DDSPublisher](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY\_NAME.

This operation retrieves a [DDSPublisher](#) within the [DDSDomainParticipant](#) given the entity's name. If there are several [DDSPublisher](#) with the same name within the [DDSDomainParticipant](#), this function returns the first matching occurrence.

**Parameters**

|                       |                                                                          |
|-----------------------|--------------------------------------------------------------------------|
| <i>publisher_name</i> | <<in>> Entity name of the <a href="#">DDSPublisher</a> . Cannot be NULL. |
|-----------------------|--------------------------------------------------------------------------|

**Returns**

The first [DDSPublisher](#) found with the specified name or NULL if it is not found.

**MT Safety:**

UNSAFE. It is not safe to lookup a [DDSPublisher](#) in one thread while another thread is simultaneously creating or destroying that [DDSPublisher](#).

**lookup\_subscriber\_by\_name()**

```
virtual DDSSubscriber * DDSDomainParticipant::lookup_subscriber_by_name (  
    const char * subscriber_name) [pure virtual]
```

Retrieves a [DDSSubscriber](#) by its entity name within this [DDSDomainParticipant](#)

This returned [DDSSubscriber](#) is either enabled or disabled.

Every [DDSSubscriber](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY\_NAME.

This operation retrieves a [DDSSubscriber](#) within the [DDSDomainParticipant](#) given the entity's name. If there are several [DDSSubscriber](#) with the same name within the [DDSDomainParticipant](#), this function returns the first matching occurrence.

**Parameters**

|                        |                                                                           |
|------------------------|---------------------------------------------------------------------------|
| <i>subscriber_name</i> | <<in>> Entity name of the <a href="#">DDSSubscriber</a> . Cannot be NULL. |
|------------------------|---------------------------------------------------------------------------|

**Returns**

The first [DDSSubscriber](#) found with the specified name or NULL if it is not found.

**MT Safety:**

UNSAFE. It is not safe to lookup a [DDSSubscriber](#) in one thread while another thread is simultaneously creating or destroying that [DDSSubscriber](#).

**lookup\_datawriter\_by\_name()**

```
virtual DDSDataWriter * DDSDomainParticipant::lookup_datawriter_by_name (  
    const char * datawriter_full_name) [pure virtual]
```

Retrieves a [DDSDataWriter](#) by its entity name within this [DDSDomainParticipant](#)

This returned [DDSDataWriter](#) is either enabled or disabled.

Every [DDSDataWriter](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY\_NAME.

This operation retrieves a [DDSDataWriter](#) within the [DDSDomainParticipant](#) given the entity's name. If there are several [DDSDataWriter](#) with the same name within the [DDSDomainParticipant](#), this function returns the first matching occurrence.

## Parameters

|                             |                                                                           |
|-----------------------------|---------------------------------------------------------------------------|
| <i>datawriter_full_name</i> | <<in>> Entity name of the <a href="#">DDSDataWriter</a> . Cannot be NULL. |
|-----------------------------|---------------------------------------------------------------------------|

## Returns

The first [DDSDataWriter](#) found with the specified name or NULL if it is not found.

## MT Safety:

UNSAFE. It is not safe to lookup a [DDSDataWriter](#) in one thread while another thread is simultaneously creating or destroying that [DDSDataWriter](#).

**lookup\_datareader\_by\_name()**

```
virtual DDSDataReader * DDSDomainParticipant::lookup_datareader_by_name (
    const char * datareader_full_name) [pure virtual]
```

Retrieves a [DDSDataReader](#) by its entity name within this [DDSDomainParticipant](#)

This returned [DDSDataReader](#) is either enabled or disabled.

Every [DDSDataReader](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY\_NAME.

This operation retrieves a [DDSDataReader](#) within the [DDSDomainParticipant](#) given the entity's name. If there are several [DDSDataReader](#) with the same name within the [DDSDomainParticipant](#), this function returns the first matching occurrence.

## Parameters

|                             |                                                                           |
|-----------------------------|---------------------------------------------------------------------------|
| <i>datareader_full_name</i> | <<in>> Entity name of the <a href="#">DDSDataReader</a> . Cannot be NULL. |
|-----------------------------|---------------------------------------------------------------------------|

## Returns

The first [DDSDataReader](#) found with the specified name or NULL if it is not found.

## MT Safety:

UNSAFE. It is not safe to lookup a [DDSDataReader](#) in one thread while another thread is simultaneously creating or destroying that [DDSDataReader](#).



**assert\_liveliness()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::assert_liveliness () [pure virtual]
```

Manually asserts the liveliness of this [DDSDomainParticipant](#).

This is used in combination with the [DDS\\_LivelinessQosPolicy](#) to indicate to RTI Connex DDS Micro that the entity remains active.

You need to use this operation if the [DDSDomainParticipant](#) contains [DDSDataWriter](#) entities with the [DDS\\_LivelinessQosPolicy::kind](#) set to [DDS\\_MANUAL\\_BY\\_PARTICIPANT\\_LIVELINESS\\_QOS](#) and it only affects the liveliness of those [DDSDataWriter](#) entities. Otherwise, it has no effect.

**Note:** writing data via the [FooDataWriter::write](#) or [FooDataWriter::write\\_w\\_timestamp](#) operation asserts liveliness on the [DDSDataWriter](#) itself and its [DDSDomainParticipant](#). Consequently the use of [assert\\_liveliness\(\)](#) is only needed if the application is not writing data regularly.

**Returns**

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_NOT\\_ENABLED](#)

**See also**

[DDS\\_LivelinessQosPolicy](#)

**delete\_contained\_entities()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::delete_contained_entities () [pure virtual]
```

Delete all the entities that were created by means of the "create" operations on the [DDSDomainParticipant](#).

This operation deletes all contained [DDSPublisher](#), [DDSSubscriber](#), and [DDSTopic](#).

Prior to deleting each contained entity, this operation will recursively call the corresponding [delete\\_contained\\_entities](#) operation on each contained entity (if applicable). This pattern is applied recursively. In this manner the operation [delete\\_contained\\_entities\(\)](#) on the [DDSDomainParticipant](#) will end up deleting all the entities recursively contained in the [DDSDomainParticipant](#), that is also the [DDSDataWriter](#), and [DDSDataReader](#).

The operation will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) if any of the contained entities is in a state where it cannot be deleted.

If [delete\\_contained\\_entities\(\)](#) completes successfully, the application may delete the [DDSDomainParticipant](#) knowing that it has no contained entities.

**Returns**

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**register\_type()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::register_type (
    const char * type_name,
    DDS_TypePluginI * plugin) [pure virtual]
```

<<*eXtension*>> Allows an application to communicate to RTI Connex DDS Micro the existence of a data type.

The *generated* implementation of the operation embeds all the knowledge that has to be communicated to the middle-ware in order to make it able to manage the contents of data of that type. This includes in particular the key definition that will allow RTI Connex DDS Micro to distinguish different instances of the same type.

The same [DDS\\_TypePluginI](#) can be registered multiple times with a [DDSDomainParticipant](#) using the same or different values for the `type_name`. If `register_type` is called multiple times on the same [DDS\\_TypePluginI](#) with the same [DDSDomainParticipant](#) and `type_name`, the second (and subsequent) registrations are ignored by the operation fails with [DDS\\_RETCODE\\_OK](#).

**Precondition**

Cannot use the same `type_name` to register two different [DDS\\_TypePluginI](#) with the same [DDSDomainParticipant](#), or else the operation will fail and [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) will be returned.

**Parameters**

|                  |                                                                                                                                                                                                                                                                                                      |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>type_name</i> | << <i>in</i> >> the type name under with the data type <code>Foo</code> is registered with the participant; this type name is used when creating a new <a href="#">DDSTopic</a> . (See <a href="#">DDSDomainParticipant::create_topic</a> .) The name may not be NULL or longer than 255 characters. |
| <i>plugin</i>    | << <i>in</i> >> the <code>Foo</code> type plugin to register the data type with. Cannot be NULL.                                                                                                                                                                                                     |

**Returns**

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) or [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#).

**See also**

[DDSDomainParticipant::create\\_topic](#)

**unregister\_type()**

```
virtual DDS_TypePluginI * DDSDomainParticipant::unregister_type (
    const char * type_name) [pure virtual]
```

Allows an application to unregister a data type from RTI Connex DDS Micro. After calling `unregister_type`, no further communication using that type is possible.

The *generated* implementation of the operation removes all the information about a type from RTI Connex DDS Micro. No further communication using that type is possible.

**Precondition**

A type with `type_name` is registered with the participant and all [DDSTopic](#) objects referencing the type have been destroyed. If the type is not registered with the participant, or if any [DDSTopic](#) is associated with the type, the operation will fail with [DDS\\_RETCODE\\_ERROR](#).

**Postcondition**

All information about the type is removed from RTI Connex DDS Micro. No further communication using this type is possible.

#### Parameters

|                  |                                                                                                                                                                                                                                 |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>type_name</i> | << <i>in</i> >> the type name under with the data type <code>Foo</code> is registered with the participant. The name should match a name that has been previously used to register a type with the participant. Cannot be NULL. |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_BAD_PARAMETER` or `DDS_RETCODE_ERROR`

#### See also

[DDSDomainParticipant::register\\_type](#)

### **set\_listener()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_listener (  
    const DDSDomainParticipantListener * listener,  
    DDS_StatusMask mask) [pure virtual]
```

Sets the participant listener.

#### Parameters

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>listener</i> | << <i>in</i> >> Listener to be installed on entity.                            |
| <i>mask</i>     | << <i>in</i> >> Changes of communication status to be invoked on the listener. |

#### MT Safety:

Unsafe. This method is not synchronized with the listener callbacks, so it is possible to set a new listener on a participant when the old listener is in a callback.

Care should therefore be taken not to delete any listener that has been set on an enabled participant unless some application-specific means are available of ensuring that the old listener cannot still be in use.

#### Returns

One of the [Standard Return Codes](#)

#### See also

[set\\_listener \(abstract\)](#)

**get\_listener()**

```
virtual DDSDomainParticipantListener * DDSDomainParticipant::get_listener () [pure virtual]
```

Get the participant listener.

**Returns**

Existing listener attached to the [DDSDomainParticipant](#).

**get\_discovered\_participants()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_discovered_participants (
    DDS_InstanceHandleSeq & participant_handles) [pure virtual]
```

Returns a sequence containing [DDS\\_InstanceHandle\\_t](#) for all the remote [DDSDomainParticipant](#) that have been discovered by the [DDSDomainParticipant](#).

**Parameters**

|                            |                                                                                           |
|----------------------------|-------------------------------------------------------------------------------------------|
| <i>participant_handles</i> | <<out>> sequence of <a href="#">DDS_InstanceHandle_t</a> where the result will be stored. |
|----------------------------|-------------------------------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**get\_discovered\_participant\_data()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_discovered_participant_data (
    DDS_ParticipantBuiltinTopicData & participant_data,
    const DDS_InstanceHandle_t & participant_handle) [pure virtual]
```

Returns information about a remote [DDSDomainParticipant](#) that has been discovered by the [DDSDomainParticipant](#).

**Parameters**

|                           |                                                                                                           |
|---------------------------|-----------------------------------------------------------------------------------------------------------|
| <i>participant_data</i>   | <<out>> information about the remote <a href="#">DDSDomainParticipant</a> .                               |
| <i>participant_handle</i> | <<in>> <a href="#">DDS_InstanceHandle_t</a> identifying the remote <a href="#">DDSDomainParticipant</a> . |

**Returns**

One of the [Standard Return Codes](#)

**get\_current\_time()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_current_time (
    DDS_Time_t & current_time) [pure virtual]
```

Returns the current value of the time.

The current value of the time that RTI Connex DDS Micro uses to time-stamp [DDSDataWriter](#) and to set the reception-timestamp for the data updates that it receives.

**Parameters**

|                     |                                                  |
|---------------------|--------------------------------------------------|
| <i>current_time</i> | << <i>inout</i> >> Current time to be filled up. |
|---------------------|--------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**add\_peer()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::add_peer (  
    const char * peer) [pure virtual]
```

<<*eXtension*>> Attempt to contact one or more additional peer participants.

Add the given peer description to the list of peers with which this [DDSDomainParticipant](#) will try to communicate.

This method may be called at any time after this [DDSDomainParticipant](#) has been created, before or after it has been enabled. If it is called after [DDSEntity::enable](#), an attempt will be made to contact the new peer(s) immediately. If it is called before, the peer description will simply be added to the list that was populated by [DDS\\_DiscoveryQosPolicy::initial\\_peers](#); the first attempted contact will take place after this [DDSDomainParticipant](#) is enabled.

Adding a peer description with this method does not guarantee that any peer(s) discovered as a result will exactly correspond to those described:

- This [DDSDomainParticipant](#) will attempt to discover peer participants at the given locations but may not succeed if no such participants are available. Such a situation will not result in an error result from this method, which will not wait for contact attempt(s) to be made.
- If remote participants such as are described by the given peer description are discovered, the distributed application is configured with asymmetric peer lists, and [DDS\\_DiscoveryQosPolicy::accept\\_unknown\\_peers](#) is set to [DDS\\_BOOLEAN\\_TRUE](#) this [DDSDomainParticipant](#) may actually discover *more* peers than are described in the given peer description.

Adding a peer description with this method has no effect on the [DDS\\_DiscoveryQosPolicy::initial\\_peers](#).

**Parameters**

|             |                                                  |
|-------------|--------------------------------------------------|
| <i>peer</i> | << <i>in</i> >> New peer descriptor to be added. |
|-------------|--------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**See also**

[DDS\\_DiscoveryQosPolicy::initial\\_peers](#)  
[::DDS\\_PARTICIPANT\\_TOPIC\\_NAME](#)

**get\_default\_flowcontroller\_property()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_flowcontroller_property (
    DDS_FlowControllerProperty_t & prop) [pure virtual]
```

<<*eXtension*>> Copies the default [DDS\\_FlowControllerProperty\\_t](#) values for this domain participant into the given [DDS\\_FlowControllerProperty\\_t](#) instance.

The retrieved `property` will match the set of values specified on the last successful call to [DDSDomainParticipant::set\\_default\\_flowcontroller\\_property](#) or else, if the call was never made, the default values listed in [DDS\\_FlowControllerProperty\\_t](#).

**MT Safety:**

UNSAFE. It is not safe to retrieve the default flow controller properties from a `DomainParticipant` while another thread may be simultaneously calling [DDSDomainParticipant::set\\_default\\_flowcontroller\\_property](#)

**Parameters**

|             |                                                   |
|-------------|---------------------------------------------------|
| <i>prop</i> | << <i>in</i> >> Default property to be retrieved. |
|-------------|---------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**See also**

[DDS\\_FLOW\\_CONTROLLER\\_PROPERTY\\_DEFAULT](#)  
[DDSDomainParticipant::create\\_flowcontroller](#)

**set\_default\_flowcontroller\_property()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_flowcontroller_property (
    const DDS_FlowControllerProperty_t & prop) [pure virtual]
```

<<*eXtension*>> Set the default [DDS\\_FlowControllerProperty\\_t](#) values for this domain participant.

This default value will be used for newly created [DDSFlowController](#) if [DDS\\_FLOW\\_CONTROLLER\\_PROPERTY\\_DEFAULT](#) is specified as the `property` parameter when [DDSDomainParticipant::create\\_flowcontroller](#) is called.

**Precondition**

The specified property values must be consistent, or else the operation will have no effect and fail with [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

**MT Safety:**

UNSAFE. It is not safe to set the default flow controller properties for a `DomainParticipant` while another thread may be simultaneously calling [DDSDomainParticipant::set\\_default\\_flowcontroller\\_property](#), [DDSDomainParticipant::get\\_default\\_flowcontroller\\_property](#) or calling [DDSDomainParticipant::create\\_flowcontroller](#) with [DDS\\_FLOW\\_CONTROLLER\\_PROPERTY\\_DEFAULT](#) as the `qos` parameter.

## Parameters

|             |                                                                                                                                                                                                                                                                                                                                                           |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>prop</i> | << <i>in</i> >> Default property to be set. The special value <a href="#">DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT</a> may be passed as <i>property</i> to indicate that the default property should be reset to the default values the factory would use if <a href="#">DDSDomainParticipant::set_default_flowcontroller_property</a> had never been called. |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

## See also

[DDS\\_FLOW\\_CONTROLLER\\_PROPERTY\\_DEFAULT](#)  
[DDSDomainParticipant::create\\_flowcontroller](#)

**create\_flowcontroller()**

```
virtual DDSFlowController * DDSDomainParticipant::create_flowcontroller (
    const char * name,
    const DDS\_FlowControllerProperty\_t & prop) [pure virtual]
```

<<*eXtension*>> Creates a [DDSFlowController](#) with the desired property.

## MT Safety:

UNSAFE. If [DDS\\_FLOW\\_CONTROLLER\\_PROPERTY\\_DEFAULT](#) is used for *property*, it is not safe to create the flow controller while another thread may be simultaneously calling [DDSDomainParticipant::set\\_default\\_flowcontroller\\_property](#) or trying to lookup that flow controller with [DDSDomainParticipant::lookup\\_flowcontroller](#).

## Parameters

|             |                                                                                                                                                                                                                                                                                                                                                                          |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>name</i> | << <i>in</i> >> name of the <a href="#">DDSFlowController</a> to create. A <a href="#">DDSDataWriter</a> is associated with a <a href="#">DDSFlowController</a> by name. Limited to 255 characters.                                                                                                                                                                      |
| <i>prop</i> | << <i>in</i> >> property to be used for creating the new <a href="#">DDSFlowController</a> . The special value <a href="#">DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT</a> can be used to indicate that the <a href="#">DDSFlowController</a> should be created with the default <a href="#">DDS_FlowControllerProperty_t</a> set in the <a href="#">DDSDomainParticipant</a> . |

## Returns

Newly created flow controller object or NULL on failure.

## See also

[DDS\\_FlowControllerProperty\\_t](#) for rules on consistency among property  
[DDS\\_FLOW\\_CONTROLLER\\_PROPERTY\\_DEFAULT](#)  
[DDSDomainParticipant::get\\_default\\_flowcontroller\\_property](#)

The created [DDSFlowController](#) is associated with a [DDSDataWriter](#) via [DDS\\_PublishModeQosPolicy::flow\\_controller\\_name](#). A single FlowController may service multiple DataWriters instances, even if they belong to a different [DDSPublisher](#). The *property* determines how the FlowController shapes the network traffic.

## Precondition

The specified *property* must be consistent, or the operation will fail and no [DDSFlowController](#) will be created.

**delete\_flowcontroller()**

```
virtual DDS_ReturnCode_t DDSDomainParticipant::delete_flowcontroller (
    DDSFlowController * fc) [pure virtual]
```

<<*eXtension*>> Deletes an existing [DDSFlowController](#).

**Precondition**

The [DDSFlowController](#) must not have any attached [DDSDataWriter](#) objects. If there are any attached [DDSDataWriter](#) objects, it will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

The [DDSFlowController](#) must have been created by this [DDSDomainParticipant](#), or else it will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**Postcondition**

The [DDSFlowController](#) is deleted if this method completes successfully.

**MT Safety:**

UNSAFE. It is not safe to delete an entity while another thread may be simultaneously calling an API that uses the entity.

**Parameters**

|           |                                                                      |
|-----------|----------------------------------------------------------------------|
| <i>fc</i> | << <i>in</i> >> The <a href="#">DDSFlowController</a> to be deleted. |
|-----------|----------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**lookup\_flowcontroller()**

```
virtual DDSFlowController * DDSDomainParticipant::lookup_flowcontroller (
    const char * name) [pure virtual]
```

<<*eXtension*>> Looks up an existing locally-created [DDSFlowController](#), based on its name.

Looks up a previously created [DDSFlowController](#), including the built-in ones. Once a [DDSFlowController](#) has been deleted, subsequent lookups will fail.

**MT Safety:**

UNSAFE. It is not safe to lookup a flow controller description while another thread is creating that flow controller.



## Parameters

|             |                                                                                                                     |
|-------------|---------------------------------------------------------------------------------------------------------------------|
| <i>name</i> | << <i>in</i> >> Name of <a href="#">DDSFlowController</a> to search for. Limited to 255 characters. Cannot be NULL. |
|-------------|---------------------------------------------------------------------------------------------------------------------|

## Returns

The flow controller if it has already been created locally, or NULL otherwise.

## 7.99 DDSDomainParticipantFactory Class Reference

<<*singleton*>> <<*interface*>> Allows creation and destruction of [DDSDomainParticipant](#) objects.

```
#include <dds_cpp_domain.hxx>
```

### Public Member Functions

- virtual [DDSDomainParticipant](#) \* [create\\_participant](#) ([DDS\\_DomainId\\_t](#) domainId, const [DDS\\_DomainParticipantQos](#) &qos, [DDSDomainParticipantListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
*Creates a new [DDSDomainParticipant](#) object.*
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_participant](#) ([DDSDomainParticipant](#) \*a\_participant)=0  
*Deletes an existing [DDSDomainParticipant](#).*
- virtual [DDSDomainParticipant](#) \* [create\\_participant\\_from\\_config](#) (const char \*configuration\_name)=0  
*Creates a new [DDSDomainParticipant](#) given its configuration name from a description provided in configuration file.*
- virtual [DDSDomainParticipant](#) \* [lookup\\_participant](#) ([DDS\\_DomainId\\_t](#) domainId)=0  
*Locates an existing [DDSDomainParticipant](#).*
- virtual [DDSDomainParticipant](#) \* [lookup\\_participant\\_by\\_name](#) (const char \*participant\_name)=0  
*Retrieves a [DDSDomainParticipant](#) by its entity name in the within the [DDSDomainParticipantFactory](#)*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_default\\_participant\\_qos](#) (const [DDS\\_DomainParticipantQos](#) &qos)=0  
*Sets the default [DDS\\_DomainParticipantQos](#) values for this domain participant factory.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_default\\_participant\\_qos](#) ([DDS\\_DomainParticipantQos](#) &qos)=0  
*Initializes the [DDS\\_DomainParticipantQos](#) instance with default values.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_qos](#) ([DDS\\_DomainParticipantFactoryQos](#) &qos)=0  
*Gets the value for participant factory QoS.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_qos](#) (const [DDS\\_DomainParticipantFactoryQos](#) &qos)=0  
*Sets the value for a participant factory QoS.*
- virtual [RTRegistry](#) \* [get\\_registry](#) ()=0  
*Retrieves the singleton registry instance associated with the factory.*

### Static Public Member Functions

- static [DDSDomainParticipantFactory](#) \* [get\\_instance](#) ()  
*Gets the singleton instance of this class.*
- static [DDS\\_ReturnCode\\_t](#) [finalize\\_instance](#) ()  
*<<eXtension>> Destroys the singleton instance of this class.*

### 7.99.1 Detailed Description

`<<singleton>>` `<<interface>>` Allows creation and destruction of [DDSDomainParticipant](#) objects.

The sole purpose of this class is to allow the creation and destruction of [DDSDomainParticipant](#) objects. This class itself is a `<<singleton>>`, and accessed via the [get\\_instance\(\)](#) method, and destroyed with [finalize\\_instance\(\)](#) method.

A single application can participate in multiple domains by instantiating multiple [DDSDomainParticipant](#) objects.

An application may even instantiate multiple participants in the same domain. Participants in the same domain exchange data in the same way regardless of whether they are in the same application or different applications or on the same node or different nodes; their location is transparent.

There are two important caveats:

- With multiple participants in the same domain on the same node (in the same application or different applications), each participant must be assigned its own receive ports. The receive ports are calculated based on the domain ID and the participant index set in the [DDS\\_WireProtocolQosPolicy](#). The default index (-1) causes the participant to automatically search for the next available index, starting at 0, when it is created. If an index larger or equal to 0 is assigned then only that value will be used. If another participant with the same index already exists the participant creation will fail.

NOTE: An exception to this rule is multicast ports. Multicast ports are shared between participants in the same domain on the same host). Thus, the index value is ignored.

- You cannot mix entities from different participants. For example, you cannot delete a topic on a different participant than you created it from, and you cannot ask a subscriber to create a reader for a topic created from a participant different than the subscriber's own participant. (Note that it is permissible for an application built on top of RTI Connext DDS Micro to know about entities from different participants. For example, an application could keep references to a reader from one domain and a writer from another and then bridge the domains by writing the data received in the reader callback.)

See also

[DDSDomainParticipant](#)

### 7.99.2 Member Function Documentation

#### **get\_instance()**

```
static DDSDomainParticipantFactory * DDSDomainParticipantFactory::get\_instance () [static]
```

Gets the singleton instance of this class.

Returns

the [DDSDomainParticipantFactory](#) instance.

**MT Safety:**

UNSAFE on the FIRST call to [DDSDomainParticipantFactory::get\\_instance\(\)](#). It is not safe for two threads to simultaneously make the first call to get an instance of the factory. Subsequent calls are thread safe.

[DDSTheParticipantFactory](#) can be used as an alias for the singleton factory returned by this operation.

**Returns**

the singleton [DDSDomainParticipantFactory](#) instance.

**See also**

[DDSTheParticipantFactory](#)

**finalize\_instance()**

```
static DDS_ReturnCode_t DDSDomainParticipantFactory::finalize_instance () [static]
```

<<*eXtension*>> Destroys the singleton instance of this class.

Only necessary to explicitly reclaim resources used by the participant factory singleton. Note that on many OSes, these resources are automatically reclaimed by the OS when the program terminates. Some memory checker tools still flag these as unreclaimed however. So this method provides a way to clean up memory used by the participant factory.

**Precondition**

All participants created from the factory have been deleted.

**Postcondition**

All resources belonging to the factory reclaimed. Another call to [DDSDomainParticipantFactory::get\\_instance](#) will return a new lifecycle of the singleton.

**MT Safety:**

UNSAFE for two threads to simultaneously call [DDSDomainParticipantFactory::finalize\\_instance\(\)](#) or [DDSDomainParticipantFactory::get\\_instance\(\)](#) at the same time.

**Returns**

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#)

**create\_participant()**

```
virtual DDSDomainParticipant * DDSDomainParticipantFactory::create_participant (
    DDS_DomainId_t domainId,
    const DDS_DomainParticipantQos & qos,
    DDSDomainParticipantListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Creates a new [DDSDomainParticipant](#) object.

**Precondition**

The specified QoS policies must be consistent, or the operation will fail and no [DDSDomainParticipant](#) will be created.

If you want to create multiple participants on a given host in the same domain, make sure each one has a different participant index (set in the [DDS\\_WireProtocolQoSPolicy](#)). This in turn will ensure each participant uses a different port number (since the unicast port numbers are calculated from the participant index and the domain ID).

Note that if there is a single participant per host in a given domain, the participant index can be left at the default value (-1).

**Parameters**

|                 |                                                                                                                                                                                                                                                                                                               |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>domainId</i> | <<in>> ID of the domain that the application intends to join. <b>[range]</b> [ $\geq 0$ ], and does not violate guidelines stated in <a href="#">DDS_RtpsWellKnownPorts_t</a> .                                                                                                                               |
| <i>qos</i>      | <<in>> the domain participant's QoS. The special value <a href="#">DDS_PARTICIPANT_QOS_DEFAULT</a> can be used to indicate that the <a href="#">DDSDomainParticipant</a> should be created with the default <a href="#">DDS_DomainParticipantQos</a> set in the <a href="#">DDSDomainParticipantFactory</a> . |
| <i>listener</i> | <<in>> the domain participant's listener.                                                                                                                                                                                                                                                                     |
| <i>mask</i>     | <<in>> Changes of communication status to be invoked on the listener.                                                                                                                                                                                                                                         |

**Returns**

domain participant or NULL on failure

**See also**

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS\\_DomainParticipantQos](#) for rules on consistency among QoS

[DDS\\_PARTICIPANT\\_QOS\\_DEFAULT](#)

**delete\_participant()**

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::delete_participant (
    DDSDomainParticipant * a_participant) [pure virtual]
```

Deletes an existing [DDSDomainParticipant](#).

**Precondition**

All domain entities belonging to the participant must have already been deleted. Otherwise it fails with the error [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**Postcondition**

Listener installed on the [DDSDomainParticipant](#) will not be called after this method returns successfully.

**Parameters**

|                      |                                                            |
|----------------------|------------------------------------------------------------|
| <i>a_participant</i> | <<in>> <a href="#">DDSDomainParticipant</a> to be deleted. |
|----------------------|------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**create\_participant\_from\_config()**

```
virtual DDSDomainParticipant * DDSDomainParticipantFactory::create_participant_from_config (
    const char * configuration_name) [pure virtual]
```

Creates a new [DDSDomainParticipant](#) given its configuration name from a description provided in configuration file.

This operation creates a [DDSDomainParticipant](#) and all the contained entities ([DDSTopic](#), [DDSPublisher](#), [DDSSubscriber](#), [DDSDataWriter](#), [DDSDataReader](#)) from a description given in a configuration file.

The configuration name is the fully qualified name of the application, consisting of the name of the application library plus the name of application configuration.

For example the name "MyApplicationLibrary::PublicationParticipant" can be used to create the application from the configuration with library name "MyApplicationLibrary" and participant name "PublicationParticipant"

The entities belonging to the newly created [DDSDomainParticipant](#) can be retrieved with the help of lookup operations such as: [DDSDomainParticipant::lookup\\_publisher\\_by\\_name](#).

**Parameters**

|                           |                                                                                                              |
|---------------------------|--------------------------------------------------------------------------------------------------------------|
| <i>configuration_name</i> | <<in>> Name of RTI Connexx DDS Micro application configuration in the configuration file.<br>Cannot be NULL. |
|---------------------------|--------------------------------------------------------------------------------------------------------------|

**Returns**

Pointer to [DDSDomainParticipant](#) or NULL on failure

**lookup\_participant()**

```
virtual DDSDomainParticipant * DDSDomainParticipantFactory::lookup_participant (
    DDS_DomainId_t domainId) [pure virtual]
```

Locates an existing [DDSDomainParticipant](#).

If no such [DDSDomainParticipant](#) exists, the operation will return NULL value.

If multiple [DDSDomainParticipant](#) entities belonging to that domainId exist, then the operation will return one of them. It is not specified which one.

**Parameters**

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>domainId</i> | <<in>> ID of the domain participant to lookup. |
|-----------------|------------------------------------------------|

**Returns**

domain participant if it exists, or NULL

**lookup\_participant\_by\_name()**

```
virtual DDSDomainParticipant * DDSDomainParticipantFactory::lookup_participant_by_name (
    const char * participant_name) [pure virtual]
```

Retrieves a [DDSDomainParticipant](#) by its entity name in the within the [DDSDomainParticipantFactory](#)

This returned [DDSDomainParticipant](#) is either enabled or disabled.

Every [DDSDomainParticipant](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY\_NAME.

This operation retrieves a [DDSDomainParticipant](#) within the [DDSDomainParticipantFactory](#) given the entity's name. If there are several [DDSDomainParticipant](#) with the same name within the [DDSDomainParticipantFactory](#), this function returns the first matching occurrence.

**Parameters**

|                         |                                                                                  |
|-------------------------|----------------------------------------------------------------------------------|
| <i>participant_name</i> | <<in>> Entity name of the <a href="#">DDSDomainParticipant</a> . Cannot be NULL. |
|-------------------------|----------------------------------------------------------------------------------|

**Returns**

The first [DDSDomainParticipant](#) found with the specified name or NULL if it is not found.

**MT Safety:**

UNSAFE. It is not safe to lookup a [DDSDomainParticipant](#) in one thread while another thread is simultaneously creating or destroying that [DDSDomainParticipant](#).

**set\_default\_participant\_qos()**

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::set_default_participant_qos (
    const DDS_DomainParticipantQos & qos) [pure virtual]
```

Sets the default [DDS\\_DomainParticipantQos](#) values for this domain participant factory.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

## Parameters

|            |                                                                                                                                                                                                                                                                                                                                                                                |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>inout</i> >> Qos to be used by the <a href="#">DDSDomainParticipant</a> . The special value <a href="#">DDS_PARTICIPANT_QOS_DEFAULT</a> may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would use if <a href="#">DDSDomainParticipantFactory::set_default_participant_qos</a> had never been called. |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#)

## See also

[DDS\\_PARTICIPANT\\_QOS\\_DEFAULT](#)  
[DDSDomainParticipantFactory::create\\_participant](#)

**get\_default\_participant\_qos()**

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::get_default_participant_qos (
    DDS_DomainParticipantQos & qos) [pure virtual]
```

Initializes the [DDS\\_DomainParticipantQos](#) instance with default values.

The retrieved *qos* will match the set of values specified on the last successful call to [DDSDomainParticipantFactory::set\\_default\\_participant\\_qos](#) or else, if the call was never made, the default values listed in [DDS\\_DomainParticipantQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

## Parameters

|            |                                               |
|------------|-----------------------------------------------|
| <i>qos</i> | << <i>out</i> >> the domain participant's QoS |
|------------|-----------------------------------------------|

## Returns

One of the [Standard Return Codes](#)

## See also

[DDS\\_PARTICIPANT\\_QOS\\_DEFAULT](#)  
[DDSDomainParticipantFactory::create\\_participant](#)

**get\_qos()**

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::get_qos (
    DDS_DomainParticipantFactoryQos & qos) [pure virtual]
```

Gets the value for participant factory QoS.

## Parameters

|            |                                         |
|------------|-----------------------------------------|
| <i>qos</i> | << <i>inout</i> >> QoS to be filled up. |
|------------|-----------------------------------------|

## Returns

One of the [Standard Return Codes](#)

**set\_qos()**

```
virtual DDS_ReturnCode_t DDSDomainParticipantFactory::set_qos (
    const DDS_DomainParticipantFactoryQos & qos) [pure virtual]
```

Sets the value for a participant factory QoS.

The [DDS\\_DomainParticipantFactoryQos::entity\\_factory](#) can be changed. The other policies are immutable.

Note that despite having QoS, the [DDSDomainParticipantFactory](#) is not an [DDSEntity](#).

Note that this function may cause RTI Connex DDS Micro to free and reallocate memory, depending on the QoS policies that are changed.

## Parameters

|            |                                                                                                                                                                                                                                                                                          |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> Set of policies to be applied to <a href="#">DDSDomainParticipantFactory</a> . Policies must be consistent. Immutable policies can only be changed before calling any other RTI Connex DDS Micro methods except for <a href="#">DDSDomainParticipantFactory::get_qos</a> |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#) if immutable policy is changed, or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#) if policies are inconsistent

## See also

[DDS\\_DomainParticipantFactoryQos](#) for rules on consistency among QoS

**get\_registry()**

```
virtual RTRegistry * DDSDomainParticipantFactory::get_registry () [pure virtual]
```

Retrieves the singleton registry instance associated with the factory.

## Returns

the registry instance or NULL on failure.



## 7.100 DDSDomainParticipantListener Class Reference

<<interface>> Listener for participant status.

```
#include <dds_cpp_domain.hxx>
```

### Public Member Functions

- virtual void `on_data_available` (DDSDataReader \*)  
*Handle the `DDS_DATA_AVAILABLE_STATUS` communication status.*
- virtual void `on_requested_deadline_missed` (DDSDataReader \*reader, const DDS\_RequestedDeadlineMissedStatus &status)  
*Handles the `DDS_REQUESTED_DEADLINE_MISSED_STATUS` communication status.*
- virtual void `on_liveliness_changed` (DDSDataReader \*reader, const DDS\_LivelinessChangedStatus &status)  
*Handles the `DDS_LIVELINESS_CHANGED_STATUS` communication status.*
- virtual void `on_requested_incompatible_qos` (DDSDataReader \*reader, const DDS\_RequestedIncompatibleQosStatus &status)  
*Handles the `DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS` communication status.*
- virtual void `on_sample_rejected` (DDSDataReader \*reader, const DDS\_SampleRejectedStatus &status)  
*Handles the `DDS_SAMPLE_REJECTED_STATUS` communication status.*
- virtual void `on_subscription_matched` (DDSDataReader \*reader, const DDS\_SubscriptionMatchedStatus &status)  
*Handles the `DDS_SUBSCRIPTION_MATCHED_STATUS` communication status.*
- virtual void `on_sample_lost` (DDSDataReader \*reader, const DDS\_SampleLostStatus &status)  
*Handles the `DDS_SAMPLE_LOST_STATUS` communication status.*
- virtual void `on_instance_replaced` (DDSDataReader \*reader, const DDS\_DataReaderInstanceReplacedStatus &status)  
*Handles the `DDS_INSTANCE_REPLACED_STATUS` communication status.*
- virtual DDS\_Boolean `on_before_sample_deserialize` (DDSDataReader \*reader, NDDS\_Type\_Plugin \*, CDR\_↵ Stream\_t \*stream, DDS\_Boolean \*)  
*Callback to filter a received sample based on serialized data*
- virtual DDS\_Boolean `on_before_sample_commit` (DDSDataReader \*, const void \*const, const struct DDS\_SampleInfo \*const, DDS\_Boolean \*)  
*Callback to filter a received sample based on deserialized data*
- virtual void `on_data_on_readers` (DDSSubscriber \*)  
*Handles the `DDS_DATA_ON_READERS_STATUS` communication status.*
- virtual void `on_offered_deadline_missed` (DDSDataWriter \*writer, const struct DDS\_OfferedDeadlineMissedStatus &status)  
*Handles the `DDS_OFFERED_DEADLINE_MISSED_STATUS` status.*
- virtual void `on_liveliness_lost` (DDSDataWriter \*writer, const struct DDS\_LivelinessLostStatus &status)  
*Handles the `DDS_LIVELINESS_LOST_STATUS` status.*
- virtual void `on_offered_incompatible_qos` (DDSDataWriter \*writer, const struct DDS\_OfferedIncompatibleQosStatus &status)  
*Handles the `DDS_OFFERED_INCOMPATIBLE_QOS_STATUS` status.*
- virtual void `on_publication_matched` (DDSDataWriter \*writer, const struct DDS\_PublicationMatchedStatus &status)  
*Handles the `DDS_PUBLICATION_MATCHED_STATUS` status.*
- virtual void `on_reliable_reader_activity_changed` (DDSDataWriter \*writer, const struct DDS\_ReliableReaderActivityChangedStatus &status)  
*<<eXtension>> A matched reliable reader has become active or become inactive.*
- virtual void `on_inconsistent_topic` (DDSTopic \*topic, const DDS\_InconsistentTopicStatus &status)  
*Handle the `DDS_INCONSISTENT_TOPIC_STATUS` status.*

### 7.100.1 Detailed Description

`<<interface>>` Listener for participant status.

Entity:

[DDSDomainParticipant](#)

Status:

[Status Kinds](#)

This is the interface that can be implemented by an application-provided class and then registered with the [DDSDomainParticipant](#) such that the application can be notified by RTI Connex DDS Micro of relevant status changes.

The [DDSDomainParticipantListener](#) interface extends all other Listener interfaces and has no additional operation beyond the ones defined by the more general listeners.

The purpose of the [DDSDomainParticipantListener](#) is to be the listener of last resort that is notified of all status changes not captured by more specific listeners attached to the [DDSDomainEntity](#) objects. When a relevant status change occurs, RTI Connex DDS Micro will first attempt to notify the listener attached to the concerned [DDSDomainEntity](#) if one is installed. Otherwise, RTI Connex DDS Micro will notify the Listener attached to the [DDSDomainParticipant](#).

*Important:* Because a [DDSDomainParticipantListener](#) may receive callbacks pertaining to many different entities, it is possible for the same listener to receive multiple callbacks simultaneously in different threads. (Such is not the case for listeners of other types.) It is therefore critical that users of this listener provide their own protection for any thread-unsafe activities undertaken in a [DDSDomainParticipantListener](#) callback.

See also

[DDSLISTENER](#)

[DDSDomainParticipant::set\\_listener](#)

RTI Connex DDS Micro C++ API avoids the use of multiple inheritance for certification purposes. For this reason, [DDSDomainParticipantListener](#) is not declared as a sub-class of [DDSPublisherListener](#), [DDSSubscriberListener](#), and [DDSTopicListener](#), but it explicitly declares all supported call-backs.

### 7.100.2 Member Function Documentation

#### **on\_data\_available()**

```
virtual void DDSDomainParticipantListener::on_data_available (  
    DDSDataReader * ) [inline], [virtual]
```

Handle the [DDS\\_DATA\\_AVAILABLE\\_STATUS](#) communication status.

**on\_requested\_deadline\_missed()**

```
virtual void DDSDomainParticipantListener::on_requested_deadline_missed (  
    DDSDataReader * reader,  
    const DDS_RequestedDeadlineMissedStatus & status) [inline], [virtual]
```

Handles the [DDS\\_REQUESTED\\_DEADLINE\\_MISSED\\_STATUS](#) communication status.

**on\_liveliness\_changed()**

```
virtual void DDSDomainParticipantListener::on_liveliness_changed (  
    DDSDataReader * reader,  
    const DDS_LivelinessChangedStatus & status) [inline], [virtual]
```

Handles the [DDS\\_LIVELINESS\\_CHANGED\\_STATUS](#) communication status.

**on\_requested\_incompatible\_qos()**

```
virtual void DDSDomainParticipantListener::on_requested_incompatible_qos (  
    DDSDataReader * reader,  
    const DDS_RequestedIncompatibleQosStatus & status) [inline], [virtual]
```

Handles the [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) communication status.

**on\_sample\_rejected()**

```
virtual void DDSDomainParticipantListener::on_sample_rejected (  
    DDSDataReader * reader,  
    const DDS_SampleRejectedStatus & status) [inline], [virtual]
```

Handles the [DDS\\_SAMPLE\\_REJECTED\\_STATUS](#) communication status.

**on\_subscription\_matched()**

```
virtual void DDSDomainParticipantListener::on_subscription_matched (  
    DDSDataReader * reader,  
    const DDS_SubscriptionMatchedStatus & status) [inline], [virtual]
```

Handles the [DDS\\_SUBSCRIPTION\\_MATCHED\\_STATUS](#) communication status.

**on\_sample\_lost()**

```
virtual void DDSDomainParticipantListener::on_sample_lost (  
    DDSDataReader * reader,  
    const DDS_SampleLostStatus & status) [inline], [virtual]
```

Handles the [DDS\\_SAMPLE\\_LOST\\_STATUS](#) communication status.

**on\_instance\_replaced()**

```
virtual void DDSDomainParticipantListener::on_instance_replaced (
    DDSDataReader * reader,
    const DDS_DataReaderInstanceReplacedStatus & status) [inline], [virtual]
```

Handles the [DDS\\_INSTANCE\\_REPLACED\\_STATUS](#) communication status.

This listener may only be called when the instance replacement policy is set to [DDS\\_REPLACE\\_OLDEST\\_INSTANCE\\_REPLACEMENT\\_QOS](#). When a new instance is detected and resources are exhausted the oldest instance will be removed even if samples with a not read state exists for the instance. Thus, this is a forceful removal of samples and the instance. The freed instance resources will be re-used for the newly detected instance. If there are outstanding loans for the replaced instance the samples can not be freed until returned. However, when the loan is returned the samples are automatically freed. Note that the instance will not be rejected.

**on\_before\_sample\_deserialize()**

```
virtual DDS_Boolean DDSDomainParticipantListener::on_before_sample_deserialize (
    DDSDataReader * reader,
    NDDS_Type_Plugin * ,
    CDR_Stream_t * stream,
    DDS_Boolean * ) [inline], [virtual]
```

Callback to filter a received sample based on serialized data

When a DataReader receives a (serialized) sample, it will deserialize it before storing it in its queue. [On\\_before\\_sample\\_deserialize\(\)](#) is called before deserialization. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filter out of the DataReader's queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the DataReader will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to [on\\_before\\_sample\\_commit\(\)](#), the sample would be dropped with less processing (without deserialization) by the subscribing application, but with the added complexity of filtering on serialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

**on\_before\_sample\_commit()**

```
virtual DDS_Boolean DDSDomainParticipantListener::on_before_sample_commit (
    DDSDataReader * ,
    const void * const ,
    const struct DDS_SampleInfo * const ,
    DDS_Boolean * ) [inline], [virtual]
```

Callback to filter a received sample based on deserialized data

When a `DataReader` receives a (serialized) sample, it will deserialize it before storing it in its queue. `On_before_sample_commit()` is called after deserialization, but before the sample is stored into the `DataReader`'s queue. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filter out of the `DataReader`'s queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the `DataReader` will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to `on_before_sample_deserialize()`, the sample would be dropped with more processing (with deserialization) by the subscribing application, but with the lesser complexity of filtering on deserialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

The following fields are valid in the `DDS_SampleInfo` structure:

- `source_timestamp`
- `instance_handle`
- `publication_handle`
- `valid_data`
- `encapsulation_id`

### **on\_data\_on\_readers()**

```
virtual void DDSDomainParticipantListener::on_data_on_readers (
    DDSSubscriber * ) [inline], [virtual]
```

Handles the `DDS_DATA_ON_READERS_STATUS` communication status.

### **on\_offered\_deadline\_missed()**

```
virtual void DDSDomainParticipantListener::on_offered_deadline_missed (
    DDSDataWriter * writer,
    const struct DDS_OfferedDeadlineMissedStatus & status) [inline], [virtual]
```

Handles the `DDS_OFFERED_DEADLINE_MISSED_STATUS` status.

This callback is called when the deadline that the `DDSDataWriter` has committed through its `DEADLINE` qos policy was not respected for a specific instance. This callback is called for each deadline period elapsed during which the `DDSDataWriter` failed to provide data for an instance.

#### **Parameters**

|               |                                                                                        |
|---------------|----------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <code>DDSDataWriter</code> that triggers the listener callback |
| <i>status</i> | <<out>> Current deadline missed status of locally created <code>DDSDataWriter</code>   |

**on\_liveliness\_lost()**

```
virtual void DDSDomainParticipantListener::on_liveliness_lost (
    DDSDataWriter * writer,
    const struct DDS_LivelinessLostStatus & status) [inline], [virtual]
```

Handles the [DDS\\_LIVELINESS\\_LOST\\_STATUS](#) status.

This callback is called when the liveliness that the [DDSDataWriter](#) has committed through its [LIVELINESS](#) qos policy was not respected; this [DDSDataReader](#) entities will consider the [DDSDataWriter](#) as no longer "alive/active". This callback will not be called when an already not alive [DDSDataWriter](#) simply remains not alive for another liveliness period.

**Parameters**

|               |                                                                                           |
|---------------|-------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDataWriter</a> that triggers the listener callback |
| <i>status</i> | <<out>> Current liveliness lost status of locally created <a href="#">DDSDataWriter</a>   |

**on\_offered\_incompatible\_qos()**

```
virtual void DDSDomainParticipantListener::on_offered_incompatible_qos (
    DDSDataWriter * writer,
    const struct DDS_OfferedIncompatibleQosStatus & status) [inline], [virtual]
```

Handles the [DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) status.

This callback is called when the [DDS\\_DataWriterQos](#) of the [DDSDataWriter](#) was incompatible with what was requested by a [DDSDataReader](#). This callback is called when a [DDSDataWriter](#) has discovered a [DDSDataReader](#) for the same [DDSTopic](#) and common partition, but with a requested QoS that is incompatible with that offered by the [DDSDataWriter](#).

**Parameters**

|               |                                                                                           |
|---------------|-------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDataWriter</a> that triggers the listener callback |
| <i>status</i> | <<out>> Current incompatible qos status of locally created <a href="#">DDSDataWriter</a>  |

**on\_publication\_matched()**

```
virtual void DDSDomainParticipantListener::on_publication_matched (
    DDSDataWriter * writer,
    const struct DDS_PublicationMatchedException & status) [inline], [virtual]
```

Handles the [DDS\\_PUBLICATION\\_MATCHED\\_STATUS](#) status.

This callback is called when the [DDSDataWriter](#) has found a [DDSDataReader](#) that matches the [DDSTopic](#), has a common partition and compatible QoS, or has ceased to be matched with a [DDSDataReader](#) that was previously considered to be matched.

## Parameters

|               |                                                                                            |
|---------------|--------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback |
| <i>status</i> | <<out>> Current publication match status of locally created <a href="#">DDSDDataWriter</a> |

**on\_reliable\_reader\_activity\_changed()**

```
virtual void DDSDomainParticipantListener::on_reliable_reader_activity_changed (
    DDSDDataWriter * writer,
    const struct DDS\_ReliableReaderActivityChangedStatus & status) [inline], [virtual]
```

<<eXtension>> A matched reliable reader has become active or become inactive.

A reliable writer considers a matched reliable reader to be active when it receives ACKNACK messages from the reader in response to its HEARTBEAT messages. The writer considers an active reader to have become inactive when it has sent a number of consecutive periodic HEARTBEATs equal to [DDS\\_RtpsReliableWriterProtocol\\_t::max\\_heartbeat\\_retries](#) without receiving an ACKNACK from the reader. An inactive reader becomes active once the writer receives an ACKNACK from it.

This callback is called when the transition from active to inactive, and vice versa, has occurred for a matched reliable reader.

## Parameters

|               |                                                                                                           |
|---------------|-----------------------------------------------------------------------------------------------------------|
| <i>writer</i> | <<out>> Locally created <a href="#">DDSDDataWriter</a> that triggers the listener callback                |
| <i>status</i> | <<out>> Current reliable reader activity changed status of locally created <a href="#">DDSDDataWriter</a> |

**on\_inconsistent\_topic()**

```
virtual void DDSDomainParticipantListener::on_inconsistent_topic (
    DDSTopic * topic,
    const DDS\_InconsistentTopicStatus & status) [inline], [virtual]
```

Handle the [DDS\\_INCONSISTENT\\_TOPIC\\_STATUS](#) status.

This callback is called when a remote [DDSTopic](#) is discovered but is inconsistent with the locally created [DDSTopic](#) of the same topic name.

## Parameters

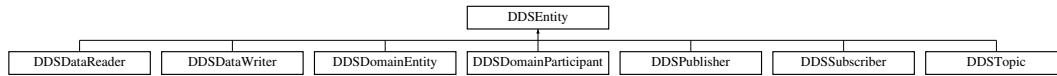
|               |                                                                                      |
|---------------|--------------------------------------------------------------------------------------|
| <i>topic</i>  | <<out>> Locally created <a href="#">DDSTopic</a> that triggers the listener callback |
| <i>status</i> | <<out>> Current inconsistent status of locally created <a href="#">DDSTopic</a>      |

## 7.101 DDSEntity Class Reference

[`<<interface>>`](#) Abstract base class for all the DDS objects that support QoS policies, and a listener.

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSEntity:



### Public Member Functions

- [`DDS\_ReturnCode\_t enable \(\)`](#)  
*Enables the [`DDSEntity`](#).*
- [`DDSStatusCondition \* get\_statuscondition \(\)`](#)  
*Return the [`DDSStatusCondition`](#) associated with a particular [`DDSEntity`](#).*
- [`DDS\_StatusMask get\_status\_changes \(\)`](#)  
*Retrieves the list of communication statuses in the [`DDSEntity`](#) that are triggered.*
- [`DDS\_InstanceHandle\_t get\_instance\_handle \(\)`](#)  
*Allows access to the [`DDS\_InstanceHandle\_t`](#) associated with the [`DDSEntity`](#).*

### 7.101.1 Detailed Description

[`<<interface>>`](#) Abstract base class for all the DDS objects that support QoS policies, and a listener.

All operations except for `set_qos()`, `get_qos()`, `set_listener()`, `get_listener()` and [`enable\(\)`](#), may return the value [`DDS\_RETCODE\_NOT\_ENABLED`](#).

QoS:

[QoS Policies](#)

Status:

[Status Kinds](#)

Listener:

[DDSListener](#)

### 7.101.2 Abstract operations

Each derived entity provides the following operations specific to its role in RTI Connex DDS Micro.



### 7.101.2.1 `set_qos` (abstract)

This operation sets the QoS policies of the [DDSEntity](#).

This operation must be provided by each of the derived [DDSEntity](#) classes ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), and [DDSDataReader](#)) so that the policies that are meaningful to each [DDSEntity](#) can be set.

#### Precondition

Certain policies are immutable (see [QoS Policies](#)): they can only be set at [DDSEntity](#) creation time or before the entity is enabled. If `set_qos()` is invoked after the [DDSEntity](#) is enabled and it attempts to change the value of an immutable policy, the operation will fail and return [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#).

Certain values of QoS policies can be incompatible with the settings of the other policies. The `set_qos()` operation will also fail if it specifies a set of values that, once combined with the existing values, would result in an inconsistent set of policies. In this case, the operation will fail and return [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#).

If the application supplies a non-default value for a QoS policy that is not supported by the implementation of the service, the `set_qos` operation will fail and return [DDS\\_RETCODE\\_UNSUPPORTED](#).

#### Postcondition

The existing set of policies is only changed if the `set_qos()` operation succeeds. This is indicated by a return code of [DDS\\_RETCODE\\_OK](#). In all other cases, none of the policies are modified.

Each derived [DDSEntity](#) class ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), [DDSDataReader](#)) has a corresponding special value of the QoS ([DDS\\_PARTICIPANT\\_QOS\\_DEFAULT](#), [DDS\\_PUBLISHER\\_QOS\\_DEFAULT](#), [DDS\\_SUBSCRIBER\\_QOS\\_DEFAULT](#), [DDS\\_TOPIC\\_QOS\\_DEFAULT](#), [DDS\\_DATAWRITER\\_QOS\\_DEFAULT](#), [DDS\\_DATAREADER\\_QOS\\_DEFAULT](#)). This special value may be used as a parameter to the `set_qos` operation to indicate that the QoS of the [DDSEntity](#) should be changed to match the current default QoS set in the [DDSEntity](#)'s factory. The operation `set_qos` cannot modify the immutable QoS, so a successful return of the operation indicates that the mutable QoS for the Entity has been modified to match the current default for the [DDSEntity](#)'s factory.

The set of policies specified in the `qos` parameter are applied on top of the existing QoS, replacing the values of any policies previously set.

Possible error codes returned in addition to [Standard Return Codes](#) : [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#).

### 7.101.2.2 `get_qos` (abstract)

This operation allows access to the existing set of QoS policies for the [DDSEntity](#). This operation must be provided by each of the derived [DDSEntity](#) classes ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), and [DDSDataReader](#)), so that the policies that are meaningful to each [DDSEntity](#) can be retrieved.

Possible error codes are [Standard Return Codes](#).

### 7.101.2.3 set\_listener (abstract)

This operation installs a [DDSLISTENER](#) on the [DDSEntity](#). The listener will only be invoked on the changes of communication status indicated by the specified `mask`.

This operation must be provided by each of the derived [DDSEntity](#) classes ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), and [DDSDataReader](#)), so that the listener is of the concrete type suitable to the particular [DDSEntity](#).

It is permitted to use NULL as the value of the listener. The NULL listener behaves as if the `mask` is [DDS\\_STATUS\\_MASK\\_NONE](#).

#### Postcondition

Only one listener can be attached to each [DDSEntity](#). If a listener was already set, the operation `set_listener()` will replace it with the new one. Consequently, if the value NULL is passed for the listener parameter to the `set_listener` operation, any existing listener will be removed.

### 7.101.2.4 get\_listener (abstract)

This operation allows access to the existing [DDSLISTENER](#) attached to the [DDSEntity](#).

This operation must be provided by each of the derived [DDSEntity](#) classes ([DDSDomainParticipant](#), [DDSTopic](#), [DDSPublisher](#), [DDSDataWriter](#), [DDSSubscriber](#), and [DDSDataReader](#)) so that the listener is of the concrete type suitable to the particular [DDSEntity](#).

If no listener is installed on the [DDSEntity](#), this operation will return NULL.

## 7.101.3 Member Function Documentation

### enable()

```
DDS_ReturnCode_t DDSEntity::enable ()
```

Enables the [DDSEntity](#).

This operation enables the Entity. Entity objects can be created either enabled or disabled. This is controlled by the value of the [ENTITY\\_FACTORY](#) QoS policy on the corresponding factory for the [DDSEntity](#).

By default, [ENTITY\\_FACTORY](#) is set so that it is not necessary to explicitly call [DDSEntity::enable](#) on newly created entities.

The [DDSEntity::enable](#) operation is idempotent. Calling enable on an already enabled Entity returns OK and has no effect.

If a [DDSEntity](#) has not yet been enabled, the following kinds of operations may be invoked on it:

- set or get the QoS policies (including default QoS policies) and listener
- 'factory' operations

- 'lookup' operations

Other operations may explicitly state that they may be called on disabled entities; those that do not will return the error [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

It is legal to delete an [DDSEntity](#) that has not been enabled by calling the proper operation on its factory.

Entities created from a factory that is disabled are created disabled, regardless of the setting of the [DDS\\_EntityFactoryQosPolicy](#).

Calling enable on an Entity whose factory is not enabled will fail and return [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

If [DDS\\_EntityFactoryQosPolicy::autoenable\\_created\\_entities](#) is TRUE, the enable operation on a factory will automatically enable all entities created from that factory.

Listeners associated with an entity are not called until the entity is enabled.

#### Returns

One of the [Standard Return Codes](#), [Standard Return Codes](#) or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

### **get\_statuscondition()**

```
DDSStatusCondition * DDSEntity::get_statuscondition ()
```

Return the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).

#### Returns

The [DDSStatusCondition](#) that belongs to the specified [DDSEntity](#).

### **get\_status\_changes()**

```
DDS_StatusMask DDSEntity::get_status_changes ()
```

Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.

That is, the list of statuses whose value has changed since the last time the application read the status using the `get_*_status()` method.

When the entity is first created or if the entity is not enabled, all communication statuses are in the "untriggered" state so the list returned by the `get_status_changes` operation will be empty.

The list of statuses returned by the `get_status_changes` operation refers to the status that are triggered on the Entity itself and does not include statuses that apply to contained entities.

#### Returns

list of communication statuses in the [DDSEntity](#) that are triggered.

#### See also

[Status Kinds](#)

**get\_instance\_handle()**

```
DDS_InstanceHandle_t DDSEntity::get_instance_handle ()
```

Allows access to the `DDS_InstanceHandle_t` associated with the `DDSEntity`.

This operation returns the `DDS_InstanceHandle_t` that represents the `DDSEntity`.

**Returns**

the instance handle associated with this entity.

**7.102 DDSFlowController Class Reference**

**<<interface>>** A flow controller is the object responsible for shaping the network traffic by determining when attached asynchronous `DDSDataWriter` instances are allowed to write data.

```
#include <dds_cpp_flowcontroller.hxx>
```

**Public Member Functions**

- virtual `DDS_ReturnCode_t set_property` (const struct `DDS_FlowControllerProperty_t` &prop)=0  
*Sets the `DDSFlowController` property.*
- virtual `DDS_ReturnCode_t get_property` (struct `DDS_FlowControllerProperty_t` &prop)=0  
*Gets the `DDSFlowController` property.*
- virtual `DDS_ReturnCode_t trigger_flow` ()=0  
*Provides an external trigger to the `DDSFlowController`.*
- virtual const char \* `get_name` ()=0  
*Returns the name of the `DDSFlowController`.*
- virtual `DDSDomainParticipant` \* `get_participant` ()=0  
*Returns the `DDSDomainParticipant` to which the `DDSFlowController` belongs.*

**7.102.1 Detailed Description**

**<<interface>>** A flow controller is the object responsible for shaping the network traffic by determining when attached asynchronous `DDSDataWriter` instances are allowed to write data.

QoS:

`DDS_FlowControllerProperty_t`

**7.102.2 Member Function Documentation****set\_property()**

```
virtual DDS_ReturnCode_t DDSFlowController::set_property (
    const struct DDS_FlowControllerProperty_t & prop) [pure virtual]
```

Sets the `DDSFlowController` property.

This operation modifies the property of the `DDSFlowController`.

Once a `DDSFlowController` has been instantiated, only the `DDS_FlowControllerProperty_t::token_bucket` can be changed. The `DDS_FlowControllerProperty_t::scheduling_policy` is immutable.

A new `DDS_FlowControllerTokenBucketProperty_t::period` only takes effect at the next scheduled token distribution time (as determined by its previous value).

## Parameters

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>prop</i> | <<in>> The new <a href="#">DDS_FlowControllerProperty_t</a> . Property must be consistent. Immutable fields cannot be changed after <a href="#">DDSFlowController</a> has been created. The special value <a href="#">DDS_FLOW_CONTROLLER_PROPERTY_DEFAULT</a> can be used to indicate that the property of the <a href="#">DDSFlowController</a> should be changed to match the current default <a href="#">DDS_FlowControllerProperty_t</a> set in the <a href="#">DDSDomainParticipant</a> . |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#).

## See also

[DDS\\_FlowControllerProperty\\_t](#) for rules on consistency among property values.

**get\_property()**

```
virtual DDS\_ReturnCode\_t DDSFlowController::get_property (
    struct DDS\_FlowControllerProperty\_t & prop) [pure virtual]
```

Gets the [DDSFlowController](#) property.

## Parameters

|             |                                                           |
|-------------|-----------------------------------------------------------|
| <i>prop</i> | <<in>> <a href="#">DDSFlowController</a> to be filled in. |
|-------------|-----------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#)

**trigger\_flow()**

```
virtual DDS\_ReturnCode\_t DDSFlowController::trigger_flow () [pure virtual]
```

Provides an external trigger to the [DDSFlowController](#).

## Returns

One of the [Standard Return Codes](#)

Typically, a [DDSFlowController](#) uses an internal trigger to periodically replenish its tokens. The period by which this trigger is called is determined by the [DDS\\_FlowControllerTokenBucketProperty\\_t::period](#) property setting.

This function provides an additional, external trigger to the [DDSFlowController](#). This trigger adds [DDS\\_FlowControllerTokenBucketProperty\\_t](#) tokens each time it is called (subject to the other property settings of the [DDSFlowController](#)).

An *on-demand* [DDSFlowController](#) can be created with a [DDS\\_DURATION\\_INFINITE](#) as [DDS\\_FlowControllerTokenBucketProperty\\_t::period](#) in which case the only trigger source is external (i.e. the FlowController is solely triggered by the user on demand).

[DDSFlowController::trigger\\_flow](#) can be called on both strict *on-demand* FlowController and hybrid FlowController (internally and externally triggered).

**get\_name()**

```
virtual const char * DDSFlowController::get_name () [pure virtual]
```

Returns the name of the [DDSFlowController](#).

**Returns**

The name of the [DDSFlowController](#).

**get\_participant()**

```
virtual DDSDomainParticipant * DDSFlowController::get_participant () [pure virtual]
```

Returns the [DDSDomainParticipant](#) to which the [DDSFlowController](#) belongs.

**Returns**

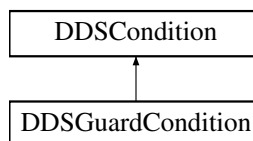
The [DDSDomainParticipant](#) to which the [DDSFlowController](#) belongs.

**7.103 DDSGuardCondition Class Reference**

<<*interface*>> A specific [DDSCondition](#) whose `trigger_value` is completely under the control of the application.

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSGuardCondition:

**Public Member Functions**

- [DDS\\_ReturnCode\\_t set\\_trigger\\_value](#) (bool value)  
*Set the guard condition trigger value.*

**7.103.1 Detailed Description**

<<*interface*>> A specific [DDSCondition](#) whose `trigger_value` is completely under the control of the application.

The [DDSGuardCondition](#) provides a way for an application to manually wake up a [DDSWaitSet](#). This is accomplished by attaching the [DDSGuardCondition](#) to the [DDSWaitSet](#) and then setting the `trigger_value` by means of the [DDSGuardCondition::set\\_trigger\\_value](#) operation.

**See also**

[DDSWaitSet](#)

### 7.103.2 Member Function Documentation

#### set\_trigger\_value()

```
DDS_ReturnCode_t DDSGuardCondition::set_trigger_value (
    bool value)
```

Set the guard condition trigger value.

#### Parameters

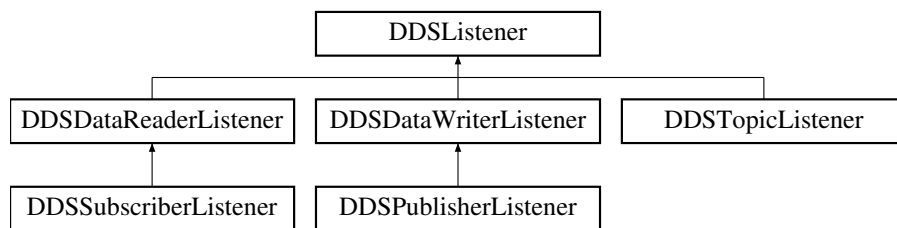
|              |                                        |
|--------------|----------------------------------------|
| <i>value</i> | << <i>in</i> >> the new trigger value. |
|--------------|----------------------------------------|

## 7.104 DDSListener Class Reference

<<*interface*>> Abstract base class for all Listener interfaces.

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSListener:



### 7.104.1 Detailed Description

<<*interface*>> Abstract base class for all Listener interfaces.

Entity:

[DDSEntity](#)

QoS:

[QoS Policies](#)

Status:

[Status Kinds](#)

All the supported kinds of concrete [DDSListener](#) interfaces (one per concrete [DDSEntity](#) type) derive from this root and add methods whose prototype depends on the concrete Listener.

Listeners provide a way for RTI Connex DDS Micro to asynchronously alert the application when there are relevant status changes.

Almost every application will have to implement listener interfaces.

Each dedicated listener presents a list of operations that correspond to the relevant communication status changes to which an application may respond.

The same [DDSListener](#) instance may be shared among multiple entities if you so desire. Consequently, the provided parameter contains a reference to the concerned [DDSEntity](#).

### 7.104.2 Access to Plain Communication Status

The general mapping between the plain communication statuses (see [Status Kinds](#)) and the listeners' operations is as follows:

- For each communication status, there is a corresponding operation whose name is `on_<communication_status>()`, which takes a parameter of type `<communication_status>` as listed in [Status Kinds](#).
- `on_<communication_status>` is available on the relevant [DDSEntity](#) as well as those that embed it, as expressed in the following figure:
- When the application attaches a listener on an entity, it must set a mask. The mask indicates to RTI Connex DDS Micro which operations are enabled within the listener (cf. operation [DDSEntity](#) `set_listener()` ).
- When a plain communication status changes, RTI Connex DDS Micro triggers the most specific relevant listener operation that is enabled. In case the most specific relevant listener operation corresponds to an application-installed 'nil' listener the operation will be considered handled by a NO-OP operation that does not reset the communication status.

This behavior allows the application to set a default behavior (e.g., in the listener associated with the [DDSDomainParticipant](#)) and to set dedicated behaviors only where needed.

### 7.104.3 Access to Read Communication Status

The two statuses related to data arrival are treated slightly differently. Since they constitute the core purpose of the Data Distribution Service, there is no need to provide a default mechanism (as is done for the plain communication statuses above).

The rule is as follows. Each time the read communication status changes:

- First, RTI Connex DDS Micro tries to trigger the [DDSSubscriberListener::on\\_data\\_on\\_readers](#) with a parameter of the related [DDSSubscriber](#);
- If this does not succeed (there is no listener or the operation is not enabled), RTI Connex DDS Micro tries to trigger [DDSDataReaderListener::on\\_data\\_available](#) on all the related [DDSDataReaderListener](#) objects, with a parameter of the related [DDSDataReader](#).

The rationale is that either the application is interested in relations among data arrivals and it must use the first option or it wants to treat each [DDSDataReader](#) independently and it may choose the second option (and then get the data by calling [FooDataReader::read\\_next\\_sample](#) or [FooDataReader::take\\_next\\_sample](#) on the related [DDSDataReader](#)).

Note that if [DDSSubscriberListener::on\\_data\\_on\\_readers](#) is called, RTI Connex DDS Micro will *not* try to call [DDSDataReaderListener::on\\_data\\_available](#).

The operations that are allowed in [DDSListener](#) callbacks depend the [DDSEntity](#) to which the [DDSListener](#) is attached -- or in the case of a [DDSDataWriter](#) or [DDSDataReader](#) listener, on the parent [DDSPublisher](#) or [DDSSubscriber](#).

The following operations are *not* allowed:



- Within any listener callback, deleting the entity to which the [DDSListener](#) is attached
- Within a [DDSTopic](#) listener callback, any operations on any subscribers, readers, publishers or writers

An attempt to call a disallowed method from within a callback will result in [DDS\\_RETCODE\\_ILLEGAL\\_OPERATION](#).

The following are *not* allowed:

- Within any listener callback, creating any entity
- Within any listener callback, deleting any entity
- Within any listener callback, enabling any entity
- Within any listener callback, setting the QoS of any entities
- Within a [DDSDDataReader](#) or [DDSSubscriber](#) listener callback, invoking any operation on any other [DDSSubscriber](#) or on any [DDSDDataReader](#) belonging to another [DDSSubscriber](#).
- Within a [DDSDDataReader](#) or [DDSSubscriber](#) listener callback, invoking any operation on any [DDSPublisher](#) (or on any [DDSDDataWriter](#) belonging to such a [DDSPublisher](#)).
- Within a [DDSDDataWriter](#) of [DDSPublisher](#) listener callback, invoking any operation on another Publisher or on a [DDSDDataWriter](#) belonging to another [DDSPublisher](#).
- Within a [DDSDDataWriter](#) of [DDSPublisher](#) listener callback, invoking any operation on any [DDSSubscriber](#) or [DDSDDataReader](#).

An attempt to call a disallowed method from within a callback will result in [DDS\\_RETCODE\\_ILLEGAL\\_OPERATION](#).

See also

[Status Kinds](#)

## 7.105 DDSPropertyQosPolicyHelper Class Reference

Policy helpers that facilitate management of the properties in the input policy.

```
#include <dds_cpp_infrastructure.hxx>
```

### Static Public Member Functions

- static [DDS\\_ReturnCode\\_t](#) [assert\\_property](#) ([DDS\\_PropertyQosPolicy](#) &policy, const char \*name, const char \*value, [DDS\\_Boolean](#) propagate)  
*Asserts the property identified by name in the input policy.*
- static [DDS\\_ReturnCode\\_t](#) [add\\_property](#) ([DDS\\_PropertyQosPolicy](#) &policy, const char \*name, const char \*value, [DDS\\_Boolean](#) propagate)  
*Adds a new property to the input policy.*
- static struct [DDS\\_Property\\_t](#) \* [lookup\\_property](#) ([DDS\\_PropertyQosPolicy](#) &policy, const char \*name)  
*Searches for a property in the input policy given its name.*
- static [DDS\\_ReturnCode\\_t](#) [remove\\_property](#) ([DDS\\_PropertyQosPolicy](#) &policy, const char \*name)  
*Removes a property from the input policy.*

### 7.105.1 Detailed Description

Policy helpers that facilitate management of the properties in the input policy.

### 7.105.2 Member Function Documentation

#### assert\_property()

```
static DDS_ReturnCode_t DDSPropertyQosPolicyHelper::assert_property (
    DDS_PropertyQosPolicy & policy,
    const char * name,
    const char * value,
    DDS_Boolean propagate) [static]
```

Asserts the property identified by name in the input policy.

If the property already exists, this function replaces its current value with the new one.

If the property identified by name does not exist, this function adds it to the property set.

This function increases the maximum number of elements of the policy sequence when this number is not enough to store the new property.

#### Precondition

policy, name and value cannot be NULL.

#### Parameters

|                  |                                                                                              |
|------------------|----------------------------------------------------------------------------------------------|
| <i>policy</i>    | <<in>> Input policy.                                                                         |
| <i>name</i>      | <<in>> Property name.                                                                        |
| <i>value</i>     | <<in>> Property value.                                                                       |
| <i>propagate</i> | <<in>> Indicates if the property will be propagated on discovery. Must be DDS_BOOLEAN_FALSE. |

#### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#)

## add\_property()

```
static DDS_ReturnCode_t DDSPropertyQosPolicyHelper::add_property (
    DDS_PropertyQosPolicy & policy,
    const char * name,
    const char * value,
    DDS_Boolean propagate) [static]
```

Adds a new property to the input policy.

This function will allocate memory to store the (name, value) pair. The memory allocated is owned by RTI Connex DDS Micro.

If the maximum number of elements of the policy sequence is not enough to store the new property, this function will increase it.

If the property already exists the function fails with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

### Precondition

policy, name and value cannot be NULL.

The property is not in the policy.

### Parameters

|                  |                                                                                                       |
|------------------|-------------------------------------------------------------------------------------------------------|
| <i>policy</i>    | << <i>in</i> >> Input policy.                                                                         |
| <i>name</i>      | << <i>in</i> >> Property name.                                                                        |
| <i>value</i>     | << <i>in</i> >> Property value.                                                                       |
| <i>propagate</i> | << <i>in</i> >> Indicates if the property will be propagated on discovery. Must be DDS_BOOLEAN_FALSE? |

### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#)

## lookup\_property()

```
static struct DDS_Property_t * DDSPropertyQosPolicyHelper::lookup_property (
    DDS_PropertyQosPolicy & policy,
    const char * name) [static]
```

Searches for a property in the input policy given its name.

### Precondition

policy, name and value cannot be NULL.

## Parameters

|               |                                |
|---------------|--------------------------------|
| <i>policy</i> | << <i>in</i> >> Input policy.  |
| <i>name</i>   | << <i>in</i> >> Property name. |

## Returns

The function returns the first property with the given name. If such a property does not exist, the function returns NULL.

**remove\_property()**

```
static DDS_ReturnCode_t DDSPropertyQosPolicyHelper::remove_property (
    DDS_PropertyQosPolicy & policy,
    const char * name) [static]
```

Removes a property from the input policy.

If the property does not exist, the function fails with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

## Precondition

policy and name cannot be NULL.

The property is in the policy.

## Parameters

|               |                                |
|---------------|--------------------------------|
| <i>policy</i> | << <i>in</i> >> Input policy.  |
| <i>name</i>   | << <i>in</i> >> Property name. |

## Returns

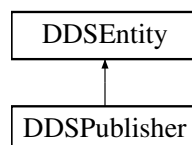
One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#)

**7.106 DDSPublisher Class Reference**

<<*interface*>> A publisher is the object responsible for the actual dissemination of publications.

```
#include <dds_cpp_publication.hxx>
```

Inheritance diagram for DDSPublisher:



## Public Member Functions

- virtual [DDSDDataWriter](#) \* [create\\_datawriter](#) ([DDSTopic](#) \*topic, const [DDS\\_DataWriterQos](#) &qos, [DDSDDataWriterListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
Creates a [DDSDDataWriter](#) that will be attached and belong to the [DDSPublisher](#).
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_datawriter](#) ([DDSDDataWriter](#) \*a\_datawriter)=0  
Deletes a [DDSDDataWriter](#) that belongs to the [DDSPublisher](#).
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_default\\_datawriter\\_qos](#) ([DDS\\_DataWriterQos](#) &qos)=0  
Copies the default [DDS\\_DataWriterQos](#) values into the provided [DDS\\_DataWriterQos](#) instance.
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_default\\_datawriter\\_qos](#) (const [DDS\\_DataWriterQos](#) &qos)=0  
Sets the default [DDS\\_DataWriterQos](#) values for this publisher.
- virtual [DDSDDataWriter](#) \* [lookup\\_datawriter](#) (const char \*topic\_name)=0  
Retrieves the [DDSDDataWriter](#) for a specific [DDSTopic](#).
- virtual [DDSDDataWriter](#) \* [lookup\\_datawriter\\_by\\_name](#) (const char \*writer\_name)=0  
Retrieves a [DDSDDataWriter](#) by its entity name within this [DDSPublisher](#)
- virtual [DDSDomainParticipant](#) \* [get\\_participant](#) ()=0  
Returns the [DDSDomainParticipant](#) to which the [DDSPublisher](#) belongs.
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_qos](#) ([DDS\\_PublisherQos](#) &qos)=0  
Gets the publisher QoS.
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_qos](#) (const [DDS\\_PublisherQos](#) &qos)=0  
Sets the publisher QoS.
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_contained\\_entities](#) ()=0  
Deletes all the entities that were created by means of the "create" operation on the [DDSPublisher](#).
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_listener](#) (const [DDSPublisherListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
Sets the publisher listener.
- virtual [DDSPublisherListener](#) \* [get\\_listener](#) ()=0  
Get the publisher listener.

## Public Member Functions inherited from [DDSEntity](#)

- [DDS\\_ReturnCode\\_t](#) [enable](#) ()  
Enables the [DDSEntity](#).
- [DDSStatusCondition](#) \* [get\\_statuscondition](#) ()  
Return the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).
- [DDS\\_StatusMask](#) [get\\_status\\_changes](#) ()  
Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.
- [DDS\\_InstanceHandle\\_t](#) [get\\_instance\\_handle](#) ()  
Allows access to the [DDS\\_InstanceHandle\\_t](#) associated with the [DDSEntity](#).

### 7.106.1 Detailed Description

<<[interface](#)>> A publisher is the object responsible for the actual dissemination of publications.

QoS:

[DDS\\_PublisherQos](#)

Listener:

[DDSPublisherListener](#)

A publisher acts on the behalf of one or several [DDSDataWriter](#) objects that belong to it. When it is informed of a change to the data associated with one of its [DDSDataWriter](#) objects, it decides when it is appropriate to actually send the data-update message. In making this decision, it considers any extra information that goes with the data (timestamp, writer, etc.) as well as the QoS of the [DDSPublisher](#) and the [DDSDataWriter](#).

The following operations may be called even if the [DDSPublisher](#) is not enabled. Other operations will fail with the value [DDS\\_RETCODE\\_NOT\\_ENABLED](#) if called on a disabled [DDSPublisher](#):

- The base-class operations [DDSEntity::enable](#)
- [DDSPublisher::create\\_datawriter](#), [DDSPublisher::delete\\_datawriter](#), [DDSPublisher::delete\\_contained\\_entities](#),

## 7.106.2 Member Function Documentation

### create\_datawriter()

```
virtual DDSDataWriter * DDSPublisher::create\_datawriter (
    DDSTopic * topic,
    const DDS\_DataWriterQos & qos,
    DDSDataWriterListener * listener,
    DDS\_StatusMask mask) [pure virtual]
```

Creates a [DDSDataWriter](#) that will be attached and belong to the [DDSPublisher](#).

#### Precondition

If publisher is enabled, topic must have been enabled. Otherwise, this operation will fail and no [DDSDataWriter](#) will be created.

The given [DDSTopic](#) must have been created from the same participant as this publisher. If it was created from a different participant, this method will fail.

#### Parameters

|              |                                                                                                                                                                                                                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>topic</i> | <<in>> The <a href="#">DDSTopic</a> that the <a href="#">DDSDataWriter</a> will be associated with. Cannot be NULL.                                                                                                                                                                                                   |
| <i>qos</i>   | <<in>> QoS to be used for creating the new <a href="#">DDSDataWriter</a> . The special value <a href="#">DDS_DATAWRITER_QOS_DEFAULT</a> can be used to indicate that the <a href="#">DDSDataWriter</a> should be created with the default <a href="#">DDS_DataWriterQos</a> set in the <a href="#">DDSPublisher</a> . |

#### Parameters

|                 |                                                                        |
|-----------------|------------------------------------------------------------------------|
| <i>listener</i> | <<in>> The listener of the <a href="#">DDSDataWriter</a> .             |
| <i>mask</i>     | <<in>>. Changes of communication status to be invoked on the listener. |

**Returns**

A [DDSDataWriter](#) of a derived class specific to the data type associated with the [DDSTopic](#) or NULL if an error occurred.

**See also**

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS\\_DataWriterQos](#) for rules on consistency among QoS

[DDS\\_DATAWRITER\\_QOS\\_DEFAULT](#)

**delete\_datawriter()**

```
virtual DDS_ReturnCode_t DDSPublisher::delete_datawriter (
    DDSDataWriter * a_datawriter) [pure virtual]
```

Deletes a [DDSDataWriter](#) that belongs to the [DDSPublisher](#).

The deletion of the [DDSDataWriter](#) will automatically unregister all instances.

**Precondition**

If the [DDSDataWriter](#) does not belong to the [DDSPublisher](#), the operation will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**Postcondition**

Listener installed on the [DDSDataWriter](#) will not be called after this method completes successfully.

**Parameters**

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>a_datawriter</i> | <<in>> The <a href="#">DDSDataWriter</a> to be deleted. |
|---------------------|---------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#) or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**get\_default\_datawriter\_qos()**

```
virtual DDS_ReturnCode_t DDSPublisher::get_default_datawriter_qos (
    DDS_DataWriterQos & qos) [pure virtual]
```

Copies the default [DDS\\_DataWriterQos](#) values into the provided [DDS\\_DataWriterQos](#) instance.

The retrieved qos will match the set of values specified on the last successful call to [DDSPublisher::set\\_default\\_datawriter\\_qos](#) or else, if the call was never made, the default values from its owning [DDSDomainParticipant](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

**MT Safety:**

UNSAFE. It is not safe to retrieve the default QoS value from a [DDSPublisher](#) while another thread may be simultaneously calling [DDSPublisher::get\\_default\\_datawriter\\_qos](#) or calling [DDSPublisher::create\\_datawriter](#) with [DDS\\_DATAWRITER\\_QOS\\_DEFAULT](#) as the qos parameter.

## Parameters

|            |                                                                       |
|------------|-----------------------------------------------------------------------|
| <i>qos</i> | << <i>inout</i> >> <a href="#">DDS_DataWriterQos</a> to be filled-up. |
|------------|-----------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#)

## See also

[DDS\\_DATAWRITER\\_QOS\\_DEFAULT](#)

[DDSPublisher::create\\_datawriter](#)

**set\_default\_datawriter\_qos()**

```
virtual DDS\_ReturnCode\_t DDSPublisher::set_default_datawriter_qos (
    const DDS\_DataWriterQos & qos) [pure virtual]
```

Sets the default [DDS\\_DataWriterQos](#) values for this publisher.

This call causes the default values inherited from the owning [DDSDomainParticipant](#) to be overridden.

This default value will be used for newly created [DDSDDataWriter](#) if [DDS\\_DATAWRITER\\_QOS\\_DEFAULT](#) is specified as the *qos* parameter when [DDSPublisher::create\\_datawriter](#) is called.

## Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

## MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a [DDSPublisher](#) while another thread may be simultaneously calling [DDSPublisher::get\\_default\\_datawriter\\_qos](#) or calling [DDSPublisher::create\\_datawriter](#) with [DDS\\_DATAWRITER\\_QOS\\_DEFAULT](#) as the *qos* parameter.

## Parameters

|            |                                                                                                                                                                                                                                                                                                                            |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> Default qos to be set. The special value <a href="#">DDS_DATAREADER_QOS_DEFAULT</a> may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would have used if <a href="#">DDSPublisher::set_default_datawriter_qos</a> had never been called. |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

**lookup\_datawriter()**

```
virtual DDSDDataWriter * DDSPublisher::lookup_datawriter (
    const char * topic_name) [pure virtual]
```

Retrieves the [DDSDDataWriter](#) for a specific [DDSTopic](#).

This returned [DDSDDataWriter](#) is either enabled or disabled.



**Parameters**

|                   |                                                                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <i>topic_name</i> | <<in>> Name of the <a href="#">DDSTopic</a> associated with the <a href="#">DDSDataWriter</a> that is to be looked up. Cannot be NULL. |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------|

**Returns**

A [DDSDataWriter](#) that belongs to the [DDSPublisher](#) attached to the [DDSTopic](#) with `topic_name`. If no such [DDSDataWriter](#) exists, this operation returns NULL.

If more than one [DDSDataWriter](#) is attached to the [DDSPublisher](#) with the same `topic_name`, then this operation may return any one of them.

**MT Safety:**

UNSAFE. It is not safe to lookup a [DDSDataWriter](#) in one thread while another thread is simultaneously creating or destroying that [DDSDataWriter](#).

**lookup\_datawriter\_by\_name()**

```
virtual DDSDataWriter * DDSPublisher::lookup_datawriter_by_name (
    const char * writer_name) [pure virtual]
```

Retrieves a [DDSDataWriter](#) by its entity name within this [DDSPublisher](#)

This returned [DDSDataWriter](#) is either enabled or disabled.

Every [DDSDataWriter](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY\_NAME.

This operation retrieves a [DDSDataWriter](#) within the [DDSPublisher](#) given the entity's name. If there are several [DDSDataWriter](#) with the same name within the [DDSPublisher](#), this function returns the first matching occurrence.

**Parameters**

|                    |                                                                           |
|--------------------|---------------------------------------------------------------------------|
| <i>writer_name</i> | <<in>> Entity name of the <a href="#">DDSDataWriter</a> . Cannot be NULL. |
|--------------------|---------------------------------------------------------------------------|

**Returns**

The first [DDSDataWriter](#) found with the specified name or NULL if it is not found.

**MT Safety:**

UNSAFE. It is not safe to lookup a [DDSDataWriter](#) in one thread while another thread is simultaneously creating or destroying that [DDSDataWriter](#).

**get\_participant()**

```
virtual DDSDomainParticipant * DDSPublisher::get_participant () [pure virtual]
```

Returns the [DDSDomainParticipant](#) to which the [DDSPublisher](#) belongs.

**Returns**

the [DDSDomainParticipant](#) to which the [DDSPublisher](#) belongs.

**get\_qos()**

```
virtual DDS_ReturnCode_t DDSPublisher::get_qos (
    DDS_PublisherQos & qos) [pure virtual]
```

Gets the publisher QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

**Parameters**

|            |                                                                   |
|------------|-------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> <a href="#">DDS_PublisherQos</a> to be filled in. |
|------------|-------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**See also**

[get\\_qos \(abstract\)](#)

**set\_qos()**

```
virtual DDS_ReturnCode_t DDSPublisher::set_qos (
    const DDS_PublisherQos & qos) [pure virtual]
```

Sets the publisher QoS.

This operation modifies the QoS of the [DDSPublisher](#).

The [DDS\\_PublisherQos::entity\\_factory](#) can be changed. The other policies are immutable.

## Parameters

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> <a href="#">DDS_PublisherQos</a> to be set to. Policies must be consistent. Policies cannot be changed after <a href="#">DDSPublisher</a> is enabled. The special value <a href="#">DDS_PUBLISHER_QOS_DEFAULT</a> can be used to indicate that the QoS of the <a href="#">DDSPublisher</a> should be changed to match the current default <a href="#">DDS_PublisherQos</a> set in the <a href="#">DDSDomainParticipant</a> . |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#).

## See also

[DDS\\_PublisherQos](#) for rules on consistency among QoS  
[set\\_qos](#) (abstract)

**delete\_contained\_entities()**

```
virtual DDS_ReturnCode_t DDSPublisher::delete_contained_entities () [pure virtual]
```

Deletes all the entities that were created by means of the "create" operation on the [DDSPublisher](#).

Deletes all contained [DDSDataWriter](#) objects. Once [DDSPublisher::delete\\_contained\\_entities](#) completes successfully, the application may delete the [DDSPublisher](#), knowing that it has no contained [DDSDataWriter](#) objects.

The operation will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) if any of the contained entities is in a state where it cannot be deleted.

## Returns

One of the [Standard Return Codes](#) or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**set\_listener()**

```
virtual DDS_ReturnCode_t DDSPublisher::set_listener (
    const DDSPublisherListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Sets the publisher listener.

## Parameters

|                 |                                                                                                           |
|-----------------|-----------------------------------------------------------------------------------------------------------|
| <i>listener</i> | << <i>in</i> >> <a href="#">DDSPublisherListener</a> to set to.                                           |
| <i>mask</i>     | << <i>in</i> >> <a href="#">DDS_StatusMask</a> associated with the <a href="#">DDSPublisherListener</a> . |

## Returns

One of the [Standard Return Codes](#)

## See also

[set\\_listener](#) (abstract)

**get\_listener()**

```
virtual DDSPublisherListener * DDSPublisher::get_listener () [pure virtual]
```

Get the publisher listener.

**Returns**

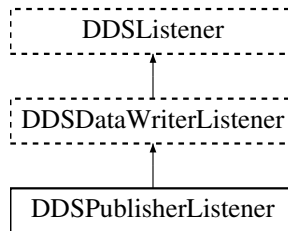
DDSPublisherListener of the DDSPublisher.

**7.107 DDSPublisherListener Class Reference**

<<interface>> DDSListener for DDSPublisher status.

```
#include <dds_cpp_publication.hxx>
```

Inheritance diagram for DDSPublisherListener:

**Additional Inherited Members****Public Member Functions inherited from DDSDataWriterListener**

- virtual void `on_offered_deadline_missed` (DDSDataWriter \*writer, const DDS\_OfferedDeadlineMissedStatus &status)  
*Handles the DDS\_OFFERED\_DEADLINE\_MISSED\_STATUS status.*
- virtual void `on_liveliness_lost` (DDSDataWriter \*writer, const DDS\_LivelinessLostStatus &status)  
*Handles the DDS\_LIVELINESS\_LOST\_STATUS status.*
- virtual void `on_offered_incompatible_qos` (DDSDataWriter \*writer, const DDS\_OfferedIncompatibleQosStatus &status)  
*Handles the DDS\_OFFERED\_INCOMPATIBLE\_QOS\_STATUS status.*
- virtual void `on_publication_matched` (DDSDataWriter \*writer, const DDS\_PublicationMatchedStatus &status)  
*Handles the DDS\_PUBLICATION\_MATCHED\_STATUS status.*
- virtual void `on_reliable_reader_activity_changed` (DDSDataWriter \*writer, const DDS\_ReliableReaderActivityChangedStatus &status)  
 <<eXtension>> A matched reliable reader has become active or become inactive.
- virtual void `on_reliable_sample_unacknowledged` (DDSDataWriter \*writer, const DDS\_ReliableSampleUnacknowledgedStatus &status)  
 <<eXtension>> A matched reliable reader has become active or become inactive.
- virtual void `on_sample_removed` (DDSDataWriter \*writer, const DDS\_Cookie\_t &cookie)  
 <<eXtension>> Called when a sample is removed from the DataWriter queue.

### 7.107.1 Detailed Description

<<*interface*>> [DDSListener](#) for [DDSPublisher](#) status.

Entity:

[DDSPublisher](#)

Status:

[DDS\\_LIVELINESS\\_LOST\\_STATUS](#), [DDS\\_LivelinessLostStatus](#);  
[DDS\\_OFFERED\\_DEADLINE\\_MISSED\\_STATUS](#), [DDS\\_OfferedDeadlineMissedStatus](#);  
[DDS\\_OFFERED\\_INCOMPATIBLE\\_QOS\\_STATUS](#), [DDS\\_OfferedIncompatibleQosStatus](#);  
[DDS\\_PUBLICATION\\_MATCHED\\_STATUS](#), [DDS\\_PublicationMatchedStatus](#);

See also

[DDSListener](#)

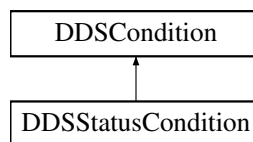
[Status Kinds](#)

## 7.108 DDSStatusCondition Class Reference

<<*interface*>> A specific [DDSCondition](#) that is associated with each [DDSEntity](#).

```
#include <dds_cpp_infrastructure.hxx>
```

Inheritance diagram for DDSStatusCondition:



### Public Member Functions

- [DDS\\_StatusMask get\\_enabled\\_statuses](#) ()  
*Get the list of statuses enabled on an [DDSEntity](#).*
- [DDS\\_ReturnCode\\_t set\\_enabled\\_statuses](#) ([DDS\\_StatusMask](#) mask)  
*This operation defines the list of communication statuses that determine the *trigger\_value* of the [DDSStatusCondition](#).*

### Friends

- class [DDSEntity](#)

### 7.108.1 Detailed Description

<<*interface*>> A specific [DDSCondition](#) that is associated with each [DDSEntity](#).

The `trigger_value` of the [DDSStatusCondition](#) depends on the communication status of that entity (e.g., arrival of data, loss of information, etc.), 'filtered' by the set of `enabled_statuses` on the [DDSStatusCondition](#).

This release supports the following subset of communication statuses that can be associated with a [DDSStatusCondition](#) :

[DDS\\_DATA\\_AVAILABLE\\_STATUS](#)

See also

[Status Kinds](#)

[DDSWaitSet](#), [DDSCondition](#)

[DDSListener](#)

### 7.108.2 Member Function Documentation

#### **get\_enabled\_statuses()**

```
DDS_StatusMask DDSStatusCondition::get_enabled_statuses ()
```

Get the list of statuses enabled on an [DDSEntity](#).

Returns

list of enabled statuses.

#### **set\_enabled\_statuses()**

```
DDS_ReturnCode_t DDSStatusCondition::set_enabled_statuses (
    DDS_StatusMask mask)
```

This operation defines the list of communication statuses that determine the `trigger_value` of the [DDSStatusCondition](#).

This operation may change the `trigger_value` of the [DDSStatusCondition](#).

[DDSWaitSet](#) objects' behavior depends on the changes of the `trigger_value` of their attached conditions. Therefore, any [DDSWaitSet](#) to which the [DDSStatusCondition](#) is attached is potentially affected by this operation.

If this function is not invoked, the default list of enabled statuses includes all the statuses.

## Parameters

|             |                                                                                  |
|-------------|----------------------------------------------------------------------------------|
| <i>mask</i> | << <i>in</i> >> the list of enables statuses (see <a href="#">Status Kinds</a> ) |
|-------------|----------------------------------------------------------------------------------|

## Returns

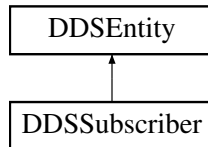
One of the [Standard Return Codes](#)

## 7.109 DDSSubscriber Class Reference

<<*interface*>> A subscriber is the object responsible for actually receiving data from a subscription.

```
#include <dds_cpp_subscription.hxx>
```

Inheritance diagram for DDSSubscriber:



### Public Member Functions

- virtual [DDSDataReader](#) \* [create\\_datareader](#) ([DDSTopicDescription](#) \*topic, const [DDS\\_DataReaderQos](#) &qos, [DDSDataReaderListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
Creates a [DDSDataReader](#) that will be attached and belong to the [DDSSubscriber](#).
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_datareader](#) ([DDSDataReader](#) \*a\_datareader)=0  
Deletes a [DDSDataReader](#) that belongs to the [DDSSubscriber](#).
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_default\\_datareader\\_qos](#) ([DDS\\_DataReaderQos](#) &qos)=0  
Copies the default [DDS\\_DataReaderQos](#) values into the provided [DDS\\_DataReaderQos](#) instance.
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_default\\_datareader\\_qos](#) (const [DDS\\_DataReaderQos](#) &qos)=0  
Sets the default [DDS\\_DataReaderQos](#) values for this subscriber.
- virtual [DDS\\_ReturnCode\\_t](#) [delete\\_contained\\_entities](#) ()=0  
Deletes all the entities that were created by means of the "create" operation on the [DDSSubscriber](#).
- virtual [DDSDataReader](#) \* [lookup\\_datareader](#) (const char \*topic\_name)=0  
Retrieves an existing [DDSDataReader](#).
- virtual [DDSDataReader](#) \* [lookup\\_datareader\\_by\\_name](#) (const char \*reader\_name)=0  
Retrieves a [DDSDataReader](#) by its entity name within this [DDSSubscriber](#)
- virtual [DDSDomainParticipant](#) \* [get\\_participant](#) ()=0  
Returns the [DDSDomainParticipant](#) to which the [DDSSubscriber](#) belongs.
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_qos](#) (const [DDS\\_SubscriberQos](#) &qos)=0  
Sets the subscriber QoS.
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_qos](#) ([DDS\\_SubscriberQos](#) &qos)=0  
Gets the subscriber QoS.
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_listener](#) (const [DDSSubscriberListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
Sets the subscriber listener.
- virtual [DDSSubscriberListener](#) \* [get\\_listener](#) ()=0  
Get the subscriber listener.

**Public Member Functions inherited from [DDSEntity](#)**

- [DDS\\_ReturnCode\\_t enable](#) ()  
*Enables the [DDSEntity](#).*
- [DDSStatusCondition \\* get\\_statuscondition](#) ()  
*Return the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).*
- [DDS\\_StatusMask get\\_status\\_changes](#) ()  
*Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.*
- [DDS\\_InstanceHandle\\_t get\\_instance\\_handle](#) ()  
*Allows access to the [DDS\\_InstanceHandle\\_t](#) associated with the [DDSEntity](#).*

**7.109.1 Detailed Description**

<<[interface](#)>> A subscriber is the object responsible for actually receiving data from a subscription.

QoS:

[DDS\\_SubscriberQos](#)

Status:

[DDS\\_DATA\\_ON\\_READERS\\_STATUS](#)

Listener:

[DDSSubscriberListener](#)

A subscriber acts on the behalf of one or several [DDSDataReader](#) objects that are related to it. When it receives data (from the other parts of the system), it builds the list of concerned [DDSDataReader](#) objects and then indicates to the application that data is available through its listener.

The application can access the list of concerned [DDSDataReader](#) objects through the operation [get\\_datareaders\(\)](#) and then access the data available through operations on the [DDSDataReader](#).

The following operations may be called even if the [DDSSubscriber](#) is not enabled. Other operations will the value [DDS\\_RETCODE\\_NOT\\_ENABLED](#) if called on a disabled [DDSSubscriber](#):

- The base-class operations [DDSEntity::enable](#)
- [DDSSubscriber::create\\_datareader](#), [DDSSubscriber::delete\\_datareader](#), [DDSSubscriber::delete\\_contained\\_entities](#).

All operations except for the base-class operations [enable\(\)](#) and [create\\_datareader\(\)](#) may fail with [DDS\\_RETCODE\\_NOT\\_ENABLED](#).



7.109.2 Member Function Documentation

create\_datareader()

```
virtual DDSDataReader * DDSSubscriber::create_datareader (
    DDSTopicDescription * topic,
    const DDS_DataReaderQos & qos,
    DDSDataReaderListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Creates a [DDSDataReader](#) that will be attached and belong to the [DDSSubscriber](#).

Precondition

If subscriber is enabled, the topic must be enabled. If it is not, this operation will fail and no [DDSDataReader](#) will be created.

The given [DDSTopicDescription](#) must have been created from the same participant as this subscriber. If it was created from a different participant, this method will return NULL.

Parameters

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>topic</i>    | << <i>in</i> >> The <a href="#">DDSTopicDescription</a> that the <a href="#">DDSDataReader</a> will be associated with. Cannot be NULL.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>qos</i>      | << <i>in</i> >> The qos of the <a href="#">DDSDataReader</a> . The special value <a href="#">DDS_DATAREADER_QOS_DEFAULT</a> can be used to indicate that the <a href="#">DDSDataReader</a> should be created with the default <a href="#">DDS_DataReaderQos</a> set in the <a href="#">DDSSubscriber</a> . If <a href="#">DDSTopicDescription</a> is of type <a href="#">DDSTopic</a> , the special value <a href="#">DDS_DATAREADER_QOS_USE_TOPIC_QOS</a> can be used to indicate that the <a href="#">DDSDataReader</a> should be created with the combination of the default <a href="#">DDS_DataReaderQos</a> set on the <a href="#">DDSSubscriber</a> and the <a href="#">DDS_TopicQos</a> . |
| <i>listener</i> | << <i>in</i> >> The listener of the <a href="#">DDSDataReader</a> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <i>mask</i>     | << <i>in</i> >>. Changes of communication status to be invoked on the listener.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

Returns

A [DDSDataReader](#) of a derived class specific to the data-type associated with the [DDSTopic](#) or NULL if an error occurred.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS\\_DataReaderQos](#) for rules on consistency among QoS

**delete\_datareader()**

```
virtual DDS_ReturnCode_t DDSSubscriber::delete_datareader (
    DDSDataReader * a_datareader) [pure virtual]
```

Deletes a [DDSDataReader](#) that belongs to the [DDSSubscriber](#).

**Precondition**

If the [DDSDataReader](#) does not belong to the [DDSSubscriber](#), or if there are any existing objects that are attached to the [DDSDataReader](#), or if there are outstanding loans on samples (as a result of a call to [read\(\)](#), [take\(\)](#), or one of the variants thereof), the operation fails with the error [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**Postcondition**

Listener installed on the [DDSDataReader](#) will not be called after this method completes successfully.

**Parameters**

|                     |                                                                  |
|---------------------|------------------------------------------------------------------|
| <i>a_datareader</i> | << <i>in</i> >> The <a href="#">DDSDataReader</a> to be deleted. |
|---------------------|------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#) or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

**get\_default\_datareader\_qos()**

```
virtual DDS_ReturnCode_t DDSSubscriber::get_default_datareader_qos (
    DDS_DataReaderQos & qos) [pure virtual]
```

Copies the default [DDS\\_DataReaderQos](#) values into the provided [DDS\\_DataReaderQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSSubscriber::set\\_default\\_datareader\\_qos](#), or else, if the call was never made, the default values from its owning [DDSDomainParticipant](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

**MT Safety:**

UNSAFE. It is not safe to retrieve the default QoS value from a subscriber while another thread may be simultaneously calling [DDSSubscriber::get\\_default\\_datareader\\_qos](#) or calling [DDSSubscriber::create\\_datareader](#) with [DDS\\_DATAREADER\\_QOS\\_DEFAULT](#) as the `qos` parameter.

**Parameters**

|            |                                                                       |
|------------|-----------------------------------------------------------------------|
| <i>qos</i> | << <i>inout</i> >> <a href="#">DDS_DataReaderQos</a> to be filled-up. |
|------------|-----------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**See also**

[DDS\\_DATAREADER\\_QOS\\_DEFAULT](#)  
[DDSSubscriber::create\\_datareader](#)

## set\_default\_datareader\_qos()

```
virtual DDS_ReturnCode_t DDSSubscriber::set_default_datareader_qos (
    const DDS_DataReaderQos & qos) [pure virtual]
```

Sets the default [DDS\\_DataReaderQos](#) values for this subscriber.

This call causes the default values inherited from the owning [DDSDomainParticipant](#) to be overridden.

This default value will be used for newly created [DDSDataReader](#) if [DDS\\_DATAREADER\\_QOS\\_DEFAULT](#) is specified as the `qos` parameter when [DDSSubscriber::create\\_datareader](#) is called.

### Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

### MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a subscriber while another thread may be simultaneously calling [DDSSubscriber::get\\_default\\_datareader\\_qos](#) or calling [DDSSubscriber::create\\_datareader](#) with [DDS\\_DATAREADER\\_QOS\\_DEFAULT](#) as the `qos` parameter.

### Parameters

|            |                                                                                                                                                                                                                                                                                                                                                                        |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> The default <a href="#">DDS_DataReaderQos</a> to be set to. The special value <a href="#">DDS_DATAREADER_QOS_DEFAULT</a> may be passed as <code>qos</code> to indicate that the default QoS should be reset back to the initial values the factory would have used if <a href="#">DDSSubscriber::set_default_datareader_qos</a> had never been called. |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#)

## delete\_contained\_entities()

```
virtual DDS_ReturnCode_t DDSSubscriber::delete_contained_entities () [pure virtual]
```

Deletes all the entities that were created by means of the "create" operation on the [DDSSubscriber](#).

Deletes all contained [DDSDataReader](#) objects. This pattern is applied recursively. In this manner, the operation [DDSSubscriber::delete\\_contained\\_entities](#) on the [DDSSubscriber](#) will end up deleting all the entities recursively contained in the [DDSSubscriber](#), that is also the objects belonging to the contained [DDSDataReader](#).

The operation will fail with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) if any of the contained entities is in a state where it cannot be deleted. This will occur, for example, if a contained [DDSDataReader](#) cannot be deleted because the application has called a [FooDataReader::read](#) or [FooDataReader::take](#) operation and has not called the corresponding [FooDataReader::return\\_loan](#) operation to return the loaned samples.

Once [DDSSubscriber::delete\\_contained\\_entities](#) completes successfully, the application may delete the [DDSSubscriber](#), knowing that it has no contained [DDSDataReader](#) objects.

### Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#)

**lookup\_datareader()**

```
virtual DDSDataReader * DDSSubscriber::lookup_datareader (
    const char * topic_name) [pure virtual]
```

Retrieves an existing [DDSDataReader](#).

Use this operation on the built-in [DDSSubscriber](#) ([Discovery](#)) to access the built-in [DDSDataReader](#) entities for the built-in topics.

The built-in [DDSDataReader](#) is created when this operation is called on a built-in topic for the first time. The built-in [DDSDataReader](#) is deleted automatically when the [DDSDomainParticipant](#) is deleted.

To ensure that builtin [DDSDataReader](#) entities receive all the discovery traffic, it is suggested that you lookup the builtin [DDSDataReader](#) before the [DDSDomainParticipant](#) is enabled. Looking up builtin [DDSDataReader](#) may implicitly register builtin transports due to creation of [DDSDataReader](#). Therefore, if you want to modify builtin transport properties, do so *before* using this operation.

Therefore the suggested sequence when looking up builtin DataReaders is:

- Create a disabled [DDSDomainParticipant](#).
- (optional) Modify builtin transport properties
- Call `DDS_DomainParticipant_get_builtin_subscriber()`.
- Call `lookup_datareader()`.
- Call `enable()` on the DomainParticipant.

**Parameters**

|                   |                                                                                                                                                  |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>topic_name</i> | << <i>in</i> >> Name of the <a href="#">DDSTopicDescription</a> that the retrieved <a href="#">DDSDataReader</a> is attached to. Cannot be NULL. |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|

**Returns**

A [DDSDataReader](#) that belongs to the [DDSSubscriber](#) attached to the [DDSTopicDescription](#) with *topic\_name*. If no such [DDSDataReader](#) exists, this operation returns NULL.

The returned [DDSDataReader](#) may be enabled or disabled.

If more than one [DDSDataReader](#) is attached to the [DDSSubscriber](#), this operation may return any one of them.

**MT Safety:**

UNSAFE. It is not safe to lookup a [DDSDataReader](#) in one thread while another thread is simultaneously creating or destroying that [DDSDataReader](#).

**lookup\_datareader\_by\_name()**

```
virtual DDSDataReader * DDSSubscriber::lookup_datareader_by_name (  
    const char * reader_name) [pure virtual]
```

Retrieves a [DDSDataReader](#) by its entity name within this [DDSSubscriber](#)

This returned [DDSDataReader](#) is either enabled or disabled.

Every [DDSDataReader](#) in the system has an entity name which is configured and stored in the EntityName policy, ENTITY\_NAME.

This operation retrieves a [DDSDataReader](#) within the [DDSSubscriber](#) given the entity's name. If there are several [DDSDataReader](#) with the same name within the [DDSSubscriber](#), this function returns the first matching occurrence.

## Parameters

|                    |                                                                            |
|--------------------|----------------------------------------------------------------------------|
| <i>reader_name</i> | <<in>> Entity name of the <a href="#">DDSDDataReader</a> . Cannot be NULL. |
|--------------------|----------------------------------------------------------------------------|

## Returns

The first [DDSDDataReader](#) found with the specified name or NULL if it is not found.

## MT Safety:

UNSAFE. It is not safe to lookup a [DDSDDataReader](#) in one thread while another thread is simultaneously creating or destroying that [DDSDDataReader](#).

**get\_participant()**

```
virtual DDSDomainParticipant * DDSSubscriber::get_participant () [pure virtual]
```

Returns the [DDSDomainParticipant](#) to which the [DDSSubscriber](#) belongs.

## Returns

the [DDSDomainParticipant](#) to which the [DDSSubscriber](#) belongs.

**set\_qos()**

```
virtual DDS\_ReturnCode\_t DDSSubscriber::set_qos (
    const DDS\_SubscriberQos & qos) [pure virtual]
```

Sets the subscriber QoS.

This operation modifies the QoS of the [DDSSubscriber](#).

The [DDS\\_SubscriberQos::entity\\_factory](#) can be changed. The other policies are immutable.

## Parameters

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>qos</i> | <<in>> <a href="#">DDS_SubscriberQos</a> to be set to. Policies must be consistent. Policies cannot be changed after <a href="#">DDSSubscriber</a> is enabled. The special value <a href="#">DDS_SUBSCRIBER_QOS_DEFAULT</a> can be used to indicate that the QoS of the <a href="#">DDSSubscriber</a> should be changed to match the current default <a href="#">DDS_SubscriberQos</a> set in the <a href="#">DDSDomainParticipant</a> . |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#), or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#).

## See also

[DDS\\_SubscriberQos](#) for rules on consistency among QoS

[set\\_qos](#) (abstract)

**get\_qos()**

```
virtual DDS_ReturnCode_t DDSSubscriber::get_qos (
    DDS_SubscriberQos & qos) [pure virtual]
```

Gets the subscriber QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

**Parameters**

|            |                                                                    |
|------------|--------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> <a href="#">DDS_SubscriberQos</a> to be filled in. |
|------------|--------------------------------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**See also**

[get\\_qos \(abstract\)](#)

**set\_listener()**

```
virtual DDS_ReturnCode_t DDSSubscriber::set_listener (
    const DDSSubscriberListener * listener,
    DDS_StatusMask mask) [pure virtual]
```

Sets the subscriber listener.

**Parameters**

|                 |                                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------|
| <i>listener</i> | << <i>in</i> >> <a href="#">DDSSubscriberListener</a> to set to.                                           |
| <i>mask</i>     | << <i>in</i> >> <a href="#">DDS_StatusMask</a> associated with the <a href="#">DDSSubscriberListener</a> . |

**Returns**

One of the [Standard Return Codes](#)

**See also**

[set\\_listener \(abstract\)](#)

**get\_listener()**

```
virtual DDSSubscriberListener * DDSSubscriber::get_listener () [pure virtual]
```

Get the subscriber listener.

**Returns**

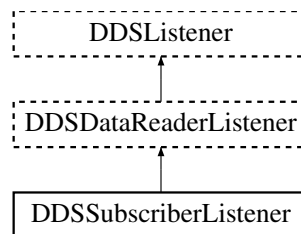
[DDSSubscriberListener](#) of the [DDSSubscriber](#).

**7.110 DDSSubscriberListener Class Reference**

<<[interface](#)>> [DDSListener](#) for status about a subscriber.

```
#include <dds_cpp_subscription.hxx>
```

Inheritance diagram for DDSSubscriberListener:

**Public Member Functions**

- virtual void [on\\_data\\_on\\_readers](#) ([DDSSubscriber](#) \*)  
*Handles the [DDS\\_DATA\\_ON\\_READERS\\_STATUS](#) communication status.*

**Public Member Functions inherited from [DDSDDataReaderListener](#)**

- virtual void [on\\_data\\_available](#) ([DDSDDataReader](#) \*)  
*Handle the [DDS\\_DATA\\_AVAILABLE\\_STATUS](#) communication status.*
- virtual void [on\\_requested\\_deadline\\_missed](#) ([DDSDDataReader](#) \*, const [DDS\\_RequestedDeadlineMissedStatus](#) &)  
*Handles the [DDS\\_REQUESTED\\_DEADLINE\\_MISSED\\_STATUS](#) communication status.*
- virtual void [on\\_liveliness\\_changed](#) ([DDSDDataReader](#) \*, const [DDS\\_LivelinessChangedStatus](#) &)  
*Handles the [DDS\\_LIVELINESS\\_CHANGED\\_STATUS](#) communication status.*
- virtual void [on\\_requested\\_incompatible\\_qos](#) ([DDSDDataReader](#) \*, const [DDS\\_RequestedIncompatibleQosStatus](#) &)  
*Handles the [DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#) communication status.*
- virtual void [on\\_sample\\_rejected](#) ([DDSDDataReader](#) \*, const [DDS\\_SampleRejectedStatus](#) &)  
*Handles the [DDS\\_SAMPLE\\_REJECTED\\_STATUS](#) communication status.*
- virtual void [on\\_subscription\\_matched](#) ([DDSDDataReader](#) \*, const [DDS\\_SubscriptionMatchedStatus](#) &)



Handles the [DDS\\_SUBSCRIPTION\\_MATCHED\\_STATUS](#) communication status.

- virtual void [on\\_sample\\_lost](#) (DDSDataReader \*, const [DDS\\_SampleLostStatus](#) &)

Handles the [DDS\\_SAMPLE\\_LOST\\_STATUS](#) communication status.

- virtual void [on\\_instance\\_replaced](#) (DDSDataReader \*, const [DDS\\_DataReaderInstanceReplacedStatus](#) &)

Handles the [DDS\\_INSTANCE\\_REPLACED\\_STATUS](#) communication status.

- virtual [DDS\\_Boolean](#) [on\\_before\\_sample\\_deserialize](#) (DDSDataReader \*, NDDS\_Type\_Plugin \*, CDR\_Stream↔\_t \*, [DDS\\_Boolean](#) \*)

Callback to filter a received sample based on serialized data

- virtual [DDS\\_Boolean](#) [on\\_before\\_sample\\_commit](#) (DDSDataReader \*, const void \*const, const struct [DDS\\_SampleInfo](#) \*const, [DDS\\_Boolean](#) \*)

Callback to filter a received sample based on deserialized data

### 7.110.1 Detailed Description

<<[interface](#)>> [DDSListener](#) for status about a subscriber.

Entity:

[DDSSubscriber](#)

Status:

[DDS\\_DATA\\_AVAILABLE\\_STATUS](#);  
[DDS\\_DATA\\_ON\\_READERS\\_STATUS](#);  
[DDS\\_LIVELINESS\\_CHANGED\\_STATUS](#), [DDS\\_LivelinessChangedStatus](#);  
[DDS\\_REQUESTED\\_DEADLINE\\_MISSED\\_STATUS](#), [DDS\\_RequestedDeadlineMissedStatus](#);  
[DDS\\_REQUESTED\\_INCOMPATIBLE\\_QOS\\_STATUS](#), [DDS\\_RequestedIncompatibleQosStatus](#);  
[DDS\\_SAMPLE\\_LOST\\_STATUS](#), [DDS\\_SampleLostStatus](#);  
[DDS\\_SAMPLE\\_REJECTED\\_STATUS](#), [DDS\\_SampleRejectedStatus](#);  
[DDS\\_SUBSCRIPTION\\_MATCHED\\_STATUS](#), [DDS\\_SubscriptionMatchedStatus](#);

See also

[DDSListener](#)

[Status Kinds](#)

### 7.110.2 Member Function Documentation

#### [on\\_data\\_on\\_readers\(\)](#)

```
virtual void DDSSubscriberListener::on_data_on_readers (
    DDSSubscriber * ) [inline], [virtual]
```

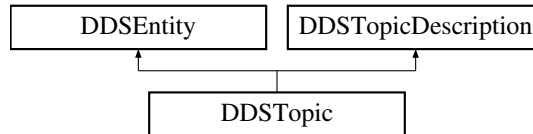
Handles the [DDS\\_DATA\\_ON\\_READERS\\_STATUS](#) communication status.

## 7.111 DDSTopic Class Reference

<<*interface*>> The most basic description of the data to be published and subscribed.

```
#include <dds_cpp_topic.hxx>
```

Inheritance diagram for DDSTopic:



### Public Member Functions

- virtual [DDSTopicDescription](#) \* [as\\_topicdescription](#) ()=0  
*Access a [DDSTopic](#)'s [DDSTopicDescription](#) supertype instance.*
- virtual [DDSEntity](#) \* [as\\_entity](#) ()=0  
*Access a [DDSTopic](#)'s [DDSEntity](#) supertype instance.*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_qos](#) (const [DDS\\_TopicQos](#) &qos)=0  
*Set the topic QoS.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_qos](#) ([DDS\\_TopicQos](#) &qos)=0  
*Get the topic QoS.*
- virtual [DDS\\_ReturnCode\\_t](#) [get\\_inconsistent\\_topic\\_status](#) ([DDS\\_InconsistentTopicStatus](#) &status)=0  
*Allows the application to retrieve the [DDS\\_INCONSISTENT\\_TOPIC\\_STATUS](#) status of a [DDSTopic](#).*
- virtual [DDS\\_ReturnCode\\_t](#) [set\\_listener](#) (const [DDSTopicListener](#) \*listener, [DDS\\_StatusMask](#) mask)=0  
*Set the topic listener.*
- virtual [DDSTopicListener](#) \* [get\\_listener](#) ()=0  
*Get the topic listener.*
- virtual const char \* [get\\_type\\_name](#) ()=0  
*Get the associated `type_name`.*
- virtual const char \* [get\\_name](#) ()=0  
*Get the name used to create this [DDSTopicDescription](#) .*
- virtual [DDSDomainParticipant](#) \* [get\\_participant](#) ()=0  
*Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.*

### Public Member Functions inherited from [DDSEntity](#)

- [DDS\\_ReturnCode\\_t](#) [enable](#) ()  
*Enables the [DDSEntity](#).*
- [DDSStatusCondition](#) \* [get\\_statuscondition](#) ()  
*Return the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).*
- [DDS\\_StatusMask](#) [get\\_status\\_changes](#) ()  
*Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.*
- [DDS\\_InstanceHandle\\_t](#) [get\\_instance\\_handle](#) ()  
*Allows access to the [DDS\\_InstanceHandle\\_t](#) associated with the [DDSEntity](#).*

## Static Public Member Functions

- static [DDSTopic](#) \* [narrow](#) ([DDSTopicDescription](#) \*topic\_description)  
*Narrow the given [DDSTopicDescription](#) pointer to a [DDSTopic](#) pointer.*

### 7.111.1 Detailed Description

<<[interface](#)>> The most basic description of the data to be published and subscribed.

QoS:

[DDS\\_TopicQos](#)

Status:

[DDS\\_INCONSISTENT\\_TOPIC\\_STATUS](#), [DDS\\_InconsistentTopicStatus](#)

Listener:

[DDSTopicListener](#)

A [DDSTopic](#) is identified by its name, which must be unique in the whole domain. In addition (by virtue of extending [DDSTopicDescription](#)) it fully specifies the type of the data that can be communicated when publishing or subscribing to the [DDSTopic](#).

[DDSTopic](#) is the only [DDSTopicDescription](#) that can be used for publications and therefore associated with a [DDSDataWriter](#).

The following operations may be called even if the [DDSTopic](#) is not enabled. Other operations will fail with the value [DDS\\_RETCODE\\_NOT\\_ENABLED](#) if called on a disabled [DDSTopic](#):

- [get\\_inconsistent\\_topic\\_status\(\)](#)

### 7.111.2 Member Function Documentation

#### [narrow\(\)](#)

```
static DDSTopic * DDSTopic::narrow (  
    DDSTopicDescription * topic_description) [static]
```

Narrow the given [DDSTopicDescription](#) pointer to a [DDSTopic](#) pointer.

Returns

[DDSTopic](#) if this [DDSTopicDescription](#) is a [DDSTopic](#). Otherwise, return NULL.

**as\_topicdescription()**

```
virtual DDS::TopicDescription * DDS::as_topicdescription () [pure virtual]
```

Access a [DDSTopic](#)'s [DDS::TopicDescription](#) supertype instance.

**Returns**

[DDSTopic](#)'s supertype [DDS::TopicDescription](#) instance

**as\_entity()**

```
virtual DDS::Entity * DDS::as_entity () [pure virtual]
```

Access a [DDSTopic](#)'s [DDS::Entity](#) supertype instance.

**Returns**

[DDSTopic](#)'s supertype [DDS::Entity](#) instance

**set\_qos()**

```
virtual DDS_ReturnCode_t DDS::set_qos (
    const DDS::TopicQos & qos) [pure virtual]
```

Set the topic QoS.

**Parameters**

|            |                                                                             |
|------------|-----------------------------------------------------------------------------|
| <i>qos</i> | << <i>in</i> >> Set of policies to be applied to <a href="#">DDSTopic</a> . |
|------------|-----------------------------------------------------------------------------|

Policies must be consistent. Policies cannot be changed after [DDSTopic](#) is enabled. The special value [DDS\\_TOPIC\\_QOS\\_DEFAULT](#) can be used to indicate that the QoS of the [DDSTopic](#) should be changed to match the current default [DDS::TopicQos](#) set in the [DDSDomainParticipant](#).

**Returns**

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_IMMUTABLE\\_POLICY](#) if immutable policy is changed, or [DDS\\_RETCODE\\_INCONSISTENT\\_POLICY](#) if policies are inconsistent

**See also**

[DDS::TopicQos](#) for rules on consistency among QoS  
[set\\_qos](#) (abstract)

**get\_qos()**

```
virtual DDS_ReturnCode_t DDS::get_qos (
    DDS::TopicQos & qos) [pure virtual]
```

Get the topic QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

**Parameters**

|            |                                         |
|------------|-----------------------------------------|
| <i>qos</i> | << <i>inout</i> >> QoS to be filled up. |
|------------|-----------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**See also**

[get\\_qos \(abstract\)](#)

**get\_inconsistent\_topic\_status()**

```
virtual DDS_ReturnCode_t DDSTopic::get_inconsistent_topic_status (  
    DDS_InconsistentTopicStatus & status) [pure virtual]
```

Allows the application to retrieve the [DDS\\_INCONSISTENT\\_TOPIC\\_STATUS](#) status of a [DDSTopic](#).

Retrieve the current [DDS\\_InconsistentTopicStatus](#)

**Parameters**

|               |                                            |
|---------------|--------------------------------------------|
| <i>status</i> | << <i>inout</i> >> Status to be retrieved. |
|---------------|--------------------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**See also**

[DDS\\_InconsistentTopicStatus](#)

**set\_listener()**

```
virtual DDS_ReturnCode_t DDSTopic::set_listener (  
    const DDSTopicListener * listener,  
    DDS_StatusMask mask) [pure virtual]
```

Set the topic listener.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>listener</i> | << <i>in</i> >> Listener to be installed on entity.                            |
| <i>mask</i>     | << <i>in</i> >> Changes of communication status to be invoked on the listener. |

**Returns**

One of the [Standard Return Codes](#)

**get\_listener()**

```
virtual DDSTopicListener * DDSTopic::get_listener () [pure virtual]
```

Get the topic listener.

**Returns**

Existing listener attached to the [DDSTopic](#).

**get\_type\_name()**

```
virtual const char * DDSTopic::get_type_name () [pure virtual]
```

Get the associated `type_name`.

The type name defines a locally unique type for the publication or the subscription.

The `type_name` corresponds to a unique string used to register a type via the [DDSDomainParticipant::register\\_type](#) method.

Thus, the `type_name` implies an association with a corresponding type and this [DDSTopicDescription](#).

**Returns**

the type name. The returned type name is valid until the [DDSTopicDescription](#) is deleted.

**Postcondition**

The result is NULL if self is NULL, otherwise result is non-NULL.

Implements [DDSTopicDescription](#).

**get\_name()**

```
virtual const char * DDSTopic::get_name () [pure virtual]
```

Get the name used to create this [DDSTopicDescription](#) .

**Returns**

the name used to create this [DDSTopicDescription](#). The returned topic name is valid until the [DDSTopicDescription](#) is deleted.

**Postcondition**

The result is NULL if self is NULL, otherwise result is non-NULL.

Implements [DDSTopicDescription](#).

**get\_participant()**

```
virtual DDSDomainParticipant * DDSTopic::get_participant () [pure virtual]
```

Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

**Returns**

The [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

**Postcondition**

The result is NULL if self is NULL, otherwise result is non-NULL.

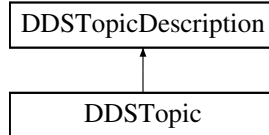
Implements [DDSTopicDescription](#).

**7.112 DDSTopicDescription Class Reference**

<<[interface](#)>> Base class for [DDSTopic](#).

```
#include <dds_cpp_topic.hxx>
```

Inheritance diagram for [DDSTopicDescription](#):

**Public Member Functions**

- virtual const char \* [get\\_type\\_name](#) ()=0  
*Get the associated `type_name`.*
- virtual const char \* [get\\_name](#) ()=0  
*Get the name used to create this [DDSTopicDescription](#).*
- virtual [DDSDomainParticipant](#) \* [get\\_participant](#) ()=0  
*Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.*

**7.112.1 Detailed Description**

<<[interface](#)>> Base class for [DDSTopic](#).

[DDSTopicDescription](#) represents the fact that both publications and subscriptions are tied to a single data-type. Its attribute `type_name` defines a unique resulting type for the publication or the subscription and therefore creates an implicit association with a type.

[DDSTopicDescription](#) has also a `name` that allows it to be retrieved locally.

### 7.112.2 Member Function Documentation

#### get\_type\_name()

```
virtual const char * DDSTopicDescription::get_type_name () [pure virtual]
```

Get the associated `type_name`.

The type name defines a locally unique type for the publication or the subscription.

The `type_name` corresponds to a unique string used to register a type via the [DDSDomainParticipant::register\\_type](#) method.

Thus, the `type_name` implies an association with a corresponding type and this [DDSTopicDescription](#).

#### Returns

the type name. The returned type name is valid until the [DDSTopicDescription](#) is deleted.

#### Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implemented in [DDSTopic](#).

#### get\_name()

```
virtual const char * DDSTopicDescription::get_name () [pure virtual]
```

Get the name used to create this [DDSTopicDescription](#) .

#### Returns

the name used to create this [DDSTopicDescription](#). The returned topic name is valid until the [DDSTopicDescription](#) is deleted.

#### Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implemented in [DDSTopic](#).



**get\_participant()**

```
virtual DDSDomainParticipant * DDSTopicDescription::get_participant () [pure virtual]
```

Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

**Returns**

The [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

**Postcondition**

The result is NULL if self is NULL, otherwise result is non-NULL.

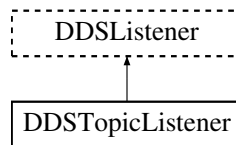
Implemented in [DDSTopic](#).

**7.113 DDSTopicListener Class Reference**

**<<interface>>** [DDSListener](#) for [DDSTopic](#) entities.

```
#include <dds_cpp_topic.hxx>
```

Inheritance diagram for DDSTopicListener:

**Public Member Functions**

- virtual void [on\\_inconsistent\\_topic](#) ([DDSTopic](#) \*topic, const [DDS\\_InconsistentTopicStatus](#) &status)  
Handle the [DDS\\_INCONSISTENT\\_TOPIC\\_STATUS](#) status.

**7.113.1 Detailed Description**

**<<interface>>** [DDSListener](#) for [DDSTopic](#) entities.

**Entity:**

[DDSTopic](#)

**Status:**

[DDS\\_INCONSISTENT\\_TOPIC\\_STATUS](#), [DDS\\_InconsistentTopicStatus](#)

This is the interface that can be implemented by an application-provided class and then registered with the [DDSTopic](#) such that the application can be notified by RTI Connex Micro of relevant status changes.

**See also**

[Status Kinds](#)  
[DDSListener](#)

7.113.2 Member Function Documentation

on\_inconsistent\_topic()

```
virtual void DDSListener::on_inconsistent_topic (
    DDS::Topic * topic,
    const DDS_InconsistentTopicStatus & status) [inline], [virtual]
```

Handle the `DDS_INCONSISTENT_TOPIC_STATUS` status.

This callback is called when a remote `DDS::Topic` is discovered but is inconsistent with the locally created `DDS::Topic` of the same topic name.

Parameters

|                     |                                                                                     |
|---------------------|-------------------------------------------------------------------------------------|
| <code>topic</code>  | <<out>> Locally created <code>DDS::Topic</code> that triggers the listener callback |
| <code>status</code> | <<out>> Current inconsistent status of locally created <code>DDS::Topic</code>      |

7.114 DDSWaitSet Class Reference

<<interface>> Allows an application to wait until one or more of the attached `DDSCondition` objects has a `trigger_value` of `DDS_BOOLEAN_TRUE` or else until the timeout expires.

```
#include <dds_cpp_infrastructure.hxx>
```

Public Member Functions

- `DDS_ReturnCode_t wait (DDSConditionSeq &active_conditions, const DDS_Duration_t &timeout)`  
*Allows an application thread to wait for the occurrence of certain conditions.*
- `DDS_ReturnCode_t attach_condition (DDSCondition *cond)`  
*Attaches a `DDSCondition` to the `DDSWaitSet`.*
- `DDS_ReturnCode_t detach_condition (DDSCondition *cond)`  
*Detaches a `DDSCondition` from the `DDSWaitSet`.*
- `DDS_ReturnCode_t get_conditions (DDSConditionSeq &attached_conditions)`  
*Retrieves the list of attached `DDSCondition` (s).*

7.114.1 Detailed Description

<<interface>> Allows an application to wait until one or more of the attached `DDSCondition` objects has a `trigger_value` of `DDS_BOOLEAN_TRUE` or else until the timeout expires.

### 7.114.2 Usage

[DDSCondition](#) (s) (in conjunction with wait-sets) provide an alternative mechanism to allow the middleware to communicate communication status changes (including arrival of data) to the application.

This mechanism is wait-based. Its general use pattern is as follows:

- The application indicates which relevant information it wants to get by creating [DDSCondition](#) objects ([DDSStatusCondition](#)) and attaching them to a [DDSWaitSet](#).
- It then waits on that [DDSWaitSet](#) until the `trigger_value` of one or several [DDSCondition](#) objects become [DDS\\_BOOLEAN\\_TRUE](#).
- It then uses the result of the wait (i.e., `active_conditions`, the list of [DDSCondition](#) objects with `trigger_value == DDS_BOOLEAN_TRUE`) to actually get the information:
  - by calling [DDSEntity::get\\_status\\_changes](#) and then `get_<communication_status>()` on the relevant [DDSEntity](#), if the condition is a [DDSStatusCondition](#) and the status changes, refer to plain communication status;
  - by calling [DDSEntity::get\\_status\\_changes](#) and then [FooDataReader::read\(\)](#) or [FooDataReader::take](#) on the relevant [DDSDataReader](#), if the condition is a [DDSStatusCondition](#) and the status changes refers to [DDS\\_DATA\\_AVAILABLE\\_STATUS](#);

Usually the first step is done in an initialization phase, while the others are put in the application main loop.

As there is no extra information passed from the middleware to the application when a wait returns (only the list of triggered [DDSCondition](#) objects), [DDSCondition](#) objects are meant to embed all that is needed to react properly when enabled. In particular, [DDSEntity](#)-related conditions are related to exactly one [DDSEntity](#) and cannot be shared.

The result of a [DDSWaitSet::wait](#) operation depends on the state of the [DDSWaitSet](#), which in turn depends on whether at least one attached [DDSCondition](#) has a `trigger_value` of [DDS\\_BOOLEAN\\_TRUE](#). If the wait operation is called on [DDSWaitSet](#) with state `BLOCKED`, it will block the calling thread. If wait is called on a [DDSWaitSet](#) with state `UNBLOCKED`, it will return immediately. In addition, when the [DDSWaitSet](#) transitions from `BLOCKED` to `UNBLOCKED` it wakes up any threads that had called wait on it.

A key aspect of the [DDSCondition/DDSWaitSet](#) mechanism is the setting of the `trigger_value` of each [DDSCondition](#).

### 7.114.3 Trigger State of a `::DDSStatusCondition`

The `trigger_value` of a [DDSStatusCondition](#) is the boolean OR of the `ChangedStatusFlag` of all the communication statuses (see [Status Kinds](#)) to which it is sensitive. That is, `trigger_value == DDS_BOOLEAN_FALSE` only if all the values of the `ChangedStatusFlags` are [DDS\\_BOOLEAN\\_FALSE](#).

The sensitivity of the [DDSStatusCondition](#) to a particular communication status is controlled by the list of `enabled_←_statuses` set on the condition by means of the [DDSStatusCondition::set\\_enabled\\_statuses](#) operation.

#### 7.114.4 Trigger State of a ::DDSGuardCondition

The `trigger_value` of a [DDSGuardCondition](#) is completely controlled by the application via the operation [DDSGuardCondition::set\\_trigger\\_value](#).

See also

[Status Kinds](#)

[DDSStatusCondition](#), [DDSGuardCondition](#)

[DDSListener](#)

#### 7.114.5 Member Function Documentation

##### **wait()**

```
DDS_ReturnCode_t DDSWaitSet::wait (
    DDSConditionSeq & active_conditions,
    const DDS_Duration_t & timeout)
```

Allows an application thread to wait for the occurrence of certain conditions.

If none of the conditions attached to the [DDSWaitSet](#) have a `trigger_value` of [DDS\\_BOOLEAN\\_TRUE](#), the wait operation will block suspending the calling thread.

The result of the wait operation is the list of all the attached conditions that have a `trigger_value` of [DDS\\_BOOLEAN\\_TRUE](#) (i.e., the conditions that unblocked the wait).

Note: The resolution of the `timeout` period is constrained by the resolution of the system clock.

The wait operation takes a `timeout` argument that specifies the maximum duration for the wait. If this duration is exceeded and none of the attached [DDSCondition](#) objects is [DDS\\_BOOLEAN\\_TRUE](#), wait will return with the return code [DDS\\_RETCODE\\_TIMEOUT](#). In this case, the resulting list of conditions will be empty.

Note: The resolution of the `timeout` period is constrained by the resolution of the system clock.

[DDS\\_RETCODE\\_TIMEOUT](#) will *not* be returned when the `timeout` duration is exceeded if attached [DDSCondition](#) objects are [DDS\\_BOOLEAN\\_TRUE](#), or in the case of a [DDSWaitSet](#) waiting for more than one trigger event, if one or more trigger events have occurred.

It is not allowable for more than one application thread to be waiting on the same [DDSWaitSet](#). If the wait operation is invoked on a [DDSWaitSet](#) that already has a thread blocking on it, the operation will return immediately with the value [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

##### Parameters

|                          |                                                                                                                                                                                                                                                                 |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>active_conditions</i> | << <i>inout</i> >> a valid non-NULL <a href="#">DDSConditionSeq</a> object. Note that RTI Connexx DDS Micro will not allocate a new object if <code>active_conditions</code> is NULL; the method will return <a href="#">DDS_RETCODE_PRECONDITION_NOT_MET</a> . |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Parameters

|                |                                |
|----------------|--------------------------------|
| <i>timeout</i> | << <i>in</i> >> a wait timeout |
|----------------|--------------------------------|

#### Returns

One of the [Standard Return Codes](#) or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) or [DDS\\_RETCODE\\_TIMEOUT](#).

### **attach\_condition()**

```
DDS_ReturnCode_t DDSWaitSet::attach_condition (  
    DDSCondition * cond)
```

Attaches a [DDSCondition](#) to the [DDSWaitSet](#).

It is possible to attach a [DDSCondition](#) on a [DDSWaitSet](#) that is currently being waited upon (via the wait operation). In this case, if the [DDSCondition](#) has a `trigger_value` of [DDS\\_BOOLEAN\\_TRUE](#), then attaching the condition will unblock the [DDSWaitSet](#).

#### Parameters

|             |                                           |
|-------------|-------------------------------------------|
| <i>cond</i> | << <i>in</i> >> Contition to be attached. |
|-------------|-------------------------------------------|

#### Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#).

### **detach\_condition()**

```
DDS_ReturnCode_t DDSWaitSet::detach_condition (  
    DDSCondition * cond)
```

Detaches a [DDSCondition](#) from the [DDSWaitSet](#).

If the [DDSCondition](#) was not attached to the [DDSWaitSet](#) the operation will return [DDS\\_RETCODE\\_BAD\\_PARAMETER](#).

#### Parameters

|             |                                           |
|-------------|-------------------------------------------|
| <i>cond</i> | << <i>in</i> >> Condition to be detached. |
|-------------|-------------------------------------------|

#### Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

### **get\_conditions()**

```
DDS_ReturnCode_t DDSWaitSet::get_conditions (  
    DDSConditionSeq & attached_conditions)
```

Retrieves the list of attached [DDSCondition](#) (s).

## Parameters

|                                  |                                                                                                           |
|----------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>attached_conditions</code> | <<inout>> a <a href="#">DDSConditionSeq</a> object where the list of attached conditions will be returned |
|----------------------------------|-----------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), or [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#).

## 7.115 DPDE\_DiscoveryPluginProperty Struct Reference

<<eXtension>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery.

```
#include <disc_dpde_discovery_plugin.h>
```

## Public Attributes

- struct [DDS\\_Duration\\_t participant\\_liveliness\\_assert\\_period](#)  
*The period to assert liveliness for the participant.*
- struct [DDS\\_Duration\\_t participant\\_liveliness\\_lease\\_duration](#)  
*The liveliness lease duration for the participant.*
- [DDS\\_Long initial\\_participant\\_announcements](#)  
*The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.*
- struct [DDS\\_Duration\\_t initial\\_participant\\_announcement\\_period](#)  
*The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.*
- [DDS\\_Boolean cache\\_serialized\\_samples](#)  
*Cache serialized discovery samples for locally created data readers and data writers.*
- [DDS\\_Long max\\_participant\\_locators](#)  
*The maximum number of locators the RTI Connex DDS Micro can send to to discover other participants in the same domain.*
- [DDS\\_Long max\\_locators\\_per\\_discovered\\_participant](#)  
*The maximum number of locators the RTI Connex DDS Micro can keep track of per discovered participant.*
- [DDS\\_Long max\\_samples\\_per\\_builtin\\_endpoint\\_reader](#)  
*The maximum number of samples allocated per builtin endpoint DataReader.*
- [DDS\\_Long max\\_samples\\_per\\_remote\\_builtin\\_endpoint\\_writer](#)  
*The maximum number of samples allocated per remote builtin writer.*
- [DDS\\_Long builtin\\_writer\\_max\\_heartbeat\\_retries](#)  
*Maximum number of heartbeat retries for built-in endpoints.*
- struct [DDS\\_Duration\\_t builtin\\_writer\\_heartbeat\\_period](#)  
*Heartbeat period for built-in endpoints.*
- [DDS\\_Long builtin\\_writer\\_heartbeats\\_per\\_max\\_samples](#)  
*Number of heartbeats per max\_samples built-in endpoints.*
- struct [DDS\\_Duration\\_t builtin\\_endpoint\\_reader\\_nack\\_period](#)  
*The period at which NACKs are sent for the builtin endpoint readers.*
- [DDS\\_ReliabilityQosPolicyKind participant\\_message\\_reader\\_reliability\\_kind](#)  
*Reliability policy for a built-in participant message reader.*

### 7.115.1 Detailed Description

<<*eXtension*>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery.

### 7.115.2 Member Data Documentation

#### participant\_liveliness\_assert\_period

```
struct DDS_Duration_t DPDE_DiscoveryPluginProperty::participant_liveliness_assert_period
```

The period to assert liveliness for the participant.

Specifies how often the DomainParticipant's discovery DataWriter will assert the liveliness of the DomainParticipant to its peers.

Should be strictly less than participant\_liveliness\_lease\_duration.

**[default]** 30 seconds

**[range]** [1 nanosec, 1 year], < participant\_liveliness\_lease\_duration

#### participant\_liveliness\_lease\_duration

```
struct DDS_Duration_t DPDE_DiscoveryPluginProperty::participant_liveliness_lease_duration
```

The liveliness lease duration for the participant.

This is the same as the expiration time of the DomainParticipant as defined in the RTPS protocol.

If the participant has not refreshed its own liveliness to other participants at least once within this period, it may be considered as stale by other participants in the network.

Should be strictly greater than participant\_liveliness\_assert\_period.

**[default]** 100 seconds

**[range]** [1 nanosec, 1 year], > participant\_liveliness\_assert\_period

#### initial\_participant\_announcements

```
DDS_Long DPDE_DiscoveryPluginProperty::initial_participant_announcements
```

The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.

The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.

Also, when a new remote participant appears, the local participant will announce itself to the peers a number of times controlled by this parameter.

**[default]** 5

**initial\_participant\_announcement\_period**

```
struct DDS_Duration_t DPDE_DiscoveryPluginProperty::initial_participant_announcement_period
```

The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.

**[default]** 1 second

**cache\_serialized\_samples**

```
DDS_Boolean DPDE_DiscoveryPluginProperty::cache_serialized_samples
```

Cache serialized discovery samples for locally created data readers and data writers.

When a data reader or data writer is enabled locally, the discovery plugin announces this to other participants in its domain. When this value is set to [DDS\\_BOOLEAN\\_TRUE](#) the serialized sample is stored in case a new participant is discovered and the information must be sent again. This may improve discovery performance, but increases memory usage. The default value is [DDS\\_BOOLEAN\\_FALSE](#) which uses less memory.

**[default]** DDS\_BOOLEAN\_FALSE

**max\_participant\_locators**

```
DDS_Long DPDE_DiscoveryPluginProperty::max_participant_locators
```

The maximum number of locators the RTI Connext DDS Micro can send to to discover other participants in the same domain.

The discovery plugin sends participant announcements to a number of locators as specified in the initial peer list. The maximum number of locators to send too is specified here.

The value [DDS\\_LENGTH\\_AUTO](#) calculates the maximum number of locators as  $\text{DomainParticipantQos.resource\_limits.remote\_participant\_allocation} + (\text{DPDE\_MAX\_ANON\_PARTICIPANT} * (\text{initial\_peers length} + 1))$

**[default]** DDS\_LENGTH\_AUTO

**max\_locators\_per\_discovered\_participant**

```
DDS_Long DPDE_DiscoveryPluginProperty::max_locators_per_discovered_participant
```

The maximum number of locators the RTI Connext DDS Micro can keep track of per discovered participant.

When a participant discover another participant, it must keep track of which addresses to respond to for each participant. NOTE: If two participants respond with the same addresses, for example multicast, only one address is saved.

**[default]** 4



**max\_samples\_per\_builtin\_endpoint\_reader**

[DDS\\_Long](#) `DPDE_DiscoveryPluginProperty::max_samples_per_builtin_endpoint_reader`

The maximum number of samples allocated per builtin endpoint DataReader.

The builtin endpoint DataReaders are reliable and may cache samples received out-of-order to improve discovery performance. This resource-limit determines the maximum number of outstanding samples at any given time.

**[default]** 8

**[range]** [1, 100000000]

**max\_samples\_per\_remote\_builtin\_endpoint\_writer**

[DDS\\_Long](#) `DPDE_DiscoveryPluginProperty::max_samples_per_remote_builtin_endpoint_writer`

The maximum number of samples allocated per remote builtin writer.

The builtin endpoint DataReaders are reliable and may cache samples received out-of-order to improve discovery performance. This resource-limit determines the maximum number of outstanding samples at any given time per remote builtin endpoint DataWriter. Note that this limit is per builtin endpoint DataReader.

The default value `DDS_MAX_UNLIMITED` allows a remote builtin endpoint DataWriter to potentially use all samples while missing samples are being resent. This may cause samples received by other remote builtin endpoint datawriters to be rejected until the remote builtin endpoint DataWriter using all the samples frees up resources.

The following values are supported for this member:

- **DDS\_MAX\_UNLIMITED.** This allows each remote builtin endpoint DataWriter to use up to [DPDE\\_DiscoveryPluginProperty::max\\_sa](#) samples.
- [1, 256]. When a positive value is specified [DPDE\\_DiscoveryPluginProperty::max\\_samples\\_per\\_builtin\\_endpoint\\_reader](#) is automatically adjusted to [DPDE\\_DiscoveryPluginProperty::max\\_samples\\_per\\_remote\\_builtin\\_endpoint\\_writer](#) \* [DDS\\_DomainParticipantResourceLimitsQosPolicy::remote\\_participant\\_allocation](#).

Note that the value of [DPDE\\_DiscoveryPluginProperty::max\\_samples\\_per\\_builtin\\_endpoint\\_reader](#) is still limited to [1, 100000000] and a run-time error will occur if the value is exceeded.

**[default]** `DDS_MAX_UNLIMITED`

**[range]** `DDS_MAX_UNLIMITED`, [1, 256]

**builtin\_writer\_max\_heartbeat\_retries**

[DDS\\_Long](#) `DPDE_DiscoveryPluginProperty::builtin_writer_max_heartbeat_retries`

Maximum number of heartbeat retries for built-in endpoints.

**[default]** `DDS_LENGTH_UNLIMITED`

**[range]** [1, 100000000] or `DDS_LENGTH_UNLIMITED`.

**builtin\_writer\_heartbeat\_period**

```
struct DDS_Duration_t DPDE_DiscoveryPluginProperty::builtin_writer_heartbeat_period
```

Heartbeat period for built-in endpoints.

**[default]** 100 ms.

**[range]** [1 nanosec, 1 year]

**builtin\_writer\_heartbeats\_per\_max\_samples**

```
DDS_Long DPDE_DiscoveryPluginProperty::builtin_writer_heartbeats_per_max_samples
```

Number of heartbeats per max\_samples built-in endpoints.

**[default]** DDS\_LENGTH\_UNLIMITED

**[range]** [1, local\_writer\_allocation] or DDS\_LENGTH\_UNLIMITED.

**builtin\_endpoint\_reader\_nack\_period**

```
struct DDS_Duration_t DPDE_DiscoveryPluginProperty::builtin_endpoint_reader_nack_period
```

The period at which NACKs are sent for the builtin endpoint readers.

Please refer to [DDS\\_RtpsReliableReaderProtocol\\_t::nack\\_period](#) for a detailed description.

**participant\_message\_reader\_reliability\_kind**

```
DDS_ReliabilityQosPolicyKind DPDE_DiscoveryPluginProperty::participant_message_reader_reliability↔  
_kind
```

Reliability policy for a built-in participant message reader.

For details, refer to the [DDS\\_ReliabilityQosPolicyKind](#).

**[default]** [DDS\\_BEST\\_EFFORT\\_RELIABILITY\\_QOS](#)

**7.116 DPDEDiscoveryFactory Class Reference**

[<<eXtension>>](#) Discovery plug-in used for asserting remote entities.

```
#include <dds_cpp_dpde.hxx>
```

## Static Public Member Functions

- static struct RT\_ComponentFactoryI \* [get\\_interface](#) ()

*Gets the component factory necessary when registering a DPDE component with RTI Connex DDS Micro*

### 7.116.1 Detailed Description

<<[eXtension](#)>> Discovery plug-in used for asserting remote entities.

### 7.116.2 Member Function Documentation

#### **get\_interface()**

```
static struct RT_ComponentFactoryI * DPDEDiscoveryFactory::get_interface () [static]
```

Gets the component factory necessary when registering a DPDE component with RTI Connex DDS Micro

## 7.117 DPSE\_DiscoveryPluginProperty Struct Reference

<<[eXtension](#)>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. This includes all discovery timing properties for participant discovery.

```
#include <disc_dpse_dpseDiscovery.h>
```

### Public Attributes

- struct [DDS\\_Duration\\_t participant\\_liveliness\\_assert\\_period](#)  
*The period to assert liveliness for the participant.*
- struct [DDS\\_Duration\\_t participant\\_liveliness\\_lease\\_duration](#)  
*The liveliness lease duration for the participant.*
- [DDS\\_Long initial\\_participant\\_announcements](#)  
*The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.*
- struct [DDS\\_Duration\\_t initial\\_participant\\_announcement\\_period](#)  
*The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.*
- [DDS\\_Long max\\_participant\\_locators](#)  
*The maximum number of locators RTI Connex DDS Micro can send to to discover other participants in the same domain.*
- [DDS\\_Long max\\_locators\\_per\\_discovered\\_participant](#)  
*The maximum number of locators RTI Connex DDS Micro can keep track of per discovered participant.*
- [DDS\\_ReliabilityQosPolicyKind participant\\_message\\_reader\\_reliability\\_kind](#)  
*Reliability policy for a built-in participant message reader.*

### 7.117.1 Detailed Description

<<eXtension>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. This includes all discovery timing properties for participant discovery.

### 7.117.2 Member Data Documentation

#### participant\_liveliness\_assert\_period

```
struct DDS_Duration_t DPSE_DiscoveryPluginProperty::participant_liveliness_assert_period
```

The period to assert liveliness for the participant.

Specifies how often the DomainParticipant's discovery DataWriter will assert the liveliness of the DomainParticipant to its peers.

Should be strictly less than participant\_liveliness\_lease\_duration.

**[default]** 30 seconds

**[range]** [1 nanosec, 1 year], < participant\_liveliness\_lease\_duration

#### participant\_liveliness\_lease\_duration

```
struct DDS_Duration_t DPSE_DiscoveryPluginProperty::participant_liveliness_lease_duration
```

The liveliness lease duration for the participant.

This is the same as the expiration time of the DomainParticipant as defined in the RTPS protocol.

If the participant has not refreshed its own liveliness to other participants at least once within this period, it may be considered as stale by other participants in the network.

Should be strictly greater than participant\_liveliness\_assert\_period.

**[default]** 100 seconds

**[range]** [1 nanosec, 1 year], > participant\_liveliness\_assert\_period

#### initial\_participant\_announcements

```
DDS_Long DPSE_DiscoveryPluginProperty::initial_participant_announcements
```

The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.

The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.

Also, when a new remote participant appears, the local participant will announce itself to the peers a number of times controlled by this parameter.

**[default]** 5

**initial\_participant\_announcement\_period**

```
struct DDS_Duration_t DPSE_DiscoveryPluginProperty::initial_participant_announcement_period
```

The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.

**[default]** 1 second

**max\_participant\_locators**

```
DDS_Long DPSE_DiscoveryPluginProperty::max_participant_locators
```

The maximum number of locators RTI Connex DDS Micro can send to to discover other participants in the same domain.

The discovery plugin sends participant announcements to a number of locators as specified in the initial peer list. The maximum number of locators to send too is specified here.

The value DDS\_LENGTH\_AUTO calculates the maximum number of locators as  $\text{DomainParticipantQos.resource\_limits.remote\_participant\_allocation} + (\text{DPDE\_MAX\_ANON\_PARTICIPANT} * (\text{initial\_peers length} + 1))$ ;

**[default]** DDS\_LENGTH\_AUTO;

**max\_locators\_per\_discovered\_participant**

```
DDS_Long DPSE_DiscoveryPluginProperty::max_locators_per_discovered_participant
```

The maximum number of locators RTI Connex DDS Micro can keep track of per discovered participant.

When a participant discover another participant, it must keep track of which addresses to respond to for each participant. NOTE: If two participants respond with the same addresses, for example multicast, only one address is saved.

**[default]** 4;

**participant\_message\_reader\_reliability\_kind**

```
DDS_ReliabilityQosPolicyKind DPSE_DiscoveryPluginProperty::participant_message_reader_reliability_↔  
_kind
```

Reliability policy for a built-in participant message reader.

For details, refer to the [DDS\\_ReliabilityQosPolicyKind](#).

**[default]** [DDS\\_BEST\\_EFFORT\\_RELIABILITY\\_QOS](#)

## 7.118 DPSEDiscoveryFactory Class Reference

**<<eXtension>>** A discovery factory that will be used by the [DDSDomainParticipantFactory](#) to create a discovery plugin.

```
#include <dds_cpp_dpse.hxx>
```

### Static Public Member Functions

- static struct RT\_ComponentFactoryI \* [get\\_interface](#) ()  
*Gets the singleton instance of the DPSE discovery plugin factory. The singleton instance of the factory is used when creating a domain participant.*

#### 7.118.1 Detailed Description

**<<eXtension>>** A discovery factory that will be used by the [DDSDomainParticipantFactory](#) to create a discovery plugin.

#### 7.118.2 Member Function Documentation

##### [get\\_interface\(\)](#)

```
static struct RT_ComponentFactoryI * DPSEDiscoveryFactory::get_interface () [static]
```

Gets the singleton instance of the DPSE discovery plugin factory. The singleton instance of the factory is used when creating a domain participant.

This function gets the single singleton instance of the DPSE discovery plugin factory that is used by the middleware to create a DPSE plugin. In the future, as more discovery plug-ins are supported, they will be registered with the middleware by installing different discovery factory instances with the [DDSDomainParticipantFactory](#). The singleton instance of the factory is registered with a name, and that name is specified in the [DDSDomainParticipant DDS\\_DiscoveryQosPolicy](#). This is used to create a discovery plug-in instance when creating a domain participant.

##### Returns

Pointer to DPSE discovery plugin factory

##### See also

[DDS\\_DiscoveryQosPolicy](#)

## 7.119 DPSEDiscoveryPlugin Class Reference

**<<eXtension>>** Discovery plug-in used for asserting remote entities.

```
#include <dds_cpp_dpse.hxx>
```

## Static Public Member Functions

- static `DDS_ReturnCode_t RemoteParticipant_assert (DDSDomainParticipant *const participant, const char *rem_participant_name)`  
*Assert remote participant by passing in the name of that participant.*
- static `DDS_ReturnCode_t RemotePublication_assert (DDSDomainParticipant *const participant, const char *const rem_participant_name, const struct DDS_PublicationBuiltinTopicData *const data, NDDS_TypePluginKeyKind key_kind)`  
*Assert remote publication by passing in the builtin data that describes the type, topic, and QoS of that remote publication.*
- static `DDS_ReturnCode_t RemoteSubscription_assert (DDSDomainParticipant *const participant, const char *const rem_participant_name, const struct DDS_SubscriptionBuiltinTopicData *const data, NDDS_TypePluginKeyKind key_kind)`  
*Assert remote subscription by passing in the builtin data that describes the type, topic, and QoS of that remote subscription.*

### 7.119.1 Detailed Description

<<eXtension>> Discovery plug-in used for asserting remote entities.

### 7.119.2 Member Function Documentation

#### RemoteParticipant\_assert()

```
static DDS_ReturnCode_t DPSEDiscoveryPlugin::RemoteParticipant_assert (
    DDSDomainParticipant *const participant,
    const char * rem_participant_name) [static]
```

Assert remote participant by passing in the name of that participant.

DPSE discovery requires that you pre-configure your application with information about any remote participants that you intend to discover. The only information that needs to be pre-configured is the name of the remote `DDSDomainParticipant` that your application intends to discover.

#### RemotePublication\_assert()

```
static DDS_ReturnCode_t DPSEDiscoveryPlugin::RemotePublication_assert (
    DDSDomainParticipant *const participant,
    const char *const rem_participant_name,
    const struct DDS_PublicationBuiltinTopicData *const data,
    NDDS_TypePluginKeyKind key_kind) [static]
```

Assert remote publication by passing in the builtin data that describes the type, topic, and QoS of that remote publication.

DPSE discovery requires that you pre-configure your application with information about any remote publications that you intend to discover and receive data from. The `DDS_PublicationBuiltinTopicData` field must include the topic name, type name, and QoS of the remote `DDSDataWriter` that you intend to receive communications from.

Additionally, this method takes a `key_kind` argument that must match the key kind returned by the type plugin.

**RemoteSubscription\_assert()**

```
static DDS_ReturnCode_t DPSEDiscoveryPlugin::RemoteSubscription_assert (
    DDSDomainParticipant *const participant,
    const char *const rem_participant_name,
    const struct DDS_SubscriptionBuiltinTopicData *const data,
    NDDS_TypePluginKeyKind key_kind) [static]
```

Assert remote subscription by passing in the builtin data that describes the type, topic, and QoS of that remote subscription.

DPSE discovery requires that you pre-configure your application with information about any remote subscriptions that you intend to discover and send data to. The [DDS\\_SubscriptionBuiltinTopicData](#) parameter must include the topic name, type name, and QoS of the remote [DDSDDataReader](#) that your application intends to send data to. Additionally, if the remote [DDSDDataReader](#) is listening on a non-default port, you must set the remote locator (including the non-default port) in the [DDS\\_SubscriptionBuiltinTopicData](#) that you intend to use to contact the remote [DDSDDataReader](#).

Additionally, this method takes a `key_kind` argument that must match the key kind returned by the type plugin.

**7.120 FooDataReader Class Reference**

Declares the interface required to support a user data type-specific data reader.

```
#include <dds_cpp_data.hxx>
```

**Public Member Functions**

- [DDS\\_ReturnCode\\_t](#) [return\\_loan](#) ([FooSeq](#) &received\_data, [DDS\\_SampleInfoSeq](#) &info\_seq)  
*Indicates to the [DDSDDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the [DDSDDataReader](#).*
- [DDS\\_ReturnCode\\_t](#) [read\\_next\\_sample](#) ([Foo](#) &received\_data, [DDS\\_SampleInfo](#) &sample\_info)  
*Copies the next not-previously-accessed data value from the [DDSDDataReader](#).*
- [DDS\\_ReturnCode\\_t](#) [take\\_next\\_sample](#) ([Foo](#) &received\_data, [DDS\\_SampleInfo](#) &sample\_info)  
*Copies the next not-previously-accessed data value from the [DDSDDataReader](#).*
- [DDS\\_ReturnCode\\_t](#) [read](#) ([FooSeq](#) &received\_data, [DDS\\_SampleInfoSeq](#) &info\_seq, [DDS\\_Long](#) max\_samples, [DDS\\_SampleStateMask](#) sample\_states, [DDS\\_ViewStateMask](#) view\_states, [DDS\\_InstanceStateMask](#) instance\_states)  
*Access a collection of data samples from the [DDSDDataReader](#).*
- [DDS\\_ReturnCode\\_t](#) [take](#) ([FooSeq](#) &received\_data, [DDS\\_SampleInfoSeq](#) &info\_seq, [DDS\\_Long](#) max\_samples, [DDS\\_SampleStateMask](#) sample\_states, [DDS\\_ViewStateMask](#) view\_states, [DDS\\_InstanceStateMask](#) instance\_states)  
*Access a collection of data-samples from the [DDSDDataReader](#).*
- [DDS\\_ReturnCode\\_t](#) [read\\_instance](#) ([FooSeq](#) &received\_data, [DDS\\_SampleInfoSeq](#) &info\_seq, [DDS\\_Long](#) max\_samples, const [DDS\\_InstanceHandle\\_t](#) &a\_handle, [DDS\\_SampleStateMask](#) sample\_states, [DDS\\_ViewStateMask](#) view\_states, [DDS\\_InstanceStateMask](#) instance\_states)  
*Access a collection of data samples from the [DDSDDataReader](#).*
- [DDS\\_ReturnCode\\_t](#) [take\\_instance](#) ([FooSeq](#) &received\_data, [DDS\\_SampleInfoSeq](#) &info\_seq, [DDS\\_Long](#) max\_samples, const [DDS\\_InstanceHandle\\_t](#) &a\_handle, [DDS\\_SampleStateMask](#) sample\_states, [DDS\\_ViewStateMask](#) view\_states, [DDS\\_InstanceStateMask](#) instance\_states)  
*Access a collection of data samples from the [DDSDDataReader](#).*
- [DDS\\_InstanceHandle\\_t](#) [lookup\\_instance](#) (const [Foo](#) &key\_holder)  
*Retrieve the instance `handle` that corresponds to an instance `key_holder`.*
- [DDS\\_ReturnCode\\_t](#) [is\\_data\\_consistent](#) ([DDS\\_Boolean](#) &is\_data\_consistent, const [Foo](#) &sample, const struct [DDS\\_SampleInfo](#) &sample\_info)  
*Checks if the sample has been overwritten by the `DataWriter`.*



## Static Public Member Functions

- static `FooDataReader * narrow (DDSDDataReader *reader)`  
Narrow the given `DDSDDataReader` pointer to a `FooDataReader` pointer.

### 7.120.1 Detailed Description

Declares the interface required to support a user data type-specific data reader.

### 7.120.2 Member Function Documentation

#### `narrow()`

```
static FooDataReader * FooDataReader::narrow (
    DDSDDataReader * reader) [static]
```

Narrow the given `DDSDDataReader` pointer to a `FooDataReader` pointer.

#### `return_loan()`

```
DDS_ReturnCode_t FooDataReader::return_loan (
    FooSeq & received_data,
    DDS_SampleInfoSeq & info_seq)
```

Indicates to the `DDSDDataReader` that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the `DDSDDataReader`.

This operation indicates to the `DDSDDataReader` that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the `DDSDDataReader`.

The `received_data` and `info_seq` must belong to a single related "pair"; that is, they should correspond to a pair returned from a single call to `read` or `take`. The `received_data` and `info_seq` must also have been obtained from the same `DDSDDataReader` to which they are returned. If either of these conditions is not met, the operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`.

The operation `FooDataReader::return_loan` allows implementations of the `read` and `take` operations to "loan" buffers from the `DDSDDataReader` to the application and in this manner provide "zerocopy" access to the data. During the loan, the `DDSDDataReader` will guarantee that the data and sample-information are not modified.

It is not necessary for an application to return the loans immediately after the `read` or `take` calls. However, as these buffers correspond to internal resources inside the `DDSDDataReader`, the application should not retain them indefinitely.

The use of `FooDataReader::return_loan` is only necessary if the `read` or `take` calls "loaned" buffers to the application. This only occurs if the `received_data` and `info_seq` collections had `max_len=0` at the time `read` or `take` was called.

The application may also examine the "owns" property of the collection to determine where there is an outstanding loan. However, calling `FooDataReader::return_loan` on a collection that does not have a loan is safe and has no side effects.

If the collections had a loan, upon completion of `FooDataReader::return_loan`, the collections will have `max_len=0`.

Similar to `read`, this operation must be provided on the specialized class that is generated for the particular application data-type that is being taken.

## Parameters

|                      |                                                                                                                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i> | << <i>in</i> >> user data type-specific <a href="#">FooSeq</a> object where the received data samples was obtained from earlier invocation of read or take on the <a href="#">DDSDDataReader</a> . |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Parameters

|                 |                                                                                                                                                                                        |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>info_seq</i> | << <i>in</i> >> a <a href="#">DDS_SampleInfoSeq</a> object where the received sample info was obtained from earlier invocation of read or take on the <a href="#">DDSDDataReader</a> . |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

**read\_next\_sample()**

```
DDS_ReturnCode_t FooDataReader::read_next_sample (
    Foo & received_data,
    DDS_SampleInfo & sample_info)
```

Copies the next not-previously-accessed data value from the [DDSDDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDDataReader](#). This operation also copies the corresponding [DDS\\_SampleInfo](#). The implied order among the samples stored in the [DDSDDataReader](#) is the same as for the [FooDataReader::read](#) operation.

The [FooDataReader::read\\_next\\_sample](#) operation is semantically equivalent to the [FooDataReader::read](#) operation, where the input data sequences has max\_len=1, the sample\_states=NOT\_READ, the view\_states=ANY\_VIEW\_STATE, and the instance\_states=ANY\_INSTANCE\_STATE.

The [FooDataReader::read\\_next\\_sample](#) operation provides a simplified API to 'read' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDDataReader](#), the operation will fail with [DDS\\_RETCODE\\_NO\\_DATA](#) and nothing is copied.

## Parameters

|                      |                                                                                                                                                                                                   |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i> | << <i>inout</i> >> user data type-specific Foo object where the next received data sample will be returned. The received_data must have been fully allocated. Otherwise, this operation may fail. |
| <i>sample_info</i>   | << <i>inout</i> >> a <a href="#">DDS_SampleInfo</a> object where the next received sample info will be returned.                                                                                  |

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

## See also

[FooDataReader::read](#)

## take\_next\_sample()

```
DDS_ReturnCode_t FooDataReader::take_next_sample (
    Foo & received_data,
    DDS_SampleInfo & sample_info)
```

Copies the next not-previously-accessed data value from the [DDSDDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDDataReader](#) and 'removes' it from the [DDSDDataReader](#) so that it is no longer accessible. This operation also copies the corresponding [DDS\\_SampleInfo](#). This operation is analogous to the [FooDataReader::read\\_next\\_sample](#) except for the fact that the sample is removed from the [DDSDDataReader](#).

The [FooDataReader::take\\_next\\_sample](#) operation is semantically equivalent to the [FooDataReader::take](#) operation, where the input data sequences has max\_len=1, the sample\_states=NOT\_READ, the view\_states=ANY\_VIEW\_STATE, and the instance\_states=ANY\_INSTANCE\_STATE.

The [FooDataReader::read\\_next\\_sample](#) operation provides a simplified API to 'take' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDDataReader](#), the operation will fail with [DDS\\_RETCODE\\_NO\\_DATA](#) and nothing is copied.

### Parameters

|                      |                                                                                                                                                                                          |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i> | <<inout>> user data type-specific Foo object where the next received data sample will be returned. The received_data must have been fully allocated. Otherwise, this operation may fail. |
| <i>sample_info</i>   | <<inout>> a <a href="#">DDS_SampleInfo</a> object where the next received sample info will be returned.                                                                                  |

### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

### See also

[FooDataReader::take](#)

## read()

```
DDS_ReturnCode_t FooDataReader::read (
    FooSeq & received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states)
```

Access a collection of data samples from the [DDSDDataReader](#).

This operation offers the same functionality and API as [FooDataReader::take](#) except that the samples returned remain in the [DDSDDataReader](#) such that they can be retrieved again by means of a read or take operation.

Please refer to the documentation of [FooDataReader::take\(\)](#) for details on the number of samples returned within the `received_data` and `info_seq` as well as the order in which the samples appear in these sequences.

The act of reading a sample changes its `sample_state` to [DDS\\_READ\\_SAMPLE\\_STATE](#). If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to be [DDS\\_NOT\\_NEW\\_VIEW\\_STATE](#). It will not affect the `instance_state` of the instance.

*Important:* If the samples "returned" by this method are loaned from RTI Connext DDS Micro (see [FooDataReader::take](#) for more information on memory loaning), it is important that their contents not be changed. Because the memory in which the data is stored belongs to the middleware, any modifications made to the data will be seen the next time the same samples are read or taken; the samples will no longer reflect the state that was received from the network.

#### Parameters

|                        |                                                                                                                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i>   | << <i>inout</i> >> User data type-specific <a href="#">FooSeq</a> object where the received data samples will be returned.                                                              |
| <i>info_seq</i>        | << <i>inout</i> >> A <a href="#">DDS_SampleInfoSeq</a> object where the received sample info will be returned.                                                                          |
| <i>max_samples</i>     | << <i>in</i> >> The maximum number of samples to be returned. If the special value <a href="#">DDS_LENGTH_UNLIMITED</a> is provided, as many samples will be returned as are available. |
| <i>sample_states</i>   | << <i>in</i> >> Data samples matching one of these <code>sample_states</code> are returned.                                                                                             |
| <i>view_states</i>     | << <i>in</i> >> Data samples matching one of these <code>view_state</code> are returned.                                                                                                |
| <i>instance_states</i> | << <i>in</i> >> Data samples matching ones of these <code>instance_state</code> are returned.                                                                                           |

#### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

#### See also

[FooDataReader::take](#),  
[DDS\\_LENGTH\\_UNLIMITED](#)

#### take()

```
DDS_ReturnCode_t FooDataReader::take (
    FooSeq & received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states)
```

Access a collection of data-samples from the [DDSDDataReader](#).

The operation will return the list of samples received by the `DDSDataReader` since the last `FooDataReader::take` operation that match the specified `DDS_SampleStateMask`, `DDS_ViewStateMask`, and `DDS_InstanceStateMask`.

This operation may fail with `DDS_RETCODE_ERROR` if the `DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads` limit has been exceeded.

The actual number of samples returned depends on the information that has been received by the middleware as well as the `DDS_HistoryQosPolicy`, `DDS_ResourceLimitsQosPolicy`, `DDS_DataReaderResourceLimitsQosPolicy`, and the characteristics of the data-type that is associated with the `DDSDataReader`:

- When the `DDS_HistoryQosPolicy::kind` is `DDS_KEEP_LAST_HISTORY_QOS`, the call will return a maximum of `DDS_HistoryQosPolicy::depth` samples for each ALIVE instance and `DDS_HistoryQosPolicy::depth + 1` samples for each NOT\_ALIVE instance. The extra sample is an invalid sample that indicates the instance state transition from ALIVE to NOT\_ALIVE.
- When the `DDS_HistoryQosPolicy::kind` is `DDS_KEEP_ALL_HISTORY_QOS`, the maximum number of samples returned is limited to `DDS_ResourceLimitsQosPolicy::max_samples + 1` for each NOT\_ALIVE instance.
- When `max_samples_per_instance` is not `DDS_LENGTH_UNLIMITED`, the number of samples returned is also limited by the product (`DDS_ResourceLimitsQosPolicy::max_samples_per_instance * DDS_ResourceLimitsQosPolicy::max_instances`).

If the read or take succeeds, and the number of samples returned has been limited (by means of a maximum limit, as listed above, or insufficient `DDS_SampleInfo` resources), the call will complete successfully and provide those samples the reader is able to return. The user may need to make additional calls, or return outstanding loaned buffers in the case of insufficient resources, in order to access remaining samples.

Note that when the `DDSTopic` associated with the `DDSDataReader` is bound to a data-type that has no key definition, then there will be at most one instance in the `DDSDataReader`. So the per-sample limits will apply.

The act of *taking* a sample removes it from RTI Connex DDS Micro so it cannot be read or taken again. If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the sample's instance to `DDS_NOT_NEW_VIEW_STATE`. It will not affect the `instance_state` of the sample's instance.

After `FooDataReader::take` completes, `received_data` and `info_seq` will be of the same length and contain the received data.

If the sequences are empty (maximum size equal 0) when the `FooDataReader::take` is called, the samples returned in the `received_data` and the corresponding `info_seq` are 'loaned' to the application from buffers provided by the `DDSDataReader`. The application can use them as desired and has guaranteed exclusive access to them.

Once the application completes its use of the samples it must 'return the loan' to the `DDSDataReader` by calling the `FooDataReader::return_loan` operation.

*Note:* While you must call `FooDataReader::return_loan` at some point, you do *not* have to do so before the next `FooDataReader::take` call. However, failure to return the loan will eventually deplete the `DDSDataReader` of the buffers it needs to receive new samples and eventually samples will start to be lost. The total number of buffers available to the `DDSDataReader` is specified by the `DDS_ResourceLimitsQosPolicy`.

If the sequences are not empty (maximum size not equal 0) when the `FooDataReader::take` is called, samples are copied to `received_data` and `info_seq`. The application will not need to call `FooDataReader::return_loan`.

The samples returned to the caller is a list where samples belonging to the same data instance are consecutive (i.e. instance access scope).

Samples belonging to the same instance will appear in the relative order in which there were received (i.e. reception timestamp destination order, where earlier samples are ahead of later samples).

If the `DDSDataReader` has no samples that meet the constraints, the method will fail with `DDS_RETCODE_NO_DATA`.

In addition to the collection of samples, the read and take operations also use a collection of `DDS_SampleInfo` structures.

### 7.120.3 SEQUENCES USAGE IN TAKE AND READ

The initial (input) properties of the `received_data` and `info_seq` collections will determine the precise behavior of the read or take operation. For the purposes of this description, the collections are modeled as having these properties:

- whether the collection container owns the memory of the elements within (`owns`, see `::FooSeq::has_ownership`)
- the current-length (`len`, see `::FooSeq::length()`)
- the maximum length (`max_len`, see `::FooSeq::maximum()`)

The initial values of the `owns`, `len` and `max_len` properties for the `received_data` and `info_seq` collections govern the behavior of the read and take operations as specified by the following rules:

1. The values of `owns`, `len` and `max_len` properties for the two collections must be identical. Otherwise read/take will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`.
2. On successful output, the values of `owns`, `len` and `max_len` will be the same for both collections.
3. If the initial `max_len==0`, then the `received_data` and `info_seq` collections will be filled with elements that are loaned by the `DDSDataReader`. On output, `owns` will be `FALSE`, `len` will be set to the number of values returned, and `max_len` will be set to a value verifying `max_len >= len`. The use of this variant allows for zero-copy access to the data and the application will need to return the loan to the `DDSDataWriter` using `FooDataReader::return_loan`.
4. If the initial `max_len>0` and `owns==FALSE`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential hard-to-detect memory leaks caused by an application forgetting to return the loan.
5. If initial `max_len>0` and `owns==TRUE`, then the read or take operation will copy the `received_data` values and `DDS_SampleInfo` values into the elements already inside the collections. On output, `owns` will be `TRUE`, `len` will be set to the number of values copied and `max_len` will remain unchanged. The use of this variant forces a copy but the application can control where the copy is placed and the application will not need to return the loan. The number of samples copied depends on the relative values of `max_len` and `max_samples`:
  - If `max_samples == LENGTH_UNLIMITED`, then at most `max_len` values will be copied. The use of this variant lets the application limit the number of samples returned to what the sequence can accommodate.
  - If `max_samples <= max_len`, then at most `max_samples` values will be copied. The use of this variant lets the application limit the number of samples returned to fewer than what the sequence can accommodate.
  - If `max_samples > max_len`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential confusion where the application expects to be able to access up to `max_samples`, but that number can never be returned, even if they are available in the `DDSDataReader`, because the output sequence cannot accommodate them.

As described above, upon completion, the `received_data` and `info_seq` collections may contain elements loaned from the `DDSDataReader`. If this is the case, the application will need to use `FooDataReader::return_loan` to return the loan once it is no longer using the `received_data` in the collection. When `FooDataReader::return_loan` completes, the collection will have `owns=FALSE` and `max_len=0`. The application can determine whether it is necessary to return the loan or not based on how the state of the collections when the read/take operation was called or by accessing the `owns` property. However, in many cases it may be simpler to always call `FooDataReader::return_loan`, as this operation is harmless (i.e., it leaves all elements unchanged) if the collection does not have a loan.

To avoid potential memory leaks, the implementation of the Foo and [DDS\\_SampleInfo](#) collections should disallow changing the length of a collection for which `owns==FALSE`. Furthermore, deleting a collection for which `owns==FALSE` should be considered an error.

On output, the collection of Foo values and the collection of [DDS\\_SampleInfo](#) structures are of the same length and are in a one-to-one correspondence. Each [DDS\\_SampleInfo](#) provides information, such as the `source_timestamp`, the `sample_state`, `view_state`, and `instance_state`, etc., about the corresponding sample.

Some elements in the returned collection may not have valid data. If the `instance_state` in the [DDS\\_SampleInfo](#) is [DDS\\_NOT\\_ALIVE\\_DISPOSED\\_INSTANCE\\_STATE](#) or [DDS\\_NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#), then the last sample for that instance in the collection (that is, the one whose [DDS\\_SampleInfo](#) has `sample_rank==0`) does not contain valid data.

Samples that contain no data do not count towards the limits imposed by the [DDS\\_ResourceLimitsQosPolicy](#). The act of reading/taking a sample sets its `sample_state` to [DDS\\_READ\\_SAMPLE\\_STATE](#).

If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to [DDS\\_NOT\\_NEW\\_VIEW\\_STATE](#). It will not affect the `instance_state` of the instance.

This operation must be provided on the specialized class that is generated for the particular application data-type that is being read (Foo). If the [DDSDataReader](#) has no samples that meet the constraints, the operations fails with [DDS\\_RETCODE\\_NO\\_DATA](#).

#### Parameters

|                      |                                                                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i> | << <i>inout</i> >> User data type-specific <a href="#">FooSeq</a> object where the received data samples will be returned. |
|----------------------|----------------------------------------------------------------------------------------------------------------------------|

#### Parameters

|                        |                                                                                                                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>info_seq</i>        | << <i>inout</i> >> A <a href="#">DDS_SampleInfoSeq</a> object where the received sample info will be returned.                                                                          |
| <i>max_samples</i>     | << <i>in</i> >> The maximum number of samples to be returned. If the special value <a href="#">DDS_LENGTH_UNLIMITED</a> is provided, as many samples will be returned as are available. |
| <i>sample_states</i>   | << <i>in</i> >> Data samples matching one of these <code>sample_states</code> are returned.                                                                                             |
| <i>view_states</i>     | << <i>in</i> >> Data samples matching one of these <code>view_state</code> are returned.                                                                                                |
| <i>instance_states</i> | << <i>in</i> >> Data samples matching one of these <code>instance_state</code> are returned.                                                                                            |

#### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

#### See also

[FooDataReader::read](#)

[DDS\\_LENGTH\\_UNLIMITED](#)

**read\_instance()**

```
DDS_ReturnCode_t FooDataReader::read_instance (
    FooSeq & received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    const DDS_InstanceHandle_t & a_handle,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states)
```

Access a collection of data samples from the [DDSDDataReader](#).

This operation accesses a collection of data values from the [DDSDDataReader](#). The behavior is identical to [FooDataReader::read](#), except that all samples returned belong to the single specified instance whose handle is `a_handle`.

Upon successful completion, the data collection will contain samples all belonging to the same instance. The corresponding [DDS\\_SampleInfo](#) verifies `DDS_SampleInfo::instance_handle == a_handle`.

The [FooDataReader::read\\_instance](#) operation is semantically equivalent to the [FooDataReader::read](#) operation, except in building the collection, the [DDSDDataReader](#) will check that the sample belongs to the specified instance and otherwise it will not place the sample in the returned collection.

The behavior of the [FooDataReader::read\\_instance](#) operation follows the same rules as the [FooDataReader::read](#) operation regarding the pre-conditions and post-conditions for the `received_data` and `sample_info`. Similar to the [FooDataReader::read](#), the [FooDataReader::read\\_instance](#) operation may 'loan' elements to the output collections, which must then be returned by means of [FooDataReader::return\\_loan](#).

Similar to the [FooDataReader::read](#), this operation must be provided on the specialized class that is generated for the particular application data-type that is being taken.

If the [DDSDDataReader](#) has no samples that meet the constraints, the method will fail with [DDS\\_RETCODE\\_NO\\_DATA](#).

This operation may fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#) if the [DDS\\_InstanceHandle\\_t](#) `a_handle` does not correspond to an existing data-object known to the [DDSDDataReader](#).

**Parameters**

|                        |                                                                                                                                                                                                                                                           |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i>   | << <i>inout</i> >> user data type-specific <a href="#">FooSeq</a> object where the received data samples will be returned.                                                                                                                                |
| <i>info_seq</i>        | << <i>inout</i> >> a <a href="#">DDS_SampleInfoSeq</a> object where the received sample info will be returned.                                                                                                                                            |
| <i>max_samples</i>     | << <i>in</i> >> The maximum number of samples to be returned. If the special value <code>DDS_LENGTH_UNLIMITED</code> is provided, as many samples will be returned as are available.                                                                      |
| <i>a_handle</i>        | << <i>in</i> >> The specified instance to return samples for. The method will fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> if the <code>handle</code> does not correspond to an existing data-object known to the <a href="#">DDSDDataReader</a> . |
| <i>sample_states</i>   | << <i>in</i> >> data samples matching ones of these <code>sample_states</code> are returned                                                                                                                                                               |
| <i>view_states</i>     | << <i>in</i> >> data samples matching ones of these <code>view_state</code> are returned                                                                                                                                                                  |
| <i>instance_states</i> | << <i>in</i> >> data samples matching ones of these <code>instance_state</code> are returned                                                                                                                                                              |



## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

## See also

[FooDataReader::read](#)

[DDS\\_LENGTH\\_UNLIMITED](#)

## take\_instance()

```
DDS_ReturnCode_t FooDataReader::take_instance (
    FooSeq & received_data,
    DDS_SampleInfoSeq & info_seq,
    DDS_Long max_samples,
    const DDS_InstanceHandle_t & a_handle,
    DDS_SampleStateMask sample_states,
    DDS_ViewStateMask view_states,
    DDS_InstanceStateMask instance_states)
```

Access a collection of data samples from the [DDSDDataReader](#).

This operation accesses a collection of data values from the [DDSDDataReader](#). The behavior is identical to [FooDataReader::take](#), except for that all samples returned belong to the single specified instance whose handle is `a_handle`.

The semantics are the same for the [FooDataReader::take](#) operation, except in building the collection, the [DDSDDataReader](#) will check that the sample belongs to the specified instance, and otherwise it will not place the sample in the returned collection.

The behavior of the [FooDataReader::take\\_instance](#) operation follows the same rules as the [FooDataReader::read](#) operation regarding the pre-conditions and post-conditions for the `received_data` and `sample_info`. Similar to the [FooDataReader::read](#), the [FooDataReader::take\\_instance](#) operation may 'loan' elements to the output collections, which must then be returned by means of [FooDataReader::return\\_loan](#).

Similar to the [FooDataReader::read](#), this operation must be provided on the specialized class that is generated for the particular application data-type that is being taken.

If the [DDSDDataReader](#) has no samples that meet the constraints, the method fails with [DDS\\_RETCODE\\_NO\\_DATA](#).

This operation may fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#) if the [DDS\\_InstanceHandle\\_t](#) `a_handle` does not correspond to an existing data-object known to the [DDSDDataReader](#).

## Parameters

|                      |                                                                                                                                                                                |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>received_data</i> | <<inout>> user data type-specific <a href="#">FooSeq</a> object where the received data samples will be returned.                                                              |
| <i>info_seq</i>      | <<inout>> a <a href="#">DDS_SampleInfoSeq</a> object where the received sample info will be returned.                                                                          |
| <i>max_samples</i>   | <<in>> The maximum number of samples to be returned. If the special value <a href="#">DDS_LENGTH_UNLIMITED</a> is provided, as many samples will be returned as are available. |

|                        |                                                                                                                                                                                                                                                           |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>a_handle</i>        | << <i>in</i> >> The specified instance to return samples for. The method will fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> if the <code>handle</code> does not correspond to an existing data-object known to the <a href="#">DDSDDataReader</a> . |
| <i>sample_states</i>   | << <i>in</i> >> data samples matching ones of these <code>sample_states</code> are returned                                                                                                                                                               |
| <i>view_states</i>     | << <i>in</i> >> data samples matching ones of these <code>view_state</code> are returned                                                                                                                                                                  |
| <i>instance_states</i> | << <i>in</i> >> data samples matching ones of these <code>instance_state</code> are returned                                                                                                                                                              |

#### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#), [DDS\\_RETCODE\\_NO\\_DATA](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

#### See also

[FooDataReader::take](#)

[DDS\\_LENGTH\\_UNLIMITED](#)

### lookup\_instance()

```
DDS_InstanceHandle_t FooDataReader::lookup_instance (
    const Foo & key_holder)
```

Retrieve the instance `handle` that corresponds to an instance `key_holder`.

Useful for keyed data types.

This operation takes as a parameter an instance and returns a handle that can be used in subsequent operations that accept an instance handle as an argument. The instance parameter is only used for the purpose of examining the fields that define the key. This operation does not register the instance in question. If the instance has not been previously registered, or if for any other reason the Service is unable to provide an instance handle, the Service will return the special value `HANDLE_NIL`.

#### Parameters

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| <i>key_holder</i> | << <i>in</i> >> a user data type specific key holder. |
|-------------------|-------------------------------------------------------|

#### Returns

the instance handle associated with this instance. If `Foo` has no key, this method has no effect and returns [DDS\\_HANDLE\\_NIL](#)

is\_data\_consistent()

```
DDS_ReturnCode_t FooDataReader::is_data_consistent (
    DDS_Boolean & is_data_consistent,
    const Foo & sample,
    const struct DDS_SampleInfo & sample_info)
```

Checks if the sample has been overwritten by the DataWriter.

When a sample is received via Zero Copy transfer over shared memory, the sample can be reused by the DataWriter once it is removed from the DataWriter's send queue. Since there is no synchronization between the DataReader and the DataWriter, the sample could be overwritten by the DataWriter before it is processed by the DataReader. The [FooDataReader::is\\_data\\_consistent](#) operation can be used after processing the sample to check if it was overwritten by the DataWriter.

A precondition for using this operation is to set [DDS\\_DataWriterShmemRefTransferModeSettings::enable\\_data\\_consistency\\_check](#) to true.

Parameters

|                           |                                                                                                                          |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>is_data_consistent</i> | << <i>inout</i> >> Set to true if the sample is consistent (i.e., the sample has not been overwritten by the DataWriter) |
| <i>sample</i>             | << <i>in</i> >> Sample to be validated                                                                                   |
| <i>sample_info</i>        | << <i>in</i> >> <a href="#">DDS_SampleInfo</a> object received with the sample                                           |

Returns

One of the [Standard Return Codes](#) or [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#).

References [is\\_data\\_consistent\(\)](#).

Referenced by [is\\_data\\_consistent\(\)](#).

7.121 FooDataWriter Class Reference

Declares the interface required to support a user data type-specific data writer.

```
#include <dds_cpp_data.hxx>
```

## Public Member Functions

- [DDSDataWriter \\* as\\_datawriter](#) ()  
*Widen the given [FooDataWriter](#) pointer to a [DDSDataWriter](#) pointer.*
- [DDS\\_ReturnCode\\_t write](#) (const Foo &instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle)  
*Modifies the value of a data instance.*
- [DDS\\_InstanceHandle\\_t register\\_instance](#) (const Foo &instance\_data)  
*Informs RTI Connex Micro that the application will be modifying a particular instance.*
- [DDS\\_InstanceHandle\\_t register\\_instance\\_w\\_timestamp](#) (const Foo &instance\_data, const [DDS\\_Time\\_t](#) &source\_timestamp)  
*Performs the same functions as [register\\_instance](#) except that the application provides the value for the `source_timestamp`.*
- [DDS\\_ReturnCode\\_t unregister\\_instance](#) (const Foo &instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle)  
*Reverses the action of [FooDataWriter::register\\_instance](#).*
- [DDS\\_ReturnCode\\_t unregister\\_instance\\_w\\_timestamp](#) (const Foo &instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle, const [DDS\\_Time\\_t](#) &source\_timestamp)  
*Performs the same function as [FooDataWriter::unregister\\_instance](#) except that it also provides the value for the `source_timestamp`.*
- [DDS\\_ReturnCode\\_t dispose](#) (const Foo &instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle)  
*Requests the middleware to delete the data.*
- [DDS\\_ReturnCode\\_t dispose\\_w\\_timestamp](#) (const Foo &instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle, const [DDS\\_Time\\_t](#) &source\_timestamp)  
*Performs the same functions as [dispose](#) except that the application provides the value for the `source_timestamp` that is made available to [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS\\_SampleInfo](#).*
- [DDS\\_ReturnCode\\_t write\\_w\\_timestamp](#) (const Foo &instance\_data, const [DDS\\_InstanceHandle\\_t](#) &handle, const [DDS\\_Time\\_t](#) &source\_timestamp)  
*Performs the same function as [FooDataWriter::write](#) except that it also provides the value for the `source_timestamp`.*
- [DDS\\_ReturnCode\\_t get\\_loan](#) (Foo \*&sample)  
*Gets a sample managed by the DataWriter.*
- [DDS\\_ReturnCode\\_t discard\\_loan](#) (Foo &sample)  
*Returns a loaned sample back to the DataWriter.*
- Foo \* [create\\_data](#) ()  
*Creates a data sample and initializes it.*
- bool [delete\\_data](#) (Foo \*sample)  
*Destroys a user data type instance.*

## Static Public Member Functions

- static [FooDataWriter \\* narrow](#) ([DDSDataWriter](#) \*writer)  
*Narrow the given [DDSDataWriter](#) pointer to a [FooDataWriter](#) pointer.*

### 7.121.1 Detailed Description

Declares the interface required to support a user data type-specific data writer.

### 7.121.2 Member Function Documentation

#### narrow()

```
static FooDataWriter * FooDataWriter::narrow (
    DDSDataWriter * writer) [static]
```

Narrow the given [DDSDataWriter](#) pointer to a [FooDataWriter](#) pointer.

Check if the given `writer` is of type [FooDataWriter](#).

#### Parameters

|               |                                                                                                                                                                        |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>writer</i> | <<in>> Base-class <a href="#">DDSDataWriter</a> to be converted to the auto-generated class <a href="#">FooDataWriter</a> that extends <a href="#">DDSDataWriter</a> . |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### Returns

[FooDataWriter](#) if `writer` is of type `Foo`. Return NULL otherwise.

#### as\_datawriter()

```
DDSDataWriter * FooDataWriter::as_datawriter ()
```

Widen the given [FooDataWriter](#) pointer to a [DDSDataWriter](#) pointer.

#### Returns

[DDSDataWriter](#).

#### write()

```
DDS_ReturnCode_t FooDataWriter::write (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle)
```

Modifies the value of a data instance.

When this operation is used, RTI Connex Micro will automatically supply the value of the `source_timestamp` that is made available to [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS\\_SampleInfo](#). (Refer to [DDS\\_SampleInfo](#) details).

As a side effect, this operation asserts liveness on the [DDSDataWriter](#) itself, the [DDSPublisher](#) and the [DDSDomainParticipant](#).

Note that the special value [DDS\\_HANDLE\\_NIL](#) can be used for the parameter `handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the key).

If `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#).

RTI Connex DDS Micro will not detect the error when the `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the `key`). RTI Connex DDS Micro will treat as if the `write()` operation is for the instance as indicated by the `handle`.

This operation may block if the [RELIABILITY](#) `kind` is set to [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#) and the modification would cause data to be lost or else cause one of the limits specified in the [RESOURCE\\_LIMITS](#) to be exceeded.

Specifically, this operation may block in the following situations (note that the list may not be exhaustive), even if its [DDS\\_HistoryQosPolicyKind](#) is [DDS\\_KEEP\\_LAST\\_HISTORY\\_QOS](#):

- If  $(\text{DDS\_ResourceLimitsQosPolicy::max\_samples} < \text{DDS\_ResourceLimitsQosPolicy::max\_instances} * \text{DDS\_HistoryQosPolicy::depth})$ , then in the situation where the `max_samples` resource limit is exhausted, RTI Connex DDS Micro is allowed to discard samples of some other instance, as long as at least one sample remains for such an instance. If it is still not possible to make space available to store the modification, the writer is allowed to block.
- If  $(\text{DDS\_ResourceLimitsQosPolicy::max\_samples} < \text{DDS\_ResourceLimitsQosPolicy::max\_instances})$ , then the DataWriter may block regardless of the [DDS\\_HistoryQosPolicy::depth](#).

If this operation *does* block for any of the above reasons, the [RELIABILITY](#) `max_blocking_time` configures the maximum time the write operation may block (waiting for space to become available). If `max_blocking_time` elapses before the [DDSDataWriter](#) is able to store the modification without exceeding the limits, the operation will time out ([DDS\\_RETCODE\\_TIMEOUT](#)).

If there are no instance resources left, this operation may fail with [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#). Calling [FooDataWriter::unregister\\_instance](#) may help freeing up some resources.

#### Parameters

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i> | <<in>> The data to write.                                                                                                                                                                                                                                                                                                                                                                                                    |
| <i>handle</i>        | <<in>> Either the handle returned by a previous call to <a href="#">FooDataWriter::register_instance</a> , or else the special value <a href="#">DDS_HANDLE_NIL</a> . If <i>Foo</i> has a key and <code>handle</code> is not <a href="#">DDS_HANDLE_NIL</a> , <code>handle</code> must represent a registered instance of type <i>Foo</i> . Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> . |

#### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

#### See also

[DDSDataReader](#)

[FooDataWriter::write\\_w\\_timestamp](#)

## register\_instance()

```
DDS_InstanceHandle_t FooDataWriter::register_instance (
    const Foo & instance_data)
```

Informs RTI Connex DDS Micro that the application will be modifying a particular instance.

This operation is only useful for keyed data types. Using it for non-keyed types causes no effect and returns [DDS\\_HANDLE\\_NIL](#). The operation takes as a parameter an instance (of which only the key value is examined) and returns a `handle` that can be used in successive [write\(\)](#) or [dispose\(\)](#) operations.

The operation gives RTI Connex DDS Micro an opportunity to pre-configure itself to improve performance.

The use of this operation by an application is optional even for keyed types. If an instance has not been pre-registered, the application can use the special value [DDS\\_HANDLE\\_NIL](#) as the [DDS\\_InstanceHandle\\_t](#) paramater to the write or dispose operation and RTI Connex DDS Micro will auto-register the instance.

For best performance, the operation should be invoked prior to calling any operation that modifies the instance, such as [FooDataWriter::write](#), [FooDataWriter::write\\_w\\_timestamp](#), [FooDataWriter::dispose](#) and [FooDataWriter::dispose\\_w\\_timestamp](#) and the handle used in conjunction with the data for those calls.

When this operation is used, RTI Connex DDS Micro will automatically supply the value of the `source_timestamp` that is used.

This operation may fail and return [DDS\\_HANDLE\\_NIL](#) if [DDS\\_ResourceLimitsQosPolicy::max\\_instances](#) limit has been exceeded.

The operation is **idempotent**. If it is called for an already registered instance, it just returns the already allocated handle. This may be used to lookup and retrieve the handle allocated to a given instance.

This operation can only be called after [DDSDataWriter](#) has been enabled. Otherwise, [DDS\\_HANDLE\\_NIL](#) will be returned.

### Parameters

|                      |                                                                                                                                                  |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i> | << <i>in</i> >> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the function. . |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|

### Returns

For keyed data type, a handle that can be used in the calls that take a [DDS\\_InstanceHandle\\_t](#), such as `write`, `dispose`, `unregister_instance`, or return [DDS\\_HANDLE\\_NIL](#) on failure. If the `instance_data` is of a data type that has no keys, this function always return [DDS\\_HANDLE\\_NIL](#).

### See also

[FooDataWriter::unregister\\_instance](#)

**register\_instance\_w\_timestamp()**

```
DDS_InstanceHandle_t FooDataWriter::register_instance_w_timestamp (
    const Foo & instance_data,
    const DDS_Time_t & source_timestamp)
```

Performs the same functions as `register_instance` except that the application provides the value for the `source_timestamp`.

The provided `source_timestamp` potentially affects the relative order in which readers observe events from multiple writers.

This operation may fail and return `DDS_HANDLE_NIL` if `DDS_ResourceLimitsQosPolicy::max_instances` limit has been exceeded.

This operation can only be called after `DDSDDataWriter` has been enabled. Otherwise, `DDS_HANDLE_NIL` will be returned.

**Parameters**

|                         |                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i>    | <<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the function.                                                                                                                                                                                                                                                                 |
| <i>source_timestamp</i> | <<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. |

**Returns**

For keyed data type, return a handle that can be used in the calls that take a `DDS_InstanceHandle_t`, such as *write*, *dispose*, *unregister\_instance*, or return `DDS_HANDLE_NIL` on failure. If the `instance_data` is of a data type that has no keys, this function always return `DDS_HANDLE_NIL`.

**See also**

[FooDataWriter::unregister\\_instance](#)

**unregister\_instance()**

```
DDS_ReturnCode_t FooDataWriter::unregister_instance (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle)
```

Reverses the action of `FooDataWriter::register_instance`.

This operation is useful only for keyed data types. Using it for non-keyed types causes no effect and reports no error. The operation takes as a parameter an instance (of which only the key value is examined) and a handle.



This operation should only be called on an instance that is currently registered. This includes instances that have been auto-registered by calling operations such as `write` or `dispose` as described in [FooDataWriter::register\\_instance](#). Otherwise, this operation may fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#).

This only need be called just once per instance, regardless of how many times `register_instance` was called for that instance.

When this operation is used, RTI Connex DDS Micro will automatically supply the value of the `source_timestamp` that is used.

This operation informs RTI Connex DDS Micro that the [DDSDataWriter](#) is no longer going to provide any information about the instance. This operation also indicates that RTI Connex DDS Micro can locally remove all information regarding that instance. The application should not attempt to use the `handle` previously allocated to that instance after calling [FooDataWriter::unregister\\_instance\(\)](#).

The special value [DDS\\_HANDLE\\_NIL](#) can be used for the parameter `handle`. This indicates that the identity of the instance should be automatically deduced from the `instance_data` (by means of the `key`).

If `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#).

RTI Connex DDS Micro will not detect the error when the `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the `key`). RTI Connex DDS Micro will treat as if the [unregister\\_instance\(\)](#) operation is for the instance as indicated by the `handle`.

If after a [FooDataWriter::unregister\\_instance](#), the application wants to modify ([FooDataWriter::write](#) or [FooDataWriter::dispose](#)) an instance, it has to register it again, or else use the special `handle` value [DDS\\_HANDLE\\_NIL](#).

This operation does not indicate that the instance is deleted (that is the purpose of [FooDataWriter::dispose](#)). The operation [FooDataWriter::unregister\\_instance](#) just indicates that the [DDSDataWriter](#) no longer has anything to say about the instance. [DDSDataReader](#) entities that are reading the instance may receive a sample with [DDS\\_NOT\\_ALIVE\\_NO\\_WRITERS\\_INSTANCE\\_STATE](#) for the instance, unless there are other [DDSDataWriter](#) objects writing that same instance.

This operation can affect the ownership of the data instance (see [OWNERSHIP](#)). If the [DDSDataWriter](#) was the exclusive owner of the instance, then calling [unregister\\_instance\(\)](#) will relinquish that ownership.

If [DDS\\_ReliabilityQosPolicy::kind](#) is set to [DDS\\_RELIABLE\\_RELIABILITY\\_QOS](#) and the unregistration would overflow the resource limits of this writer or of a reader, this operation may block for up to [DDS\\_ReliabilityQosPolicy::max\\_blocking\\_time](#); if this writer is still unable to unregister after that period, this method will fail with [DDS\\_RETCODE\\_TIMEOUT](#).

#### Parameters

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i> | <<in>> The instance that should be unregistered. If <i>Foo</i> has a key and <code>instance_handle</code> is <a href="#">DDS_HANDLE_NIL</a> , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used. If <code>instance_data</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> .                             |
| <i>handle</i>        | <<in>> represents the instance to be unregistered. If <i>Foo</i> has a key and <code>handle</code> is <a href="#">DDS_HANDLE_NIL</a> , <code>handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If <i>Foo</i> has no key, <code>handle</code> is not used. If <code>handle</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> . |

**Returns**

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#)

**See also**

[FooDataWriter::register\\_instance](#)

[FooDataWriter::unregister\\_instance\\_w\\_timestamp](#)

**unregister\_instance\_w\_timestamp()**

```
DDS_ReturnCode_t FooDataWriter::unregister_instance_w_timestamp (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp)
```

Performs the same function as [FooDataWriter::unregister\\_instance](#) except that it also provides the value for the `source_timestamp`.

The provided `source_timestamp` potentially affects the relative order in which readers observe events from multiple writers.

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the [FooDataWriter::unregister\\_instance](#) operation.

This operation may block and may time out ([DDS\\_RETCODE\\_TIMEOUT](#)) under the same circumstances described for the `unregister_instance` operation.

**Parameters**

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i>    | <<in>> The instance that should be unregistered. If <i>Foo</i> has a key and <code>instance_handle</code> is <a href="#">DDS_HANDLE_NIL</a> , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used. If <code>instance_data</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> .                             |
| <i>handle</i>           | <<in>> represents the instance to be unregistered. If <i>Foo</i> has a key and <code>handle</code> is <a href="#">DDS_HANDLE_NIL</a> , <code>handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If <i>Foo</i> has no key, <code>handle</code> is not used. If <code>handle</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> . |
| <i>source_timestamp</i> | <<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers.                                                                             |

**Returns**

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

**See also**

[FooDataWriter::register\\_instance](#)

[FooDataWriter::unregister\\_instance](#)

**dispose()**

```
DDS_ReturnCode_t FooDataWriter::dispose (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle)
```

Requests the middleware to delete the data.

This operation is useful only for keyed data types. Using it for non-keyed types has no effect and reports no error.

The actual deletion is postponed until there is no more use for that data in the whole system.

Applications are made aware of the deletion by means of operations on the [DDSDataReader](#) objects that already knew that instance. [DDSDataReader](#) objects that didn't know the instance will never see it.

This operation does not modify the value of the instance. The `instance_data` parameter is passed just for the purposes of identifying the instance.

When this operation is used, RTI Connex DDS Micro will automatically supply the value of the `source_timestamp` that is made available to [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS\\_SampleInfo](#).

The constraints on the values of the handle parameter and the corresponding error behavior are the same specified for the [FooDataWriter::unregister\\_instance](#) operation.

The special value [DDS\\_HANDLE\\_NIL](#) can be used for the parameter `instance_handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the key).

If `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#).

RTI Connex DDS Micro will not detect the error when the `handle` is any value other than [DDS\\_HANDLE\\_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the key). RTI Connex DDS Micro will treat as if the [dispose\(\)](#) operation is for the instance as indicated by the `handle`.

This operation may block and time out ([DDS\\_RETCODE\\_TIMEOUT](#)) under the same circumstances described for [FooDataWriter::write\(\)](#).

If there are no instance resources left, this operation may fail with [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#). Calling [FooDataWriter::unregister\\_instance](#) may help freeing up some resources.

**Parameters**

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i> | <<in>> The data to dispose. If <i>Foo</i> has a key and <code>instance_handle</code> is <a href="#">DDS_HANDLE_NIL</a> , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used. If <i>Foo</i> has a key, <code>instance_data</code> can be NULL only if <code>instance_handle</code> is not <a href="#">DDS_HANDLE_NIL</a> . Otherwise, this method will fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> .                                                                                                                                                  |
| <i>handle</i>        | <<in>> Either the handle returned by a previous call to <a href="#">FooDataWriter::register_instance</a> , or else the special value <a href="#">DDS_HANDLE_NIL</a> . If <i>Foo</i> has a key and <code>instance_handle</code> is <a href="#">DDS_HANDLE_NIL</a> , <code>instance_handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If <i>Foo</i> has no key, <code>instance_handle</code> is not used. If <code>handle</code> is used, it must represent a registered instance of type <i>Foo</i> . Otherwise, this method fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> . |

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

## See also

[FooDataWriter::dispose\\_w\\_timestamp](#)

**dispose\_w\_timestamp()**

```
DDS_ReturnCode_t FooDataWriter::dispose_w_timestamp (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp)
```

Performs the same functions as `dispose` except that the application provides the value for the `source_timestamp` that is made available to [DDSDDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS\\_SampleInfo](#).

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the [FooDataWriter::dispose](#) operation.

This operation may block and time out ([DDS\\_RETCODE\\_TIMEOUT](#)) under the same circumstances described for [FooDataWriter::write](#).

If there are no instance resources left, this operation may fail with [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#). Calling [FooDataWriter::unregister\\_instance](#) may help freeing up some resources.

## Parameters

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i>    | <<in>> The data to dispose. If <i>Foo</i> has a key and <i>instance_handle</i> is <a href="#">DDS_HANDLE_NIL</a> , only the fields that represent the key are examined by the function. Otherwise, <i>instance_data</i> is not used.                                                                                                                                                                                                                                                                                                                                                           |
| <i>handle</i>           | <<in>> Either the handle returned by a previous call to <a href="#">FooDataWriter::register_instance</a> , or else the special value <a href="#">DDS_HANDLE_NIL</a> . If <i>Foo</i> has a key and <i>instance_handle</i> is <a href="#">DDS_HANDLE_NIL</a> , <i>instance_handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If <i>Foo</i> has no key, <i>instance_handle</i> is not used. If <i>handle</i> is used, it must represent a registered instance of type <i>Foo</i> . Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> |
| <i>source_timestamp</i> | <<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the <a href="#">DDSDDataReader</a> objects by means of the <code>source_timestamp</code> attribute inside the <a href="#">DDS_SampleInfo</a> .       |

## Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

## See also

[FooDataWriter::dispose](#)

**write\_w\_timestamp()**

```
DDS_ReturnCode_t FooDataWriter::write_w_timestamp (
    const Foo & instance_data,
    const DDS_InstanceHandle_t & handle,
    const DDS_Time_t & source_timestamp)
```

Performs the same function as [FooDataWriter::write](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS\\_SampleInfo](#). (Refer to [DDS\\_SampleInfo](#) details)

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the [FooDataWriter::write](#) operation.

This operation may block and time out ([DDS\\_RETCODE\\_TIMEOUT](#)) under the same circumstances described for [FooDataWriter::write](#).

If there are no instance resources left, this operation may fail with [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#). Calling [FooDataWriter::unregister\\_instance](#) may help free up some resources.

This operation may fail with [DDS\\_RETCODE\\_BAD\\_PARAMETER](#) under the same circumstances described for the write operation.

**Parameters**

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance_data</i>    | <<in>> The data to write.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <i>handle</i>           | <<in>> Either the handle returned by a previous call to <a href="#">FooDataWriter::register_instance</a> , or else the special value <a href="#">DDS_HANDLE_NIL</a> . If <i>Foo</i> has a key and <code>handle</code> is not <a href="#">DDS_HANDLE_NIL</a> , <code>handle</code> must represent a registered instance of type <i>Foo</i> . Otherwise, this method may fail with <a href="#">DDS_RETCODE_BAD_PARAMETER</a> .                                                                                                                                                            |
| <i>source_timestamp</i> | <<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the <a href="#">DDSDataReader</a> objects by means of the <code>source_timestamp</code> attribute inside the <a href="#">DDS_SampleInfo</a> . |

**Returns**

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_TIMEOUT](#), [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#) or [DDS\\_RETCODE\\_NOT\\_ENABLED](#).

**See also**

[FooDataWriter::write](#)  
[DDSDataReader](#)

**get\_loan()**

```
DDS_ReturnCode_t FooDataWriter::get_loan (
    Foo *& sample)
```

Gets a sample managed by the DataWriter.

This operation is supported while using Zero Copy transfer or FlatData language binding.

[FooDataWriter::get\\_loan](#) fails with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) if the above conditions are not satisfied.

The loaned sample is obtained from a DataWriter-managed sample pool and is uninitialized by default. An initialized sample can be obtained by setting [DDS\\_DataWriterResourceLimitsQosPolicy::initialize\\_writer\\_loaned\\_sample](#) to [DDS\\_BOOLEAN\\_TRUE](#). The [DDS\\_DataWriterResourceLimitsQosPolicy::writer\\_loaned\\_sample\\_allocation](#) settings can be used to configure the DataWriter-managed sample pool.

[FooDataWriter::get\\_loan](#) fails with [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#) if [DDS\\_DataWriterResourceLimitsQosPolicy::writer\\_loaned\\_samples](#) have been loaned, and none of those samples has been written with [FooDataWriter::write](#) or discarded via [FooDataWriter::discard\\_loan](#).

Samples returned from [FooDataWriter::get\\_loan](#) have an associated state. Due to the optimized nature of the write operation while using Zero Copy transfer over shared memory or FlatData language binding, this sample state is used to control when a sample is available for reuse after the write operation. The possible sample states are free, allocated, removed or serialized. A sample that has never been allocated is "free". [FooDataWriter::get\\_loan](#) takes a "free" or "removed" sample and makes it "allocated". When a sample is written, its state transitions from "allocated" to "serialized", and the DataWriter takes responsibility for returning the sample back to its sample pool. The sample remains in the "serialized" state until it is removed from the DataWriter queue.

Sample removal behavior differs slightly when using the Shared Memory transport or the Zero Copy v2 transport. When using the Shared Memory transport with a reliable DataWriter, the sample is removed from the DataWriter's queue when the sample is acknowledged by all DataReaders. For a best-effort DataWriter, the sample is removed from the queue immediately after the write operation. After the sample is removed from the DataWriter queue, the sample is put back into the sample pool, and its state transitions from "serialized" to "removed". At this time, a new call to [FooDataWriter::get\\_loan](#) may return the same sample.

When using Zero Copy v2 transport, Best Effort and Reliable DataWriters behave similarly. The samples are kept in the serialized state until the sample HISTORY.depth is reached. At that point they are removed from the DataWriter's queue and put back into the sample pool. Since Zero Copy v2 provides automatic sample synchronization if the sample is currently being read by a DataReader with the [FooDataReader::take](#) or [FooDataReader::read](#) APIs, the sample will not be removed from the DataWriter queue until the DataReader returns the loan.

A loaned sample should not be reused to write a new value after the first write operation. Instead, a new sample from [FooDataWriter::get\\_loan](#) should be used to write the new value.

A loaned sample that has not been written can be returned to the DataWriter's sample pool by using [FooDataWriter::discard\\_loan](#).

A DataWriter writing a type annotated with `@transfer_mode(SHMEM_REF)` or `@language_binding(FLAT_DATA)` cannot write unmanaged samples that have not been returned from [FooDataWriter::get\\_loan](#).

**Parameters**

|               |                                                                                                 |
|---------------|-------------------------------------------------------------------------------------------------|
| <i>sample</i> | <<inout>> reference to a user data type pointer. The loaned sample is returned via this sample. |
|---------------|-------------------------------------------------------------------------------------------------|

See also

[FooDataWriter::discard\\_loan](#)

### discard\_loan()

```
DDS_ReturnCode_t FooDataWriter::discard_loan (
    Foo & sample)
```

Returns a loaned sample back to the DataWriter.

This operation is supported while using Zero Copy transfer or the FlatData language binding.

[FooDataWriter::get\\_loan](#) fails with [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) if the above conditions are not satisfied.

A loaned sample that hasn't been written can be returned to the DataWriter with this operation.

#### Parameters

|               |                                                |
|---------------|------------------------------------------------|
| <i>sample</i> | << <i>in</i> >> loaned sample to be discarded. |
|---------------|------------------------------------------------|

#### Returns

One of the [Standard Return Codes](#)

#### See also

[FooDataWriter::get\\_loan](#)

### create\_data()

```
Foo * FooDataWriter::create_data ()
```

Creates a data sample and initializes it.

The behavior of this API is identical to [FooTypeSupport::create\\_data](#).

#### See also

[FooDataWriter::delete\\_data](#)

### delete\_data()

```
bool FooDataWriter::delete_data (
    Foo * sample)
```

Destroys a user data type instance.

The behavior of this API is identical to [FooTypeSupport::delete\\_data](#).

## Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>sample</i> | << <i>in</i> >> Cannot be NULL. |
|---------------|---------------------------------|

## Returns

DDS\_BOOLEAN\_TRUE on success.

## See also

[FooDataWriter::create\\_data](#)

## 7.122 FooSeq Struct Reference

<<*interface*>> <<*generic*>> A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo".

```
#include <dds_cpp_sequence_defn.hxx>
```

### Public Member Functions

- [FooSeq](#) & [operator=](#) (const struct [FooSeq](#) &src\_seq)  
*Copy elements from another sequence, resizing the sequence if necessary.*
- bool [copy\\_no\\_alloc](#) (const struct [FooSeq](#) &src\_seq)  
*Copy elements from another sequence, only if the destination sequence has enough capacity.*
- bool [from\\_array](#) (const Foo array[], [DDS\\_Long](#) length)  
*Copy elements from an array of elements, resizing the sequence if necessary. The original contents of the sequence (if any) are replaced.*
- bool [to\\_array](#) (Foo array[], [DDS\\_Long](#) length)  
*Copy elements to an array of elements. The original contents of the array (if any) are replaced.*
- Foo & [operator\[\]](#) ([DDS\\_Long](#) i)  
*Get the i-th element of the sequence.*
- const Foo & [operator\[\]](#) ([DDS\\_Long](#) i) const  
*Get the i-th element for a const sequence.*
- [DDS\\_Long](#) [length](#) () const  
*Get the logical length of this sequence.*
- bool [length](#) ([DDS\\_Long](#) new\_length)  
*Change the length of this sequence.*
- bool [ensure\\_length](#) ([DDS\\_Long](#) length, [DDS\\_Long](#) max)  
*Set the sequence to the desired length, and resize the sequence if necessary.*
- [DDS\\_Long](#) [maximum](#) () const  
*Get the current maximum number of elements that can be stored in this sequence.*
- bool [maximum](#) ([DDS\\_Long](#) new\_max)  
*Resize this sequence to a new desired maximum.*
- bool [loan\\_contiguous](#) (Foo \*buffer, [DDS\\_Long](#) new\_length, [DDS\\_Long](#) new\_max)  
*Loan a contiguous buffer to this sequence.*



- bool [unloan](#) ()  
*Return the loaned buffer in the sequence and set the maximum to 0.*
- Foo \* [get\\_contiguous\\_buffer](#) () const  
*Access the internal buffer of a sequence.*
- bool [has\\_ownership](#) ()  
*Return the value of the owned flag.*
- ~[FooSeq](#) ()  
*Deallocate this sequence's buffer.*
- [FooSeq](#) (DDS\_Long new\_max=0)  
*Create a sequence with the given maximum.*
- [FooSeq](#) (const struct [FooSeq](#) &foo\_seq)  
*Create a sequence by copying from an existing sequence.*

## Public Attributes

- Foo \* [\\_contiguous\\_buffer](#)  
*Pointer to array of contiguous or discontiguous data.*
- DDS\_UnsignedLong [\\_maximum](#)  
*Maximum size of the sequence.*
- DDS\_UnsignedLong [\\_length](#)  
*Actual length of the sequence that contains data.*
- DDS\_Long [\\_element\\_size](#)  
*Size of data element in the sequence.*
- void \* [\\_token1](#)  
*Reserved for internal use.*
- void \* [\\_token2](#)  
*Reserved for internal use.*
- DDS\_UnsignedLong [max\\_alloc\\_size](#)  
*The maximum allocation for pointer elements (string,wstring) Only in debug librarires.*
- DDS\_Char [\\_flags](#)  
*Reserved for internal use.*

### 7.122.1 Detailed Description

<<[interface](#)>> <<[generic](#)>> A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo".

The sequence offers a subset of the methods defined by the standard OMG IDL to C++ mapping for sequences. We refer to an IDL sequence<Foo> as [FooSeq](#).

The state of a sequence is described by the properties 'maximum', and 'length'.

- The 'maximum' represents the size of the underlying buffer; this is the maximum number of elements it can possibly hold. It is returned by the [FooSeq::maximum\(\)](#) operation.
- The 'length' represents the actual number of elements it currently holds. It is returned by the [FooSeq::length\(\)](#) operation.

### 7.122.2 Constructor & Destructor Documentation

#### ~FooSeq()

```
FooSeq::~~FooSeq ()
```

Deallocate this sequence's buffer.

#### Precondition

(owned == [DDS\\_BOOLEAN\\_TRUE](#)). If this precondition is not met, no memory will be freed and an error will be logged.

#### Postcondition

maximum == 0 and the underlying buffer is freed.

#### See also

[FooSeq::maximum\(\)](#), [FooSeq::unloan](#)

#### FooSeq() [1/2]

```
FooSeq::FooSeq (
    DDS\_Long new_max = 0)
```

Create a sequence with the given maximum.

This is a constructor for the sequence. The constructor will automatically allocate memory to hold new\_max elements of type Foo.

This constructor will be used when the application creates a sequence using one of the following:

```
FooSeq mySeq(5);
// or
FooSeq mySeq;
// or
FooSeq* mySeqPtr = new FooSeq(5);
```

#### Postcondition

maximum == new\_max  
length == 0  
owned == [DDS\\_BOOLEAN\\_TRUE](#),

#### Parameters

|                |                                                                          |
|----------------|--------------------------------------------------------------------------|
| <i>new_max</i> | Must be >= 0. Otherwise the sequence will be initialized to a new_max=0. |
|----------------|--------------------------------------------------------------------------|

Referenced by [copy\\_no\\_alloc\(\)](#), [FooSeq\(\)](#), and [operator=\(\)](#).

**FooSeq()** [2/2]

```
FooSeq::FooSeq (
    const struct FooSeq & foo_seq)
```

Create a sequence by copying from an existing sequence.

This is a constructor for the sequence. The constructor will automatically allocate memory to hold `foo_seq::maximum()` elements of type `Foo` and will copy the current contents of `foo_seq` into the new sequence.

This constructor will be used when the application creates a sequence using one of the following:

```
FooSeq mySeq(foo_seq);
// or
FooSeq mySeq = foo_seq;
// or
FooSeq *mySeqPtr = new FooSeq(foo_seq);
```

**Postcondition**

```
this::maximum == foo_seq::maximum
this::length == foo_seq::length
this[i] == foo_seq[i] for 0 <= i < foo_seq::length
this::owned == DDS_BOOLEAN_TRUE
```

**Note**

If the pre-conditions are not met, the constructor will initialize the new sequence to a maximum of zero.

References [FooSeq\(\)](#).

**7.122.3 Member Function Documentation****operator=()**

```
FooSeq & FooSeq::operator= (
    const struct FooSeq & src_seq)
```

Copy elements from another sequence, resizing the sequence if necessary.

This method invokes copies the content of the specified sequence, after ensuring that this sequence has enough capacity to hold the elements to be copied.

NOTE: If this method is called on a sequence that has been created with `set_maximum_w_max` or `ensure_length_w_max`, it will automatically call `copy_w_max()`.

This operator is invoked when the following expression appears in the code:

```
target_seq = src_seq
```

*Important:* This method *will* allocate memory if `this::maximum < src_seq::length`.

Therefore, to programatically detect the successful completion of the operator it is recommended that the application first sets the length of this sequence to zero, makes the assignment, and then checks that the length of this sequence matches that of `src_seq`.

## Parameters

|                      |                                                 |
|----------------------|-------------------------------------------------|
| <code>src_seq</code> | << <i>in</i> >> the sequence from which to copy |
|----------------------|-------------------------------------------------|

References [FooSeq\(\)](#).

**copy\_no\_alloc()**

```
bool FooSeq::copy_no_alloc (
    const struct FooSeq & src_seq)
```

Copy elements from another sequence, only if the destination sequence has enough capacity.

Fill the elements in this sequence by copying the corresponding elements in `src_seq`. The original contents in this sequence are replaced via the element assignment operation (`Foo_copy()` function). By default, elements are discarded; 'delete' is not invoked on the discarded elements.

## Precondition

```
this::maximum >= src_seq::length
this::owned == DDS_BOOLEAN_TRUE
```

## Postcondition

```
this::length == src_seq::length
this[i] == src_seq[i] for 0 <= i < target_seq::length
this::owned == DDS_BOOLEAN_TRUE
```

## Parameters

|                      |                                                 |
|----------------------|-------------------------------------------------|
| <code>src_seq</code> | << <i>in</i> >> the sequence from which to copy |
|----------------------|-------------------------------------------------|

## Returns

[DDS\\_BOOLEAN\\_TRUE](#) if the sequence was successfully copied; [DDS\\_BOOLEAN\\_FALSE](#) otherwise.

## Note

If the pre-conditions are not met, the operator will print a message to stdout and leave this sequence unchanged.

## See also

[FooSeq::operator=](#)

References [FooSeq\(\)](#).

**from\_array()**

```
bool FooSeq::from_array (
    const Foo array[],
    DDS_Long length)
```

Copy elements from an array of elements, resizing the sequence if necessary. The original contents of the sequence (if any) are replaced.

Fill the elements in this sequence by copying the corresponding elements in `array`. The original contents in this sequence are replaced via the element assignment operation (`Foo_copy()` function). By default, elements are discarded; 'delete' is not invoked on the discarded elements.

**Precondition**

`this::owned == DDS_BOOLEAN_TRUE`

**Postcondition**

`this::length == length`

`this[i] == array[i]` for  $0 \leq i < \text{length}$

`this::owned == DDS_BOOLEAN_TRUE`

**Parameters**

|               |                                                       |
|---------------|-------------------------------------------------------|
| <i>array</i>  | <<in>> The array of elements to be copy elements from |
| <i>length</i> | <<in>> The length of the array.                       |

**Returns**

`DDS_BOOLEAN_TRUE` if the array was successfully copied; `DDS_BOOLEAN_FALSE` otherwise.

References `length()`.

**to\_array()**

```
bool FooSeq::to_array (
    Foo array[],
    DDS_Long length)
```

Copy elements to an array of elements. The original contents of the array (if any) are replaced.

Copy the elements of this sequence to the corresponding elements in the array. The original contents of the array are replaced via the element assignment operation (`Foo_copy()` function). By default, elements are discarded; 'delete' is not invoked on the discarded elements.

## Parameters

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>array</i>  | <<in>> The array of elements to be filled with elements from this sequence |
| <i>length</i> | <<in>> The number of elements to be copied.                                |

## Returns

[DDS\\_BOOLEAN\\_TRUE](#) if the elements of the sequence were successfully copied; [DDS\\_BOOLEAN\\_FALSE](#) otherwise.

References [length\(\)](#).

**operator[]()** [1/2]

```
Foo & FooSeq::operator[] (
    DDS_Long i)
```

Get the *i*-th element of the sequence.

This is the operator that is invoked when the application indexes into a non-const sequence:

```
myElement = mySequence[i];
mySequence[i] = myElement;
```

## Parameters

|          |                                                                                             |
|----------|---------------------------------------------------------------------------------------------|
| <i>i</i> | index of element to access, must be $\geq 0$ and less than <a href="#">FooSeq::length()</a> |
|----------|---------------------------------------------------------------------------------------------|

## Returns

the *i*-th element

**operator[]()** [2/2]

```
const Foo & FooSeq::operator[] (
    DDS_Long i) const
```

Get the *i*-th element for a `const` sequence.

This is the operator that is invoked when the application indexes into a const sequence:

```
myElement = mySequence[i];
```

## Parameters

|          |                                                                                             |
|----------|---------------------------------------------------------------------------------------------|
| <i>i</i> | index of element to access, must be $\geq 0$ and less than <a href="#">FooSeq::length()</a> |
|----------|---------------------------------------------------------------------------------------------|

## Returns

the *i*-th element

**length()** [1/2]

```
DDS_Long FooSeq::length () const
```

Get the logical length of this sequence.

Get the length that was last set, or zero if the length has never been set.

**Returns**

the length of the sequence

Referenced by [ensure\\_length\(\)](#), [from\\_array\(\)](#), and [to\\_array\(\)](#).

**length()** [2/2]

```
bool FooSeq::length (  
    DDS_Long new_length)
```

Change the length of this sequence.

This method does not allocate/deallocate memory.

The new length must not exceed the maximum of this sequence as returned by the [FooSeq::maximum\(\)](#) operation. (Note that, if necessary, the maximum of this sequence can be increased manually by using the [FooSeq::maximum\(DDS\\_Long\)](#) operation.)

The elements of the sequence are not modified by this operation. If the new length is larger than the original length, the new elements will be uninitialized; if the length is decreased, the old elements that are beyond the new length will physically remain in the sequence but will not be accessible.

**Postcondition**

length = new\_length.

**Parameters**

|                   |                                                                                                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>new_length</i> | the new desired length. This value must be non-negative and cannot exceed maximum of the sequence. In other words $0 \leq \text{new\_length} \leq \text{maximum}$ |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Returns**

[DDS\\_BOOLEAN\\_TRUE](#) on success or [DDS\\_BOOLEAN\\_FALSE](#) on failure

**ensure\_length()**

```
bool FooSeq::ensure_length (
    DDS_Long length,
    DDS_Long max)
```

Set the sequence to the desired length, and resize the sequence if necessary.

If the current maximum is greater than the desired length, then sequence is not resized.

Otherwise if this sequence owns its buffer, the sequence is resized to the new maximum by freeing and re-allocating the buffer. However, if the sequence does not own its buffer, this operation will fail.

This function allows user to avoid unnecessary buffer re-allocation.

**Precondition**

length <= max  
owned == [DDS\\_BOOLEAN\\_TRUE](#) if sequence needs to be resized

**Postcondition**

length == length  
maximum == max if resized

**Parameters**

|               |                                                                                              |
|---------------|----------------------------------------------------------------------------------------------|
| <i>length</i> | <<in>> The new length that should be set. Must be >= 0.                                      |
| <i>max</i>    | <<in>> If sequence need to be resized, this is the maximum that should be set. max >= length |

**Returns**

[DDS\\_BOOLEAN\\_TRUE](#) on success, [DDS\\_BOOLEAN\\_FALSE](#) if the preconditions are not met. In that case the sequence is not modified.

References [length\(\)](#).

**maximum()** [1/2]

```
DDS_Long FooSeq::maximum () const
```

Get the current maximum number of elements that can be stored in this sequence.

The `maximum` of the sequence represents the maximum number of elements that the underlying buffer can hold. It does not represent the current number of elements.

The `maximum` is a non-negative number. It is initialized when the sequence is first created and can only be changed by mean of the [FooSeq::maximum\(DDS\\_Long\)](#) operation.

**Returns**

the current maximum of the sequence.

**See also**

[FooSeq::length\(\)](#)



**maximum()** [2/2]

```
bool FooSeq::maximum (
    DDS_Long new_max)
```

Resize this sequence to a new desired maximum.

This operation does nothing if the new desired maximum matches the current maximum.

If this sequence owns its buffer and the new maximum is not equal to the old maximum, then the existing buffer will be freed and re-allocated.

**Precondition**

owned == [DDS\\_BOOLEAN\\_TRUE](#)

**Postcondition**

owned == [DDS\\_BOOLEAN\\_TRUE](#)  
length == MINIMUM(original length, new\_max)

**Parameters**

|                |                    |
|----------------|--------------------|
| <i>new_max</i> | Must be $\geq 0$ . |
|----------------|--------------------|

**Returns**

[DDS\\_BOOLEAN\\_TRUE](#) on success, [DDS\\_BOOLEAN\\_FALSE](#) if the preconditions are not met. In that case the sequence is not modified.

**loan\_contiguous()**

```
bool FooSeq::loan_contiguous (
    Foo * buffer,
    DDS_Long new_length,
    DDS_Long new_max)
```

Loan a contiguous buffer to this sequence.

This operation changes the `owned` flag of the sequence to [DDS\\_BOOLEAN\\_FALSE](#) and also sets the underlying buffer used by the sequence. See the user's manual for more information about sequences and memory ownership.

Use this method if you want to manage the memory used by the sequence yourself. You must provide an array of elements and integers indicating how many elements are allocated in that array (i.e. the maximum) and how many elements are valid (i.e. the length). The sequence will subsequently use the memory you provide and will not permit it to be freed by a call to [FooSeq::maximum\(DDS\\_Long\)](#).

Once you have loaned a buffer to a sequence, make sure that you don't free it before calling [FooSeq::unloan](#): the next time you access the sequence, you will be accessing freed memory!

You can use this method to wrap stack memory with a sequence interface, thereby avoiding dynamic memory allocation. Create a [FooSeq](#) and an array of type `Foo` and then loan the array to the sequence:

```
Foo fooArray[10];
::FooSeq fooSeq;
fooSeq.loan_contiguous(fooArray, 0, 10);
```

By default, a sequence you create owns its memory unless you explicitly loan memory of your own to it. In a very few cases, RTI Connex Micro DDS will return a sequence to you that has a loan; those cases are documented as such. For example, if you call [FooDataReader::read](#) or [FooDataReader::take](#) and pass in sequences with no loan and no memory allocated, RTI Connex Micro DDS will loan memory to your sequences which must be unloaned with [FooDataReader::return\\_loan](#). See the documentation of those methods for more information.

**Precondition**

`FooSeq::maximum() == 0`; i.e. the sequence has no memory allocated to it.

`FooSeq::has_ownership == DDS_BOOLEAN_TRUE`; i.e. the sequence does not already have an outstanding loan

**Postcondition**

The sequence will store its elements in the buffer provided.

`FooSeq::has_ownership == DDS_BOOLEAN_FALSE`

`FooSeq::length() == new_length`

`FooSeq::maximum() == new_max`

**Parameters**

|                   |                                                                                                                                                                                                    |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>buffer</i>     | The new buffer that the sequence will use. Must point to enough memory to hold <code>new_max</code> elements of type <code>Foo</code> . It may be <code>NULL</code> if <code>new_max == 0</code> . |
| <i>new_length</i> | The desired new length for the sequence. It must be the case that that <code>0 &lt;= new_length &lt;= new_max</code> .                                                                             |
| <i>new_max</i>    | The allocated number of elements that could fit in the loaned buffer.                                                                                                                              |

**Returns**

`DDS_BOOLEAN_TRUE` if `buffer` is successfully loaned to this sequence or `DDS_BOOLEAN_FALSE` otherwise. Failure only occurs due to failing to meet the pre-conditions. Upon failure the sequence remains unmodified.

**See also**

[FooSeq::unloan](#)

**unloan()**

```
bool FooSeq::unloan ()
```

Return the loaned buffer in the sequence and set the maximum to 0.

This method affects only the state of this sequence; it does not change the contents of the buffer in any way.

Only the user who originally loaned a buffer should return that loan, as the user may have dependencies on that memory known only to them. Unloaning someone else's buffer may cause unspecified problems. For example, suppose a sequence is loaning memory from a custom memory pool. A user of the sequence likely has no way to release the memory back into the pool, so unloaning the sequence buffer would result in a resource leak. If the user were to then re-loan a different buffer, the original creator of the sequence would have no way to discover, when freeing the sequence, that the loan no longer referred to its own memory and would thus not free the user's memory properly, exacerbating the situation and leading to undefined behavior.

**Precondition**

owned == [DDS\\_BOOLEAN\\_FALSE](#)

**Postcondition**

owned == [DDS\\_BOOLEAN\\_TRUE](#)  
maximum == 0

**Returns**

[DDS\\_BOOLEAN\\_TRUE](#) if the preconditions were met. Otherwise [DDS\\_BOOLEAN\\_FALSE](#). The function only fails if the pre-conditions are not met, in which case it leaves the sequence unmodified.

**See also**

[FooSeq::loan\\_contiguous](#) [FooSeq::maximum\(DDS\\_Long\)](#)

**get\_contiguous\_buffer()**

```
Foo * FooSeq::get_contiguous_buffer () const
```

Access the internal buffer of a sequence.

Use this function to access the memory buffer that a sequence uses to store the values of its elements.

**has\_ownership()**

```
bool FooSeq::has_ownership ()
```

Return the value of the owned flag.

**Returns**

[DDS\\_BOOLEAN\\_TRUE](#) if sequence owns the underlying buffer, or [DDS\\_BOOLEAN\\_FALSE](#) if it has an outstanding loan.

**7.122.4 Member Data Documentation****\_contiguous\_buffer**

```
Foo* FooSeq::_contiguous_buffer
```

Pointer to array of contiguous or discontiguous data.

**\_maximum**

```
DDS_UnsignedLong FooSeq::_maximum
```

Maximum size of the sequence.

The allocated length of this sequence. It applies to whichever of the above buffers is non-NULL, if any. If both a NULL, its value must be 0.

If `_maximum == 0`, `_owned == true`.

**\_length**

```
DDS_UnsignedLong FooSeq::_length
```

Actual length of the sequence that contains data.

The current logical length of this sequence, i.e. the number of valid elements it contains. It applies to whichever of the above buffers is non-null, if any. If both are NULL, its value must be 0.

**\_element\_size**

```
DDS_Long FooSeq::_element_size
```

Size of data element in the sequence.

Each element in the sequence has this size.

**\_token1**

```
void* FooSeq::_token1
```

Reserved for internal use.

**\_token2**

```
void* FooSeq::_token2
```

Reserved for internal use.

**max\_alloc\_size**

```
DDS_UnsignedLong FooSeq::max_alloc_size
```

The maximum allocation for pointer elements (string,wstring) Only in debug librarires.

## **\_flags**

DDS\_Char FooSeq::\_flags

Reserved for internal use.

## **7.123 FooTypeSupport Class Reference**

**<<interface>>** **<<generic>>** User data type specific interface.

```
#include <dds_cpp_data.hxx>
```

### **Static Public Member Functions**

- static bool **initialize\_data** (Foo \*sample)  
**<<eXtension>>** Initialize data type.
- static bool **finalize\_data** (Foo \*sample)  
**<<eXtension>>** Finalize data type.
- static Foo \* **create\_data** ()  
**<<eXtension>>** Create a data type and initialize it.
- static DDS\_ReturnCode\_t **delete\_data** (Foo \*sample)  
**<<eXtension>>** Destroy a user data type instance.
- static bool **copy\_data** (Foo \*dst\_data, Foo \*src\_data)  
**<<eXtension>>** Copy data type.
- static DDS\_ReturnCode\_t **register\_type** (DDSDomainParticipant \*participant, const char \*type\_name)  
Allows an application to communicate to RTI Connex DDS Micro the existence of a data type.
- static DDS\_ReturnCode\_t **unregister\_type** (DDSDomainParticipant \*participant, const char \*type\_name)  
Allows an application to unregister a data type from RTI Connex DDS Micro. After calling **unregister\_type**, no further communication using that type is possible.
- static DDS\_ReturnCode\_t **serialize\_data\_to\_cdr\_buffer\_ex** (char \*buffer, unsigned int &length, const TData \*a\_data, DDS\_DataRepresentationId\_t representation)  
**<<eXtension>>** Serializes the input sample into a buffer of octets.
- static DDS\_ReturnCode\_t **serialize\_data\_to\_cdr\_buffer** (char \*buffer, unsigned int &length, const TData \*a\_data)  
**<<eXtension>>** Serializes the input sample into a CDR buffer of octets.
- static DDS\_ReturnCode\_t **deserialize\_data\_from\_cdr\_buffer** (TData \*a\_data, const char \*buffer, unsigned int length)  
**<<eXtension>>** Deserializes a sample from a buffer of octets.

### **7.123.1 Detailed Description**

**<<interface>>** **<<generic>>** User data type specific interface.

### **7.123.2 Member Function Documentation**

#### **initialize\_data()**

```
static bool FooTypeSupport::initialize_data (
    Foo * sample) [static]
```

**<<eXtension>>** Initialize data type.

The *generated* implementation of the operation knows how to initialize a data type. This method is typically called to initialize a data type that is allocated on the stack.

## Parameters

|               |                                    |
|---------------|------------------------------------|
| <i>sample</i> | << <i>inout</i> >> Cannot be NULL. |
|---------------|------------------------------------|

## Returns

One of the [Standard Return Codes](#)

## See also

[FooTypeSupport::finalize\\_data](#)

**finalize\_data()**

```
static bool FooTypeSupport::finalize_data (  
    Foo * sample) [static]
```

<<*eXtension*>> Finalize data type.

The *generated* implementation of the operation knows how to finalize a data type. This method is typically called to finalize a data type that has previously been initialized.

## Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>sample</i> | << <i>in</i> >> Cannot be NULL. |
|---------------|---------------------------------|

## Returns

One of the [Standard Return Codes](#)

## See also

[FooTypeSupport::initialize\\_data](#)

**create\_data()**

```
static Foo * FooTypeSupport::create_data () [static]
```

<<*eXtension*>> Create a data type and initialize it.

The *generated* implementation of the operation knows how to instantiate a data type and initialize it properly.

All memory for the type is deeply allocated.

## Returns

newly created data type

## See also

[FooTypeSupport::delete\\_data](#)

**delete\_data()**

```
static DDS_ReturnCode_t FooTypeSupport::delete_data (  
    Foo * sample) [static]
```

<<*eXtension*>> Destroy a user data type instance.

The *generated* implementation of the operation knows how to destroy a data type and return all resources.

**Parameters**

|               |                                 |
|---------------|---------------------------------|
| <i>sample</i> | << <i>in</i> >> Cannot be NULL. |
|---------------|---------------------------------|

**Returns**

One of the [Standard Return Codes](#)

**See also**

[FooTypeSupport::create\\_data](#)

**copy\_data()**

```
static bool FooTypeSupport::copy_data (  
    Foo * dst_data,  
    Foo * src_data) [static]
```

<<*eXtension*>> Copy data type.

The *generated* implementation of the operation knows how to copy value of a data type.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>dst_data</i> | << <i>inout</i> >> Data type to copy value to. Cannot be NULL. |
| <i>src_data</i> | << <i>in</i> >> Data type to copy value from. Cannot be NULL.  |

**Returns**

One of the [Standard Return Codes](#)

**register\_type()**

```
static DDS_ReturnCode_t FooTypeSupport::register_type (
    DDSDomainParticipant * participant,
    const char * type_name) [static]
```

Allows an application to communicate to RTI Connex DDS Micro the existence of a data type.

The *generated* implementation of the operation embeds all the knowledge that has to be communicated to the middle-ware in order to make it able to manage the contents of data of that type. This includes in particular the key definition that will allow RTI Connex DDS Micro to distinguish different instances of the same type.

The same [DDS\\_TYPESUPPORT\\_CPP](#) can be registered multiple times with a [DDSDomainParticipant](#) using the same or different values for the `type_name`. If `register_type` is called multiple times on the same [DDS\\_TYPESUPPORT\\_CPP](#) with the same [DDSDomainParticipant](#) and `type_name`, the second (and subsequent) registrations return [DDS\\_RETCODE\\_OK](#). A type must be registered and unregistered an equal number of times.

**Precondition**

Cannot use the same `type_name` to register two different [DDS\\_TYPESUPPORT\\_CPP](#) with the same [DDSDomainParticipant](#), or else the operation will fail and [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) will be returned.

**Parameters**

|                    |                                                                                                                                                                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>participant</i> | << <i>in</i> >> the <a href="#">DDSDomainParticipant</a> to register the data type <code>Foo</code> with. Cannot be NULL.                                                                                                                                                                    |
| <i>type_name</i>   | << <i>in</i> >> the type name under with the data type <code>Foo</code> is registered with the participant; this type name is used when creating a new <a href="#">DDSTopic</a> . (See <a href="#">DDSDomainParticipant::create_topic</a> .) The name may not be longer than 255 characters. |

**Returns**

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_PRECONDITION\\_NOT\\_MET](#) or [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#).

**MT Safety:**

UNSAFE on the FIRST call. It is not safe for two threads to simultaneously make the first call to register a type. Subsequent calls are thread safe.

**See also**

[DDSDomainParticipant::create\\_topic](#)



## unregister\_type()

```
static DDS_ReturnCode_t FooTypeSupport::unregister_type (
    DDSDomainParticipant * participant,
    const char * type_name) [static]
```

Allows an application to unregister a data type from RTI Connex DDS Micro. After calling `unregister_type`, no further communication using that type is possible.

The *generated* implementation of the operation removes all the information about a type from RTI Connex DDS Micro. No further communication using that type is possible.

### Precondition

A type with `type_name` is registered with the participant and all [DDSTopic](#) objects referencing the type have been destroyed. If the type is not registered with the participant, or if any [DDSTopic](#) is associated with the type, the operation will fail with [DDS\\_RETCODE\\_ERROR](#).

### Postcondition

All information about the type is removed from RTI Connex DDS Micro. No further communication using this type is possible.

### Parameters

|                    |                                                                                                                                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>participant</i> | << <i>in</i> >> the <a href="#">DDSDomainParticipant</a> to unregister the data type <code>Foo</code> from. Cannot be NULL.                                                                                                      |
| <i>type_name</i>   | << <i>in</i> >> the type name under which the data type <code>Foo</code> is registered with the participant. The name should match a name that has been previously used to register a type with the participant. Cannot be NULL. |

### Returns

One of the [Standard Return Codes](#), [DDS\\_RETCODE\\_BAD\\_PARAMETER](#) or [DDS\\_RETCODE\\_ERROR](#)

### MT Safety:

SAFE.

### See also

[DDSDomainParticipant::register\\_type](#)

## serialize\_data\_to\_cdr\_buffer\_ex()

```
static DDS_ReturnCode_t FooTypeSupport::serialize_data_to_cdr_buffer_ex (
    char * buffer,
    unsigned int & length,
    const TData * a_data,
    DDS_DataRepresentationId_t representation) [static]
```

<<*eXtension*>> Serializes the input sample into a buffer of octets.

This method serializes a sample into a buffer of octets using the input data representation. See [FooTypeSupport::serialize\\_data\\_to\\_cdr\\_buffer\\_ex](#) for details.

## Parameters

|                       |                                                             |
|-----------------------|-------------------------------------------------------------|
| <i>a_data</i>         | << <i>in</i> >>. Input sample. Cannot be NULL.              |
| <i>buffer</i>         | << <i>out</i> >>. Serialization buffer.                     |
| <i>length</i>         | << <i>inout</i> >>. Serialization buffer length.            |
| <i>representation</i> | << <i>in</i> >>. Representation used to serialize the data. |

## Returns

One of the [Standard Return Codes](#)

**serialize\_data\_to\_cdr\_buffer()**

```
static DDS_ReturnCode_t FooTypeSupport::serialize_data_to_cdr_buffer (
    char * buffer,
    unsigned int & length,
    const TData * a_data) [static]
```

<<*eXtension*>> Serializes the input sample into a CDR buffer of octets.

This method serializes a sample into a buffer of octets and it uses CDR as the data representation. Calling this method is equivalent to calling `FooTypeSupport::serialize_data_to_cdr_buffer_ex` with `DDS_AUTO_DATA_REPRESENTATION` as the representation.

The input buffer must be big enough to store the serialized representation of the sample. Otherwise, the method will return an error.

To determine the minimum size of the input buffer, the user must call this method with the buffer set to NULL.

## Parameters

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>a_data</i> | << <i>in</i> >>. Input sample. Cannot be NULL.                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>buffer</i> | << <i>out</i> >>. Serialization buffer.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <i>length</i> | << <i>inout</i> >>. When <i>buffer</i> is set to NULL, after the method executes, <i>length</i> will contain a buffer size big enough to hold the serialized data. When <i>buffer</i> is not NULL, <i>length</i> must contain the size of the input buffer when the method is invoked. After the method executes, <i>length</i> will be updated to contain the actual size of the serialized content, which may be smaller than the size obtained when <i>buffer</i> is set to NULL. |

## Returns

One of the [Standard Return Codes](#)

**deserialize\_data\_from\_cdr\_buffer()**

```
static DDS_ReturnCode_t FooTypeSupport::deserialize_data_from_cdr_buffer (
    TData * a_data,
    const char * buffer,
    unsigned int length) [static]
```

<<*eXtension*>> Deserializes a sample from a buffer of octets.

This method deserializes a sample from a CDR buffer of octets.

The content of the buffer generated by the method `FooTypeSupport::serialize_data_to_cdr_buffer` can be provided to this method to get the sample back.

**Parameters**

|               |                                                                                       |
|---------------|---------------------------------------------------------------------------------------|
| <i>a_data</i> | << <i>in</i> >>. Output sample. Cannot be NULL.                                       |
| <i>buffer</i> | << <i>in</i> >>. Deserialization buffer. Cannot be NULL.                              |
| <i>length</i> | << <i>in</i> >>. Length of the serialized representation of the sample in the buffer. |

**Returns**

One of the [Standard Return Codes](#)

## 7.124 NDDS\_Discovery\_Property Struct Reference

Specifies internal information used to store discovery information about a [DDSDomainParticipant](#) in the registry.

```
#include <dds_c_domain.h>
```

### 7.124.1 Detailed Description

Specifies internal information used to store discovery information about a [DDSDomainParticipant](#) in the registry.

## 7.125 NDDS\_Type\_PluginVersion Struct Reference

Data type used to keep track of versioning information about a [DDS\\_TypePluginI](#).

```
#include <dds_c_type.h>
```

**Public Attributes**

- char [majorRev](#)  
*Major version number.*
- char [minorRev](#)  
*Minor version number.*

### 7.125.1 Detailed Description

Data type used to keep track of versioning information about a [DDS\\_TypePluginI](#).

### 7.125.2 Member Data Documentation

**majorRev**

```
char NDDS_Type_PluginVersion::majorRev
```

Major version number.

**minorRev**

```
char NDDS_Type_PluginVersion::minorRev
```

Minor version number.

**7.126 NETIO\_Address Struct Reference**

```
#include <netio_address.h>
```

**Public Attributes**

- [RTI\\_INT32](#) kind  
*The address kind.*
- [RTI\\_UINT32](#) port  
*The address port.*
- union [NETIO\\_AddressValue](#) value  
*The transport address.*

**7.126.1 Detailed Description**

<<[eXtension](#)>> The NETIO Address.

**7.126.2 Member Data Documentation****kind**

```
RTI\_INT32 NETIO_Address::kind
```

The address kind.

**port**

```
RTI\_UINT32 NETIO_Address::port
```

The address port.

**value**

```
union NETIO\_AddressValue NETIO_Address::value
```

The transport address.

## 7.127 NETIO\_AddressSeq Struct Reference

<<*eXtension*>> The [NETIO\\_AddressSeq](#) sequence type.

```
#include <netio_address.h>
```

### 7.127.1 Detailed Description

<<*eXtension*>> The [NETIO\\_AddressSeq](#) sequence type.

## 7.128 NETIO\_AddressUInt32 Struct Reference

<<*eXtension*>> [NETIO\\_AddressUInt32](#)

```
#include <netio_address.h>
```

### Public Attributes

- [RTI\\_UINT32](#) value [4]  
*Array of unsigned 32-bit integers.*

### 7.128.1 Detailed Description

<<*eXtension*>> [NETIO\\_AddressUInt32](#)

### 7.128.2 Member Data Documentation

#### value

```
RTI\_UINT32 NETIO_AddressUInt32::value[4]
```

Array of unsigned 32-bit integers.

## 7.129 NETIO\_AddressValue Union Reference

<<*eXtension*>> [NETIO\\_AddresValue](#)

```
#include <netio_address.h>
```

**Public Attributes**

- struct NETIO\_AddressIpv6 [address](#)  
*Address as array of octets address.octet[0-15].*
- struct NETIO\_AddressUInt32 [as\\_uint32](#)  
*Address as array of unsigned 32-bit integers.*

**7.129.1 Detailed Description**

<<*eXtension*>> NETIO\_AddresValue

**7.129.2 Member Data Documentation****address**

```
struct NETIO_AddressIpv6 NETIO_AddressValue::address
```

Address as array of octets address.octet[0-15].

**as\_uint32**

```
struct NETIO_AddressUInt32 NETIO_AddressValue::as_uint32
```

Address as array of unsigned 32-bit integers.

**7.130 NETIO\_DGRAM\_Interface Struct Reference**

<<*eXtension*>> Definition of the NETIO\_DGRAM Interface.

```
#include <netio_dgram.h>
```

**Public Attributes**

- NETIO\_DGRAM\_Interface\_createFunc [create\\_instance](#)  
*Creates an instance of the user defined datagram Interface.*
- NETIO\_DGRAMUserInterface\_deleteFunc [delete\\_instance](#)  
*Deletes an instance of the user defined datagram Interface. Not in CERT.*
- NETIO\_DGRAMUserInterface\_get\_interface\_listFunc [get\\_interface\\_list](#)  
*Read available interfaces from the user DGRAM Interface.*
- NETIO\_Interface\_release\_addressFunc [release\\_address](#)  
*Release addresses to the DGRAMInterfaceUser.*
- NETIO\_DGRAMUserInterface\_resolve\_addressFunc [resolve\\_address](#)  
*Resolve an address string from DGRAM user interface.*
- NETIO\_Interface\_sendFunc [send](#)  
*Send a message with the DGRAM user interface.*
- NETIO\_Interface\_get\_route\_tableFunc [get\\_route\\_table](#)  
*Read the route table from the DGRAM user interface.*
- NETIO\_DGRAMUserInterface\_bind\_addressFunc [bind\\_address](#)  
*Bind to an address.*

### 7.130.1 Detailed Description

<<*eXtension*>> Definition of the NETIO\_DGRAM Interface.

### 7.130.2 Member Data Documentation

#### create\_instance

NETIO\_DGRAM\_Interface\_createFunc NETIO\_DGRAM\_InterfaceI::create\_instance

Creates an instance of the user defined datagram Interface.

A user defined datagram function that is called by RTI Connex DDS Micro to create an instance of the NETIO\_DGRAM transport.

##### Parameters

|    |                 |                                      |
|----|-----------------|--------------------------------------|
| in | <i>upstream</i> | The upstream interface.              |
| in | <i>property</i> | The user defined interface property. |

##### Returns

The function must return a pointer to a new instance on success and NULL on failure.

#### delete\_instance

NETIO\_DGRAMUserInterface\_deleteFunc NETIO\_DGRAM\_InterfaceI::delete\_instance

Deletes an instance of the user defined datagram Interface. Not in CERT.

A user defined datagram function that is called by RTI Connex DDS Micro to create an instance of the NETIO\_DGRAM transport.

##### Parameters

|    |                 |                         |
|----|-----------------|-------------------------|
| in | <i>upstream</i> | The upstream interface. |
|----|-----------------|-------------------------|

##### Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

#### get\_interface\_list

NETIO\_DGRAMUserInterface\_get\_interface\_listFunc NETIO\_DGRAM\_InterfaceI::get\_interface\_list

Read available interfaces from the user DGRAM Interface.

A user defined datagram function that is called by RTI Connex DDS Micro to read the available interfaces from a transport.

**Parameters**

|    |                   |                             |
|----|-------------------|-----------------------------|
| in | <i>netio_intf</i> | The DGRAM interface to use. |
| in | <i>if_table</i>   | The interface table.        |

**Returns**

TRUE on success, FALSE on failure.

**release\_address**

NETIO\_Interface\_release\_addressFunc NETIO\_DGRAM\_InterfaceI::release\_address

Release addresses to the DGRAMInterfaceUser.

A user defined datagram function that is called by RTI Connex DDS Micro when an address is no longer in use.

**Parameters**

|    |                 |                             |
|----|-----------------|-----------------------------|
| in | <i>self</i>     | The DGRAM interface to use. |
| in | <i>src_addr</i> | The address to release.     |

**Returns**

TRUE on success, FALSE on failure.

**resolve\_address**

NETIO\_DGRAMUserInterface\_resolve\_addressFunc NETIO\_DGRAM\_InterfaceI::resolve\_address

Resolve an address string from DGRAM user interface.

A user defined datagram function that is called by RTI Connex DDS Micro to convert an address in string format to a numerical value. The *if\_entry* *multicast\_group* decides if the address is a multicast group. The *address\_value* is automatically marked as a multicast address if the *resolve\_address* function returns RTI\_TRUE.

If *resolve\_address* is NULL when calling [NETIO\\_DGRAM\\_InterfaceFactory\\_register](#), RTI Connex DDS Micro will resolve addresses in the dotted IPv4 notation "a.b.c.d" where a, b, c, and d are decimal integers between 0-255.

**Parameters**

|     |                       |                                                       |
|-----|-----------------------|-------------------------------------------------------|
| in  | <i>netio_intf</i>     | Interface.                                            |
| in  | <i>if_entry</i>       | The interface entry requested to resolve the address. |
| in  | <i>address_string</i> | Address to convert.                                   |
| out | <i>address_value</i>  | Converted address.                                    |
| out | <i>is_invalid</i>     | If the address is valid or not.                       |

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure.



## send

```
NETIO_Interface_sendFunc NETIO_DGRAM_InterfaceI::send
```

Send a message with the DGRAM user interface.

A user defined datagram function that is called by RTI Connex DDS Micro to forward a packet from a source interface to a destination with the user defined DGRAM interface.

### Parameters

|    |                    |                                         |
|----|--------------------|-----------------------------------------|
| in | <i>self</i>        | DGRAM interface to send from.           |
| in | <i>source</i>      | The source interface for the packet.    |
| in | <i>destination</i> | The destination address for the packet. |
| in | <i>packet</i>      | The packet to send.                     |

### Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

## get\_route\_table

```
NETIO_Interface_get_route_tableFunc NETIO_DGRAM_InterfaceI::get_route_table
```

Read the route table from the DGRAM user interface.

A user defined datagram function that will be used by RTI Connex DDS Micro to get the route table from the DGRAM interface. The route table is a sequence of address and netmask pairs that informs RTI Connex DDS Micro which locators can be reached from the transport.

### Parameters

|         |                   |                                                         |
|---------|-------------------|---------------------------------------------------------|
| in      | <i>netio_intf</i> | The NETIO interface.                                    |
| in, out | <i>address</i>    | Sequence of NETIO addresses this interface understands. |
| in, out | <i>netmask</i>    | Sequence of the corresponding netmasks.                 |

### Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

## bind\_address

```
NETIO_DGRAMUserInterface_bind_addressFunc NETIO_DGRAM_InterfaceI::bind_address
```

Bind to an address.

A user defined datagram function called by RTI Connex DDS Micro to starting listening on the specified address.

## Parameters

|    |                  |                            |
|----|------------------|----------------------------|
| in | <i>user_intf</i> | DGRAM interface to to use. |
| in | <i>source</i>    | The address to bind.       |

## Returns

RTI\_TRUE if successful, RTI\_FALSE if not.

## 7.131 NETIO\_DGRAM\_InterfaceMultiCastGroup Struct Reference

<<*eXtension*>> Definition of a valid multicast group address range.

```
#include <netio_dgram.h>
```

## Public Attributes

- struct [NETIO\\_Address](#) *address\_low*  
*The lowest multicast address.*
- struct [NETIO\\_Address](#) *address\_high*  
*The highest multicast address.*
- struct [NETIO\\_Netmask](#) *netmask*  
*The netmask to apply to the group.*

### 7.131.1 Detailed Description

<<*eXtension*>> Definition of a valid multicast group address range.

### 7.131.2 Member Data Documentation

**address\_low**

```
struct NETIO\_Address NETIO_DGRAM_InterfaceMultiCastGroup::address_low
```

The lowest multicast address.

**address\_high**

```
struct NETIO\_Address NETIO_DGRAM_InterfaceMultiCastGroup::address_high
```

The highest multicast address.

## netmask

```
struct NETIO_Netmask NETIO_DGRAM_InterfaceMultiCastGroup::netmask
```

The netmask to apply to the group.

The netmask is a sequence of N consecutive 1s, starting with the left most bit. In order to determine if an address is a multicast address the netmask is first applied to the address and the high and low address of the multicast group. If the masked address is within the multicast group it is treated as a multicast address.

## 7.132 NETIO\_DGRAM\_InterfaceRouteEntry Struct Reference

<<eXtension>> Definition of a route entry.

```
#include <netio_dgram.h>
```

### Public Attributes

- struct [NETIO\\_Address address](#)  
*The address of the entry.*
- struct [NETIO\\_Netmask netmask](#)  
*The netmask of the entry used for comparing addresses.*

### 7.132.1 Detailed Description

<<eXtension>> Definition of a route entry.

### 7.132.2 Member Data Documentation

#### address

```
struct NETIO_Address NETIO_DGRAM_InterfaceRouteEntry::address
```

The address of the entry.

The [NETIO\\_DGRAM\\_InterfaceRouteEntry](#) structure provides a convenient method to add routes to a NETIO\_DGRAM transport. The address attribute is the addresses that can be reached after the netmask attribute has been applied.

#### netmask

```
struct NETIO_Netmask NETIO_DGRAM_InterfaceRouteEntry::netmask
```

The netmask of the entry used for comparing addresses.

## 7.133 NETIO\_DGRAM\_InterfaceTableEntry Struct Reference

<<*eXtension*>> Definition of an interface.

```
#include <netio_dgram.h>
```

### Public Attributes

- [RTI\\_INT32 locator\\_kind](#)  
*Locator type supported by the interface, such as UDPv4.*
- [RTI\\_UINT32 flags](#)  
*The flags attribute is a bitmap used to indicate capabilities of the transport.*
- [RTI\\_UINT32 mtu](#)  
*The maximum message size that can be sent and received by the transport.*
- struct [NETIO\\_Address address](#)  
*The address of the interface.*
- struct [NETIO\\_DGRAM\\_InterfaceMultiCastGroup multicast\\_group](#)  
*An optional multicast group for the interface.*
- const char \* [ifname](#)  
*The interface name is not used by RTI Connex DDS Micro.*

### 7.133.1 Detailed Description

<<*eXtension*>> Definition of an interface.

### 7.133.2 Member Data Documentation

#### locator\_kind

```
RTI_INT32 NETIO_DGRAM_InterfaceTableEntry::locator_kind
```

Locator type supported by the interface, such as UDPv4.

The locator\_kind describes the transport protocol. For example, a locator\_kind equal to 1 is reserved for DDS UDPv4 transport. Note the locator\_kind is used discovery and if a locator\_kind is discovered, but unsupported, no communication will take place using locators of that kind.

Note that the locator\_kind must be less than [NETIO\\_ADDRESS\\_RTPS\\_RESERVED\\_LOW](#) and greater than [NETIO\\_ADDRESS\\_RTPS\\_RESERVED\\_HIGH](#).

#### flags

```
RTI_UINT32 NETIO_DGRAM_InterfaceTableEntry::flags
```

The flags attribute is a bitmap used to indicate capabilities of the transport.

**mtu**

```
RTI_UINT32 NETIO_DGRAM_InterfaceTableEntry::mtu
```

The maximum message size that can be sent and received by the transport.

The mtu should be set to the maximum number of octets that can be sent and received in a message over the transport, excluding overhead required by the transport. RTI Connex DDS Micro uses the smallest mtu across all available transports to determine the maximum number of octets that can be sent in a single message.

**address**

```
struct NETIO_Address NETIO_DGRAM_InterfaceTableEntry::address
```

The address of the interface.

This is the address that is announced to other DDS DomainParticipants that this transport can receive messages on.

Note: This address must not be within the multicast group.

**multicast\_group**

```
struct NETIO_DGRAM_InterfaceMultiCastGroup NETIO_DGRAM_InterfaceTableEntry::multicast_group
```

An optional multicast group for the interface.

This attribute can be used to specify a group of addresses that should be considered multicast addresses. Multicast addresses are address that is sent to once, but received by many and can be a more efficient mechanism to distribute data. RTI Connex DDS Micro will prioritize sending to multicast addresses over unicast addresses.

Note the following restrictions:

- All the multicast\_group.netmask.mask array elements must be 0
- The multicast\_group.netmask.bits must be in the range [0,128]

**ifname**

```
const char* NETIO_DGRAM_InterfaceTableEntry::ifname
```

The interface name is not used by RTI Connex DDS Micro.

**7.134 NETIO\_DGRAM\_InterfaceTableEntrySeq Struct Reference**

A sequence of NETIO\_DGRAMInterfaceTableEntry elements.

```
#include <netio_dgram.h>
```

### 7.134.1 Detailed Description

A sequence of NETIO\_DGRAMInterfaceTableEntry elements.

## 7.135 NETIO\_Interface Struct Reference

Base-class definition for all NETIO interfaces.

```
#include <netio_interface.h>
```

### 7.135.1 Detailed Description

Base-class definition for all NETIO interfaces.

[`<<eXtension>> NETIO\_Interface`](#) base-class.

All NETIO layers are RT components and are derived from RT components. All NETIO layers also share a common base class, [`NETIO\_Interface`](#) and share common properties.

## 7.136 NETIO\_InterfaceI Struct Reference

```
#include <netio_interface.h>
```

### 7.136.1 Detailed Description

[`<<eXtension>> NETIO\_InterfaceI`](#). This type is internal to RTI

## 7.137 NETIO\_Netmask Struct Reference

[`<<eXtension>> NETIO\_Netmask`](#).

```
#include <netio_address.h>
```

### Public Attributes

- [`RTI\_UINT32`](#) bits  
*Number of bits in Netmask.*
- [`RTI\_UINT32`](#) mask [NETIO\_NETMASK\_MASK\_LENGTH]  
*The Netmask as a sequence of consecuteive 1s from left to right.*

### 7.137.1 Detailed Description

<<*eXtension*>> [NETIO\\_Netmask](#).

### 7.137.2 Member Data Documentation

#### bits

[RTI\\_UINT32](#) [NETIO\\_Netmask::bits](#)

Number of bits in Netmask.

#### mask

[RTI\\_UINT32](#) [NETIO\\_Netmask::mask](#)[[NETIO\\_NETMASK\\_MASK\\_LENGTH](#)]

The Netmask as a sequence of consecutive 1s from left to right.

## 7.138 NETIO\_NetmaskSeq Struct Reference

<<*eXtension*>> The [NETIO\\_NetmaskSeq](#) sequence type.

```
#include <netio_address.h>
```

### 7.138.1 Detailed Description

<<*eXtension*>> The [NETIO\\_NetmaskSeq](#) sequence type.

## 7.139 NETIO\_Packet Struct Reference

Implementation of the abstract [NETIO\\_Packet](#) type.

```
#include <netio_interface.h>
```

### 7.139.1 Detailed Description

Implementation of the abstract [NETIO\\_Packet](#) type.

<<*eXtension*>> The NETIO Packet API.

The NETIO layers passed [NETIO\\_Packet](#) structures upstream/downstream. Any meta-data about the packet must be encapsulated either in the packet payload or the packet info field.

## 7.140 NETIO\_SharedMemorySegmentHeader Struct Reference

```
#include <netio_shmem_segment.h>
```

### 7.140.1 Detailed Description

Opaque handle initialized by the attach methods and used to interact with a shared memory segment.

## 7.141 NETIO\_SHMEMInterfaceFactoryProperty Struct Reference

<<eXtension>> Properties for the SHMEM Transport.

```
#include <netio_shmem.h>
```

### Public Attributes

- [RTI\\_INT32 received\\_message\\_count\\_max](#)  
*Max number of received message sizes that can be residing inside the shared memory transport concurrent queue.*
- [RTI\\_INT32 receive\\_buffer\\_size](#)  
*The size of the receive socket buffer.*
- [RTI\\_INT32 message\\_size\\_max](#)  
*The maximum size of the message which can be received.*
- struct [OSAPI\\_ThreadProperty rcv\\_thread\\_property](#)  
*Thread properties for each receive thread created by this NETIO interface.*
- struct [DDS\\_ProductVersion pro\\_minimum\\_compatibility\\_version](#)  
*The minimum Connex Pro version that the shared memory transport should be compatible with.*

### 7.141.1 Detailed Description

<<eXtension>> Properties for the SHMEM Transport.

### 7.141.2 Member Data Documentation

#### received\_message\_count\_max

```
RTI_INT32 NETIO_SHMEMInterfaceFactoryProperty::received_message_count_max
```

Max number of received message sizes that can be residing inside the shared memory transport concurrent queue.



**receive\_buffer\_size**

```
RTI_INT32 NETIO_SHMEMInterfaceFactoryProperty::receive_buffer_size
```

The size of the receive socket buffer.

**message\_size\_max**

```
RTI_INT32 NETIO_SHMEMInterfaceFactoryProperty::message_size_max
```

The maximum size of the message which can be received.

**recv\_thread\_property**

```
struct OSAPI_ThreadProperty NETIO_SHMEMInterfaceFactoryProperty::recv_thread_property
```

Thread properties for each receive thread created by this NETIO interface.

**pro\_minimum\_compatibility\_version**

```
struct DDS_ProductVersion NETIO_SHMEMInterfaceFactoryProperty::pro_minimum_compatibility_version
```

The minimum Connex Pro version that the shared memory transport should be compatible with.

## 7.142 OSAPI\_LogProperty Struct Reference

Configuration of logging functionality.

```
#include <osapi_log.h>
```

**Public Attributes**

- [RTI\\_SIZE\\_T max\\_buffer\\_size](#)  
*The maximum number of bytes allocated to the log buffer.*
- [RTI\\_UINT32 log\\_detail](#)  
*Bitmap to control the fidelity of what is stored in the log entry.*
- [OSAPI\\_Log\\_write\\_buffer\\_T write\\_buffer](#)  
*Function pointer to output a log/trace buffer.*

### 7.142.1 Detailed Description

Configuration of logging functionality.

### 7.142.2 Member Data Documentation

#### max\_buffer\_size

`RTI_SIZE_T OSAPI_LogProperty::max_buffer_size`

The maximum number of bytes allocated to the log buffer.

Log entries are of variable length. When this limit has been reached, no more log entries can be stored unless the buffer is cleared.

#### log\_detail

`RTI_UINT32 OSAPI_LogProperty::log_detail`

Bitmap to control the fidelity of what is stored in the log entry.

#### write\_buffer

`OSAPI_Log_write_buffer_T OSAPI_LogProperty::write_buffer`

Function pointer to output a log/trace buffer.

## 7.143 OSAPI\_SharedMemorySegmentHandle Struct Reference

```
#include <netio_zcopy_shm_segment.h>
```

### Public Attributes

- struct `OSAPI_SharedMemorySegmentHeader` \* `ptr_header`
- void \* `ptr_user_data`
- `OSAPI_SHMEM_MODE` mode

### 7.143.1 Detailed Description

Opaque handle initialized by the create and attach methods and used to interact with a shared memory segment.

### 7.143.2 Member Data Documentation

#### ptr\_header

`struct OSAPI_SharedMemorySegmentHeader* OSAPI_SharedMemorySegmentHandle::ptr_header`

Pointer to segment header in shared memory

**ptr\_user\_data**

```
void* OSAPI_SharedMemorySegmentHandle::ptr_user_data
```

Pointer to where user data can be stored in shared memory

**mode**

```
OSAPI_SHMEM_MODE OSAPI_SharedMemorySegmentHandle::mode
```

Mode requested by the caller of create or attach

## 7.144 OSAPI\_SharedMemorySegmentHeader Struct Reference

```
#include <netio_zcopy_shm_segment.h>
```

**Public Attributes**

- [RTI\\_UINT64](#) size
- [OSAPI\\_SHMEM\\_MODE](#) mode

### 7.144.1 Detailed Description

Opaque header at beginning of shared memory segment to share information about the segment between processes.

### 7.144.2 Member Data Documentation

**size**

```
RTI_UINT64 OSAPI_SharedMemorySegmentHeader::size
```

Size requested by the caller of create

**mode**

```
OSAPI_SHMEM_MODE OSAPI_SharedMemorySegmentHeader::mode
```

Mode requested by the caller of create

## 7.145 OSAPI\_SystemI Struct Reference

System API.

```
#include <osapi_system.h>
```

### Public Attributes

- [OSAPI\\_System\\_start\\_timer\\_T start\\_timer](#)  
*Implementation of function to start a timer.*
- [OSAPI\\_System\\_get\\_timer\\_resolution\\_T get\\_timer\\_resolution](#)  
*Implementation of function to get timer resolution.*
- [OSAPI\\_System\\_get\\_time\\_T get\\_time](#)  
*Implementation of function to get the current time.*
- [OSAPI\\_System\\_generate\\_uuid\\_T generate\\_uuid](#)  
*Implementation of function to generate a UUID.*
- [OSAPI\\_System\\_get\\_hostname\\_T get\\_hostname](#)  
*Implementation of function to get the hostname.*
- [OSAPI\\_System\\_get\\_ticktime\\_T get\\_ticktime](#)  
*Implementation of function to get the tick time.*

### 7.145.1 Detailed Description

System API.

The methods in this interface are called by the corresponding OSAPI\_System function. For example, OSAPI\_System↔\_get\_time() calls system->[get\\_time\(\)](#).

### 7.145.2 Member Data Documentation

#### start\_timer

```
OSAPI_System_start_timer_T OSAPI_SystemI::start_timer
```

Implementation of function to start a timer.

#### get\_timer\_resolution

```
OSAPI_System_get_timer_resolution_T OSAPI_SystemI::get_timer_resolution
```

Implementation of function to get timer resolution.

**get\_time**

```
OSAPI_System_get_time_T OSAPI_SystemI::get_time
```

Implementation of function to get the current time.

**generate\_uuid**

```
OSAPI_System_generate_uuid_T OSAPI_SystemI::generate_uuid
```

Implementation of function to generate a UUID.

**get\_hostname**

```
OSAPI_System_get_hostname_T OSAPI_SystemI::get_hostname
```

Implementation of function to get the hostname.

**get\_ticktime**

```
OSAPI_System_get_ticktime_T OSAPI_SystemI::get_ticktime
```

Implementation of function to get the tick time.

## 7.146 OSAPI\_SystemProperty Struct Reference

The System Properties.

```
#include <osapi_system.h>
```

**Public Attributes**

- struct [OSAPI\\_TimerProperty timer\\_property](#)  
*Timer properties.*
- char [hostname](#) [[OSAPI\\_SYSTEM\\_MAX\\_HOSTNAME](#)]  
*The hostname for the node.*
- [RTI\\_INT32 max\\_user\\_blocking\\_threads](#)  
*The maximum number of threads which can block at the same time.*
- [RTI\\_INT32 max\\_timers](#)  
*The maximum number timers the OS platform support library is required to support.*

### 7.146.1 Detailed Description

The System Properties.

System properties

### 7.146.2 Member Data Documentation

#### timer\_property

```
struct OSAPI_TimerProperty OSAPI_SystemProperty::timer_property
```

Timer properties.

The timer properties control the time thread running the internal RTI Connex DDS Micro timer tick. Note that each port of RTI Connex DDS Micro may choose to implement the timer tick differently. Please refer to platform specific information for details.

#### hostname

```
char OSAPI_SystemProperty::hostname[OSAPI_SYSTEM_MAX_HOSTNAME]
```

The hostname for the node.

RTI Connex DDS Micro uses this name when it announces itself on the network. This name is used by other RTI products. The hostname must not exceed OSAPI\_SYSTEM\_MAX\_HOSTNAME, including the \0 character.

#### max\_user\_blocking\_threads

```
RTI_INT32 OSAPI_SystemProperty::max_user_blocking_threads
```

The maximum number of threads which can block at the same time.

RTI Connex DDS Micro supports blocking operations, such as [DDS\\_KEEP\\_ALL\\_HISTORY\\_QOS](#), and may block the user thread in certain conditions, e.g. if out of resources. The maximum number of threads that RTI Connex DDS Micro can block at the same time is controlled by this property.

If a blocking operation is unable to acquire a blocking resource, the operation will return with [DDS\\_RETCODE\\_OUT\\_OF\\_RESOURCES](#).

Note that receive threads created by a transport and threads calling DDS APIs are both considered user threads.

**[default]** 32

**[range]** [1, INT\_MAX]

## max\_timers

`RTI_INT32 OSAPI_SystemProperty::max_timers`

The maximum number timers the OS platform support library is required to support.

RTI Connex DDS Micro implements its own software timers to keep track of Qos policies such as [DDS\\_DeadlineQosPolicy](#). These software timers are driven by the underlying platform RTI Connex DDS Micro is executing on. This property configures the maximum number of timers that the OS is required to support.

RTI Connex DDS Micro allocated uses these timers as follows:

- One timer is allocated per [DDSDomainParticipant](#)
- One timer is allocated per [DDSDomainParticipant](#) for each [Zero Copy v2 API](#) transport enabled on a [DDSDomainParticipant](#).

The default value supports 8 DomainParticipants without [Zero Copy v2 API](#) enabled and 4 DomainParticipants with `netio_zcopy` enabled.

**[default]** 8

**[range]** [1, INT\_MAX]

## 7.147 OSAPI\_SystemTime Struct Reference

System time.

```
#include <osapi_time.h>
```

### Public Attributes

- [RTI\\_INT64 sec](#)  
*Seconds.*
- [RTI\\_UINT32 nanosec](#)  
*Nanoseconds.*

### 7.147.1 Detailed Description

System time.

Expresses time in seconds and nanoseconds. Assumes that Nanoseconds is less than 1000000000.

### 7.147.2 Member Data Documentation

#### sec

[RTI\\_INT64](#) OSAPI\_SystemTime::sec

Seconds.

#### nanosec

[RTI\\_UINT32](#) OSAPI\_SystemTime::nanosec

Nanoseconds.

## 7.148 OSAPI\_ThreadProperty Struct Reference

Properties for thread creation.

```
#include <osapi_thread.h>
```

### Public Attributes

- [RTI\\_UINT32](#) `stack_size`

*Hint on required stack-size in bytes. The special value `OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE` means do not set the stack-size and rely on the OS defaults.*

- [RTI\\_INT32](#) `priority`

*Hint on required priority. A value  $\geq 0$  is used as is, and negative value is mapped according to the platform documentation.*

- [OSAPI\\_ThreadOptions](#) `options`

*Hint for thread creation to the OS.*

### 7.148.1 Detailed Description

Properties for thread creation.

### 7.148.2 Member Data Documentation

#### stack\_size

[RTI\\_UINT32](#) OSAPI\_ThreadProperty::stack\_size

Hint on required stack-size in bytes. The special value `OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE` means do not set the stack-size and rely on the OS defaults.



**priority**

```
RTI_INT32 OSAPI_ThreadProperty::priority
```

Hint on required priority. A value  $\geq 0$  is used as is, and negative value is mapped according to the platform documentation.

**options**

```
OSAPI_ThreadOptions OSAPI_ThreadProperty::options
```

Hint for thread creation to the OS.

**7.149 OSAPI\_TimerProperty Struct Reference**

Timer properies.

```
#include <osapi_timer.h>
```

**Public Attributes**

- struct [OSAPI\\_ThreadProperty thread](#)  
*The thread settings for the timer thread, if a thread is created.*

**7.149.1 Detailed Description**

Timer properies.

**7.149.2 Member Data Documentation****thread**

```
struct OSAPI\_ThreadProperty OSAPI_TimerProperty::thread
```

The thread settings for the timer thread, if a thread is created.

**7.150 PSK\_ErrListener Struct Reference**

A structure for holding callback pointers in case of an error.

```
#include <psk_ssl_transform.h>
```

**Public Attributes**

- `int(* callback )(const char *msg, size_t msg_len, void *user\_ptr)`  
*Prototype for callback to be invoked in case of error.*
- `void * user\_ptr`  
*Pointer to be used in error callback.*

**7.150.1 Detailed Description**

A structure for holding callback pointers in case of an error.

**7.150.2 Member Data Documentation****callback**

```
int(* PSK_ErrListener::callback) (const char *msg, size_t msg_len, void *user\_ptr)
```

Prototype for callback to be invoked in case of error.

This method is invoked for any error that has been recorded in the PSK Transformation PSL. If its return value is not 0, the queue of errors continues to be processed (with the associated callback invocations). If its return value is 0, the walking over errors is cancelled.

**user\_ptr**

```
void* PSK_ErrListener::user_ptr
```

Pointer to be used in error callback.

**7.151 RHSMHistoryFactory Class Reference**

<<[eXtension](#)>> A reader history factory that will be used by the DataReader to create a history cache.

```
#include <dds_cpp_rh_sm.hxx>
```

**Static Public Member Functions**

- `static struct RT_ComponentFactoryI * get\_interface ()`  
*Gets the singleton instance of the RHSM reader history factory.*

**7.151.1 Detailed Description**

<<[eXtension](#)>> A reader history factory that will be used by the DataReader to create a history cache.

### 7.151.2 Member Function Documentation

#### get\_interface()

```
static struct RT_ComponentFactoryI * RHSMHistoryFactory::get_interface () [static]
```

Gets the singleton instance of the RHSM reader history factory.

This function gets the singleton instance of the RHSM history plugin factory that is used by the middleware to create a ReaderHistory plugin. The singleton instance of the factory must be registered with the name "rh" in the Domain↔ ParticipantFactory.

#### Returns

Pointer to RHSM history plugin factory

### 7.152 RTPS\_Checksum Union Reference

<<eXtension>> RTPS checksum type

```
#include <rtps_checksum.h>
```

#### Public Attributes

- [RTI\\_UINT32 checksum32](#)  
*A 32-bit unsigned checksum in host endianness order.*
- [RTI\\_UINT64 checksum64](#)  
*A 64-bit unsigned integer checksum in host endianness order.*
- [RTI\\_UINT8 checksum128](#) [16]  
*A 128-bit checksum as a 16-byte array.*

#### 7.152.1 Detailed Description

<<eXtension>> RTPS checksum type

### 7.153 RTPS\_ChecksumClass Struct Reference

<<eXtension>> Definition of a checksum function

```
#include <rtps_checksum.h>
```

**Public Attributes**

- [RTPS\\_ChecksumClassId\\_T class\\_id](#)  
*The checksum class ID.*
- void \* [context](#)  
*User defined context.*
- [RTPS\\_ChecksumCalculate\\_T checksum\\_calculate](#)  
*User defined function to calculate a checksum.*

**7.153.1 Detailed Description**

<<[eXtension](#)>> Definition of a checksum function

**7.154 RTPS\_ChecksumProperty Struct Reference**

<<[eXtension](#)>> Checksum properties for the RTPS plugin

```
#include <rtps_checksum.h>
```

**Public Attributes**

- [RTPS\\_ChecksumClass\\_T builtin\\_checksum32\\_class](#)  
*The builtin 32-bit checksum class.*
- [RTPS\\_ChecksumClass\\_T builtin\\_checksum64\\_class](#)  
*The builtin 64-bit checksum class.*
- [RTPS\\_ChecksumClass\\_T builtin\\_checksum128\\_class](#)  
*The builtin 128-bit checksum class.*
- [RTPS\\_ChecksumTxMode\\_T checksum\\_tx\\_mode](#)  
*The transmit mode for checksums.*
- [RTI\\_BOOL allow\\_builtin\\_override](#)  
*If TRUE, allow overriding the builtin checksum functions.*

**7.154.1 Detailed Description**

<<[eXtension](#)>> Checksum properties for the RTPS plugin

**7.155 RTPS\_InterfaceFactoryProperty Struct Reference**

<<[eXtension](#)>> Properties for the RTPS Transport.

```
#include <rtps_rtps.h>
```

## Public Attributes

- struct [RTPS\\_ChecksumProperty checksum](#)  
*The checksum properties for RTPS.*

### 7.155.1 Detailed Description

<<[eXtension](#)>> Properties for the RTPS Transport.

## 7.156 RTRegistry Class Reference

<<[eXtension](#)>> A run-time component factory.

```
#include <dds_cpp_rt.hxx>
```

## Public Member Functions

- bool [register\\_component](#) (const char \*name, RT\_ComponentFactoryI \*intf, RT\_ComponentFactoryProperty \*property, RT\_ComponentFactoryListener \*listener)  
*Register a factory of RT components*
- bool [unregister](#) (const char \*name, RT\_ComponentFactoryProperty \*\*property, RT\_ComponentFactoryListener \*\*listener)  
*Remove a component factory from the registry*

### 7.156.1 Detailed Description

<<[eXtension](#)>> A run-time component factory.

### 7.156.2 Member Function Documentation

#### **register\_component()**

```
bool RTRegistry::register_component (
    const char * name,
    RT_ComponentFactoryI * intf,
    RT_ComponentFactoryProperty * property,
    RT_ComponentFactoryListener * listener)
```

Register a factory of RT components

In order to make a component available, it is necessary to register a factory for the components. The factory is responsible for creating and deleting components.

The name of the factory must be unique. Otherwise, if a factory already exists with the the same name, the registration will fail.

## Parameters

|                 |                                                                        |
|-----------------|------------------------------------------------------------------------|
| <i>name</i>     | << <i>in</i> >> Name of the component factory                          |
| <i>intf</i>     | << <i>in</i> >> Pointer to the implementation of the factory interface |
| <i>property</i> | << <i>in</i> >> Properties with which to create the factory            |
| <i>listener</i> | << <i>in</i> >> The listener to install with the factory               |

## Returns

RTI\_TRUE on success, RTI\_FALSE on failure

## See also

[RTRegistry::unregister](#)

**unregister()**

```
bool RTRegistry::unregister (
    const char * name,
    RT_ComponentFactoryProperty ** property,
    RT_ComponentFactoryListener ** listener)
```

Remove a component factory from the registry

After a component factory is unregistered, it will no longer be possible to create or delete components with the factory.

## Parameters

|                 |                                                               |
|-----------------|---------------------------------------------------------------|
| <i>name</i>     | << <i>in</i> >> Name of the component factory                 |
| <i>property</i> | << <i>in</i> >> Properties with which the factory was created |
| <i>listener</i> | << <i>in</i> >> The listener installed with the factory       |

## Returns

RTI\_TRUE on success, RTI\_FALSE on failure

## See also

[RTRegistry::register\\_component](#)

**7.157 SHMEMInterfaceFactory Class Reference**

<<*eXtension*>> A SHMEM transport factory that will be used by a DomainParticipant to create a SHMEM interface. Only one SHMEM interface can be registered with the DomainParticipantFactory.

```
#include <dds_cpp_netio_shmem.hxx>
```

## Static Public Member Functions

- static struct RT\_ComponentFactoryI \* [get\\_interface](#) ()  
*Gets the singleton instance of the SHMEM interface factory.*

### 7.157.1 Detailed Description

<<[eXtension](#)>> A SHMEM transport factory that will be used by a DomainParticipant to create a SHMEM interface. Only one SHMEM interface can be registered with the DomainParticipantFactory.

### 7.157.2 Member Function Documentation

#### **get\_interface()**

```
static struct RT_ComponentFactoryI * SHMEMInterfaceFactory::get_interface () [static]
```

Gets the singleton instance of the SHMEM interface factory.

This function gets the singleton instance of the SHMEM factory that is used by the middleware to create a SHMEM transport instance.

#### Returns

Pointer to SHMEM factory instance

## 7.158 UDP\_InterfaceFactoryProperty Struct Reference

<<[eXtension](#)>> Properties for the UDP Transport.

```
#include <netio_udp.h>
```

### Public Attributes

- struct REDA\_StringSeq [allow\\_interface](#)  
*Sequence of allowed interface names.*
- struct REDA\_StringSeq [deny\\_interface](#)  
*Sequence of denied interface names. This list is checked after the allow\_interface list.*
- [RTI\\_INT32 max\\_send\\_buffer\\_size](#)  
*The size of the send socket buffer.*
- [RTI\\_INT32 max\\_receive\\_buffer\\_size](#)  
*The size of the receive socket buffer.*
- [RTI\\_INT32 max\\_message\\_size](#)  
*The maximum size of the message which can be sent or received.*
- [RTI\\_INT32 max\\_send\\_message\\_size](#)  
*The maximum size of the message which can be sent. This is only a hint and not enforced.*

- [RTI\\_INT32 multicast\\_ttl](#)  
*The maximum TTL.*
- struct [UDP\\_NatEntrySeq nat](#)  
*Configure network address translation (NAT).*
- struct [UDP\\_InterfaceTableEntrySeq if\\_table](#)  
*The interface table if interfaces are added manually.*
- [REDA\\_String\\_T multicast\\_interface](#)  
*The network interface to use to send to multicast.*
- [RTI\\_BOOL is\\_default\\_interface](#)  
*If this should be considered the default UDP interface if no other UDP interface is found to handle a route.*
- [RTI\\_BOOL disable\\_auto\\_interface\\_config](#)  
*Disable reading of available network interfaces using system information and instead rely on the manually configured interface table.*
- [RTI\\_BOOL multicast\\_loopback\\_disabled](#)  
*Prevents the transport plugin from putting multicast packets onto the loopback interface.*
- struct [OSAPI\\_ThreadProperty recv\\_thread](#)  
*Thread properties for each receive thread created by this NETIO interface.*
- [RTI\\_BOOL enable\\_interface\\_bind](#)  
*Bind receive sockets to specific interfaces.*
- [RTI\\_BOOL disable\\_multicast\\_bind](#)  
*Disable a multicast receive socket binding to a multicast address.*
- [RTI\\_BOOL disable\\_multicast\\_interface\\_select](#)  
*Do not select outgoing interfaces when sending to multicast groups.*
- struct [UDP\\_TransformRuleSeq source\\_rules](#)  
*Rules for how to transform received UDP payloads based on the source address.*
- struct [UDP\\_TransformRuleSeq destination\\_rules](#)  
*Rules for how to transform sent UDP payloads based on the destination address.*
- [UDP\\_TransformUdpMode\\_T transform\\_udp\\_mode](#)  
*Determines how regular UDP is supported when transformations are supported.*
- [RTI\\_INT32 transform\\_locator\\_kind](#)  
*The locator to use for locators that have transformations.*

### 7.158.1 Detailed Description

<<*eXtension*>> Properties for the UDP Transport.

### 7.158.2 Member Data Documentation

#### **allow\_interface**

```
struct REDA_StringSeq UDP_InterfaceFactoryProperty::allow_interface
```

Sequence of allowed interface names.



**deny\_interface**

```
struct REDA_StringSeq UDP_InterfaceFactoryProperty::deny_interface
```

Sequence of denied interface names. This list is checked after the `allow_interface` list.

**max\_send\_buffer\_size**

```
RTI_INT32 UDP_InterfaceFactoryProperty::max_send_buffer_size
```

The size of the send socket buffer.

**max\_receive\_buffer\_size**

```
RTI_INT32 UDP_InterfaceFactoryProperty::max_receive_buffer_size
```

The size of the receive socket buffer.

**max\_message\_size**

```
RTI_INT32 UDP_InterfaceFactoryProperty::max_message_size
```

The maximum size of the message which can be sent or received.

**max\_send\_message\_size**

```
RTI_INT32 UDP_InterfaceFactoryProperty::max_send_message_size
```

The maximum size of the message which can be sent. This is only a hint and not enforced.

This parameters is useful to UDP transformation functions which must allocate a buffer to put the result in.

**multicast\_ttl**

```
RTI_INT32 UDP_InterfaceFactoryProperty::multicast_ttl
```

The maximum TTL.

**nat**

```
struct UDP_NatEntrySeq UDP_InterfaceFactoryProperty::nat
```

Configure network address translation (NAT).

Configure network address translation (NAT). This feature only supports translation between a private and public IP address; UDP ports are not translated. Does not support any hole punching technique or WAN server, thus is only useful when the private and public address mapping is static. Please refer to [UDP\\_NatEntrySeq](#) for the definition of the sequence element.

**if\_table**

```
struct UDP_InterfaceTableEntrySeq UDP_InterfaceFactoryProperty::if_table
```

The interface table if interfaces are added manually.

**multicast\_interface**

```
REDA_String_T UDP_InterfaceFactoryProperty::multicast_interface
```

The network interface to use to send to multicast.

**is\_default\_interface**

```
RTI_BOOL UDP_InterfaceFactoryProperty::is_default_interface
```

If this should be considered the default UDP interface if no other UDP interface is found to handle a route.

**disable\_auto\_interface\_config**

```
RTI_BOOL UDP_InterfaceFactoryProperty::disable_auto_interface_config
```

Disable reading of available network interfaces using system information and instead rely on the manually configured interface table.

**multicast\_loopback\_disabled**

```
RTI_BOOL UDP_InterfaceFactoryProperty::multicast_loopback_disabled
```

Prevents the transport plugin from putting multicast packets onto the loopback interface.

If multicast loopback is disabled (this value is set to 1), then when sending multicast packets, RTI Connex DDS Micro will not put a copy of the packets on the loopback interface. This prevents applications on the same node (including itself) from receiving those packets.

This value is set to 0 by default, meaning multicast loopback is enabled.

Disabling multicast loopback (setting this value to 1) may result in minor performance gains when using multicast.

**recv\_thread**

```
struct OSAPI_ThreadProperty UDP_InterfaceFactoryProperty::recv_thread
```

Thread properties for each receive thread created by this NETIO interface.

### **enable\_interface\_bind**

```
RTI_BOOL UDP_InterfaceFactoryProperty::enable_interface_bind
```

Bind receive sockets to specific interfaces.

When this is set to 1, the UDP transport binds each receive port to a specific interface when the `allow_interface/deny_interface` lists are non-empty. This allows multiple UDP transports to be used by a single DomainParticipant at the expense of an increased number of threads. This property is ignored when transformations are enabled and the `allow_interface/deny_interface` lists are non-empty.

The default value is 0.

### **disable\_multicast\_bind**

```
RTI_BOOL UDP_InterfaceFactoryProperty::disable_multicast_bind
```

Disable a multicast receive socket binding to a multicast address.

NOTE: This option has no effect on Windows. On Windows a multicast socket is only bound to the UDP port.

The default value is 0, meaning that a multicast socket is bound to both the UDP port and multicast address.

When this is set to 1, the UDP transport binds a multicast receive socket only to the UDP port.

### **disable\_multicast\_interface\_select**

```
RTI_BOOL UDP_InterfaceFactoryProperty::disable_multicast_interface_select
```

Do not select outgoing interfaces when sending to multicast groups.

The default value is 0 and RTI Connex DDS Micro will select the outgoing interfaces when sending to multicast either using the `allow_interface` and `deny_interface` properties, or the `multicast_interface` property.

When this is set to 1, RTI Connex DDS Micro will not select the outgoing interface when sending to multicast groups. This may be useful if the OS has fine grained control to select which interface should be used to send to multicast.

### **source\_rules**

```
struct UDP_TransformRuleSeq UDP_InterfaceFactoryProperty::source_rules
```

Rules for how to transform received UDP payloads based on the source address.

### **destination\_rules**

```
struct UDP_TransformRuleSeq UDP_InterfaceFactoryProperty::destination_rules
```

Rules for how to transform sent UDP payloads based on the destination address.

**transform\_udp\_mode**

```
UDP_TransformUdpMode_T UDP_InterfaceFactoryProperty::transform_udp_mode
```

Determines how regular UDP is supported when transformations are supported.

**transform\_locator\_kind**

```
RTI_INT32 UDP_InterfaceFactoryProperty::transform_locator_kind
```

The locator to use for locators that have transformations.

When transformation rules have been enabled, they are announced as a vendor-specific locator. This property overrides this value. NOTE: Changing this value may prevent communication. NOTE: Must be greater or equal than NETIO\_↔ ADDRESS\_KIND\_USER and not equal to NETIO\_ADDRESS\_KIND\_SHMEM or NETIO\_ADDRESS\_KIND\_INTRA.

**7.159 UDP\_InterfaceTableEntry Struct Reference**

Generic structure to describe a network interface.

```
#include <netio_udp.h>
```

**Public Attributes**

- [RTI\\_UINT32 flags](#)  
*Flags to indicate if the interface is up etc.*
- [RTI\\_UINT32 address](#)  
*The address of the interface as configured by the OS.*
- [RTI\\_UINT32 netmask](#)  
*The netmask of the interface as configured by the OS.*
- char [ifname](#) [UDP\_INTERFACE\_MAX\_IFNAME]  
*The name of the interface as configured by the OS.*

**7.159.1 Detailed Description**

Generic structure to describe a network interface.

**7.159.2 Member Data Documentation****flags**

```
RTI_UINT32 UDP_InterfaceTableEntry::flags
```

Flags to indicate if the interface is up etc.

**address**

```
RTI_UINT32 UDP_InterfaceTableEntry::address
```

The address of the interface as configured by the OS.

**netmask**

```
RTI_UINT32 UDP_InterfaceTableEntry::netmask
```

The netmask of the interface as configured by the OS.

**ifname**

```
char UDP_InterfaceTableEntry::ifname[UDP_INTERFACE_MAX_IFNAME]
```

The name of the interface as configured by the OS.

## 7.160 UDP\_InterfaceTableEntrySeq Struct Reference

<<*eXtension*>> Declares IDL sequence<UDP\_InterfaceTableEntry>

```
#include <netio_udp.h>
```

### 7.160.1 Detailed Description

<<*eXtension*>> Declares IDL sequence<UDP\_InterfaceTableEntry>

## 7.161 UDP\_NatEntry Struct Reference

<<*eXtension*>> Properties for a Network Address Translation entry

```
#include <netio_udp.h>
```

### 7.161.1 Detailed Description

<<*eXtension*>> Properties for a Network Address Translation entry

Adding [UDP\\_NatEntry](#) entries to the [UDP\\_InterfaceFactoryProperty::nat](#) sequence enables translation between local and public network addresses. Please refer to [Sequence Support](#) for more information on how to manages sequences in RTI Connex DDS Micro.

## 7.162 UDP\_NatEntrySeq Struct Reference

<<[eXtension](#)>> Declares IDL sequence<[UDP\\_NatEntry](#)>

```
#include <netio_udp.h>
```

### 7.162.1 Detailed Description

<<[eXtension](#)>> Declares IDL sequence<[UDP\\_NatEntry](#)>

## 7.163 UDPInterfaceFactory Class Reference

<<[eXtension](#)>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory.

```
#include <netio_udp_cpp.hxx>
```

### Static Public Member Functions

- static struct RT\_ComponentFactoryI \* [get\\_interface](#) ()  
*Gets the singleton instance of the UDP interface factory.*

### 7.163.1 Detailed Description

<<[eXtension](#)>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory.

### 7.163.2 Member Function Documentation

#### **get\_interface()**

```
static struct RT_ComponentFactoryI * UDPInterfaceFactory::get_interface () [static]
```

Gets the singleton instance of the UDP interface factory.

This function gets the singleton instance of the UDP factory that is used by the middleware to create a UDP transport instance. [UDP\\_InterfaceFactoryProperty](#)

#### Returns

Pointer to UDP factory instance()

## 7.164 UDPInterfaceTable Class Reference

```
#include <netio_udp_cpp.hxx>
```

### Static Public Member Functions

- static bool [add\\_entry](#) (struct [UDP\\_InterfaceTableEntrySeq](#) \*seq, [RTI\\_UINT32](#) address, [RTI\\_UINT32](#) netmask, const char \*ifname, [RTI\\_UINT32](#) flags)  
*Add an entry to the list of available interfaces.*

#### 7.164.1 Detailed Description

## 7.165 WHSMHistoryFactory Class Reference

<<[eXtension](#)>> A writer history factory that will be used by the DataWriter to create a history cache.

```
#include <dds_cpp_wh_sm.hxx>
```

### Static Public Member Functions

- static struct [RT\\_ComponentFactoryI](#) \* [get\\_interface](#) ()  
*Gets the singleton instance of the WHSM writer history factory.*

#### 7.165.1 Detailed Description

<<[eXtension](#)>> A writer history factory that will be used by the DataWriter to create a history cache.

#### 7.165.2 Member Function Documentation

##### **get\_interface()**

```
static struct RT\_ComponentFactoryI * WHSMHistoryFactory::get_interface () [static]
```

Gets the singleton instance of the WHSM writer history factory.

This function gets the singleton instance of the WHSM history plugin factory that is used by the middleware to create a WriterHistory plugin. The singleton instance of the factory must be registered with the name "wh" in the DomainParticipantFactory.

##### Returns

Pointer to WHSM history plugin factory

## 7.166 ZCOPY\_NotifInterfaceFactoryProperty Struct Reference

Notification interface property.

```
#include <netio_zcopy_notif_interface.h>
```

### Public Attributes

- [RTI\\_UINT32 max\\_samples\\_per\\_notif](#)  
*Maximum samples to process per notification per DataReader.*
- struct [ZCOPY\\_NotifUserInterfaceI](#) \* [user\\_intf](#)  
*The downstream transport interface.*
- void \* [user\\_property](#)  
*Pass-through pointer to the downstream transport. This must be valid for the life-cycle of the transport.*

### 7.166.1 Detailed Description

Notification interface property.

### 7.166.2 Member Data Documentation

#### max\_samples\_per\_notif

```
RTI_UINT32 ZCOPY_NotifInterfaceFactoryProperty::max_samples_per_notif
```

Maximum samples to process per notification per DataReader.

This setting optimizes performance when sending large amounts of data. However, setting it to an excessively high value may prevent other threads from running effectively.

**[default]** 1.

#### user\_intf

```
struct ZCOPY_NotifUserInterfaceI* ZCOPY_NotifInterfaceFactoryProperty::user_intf
```

The downstream transport interface.

**[default]** NULL.

#### user\_property

```
void* ZCOPY_NotifInterfaceFactoryProperty::user_property
```

Pass-through pointer to the downstream transport. This must be valid for the life-cycle of the transport.

**[default]** NULL.



## 7.167 ZCOPY\_NotifMechanismProperty Struct Reference

Notification mechanism property for the default implementation.

```
#include <netio_zcopy_default_notif_mech.h>
```

### Public Attributes

- [RTI\\_UINT32 intf\\_addr](#)  
*The interface address for this implementation. The value should be globally unique.*
- struct [OSAPI\\_ThreadProperty thread\\_prop](#)  
*The thread property for the receive thread in the notification mechanism.*
- [RTI\\_UINT32 max\\_receive\\_ports](#)  
*The maximum number of receive ports.*
- [RTI\\_UINT32 max\\_routes](#)  
*The maximum number of outgoing routes this notification mechanism can handle.*

### 7.167.1 Detailed Description

Notification mechanism property for the default implementation.

### 7.167.2 Member Data Documentation

#### **intf\_addr**

```
RTI\_UINT32 ZCOPY_NotifMechanismProperty::intf_addr
```

The interface address for this implementation. The value should be globally unique.

**[default]** 0.

#### **thread\_prop**

```
struct OSAPI\_ThreadProperty ZCOPY_NotifMechanismProperty::thread_prop
```

The thread property for the receive thread in the notification mechanism.

**[default]** [OSAPI\\_THREAD\\_PROPERTY\\_DEFAULT](#).

#### **max\_receive\_ports**

```
RTI\_UINT32 ZCOPY_NotifMechanismProperty::max_receive_ports
```

The maximum number of receive ports.

**[default]** 2.

**max\_routes**

`RTI_UINT32 ZCOPY_NotifMechanismProperty::max_routes`

The maximum number of outgoing routes this notification mechanism can handle.

[default] 32.

**7.168 ZCOPY\_NotifUserInterface Struct Reference**

Notification mechanism user interface.

```
#include <netio_zcopy_notif_mechanism_intf.h>
```

**Public Attributes**

- [ZCOPY\\_NotifUserInterface\\_createFunc create\\_instance](#)  
*Create an instance of the notification mechanism interface with upstream as the owner.*
- [ZCOPY\\_NotifUserInterface\\_deleteFunc delete\\_instance](#)  
*Delete an instance of the notification mechanism interface.*
- [NETIO\\_Interface\\_resolve\\_addressFunc resolve\\_address](#)  
*Resolve an address string from the notification mechanism interface.*
- [NETIO\\_Interface\\_get\\_route\\_tableFunc get\\_route\\_table](#)  
*Get the notification mechanism interface route table.*
- [ZCOPY\\_NotifUserInterface\\_reserve\\_addressFunc reserve\\_address](#)  
*Reserve an address with the notification mechanism interface.*
- [ZCOPY\\_NotifUserInterface\\_release\\_addressFunc release\\_address](#)  
*Release addresses to the notification mechanism interface.*
- [ZCOPY\\_NotifUserInterface\\_add\\_routeFunc add\\_route](#)  
*Add a route on the notification mechanism interface.*
- [ZCOPY\\_NotifUserInterface\\_delete\\_routeFunc delete\\_route](#)  
*Delete a route on the notification mechanism interface.*
- [ZCOPY\\_NotifUserInterface\\_bindFunc bind](#)  
*Bind on the notification mechanism interface.*
- [ZCOPY\\_NotifUserInterface\\_unbindFunc unbind](#)  
*Unbind on the notification mechanism interface.*
- [ZCOPY\\_NotifUserInterface\\_sendFunc send](#)  
*Send a notification using the notification mechanism interface.*
- [ZCOPY\\_NotifUserInterface\\_notify\\_portFunc notify\\_rcv\\_port](#)  
*Notify the receive port.*

**7.168.1 Detailed Description**

Notification mechanism user interface.

**7.168.2 Member Data Documentation****create\_instance**

`ZCOPY_NotifUserInterface_createFunc ZCOPY_NotifUserInterfaceI::create_instance`

Create an instance of the notification mechanism interface with upstream as the owner.

**Parameters**

|                          |                                      |
|--------------------------|--------------------------------------|
| <i>upstream[in]</i>      | The upstream notification interface. |
| <i>user_property[in]</i> | The user property.                   |

**Returns**

The user interface instance or NULL if an error occurred

**delete\_instance**

`ZCOPY_NotifUserInterface_deleteFunc ZCOPY_NotifUserInterfaceI::delete_instance`

Delete an instance of the notification mechanism interface.

Not available in CERT.

**Parameters**

|                      |                              |
|----------------------|------------------------------|
| <i>user_intf[in]</i> | The user interface instance. |
|----------------------|------------------------------|

**resolve\_address**

`NETIO_Interface_resolve_addressFunc ZCOPY_NotifUserInterfaceI::resolve_address`

Resolve an address string from the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to determine if the address string `address_string` is a valid address and return the result in `address_value`.

**Parameters**

|     |                       |                                                       |
|-----|-----------------------|-------------------------------------------------------|
| in  | <i>netio_intf</i>     | Interface.                                            |
| in  | <i>netio_intf</i>     | The interface entry requested to resolve the address. |
| in  | <i>address_string</i> | The address to convert.                               |
| out | <i>address_value</i>  | The converted address.                                |
| out | <i>invalid</i>        | If the address is valid or not.                       |

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure.

**get\_route\_table**

`NETIO_Interface_get_route_tableFunc ZCOPY_NotifUserInterfaceI::get_route_table`

Get the notification mechanism interface route table.

Instruct the notification mechanism interface `netio_intf` to return a sequence of address and netmask pairs this interface can send to.

**Parameters**

|         |                   |                                                         |
|---------|-------------------|---------------------------------------------------------|
| in      | <i>netio_intf</i> | The Notif interface.                                    |
| in, out | <i>address</i>    | Sequence of NETIO addresses this interface understands. |
| in, out | <i>netmask</i>    | Sequence of the corresponding netmasks.                 |

**Returns**

RTI\_TRUE on success, RTI\_FALSE on failure.;

**reserve\_address**

[ZCOPY\\_NotifUserInterface\\_reserve\\_addressFunc](#) ZCOPY\_NotifUserInterfaceI::reserve\_address

Reserve an address with the notification mechanism interface.

Instruct the notification mechanism interface specified by *user\_intf* to set up resources for listening to messages on the address *src\_addr* and return a *port\_entry\_out*. The *port\_entry\_out* will be provided to a bind call.

**Parameters**

|     |                       |                                             |
|-----|-----------------------|---------------------------------------------|
| in  | <i>user_intf</i>      | The user interface instance.                |
| in  | <i>src_addr</i>       | The address to reserve.                     |
| out | <i>port_entry_out</i> | The port entry associated with the address. |

**release\_address**

[ZCOPY\\_NotifUserInterface\\_release\\_addressFunc](#) ZCOPY\_NotifUserInterfaceI::release\_address

Release addresses to the notification mechanism interface.

Instruct the notification mechanism interface specified by *user\_intf* to release resources for listening to messages on the address *src\_addr*.

**Parameters**

|    |                   |                                             |
|----|-------------------|---------------------------------------------|
| in | <i>user_intf</i>  | The user interface instance.                |
| in | <i>src_addr</i>   | The address to release.                     |
| in | <i>port_entry</i> | The port entry associated with the address. |

**Returns**

RTI\_TRUE on success, RTI\_FALSE otherwise

**add\_route**

[ZCOPY\\_NotifUserInterface\\_add\\_routeFunc](#) ZCOPY\_NotifUserInterfaceI::add\_route

Add a route on the notification mechanism interface.

Instruct the notification mechanism interface specified by *user\_intf* to add a route from the source to the destination. The *route\_entry\_out* will be provided to the user later when calling send.

**Parameters**

|     |                        |                                            |
|-----|------------------------|--------------------------------------------|
| in  | <i>user_intf</i>       | The user interface instance.               |
| in  | <i>source</i>          | The source address.                        |
| in  | <i>destination</i>     | The destination address.                   |
| out | <i>route_entry_out</i> | The route entry associated with the route. |

**Returns**

RTI\_TRUE on success, RTI\_FALSE otherwise

**delete\_route**

`ZCOPY_NotifUserInterface_delete_routeFunc` `ZCOPY_NotifUserInterfaceI::delete_route`

Delete a route on the notification mechanism interface.

Instruct the notification mechanism interface specified by *user\_intf* to remove a route from the source to the destination.

**Parameters**

|    |                    |                                            |
|----|--------------------|--------------------------------------------|
| in | <i>user_intf</i>   | The user interface instance.               |
| in | <i>source</i>      | The source address.                        |
| in | <i>destination</i> | The destination address.                   |
| in | <i>route_entry</i> | The route entry associated with the route. |

**Returns**

RTI\_TRUE on success, RTI\_FALSE otherwise.

**bind**

`ZCOPY_NotifUserInterface_bindFunc` `ZCOPY_NotifUserInterfaceI::bind`

Bind on the notification mechanism interface.

Instruct the notification mechanism interface specified by *user\_intf* to start listening for messages on the address specified by *src\_addr* on the given *port\_entry*.

**Parameters**

|    |                   |                                                   |
|----|-------------------|---------------------------------------------------|
| in | <i>user_intf</i>  | The user interface instance.                      |
| in | <i>src_addr</i>   | The remote address.                               |
| in | <i>dst_addr</i>   | The local address.                                |
| in | <i>port_entry</i> | The port entry associated with the local address. |
| in | <i>bind_entry</i> | The bind entry associated with the bind.          |

**Returns**

RTI\_TRUE on success, RTI\_FALSE otherwise

## unbind

`ZCOPY_NotifUserInterface_unbindFunc` `ZCOPY_NotifUserInterfaceI::unbind`

Unbind on the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to stop listening for messages on the address specified by `src_addr`.

### Parameters

|     |                       |                                                   |
|-----|-----------------------|---------------------------------------------------|
| in  | <i>user_intf</i>      | The user interface instance.                      |
| in  | <i>src_addr</i>       | The remote address.                               |
| in  | <i>dst_addr</i>       | The local address.                                |
| in  | <i>port_entry</i>     | The port entry associated with the local address. |
| out | <i>bind_entry_out</i> | The bind entry associated with the bind.          |

### Returns

RTI\_TRUE on success, RTI\_FALSE otherwise.

## send

`ZCOPY_NotifUserInterface_sendFunc` `ZCOPY_NotifUserInterfaceI::send`

Send a notification using the notification mechanism interface.

Instruct the notification mechanism interface specified by `user_intf` to send a notification to the destination using the `route_entry`.

### Parameters

|    |                    |                                            |
|----|--------------------|--------------------------------------------|
| in | <i>user_intf</i>   | The user interface instance.               |
| in | <i>source</i>      | The source address.                        |
| in | <i>destination</i> | The destination address.                   |
| in | <i>route_entry</i> | The route entry associated with the route. |

### Returns

RTI\_TRUE on success, RTI\_FALSE otherwise.

## notify\_recv\_port

`ZCOPY_NotifUserInterface_notify_portFunc` `ZCOPY_NotifUserInterfaceI::notify_recv_port`

Notify the receive port.

Instruct the notification mechanism interface specified by `user_intf` to notify the receive port specified by `port_entry`.

Used by the reader to send notification on a specific receive port.

## Parameters

|    |                   |                              |
|----|-------------------|------------------------------|
| in | <i>user_intf</i>  | The user interface instance. |
| in | <i>port_entry</i> | The port entry to notify.    |

## Returns

RTI\_TRUE on success, RTI\_FALSE otherwise.

## 8 File Documentation

### 8.1 app\_gen\_log.h File Reference

```
#include "osapi/osapi_log.h"
```

## Macros

- #define APPGEN\_LOG\_ALLOCATE\_EC (APPGEN\_LOG\_BASE + 1)  
*Failed to allocate memory for an Application Generator interface.*
- #define APPGEN\_LOG\_COPY\_ENTITY\_NAME\_EC (APPGEN\_LOG\_BASE + 2)  
*Cannot copy a name into an entity name. Name too long.*
- #define APPGEN\_LOG\_CREATE\_ENTITY\_NAME\_EC (APPGEN\_LOG\_BASE + 3)  
*Cannot generate an entity name. Name too long.*
- #define APPGEN\_LOG\_MULTIPLICITY\_EC (APPGEN\_LOG\_BASE + 4)  
*Cannot create entity name because multiplicity is wrong (0)*
- #define APPGEN\_LOG\_FACTORY\_REGISTER\_EC (APPGEN\_LOG\_BASE + 5)  
*Cannot register a factory.*
- #define APPGEN\_LOG\_FACTORY\_UNREGISTER\_EC (APPGEN\_LOG\_BASE + 5)  
*Cannot unregister a factory.*
- #define APPGEN\_LOG\_FACTORY\_TOPIC\_NOT\_FOUND\_EC (APPGEN\_LOG\_BASE + 6)  
*Topic not found. Wrong topic name in writer configuration.*
- #define APPGEN\_LOG\_DDS\_ENTITY\_EC (APPGEN\_LOG\_BASE + 7)  
*Failed to create/delete/lookup DDS entity.*
- #define APPGEN\_LOG\_API\_ERROR\_EC (APPGEN\_LOG\_BASE + 8)  
*A call to a DDS api returned an error.*
- #define APPGEN\_LOG\_APP\_NAME\_ERROR\_EC (APPGEN\_LOG\_BASE + 9)  
*Wrong application or participant name. Not a qualified name.*
- #define APPGEN\_LOG\_LIB\_NOT\_FOUND\_EC (APPGEN\_LOG\_BASE + 10)  
*Participant cannot be created because the library name is not found in the configuration.*
- #define APPGEN\_LOG\_APP\_PARTICIPANT\_NOT\_FOUND\_EC (APPGEN\_LOG\_BASE + 11)  
*Participant cannot be created because the participant name is not found in the configuration.*
- #define APPGEN\_LOG\_APP\_CONFIG\_NOT\_CORRECT\_EC (APPGEN\_LOG\_BASE + 12)  
*Application generation configuration is not correct or not consistent.*
- #define APPGEN\_LOG\_APP\_CONFIG\_POINTER\_NOT\_CORRECT\_EC (APPGEN\_LOG\_BASE + 13)  
*Application generation configuration is not correct or not consistent.*

## 8.2 cdr\_log.h File Reference

```
#include "osapi/osapi_log.h"
```

### Macros

- #define [CDR\\_LOG\\_STREAM\\_ALLOC\\_EC](#) (CDR\_LOG\_BASE + 1)  
*Failed to allocate stream or stream buffer.*
- #define [CDR\\_LOG\\_SET\\_OFFSET\\_EC](#) (CDR\_LOG\_BASE + 2)  
*Failed to set stream's current offset.*
- #define [CDR\\_LOG\\_INCR\\_OFFSET\\_EC](#) (CDR\_LOG\_BASE + 3)  
*Failed to increment stream's current offset.*
- #define [CDR\\_LOG\\_UNSUPPORTED\\_OPTIONS\\_EC](#) (CDR\_LOG\_BASE + 4)  
*Unsupported CDR encapsulations were received. The sample is dropped.*

## 8.3 db\_log.h File Reference

DB module log codes.

```
#include "osapi/osapi_log.h"
```

### Macros

- #define [DB\\_LOG\\_SORTED\\_ALLOC\\_EC](#) (DB\_LOG\_BASE + 1)  
*Not sufficient memory to allocate sorted list for an index.*
- #define [DB\\_LOG\\_NAME\\_TOO\\_LONG\\_EC](#) (DB\_LOG\_BASE + 2)  
*The specified database name is too long.*
- #define [DB\\_LOG\\_ILLEGAL\\_TABLE\\_SIZE\\_EC](#) (DB\_LOG\_BASE + 3)  
*Illegal table size specified.*
- #define [DB\\_LOG\\_ILLEGAL\\_LOCK\\_MODE\\_EC](#) (DB\_LOG\_BASE + 4)  
*Illegal combination of lock mode and mutex given.*
- #define [DB\\_LOG\\_MUTEX\\_ALLOC\\_EC](#) (DB\_LOG\_BASE + 5)  
*Failed to allocate database mutex.*
- #define [DB\\_LOG\\_ALLOC\\_TABLE\\_POOL\\_EC](#) (DB\_LOG\_BASE + 6)  
*Failed to allocate buffer pool.*
- #define [DB\\_LOG\\_TABLES\\_INUSE\\_EC](#) (DB\_LOG\_BASE + 7)  
*Table is in use.*
- #define [DB\\_LOG\\_TABLE\\_NAME\\_TOO\\_LONG\\_EC](#) (DB\_LOG\_BASE + 8)  
*Specified table name too long.*
- #define [DB\\_LOG\\_ILLEGAL\\_RECORD\\_COUNT\\_EC](#) (DB\_LOG\_BASE + 9)  
*Illegal record count specified.*
- #define [DB\\_LOG\\_TABLE\\_EXISTS\\_EC](#) (DB\_LOG\_BASE + 10)  
*Specified table already exists.*



- #define `DB_LOG_OUT_OF_TABLES_EC` (`DB_LOG_BASE` + 11)  
*Table resources exceeded.*
- #define `DB_LOG_OUT_OF_RECORDS_EC` (`DB_LOG_BASE` + 12)  
*Table record resources exceeded.*
- #define `DB_LOG_OUT_OF_INDICES_EC` (`DB_LOG_BASE` + 13)  
*Table index resources exceeded.*
- #define `DB_LOG_OUT_OF_CURSORS_EC` (`DB_LOG_BASE` + 14)  
*Table cursor resources exceeded.*
- #define `DB_LOG_RECORDS_INUSE_EC` (`DB_LOG_BASE` + 15)  
*Cannot delete table, records are still in table.*
- #define `DB_LOG_CURSORS_INUSE_EC` (`DB_LOG_BASE` + 16)  
*Cannot delete table, cursors are still in use.*
- #define `DB_LOG_INDEX_INUSE_EC` (`DB_LOG_BASE` + 17)  
*Cannot delete table, indices are still in use.*
- #define `DB_LOG_ALLOC_CURSOR_POOL_EC` (`DB_LOG_BASE` + 18)  
*Failed to allocate cursor buffer pool.*
- #define `DB_LOG_ALLOC_INDEX_POOL_EC` (`DB_LOG_BASE` + 19)  
*Failed to allocate index buffer pool.*
- #define `DB_LOG_ALLOC_RECORD_POOL_EC` (`DB_LOG_BASE` + 20)  
*Failed to allocate record buffer pool.*
- #define `DB_LOG_ALLOC_DATABASE_EC` (`DB_LOG_BASE` + 21)  
*Failed to allocate database.*
- #define `DB_LOG_TABLE_NOT_INUSE_EC` (`DB_LOG_BASE` + 22)  
*An attempt was made to delete a table not in use.*
- #define `DB_LOG_RECORD_ALREADY_EXISTS_EC` (`DB_LOG_BASE` + 23)  
*An attempt was made to insert an already existing record.*
- #define `DB_LOG_RECORD_DOES_NOT_EXIST_EC` (`DB_LOG_BASE` + 24)  
*An attempt was made to delete a record that does not exist.*
- #define `DB_LOG_CURSOR_INVALIDATED_EC` (`DB_LOG_BASE` + 25)  
*An attempt was made to use a cursor that has been invalidated.*
- #define `DB_LOG_LOCK_FAILURE_EC` (`DB_LOG_BASE` + 26)  
*Failed to lock the database.*
- #define `DB_LOG_UNLOCK_FAILURE_EC` (`DB_LOG_BASE` + 27)  
*Failed to unlock the database.*
- #define `DB_LOG_OUT_OF_CTORTOR_EC` (`DB_LOG_BASE` + 28)  
*Constructor/Destructor memory could not be allocated.*

### 8.3.1 Detailed Description

DB module log codes.

This file defines log codes for all messages logged by the database modules. All the log codes in this file are publicly documented.

## 8.4 dds\_c\_config.h File Reference

DDS C configuration file.

```
#include "netio/netio_config.h"
#include "osapi/osapi_config.h"
```

### 8.4.1 Detailed Description

DDS C configuration file.

## 8.5 dds\_c\_flowcontroller.h File Reference

DDS Flow Controller.

```
#include "netio/netio_flowcontroller.h"
#include "dds_c/dds_c_common.h"
#include "dds_c/dds_c_infrastructure.h"
```

### Classes

- struct [DDS\\_FlowControllerTokenBucketProperty\\_t](#)  
*DDSFlowController uses the popular token bucket approach for open loop network flow control. The flow control characteristics are determined by the token bucket properties.*
- struct [DDS\\_FlowControllerProperty\\_t](#)  
*Determines the flow control characteristics of the DDSFlowController.*

### Enumerations

- enum [DDS\\_FlowControllerSchedulingPolicy](#) { [DDS\\_RR\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#), [DDS\\_EDF\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#), [DDS\\_HPF\\_FLOW\\_CONTROLLER\\_SCHED\\_POLICY](#) }  
*Kinds of flow controller scheduling policy.*

### Variables

- const char \*const [DDS\\_DEFAULT\\_FLOW\\_CONTROLLER\\_NAME](#)  
*[default] Special value of DDS\_PublishModeQosPolicy::flow\_controller\_name that refers to the built-in default flow controller.*
- const char \*const [DDS\\_FIXED\\_RATE\\_FLOW\\_CONTROLLER\\_NAME](#)  
*Special value of DDS\_PublishModeQosPolicy::flow\_controller\_name that refers to the built-in fixed-rate flow controller.*
- const char \*const [DDS\\_ON\\_DEMAND\\_FLOW\\_CONTROLLER\\_NAME](#)  
*Special value of DDS\_PublishModeQosPolicy::flow\_controller\_name that refers to the built-in on-demand flow controller.*
- const struct [DDS\\_FlowControllerProperty\\_t](#) [DDS\\_FLOW\\_CONTROLLER\\_PROPERTY\\_DEFAULT](#)  
*<<eXtension>> Special value for creating a DDSFlowController with default property.*

### 8.5.1 Detailed Description

DDS Flow Controller.

## 8.6 dds\_c\_log.h File Reference

DDS\_C module log codes.

```
#include "dds_c/dds_c_config.h"
#include "osapi/osapi_log.h"
```

### Macros

- #define [DDSC\\_LOG\\_INVALID\\_DURATION\\_EC](#) (DDSC\_LOG\_BASE + 1)  
*The specified duration is not valid.*
- #define [DDSC\\_LOG\\_INVALID\\_PARTICIPANT\\_NAME\\_EC](#) (DDSC\_LOG\_BASE + 2)  
*An invalid participant name was specified with an unknown GUID.*
- #define [DDSC\\_LOG\\_INVALID\\_PARTICIPANT\\_GUID\\_PREFIX\\_EC](#) (DDSC\_LOG\_BASE + 3)  
*An invalid participant GUID prefix was specified, typically the GUID prefix does not match an already detected participant.*
- #define [DDSC\\_LOG\\_SYS\\_GETTIME\\_EC](#) (DDSC\_LOG\_BASE + 4)  
*A call to OSAPI\_System\_get\_time failed.*
- #define [DDSC\\_LOG\\_GET\\_NEXT\\_OBJECT\\_ID\\_EC](#) (DDSC\_LOG\_BASE + 5)  
*Failed to get the next automatically generated object ID for an entity's GUID. This typically means the object id pool has been exhausted.*
- #define [DDSC\\_LOG\\_SET\\_ENTITY\\_NAME\\_EC](#) (DDSC\_LOG\_BASE + 6)  
*Failed to set name string for a DDS entity in the DomainParticipantQos entity\_name policy.*
- #define [DDSC\\_LOG\\_SYS\\_GET\\_HOSTNAME\\_EC](#) (DDSC\_LOG\_BASE + 7)  
*A call to OSAPI\_System\_get\_hostname failed.*
- #define [DDSC\\_LOG\\_IO\\_SNPRINTF\\_FAILED\\_EC](#) (DDSC\_LOG\_BASE + 8)  
*A call to OSAPI\_Stdio\_snprintf failed. Typically this means the destination buffer was too small.*
- #define [DDSC\\_LOG\\_TOPIC\\_FIND\\_EC](#) (DDSC\_LOG\_BASE + 9)  
*Failed to find a topic created by a DomainParticipant.*
- #define [DDSC\\_LOG\\_UNKNOWN\\_REMOTE\\_PARTICIPANT\\_NAME\\_EC](#) (DDSC\_LOG\_BASE + 10)  
*Endpoint discovery failed because the name of the remote participant parent for an endpoint was not found.*
- #define [DDSC\\_LOG\\_UNKNOWN\\_REMOTE\\_PARTICIPANT\\_KEY\\_EC](#) (DDSC\_LOG\_BASE + 11)  
*Endpoint discovery failed because the key of the remote participant parent for an endpoint was not found. Note that the key is logged as 4 integers in host endianness format.*
- #define [DDSC\\_LOG\\_REMOTE\\_PARTICIPANT\\_KEY\\_NOT\\_EQUAL\\_EC](#) (DDSC\_LOG\_BASE + 12)  
*Failed endpoint discovery when key does not match the remote participant.*
- #define [DDSC\\_LOG\\_INVALID\\_ENDPOINT\\_GUID\\_EC](#) (DDSC\_LOG\_BASE + 13)  
*Failed endpoint discovery due to an invalid or unknown endpoint GUID.*
- #define [DDSC\\_LOG\\_ENDPOINT\\_NOT\\_CHILD\\_OF\\_PARTICIPANT\\_EC](#) (DDSC\_LOG\_BASE + 14)  
*Failed endpoint discovery when an endpoint is determined to belong to a different remote participant.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_DOES\\_NOT\\_EXIST\\_EC](#) (DDSC\_LOG\_BASE + 15)  
*Failed participant discovery because a remote participant that should have been already asserted locally was not found.*

- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 16)  
*Did not find a remote participant when asserting participant liveness for it. Note that the key is logged as 4 integers in host endianness format.*
- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC` (`DDSC_LOG_BASE` + 17)  
*Failed to assert participant liveness to a remote participant.*
- #define `DDSC_LOG_PARTICIPANT_LOOKUP_EC` (`DDSC_LOG_BASE` + 18)  
*Failed to find a remote participant as previously discovered.*
- #define `DDSC_LOG_REMOVE_PUBLICATION_EC` (`DDSC_LOG_BASE` + 19)  
*Failed to remove resources for a remote publication.*
- #define `DDSC_LOG_REMOVE_SUBSCRIPTION_EC` (`DDSC_LOG_BASE` + 20)  
*Failed to remove resources for a remote subscription.*
- #define `DDSC_LOG_FIND_PUBLICATION_PARENT_EC` (`DDSC_LOG_BASE` + 21)  
*Cannot determine the participant of a remote publication.*
- #define `DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC` (`DDSC_LOG_BASE` + 22)  
*Cannot determine the participant of a remote subscription.*
- #define `DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC` (`DDSC_LOG_BASE` + 23)  
*Failed to create DomainParticipant due to running out of participant IDs. This is typically caused by all UDP ports being used.*
- #define `DDSC_LOG_RESERVE_LOCATORS_EC` (`DDSC_LOG_BASE` + 24)  
*Failed to reserve endpoint locators.*
- #define `DDSC_LOG_TIMER_CREATE_TIMEOUT_EC` (`DDSC_LOG_BASE` + 25)  
*Failed to create a timeout.*
- #define `DDSC_LOG_ILLEGAL_OBJECTID_EC` (`DDSC_LOG_BASE` + 26)  
*Illegal object id specified.*
- #define `DDSC_LOG_TOPIC_NARROW_EC` (`DDSC_LOG_BASE` + 27)  
*Failed to narrow a TopicDescription to the named Topic.*
- #define `DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC` (`DDSC_LOG_BASE` + 28)  
*Failed to update state of StatusCondition.*
- #define `DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC` (`DDSC_LOG_BASE` + 29)  
*Failed to remove a condition reference from a waitset.*
- #define `DDSC_LOG_WS_ADD_COND_REFERENCE_EC` (`DDSC_LOG_BASE` + 30)  
*Failed to add a condition reference to a waitset.*
- #define `DDSC_LOG_RELEASE_META_MC_EC` (`DDSC_LOG_BASE` + 31)  
*Failed to release resources for multicast discovery locators.*
- #define `DDSC_LOG_RELEASE_META_UC_EC` (`DDSC_LOG_BASE` + 32)  
*Failed to release resources for unicast discovery locators.*
- #define `DDSC_LOG_RELEASE_USER_MC_EC` (`DDSC_LOG_BASE` + 33)  
*Failed to release resources for multicast user locators.*
- #define `DDSC_LOG_RELEASE_USER_UC_EC` (`DDSC_LOG_BASE` + 34)  
*Failed to release resources for unicast user locators.*
- #define `DDSC_LOG_INVALID_DOMAINID_EC` (`DDSC_LOG_BASE` + 35)  
*The domain ID specified exceeds what is allowed based on the parameters specified in `dp_qos.protocol.rtps_well_known_ports`.*
- #define `DDSC_LOG_ON_TYPE_REGISTERED_FAILURE_EC` (`DDSC_LOG_BASE` + 36)  
*Error occurred during the `on_type_registered` call back.*
- #define `DDSC_LOG_ON_TYPE_UNREGISTERED_FAILURE_EC` (`DDSC_LOG_BASE` + 37)  
*Error occurred during the `on_type_unregistered` call back.*
- #define `DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC` (`DDSC_LOG_BASE` + 38)

- Error occurred during the on\_type\_unregistered call back.*

  - #define `DDSC_LOG_MIN_DISCOVERY_MTU_EC` (`DDSC_LOG_BASE` + 39)
  - Minimum MTU across all Discovery transports must be greater or equal to a packet containing serialized Participant↔BuiltinTopicData.*
  - #define `DDSC_LOG_MIN_MTU_SIZE_EC` (`DDSC_LOG_BASE` + 40)
  - MTU for a transport set lower or equal to Protocol Overhead of 448 bytes.*
  - #define `DDSC_LOG_GET_MTU_EC` (`DDSC_LOG_BASE` + 40)
  - Error getting the mtu from the route resolver.*
  - #define `DDSC_LOG_GET_MTU_NO_ROUTES_EC` (`DDSC_LOG_BASE` + 41)
  - MTU could not be found because of no routes.*
  - #define `DDSC_LOG_RESOLVE_LOCATORS_PRIORITY_EC` (`DDSC_LOG_BASE` + 42)
  - Failed to resolve locators based on priority.*
  - #define `DDSC_LOG_REMOTE_PUBLICATION_DATA_CHANGED_EC` (`DDSC_LOG_BASE` + 43)
  - The publication builtin topic data for a remote publication changed after the remote publication was asserted.*
  - #define `DDSC_LOG_REMOTE_SUBSCRIPTION_DATA_CHANGED_EC` (`DDSC_LOG_BASE` + 44)
  - Fields in subscription builtin topic data for a remote subscription which are not supported to dynamically change were updated after the remote subscription was asserted.*
  - #define `DDSC_LOG_DATABASE_CREATE_EC` (`DDSC_LOG_BASE` + 100)
  - Failed to create database.*
  - #define `DDSC_LOG_DATABASE_DELETE_EC` (`DDSC_LOG_BASE` + 101)
  - Failed to delete database.*
  - #define `DDSC_LOG_TABLE_CREATE_EC` (`DDSC_LOG_BASE` + 102)
  - Failed to create database table of the specified name.*
  - #define `DDSC_LOG_TABLE_INUSE_EC` (`DDSC_LOG_BASE` + 103)
  - Failed to delete a database table because it is not empty.*
  - #define `DDSC_LOG_TABLE_DELETE_EC` (`DDSC_LOG_BASE` + 104)
  - Failed to delete a database table.*
  - #define `DDSC_LOG_TABLE_SELECT_EC` (`DDSC_LOG_BASE` + 105)
  - A selection operation failed on the specified database table.*
  - #define `DDSC_LOG_CREATE_INDEX_EC` (`DDSC_LOG_BASE` + 106)
  - Failed to create an index on a database table.*
  - #define `DDSC_LOG_DELETE_INDEX_EC` (`DDSC_LOG_BASE` + 107)
  - Failed to delete an index on a database table.*
  - #define `DDSC_LOG_DB_CURSOR_INVALIDATED_EC` (`DDSC_LOG_BASE` + 108)
  - A database table cursor was invalidated while in use.*
  - #define `DDSC_LOG_SUBSCRIPTION_RECORD` 1
  - Database record for a Subscription.*
  - #define `DDSC_LOG_PUBLICATION_RECORD` 2
  - Database record for a Publication.*
  - #define `DDSC_LOG_PARTICIPANT_RECORD` 3
  - Database record for a Remote Participant.*
  - #define `DDSC_LOG_TOPIC_RECORD` 4
  - Database record for a Topic.*
  - #define `DDSC_LOG_DATAWRITER_RECORD` 5
  - Database record for a DataWriter.*
  - #define `DDSC_LOG_DATAREADER_RECORD` 6
  - Database record for a DataReader.*

- `#define DDSC_LOG_PUBLISHER_RECORD 7`  
*Database record for a Publisher.*
- `#define DDSC_LOG_SUBSCRIBER_RECORD 8`  
*Database record for a Subscriber.*
- `#define DDSC_LOG_ROUTE_RECORD 9`  
*Database record for a route record.*
- `#define DDSC_LOG_BIND_RECORD 10`  
*Database record for a bind record.*
- `#define DDSC_LOG_TYPE_RECORD 11`  
*Database record for a type plugin.*
- `#define DDSC_LOG_REMOTE_ENDPOINT_TRUST_STATE_RECORD 12`  
*Database record for a remote trust endpoint.*
- `#define DDSC_LOG_MATCHED_DW_RECORD 13`  
*Database record for a matched remote data writer.*
- `#define DDSC_LOG_STRING_RECORD 14`  
*Database record for a string property.*
- `#define DDSC_LOG_RECORD_CREATE_EC (DDSC_LOG_BASE + 109)`  
*Failed to create a database record of the specified kind.*
- `#define DDSC_LOG_RECORD_DELETE_EC (DDSC_LOG_BASE + 110)`  
*Failed to delete a database record of the specified kind.*
- `#define DDSC_LOG_RECORD_INSERT_EC (DDSC_LOG_BASE + 111)`  
*Failed to insert a database record of the specified kind.*
- `#define DDSC_LOG_RECORD_ERROR_EC (DDSC_LOG_BASE + 112)`  
*Unknown error for database record of the specified kind.*
- `#define DDSC_LOG_RECORD_EXISTS_EC (DDSC_LOG_BASE + 113)`  
*A database record of the specified kind already exists.*
- `#define DDSC_LOG_RECORD_LOOKUP_EC (DDSC_LOG_BASE + 114)`  
*A lookup of a database record of the specified kind failed.*
- `#define DDSC_LOG_RECORD_NOT_EXISTS_EC (DDSC_LOG_BASE + 115)`  
*A database record of the specified kind does not exist.*
- `#define DDSC_LOG_RECORD_SELECT_EC (DDSC_LOG_BASE + 116)`  
*A database select on the specified record kind failed.*
- `#define DDSC_LOG_RECORD_REMOVE_EC (DDSC_LOG_BASE + 117)`  
*Removal a database record of the specified kind failed.*
- `#define DDSC_LOG_RECORD_INITIALIZE_EC (DDSC_LOG_BASE + 118)`  
*A database record of the specified kind could not be initialized.*
- `#define DDSC_LOG_RECORD_FINALIZE_EC (DDSC_LOG_BASE + 119)`  
*A database record of the specified kind could not be finalized.*
- `#define DDSC_LOG_RECORD_RESET_EC (DDSC_LOG_BASE + 120)`  
*A database record of the specified kind could not be reset.*
- `#define DDSC_LOG_PUBLICATIONDATA_OBJECT 1`  
*DDS PublicationBuiltinTopicData object.*
- `#define DDSC_LOG_SUBSCRIPTIONDATA_OBJECT 2`  
*DDS SubscriptionBuiltinTopicData object.*
- `#define DDSC_LOG_PARTICIPANTDATA_OBJECT 3`  
*DDS ParticipantBuiltinTopicData object.*
- `#define DDSC_LOG_DATAREADERQOS_OBJECT 4`

- *DDS DataReaderQos object.*
- #define [DDSC\\_LOG\\_DATAWRITERQOS\\_OBJECT](#) 5
- *DDS DataWriterQos object.*
- #define [DDSC\\_LOG\\_TOPICQOS\\_OBJECT](#) 6
- *DDS TopicQos object.*
- #define [DDSC\\_LOG\\_PUBLISHERQOS\\_OBJECT](#) 7
- *DDS PublisherQos object.*
- #define [DDSC\\_LOG\\_SUBSCRIBERQOS\\_OBJECT](#) 8
- *DDS SubscriberQos object.*
- #define [DDSC\\_LOG\\_PARTICIPANTQOS\\_OBJECT](#) 9
- *DDS DomainParticipantQos object.*
- #define [DDSC\\_LOG\\_ENTITY\\_OBJECT](#) 10
- *DDS Entity object.*
- #define [DDSC\\_LOG\\_PARTICIPANTFACTORYQOS\\_OBJECT](#) 11
- *DDS DomainParticipantFactoryQos object.*
- #define [DDSC\\_LOG\\_TYPE\\_OBJECT](#) 12
- *A DDS Type object.*
- #define [DDSC\\_LOG\\_BINDRESOLVER\\_OBJECT](#) 13
- *A NETIO BindResolver object.*
- #define [DDSC\\_LOG\\_ROUTERESOLVER\\_OBJECT](#) 14
- *A NETIO RouteResolver object.*
- #define [DDSC\\_LOG\\_ADDRESSRESOLVER\\_OBJECT](#) 15
- *A NETIO AddressResolver object.*
- #define [DDSC\\_LOG\\_DATAWRITERIO\\_OBJECT](#) 16
- *A NETIO DataWriterInterface object.*
- #define [DDSC\\_LOG\\_DATAREADERIO\\_OBJECT](#) 17
- *A NETIO DataReaderInterface object.*
- #define [DDSC\\_LOG\\_LOG\\_OBJECT](#) 18
- *A OSAPI Log object.*
- #define [DDSC\\_LOG\\_SYSTEM\\_OBJECT](#) 19
- *A OSAPI system object.*
- #define [DDSC\\_LOG\\_MUTEX\\_OBJECT](#) 20
- *A OSAPI mutex object.*
- #define [DDSC\\_LOG\\_CONDREF\\_OBJECT](#) 21
- *A DDS Waitset condition reference object.*
- #define [DDSC\\_LOG\\_WSREF\\_OBJECT](#) 22
- *A DDS Condition waitset reference object.*
- #define [DDSC\\_LOG\\_MD5STREAM\\_OBJECT](#) 23
- *A MD5 stream object.*
- #define [DDSC\\_LOG\\_CONDITION\\_OBJECT](#) 24
- *A DDS Condition object.*
- #define [DDSC\\_LOG\\_RT\\_OBJECT](#) 25
- *A RT Condition object.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_POOL\\_OBJECT](#) 26
- *A Participant pool object.*
- #define [DDSC\\_LOG\\_TIMER\\_OBJECT](#) 27
- *A OSAPI Timer object.*

- #define [DDSC\\_LOG\\_TIMEROUT\\_OBJECT](#) 28  
*A OSAPI\_Timer timeout object.*
- #define [DDSC\\_LOG\\_TOPIC\\_OBJECT](#) 29  
*A DDS Topic object.*
- #define [DDSC\\_LOG\\_WAITSET\\_OBJECT](#) 30  
*A DDS WaitSet object.*
- #define [DDSC\\_LOG\\_UUID\\_OBJECT](#) 31  
*A UUID object.*
- #define [DDSC\\_LOG\\_MEMPOOL\\_OBJECT](#) 32  
*A REDA\_MemPool object.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_PROPERTIES](#) 33  
*A DomainParticipant's string properties sequence.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_BINARY\\_PROPERTIES](#) 34  
*A DomainParticipant's binary properties sequence.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_BUILTIN\\_PUBLISHER](#) 35  
*A DomainParticipant's built-in Publisher.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_BUILTIN\\_SUBSCRIBER](#) 36  
*A DomainParticipant's built-in Subscriber.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_GENERIC\\_MESSAGE\\_OBJECT](#) 37  
*A built-in Inter-Participant Channel data type object.*
- #define [DDSC\\_LOG\\_DATA HOLDER\\_SEQ\\_OBJECT](#) 38  
*A Trust data holder sequence object.*
- #define [DDSC\\_LOG\\_PROPERTYQOSPOLICY\\_OBJECT](#) 39  
*A Property QoS policy object.*
- #define [DDSC\\_LOG\\_STRING\\_OBJECT](#) 40  
*A String object.*
- #define [DDSC\\_LOG\\_AUTHPOOL\\_OBJECT](#) 41  
*An authentication pool object.*
- #define [DDSC\\_LOG\\_BUFFER\\_OBJECT](#) 42  
*A Log buffer object.*
- #define [DDSC\\_LOG\\_STRINGMANAGER\\_OBJECT](#) 43  
*A String Manager object.*
- #define [DDSC\\_LOG\\_OBJECT\\_INITIALIZE\\_EC](#) (DDSC\_LOG\_BASE + 200)  
*Out of resources to initialize object of the specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_ALLOCATE\\_EC](#) (DDSC\_LOG\_BASE + 201)  
*Out of resources to allocate an object of the specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_FINALIZE\\_EC](#) (DDSC\_LOG\_BASE + 202)  
*Failed to finalize object of specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_DELETE\\_EC](#) (DDSC\_LOG\_BASE + 203)  
*Failed to delete object of specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_COPY\\_EC](#) (DDSC\_LOG\_BASE + 204)  
*Failed to copy object of specified kind.*
- #define [DDSC\\_LOG\\_OBJECT\\_REFCOUNT\\_EC](#) (DDSC\_LOG\_BASE + 205)  
*Failed to delete/finalize an object because other objects are referencing it.*
- #define [DDSC\\_LOG\\_OBJECT\\_GET\\_PROPERTY\\_EC](#) (DDSC\_LOG\_BASE + 206)  
*Failed to get the object properties.*
- #define [DDSC\\_LOG\\_OBJECT\\_SET\\_PROPERTY\\_EC](#) (DDSC\_LOG\_BASE + 207)



- Failed to set the object properties.*

  - #define `DDSC_LOG_OBJECT_EMPTY_EC` (`DDSC_LOG_BASE` + 208)

*An object is empty, typically applies only to buffer-pool objects.*
- #define `DDSC_LOG_OBJECT_INUSECOUNT_EC` (`DDSC_LOG_BASE` + 209)

*Failed to delete/finalize an object because other objects are using it.*
- #define `DDSC_LOG_OBJECT_CREATE_EC` (`DDSC_LOG_BASE` + 210)

*Failed to create an object.*
- #define `DDSC_LOG_NETMASK_SEQUENCE` 1

*A NETIO Netmask sequence.*
- #define `DDSC_LOG_ROUTE_SEQUENCE` 2

*A NETIO Route sequence.*
- #define `DDSC_LOG_RESERVED_SEQUENCE` 3

*A NETIO Reserved address sequence.*
- #define `DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE` 4

*A DDS sequence of enabled user NETIO transports.*
- #define `DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE` 5

*A DDS sequence of enabled transports.*
- #define `DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE` 6

*A DDS sequence of enabled discovery transports.*
- #define `DDSC_LOG_INITIAL_PEER_SEQUENCE` 7

*A DDS sequence of initial peers.*
- #define `DDSC_LOG_DESTINATION_SEQUENCE` 8

*A NETIO sequence of destinations to send to.*
- #define `DDSC_LOG_METAUNICAST_SEQUENCE` 9

*A DDS sequence of meta-traffic unicast locators.*
- #define `DDSC_LOG_METAMULTICAST_SEQUENCE` 10

*A DDS sequence of meta-traffic multicast locators.*
- #define `DDSC_LOG_USERUNICAST_SEQUENCE` 11

*A DDS sequence of user-traffic unicast locators.*
- #define `DDSC_LOG_USERMULTICAST_SEQUENCE` 12

*A DDS sequence of user-traffic multicast locators.*
- #define `DDSC_LOG_READTAKE_SEQUENCE` 13

*A DDS sequence used in a read/take call.*
- #define `DDSC_LOG_DATAHOLDER_SEQUENCE` 14

*A DDS sequence of DataHolder objects.*
- #define `DDSC_LOG_COOKIE_PAYLOAD_SEQUENCE` 15

*A cookie octet sequence.*
- #define `DDSC_LOG_REPRESENTATION_ID_SEQUENCE` 16

*a sequence for data representation IDs*
- #define `DDSC_LOG_SEQUENCE_SETMAX_EC` (`DDSC_LOG_BASE` + 300)

*Failed to set the maximum length of a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_SETLENGTH_EC` (`DDSC_LOG_BASE` + 301)

*Failed to set the length of a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_GETREF_EC` (`DDSC_LOG_BASE` + 302)

*Failed to get a reference at the specified index for a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_INITIALIZE_EC` (`DDSC_LOG_BASE` + 303)

*Failed to initialize a sequence of the specified kind.*

- #define `DDSC_LOG_SEQUENCE_FINALIZE_EC` (`DDSC_LOG_BASE` + 304)  
*Failed to finalize a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_COPY_EC` (`DDSC_LOG_BASE` + 305)  
*Failed to copy a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_INVALID_EC` (`DDSC_LOG_BASE` + 306)  
*The sequence of the specified kind was invalid in the context it is used.*
- #define `DDSC_LOG_DISCOVERY_COMPONENT` 1  
*A component of discovery kind.*
- #define `DDSC_LOG_RTPS_COMPONENT` 2  
*A component of RTPS kind.*
- #define `DDSC_LOG_READERHISTORY_COMPONENT` 3  
*A component of reader-history kind.*
- #define `DDSC_LOG_WRITERHISTORY_COMPONENT` 4  
*A component of writer-history kind.*
- #define `DDSC_LOG_DATAREADERIO_COMPONENT` 5  
*A component of DataReaderInterface kind.*
- #define `DDSC_LOG_DATAWRITERIO_COMPONENT` 6  
*A component of DataWriterInterface kind.*
- #define `DDSC_LOG_NETIO_COMPONENT` 7  
*A component of NETIO kind.*
- #define `DDSC_LOG_TRANSPORT_COMPONENT` 8  
*A component of transport kind.*
- #define `DDSC_LOG_APPGEN_COMPONENT` 9  
*A component of application generation kind.*
- #define `DDSC_LOG_COMPONENT_LOOKUP_EC` (`DDSC_LOG_BASE` + 400)  
*Did not find a component factory with the given name in the registry.*
- #define `DDSC_LOG_COMPONENT_CREATE_EC` (`DDSC_LOG_BASE` + 401)  
*Could not create a component of the specified kind using the specified factory.*
- #define `DDSC_LOG_COMPONENT_DELETE_EC` (`DDSC_LOG_BASE` + 402)  
*Could not delete a component of the specified kind using the specified factory.*
- #define `DDSC_LOG_HISTORY_RESOURCE` 1  
*Exceeded resource limits for writer history queue.*
- #define `DDSC_LOG_SAMPLE_RESOURCES` 2  
*Failed to get resource for new sample due to resource limit.*
- #define `DDSC_LOG_PARTICIPANT_RESOURCES` 3  
*Failed to allocate a new participant.*
- #define `DDSC_LOG_RESOURCE_EXCEEDED_EC` (`DDSC_LOG_BASE` + 500)  
*Could not allocate a resource of the specified kind.*
- #define `DDSC_LOG_TOPIC_QOS` 1  
*The TopicQos kind.*
- #define `DDSC_LOG_DATAREADER_QOS` 2  
*The DataReaderQos kind.*
- #define `DDSC_LOG_DATAWRITER_QOS` 3  
*The DataWriterQos kind.*
- #define `DDSC_LOG_SUBSCRIBER_QOS` 4  
*The SubscriberQos kind.*
- #define `DDSC_LOG_PUBLISHER_QOS` 5

- *The PublisherQos kind.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_QOS](#) 6
- *The DomainParticipantQos kind.*
- #define [DDSC\\_LOG\\_PARTICIPANTFACTORY\\_QOS](#) 7
- *The DomainParticipantFactoryQos kind.*
- #define [DDSC\\_LOG\\_DEFAULTTOPIC\\_QOS](#) 8
- *The default TopicQos kind.*
- #define [DDSC\\_LOG\\_DEFAULTDATAREADER\\_QOS](#) 9
- *The default DataReaderQos kind.*
- #define [DDSC\\_LOG\\_DEFAULTDATAWRITER\\_QOS](#) 10
- *The default DataWriterQos kind.*
- #define [DDSC\\_LOG\\_DEFAULTSUBSCRIBER\\_QOS](#) 11
- *The default SubscriberQos kind.*
- #define [DDSC\\_LOG\\_DEFAULTPUBLISHER\\_QOS](#) 12
- *The default PublisherQos kind.*
- #define [DDSC\\_LOG\\_DEFAULTPARTICIPANT\\_QOS](#) 13
- *The default PublisherQos kind.*
- #define [DDSC\\_LOG\\_DEADLINE\\_QOS\\_POLICY](#) 14
- *The deadline qos policy kind.*
- #define [DDSC\\_LOG\\_LIVELINESS\\_QOS\\_POLICY](#) 15
- *The liveliness qos policy kind.*
- #define [DDSC\\_LOG\\_HISTORY\\_QOS\\_POLICY](#) 16
- *The history qos policy kind.*
- #define [DDSC\\_LOG\\_RESOURCE\\_LIMIT\\_QOS\\_POLICY](#) 17
- *The resource limits qos policy kind.*
- #define [DDSC\\_LOG\\_PROTOCOL\\_QOS\\_POLICY](#) 18
- *The protocol qos policy kind.*
- #define [DDSC\\_LOG\\_TYPE\\_SUPPORT\\_QOS\\_POLICY](#) 19
- *The type-support qos policy kind.*
- #define [DDSC\\_LOG\\_RELIABILITY\\_QOS\\_POLICY](#) 20
- *The reliability qos policy kind.*
- #define [DDSC\\_LOG\\_DURABILITY\\_QOS\\_POLICY](#) 21
- *The durability qos policy kind.*
- #define [DDSC\\_LOG\\_OWNERSHIP\\_QOS\\_POLICY](#) 22
- *The ownership qos policy kind.*
- #define [DDSC\\_LOG\\_OWNERSHIP\\_STRENGTH\\_QOS\\_POLICY](#) 23
- *The ownership-strength qos policy kind.*
- #define [DDSC\\_LOG\\_TRANSPORT\\_QOS\\_POLICY](#) 24
- *The transport qos policy kind.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_ID\\_QOS\\_POLICY](#) 25
- *The participant id qos policy kind.*
- #define [DDSC\\_LOG\\_HEARTBEATS\\_QOS\\_POLICY](#) 26
- *The heartbeat qos policy kind.*
- #define [DDSC\\_LOG\\_DATAWRITER\\_RESOURCE\\_QOS\\_POLICY](#) 27
- *The DataWriterQos writer\_resource\_limits policy.*
- #define [DDSC\\_LOG\\_DATAREADER\\_RESOURCE\\_QOS\\_POLICY](#) 28
- *The DataReaderQos reader\_resource\_limits policy.*

- #define [DDSC\\_LOG\\_DESTINATION\\_ORDER\\_POLICY](#) 29  
*The DestinationOrderQos destination\_order policy.*
- #define [DDSC\\_LOG\\_PROPERTY\\_QOS\\_POLICY](#) 30  
*The PropertyQos policy.*
- #define [DDSC\\_LOG\\_PUBLISH\\_MODE\\_QOS\\_POLICY](#) 31  
*The PublishModeQos policy.*
- #define [DDSC\\_LOG\\_DATA\\_REPRESENTATION\\_QOS\\_POLICY](#) 33  
*The DataRepresentationQos policy.*
- #define [DDSC\\_LOG\\_LATENCY\\_BUDGET\\_QOS\\_POLICY](#) 34  
*The LatencyBudgetQos policy.*
- #define [DDSC\\_LOG\\_CONTENT\\_FILTER\\_QOS\\_POLICY](#) 35  
*The ContentFilterQos policy.*
- #define [DDSC\\_LOG\\_QOS\\_INCONSISTENT\\_POLICY\\_EC](#) (DDSC\_LOG\_BASE + 501)  
*An inconsistent Qos policy for the specified Qos kind was found.*
- #define [DDSC\\_LOG\\_QOS\\_INCONSISTENT\\_POLICIES\\_EC](#) (DDSC\_LOG\_BASE + 502)  
*Inconsistency between two Qos policies for the specified Qos kind was found.*
- #define [DDSC\\_LOG\\_QOS\\_INCONSISTENT\\_EC](#) (DDSC\_LOG\_BASE + 503)  
*Failed to create an entity or set a qos due to inconsistent policy.*
- #define [DDSC\\_LOG\\_QOS\\_COPY\\_EC](#) (DDSC\_LOG\_BASE + 504)  
*Failed to copy a Qos of the specified kind.*
- #define [DDSC\\_LOG\\_QOS\\_INITIALIZE\\_EC](#) (DDSC\_LOG\_BASE + 505)  
*Failed to initialize a Qos of the specified kind.*
- #define [DDSC\\_LOG\\_QOS\\_FINALIZE\\_EC](#) (DDSC\_LOG\_BASE + 506)  
*Failed to finalize a Qos of the specified kind.*
- #define [DDSC\\_LOG\\_QOS\\_SET\\_EC](#) (DDSC\_LOG\_BASE + 507)  
*Failed to set a Qos of the specified kind.*
- #define [DDSC\\_LOG\\_QOS\\_SET\\_ON\\_ENABLED\\_EC](#) (DDSC\_LOG\_BASE + 508)  
*Failed to set a Qos of the specified kind because the entity is already enabled.*
- #define [DDSC\\_LOG\\_QOS\\_IMMUTABLE\\_EC](#) (DDSC\_LOG\_BASE + 509)  
*Failed to set a Qos of the specified kind the immutable Qos policies have been changed.*
- #define [DDSC\\_LOG\\_QOS\\_CHANGED\\_EC](#) (DDSC\_LOG\_BASE + 510)  
*A discovered Qos changed (the entity already existed)*
- #define [DDSC\\_LOG\\_QOS\\_GET\\_EC](#) (DDSC\_LOG\_BASE + 511)  
*Failed to get a Qos of the specified kind.*
- #define [DDSC\\_LOG\\_TOPIC\\_LISTENER](#) 1  
*The topic listener kind.*
- #define [DDSC\\_LOG\\_DATAREADER\\_LISTENER](#) 2  
*The datareader listener kind.*
- #define [DDSC\\_LOG\\_DATAWRITER\\_LISTENER](#) 3  
*The datawriter listener kind.*
- #define [DDSC\\_LOG\\_SUBSCRIBER\\_LISTENER](#) 4  
*The subscriber listener kind.*
- #define [DDSC\\_LOG\\_PUBLISHER\\_LISTENER](#) 5  
*The publisher listener kind.*
- #define [DDSC\\_LOG\\_PARTICIPANT\\_LISTENER](#) 6  
*The participant listener kind.*
- #define [DDSC\\_LOG\\_PARTICIPANTFACTORY\\_LISTENER](#) 7

- The participant-factory listener kind.*

  - #define `DDSC_LOG_LISTENER_INCONSISTENT_EC` (`DDSC_LOG_BASE` + 512)

*Failed to create an entity due to inconsistent listener and status mask.*
- #define `DDSC_LOG_LISTENER_SET_EC` (`DDSC_LOG_BASE` + 513)

*Failed to set the listener of the specified kind.*
- #define `DDSC_LOG_LISTENER_GET_EC` (`DDSC_LOG_BASE` + 514)

*Failed to get the listener of the specified kind.*
- #define `DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC` (`DDSC_LOG_BASE` + 515)

*Illegal combination of NULL listener and non-NONE status mask when setting a listener for an Entity.*
- #define `DDSC_LOG_TOPIC_ENTITY` 1

*The topic entity kind.*
- #define `DDSC_LOG_DATAWRITER_ENTITY` 2

*The datawriter entity kind.*
- #define `DDSC_LOG_DATAREADER_ENTITY` 3

*The datareader entity kind.*
- #define `DDSC_LOG_PUBLISHER_ENTITY` 4

*The publisher entity kind.*
- #define `DDSC_LOG_SUBSCRIBER_ENTITY` 5

*The subscriber entity kind.*
- #define `DDSC_LOG_PARTICIPANT_ENTITY` 6

*The participant entity kind.*
- #define `DDSC_LOG_PUBLICATION_ENTITY` 7

*The publication entity kind.*
- #define `DDSC_LOG_SUBSCRIPTION_ENTITY` 8

*The subscription entity kind.*
- #define `DDSC_LOG_ENTITY_ENABLE_EC` (`DDSC_LOG_BASE` + 600)

*Failed to enable an entity of the specified kind.*
- #define `DDSC_LOG_ENTITY_NOT_EMPTY_EC` (`DDSC_LOG_BASE` + 601)

*Failed to an delete/finalize an entity of the specified kind because it is not empty.*
- #define `DDSC_LOG_ENTITY_FINALIZE_EC` (`DDSC_LOG_BASE` + 602)

*Failed to finalize an entity of the specified kind.*
- #define `DDSC_LOG_ENTITY_INITIALIZE_EC` (`DDSC_LOG_BASE` + 603)

*Failed to initialize an entity of the specified kind.*
- #define `DDSC_LOG_ENTITY_NOT_ENABLED_EC` (`DDSC_LOG_BASE` + 604)

*An operation was attempted on an entity that is not enabled.*
- #define `DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC` (`DDSC_LOG_BASE` + 605)

*Entities are in different factories.*
- #define `DDSC_LOG_ENTITY_INVALID_PROPERTY_EC` (`DDSC_LOG_BASE` + 606)

*An invalid property was specified for the entity.*
- #define `DDSC_LOG_DATAWRITER_CDR` 1

*The datawriter CDR kind.*
- #define `DDSC_LOG_DATAREADER_CDR` 2

*The datareader CDR kind.*
- #define `DDSC_LOG_DATAWRITER_INLINE` 3

*The datawriter inline kind.*
- #define `DDSC_LOG_CDR_POOL_ALLOC_EC` (`DDSC_LOG_BASE` + 700)

*Failed to allocate a pool of the specified kind.*

- #define `DDSC_LOG_CDR_BUFFER_SET_EC` (`DDSC_LOG_BASE` + 701)  
*Failed to set the CDR buffer for a packet.*
- #define `DDSC_LOG_CDR_POOL_DELETE_EC` (`DDSC_LOG_BASE` + 702)  
*Failed to delete the CDR pool.*
- #define `DDSC_LOG_CDR_SERIALIZE_PID_EC` (`DDSC_LOG_BASE` + 703)  
*Failed to serialize a parameter ID.*
- #define `DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC` (`DDSC_LOG_BASE` + 704)  
*Failed to serialize a parameter length.*
- #define `DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC` (`DDSC_LOG_BASE` + 705)  
*Failed to serialize a key-hash.*
- #define `DDSC_LOG_CDR_SERIALIZE_DATA_EC` (`DDSC_LOG_BASE` + 706)  
*Failed to serialize payload data.*
- #define `DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC` (`DDSC_LOG_BASE` + 707)  
*Deserialized an invalid parameter length for a specific parameter ID.*
- #define `DDSC_LOG_CDR_DESERIALIZE_PID_EC` (`DDSC_LOG_BASE` + 708)  
*Failed to deserialize the ID of an inline parameter.*
- #define `DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC` (`DDSC_LOG_BASE` + 709)  
*Failed to deserialize the length of an inline parameter.*
- #define `DDSC_LOG_CDR_INCREMENT_POS_EC` (`DDSC_LOG_BASE` + 710)  
*Failed to increment to the position of the next inline parameter.*
- #define `DDSC_LOG_CDR_SET_POS_EC` (`DDSC_LOG_BASE` + 711)  
*Failed to set the reception stream position.*
- #define `DDSC_LOG_CDR_DESERIALIZE_HEADER_EC` (`DDSC_LOG_BASE` + 712)  
*Failed to deserialize the encapsulation header.*
- #define `DDSC_LOG_CDR_DESERIALIZE_DATA_EC` (`DDSC_LOG_BASE` + 713)  
*Failed to deserialize CDR payload data.*
- #define `DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC` (`DDSC_LOG_BASE` + 714)  
*Failed to initialize CDR sample.*
- #define `DDSC_LOG_CDR_FINALIZE_SAMPLE_EC` (`DDSC_LOG_BASE` + 715)  
*Failed to finalize sample.*
- #define `DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC` (`DDSC_LOG_BASE` + 716)  
*Failed to serialize the status info parameter.*
- #define `DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC` (`DDSC_LOG_BASE` + 717)  
*Failed to deserialize a key-hash.*
- #define `DDSC_LOG_CDR_DESERIALIZE_KEY_EC` (`DDSC_LOG_BASE` + 718)  
*Failed to deserialize CDR payload key.*
- #define `DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 800)  
*Failed to add a route to an anonymous participant discovery datawriter.*
- #define `DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 801)  
*Failed to add a route to a topic from a datawriter.*
- #define `DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC` (`DDSC_LOG_BASE` + 802)  
*Failed to delete a route to a topic.*
- #define `DDSC_LOG_NETIO_FORWARD_TOPIC_EC` (`DDSC_LOG_BASE` + 803)  
*Failed to forward a topic.*
- #define `DDSC_LOG_NETIO_NETIO_KIND` 1  
*Any NETIO interface kind, typically UDP.*
- #define `DDSC_LOG_INTRA_NETIO_KIND` 2

- The intra interface kind.*

  - #define `DDSC_LOG_RTPS_NETIO_KIND` 3
- The RTPS interface kind.*

  - #define `DDSC_LOG_DATAREADER_NETIO_KIND` 4
- The DataReader interface kind.*

  - #define `DDSC_LOG_DATAWRITER_NETIO_KIND` 5
- The DataWriter interface kind.*

  - #define `DDSC_LOG_NETIO_BIND_EXTERNAL_EC` (`DDSC_LOG_BASE` + 804)

*Failed to bind two external interface of the specified kind.*
- #define `DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC` (`DDSC_LOG_BASE` + 805)

*Failed to unbind two external interfaces of the specified kind.*
- #define `DDSC_LOG_NETIO_BIND_EC` (`DDSC_LOG_BASE` + 806)

*Failed to bind an interface to a peer interface.*
- #define `DDSC_LOG_NETIO_UNBIND_EC` (`DDSC_LOG_BASE` + 807)

*Failed to unbind an interface from a peer interface.*
- #define `DDSC_LOG_NETIO_ADD_ROUTE_EC` (`DDSC_LOG_BASE` + 808)

*Failed to add a route from an interface to a peer interface.*
- #define `DDSC_LOG_NETIO_DELETE_ROUTE_EC` (`DDSC_LOG_BASE` + 809)

*Failed to delete a route from an interface to a peer interface.*
- #define `DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC` (`DDSC_LOG_BASE` + 810)

*Failed to get an external interface for the specified interface kind.*
- #define `DDSC_LOG_NETIO_NO_ROUTE_EC` (`DDSC_LOG_BASE` + 811)

*A DataReader failed a bind due to no existing route.*
- #define `DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC` (`DDSC_LOG_BASE` + 812)

*Lookup a route to a destination failed.*
- #define `DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC` (`DDSC_LOG_BASE` + 813)

*Failed to get the route table for an interface.*
- #define `DDSC_LOG_NETIO_SEND_FAILED_EC` (`DDSC_LOG_BASE` + 814)

*Failure when sending on an interface.*
- #define `DDSC_LOG_NETIO_SET_STATE_EC` (`DDSC_LOG_BASE` + 815)

*Failed to set an interface state.*
- #define `DDSC_LOG_NETIO_PEER_LOOKUP_EC` (`DDSC_LOG_BASE` + 816)

*Datawriter did not find a peer.*
- #define `DDSC_LOG_NETIO_FORCED_REMOVE_EC` (`DDSC_LOG_BASE` + 817)

*Forced removal of sample downstream failed.*
- #define `DDSC_LOG_PACKET_INIT_EC` (`DDSC_LOG_BASE` + 818)

*Failed to initialize a packet.*
- #define `DDSC_LOG_PACKET_SET_HEAD_EC` (`DDSC_LOG_BASE` + 819)

*Failed to set the head of a packet.*
- #define `DDSC_LOG_PACKET_SET_TAIL_EC` (`DDSC_LOG_BASE` + 820)

*Failed to set the tail of a packet.*
- #define `DDSC_LOG_NETIO_DISCOVERY_ENABLED_TRANSPORT_EC` (`DDSC_LOG_BASE` + 821)

*Configured discovery enabled transport is not valid.*
- #define `DDSC_LOG_DW_ACKNACK_FAILED_EC` (`DDSC_LOG_BASE` + 900)

*Failed to ACKNACK sample in the writer history.*
- #define `DDSC_LOG_DW_COMMIT_EC` (`DDSC_LOG_BASE` + 901)

*Failed to commit a sample to the writer queue.*



- #define `DDSC_LOG_DW_KEYHASH_CREATE_EC` (`DDSC_LOG_BASE` + 902)  
*Failed to create keyhash of instance handle.*
- #define `DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC` (`DDSC_LOG_BASE` + 903)  
*Failed a write due to an invalid key kind for the type being written.*
- #define `DDSC_LOG_DW_CREATE_TYPED_WRITER_EC` (`DDSC_LOG_BASE` + 904)  
*Failed to create a typed writer.*
- #define `DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC` (`DDSC_LOG_BASE` + 905)  
*Failed to register the key of an instance.*
- #define `DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC` (`DDSC_LOG_BASE` + 906)  
*Failed to create a flow-controller.*
- #define `DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC` (`DDSC_LOG_BASE` + 907)  
*The data-sample is larger then supported by the transport, but the flow-controller has been compiled out.*
- #define `DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC` (`DDSC_LOG_BASE` + 908)  
*Failed to reserve an instance to publish discovery data for a user created data writer. This typically means Domain↵ ParticipantQos.resource\_limits.remote\_writer\_allocation is too small.*
- #define `DDSC_LOG_DW_INCOMPATIBLE_PADDING_EC` (`DDSC_LOG_BASE` + 909)  
*A DataReader incompatible with padding bits in encapsulation header was discovered.*
- #define `DDSC_LOG_DR_CREATE_TYPED_READER_EC` (`DDSC_LOG_BASE` + 1000)  
*Failed to create a typed datareader.*
- #define `DDSC_LOG_DR_COPY_DATA_SAMPLE_EC` (`DDSC_LOG_BASE` + 1001)  
*Failed to copy a sample upon reception, read, or take.*
- #define `DDSC_LOG_DR_COMMIT_SAMPLE_EC` (`DDSC_LOG_BASE` + 1002)  
*Failed to commit a sample to be made available to be read or taken.*
- #define `DDSC_LOG_DR_FILTER_ERROR_EC` (`DDSC_LOG_BASE` + 1003)  
*A datareader filter function failed.*
- #define `DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC` (`DDSC_LOG_BASE` + 1004)  
*Failed to deserialize a key-hash parameter.*
- #define `DDSC_LOG_DR_GET_ENTRY_FAILED_EC` (`DDSC_LOG_BASE` + 1005)  
*Failed to get a Reader History entry for a received sample.*
- #define `DDSC_LOG_DR_COMMIT_ENTRY_EC` (`DDSC_LOG_BASE` + 1006)  
*Failed to commit a receive sample to Reader History to be read or taken.*
- #define `DDSC_LOG_DR_UNREGISTER_KEY_EC` (`DDSC_LOG_BASE` + 1007)  
*A DataReader failed to unregister an instance.*
- #define `DDSC_LOG_DR_DISPOSE_KEY_EC` (`DDSC_LOG_BASE` + 1008)  
*A DataReader failed to dispose an instance.*
- #define `DDSC_LOG_DR_READ_TAKE_FAILURE_EC` (`DDSC_LOG_BASE` + 1009)  
*A call to a reader/take function failed.*
- #define `DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC` (`DDSC_LOG_BASE` + 1010)  
*Failed to create keyhash of instance handle.*
- #define `DDSC_LOG_DR_INSTANCE_MAPPING_EXHAUSTED_EC` (`DDSC_LOG_BASE` + 1011)  
*Failed to create keyhash of instance handle.*
- #define `DDSC_LOG_DR_INSTANCE_ASSERTION_FAILED_EC` (`DDSC_LOG_BASE` + 1012)  
*Failed to reserve an instance to publish discovery data for a user created data reader. This typically means Domain↵ ParticipantQos.resource\_limits.remote\_reader\_allocation is too small.*
- #define `DDSC_LOG_DR_ON_SAMPLE_REMOVED_EC` (`DDSC_LOG_BASE` + 1013)  
*Failed to remove a sample when it was being pushed out from the Reader History.*
- #define `DDSC_LOG_TYPE_NAME_CMP_EC` (`DDSC_LOG_BASE` + 1100)



- *Two type names are incompatible.*
- #define DDSC\_LOG\_TOPIC\_NAME\_CMP\_EC (DDSC\_LOG\_BASE + 1101)
- *Two topic names are incompatible.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_GET\_SERIALIZED\_SAMPLE\_MAX\_SIZE 1
- *The get\_serialized\_sample\_max\_size function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_SERIALIZE\_DATA 2
- *The serialize\_data function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_DESERIALIZE\_DATA 3
- *The deserialize\_data function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_CREATE\_SAMPLE 4
- *The create\_sample function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_COPY\_SAMPLE 5
- *The copy\_sample function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_DELETE\_SAMPLE 6
- *The delete\_sample function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_GET\_KEY\_KIND 7
- *The get\_key\_kind function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_INSTANCE\_TO\_KEYHASH 8
- *The instance\_to\_keyhash function kind.*
- #define DDSC\_LOG\_TYPE\_FUNCTION\_NULL\_EC (DDSC\_LOG\_BASE + 1102)
- *Invalid type plugin, The specified function pointer is NULL.*
- #define DDSC\_LOG\_TOPIC\_TOO\_LONG\_EC (DDSC\_LOG\_BASE + 1103)
- *Failed to create a topic because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)*
- #define DDSC\_LOG\_TYPE\_TOO\_LONG\_EC (DDSC\_LOG\_BASE + 1104)
- *Failed to create a type because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)*
- #define DDSC\_LOG\_LOOKUP\_TYPE\_PLUGIN\_EC (DDSC\_LOG\_BASE + 1105)
- *A type-plugin for the given type could not be found.*
- #define DDSC\_LOG\_TYPE\_KEY\_TYPE\_EC (DDSC\_LOG\_BASE + 1106)
- *Two types have incompatible keys.*
- #define DDSC\_LOG\_TYPE\_PLUGIN\_GET\_BUFFER\_EC (DDSC\_LOG\_BASE + 1107)
- *Type-plugin cannot return a buffer.*
- #define DDSC\_LOG\_TYPE\_PLUGIN\_INCONSISTENT\_DR\_EC (DDSC\_LOG\_BASE + 1108)
- *Inconsistent type-plugin DataReader.*
- #define DDSC\_LOG\_DUPLICATE\_GUID\_PREFIX\_EC (DDSC\_LOG\_BASE + 1109)
- *A DomainParticipant with the specified GUID already exists in the DomainParticipant factory.*
- #define DDSC\_LOG\_TYPE\_MANAGED\_POOL\_FAILURE\_EC (DDSC\_LOG\_BASE + 1110)
- *Managed pool function failed.*
- #define DDSC\_LOG\_TYPE\_GET\_SAMPLE\_ID\_EC (DDSC\_LOG\_BASE + 1111)
- *Managed pool failed to get sample id.*
- #define DDSC\_LOG\_PARTICIPANT\_NAME\_EMPTY\_EC (DDSC\_LOG\_BASE + 1112)
- *A DomainParticipant with enable\_participant\_discovery\_by\_name set to to TRUE but an empty name is created.*
- #define DDSC\_LOG\_DUPLICATE\_PARTICIPANT\_NAME\_EC (DDSC\_LOG\_BASE + 1113)
- *A DomainParticipant with enable\_participant\_discovery\_by\_name set to to TRUE, but a local participant with the same DomainParticipantQos.participant\_name.name already exists.*
- #define DDSC\_LOG\_DUPLICATE\_REMOTE\_PARTICIPANT\_NAME\_EC (DDSC\_LOG\_BASE + 1114)
- *A remote participant is discovered with the same name as the a local participant name in the same domain ID.*
- #define DDSC\_LOG\_REMOTE\_PARTICIPANT\_NAME\_EMPTY\_EC (DDSC\_LOG\_BASE + 1115)

- A remote participant without a name is discovered.*

  - #define `DDSC_LOG_REMOTE_PARTICIPANT_RESTART_EC` (`DDSC_LOG_BASE` + 1116)
- Failed to reset a remote participant.*

  - #define `DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC` (`DDSC_LOG_BASE` + 1200)
- Discovery plugin failed its update after a local DomainParticipant was enabled.*

  - #define `DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC` (`DDSC_LOG_BASE` + 1201)
- Discovery plugin failed its update before a local DomainParticipant was created.*

  - #define `DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC` (`DDSC_LOG_BASE` + 1202)
- Discovery plugin failed its update after a local DomainParticipant was created.*

  - #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC` (`DDSC_LOG_BASE` + 1203)
- Discovery plugin failed its update after a local DataReader was enabled.*

  - #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC` (`DDSC_LOG_BASE` + 1204)
- Discovery plugin failed its update after a local DataReader was deleted.*

  - #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC` (`DDSC_LOG_BASE` + 1205)
- Discovery plugin failed its update after a local DataWriter was enabled.*

  - #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC` (`DDSC_LOG_BASE` + 1206)
- Discovery plugin failed its update after a local DataWriter was deleted.*

  - #define `DDSC_LOG_DISC_BEFORE_REMOTE_PARTICIPANT_DELETED_EC` (`DDSC_LOG_BASE` + 1207)
- Discovery plugin failed its update after being notified that a remote DomainParticipant was about to be deleted.*

  - #define `DDSC_LOG_DISC_ADD_PEER_EC` (`DDSC_LOG_BASE` + 1208)
- Failed to add a peer with discovery plugin.*

  - #define `DDSC_LOG_DESERIALIZE_UNKNOWN_PID_EC` (`DDSC_LOG_BASE` + 1209)
- Deserialized a parameter of unknown ID.*

  - #define `DDSC_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (`DDSC_LOG_BASE` + 1210)
- Failed to serialize Builtin Endpoint Mask parameter.*

  - #define `DDSC_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (`DDSC_LOG_BASE` + 1211)
- Failed to deserialize Builtin Endpoint Mask parameter.*

  - #define `DDSC_LOG_SERIALIZE_TOPIC_NAME_EC` (`DDSC_LOG_BASE` + 1212)
- Failed to serialize Topic Name parameter.*

  - #define `DDSC_LOG_CDR_SET_OFFSET_EC` (`DDSC_LOG_BASE` + 1213)
- Failed to set current offset of stream.*

  - #define `DDSC_LOG_SERIALIZE_ENTITY_NAME_EC` (`DDSC_LOG_BASE` + 1214)
- Failed to serialize Entity Name parameter.*

  - #define `DDSC_LOG_SERIALIZE_TYPE_NAME_EC` (`DDSC_LOG_BASE` + 1215)
- Failed to serialize Type Name parameter.*

  - #define `DDSC_LOG_SERIALIZE_GUID_EC` (`DDSC_LOG_BASE` + 1216)
- Failed to serialize GUID key.*

  - #define `DDSC_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (`DDSC_LOG_BASE` + 1217)
- Failed to serialize Default Unicast Locator parameter.*

  - #define `DDSC_LOG_DESERIALIZE_LOCATOR_EC` (`DDSC_LOG_BASE` + 1218)
- Failed to deserialize a locator of the specified kind.*

  - #define `DDSC_LOG_LOCATORS_FULL_EC` (`DDSC_LOG_BASE` + 1219)
- A locator sequence is full, the remaining locators of the specified kind are dropped.*

  - #define `DDSC_LOG_UNSUPPORTED_LOCATOR_EC` (`DDSC_LOG_BASE` + 1220)
- The locator kind is not supported by any transport registered with the domain participant and is dropped.*

  - #define `DDSC_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (`DDSC_LOG_BASE` + 1221)
- Failed to serialize Protocol Version parameter.*

- #define `DDSC_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (`DDSC_LOG_BASE` + 1222)  
*Failed to deserialize Protocol Version parameter.*
- #define `DDSC_LOG_DESERIALIZE_VENDOR_ID_EC` (`DDSC_LOG_BASE` + 1223)  
*Failed to deserialize Vendor ID parameter.*
- #define `DDSC_LOG_SERIALIZE_VENDOR_ID_EC` (`DDSC_LOG_BASE` + 1224)  
*Failed to serialize Vendor ID parameter.*
- #define `DDSC_LOG_SERIALIZE_PRODUCT_VERSION_EC` (`DDSC_LOG_BASE` + 1225)  
*Failed to serialize Product Version parameter.*
- #define `DDSC_LOG_SERIALIZE_LEASE_DURATION_EC` (`DDSC_LOG_BASE` + 1226)  
*Failed to serialize Lease Duration parameter.*
- #define `DDSC_LOG_DATAWRITER_ADD_PEER_FAILED_EC` (`DDSC_LOG_BASE` + 1227)  
*Failed to add a remote peer to a local DataWriter.*
- #define `DDSC_LOG_DATAREADER_ADD_PEER_FAILED_EC` (`DDSC_LOG_BASE` + 1228)  
*Failed to add a remote peer to a local DataReader.*
- #define `DDSC_LOG_DISC_REMOTE_PARTICIPANT_REMOVE_EC` (`DDSC_LOG_BASE` + 1229)  
*Failed to remote/reset remote participant after liveliness expired.*
- #define `DDSC_LOG_DESERIALIZE_PARTITION_SEQ_EC` (`DDSC_LOG_BASE` + 1230)  
*Failed to deserialize partition seq.*
- #define `DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC` (`DDSC_LOG_BASE` + 1231)  
*Failed to initialize discovery queue.*
- #define `DDSC_LOG_QUEUE_PUBLICATION_DISCOVERY_EC` (`DDSC_LOG_BASE` + 1232)  
*Failed to queue a publication discovery message.*
- #define `DDSC_LOG_QUEUE_SUBSCRIPTION_DISCOVERY_EC` (`DDSC_LOG_BASE` + 1233)  
*Failed to queue a subscription discovery message.*
- #define `DDSC_LOG_QUEUE_FINALIZE_EC` (`DDSC_LOG_BASE` + 1234)  
*Failed finalize discovery queue.*
- #define `DDSC_LOG_IPC_INIT_FAILED_EC` (`DDSC_LOG_BASE` + 1300)  
*Failed to initialize Builtin IPC entities.*
- #define `DDSC_LOG_IPC_ALLOC_FAILED_EC` (`DDSC_LOG_BASE` + 1301)  
*Failed to allocate memory for one of the Builtin IPC entities.*
- #define `DDSC_LOG_IPC_FINALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1302)  
*Failed to finalize all resources A potential memory leak might have been caused by unexpected errors during finalization of acquired resources.*
- #define `DDSC_LOG_IPC_CHANNEL_TYPE_REGISTER_EC` (`DDSC_LOG_BASE` + 1303)  
*Failed to register type for a Builtin IPC channel.*
- #define `DDSC_LOG_IPC_CHANNEL_TOPIC_CREATE_EC` (`DDSC_LOG_BASE` + 1304)  
*Failed to create topic for a Builtin IPC channel.*
- #define `DDSC_LOG_IPC_CHANNEL_READER_CREATE_EC` (`DDSC_LOG_BASE` + 1305)  
*Failed to create reader for a Builtin IPC channel.*
- #define `DDSC_LOG_IPC_CHANNEL_WRITER_CREATE_EC` (`DDSC_LOG_BASE` + 1306)  
*Failed to create writer for a Builtin IPC channel.*
- #define `DDSC_LOG_IPC_CHANNEL_TYPE_PLUGIN_EC` (`DDSC_LOG_BASE` + 1307)  
*Failed to find type plugin for a Builtin IPC channel.*
- #define `DDSC_LOG_IPC_CHANNEL_UNKNOWN_EC` (`DDSC_LOG_BASE` + 1308)  
*Unknown builtin IPC channel requested.*
- #define `DDSC_LOG_IPC_UPDATE_QOS_FAILED_EC` (`DDSC_LOG_BASE` + 1309)  
*Failed to update DomainParticipantQos to account for built-in IPC endpoints.*

- #define `DDSC_LOG_IPC_CHANNEL_ADD_ANON_ROUTE_EC` (`DDSC_LOG_BASE` + 1310)  
*Failed to add an anonymous route to a built-in DataWriter or DataReader.*
- #define `DDSC_LOG_IPC_CHANNEL_DELETE_ANON_ROUTE_EC` (`DDSC_LOG_BASE` + 1311)  
*Failed to delete an anonymous route to a built-in DataWriter or DataReader.*
- #define `DDSC_LOG_IPC_CHANNEL_ADD_PEER_EC` (`DDSC_LOG_BASE` + 1312)  
*Failed to add an inter-participant channel peer.*
- #define `DDSC_LOG_IPC_CHANNEL_REMOVE_PEER_EC` (`DDSC_LOG_BASE` + 1313)  
*Failed to remove an inter-participant channel peer.*
- #define `DDSC_LOG_IPC_CHANNEL_RETURN_LOAN_EC` (`DDSC_LOG_BASE` + 1314)  
*Failed to return a loaned inter-participant sample.*
- #define `DDSC_LOG_IPC_CHANNEL_PROCESS_SAMPLE_EC` (`DDSC_LOG_BASE` + 1315)  
*Failed to process inter-participant message.*
- #define `DDSC_LOG_IPC_ASSERT_ROUTES_EC` (`DDSC_LOG_BASE` + 1316)  
*Failed to assert inter-participant routes.*
- #define `DDSC_LOG_IPC_CHANNEL_WRITE_SAMPLE_EC` (`DDSC_LOG_BASE` + 1317)  
*Failed to write inter-participant message.*
- #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_HANDSHAKE_FAILED_EC` (`DDSC_LOG_BASE` + 1318)  
*Failed to process handshake authentication.*
- #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1319)  
*Failed to process participant interceptor tokens.*
- #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC` (`DDSC_LOG_BASE` + 1320)  
*Failed to process endpoint interceptor tokens.*
- #define `DDSC_LOG_IPC_CHANNEL_FAILED_PROCESS_AUTH_REQUEST_FAILED_EC` (`DDSC_LOG_BASE` + 1321)  
*Failed to process authentication request.*
- #define `DDSC_LOG_IPC_CHANNEL_CONFIG_INITIALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1322)  
*Failed to initialize inter-participant channel.*
- #define `DDSC_LOG_IPC_CHANNEL_CONFIG_FINALIZE_FAILED_EC` (`DDSC_LOG_BASE` + 1323)
- #define `DDSC_LOG_IPC_CHANNEL_CONFIG_SET_FAILED_EC` (`DDSC_LOG_BASE` + 1324)  
*Failed to get inter-participant channel configuration.*
- #define `DDSC_LOG_IPC_CHANNEL_SET_LISTENERS_FAILED_EC` (`DDSC_LOG_BASE` + 1325)  
*Failed to set inter-participant channel listeners.*
- #define `DDSC_LOG_IPC_CHANNEL_ENABLE_FAILED_EC` (`DDSC_LOG_BASE` + 1326)  
*Failed to enable inter-participant channel entities.*
- #define `DDSC_LOG_IPC_CHANNEL_CREATE_FAILED_EC` (`DDSC_LOG_BASE` + 1327)  
*Failed to create inter-participant channel.*
- #define `DDSC_LOG_TRUST_UNKNOWN_GMCLASSID_EC` (`DDSC_LOG_BASE` + 1401)  
*Unknown generic message class id.*
- #define `DDSC_LOG_TRUST_UNKNOWN_SERVICEID_EC` (`DDSC_LOG_BASE` + 1402)  
*Unknown builtin service id.*
- #define `DDSC_LOG_TRUST_IGNORED_REMOTE_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 1403)  
*Ignored remote participant.*
- #define `DDSC_LOG_TRUST_IGNORED_HANDSHAKE_MESSAGE_EC` (`DDSC_LOG_BASE` + 1404)  
*Ignored handshake message from remote participant.*
- #define `DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC` (`DDSC_LOG_BASE` + 1405)

- Unexpected validation result.*

  - #define DDSC\_LOG\_TRUST\_UNAUTHORIZED\_REMOTE\_PARTICIPANT\_EC (DDSC\_LOG\_BASE + 1406)

*A remote participant failed the authorization process.*
- #define DDSC\_LOG\_TRUST\_FAILED\_VALIDATE\_LOCAL\_IDENTITY\_EC (DDSC\_LOG\_BASE + 1407)

*Failed to validate local identity.*
- #define DDSC\_LOG\_TRUST\_INVALID\_IDENTITY\_HANDLE\_EC (DDSC\_LOG\_BASE + 1408)

*Invalid local trust identity.*
- #define DDSC\_LOG\_TRUST\_GET\_IDENTITY\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1409)

*Failed to get identity token.*
- #define DDSC\_LOG\_TRUST\_INVALID\_PERMISSIONS\_HANDLE\_EC (DDSC\_LOG\_BASE + 1410)

*Invalid local trust permissions.*
- #define DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_GUID\_EC (DDSC\_LOG\_BASE + 1411)

*Invalid DomainParticipant GUID configuration.*
- #define DDSC\_LOG\_TRUST\_GET\_PARTICIPANT\_TRUST\_ATTRIBUTES\_FAILED\_EC (DDSC\_LOG\_BASE + 1412)

*Failed to get DomainParticipant trust attributes.*
- #define DDSC\_LOG\_TRUST\_CHECK\_CREATE\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1413)

*Failed to create access control DomainParticipant.*
- #define DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1414)

*Failed to get permissions credential token.*
- #define DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1415)

*Failed to get permissions token.*
- #define DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_HANDLE\_EC (DDSC\_LOG\_BASE + 1416)

*Invalid interceptor handle.*
- #define DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1417)

*Failed to register local DomainParticipant.*
- #define DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1418)

*Failed to return permissions credential token.*
- #define DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1419)

*Failed to return permissions token.*
- #define DDSC\_LOG\_TRUST\_RETURN\_IDENTITY\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1420)

*Failed to return identity token.*
- #define DDSC\_LOG\_TRUST\_UNREGISTER\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1421)

*Failed to unregister local participant.*
- #define DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_HANDLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1422)

*Failed to return permissions handle.*
- #define DDSC\_LOG\_TRUST\_RETURN\_IDENTITY\_HANDLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1423)

*Failed to return identity handle.*
- #define DDSC\_LOG\_TRUST\_RETURN\_SHAREDSECRET\_HANDLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1424)

*Failed to return shared secret handle.*
- #define DDSC\_LOG\_TRUST\_GET\_AUTHENTICATED\_PEER\_CREDENTIAL\_TOKEN\_FAILED\_EC (DDSC\_LOG\_BASE + 1425)

*Failed to get authenticated peer credential token.*
- #define DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1426)

*Failed to check access control remote DomainParticipant.*

- `#define DDSC_LOG_TRUST_GET_SHARED_SECRET_FAILED_EC (DDSC_LOG_BASE + 1427)`  
*Failed to get shared secret.*
- `#define DDSC_LOG_TRUST_PREPARE_LOCAL_PARTICIPANT_STATE_FAILED_EC (DDSC_LOG_BASE + 1428)`  
*Failed to validate local DomainParticipant trust.*
- `#define DDSC_LOG_TRUST_DELETE_TRUST_PLUGINS_EC (DDSC_LOG_BASE + 1429)`  
*Failed to delete trust plugins.*
- `#define DDSC_LOG_TRUST_SET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1430)`  
*Failed to set permissions credential and token.*
- `#define DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_PARTICIPANT_FAILED_EC (DDSC_LOG_BASE + 1431)`  
*Failed to register matched remote DomainParticipant.*
- `#define DDSC_LOG_TRUST_GET_TOPIC_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1432)`  
*Failed to get topic trust attributes.*
- `#define DDSC_LOG_TRUST_CHECK_CREATE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1433)`  
*Failed to create trust check DataWriter.*
- `#define DDSC_LOG_TRUST_RETURN_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1434)`  
*Failed to return DataWriter trust attributes.*
- `#define DDSC_LOG_TRUST_GET_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1435)`  
*Failed to get DataWriter trust attributes.*
- `#define DDSC_LOG_TRUST_GET_DATAREADER_TRUST_ATTRIBUTES_FAILED_EC (DDSC_LOG_BASE + 1436)`  
*Failed to get DataReader trust attributes.*
- `#define DDSC_LOG_TRUST_CHECK_CREATE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1437)`  
*Failed to create trust check DataReader.*
- `#define DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REPLY_EC (DDSC_LOG_BASE + 1438)`  
*Handshake begin reply failed.*
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_PARTICIPANT_INTERCEPTOR_TOKENS_EC (DDSC_LOG_BASE + 1439)`  
*Failed to create local DomainParticipant interceptor tokens.*
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_DW_TOKEN_EC (DDSC_LOG_BASE + 1440)`  
*Failed to create local DataWriter interceptor token.*
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_DR_TOKEN_EC (DDSC_LOG_BASE + 1441)`  
*Failed to create local DataReader interceptor token.*
- `#define DDSC_LOG_TRUST_RETURN_HANDSHAKE_HANDLE_FAILED_EC (DDSC_LOG_BASE + 1442)`  
*Failed to return handshake handle.*
- `#define DDSC_LOG_TRUST_SET_REMOTE_PARTICIPANT_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1443)`  
*Failed to set remote DomainParticipant interceptor tokens.*
- `#define DDSC_LOG_TRUST_SET_REMOTE_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1444)`  
*Failed to set remote DataReader interceptor tokens.*
- `#define DDSC_LOG_TRUST_SET_REMOTE_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1445)`  
*Failed to set remote DataWriter interceptor tokens.*

- #define `DDSC_LOG_TRUST_REGISTER_LOCAL_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1446)  
*Failed to register local DataReader.*
- #define `DDSC_LOG_TRUST_REGISTER_LOCAL_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1447)  
*Failed to register local DataWriter.*
- #define `DDSC_LOG_TRUST_CHECK_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1448)  
*Failed to check remote DataWriter.*
- #define `DDSC_LOG_TRUST_CHECK_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1449)  
*Failed to check remote DataReader.*
- #define `DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1450)  
*Failed to register matched remote DataWriter.*
- #define `DDSC_LOG_TRUST_REGISTER_MATCHED_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1451)  
*Failed to register matched remote DataReader.*
- #define `DDSC_LOG_TRUST_UNREGISTER_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1452)  
*Failed to unregister DataReader.*
- #define `DDSC_LOG_TRUST_UNREGISTER_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1453)  
*Failed to unregister DataWriter.*
- #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_IDENTITY_EC` (`DDSC_LOG_BASE` + 1454)  
*Failed to validate remote identity.*
- #define `DDSC_LOG_TRUST_BEGIN_HANDSHAKE_REQUEST_EC` (`DDSC_LOG_BASE` + 1455)  
*Handshake begin request failed.*
- #define `DDSC_LOG_TRUST_PROCESS_HANDSHAKE_EC` (`DDSC_LOG_BASE` + 1456)  
*Handshake process failed.*
- #define `DDSC_LOG_TRUST_CREATE_TRUST_PLUGINS_FAILED_EC` (`DDSC_LOG_BASE` + 1457)  
*Failed to create trust plugins.*
- #define `DDSC_LOG_TRUST_PARSE_PROPERTY_FAILED_EC` (`DDSC_LOG_BASE` + 1458)  
*Failed to parse trust property.*
- #define `DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_READY_FAILED_EC` (`DDSC_LOG_BASE` + 1459)  
*Error after remote trust DomainParticipant is ready.*
- #define `DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_EC` (`DDSC_LOG_BASE` + 1460)  
*Failed to remove remote DomainParticipant.*
- #define `DDS_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC` (`DDSC_LOG_BASE` + 1461)  
*Failed to assert DomainParticipant generic message.*
- #define `DDSC_LOG_TRUST_SEND_HANDSHAKE_EC` (`DDSC_LOG_BASE` + 1462)  
*Failed to send handshake message.*
- #define `DDSC_LOG_VALIDATE_REMOTE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE` + 1463)  
*Failed to validate trust remote DomainParticipant.*
- #define `DDSC_LOG_TRUST_BIND_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1464)  
*Failed to get remote DataReader bind properties.*
- #define `DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC` (`DDSC_LOG_BASE` + 1465)  
*Failed to register remote DataReader.*
- #define `DDSC_LOG_TRUST_BIND_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1466)  
*Failed to get remote DataWriter bind properties.*
- #define `DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC` (`DDSC_LOG_BASE` + 1467)



- Failed to register remote DataWriter.*

  - #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAWRITER_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1468`)
- Failed to validate remote DataWriter.*

  - #define `DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAREADER_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1469`)
- Failed to validate remote DataReader.*

  - #define `DDSC_LOG_TRUST_VALIDATE_LOCAL_DATAWRITER_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1470`)
- Failed to validate local DataWriter.*

  - #define `DDSC_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC` (`DDSC_LOG_BASE + 1471`)
- Invalid DomainParticipant trust generic message.*

  - #define `DDSC_LOG_TRUST_PREPARE_AUTH_REQUEST_EC` (`DDSC_LOG_BASE + 1472`)
- Failed to prepare authentication request.*

  - #define `DDSC_LOG_TRUST_PREPARE_AUTH_HANDSHAKE_MESSAGE_EC` (`DDSC_LOG_BASE + 1473`)
- Failed to prepare authentication handshake message.*

  - #define `DDSC_LOG_TRUST_PREPARE_ENDPOINT_INTERCEPTOR_MESSAGE_EC` (`DDSC_LOG_BASE + 1474`)
- Failed to prepare endpoint interceptor message.*

  - #define `DDSC_LOG_TRUST_INVALID_GENERIC_MESSAGE_REPLY_EC` (`DDSC_LOG_BASE + 1475`)
- Invalid trust generic message reply.*

  - #define `DDSC_LOG_FINALIZE_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1476`)
- Failed to finalize trust.*

  - #define `DDSC_LOG_TRUST_DATA_HOLDER_COPY_FAILED_EC` (`DDSC_LOG_BASE + 1477`)
- Failed to copy trust data holder.*

  - #define `DDSC_LOG_TRUST_INVALID_SAMPLE_MAX_SERIALIZED_SIZE_EC` (`DDSC_LOG_BASE + 1478`)
- Invalid sample serialized size.*

  - #define `DDSC_LOG_TRUST_ALLOC_FAILED_EC` (`DDSC_LOG_BASE + 1479`)
- Failed to allocate memory.*

  - #define `DDSC_LOG_TRUST_SERIALIZE_DATA_FAILED_EC` (`DDSC_LOG_BASE + 1480`)
- Failed to serialize trust data.*

  - #define `DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1481`)
- Failed to initialize trust.*

  - #define `DDSC_LOG_TRUST_INITIALIZE_PARTICIPANT_BUILTIN_DATA_FAILED_EC` (`DDSC_LOG_BASE + 1482`)
- Failed to initialize DomainParticipant trust data.*

  - #define `DDSC_LOG_TRUST_INVALID_PARTICIPANT_GENERIC_MESSAGE_EC` (`DDSC_LOG_BASE + 1483`)
- Invalid DomainParticipant trust generic message.*

  - #define `DDSC_LOG_TRUST_ADVANCE_SN_FAILED_EC` (`DDSC_LOG_BASE + 1484`)
- Failed to advance trust sequence number.*

  - #define `DDSC_LOG_TRUST_WRITE_SAMPLE_FAILED_EC` (`DDSC_LOG_BASE + 1485`)
- Failed to write sample.*

  - #define `DDSC_LOG_TRUST_RESEND_HANDSHAKE_IN_PROGRESS_EC` (`DDSC_LOG_BASE + 1486`)
- Failed to resend handshake because it is already in progress.*

  - #define `DDSC_LOG_TRUST_RESEND_HANDSHAKE_STOP_FAILED_EC` (`DDSC_LOG_BASE + 1487`)
- Failed to stop periodic handshake.*



- `#define DDSC_LOG_TRUST_RELEASE_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1488)`  
*Failed to release DomainParticipant trust generic message.*
- `#define DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_START_FAILED_EC (DDSC_LOG_BASE + 1489)`  
*Failed to start periodic handshake.*
- `#define DDSC_LOG_TRUST_UNEXPECTED_REMOTE_PARTICIPANT_STATUS_EC (DDSC_LOG_BASE + 1490)`  
*Invalid remote DomainParticipant trust status.*
- `#define DDSC_LOG_TRUST_SEND_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1491)`  
*Fail to send DomainParticipant interceptor state.*
- `#define DDSC_LOG_TRUST_PARTICIPANT_AUTHENTICATION_FAILED_EC (DDSC_LOG_BASE + 1492)`  
*DomainParticipant authentication failed.*
- `#define DDSC_LOG_TRUST_ASSERT_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1493)`  
*Failed to assert DomainParticipant generic message.*
- `#define DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC (DDSC_LOG_BASE + 1494)`  
*Failed to send handshake message.*
- `#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_EC (DDSC_LOG_BASE + 1495)`  
*Invalid interceptor tokens.*
- `#define DDSC_LOG_TRUST_INVALID_INTERCEPTOR_TOKENS_DESTINATION_EC (DDSC_LOG_BASE + 1496)`  
*Invalid interceptor token destination.*
- `#define DDSC_LOG_TRUST_ENDPOINT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC (DDSC_LOG_BASE + 1497)`  
*Failed to copy endpoint trust attributes.*
- `#define DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC (DDSC_LOG_BASE + 1498)`  
*Failed to send tokens to remote DomainParticipant.*
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_DATAWRITER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1499)`  
*Failed to create tokens for the remote DataReader.*
- `#define DDSC_LOG_TRUST_CREATE_LOCAL_DATAREADER_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1500)`  
*Failed to create tokens for the remote DataWriter.*
- `#define DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1501)`  
*Failed to return interceptor tokens.*
- `#define DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_CREDENTIAL_TOKEN_FAILED_EC (DDSC_LOG_BASE + 1502)`  
*Failed to reutrn authenticated peer credential token.*
- `#define DDSC_LOG_TRUST_PROCESS_REMOTE_INTERCEPTOR_TOKENS_FAILED_EC (DDSC_LOG_BASE + 1503)`  
*Failed to process interceptor tokens.*
- `#define DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAWRITER_FAILED_EC (DDSC_LOG_BASE + 1504)`  
*Failed to unregister remote DataWriter.*
- `#define DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAREADER_FAILED_EC (DDSC_LOG_BASE + 1505)`

- Failed to unregister remote DataReader.*

  - #define `DDSC_LOG_TRUST_INVALID_SHARED_SECRET_EC` (`DDSC_LOG_BASE + 1506`)

*Invalid shared secret.*
- #define `DDSC_LOG_TRUST_PARTICIPANT_UNEXPECTED_AUTHENTICATION_ERROR_EC` (`DDSC_LOG_BASE + 1507`)

*Unexpected DomainParticipant authentication error.*
- #define `DDSC_LOG_TRUST_PREPARE_LOCAL_DP_TOKENS_EC` (`DDSC_LOG_BASE + 1508`)

*Failed to prepare local DomainParticipant interceptor tokens.*
- #define `DDSC_LOG_TRUST_ASSERT_PROPERTY_FAILED_EC` (`DDSC_LOG_BASE + 1509`)

*Failed to assert trust property.*
- #define `DDSC_LOG_TRUST_PARTICIPANT_INTERNAL_ATTRIBUTES_CREATE_FAILED_EC` (`DDSC_LOG_BASE + 1510`)

*Failed to copy DomainParticipant trust attributes.*
- #define `DDSC_LOG_TRUST_INVALID_PARTICIPANT_INTERNAL_ATTRIBUTES_EC` (`DDSC_LOG_BASE + 1511`)

*Invalid DomainParticipant trust attributes.*
- #define `DDSC_LOG_TRUST_TRANSFORM_OUTGOING_SERIALIZED_PAYLOAD_FAILED_EC` (`DDSC_LOG_BASE + 1512`)

*Failed to encode outgoing serialized payload.*
- #define `DDSC_LOG_TRUST_TRANSFORM_INCOMING_SERIALIZED_PAYLOAD_FAILED_EC` (`DDSC_LOG_BASE + 1513`)

*Failed to decode incoming serialized payload.*
- #define `DDSC_LOG_TRUST_FINALIZE_AC_PLUGIN_PROPERTIES_FAILED_EC` (`DDSC_LOG_BASE + 1514`)

*Failed to finalize trust properties.*
- #define `DDSC_LOG_TRUST_AFTER_REMOTE_PARTICIPANT_AUTHENTICATED_FAILED_EC` (`DDSC_LOG_BASE + 1515`)

*Error after remote trust DomainParticipant is authenticated.*
- #define `DDSC_LOG_TRUST_CACHE_REMOTE_PARTICIPANT_SAMPLE_FAILED_EC` (`DDSC_LOG_BASE + 1516`)

*Failed to copy remote DomainParticipant sample.*
- #define `DDSC_LOG_TRUST_FINALIZE_AUTH_HANDSHAKE_STATE_FAILED_EC` (`DDSC_LOG_BASE + 1517`)

*Failed to finalize authentication handshake.*
- #define `DDSC_LOG_TRUST_FINALIZE_REMOTE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1518`)

*Failed to finalize remote trust DomainParticipant record.*
- #define `DDSC_LOG_TRUST_RESET_REMOTE_PARTICIPANT_TRUST_FAILED_EC` (`DDSC_LOG_BASE + 1519`)

*Failed to finalize remote trust DomainParticipant.*
- #define `DDSC_LOG_TRUST_ASSERT_AUTH_HANDSHAKE_STATE_FAILED_EC` (`DDSC_LOG_BASE + 1520`)

*Failed to assert authentication handshake state.*
- #define `DDSC_LOG_TRUST_INVALID_AUTH_HANDSHAKE_STATE_EC` (`DDSC_LOG_BASE + 1521`)

*Invalid authentication handshake state.*
- #define `DDSC_LOG_TRUST_SEND_AUTH_REQUEST_FAILED_EC` (`DDSC_LOG_BASE + 1522`)

*Failed to send authentication request.*
- #define `DDSC_LOG_TRUST_GET_AUTH_HANDSHAKE_STATE_FAILED_EC` (`DDSC_LOG_BASE + 1523`)

*Failed to get authentication handshake state.*

- #define DDSC\_LOG\_TRUST\_SET\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC (DDSC\_LOG\_BASE + 1524)  
*Failed to set authentication handshake state.*
- #define DDSC\_LOG\_TRUST\_REAUTH\_REMOTE\_PARTICIPANT\_FAILED\_EC (DDSC\_LOG\_BASE + 1525)  
*Failed to reauthenticate a remote DomainParticipant.*
- #define DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_MESSAGE\_FAILED\_EC (DDSC\_LOG\_BASE + 1526)  
*Failed to resend handshake message.*
- #define DDSC\_LOG\_TRUST\_PBUFS\_MAX\_EXCEEDED\_EC (DDSC\_LOG\_BASE + 1527)  
*Maximum number of buffers exceeded.*
- #define DDSC\_LOG\_TRUST\_CREATE\_TRUST\_PLUGIN\_EC (DDSC\_LOG\_BASE + 1528)  
*Failed to create trust plugin.*
- #define DDSC\_LOG\_TRUST\_LOOKUP\_FACTORY\_FAILED\_EC (DDSC\_LOG\_BASE + 1529)  
*Failed to lookup trust plugin factory.*
- #define DDSC\_LOG\_NAME\_LOOKUP\_EC (DDSC\_LOG\_BASE + 1700)  
*Fully qualified name does not contain substring "::.".*
- #define DDSC\_LOG\_ENTITY\_NAME\_TOO\_LONG\_EC (DDSC\_LOG\_BASE + 1701)  
*Failed to get an entity because the specified name exceeds the maximum length of 255 octets (excluding the terminating NUL)*
- #define DDSC\_LOG\_MESSAGE\_DATA\_UNKNOWN\_LIVENESS\_KIND\_EC (DDSC\_LOG\_BASE + 1800)  
*Unknown inter-participant message liveliness kind received.*
- #define DDSC\_LOG\_MESSAGE\_DATA\_WRONG\_LIVENESS\_KIND\_EC (DDSC\_LOG\_BASE + 1801)  
*Wrong inter-participant message liveliness kind received.*
- #define DDSC\_LOG\_UPDATE\_LEASE\_DURATION\_TIMER\_EC (DDSC\_LOG\_BASE + 1802)  
*Failed to update DataWriter lease duration.*
- #define DDSC\_LOG\_PARTMESSAGE\_WRITE\_SAMPLE\_FAILED\_EC (DDSC\_LOG\_BASE + 1803)  
*Failed to write inter-participant liveliness message.*
- #define DDSC\_LOG\_SEND\_LIVENESS\_FAILED\_EC (DDSC\_LOG\_BASE + 1804)  
*Failed to send inter-participant liveliness message.*
- #define DDSC\_LOG\_UPDATE\_LIVENESS\_FAILED\_EC (DDSC\_LOG\_BASE + 1805)  
*Failed to update DataWriter liveliness.*
- #define DDSC\_LOG\_REFRESH\_LIVENESS\_FAILED\_EC (DDSC\_LOG\_BASE + 1806)  
*Failed to refresh endpoint liveliness.*
- #define DDSC\_LOG\_TRUST\_INCOMPATIBLE\_ENDPOINT\_TRUST\_ATTRIBUTES\_EC (DDSC\_LOG\_BASE + 1807)  
*Two security endpoints are incompatible.*
- #define DDSC\_LOG\_FLOW\_CONTROLLER\_CREATE\_EC (DDSC\_LOG\_BASE + 1900)  
*Failed to create flow controller.*
- #define DDSC\_LOG\_FLOW\_CONTROLLER\_DELETE\_EC (DDSC\_LOG\_BASE + 1901)  
*Failed to delete flow controller.*
- #define DDSC\_LOG\_XTYPES\_LOANED\_SAMPLE\_STATE\_ERROR\_EC (DDSC\_LOG\_BASE + 2000)  
*Extensible types sample state error.*
- #define DDSC\_LOG\_INTERPRETER\_SUPPORT\_UNEXPECTED\_ERROR\_EC (DDSC\_LOG\_BASE + 2001)  
*Interpreter unexpected error.*
- #define DDSC\_LOG\_MEMORY\_MANAGER\_OUT\_OF\_RESOURCES\_EC (DDSC\_LOG\_BASE + 2002)  
*Memory manager out of resources.*
- #define DDSC\_LOG\_MEMORY\_MANAGER\_NOT\_OWNER\_EC (DDSC\_LOG\_BASE + 2003)  
*Memory manager is not the owner of the data passed in.*
- #define DDSC\_LOG\_MEMORY\_MANAGER\_DELETE\_EC (DDSC\_LOG\_BASE + 2004)

- The wire manager was not able to delete a resource.*

  - #define `DDSC_LOG_BUFFER_MAX_EXCEEDED_EC` (`DDSC_LOG_BASE` + 2101)

*Mamimum size of buffer excedeed.*
- #define `DDSC_LOG_INVALID_BUFFER_LENGTH_EC` (`DDSC_LOG_BASE` + 2102)

*Invalid buffer length.*
- #define `DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC` (`DDSC_LOG_BASE` + 2201)

*Matching error because Presentation QoS is not compatible.*
- #define `DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC` (`DDSC_LOG_BASE` + 2202)

*Two endpoints did not match because of incompatible partition QoS.*
- #define `DDSC_LOG_CHECKSUM_REQUIRED_EC` (`DDSC_LOG_BASE` + 2301)

*A participant requires checksum, but the discovered participant does not send a checksum.*
- #define `DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC` (`DDSC_LOG_BASE` + 2302)

*A participant does not support checksum's sent by a discovered participant.*
- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC` (`DDSC_LOG_BASE` + 2303)

*The checksum class id for a custom checksum does match.*
- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC` (`DDSC_LOG_BASE` + 2304)

*The computed\_crc\_kind is inconsistent with what is allowed or supported.*
- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC` (`DDSC_LOG_BASE` + 2305)

*The allowed\_crc\_mask is inconsistent with what is supported.*
- #define `DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROP_EC` (`DDSC_LOG_BASE` + 2306)

*The token bucket properties for the flow controller are inconsistent.*
- #define `DDSC_LOG_LOCK_EC` (`DDSC_LOG_BASE` + 2401)

*Failed to take a lock.*
- #define `DDSC_LOG_UNLOCK_EC` (`DDSC_LOG_BASE` + 2402)

*Failed to give a lock.*
- #define `DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC` (`DDSC_LOG_BASE` + 2501)

*Failed to find the filter plugin factory specified in a DomainParticipant's QoS.*
- #define `DDSC_LOG_ENABLE_WRITER_FILTERING_EC` (`DDSC_LOG_BASE` + 2502)

*A local writer failed to enable writer filtering for a reader.*
- #define `DDSC_LOG_CREATE_WRITER_FILTER_EC` (`DDSC_LOG_BASE` + 2503)

*A local writer failed to create its writer filter.*
- #define `DDSC_LOG_EVALUATE_WRITER_FILTER_EC` (`DDSC_LOG_BASE` + 2504)

*A local writer failed to evaluate a sample against the filters of its matched readers.*
- #define `DDSC_LOG_APPLY_READER_FILTER_EC` (`DDSC_LOG_BASE` + 2505)

*A local writer failed to check if a reader had filtered a sample.*
- #define `DDSC_LOG_COMPILE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2506)

*Failed to compile the content filter configured in a DataReader's QoS.*
- #define `DDSC_LOG_CONTENT_FILTERING_NOT_ENABLED_EC` (`DDSC_LOG_BASE` + 2507)

*Reader configured a content filter but content filtering is not enabled.*
- #define `DDSC_LOG_EVALUATE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2508)

*Failed to evaluate a sample against a reader's content filter.*
- #define `DDSC_LOG_DESERIALIZE_CONTENT_FILTER_INFO_EC` (`DDSC_LOG_BASE` + 2509)

*Failed to deserialize the content filter info from the in-line QoS of a sample.*
- #define `DDSC_LOG_CHANGE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2510)

*Failed to change the configured content filter of a DataReader.*
- #define `DDSC_LOG_UPDATE_REMOTE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2511)

*Failed to update the content filter of a remote subscription.*

- #define `DDSC_LOG_CREATE_FILTER_PLUGIN_EC` (`DDSC_LOG_BASE` + 2512)  
*DomainParticipant failed to create its filter plugin.*
- #define `DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC` (`DDSC_LOG_BASE` + 2513)  
*DomainParticipant failed to finalize its filter plugin.*
- #define `DDSC_LOG_DESERIALIZE_REMOTE_CONTENT_FILTER_EC` (`DDSC_LOG_BASE` + 2514)  
*Failed to deserialize the content filter property in the subscription built-in topic data for a remote subscription.*

### 8.6.1 Detailed Description

DDS\_C module log codes.

## 8.7 dds\_filter\_log.h File Reference

DDS Filter module log codes.

```
#include "osapi/osapi_log.h"
```

### Macros

- #define `DDS_FILTER_LOG_FILTER_CLASS_RECORD` 1  
*Database record for a Filter class.*
- #define `DDS_FILTER_LOG_COMPILED_FILTER_RECORD` 2  
*Database record for a compiled filter.*
- #define `DDS_FILTER_LOG_READER_RECORD` 3  
*Database record for a reader entry.*
- #define `DDS_FILTER_LOG_ROUTE_RECORD` 4  
*Database record for a route entry.*
- #define `DDS_FILTER_LOG_RECORD_SELECT_EC` (`DDS_FILTER_LOG_BASE` + 1)  
*A database select on the specified record kind failed.*
- #define `DDS_FILTER_LOG_RECORD_CREATE_EC` (`DDS_FILTER_LOG_BASE` + 2)  
*Failed to create a database record of the specified kind.*
- #define `DDS_FILTER_LOG_RECORD_DELETE_EC` (`DDS_FILTER_LOG_BASE` + 3)  
*Failed to delete a database record of the specified kind.*
- #define `DDS_FILTER_LOG_RECORD_INSERT_EC` (`DDS_FILTER_LOG_BASE` + 4)  
*Failed to insert a database record of the specified kind.*
- #define `DDS_FILTER_LOG_RECORD_REMOVE_EC` (`DDS_FILTER_LOG_BASE` + 5)  
*Failed to remove a database record of the specified kind.*
- #define `DDS_FILTER_LOG_TABLE_CREATE_EC` (`DDS_FILTER_LOG_BASE` + 6)  
*Failed to create a database table of the specified kind.*
- #define `DDS_FILTER_LOG_TABLE_DELETE_EC` (`DDS_FILTER_LOG_BASE` + 7)  
*Failed to delete a database table of the specified kind.*
- #define `DDS_FILTER_LOG_FILTER_CREATE_INSTANCE_EC` (`DDS_FILTER_LOG_BASE` + 10)  
*Failed to create an instance of the specified filter class.*
- #define `DDS_FILTER_LOG_FILTER_DELETE_INSTANCE_EC` (`DDS_FILTER_LOG_BASE` + 11)

- Failed to delete an instance of the specified filter class.*

  - #define `DDS_FILTER_LOG_FILTER_COMPILE_EC` (`DDS_FILTER_LOG_BASE + 12`)
- Failed to compile a filter of the specified class.*

  - #define `DDS_FILTER_LOG_FILTER_EVALUATE_EC` (`DDS_FILTER_LOG_BASE + 13`)
- Failed to evaluate a filter of the specified class.*

  - #define `DDS_FILTER_LOG_WRITER_DETACH_EC` (`DDS_FILTER_LOG_BASE + 14`)
- Failed to detach a writer because it still has remote readers attached.*

  - #define `DDS_FILTER_LOG_FILTER_EXPRESSION_MAX_COUNT_EC` (`DDS_FILTER_LOG_BASE + 15`)
- No more filters can be compiled because the configured maximum number of filter expressions has been reached.*

  - #define `DDS_FILTER_LOG_FILTER_CLASS_REGISTER_EC` (`DDS_FILTER_LOG_BASE + 20`)
- Failed to register a filter class because one with the same name has already been registered.*

  - #define `DDS_FILTER_LOG_FILTER_CLASS_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE + 21`)
- Failed to find a filter class registered with the specified name.*

  - #define `DDS_FILTER_LOG_FILTER_CLASS_UNREGISTER_EC` (`DDS_FILTER_LOG_BASE + 22`)
- Failed to unregister a filter class because it is still in use.*

  - #define `DDS_FILTER_LOG_CONTENT_FILTERED_TOPIC_NAME_STRING` 1
- Content filter topic name string.*

  - #define `DDS_FILTER_LOG_RELATED_TOPIC_NAME_STRING` 2
- Content filter related topic name string.*

  - #define `DDS_FILTER_LOG_FILTER_CLASS_NAME_STRING` 3
- Filter class name string.*

  - #define `DDS_FILTER_LOG_FILTER_EXPRESSION_STRING` 4
- Filter expression string.*

  - #define `DDS_FILTER_LOG_FILTER_EXPRESSION_PARAMETER_STRING` 5
- Filter expression parameter string.*

  - #define `DDS_FILTER_LOG_CONTENT_FILTER_NAME_STRING` 6
- Content filter QoS expression name string.*

  - #define `DDS_FILTER_LOG_STRING_ASSERT_EC` (`DDS_FILTER_LOG_BASE + 30`)
- Failed to assert a string in the string manager of a specified kind.*

  - #define `DDS_FILTER_LOG_STRING_DELETE_EC` (`DDS_FILTER_LOG_BASE + 31`)
- Failed to delete a string in the string manager of a specified kind.*

  - #define `DDS_FILTER_LOG_CONTENT_FILTER_NAME_TOO_LONG_EC` (`DDS_FILTER_LOG_BASE + 32`)
- Failed to construct a content filter topic name from the combination of the related topic name and the filter name because it exceeded the maximum topic name length.*

  - #define `DDS_FILTER_LOG_STRING_MANAGER_CREATE_EC` (`DDS_FILTER_LOG_BASE + 33`)
- Failed to create a string manager for a specified string kind, max\_string\_size, and memory\_allocation.*

  - #define `DDS_FILTER_LOG_FILTER_PARAMETER_COUNT_EC` (`DDS_FILTER_LOG_BASE + 34`)
- Failed to deserialize a content filter property because it contained too many expression parameters.*

  - #define `DDS_FILTER_LOG_TYPE_CODE_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE + 40`)
- Content filtering is not supported on this data type because its type support does not include a type code.*

  - #define `DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_BUFFER` 1
- Heap memory for a filter plugin factory instance.*

  - #define `DDS_FILTER_LOG_FILTER_PLUGIN_BUFFER` 2
- Heap memory for a filter plugin instance.*

  - #define `DDS_FILTER_LOG_WRITER_FILTER_BUFFER` 3
- Heap memory for a writer filter instance.*

  - #define `DDS_FILTER_LOG_INLINE_QOS_BUFFER` 4

- Heap memory for a writer to serialize a sample's in-line QoS.*

  - #define `DDS_FILTER_LOG_HEAP_ALLOC_EC` (`DDS_FILTER_LOG_BASE` + 50)
- Failed to allocate heap memory for a buffer of a given type.*

  - #define `DDS_FILTER_LOG_FILTER_PLUGIN_INIT_EC` (`DDS_FILTER_LOG_BASE` + 51)
- Failed to initialize a filter plugin.*

  - #define `DDS_FILTER_LOG_FILTER_PLUGIN_FINALIZE_EC` (`DDS_FILTER_LOG_BASE` + 52)
- Failed to finalize a filter plugin.*

  - #define `DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_IN_USE_EC` (`DDS_FILTER_LOG_BASE` + 53)
- Failed to finalize a filter plugin factory because it is still in use.*

  - #define `DDS_FILTER_LOG_WRITER_FILTER_COMPILE_EC` (`DDS_FILTER_LOG_BASE` + 61)
- A writer filter failed to compile a filter for a discovered reader.*

  - #define `DDS_FILTER_LOG_WRITER_FILTER_REMOVE_READER_EC` (`DDS_FILTER_LOG_BASE` + 62)
- Failed to remove a reader from a writer filter because it still has active routes.*

  - #define `DDS_FILTER_LOG_WRITER_FILTER_SEND_GAP_EC` (`DDS_FILTER_LOG_BASE` + 63)
- Writer failed to send a reader a GAP to indicate filtered data.*

  - #define `DDS_FILTER_LOG_WRITER_SERIALIZE_INLINE_QOS_EC` (`DDS_FILTER_LOG_BASE` + 64)
- Writer failed to serialize the content filter info into the in-line QoS.*

  - #define `DDS_FILTER_LOG_WRITER_FILTER_EVALUATE_OUT_OF_ORDER_EC` (`DDS_FILTER_LOG_BASE` + 65)
- Writer filter was evaluated out of order which requires invalidating a previously results.*

  - #define `DDS_FILTER_LOG_WRITER_FILTER_STORE_RESULT_EC` (`DDS_FILTER_LOG_BASE` + 66)
- Writer filter unexpectedly failed to store the result of a filter evaluation.*

  - #define `DDS_FILTER_LOG_SQL_UNSUPPORTED_TYPE_CODE_EC` (`DDS_FILTER_LOG_BASE` + 100)
- Unsupported type code for SQL filtering.*

  - #define `DDS_FILTER_LOG_SQL_UNEXPECTED_TOKEN_EC` (`DDS_FILTER_LOG_BASE` + 101)
- Found an unexpected token while parsing an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_MAX_PREDICATES_EXCEEDED_EC` (`DDS_FILTER_LOG_BASE` + 102)
- Failed to parse an SQL filter expression because the number of predicates in the expression exceeded the configured maximum.*

  - #define `DDS_FILTER_LOG_SQL_INVALID_TOKEN_EC` (`DDS_FILTER_LOG_BASE` + 103)
- Found an invalid token while parsing an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_PARSE_PARAMETER_EC` (`DDS_FILTER_LOG_BASE` + 104)
- Failed to parse a parameter in an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_FIELD_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 105)
- Failed to parse an SQL filter expression because a given field name did not match any field in the type code of the data being filtered.*

  - #define `DDS_FILTER_LOG_SQL_UNSUPPORTED_FIELD_TYPE_EC` (`DDS_FILTER_LOG_BASE` + 106)
- A field specified in an SQL filter expression is not a supported type for use in an SQL filter.*

  - #define `DDS_FILTER_LOG_SQL_ENUM_LABEL_NOT_FOUND_EC` (`DDS_FILTER_LOG_BASE` + 107)
- Failed to find a given enum label in an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_INCOMPATIBLE_TYPES_EC` (`DDS_FILTER_LOG_BASE` + 108)
- Comparison between two incompatible types in an SQL filter expression.*

  - #define `DDS_FILTER_LOG_SQL_INTEGER_OUT_OF_RANGE_EC` (`DDS_FILTER_LOG_BASE` + 109)
- Integer constant in an SQL expression is out of range for the type.*

### 8.7.1 Detailed Description

DDS Filter module log codes.

### 8.7.2 Macro Definition Documentation

#### DDS\_FILTER\_LOG\_TYPE\_CODE\_NOT\_FOUND\_EC

```
#define DDS_FILTER_LOG_TYPE_CODE_NOT_FOUND_EC (DDS_FILTER_LOG_BASE + 40)
```

Content filtering is not supported on this data type because its type support does not include a type code.

Content filtering requires the type code to be able to evaluate the filter expression. The type code can be automatically generated by rtiddsgen either by annotating the type in its IDL with @interpreted(true) or by using the "-interpreted 1" flag.

#### DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_FACTORY\_BUFFER

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_FACTORY_BUFFER 1
```

Heap memory for a filter plugin factory instance.

#### DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_BUFFER

```
#define DDS_FILTER_LOG_FILTER_PLUGIN_BUFFER 2
```

Heap memory for a filter plugin instance.

#### DDS\_FILTER\_LOG\_WRITER\_FILTER\_BUFFER

```
#define DDS_FILTER_LOG_WRITER_FILTER_BUFFER 3
```

Heap memory for a writer filter instance.

#### DDS\_FILTER\_LOG\_INLINE\_QOS\_BUFFER

```
#define DDS_FILTER_LOG_INLINE_QOS_BUFFER 4
```

Heap memory for a writer to serialize a sample's in-line QoS.

## 8.8 dds\_psk\_log.h File Reference

DDS\_PSK module log codes.

```
#include "osapi/osapi_log.h"
```



## Macros

- #define `DDS_PSK_TRUST_EXCEPTION_EC` (`SEC_CORE_LOG_BASE` + 3)  
*Log trust exceptions.*

### 8.8.1 Detailed Description

DDS\_PSK module log codes.

### 8.8.2 Macro Definition Documentation

#### DDS\_PSK\_TRUST\_EXCEPTION\_EC

```
#define DDS_PSK_TRUST_EXCEPTION_EC (SEC_CORE_LOG_BASE + 3)
```

Log trust exceptions.

## 8.9 disc\_dpde\_log.h File Reference

```
#include "osapi/osapi_log.h"
```

## Macros

- #define `DPDE_LOG_CDR_SET_OFFSET_EC` (`DPDE_LOG_BASE` + 1)  
*Failed to set current offset of stream.*
- #define `DPDE_LOG_SERIALIZE_ENTITY_NAME_EC` (`DPDE_LOG_BASE` + 2)  
*Failed to serialize Entity Name parameter.*
- #define `DPDE_LOG_SERIALIZE_TOPIC_NAME_EC` (`DPDE_LOG_BASE` + 3)  
*Failed to serialize Topic Name parameter.*
- #define `DPDE_LOG_SERIALIZE_TYPE_NAME_EC` (`DPDE_LOG_BASE` + 4)  
*Failed to serialize Type Name parameter.*
- #define `DPDE_LOG_SERIALIZE_GUID_EC` (`DPDE_LOG_BASE` + 5)  
*Failed to serialize GUID key.*
- #define `DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (`DPDE_LOG_BASE` + 6)  
*Failed to serialize Default Unicast Locator parameter.*
- #define `DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (`DPDE_LOG_BASE` + 7)  
*Failed to serialize Protocol Version parameter.*
- #define `DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (`DPDE_LOG_BASE` + 8)  
*Failed to deserialize Protocol Version parameter.*
- #define `DPDE_LOG_DESERIALIZE_VENDOR_ID_EC` (`DPDE_LOG_BASE` + 9)  
*Failed to deserialize Vendor ID parameter.*
- #define `DPDE_LOG_SERIALIZE_VENDOR_ID_EC` (`DPDE_LOG_BASE` + 10)  
*Failed to serialize Vendor ID parameter.*

- #define DPDE\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC (DPDE\_LOG\_BASE + 11)  
*Failed to serialize Product Version parameter.*
- #define DPDE\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC (DPDE\_LOG\_BASE + 12)  
*Failed to serialize Lease Duration parameter.*
- #define DPDE\_LOG\_CREATE\_PUBLISHER\_EC (DPDE\_LOG\_BASE + 13)  
*Failed to create Publisher for built-in endpoint DataWriters.*
- #define DPDE\_LOG\_CREATE\_SUBSCRIBER\_EC (DPDE\_LOG\_BASE + 14)  
*Failed to create Subscriber for built-in endpoint DataReaders.*
- #define DPDE\_LOG\_CREATE\_PARTICIPANT\_DISCOVERY\_EC (DPDE\_LOG\_BASE + 15)  
*Failed to create built-in endpoint for participant discovery after creating local user DomainParticipant.*
- #define DPDE\_LOG\_CREATE\_PUBLICATION\_DISCOVERY\_EC (DPDE\_LOG\_BASE + 16)  
*Failed to create built-in publication endpoint after creating local user DomainParticipant.*
- #define DPDE\_LOG\_CREATE\_SUBSCRIPTION\_DISCOVERY\_EC (DPDE\_LOG\_BASE + 17)  
*Failed to create built-in subscription endpoint after creating local DomainParticipant.*
- #define DPDE\_LOG\_DELETE\_SUBSCRIPTION\_TOPIC\_EC (DPDE\_LOG\_BASE + 18)  
*Failed to delete subscription built-in topic.*
- #define DPDE\_LOG\_DELETE\_PUBLICATION\_TOPIC\_EC (DPDE\_LOG\_BASE + 19)  
*Failed to delete publication built-in topic.*
- #define DPDE\_LOG\_DELETE\_PARTICIPANT\_TOPIC\_EC (DPDE\_LOG\_BASE + 20)  
*Failed to delete participant built-in topic.*
- #define DPDE\_LOG\_DISPOSE\_PARTICIPANT\_EC (DPDE\_LOG\_BASE + 21)  
*Failed to dispose built-in participant when deleting DomainParticipant.*
- #define DPDE\_LOG\_ALLOCATE\_DPDE\_EC (DPDE\_LOG\_BASE + 22)  
*Out of memory to allocate discovery plugin.*
- #define DPDE\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC (DPDE\_LOG\_BASE + 23)  
*Failed to serialize Builtin Endpoint Mask parameter.*
- #define DPDE\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC (DPDE\_LOG\_BASE + 24)  
*Failed to deserialize Builtin Endpoint Mask parameter.*
- #define DPDE\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC (DPDE\_LOG\_BASE + 25)  
*Failed to deserialize a parameter of unknown ID.*
- #define DPDE\_LOG\_ANNOUNCEMENT\_EC (DPDE\_LOG\_BASE + 26)  
*Failed to write a dynamic participant discovery message.*
- #define DPDE\_LOG\_UPDATE\_PARTICIPANT\_ASSERT\_PERIOD\_EC (DPDE\_LOG\_BASE + 27)  
*Failed to schedule an event to assert the next participant discovery announcement.*
- #define DPDE\_LOG\_ADVANCE\_SN\_EC (DPDE\_LOG\_BASE + 28)  
*Failed to advance the sequence number of a participant discovery announcement.*
- #define DPDE\_LOG\_ANNOUNCE\_WRITE\_EC (DPDE\_LOG\_BASE + 29)  
*Failed to write initial participant discovery announcement.*
- #define DPDE\_LOG\_SCHEDULE\_FAST\_ASSERTION\_EC (DPDE\_LOG\_BASE + 30)  
*Failed to schedule event for asserting participant discovery announcements.*
- #define DPDE\_LOG\_REGISTER\_TYPE\_EC (DPDE\_LOG\_BASE + 32)  
*Failed to register a built-in type for discovery.*
- #define DPDE\_LOG\_CREATE\_TOPIC\_EC (DPDE\_LOG\_BASE + 33)  
*Failed to create a topic for discovery.*
- #define DPDE\_LOG\_CREATE\_WRITER\_EC (DPDE\_LOG\_BASE + 34)  
*Failed to create a built-in DataWriter for discovery.*
- #define DPDE\_LOG\_CREATE\_READER\_EC (DPDE\_LOG\_BASE + 35)

- Failed to create a built-in DataReader for discovery.*

  - #define `DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 36)
- Failed to assert and complete discovery of a remote participant.*

  - #define `DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 37)
- Failed to enable and complete discovery of a remote participant.*

  - #define `DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (`DPDE_LOG_BASE` + 38)
- Failed to refresh liveness for a discovered remote participant.*

  - #define `DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 39)
- Failed to take a sample from a participant discovery DataReader.*

  - #define `DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 40)
- Received a participant discovery announcement with invalid state.*

  - #define `DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC` (`DPDE_LOG_BASE` + 41)
- Failed to return a loan on a participant discovery sample.*

  - #define `DPDE_LOG_DISPOSE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 42)
- Failed to dispose an instance for a publication.*

  - #define `DPDE_LOG_DISPOSE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 43)
- Failed to dispose an instance of a subscription.*

  - #define `DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 44)
- Failed to assert and complete discovery of a remote publication.*

  - #define `DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 45)
- Failed to assert and complete discovery of a remote subscription.*

  - #define `DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 46)
- Failed to take a sample from a publication discovery DataReader.*

  - #define `DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC` (`DPDE_LOG_BASE` + 47)
- Failed to dispose or unregister a remote publication instance.*

  - #define `DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 48)
- Received a publication discovery announcement with invalid state.*

  - #define `DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC` (`DPDE_LOG_BASE` + 49)
- Failed to return a loan on a publication discovery sample.*

  - #define `DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC` (`DPDE_LOG_BASE` + 51)
- Failed to take a sample from a subscription discovery DataReader.*

  - #define `DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC` (`DPDE_LOG_BASE` + 52)
- Received a subscription discovery sample with invalid state.*

  - #define `DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC` (`DPDE_LOG_BASE` + 53)
- Failed to return a loan on a subscription discovery sample.*

  - #define `DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC` (`DPDE_LOG_BASE` + 54)
- Failed to dispose or unregister a remote subscription instance.*

  - #define `DPDE_LOG_LOCATORS_FULL_EC` (`DPDE_LOG_BASE` + 55)
- A locator sequence is full, the remaining locators of the specified kind are dropped.*

  - #define `DPDE_LOG_DESERIALIZE_LOCATOR_EC` (`DPDE_LOG_BASE` + 56)
- Failed to deserialize a locator of the specified kind.*

  - #define `DPDE_LOG_UNSUPPORTED_LOCATOR_EC` (`DPDE_LOG_BASE` + 57)
- The locator kind is not supported by any transport registered with the domain participant and is dropped.*

  - #define `DPDE_LOG_ADD_PEER_EC` (`DPDE_LOG_BASE` + 58)
- Failed to add the specified entity as a peer.*

  - #define `DPDE_LOG_REMOVE_PEER_EC` (`DPDE_LOG_BASE` + 59)
- Failed to add the specified entity as a peer.*

- #define DPDE\_LOG\_SERIALIZE\_PRESENTATION\_EC (DPDE\_LOG\_BASE + 60)  
*Failed to serialize Presentation parameter.*
- #define DPDE\_LOG\_UNREGISTER\_TYPE\_EC (DPDE\_LOG\_BASE + 61)  
*Failed to unregister a built-in type for discovery.*
- #define DPDE\_LOG\_ADD\_REMOTE\_PARTICIPANT\_ROUTES\_FAILED\_EC (DPDE\_LOG\_BASE + 62)  
*Failed to add routes to a remote participant.*
- #define DPDE\_LOG\_DELETE\_REMOTE\_PARTICIPANT\_ROUTES\_FAILED\_EC (DPDE\_LOG\_BASE + 63)  
*Failed to delete routes to a remote participant.*
- #define DPDE\_LOG\_REMOVE\_REMOTE\_PARTICIPANT\_EC (DPDE\_LOG\_BASE + 64)  
*Failed remove a remote participant.*
- #define DPDE\_LOG\_CHECKSUM\_INCOMPATIBLE\_PARTICIPANT\_IGNORED\_EC (DPDE\_LOG\_BASE + 65)  
*A participant with incompatible checksum was discovered and ignored.*

## 8.10 disc\_dpse\_log.h File Reference

```
#include "osapi/osapi_log.h"
```

### Macros

- #define DPSE\_LOG\_INVALID\_PEER\_ADDRESS\_EC (DPSE\_LOG\_BASE + 1)  
*A peer address string specifies an invalid address.*
- #define DPSE\_LOG\_CREATE\_DISCOVERY\_PUBLISHER\_EC (DPSE\_LOG\_BASE + 2)  
*Failed to create the publisher for a participant discovery writer.*
- #define DPSE\_LOG\_CREATE\_DISCOVERY\_SUBSCRIBER\_EC (DPSE\_LOG\_BASE + 3)  
*Failed to create the subscriber for a participant discovery datareader.*
- #define DPSE\_LOG\_REGISTER\_TYPE\_EC (DPSE\_LOG\_BASE + 4)  
*Failed to register a built-in participant discovery type.*
- #define DPSE\_LOG\_CREATE\_TOPIC\_EC (DPSE\_LOG\_BASE + 5)  
*Failed to create a topic for the built-in participant discovery topic.*
- #define DPSE\_LOG\_CREATE\_WRITER\_EC (DPSE\_LOG\_BASE + 6)  
*Failed to create a DataWriter for participant discovery.*
- #define DPSE\_LOG\_CREATE\_READER\_EC (DPSE\_LOG\_BASE + 7)  
*Failed to create a DataReader for participant discovery.*
- #define DPSE\_LOG\_DISPOSE\_EC (DPSE\_LOG\_BASE + 8)  
*Failed to dispose a participant discovery instance.*
- #define DPSE\_LOG\_ANNOUNCE\_LOCAL\_PARTICIPANT\_DELETION\_EC (DPSE\_LOG\_BASE + 9)  
*Failed to dispose a participant discovery instance upon deletion.*
- #define DPSE\_LOG\_DELETE\_WRITER\_EC (DPSE\_LOG\_BASE + 10)  
*Failed to delete a participant discovery DataWriter.*
- #define DPSE\_LOG\_DELETE\_PUBLISHER\_EC (DPSE\_LOG\_BASE + 11)  
*Failed to delete a participant discovery Publisher.*
- #define DPSE\_LOG\_DELETE\_READER\_EC (DPSE\_LOG\_BASE + 12)  
*Failed to delete a participant discovery DataReader.*
- #define DPSE\_LOG\_DELETE\_TOPIC\_EC (DPSE\_LOG\_BASE + 13)

- Failed to delete a participant discovery Topic.*

  - #define `DPSE_LOG_DELETE_SUBSCRIBER_EC` (`DPSE_LOG_BASE` + 14)
- Failed to delete a participant discovery Subscriber.*

  - #define `DPSE_KEYLIST_OBJECT` (1)
- The list keeping track of asserted participant keys.*

  - #define `DPSE_PARTICIPANTMAMES_OBJECT` (2)
- The list keeping track of asserted participant names.*

  - #define `DPSE_PARTICIPANTNAMEINDEX_OBJECT` (3)
- The index keeping track of asserted participant names.*

  - #define `DPSE_PARTICIPANTANNOUCEMENT_OBJECT` (4)
- The participant announcement timeout object.*

  - #define `DPSE_DISCOVERYPLUGIN_OBJECT` (5)
- A discovery plugin object.*

  - #define `DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT` (6)
- ParticipantBuiltinTopicData.*

  - #define `DPSE_PARTICIPANTENTITYNAME_OBJECT` (7)
- ParticipantBuiltinTopicData entity-name object.*

  - #define `DPSE_ENABLEDPARTICIPANTINDEX_OBJECT` (8)
- The index keeping track of enabled participant names.*

  - #define `DPSE_PARTICIPANTPOOL_OBJECT` (10)
- The pool of asserted participants.*

  - #define `DPSE_LOG_OBJECT_ALLOCATE_EC` (`DPSE_LOG_BASE` + 15)
- Failed to allocate an object of the specified kind.*

  - #define `DPSE_LOG_OBJECT_INITIALIZE_EC` (`DPSE_LOG_BASE` + 16)
- Failed to initialize an object of the specified kind.*

  - #define `DPSE_LOG_OBJECT_FINALIZE_EC` (`DPSE_LOG_BASE` + 17)
- Failed to finalize an object of the specified kind.*

  - #define `DPSE_LOG_OBJECT_DELETE_EC` (`DPSE_LOG_BASE` + 18)
- Failed to delete an object of the specified kind.*

  - #define `DPSE_LOG_OBJECT_INVALID_EC` (`DPSE_LOG_BASE` + 19)
- The object was invalid in the context it was used.*

  - #define `DPSE_LOG_CDR_SET_POSITION_EC` (`DPSE_LOG_BASE` + 20)
- Failed to set the CDR stream position.*

  - #define `DPSE_LOG_CDR_LONG_KIND` 1
- CDR Long kind.*

  - #define `DPSE_LOG_CDR_PID_KIND` 2
- CDR PID kind, the pid argument specifies which pid.*

  - #define `DPSE_LOG_CDR_SERIALIZE_EC` (`DPSE_LOG_BASE` + 21)
- Failed to serialize the specified kind.*

  - #define `DPSE_LOG_CDR_DESERIALIZE_EC` (`DPSE_LOG_BASE` + 22)
- Failed to deserialize the specified kind.*

  - #define `DPSE_LOG_SERIALIZE_GUID_EC` (`DPSE_LOG_BASE` + 23)
- Failed to serialize a GUID key parameter.*

  - #define `DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (`DPSE_LOG_BASE` + 24)
- Failed to serialize a Builtin Endpoint Mask parameter.*

  - #define `DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (`DPSE_LOG_BASE` + 25)
- Failed to serialize a Protocol Version parameter.*

- `#define DPSE_LOG_SERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 26)`  
*Failed to serialize a Vendor ID parameter.*
- `#define DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DPSE_LOG_BASE + 27)`  
*Failed to serialize a Default Unicast Locator parameter.*
- `#define DPSE_LOG_SERIALIZE_META_UNICAST_EC (DPSE_LOG_BASE + 28)`  
*Failed to serialize a Meta Unicast Locator parameter.*
- `#define DPSE_LOG_SERIALIZE_META_MULTICAST_EC (DPSE_LOG_BASE + 29)`  
*Failed to serialize a Meta Multicast Locator parameter.*
- `#define DPSE_LOG_SERIALIZE_LEASE_DURATION_EC (DPSE_LOG_BASE + 30)`  
*Failed to serialize a Lease Duration parameter.*
- `#define DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 31)`  
*Failed to serialize a Product Version parameter.*
- `#define DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 32)`  
*Failed to serialize a Product Version parameter.*
- `#define DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 33)`  
*Failed to serialize a Participant Name parameter.*
- `#define DPSE_LOG_DESERIALIZE_GUID_EC (DPSE_LOG_BASE + 34)`  
*Failed to deserialize a GUID key parameter.*
- `#define DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DPSE_LOG_BASE + 35)`  
*Failed to deserialize a Builtin Endpoint Mask parameter.*
- `#define DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DPSE_LOG_BASE + 36)`  
*Failed to deserialize a Protocol Version parameter.*
- `#define DPSE_LOG_DESERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 37)`  
*Failed to deserialize a Vendor ID parameter.*
- `#define DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC (DPSE_LOG_BASE + 38)`  
*Cannot deserialize another locator parameter, having reached maximum of 4 unicast or 4 multicast locators.*
- `#define DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 39)`  
*Failed to deserialize a Participant Name parameter.*
- `#define DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC (DPSE_LOG_BASE + 40)`  
*Failed to deserialize a parameter with an unknown ID.*
- `#define DPSE_LOG_ANNOUNCEMENT_EC (DPSE_LOG_BASE + 41)`  
*Failed to send a participant discovery announcement.*
- `#define DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC (DPSE_LOG_BASE + 42)`  
*Failed to schedule an event to send the next participant discovery announcement.*
- `#define DPSE_LOG_ADVANCE_SN_EC (DPSE_LOG_BASE + 43)`  
*Failed to advance the sequence number of a participant discovery announcement.*
- `#define DPSE_LOG_ANNOUNCE_WRITE_EC (DPSE_LOG_BASE + 44)`  
*Failed to write a participant discovery announcement message.*
- `#define DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC (DPSE_LOG_BASE + 45)`  
*Failed to schedule an event to send the next participant discovery announcement.*
- `#define DPSE_LOG_PARTICIPANT_TAKE_EC (DPSE_LOG_BASE + 46)`  
*Failed to take a sample from a participant discovery DataReader.*
- `#define DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 47)`  
*Failed to assert remote participant.*
- `#define DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 48)`  
*Failed to assert and complete discovery of a remote participant.*
- `#define DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 49)`

- Failed to add an anonymous route.*

  - #define `DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC` (`DPSE_LOG_BASE` + 50)
- Failed to delete an anonymous route.*

  - #define `DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 51)
- Exceeded resource limit, remote participant allocation.*

  - #define `DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 52)
- Failed to refresh liveliness for a remote participant.*

  - #define `DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC` (`DPSE_LOG_BASE` + 53)
- Failed to reset liveliness for a remote participant.*

  - #define `DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC` (`DPSE_LOG_BASE` + 54)
- Received a participant discovery announcement with invalid state.*

  - #define `DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC` (`DPSE_LOG_BASE` + 55)
- Failed to return a loan on a participant discovery announcement.*

  - #define `DPSE_LOG_SERIALIZE_MULTICAST_EC` (`DPSE_LOG_BASE` + 56)
- Failed to serialize a Multicast Locator parameter.*

  - #define `DPSE_LOG_GET_DDS_PROPERTIES_EC` (`DPSE_LOG_BASE` + 57)
- Failed to serialize a Multicast Locator parameter.*

  - #define `DPSE_TOPIC_ENTITY` 1
- A DDS Topic entity.*

  - #define `DPSE_PUBLISHER_ENTITY` 2
- A DDS Publisher entity.*

  - #define `DPSE_SUBSCRIBER_ENTITY` 3
- A DDS Subscriber entity.*

  - #define `DPSE_LOG_ENTITY_ENABLE_EC` (`DPSE_LOG_BASE` + 58)
- Failed to enable the specified entity kind.*

  - #define `DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE` 1
- The default unicast locator sequence.*

  - #define `DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE` 2
- The default multicast locator sequence.*

  - #define `DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE` 3
- The meta unicast locator sequence.*

  - #define `DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE` 4
- The meta multicast locator sequence.*

  - #define `DPSE_LOG_SEQUENCE_SETMAX_EC` (`DPSE_LOG_BASE` + 59)
- Failed to set the maximum length of a sequence of the specified kind.*

  - #define `DPSE_LOG_SEQUENCE_SETLENGTH_EC` (`DPSE_LOG_BASE` + 60)
- Failed to set the length of a sequence of the specified kind.*

  - #define `DPSE_LOG_SEQUENCE_GETREF_EC` (`DPSE_LOG_BASE` + 61)
- Failed to get a reference at the specified index for a sequence of the specified kind.*

  - #define `DPSE_LOG_SEQUENCE_INITIALIZE_EC` (`DPSE_LOG_BASE` + 62)
- Failed to initialize a sequence of the specified kind.*

  - #define `DPSE_LOG_SEQUENCE_FINALIZE_EC` (`DPSE_LOG_BASE` + 63)
- Failed to finalize a sequence of the specified kind.*

  - #define `DPSE_LOG_SEQUENCE_COPY_EC` (`DPSE_LOG_BASE` + 64)
- Failed to copy a sequence of the specified kind.*

  - #define `DPSE_LOG_GET_TIMER_EC` (`DPSE_LOG_BASE` + 65)
- Failed to get timer.*

- #define `DPSE_LOG_UNSUPPORTED_LOCATOR_EC` (`DPSE_LOG_BASE` + 66)  
*The locator kind is not supported by any transport registered with the domain participant and is dropped.*
- #define `DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC` (`DPSE_LOG_BASE` + 67)  
*A participant with incompatible checksum was discovered and ignored.*
- #define `DPSE_LOG_SERIALIZE_PROP_EC` (`DPSE_LOG_BASE` + 68)  
*A Participant failed to serialize its properties.*

### 8.10.1 Macro Definition Documentation

#### DPSE\_KEYLIST\_OBJECT

```
#define DPSE_KEYLIST_OBJECT (1)
```

The list keeping track of asserted participant keys.

#### DPSE\_PARTICIPANTNAMES\_OBJECT

```
#define DPSE_PARTICIPANTNAMES_OBJECT (2)
```

The list keeping track of asserted participant names.

#### DPSE\_PARTICIPANTNAMEINDEX\_OBJECT

```
#define DPSE_PARTICIPANTNAMEINDEX_OBJECT (3)
```

The index keeping track of asserted participant names.

#### DPSE\_PARTICIPANTANNOUNCEMENT\_OBJECT

```
#define DPSE_PARTICIPANTANNOUNCEMENT_OBJECT (4)
```

The participant announcement timeout object.

#### DPSE\_DISCOVERYPLUGIN\_OBJECT

```
#define DPSE_DISCOVERYPLUGIN_OBJECT (5)
```

A discovery plugin object.

#### DPSE\_PARTICIPANTBUILTINTOPICDATA\_OBJECT

```
#define DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT (6)
```

ParticipantBuiltinTopicData.



**DPSE\_PARTICIPANTENTITYNAME\_OBJECT**

```
#define DPSE_PARTICIPANTENTITYNAME_OBJECT (7)
```

ParticipantBuiltinTopicData entity-name object.

**DPSE\_ENABLEDPARTICIPANTINDEX\_OBJECT**

```
#define DPSE_ENABLEDPARTICIPANTINDEX_OBJECT (8)
```

The index keeping track of enabled participant names.

**DPSE\_PARTICIPANTPOOL\_OBJECT**

```
#define DPSE_PARTICIPANTPOOL_OBJECT (10)
```

The pool of asserted participants.

**DPSE\_LOG\_CDR\_LONG\_KIND**

```
#define DPSE_LOG_CDR_LONG_KIND 1
```

CDR Long kind.

**DPSE\_LOG\_CDR\_PID\_KIND**

```
#define DPSE_LOG_CDR_PID_KIND 2
```

CDR PID kind, the pid argument specifies which pid.

**DPSE\_TOPIC\_ENTITY**

```
#define DPSE_TOPIC_ENTITY 1
```

A DDS Topic entity.

**DPSE\_PUBLISHER\_ENTITY**

```
#define DPSE_PUBLISHER_ENTITY 2
```

A DDS Publisher entity.

**DPSE\_SUBSCRIBER\_ENTITY**

```
#define DPSE_SUBSCRIBER_ENTITY 3
```

A DDS Subscriber entity.

**DPSE\_LOG\_DEFAULT\_UNICAST\_LOCATOR\_SEQUENCE**

```
#define DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE 1
```

The default unicast locator sequence.

**DPSE\_LOG\_DEFAULT\_MULTICAST\_LOCATOR\_SEQUENCE**

```
#define DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE 2
```

The default multicast locator sequence.

**DPSE\_LOG\_META\_UNICAST\_LOCATOR\_SEQUENCE**

```
#define DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE 3
```

The meta unicast locator sequence.

**DPSE\_LOG\_META\_MULTICAST\_LOCATOR\_SEQUENCE**

```
#define DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE 4
```

The meta multicast locator sequence.

**8.11 netio\_log.h File Reference**

NETIO module log codes.

```
#include "osapi/osapi_log.h"
```

## Macros

- #define `NETIO_LOG_GETHOST_BYNAME_EC` (`NETIO_LOG_BASE` + 1)  
*Failed to get host address from a string representation.*
- #define `NETIO_LOG_BIND_NEW_EC` (`NETIO_LOG_BASE` + 2)  
*Failed to allocate memory for a bind-resolver.*
- #define `NETIO_LOG_BIND_NEW_RTABLE_EC` (`NETIO_LOG_BASE` + 3)  
*Failed to create a database table for routes of a bind-resolver.*
- #define `NETIO_LOG_BIND_CREATE_RTABLE_EC` (`NETIO_LOG_BASE` + 4)  
*Failed to create route record.*
- #define `NETIO_LOG_BIND_DELETE_RTABLE_EC` (`NETIO_LOG_BASE` + 5)  
*Failed to delete a database table for routes of a bind\_resolver.*
- #define `NETIO_LOG_BIND_SET_LENGTH_EC` (`NETIO_LOG_BASE` + 6)  
*Failed to set the length for a sequence of resolved addresses.*
- #define `NETIO_LOG_BIND_SET_MAX_EC` (`NETIO_LOG_BASE` + 7)  
*Failed to set the maximum length of an internal sequence of addresses.*
- #define `NETIO_LOG_BIND_EXTERNAL_FAILED_EC` (`NETIO_LOG_BASE` + 8)  
*An interface failed on bind\_external.*
- #define `NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC` (`NETIO_LOG_BASE` + 10)  
*An interface failed on unbind\_external.*
- #define `NETIO_LOG_ROUTE_RTABLE_ALLOC_EC` (`NETIO_LOG_BASE` + 11)  
*Failed to allocate memory for a route resolver.*
- #define `NETIO_LOG_ROUTE_RTABLE_CREATE_EC` (`NETIO_LOG_BASE` + 12)  
*Failed to create a database table for routes of a route-resolver.*
- #define `NETIO_LOG_ROUTE_RTABLE_DELETE_EC` (`NETIO_LOG_BASE` + 13)  
*Failed to delete a database table for routes of a route-resolver.*
- #define `NETIO_LOG_ROUTE_RTABLE_ADD_EC` (`NETIO_LOG_BASE` + 14)  
*Failed to create route resolver record.*
- #define `NETIO_LOG_ROUTE_RTABLE_UPDATE_EC` (`NETIO_LOG_BASE` + 15)  
*Failed to find routes to update for a route-resolver.*
- #define `NETIO_LOG_AR_ALLOC_FAILED_EC` (`NETIO_LOG_BASE` + 16)  
*Failed to allocate memory for address resolver.*
- #define `NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC` (`NETIO_LOG_BASE` + 17)  
*An attempt was made to add an existing interface with a different port resolver to the address resolver.*
- #define `NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC` (`NETIO_LOG_BASE` + 18)  
*An interface failed to resolve an address.*
- #define `NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC` (`NETIO_LOG_BASE` + 19)  
*An interface failed to resolve a port.*
- #define `NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC` (`NETIO_LOG_BASE` + 21)  
*NETIO\_AddressResolver\_resolve/get\_next was called with an uninitialized context.*
- #define `NETIO_LOG_AR_INVALID_TOKEN_EC` (`NETIO_LOG_BASE` + 22)  
*An invalid token was encountered in the address string.*
- #define `NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC` (`NETIO_LOG_BASE` + 23)  
*The length of the address exceeded the address buffer passed in.*
- #define `NETIO_LOG_INVALID_ADDRESS_INDEX_EC` (`NETIO_LOG_BASE` + 24)  
*An address index was not parsed correctly.*
- #define `NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC` (`NETIO_LOG_BASE` + 25)

- Illegal route operation attempted.*

  - #define `NETIO_LOG_SET_NAME_EC` (`UDP_LOG_BASE` + 26)

*Failed to set the name of a runtime component interface.*
- #define `NETIO_LOG_PACKET_SET_HEAD_EC` (`UDP_LOG_BASE` + 27)

*Failed to set packet's head.*
- #define `NETIO_LOG_PACKET_SET_TAIL_EC` (`UDP_LOG_BASE` + 28)

*Failed to set packet's tail.*
- #define `NETIO_LOG_PACKET_INITIALIZE_EC` (`UDP_LOG_BASE` + 29)

*Failed to initialize packet.*
- #define `NETIO_LOG_LOOP_DELETE_TABLE_EC` (`NETIO_LOG_BASE` + 100)

*Failed to delete a database table for a NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_CREATE_TABLE_EC` (`NETIO_LOG_BASE` + 101)

*Failed to create a database table for a NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_SELECT_TABLE_EC` (`NETIO_LOG_BASE` + 102)

*Failed to select a valid record when sending with the NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_FWD_EC` (`NETIO_LOG_BASE` + 103)

*Failed to send forward with the NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_BINDX_DB_EC` (`NETIO_LOG_BASE` + 104)

*Failed to bind an external interface with the NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_UNBINDX_DB_EC` (`NETIO_LOG_BASE` + 105)

*Failed to unbind an external interface with the NETIO loopback interface.*
- #define `NETIO_LOG_LOOP_SET_LENGTH_EC` (`NETIO_LOG_BASE` + 106)

*Failed to set the length of an address sequence.*
- #define `NETIO_LOG_LOOP_INVALID_PROPERTY_EC` (`NETIO_LOG_BASE` + 107)

*Invalid property passed in when creating a loopback interface.*
- #define `NETIO_LOG_LOOP_INITIALIZE_FAILED_EC` (`NETIO_LOG_BASE` + 108)

*Failed to initialize the loopback interface.*
- #define `NETIO_LOG_LOOP_CURSOR_ERROR_EC` (`NETIO_LOG_BASE` + 109)

*An error occurred with a cursor while iterating over a table.*
- #define `NETIO_LOG_LOOP_INVALID_FACTORY_EC` (`NETIO_LOG_BASE` + 110)

*An invalid factory was passed in to create a loopback interface.*
- #define `NETIO_LOG_LOOP_INVALID_COMPONENT_EC` (`NETIO_LOG_BASE` + 111)

*An invalid component was passed in to delete a loopback interface.*
- #define `UDP_LOG_SOCKET_INIT_EC` (`UDP_LOG_BASE` + 200)

*Failed to initialize use of sockets.*
- #define `UDP_LOG_GETHOSTNAME_EC` (`UDP_LOG_BASE` + 201)

*Failed to get the host address given the host name.*
- #define `UDP_LOG_SOCKET_SET_MCAST_EC` (`UDP_LOG_BASE` + 202)

*Failed to set a socket for multicast loopback.*
- #define `UDP_LOG_SOCKET_SET_MCASTIF_EC` (`UDP_LOG_BASE` + 203)

*Failed to set a socket for multicast bound to a specific interface.*
- #define `UDP_LOG_SOCKET_CREATE_EC` (`UDP_LOG_BASE` + 204)

*Failed to create a new socket, the sysrc code is the OS return code and reason for failure.*
- #define `UDP_LOG_SOCKET_SETFD_EC` (`UDP_LOG_BASE` + 205)

*Failed to set the close-on-exec flag for a socket.*
- #define `UDP_LOG_SOCKET_SNDBUF_EC` (`UDP_LOG_BASE` + 206)

*Failed to set the socket send buffer size.*

- `#define UDP_LOG_SOCKET_TTL_EC (UDP_LOG_BASE + 207)`  
*Failed to set multicast TTL for a socket.*
- `#define UDP_LOG_PACKET_INIT_EC (UDP_LOG_BASE + 208)`  
*Failed to initialize a receive packet.*
- `#define UDP_LOG_PACKET_HEAD_EC (UDP_LOG_BASE + 209)`  
*Failed to set the head of a receive packet.*
- `#define UDP_LOG_PACKET_FWD_EC (UDP_LOG_BASE + 210)`  
*Failed reception upstream for a received packet.*
- `#define UDP_LOG_NAT_DELETE_EC (UDP_LOG_BASE + 211)`  
*Failed to delete a database record or index from an internal NAT.*
- `#define UDP_LOG_FINALIZE_EC (UDP_LOG_BASE + 212)`  
*Failed to finalize the parent NETIO interface.*
- `#define UDP_LOG_SOCKET_REUSE_PORT_EC (UDP_LOG_BASE + 213)`  
*Failed to set socket to reuse a port or address.*
- `#define UDP_LOG_SOCKET_BIND_EC (UDP_LOG_BASE + 214)`  
*Failed a socket bind.*
- `#define UDP_LOG_SOCKET_ADDGRP_EC (UDP_LOG_BASE + 215)`  
*Failed to add membership to a multicast group for a socket.*
- `#define UDP_LOG_PORT_ENTRY_EC (UDP_LOG_BASE + 216)`  
*Failed to create a bind entry for a receive port.*
- `#define UDP_LOG_PORT_ALLOC_EC (UDP_LOG_BASE + 217)`  
*Failed to allocate memory for a receive buffer of a specific port.*
- `#define UDP_LOG_SOCKET_RXBUF_EC (UDP_LOG_BASE + 218)`  
*Failed to set the receive buffer size of a socket.*
- `#define UDP_LOG_PORT_ADD_EC (UDP_LOG_BASE + 219)`  
*Failed to add a port entry record to a database table.*
- `#define UDP_LOG_PORT_THREAD_EC (UDP_LOG_BASE + 220)`  
*Failed to create a receive thread for a receive port.*
- `#define UDP_LOG_SOCKET_IFLIST_EC (UDP_LOG_BASE + 221)`  
*Failed to get a list of interface addresses.*
- `#define UDP_LOG_SOCKET_IFFLAGS_EC (UDP_LOG_BASE + 222)`  
*Failed to get the network interface flags or mask of a socket.*
- `#define UDP_LOG_SOCKET_CLOSE_EC (UDP_LOG_BASE + 223)`  
*Failed to close a socket.*
- `#define UDP_LOG_LOADLIB_EC (UDP_LOG_BASE + 224)`  
*Failed to load a dynamic library.*
- `#define UDP_LOG_GET_BUF_SIZE_EC (UDP_LOG_BASE + 225)`  
*Failed to get interface buffer size.*
- `#define UDP_LOG_GET_BUF_ALLOC_EC (UDP_LOG_BASE + 226)`  
*Out of memory to allocate interface buffer.*
- `#define UDP_LOG_GET_IFLIST_EC (UDP_LOG_BASE + 227)`  
*Failed to get an interface list.*
- `#define UDP_LOG_GET_NAMEINFO_EC (UDP_LOG_BASE + 228)`  
*Failed getnameinfo()*
- `#define UDP_LOG_MULTICAST_ENABLE_EC (UDP_LOG_BASE + 229)`  
*Failed to enable multicast loopback.*
- `#define UDP_LOG_UDP_CREATE_TABLE_EC (UDP_LOG_BASE + 230)`

- Failed to create a database table for a UDP interface.*

  - #define `UDP_LOG_UDP_CREATE_INDEX_EC` (`UDP_LOG_BASE` + 231)
- Failed to create an index for a UDP interface.*

  - #define `UDP_LOG_RECORD_EC` (`UDP_LOG_BASE` + 232)
- Failed to create or insert a database record for a UDP interface.*

  - #define `UDP_LOG_PROPERTY_FINALIZE_EC` (`UDP_LOG_BASE` + 233)
- Failed to finalize UDP interface factory property.*

  - #define `UDP_LOG_ALLOC_EC` (`UDP_LOG_BASE` + 234)
- Failed to allocate memory for a UDP interface.*

  - #define `UDP_LOG_SEND_PING_EC` (`UDP_LOG_BASE` + 235)
- Failed sending a ping message, upon adding a route or waking up a receive thread.*

  - #define `UDP_LOG_DROPGRP_EC` (`UDP_LOG_BASE` + 236)
- Failed to drop membership from a multicast group.*

  - #define `UDP_LOG_GET_LENGTH_EC` (`UDP_LOG_BASE` + 237)
- Length of address sequence exceeds its maximum.*

  - #define `UDP_LOG_SET_LENGTH_EC` (`UDP_LOG_BASE` + 240)
- Failed to set the length of an address sequence.*

  - #define `UDP_LOG_WSA_STARTUP_EC` (`UDP_LOG_BASE` + 241)
- Failed WSStartup()*

  - #define `UDP_LOG_WINSOCK_INCOMPATIBLE_EC` (`UDP_LOG_BASE` + 242)
- Incompatible version of Winsock, must not be older than 2.0.*

  - #define `UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC` (`UDP_LOG_BASE` + 243)
- Inconsistent max\_message\_size.*

  - #define `UDP_LOG_INITIALIZE_FAILED_EC` (`UDP_LOG_BASE` + 244)
- Failed to initialize the loopback interface.*

  - #define `UDP_LOG_PORT_NOT_FOUND_EC` (`UDP_LOG_BASE` + 245)
- Did not find the specified thread port.*

  - #define `UDP_LOG_CURSOR_ERROR_EC` (`UDP_LOG_BASE` + 246)
- An error occurred while iterating over a cursor.*

  - #define `UDP_LOG_SEND_ERROR_EC` (`UDP_LOG_BASE` + 247)
- Failed to send message.*

  - #define `UDP_LOG_MULTICAST_NOT_ENABLED_EC` (`UDP_LOG_BASE` + 248)
- Multicast address ignored because multicast supported is not compiled in.*

  - #define `UDP_LOG_TRANSPORT_NO_INTERFACES_EC` (`UDP_LOG_BASE` + 249)
- Transport does not have any interfaces.*

  - #define `UDP_LOG_RECV_ERROR_EC` (`UDP_LOG_BASE` + 250)
- Failed to receive message.*

  - #define `UDP_TRANSFORM_LOG_KIND_RULE` (1)
- Rule object.*

  - #define `UDP_TRANSFORM_LOG_KIND_ENTRY` (2)
- Entry object.*

  - #define `UDP_TRANSFORM_LOG_KIND_FACTORY` (3)
- Factory object.*

  - #define `UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX` (4)
- Factory object.*

  - #define `UDP_TRANSFORM_LOG_KIND_DST_INDEX` (5)
- Factory object.*

- `#define UDP_TRANSFORM_LOG_KIND_SRC_INDEX (6)`  
*Factory object.*
- `#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE (7)`  
*Rule object.*
- `#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION (8)`  
*Entry object.*
- `#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETMASK_INDEX (9)`  
*Mask object.*
- `#define UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC (UDP_LOG_BASE + 300)`  
*The returned factory reference is NULL.*
- `#define UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC (UDP_LOG_BASE + 301)`  
*Could not find the factory used as part of a transform rule.*
- `#define UDP_TRANSFORM_LOG_ALLOC_EC (UDP_LOG_BASE + 302)`  
*Failed to allocate entry of the specified kind.*
- `#define UDP_TRANSFORM_LOG_CREATE_EC (UDP_LOG_BASE + 303)`  
*Failed to create a transformation of the specified kind.*
- `#define UDP_TRANSFORM_LOG_ADD_EC (UDP_LOG_BASE + 304)`  
*Failed to add object of specified kind.*
- `#define UDP_TRANSFORM_LOG_FIND_EC (UDP_LOG_BASE + 305)`  
*Failed to find a transformation.*
- `#define UDP_TRANSFORM_LOG_DUPLICATE_EC (UDP_LOG_BASE + 306)`  
*Duplicate object.*
- `#define UDP_TRANSFORM_LOG_TRANSFORM_EC (UDP_LOG_BASE + 307)`  
*Source or destination transformation function failed.*
- `#define UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC (UDP_LOG_BASE + 308)`  
*Invalid UDP mode specified.*
- `#define UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC (UDP_LOG_BASE + 309)`  
*Invalid transformation UDP locator kind specified.*

### 8.11.1 Detailed Description

NETIO module log codes.

### 8.11.2 Macro Definition Documentation

#### UDP\_TRANSFORM\_LOG\_KIND\_RULE

```
#define UDP_TRANSFORM_LOG_KIND_RULE (1)
```

Rule object.

#### UDP\_TRANSFORM\_LOG\_KIND\_ENTRY

```
#define UDP_TRANSFORM_LOG_KIND_ENTRY (2)
```

Entry object.

**UDP\_TRANSFORM\_LOG\_KIND\_FACTORY**

```
#define UDP_TRANSFORM_LOG_KIND_FACTORY (3)
```

Factory object.

**UDP\_TRANSFORM\_LOG\_KIND\_FACTORY\_INDEX**

```
#define UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX (4)
```

Factory object.

**UDP\_TRANSFORM\_LOG\_KIND\_DST\_INDEX**

```
#define UDP_TRANSFORM_LOG_KIND_DST_INDEX (5)
```

Factory object.

**UDP\_TRANSFORM\_LOG\_KIND\_SRC\_INDEX**

```
#define UDP_TRANSFORM_LOG_KIND_SRC_INDEX (6)
```

Factory object.

**UDP\_TRANSFORM\_LOG\_KIND\_TRANSFORM\_SOURCE**

```
#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE (7)
```

Rule object.

**UDP\_TRANSFORM\_LOG\_KIND\_TRANSFORM\_DESTINATION**

```
#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION (8)
```

Entry object.

**UDP\_TRANSFORM\_LOG\_KIND\_TRANSFORM\_NETMASK\_INDEX**

```
#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETMASK_INDEX (9)
```

Mask object.



## 8.12 netio\_shmem\_log.h File Reference

NETIO Zero Copy Shared Data module log codes.

```
#include "osapi/osapi_log.h"
```

### Macros

- #define [NETIO\\_SHMEM\\_LOG\\_DELETE\\_TABLE\\_EC](#) (SHMEM\_LOG\_BASE + 1)  
*Failed to delete a database table for a NETIO shared memory interface.*
- #define [NETIO\\_SHMEM\\_LOG\\_CREATE\\_TABLE\\_EC](#) (SHMEM\_LOG\_BASE + 2)  
*Failed to create a database table for a NETIO shared memory interface.*
- #define [NETIO\\_SHMEM\\_LOG\\_SELECT\\_TABLE\\_EC](#) (SHMEM\_LOG\_BASE + 3)  
*Failed to select a valid record when sending with the NETIO shared memory.*
- #define [NETIO\\_SHMEM\\_LOG\\_FWD\\_EC](#) (SHMEM\_LOG\_BASE + 4)  
*Failed to send forward with the NETIO shared memory interface.*
- #define [NETIO\\_SHMEM\\_LOG\\_BINDX\\_DB\\_EC](#) (SHMEM\_LOG\_BASE + 5)  
*Failed to bind an external interface with the NETIO shared memory interface.*
- #define [NETIO\\_SHMEM\\_LOG\\_UNBINDX\\_DB\\_EC](#) (SHMEM\_LOG\_BASE + 6)  
*Failed to unbind an external interface with the NETIO shared memory interface.*
- #define [NETIO\\_SHMEM\\_LOG\\_SET\\_LENGTH\\_EC](#) (SHMEM\_LOG\_BASE + 7)  
*Failed to set the length of an address sequence.*
- #define [NETIO\\_SHMEM\\_LOG\\_INVALID\\_PROPERTY\\_EC](#) (SHMEM\_LOG\_BASE + 8)  
*Invalid property passed in when creating a shared memory interface.*
- #define [NETIO\\_SHMEM\\_LOG\\_INITIALIZE\\_FAILED\\_EC](#) (SHMEM\_LOG\_BASE + 9)  
*Failed to initialize the shared memory interface.*
- #define [NETIO\\_SHMEM\\_LOG\\_CURSOR\\_ERROR\\_EC](#) (SHMEM\_LOG\_BASE + 10)  
*An error occurred with a cursor while iterating over a table.*
- #define [NETIO\\_SHMEM\\_LOG\\_INVALID\\_FACTORY\\_EC](#) (SHMEM\_LOG\_BASE + 11)  
*An invalid factory was passed in to create a shared memory interface.*
- #define [NETIO\\_SHMEM\\_LOG\\_INVALID\\_COMPONENT\\_EC](#) (SHMEM\_LOG\_BASE + 12)  
*An invalid component was passed in to delete a shared memory interface.*
- #define [NETIO\\_SHMEM\\_LOG\\_INCOMPATIBLE\\_COOKIE\\_EC](#) (SHMEM\_LOG\_BASE + 13)  
*Found incompatible cookie in shared memory header.*
- #define [NETIO\\_SHMEM\\_LOG\\_INCOMPATIBLE\\_VERSION\\_EC](#) (SHMEM\_LOG\_BASE + 14)  
*Found incompatible major version in shared memory header.*
- #define [NETIO\\_SHMEM\\_LOG\\_INCOMPATIBLE\\_HEADER\\_SIZE\\_EC](#) (SHMEM\_LOG\_BASE + 15)  
*Found incompatible header size in shared memory header.*
- #define [NETIO\\_SHMEM\\_LOG\\_INCOMPATIBLE\\_HEADER\\_EC](#) (SHMEM\_LOG\_BASE + 16)  
*Attempted to attach to shared memory header transport concurrent queue that is incompatible.*
- #define [NETIO\\_SHMEM\\_LOG\\_ADDRESS\\_IN\\_USE\\_EC](#) (SHMEM\_LOG\_BASE + 17)  
*Failed to create or attach to a shared memory segment.*
- #define [NETIO\\_SHMEM\\_LOG\\_CHECK\\_SYSTEM\\_SETTINGS\\_EC](#) (SHMEM\_LOG\_BASE + 18)  
*Possible failure due to misconfigured system settings on Darwin operating systems.*
- #define [NETIO\\_SHMEM\\_LOG\\_SHMEM\\_MUTEX\\_LOCK\\_FAILED\\_EC](#) (SHMEM\_LOG\_BASE + 19)  
*Failed to lock shared memory mutex protecting the shared memory segment.*

- `#define NETIO_SHMEM_LOG_SHMEM_MUTEX_UNLOCK_FAILED_EC (SHMEM_LOG_BASE + 20)`  
*Failed to unlock shared memory mutex protecting the shared memory segment.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_ALLOCATE_STRUCT_EC (SHMEM_LOG_BASE + 21)`  
*Failed to allocate memory needed by shared memory transport.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_INITIALIZE_EC (SHMEM_LOG_BASE + 22)`  
*Failed to initialize shared memory transport.*
- `#define NETIO_SHMEM_LOG_CREATE_ATTACH_INFINITE_EC (SHMEM_LOG_BASE + 23)`  
*Failed to create or attach. Possible shared memory key conflict.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_INIT_MUTEX_EC (SHMEM_LOG_BASE + 24)`  
*Failed to create a new shared memory mutex or failed to attach to an existing one.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_INIT_SIG_SEM_EC (SHMEM_LOG_BASE + 25)`  
*Failed to create signaling semaphore. A Domain Participant's shared memory transport blocks on a signaling semaphore when waiting for data.*
- `#define NETIO_SHMEM_LOG_FAILED_TAKE_SIGN_SEM_EC (SHMEM_LOG_BASE + 27)`  
*Failed to take a shared memory signaling semaphore.*
- `#define NETIO_SHMEM_LOG_SIG_SEM_SIGNAL_FAILED_EC (SHMEM_LOG_BASE + 28)`  
*Failed to give a shared memory signaling semaphore.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_INIT_SEGMENT_ADDR_EC (SHMEM_LOG_BASE + 29)`  
*Failed to attach to a shared memory segment because it doesn't exist.*
- `#define NETIO_SHMEM_LOG_PACKET_INIT_EC (SHMEM_LOG_BASE + 30)`  
*Failed to initialize a receive packet.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_INIT_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 31)`  
*Failed to create a concurrent queue.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_ATTACH_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 32)`  
*Failed to attach to a concurrent queue.*
- `#define NETIO_SHMEM_LOG_INCOMPATIBLE_CONCURRENT_Q_EC (SHMEM_LOG_BASE + 33)`  
*Attempting to attach to an incompatible concurrent queue with incompatible message\_size\_max.*
- `#define NETIO_SHMEM_LOG_FAILED_TO_GET_TIMESTAMP_EC (SHMEM_LOG_BASE + 33)`  
*Failed to get timestamp.*
- `#define NETIO_SHMEM_LOG_NONFATAL_SIG_SEM_RC_EC (SHMEM_LOG_BASE + 34)`  
*Failed to unlock shared memory mutex.*
- `#define NETIO_SHMEM_LOG_INCONSISTENT_MESSAGE_SIZE_EC (SHMEM_LOG_BASE + 35)`  
*Failed to unlock shared memory mutex.*
- `#define NETIO_SHMEM_LOG_RECORD_EC (SHMEM_LOG_BASE + 36)`  
*Database operation on record failed.*
- `#define NETIO_SHMEM_LOG_PACKET_HEAD_EC (SHMEM_LOG_BASE + 37)`  
*Failed to set packet head.*
- `#define NETIO_SHMEM_LOG_PACKET_FWD_EC (SHMEM_LOG_BASE + 38)`  
*Failed reception upstream for a received packet.*
- `#define NETIO_SHMEM_LOG_QUEUE_FULL_EC (SHMEM_LOG_BASE + 39)`  
*Failed reception upstream for a received packet.*
- `#define NETIO_SHMEM_LOG_ROUTE_LOOKUP_FAILED_EC (SHMEM_LOG_BASE + 40)`  
*Failed to evict find an entry in the route table.*
- `#define NETIO_SHMEM_LOG_ROUTE_DELETE_FAILED_EC (SHMEM_LOG_BASE + 41)`  
*Failed to delete an entry from the route table.*
- `#define NETIO_SHMEM_LOG_IGNORE_TRANSPORT_COMPATIBILITY_EC (SHMEM_LOG_BASE + 42)`  
*The shared memory transport is ignoring the requested transport minimum compatibility version because it cannot be backward compatible.*

- `#define NETIO_SHMEM_LOG_UNSUPPORTED_TRANSPORT_COMPATIBILITY_EC (SHMEM_LOG_BASE + 43)`  
*Unsupported minimum compatibility version because it requires a version of the shared memory transport which is not supported.*
- `#define NETIO_SHMEM_LOG_CQ_INT_REPRESENTATION_EC (SHMEM_LOG_BASE + 200)`  
*Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*
- `#define NETIO_SHMEM_LOG_CQ_UNLINKED_EC (SHMEM_LOG_BASE + 201)`  
*Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*
- `#define NETIO_SHMEM_LOG_CQ_INVALID_SIGNATURE_EC (SHMEM_LOG_BASE + 202)`  
*Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*
- `#define NETIO_SHMEM_LOG_CQ_INCOMPATIBLE_VERSION_EC (SHMEM_LOG_BASE + 203)`  
*Error when attaching to concurrent queue. Inconsistent representation of size of integers between concurrent queue and attaching application.*
- `#define NETIO_SHMEM_LOG_CQ_INCONSISTENT_STATE_EC (SHMEM_LOG_BASE + 204)`  
*Found an inconsistent concurrent queue state while performing a a read or write. The queue memory may have been corrupted or otherwise tampered with.*

### 8.12.1 Detailed Description

NETIO Zero Copy Shared Data module log codes.

## 8.13 netio\_zcopy\_log.h File Reference

ZeroCopy module log codes.

```
#include "osapi/osapi_log.h"
```

### 8.13.1 Detailed Description

ZeroCopy module log codes.

## 8.14 netio\_zcopy\_notif\_interface.h File Reference

```
#include "rt/rt_rt.h"  
#include "netio_zcopy/netio_zcopy_notif_mechanism_intf.h"
```

### Classes

- struct [ZCOPY\\_NotifInterfaceFactoryProperty](#)  
*Notification interface property.*

## Macros

- `#define ZCOPY_NotifInterfaceFactoryProperty_INITIALIZER`  
Constant to initialize a `ZCOPY_NotifInterfaceFactoryProperty`.

## Functions

- `RTI_BOOL ZCOPY_NotifInterface_receive (NETIO_Interface_T *netio_intf, const struct NETIO_Address *port, RTI_BOOL *has_more_data_out)`  
Definition of the `NETIO_Interface` receive method.
- `RTI_BOOL ZCOPY_NotifInterfaceFactory_register (RT_Registry_T *registry, const char *name, struct ZCOPY_NotifInterfaceFactoryProperty *property)`  
Register the notification interface factory with a factory property.
- `RTI_BOOL ZCOPY_NotifInterfaceFactory_unregister (RT_Registry_T *registry, const char *name)`  
Unregister the notification interface factory.

### 8.14.1 Function Documentation

#### ZCOPY\_NotifInterface\_receive()

```
RTI_BOOL ZCOPY_NotifInterface_receive (
    NETIO_Interface_T * netio_intf,
    const struct NETIO_Address * port,
    RTI_BOOL * has_more_data_out) [extern]
```

Definition of the `NETIO_Interface` receive method.

#### Parameters

|     |                                |                                           |
|-----|--------------------------------|-------------------------------------------|
| in  | <code>netio_intf</code>        | The notification interface to receive on. |
| in  | <code>port</code>              | The port to receive on.                   |
| out | <code>has_more_data_out</code> | Whether there is more data to receive.    |

#### Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

See Also `::DDSNETIO_Interface::receive`, `NETIO_Interface`.

#### ZCOPY\_NotifInterfaceFactory\_register()

```
RTI_BOOL ZCOPY_NotifInterfaceFactory_register (
    RT_Registry_T * registry,
    const char * name,
    struct ZCOPY_NotifInterfaceFactoryProperty * property) [extern]
```

Register the notification interface factory with a factory property.

**Parameters**

|    |                 |                                            |
|----|-----------------|--------------------------------------------|
| in | <i>registry</i> | The registry used to register the factory. |
| in | <i>name</i>     | The name used to register the factory.     |
| in | <i>property</i> | The property used to register the factory. |

**Returns**

RTI\_TRUE if the factory was registered successfully, RTI\_FALSE otherwise.

**ZCOPY\_NotifInterfaceFactory\_unregister()**

```
RTI_BOOL ZCOPY_NotifInterfaceFactory_unregister (  
    RT_Registry_T * registry,  
    const char * name) [extern]
```

Unregister the notification interface factory.

**Parameters**

|    |                 |                                              |
|----|-----------------|----------------------------------------------|
| in | <i>registry</i> | The registry used to unregister the factory. |
| in | <i>name</i>     | The name used to unregister the factory.     |

**Returns**

RTI\_TRUE if the factory was unregistered successfully, RTI\_FALSE otherwise.

**8.15 netio\_zcopy\_whsq\_log.h File Reference**

WSHQ module log codes.

```
#include "osapi/osapi_log.h"
```

**Macros**

- #define **WSHQ\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC** (WSHQ\_LOG\_BASE + 1)  
*KEEP\_ALL History kind is not supported.*
- #define **WSHQ\_LOG\_UNLIMITED\_HISTORY\_NOT\_SUPPORTED\_EC** (WSHQ\_LOG\_BASE + 2)  
*Unlimited length resource limits unsupported.*
- #define **WSHQ\_LOG\_MAX\_SAMPLES\_TOO\_SMALL\_EC** (WSHQ\_LOG\_BASE + 3)  
*DataWriterQos.resource\_limits.max\_samples set too small.*
- #define **WSHQ\_LOG\_HISTORY\_OBJECT** 1  
*A history object.*

- #define [WHSQ\\_LOG\\_OBJECT\\_ALLOCATE\\_EC](#) (WHSQ\_LOG\_BASE + 4)  
*Failed to allocate object of the specified kind.*
- #define [WHSQ\\_LOG\\_PURGE\\_FAILED\\_EC](#) (WHSQ\_LOG\_BASE + 5)  
*Failed to purge a sample from the shared queue.*
- #define [WHSQ\\_LOG\\_WH\\_CREATE\\_FAILED\\_EC](#) (WHSQ\_LOG\_BASE + 6)  
*Failed to create the underlying writer history.*
- #define [WHSQ\\_LOG\\_OBJECT\\_DELETE\\_EC](#) (WHSQ\_LOG\_BASE + 7)  
*Failed to delete object of the specified kind.*
- #define [WHSQ\\_LOG\\_CONFIGURATION\\_EC](#) (WHSQ\_LOG\_BASE + 8)  
*Listener or Property not configured when creating the component.*
- #define [WHSQ\\_LOG\\_NO\\_DDS\\_FACTORY\\_EC](#) (WHSQ\_LOG\_BASE + 9)  
*No factory for forwarding when creating a writer history instance.*
- #define [WHSQ\\_LOG\\_COMMIT\\_FAILED\\_EC](#) (WHSQ\_LOG\_BASE + 10)  
*Committing a Sample failed.*

### 8.15.1 Detailed Description

WHSQ module log codes.

### 8.15.2 Macro Definition Documentation

#### WHSQ\_LOG\_HISTORY\_OBJECT

```
#define WHSQ_LOG_HISTORY_OBJECT 1
```

A history object.

## 8.16 osapi\_log.h File Reference

OSAPI Log API.

```
#include "osapi/osapi_dll.h"
#include "osapi/osapi_heap.h"
#include "osapi/osapi_time.h"
#include "osapi/osapi_log_impl.h"
```

### Classes

- struct [OSAPI\\_LogProperty](#)  
*Configuration of logging functionality.*

## Macros

- #define `OSAPI_LOG_BASE` (0)  
*Log Id base-number.*
- #define `REDA_LOG_BASE` (1 << 16)  
*REDA Module.*
- #define `DB_LOG_BASE` (2 << 16)  
*DB Module.*
- #define `RT_LOG_BASE` (3 << 16)  
*RT Module.*
- #define `NETIO_LOG_BASE` (4 << 16)  
*NETIO Module.*
- #define `UDP_LOG_BASE` `NETIO_LOG_BASE`  
*UDP Module, same as NETIO.*
- #define `SHMEM_LOG_BASE` `NETIO_LOG_BASE` + 10000  
*NETIO\_SHMEM Module, same as NETIO.*
- #define `SDM_LOG_BASE` `NETIO_LOG_BASE` + 20000  
*SDM Module, same as NETIO.*
- #define `CDR_LOG_BASE` (5 << 16)  
*CDR Module.*
- #define `XCDR_LOG_BASE` `CDR_LOG_BASE` + 20000  
*CDR Module.*
- #define `RTPS_LOG_BASE` (6 << 16)  
*RTPS Module.*
- #define `DDSC_LOG_BASE` (7 << 16)  
*DDS\_C Module.*
- #define `RHSM_LOG_BASE` (8 << 16)  
*RHSM Module.*
- #define `WHSM_LOG_BASE` (9 << 16)  
*WHSM Module.*
- #define `DPSE_LOG_BASE` (10 << 16)  
*DPSE Module.*
- #define `DPDE_LOG_BASE` (11 << 16)  
*DPDE Module.*
- #define `SEC_CORE_LOG_BASE` (12 << 16)  
*DDDS Security plugin.*
- #define `APPGEN_LOG_BASE` (13 << 16)  
*APPGEN Module.*
- #define `NETIO_ZCOPY_LOG_BASE` (14 << 16)  
*NETIO Zero Copy Module.*
- #define `DDS_FILTER_LOG_BASE` (15 << 16)  
*DDS Filter Module.*
- #define `OSAPI_LOG_GET_NEXT_OBJECT_ID_EC` (`OSAPI_LOG_BASE` + 1)  
*Retrieving the next error code failed.*
- #define `OSAPI_LOG_SYSTEM_SET_PROPERTY_EC` (`OSAPI_LOG_BASE` + 2)  
*An error occurred while setting the system properties.*
- #define `OSAPI_LOG_SYSTEM_TIMER_START_EC` (`OSAPI_LOG_BASE` + 4)

- An error occurred when starting the system timer.*

  - #define `OSAPI_LOG_SYSTEM_TIMER_STOP_EC` (`OSAPI_LOG_BASE` + 5)
- An error occurred when stopping the system timer.*

  - #define `OSAPI_LOG_THREAD_NEW_EC` (`OSAPI_LOG_BASE` + 6)
- An error when allocating the a thread object.*

  - #define `OSAPI_LOG_THREAD_CREATE_EC` (`OSAPI_LOG_BASE` + 7)
- An error when creating the a thread object.*

  - #define `OSAPI_LOG_THREAD_SEM_EC` (`OSAPI_LOG_BASE` + 8)
- An error when creating thread sync semaphore.*

  - #define `OSAPI_LOG_THREAD_EXEC_CREATE_EC` (`OSAPI_LOG_BASE` + 9)
- Failed to signal that a thread has been created.*

  - #define `OSAPI_LOG_THREAD_EXEC_START_EC` (`OSAPI_LOG_BASE` + 10)
- Failed to signal the start a created thread.*

  - #define `OSAPI_LOG_THREAD_START_EC` (`OSAPI_LOG_BASE` + 11)
- Failed to start a thread.*

  - #define `OSAPI_LOG_THREAD_DESTROY_EC` (`OSAPI_LOG_BASE` + 12)
- Failed to destroy a thread.*

  - #define `OSAPI_LOG_THREAD_DESTROY_NO_START_EC` (`OSAPI_LOG_BASE` + 13)
- Failed to start an unstarted thread being destroyed.*

  - #define `OSAPI_LOG_THREAD_DESTROY_NO_WAKEUP_EC` (`OSAPI_LOG_BASE` + 14)
- Failed wakeup of a thread being destroyed.*

  - #define `OSAPI_LOG_THREAD_INIT_EC` (`OSAPI_LOG_BASE` + 15)
- Failed initializing a thread.*

  - #define `OSAPI_LOG_THREAD_SCHEDPARAM_EC` (`OSAPI_LOG_BASE` + 16)
- Failed to set scheduling policy of a thread.*

  - #define `OSAPI_LOG_THREAD_GET_POLICY_EC` (`OSAPI_LOG_BASE` + 17)
- Failed to get the scheduling policy of a thread.*

  - #define `OSAPI_LOG_THREAD_POLICY_DIFFER_EC` (`OSAPI_LOG_BASE` + 18)
- Mismatch of scheduling policy of a created thread and the application thread.*

  - #define `OSAPI_LOG_THREAD_PRIORITY_MAP_EC` (`OSAPI_LOG_BASE` + 19)
- Failed to map to native thread priority values.*

  - #define `OSAPI_LOG_TIMER_DELETE_EC` (`OSAPI_LOG_BASE` + 20)
- Failed to delete the Timer object.*

  - #define `OSAPI_LOG_TIMER_TICK_MUTEX_EC` (`OSAPI_LOG_BASE` + 22)
- Failed taking or giving the Timer mutex.*

  - #define `OSAPI_LOG_TIMER_GET_USER_DATA_EPOCH_EC` (`OSAPI_LOG_BASE` + 27)
- Failed to return user data for a timeout due to mismatched epochs.*

  - #define `OSAPI_LOG_TIMER_NEW_EC` (`OSAPI_LOG_BASE` + 28)
- Failed to allocate memory for a new Timer.*

  - #define `OSAPI_LOG_TIMER_NEW_ENTRY_EC` (`OSAPI_LOG_BASE` + 29)
- Failed to allocate memory for a new Timer entry.*

  - #define `OSAPI_LOG_TIMER_NEW_WHEEL_EC` (`OSAPI_LOG_BASE` + 30)
- Failed to allocate memory for a new Timer wheel.*

  - #define `OSAPI_LOG_TIMER_NEW_MUTEX_EC` (`OSAPI_LOG_BASE` + 31)
- Failed to create a new Timer mutex.*

  - #define `OSAPI_LOG_TIMER_NEW_START_TIMER_EC` (`OSAPI_LOG_BASE` + 32)
- Failed to start a new Timer being created.*



- `#define OSAPI_LOG_TIMER_DELETE_STOP_TIMER_EC (OSAPI_LOG_BASE + 33)`  
*Failed to stop a Timer being deleted.*
- `#define OSAPI_LOG_TIMER_DELETE_MUTEX_EC (OSAPI_LOG_BASE + 34)`  
*Failed to delete the Timer mutex.*
- `#define OSAPI_LOG_TIMER_MUTEX_EC (OSAPI_LOG_BASE + 35)`  
*Failed to take or give a Timer mutex.*
- `#define OSAPI_LOG_SEMAPHORE_DELETE_EC (OSAPI_LOG_BASE + 36)`  
*Failed to delete a semaphore.*
- `#define OSAPI_LOG_SEMAPHORE_NEW_EC (OSAPI_LOG_BASE + 37)`  
*Failed to create a semaphore.*
- `#define OSAPI_LOG_SEMAPHORE_NEW_INIT_EC (OSAPI_LOG_BASE + 38)`  
*Failed to initialize a new semaphore.*
- `#define OSAPI_LOG_SEMAPHORE_GIVE_EC (OSAPI_LOG_BASE + 39)`  
*Failed to give a semaphore.*
- `#define OSAPI_LOG_SEMAPHORE_TAKE_EC (OSAPI_LOG_BASE + 40)`  
*Failed to take a semaphore.*
- `#define OSAPI_LOG_MUTEX_DELETE_EC (OSAPI_LOG_BASE + 41)`  
*Failed to delete a mutex.*
- `#define OSAPI_LOG_MUTEX_NEW_EC (OSAPI_LOG_BASE + 42)`  
*Failed to create a mutex.*
- `#define OSAPI_LOG_MUTEX_TAKE_EC (OSAPI_LOG_BASE + 43)`  
*Failed to take a mutex.*
- `#define OSAPI_LOG_MUTEX_GIVE_EC (OSAPI_LOG_BASE + 44)`  
*Failed to give a mutex.*
- `#define OSAPI_LOG_MUTEX_INIT_EC (OSAPI_LOG_BASE + 45)`  
*Failed to initialize a mutex.*
- `#define OSAPI_LOG_HEAP_INTERNAL_ALLOCATE_EC (OSAPI_LOG_BASE + 46)`  
*Failed to allocate a buffer from the heap.*
- `#define OSAPI_LOG_SYSTEM_GET_TIME_EC (OSAPI_LOG_BASE + 48)`  
*Failed to get current system time.*
- `#define OSAPI_LOG_LAST_RECORDED_ERROR_EC (OSAPI_LOG_BASE + 49)`  
*Return the last recorded error-code for the calling thread.*
- `#define OSAPI_LOG_SET_THREAD_NAME_EC (OSAPI_LOG_BASE + 50)`  
*Failed to set the thread name in the OS.*
- `#define OSAPI_LOG_SEM_OPEN_EC (OSAPI_LOG_BASE + 52)`  
*Failure during call VxWorks semOpen(...) or Posix sem\_open(...) when managing shared memory segments, semaphores, or mutexes.*
- `#define OSAPI_LOG_SEM_INFO_GET_EC (OSAPI_LOG_BASE + 53)`  
*Failure during VxWorks call semInfoGet when managing shared memory segments, semaphores, or mutexes.*
- `#define OSAPI_LOG_SD_UNMAP_EC (OSAPI_LOG_BASE + 54)`  
*Failure during VxWorks call sdUnmap when managing shared memory segments, semaphores, or mutexes.*
- `#define OSAPI_LOG_SD_OPEN_EC (OSAPI_LOG_BASE + 55)`  
*Failure during VxWorks call sdOpen when managing shared memory segments, semaphores, or mutexes.*
- `#define OSAPI_LOG_SEM_GIVE_EC (OSAPI_LOG_BASE + 56)`  
*Failure during VxWorks call semGive when managing shared memory segments, semaphores, or mutexes.*
- `#define OSAPI_LOG_SEM_TAKE_EC (OSAPI_LOG_BASE + 57)`  
*Failure during VxWorks call semTake when managing shared memory segments, semaphores, or mutexes.*

- #define `OSAPI_LOG_UNKNOWN_SEMaphore_TYPE_EC` (`OSAPI_LOG_BASE` + 58)  
*Trying to perform an operation an unknown semmutex type.*
- #define `OSAPI_LOG_SEGMENT_KEY_ALREADY_EXISTS_EC` (`OSAPI_LOG_BASE` + 59)  
*Failed to create segment due to a segment already existing.*
- #define `OSAPI_LOG_SEGMENT_KEY_DOESNT_EXIST_EC` (`OSAPI_LOG_BASE` + 60)  
*Segment trying to be attached to does not exist.*
- #define `OSAPI_LOG_SEGMENT_DETACH_FAILED_EC` (`OSAPI_LOG_BASE` + 61)  
*Failed to detach from shared memory segment.*
- #define `OSAPI_LOG_SHMEM_ATTACH_FAILED_EC` (`OSAPI_LOG_BASE` + 62)  
*Attaching to shared memory segment/mutex/signaling semaphore failed.*
- #define `OSAPI_LOG_SHMEM_GIVE_FAILED_EC` (`OSAPI_LOG_BASE` + 63)  
*Failed to give shared memory mutex/signaling semaphore.*
- #define `OSAPI_LOG_SHMEM_TAKE_FAILED_EC` (`OSAPI_LOG_BASE` + 64)  
*Failed to take shared memory mutex/signaling semaphore.*
- #define `OSAPI_LOG_SHMEM_SEM_DELETE_FAILED_EC` (`OSAPI_LOG_BASE` + 65)  
*Failed to delete shared memory semaphore or mutex.*
- #define `OSAPI_LOG_SHMEM_SEM_MUTEX_CREATED_EC` (`OSAPI_LOG_BASE` + 66)  
*Error during shared memory semaphore creation/deletion.*
- #define `OSAPI_LOG_SHMEM_SEGMENT_FILE_MAP_FAILED_EC` (`OSAPI_LOG_BASE` + 67)  
*Windows specific API MapViewOfFile failed.*
- #define `OSAPI_LOG_SHMEM_SET_DESCRIPTOR_EC` (`OSAPI_LOG_BASE` + 68)  
*Windows specific error during setting security descriptor.*
- #define `OSAPI_LOG_SHMEM_INIT_DESCRIPTOR_EC` (`OSAPI_LOG_BASE` + 69)  
*Windows specific error during initialization of security descriptor.*
- #define `OSAPI_LOG_SHMEM_FTRUNCATE_EC` (`OSAPI_LOG_BASE` + 70)  
*Posix specific error during ftruncate(...) call.*
- #define `OSAPI_LOG_SHMEM_MMAP_EC` (`OSAPI_LOG_BASE` + 71)  
*Posix specific error during mmap(...) call.*
- #define `OSAPI_LOG_SHMEM_SHM_OPEN_EC` (`OSAPI_LOG_BASE` + 72)  
*Posix specific error during shm\_open(...) call.*
- #define `OSAPI_LOG_SHMEM_MUNMAP_EC` (`OSAPI_LOG_BASE` + 73)  
*Posix specific error during munmap(...) call.*
- #define `OSAPI_LOG_CLOSE_EC` (`OSAPI_LOG_BASE` + 74)  
*Posix specific error during close(...) call.*
- #define `OSAPI_LOG_SHMEM_UNLINK_EC` (`OSAPI_LOG_BASE` + 76)  
*Posix specific error during unlink(...) call.*
- #define `OSAPI_LOG_EXHAUSTED_POSIX_SEM_LIMIT_EC` (`OSAPI_LOG_BASE` + 77)  
*System ran out of space to create semaphores. Consider increasing Posix sem limit.*
- #define `OSAPI_LOG_SHMEM_SEM_WAIT_EC` (`OSAPI_LOG_BASE` + 78)  
*Posix specific error during unlink(...) call.*
- #define `OSAPI_LOG_SHMEM_SEM_UNLINK_EC` (`OSAPI_LOG_BASE` + 79)  
*Posix specific error during unlink(...) call.*
- #define `OSAPI_LOG_SHMEM_POSIX_GIVE_EC` (`OSAPI_LOG_BASE` + 80)  
*Posix specific error during os specific implementation of a semaphore mutex give.*
- #define `OSAPI_LOG_THREAD_SEMPOOL_EXCEEDED_EC` (`OSAPI_LOG_BASE` + 90)  
*Failed to add a semaphore to the thread semaphore pool.*
- #define `OSAPI_LOG_THREAD_SEMPOOL_INVALID_SEM_EC` (`OSAPI_LOG_BASE` + 91)

- Failed to delete a semaphore from the thread semaphore pool.*

  - #define `OSAPI_LOG_SHMEM_SIZE_EXCEEDED_EC` (`OSAPI_LOG_BASE` + 92)

*Trying to create a shared memory segment that is greater than  $(2^{31}) - 1$ . The middleware cannot support shared memory segments created than  $\sim 2$  GB even though the operating system may.*
- #define `OSAPI_LOG_MATH_OFV_EC` (`OSAPI_LOG_BASE` + 93)

*A math overflow occurred.*
- #define `OSAPI_LOG_INVALID_TIMER_RES_EC` (`OSAPI_LOG_BASE` + 94)

*An invalid timer resolution was specified.*
- #define `OSAPI_LOG_THREAD_JOIN_EC` (`OSAPI_LOG_BASE` + 95)

*Failed to join a thread.*
- #define `OSAPI_LOG_THREAD_GET_MAX_PRIORITY_EC` (`OSAPI_LOG_BASE` + 96)

*Failed to get the maximum priority value for a scheduling policy.*
- #define `OSAPI_LOG_SHMEM_FSTAT_EC` (`OSAPI_LOG_BASE` + 97)

*POSIX specific error during `fstat(...)` call.*
- #define `OSAPI_LOG_SHMEM_MUTEX_INIT_TIMEOUT_EC` (`OSAPI_LOG_BASE` + 97)

*Timed out waiting for a shared memory mutex to be initialized.*
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_INIT_EC` (`OSAPI_LOG_BASE` + 200)

*Failed to create a pthread mutex attribute.*
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPSHARED_EC` (`OSAPI_LOG_BASE` + 201)

*Failed to set the pthread mutex attribute to be process shared.*
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETROBUST_EC` (`OSAPI_LOG_BASE` + 202)

*Failed to set the pthread mutex attribute to be robust.*
- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_DESTROY_EC` (`OSAPI_LOG_BASE` + 203)

*Failed to destroy the pthread mutex attribute.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_INIT_EC` (`OSAPI_LOG_BASE` + 204)

*Failed to initialize a pthread mutex.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_DESTROY_EC` (`OSAPI_LOG_BASE` + 205)

*Failed to destroy a pthread mutex.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_LOCK_EC` (`OSAPI_LOG_BASE` + 206)

*Failed to lock a pthread mutex.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_UNLOCK_EC` (`OSAPI_LOG_BASE` + 207)

*Failed to unlock a pthread mutex.*
- #define `OSAPI_LOG_PTHREAD_MUTEX_CONSISTENT_EC` (`OSAPI_LOG_BASE` + 208)

*Failed to set the pthread mutex to be consistent.*
- #define `OSAPI_LOG_PTHREAD_CONDATTR_INIT_EC` (`OSAPI_LOG_BASE` + 209)

*Failed to create a pthread condition variable attribute.*
- #define `OSAPI_LOG_PTHREAD_CONDATTR_SETPSHARED_EC` (`OSAPI_LOG_BASE` + 210)

*Failed to set the pthread condition variable attribute to be process shared.*
- #define `OSAPI_LOG_PTHREAD_CONDATTR_DESTROY_EC` (`OSAPI_LOG_BASE` + 211)

*Failed to destroy the pthread condition variable attribute.*
- #define `OSAPI_LOG_PTHREAD_COND_INIT_EC` (`OSAPI_LOG_BASE` + 212)

*Failed to initialize a pthread condition variable.*
- #define `OSAPI_LOG_PTHREAD_COND_DESTROY_EC` (`OSAPI_LOG_BASE` + 213)

*Failed to destroy a pthread condition variable.*
- #define `OSAPI_LOG_PTHREAD_COND_WAIT_EC` (`OSAPI_LOG_BASE` + 214)

*Failed to wait on a pthread condition variable.*
- #define `OSAPI_LOG_PTHREAD_COND_SIGNAL_EC` (`OSAPI_LOG_BASE` + 215)

*Failed to signal a pthread condition variable.*

- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETTYPE_EC` (`OSAPI_LOG_BASE` + 216)

*Failed to set the type of a pthread mutex attribute.*

- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPROTOCOL_EC` (`OSAPI_LOG_BASE` + 217)

*Failed to set the protocol of a pthread mutex attribute.*

- #define `OSAPI_LOG_PTHREAD_MUTEXATTR_SETPRIOCEILING_EC` (`OSAPI_LOG_BASE` + 218)

*Failed to set the priority ceiling of a pthread mutex attribute.*

## Typedefs

- typedef void(\* `OSAPI_Log_write_buffer_T`) (const char \*buffer, `RTI_SIZE_T` length)

*Optional user-defined function to output a log/trace buffer.*

- typedef void(\* `OSAPI_LogHandler_T`) (void \*param, `OSAPI_LogEntry_T` \*entry)

*Optional user-defined function for processing new log messages.*

## Enumerations

- enum `OSAPI_LogVerbosity_T` { `OSAPI_LOG_VERBOSITY_SILENT` = 0 , `OSAPI_LOG_VERBOSITY_ERROR` = 1 , `OSAPI_LOG_VERBOSITY_WARNING` = 2 , `OSAPI_LOG_VERBOSITY_DEBUG` = 3 }

*Logging verbosity.*

## Functions

- `RTI_BOOL OSAPI_Log_set_log_handler` (`OSAPI_LogHandler_T` handler, void \*param)  
*Install a log handler.*
- `RTI_BOOL OSAPI_Log_get_log_handler` (`OSAPI_LogHandler_T` \*handler, void \*\*param)  
*Return the current log handler.*
- `RTI_BOOL OSAPI_Log_set_display_handler` (`OSAPI_LogDisplay_T` handler, void \*param)  
*Install a display handler.*
- `RTI_BOOL OSAPI_Log_get_display_handler` (`OSAPI_LogDisplay_T` \*handler, void \*\*param)  
*Return the current display function.*
- void `OSAPI_Log_set_verbosity` (`OSAPI_LogVerbosity_T` verbosity)  
*Set the log verbosity.*
- `RTI_BOOL OSAPI_Log_clear` (void)  
*Clear the log buffer.*
- void `OSAPI_Log_get_property` (struct `OSAPI_LogProperty` \*property)  
*Get the current log properties.*
- `RTI_BOOL OSAPI_Log_set_property` (struct `OSAPI_LogProperty` \*property)  
*Set the log properties.*
- `OSPSLDIExport RTI_INT32 OSAPI_Log_get_last_error_code` (void)  
*Returns the error code for a function that failed.*

### 8.16.1 Detailed Description

OSAPI Log API.

## 8.16.2 Macro Definition Documentation

### OSAPI\_LOG\_BASE

```
#define OSAPI_LOG_BASE (0)
```

Log Id base-number.

### REDA\_LOG\_BASE

```
#define REDA_LOG_BASE (1 << 16)
```

REDA Module.

### DB\_LOG\_BASE

```
#define DB_LOG_BASE (2 << 16)
```

DB Module.

### RT\_LOG\_BASE

```
#define RT_LOG_BASE (3 << 16)
```

RT Module.

### NETIO\_LOG\_BASE

```
#define NETIO_LOG_BASE (4 << 16)
```

NETIO Module.

### UDP\_LOG\_BASE

```
#define UDP_LOG_BASE NETIO\_LOG\_BASE
```

UDP Module, same as NETIO.

### SHMEM\_LOG\_BASE

```
#define SHMEM_LOG_BASE NETIO\_LOG\_BASE + 10000
```

NETIO\_SHMEM Module, same as NETIO.

**SDM\_LOG\_BASE**

```
#define SDM_LOG_BASE NETIO_LOG_BASE + 20000
```

SDM Module, same as NETIO.

**CDR\_LOG\_BASE**

```
#define CDR_LOG_BASE (5 << 16)
```

CDR Module.

**XCDR\_LOG\_BASE**

```
#define XCDR_LOG_BASE CDR_LOG_BASE + 20000
```

CDR Module.

**RTPS\_LOG\_BASE**

```
#define RTPS_LOG_BASE (6 << 16)
```

RTPS Module.

**DDSC\_LOG\_BASE**

```
#define DDSC_LOG_BASE (7 << 16)
```

DDS\_C Module.

**RHSM\_LOG\_BASE**

```
#define RHSM_LOG_BASE (8 << 16)
```

RHSM Module.

**WHSM\_LOG\_BASE**

```
#define WHSM_LOG_BASE (9 << 16)
```

WHSM Module.

**DPSE\_LOG\_BASE**

```
#define DPSE_LOG_BASE (10 << 16)
```

DPSE Module.

**DPDE\_LOG\_BASE**

```
#define DPDE_LOG_BASE (11 << 16)
```

DPDE Module.

**SEC\_CORE\_LOG\_BASE**

```
#define SEC_CORE_LOG_BASE (12 << 16)
```

DDDS Security plugin.

**APPGEN\_LOG\_BASE**

```
#define APPGEN_LOG_BASE (13 << 16)
```

APPGEN Module.

**NETIO\_ZCOPY\_LOG\_BASE**

```
#define NETIO_ZCOPY_LOG_BASE (14 << 16)
```

NETIO Zero Copy Module.

**DDS\_FILTER\_LOG\_BASE**

```
#define DDS_FILTER_LOG_BASE (15 << 16)
```

DDS Filter Module.

**OSAPI\_LOG\_MATH\_OFV\_EC**

```
#define OSAPI_LOG_MATH_OFV_EC (OSAPI_LOG_BASE + 93)
```

A math overflow occurred.

**OSAPI\_LOG\_INVALID\_TIMER\_RES\_EC**

```
#define OSAPI_LOG_INVALID_TIMER_RES_EC (OSAPI_LOG_BASE + 94)
```

An invalid timer resolution was specified.

**8.17 osapi\_process.h File Reference**

process related utility

```
#include "osapi/osapi_dll.h"
#include "osapi/osapi_types.h"
```

**Functions**

- OSPSLDIIExport [RTI\\_BOOL OSAPI\\_Process\\_is\\_alive](#) (OSAPI\_ProcessId pid)  
*Returns true if a process is alive.*
- OSPSLDIIExport OSAPI\_ProcessId [OSAPI\\_Process\\_getpid](#) (void)  
*Return the process ID, which is unique for the application.*

**8.17.1 Detailed Description**

process related utility

**8.18 osapi\_semaphore.h File Reference**

Semaphore interface definition.

```
#include "osapi/osapi_dll.h"
#include "osapi/osapi_types.h"
```

**Macros**

- #define [OSAPI\\_SEMAPHORE\\_TIMEOUT\\_INFINITE](#) -1  
*If [OSAPI\\_Semaphore\\_take](#) is called with [OSAPI\\_SEMAPHORE\\_TIMEOUT\\_INFINITE](#) as timeout, [OSAPI\\_Semaphore\\_take](#) will not return until the semaphore is signaled.*
- #define [OSAPI\\_SEMAPHORE\\_RESULT\\_OK](#) 0  
*If [OSAPI\\_Semaphore\\_take](#) succeeds and the semaphore is signaled within the timeout period, [OSAPI\\_Semaphore\\_take](#) returns TRUE and sets the failure reason to [OSAPI\\_SEMAPHORE\\_RESULT\\_OK](#).*
- #define [OSAPI\\_SEMAPHORE\\_RESULT\\_TIMEOUT](#) 1  
*If [OSAPI\\_Semaphore\\_take](#) was called with a timeout value different from [OSAPI\\_SEMAPHORE\\_TIMEOUT\\_INFINITE](#), and the semaphore was not signaled before the timeout expired, waiting for the semaphore to unblock timed out, [OSAPI\\_Semaphore\\_take](#) returns FALSE and sets the failure reason to [OSAPI\\_SEMAPHORE\\_RESULT\\_TIMEOUT](#).*
- #define [OSAPI\\_SEMAPHORE\\_RESULT\\_ERROR](#) 2  
*If [OSAPI\\_Semaphore\\_take](#) fails for an unknown reason, [OSAPI\\_Semaphore\\_take](#) returns FALSE and sets the failure reason to [OSAPI\\_SEMAPHORE\\_RESULT\\_ERROR](#).*



## Typedefs

- typedef struct OSAPI\_Semaphore [OSAPI\\_Semaphore\\_T](#)  
*Abstract Semaphore type.*

## Functions

- OSPLDIIExport [OSAPI\\_Semaphore\\_T](#) \* [OSAPI\\_Semaphore\\_new](#) (void)  
*Create a Semaphore.*
- OSPLDIIExport [RTI\\_BOOL](#) [OSAPI\\_Semaphore\\_delete](#) ([OSAPI\\_Semaphore\\_T](#) \*self)  
*Delete a semaphore.*
- OSPLDIIExport [RTI\\_BOOL](#) [OSAPI\\_Semaphore\\_take](#) ([OSAPI\\_Semaphore\\_T](#) \*self, [RTI\\_INT32](#) timeout\_ms, [RTI\\_INT32](#) \*fail\_reason)  
*Take a Semaphore.*
- OSPLDIIExport [RTI\\_BOOL](#) [OSAPI\\_Semaphore\\_give](#) ([OSAPI\\_Semaphore\\_T](#) \*self)  
*Give a Semaphore.*

### 8.18.1 Detailed Description

Semaphore interface definition.

### 8.18.2 Function Documentation

#### OSAPI\_Semaphore\_new()

```
OSPLDIIExport OSAPI\_Semaphore\_T * OSAPI_Semaphore_new (  
    void )
```

Create a Semaphore.

The created semaphore is a binary semaphore with an initial semaphore value set to 0 and a maximum value of 1. Thus, a call to [OSAPI\\_Semaphore\\_take](#) will block until the semaphore is signaled with a call to [OSAPI\\_Semaphore\\_give](#).

#### Returns

Pointer to a semaphore with a value of 1 on success, NULL on failure.

#### OSAPI\_Semaphore\_take()

```
OSPLDIIExport RTI\_BOOL OSAPI_Semaphore_take (  
    OSAPI\_Semaphore\_T * self,  
    RTI\_INT32 timeout_ms,  
    RTI\_INT32 * fail_reason)
```

Take a Semaphore.

## Parameters

|     |                    |                                                                                                                                                                                                                                                                                      |
|-----|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>self</i>        | Semaphore previously created with OSAPI_Semaphore_new.                                                                                                                                                                                                                               |
| in  | <i>timeout_ms</i>  | Timeout in ms. The take call will wait at most timeout ms before returning. If OSAPI_SEMAPHORE_TIMEOUT_INFINITE is specified, the call will not return until the semaphore has a value of 1 or higher.                                                                               |
| out | <i>fail_reason</i> | Additional information when the call returns. If RTI_TRUE is returned, but timed out, fail_reason is set to OSAPI_SEMAPHORE_RESULT_TIMEOUT, otherwise fail_reason is set to OSAPI_SEMAPHORE_RESULT_OK. If RTI_FALSE is returned, fail_reason is set to OSAPI_SEMAPHORE_RESULT_ERROR. |

## Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

References [RTI\\_INT32](#).

**OSAPI\_Semaphore\_give()**

```
OSPSLDllExport RTI_BOOL OSAPI_Semaphore_give (  
    OSAPI_Semaphore_T * self)
```

Give a Semaphore.

A mutex can only be given if it is owned by the calling thread. The mutex must be given as many times as it has been taken.

## Parameters

|    |             |                                                        |
|----|-------------|--------------------------------------------------------|
| in | <i>self</i> | Semaphore previously created with OSAPI_Semaphore_new. |
|----|-------------|--------------------------------------------------------|

## Returns

RTI\_TRUE on success, RTI\_FALSE on failure.

References [RTI\\_INT32](#), and [RTI\\_UINT32](#).

**8.19 osapi\_stdio.h File Reference**

Standard I/O interface definition.

```
#include "osapi/osapi_dll.h"  
#include "osapi/osapi_types.h"
```

### 8.19.1 Detailed Description

Standard I/O interface definition.

## 8.20 osapi\_task.h File Reference

Task interface definition.

```
#include "osapi/osapi_config.h"
#include "osapi/osapi_dll.h"
#include "osapi/osapi_types.h"
#include "osapi/osapi_system.h"
#include "osapi/osapi_task.h"
#include "osapi/osapi_time.h"
#include "osapi/osapi_thread.h"
```

### 8.20.1 Detailed Description

Task interface definition.

## 8.21 osapi\_time.h File Reference

Time API definition.

```
#include "osapi/osapi_dll.h"
#include "osapi/osapi_types.h"
```

### Classes

- struct [OSAPI\\_SystemTime](#)  
*System time.*

### Typedefs

- typedef struct OSAPI\_SystemTime [OSAPI\\_SystemTime](#)  
*System time.*

### 8.21.1 Detailed Description

Time API definition.

This file implements functions to convert between time formats.

## 8.22 reda\_log.h File Reference

REDA module log codes.

```
#include "osapi/osapi_log.h"
```

### Macros

- #define REDA\_LOG\_HEAP\_MGR\_SAMPLE\_BUFFERPOOL 1  
*Sample buffer pool object.*
- #define REDA\_LOG\_HEAP\_MGR\_LISTNODE BUFFERPOOL 2  
*Listnode buffer pool object.*
- #define REDA\_LOG\_HEAP\_MGR\_OBJECT\_MEMORY 3  
*Heap manager object.*
- #define REDA\_LOG\_HEAP\_MGR\_INDEXER 4  
*REDA indexer object.*
- #define REDA\_LOG\_HEAP\_MGR\_MUTEX 5  
*REDA heap manager object.*
- #define REDA\_LOG\_BUFFERPOOL\_OUT\_OF\_RESOURCES\_EC (REDA\_LOG\_BASE + 1)  
*Not sufficient memory to allocate buffer-pool.*
- #define REDA\_LOG\_BUFFERPOOL\_BUFFER\_INITIALIZATION\_FAILED\_EC (REDA\_LOG\_BASE + 2)  
*The buffer initialization method failed.*
- #define REDA\_LOG\_BUFFERPOOL\_NOT\_EMPTY\_EC (REDA\_LOG\_BASE + 3)  
*Cannot delete buffer-pool due to buffer(s) not returned to pool.*
- #define REDA\_LOG\_BUFFERPOOL\_DOUBLE\_FREE\_EC (REDA\_LOG\_BASE + 4)  
*A buffer was freed more than once.*
- #define REDA\_LOG\_MEMPOOL\_INCONSISTENT\_PROPERTIES\_EC (REDA\_LOG\_BASE + 5)  
*Inconsistent properties specified for a REDA\_MemPool.*
- #define REDA\_LOG\_MEMPOOL\_OUT\_OF\_RESOURCES\_EC (REDA\_LOG\_BASE + 6)  
*Insufficient memory to allocate a new REDA\_MemPool.*
- #define REDA\_LOG\_MEMPOOL\_POOL\_ALLOCATION\_EC (REDA\_LOG\_BASE + 7)  
*Error during allocation of the memory pool used by a REDA\_MemPool.*
- #define REDA\_LOG\_BUFFERPOOL\_NULL\_POINTER\_EC (REDA\_LOG\_BASE + 8)  
*A pointer in a pool's linked list was NULL.*
- #define REDA\_LOG\_SEQUENCE\_COPY\_FAILED\_EC (REDA\_LOG\_BASE + 100)  
*An error occurred while copying a sequence.*
- #define REDA\_LOG\_SEQUENCE\_ALLOC\_FAILED\_EC (REDA\_LOG\_BASE + 101)  
*An error occurred while allocating space for a sequence.*
- #define REDA\_LOG\_SEQUENCE\_SET\_MAX\_FAILED\_EC (REDA\_LOG\_BASE + 102)  
*An error occurred while setting the maximum sequence size.*
- #define REDA\_LOG\_SEQUENCE\_REALLOCATION\_EC (REDA\_LOG\_BASE + 103)  
*An attempt was made to resize an already allocated sequence.*
- #define REDA\_LOG\_SEQUENCE\_INVALID\_OPERATION\_EC (REDA\_LOG\_BASE + 104)  
*An error occurred while operating on two sequences (copy/compare)*
- #define REDA\_LOG\_SEQUENCE\_INVALID\_LENGTH\_EC (REDA\_LOG\_BASE + 105)  
*An attempt was made to set an illegal length (length < 0 or length > max\_length)*

- #define REDA\_LOG\_SEQUENCE\_INDEX\_OUT\_OF\_BOUNDS\_EC (REDA\_LOG\_BASE + 106)  
*An illegal sequence index was specified (index < 0 or index > length)*
- #define REDA\_LOG\_STRING\_ALLOC\_FAILED\_EC (REDA\_LOG\_BASE + 200)  
*An error occurred while allocating memory for a string.*
- #define REDA\_LOG\_INDEX\_FULL\_EC (REDA\_LOG\_BASE + 300)  
*An attempt was made to add an element to a full index.*
- #define REDA\_LOG\_INDEX\_ENTRY\_EXISTS\_EC (REDA\_LOG\_BASE + 301)  
*An attempt was made to add an existing element to an index.*
- #define REDA\_LOG\_INVALID\_LIST\_NODE\_EC (REDA\_LOG\_BASE + 302)  
*An attempt was made to add an existing element to an index.*
- #define REDA\_LOG\_BUFFER\_OUT\_OF\_MEMORY\_EC (REDA\_LOG\_BASE + 307)  
*Failed to allocate buffer of a specified kind.*
- #define REDA\_LOG\_HEAP\_MGR\_ALLOC\_EC (REDA\_LOG\_BASE + 308)  
*Allocation error in REDA Heap Manager.*
- #define REDA\_LOG\_HEAP\_MGR\_GET\_BUFFER\_EC (REDA\_LOG\_BASE + 309)  
*Error when there are no more available loanable buffers.*
- #define REDA\_LOG\_HEAP\_MGR\_MUTEX\_FAILURE\_EC (REDA\_LOG\_BASE + 310)  
*Error during locking or unlocking of a mutex.*

### 8.22.1 Detailed Description

REDA module log codes.

## 8.23 rh\_sm\_log.h File Reference

RH module log codes.

```
#include "osapi/osapi_log.h"
```

### Macros

- #define RHSM\_LOG\_OUTSTANDING\_SAMPLE\_EC (RHSM\_LOG\_BASE + 1)  
*Failed to delete Reader History due to outstanding, unremoved sample.*
- #define RHSM\_LOG\_NO\_PROPERTY\_QOS\_EC (RHSM\_LOG\_BASE + 2)  
*Failed to create Reader History due to unspecified property Qos.*
- #define RHSM\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC (RHSM\_LOG\_BASE + 3)  
*KEEP\_ALL History kind is not supported.*
- #define RHSM\_LOG\_MAX\_SAMPLE\_TOO\_SMALL\_EC (RHSM\_LOG\_BASE + 4)  
*DataReaderQos.resource\_limits.max\_samples set too small.*
- #define RHSM\_LOG\_TBF\_NOT\_SUPPORTED\_EC (RHSM\_LOG\_BASE + 5)  
*Time-based filtering is not supported.*
- #define RHSM\_LOG\_SAMPLE\_INFO\_POOL\_EC (RHSM\_LOG\_BASE + 6)  
*Failed to allocate a buffer pool for sample info.*
- #define RHSM\_LOG\_SAMPLE\_POOL\_EC (RHSM\_LOG\_BASE + 7)

- Failed to allocate a buffer pool for sample pointers.*

  - #define `RHSM_LOG_SAMPLE_PTR_ARRAY_EC` (`RHSM_LOG_BASE` + 8)
- Failed to get sample buffer.*

  - #define `RHSM_LOG_INFO_ARRAY_EC` (`RHSM_LOG_BASE` + 9)
- Failed to get sample info buffer.*

  - #define `RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC` (`RHSM_LOG_BASE` + 10)
- Failed to get time for reception timestamp.*

  - #define `RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC` (`RHSM_LOG_BASE` + 11)
- An invalid instance replacement policy was specified.*

  - #define `RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC` (`RHSM_LOG_BASE` + 12)
- Failed to remove the oldest sample.*

  - #define `RHSM_LOG_SAMPLE_POOL_EMPTY_EC` (`RHSM_LOG_BASE` + 13)
- Sample pool empty, may have exceeded.*

  - #define `RHSM_LOG_RW_PRUNE_FAILED_EC` (`RHSM_LOG_BASE` + 14)
- Failed to prune remote writer entry.*

  - #define `RHSM_LOG_ENTRY_RESERVATION_FAILED_EC` (`RHSM_LOG_BASE` + 15)
- Failed to reserve an entry.*

  - #define `RHSM_LOG_RETURN_SAMPLE_EC` (`RHSM_LOG_BASE` + 16)
- Failed to return a sample.*

  - #define `RHSM_LOG_HISTORY_UPDATE_EC` (`RHSM_LOG_BASE` + 17)
- Failed to update the history queue.*

  - #define `RHSM_LOG_GET_TIME_EC` (`RHSM_LOG_BASE` + 18)
- Failed to get the system time.*

  - #define `RHSM_LOG_HISTORY_OBJECT` 1
- A history object.*

  - #define `RHSM_LOG_REMOVELIST_OBJECT` 2
- A removal list object.*

  - #define `RHSM_LOG_RWPOOL_OBJECT` 3
- A remote writer pool object.*

  - #define `RHSM_LOG_RWINDEX_OBJECT` 4
- A remote writer index object.*

  - #define `RHSM_LOG_KEY_OBJECT` 5
- A Key object.*

  - #define `RHSM_LOG_RW_OBJECT` 6
- A Key object.*

  - #define `RHSM_LOG_KEYPOOL_OBJECT` 7
- A Key pool object.*

  - #define `RHSM_LOG_OWNER_OBJECT` 8
- A key owner object.*

  - #define `RHSM_LOG_SAMPLEPOOL_OBJECT` 9
- A sample pool object.*

  - #define `RHSM_LOG_KEYINDEXPOOL_OBJECT` 10
- A key index pool.*

  - #define `RHSM_LOG_SAMPLEPTPOOL_OBJECT` 11
- A sample ptr pool object.*

  - #define `RHSM_LOG_SAMPLEINFO_OBJECT` 12
- A sample info pool.*

- `#define RHSM_LOG_INSTANCEHANDLE_OBJECT 13`  
*A sample info pool.*
- `#define RHSM_LOG_OBJECT_ALLOCATE_EC (RHSM_LOG_BASE + 19)`  
*Failed to allocate object of the specified kind.*
- `#define RHSM_LOG_OBJECT_DELETE_EC (RHSM_LOG_BASE + 20)`  
*Failed to delete object of the specified kind.*
- `#define RHSM_LOG_OBJECT_INDEX_EC (RHSM_LOG_BASE + 21)`  
*Failed to index an object of the specified kind.*
- `#define RHSM_LOG_OBJECT_INVALID_EC (RHSM_LOG_BASE + 22)`  
*Invalid object specified.*
- `#define RHSM_LOG_NO_PROPERTY_EC (RHSM_LOG_BASE + 23)`  
*No property when creating a reader history instance.*
- `#define RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 24)`  
*Destination ordering by SOURCE\_TIMESTAMP ordering is not supported.*
- `#define RHSM_LOG_RECOMMIT_FAILURE_EC (RHSM_LOG_BASE + 25)`  
*Failed to recommit sample.*

### 8.23.1 Detailed Description

RH module log codes.

### 8.23.2 Macro Definition Documentation

#### **RHSM\_LOG\_HISTORY\_OBJECT**

```
#define RHSM_LOG_HISTORY_OBJECT 1
```

A history object.

#### **RHSM\_LOG\_REMOVELIST\_OBJECT**

```
#define RHSM_LOG_REMOVELIST_OBJECT 2
```

A removal list object.

#### **RHSM\_LOG\_RWPOOL\_OBJECT**

```
#define RHSM_LOG_RWPOOL_OBJECT 3
```

A remote writer pool object.

**RHSM\_LOG\_RWINDEX\_OBJECT**

```
#define RHSM_LOG_RWINDEX_OBJECT 4
```

A remote writer index object.

**RHSM\_LOG\_KEY\_OBJECT**

```
#define RHSM_LOG_KEY_OBJECT 5
```

A Key object.

**RHSM\_LOG\_RW\_OBJECT**

```
#define RHSM_LOG_RW_OBJECT 6
```

A Key object.

**RHSM\_LOG\_KEYPOOL\_OBJECT**

```
#define RHSM_LOG_KEYPOOL_OBJECT 7
```

A Key pool object.

**RHSM\_LOG\_OWNER\_OBJECT**

```
#define RHSM_LOG_OWNER_OBJECT 8
```

A key owner object.

**RHSM\_LOG\_SAMPLEPOOL\_OBJECT**

```
#define RHSM_LOG_SAMPLEPOOL_OBJECT 9
```

A sample pool object.

**RHSM\_LOG\_KEYINDEXPOOL\_OBJECT**

```
#define RHSM_LOG_KEYINDEXPOOL_OBJECT 10
```

A key index pool.



### RHSM\_LOG\_SAMPLEPTPOOL\_OBJECT

```
#define RHSM_LOG_SAMPLEPTPOOL_OBJECT 11
```

A sample ptr pool object.

### RHSM\_LOG\_SAMPLEINFO\_OBJECT

```
#define RHSM_LOG_SAMPLEINFO_OBJECT 12
```

A sample info pool.

### RHSM\_LOG\_INSTANCEHANDLE\_OBJECT

```
#define RHSM_LOG_INSTANCEHANDLE_OBJECT 13
```

A sample info pool.

## 8.24 rt\_log.h File Reference

```
#include "osapi/osapi_log.h"
```

### Macros

- `#define RT_LOG_SET_IMMUTABLE_PROPERTY_EC (RT_LOG_BASE + 1)`  
*Failed to set registry property due to registry already being enabled.*
- `#define RT_LOG_REGISTRY_INIT_FAILURE_EC (RT_LOG_BASE + 2)`  
*Failed to initialize registry due to failed table creation.*
- `#define RT_LOG_REGISTRY_FINALIZE_EC (RT_LOG_BASE + 3)`  
*Failed to finalize registry due to failed table deletion.*
- `#define RT_LOG_REGISTRY_EXISTS_EC (RT_LOG_BASE + 4)`  
*An attempt was made to register a factory that already existed.*
- `#define RT_LOG_REGISTRY_REGISTER_EC (RT_LOG_BASE + 5)`  
*Error registering a component factory. May have exceeded RT\_RegistryProperty.max\_factories.*
- `#define RT_LOG_REGISTRY_INIT_FACTORY_EC (RT_LOG_BASE + 7)`  
*A registered factory failed to initialize.*
- `#define RT_LOG_REGISTRY_NAME_TOO_LONG_EC (RT_LOG_BASE + 8)`  
*Factory name longer than maximum length of 8.*
- `#define RT_LOG_REGISTRY_INCONSISTENT_CID_EC (RT_LOG_BASE + 9)`  
*Factory name exists, but the class ID is not of the requested type.*
- `#define RT_LOG_REGISTRY_NOT_INITIALIZED_EC (RT_LOG_BASE + 10)`  
*An attempt was made to operate on an uninitialized registry.*

## 8.25 rtps\_config.h File Reference

RTPS Configuration.

### 8.25.1 Detailed Description

RTPS Configuration.

Configurable build flags for RTPS functionality

## 8.26 rtps\_dll.h File Reference

RTPS Configuration.

```
#include "osapi/osapi_config.h"  
#include "rtps/rtps_config.h"
```

### 8.26.1 Detailed Description

RTPS Configuration.

Configurable build flags for RTPS functionality

## 8.27 rtps\_log.h File Reference

RTPS module log codes.

```
#include "osapi/osapi_log.h"
```

## Macros

- #define `RTPS_LOG_INITIALIZE_INTERFACE_EC` (`RTPS_LOG_BASE` + 1)  
*Failed to initialize an RTPS interface.*
- #define `RTPS_LOG_ALLOCATE_EC` (`RTPS_LOG_BASE` + 2)  
*Failed to allocate heap memory for internal resources.*
- #define `RTPS_LOG_CREATE_ROUTE_TABLE_EC` (`RTPS_LOG_BASE` + 3)  
*Failed to create a database table for storing RTPS route info.*
- #define `RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC` (`RTPS_LOG_BASE` + 4)  
*Failed to create database index for route-peer info.*
- #define `RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC` (`RTPS_LOG_BASE` + 5)  
*Failed to create database index for route-transport info.*
- #define `RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC` (`RTPS_LOG_BASE` + 6)  
*Failed to create database index for bind-peer info.*
- #define `RTPS_LOG_INDEX_ADD_ENTRY_EC` (`RTPS_LOG_BASE` + 8)  
*Failed to add an entry to a database index.*
- #define `RTPS_LOG_DB_SELECT_ALL_EC` (`RTPS_LOG_BASE` + 9)  
*Failed to select all entries of a database table.*
- #define `RTPS_LOG_DB_SELECT_MATCH_EC` (`RTPS_LOG_BASE` + 10)  
*Matching entry not found in a database table.*
- #define `RTPS_LOG_DB_SELECT_RANGE_EC` (`RTPS_LOG_BASE` + 11)  
*No matching entries found in a database table.*
- #define `RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC` (`RTPS_LOG_BASE` + 12)  
*Failed to create database entry for route.*
- #define `RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC` (`RTPS_LOG_BASE` + 13)  
*Failed to insert entry into route table.*
- #define `RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC` (`RTPS_LOG_BASE` + 14)  
*Failed to remove entry from route table.*
- #define `RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC` (`RTPS_LOG_BASE` + 15)  
*Failed to delete entry from route table.*
- #define `RTPS_LOG_DB_CREATE_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 16)  
*Failed to create database entry for bind.*
- #define `RTPS_LOG_DB_INSERT_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 17)  
*Failed to insert an entry into the bind table.*
- #define `RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 18)  
*Failed to remove an entry from the bind table.*
- #define `RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 19)  
*Failed to remove an entry from the external bind table.*
- #define `RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 20)  
*Failed to delete entry from external bind table.*
- #define `RTPS_LOG_SEND_EC` (`RTPS_LOG_BASE` + 21)  
*Failed to send a packet due to a lower module failing to send the packet.*
- #define `RTPS_LOG_RECEIVE_EC` (`RTPS_LOG_BASE` + 22)  
*Failed to receive a packet due to a higher module failing to receive the packet.*
- #define `RTPS_LOG_ROUTE_PACKET_EC` (`RTPS_LOG_BASE` + 23)  
*Failed to send a packet, when routing to a lower module or peer.*
- #define `RTPS_LOG_FORWARD_UPSTREAM_EC` (`RTPS_LOG_BASE` + 24)

- Failed to receive a packet, when forwarding to a higher upstream module.*

  - #define `RTPS_LOG_BAD_PARAMETER_EC` (`RTPS_LOG_BASE` + 25)

*Bad parameter to an RTPS interface function.*
- #define `RTPS_LOG_NOT_ENABLED_EC` (`RTPS_LOG_BASE` + 26)

*Failed because interface is not enabled.*
- #define `RTPS_LOG_READER_UNSUPPORTED_EC` (`RTPS_LOG_BASE` + 27)

*RTPS reader does not support send.*
- #define `RTPS_LOG_INTERFACE_MISMATCH_EC` (`RTPS_LOG_BASE` + 28)

*Upstream interface does not match source or destination of packet being sent or received.*
- #define `RTPS_LOG_FULL_SEND_WINDOW_EC` (`RTPS_LOG_BASE` + 29)

*Failed to send due to reaching maximum send window size.*
- #define `RTPS_LOG_NETIO_PACKET_SET_TAIL_EC` (`RTPS_LOG_BASE` + 31)

*Failed to set tail of packet to send.*
- #define `RTPS_LOG_FUNC_UNSUPPORTED_EC` (`RTPS_LOG_BASE` + 32)

*RTPS interface does not support this function.*
- #define `RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC` (`RTPS_LOG_BASE` + 33)

*Out of transport entries.*
- #define `RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC` (`RTPS_LOG_BASE` + 34)

*Out of peer entries.*
- #define `RTPS_LOG_ASSERT_PEER_EC` (`RTPS_LOG_BASE` + 35)

*Failed to assert a remote writer or reader.*
- #define `RTPS_LOG_ASSERT_TRANSPORT_EC` (`RTPS_LOG_BASE` + 36)

*Failed to assert a downstream transport.*
- #define `RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC` (`RTPS_LOG_BASE` + 37)

*Failed to lookup an external interface.*
- #define `RTPS_LOG_NONEXISTENT_ROUTE_EC` (`RTPS_LOG_BASE` + 38)

*Failed to delete a nonexistent route.*
- #define `RTPS_LOG_NONEXISTENT_BIND_EC` (`RTPS_LOG_BASE` + 39)

*Failed to remove a nonexistent bind entry.*
- #define `RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC` (`RTPS_LOG_BASE` + 40)

*Failed to remove a nonexistent external bind entry.*
- #define `RTPS_LOG_STALE_ACK_EPOCH_EC` (`RTPS_LOG_BASE` + 41)

*Dropped an ACKNACK submessage with an old epoch.*
- #define `RTPS_LOG_STALE_HB_EPOCH_EC` (`RTPS_LOG_BASE` + 42)

*Dropped a HEARTBEAT submessage with an old epoch.*
- #define `RTPS_LOG_ACK_EC` (`RTPS_LOG_BASE` + 43)

*Failed an acknack() upstream.*
- #define `RTPS_LOG_REQUEST_EC` (`RTPS_LOG_BASE` + 45)

*Failed a request() upstream.*
- #define `RTPS_LOG_RETURN_LOAN_EC` (`RTPS_LOG_BASE` + 46)

*Failed a return\_loan() upstream.*
- #define `RTPS_LOG_NETIO_PACKET_SET_HEAD_EC` (`RTPS_LOG_BASE` + 47)

*Failed to set the head of a packet.*
- #define `RTPS_LOG_SEND_HEARTBEAT_EC` (`RTPS_LOG_BASE` + 48)

*Failed to send a HEARTBEAT submessage.*
- #define `RTPS_LOG_SHIFT_BITMAP_EC` (`RTPS_LOG_BASE` + 49)

*Failed to shift a bitmap.*

- #define `RTPS_LOG_FULLY_ACKED_READER_EC` (`RTPS_LOG_BASE` + 50)  
*A Reader is fully acknowledged and does not need to respond to a final HEARTBEAT.*
- #define `RTPS_LOG_DATA_OUT_OF_RANGE_EC` (`RTPS_LOG_BASE` + 51)  
*Dropping a DATA submessage whose sequence number is outside of a Reader's receive window.*
- #define `RTPS_LOG_DATA_ALREADY_RECEIVED_EC` (`RTPS_LOG_BASE` + 52)  
*Dropping a DATA submessage whose sequence number was previously received.*
- #define `RTPS_LOG_INTERFACE_NOT_ENABLED_EC` (`RTPS_LOG_BASE` + 53)  
*Failed to receive a message because interface is not enabled.*
- #define `RTPS_LOG_INVALID_PACKET_EC` (`RTPS_LOG_BASE` + 54)  
*Dropped a message with an invalid packet header.*
- #define `RTPS_LOG_UNSUPPORTED_INTERFACE_EC` (`RTPS_LOG_BASE` + 55)  
*Received a message on an unsupported interface.*
- #define `RTPS_LOG_UNKNOWN_SUBMESSAGE_EC` (`RTPS_LOG_BASE` + 56)  
*Received a submessage with an unknown ID.*
- #define `RTPS_LOG_PROCESS_ACKNACK_EC` (`RTPS_LOG_BASE` + 57)  
*Failed to process received ACKNACK submessage.*
- #define `RTPS_LOG_PROCESS_DATA_EC` (`RTPS_LOG_BASE` + 58)  
*Failed to process received DATA submessage.*
- #define `RTPS_LOG_PROCESS_GAP_EC` (`RTPS_LOG_BASE` + 59)  
*Failed to process received GAP submessage.*
- #define `RTPS_LOG_PROCESS_HEARTBEAT_EC` (`RTPS_LOG_BASE` + 60)  
*Failed to process received HEARTBEAT submessage.*
- #define `RTPS_LOG_CREATE_HB_EVENT_EC` (`RTPS_LOG_BASE` + 61)  
*Failed to create periodic HEARTBEAT event.*
- #define `RTPS_LOG_CREATE_EVENT_TIMER_EC` (`RTPS_LOG_BASE` + 62)  
*Failed to create periodic HEARTBEAT event timer.*
- #define `RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC` (`RTPS_LOG_BASE` + 63)  
*Failed to set bitmap bit due to out-of-range sequence number.*
- #define `RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC` (`RTPS_LOG_BASE` + 65)  
*Invalid range of sequence numbers to fill in bitmap.*
- #define `RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC` (`RTPS_LOG_BASE` + 66)  
*Deserialized an invalid out-of-bounds bitcount for a bitmap.*
- #define `RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC` (`RTPS_LOG_BASE` + 67)  
*Failed to shift bitmap due to invalid starting sequence number.*
- #define `RTPS_LOG_SERIALIZE_HOST_ID_EC` (`RTPS_LOG_BASE` + 68)  
*Failed to serialize host ID of GUID.*
- #define `RTPS_LOG_SERIALIZE_APP_ID_EC` (`RTPS_LOG_BASE` + 69)  
*Failed to serialize app ID of GUID.*
- #define `RTPS_LOG_SERIALIZE_INSTANCE_ID_EC` (`RTPS_LOG_BASE` + 70)  
*Failed to serialized instance ID of GUID.*
- #define `RTPS_LOG_SERIALIZE_OBJECT_ID_EC` (`RTPS_LOG_BASE` + 71)  
*Failed to serialize object ID of GUID.*
- #define `RTPS_LOG_DESERIALIZE_HOST_ID_EC` (`RTPS_LOG_BASE` + 72)  
*Failed to deserialize host ID of GUID.*
- #define `RTPS_LOG_DESERIALIZE_APP_ID_EC` (`RTPS_LOG_BASE` + 73)  
*Failed to deserialize app ID of GUID.*
- #define `RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC` (`RTPS_LOG_BASE` + 74)

- Failed to deserialize instance ID of GUID.*

  - #define [RTPS\\_LOG\\_DESERIALIZE\\_OBJECT\\_ID\\_EC](#) (RTPS\_LOG\_BASE + 75)
- Failed to deserialize object ID of GUID.*

  - #define [RTPS\\_LOG\\_DB\\_CREATE\\_EXT\\_BIND\\_ENTRY\\_EC](#) (RTPS\_LOG\_BASE + 76)
- Failed to create database entry for external bind table.*

  - #define [RTPS\\_LOG\\_BITMAP\\_SET\\_BIT\\_EC](#) (RTPS\_LOG\_BASE + 77)
- Failed to set bit in bitmap.*

  - #define [RTPS\\_LOG\\_UNSUPPORTED\\_INFO\\_REPLY\\_EC](#) (RTPS\_LOG\_BASE + 78)
- Received and dropped currently unsupported INFO\_REPLY submessage.*

  - #define [RTPS\\_LOG\\_UNSUPPORTED\\_INFO\\_REPLY\\_IP4\\_EC](#) (RTPS\_LOG\_BASE + 79)
- Received and dropped currently unsupported INFO\_REPLY\_IP4 submessage.*

  - #define [RTPS\\_LOG\\_UNSUPPORTED\\_INFO\\_SRC\\_EC](#) (RTPS\_LOG\_BASE + 80)
- Received and dropped currently unsupported INFO\_SRC submessage.*

  - #define [RTPS\\_LOG\\_DELETE\\_INDEX\\_EC](#) (RTPS\_LOG\_BASE + 81)
- Failed to delete index of a database table.*

  - #define [RTPS\\_LOG\\_DELETE\\_TABLE\\_EC](#) (RTPS\_LOG\_BASE + 82)
- Failed to delete a database table.*

  - #define [RTPS\\_LOG\\_DB\\_LOCK\\_EC](#) (RTPS\_LOG\_BASE + 83)
- Failed to take database lock.*

  - #define [RTPS\\_LOG\\_DB\\_UNLOCK\\_EC](#) (RTPS\_LOG\_BASE + 84)
- Failed to give database lock.*

  - #define [RTPS\\_LOG\\_CREATE\\_BIND\\_TABLE\\_EC](#) (RTPS\_LOG\_BASE + 85)
- Failed to create a database table for storing RTPS bind info.*

  - #define [RTPS\\_LOG\\_STATUS\\_CHANGE\\_EC](#) (RTPS\_LOG\_BASE + 86)
- Failed to update status for reliable reader activity changed.*

  - #define [RTPS\\_LOG\\_SET\\_GAP\\_EC](#) (RTPS\_LOG\_BASE + 87)
- Failed to update status for reliable reader activity changed.*

  - #define [RTPS\\_LOG\\_WINDOW\\_POOL\\_ALLOC\\_EC](#) (RTPS\_LOG\_BASE + 88)
- Out of memory to allocate send window pool.*

  - #define [RTPS\\_LOG\\_GET\\_WINDOW\\_BUFFER\\_EC](#) (RTPS\_LOG\_BASE + 89)
- Failed to get buffer from send window pool.*

  - #define [RTPS\\_LOG\\_INSERT\\_WINDOW\\_FULL\\_EC](#) (RTPS\_LOG\_BASE + 90)
- Cannot insert sample into full window.*

  - #define [RTPS\\_LOG\\_WINDOW\\_INSERT\\_EC](#) (RTPS\_LOG\_BASE + 91)
- Failed to insert sample into send window.*

  - #define [RTPS\\_LOG\\_TIMER\\_DELETE\\_TIMEOUT\\_EC](#) (RTPS\_LOG\_BASE + 92)
- Failed to delete a timeout event.*

  - #define [RTPS\\_LOG\\_BUFFERPOOL\\_DELETE\\_EC](#) (RTPS\_LOG\_BASE + 93)
- Failed to delete a buffer pool.*

  - #define [RTPS\\_LOG\\_DIRECT\\_SEND\\_EC](#) (RTPS\_LOG\_BASE + 94)
- Failed to send a message to a specific peer.*

  - #define [RTPS\\_LOG\\_PACKET\\_INITIALIZE\\_EC](#) (RTPS\_LOG\_BASE + 95)
- RTPS writer failed to initialize a packet.*

  - #define [RTPS\\_LOG\\_WRITER\\_REQUEST\\_SENT\\_HISTORY\\_EC](#) (RTPS\_LOG\_BASE + 96)
- RTPS reliable writer failed to request and resend history.*

  - #define [RTPS\\_LOG\\_BITMAP\\_SHIFT\\_EC](#) (RTPS\_LOG\_BASE + 97)
- Failed to shift bitmap to new lead sequence number.*

- #define `RTPS_LOG_WINDOW_ADVANCE_EC` (`RTPS_LOG_BASE` + 98)  
*Send window of a remote reader failed to advance.*
- #define `RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC` (`RTPS_LOG_BASE` + 99)  
*Failed when advancing window ahead of unacknowledged sequence number.*
- #define `RTPS_LOG_LOOKUP_PEER_EC` (`RTPS_LOG_BASE` + 100)  
*Failed to find peer that should exist.*
- #define `RTPS_LOG_READER_SEND_ACKNACK_EC` (`RTPS_LOG_BASE` + 101)  
*RTPS reader failed to send an ACKNACK message.*
- #define `RTPS_LOG_FINALIZE_INTERFACE_EC` (`RTPS_LOG_BASE` + 102)  
*Failed to finalize an RTPS interface.*
- #define `RTPS_LOG_INDEXER_REMOVE_ENTRY_EC` (`RTPS_LOG_BASE` + 103)  
*Failed to remove index entry for external interface upon deleting RTPS interface.*
- #define `RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC` (`RTPS_LOG_BASE` + 104)  
*Failed when looking up external bind entry from database.*
- #define `RTPS_LOG_SEQ_INIT_EC` (`RTPS_LOG_BASE` + 105)  
*Failed to initialize a local sequence.*
- #define `RTPS_LOG_SEQ_MAX_EC` (`RTPS_LOG_BASE` + 106)  
*Failed to set maximum of a local sequence.*
- #define `RTPS_LOG_SEQ_LEN_EC` (`RTPS_LOG_BASE` + 107)  
*Failed to set length of a local sequence.*
- #define `RTPS_LOG_INTF_MODE_UNDEF_EC` (`RTPS_LOG_BASE` + 108)  
*Initializing an RTPS interface with undefined mode.*
- #define `RTPS_LOG_DELETE_BUFFER_POOL_EC` (`RTPS_LOG_BASE` + 109)  
*Failed to delete buffer pool.*
- #define `RTPS_LOG_CREATE_BUFFER_POOL_EC` (`RTPS_LOG_BASE` + 110)  
*Failed to create buffer pool.*
- #define `RTPS_LOG_CREATE_PEERS_INDEX_EC` (`RTPS_LOG_BASE` + 111)  
*Failed to create index of peers.*
- #define `RTPS_LOG_DELETE_INDEXER_EC` (`RTPS_LOG_BASE` + 112)  
*Failed to delete indexer.*
- #define `RTPS_LOG_GET_PEER_BUFFER_EC` (`RTPS_LOG_BASE` + 113)  
*Failed to get buffer from peer buffer pool.*
- #define `RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC` (`RTPS_LOG_BASE` + 114)  
*Failed to create database index for direct sends.*
- #define `RTPS_LOG_DB_CREATE_GROUP_INDEX_EC` (`RTPS_LOG_BASE` + 115)  
*Failed to create database index for group sends.*
- #define `RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC` (`RTPS_LOG_BASE` + 116)  
*Insufficient buffer in packet.*
- #define `RTPS_LOG_TRANSFORM_RTPS_EC` (`RTPS_LOG_BASE` + 117)  
*Failed to transform rtps message.*
- #define `RTPS_LOG_RESTORE_TRUST_LOAN_BUFFER_EC` (`RTPS_LOG_BASE` + 118)  
*Failed to restore trust loan buffer.*
- #define `RTPS_LOG_PROCESS_DATA_BATCH_EC` (`RTPS_LOG_BASE` + 136)  
*Failed to process received DATA BATCH submessage.*
- #define `RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC` (`RTPS_LOG_BASE` + 137)  
*Failed to process received HEARTBEAT\_BATCH submessage.*
- #define `RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC` (`RTPS_LOG_BASE` + 138)

*The reservation count is zero.*

- #define [RTPS\\_LOG\\_TRUST\\_REMOTE\\_PARTICIPANT\\_HANDLE\\_LOOKUP\\_FAILED\\_EC](#) (RTPS\_LOG\_BASE + 139)

*Failed to lookup a remote participant's interceptor handle.*

- #define [RTPS\\_LOG\\_TRUST\\_REMOTE\\_PARTICIPANT\\_HANDLE\\_CREATE\\_FAILED\\_EC](#) (RTPS\_LOG\_BASE + 140)

*Failed to create a record to store a remote participant's interceptor handle.*

- #define [RTPS\\_LOG\\_TRUST\\_REMOTE\\_PARTICIPANT\\_HANDLE\\_INSERT\\_FAILED\\_EC](#) (RTPS\_LOG\_BASE + 141)

*Failed to insert a record in the table storing interceptor handles of matched remote participants.*

- #define [RTPS\\_LOG\\_TRUST\\_INTERCEPTOR\\_HANDLE\\_LIST\\_FULL\\_EC](#) (RTPS\_LOG\_BASE + 142)

*Interceptor handle list out of resources.*

- #define [RTPS\\_LOG\\_TRUST\\_INCONSISTENT\\_INTERCEPTOR\\_HANDLE\\_EC](#) (RTPS\_LOG\_BASE + 143)

*Inconsistent handles found for remote peer.*

- #define [RTPS\\_LOG\\_TRUST\\_BIND\\_FAILED\\_EC](#) (RTPS\_LOG\_BASE + 144)

*Failed to bind resources for trust behavior.*

- #define [RTPS\\_LOG\\_TRUST\\_SUBMSG\\_CONVERSION\\_FAILED\\_EC](#) (RTPS\_LOG\_BASE + 145)

*Failed to bind resources for trust behavior.*

- #define [RTPS\\_LOG\\_TRUST\\_INVALID\\_SUBMSG\\_EC](#) (RTPS\_LOG\_BASE + 146)

*A malformed trust message was received.*

- #define [RTPS\\_LOG\\_TRUST\\_UNEXPECTED\\_READER\\_STATE\\_EC](#) (RTPS\_LOG\_BASE + 147)

*The RTPS reader was in an unexpected state.*

- #define [RTPS\\_LOG\\_DB\\_CREATE\\_SELECTED\\_GROUP\\_INDEX\\_EC](#) (RTPS\_LOG\_BASE + 148)

*Failed to create database index for the selected route group.*

- #define [RTPS\\_LOG\\_LOCK\\_EC](#) (RTPS\_LOG\_BASE + 149)

*Failed to take a lock.*

- #define [RTPS\\_LOG\\_UNLOCK\\_EC](#) (RTPS\_LOG\_BASE + 150)

*Failed to give a lock.*

- #define [RTPS\\_LOG\\_SCHEDULE\\_PACKET\\_EC](#) (RTPS\_LOG\_BASE + 151)

*Failed to schedule a packet.*

- #define [RTPS\\_LOG\\_SEQ\\_FINALIZE\\_EC](#) (RTPS\_LOG\_BASE + 152)

*Failed to finalize a sequence.*

- #define [RTPS\\_LOG\\_CHECKSUM\\_ILLEGAL\\_OVERRIDE\\_CHECKSUM128\\_EC](#) (RTPS\_LOG\_BASE + 200)

*The property tries to override the BUILTIN128 checksum without setting allow\_builtin\_override to TRUE.*

- #define [RTPS\\_LOG\\_CHECKSUM\\_ILLEGAL\\_OVERRIDE\\_CHECKSUM32\\_EC](#) (RTPS\_LOG\_BASE + 201)

*The property tries to override the BUILTIN32 checksum without setting allow\_builtin\_override to TRUE.*

- #define [RTPS\\_LOG\\_CHECKSUM\\_ILLEGAL\\_OVERRIDE\\_CHECKSUM64\\_EC](#) (RTPS\_LOG\_BASE + 202)

*The property tries to override the BUILTIN64 checksum without setting allow\_builtin\_override to TRUE.*

- #define [RTPS\\_LOG\\_CHECKSUM\\_INVALID\\_OVERRIDE\\_BUILTIN128\\_EC](#) (RTPS\_LOG\_BASE + 203)

*The property override of the BUILTIN128 checksum is invalid. It is not legal to reuse any of the built-in functions.*

- #define [RTPS\\_LOG\\_CHECKSUM\\_INVALID\\_OVERRIDE\\_BUILTIN32\\_EC](#) (RTPS\_LOG\_BASE + 204)

*The property override of the BUILTIN32 checksum is invalid. It is not legal to reuse any of the built-in functions functions.*

- #define [RTPS\\_LOG\\_CHECKSUM\\_INVALID\\_OVERRIDE\\_BUILTIN64\\_EC](#) (RTPS\_LOG\_BASE + 205)

*The property override of the BUILTIN64 checksum is invalid. It is not legal to reuse any of the built-in functions functions.*

- #define [RTPS\\_LOG\\_CHECKSUM\\_CHECKSUM\\_CALCULATION\\_FAILED\\_EC](#) (RTPS\_LOG\_BASE + 210)

*RTPS\_Interface\_calculate\_checksum() failed to calculate the checksum.*

- #define [RTPS\\_LOG\\_UNSUPPORTED\\_MANDATORY\\_HDR\\_EXT\\_PID\\_EC](#) (RTPS\_LOG\_BASE + 211)



- The HeaderExtension contained a mandatory PID that was not understood.*

  - #define `RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC` (`RTPS_LOG_BASE` + 212)

*An invalid HeaderExtension PID was detected.*
- #define `RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC` (`RTPS_LOG_BASE` + 213)

*Failed to decode the HeaderExtension PID list.*
- #define `RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC` (`RTPS_LOG_BASE` + 214)

*The HeaderExtension length is inconsistent with the decoded encoded HeaderExtension length.*
- #define `RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC` (`RTPS_LOG_BASE` + 215)

*A RTPS message which was longer than the indicated message length in the RTPS header was received. This causes the rest of the RTPS message to be invalidated and the remaining submessages to be ignored.*
- #define `RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC` (`RTPS_LOG_BASE` + 216)

*The CRC32 submsg flags are invalid.*
- #define `RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC` (`RTPS_LOG_BASE` + 217)

*A header extension was received, but the checksum length is not present.*
- #define `RTPS_LOG_CHECKSUM_UNSUPPORTED_EC` (`RTPS_LOG_BASE` + 218)

*Unsupported checksum bits received, message dropped.*
- #define `RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC` (`RTPS_LOG_BASE` + 219)

*Checksum length could not be deserialized.*
- #define `RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC` (`RTPS_LOG_BASE` + 220)

*The payload length is less than the deserialized checksum length.*
- #define `RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC` (`RTPS_LOG_BASE` + 221)

*Checksum length could not be deserialized.*
- #define `RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC` (`RTPS_LOG_BASE` + 222)

*The checksum is invalid.*
- #define `RTPS_LOG_CHECKSUM_TOO_MANY_SG_BUFFERS_EC` (`RTPS_LOG_BASE` + 223)

*Found too many scatter-gather buffers when calculating checksum.*
- #define `RTPS_LOG_SEND_LIVELINESS_EC` (`RTPS_LOG_BASE` + 224)

*Failure to send Liveliness packets.*
- #define `RTPS_LOG_DELETE_RECORD_EC` (`RTPS_LOG_BASE` + 225)

*Failure to delete a record.*
- #define `RTPS_LOG_APPLY_CONTENT_FILTER_EC` (`RTPS_LOG_BASE` + 300)

*Failed to apply a content filter to a message.*
- #define `RTPS_LOG_UPDATE_RELIABLE_PEERS_FILTER_EC` (`RTPS_LOG_BASE` + 301)

*Failed to update the state of reliable peers while applying filtering.*
- #define `RTPS_LOG_ADD_FILTERED_ROUTE_EC` (`RTPS_LOG_BASE` + 302)

*Failed to add a filtered route.*
- #define `RTPS_LOG_DELETE_FILTERED_ROUTE_EC` (`RTPS_LOG_BASE` + 303)

*Failed to delete a filtered route.*

### 8.27.1 Detailed Description

RTPS module log codes.

Log codes of the RTPS module

### 8.27.2 Macro Definition Documentation

#### RTPS\_LOG\_DB\_REMOVE\_ROUTE\_ENTRY\_EC

```
#define RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 14)
```

Failed to remove entry from route table.

Failure reason given by database return code (dbrc)

## 8.28 rtps\_rtps\_impl.h File Reference

Implementation of RTPS interface functions and types.

```
#include "reda/reda_sequenceNumber.h"
#include "osapi/osapi_config.h"
```

### 8.28.1 Detailed Description

Implementation of RTPS interface functions and types.

RTPS protocol defined types, implemented in C.

## 8.29 wh\_sm\_log.h File Reference

WH module log codes.

```
#include "osapi/osapi_log.h"
```

### Macros

- #define WHSM\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC (WHSM\_LOG\_BASE + 1)  
*KEEP\_ALL History kind is not supported.*
- #define WHSM\_LOG\_UNLIMITED\_HISTORY\_NOT\_SUPPORTED\_EC (WHSM\_LOG\_BASE + 2)  
*Unlimited length resource limits unsupported.*
- #define WHSM\_LOG\_MAX\_SAMPLES\_TOO\_SMALL\_EC (WHSM\_LOG\_BASE + 3)  
*DataWriterQos.resource\_limits.max\_samples set too small.*
- #define WHSM\_LOG\_HISTORY\_OBJECT 1  
*A history object.*
- #define WHSM\_LOG\_KEY\_OBJECT 2  
*A key object.*
- #define WHSM\_LOG\_SAMPLE\_OBJECT 2  
*A sample object.*

- `#define WHSM_LOG_KEYPOOL_OBJECT 3`  
*A key-pool object.*
- `#define WHSM_LOG_SAMPLEPOOL_OBJECT 4`  
*A sample-pool object.*
- `#define WHSM_LOG_KEYINDEX_OBJECT 5`  
*A key index pool.*
- `#define WHSM_LOG_HISTORYINDEX_OBJECT 6`  
*A key index pool.*
- `#define WHSM_LOG_SAMPLEINDEX_OBJECT 7`  
*A key index pool.*
- `#define WHSM_LOG_OBJECT_ALLOCATE_EC (WHSM_LOG_BASE + 4)`  
*Failed to allocate object of the specified kind.*
- `#define WHSM_LOG_OBJECT_DELETE_EC (WHSM_LOG_BASE + 5)`  
*Failed to delete object of the specified kind.*
- `#define WHSM_LOG_OBJECT_INDEX_EC (WHSM_LOG_BASE + 6)`  
*Failed to index an object of the specified kind.*
- `#define WHSM_LOG_NO_PROPERTY_EC (WHSM_LOG_BASE + 7)`  
*No property when creating a writer history instance.*
- `#define WHSM_LOG_INVALID_INDEX_OBJECT_EC (WHSM_LOG_BASE + 8)`  
*The sample removed from an index did not match sample's key.*

### 8.29.1 Detailed Description

WH module log codes.

### 8.29.2 Macro Definition Documentation

#### WHSM\_LOG\_HISTORY\_OBJECT

```
#define WHSM_LOG_HISTORY_OBJECT 1
```

A history object.

#### WHSM\_LOG\_KEY\_OBJECT

```
#define WHSM_LOG_KEY_OBJECT 2
```

A key object.

#### WHSM\_LOG\_SAMPLE\_OBJECT

```
#define WHSM_LOG_SAMPLE_OBJECT 2
```

A sample object.

**WHSM\_LOG\_KEYPOOL\_OBJECT**

```
#define WHSM_LOG_KEYPOOL_OBJECT 3
```

A key-pool object.

**WHSM\_LOG\_SAMPLEPOOL\_OBJECT**

```
#define WHSM_LOG_SAMPLEPOOL_OBJECT 4
```

A sample-pool object.

**WHSM\_LOG\_KEYINDEX\_OBJECT**

```
#define WHSM_LOG_KEYINDEX_OBJECT 5
```

A key index pool.

**WHSM\_LOG\_HISTORYINDEX\_OBJECT**

```
#define WHSM_LOG_HISTORYINDEX_OBJECT 6
```

A key index pool.

**WHSM\_LOG\_SAMPLEINDEX\_OBJECT**

```
#define WHSM_LOG_SAMPLEINDEX_OBJECT 7
```

A key index pool.



## Index

- `_contiguous_buffer`
    - `FooSeq`, [816](#)
  - `_element_size`
    - `FooSeq`, [817](#)
  - `_flags`
    - `FooSeq`, [817](#)
  - `_length`
    - `FooSeq`, [817](#)
  - `_maximum`
    - `FooSeq`, [816](#)
  - `_token1`
    - `FooSeq`, [817](#)
  - `_token2`
    - `FooSeq`, [817](#)
  - `~FooSeq`
    - `FooSeq`, [807](#)
- `accept_unknown_peers`
  - `DDS_DiscoveryQosPolicy`, [527](#)
- `access_scope`
  - `DDS_PresentationQosPolicy`, [582](#)
- `active_count`
  - `DDS_ReliableReaderActivityChangedStatus`, [601](#)
- `active_count_change`
  - `DDS_ReliableReaderActivityChangedStatus`, [602](#)
- `add_entry`
  - UDP Transport API, [153](#)
- `add_peer`
  - `DDSDomainParticipant`, [706](#)
- `add_property`
  - `DDSPropertyQosPolicyHelper`, [735](#)
- `add_route`
  - `ZCOPY_NotifUserInterfaceI`, [865](#)
- `address`
  - `DDS_Locator_t`, [565](#)
  - `NETIO_AddressValue`, [827](#)
  - `NETIO_DGRAM_InterfaceRouteEntry`, [832](#)
  - `NETIO_DGRAM_InterfaceTableEntry`, [834](#)
  - `UDP_InterfaceTableEntry`, [857](#)
- `address_high`
  - `NETIO_DGRAM_InterfaceMultiCastGroup`, [831](#)
- `address_low`
  - `NETIO_DGRAM_InterfaceMultiCastGroup`, [831](#)
- `alive_count`
  - `DDS_LivelinessChangedStatus`, [559](#)
- `alive_count_change`
  - `DDS_LivelinessChangedStatus`, [560](#)
- `allow_builtin_override`
  - RTPS Transport API, [158](#)
- `allow_interface`
  - `UDP_InterfaceFactoryProperty`, [853](#)
- `allowed_crc_mask`
  - `DDS_ChecksumProperty`, [497](#)
  - `DDS_ChecksumProperty_t`, [498](#)
  - `DDS_WireProtocolQosPolicy`, [643](#)
- `app_gen_log.h`, [868](#)
- `APPGEN`, [482](#)
  - `APPGEN_APP_PARTICIPANT_NOT_FOUND_EC`, [484](#)
  - `APPGEN_FACTORY_REGISTER_EC`, [483](#)
  - `APPGEN_FACTORY_UNREGISTER_EC`, [483](#)
  - `APPGEN_LOG_ALLOCATE_EC`, [483](#)
  - `APPGEN_LOG_API_ERROR_EC`, [484](#)
  - `APPGEN_LOG_APP_CONFIG_NOT_CORRECT_EC`, [484](#)
  - `APPGEN_LOG_APP_CONFIG_POINTER_NOT_CORRECT_EC`, [484](#)
  - `APPGEN_LOG_APP_NAME_ERROR_EC`, [484](#)
  - `APPGEN_LOG_COPY_ENTITY_NAME_EC`, [483](#)
  - `APPGEN_LOG_CREATE_ENTITY_NAME_EC`, [483](#)
  - `APPGEN_LOG_DDS_ENTITY_EC`, [484](#)
  - `APPGEN_LOG_FACTORY_TOPIC_NOT_FOUND_EC`, [483](#)
  - `APPGEN_LOG_LIB_NOT_FOUND_EC`, [484](#)
  - `APPGEN_MULTIPLICITY_EC`, [483](#)
- `APPGEN_APP_PARTICIPANT_NOT_FOUND_EC`
  - `APPGEN`, [484](#)
- `APPGEN_Factory_get_interface`
  - Application Generation API, [225](#)
- `APPGEN_Factory_register`
  - Application Generation API, [225](#)
- `APPGEN_FACTORY_REGISTER_EC`
  - `APPGEN`, [483](#)
- `APPGEN_Factory_unregister`
  - Application Generation API, [225](#)
- `APPGEN_FACTORY_UNREGISTER_EC`
  - `APPGEN`, [483](#)
- `APPGEN_LOG_ALLOCATE_EC`
  - `APPGEN`, [483](#)
- `APPGEN_LOG_API_ERROR_EC`
  - `APPGEN`, [484](#)
- `APPGEN_LOG_APP_CONFIG_NOT_CORRECT_EC`
  - `APPGEN`, [484](#)
- `APPGEN_LOG_APP_CONFIG_POINTER_NOT_CORRECT_EC`
  - `APPGEN`, [484](#)
- `APPGEN_LOG_APP_NAME_ERROR_EC`
  - `APPGEN`, [484](#)
- `APPGEN_LOG_BASE`
  - `osapi_log.h`, [932](#)
- `APPGEN_LOG_COPY_ENTITY_NAME_EC`
  - `APPGEN`, [483](#)
- `APPGEN_LOG_CREATE_ENTITY_NAME_EC`

- APPGEN, [483](#)
- APPGEN\_LOG\_DDS\_ENTITY\_EC
  - APPGEN, [484](#)
- APPGEN\_LOG\_FACTORY\_TOPIC\_NOT\_FOUND\_EC
  - APPGEN, [483](#)
- APPGEN\_LOG\_LIB\_NOT\_FOUND\_EC
  - APPGEN, [484](#)
- APPGEN\_MULTIPLICITY\_EC
  - APPGEN, [483](#)
- Application Generation API, [224](#)
  - APPGEN\_Factory\_get\_interface, [225](#)
  - APPGEN\_Factory\_register, [225](#)
  - APPGEN\_Factory\_unregister, [225](#)
- as\_datawriter
  - FooDataWriter, [794](#)
- as\_entity
  - DDSDataReader, [648](#)
  - DDSDomainParticipant, [688](#)
  - DDSTopic, [761](#)
- as\_topicdescription
  - DDSTopic, [760](#)
- as\_uint32
  - NETIO\_AddressValue, [827](#)
- assert\_liveliness
  - DDSDataWriter, [669](#)
  - DDSDomainParticipant, [701](#)
- assert\_property
  - DDSPropertyQosPolicyHelper, [735](#)
- attach\_condition
  - DDSWaitSet, [770](#)
- autoenable\_created\_entities
  - DDS\_EntityFactoryQosPolicy, [543](#)
- bind
  - ZCOPY\_NotifUserInterfacel, [866](#)
- bind\_address
  - NETIO\_DGRAM\_Interfacel, [830](#)
- bits
  - NETIO\_Netmask, [836](#)
- builtin\_checksum128\_class
  - RTPS Transport API, [158](#)
- builtin\_checksum32\_class
  - RTPS Transport API, [157](#)
- builtin\_checksum64\_class
  - RTPS Transport API, [157](#)
- builtin\_endpoint\_reader\_nack\_period
  - DPDE\_DiscoveryPluginProperty, [775](#)
- builtin\_multicast\_port\_offset
  - DDS\_RtpsWellKnownPorts\_t, [614](#)
- builtin\_unicast\_port\_offset
  - DDS\_RtpsWellKnownPorts\_t, [614](#)
- builtin\_writer\_heartbeat\_period
  - DPDE\_DiscoveryPluginProperty, [774](#)
- builtin\_writer\_heartbeats\_per\_max\_samples
  - DPDE\_DiscoveryPluginProperty, [775](#)
- builtin\_writer\_max\_heartbeat\_retries
  - DPDE\_DiscoveryPluginProperty, [774](#)
- bytes\_per\_token
  - DDS\_FlowControllerTokenBucketProperty\_t, [552](#)
- cache\_serialized\_samples
  - DPDE\_DiscoveryPluginProperty, [773](#)
- callback
  - PSK\_ErrListener, [847](#)
- CDR, [278](#)
  - CDR\_LOG\_INCR\_OFFSET\_EC, [279](#)
  - CDR\_LOG\_SET\_OFFSET\_EC, [278](#)
  - CDR\_LOG\_STREAM\_ALLOC\_EC, [278](#)
  - CDR\_LOG\_UNSUPPORTED\_OPTIONS\_EC, [279](#)
- CDR Stream API, [142](#)
  - CDR\_Stream\_check\_size, [143](#)
  - CDR\_Stream\_get\_current\_position\_offset, [142](#)
  - CDR\_Stream\_get\_current\_position\_ptr, [144](#)
  - CDR\_Stream\_increment\_current\_position, [143](#)
  - CDR\_Stream\_is\_byte\_swapped, [144](#)
  - CDR\_Stream\_set\_current\_position\_offset, [143](#)
- cdr\_log.h, [869](#)
- CDR\_LOG\_BASE
  - osapi\_log.h, [931](#)
- CDR\_LOG\_INCR\_OFFSET\_EC
  - CDR, [279](#)
- CDR\_LOG\_SET\_OFFSET\_EC
  - CDR, [278](#)
- CDR\_LOG\_STREAM\_ALLOC\_EC
  - CDR, [278](#)
- CDR\_LOG\_UNSUPPORTED\_OPTIONS\_EC
  - CDR, [279](#)
- CDR\_Stream\_check\_size
  - CDR Stream API, [143](#)
- CDR\_Stream\_get\_current\_position\_offset
  - CDR Stream API, [142](#)
- CDR\_Stream\_get\_current\_position\_ptr
  - CDR Stream API, [144](#)
- CDR\_Stream\_increment\_current\_position
  - CDR Stream API, [143](#)
- CDR\_Stream\_is\_byte\_swapped
  - CDR Stream API, [144](#)
- CDR\_Stream\_set\_current\_position\_offset
  - CDR Stream API, [143](#)
- check\_crc
  - DDS\_WireProtocolQosPolicy, [642](#)
- checksum
  - DDS\_ParticipantBuiltinTopicData, [576](#)
  - RTPS Transport API, [158](#)
- checksum128
  - RTPS Transport API, [157](#)
- checksum32
  - RTPS Transport API, [157](#)

- checksum64
  - RTPS Transport API, [157](#)
- checksum\_calculate
  - RTPS Transport API, [157](#)
- checksum\_tx\_mode
  - RTPS Transport API, [158](#)
- class\_id
  - RTPS Transport API, [157](#)
- coherent\_access
  - DDS\_PresentationQosPolicy, [582](#)
- Compliance Configuration, [139](#)
- compute\_crc
  - DDS\_WireProtocolQosPolicy, [641](#)
- computed\_crc\_kind
  - DDS\_ChecksumProperty, [497](#)
  - DDS\_ChecksumProperty\_t, [498](#)
  - DDS\_WireProtocolQosPolicy, [642](#)
- Conditions and WaitSets, [132](#)
- Content Filter Plugin, [29](#)
  - DDS\_SQL\_CONTENT\_FILTER\_CLASS, [30](#)
- CONTENT\_FILTER, [104](#)
- content\_filter
  - DDS\_DataReaderQos, [508](#)
  - DDS\_SubscriptionBuiltinTopicData, [629](#)
- content\_filtered\_topic\_name
  - DDS\_ContentFilterProperty, [499](#)
- context
  - RTPS Transport API, [157](#)
- copy\_data
  - FooTypeSupport, [820](#)
- copy\_no\_alloc
  - FooSeq, [809](#)
- create\_data
  - FooDataWriter, [804](#)
  - FooTypeSupport, [819](#)
- create\_datareader
  - DDSSubscriber, [750](#)
- create\_datawriter
  - DDSPublisher, [739](#)
- create\_flowcontroller
  - DDSDomainParticipant, [708](#)
- create\_instance
  - NETIO\_DGRAM\_Interfaces, [828](#)
  - ZCOPY\_NotifUserInterface, [863](#)
- create\_participant
  - DDSDomainParticipantFactory, [712](#)
- create\_participant\_from\_config
  - DDSDomainParticipantFactory, [714](#)
- create\_publisher
  - DDSDomainParticipant, [688](#)
- create\_subscriber
  - DDSDomainParticipant, [690](#)
- create\_topic
  - DDSDomainParticipant, [692](#)
- current\_count
  - DDS\_PublicationMatchedStatus, [594](#)
  - DDS\_SubscriptionMatchedStatus, [630](#)
- current\_count\_change
  - DDS\_PublicationMatchedStatus, [595](#)
  - DDS\_SubscriptionMatchedStatus, [630](#)
- Data Sample, [73](#)
- DATA\_READER\_PROTOCOL, [125](#)
- DATA\_REPRESENTATION, [111](#)
  - DDS\_AUTO\_DATA\_REPRESENTATION, [113](#)
  - DDS\_DataRepresentationId\_t, [114](#)
  - DDS\_XCDR2\_DATA\_REPRESENTATION, [113](#)
  - DDS\_XCDR\_DATA\_REPRESENTATION, [112](#)
  - DDS\_XML\_DATA\_REPRESENTATION, [112](#)
  - T, [113](#), [114](#)
- DATA\_WRITER\_PROTOCOL, [125](#)
- DATA\_WRITER\_TRANSFER\_MODE, [131](#)
  - transfer\_mode, [132](#)
- DataReader, [70](#)
  - DDS\_NOT\_REJECTED, [71](#)
  - DDS\_REJECTED\_BY\_INSTANCES\_LIMIT, [71](#)
  - DDS\_REJECTED\_BY\_REMOTE\_WRITERS\_LIMIT, [71](#)
  - DDS\_REJECTED\_BY\_SAMPLES\_LIMIT, [71](#)
  - DDS\_REJECTED\_BY\_SAMPLES\_PER\_INSTANCE\_LIMIT, [71](#)
  - DDS\_REJECTED\_BY\_SAMPLES\_PER\_REMOTE\_WRITER\_LIMIT, [71](#)
  - DDS\_SAMPLE\_LOST\_BY\_DATAWRITER, [72](#)
  - DDS\_SAMPLE\_LOST\_BY\_HISTORY\_DEPTH\_LIMIT, [72](#)
  - DDS\_SAMPLE\_LOST\_BY\_MANAGEDPOOL\_OVERWRITE, [72](#)
  - DDS\_SAMPLE\_LOST\_BY\_MAX\_SAMPLES\_LIMIT, [72](#)
  - DDS\_SAMPLE\_LOST\_BY\_META\_SAMPLE\_LIMIT, [72](#)
  - DDS\_SAMPLE\_LOST\_BY\_NOT\_READ\_ON\_CACHE\_DELETION, [72](#)
  - DDS\_SAMPLE\_LOST\_NOT\_LOST, [72](#)
  - DDS\_SampleLostStatusKind, [71](#)
  - DDS\_SampleRejectedStatusKind, [71](#)
- DataReader Resource Limits, [119](#)
  - DDS\_DataReaderResourceLimitsInstanceReplacementKind, [119](#)
  - DDS\_NO\_INSTANCE\_REPLACEMENT\_QOS, [120](#)
  - DDS\_REPLACE\_OLDEST\_INSTANCE\_REPLACEMENT\_QOS, [120](#)
- DataReaderQos, [234](#)
- DataWriter, [65](#)
  - DDS\_DataWriterListener\_ReliableReaderActivityChangedCallback, [66](#)



- DDS\_DataWriterListener\_ReliableSampleUnacknowledgedCallback, [66](#)
- DDS\_DataWriterListener\_SampleRemovedCallback, [67](#)
- DDS\_ReliableReaderActivityChangedStatus\_INITIALIZED, [66](#)
- DataWriter Resource Limits, [120](#)
- DataWriterQos, [239](#)
- DB, [270](#)
  - DB\_LOG\_ALLOC\_CURSOR\_POOL\_EC, [274](#)
  - DB\_LOG\_ALLOC\_DATABASE\_EC, [275](#)
  - DB\_LOG\_ALLOC\_INDEX\_POOL\_EC, [274](#)
  - DB\_LOG\_ALLOC\_RECORD\_POOL\_EC, [275](#)
  - DB\_LOG\_ALLOC\_TABLE\_POOL\_EC, [273](#)
  - DB\_LOG\_CURSOR\_INVALIDATED\_EC, [275](#)
  - DB\_LOG\_CURSORS\_INUSE\_EC, [274](#)
  - DB\_LOG\_ILLEGAL\_LOCK\_MODE\_EC, [272](#)
  - DB\_LOG\_ILLEGAL\_RECORD\_COUNT\_EC, [273](#)
  - DB\_LOG\_ILLEGAL\_TABLE\_SIZE\_EC, [272](#)
  - DB\_LOG\_INDEX\_INUSE\_EC, [274](#)
  - DB\_LOG\_LOCK\_FAILURE\_EC, [275](#)
  - DB\_LOG\_MUTEX\_ALLOC\_EC, [272](#)
  - DB\_LOG\_NAME\_TOO\_LONG\_EC, [272](#)
  - DB\_LOG\_OUT\_OF\_CTORDTOR\_EC, [276](#)
  - DB\_LOG\_OUT\_OF\_CURSORS\_EC, [274](#)
  - DB\_LOG\_OUT\_OF\_INDICES\_EC, [274](#)
  - DB\_LOG\_OUT\_OF\_RECORDS\_EC, [273](#)
  - DB\_LOG\_OUT\_OF\_TABLES\_EC, [273](#)
  - DB\_LOG\_RECORD\_ALREADY\_EXISTS\_EC, [275](#)
  - DB\_LOG\_RECORD\_DOES\_NOT\_EXIST\_EC, [275](#)
  - DB\_LOG\_RECORDS\_INUSE\_EC, [274](#)
  - DB\_LOG\_SORTED\_ALLOC\_EC, [272](#)
  - DB\_LOG\_TABLE\_EXISTS\_EC, [273](#)
  - DB\_LOG\_TABLE\_NAME\_TOO\_LONG\_EC, [273](#)
  - DB\_LOG\_TABLE\_NOT\_INUSE\_EC, [275](#)
  - DB\_LOG\_TABLES\_INUSE\_EC, [273](#)
  - DB\_LOG\_UNLOCK\_FAILURE\_EC, [276](#)
- db\_log.h, [869](#)
- DB\_LOG\_ALLOC\_CURSOR\_POOL\_EC, [274](#)
- DB\_LOG\_ALLOC\_DATABASE\_EC, [275](#)
- DB\_LOG\_ALLOC\_INDEX\_POOL\_EC, [274](#)
- DB\_LOG\_ALLOC\_RECORD\_POOL\_EC, [275](#)
- DB\_LOG\_ALLOC\_TABLE\_POOL\_EC, [273](#)
- DB\_LOG\_BASE
  - osapi\_log.h, [930](#)
- DB\_LOG\_CURSOR\_INVALIDATED\_EC, [275](#)
- DB\_LOG\_CURSORS\_INUSE\_EC, [274](#)
- DB\_LOG\_ILLEGAL\_LOCK\_MODE\_EC, [272](#)
- DB\_LOG\_ILLEGAL\_RECORD\_COUNT\_EC, [273](#)
- DB\_LOG\_ILLEGAL\_TABLE\_SIZE\_EC, [272](#)
- DB\_LOG\_INDEX\_INUSE\_EC, [274](#)
- DB\_LOG\_LOCK\_FAILURE\_EC, [275](#)
- DB\_LOG\_MUTEX\_ALLOC\_EC, [272](#)
- DB\_LOG\_NAME\_TOO\_LONG\_EC, [272](#)
- DB\_LOG\_OUT\_OF\_CTORDTOR\_EC, [276](#)
- DB\_LOG\_OUT\_OF\_CURSORS\_EC, [274](#)
- DB\_LOG\_OUT\_OF\_INDICES\_EC, [274](#)
- DB\_LOG\_OUT\_OF\_RECORDS\_EC, [273](#)
- DB\_LOG\_OUT\_OF\_TABLES\_EC, [273](#)
- DB\_LOG\_RECORD\_ALREADY\_EXISTS\_EC, [275](#)
- DB\_LOG\_RECORD\_DOES\_NOT\_EXIST\_EC, [275](#)
- DB\_LOG\_RECORDS\_INUSE\_EC, [274](#)
- DB\_LOG\_SORTED\_ALLOC\_EC, [272](#)
- DB\_LOG\_TABLE\_EXISTS\_EC, [273](#)
- DB\_LOG\_TABLE\_NAME\_TOO\_LONG\_EC, [273](#)
- DB\_LOG\_TABLE\_NOT\_INUSE\_EC, [275](#)
- DB\_LOG\_TABLES\_INUSE\_EC, [273](#)
- DB\_LOG\_UNLOCK\_FAILURE\_EC, [276](#)
- DDS API, [26](#)
- DDS-Specific Primitive Types, [46](#)
  - DDS\_Boolean, [51](#)
  - DDS\_BOOLEAN\_FALSE, [48](#)
  - DDS\_BOOLEAN\_TRUE, [48](#)
  - DDS\_Char, [48](#)
  - DDS\_Double, [51](#)
  - DDS\_Enum, [51](#)
  - DDS\_Float, [50](#)
  - DDS\_Long, [49](#)
  - DDS\_LongDouble, [51](#)
  - DDS\_LongLong, [50](#)

- DDS\_Octet, [48](#)
- DDS\_Short, [49](#)
- DDS\_String, [52](#)
- DDS\_UnsignedLong, [49](#)
- DDS\_UnsignedLongLong, [50](#)
- DDS\_UnsignedShort, [49](#)
- DDS\_Wchar, [48](#)
- DDS\_Wstring, [52](#)
- DDS\_ALIVE\_INSTANCE\_STATE
  - Instance States, [78](#)
- DDS\_ANY\_INSTANCE\_STATE
  - Instance States, [78](#)
- DDS\_ANY\_SAMPLE\_STATE
  - Sample States, [74](#)
- DDS\_ANY\_VIEW\_STATE
  - View States, [76](#)
- DDS\_ASYNCHRONOUS\_PUBLISH\_MODE\_QOS
  - PUBLISH\_MODE, [129](#)
- DDS\_AUTO\_DATA\_REPRESENTATION
  - DATA\_REPRESENTATION, [113](#)
- DDS\_AUTOMATIC\_LIVELINESS\_QOS
  - LIVELINESS, [108](#)
- DDS\_AUTOMATIC\_PUBLISH\_MODE\_QOS
  - PUBLISH\_MODE, [129](#)
- DDS\_BEST\_EFFORT\_RELIABILITY\_QOS
  - RELIABILITY, [109](#)
- DDS\_Boolean
  - DDS-Specific Primitive Types, [51](#)
- DDS\_BOOLEAN\_FALSE
  - DDS-Specific Primitive Types, [48](#)
- DDS\_BOOLEAN\_TRUE
  - DDS-Specific Primitive Types, [48](#)
- dds\_builtin\_endpoints
  - DDS\_ParticipantBuiltinTopicData, [575](#)
- DDS\_BuiltinTopicKey\_t, [495](#)
  - Discovery, [39](#)
  - value, [496](#)
- DDS\_BY\_RECEPTION\_TIMESTAMP\_DESTINATIONORDER\_QOS
  - DESTINATION\_ORDER, [118](#)
- DDS\_BY\_SOURCE\_TIMESTAMP\_DESTINATIONORDER\_QOS
  - DESTINATION\_ORDER, [118](#)
- DDS\_C, [336](#)
  - DDS\_LOG\_IPC\_ASSERT\_ROUTES\_EC, [421](#)
  - DDS\_LOG\_TRUST\_ASSERT\_PARTICIPANT\_GENERIC\_MESSAGE\_FILTERING\_NOT\_ENABLED\_EC, [432](#)
  - DDSC\_LOG\_ADDRESSRESOLVER\_OBJECT, [377](#)
  - DDSC\_LOG\_APPGEN\_COMPONENT, [387](#)
  - DDSC\_LOG\_APPLY\_READER\_FILTER\_EC, [446](#)
  - DDSC\_LOG\_AUTHPOOL\_OBJECT, [380](#)
  - DDSC\_LOG\_BIND\_RECORD, [372](#)
  - DDSC\_LOG\_BINDRESOLVER\_OBJECT, [376](#)
  - DDSC\_LOG\_BUFFER\_MAX\_EXCEEDED\_EC, [444](#)
  - DDSC\_LOG\_BUFFER\_OBJECT, [381](#)
  - DDSC\_LOG\_CDR\_BUFFER\_SET\_EC, [399](#)
  - DDSC\_LOG\_CDR\_DESERIALIZE\_DATA\_EC, [401](#)
  - DDSC\_LOG\_CDR\_DESERIALIZE\_HEADER\_EC, [401](#)
  - DDSC\_LOG\_CDR\_DESERIALIZE\_KEY\_EC, [402](#)
  - DDSC\_LOG\_CDR\_DESERIALIZE\_KEYHASH\_EC, [401](#)
  - DDSC\_LOG\_CDR\_DESERIALIZE\_PID\_EC, [400](#)
  - DDSC\_LOG\_CDR\_DESERIALIZE\_PID\_LENGTH\_EC, [400](#)
  - DDSC\_LOG\_CDR\_FINALIZE\_SAMPLE\_EC, [401](#)
  - DDSC\_LOG\_CDR\_INCREMENT\_POS\_EC, [400](#)
  - DDSC\_LOG\_CDR\_INITIALIZE\_SAMPLE\_EC, [401](#)
  - DDSC\_LOG\_CDR\_POOL\_ALLOC\_EC, [399](#)
  - DDSC\_LOG\_CDR\_POOL\_DELETE\_EC, [399](#)
  - DDSC\_LOG\_CDR\_SERIALIZE\_DATA\_EC, [400](#)
  - DDSC\_LOG\_CDR\_SERIALIZE\_KEYHASH\_EC, [400](#)
  - DDSC\_LOG\_CDR\_SERIALIZE\_PID\_EC, [399](#)
  - DDSC\_LOG\_CDR\_SERIALIZE\_PID\_LENGTH\_EC, [400](#)
  - DDSC\_LOG\_CDR\_SERIALIZE\_STATUS\_INFO\_EC, [401](#)
  - DDSC\_LOG\_CDR\_SET\_OFFSET\_EC, [416](#)
  - DDSC\_LOG\_CDR\_SET\_POS\_EC, [401](#)
  - DDSC\_LOG\_CHANGE\_CONTENT\_FILTER\_EC, [447](#)
  - DDSC\_LOG\_CHECKSUM\_INCOMPATIBLE\_EC, [445](#)
  - DDSC\_LOG\_CHECKSUM\_INCONSISTENT\_ALLOW\_EC, [445](#)
  - DDSC\_LOG\_CHECKSUM\_INCONSISTENT\_CLASS\_EC, [445](#)
  - DDSC\_LOG\_CHECKSUM\_INCONSISTENT\_COMPUTE\_EC, [445](#)
  - DDSC\_LOG\_CHECKSUM\_REQUIRED\_EC, [444](#)
  - DDSC\_LOG\_COMPILE\_CONTENT\_FILTER\_EC, [446](#)
  - DDSC\_LOG\_COMPONENT\_CREATE\_EC, [387](#)
  - DDSC\_LOG\_COMPONENT\_DELETE\_EC, [387](#)
  - DDSC\_LOG\_COMPONENT\_LOOKUP\_EC, [387](#)
  - DDSC\_LOG\_CONDITION\_OBJECT, [378](#)
  - DDSC\_LOG\_CONDREF\_OBJECT, [378](#)
  - DDSC\_LOG\_CONTENT\_FILTER\_QOS\_POLICY, [393](#)
  - DDSC\_LOG\_CREATE\_FILTER\_PLUGIN\_EC, [447](#)
  - DDSC\_LOG\_CREATE\_INDEX\_EC, [370](#)
  - DDSC\_LOG\_CREATE\_WRITER\_FILTER\_EC, [446](#)
  - DDSC\_LOG\_DATA\_HOLDER\_SEQ\_OBJECT, [380](#)
  - DDSC\_LOG\_DATA\_REPRESENTATION\_QOS\_POLICY, [393](#)
  - DDSC\_LOG\_DATABASE\_CREATE\_EC, [369](#)
  - DDSC\_LOG\_DATABASE\_DELETE\_EC, [369](#)
  - DDSC\_LOG\_COOKIE\_PAYLOAD\_SEQUENCE, [384](#)
  - DDSC\_LOG\_CREATE\_FILTER\_PLUGIN\_EC, [447](#)
  - DDSC\_LOG\_CREATE\_INDEX\_EC, [370](#)
  - DDSC\_LOG\_CREATE\_WRITER\_FILTER\_EC, [446](#)
  - DDSC\_LOG\_DATA\_HOLDER\_SEQ\_OBJECT, [380](#)
  - DDSC\_LOG\_DATA\_REPRESENTATION\_QOS\_POLICY, [393](#)
  - DDSC\_LOG\_DATABASE\_CREATE\_EC, [369](#)
  - DDSC\_LOG\_DATABASE\_DELETE\_EC, [369](#)
  - DDSC\_LOG\_COOKIE\_PAYLOAD\_SEQUENCE, [384](#)

DDSC\_LOG\_DATAHOLDER\_SEQUENCE, [384](#)  
 DDSC\_LOG\_DATAREADER\_ADD\_PEER\_FAILED\_EC, [418](#)  
 DDSC\_LOG\_DATAREADER\_CDR, [399](#)  
 DDSC\_LOG\_DATAREADER\_ENTITY, [397](#)  
 DDSC\_LOG\_DATAREADER\_LISTENER, [395](#)  
 DDSC\_LOG\_DATAREADER\_NETIO\_KIND, [403](#)  
 DDSC\_LOG\_DATAREADER\_QOS, [388](#)  
 DDSC\_LOG\_DATAREADER\_RECORD, [371](#)  
 DDSC\_LOG\_DATAREADER\_RESOURCE\_QOS\_POLICY, [392](#)  
 DDSC\_LOG\_DATAREADERIO\_COMPONENT, [386](#)  
 DDSC\_LOG\_DATAREADERIO\_OBJECT, [377](#)  
 DDSC\_LOG\_DATAREADERQOS\_OBJECT, [375](#)  
 DDSC\_LOG\_DATAWRITER\_ADD\_PEER\_FAILED\_EC, [418](#)  
 DDSC\_LOG\_DATAWRITER\_CDR, [399](#)  
 DDSC\_LOG\_DATAWRITER\_ENTITY, [397](#)  
 DDSC\_LOG\_DATAWRITER\_INLINE, [399](#)  
 DDSC\_LOG\_DATAWRITER\_LISTENER, [395](#)  
 DDSC\_LOG\_DATAWRITER\_NETIO\_KIND, [403](#)  
 DDSC\_LOG\_DATAWRITER\_QOS, [389](#)  
 DDSC\_LOG\_DATAWRITER\_RECORD, [371](#)  
 DDSC\_LOG\_DATAWRITER\_RESOURCE\_QOS\_POLICY, [392](#)  
 DDSC\_LOG\_DATAWRITERIO\_COMPONENT, [386](#)  
 DDSC\_LOG\_DATAWRITERIO\_OBJECT, [377](#)  
 DDSC\_LOG\_DATAWRITERQOS\_OBJECT, [375](#)  
 DDSC\_LOG\_DB\_CURSOR\_INVALIDATED\_EC, [370](#)  
 DDSC\_LOG\_DEADLINE\_QOS\_POLICY, [390](#)  
 DDSC\_LOG\_DEFAULTDATAREADER\_QOS, [389](#)  
 DDSC\_LOG\_DEFAULTDATAWRITER\_QOS, [390](#)  
 DDSC\_LOG\_DEFAULTPARTICIPANT\_QOS, [390](#)  
 DDSC\_LOG\_DEFAULTPUBLISHER\_QOS, [390](#)  
 DDSC\_LOG\_DEFAULTSUBSCRIBER\_QOS, [390](#)  
 DDSC\_LOG\_DEFAULTTOPIC\_QOS, [389](#)  
 DDSC\_LOG\_DELETE\_INDEX\_EC, [370](#)  
 DDSC\_LOG\_DESERIALIZE\_BAD\_PID\_LENGTH\_EC, [400](#)  
 DDSC\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC, [416](#)  
 DDSC\_LOG\_DESERIALIZE\_CONTENT\_FILTER\_INFO\_EC, [447](#)  
 DDSC\_LOG\_DESERIALIZE\_LOCATOR\_EC, [417](#)  
 DDSC\_LOG\_DESERIALIZE\_PARTITION\_SEQ\_EC, [418](#)  
 DDSC\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC, [417](#)  
 DDSC\_LOG\_DESERIALIZE\_REMOTE\_CONTENT\_FILTER\_INFO\_EC, [447](#)  
 DDSC\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC, [415](#)  
 DDSC\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC, [417](#)  
 DDSC\_LOG\_DESTINATION\_ORDER\_POLICY, [392](#)  
 DDSC\_LOG\_DESTINATION\_SEQUENCE, [383](#)  
 DDSC\_LOG\_DISC\_ADD\_PEER\_EC, [415](#)  
 DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAREADER\_DELETED\_EC, [415](#)  
 DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAREADER\_ENABLED\_EC, [414](#)  
 DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAWRITER\_DELETED\_EC, [415](#)  
 DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAWRITER\_ENABLED\_EC, [415](#)  
 DDSC\_LOG\_DISC\_AFTER\_LOCAL\_PARTICIPANT\_CREATED\_EC, [414](#)  
 DDSC\_LOG\_DISC\_BEFORE\_LOCAL\_PARTICIPANT\_CREATED\_EC, [414](#)  
 DDSC\_LOG\_DISC\_BEFORE\_REMOTE\_PARTICIPANT\_DELETED\_EC, [415](#)  
 DDSC\_LOG\_DISC\_LOCAL\_PARTICIPANT\_ENABLED\_EC, [414](#)  
 DDSC\_LOG\_DISC\_REMOTE\_PARTICIPANT\_REMOVE\_EC, [418](#)  
 DDSC\_LOG\_DISCOVERY\_COMPONENT, [386](#)  
 DDSC\_LOG\_DR\_COMMIT\_ENTRY\_EC, [408](#)  
 DDSC\_LOG\_DR\_COMMIT\_SAMPLE\_EC, [408](#)  
 DDSC\_LOG\_DR\_COPY\_DATA\_SAMPLE\_EC, [408](#)  
 DDSC\_LOG\_DR\_CREATE\_TYPED\_READER\_EC, [407](#)  
 DDSC\_LOG\_DR\_DESERIALIZE\_KEYHASH\_EC, [408](#)  
 DDSC\_LOG\_DR\_DISPOSE\_KEY\_EC, [409](#)  
 DDSC\_LOG\_DR\_FILTER\_ERROR\_EC, [408](#)  
 DDSC\_LOG\_DR\_GET\_ENTRY\_FAILED\_EC, [408](#)  
 DDSC\_LOG\_DR\_INSTANCE\_ASSERTION\_FAILED\_EC, [409](#)  
 DDSC\_LOG\_DR\_INSTANCE\_MAPPING\_EXHAUSTED\_EC, [409](#)  
 DDSC\_LOG\_DR\_INSTANCE\_TO\_KEYHASH\_EC, [409](#)  
 DDSC\_LOG\_DR\_ON\_SAMPLE\_REMOVED\_EC, [409](#)  
 DDSC\_LOG\_DR\_READ\_TAKE\_FAILURE\_EC, [409](#)  
 DDSC\_LOG\_DR\_UNREGISTER\_KEY\_EC, [408](#)  
 DDSC\_LOG\_DUPLICATE\_GUID\_PREFIX\_EC, [412](#)  
 DDSC\_LOG\_DUPLICATE\_PARTICIPANT\_NAME\_EC, [413](#)  
 DDSC\_LOG\_DUPLICATE\_REMOTE\_PARTICIPANT\_NAME\_EC, [413](#)  
 DDSC\_LOG\_DURABILITY\_QOS\_POLICY, [391](#)  
 DDSC\_LOG\_DW\_ACKNACK\_FAILED\_EC, [406](#)  
 DDSC\_LOG\_DW\_COMMIT\_EC, [406](#)  
 DDSC\_LOG\_DW\_CREATE\_TYPED\_WRITER\_EC, [406](#)  
 DDSC\_LOG\_DW\_FLOW\_CONTROLLER\_REQUIRED\_EC, [407](#)  
 DDSC\_LOG\_DW\_HISTORY\_REGISTER\_KEY\_EC,

- 406
- DDSC\_LOG\_DW\_ILLEGAL\_KEY\_KIND\_EC, 406
- DDSC\_LOG\_DW\_INCOMPATIBLE\_PADDING\_EC, 407
- DDSC\_LOG\_DW\_INSTANCE\_ASSERTION\_FAILED\_EC, 407
- DDSC\_LOG\_DW\_KEYHASH\_CREATE\_EC, 406
- DDSC\_LOG\_DW\_UNKNOWN\_FLOW\_CONTROLLER\_EC, 406
- DDSC\_LOG\_ENABLE\_WRITER\_FILTERING\_EC, 446
- DDSC\_LOG\_ENABLED\_DISCOVERY\_TRANSPORT\_SEQUENCE, 383
- DDSC\_LOG\_ENABLED\_TRANSPORT\_SEQUENCE, 383
- DDSC\_LOG\_ENABLED\_USER\_TRANSPORT\_SEQUENCE, 383
- DDSC\_LOG\_ENDPOINT\_NOT\_CHILD\_OF\_PARTICIPANT\_EC, 364
- DDSC\_LOG\_ENTITY\_DIFFERENT\_FACTORY\_EC, 398
- DDSC\_LOG\_ENTITY\_ENABLE\_EC, 398
- DDSC\_LOG\_ENTITY\_FINALIZE\_EC, 398
- DDSC\_LOG\_ENTITY\_INITIALIZE\_EC, 398
- DDSC\_LOG\_ENTITY\_INVALID\_PROPERTY\_EC, 398
- DDSC\_LOG\_ENTITY\_NAME\_TOO\_LONG\_EC, 442
- DDSC\_LOG\_ENTITY\_NOT\_EMPTY\_EC, 398
- DDSC\_LOG\_ENTITY\_NOT\_ENABLED\_EC, 398
- DDSC\_LOG\_ENTITY\_OBJECT, 376
- DDSC\_LOG\_EVALUATE\_CONTENT\_FILTER\_EC, 447
- DDSC\_LOG\_EVALUATE\_WRITER\_FILTER\_EC, 446
- DDSC\_LOG\_FAILED\_UPDATE\_STATUS\_CONDITION\_EC, 366
- DDSC\_LOG\_FILTER\_PLUGIN\_FACTORY\_NOT\_FOUND\_EC, 446
- DDSC\_LOG\_FINALIZE\_FILTER\_PLUGIN\_EC, 447
- DDSC\_LOG\_FINALIZE\_TRUST\_FAILED\_EC, 434
- DDSC\_LOG\_FIND\_PUBLICATION\_PARENT\_EC, 365
- DDSC\_LOG\_FIND\_SUBSCRIPTION\_PARENT\_EC, 366
- DDSC\_LOG\_FLOW\_CONTROLLER\_CREATE\_EC, 443
- DDSC\_LOG\_FLOW\_CONTROLLER\_DELETE\_EC, 443
- DDSC\_LOG\_FLOWCONTROLLER\_INCONSISTENT\_PROPERTIES\_EC, 445
- DDSC\_LOG\_GET\_MTU\_EC, 368
- DDSC\_LOG\_GET\_MTU\_NO\_ROUTES\_EC, 368
- DDSC\_LOG\_GET\_NEXT\_OBJECT\_ID\_EC, 363
- DDSC\_LOG\_GET\_SERIALIZED\_KEY\_SIZE\_EC, 368
- DDSC\_LOG\_HEARTBEATS\_QOS\_POLICY, 392
- DDSC\_LOG\_HISTORY\_QOS\_POLICY, 390
- DDSC\_LOG\_HISTORY\_RESOURCE, 387
- DDSC\_LOG\_ILLEGAL\_OBJECTID\_EC, 366
- DDSC\_LOG\_INCOMPATIBLE\_PARTITION\_QOS\_EC, 444
- DDSC\_LOG\_INCOMPATIBLE\_PRESENTATION\_QOS\_EC, 444
- DDSC\_LOG\_INITIAL\_PEER\_SEQUENCE, 383
- DDSC\_LOG\_INITIALIZE\_DISCOVERY\_QUEUE\_EC, 448
- DDSC\_LOG\_INTERPRETER\_SUPPORT\_UNEXPECTED\_ERROR\_EC, 443
- DDSC\_LOG\_INTRA\_NETIO\_KIND, 402
- DDSC\_LOG\_INVALID\_BUFFER\_LENGTH\_EC, 444
- DDSC\_LOG\_INVALID\_DOMAINID\_EC, 367
- DDSC\_LOG\_INVALID\_DURATION\_EC, 363
- DDSC\_LOG\_INVALID\_ENDPOINT\_GUID\_EC, 364
- DDSC\_LOG\_INVALID\_PARTICIPANT\_GUID\_PREFIX\_EC, 363
- DDSC\_LOG\_INVALID\_PARTICIPANT\_NAME\_EC, 363
- DDSC\_LOG\_IO\_SNPRINTF\_FAILED\_EC, 364
- DDSC\_LOG\_IPC\_ALLOC\_FAILED\_EC, 419
- DDSC\_LOG\_IPC\_CHANNEL\_ADD\_ANON\_ROUTE\_EC, 420
- DDSC\_LOG\_IPC\_CHANNEL\_ADD\_PEER\_EC, 421
- DDSC\_LOG\_IPC\_CHANNEL\_CONFIG\_FINALIZE\_FAILED\_EC, 422
- DDSC\_LOG\_IPC\_CHANNEL\_CONFIG\_INITIALIZE\_FAILED\_EC, 422
- DDSC\_LOG\_IPC\_CHANNEL\_CONFIG\_SET\_FAILED\_EC, 422
- DDSC\_LOG\_IPC\_CHANNEL\_CREATE\_FAILED\_EC, 423
- DDSC\_LOG\_IPC\_CHANNEL\_DELETE\_ANON\_ROUTE\_EC, 421
- DDSC\_LOG\_IPC\_CHANNEL\_ENABLE\_FAILED\_EC, 423
- DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_AUTH\_HANDSHAKE\_EC, 422
- DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_AUTH\_REQUEST\_EC, 422
- DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_ENDPOINT\_INTERACTION\_EC, 422
- DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_PARTICIPANT\_INTERACTION\_EC, 422
- DDSC\_LOG\_IPC\_CHANNEL\_PROCESS\_SAMPLE\_EC, 421
- DDSC\_LOG\_IPC\_CHANNEL\_READER\_CREATE\_EC, 420
- DDSC\_LOG\_IPC\_CHANNEL\_REMOVE\_PEER\_EC, 421

- DDSC\_LOG\_IPC\_CHANNEL\_RETURN\_LOAN\_EC, 421
- DDSC\_LOG\_IPC\_CHANNEL\_SET\_LISTENERS\_FAILED\_EC, 423
- DDSC\_LOG\_IPC\_CHANNEL\_TOPIC\_CREATE\_EC, 420
- DDSC\_LOG\_IPC\_CHANNEL\_TYPE\_PLUGIN\_EC, 420
- DDSC\_LOG\_IPC\_CHANNEL\_TYPE\_REGISTER\_EC, 419
- DDSC\_LOG\_IPC\_CHANNEL\_UNKNOWN\_EC, 420
- DDSC\_LOG\_IPC\_CHANNEL\_WRITE\_SAMPLE\_EC, 421
- DDSC\_LOG\_IPC\_CHANNEL\_WRITER\_CREATE\_EC, 420
- DDSC\_LOG\_IPC\_FINALIZE\_FAILED\_EC, 419
- DDSC\_LOG\_IPC\_INIT\_FAILED\_EC, 419
- DDSC\_LOG\_IPC\_UPDATE\_QOS\_FAILED\_EC, 420
- DDSC\_LOG\_LATENCY\_BUDGET\_QOS\_POLICY, 393
- DDSC\_LOG\_LISTENER\_GET\_EC, 396
- DDSC\_LOG\_LISTENER\_INCONSISTENT\_EC, 396
- DDSC\_LOG\_LISTENER\_SET\_EC, 396
- DDSC\_LOG\_LISTENER\_SET\_ILLEGAL\_NULL\_EC, 396
- DDSC\_LOG\_LIVELINESS\_QOS\_POLICY, 390
- DDSC\_LOG\_LOCATORS\_FULL\_EC, 417
- DDSC\_LOG\_LOCK\_EC, 445
- DDSC\_LOG\_LOG\_OBJECT, 377
- DDSC\_LOG\_LOOKUP\_TYPE\_PLUGIN\_EC, 411
- DDSC\_LOG\_MATCHED\_DW\_RECORD, 372
- DDSC\_LOG\_MAX\_PARTICIPANT\_ID\_REACHED\_EC, 366
- DDSC\_LOG\_MD5STREAM\_OBJECT, 378
- DDSC\_LOG\_MEMORY\_MANAGER\_DELETE\_EC, 444
- DDSC\_LOG\_MEMORY\_MANAGER\_NOT\_OWNER\_EC, 444
- DDSC\_LOG\_MEMORY\_MANAGER\_OUT\_OF\_RESOURCES\_EC, 443
- DDSC\_LOG\_MEMPOOL\_OBJECT, 379
- DDSC\_LOG\_MESSAGE\_DATA\_UNKNOWN\_LIVELINESS\_KIND\_EC, 442
- DDSC\_LOG\_MESSAGE\_DATA\_WRONG\_LIVELINESS\_KIND\_EC, 442
- DDSC\_LOG\_METAMULTICAST\_SEQUENCE, 384
- DDSC\_LOG\_METAUNICAST\_SEQUENCE, 384
- DDSC\_LOG\_MIN\_DISCOVERY\_MTU\_EC, 368
- DDSC\_LOG\_MIN\_MTU\_SIZE\_EC, 368
- DDSC\_LOG\_MUTEX\_OBJECT, 377
- DDSC\_LOG\_NAME\_LOOKUP\_EC, 441
- DDSC\_LOG\_NETIO\_ADD\_ANON\_TOPIC\_ROUTE\_EC, 402
- DDSC\_LOG\_NETIO\_ADD\_ROUTE\_EC, 404
- DDSC\_LOG\_NETIO\_ADD\_TOPIC\_ROUTE\_EC, 402
- DDSC\_LOG\_NETIO\_BIND\_EC, 403
- DDSC\_LOG\_NETIO\_BIND\_EXTERNAL\_EC, 403
- DDSC\_LOG\_NETIO\_COMPONENT, 387
- DDSC\_LOG\_NETIO\_DELETE\_ROUTE\_EC, 404
- DDSC\_LOG\_NETIO\_DELETE\_TOPIC\_ROUTE\_EC, 402
- DDSC\_LOG\_NETIO\_DISCOVERY\_ENABLED\_TRANSPORT\_EC, 405
- DDSC\_LOG\_NETIO\_FORCED\_REMOVE\_EC, 405
- DDSC\_LOG\_NETIO\_FORWARD\_TOPIC\_EC, 402
- DDSC\_LOG\_NETIO\_GET\_EXTERNAL\_INTF\_EC, 404
- DDSC\_LOG\_NETIO\_GET\_ROUTE\_TABLE\_FAILED\_EC, 404
- DDSC\_LOG\_NETIO\_NETIO\_KIND, 402
- DDSC\_LOG\_NETIO\_NO\_ROUTE\_EC, 404
- DDSC\_LOG\_NETIO\_PEER\_LOOKUP\_EC, 405
- DDSC\_LOG\_NETIO\_ROUTE\_LOOKUP\_FAILED\_EC, 404
- DDSC\_LOG\_NETIO\_SEND\_FAILED\_EC, 404
- DDSC\_LOG\_NETIO\_SET\_STATE\_EC, 405
- DDSC\_LOG\_NETIO\_UNBIND\_EC, 403
- DDSC\_LOG\_NETIO\_UNBIND\_EXTERNAL\_EC, 403
- DDSC\_LOG\_NETMASK\_SEQUENCE, 382
- DDSC\_LOG\_OBJECT\_ALLOCATE\_EC, 381
- DDSC\_LOG\_OBJECT\_COPY\_EC, 381
- DDSC\_LOG\_OBJECT\_CREATE\_EC, 382
- DDSC\_LOG\_OBJECT\_DELETE\_EC, 381
- DDSC\_LOG\_OBJECT\_EMPTY\_EC, 382
- DDSC\_LOG\_OBJECT\_FINALIZE\_EC, 381
- DDSC\_LOG\_OBJECT\_GET\_PROPERTY\_EC, 382
- DDSC\_LOG\_OBJECT\_INITIALIZE\_EC, 381
- DDSC\_LOG\_OBJECT\_INUSECOUNT\_EC, 382
- DDSC\_LOG\_OBJECT\_REFCOUNT\_EC, 382
- DDSC\_LOG\_OBJECT\_SET\_PROPERTY\_EC, 382
- DDSC\_LOG\_ON\_TYPE\_REGISTERED\_FAILURE\_EC, 368
- DDSC\_LOG\_ON\_TYPE\_UNREGISTERED\_FAILURE\_EC, 368
- DDSC\_LOG\_OWNERSHIP\_QOS\_POLICY, 391
- DDSC\_LOG\_OWNERSHIP\_STRENGTH\_QOS\_POLICY, 391
- DDSC\_LOG\_PACKET\_INIT\_EC, 405
- DDSC\_LOG\_PACKET\_SET\_HEAD\_EC, 405
- DDSC\_LOG\_PACKET\_SET\_TAIL\_EC, 405
- DDSC\_LOG\_PARTICIPANT\_BINARY\_PROPERTIES, 379
- DDSC\_LOG\_PARTICIPANT\_BUILTIN\_PUBLISHER, 380
- DDSC\_LOG\_PARTICIPANT\_BUILTIN\_SUBSCRIBER, 380
- DDSC\_LOG\_PARTICIPANT\_DOES\_NOT\_EXIST\_EC, 380

- 365
- DDSC\_LOG\_PARTICIPANT\_ENTITY, 397
- DDSC\_LOG\_PARTICIPANT\_GENERIC\_MESSAGE\_OBJECT, 380
- DDSC\_LOG\_PARTICIPANT\_ID\_QOS\_POLICY, 392
- DDSC\_LOG\_PARTICIPANT\_LISTENER, 396
- DDSC\_LOG\_PARTICIPANT\_LOOKUP\_EC, 365
- DDSC\_LOG\_PARTICIPANT\_NAME\_EMPTY\_EC, 412
- DDSC\_LOG\_PARTICIPANT\_POOL\_OBJECT, 378
- DDSC\_LOG\_PARTICIPANT\_PROPERTIES, 379
- DDSC\_LOG\_PARTICIPANT\_QOS, 389
- DDSC\_LOG\_PARTICIPANT\_RECORD, 371
- DDSC\_LOG\_PARTICIPANT\_RESOURCES, 388
- DDSC\_LOG\_PARTICIPANTDATA\_OBJECT, 375
- DDSC\_LOG\_PARTICIPANTFACTORY\_LISTENER, 396
- DDSC\_LOG\_PARTICIPANTFACTORY\_QOS, 389
- DDSC\_LOG\_PARTICIPANTFACTORYQOS\_OBJECT, 376
- DDSC\_LOG\_PARTICIPANTQOS\_OBJECT, 376
- DDSC\_LOG\_PARTMESSAGE\_WRITE\_SAMPLE\_FAILED\_EC, 442
- DDSC\_LOG\_PROPERTY\_QOS\_POLICY, 393
- DDSC\_LOG\_PROPERTYQOSPOLICY\_OBJECT, 380
- DDSC\_LOG\_PROTOCOL\_QOS\_POLICY, 391
- DDSC\_LOG\_PUBLICATION\_ENTITY, 397
- DDSC\_LOG\_PUBLICATION\_RECORD, 371
- DDSC\_LOG\_PUBLICATIONDATA\_OBJECT, 375
- DDSC\_LOG\_PUBLISH\_MODE\_QOS\_POLICY, 393
- DDSC\_LOG\_PUBLISHER\_ENTITY, 397
- DDSC\_LOG\_PUBLISHER\_LISTENER, 395
- DDSC\_LOG\_PUBLISHER\_QOS, 389
- DDSC\_LOG\_PUBLISHER\_RECORD, 371
- DDSC\_LOG\_PUBLISHERQOS\_OBJECT, 376
- DDSC\_LOG\_QOS\_CHANGED\_EC, 395
- DDSC\_LOG\_QOS\_COPY\_EC, 394
- DDSC\_LOG\_QOS\_FINALIZE\_EC, 394
- DDSC\_LOG\_QOS\_GET\_EC, 395
- DDSC\_LOG\_QOS\_IMMUTABLE\_EC, 394
- DDSC\_LOG\_QOS\_INCONSISTENT\_EC, 394
- DDSC\_LOG\_QOS\_INCONSISTENT\_POLICIES\_EC, 393
- DDSC\_LOG\_QOS\_INCONSISTENT\_POLICY\_EC, 393
- DDSC\_LOG\_QOS\_INITIALIZE\_EC, 394
- DDSC\_LOG\_QOS\_SET\_EC, 394
- DDSC\_LOG\_QOS\_SET\_ON\_ENABLED\_EC, 394
- DDSC\_LOG\_QUEUE\_FINALIZE\_EC, 419
- DDSC\_LOG\_QUEUE\_PUBLICATION\_DISCOVERY\_EC, 419
- DDSC\_LOG\_QUEUE\_SUBSCRIPTION\_DISCOVERY\_EC, 419
- DDSC\_LOG\_READERHISTORY\_COMPONENT, 386
- DDSC\_LOG\_READTAKE\_SEQUENCE, 384
- DDSC\_LOG\_RECORD\_CREATE\_EC, 372
- DDSC\_LOG\_RECORD\_DELETE\_EC, 373
- DDSC\_LOG\_RECORD\_ERROR\_EC, 373
- DDSC\_LOG\_RECORD\_EXISTS\_EC, 373
- DDSC\_LOG\_RECORD\_FINALIZE\_EC, 374
- DDSC\_LOG\_RECORD\_INITIALIZE\_EC, 374
- DDSC\_LOG\_RECORD\_INSERT\_EC, 373
- DDSC\_LOG\_RECORD\_LOOKUP\_EC, 374
- DDSC\_LOG\_RECORD\_NOT\_EXISTS\_EC, 374
- DDSC\_LOG\_RECORD\_REMOVE\_EC, 374
- DDSC\_LOG\_RECORD\_RESET\_EC, 374
- DDSC\_LOG\_RECORD\_SELECT\_EC, 374
- DDSC\_LOG\_REFRESH\_LIVELINESS\_FAILED\_EC, 443
- DDSC\_LOG\_REFRESH\_REM\_PARTICIPANT\_EC, 365
- DDSC\_LOG\_REFRESH\_REM\_PARTICIPANT\_TIMEOUT\_EC, 365
- DDSC\_LOG\_RELEASE\_META\_MC\_EC, 367
- DDSC\_LOG\_RELEASE\_META\_UC\_EC, 367
- DDSC\_LOG\_RELEASE\_USER\_MC\_EC, 367
- DDSC\_LOG\_RELEASE\_USER\_UC\_EC, 367
- DDSC\_LOG\_RELIABILITY\_QOS\_POLICY, 391
- DDSC\_LOG\_REMOTE\_ENDPOINT\_TRUST\_STATE\_RECORD, 372
- DDSC\_LOG\_REMOTE\_PARTICIPANT\_KEY\_NOT\_EQUAL\_EC, 364
- DDSC\_LOG\_REMOTE\_PARTICIPANT\_NAME\_EMPTY\_EC, 413
- DDSC\_LOG\_REMOTE\_PARTICIPANT\_RESTART\_EC, 414
- DDSC\_LOG\_REMOTE\_PUBLICATION\_DATA\_CHANGED\_EC, 369
- DDSC\_LOG\_REMOTE\_SUBSCRIPTION\_DATA\_CHANGED\_EC, 369
- DDSC\_LOG\_REMOVE\_PUBLICATION\_EC, 365
- DDSC\_LOG\_REMOVE\_SUBSCRIPTION\_EC, 365
- DDSC\_LOG\_REPRESENTATION\_ID\_SEQUENCE, 385
- DDSC\_LOG\_RESERVE\_LOCATORS\_EC, 366
- DDSC\_LOG\_RESERVED\_SEQUENCE, 383
- DDSC\_LOG\_RESOLVE\_LOCATORS\_PRIORITY\_EC, 369
- DDSC\_LOG\_RESOURCE\_EXCEEDED\_EC, 388
- DDSC\_LOG\_RESOURCE\_LIMIT\_QOS\_POLICY, 391
- DDSC\_LOG\_ROUTE\_RECORD, 372
- DDSC\_LOG\_ROUTE\_SEQUENCE, 383
- DDSC\_LOG\_ROUTERESOLVER\_OBJECT, 377
- DDSC\_LOG\_RT\_OBJECT, 378
- DDSC\_LOG\_RTPS\_COMPONENT, 386



- DDSC\_LOG\_RTPS\_NETIO\_KIND, [403](#)
- DDSC\_LOG\_SAMPLE\_RESOURCES, [388](#)
- DDSC\_LOG\_SEND\_LIVELINESS\_FAILED\_EC, [442](#)
- DDSC\_LOG\_SEQUENCE\_COPY\_EC, [385](#)
- DDSC\_LOG\_SEQUENCE\_FINALIZE\_EC, [385](#)
- DDSC\_LOG\_SEQUENCE\_GETREF\_EC, [385](#)
- DDSC\_LOG\_SEQUENCE\_INITIALIZE\_EC, [385](#)
- DDSC\_LOG\_SEQUENCE\_INVALID\_EC, [386](#)
- DDSC\_LOG\_SEQUENCE\_SETLENGTH\_EC, [385](#)
- DDSC\_LOG\_SEQUENCE\_SETMAX\_EC, [385](#)
- DDSC\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC, [415](#)
- DDSC\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC, [416](#)
- DDSC\_LOG\_SERIALIZE\_ENTITY\_NAME\_EC, [416](#)
- DDSC\_LOG\_SERIALIZE\_GUID\_EC, [416](#)
- DDSC\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC, [418](#)
- DDSC\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC, [418](#)
- DDSC\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC, [417](#)
- DDSC\_LOG\_SERIALIZE\_TOPIC\_NAME\_EC, [416](#)
- DDSC\_LOG\_SERIALIZE\_TYPE\_NAME\_EC, [416](#)
- DDSC\_LOG\_SERIALIZE\_VENDOR\_ID\_EC, [417](#)
- DDSC\_LOG\_SET\_ENTITY\_NAME\_EC, [363](#)
- DDSC\_LOG\_STRING\_OBJECT, [380](#)
- DDSC\_LOG\_STRING\_RECORD, [372](#)
- DDSC\_LOG\_STRINGMANAGER\_OBJECT, [381](#)
- DDSC\_LOG\_SUBSCRIBER\_ENTITY, [397](#)
- DDSC\_LOG\_SUBSCRIBER\_LISTENER, [395](#)
- DDSC\_LOG\_SUBSCRIBER\_QOS, [389](#)
- DDSC\_LOG\_SUBSCRIBER\_RECORD, [371](#)
- DDSC\_LOG\_SUBSCRIBERQOS\_OBJECT, [376](#)
- DDSC\_LOG\_SUBSCRIPTION\_ENTITY, [397](#)
- DDSC\_LOG\_SUBSCRIPTION\_RECORD, [370](#)
- DDSC\_LOG\_SUBSCRIPTIONDATA\_OBJECT, [375](#)
- DDSC\_LOG\_SYS\_GET\_HOSTNAME\_EC, [363](#)
- DDSC\_LOG\_SYS\_GETTIME\_EC, [363](#)
- DDSC\_LOG\_SYSTEM\_OBJECT, [377](#)
- DDSC\_LOG\_TABLE\_CREATE\_EC, [369](#)
- DDSC\_LOG\_TABLE\_DELETE\_EC, [370](#)
- DDSC\_LOG\_TABLE\_INUSE\_EC, [370](#)
- DDSC\_LOG\_TABLE\_SELECT\_EC, [370](#)
- DDSC\_LOG\_TIMER\_CREATE\_TIMEOUT\_EC, [366](#)
- DDSC\_LOG\_TIMER\_OBJECT, [378](#)
- DDSC\_LOG\_TIMEROUT\_OBJECT, [379](#)
- DDSC\_LOG\_TOPIC\_ENTITY, [396](#)
- DDSC\_LOG\_TOPIC\_FIND\_EC, [364](#)
- DDSC\_LOG\_TOPIC\_LISTENER, [395](#)
- DDSC\_LOG\_TOPIC\_NAME\_CMP\_EC, [410](#)
- DDSC\_LOG\_TOPIC\_NARROW\_EC, [366](#)
- DDSC\_LOG\_TOPIC\_OBJECT, [379](#)
- DDSC\_LOG\_TOPIC\_QOS, [388](#)
- DDSC\_LOG\_TOPIC\_RECORD, [371](#)
- DDSC\_LOG\_TOPIC\_TOO\_LONG\_EC, [411](#)
- DDSC\_LOG\_TOPICQOS\_OBJECT, [375](#)
- DDSC\_LOG\_TRANSPORT\_COMPONENT, [387](#)
- DDSC\_LOG\_TRANSPORT\_QOS\_POLICY, [392](#)
- DDSC\_LOG\_TRUST\_ADVANCE\_SN\_FAILED\_EC, [435](#)
- DDSC\_LOG\_TRUST\_AFTER\_REMOTE\_PARTICIPANT\_AUTHENTICATED\_EC, [439](#)
- DDSC\_LOG\_TRUST\_AFTER\_REMOTE\_PARTICIPANT\_READY\_FAILED\_EC, [431](#)
- DDSC\_LOG\_TRUST\_ALLOC\_FAILED\_EC, [434](#)
- DDSC\_LOG\_TRUST\_ASSERT\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC, [440](#)
- DDSC\_LOG\_TRUST\_ASSERT\_PARTICIPANT\_GENERIC\_MESSAGE\_EC, [436](#)
- DDSC\_LOG\_TRUST\_ASSERT\_PROPERTY\_FAILED\_EC, [438](#)
- DDSC\_LOG\_TRUST\_BEGIN\_HANDSHAKE\_REPLY\_EC, [428](#)
- DDSC\_LOG\_TRUST\_BEGIN\_HANDSHAKE\_REQUEST\_EC, [431](#)
- DDSC\_LOG\_TRUST\_BIND\_REMOTE\_DATAREADER\_FAILED\_EC, [432](#)
- DDSC\_LOG\_TRUST\_BIND\_REMOTE\_DATAWRITER\_FAILED\_EC, [432](#)
- DDSC\_LOG\_TRUST\_CACHE\_REMOTE\_PARTICIPANT\_SAMPLE\_FAILED\_EC, [439](#)
- DDSC\_LOG\_TRUST\_CHECK\_CREATE\_DATAREADER\_FAILED\_EC, [428](#)
- DDSC\_LOG\_TRUST\_CHECK\_CREATE\_DATAWRITER\_FAILED\_EC, [428](#)
- DDSC\_LOG\_TRUST\_CHECK\_CREATE\_PARTICIPANT\_FAILED\_EC, [425](#)
- DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_DATAREADER\_FAILED\_EC, [430](#)
- DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_DATAWRITER\_FAILED\_EC, [430](#)
- DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_PARTICIPANT\_FAILED\_EC, [427](#)
- DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DATAREADER\_INTERCEPTOR\_EC, [437](#)
- DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DATAWRITER\_INTERCEPTOR\_EC, [437](#)
- DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DR\_TOKEN\_EC, [429](#)
- DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DW\_TOKEN\_EC, [429](#)
- DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_PARTICIPANT\_INTERCEPTOR\_EC, [428](#)
- DDSC\_LOG\_TRUST\_CREATE\_TRUST\_PLUGIN\_EC, [441](#)
- DDSC\_LOG\_TRUST\_CREATE\_TRUST\_PLUGINS\_FAILED\_EC, [431](#)

DDSC\_LOG\_TRUST\_DATA\_HOLDER\_COPY\_FAILED\_EC, DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_TOKENS\_EC,  
 434 436  
 DDSC\_LOG\_TRUST\_DELETE\_TRUST\_PLUGINS\_EC, DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_GENERIC\_MESSAGE,  
 427 435  
 DDSC\_LOG\_TRUST\_ENDPOINT\_INTERNAL\_ATTRIBUTES\_FAILED\_EC, DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_GUID\_EC,  
 437 424  
 DDSC\_LOG\_TRUST\_FAILED\_VALIDATE\_LOCAL\_IDENTITY\_EC, DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_INTERNAL\_ATTRIBUTES,  
 424 439  
 DDSC\_LOG\_TRUST\_FINALIZE\_AC\_PLUGIN\_PROPERTIES\_FAILED\_EC, DDSC\_LOG\_TRUST\_INVALID\_PERMISSIONS\_HANDLE\_EC,  
 439 424  
 DDSC\_LOG\_TRUST\_FINALIZE\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC, DDSC\_LOG\_TRUST\_INVALID\_SAMPLE\_MAX\_SERIALIZED\_SIZE\_EC,  
 440 434  
 DDSC\_LOG\_TRUST\_FINALIZE\_REMOTE\_PARTICIPANT\_TRUST\_FAILED\_EC, DDSC\_LOG\_TRUST\_INVALID\_SHARED\_SECRET\_EC,  
 440 438  
 DDSC\_LOG\_TRUST\_GET\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC, DDSC\_LOG\_TRUST\_LOOKUP\_FACTORY\_FAILED\_EC,  
 440 441  
 DDSC\_LOG\_TRUST\_GET\_AUTHENTICATED\_PEER\_CREDENTIALS\_FAILED\_EC, DDSC\_LOG\_TRUST\_LOOKUP\_FACTORY\_PROPERTY\_FAILED\_EC,  
 426 431  
 DDSC\_LOG\_TRUST\_GET\_DATAREADER\_TRUST\_ATTRIBUTES\_FAILED\_EC, DDSC\_LOG\_TRUST\_PARTICIPANT\_AUTHENTICATION\_FAILED\_EC,  
 428 436  
 DDSC\_LOG\_TRUST\_GET\_DATAWRITER\_TRUST\_ATTRIBUTES\_FAILED\_EC, DDSC\_LOG\_TRUST\_PARTICIPANT\_INTERNAL\_ATTRIBUTES\_CREATION\_FAILED\_EC,  
 428 439  
 DDSC\_LOG\_TRUST\_GET\_IDENTITY\_TOKEN\_FAILED\_EC, DDSC\_LOG\_TRUST\_PARTICIPANT\_UNEXPECTED\_AUTHENTICATION\_FAILED\_EC,  
 424 438  
 DDSC\_LOG\_TRUST\_GET\_PARTICIPANT\_TRUST\_ATTRIBUTES\_FAILED\_EC, DDSC\_LOG\_TRUST\_PBUFS\_MAX\_EXCEEDED\_EC,  
 425 441  
 DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC, DDSC\_LOG\_TRUST\_PREPARE\_AUTH\_HANDSHAKE\_MESSAGE\_FAILED\_EC,  
 425 433  
 DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_TOKEN\_FAILED\_EC, DDSC\_LOG\_TRUST\_PREPARE\_AUTH\_REQUEST\_EC,  
 425 433  
 DDSC\_LOG\_TRUST\_GET\_SHARED\_SECRET\_FAILED\_EC, DDSC\_LOG\_TRUST\_PREPARE\_ENDPOINT\_INTERCEPTOR\_MESSAGE\_FAILED\_EC,  
 427 433  
 DDSC\_LOG\_TRUST\_GET\_TOPIC\_TRUST\_ATTRIBUTES\_FAILED\_EC, DDSC\_LOG\_TRUST\_PREPARE\_LOCAL\_DP\_TOKENS\_EC,  
 427 438  
 DDSC\_LOG\_TRUST\_IGNORED\_HANDSHAKE\_MESSAGE\_EC, DDSC\_LOG\_TRUST\_PREPARE\_LOCAL\_PARTICIPANT\_STATE\_FAILED\_EC,  
 423 427  
 DDSC\_LOG\_TRUST\_IGNORED\_REMOTE\_PARTICIPANT\_TRUST\_ATTRIBUTES\_EC, DDSC\_LOG\_TRUST\_PROCESS\_HANDSHAKE\_EC,  
 423 431  
 DDSC\_LOG\_TRUST\_INCOMPATIBLE\_ENDPOINT\_TRUST\_ATTRIBUTES\_EC, DDSC\_LOG\_TRUST\_PROCESS\_REMOTE\_INTERCEPTOR\_TOKENS\_FAILED\_EC,  
 443 438  
 DDSC\_LOG\_TRUST\_INITIALIZE\_PARTICIPANT\_BUILTIN\_DATA\_FAILED\_EC, DDSC\_LOG\_TRUST\_REAUTH\_REMOTE\_PARTICIPANT\_FAILED\_EC,  
 435 441  
 DDSC\_LOG\_TRUST\_INITIALIZE\_PARTICIPANT\_TRUST\_ATTRIBUTES\_FAILED\_EC, DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_DATAREADER\_FAILED\_EC,  
 434 429  
 DDSC\_LOG\_TRUST\_INVALID\_AUTH\_HANDSHAKE\_STATE\_EC, DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_DATAWRITER\_FAILED\_EC,  
 440 430  
 DDSC\_LOG\_TRUST\_INVALID\_GENERIC\_MESSAGE\_REPLY\_EC, DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_PARTICIPANT\_FAILED\_EC,  
 434 425  
 DDSC\_LOG\_TRUST\_INVALID\_IDENTITY\_HANDLE\_EC, DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_DATAREADER\_FAILED\_EC,  
 424 430  
 DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_HANDLE\_EC, DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_DATAWRITER\_FAILED\_EC,  
 425 430  
 DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_TOKENS\_DESTINATION\_FAILED\_EC, DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_PARTICIPANT\_FAILED\_EC,  
 437 427



|                                                                     |                                                            |
|---------------------------------------------------------------------|------------------------------------------------------------|
| DDSC_LOG_TRUST_REGISTER_REMOTE_DATAREADER_FAILED_EC,                | DDSC_LOG_TRUST_SET_REMOTE_DATAREADER_INTERCEPTOR,          |
| 432                                                                 | 429                                                        |
| DDSC_LOG_TRUST_REGISTER_REMOTE_DATAWRITER_FAILED_EC,                | DDSC_LOG_TRUST_SET_REMOTE_DATAWRITER_INTERCEPTOR,          |
| 432                                                                 | 429                                                        |
| DDSC_LOG_TRUST_RELEASE_PARTICIPANT_GENERIC_MESSAGE_FAILED_EC,       | DDSC_LOG_TRUST_SET_REMOTE_PARTICIPANT_INTERCEPTOR,         |
| 435                                                                 | 429                                                        |
| DDSC_LOG_TRUST_REMOVE_REMOTE_PARTICIPANT_FAILED_EC,                 | DDSC_LOG_TRUST_TRANFORM_INCOMING_SERIALIZED_PAYLOAD,       |
| 431                                                                 | 439                                                        |
| DDSC_LOG_TRUST_RESEND_HANDSHAKE_IN_PROGRESS_FAILED_EC,              | DDSC_LOG_TRUST_TRANFORM_OUTGOING_SERIALIZED_PAYLOAD,       |
| 435                                                                 | 439                                                        |
| DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_FAILED_EC,                  | DDSC_LOG_TRUST_UNAUTHORIZED_REMOTE_PARTICIPANT_EC,         |
| 441                                                                 | 424                                                        |
| DDSC_LOG_TRUST_RESEND_HANDSHAKE_MESSAGE_START_FAILED_EC,            | DDSC_LOG_TRUST_UNEXPECTED_REMOTE_PARTICIPANT_STATE,        |
| 436                                                                 | 436                                                        |
| DDSC_LOG_TRUST_RESEND_HANDSHAKE_STOP_FAILED_EC,                     | DDSC_LOG_TRUST_UNEXPECTED_VALIDATION_RESULT_EC,            |
| 435                                                                 | 424                                                        |
| DDSC_LOG_TRUST_RESET_REMOTE_PARTICIPANT_TRUST_FAILED_EC,            | DDSC_LOG_TRUST_UNKNOWN_GMCLASSID_EC,                       |
| 440                                                                 | 423                                                        |
| DDSC_LOG_TRUST_RETURN_AUTHENTICATED_PEER_OBSOLETE_TOKEN_UNKNOWN_EC, | DDSC_LOG_TRUST_UNKNOWN_SERVICEID_EC,                       |
| 437                                                                 | 423                                                        |
| DDSC_LOG_TRUST_RETURN_DATAWRITER_TRUST_ATTRIBUTES_FAILED_EC,        | DDSC_LOG_TRUST_UNREGISTER_DATAREADER_FAILED_EC,            |
| 428                                                                 | 430                                                        |
| DDSC_LOG_TRUST_RETURN_HANDSHAKE_HANDLE_FAILED_EC,                   | DDSC_LOG_TRUST_UNREGISTER_DATAWRITER_FAILED_EC,            |
| 429                                                                 | 430                                                        |
| DDSC_LOG_TRUST_RETURN_IDENTITY_HANDLE_FAILED_EC,                    | DDSC_LOG_TRUST_UNREGISTER_PARTICIPANT_FAILED_EC,           |
| 426                                                                 | 426                                                        |
| DDSC_LOG_TRUST_RETURN_IDENTITY_TOKEN_FAILED_EC,                     | DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAREADER_FAILED_EC,     |
| 426                                                                 | 438                                                        |
| DDSC_LOG_TRUST_RETURN_INTERCEPTOR_TOKENS_FAILED_EC,                 | DDSC_LOG_TRUST_UNREGISTER_REMOTE_DATAWRITER_FAILED_EC,     |
| 437                                                                 | 438                                                        |
| DDSC_LOG_TRUST_RETURN_PERMISSIONS_CREDENTIALS_TOKEN_FAILED_EC,      | DDSC_LOG_TRUST_VALIDATE_LOCAL_DATAWRITER_TRUST_FAILED_EC,  |
| 425                                                                 | 433                                                        |
| DDSC_LOG_TRUST_RETURN_PERMISSIONS_HANDLE_FAILED_EC,                 | DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAREADER_TRUST_FAILED_EC, |
| 426                                                                 | 433                                                        |
| DDSC_LOG_TRUST_RETURN_PERMISSIONS_TOKEN_FAILED_EC,                  | DDSC_LOG_TRUST_VALIDATE_REMOTE_DATAWRITER_TRUST_FAILED_EC, |
| 426                                                                 | 433                                                        |
| DDSC_LOG_TRUST_RETURN_SHAREDSECRET_HANDLE_FAILED_EC,                | DDSC_LOG_TRUST_VALIDATE_REMOTE_IDENTITY_EC,                |
| 426                                                                 | 431                                                        |
| DDSC_LOG_TRUST_SEND_AUTH_REQUEST_FAILED_EC,                         | DDSC_LOG_TRUST_WRITE_SAMPLE_FAILED_EC,                     |
| 440                                                                 | 435                                                        |
| DDSC_LOG_TRUST_SEND_ENDPOINT_INTERCEPTOR_STATE_FAILED_EC,           | DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE,                        |
| 437                                                                 | 410                                                        |
| DDSC_LOG_TRUST_SEND_HANDSHAKE_EC,                                   | DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE,                      |
| 432                                                                 | 410                                                        |
| DDSC_LOG_TRUST_SEND_HANDSHAKE_MESSAGE_FAILED_EC,                    | DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE,                      |
| 436                                                                 | 410                                                        |
| DDSC_LOG_TRUST_SEND_PARTICIPANT_INTERCEPTOR_STATE_FAILED_EC,        | DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA,                   |
| 436                                                                 | 410                                                        |
| DDSC_LOG_TRUST_SERIALIZE_DATA_FAILED_EC,                            | DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND,                       |
| 434                                                                 | 411                                                        |
| DDSC_LOG_TRUST_SET_AUTH_HANDSHAKE_STATE_FAILED_EC,                  | DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE,     |
| 441                                                                 | 410                                                        |
| DDSC_LOG_TRUST_SET_PERMISSIONS_CREDENTIAL_TOKEN_FAILED_EC,          | DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH,                |
| 427                                                                 | 411                                                        |

- DDSC\_LOG\_TYPE\_FUNCTION\_NULL\_EC, [411](#)
- DDSC\_LOG\_TYPE\_FUNCTION\_SERIALIZE\_DATA, [410](#)
- DDSC\_LOG\_TYPE\_GET\_SAMPLE\_ID\_EC, [412](#)
- DDSC\_LOG\_TYPE\_KEY\_TYPE\_EC, [411](#)
- DDSC\_LOG\_TYPE\_MANAGED\_POOL\_FAILURE\_EC, [412](#)
- DDSC\_LOG\_TYPE\_NAME\_CMP\_EC, [409](#)
- DDSC\_LOG\_TYPE\_OBJECT, [376](#)
- DDSC\_LOG\_TYPE\_PLUGIN\_GET\_BUFFER\_EC, [412](#)
- DDSC\_LOG\_TYPE\_PLUGIN\_INCONSISTENT\_DR\_EC, [412](#)
- DDSC\_LOG\_TYPE\_RECORD, [372](#)
- DDSC\_LOG\_TYPE\_SUPPORT\_QOS\_POLICY, [391](#)
- DDSC\_LOG\_TYPE\_TOO\_LONG\_EC, [411](#)
- DDSC\_LOG\_UNKNOWN\_REMOTE\_PARTICIPANT\_KEY\_EC, [364](#)
- DDSC\_LOG\_UNKNOWN\_REMOTE\_PARTICIPANT\_NAME\_EC, [364](#)
- DDSC\_LOG\_UNLOCK\_EC, [445](#)
- DDSC\_LOG\_UNSUPPORTED\_LOCATOR\_EC, [417](#)
- DDSC\_LOG\_UPDATE\_LEASE\_DURATION\_TIMER\_EC, [442](#)
- DDSC\_LOG\_UPDATE\_LIVELINESS\_FAILED\_EC, [442](#)
- DDSC\_LOG\_UPDATE\_REMOTE\_CONTENT\_FILTER\_EC, [447](#)
- DDSC\_LOG\_USERMULTICAST\_SEQUENCE, [384](#)
- DDSC\_LOG\_USERUNICAST\_SEQUENCE, [384](#)
- DDSC\_LOG\_UUID\_OBJECT, [379](#)
- DDSC\_LOG\_VALIDATE\_REMOTE\_PARTICIPANT\_TRUST\_FAILED\_EC, [432](#)
- DDSC\_LOG\_WAITSET\_OBJECT, [379](#)
- DDSC\_LOG\_WRITERHISTORY\_COMPONENT, [386](#)
- DDSC\_LOG\_WS\_ADD\_COND\_REFERENCE\_EC, [367](#)
- DDSC\_LOG\_WS\_REMOVE\_COND\_REFERENCE\_EC, [367](#)
- DDSC\_LOG\_WSREF\_OBJECT, [378](#)
- DDSC\_LOG\_XTYPES\_LOANED\_SAMPLE\_STATE\_ERROR\_EC, [443](#)
- DDSC\_TRUST\_INVALID\_PARTICIPANT\_GENERIC\_MESSAGE\_EC, [433](#)
- dds\_c\_config.h, [871](#)
- dds\_c\_flowcontroller.h, [871](#)
- dds\_c\_log.h, [872](#)
- DDS\_Char
  - DDS-Specific Primitive Types, [48](#)
- DDS\_CHECKSUM\_AUTO
  - WIRE\_PROTOCOL, [122](#)
- DDS\_CHECKSUM\_BUILTIN128
  - WIRE\_PROTOCOL, [122](#)
- DDS\_CHECKSUM\_BUILTIN32
  - WIRE\_PROTOCOL, [122](#)
- DDS\_CHECKSUM\_BUILTIN64
  - WIRE\_PROTOCOL, [122](#)
- DDS\_CHECKSUM\_NONE
  - WIRE\_PROTOCOL, [122](#)
- DDS\_ChecksumKind\_t
  - WIRE\_PROTOCOL, [123](#)
- DDS\_ChecksumKindMask\_t
  - WIRE\_PROTOCOL, [123](#)
- DDS\_ChecksumProperty, [496](#)
  - allowed\_crc\_mask, [497](#)
  - computed\_crc\_kind, [497](#)
  - require\_crc, [497](#)
- DDS\_ChecksumProperty\_t, [497](#)
  - allowed\_crc\_mask, [498](#)
  - computed\_crc\_kind, [498](#)
  - require\_crc, [498](#)
- DDS\_CONTENT\_FILTER\_CLASS\_NAME\_MAX\_LENGTH
  - FILTER, [102](#)
- DDS\_CONTENT\_FILTER\_EXPRESSION\_MAX\_LENGTH
  - FILTER, [103](#)
- DDS\_CONTENT\_FILTER\_MAX\_PARAMETERS
  - FILTER, [103](#)
- DDS\_CONTENT\_FILTER\_NAME\_MAX\_LENGTH
  - FILTER, [102](#)
- DDS\_ContentFilterProperty, [498](#)
  - content\_filtered\_topic\_name, [499](#)
  - expression\_parameters, [499](#)
  - filter\_class\_name, [499](#)
  - filter\_expression, [499](#)
  - filter\_name, [499](#)
- DDS\_ContentFilterQosPolicy, [500](#)
  - expression\_parameters, [502](#)
  - filter\_class\_name, [501](#)
  - filter\_expression, [502](#)
  - filter\_name, [501](#)
- DDS\_DATA\_AVAILABLE\_STATUS
  - Status Kinds, [94](#)
- DDS\_DATA\_ON\_READERS\_STATUS
  - Status Kinds, [94](#)
- DDS\_DATA\_WRITER\_SAMPLE\_REMOVED\_STATUS
  - Status Kinds, [97](#)
- DDS\_DATAREADER
  - DDS\_DATAREADER\_CPP
    - User Data Type Support, [55](#)
  - DDS\_DATAREADER\_ENTITY\_KIND
    - Infrastructure Module, [80](#)
  - DDS\_DATAREADER\_QOS\_DEFAULT
    - Subscriber, [69](#)
  - DDS\_DATAREADER\_QOS\_USE\_TOPIC\_QOS
    - Subscriber, [69](#)
  - DDS\_DataReaderInstanceReplacedStatus, [502](#)
    - last\_instance\_handle, [503](#)
    - last\_replacement\_instance, [503](#)

- lost\_samples, 503
- publication\_handle, 503
- total\_count, 503
- total\_count\_change, 503
- DDS\_DataReaderProtocolQosPolicy, 504
  - rtps\_object\_id, 504
  - rtps\_reliable\_reader, 504
- DDS\_DataReaderQos, 505
  - content\_filter, 508
  - deadline, 506
  - destination\_order, 507
  - durability, 507
  - history, 506
  - liveliness, 506
  - ownership, 506
  - property, 508
  - protocol, 507
  - reader\_resource\_limits, 507
  - reliability, 506
  - representation, 507
  - resource\_limits, 506
  - subscription\_name, 508
  - transport, 507
  - user\_data, 507
- DDS\_DataReaderResourceLimitsInstanceReplacementKind
  - DataReader Resource Limits, 119
- DDS\_DataReaderResourceLimitsQosPolicy, 508
  - instance\_replacement, 510
  - max\_fragmented\_samples, 511
  - max\_fragmented\_samples\_per\_remote\_writer, 511
  - max\_outstanding\_reads, 510
  - max\_remote\_writers, 509
  - max\_remote\_writers\_per\_instance, 509
  - max\_routes\_per\_writer, 510
  - max\_samples\_per\_remote\_writer, 510
  - shmем\_ref\_transfer\_mode\_attached\_segment\_allocation, 511
- DDS\_DataRepresentationId\_t
  - DATA\_REPRESENTATION, 114
- DDS\_DataRepresentationQosPolicy, 512
- DDS\_DATAWRITER\_CPP
  - User Data Type Support, 55
- DDS\_DATAWRITER\_ENTITY\_KIND
  - Infrastructure Module, 80
- DDS\_DATAWRITER\_QOS\_DEFAULT
  - Publisher, 65
- DDS\_DataWriterListener\_ReliableReaderActivityChangedCallback
  - Writer, 66
- DDS\_DataWriterListener\_ReliableSampleUnacknowledgedCallback
  - Writer, 66
- DDS\_DataWriterListener\_SampleRemovedCallback
  - Writer, 67
- DDS\_DataWriterProtocolQosPolicy, 513
  - rtps\_object\_id, 513
  - rtps\_reliable\_writer, 513
  - serialize\_on\_write, 514
- DDS\_DataWriterQos, 514
  - deadline, 515
  - destination\_order, 516
  - durability, 516
  - history, 516
  - liveliness, 515
  - ownership, 516
  - ownership\_strength, 516
  - property, 517
  - protocol, 517
  - publication\_name, 518
  - publish\_mode, 517
  - reliability, 516
  - representation, 517
  - resource\_limits, 516
  - transport, 517
  - user\_data, 517
  - writer\_resource\_limits, 517
- DDS\_DataWriterResourceLimitsQosPolicy, 518
  - initialize\_writer\_loaned\_sample, 519
  - max\_remote\_reader\_filters, 520
  - max\_remote\_readers, 519
  - max\_routes\_per\_reader, 519
  - writer\_loaned\_sample\_allocation, 519
- DDS\_DataWriterShmemRefTransferModeSettings, 521
  - enable\_data\_consistency\_check, 521
- DDS\_DataWriterTransferModeQosPolicy, 521
  - shmем\_ref\_settings, 522
- DDS\_DEADLINE\_QOS\_POLICY\_ID
  - QoS Policies, 101
- DDS\_DeadlineQosPolicy, 522
  - period, 523
- DDS\_DEFAULT\_FLOW\_CONTROLLER\_NAME
  - Flow Controllers, 61
- DDS\_DestinationOrderQosPolicy, 524
  - kind, 524
  - source\_timestamp\_tolerance, 524
- DDS\_DestinationOrderQosPolicyKind
  - DESTINATION\_ORDER, 117
- DDS\_DiscoveryComponent, 525
  - name, 525
- DDS\_DiscoveryQosPolicy, 525
  - accept\_unknown\_peers, 527
  - discovery, 526
  - enable\_endpoint\_discovery\_queue, 527
  - enable\_participant\_discovery\_by\_name, 527
  - enabled\_transports, 526
  - initial\_peers, 526
- DDS\_DomainId\_t
  - Domain Module, 31
- DDS\_DomainParticipantFactoryQos, 528
  - entity\_factory, 529

- resource\_limits, 529
- DDS\_DomainParticipantQos, 529
  - discovery, 530
  - entity\_factory, 530
  - filter, 531
  - participant\_name, 530
  - property, 531
  - protocol, 530
  - resource\_limits, 530
  - transports, 531
  - trust, 531
  - user\_data, 531
  - user\_traffic, 531
- DDS\_DomainParticipantResourceLimitsQosPolicy, 532
  - local\_publisher\_allocation, 534
  - local\_reader\_allocation, 534
  - local\_subscriber\_allocation, 534
  - local\_topic\_allocation, 534
  - local\_type\_allocation, 534
  - local\_writer\_allocation, 534
  - matching\_reader\_writer\_pair\_allocation, 535
  - matching\_writer\_reader\_pair\_allocation, 535
  - max\_destination\_ports, 536
  - max\_partition\_cumulative\_characters, 539
  - max\_partition\_string\_allocation, 540
  - max\_partition\_string\_size, 539
  - max\_partitions, 539
  - max\_receive\_ports, 536
  - participant\_user\_data\_max\_count, 537
  - participant\_user\_data\_max\_length, 536
  - publisher\_group\_data\_max\_count, 537
  - publisher\_group\_data\_max\_length, 537
  - reader\_user\_data\_max\_count, 539
  - reader\_user\_data\_max\_length, 538
  - remote\_participant\_allocation, 535
  - remote\_reader\_allocation, 535
  - remote\_writer\_allocation, 535
  - shmem\_ref\_transfer\_mode\_max\_segments, 536
  - subscriber\_group\_data\_max\_count, 538
  - subscriber\_group\_data\_max\_length, 538
  - topic\_data\_max\_count, 537
  - topic\_data\_max\_length, 537
  - writer\_user\_data\_max\_count, 538
  - writer\_user\_data\_max\_length, 538
- DDS\_Double
  - DDS-Specific Primitive Types, 51
- DDS\_DurabilityQosPolicy, 540
  - kind, 541
- DDS\_DurabilityQosPolicyKind
  - DURABILITY, 111
- DDS\_Duration\_equal
  - Time Support, 82
- DDS\_DURATION\_INFINITE
  - Time Support, 84
- DDS\_DURATION\_INFINITE\_NSEC
  - Time Support, 84
- DDS\_DURATION\_INFINITE\_SEC
  - Time Support, 83
- DDS\_Duration\_is\_infinite
  - Time Support, 82
- DDS\_Duration\_is\_zero
  - Time Support, 83
- DDS\_Duration\_t, 541
  - nanosec, 541
  - sec, 541
- DDS\_DURATION\_ZERO
  - Time Support, 84
- DDS\_DURATION\_ZERO\_NSEC
  - Time Support, 84
- DDS\_DURATION\_ZERO\_SEC
  - Time Support, 84
- DDS\_EDF\_FLOW\_CONTROLLER\_SCHED\_POLICY
  - Flow Controllers, 60
- DDS\_ENTITYFACTORY\_QOS\_POLICY\_ID
  - QoS Policies, 101
- DDS\_EntityFactoryQosPolicy, 542
  - autoenable\_created\_entities, 543
- DDS\_EntityKind\_t
  - Infrastructure Module, 80
- DDS\_ENTITYNAME\_QOS\_NAME\_MAX
  - ENTITY\_NAME, 126
- DDS\_EntityNameQosPolicy, 543
  - name, 544
- DDS\_Enum
  - DDS-Specific Primitive Types, 51
- DDS\_EXCLUSIVE\_OWNERSHIP\_QOS
  - OWNERSHIP, 106
- DDS\_FILTER, 485
  - DDS\_FILTER\_LOG\_COMPILED\_FILTER\_RECORD, 487
  - DDS\_FILTER\_LOG\_CONTENT\_FILTER\_NAME\_STRING, 491
  - DDS\_FILTER\_LOG\_CONTENT\_FILTER\_NAME\_TOO\_LONG\_EC, 491
  - DDS\_FILTER\_LOG\_CONTENT\_FILTERED\_TOPIC\_NAME\_STRING, 490
  - DDS\_FILTER\_LOG\_FILTER\_CLASS\_NAME\_STRING, 490
  - DDS\_FILTER\_LOG\_FILTER\_CLASS\_NOT\_FOUND\_EC, 490
  - DDS\_FILTER\_LOG\_FILTER\_CLASS\_RECORD, 487
  - DDS\_FILTER\_LOG\_FILTER\_CLASS\_REGISTER\_EC, 490
  - DDS\_FILTER\_LOG\_FILTER\_CLASS\_UNREGISTER\_EC, 490
  - DDS\_FILTER\_LOG\_FILTER\_COMPILE\_EC, 489
  - DDS\_FILTER\_LOG\_FILTER\_CREATE\_INSTANCE\_EC, 489

DDS\_FILTER\_LOG\_FILTER\_DELETE\_INSTANCE\_EC, 492  
 489  
 DDS\_FILTER\_LOG\_FILTER\_EVALUATE\_EC, 489 493  
 DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_MAX\_COUNT\_EC, 489 492  
 DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_PARAMETER\_STRING, 491 493  
 DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_STRING, 490 493  
 DDS\_FILTER\_LOG\_FILTER\_PARAMETER\_COUNT\_EC, 492 493  
 DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_FACTORY\_IN\_USE\_EC, 492 898  
 DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_FINALIZE\_EC, 492 901  
 DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_INIT\_EC, 492 901  
 DDS\_FILTER\_LOG\_HEAP\_ALLOC\_EC, 492  
 DDS\_FILTER\_LOG\_READER\_RECORD, 487  
 DDS\_FILTER\_LOG\_RECORD\_CREATE\_EC, 488  
 DDS\_FILTER\_LOG\_RECORD\_DELETE\_EC, 488  
 DDS\_FILTER\_LOG\_RECORD\_INSERT\_EC, 488  
 DDS\_FILTER\_LOG\_RECORD\_REMOVE\_EC, 488  
 DDS\_FILTER\_LOG\_RECORD\_SELECT\_EC, 488  
 DDS\_FILTER\_LOG\_RELATED\_TOPIC\_NAME\_STRING, 490  
 DDS\_FILTER\_LOG\_ROUTE\_RECORD, 488  
 DDS\_FILTER\_LOG\_SQL\_ENUM\_LABEL\_NOT\_FOUND\_EC, 495  
 DDS\_FILTER\_LOG\_SQL\_FIELD\_NOT\_FOUND\_EC, 494  
 DDS\_FILTER\_LOG\_SQL\_INCOMPATIBLE\_TYPES\_EC, 495  
 DDS\_FILTER\_LOG\_SQL\_INTEGER\_OUT\_OF\_RANGE\_EC, 495  
 DDS\_FILTER\_LOG\_SQL\_INVALID\_TOKEN\_EC, 494  
 DDS\_FILTER\_LOG\_SQL\_MAX\_PREDICATES\_EXCEEDED\_EC, 494  
 DDS\_FILTER\_LOG\_SQL\_PARSE\_PARAMETER\_EC, 494  
 DDS\_FILTER\_LOG\_SQL\_UNEXPECTED\_TOKEN\_EC, 493  
 DDS\_FILTER\_LOG\_SQL\_UNSUPPORTED\_FIELD\_TYPE\_EC, 494  
 DDS\_FILTER\_LOG\_SQL\_UNSUPPORTED\_TYPE\_CODE\_EC, 493  
 DDS\_FILTER\_LOG\_STRING\_ASSERT\_EC, 491  
 DDS\_FILTER\_LOG\_STRING\_DELETE\_EC, 491  
 DDS\_FILTER\_LOG\_STRING\_MANAGER\_CREATE\_EC, 491  
 DDS\_FILTER\_LOG\_TABLE\_CREATE\_EC, 488  
 DDS\_FILTER\_LOG\_TABLE\_DELETE\_EC, 489  
 DDS\_FILTER\_LOG\_WRITER\_DETACH\_EC, 489  
 DDS\_FILTER\_LOG\_WRITER\_FILTER\_COMPILE\_EC, 492  
 DDS\_FILTER\_LOG\_WRITER\_FILTER\_EVALUATE\_OUT\_OF\_ORDER\_EC, 493  
 DDS\_FILTER\_LOG\_WRITER\_FILTER\_REMOVE\_READER\_EC, 492  
 DDS\_FILTER\_LOG\_WRITER\_FILTER\_SEND\_GAP\_EC, 493  
 DDS\_FILTER\_LOG\_WRITER\_FILTER\_STORE\_RESULT\_EC, 493  
 DDS\_FILTER\_LOG\_WRITER\_SERIALIZE\_INLINE\_QOS\_EC, 493  
 DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_BUFFER, 901  
 DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_FACTORY\_BUFFER, 901  
 DDS\_FILTER\_LOG\_INLINE\_QOS\_BUFFER, 901  
 DDS\_FILTER\_LOG\_TYPE\_CODE\_NOT\_FOUND\_EC, 901  
 DDS\_FILTER\_LOG\_WRITER\_FILTER\_BUFFER, 901  
 DDS\_FILTER\_LOG\_BASE  
 osapi\_log.h, 932  
 DDS\_FILTER\_LOG\_COMPILED\_FILTER\_RECORD  
 DDS\_FILTER, 487  
 DDS\_FILTER\_LOG\_CONTENT\_FILTER\_NAME\_STRING  
 DDS\_FILTER, 491  
 DDS\_FILTER\_LOG\_CONTENT\_FILTER\_NAME\_TOO\_LONG\_EC  
 DDS\_FILTER, 491  
 DDS\_FILTER\_LOG\_CONTENT\_FILTERED\_TOPIC\_NAME\_STRING  
 DDS\_FILTER, 490  
 DDS\_FILTER\_LOG\_FILTER\_CLASS\_NAME\_STRING  
 DDS\_FILTER, 490  
 DDS\_FILTER\_LOG\_FILTER\_CLASS\_NOT\_FOUND\_EC  
 DDS\_FILTER, 490  
 DDS\_FILTER\_LOG\_FILTER\_CLASS\_RECORD  
 DDS\_FILTER, 487  
 DDS\_FILTER\_LOG\_FILTER\_CLASS\_REGISTER\_EC  
 DDS\_FILTER, 490  
 DDS\_FILTER\_LOG\_FILTER\_CLASS\_UNREGISTER\_EC  
 DDS\_FILTER, 490  
 DDS\_FILTER\_LOG\_FILTER\_COMPILE\_EC  
 DDS\_FILTER, 489  
 DDS\_FILTER\_LOG\_FILTER\_CREATE\_INSTANCE\_EC  
 DDS\_FILTER, 489  
 DDS\_FILTER\_LOG\_FILTER\_DELETE\_INSTANCE\_EC  
 DDS\_FILTER, 489  
 DDS\_FILTER\_LOG\_FILTER\_EVALUATE\_EC  
 DDS\_FILTER, 489  
 DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_MAX\_COUNT\_EC  
 DDS\_FILTER, 489  
 DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_PARAMETER\_STRING  
 DDS\_FILTER, 491  
 DDS\_FILTER\_LOG\_FILTER\_EXPRESSION\_STRING

- DDS\_FILTER, [490](#)
- DDS\_FILTER\_LOG\_FILTER\_PARAMETER\_COUNT\_EC
  - DDS\_FILTER, [492](#)
- DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_BUFFER
  - dds\_filter\_log.h, [901](#)
- DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_FACTORY\_BUFFER
  - dds\_filter\_log.h, [901](#)
- DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_FACTORY\_IN\_USE\_EC
  - DDS\_FILTER, [492](#)
- DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_FINALIZE\_EC
  - DDS\_FILTER, [492](#)
- DDS\_FILTER\_LOG\_FILTER\_PLUGIN\_INIT\_EC
  - DDS\_FILTER, [492](#)
- DDS\_FILTER\_LOG\_HEAP\_ALLOC\_EC
  - DDS\_FILTER, [492](#)
- DDS\_FILTER\_LOG\_INLINE\_QOS\_BUFFER
  - dds\_filter\_log.h, [901](#)
- DDS\_FILTER\_LOG\_READER\_RECORD
  - DDS\_FILTER, [487](#)
- DDS\_FILTER\_LOG\_RECORD\_CREATE\_EC
  - DDS\_FILTER, [488](#)
- DDS\_FILTER\_LOG\_RECORD\_DELETE\_EC
  - DDS\_FILTER, [488](#)
- DDS\_FILTER\_LOG\_RECORD\_INSERT\_EC
  - DDS\_FILTER, [488](#)
- DDS\_FILTER\_LOG\_RECORD\_REMOVE\_EC
  - DDS\_FILTER, [488](#)
- DDS\_FILTER\_LOG\_RECORD\_SELECT\_EC
  - DDS\_FILTER, [488](#)
- DDS\_FILTER\_LOG\_RELATED\_TOPIC\_NAME\_STRING
  - DDS\_FILTER, [490](#)
- DDS\_FILTER\_LOG\_ROUTE\_RECORD
  - DDS\_FILTER, [488](#)
- DDS\_FILTER\_LOG\_SQL\_ENUM\_LABEL\_NOT\_FOUND\_EC
  - DDS\_FILTER, [495](#)
- DDS\_FILTER\_LOG\_SQL\_FIELD\_NOT\_FOUND\_EC
  - DDS\_FILTER, [494](#)
- DDS\_FILTER\_LOG\_SQL\_INCOMPATIBLE\_TYPES\_EC
  - DDS\_FILTER, [495](#)
- DDS\_FILTER\_LOG\_SQL\_INTEGER\_OUT\_OF\_RANGE\_EC
  - DDS\_FILTER, [495](#)
- DDS\_FILTER\_LOG\_SQL\_INVALID\_TOKEN\_EC
  - DDS\_FILTER, [494](#)
- DDS\_FILTER\_LOG\_SQL\_MAX\_PREDICATES\_EXCEEDED\_EC
  - DDS\_FILTER, [494](#)
- DDS\_FILTER\_LOG\_SQL\_PARSE\_PARAMETER\_EC
  - DDS\_FILTER, [494](#)
- DDS\_FILTER\_LOG\_SQL\_UNEXPECTED\_TOKEN\_EC
  - DDS\_FILTER, [493](#)
- DDS\_FILTER\_LOG\_SQL\_UNSUPPORTED\_FIELD\_TYPE\_EC
  - DDS\_FILTER, [494](#)
- DDS\_FILTER\_LOG\_SQL\_UNSUPPORTED\_TYPE\_CODE\_EC
  - DDS\_FILTER, [493](#)
- DDS\_FILTER\_LOG\_STRING\_ASSERT\_EC
  - DDS\_FILTER, [491](#)
- DDS\_FILTER\_LOG\_STRING\_DELETE\_EC
  - DDS\_FILTER, [491](#)
- DDS\_FILTER\_LOG\_STRING\_MANAGER\_CREATE\_EC
  - DDS\_FILTER, [491](#)
- DDS\_FILTER\_LOG\_TABLE\_CREATE\_EC
  - DDS\_FILTER, [488](#)
- DDS\_FILTER\_LOG\_TABLE\_DELETE\_EC
  - DDS\_FILTER, [489](#)
- DDS\_FILTER\_LOG\_TYPE\_CODE\_NOT\_FOUND\_EC
  - dds\_filter\_log.h, [901](#)
- DDS\_FILTER\_LOG\_WRITER\_DETACH\_EC
  - DDS\_FILTER, [489](#)
- DDS\_FILTER\_LOG\_WRITER\_FILTER\_BUFFER
  - dds\_filter\_log.h, [901](#)
- DDS\_FILTER\_LOG\_WRITER\_FILTER\_COMPILE\_EC
  - DDS\_FILTER, [492](#)
- DDS\_FILTER\_LOG\_WRITER\_FILTER\_EVALUATE\_OUT\_OF\_ORDER\_EC
  - DDS\_FILTER, [493](#)
- DDS\_FILTER\_LOG\_WRITER\_FILTER\_REMOVE\_READER\_EC
  - DDS\_FILTER, [492](#)
- DDS\_FILTER\_LOG\_WRITER\_FILTER\_SEND\_GAP\_EC
  - DDS\_FILTER, [493](#)
- DDS\_FILTER\_LOG\_WRITER\_FILTER\_STORE\_RESULT\_EC
  - DDS\_FILTER, [493](#)
- DDS\_FILTER\_LOG\_WRITER\_SERIALIZE\_INLINE\_QOS\_EC
  - DDS\_FILTER, [493](#)
- DDS\_FILTER\_PLUGIN\_FACTORY\_DEFAULT\_NAME
  - FILTER, [103](#)
- DDS\_FILTER\_RESOURCE\_LIMITS\_DEFAULT
  - FILTER, [103](#)
- DDS\_FilterQosPolicy, [544](#)
  - disable\_builtin\_sql\_filter, [546](#)
  - disable\_writer\_filtering, [546](#)
  - name, [545](#)
  - resource\_limits, [545](#)
- DDS\_FilterResourceLimits, [546](#)
  - filter\_class\_max\_count, [547](#)
  - filter\_class\_max\_length, [547](#)
  - filter\_expression\_max\_count, [548](#)
  - filter\_expression\_max\_length, [547](#)
  - filter\_parameter\_max\_count\_per\_expression, [548](#)
  - filter\_parameter\_max\_length, [548](#)
  - sql\_predicate\_max\_count\_per\_expression, [548](#)
- DDS\_FIXED\_RATE\_FLOW\_CONTROLLER\_NAME
  - Flow Controllers, [62](#)
- DDS\_Float
  - DDS-Specific Primitive Types, [50](#)
- DDS\_FLOW\_CONTROLLER\_PROPERTY\_DEFAULT
  - DomainParticipant, [34](#)
- DDS\_FlowControllerProperty\_t, [549](#)
  - scheduling\_policy, [550](#)
  - token\_bucket, [550](#)
- DDS\_FlowControllerSchedulingPolicy

- Flow Controllers, [59](#)
- DDS\_FlowControllerTokenBucketProperty\_t, [550](#)
  - bytes\_per\_token, [552](#)
  - max\_tokens, [551](#)
  - period, [552](#)
  - tokens\_added\_per\_period, [551](#)
  - tokens\_leaked\_per\_period, [552](#)
- DDS\_GROUP\_PRESENTATION\_QOS
  - PRESSENTATION, [116](#)
- DDS\_GroupDataQosPolicy, [553](#)
  - value, [554](#)
- DDS\_GUID\_t, [554](#)
  - GUID Support, [85](#)
  - value, [555](#)
- DDS\_GUID\_UNKNOWN
  - GUID Support, [85](#)
- DDS\_HANDLE\_NIL
  - User Data Type Support, [57](#)
- DDS\_HISTORY\_QOS\_POLICY\_ID
  - QoS Policies, [101](#)
- DDS\_HistoryQosPolicy, [555](#)
  - depth, [556](#)
  - kind, [556](#)
- DDS\_HistoryQosPolicyKind
  - HISTORY, [110](#)
- DDS\_HPF\_FLOW\_CONTROLLER\_SCHED\_POLICY
  - Flow Controllers, [61](#)
- DDS\_INCONSISTENT\_TOPIC\_STATUS
  - Status Kinds, [92](#)
- DDS\_InconsistentTopicStatus, [557](#)
- DDS\_INSTANCE\_PRESENTATION\_QOS
  - PRESSENTATION, [116](#)
- DDS\_INSTANCE\_REPLACED\_STATUS
  - Status Kinds, [96](#)
- DDS\_InstanceHandle\_equals
  - User Data Type Support, [56](#)
- DDS\_InstanceHandle\_is\_nil
  - User Data Type Support, [56](#)
- DDS\_InstanceHandle\_t
  - User Data Type Support, [56](#)
- DDS\_InstanceHandleSeq, [557](#)
- DDS\_InstanceId\_t
  - Type Support, [134](#)
- DDS\_InstanceStateKind
  - Instance States, [77](#)
- DDS\_InstanceStateMask
  - Instance States, [77](#)
- DDS\_INTEROPERABLE\_RTPS\_WELL\_KNOWN\_PORTS
  - WIRE\_PROTOCOL, [124](#)
- DDS\_INVALID\_QOS\_POLICY\_ID
  - QoS Policies, [101](#)
- DDS\_KEEP\_ALL\_HISTORY\_QOS
  - HISTORY, [110](#)
- DDS\_KEEP\_LAST\_HISTORY\_QOS
  - HISTORY, [110](#)
- HISTORY, [110](#)
- DDS\_LatencyBudgetQosPolicy, [557](#)
  - duration, [559](#)
- DDS\_LENGTH\_AUTO
  - RESOURCE\_LIMITS, [115](#)
- DDS\_LENGTH\_UNLIMITED
  - RESOURCE\_LIMITS, [115](#)
- DDS\_LIVELINESS\_CHANGED\_STATUS
  - Status Kinds, [95](#)
- DDS\_LIVELINESS\_LOST\_STATUS
  - Status Kinds, [95](#)
- DDS\_LIVELINESS\_QOS\_POLICY\_ID
  - QoS Policies, [101](#)
- DDS\_LivelinessChangedStatus, [559](#)
  - alive\_count, [559](#)
  - alive\_count\_change, [560](#)
  - last\_publication\_handle, [560](#)
  - not\_alive\_count, [559](#)
  - not\_alive\_count\_change, [560](#)
- DDS\_LivelinessLostStatus, [560](#)
  - total\_count, [561](#)
  - total\_count\_change, [561](#)
- DDS\_LivelinessQosPolicy, [561](#)
  - kind, [564](#)
  - lease\_duration, [564](#)
- DDS\_LivelinessQosPolicyKind
  - LIVELINESS, [107](#)
- DDS\_LOCATOR\_ADDRESS\_INVALID
  - Discovery, [37](#)
- DDS\_LOCATOR\_INVALID
  - Discovery, [37](#)
- DDS\_LOCATOR\_KIND\_INVALID
  - Discovery, [37](#)
- DDS\_LOCATOR\_KIND\_RESERVED
  - Discovery, [37](#)
- DDS\_LOCATOR\_KIND\_SHMEM
  - Discovery, [37](#)
- DDS\_LOCATOR\_KIND\_UDPv4
  - Discovery, [37](#)
- DDS\_LOCATOR\_KIND\_UDPv6
  - Discovery, [37](#)
- DDS\_LOCATOR\_PORT\_INVALID
  - Discovery, [37](#)
- DDS\_Locator\_t, [564](#)
  - address, [565](#)
  - kind, [565](#)
  - port, [565](#)
- DDS\_LocatorSeq, [565](#)
- DDS\_LOG\_IPC\_ASSERT\_ROUTES\_EC
  - DDS\_C, [421](#)
- DDS\_LOG\_TRUST\_ASSERT\_PARTICIPANT\_GENERIC\_MESSAGE\_FAILURE
  - DDS\_C, [432](#)
- DDS\_Long
  - DDS-Specific Primitive Types, [49](#)



- DDS\_LongDouble
  - DDS-Specific Primitive Types, [51](#)
- DDS\_LongLong
  - DDS-Specific Primitive Types, [50](#)
- DDS\_MANUAL\_BY\_PARTICIPANT\_LIVELINESS\_QOS
  - LIVELINESS, [108](#)
- DDS\_MANUAL\_BY\_TOPIC\_LIVELINESS\_QOS
  - LIVELINESS, [108](#)
- DDS\_NEW\_VIEW\_STATE
  - View States, [76](#)
- DDS\_NO\_INSTANCE\_REPLACEMENT\_QOS
  - DataReader Resource Limits, [120](#)
- DDS\_NOT\_ALIVE\_DISPOSED\_INSTANCE\_STATE
  - Instance States, [78](#)
- DDS\_NOT\_ALIVE\_INSTANCE\_STATE
  - Instance States, [78](#)
- DDS\_NOT\_ALIVE\_NO\_WRITERS\_INSTANCE\_STATE
  - Instance States, [78](#)
- DDS\_NOT\_NEW\_VIEW\_STATE
  - View States, [76](#)
- DDS\_NOT\_READ\_SAMPLE\_STATE
  - Sample States, [74](#)
- DDS\_NOT\_REJECTED
  - DataReader, [71](#)
- DDS\_Octet
  - DDS-Specific Primitive Types, [48](#)
- DDS\_OFFERED\_DEADLINE\_MISSED\_STATUS
  - Status Kinds, [92](#)
- DDS\_OFFERED\_INCOMPATIBLE\_QOS\_STATUS
  - Status Kinds, [93](#)
- DDS\_OfferedDeadlineMissedStatus, [565](#)
  - last\_instance\_handle, [566](#)
  - total\_count, [566](#)
  - total\_count\_change, [566](#)
- DDS\_OfferedIncompatibleQosStatus, [567](#)
  - last\_policy\_id, [568](#)
  - policies, [568](#)
  - total\_count, [568](#)
  - total\_count\_change, [568](#)
- DDS\_ON\_DEMAND\_FLOW\_CONTROLLER\_NAME
  - Flow Controllers, [62](#)
- DDS\_OWNERSHIP\_QOS\_POLICY\_ID
  - QoS Policies, [101](#)
- DDS\_OwnershipQosPolicy, [568](#)
  - kind, [572](#)
- DDS\_OwnershipQosPolicyKind
  - OWNERSHIP, [106](#)
- DDS\_OWNERSHIPSTRENGTH\_QOS\_POLICY\_ID
  - QoS Policies, [101](#)
- DDS\_OwnershipStrengthQosPolicy, [573](#)
  - value, [573](#)
- DDS\_PARTICIPANT\_ENTITY\_KIND
  - Infrastructure Module, [80](#)
- DDS\_PARTICIPANT\_FACTORY\_QOS\_DEFAULT
  - DomainParticipantFactory, [32](#)
- DDS\_PARTICIPANT\_QOS\_DEFAULT
  - DomainParticipantFactory, [32](#)
- DDS\_PARTICIPANT\_TOPIC\_NAME
  - Participant Built-in Topic, [40](#)
- DDS\_ParticipantBuiltinTopicData, [574](#)
  - checksum, [576](#)
  - dds\_builtin\_endpoints, [575](#)
  - default\_multicast\_locators, [575](#)
  - default\_unicast\_locators, [575](#)
  - key, [575](#)
  - liveliness\_lease\_duration, [576](#)
  - metatraffic\_multicast\_locators, [576](#)
  - metatraffic\_unicast\_locators, [576](#)
  - Participant Built-in Topic, [40](#), [41](#)
  - participant\_name, [575](#)
  - product\_version, [576](#)
  - property, [576](#)
  - rtps\_protocol\_version, [575](#)
  - rtps\_vendor\_id, [575](#)
  - user\_data, [576](#)
- DDS\_ParticipantBuiltinTopicData\_finalize
  - Participant Built-in Topic, [41](#)
- DDS\_ParticipantBuiltinTopicData\_is\_equal
  - Participant Built-in Topic, [41](#)
- DDS\_PartitionQosPolicy, [577](#)
  - name, [579](#)
- DDS\_PresentationQosPolicy, [579](#)
  - access\_scope, [582](#)
  - coherent\_access, [582](#)
  - ordered\_access, [582](#)
- DDS\_PresentationQosPolicyAccessScopeKind
  - PRESENTATION, [116](#)
- DDS\_ProductVersion, [583](#)
  - major, [583](#)
  - minor, [583](#)
  - release, [584](#)
  - revision, [584](#)
- DDS\_ProductVersion\_t, [584](#)
  - major, [585](#)
  - minor, [585](#)
  - release, [585](#)
  - revision, [585](#)
- DDS\_PRODUCTVERSION\_UNKNOWN
  - Discovery, [39](#)
- DDS\_Property\_t, [585](#)
  - name, [586](#)
  - propagate, [586](#)
  - value, [586](#)
- DDS\_PropertyQosPolicy, [586](#)
  - value, [587](#)
- DDS\_PropertySeq, [587](#)
- DDS\_PROTOCOLVERSION
  - Discovery, [38](#)



- DDS\_ProtocolVersion, 588
  - major, 588
  - minor, 588
- DDS\_PROTOCOLVERSION\_1\_0
  - Discovery, 38
- DDS\_PROTOCOLVERSION\_1\_1
  - Discovery, 38
- DDS\_PROTOCOLVERSION\_1\_2
  - Discovery, 38
- DDS\_PROTOCOLVERSION\_2\_0
  - Discovery, 38
- DDS\_PROTOCOLVERSION\_2\_1
  - Discovery, 38
- DDS\_ProtocolVersion\_t
  - Discovery, 39
- DDS\_PSK\_DEFAULT\_PLUGIN\_NAME
  - Lightweight Security API, 227
- dds\_psk\_log.h, 901
  - DDS\_PSK\_TRUST\_EXCEPTION\_EC, 902
- DDS\_PSK\_TRUST\_EXCEPTION\_EC
  - dds\_psk\_log.h, 902
- DDS\_PskServiceFactoryProperty, 588
  - psl\_config, 589
  - psl\_get\_interface\_func, 589
  - service\_name, 589
- DDS\_PskServiceFactoryProperty\_INITIALIZER
  - Lightweight Security API, 226
- DDS\_PUBLICATION\_MATCHED\_STATUS
  - Status Kinds, 96
- DDS\_PUBLICATION\_PRIORITY\_AUTOMATIC
  - PUBLISH\_MODE, 128
- DDS\_PUBLICATION\_PRIORITY\_UNDEFINED
  - PUBLISH\_MODE, 128
- DDS\_PUBLICATION\_TOPIC\_NAME
  - Publication Built-in Topic, 42
- DDS\_PublicationBuiltinTopicData, 589
  - deadline, 591
  - destination\_order, 592
  - durability, 592
  - group\_data, 593
  - key, 591
  - latency\_budget, 592
  - liveliness, 592
  - ownership, 592
  - ownership\_strength, 592
  - participant\_key, 591
  - partition, 593
  - Publication Built-in Topic, 42, 43
  - reliability, 592
  - representation, 593
  - topic\_data, 593
  - topic\_name, 591
  - type\_name, 591
  - unicast\_locator, 593
  - user\_data, 593
- DDS\_PublicationBuiltinTopicData\_finalize
  - Publication Built-in Topic, 43
- DDS\_PublicationBuiltinTopicData\_is\_equal
  - Publication Built-in Topic, 43
- DDS\_PublicationMatchedStatus, 594
  - current\_count, 594
  - current\_count\_change, 595
  - last\_subscription\_handle, 595
  - total\_count, 594
  - total\_count\_change, 594
- DDS\_Publisher
  - Publisher, 64
- DDS\_PUBLISHER\_ENTITY\_KIND
  - Infrastructure Module, 80
- DDS\_PUBLISHER\_QOS\_DEFAULT
  - DomainParticipant, 33
- DDS\_PublisherQos, 595
  - entity\_factory, 596
  - group\_data, 596
  - partition, 596
  - publisher\_name, 596
- DDS\_PublishModeQosPolicy, 596
  - flow\_controller\_name, 598
  - kind, 598
  - priority, 598
- DDS\_PublishModeQosPolicyKind
  - PUBLISH\_MODE, 128
- DDS\_QosPolicyId\_t
  - QoS Policies, 101
- DDS\_READ\_SAMPLE\_STATE
  - Sample States, 74
- DDS\_REJECTED\_BY\_INSTANCES\_LIMIT
  - DataReader, 71
- DDS\_REJECTED\_BY\_REMOTE\_WRITERS\_LIMIT
  - DataReader, 71
- DDS\_REJECTED\_BY\_SAMPLES\_LIMIT
  - DataReader, 71
- DDS\_REJECTED\_BY\_SAMPLES\_PER\_INSTANCE\_LIMIT
  - DataReader, 71
- DDS\_REJECTED\_BY\_SAMPLES\_PER\_REMOTE\_WRITER\_LIMIT
  - DataReader, 71
- DDS\_RELIABILITY\_QOS\_POLICY\_ID
  - QoS Policies, 101
- DDS\_ReliabilityQosPolicy, 599
  - kind, 600
  - max\_blocking\_time, 600
- DDS\_ReliabilityQosPolicyKind
  - RELIABILITY, 109
- DDS\_RELIABLE\_READER\_ACTIVITY\_CHANGED\_STATUS
  - Status Kinds, 97
- DDS\_RELIABLE\_RELIABILITY\_QOS
  - RELIABILITY, 109
- DDS\_ReliableReaderActivityChangedStatus, 601

- active\_count, [601](#)
- active\_count\_change, [602](#)
- inactive\_count, [601](#)
- inactive\_count\_change, [602](#)
- last\_instance\_handle, [602](#)
- DDS\_ReliableReaderActivityChangedStatus\_INITIALIZER
  - DataWriter, [66](#)
- DDS\_ReliableSampleUnacknowledgedStatus, [602](#)
  - instance\_handle, [603](#)
  - sequence\_number, [603](#)
  - unacknowledged\_count, [603](#)
- DDS\_REPLACE\_OLDEST\_INSTANCE\_REPLACEMENT\_QOS
  - DataReader Resource Limits, [120](#)
- DDS\_REQUESTED\_DEADLINE\_MISSED\_STATUS
  - Status Kinds, [93](#)
- DDS\_REQUESTED\_INCOMPATIBLE\_QOS\_STATUS
  - Status Kinds, [93](#)
- DDS\_RequestedDeadlineMissedStatus, [603](#)
  - last\_instance\_handle, [604](#)
  - total\_count, [604](#)
  - total\_count\_change, [604](#)
- DDS\_RequestedIncompatibleQosStatus, [604](#)
  - last\_policy\_id, [605](#)
  - policies, [605](#)
  - total\_count, [605](#)
  - total\_count\_change, [605](#)
- DDS\_ResourceLimitsQosPolicy, [605](#)
  - max\_instances, [607](#)
  - max\_samples, [607](#)
  - max\_samples\_per\_instance, [607](#)
- DDS\_RETCODE\_ALREADY\_DELETED
  - Return Codes, [88](#)
- DDS\_RETCODE\_BAD\_PARAMETER
  - Return Codes, [88](#)
- DDS\_RETCODE\_ERROR
  - Return Codes, [88](#)
- DDS\_RETCODE\_ILLEGAL\_OPERATION
  - Return Codes, [88](#)
- DDS\_RETCODE\_IMMUTABLE\_POLICY
  - Return Codes, [88](#)
- DDS\_RETCODE\_INCONSISTENT\_POLICY
  - Return Codes, [88](#)
- DDS\_RETCODE\_NO\_DATA
  - Return Codes, [88](#)
- DDS\_RETCODE\_NOT\_ALLOWED\_BY\_SEC
  - Return Codes, [88](#)
- DDS\_RETCODE\_NOT\_ENABLED
  - Return Codes, [88](#)
- DDS\_RETCODE\_OK
  - Return Codes, [88](#)
- DDS\_RETCODE\_OUT\_OF\_RESOURCES
  - Return Codes, [88](#)
- DDS\_RETCODE\_PRECONDITION\_NOT\_MET
  - Return Codes, [88](#)
- DDS\_RETCODE\_TIMEOUT
  - Return Codes, [88](#)
- DDS\_RETCODE\_UNSUPPORTED
  - Return Codes, [88](#)
- DDS\_ReturnCode\_t
  - Return Codes, [87](#)
- DDS\_RR\_FLOW\_CONTROLLER\_SCHED\_POLICY
  - Flow Controllers, [59](#)
- DDS\_RTI\_BACKWARDS\_COMPATIBLE\_RTPS\_WELL\_KNOWN\_PORTS
  - WIRE\_PROTOCOL, [124](#)
- DDS\_RtpsReliableReaderProtocol\_t, [608](#)
  - nack\_period, [608](#)
- DDS\_RtpsReliableWriterProtocol\_t, [609](#)
  - heartbeat\_period, [609](#)
  - heartbeats\_per\_max\_samples, [609](#)
  - max\_heartbeat\_retries, [610](#)
  - max\_send\_window, [610](#)
- DDS\_RtpsWellKnownPorts\_t, [611](#)
  - builtin\_multicast\_port\_offset, [614](#)
  - builtin\_unicast\_port\_offset, [614](#)
  - domain\_id\_gain, [612](#)
  - participant\_id\_gain, [613](#)
  - port\_base, [612](#)
  - user\_multicast\_port\_offset, [614](#)
  - user\_unicast\_port\_offset, [614](#)
- DDS\_SAMPLE\_LOST\_BY\_DATAWRITER
  - DataReader, [72](#)
- DDS\_SAMPLE\_LOST\_BY\_HISTORY\_DEPTH\_LIMIT
  - DataReader, [72](#)
- DDS\_SAMPLE\_LOST\_BY\_MANAGEDPOOL\_OVERWRITE
  - DataReader, [72](#)
- DDS\_SAMPLE\_LOST\_BY\_MAX\_SAMPLES\_LIMIT
  - DataReader, [72](#)
- DDS\_SAMPLE\_LOST\_BY\_META\_SAMPLE\_LIMIT
  - DataReader, [72](#)
- DDS\_SAMPLE\_LOST\_BY\_NOT\_READ\_ON\_CACHE\_DELETION
  - DataReader, [72](#)
- DDS\_SAMPLE\_LOST\_NOT\_LOST
  - DataReader, [72](#)
- DDS\_SAMPLE\_LOST\_STATUS
  - Status Kinds, [94](#)
- DDS\_SAMPLE\_REJECTED\_STATUS
  - Status Kinds, [94](#)
- DDS\_SampleIdentity\_t, [615](#)
  - sequence\_number, [615](#)
  - writer\_guid, [615](#)
- DDS\_SampleInfo, [616](#)
  - encapsulation\_id, [618](#)
  - instance\_handle, [617](#)
  - instance\_state, [617](#)
  - publication\_handle, [617](#)
  - publication\_sequence\_number, [618](#)
  - reception\_timestamp, [618](#)
  - sample\_state, [617](#)

- source\_timestamp, 618
  - valid\_data, 618
  - view\_state, 617
- DDS\_SampleInfoSeq, 619
- DDS\_SampleLostStatus, 619
  - reason, 620
  - sample\_info, 620
  - total\_count, 620
  - total\_count\_change, 620
- DDS\_SampleLostStatusKind
  - DataReader, 71
- DDS\_SampleRejectedStatus, 620
  - last\_instance\_handle, 621
  - last\_reason, 621
  - total\_count, 621
  - total\_count\_change, 621
- DDS\_SampleRejectedStatusKind
  - DataReader, 71
- DDS\_SampleStateKind
  - Sample States, 74
- DDS\_SampleStateMask
  - Sample States, 74
- DDS\_SecTransform\_Configuration, 621
  - err\_listener, 622
  - provider\_name, 622
- DDS\_SequenceNumber\_compare
  - Sequence Number Support, 86
- DDS\_SequenceNumber\_t, 622
  - high, 623
  - low, 623
- DDS\_SHARED\_OWNERSHIP\_QOS
  - OWNERSHIP, 106
- DDS\_Short
  - DDS-Specific Primitive Types, 49
- DDS\_SIZE\_AUTO
  - RESOURCE\_LIMITS, 115
- DDS\_SQL\_CONTENT\_FILTER\_CLASS
  - Content Filter Plugin, 30
- DDS\_STATUS\_MASK\_ALL
  - Status Kinds, 91
- DDS\_STATUS\_MASK\_NONE
  - Status Kinds, 91
- DDS\_StatusKind
  - Status Kinds, 92
- DDS\_StatusMask
  - Status Kinds, 91
- DDS\_String
  - DDS-Specific Primitive Types, 52
- DDS\_String\_alloc
  - String Support, 137
- DDS\_String\_cmp
  - String Support, 138
- DDS\_String\_dup
  - String Support, 137
- DDS\_String\_free
  - String Support, 137
- DDS\_String\_length
  - String Support, 138
- DDS\_String\_ncmp
  - String Support, 138
- DDS\_SUBSCRIBER\_ENTITY\_KIND
  - Infrastructure Module, 80
- DDS\_SUBSCRIBER\_QOS\_DEFAULT
  - DomainParticipant, 33
- DDS\_SubscriberQos, 623
  - entity\_factory, 624
  - group\_data, 624
  - partition, 624
  - subscriber\_name, 624
- DDS\_SUBSCRIPTION\_MATCHED\_STATUS
  - Status Kinds, 96
- DDS\_SUBSCRIPTION\_TOPIC\_NAME
  - Subscription Built-in Topic, 44
- DDS\_SubscriptionBuiltinTopicData, 624
  - content\_filter, 629
  - deadline, 627
  - destination\_order, 628
  - durability, 627
  - group\_data, 629
  - key, 626
  - latency\_budget, 627
  - liveliness, 627
  - multicast\_locator, 628
  - ownership, 627
  - participant\_key, 626
  - partition, 628
  - presentation, 628
  - reliability, 627
  - representation, 628
  - Subscription Built-in Topic, 44, 45
  - topic\_data, 629
  - topic\_name, 626
  - type\_name, 626
  - unicast\_locator, 628
  - user\_data, 628
- DDS\_SubscriptionBuiltinTopicData\_finalize
  - Subscription Built-in Topic, 45
- DDS\_SubscriptionBuiltinTopicData\_is\_equal
  - Subscription Built-in Topic, 45
- DDS\_SubscriptionMatchedStatus, 629
  - current\_count, 630
  - current\_count\_change, 630
  - last\_publication\_handle, 630
  - total\_count, 630
  - total\_count\_change, 630
- DDS\_SYNCHRONOUS\_PUBLISH\_MODE\_QOS
  - PUBLISH\_MODE, 128
- DDS\_SystemResourceLimitsQosPolicy, 631

- max\_components, [632](#)
- max\_participants, [631](#)
- DDS\_Time\_greater\_than
  - Time Support, [82](#)
- DDS\_TIME\_INVALID
  - Time Support, [83](#)
- DDS\_TIME\_INVALID\_NSEC
  - Time Support, [83](#)
- DDS\_TIME\_INVALID\_SEC
  - Time Support, [83](#)
- DDS\_Time\_is\_zero
  - Time Support, [82](#)
- DDS\_Time\_t, [632](#)
  - nanosec, [633](#)
  - sec, [633](#)
- DDS\_TIME\_ZERO
  - Time Support, [82](#)
- DDS\_TOPIC\_ENTITY\_KIND
  - Infrastructure Module, [80](#)
- DDS\_TOPIC\_PRESENTATION\_QOS
  - Presentation, [116](#)
- DDS\_TOPIC\_QOS\_DEFAULT
  - DomainParticipant, [33](#)
- DDS\_TopicDataQosPolicy, [633](#)
  - value, [634](#)
- DDS\_TopicQos, [634](#)
  - topic\_data, [634](#)
- DDS\_TRANSIENT\_LOCAL\_DURABILITY\_QOS
  - Durability, [111](#)
- DDS\_TransportQosPolicy, [635](#)
  - enabled\_transports, [635](#)
- DDS\_TypePluginI, [636](#)
  - Type Support, [135](#)
- DDS\_TYPESUPPORT\_CPP
  - User Data Type Support, [55](#)
- DDS\_UNKNOWN\_ENTITY\_KIND
  - Infrastructure Module, [80](#)
- DDS\_UnsignedLong
  - DDS-Specific Primitive Types, [49](#)
- DDS\_UnsignedLongLong
  - DDS-Specific Primitive Types, [50](#)
- DDS\_UnsignedShort
  - DDS-Specific Primitive Types, [49](#)
- DDS\_UserDataQosPolicy, [636](#)
  - value, [637](#)
- DDS\_UserTrafficQosPolicy, [637](#)
  - enabled\_transports, [637](#)
- DDS\_VENDOR\_ID\_LENGTH\_MAX
  - Discovery, [38](#)
- DDS\_VendorId, [638](#)
  - vendorId, [638](#)
- DDS\_VendorId\_t, [638](#)
  - vendorId, [639](#)
- DDS\_ViewStateKind
  - View States, [76](#)
- DDS\_ViewStateMask
  - View States, [75](#)
- DDS\_VOLATILE\_DURABILITY\_QOS
  - Durability, [111](#)
- DDS\_Wchar
  - DDS-Specific Primitive Types, [48](#)
- DDS\_WireProtocolQosPolicy, [639](#)
  - allowed\_crc\_mask, [643](#)
  - check\_crc, [642](#)
  - compute\_crc, [641](#)
  - computed\_crc\_kind, [642](#)
  - participant\_id, [640](#)
  - require\_crc, [642](#)
  - rtps\_app\_id, [641](#)
  - rtps\_host\_id, [640](#)
  - rtps\_instance\_id, [641](#)
  - rtps\_well\_known\_ports, [641](#)
- DDS\_WRITEPARAMS\_DEFAULT
  - Infrastructure Module, [80](#)
- DDS\_WriteParams\_t, [643](#)
  - handle, [644](#)
  - source\_timestamp, [644](#)
- DDS\_Wstring
  - DDS-Specific Primitive Types, [52](#)
- DDS\_XCDR2\_DATA\_REPRESENTATION
  - DATA REPRESENTATION, [113](#)
- DDS\_XCDR\_DATA\_REPRESENTATION
  - DATA REPRESENTATION, [112](#)
- DDS\_XML\_DATA\_REPRESENTATION
  - DATA REPRESENTATION, [112](#)
- DDS\_XTYPES\_COMPLIANCE\_MASK\_PROPERTY
  - Property Reference, [140](#)
- DDSC\_LOG\_ADDRESSRESOLVER\_OBJECT
  - DDS\_C, [377](#)
- DDSC\_LOG\_APPGEN\_COMPONENT
  - DDS\_C, [387](#)
- DDSC\_LOG\_APPLY\_READER\_FILTER\_EC
  - DDS\_C, [446](#)
- DDSC\_LOG\_AUTHPOOL\_OBJECT
  - DDS\_C, [380](#)
- DDSC\_LOG\_BASE
  - osapi\_log.h, [931](#)
- DDSC\_LOG\_BIND\_RECORD
  - DDS\_C, [372](#)
- DDSC\_LOG\_BINDRESOLVER\_OBJECT
  - DDS\_C, [376](#)
- DDSC\_LOG\_BUFFER\_MAX\_EXCEEDED\_EC
  - DDS\_C, [444](#)
- DDSC\_LOG\_BUFFER\_OBJECT
  - DDS\_C, [381](#)
- DDSC\_LOG\_CDR\_BUFFER\_SET\_EC
  - DDS\_C, [399](#)
- DDSC\_LOG\_CDR\_DESERIALIZE\_DATA\_EC

DDS\_C, [401](#)  
 DDSC\_LOG\_CDR\_DESERIALIZE\_HEADER\_EC  
     DDS\_C, [401](#)  
 DDSC\_LOG\_CDR\_DESERIALIZE\_KEY\_EC  
     DDS\_C, [402](#)  
 DDSC\_LOG\_CDR\_DESERIALIZE\_KEYHASH\_EC  
     DDS\_C, [401](#)  
 DDSC\_LOG\_CDR\_DESERIALIZE\_PID\_EC  
     DDS\_C, [400](#)  
 DDSC\_LOG\_CDR\_DESERIALIZE\_PID\_LENGTH\_EC  
     DDS\_C, [400](#)  
 DDSC\_LOG\_CDR\_FINALIZE\_SAMPLE\_EC  
     DDS\_C, [401](#)  
 DDSC\_LOG\_CDR\_INCREMENT\_POS\_EC  
     DDS\_C, [400](#)  
 DDSC\_LOG\_CDR\_INITIALIZE\_SAMPLE\_EC  
     DDS\_C, [401](#)  
 DDSC\_LOG\_CDR\_POOL\_ALLOC\_EC  
     DDS\_C, [399](#)  
 DDSC\_LOG\_CDR\_POOL\_DELETE\_EC  
     DDS\_C, [399](#)  
 DDSC\_LOG\_CDR\_SERIALIZE\_DATA\_EC  
     DDS\_C, [400](#)  
 DDSC\_LOG\_CDR\_SERIALIZE\_KEYHASH\_EC  
     DDS\_C, [400](#)  
 DDSC\_LOG\_CDR\_SERIALIZE\_PID\_EC  
     DDS\_C, [399](#)  
 DDSC\_LOG\_CDR\_SERIALIZE\_PID\_LENGTH\_EC  
     DDS\_C, [400](#)  
 DDSC\_LOG\_CDR\_SERIALIZE\_STATUS\_INFO\_EC  
     DDS\_C, [401](#)  
 DDSC\_LOG\_CDR\_SET\_OFFSET\_EC  
     DDS\_C, [416](#)  
 DDSC\_LOG\_CDR\_SET\_POS\_EC  
     DDS\_C, [401](#)  
 DDSC\_LOG\_CHANGE\_CONTENT\_FILTER\_EC  
     DDS\_C, [447](#)  
 DDSC\_LOG\_CHECKSUM\_INCOMPATIBLE\_EC  
     DDS\_C, [445](#)  
 DDSC\_LOG\_CHECKSUM\_INCONSISTENT\_ALLOW\_EC  
     DDS\_C, [445](#)  
 DDSC\_LOG\_CHECKSUM\_INCONSISTENT\_CLASS\_EC  
     DDS\_C, [445](#)  
 DDSC\_LOG\_CHECKSUM\_INCONSISTENT\_COMPUTE\_EC  
     DDS\_C, [445](#)  
 DDSC\_LOG\_CHECKSUM\_REQUIRED\_EC  
     DDS\_C, [444](#)  
 DDSC\_LOG\_COMPILE\_CONTENT\_FILTER\_EC  
     DDS\_C, [446](#)  
 DDSC\_LOG\_COMPONENT\_CREATE\_EC  
     DDS\_C, [387](#)  
 DDSC\_LOG\_COMPONENT\_DELETE\_EC  
     DDS\_C, [387](#)  
 DDSC\_LOG\_COMPONENT\_LOOKUP\_EC  
     DDS\_C, [387](#)  
 DDSC\_LOG\_CONDITION\_OBJECT  
     DDS\_C, [378](#)  
 DDSC\_LOG\_CONDREF\_OBJECT  
     DDS\_C, [378](#)  
 DDSC\_LOG\_CONTENT\_FILTER\_QOS\_POLICY  
     DDS\_C, [393](#)  
 DDSC\_LOG\_CONTENT\_FILTERING\_NOT\_ENABLED\_EC  
     DDS\_C, [446](#)  
 DDSC\_LOG\_COOKIE\_PAYLOAD\_SEQUENCE  
     DDS\_C, [384](#)  
 DDSC\_LOG\_CREATE\_FILTER\_PLUGIN\_EC  
     DDS\_C, [447](#)  
 DDSC\_LOG\_CREATE\_INDEX\_EC  
     DDS\_C, [370](#)  
 DDSC\_LOG\_CREATE\_WRITER\_FILTER\_EC  
     DDS\_C, [446](#)  
 DDSC\_LOG\_DATA\_HOLDER\_SEQ\_OBJECT  
     DDS\_C, [380](#)  
 DDSC\_LOG\_DATA\_REPRESENTATION\_QOS\_POLICY  
     DDS\_C, [393](#)  
 DDSC\_LOG\_DATABASE\_CREATE\_EC  
     DDS\_C, [369](#)  
 DDSC\_LOG\_DATABASE\_DELETE\_EC  
     DDS\_C, [369](#)  
 DDSC\_LOG\_DATAHOLDER\_SEQUENCE  
     DDS\_C, [384](#)  
 DDSC\_LOG\_DATAREADER\_ADD\_PEER\_FAILED\_EC  
     DDS\_C, [418](#)  
 DDSC\_LOG\_DATAREADER\_CDR  
     DDS\_C, [399](#)  
 DDSC\_LOG\_DATAREADER\_ENTITY  
     DDS\_C, [397](#)  
 DDSC\_LOG\_DATAREADER\_LISTENER  
     DDS\_C, [395](#)  
 DDSC\_LOG\_DATAREADER\_NETIO\_KIND  
     DDS\_C, [403](#)  
 DDSC\_LOG\_DATAREADER\_QOS  
     DDS\_C, [388](#)  
 DDSC\_LOG\_DATAREADER\_RECORD  
     DDS\_C, [371](#)  
 DDSC\_LOG\_DATAREADER\_RESOURCE\_QOS\_POLICY  
     DDS\_C, [392](#)  
 DDSC\_LOG\_DATAREADERIO\_COMPONENT  
     DDS\_C, [386](#)  
 DDSC\_LOG\_DATAREADERIO\_OBJECT  
     DDS\_C, [377](#)  
 DDSC\_LOG\_DATAREADERQOS\_OBJECT  
     DDS\_C, [375](#)  
 DDSC\_LOG\_DATAWRITER\_ADD\_PEER\_FAILED\_EC  
     DDS\_C, [418](#)  
 DDSC\_LOG\_DATAWRITER\_CDR  
     DDS\_C, [399](#)  
 DDSC\_LOG\_DATAWRITER\_ENTITY

- DDS\_C, [397](#)
- DDSC\_LOG\_DATAWRITER\_INLINE
  - DDS\_C, [399](#)
- DDSC\_LOG\_DATAWRITER\_LISTENER
  - DDS\_C, [395](#)
- DDSC\_LOG\_DATAWRITER\_NETIO\_KIND
  - DDS\_C, [403](#)
- DDSC\_LOG\_DATAWRITER\_QOS
  - DDS\_C, [389](#)
- DDSC\_LOG\_DATAWRITER\_RECORD
  - DDS\_C, [371](#)
- DDSC\_LOG\_DATAWRITER\_RESOURCE\_QOS\_POLICY
  - DDS\_C, [392](#)
- DDSC\_LOG\_DATAWRITERIO\_COMPONENT
  - DDS\_C, [386](#)
- DDSC\_LOG\_DATAWRITERIO\_OBJECT
  - DDS\_C, [377](#)
- DDSC\_LOG\_DATAWRITERQOS\_OBJECT
  - DDS\_C, [375](#)
- DDSC\_LOG\_DB\_CURSOR\_INVALIDATED\_EC
  - DDS\_C, [370](#)
- DDSC\_LOG\_DEADLINE\_QOS\_POLICY
  - DDS\_C, [390](#)
- DDSC\_LOG\_DEFAULTDATAREADER\_QOS
  - DDS\_C, [389](#)
- DDSC\_LOG\_DEFAULTDATAWRITER\_QOS
  - DDS\_C, [390](#)
- DDSC\_LOG\_DEFAULTPARTICIPANT\_QOS
  - DDS\_C, [390](#)
- DDSC\_LOG\_DEFAULTPUBLISHER\_QOS
  - DDS\_C, [390](#)
- DDSC\_LOG\_DEFAULTSUBSCRIBER\_QOS
  - DDS\_C, [390](#)
- DDSC\_LOG\_DEFAULTTOPIC\_QOS
  - DDS\_C, [389](#)
- DDSC\_LOG\_DELETE\_INDEX\_EC
  - DDS\_C, [370](#)
- DDSC\_LOG\_DESERIALIZE\_BAD\_PID\_LENGTH\_EC
  - DDS\_C, [400](#)
- DDSC\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC
  - DDS\_C, [416](#)
- DDSC\_LOG\_DESERIALIZE\_CONTENT\_FILTER\_INFO\_EC
  - DDS\_C, [447](#)
- DDSC\_LOG\_DESERIALIZE\_LOCATOR\_EC
  - DDS\_C, [417](#)
- DDSC\_LOG\_DESERIALIZE\_PARTITION\_SEQ\_EC
  - DDS\_C, [418](#)
- DDSC\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC
  - DDS\_C, [417](#)
- DDSC\_LOG\_DESERIALIZE\_REMOTE\_CONTENT\_FILTER\_EC
  - DDS\_C, [447](#)
- DDSC\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC
  - DDS\_C, [415](#)
- DDSC\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC
  - DDS\_C, [417](#)
- DDSC\_LOG\_DESTINATION\_ORDER\_POLICY
  - DDS\_C, [392](#)
- DDSC\_LOG\_DESTINATION\_SEQUENCE
  - DDS\_C, [383](#)
- DDSC\_LOG\_DISC\_ADD\_PEER\_EC
  - DDS\_C, [415](#)
- DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAREADER\_DELETED\_EC
  - DDS\_C, [415](#)
- DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAREADER\_ENABLED\_EC
  - DDS\_C, [414](#)
- DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAWRITER\_DELETED\_EC
  - DDS\_C, [415](#)
- DDSC\_LOG\_DISC\_AFTER\_LOCAL\_DATAWRITER\_ENABLED\_EC
  - DDS\_C, [415](#)
- DDSC\_LOG\_DISC\_AFTER\_LOCAL\_PARTICIPANT\_CREATED\_EC
  - DDS\_C, [414](#)
- DDSC\_LOG\_DISC\_BEFORE\_LOCAL\_PARTICIPANT\_CREATED\_EC
  - DDS\_C, [414](#)
- DDSC\_LOG\_DISC\_BEFORE\_REMOTE\_PARTICIPANT\_DELETED\_EC
  - DDS\_C, [415](#)
- DDSC\_LOG\_DISC\_LOCAL\_PARTICIPANT\_ENABLED\_EC
  - DDS\_C, [414](#)
- DDSC\_LOG\_DISC\_REMOTE\_PARTICIPANT\_REMOVE\_EC
  - DDS\_C, [418](#)
- DDSC\_LOG\_DISCOVERY\_COMPONENT
  - DDS\_C, [386](#)
- DDSC\_LOG\_DR\_COMMIT\_ENTRY\_EC
  - DDS\_C, [408](#)
- DDSC\_LOG\_DR\_COMMIT\_SAMPLE\_EC
  - DDS\_C, [408](#)
- DDSC\_LOG\_DR\_COPY\_DATA\_SAMPLE\_EC
  - DDS\_C, [408](#)
- DDSC\_LOG\_DR\_CREATE\_TYPED\_READER\_EC
  - DDS\_C, [407](#)
- DDSC\_LOG\_DR\_DESERIALIZE\_KEYHASH\_EC
  - DDS\_C, [408](#)
- DDSC\_LOG\_DR\_DISPOSE\_KEY\_EC
  - DDS\_C, [409](#)
- DDSC\_LOG\_DR\_FILTER\_ERROR\_EC
  - DDS\_C, [408](#)
- DDSC\_LOG\_DR\_GET\_ENTRY\_FAILED\_EC
  - DDS\_C, [408](#)
- DDSC\_LOG\_DR\_INSTANCE\_ASSERTION\_FAILED\_EC
  - DDS\_C, [409](#)
- DDSC\_LOG\_DR\_INSTANCE\_MAPPING\_EXHAUSTED\_EC
  - DDS\_C, [409](#)
- DDSC\_LOG\_DR\_INSTANCE\_TO\_KEYHASH\_EC
  - DDS\_C, [409](#)
- DDSC\_LOG\_DR\_ON\_SAMPLE\_REMOVED\_EC
  - DDS\_C, [409](#)
- DDSC\_LOG\_DR\_READ\_TAKE\_FAILURE\_EC
  - DDS\_C, [409](#)
- DDSC\_LOG\_DR\_UNREGISTER\_KEY\_EC
  - DDS\_C, [409](#)

|                                               |                                                  |
|-----------------------------------------------|--------------------------------------------------|
| DDS_C, <a href="#">408</a>                    | DDS_C, <a href="#">398</a>                       |
| DDSC_LOG_DUPLICATE_GUID_PREFIX_EC             | DDSC_LOG_ENTITY_OBJECT                           |
| DDS_C, <a href="#">412</a>                    | DDS_C, <a href="#">376</a>                       |
| DDSC_LOG_DUPLICATE_PARTICIPANT_NAME_EC        | DDSC_LOG_EVALUATE_CONTENT_FILTER_EC              |
| DDS_C, <a href="#">413</a>                    | DDS_C, <a href="#">447</a>                       |
| DDSC_LOG_DUPLICATE_REMOTE_PARTICIPANT_NAME_EC | DDSC_LOG_EVALUATE_WRITER_FILTER_EC               |
| DDS_C, <a href="#">413</a>                    | DDS_C, <a href="#">446</a>                       |
| DDSC_LOG_DURABILITY_QOS_POLICY                | DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC       |
| DDS_C, <a href="#">391</a>                    | DDS_C, <a href="#">366</a>                       |
| DDSC_LOG_DW_ACKNACK_FAILED_EC                 | DDSC_LOG_FILTER_PLUGIN_FACTORY_NOT_FOUND_EC      |
| DDS_C, <a href="#">406</a>                    | DDS_C, <a href="#">446</a>                       |
| DDSC_LOG_DW_COMMIT_EC                         | DDSC_LOG_FINALIZE_FILTER_PLUGIN_EC               |
| DDS_C, <a href="#">406</a>                    | DDS_C, <a href="#">447</a>                       |
| DDSC_LOG_DW_CREATE_TYPED_WRITER_EC            | DDSC_LOG_FINALIZE_TRUST_FAILED_EC                |
| DDS_C, <a href="#">406</a>                    | DDS_C, <a href="#">434</a>                       |
| DDSC_LOG_DW_FLOW_CONTROLLER_REQUIRED_EC       | DDSC_LOG_FIND_PUBLICATION_PARENT_EC              |
| DDS_C, <a href="#">407</a>                    | DDS_C, <a href="#">365</a>                       |
| DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC           | DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC             |
| DDS_C, <a href="#">406</a>                    | DDS_C, <a href="#">366</a>                       |
| DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC               | DDSC_LOG_FLOW_CONTROLLER_CREATE_EC               |
| DDS_C, <a href="#">406</a>                    | DDS_C, <a href="#">443</a>                       |
| DDSC_LOG_DW_INCOMPATIBLE_PADDING_EC           | DDSC_LOG_FLOW_CONTROLLER_DELETE_EC               |
| DDS_C, <a href="#">407</a>                    | DDS_C, <a href="#">443</a>                       |
| DDSC_LOG_DW_INSTANCE_ASSERTION_FAILED_EC      | DDSC_LOG_FLOWCONTROLLER_INCONSISTENT_PROP_EC     |
| DDS_C, <a href="#">407</a>                    | DDS_C, <a href="#">445</a>                       |
| DDSC_LOG_DW_KEYHASH_CREATE_EC                 | DDSC_LOG_GET_MTU_EC                              |
| DDS_C, <a href="#">406</a>                    | DDS_C, <a href="#">368</a>                       |
| DDSC_LOG_DW_UNKNOWN_FLOW_CONTROLLER_EC        | DDSC_LOG_GET_MTU_NO_ROUTES_EC                    |
| DDS_C, <a href="#">406</a>                    | DDS_C, <a href="#">368</a>                       |
| DDSC_LOG_ENABLE_WRITER_FILTERING_EC           | DDSC_LOG_GET_NEXT_OBJECT_ID_EC                   |
| DDS_C, <a href="#">446</a>                    | DDS_C, <a href="#">363</a>                       |
| DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE | DDSC_LOG_GET_SERIALIZED_KEY_SIZE_EC              |
| DDS_C, <a href="#">383</a>                    | DDS_C, <a href="#">368</a>                       |
| DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE           | DDSC_LOG_HEARTBEATS_QOS_POLICY                   |
| DDS_C, <a href="#">383</a>                    | DDS_C, <a href="#">392</a>                       |
| DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCED     | DDSC_LOG_HISTORY_QOS_POLICY                      |
| DDS_C, <a href="#">383</a>                    | DDS_C, <a href="#">390</a>                       |
| DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC | DDSC_LOG_HISTORY_RESOURCE                        |
| DDS_C, <a href="#">364</a>                    | DDS_C, <a href="#">387</a>                       |
| DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC          | DDSC_LOG_ILLEGAL_OBJECTID_EC                     |
| DDS_C, <a href="#">398</a>                    | DDS_C, <a href="#">366</a>                       |
| DDSC_LOG_ENTITY_ENABLE_EC                     | DDSC_LOG_INCOMPATIBLE_PARTITION_QOS_EC           |
| DDS_C, <a href="#">398</a>                    | DDS_C, <a href="#">444</a>                       |
| DDSC_LOG_ENTITY_FINALIZE_EC                   | DDSC_LOG_INCOMPATIBLE_PRESENTATION_QOS_EC        |
| DDS_C, <a href="#">398</a>                    | DDS_C, <a href="#">444</a>                       |
| DDSC_LOG_ENTITY_INITIALIZE_EC                 | DDSC_LOG_INITIAL_PEER_SEQUENCE                   |
| DDS_C, <a href="#">398</a>                    | DDS_C, <a href="#">383</a>                       |
| DDSC_LOG_ENTITY_INVALID_PROPERTY_EC           | DDSC_LOG_INITIALIZE_DISCOVERY_QUEUE_EC           |
| DDS_C, <a href="#">398</a>                    | DDS_C, <a href="#">418</a>                       |
| DDSC_LOG_ENTITY_NAME_TOO_LONG_EC              | DDSC_LOG_INTERPRETER_SUPPORT_UNEXPECTED_ERROR_EC |
| DDS_C, <a href="#">442</a>                    | DDS_C, <a href="#">443</a>                       |
| DDSC_LOG_ENTITY_NOT_EMPTY_EC                  | DDSC_LOG_INTRA_NETIO_KIND                        |
| DDS_C, <a href="#">398</a>                    | DDS_C, <a href="#">402</a>                       |
| DDSC_LOG_ENTITY_NOT_ENABLED_EC                | DDSC_LOG_INVALID_BUFFER_LENGTH_EC                |



DDS\_C, [444](#)  
 DDSC\_LOG\_INVALID\_DOMAINID\_EC  
     DDS\_C, [367](#)  
 DDSC\_LOG\_INVALID\_DURATION\_EC  
     DDS\_C, [363](#)  
 DDSC\_LOG\_INVALID\_ENDPOINT\_GUID\_EC  
     DDS\_C, [364](#)  
 DDSC\_LOG\_INVALID\_PARTICIPANT\_GUID\_PREFIX\_EC  
     DDS\_C, [363](#)  
 DDSC\_LOG\_INVALID\_PARTICIPANT\_NAME\_EC  
     DDS\_C, [363](#)  
 DDSC\_LOG\_IO\_SNPRINTF\_FAILED\_EC  
     DDS\_C, [364](#)  
 DDSC\_LOG\_IPC\_ALLOC\_FAILED\_EC  
     DDS\_C, [419](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_ADD\_ANON\_ROUTE\_EC  
     DDS\_C, [420](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_ADD\_PEER\_EC  
     DDS\_C, [421](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_CONFIG\_FINALIZE\_FAILED\_EC  
     DDS\_C, [422](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_CONFIG\_INITIALIZE\_FAILED\_EC  
     DDS\_C, [422](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_CONFIG\_SET\_FAILED\_EC  
     DDS\_C, [422](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_CREATE\_FAILED\_EC  
     DDS\_C, [423](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_DELETE\_ANON\_ROUTE\_EC  
     DDS\_C, [421](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_ENABLE\_FAILED\_EC  
     DDS\_C, [423](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_AUTH\_HANDSHAKE\_FAILED\_EC  
     DDS\_C, [422](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_AUTH\_REQUEST\_MISMATCHED\_DW\_RECORD  
     DDS\_C, [422](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_ENDPOINT\_INTERCHANGE\_STATE\_REACHED\_EC  
     DDS\_C, [422](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_FAILED\_PROCESS\_PARTICIPANT\_INTERCHANGE\_STATE\_FAILED\_EC  
     DDS\_C, [422](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_PROCESS\_SAMPLE\_EC  
     DDS\_C, [421](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_READER\_CREATE\_EC  
     DDS\_C, [420](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_REMOVE\_PEER\_EC  
     DDS\_C, [421](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_RETURN\_LOAN\_EC  
     DDS\_C, [421](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_SET\_LISTENERS\_FAILED\_EC  
     DDS\_C, [423](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_TOPIC\_CREATE\_EC  
     DDS\_C, [420](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_TYPE\_PLUGIN\_EC  
     DDS\_C, [420](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_TYPE\_REGISTER\_EC  
     DDS\_C, [419](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_UNKNOWN\_EC  
     DDS\_C, [420](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_WRITE\_SAMPLE\_EC  
     DDS\_C, [421](#)  
 DDSC\_LOG\_IPC\_CHANNEL\_WRITER\_CREATE\_EC  
     DDS\_C, [420](#)  
 DDSC\_LOG\_IPC\_FINALIZE\_FAILED\_EC  
     DDS\_C, [419](#)  
 DDSC\_LOG\_IPC\_INIT\_FAILED\_EC  
     DDS\_C, [419](#)  
 DDSC\_LOG\_IPC\_UPDATE\_QOS\_FAILED\_EC  
     DDS\_C, [420](#)  
 DDSC\_LOG\_LATENCY\_BUDGET\_QOS\_POLICY  
     DDS\_C, [393](#)  
 DDSC\_LOG\_LISTENER\_GET\_EC  
     DDS\_C, [396](#)  
 DDSC\_LOG\_LISTENER\_INCONSISTENT\_EC  
     DDS\_C, [396](#)  
 DDSC\_LOG\_LISTENER\_SET\_EC  
     DDS\_C, [396](#)  
 DDSC\_LOG\_LISTENER\_SET\_ILLEGAL\_NULL\_EC  
     DDS\_C, [396](#)  
 DDSC\_LOG\_LIVELINESS\_QOS\_POLICY  
     DDS\_C, [390](#)  
 DDSC\_LOG\_LOCATORS\_FULL\_EC  
     DDS\_C, [417](#)  
 DDSC\_LOG\_LOCK\_EC  
     DDS\_C, [445](#)  
 DDSC\_LOG\_LOG\_OBJECT  
     DDS\_C, [377](#)  
 DDSC\_LOG\_METADATA\_PLUGIN\_EC  
     DDS\_C, [411](#)  
 DDSC\_LOG\_MATCHED\_DW\_RECORD  
     DDS\_C, [372](#)  
 DDSC\_LOG\_MAX\_PARTICIPANTS\_REACHED\_EC  
     DDS\_C, [366](#)  
 DDSC\_LOG\_METADATA\_STATE\_FAILED\_EC  
     DDS\_C, [378](#)  
 DDSC\_LOG\_MEMORY\_MANAGER\_DELETE\_EC  
     DDS\_C, [444](#)  
 DDSC\_LOG\_MEMORY\_MANAGER\_NOT\_OWNER\_EC  
     DDS\_C, [444](#)  
 DDSC\_LOG\_MEMORY\_MANAGER\_OUT\_OF\_RESOURCES\_EC  
     DDS\_C, [443](#)  
 DDSC\_LOG\_MEMPOOL\_OBJECT  
     DDS\_C, [379](#)  
 DDSC\_LOG\_MESSAGE\_DATA\_UNKNOWN\_LIVELINESS\_KIND\_EC  
     DDS\_C, [442](#)  
 DDSC\_LOG\_MESSAGE\_DATA\_WRONG\_LIVELINESS\_KIND\_EC  
     DDS\_C, [442](#)  
 DDSC\_LOG\_METAMULTICAST\_SEQUENCE  
     DDS\_C, [384](#)  
 DDSC\_LOG\_METAUNICAST\_SEQUENCE



DDS\_C, [384](#)  
 DDSC\_LOG\_MIN\_DISCOVERY\_MTU\_EC  
     DDS\_C, [368](#)  
 DDSC\_LOG\_MIN\_MTU\_SIZE\_EC  
     DDS\_C, [368](#)  
 DDSC\_LOG\_MUTEX\_OBJECT  
     DDS\_C, [377](#)  
 DDSC\_LOG\_NAME\_LOOKUP\_EC  
     DDS\_C, [441](#)  
 DDSC\_LOG\_NETIO\_ADD\_ANON\_TOPIC\_ROUTE\_EC  
     DDS\_C, [402](#)  
 DDSC\_LOG\_NETIO\_ADD\_ROUTE\_EC  
     DDS\_C, [404](#)  
 DDSC\_LOG\_NETIO\_ADD\_TOPIC\_ROUTE\_EC  
     DDS\_C, [402](#)  
 DDSC\_LOG\_NETIO\_BIND\_EC  
     DDS\_C, [403](#)  
 DDSC\_LOG\_NETIO\_BIND\_EXTERNAL\_EC  
     DDS\_C, [403](#)  
 DDSC\_LOG\_NETIO\_COMPONENT  
     DDS\_C, [387](#)  
 DDSC\_LOG\_NETIO\_DELETE\_ROUTE\_EC  
     DDS\_C, [404](#)  
 DDSC\_LOG\_NETIO\_DELETE\_TOPIC\_ROUTE\_EC  
     DDS\_C, [402](#)  
 DDSC\_LOG\_NETIO\_DISCOVERY\_ENABLED\_TRANSPORT\_EC  
     DDS\_C, [405](#)  
 DDSC\_LOG\_NETIO\_FORCED\_REMOVE\_EC  
     DDS\_C, [405](#)  
 DDSC\_LOG\_NETIO\_FORWARD\_TOPIC\_EC  
     DDS\_C, [402](#)  
 DDSC\_LOG\_NETIO\_GET\_EXTERNAL\_INTF\_EC  
     DDS\_C, [404](#)  
 DDSC\_LOG\_NETIO\_GET\_ROUTE\_TABLE\_FAILED\_EC  
     DDS\_C, [404](#)  
 DDSC\_LOG\_NETIO\_NETIO\_KIND  
     DDS\_C, [402](#)  
 DDSC\_LOG\_NETIO\_NO\_ROUTE\_EC  
     DDS\_C, [404](#)  
 DDSC\_LOG\_NETIO\_PEER\_LOOKUP\_EC  
     DDS\_C, [405](#)  
 DDSC\_LOG\_NETIO\_ROUTE\_LOOKUP\_FAILED\_EC  
     DDS\_C, [404](#)  
 DDSC\_LOG\_NETIO\_SEND\_FAILED\_EC  
     DDS\_C, [404](#)  
 DDSC\_LOG\_NETIO\_SET\_STATE\_EC  
     DDS\_C, [405](#)  
 DDSC\_LOG\_NETIO\_UNBIND\_EC  
     DDS\_C, [403](#)  
 DDSC\_LOG\_NETIO\_UNBIND\_EXTERNAL\_EC  
     DDS\_C, [403](#)  
 DDSC\_LOG\_NETMASK\_SEQUENCE  
     DDS\_C, [382](#)  
 DDSC\_LOG\_OBJECT\_ALLOCATE\_EC  
     DDS\_C, [381](#)  
 DDSC\_LOG\_OBJECT\_COPY\_EC  
     DDS\_C, [381](#)  
 DDSC\_LOG\_OBJECT\_CREATE\_EC  
     DDS\_C, [382](#)  
 DDSC\_LOG\_OBJECT\_DELETE\_EC  
     DDS\_C, [381](#)  
 DDSC\_LOG\_OBJECT\_EMPTY\_EC  
     DDS\_C, [382](#)  
 DDSC\_LOG\_OBJECT\_FINALIZE\_EC  
     DDS\_C, [381](#)  
 DDSC\_LOG\_OBJECT\_GET\_PROPERTY\_EC  
     DDS\_C, [382](#)  
 DDSC\_LOG\_OBJECT\_INITIALIZE\_EC  
     DDS\_C, [381](#)  
 DDSC\_LOG\_OBJECT\_INUSECOUNT\_EC  
     DDS\_C, [382](#)  
 DDSC\_LOG\_OBJECT\_REFCOUNT\_EC  
     DDS\_C, [382](#)  
 DDSC\_LOG\_OBJECT\_SET\_PROPERTY\_EC  
     DDS\_C, [382](#)  
 DDSC\_LOG\_ON\_TYPE\_REGISTERED\_FAILURE\_EC  
     DDS\_C, [368](#)  
 DDSC\_LOG\_ON\_TYPE\_UNREGISTERED\_FAILURE\_EC  
     DDS\_C, [368](#)  
 DDSC\_LOG\_OWNERSHIP\_QOS\_POLICY  
     DDS\_C, [391](#)  
 DDSC\_LOG\_OWNERSHIP\_STRENGTH\_QOS\_POLICY  
     DDS\_C, [391](#)  
 DDSC\_LOG\_PACKET\_INIT\_EC  
     DDS\_C, [405](#)  
 DDSC\_LOG\_PACKET\_SET\_HEAD\_EC  
     DDS\_C, [405](#)  
 DDSC\_LOG\_PACKET\_SET\_TAIL\_EC  
     DDS\_C, [405](#)  
 DDSC\_LOG\_PARTICIPANT\_BINARY\_PROPERTIES  
     DDS\_C, [379](#)  
 DDSC\_LOG\_PARTICIPANT\_BUILTIN\_PUBLISHER  
     DDS\_C, [380](#)  
 DDSC\_LOG\_PARTICIPANT\_BUILTIN\_SUBSCRIBER  
     DDS\_C, [380](#)  
 DDSC\_LOG\_PARTICIPANT\_DOES\_NOT\_EXIST\_EC  
     DDS\_C, [365](#)  
 DDSC\_LOG\_PARTICIPANT\_ENTITY  
     DDS\_C, [397](#)  
 DDSC\_LOG\_PARTICIPANT\_GENERIC\_MESSAGE\_OBJECT  
     DDS\_C, [380](#)  
 DDSC\_LOG\_PARTICIPANT\_ID\_QOS\_POLICY  
     DDS\_C, [392](#)  
 DDSC\_LOG\_PARTICIPANT\_LISTENER  
     DDS\_C, [396](#)  
 DDSC\_LOG\_PARTICIPANT\_LOOKUP\_EC  
     DDS\_C, [365](#)  
 DDSC\_LOG\_PARTICIPANT\_NAME\_EMPTY\_EC

DDS\_C, [412](#)  
DDSC\_LOG\_PARTICIPANT\_POOL\_OBJECT  
  DDS\_C, [378](#)  
DDSC\_LOG\_PARTICIPANT\_PROPERTIES  
  DDS\_C, [379](#)  
DDSC\_LOG\_PARTICIPANT\_QOS  
  DDS\_C, [389](#)  
DDSC\_LOG\_PARTICIPANT\_RECORD  
  DDS\_C, [371](#)  
DDSC\_LOG\_PARTICIPANT\_RESOURCES  
  DDS\_C, [388](#)  
DDSC\_LOG\_PARTICIPANTDATA\_OBJECT  
  DDS\_C, [375](#)  
DDSC\_LOG\_PARTICIPANTFACTORY\_LISTENER  
  DDS\_C, [396](#)  
DDSC\_LOG\_PARTICIPANTFACTORY\_QOS  
  DDS\_C, [389](#)  
DDSC\_LOG\_PARTICIPANTFACTORYQOS\_OBJECT  
  DDS\_C, [376](#)  
DDSC\_LOG\_PARTICIPANTQOS\_OBJECT  
  DDS\_C, [376](#)  
DDSC\_LOG\_PARTMESSAGE\_WRITE\_SAMPLE\_FAILED\_EC  
  DDS\_C, [442](#)  
DDSC\_LOG\_PROPERTY\_QOS\_POLICY  
  DDS\_C, [393](#)  
DDSC\_LOG\_PROPERTYQOSPOLICY\_OBJECT  
  DDS\_C, [380](#)  
DDSC\_LOG\_PROTOCOL\_QOS\_POLICY  
  DDS\_C, [391](#)  
DDSC\_LOG\_PUBLICATION\_ENTITY  
  DDS\_C, [397](#)  
DDSC\_LOG\_PUBLICATION\_RECORD  
  DDS\_C, [371](#)  
DDSC\_LOG\_PUBLICATIONDATA\_OBJECT  
  DDS\_C, [375](#)  
DDSC\_LOG\_PUBLISH\_MODE\_QOS\_POLICY  
  DDS\_C, [393](#)  
DDSC\_LOG\_PUBLISHER\_ENTITY  
  DDS\_C, [397](#)  
DDSC\_LOG\_PUBLISHER\_LISTENER  
  DDS\_C, [395](#)  
DDSC\_LOG\_PUBLISHER\_QOS  
  DDS\_C, [389](#)  
DDSC\_LOG\_PUBLISHER\_RECORD  
  DDS\_C, [371](#)  
DDSC\_LOG\_PUBLISHERQOS\_OBJECT  
  DDS\_C, [376](#)  
DDSC\_LOG\_QOS\_CHANGED\_EC  
  DDS\_C, [395](#)  
DDSC\_LOG\_QOS\_COPY\_EC  
  DDS\_C, [394](#)  
DDSC\_LOG\_QOS\_FINALIZE\_EC  
  DDS\_C, [394](#)  
DDSC\_LOG\_QOS\_GET\_EC  
  DDS\_C, [395](#)  
DDSC\_LOG\_QOS\_IMMUTABLE\_EC  
  DDS\_C, [394](#)  
DDSC\_LOG\_QOS\_INCONSISTENT\_EC  
  DDS\_C, [394](#)  
DDSC\_LOG\_QOS\_INCONSISTENT\_POLICIES\_EC  
  DDS\_C, [393](#)  
DDSC\_LOG\_QOS\_INCONSISTENT\_POLICY\_EC  
  DDS\_C, [393](#)  
DDSC\_LOG\_QOS\_INITIALIZE\_EC  
  DDS\_C, [394](#)  
DDSC\_LOG\_QOS\_SET\_EC  
  DDS\_C, [394](#)  
DDSC\_LOG\_QOS\_SET\_ON\_ENABLED\_EC  
  DDS\_C, [394](#)  
DDSC\_LOG\_QUEUE\_FINALIZE\_EC  
  DDS\_C, [419](#)  
DDSC\_LOG\_QUEUE\_PUBLICATION\_DISCOVERY\_EC  
  DDS\_C, [419](#)  
DDSC\_LOG\_QUEUE\_SUBSCRIPTION\_DISCOVERY\_EC  
  DDS\_C, [419](#)  
DDSC\_LOG\_READERHISTORY\_COMPONENT  
  DDS\_C, [386](#)  
DDSC\_LOG\_READTAKE\_SEQUENCE  
  DDS\_C, [384](#)  
DDSC\_LOG\_RECORD\_CREATE\_EC  
  DDS\_C, [372](#)  
DDSC\_LOG\_RECORD\_DELETE\_EC  
  DDS\_C, [373](#)  
DDSC\_LOG\_RECORD\_ERROR\_EC  
  DDS\_C, [373](#)  
DDSC\_LOG\_RECORD\_EXISTS\_EC  
  DDS\_C, [373](#)  
DDSC\_LOG\_RECORD\_FINALIZE\_EC  
  DDS\_C, [374](#)  
DDSC\_LOG\_RECORD\_INITIALIZE\_EC  
  DDS\_C, [374](#)  
DDSC\_LOG\_RECORD\_INSERT\_EC  
  DDS\_C, [373](#)  
DDSC\_LOG\_RECORD\_LOOKUP\_EC  
  DDS\_C, [374](#)  
DDSC\_LOG\_RECORD\_NOT\_EXISTS\_EC  
  DDS\_C, [374](#)  
DDSC\_LOG\_RECORD\_REMOVE\_EC  
  DDS\_C, [374](#)  
DDSC\_LOG\_RECORD\_RESET\_EC  
  DDS\_C, [374](#)  
DDSC\_LOG\_RECORD\_SELECT\_EC  
  DDS\_C, [374](#)  
DDSC\_LOG\_REFRESH\_LIVELINESS\_FAILED\_EC  
  DDS\_C, [443](#)  
DDSC\_LOG\_REFRESH\_REM\_PARTICIPANT\_EC  
  DDS\_C, [365](#)  
DDSC\_LOG\_REFRESH\_REM\_PARTICIPANT\_TIMEOUT\_EC

DDS\_C, [365](#)  
 DDSC\_LOG\_RELEASE\_META\_MC\_EC  
     DDS\_C, [367](#)  
 DDSC\_LOG\_RELEASE\_META\_UC\_EC  
     DDS\_C, [367](#)  
 DDSC\_LOG\_RELEASE\_USER\_MC\_EC  
     DDS\_C, [367](#)  
 DDSC\_LOG\_RELEASE\_USER\_UC\_EC  
     DDS\_C, [367](#)  
 DDSC\_LOG\_RELIABILITY\_QOS\_POLICY  
     DDS\_C, [391](#)  
 DDSC\_LOG\_REMOTE\_ENDPOINT\_TRUST\_STATE\_RECORD  
     DDS\_C, [372](#)  
 DDSC\_LOG\_REMOTE\_PARTICIPANT\_KEY\_NOT\_EQUAL\_EC  
     DDS\_C, [364](#)  
 DDSC\_LOG\_REMOTE\_PARTICIPANT\_NAME\_EMPTY\_EC  
     DDS\_C, [413](#)  
 DDSC\_LOG\_REMOTE\_PARTICIPANT\_RESTART\_EC  
     DDS\_C, [414](#)  
 DDSC\_LOG\_REMOTE\_PUBLICATION\_DATA\_CHANGED\_EC  
     DDS\_C, [369](#)  
 DDSC\_LOG\_REMOTE\_SUBSCRIPTION\_DATA\_CHANGED\_EC  
     DDS\_C, [369](#)  
 DDSC\_LOG\_REMOVE\_PUBLICATION\_EC  
     DDS\_C, [365](#)  
 DDSC\_LOG\_REMOVE\_SUBSCRIPTION\_EC  
     DDS\_C, [365](#)  
 DDSC\_LOG\_REPRESENTATION\_ID\_SEQUENCE  
     DDS\_C, [385](#)  
 DDSC\_LOG\_RESERVE\_LOCATORS\_EC  
     DDS\_C, [366](#)  
 DDSC\_LOG\_RESERVED\_SEQUENCE  
     DDS\_C, [383](#)  
 DDSC\_LOG\_RESOLVE\_LOCATORS\_PRIORITY\_EC  
     DDS\_C, [369](#)  
 DDSC\_LOG\_RESOURCE\_EXCEEDED\_EC  
     DDS\_C, [388](#)  
 DDSC\_LOG\_RESOURCE\_LIMIT\_QOS\_POLICY  
     DDS\_C, [391](#)  
 DDSC\_LOG\_ROUTE\_RECORD  
     DDS\_C, [372](#)  
 DDSC\_LOG\_ROUTE\_SEQUENCE  
     DDS\_C, [383](#)  
 DDSC\_LOG\_ROUTERESOLVER\_OBJECT  
     DDS\_C, [377](#)  
 DDSC\_LOG\_RT\_OBJECT  
     DDS\_C, [378](#)  
 DDSC\_LOG\_RTPS\_COMPONENT  
     DDS\_C, [386](#)  
 DDSC\_LOG\_RTPS\_NETIO\_KIND  
     DDS\_C, [403](#)  
 DDSC\_LOG\_SAMPLE\_RESOURCES  
     DDS\_C, [388](#)  
 DDSC\_LOG\_SEND\_LIVELINESS\_FAILED\_EC  
     DDS\_C, [442](#)  
 DDSC\_LOG\_SEQUENCE\_COPY\_EC  
     DDS\_C, [385](#)  
 DDSC\_LOG\_SEQUENCE\_FINALIZE\_EC  
     DDS\_C, [385](#)  
 DDSC\_LOG\_SEQUENCE\_GETREF\_EC  
     DDS\_C, [385](#)  
 DDSC\_LOG\_SEQUENCE\_INITIALIZE\_EC  
     DDS\_C, [385](#)  
 DDSC\_LOG\_SEQUENCE\_INVALID\_EC  
     DDS\_C, [386](#)  
 DDSC\_LOG\_SEQUENCE\_SETLENGTH\_EC  
     DDS\_C, [385](#)  
 DDSC\_LOG\_SEQUENCE\_SETMAX\_EC  
     DDS\_C, [385](#)  
 DDSC\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC  
     DDS\_C, [415](#)  
 DDSC\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC  
     DDS\_C, [416](#)  
 DDSC\_LOG\_SERIALIZE\_ENTITY\_NAME\_EC  
     DDS\_C, [416](#)  
 DDSC\_LOG\_SERIALIZE\_GUID\_EC  
     DDS\_C, [416](#)  
 DDSC\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC  
     DDS\_C, [418](#)  
 DDSC\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC  
     DDS\_C, [418](#)  
 DDSC\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC  
     DDS\_C, [417](#)  
 DDSC\_LOG\_SERIALIZE\_TOPIC\_NAME\_EC  
     DDS\_C, [416](#)  
 DDSC\_LOG\_SERIALIZE\_TYPE\_NAME\_EC  
     DDS\_C, [416](#)  
 DDSC\_LOG\_SERIALIZE\_VENDOR\_ID\_EC  
     DDS\_C, [417](#)  
 DDSC\_LOG\_SET\_ENTITY\_NAME\_EC  
     DDS\_C, [363](#)  
 DDSC\_LOG\_STRING\_OBJECT  
     DDS\_C, [380](#)  
 DDSC\_LOG\_STRING\_RECORD  
     DDS\_C, [372](#)  
 DDSC\_LOG\_STRINGMANAGER\_OBJECT  
     DDS\_C, [381](#)  
 DDSC\_LOG\_SUBSCRIBER\_ENTITY  
     DDS\_C, [397](#)  
 DDSC\_LOG\_SUBSCRIBER\_LISTENER  
     DDS\_C, [395](#)  
 DDSC\_LOG\_SUBSCRIBER\_QOS  
     DDS\_C, [389](#)  
 DDSC\_LOG\_SUBSCRIBER\_RECORD  
     DDS\_C, [371](#)  
 DDSC\_LOG\_SUBSCRIBERQOS\_OBJECT  
     DDS\_C, [376](#)  
 DDSC\_LOG\_SUBSCRIPTION\_ENTITY

DDS\_C, [397](#)  
 DDSC\_LOG\_SUBSCRIPTION\_RECORD  
     DDS\_C, [370](#)  
 DDSC\_LOG\_SUBSCRIPTIONDATA\_OBJECT  
     DDS\_C, [375](#)  
 DDSC\_LOG\_SYS\_GET\_HOSTNAME\_EC  
     DDS\_C, [363](#)  
 DDSC\_LOG\_SYS\_GETTIME\_EC  
     DDS\_C, [363](#)  
 DDSC\_LOG\_SYSTEM\_OBJECT  
     DDS\_C, [377](#)  
 DDSC\_LOG\_TABLE\_CREATE\_EC  
     DDS\_C, [369](#)  
 DDSC\_LOG\_TABLE\_DELETE\_EC  
     DDS\_C, [370](#)  
 DDSC\_LOG\_TABLE\_INUSE\_EC  
     DDS\_C, [370](#)  
 DDSC\_LOG\_TABLE\_SELECT\_EC  
     DDS\_C, [370](#)  
 DDSC\_LOG\_TIMER\_CREATE\_TIMEOUT\_EC  
     DDS\_C, [366](#)  
 DDSC\_LOG\_TIMER\_OBJECT  
     DDS\_C, [378](#)  
 DDSC\_LOG\_TIMEROUT\_OBJECT  
     DDS\_C, [379](#)  
 DDSC\_LOG\_TOPIC\_ENTITY  
     DDS\_C, [396](#)  
 DDSC\_LOG\_TOPIC\_FIND\_EC  
     DDS\_C, [364](#)  
 DDSC\_LOG\_TOPIC\_LISTENER  
     DDS\_C, [395](#)  
 DDSC\_LOG\_TOPIC\_NAME\_CMP\_EC  
     DDS\_C, [410](#)  
 DDSC\_LOG\_TOPIC\_NARROW\_EC  
     DDS\_C, [366](#)  
 DDSC\_LOG\_TOPIC\_OBJECT  
     DDS\_C, [379](#)  
 DDSC\_LOG\_TOPIC\_QOS  
     DDS\_C, [388](#)  
 DDSC\_LOG\_TOPIC\_RECORD  
     DDS\_C, [371](#)  
 DDSC\_LOG\_TOPIC\_TOO\_LONG\_EC  
     DDS\_C, [411](#)  
 DDSC\_LOG\_TOPICQOS\_OBJECT  
     DDS\_C, [375](#)  
 DDSC\_LOG\_TRANSPORT\_COMPONENT  
     DDS\_C, [387](#)  
 DDSC\_LOG\_TRANSPORT\_QOS\_POLICY  
     DDS\_C, [392](#)  
 DDSC\_LOG\_TRUST\_ADVANCE\_SN\_FAILED\_EC  
     DDS\_C, [435](#)  
 DDSC\_LOG\_TRUST\_AFTER\_REMOTE\_PARTICIPANT\_AUDITING\_FAILED\_EC  
     DDS\_C, [439](#)  
 DDSC\_LOG\_TRUST\_AFTER\_REMOTE\_PARTICIPANT\_READ\_FAILED\_EC  
     DDS\_C, [439](#)  
 DDS\_C, [431](#)  
 DDSC\_LOG\_TRUST\_ALLOC\_FAILED\_EC  
     DDS\_C, [434](#)  
 DDSC\_LOG\_TRUST\_ASSERT\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC  
     DDS\_C, [440](#)  
 DDSC\_LOG\_TRUST\_ASSERT\_PARTICIPANT\_GENERIC\_MESSAGE\_FAILED\_EC  
     DDS\_C, [436](#)  
 DDSC\_LOG\_TRUST\_ASSERT\_PROPERTY\_FAILED\_EC  
     DDS\_C, [438](#)  
 DDSC\_LOG\_TRUST\_BEGIN\_HANDSHAKE\_REPLY\_EC  
     DDS\_C, [428](#)  
 DDSC\_LOG\_TRUST\_BEGIN\_HANDSHAKE\_REQUEST\_EC  
     DDS\_C, [431](#)  
 DDSC\_LOG\_TRUST\_BIND\_REMOTE\_DATAREADER\_FAILED\_EC  
     DDS\_C, [432](#)  
 DDSC\_LOG\_TRUST\_BIND\_REMOTE\_DATAWRITER\_FAILED\_EC  
     DDS\_C, [432](#)  
 DDSC\_LOG\_TRUST\_CACHE\_REMOTE\_PARTICIPANT\_SAMPLE\_FAILED\_EC  
     DDS\_C, [439](#)  
 DDSC\_LOG\_TRUST\_CHECK\_CREATE\_DATAREADER\_FAILED\_EC  
     DDS\_C, [428](#)  
 DDSC\_LOG\_TRUST\_CHECK\_CREATE\_DATAWRITER\_FAILED\_EC  
     DDS\_C, [428](#)  
 DDSC\_LOG\_TRUST\_CHECK\_CREATE\_PARTICIPANT\_FAILED\_EC  
     DDS\_C, [425](#)  
 DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_DATAREADER\_FAILED\_EC  
     DDS\_C, [430](#)  
 DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_DATAWRITER\_FAILED\_EC  
     DDS\_C, [430](#)  
 DDSC\_LOG\_TRUST\_CHECK\_REMOTE\_PARTICIPANT\_FAILED\_EC  
     DDS\_C, [427](#)  
 DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DATAREADER\_INTERCEPTOR\_T  
     DDS\_C, [437](#)  
 DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DATAWRITER\_INTERCEPTOR\_T  
     DDS\_C, [437](#)  
 DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DR\_TOKEN\_EC  
     DDS\_C, [429](#)  
 DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_DW\_TOKEN\_EC  
     DDS\_C, [429](#)  
 DDSC\_LOG\_TRUST\_CREATE\_LOCAL\_PARTICIPANT\_INTERCEPTOR\_T  
     DDS\_C, [428](#)  
 DDSC\_LOG\_TRUST\_CREATE\_TRUST\_PLUGIN\_EC  
     DDS\_C, [441](#)  
 DDSC\_LOG\_TRUST\_CREATE\_TRUST\_PLUGINS\_FAILED\_EC  
     DDS\_C, [431](#)  
 DDSC\_LOG\_TRUST\_DATA\_HOLDER\_COPY\_FAILED\_EC  
     DDS\_C, [434](#)  
 DDSC\_LOG\_TRUST\_DELETE\_TRUST\_PLUGINS\_EC  
     DDS\_C, [427](#)  
 DDSC\_LOG\_TRUST\_ENDPOINT\_INTERNAL\_ATTRIBUTES\_CREATE\_FA  
     DDS\_C, [437](#)  
 DDSC\_LOG\_TRUST\_ENDPOINT\_INTERNAL\_ATTRIBUTES\_VALIDATE\_LOCAL\_IDENTITY\_EC  
     DDS\_C, [424](#)  
 DDSC\_LOG\_TRUST\_FINALIZE\_AC\_PLUGIN\_PROPERTIES\_FAILED\_EC  
     DDS\_C, [431](#)

DDS\_C, [439](#)  
 DDSC\_LOG\_TRUST\_FINALIZE\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC  
 DDS\_C, [440](#)  
 DDSC\_LOG\_TRUST\_FINALIZE\_REMOTE\_PARTICIPANT\_FAILED\_EC  
 DDS\_C, [440](#)  
 DDSC\_LOG\_TRUST\_GET\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC  
 DDS\_C, [440](#)  
 DDSC\_LOG\_TRUST\_GET\_AUTHENTICATED\_PEER\_CREDENTIALS\_FAILED\_EC  
 DDS\_C, [426](#)  
 DDSC\_LOG\_TRUST\_GET\_DATAREADER\_TRUST\_ATTRIBUTES\_FAILED\_EC  
 DDS\_C, [428](#)  
 DDSC\_LOG\_TRUST\_GET\_DATAWRITER\_TRUST\_ATTRIBUTES\_FAILED\_EC  
 DDS\_C, [428](#)  
 DDSC\_LOG\_TRUST\_GET\_IDENTITY\_TOKEN\_FAILED\_EC  
 DDS\_C, [424](#)  
 DDSC\_LOG\_TRUST\_GET\_PARTICIPANT\_TRUST\_ATTRIBUTES\_FAILED\_EC  
 DDS\_C, [425](#)  
 DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC  
 DDS\_C, [425](#)  
 DDSC\_LOG\_TRUST\_GET\_PERMISSIONS\_TOKEN\_FAILED\_EC  
 DDS\_C, [425](#)  
 DDSC\_LOG\_TRUST\_GET\_SHARED\_SECRET\_FAILED\_EC  
 DDS\_C, [427](#)  
 DDSC\_LOG\_TRUST\_GET\_TOPIC\_TRUST\_ATTRIBUTES\_FAILED\_EC  
 DDS\_C, [427](#)  
 DDSC\_LOG\_TRUST\_IGNORED\_HANDSHAKE\_MESSAGE\_EC  
 DDS\_C, [423](#)  
 DDSC\_LOG\_TRUST\_IGNORED\_REMOTE\_PARTICIPANT\_FAILED\_EC  
 DDS\_C, [423](#)  
 DDSC\_LOG\_TRUST\_INCOMPATIBLE\_ENDPOINT\_TRUST\_ATTRIBUTES\_FAILED\_EC  
 DDS\_C, [443](#)  
 DDSC\_LOG\_TRUST\_INITIALIZE\_PARTICIPANT\_BUILTIN\_DATA\_FAILED\_EC  
 DDS\_C, [435](#)  
 DDSC\_LOG\_TRUST\_INITIALIZE\_PARTICIPANT\_TRUST\_FAILED\_EC  
 DDS\_C, [434](#)  
 DDSC\_LOG\_TRUST\_INVALID\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC  
 DDS\_C, [440](#)  
 DDSC\_LOG\_TRUST\_INVALID\_GENERIC\_MESSAGE\_REPLY\_FAILED\_EC  
 DDS\_C, [434](#)  
 DDSC\_LOG\_TRUST\_INVALID\_IDENTITY\_HANDLE\_EC  
 DDS\_C, [424](#)  
 DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_HANDLE\_EC  
 DDS\_C, [425](#)  
 DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_TOKENS\_DESTINATION\_FAILED\_EC  
 DDS\_C, [437](#)  
 DDSC\_LOG\_TRUST\_INVALID\_INTERCEPTOR\_TOKENS\_EC  
 DDS\_C, [436](#)  
 DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_GENERIC\_MESSAGE\_EC  
 DDS\_C, [435](#)  
 DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_GUID\_EC  
 DDS\_C, [424](#)  
 DDSC\_LOG\_TRUST\_INVALID\_PARTICIPANT\_INTERNAL\_ATTRIBUTES\_FAILED\_EC  
 DDS\_C, [439](#)  
 DDSC\_LOG\_TRUST\_INVALID\_PERMISSIONS\_HANDLE\_EC  
 DDS\_C, [424](#)  
 DDSC\_LOG\_TRUST\_INVALID\_SAMPLE\_MAX\_SERIALIZED\_SIZE\_EC  
 DDS\_C, [434](#)  
 DDSC\_LOG\_TRUST\_INVALID\_SHARED\_SECRET\_EC  
 DDS\_C, [438](#)  
 DDSC\_LOG\_TRUST\_LOOKUP\_FACTORY\_FAILED\_EC  
 DDS\_C, [441](#)  
 DDSC\_LOG\_TRUST\_LOOKUP\_FACTORY\_PROPERTY\_FAILED\_EC  
 DDS\_C, [431](#)  
 DDSC\_LOG\_TRUST\_PARTICIPANT\_AUTHENTICATION\_FAILED\_EC  
 DDS\_C, [436](#)  
 DDSC\_LOG\_TRUST\_PARTICIPANT\_INTERNAL\_ATTRIBUTES\_CREATE\_FAILED\_EC  
 DDS\_C, [439](#)  
 DDSC\_LOG\_TRUST\_PARTICIPANT\_UNEXPECTED\_AUTHENTICATION\_FAILED\_EC  
 DDS\_C, [438](#)  
 DDSC\_LOG\_TRUST\_PBUFS\_MAX\_EXCEEDED\_EC  
 DDS\_C, [441](#)  
 DDSC\_LOG\_TRUST\_PREPARE\_AUTH\_HANDSHAKE\_MESSAGE\_EC  
 DDS\_C, [433](#)  
 DDSC\_LOG\_TRUST\_PREPARE\_AUTH\_REQUEST\_EC  
 DDS\_C, [433](#)  
 DDSC\_LOG\_TRUST\_PREPARE\_ENDPOINT\_INTERCEPTOR\_MESSAGE\_EC  
 DDS\_C, [433](#)  
 DDSC\_LOG\_TRUST\_PREPARE\_LOCAL\_DP\_TOKENS\_EC  
 DDS\_C, [438](#)  
 DDSC\_LOG\_TRUST\_PREPARE\_LOCAL\_PARTICIPANT\_STATE\_FAILED\_EC  
 DDS\_C, [427](#)  
 DDSC\_LOG\_TRUST\_PROCESS\_HANDSHAKE\_EC  
 DDS\_C, [431](#)  
 DDSC\_LOG\_TRUST\_PROCESS\_REMOTE\_INTERCEPTOR\_TOKENS\_FAILED\_EC  
 DDS\_C, [438](#)  
 DDSC\_LOG\_TRUST\_REAUTH\_REMOTE\_PARTICIPANT\_FAILED\_EC  
 DDS\_C, [441](#)  
 DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_DATAREADER\_FAILED\_EC  
 DDS\_C, [429](#)  
 DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_DATAWRITER\_FAILED\_EC  
 DDS\_C, [430](#)  
 DDSC\_LOG\_TRUST\_REGISTER\_LOCAL\_PARTICIPANT\_FAILED\_EC  
 DDS\_C, [425](#)  
 DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_DATAREADER\_FAILED\_EC  
 DDS\_C, [430](#)  
 DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_DATAWRITER\_FAILED\_EC  
 DDS\_C, [430](#)  
 DDSC\_LOG\_TRUST\_REGISTER\_MATCHED\_REMOTE\_PARTICIPANT\_FAILED\_EC  
 DDS\_C, [427](#)  
 DDSC\_LOG\_TRUST\_REGISTER\_REMOTE\_DATAREADER\_FAILED\_EC  
 DDS\_C, [432](#)  
 DDSC\_LOG\_TRUST\_REGISTER\_REMOTE\_DATAWRITER\_FAILED\_EC  
 DDS\_C, [432](#)  
 DDSC\_LOG\_TRUST\_RELEASE\_PARTICIPANT\_GENERIC\_MESSAGE\_FAILED\_EC  
 DDS\_C, [435](#)  
 DDSC\_LOG\_TRUST\_REMOVE\_REMOTE\_PARTICIPANT\_EC  
 DDS\_C, [431](#)  
 DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_IN\_PROGRESS\_EC  
 DDS\_C, [431](#)

DDS\_C, [435](#)  
 DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_MESSAGE\_FAILED\_EC  
 DDS\_C, [441](#)  
 DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_MESSAGE\_FAILED\_EC  
 DDS\_C, [436](#)  
 DDSC\_LOG\_TRUST\_RESEND\_HANDSHAKE\_STOP\_FAILED\_EC  
 DDS\_C, [435](#)  
 DDSC\_LOG\_TRUST\_RESET\_REMOTE\_PARTICIPANT\_TRUST\_FAILED\_EC  
 DDS\_C, [440](#)  
 DDSC\_LOG\_TRUST\_RETURN\_AUTHENTICATED\_PEER\_OBSOLETE\_EC  
 DDS\_C, [437](#)  
 DDSC\_LOG\_TRUST\_RETURN\_DATAWRITER\_TRUST\_ATTRIBUTES\_FAILED\_EC  
 DDS\_C, [428](#)  
 DDSC\_LOG\_TRUST\_RETURN\_HANDSHAKE\_HANDLE\_FAILED\_EC  
 DDS\_C, [429](#)  
 DDSC\_LOG\_TRUST\_RETURN\_IDENTITY\_HANDLE\_FAILED\_EC  
 DDS\_C, [426](#)  
 DDSC\_LOG\_TRUST\_RETURN\_IDENTITY\_TOKEN\_FAILED\_EC  
 DDS\_C, [426](#)  
 DDSC\_LOG\_TRUST\_RETURN\_INTERCEPTOR\_TOKENS\_FAILED\_EC  
 DDS\_C, [437](#)  
 DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC  
 DDS\_C, [425](#)  
 DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_HANDLE\_FAILED\_EC  
 DDS\_C, [426](#)  
 DDSC\_LOG\_TRUST\_RETURN\_PERMISSIONS\_TOKEN\_FAILED\_EC  
 DDS\_C, [426](#)  
 DDSC\_LOG\_TRUST\_RETURN\_SHAREDSECRET\_HANDLE\_FAILED\_EC  
 DDS\_C, [426](#)  
 DDSC\_LOG\_TRUST\_SEND\_AUTH\_REQUEST\_FAILED\_EC  
 DDS\_C, [440](#)  
 DDSC\_LOG\_TRUST\_SEND\_ENDPOINT\_INTERCEPTOR\_STATE\_FAILED\_EC  
 DDS\_C, [437](#)  
 DDSC\_LOG\_TRUST\_SEND\_HANDSHAKE\_EC  
 DDS\_C, [432](#)  
 DDSC\_LOG\_TRUST\_SEND\_HANDSHAKE\_MESSAGE\_FAILED\_EC  
 DDS\_C, [436](#)  
 DDSC\_LOG\_TRUST\_SEND\_PARTICIPANT\_INTERCEPTOR\_STATE\_FAILED\_EC  
 DDS\_C, [436](#)  
 DDSC\_LOG\_TRUST\_SERIALIZE\_DATA\_FAILED\_EC  
 DDS\_C, [434](#)  
 DDSC\_LOG\_TRUST\_SET\_AUTH\_HANDSHAKE\_STATE\_FAILED\_EC  
 DDS\_C, [441](#)  
 DDSC\_LOG\_TRUST\_SET\_PERMISSIONS\_CREDENTIAL\_TOKEN\_FAILED\_EC  
 DDS\_C, [427](#)  
 DDSC\_LOG\_TRUST\_SET\_REMOTE\_DATAREADER\_INTERCEPTOR\_FAILED\_EC  
 DDS\_C, [429](#)  
 DDSC\_LOG\_TRUST\_SET\_REMOTE\_DATAWRITER\_INTERCEPTOR\_FAILED\_EC  
 DDS\_C, [429](#)  
 DDSC\_LOG\_TRUST\_SET\_REMOTE\_PARTICIPANT\_INTERCEPTOR\_FAILED\_EC  
 DDS\_C, [429](#)  
 DDSC\_LOG\_TRUST\_TRANSFORM\_INCOMING\_SERIALIZED\_DATA\_FAILED\_EC  
 DDS\_C, [439](#)  
 DDSC\_LOG\_TRUST\_TRANSFORM\_OUTGOING\_SERIALIZED\_DATA\_FAILED\_EC  
 DDS\_C, [439](#)  
 DDSC\_LOG\_TRUST\_UNAUTHORIZED\_REMOTE\_PARTICIPANT\_EC  
 DDS\_C, [424](#)  
 DDSC\_LOG\_TRUST\_UNEXPECTED\_REMOTE\_PARTICIPANT\_STATUS\_EC  
 DDS\_C, [436](#)  
 DDSC\_LOG\_TRUST\_UNEXPECTED\_VALIDATION\_RESULT\_EC  
 DDS\_C, [424](#)  
 DDSC\_LOG\_TRUST\_UNKNOWN\_GMCLASSID\_EC  
 DDS\_C, [423](#)  
 DDSC\_LOG\_TRUST\_UNREGISTER\_DATAREADER\_FAILED\_EC  
 DDS\_C, [430](#)  
 DDSC\_LOG\_TRUST\_UNREGISTER\_DATAWRITER\_FAILED\_EC  
 DDS\_C, [430](#)  
 DDSC\_LOG\_TRUST\_UNREGISTER\_PARTICIPANT\_FAILED\_EC  
 DDS\_C, [426](#)  
 DDSC\_LOG\_TRUST\_UNREGISTER\_REMOTE\_DATAREADER\_FAILED\_EC  
 DDS\_C, [438](#)  
 DDSC\_LOG\_TRUST\_UNREGISTER\_REMOTE\_DATAWRITER\_FAILED\_EC  
 DDS\_C, [438](#)  
 DDSC\_LOG\_TRUST\_VALIDATE\_LOCAL\_DATAWRITER\_TRUST\_FAILED\_EC  
 DDS\_C, [433](#)  
 DDSC\_LOG\_TRUST\_VALIDATE\_REMOTE\_DATAREADER\_TRUST\_FAILED\_EC  
 DDS\_C, [433](#)  
 DDSC\_LOG\_TRUST\_VALIDATE\_REMOTE\_DATAWRITER\_TRUST\_FAILED\_EC  
 DDS\_C, [433](#)  
 DDSC\_LOG\_TRUST\_VALIDATE\_REMOTE\_IDENTITY\_EC  
 DDS\_C, [431](#)  
 DDSC\_LOG\_TRUST\_WRITE\_SAMPLE\_FAILED\_EC  
 DDS\_C, [435](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_COPY\_SAMPLE  
 DDS\_C, [410](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_CREATE\_SAMPLE  
 DDS\_C, [410](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_DELETE\_SAMPLE  
 DDS\_C, [410](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_DESERIALIZE\_DATA  
 DDS\_C, [410](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_GET\_KEY\_KIND  
 DDS\_C, [411](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_GET\_SERIALIZED\_SAMPLE\_MAX\_SIZE  
 DDS\_C, [410](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_INSTANCE\_TO\_KEYHASH  
 DDS\_C, [411](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_NULL\_EC  
 DDS\_C, [411](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_SERIALIZE\_DATA  
 DDS\_C, [410](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_SET\_SAMPLE\_ID\_EC  
 DDS\_C, [412](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_TYPE\_EC  
 DDS\_C, [411](#)  
 DDSC\_LOG\_TYPE\_FUNCTION\_UNMANAGED\_POOL\_FAILURE\_EC  
 DDS\_C, [411](#)



DDS\_C, [412](#)  
 DDSC\_LOG\_TYPE\_NAME\_CMP\_EC  
     DDS\_C, [409](#)  
 DDSC\_LOG\_TYPE\_OBJECT  
     DDS\_C, [376](#)  
 DDSC\_LOG\_TYPE\_PLUGIN\_GET\_BUFFER\_EC  
     DDS\_C, [412](#)  
 DDSC\_LOG\_TYPE\_PLUGIN\_INCONSISTENT\_DR\_EC  
     DDS\_C, [412](#)  
 DDSC\_LOG\_TYPE\_RECORD  
     DDS\_C, [372](#)  
 DDSC\_LOG\_TYPE\_SUPPORT\_QOS\_POLICY  
     DDS\_C, [391](#)  
 DDSC\_LOG\_TYPE\_TOO\_LONG\_EC  
     DDS\_C, [411](#)  
 DDSC\_LOG\_UNKNOWN\_REMOTE\_PARTICIPANT\_KEY\_EC  
     DDS\_C, [364](#)  
 DDSC\_LOG\_UNKNOWN\_REMOTE\_PARTICIPANT\_NAME\_EC  
     DDS\_C, [364](#)  
 DDSC\_LOG\_UNLOCK\_EC  
     DDS\_C, [445](#)  
 DDSC\_LOG\_UNSUPPORTED\_LOCATOR\_EC  
     DDS\_C, [417](#)  
 DDSC\_LOG\_UPDATE\_LEASE\_DURATION\_TIMER\_EC  
     DDS\_C, [442](#)  
 DDSC\_LOG\_UPDATE\_LIVELINESS\_FAILED\_EC  
     DDS\_C, [442](#)  
 DDSC\_LOG\_UPDATE\_REMOTE\_CONTENT\_FILTER\_EC  
     DDS\_C, [447](#)  
 DDSC\_LOG\_USERMULTICAST\_SEQUENCE  
     DDS\_C, [384](#)  
 DDSC\_LOG\_USERUNICAST\_SEQUENCE  
     DDS\_C, [384](#)  
 DDSC\_LOG\_UUID\_OBJECT  
     DDS\_C, [379](#)  
 DDSC\_LOG\_VALIDATE\_REMOTE\_PARTICIPANT\_TRUST\_FAILURE\_EC  
     DDS\_C, [432](#)  
 DDSC\_LOG\_WAITSET\_OBJECT  
     DDS\_C, [379](#)  
 DDSC\_LOG\_WRITERHISTORY\_COMPONENT  
     DDS\_C, [386](#)  
 DDSC\_LOG\_WS\_ADD\_COND\_REFERENCE\_EC  
     DDS\_C, [367](#)  
 DDSC\_LOG\_WS\_REMOVE\_COND\_REFERENCE\_EC  
     DDS\_C, [367](#)  
 DDSC\_LOG\_WSREF\_OBJECT  
     DDS\_C, [378](#)  
 DDSC\_LOG\_XTYPES\_LOANED\_SAMPLE\_STATE\_ERROR\_EC  
     DDS\_C, [443](#)  
 DDSC\_TRUST\_INVALID\_PARTICIPANT\_GENERIC\_MESSAGE\_EC  
     DDS\_C, [433](#)  
 DDSCondition, [644](#)  
 DDSConditionSeq, [645](#)  
 DDSDataReader, [645](#)  
     as\_entity, [648](#)  
     get\_instance\_replaced\_missed\_status, [662](#)  
     get\_listener, [650](#)  
     get\_liveliness\_changed\_status, [659](#)  
     get\_matched\_publication\_data, [660](#)  
     get\_matched\_publications, [660](#)  
     get\_qos, [649](#)  
     get\_requested\_deadline\_missed\_status, [661](#)  
     get\_requested\_incompatible\_qos\_status, [661](#)  
     get\_sample\_lost\_status, [661](#)  
     get\_sample\_rejected\_status, [660](#)  
     get\_subscriber, [648](#)  
     get\_subscription\_matched\_status, [659](#)  
     get\_topicdescription, [648](#)  
     is\_data\_consistent\_untyped, [657](#)  
     lookup\_instance\_untyped, [657](#)  
     read\_instance\_untyped, [654](#)  
     read\_next\_sample\_untyped, [656](#)  
     read\_untyped, [650](#)  
     return\_loan\_untyped, [658](#)  
     set\_listener, [649](#)  
     set\_qos, [648](#)  
     take\_instance\_untyped, [655](#)  
     take\_next\_sample\_untyped, [656](#)  
     take\_untyped, [651](#)  
 DDSDataReaderListener, [662](#)  
     on\_before\_sample\_commit, [665](#)  
     on\_before\_sample\_deserialize, [665](#)  
     on\_data\_available, [664](#)  
     on\_instance\_replaced, [665](#)  
     on\_liveliness\_changed, [664](#)  
     on\_requested\_deadline\_missed, [664](#)  
     on\_requested\_incompatible\_qos, [664](#)  
     on\_sample\_lost, [665](#)  
     on\_sample\_rejected, [664](#)  
     on\_subscription\_matched, [664](#)  
 DDSDataWriter, [666](#)  
     assert\_liveliness, [669](#)  
     dispose\_untyped, [676](#)  
     dispose\_w\_timestamp\_untyped, [677](#)  
     get\_listener, [672](#)  
     get\_liveliness\_lost\_status, [679](#)  
     get\_matched\_subscription\_data, [679](#)  
     get\_matched\_subscriptions, [679](#)  
     get\_offered\_deadline\_missed\_status, [680](#)  
     get\_offered\_incompatible\_qos\_status, [680](#)  
     get\_publication\_matched\_status, [678](#)  
     get\_publisher, [670](#)  
     get\_qos, [670](#)  
     get\_topic, [670](#)  
     register\_instance\_untyped, [672](#)  
     register\_instance\_w\_timestamp\_untyped, [673](#)  
     set\_listener, [671](#)  
     set\_qos, [671](#)

- unregister\_instance\_untyped, 673
- unregister\_instance\_w\_timestamp\_untyped, 675
- write\_untyped, 669
- write\_w\_timestamp\_untyped, 678
- DDSDataWriterListener, 681
  - on\_liveliness\_lost, 682
  - on\_offered\_deadline\_missed, 682
  - on\_offered\_incompatible\_qos, 682
  - on\_publication\_matched, 683
  - on\_reliable\_reader\_activity\_changed, 683
  - on\_reliable\_sample\_unacknowledged, 684
  - on\_sample\_removed, 684
- DDSDomainEntity, 685
- DDSDomainParticipant, 685
  - add\_peer, 706
  - as\_entity, 688
  - assert\_liveliness, 701
  - create\_flowcontroller, 708
  - create\_publisher, 688
  - create\_subscriber, 690
  - create\_topic, 692
  - delete\_contained\_entities, 702
  - delete\_flowcontroller, 708
  - delete\_publisher, 690
  - delete\_subscriber, 691
  - delete\_topic, 693
  - find\_topic, 693
  - get\_current\_time, 705
  - get\_default\_flowcontroller\_property, 706
  - get\_default\_publisher\_qos, 695
  - get\_default\_subscriber\_qos, 696
  - get\_default\_topic\_qos, 697
  - get\_discovered\_participant\_data, 705
  - get\_discovered\_participants, 705
  - get\_listener, 704
  - get\_qos, 694
  - lookup\_datareader\_by\_name, 701
  - lookup\_datawriter\_by\_name, 700
  - lookup\_flowcontroller, 709
  - lookup\_publisher\_by\_name, 699
  - lookup\_subscriber\_by\_name, 700
  - lookup\_topicdescription, 699
  - register\_type, 702
  - set\_default\_flowcontroller\_property, 707
  - set\_default\_publisher\_qos, 695
  - set\_default\_subscriber\_qos, 697
  - set\_default\_topic\_qos, 698
  - set\_listener, 704
  - set\_qos, 694
  - unregister\_type, 703
- DDSDomainParticipantFactory, 710
  - create\_participant, 712
  - create\_participant\_from\_config, 714
  - delete\_participant, 713
  - finalize\_instance, 712
  - get\_default\_participant\_qos, 716
  - get\_instance, 711
  - get\_qos, 716
  - get\_registry, 717
  - lookup\_participant, 714
  - lookup\_participant\_by\_name, 715
  - set\_default\_participant\_qos, 715
  - set\_qos, 717
- DDSDomainParticipantListener, 718
  - on\_before\_sample\_commit, 721
  - on\_before\_sample\_deserialize, 721
  - on\_data\_available, 719
  - on\_data\_on\_readers, 722
  - on\_inconsistent\_topic, 724
  - on\_instance\_replaced, 720
  - on\_liveliness\_changed, 720
  - on\_liveliness\_lost, 722
  - on\_offered\_deadline\_missed, 722
  - on\_offered\_incompatible\_qos, 723
  - on\_publication\_matched, 723
  - on\_reliable\_reader\_activity\_changed, 724
  - on\_requested\_deadline\_missed, 719
  - on\_requested\_incompatible\_qos, 720
  - on\_sample\_lost, 720
  - on\_sample\_rejected, 720
  - on\_subscription\_matched, 720
- DDSEntity, 725
  - enable, 727
  - get\_instance\_handle, 728
  - get\_status\_changes, 728
  - get\_statuscondition, 728
- DDSFlowController, 729
  - get\_name, 730
  - get\_participant, 731
  - get\_property, 730
  - set\_property, 729
  - trigger\_flow, 730
- DDSGuardCondition, 731
  - set\_trigger\_value, 732
- DDSListener, 732
- DDSPropertyQosPolicyHelper, 734
  - add\_property, 735
  - assert\_property, 735
  - lookup\_property, 736
  - remove\_property, 737
- DDSPublisher, 737
  - create\_datawriter, 739
  - delete\_contained\_entities, 744
  - delete\_datawriter, 740
  - get\_default\_datawriter\_qos, 740
  - get\_listener, 744
  - get\_participant, 742
  - get\_qos, 743



- lookup\_datawriter, [741](#)
- lookup\_datawriter\_by\_name, [742](#)
- set\_default\_datawriter\_qos, [741](#)
- set\_listener, [744](#)
- set\_qos, [743](#)
- DDSPublisherListener, [745](#)
- DDSStatusCondition, [746](#)
  - get\_enabled\_statuses, [747](#)
  - set\_enabled\_statuses, [747](#)
- DDSSubscriber, [748](#)
  - create\_datareader, [750](#)
  - delete\_contained\_entities, [752](#)
  - delete\_datareader, [750](#)
  - get\_default\_datareader\_qos, [751](#)
  - get\_listener, [756](#)
  - get\_participant, [755](#)
  - get\_qos, [755](#)
  - lookup\_datareader, [752](#)
  - lookup\_datareader\_by\_name, [753](#)
  - set\_default\_datareader\_qos, [751](#)
  - set\_listener, [756](#)
  - set\_qos, [755](#)
- DDSSubscriberListener, [757](#)
  - on\_data\_on\_readers, [758](#)
- DDSTheParticipantFactory
  - Domain Module, [31](#)
- DDSTopic, [759](#)
  - as\_entity, [761](#)
  - as\_topicdescription, [760](#)
  - get\_inconsistent\_topic\_status, [762](#)
  - get\_listener, [762](#)
  - get\_name, [763](#)
  - get\_participant, [763](#)
  - get\_qos, [761](#)
  - get\_type\_name, [763](#)
  - narrow, [760](#)
  - set\_listener, [762](#)
  - set\_qos, [761](#)
- DDSTopicDescription, [764](#)
  - get\_name, [765](#)
  - get\_participant, [765](#)
  - get\_type\_name, [765](#)
- DDSTopicListener, [766](#)
  - on\_inconsistent\_topic, [767](#)
- DDSWaitSet, [767](#)
  - attach\_condition, [770](#)
  - detach\_condition, [770](#)
  - get\_conditions, [770](#)
  - wait, [769](#)
- DEADLINE, [105](#)
- deadline
  - DDS\_DataReaderQos, [506](#)
  - DDS\_DataWriterQos, [515](#)
  - DDS\_PublicationBuiltinTopicData, [591](#)
  - DDS\_SubscriptionBuiltinTopicData, [627](#)
- default\_multicast\_locators
  - DDS\_ParticipantBuiltinTopicData, [575](#)
- default\_unicast\_locators
  - DDS\_ParticipantBuiltinTopicData, [575](#)
- delete\_contained\_entities
  - DDSDomainParticipant, [702](#)
  - DDSPublisher, [744](#)
  - DDSSubscriber, [752](#)
- delete\_data
  - FooDataWriter, [804](#)
  - FooTypeSupport, [819](#)
- delete\_datareader
  - DDSSubscriber, [750](#)
- delete\_datawriter
  - DDSPublisher, [740](#)
- delete\_flowcontroller
  - DDSDomainParticipant, [708](#)
- delete\_instance
  - NETIO\_DGRAM\_InterfaceI, [828](#)
  - ZCOPY\_NotifUserInterfaceI, [864](#)
- delete\_participant
  - DDSDomainParticipantFactory, [713](#)
- delete\_publisher
  - DDSDomainParticipant, [690](#)
- delete\_route
  - ZCOPY\_NotifUserInterfaceI, [866](#)
- delete\_subscriber
  - DDSDomainParticipant, [691](#)
- delete\_topic
  - DDSDomainParticipant, [693](#)
- deny\_interface
  - UDP\_InterfaceFactoryProperty, [853](#)
- depth
  - DDS\_HistoryQosPolicy, [556](#)
- deserialize\_data\_from\_cdr\_buffer
  - FooTypeSupport, [823](#)
- DESTINATION\_ORDER, [117](#)
  - DDS\_BY\_RECEPTION\_TIMESTAMP\_DESTINATIONORDER\_QOS, [118](#)
  - DDS\_BY\_SOURCE\_TIMESTAMP\_DESTINATIONORDER\_QOS, [118](#)
  - DDS\_DestinationOrderQosPolicyKind, [117](#)
- destination\_order
  - DDS\_DataReaderQos, [507](#)
  - DDS\_DataWriterQos, [516](#)
  - DDS\_PublicationBuiltinTopicData, [592](#)
  - DDS\_SubscriptionBuiltinTopicData, [628](#)
- destination\_rules
  - UDP\_InterfaceFactoryProperty, [856](#)
- detach\_condition
  - DDSWaitSet, [770](#)
- disable\_auto\_interface\_config
  - UDP\_InterfaceFactoryProperty, [855](#)

- disable\_builtin\_sql\_filter
  - DDS\_FilterQosPolicy, [546](#)
- disable\_multicast\_bind
  - UDP\_InterfaceFactoryProperty, [856](#)
- disable\_multicast\_interface\_select
  - UDP\_InterfaceFactoryProperty, [856](#)
- disable\_writer\_filtering
  - DDS\_FilterQosPolicy, [546](#)
- disc\_dpde\_log.h, [902](#)
- disc\_dpse\_log.h, [905](#)
  - DPSE\_DISCOVERYPLUGIN\_OBJECT, [909](#)
  - DPSE\_ENABLEDPARTICIPANTINDEX\_OBJECT, [910](#)
  - DPSE\_KEYLIST\_OBJECT, [909](#)
  - DPSE\_LOG\_CDR\_LONG\_KIND, [910](#)
  - DPSE\_LOG\_CDR\_PID\_KIND, [910](#)
  - DPSE\_LOG\_DEFAULT\_MULTICAST\_LOCATOR\_SEQUENCE, [911](#)
  - DPSE\_LOG\_DEFAULT\_UNICAST\_LOCATOR\_SEQUENCE, [911](#)
  - DPSE\_LOG\_META\_MULTICAST\_LOCATOR\_SEQUENCE, [911](#)
  - DPSE\_LOG\_META\_UNICAST\_LOCATOR\_SEQUENCE, [911](#)
  - DPSE\_PARTICIPANTANNOUCEMENT\_OBJECT, [909](#)
  - DPSE\_PARTICIPANTBUILTINTOPICDATA\_OBJECT, [909](#)
  - DPSE\_PARTICIPANTENTITYNAME\_OBJECT, [909](#)
  - DPSE\_PARTICIPANTMAMES\_OBJECT, [909](#)
  - DPSE\_PARTICIPANTNAMEINDEX\_OBJECT, [909](#)
  - DPSE\_PARTICIPANTPOOL\_OBJECT, [910](#)
  - DPSE\_PUBLISHER\_ENTITY, [910](#)
  - DPSE\_SUBSCRIBER\_ENTITY, [910](#)
  - DPSE\_TOPIC\_ENTITY, [910](#)
- discard\_loan
  - FooDataWriter, [803](#)
- DISCOVERY, [104](#)
- Discovery, [34](#)
  - DDS\_BuiltinTopicKey\_t, [39](#)
  - DDS\_LOCATOR\_ADDRESS\_INVALID, [37](#)
  - DDS\_LOCATOR\_INVALID, [37](#)
  - DDS\_LOCATOR\_KIND\_INVALID, [37](#)
  - DDS\_LOCATOR\_KIND\_RESERVED, [37](#)
  - DDS\_LOCATOR\_KIND\_SHMEM, [37](#)
  - DDS\_LOCATOR\_KIND\_UDPv4, [37](#)
  - DDS\_LOCATOR\_KIND\_UDPv6, [37](#)
  - DDS\_LOCATOR\_PORT\_INVALID, [37](#)
  - DDS\_PRODUCTVERSION\_UNKNOWN, [39](#)
  - DDS\_PROTOCOLVERSION, [38](#)
  - DDS\_PROTOCOLVERSION\_1\_0, [38](#)
  - DDS\_PROTOCOLVERSION\_1\_1, [38](#)
  - DDS\_PROTOCOLVERSION\_1\_2, [38](#)
  - DDS\_PROTOCOLVERSION\_2\_0, [38](#)
  - DDS\_PROTOCOLVERSION\_2\_1, [38](#)
  - DDS\_ProtocolVersion\_t, [39](#)
  - DDS\_VENDOR\_ID\_LENGTH\_MAX, [38](#)
- discovery
  - DDS\_DiscoveryQosPolicy, [526](#)
  - DDS\_DomainParticipantQos, [530](#)
- dispose
  - FooDataWriter, [799](#)
- dispose\_untyped
  - DDSDDataWriter, [676](#)
- dispose\_w\_timestamp
  - FooDataWriter, [801](#)
- dispose\_w\_timestamp\_untyped
  - DDSDDataWriter, [677](#)
- Documentation Guide, [24](#)
- Domain Module, [30](#)
  - DDS\_DomainId\_t, [31](#)
  - DDSTheParticipantFactory, [31](#)
  - Domain\_id\_gain
    - DDS\_RtpsWellKnownPorts\_t, [612](#)
  - DOMAIN\_PARTICIPANT\_RESOURCE\_LIMITS, [126](#)
  - DomainParticipant, [32](#)
  - DDS\_FLOW\_CONTROLLER\_PROPERTY\_DEFAULT, [34](#)
  - DDS\_PUBLISHER\_QOS\_DEFAULT, [33](#)
  - DDS\_SUBSCRIBER\_QOS\_DEFAULT, [33](#)
  - DDS\_TOPIC\_QOS\_DEFAULT, [33](#)
  - DomainParticipantFactory, [31](#)
    - DDS\_PARTICIPANT\_FACTORY\_QOS\_DEFAULT, [32](#)
    - DDS\_PARTICIPANT\_QOS\_DEFAULT, [32](#)
  - DomainParticipantFactoryQos, [228](#)
  - DomainParticipantQos, [228](#)
- DPDE, [470](#)
  - DPDE\_LOG\_ADD\_PEER\_EC, [481](#)
  - DPDE\_LOG\_ADD\_REMOTE\_PARTICIPANT\_ROUTES\_FAILED\_EC, [481](#)
  - DPDE\_LOG\_ADVANCE\_SN\_EC, [477](#)
  - DPDE\_LOG\_ALLOCATE\_DPDE\_EC, [476](#)
  - DPDE\_LOG\_ANNOUNCE\_WRITE\_EC, [477](#)
  - DPDE\_LOG\_ANNOUNCEMENT\_EC, [476](#)
  - DPDE\_LOG\_ASSERT\_REMOTE\_PARTICIPANT\_EC, [478](#)
  - DPDE\_LOG\_ASSERT\_REMOTE\_PUBLICATION\_EC, [479](#)
  - DPDE\_LOG\_ASSERT\_REMOTE\_SUBSCRIPTION\_EC, [479](#)
  - DPDE\_LOG\_CDR\_SET\_OFFSET\_EC, [473](#)
  - DPDE\_LOG\_CHECKSUM\_INCOMPATIBLE\_PARTICIPANT\_IGNORED, [482](#)
  - DPDE\_LOG\_CREATE\_PARTICIPANT\_DISCOVERY\_EC, [475](#)
  - DPDE\_LOG\_CREATE\_PUBLICATION\_DISCOVERY\_EC, [475](#)
  - DPDE\_LOG\_CREATE\_PUBLISHER\_EC, [474](#)

- DPDE\_LOG\_CREATE\_READER\_EC, [477](#)  
 DPDE\_LOG\_CREATE\_SUBSCRIBER\_EC, [475](#)  
 DPDE\_LOG\_CREATE\_SUBSCRIPTION\_DISCOVERY\_EC, [475](#)  
 DPDE\_LOG\_CREATE\_TOPIC\_EC, [477](#)  
 DPDE\_LOG\_CREATE\_WRITER\_EC, [477](#)  
 DPDE\_LOG\_DELETE\_PARTICIPANT\_TOPIC\_EC, [475](#)  
 DPDE\_LOG\_DELETE\_PUBLICATION\_TOPIC\_EC, [475](#)  
 DPDE\_LOG\_DELETE\_REMOTE\_PARTICIPANT\_ROUTES\_FAILED\_EC, [481](#)  
 DPDE\_LOG\_DELETE\_SUBSCRIPTION\_TOPIC\_EC, [475](#)  
 DPDE\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC, [476](#)  
 DPDE\_LOG\_DESERIALIZE\_LOCATOR\_EC, [480](#)  
 DPDE\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC, [474](#)  
 DPDE\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC, [476](#)  
 DPDE\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC, [474](#)  
 DPDE\_LOG\_DISPOSE\_PARTICIPANT\_EC, [476](#)  
 DPDE\_LOG\_DISPOSE\_PUBLICATION\_EC, [478](#)  
 DPDE\_LOG\_DISPOSE\_SUBSCRIPTION\_EC, [479](#)  
 DPDE\_LOG\_ENABLE\_REMOTE\_PARTICIPANT\_EC, [478](#)  
 DPDE\_LOG\_INVALID\_PARTICIPANT\_SAMPLE\_EC, [478](#)  
 DPDE\_LOG\_INVALID\_PUBLICATION\_SAMPLE\_EC, [479](#)  
 DPDE\_LOG\_INVALID\_SUBSCRIPTION\_SAMPLE\_EC, [480](#)  
 DPDE\_LOG\_LOCATORS\_FULL\_EC, [480](#)  
 DPDE\_LOG\_REFRESH\_REMOTE\_PARTICIPANT\_EC, [478](#)  
 DPDE\_LOG\_REGISTER\_TYPE\_EC, [477](#)  
 DPDE\_LOG\_REMOVE\_PEER\_EC, [481](#)  
 DPDE\_LOG\_REMOVE\_REMOTE\_PARTICIPANT\_EC, [481](#)  
 DPDE\_LOG\_REMOVE\_REMOTE\_PUBLICATION\_EC, [479](#)  
 DPDE\_LOG\_REMOVE\_REMOTE\_SUBSCRIPTION\_EC, [480](#)  
 DPDE\_LOG\_RETURN\_PARTICIPANT\_SAMPLE\_EC, [478](#)  
 DPDE\_LOG\_RETURN\_PUBLICATION\_SAMPLE\_EC, [479](#)  
 DPDE\_LOG\_RETURN\_SUBSCRIPTION\_SAMPLE\_EC, [480](#)  
 DPDE\_LOG\_SCHEDULE\_FAST\_ASSERTION\_EC, [477](#)  
 DPDE\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC, [476](#)  
 DPDE\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC, [473](#)  
 DPDE\_LOG\_SERIALIZE\_ENTITY\_NAME\_EC, [473](#)  
 DPDE\_LOG\_SERIALIZE\_GUID\_EC, [473](#)  
 DPDE\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC, [474](#)  
 DPDE\_LOG\_SERIALIZE\_PRESENTATION\_EC, [481](#)  
 DPDE\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC, [474](#)  
 DPDE\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC, [474](#)  
 DPDE\_LOG\_SERIALIZE\_TOPIC\_NAME\_EC, [473](#)  
 DPDE\_LOG\_SERIALIZE\_TYPE\_NAME\_EC, [473](#)  
 DPDE\_LOG\_SERIALIZE\_VENDOR\_ID\_EC, [474](#)  
 DPDE\_LOG\_TAKE\_PARTICIPANT\_SAMPLE\_EC, [478](#)  
 DPDE\_LOG\_TAKE\_PUBLICATION\_SAMPLE\_EC, [479](#)  
 DPDE\_LOG\_TAKE\_SUBSCRIPTION\_SAMPLE\_EC, [480](#)  
 DPDE\_LOG\_UNREGISTER\_TYPE\_EC, [481](#)  
 DPDE\_LOG\_UNSUPPORTED\_LOCATOR\_EC, [480](#)  
 DPDE\_LOG\_UPDATE\_PARTICIPANT\_ASSERT\_PERIOD\_EC, [476](#)  
 DPDE Dynamic Discovery API, [141](#)  
 DPDE\_DiscoveryPluginProperty\_INITIALIZER, [142](#)  
 DPDE\_DiscoveryPluginProperty, [771](#)  
 builtin\_endpoint\_reader\_nack\_period, [775](#)  
 builtin\_writer\_heartbeat\_period, [774](#)  
 builtin\_writer\_heartbeats\_per\_max\_samples, [775](#)  
 builtin\_writer\_max\_heartbeat\_retries, [774](#)  
 cache\_serialized\_samples, [773](#)  
 initial\_participant\_announcement\_period, [772](#)  
 initial\_participant\_announcements, [772](#)  
 max\_locators\_per\_discovered\_participant, [773](#)  
 max\_participant\_locators, [773](#)  
 max\_samples\_per\_builtin\_endpoint\_reader, [773](#)  
 max\_samples\_per\_remote\_builtin\_endpoint\_writer, [774](#)  
 participant\_liveliness\_assert\_period, [772](#)  
 participant\_liveliness\_lease\_duration, [772](#)  
 participant\_message\_reader\_reliability\_kind, [775](#)  
 DPDE\_DiscoveryPluginProperty\_INITIALIZER  
 DPDE Dynamic Discovery API, [142](#)  
 DPDE\_LOG\_ADD\_PEER\_EC  
 DPDE, [481](#)  
 DPDE\_LOG\_ADD\_REMOTE\_PARTICIPANT\_ROUTES\_FAILED\_EC  
 DPDE, [481](#)  
 DPDE\_LOG\_ADVANCE\_SN\_EC  
 DPDE, [477](#)  
 DPDE\_LOG\_ALLOCATE\_DPDE\_EC  
 DPDE, [476](#)  
 DPDE\_LOG\_ANNOUNCE\_WRITE\_EC  
 DPDE, [477](#)

DPDE\_LOG\_ANNOUNCEMENT\_EC  
     DPDE, [476](#)  
 DPDE\_LOG\_ASSERT\_REMOTE\_PARTICIPANT\_EC  
     DPDE, [478](#)  
 DPDE\_LOG\_ASSERT\_REMOTE\_PUBLICATION\_EC  
     DPDE, [479](#)  
 DPDE\_LOG\_ASSERT\_REMOTE\_SUBSCRIPTION\_EC  
     DPDE, [479](#)  
 DPDE\_LOG\_BASE  
     osapi\_log.h, [932](#)  
 DPDE\_LOG\_CDR\_SET\_OFFSET\_EC  
     DPDE, [473](#)  
 DPDE\_LOG\_CHECKSUM\_INCOMPATIBLE\_PARTICIPANT\_DISCOVERY\_EC  
     DPDE, [482](#)  
 DPDE\_LOG\_CREATE\_PARTICIPANT\_DISCOVERY\_EC  
     DPDE, [475](#)  
 DPDE\_LOG\_CREATE\_PUBLICATION\_DISCOVERY\_EC  
     DPDE, [475](#)  
 DPDE\_LOG\_CREATE\_PUBLISHER\_EC  
     DPDE, [474](#)  
 DPDE\_LOG\_CREATE\_READER\_EC  
     DPDE, [477](#)  
 DPDE\_LOG\_CREATE\_SUBSCRIBER\_EC  
     DPDE, [475](#)  
 DPDE\_LOG\_CREATE\_SUBSCRIPTION\_DISCOVERY\_EC  
     DPDE, [475](#)  
 DPDE\_LOG\_CREATE\_TOPIC\_EC  
     DPDE, [477](#)  
 DPDE\_LOG\_CREATE\_WRITER\_EC  
     DPDE, [477](#)  
 DPDE\_LOG\_DELETE\_PARTICIPANT\_TOPIC\_EC  
     DPDE, [475](#)  
 DPDE\_LOG\_DELETE\_PUBLICATION\_TOPIC\_EC  
     DPDE, [475](#)  
 DPDE\_LOG\_DELETE\_REMOTE\_PARTICIPANT\_ROUTES\_ENABLED\_EC  
     DPDE, [481](#)  
 DPDE\_LOG\_DELETE\_SUBSCRIPTION\_TOPIC\_EC  
     DPDE, [475](#)  
 DPDE\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC  
     DPDE, [476](#)  
 DPDE\_LOG\_DESERIALIZE\_LOCATOR\_EC  
     DPDE, [480](#)  
 DPDE\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC  
     DPDE, [474](#)  
 DPDE\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC  
     DPDE, [476](#)  
 DPDE\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC  
     DPDE, [474](#)  
 DPDE\_LOG\_DISPOSE\_PARTICIPANT\_EC  
     DPDE, [476](#)  
 DPDE\_LOG\_DISPOSE\_PUBLICATION\_EC  
     DPDE, [478](#)  
 DPDE\_LOG\_DISPOSE\_SUBSCRIPTION\_EC  
     DPDE, [479](#)  
 DPDE\_LOG\_ENABLE\_REMOTE\_PARTICIPANT\_EC  
     DPDE, [478](#)  
 DPDE\_LOG\_INVALID\_PARTICIPANT\_SAMPLE\_EC  
     DPDE, [478](#)  
 DPDE\_LOG\_INVALID\_PUBLICATION\_SAMPLE\_EC  
     DPDE, [479](#)  
 DPDE\_LOG\_INVALID\_SUBSCRIPTION\_SAMPLE\_EC  
     DPDE, [480](#)  
 DPDE\_LOG\_LOCATORS\_FULL\_EC  
     DPDE, [480](#)  
 DPDE\_LOG\_REFRESH\_REMOTE\_PARTICIPANT\_EC  
     DPDE, [478](#)  
 DPDE\_LOG\_REMOVE\_PEER\_EC  
     DPDE, [481](#)  
 DPDE\_LOG\_REMOVE\_REMOTE\_PARTICIPANT\_EC  
     DPDE, [481](#)  
 DPDE\_LOG\_REMOVE\_REMOTE\_PUBLICATION\_EC  
     DPDE, [479](#)  
 DPDE\_LOG\_REMOVE\_REMOTE\_SUBSCRIPTION\_EC  
     DPDE, [480](#)  
 DPDE\_LOG\_RETURN\_PARTICIPANT\_SAMPLE\_EC  
     DPDE, [478](#)  
 DPDE\_LOG\_RETURN\_PUBLICATION\_SAMPLE\_EC  
     DPDE, [479](#)  
 DPDE\_LOG\_RETURN\_SUBSCRIPTION\_SAMPLE\_EC  
     DPDE, [480](#)  
 DPDE\_LOG\_SCHEDULE\_FAST\_ASSERTION\_EC  
     DPDE, [477](#)  
 DPDE\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC  
     DPDE, [476](#)  
 DPDE\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC  
     DPDE, [473](#)  
 DPDE\_LOG\_SERIALIZE\_ENTITY\_NAME\_EC  
     DPDE, [473](#)  
 DPDE\_LOG\_SERIALIZE\_GUID\_EC  
     DPDE, [473](#)  
 DPDE\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC  
     DPDE, [474](#)  
 DPDE\_LOG\_SERIALIZE\_PRESENTATION\_EC  
     DPDE, [481](#)  
 DPDE\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC  
     DPDE, [474](#)  
 DPDE\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC  
     DPDE, [474](#)  
 DPDE\_LOG\_SERIALIZE\_TOPIC\_NAME\_EC  
     DPDE, [473](#)  
 DPDE\_LOG\_SERIALIZE\_TYPE\_NAME\_EC  
     DPDE, [473](#)  
 DPDE\_LOG\_SERIALIZE\_VENDOR\_ID\_EC  
     DPDE, [474](#)  
 DPDE\_LOG\_TAKE\_PARTICIPANT\_SAMPLE\_EC  
     DPDE, [478](#)

- DPDE\_LOG\_TAKE\_PUBLICATION\_SAMPLE\_EC  
DPDE, 479
- DPDE\_LOG\_TAKE\_SUBSCRIPTION\_SAMPLE\_EC  
DPDE, 480
- DPDE\_LOG\_UNREGISTER\_TYPE\_EC  
DPDE, 481
- DPDE\_LOG\_UNSUPPORTED\_LOCATOR\_EC  
DPDE, 480
- DPDE\_LOG\_UPDATE\_PARTICIPANT\_ASSERT\_PERIOD\_EC  
DPDE, 476
- DPDEDiscoveryFactory, 775  
get\_interface, 776
- DPSE, 457
- DPSE\_LOG\_ADD\_ANONYMOUS\_ROUTE\_EC, 467
- DPSE\_LOG\_ADVANCE\_SN\_EC, 466
- DPSE\_LOG\_ANNOUNCE\_LOCAL\_PARTICIPANT\_DELETION\_EC, 461
- DPSE\_LOG\_ANNOUNCE\_WRITE\_EC, 466
- DPSE\_LOG\_ANNOUNCEMENT\_EC, 466
- DPSE\_LOG\_ASSERT\_REMOTE\_PARTICIPANT\_EC, 467
- DPSE\_LOG\_CDR\_DESERIALIZE\_EC, 463
- DPSE\_LOG\_CDR\_SERIALIZE\_EC, 463
- DPSE\_LOG\_CDR\_SET\_POSITION\_EC, 463
- DPSE\_LOG\_CHECKSUM\_INCOMPATIBLE\_PARTICIPANT\_MESSAGES\_EC, 469
- DPSE\_LOG\_CREATE\_DISCOVERY\_PUBLISHER\_EC, 460
- DPSE\_LOG\_CREATE\_DISCOVERY\_SUBSCRIBER\_EC, 460
- DPSE\_LOG\_CREATE\_READER\_EC, 461
- DPSE\_LOG\_CREATE\_TOPIC\_EC, 461
- DPSE\_LOG\_CREATE\_WRITER\_EC, 461
- DPSE\_LOG\_DELETE\_ANONYMOUS\_ROUTE\_EC, 467
- DPSE\_LOG\_DELETE\_PUBLISHER\_EC, 461
- DPSE\_LOG\_DELETE\_READER\_EC, 462
- DPSE\_LOG\_DELETE\_SUBSCRIBER\_EC, 462
- DPSE\_LOG\_DELETE\_TOPIC\_EC, 462
- DPSE\_LOG\_DELETE\_WRITER\_EC, 461
- DPSE\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC, 465
- DPSE\_LOG\_DESERIALIZE\_GUID\_EC, 465
- DPSE\_LOG\_DESERIALIZE\_PARTICIPANT\_NAME\_EC, 465
- DPSE\_LOG\_DESERIALIZE\_PRODUCT\_VERSION\_EC, 464
- DPSE\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC, 465
- DPSE\_LOG\_DESERIALIZE\_TOO\_MANY\_LOCATORS\_EC, 465
- DPSE\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC, 466
- DPSE\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC, 465
- DPSE\_LOG\_DISPOSE\_EC, 461
- DPSE\_LOG\_ENTITY\_ENABLE\_EC, 468
- DPSE\_LOG\_GET\_DDS\_PROPERTIES\_EC, 468
- DPSE\_LOG\_GET\_TIMER\_EC, 469
- DPSE\_LOG\_INVALID\_DISCOVERY\_SAMPLE\_EC, 468
- DPSE\_LOG\_INVALID\_PEER\_ADDRESS\_EC, 460
- DPSE\_LOG\_MAX\_REMOTE\_PARTICIPANT\_EC, 467
- DPSE\_LOG\_OBJECT\_ALLOCATE\_EC, 462
- DPSE\_LOG\_OBJECT\_DELETE\_EC, 462
- DPSE\_LOG\_OBJECT\_FINALIZE\_EC, 462
- DPSE\_LOG\_OBJECT\_INITIALIZE\_EC, 462
- DPSE\_LOG\_OBJECT\_INVALID\_EC, 463
- DPSE\_LOG\_ON\_ASSERT\_REMOTE\_PARTICIPANT\_EC, 467
- DPSE\_LOG\_PARTICIPANT\_TAKE\_EC, 466
- DPSE\_LOG\_REFRESH\_REMOTE\_PARTICIPANT\_EC, 467
- DPSE\_LOG\_REGISTER\_TYPE\_EC, 460
- DPSE\_LOG\_RESET\_REMOTE\_PARTICIPANT\_EC, 467
- DPSE\_LOG\_RETURN\_DISCOVERY\_SAMPLE\_EC, 468
- DPSE\_LOG\_SCHEDULE\_FAST\_ASSERTION\_EC, 466
- DPSE\_LOG\_SEQUENCE\_COPY\_EC, 469
- DPSE\_LOG\_SEQUENCE\_FINALIZE\_EC, 469
- DPSE\_LOG\_SEQUENCE\_GETREF\_EC, 469
- DPSE\_LOG\_SEQUENCE\_INITIALIZE\_EC, 469
- DPSE\_LOG\_SEQUENCE\_SETLENGTH\_EC, 468
- DPSE\_LOG\_SEQUENCE\_SETMAX\_EC, 468
- DPSE\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC, 463
- DPSE\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC, 464
- DPSE\_LOG\_SERIALIZE\_GUID\_EC, 463
- DPSE\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC, 464
- DPSE\_LOG\_SERIALIZE\_META\_MULTICAST\_EC, 464
- DPSE\_LOG\_SERIALIZE\_META\_UNICAST\_EC, 464
- DPSE\_LOG\_SERIALIZE\_MULTICAST\_EC, 468
- DPSE\_LOG\_SERIALIZE\_PARTICIPANT\_NAME\_EC, 465
- DPSE\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC, 464
- DPSE\_LOG\_SERIALIZE\_PROP\_EC, 470
- DPSE\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC, 463
- DPSE\_LOG\_SERIALIZE\_VENDOR\_ID\_EC, 464
- DPSE\_LOG\_UNSUPPORTED\_LOCATOR\_EC, 469
- DPSE\_LOG\_UPDATE\_PARTICIPANT\_ASSERT\_PERIOD\_EC, 466

- DPSE Static Discovery API, [140](#)
  - DPSE\_DiscoveryPluginProperty\_INITIALIZER, [141](#)
- DPSE\_DISCOVERYPLUGIN\_OBJECT
  - disc\_dpse\_log.h, [909](#)
- DPSE\_DiscoveryPluginProperty, [776](#)
  - initial\_participant\_announcement\_period, [777](#)
  - initial\_participant\_announcements, [777](#)
  - max\_locators\_per\_discovered\_participant, [778](#)
  - max\_participant\_locators, [778](#)
  - participant\_liveliness\_assert\_period, [777](#)
  - participant\_liveliness\_lease\_duration, [777](#)
  - participant\_message\_reader\_reliability\_kind, [778](#)
- DPSE\_DiscoveryPluginProperty\_INITIALIZER
  - DPSE Static Discovery API, [141](#)
- DPSE\_ENABLEDPARTICIPANTINDEX\_OBJECT
  - disc\_dpse\_log.h, [910](#)
- DPSE\_KEYLIST\_OBJECT
  - disc\_dpse\_log.h, [909](#)
- DPSE\_LOG\_ADD\_ANONYMOUS\_ROUTE\_EC
  - DPSE, [467](#)
- DPSE\_LOG\_ADVANCE\_SN\_EC
  - DPSE, [466](#)
- DPSE\_LOG\_ANNOUNCE\_LOCAL\_PARTICIPANT\_DELETION\_EC
  - DPSE, [461](#)
- DPSE\_LOG\_ANNOUNCE\_WRITE\_EC
  - DPSE, [466](#)
- DPSE\_LOG\_ANNOUNCEMENT\_EC
  - DPSE, [466](#)
- DPSE\_LOG\_ASSERT\_REMOTE\_PARTICIPANT\_EC
  - DPSE, [467](#)
- DPSE\_LOG\_BASE
  - osapi\_log.h, [931](#)
- DPSE\_LOG\_CDR\_DESERIALIZE\_EC
  - DPSE, [463](#)
- DPSE\_LOG\_CDR\_LONG\_KIND
  - disc\_dpse\_log.h, [910](#)
- DPSE\_LOG\_CDR\_PID\_KIND
  - disc\_dpse\_log.h, [910](#)
- DPSE\_LOG\_CDR\_SERIALIZE\_EC
  - DPSE, [463](#)
- DPSE\_LOG\_CDR\_SET\_POSITION\_EC
  - DPSE, [463](#)
- DPSE\_LOG\_CHECKSUM\_INCOMPATIBLE\_PARTICIPANT\_DISCOVERED\_EC
  - DPSE, [469](#)
- DPSE\_LOG\_CREATE\_DISCOVERY\_PUBLISHER\_EC
  - DPSE, [460](#)
- DPSE\_LOG\_CREATE\_DISCOVERY\_SUBSCRIBER\_EC
  - DPSE, [460](#)
- DPSE\_LOG\_CREATE\_READER\_EC
  - DPSE, [461](#)
- DPSE\_LOG\_CREATE\_TOPIC\_EC
  - DPSE, [461](#)
- DPSE\_LOG\_CREATE\_WRITER\_EC
  - DPSE, [461](#)
- DPSE\_LOG\_DEFAULT\_MULTICAST\_LOCATOR\_SEQUENCE
  - disc\_dpse\_log.h, [911](#)
- DPSE\_LOG\_DEFAULT\_UNICAST\_LOCATOR\_SEQUENCE
  - disc\_dpse\_log.h, [911](#)
- DPSE\_LOG\_DELETE\_ANONYMOUS\_ROUTE\_EC
  - DPSE, [467](#)
- DPSE\_LOG\_DELETE\_PUBLISHER\_EC
  - DPSE, [461](#)
- DPSE\_LOG\_DELETE\_READER\_EC
  - DPSE, [462](#)
- DPSE\_LOG\_DELETE\_SUBSCRIBER\_EC
  - DPSE, [462](#)
- DPSE\_LOG\_DELETE\_TOPIC\_EC
  - DPSE, [462](#)
- DPSE\_LOG\_DELETE\_WRITER\_EC
  - DPSE, [461](#)
- DPSE\_LOG\_DESERIALIZE\_BUILTIN\_ENDPOINTS\_EC
  - DPSE, [465](#)
- DPSE\_LOG\_DESERIALIZE\_GUID\_EC
  - DPSE, [465](#)
- DPSE\_LOG\_DESERIALIZE\_PARTICIPANT\_NAME\_EC
  - DPSE, [465](#)
- DPSE\_LOG\_DESERIALIZE\_PRODUCT\_VERSION\_EC
  - DPSE, [464](#)
- DPSE\_LOG\_DESERIALIZE\_PROTOCOL\_VERSION\_EC
  - DPSE, [465](#)
- DPSE\_LOG\_DESERIALIZE\_TOO\_MANY\_LOCATORS\_EC
  - DPSE, [465](#)
- DPSE\_LOG\_DESERIALIZE\_UNKNOWN\_PID\_EC
  - DPSE, [466](#)
- DPSE\_LOG\_DESERIALIZE\_VENDOR\_ID\_EC
  - DPSE, [465](#)
- DPSE\_LOG\_DISPOSE\_EC
  - DPSE, [461](#)
- DPSE\_LOG\_ENTITY\_ENABLE\_EC
  - DPSE, [468](#)
- DPSE\_LOG\_GET\_DDS\_PROPERTIES\_EC
  - DPSE, [468](#)
- DPSE\_LOG\_GET\_TIMER\_EC
  - DPSE, [469](#)
- DPSE\_LOG\_INVALID\_DISCOVERY\_SAMPLE\_EC
  - DPSE, [468](#)
- DPSE\_LOG\_INVALID\_PEER\_ADDRESS\_EC
  - DPSE, [460](#)
- DPSE\_LOG\_MAX\_REMOTE\_PARTICIPANT\_EC
  - DPSE, [467](#)
- DPSE\_LOG\_META\_MULTICAST\_LOCATOR\_SEQUENCE
  - disc\_dpse\_log.h, [911](#)
- DPSE\_LOG\_META\_UNICAST\_LOCATOR\_SEQUENCE
  - disc\_dpse\_log.h, [911](#)
- DPSE\_LOG\_OBJECT\_ALLOCATE\_EC
  - DPSE, [462](#)
- DPSE\_LOG\_OBJECT\_DELETE\_EC
  - DPSE, [462](#)

- DPSE\_LOG\_OBJECT\_FINALIZE\_EC
  - DPSE, [462](#)
- DPSE\_LOG\_OBJECT\_INITIALIZE\_EC
  - DPSE, [462](#)
- DPSE\_LOG\_OBJECT\_INVALID\_EC
  - DPSE, [463](#)
- DPSE\_LOG\_ON\_ASSERT\_REMOTE\_PARTICIPANT\_EC
  - DPSE, [467](#)
- DPSE\_LOG\_PARTICIPANT\_TAKE\_EC
  - DPSE, [466](#)
- DPSE\_LOG\_REFRESH\_REMOTE\_PARTICIPANT\_EC
  - DPSE, [467](#)
- DPSE\_LOG\_REGISTER\_TYPE\_EC
  - DPSE, [460](#)
- DPSE\_LOG\_RESET\_REMOTE\_PARTICIPANT\_EC
  - DPSE, [467](#)
- DPSE\_LOG\_RETURN\_DISCOVERY\_SAMPLE\_EC
  - DPSE, [468](#)
- DPSE\_LOG\_SCHEDULE\_FAST\_ASSERTION\_EC
  - DPSE, [466](#)
- DPSE\_LOG\_SEQUENCE\_COPY\_EC
  - DPSE, [469](#)
- DPSE\_LOG\_SEQUENCE\_FINALIZE\_EC
  - DPSE, [469](#)
- DPSE\_LOG\_SEQUENCE\_GETREF\_EC
  - DPSE, [469](#)
- DPSE\_LOG\_SEQUENCE\_INITIALIZE\_EC
  - DPSE, [469](#)
- DPSE\_LOG\_SEQUENCE\_SETLENGTH\_EC
  - DPSE, [468](#)
- DPSE\_LOG\_SEQUENCE\_SETMAX\_EC
  - DPSE, [468](#)
- DPSE\_LOG\_SERIALIZE\_BUILTIN\_ENDPOINTS\_EC
  - DPSE, [463](#)
- DPSE\_LOG\_SERIALIZE\_DEFAULT\_UNICAST\_EC
  - DPSE, [464](#)
- DPSE\_LOG\_SERIALIZE\_GUID\_EC
  - DPSE, [463](#)
- DPSE\_LOG\_SERIALIZE\_LEASE\_DURATION\_EC
  - DPSE, [464](#)
- DPSE\_LOG\_SERIALIZE\_META\_MULTICAST\_EC
  - DPSE, [464](#)
- DPSE\_LOG\_SERIALIZE\_META\_UNICAST\_EC
  - DPSE, [464](#)
- DPSE\_LOG\_SERIALIZE\_MULTICAST\_EC
  - DPSE, [468](#)
- DPSE\_LOG\_SERIALIZE\_PARTICIPANT\_NAME\_EC
  - DPSE, [465](#)
- DPSE\_LOG\_SERIALIZE\_PRODUCT\_VERSION\_EC
  - DPSE, [464](#)
- DPSE\_LOG\_SERIALIZE\_PROP\_EC
  - DPSE, [470](#)
- DPSE\_LOG\_SERIALIZE\_PROTOCOL\_VERSION\_EC
  - DPSE, [463](#)
- DPSE\_LOG\_SERIALIZE\_VENDOR\_ID\_EC
  - DPSE, [464](#)
- DPSE\_LOG\_UNSUPPORTED\_LOCATOR\_EC
  - DPSE, [469](#)
- DPSE\_LOG\_UPDATE\_PARTICIPANT\_ASSERT\_PERIOD\_EC
  - DPSE, [466](#)
- DPSE\_PARTICIPANTANNOUNCEMENT\_OBJECT
  - disc\_dpse\_log.h, [909](#)
- DPSE\_PARTICIPANTBUILTINTOPICDATA\_OBJECT
  - disc\_dpse\_log.h, [909](#)
- DPSE\_PARTICIPANTENTITYNAME\_OBJECT
  - disc\_dpse\_log.h, [909](#)
- DPSE\_PARTICIPANTMAMES\_OBJECT
  - disc\_dpse\_log.h, [909](#)
- DPSE\_PARTICIPANTNAMEINDEX\_OBJECT
  - disc\_dpse\_log.h, [909](#)
- DPSE\_PARTICIPANTPOOL\_OBJECT
  - disc\_dpse\_log.h, [910](#)
- DPSE\_PUBLISHER\_ENTITY
  - disc\_dpse\_log.h, [910](#)
- DPSE\_SUBSCRIBER\_ENTITY
  - disc\_dpse\_log.h, [910](#)
- DPSE\_TOPIC\_ENTITY
  - disc\_dpse\_log.h, [910](#)
- DPSEDiscoveryFactory, [779](#)
  - get\_interface, [779](#)
- DPSEDiscoveryPlugin, [779](#)
  - RemoteParticipant\_assert, [780](#)
  - RemotePublication\_assert, [780](#)
  - RemoteSubscription\_assert, [780](#)
- DURABILITY, [110](#)
  - DDS\_DurabilityQosPolicyKind, [111](#)
  - DDS\_TRANSIENT\_LOCAL\_DURABILITY\_QOS, [111](#)
  - DDS\_VOLATILE\_DURABILITY\_QOS, [111](#)
- durability
  - DDS\_DataReaderQos, [507](#)
  - DDS\_DataWriterQos, [516](#)
  - DDS\_PublicationBuiltinTopicData, [592](#)
  - DDS\_SubscriptionBuiltinTopicData, [627](#)
- duration
  - DDS\_LatencyBudgetQosPolicy, [559](#)
- Dynamic Participant Dynamic Endpoint (DPDE), [243](#)
- Dynamic Participant Static Endpoint (DPSE), [242](#)
- enable
  - DDSEntity, [727](#)
- enable\_data\_consistency\_check
  - DDS\_DataWriterShmemRefTransferModeSettings, [521](#)
- enable\_endpoint\_discovery\_queue
  - DDS\_DiscoveryQosPolicy, [527](#)
- enable\_interface\_bind
  - UDP\_InterfaceFactoryProperty, [855](#)



- enable\_participant\_discovery\_by\_name
  - DDS\_DiscoveryQosPolicy, [527](#)
- enabled\_transports
  - DDS\_DiscoveryQosPolicy, [526](#)
  - DDS\_TransportQosPolicy, [635](#)
  - DDS\_UserTrafficQosPolicy, [637](#)
- encapsulation\_id
  - DDS\_SampleInfo, [618](#)
- ensure\_length
  - FooSeq, [812](#)
- Entity Support, [133](#)
- ENTITY\_FACTORY, [118](#)
- entity\_factory
  - DDS\_DomainParticipantFactoryQos, [529](#)
  - DDS\_DomainParticipantQos, [530](#)
  - DDS\_PublisherQos, [596](#)
  - DDS\_SubscriberQos, [624](#)
- ENTITY\_NAME, [126](#)
  - DDS\_ENTITYNAME\_QOS\_NAME\_MAX, [126](#)
  - set\_name, [127](#)
- err\_listener
  - DDS\_SecTransform\_Configuration, [622](#)
- Exception codes, [133](#)
- expression\_parameters
  - DDS\_ContentFilterProperty, [499](#)
  - DDS\_ContentFilterQosPolicy, [502](#)
- Extended Qos Support, [119](#)
- FILTER, [102](#)
  - DDS\_CONTENT\_FILTER\_CLASS\_NAME\_MAX\_LENGTH, [102](#)
  - DDS\_CONTENT\_FILTER\_EXPRESSION\_MAX\_LENGTH, [103](#)
  - DDS\_CONTENT\_FILTER\_MAX\_PARAMETERS, [103](#)
  - DDS\_CONTENT\_FILTER\_NAME\_MAX\_LENGTH, [102](#)
  - DDS\_FILTER\_PLUGIN\_FACTORY\_DEFAULT\_NAME, [103](#)
  - DDS\_FILTER\_RESOURCE\_LIMITS\_DEFAULT, [103](#)
- filter
  - DDS\_DomainParticipantQos, [531](#)
- filter\_class\_max\_count
  - DDS\_FilterResourceLimits, [547](#)
- filter\_class\_max\_length
  - DDS\_FilterResourceLimits, [547](#)
- filter\_class\_name
  - DDS\_ContentFilterProperty, [499](#)
  - DDS\_ContentFilterQosPolicy, [501](#)
- filter\_expression
  - DDS\_ContentFilterProperty, [499](#)
  - DDS\_ContentFilterQosPolicy, [502](#)
- filter\_expression\_max\_count
  - DDS\_FilterResourceLimits, [548](#)
- filter\_expression\_max\_length
  - DDS\_FilterResourceLimits, [547](#)
- filter\_name
  - DDS\_ContentFilterQosPolicy, [501](#)
- filter\_parameter\_max\_count\_per\_expression
  - DDS\_FilterResourceLimits, [548](#)
- filter\_parameter\_max\_length
  - DDS\_FilterResourceLimits, [548](#)
- finalize\_data
  - FooTypeSupport, [819](#)
- finalize\_instance
  - DDSDomainParticipantFactory, [712](#)
- find\_topic
  - DDSDomainParticipant, [693](#)
- flags
  - NETIO\_DGRAM\_InterfaceTableEntry, [833](#)
  - UDP\_InterfaceTableEntry, [857](#)
- FlatData Topic-Types, [53](#)
- Flow Controllers, [58](#)
  - DDS\_DEFAULT\_FLOW\_CONTROLLER\_NAME, [61](#)
  - DDS\_EDF\_FLOW\_CONTROLLER\_SCHED\_POLICY, [60](#)
  - DDS\_FIXED\_RATE\_FLOW\_CONTROLLER\_NAME, [62](#)
  - DDS\_FlowControllerSchedulingPolicy, [59](#)
  - DDS\_HPF\_FLOW\_CONTROLLER\_SCHED\_POLICY, [61](#)
  - DDS\_ON\_DEMAND\_FLOW\_CONTROLLER\_NAME, [62](#)
  - DDS\_RR\_FLOW\_CONTROLLER\_SCHED\_POLICY, [59](#)
- flow\_controller\_name
  - DDS\_PublishModeQosPolicy, [598](#)
- FooDataReader, [781](#)
  - is\_data\_consistent, [791](#)
  - lookup\_instance, [791](#)
  - narrow, [782](#)
  - read, [784](#)
  - read\_instance, [788](#)
  - read\_next\_sample, [783](#)
  - return\_loan, [782](#)
  - take, [785](#)
  - take\_instance, [790](#)
  - take\_next\_sample, [783](#)
- FooDataWriter, [792](#)
  - as\_datawriter, [794](#)
  - create\_data, [804](#)
  - delete\_data, [804](#)
  - discard\_loan, [803](#)
  - dispose, [799](#)
  - dispose\_w\_timestamp, [801](#)
  - get\_loan, [802](#)
  - narrow, [794](#)
  - register\_instance, [795](#)



- register\_instance\_w\_timestamp, 796
- unregister\_instance, 797
- unregister\_instance\_w\_timestamp, 799
- write, 794
- write\_w\_timestamp, 801
- FooSeq, 805
  - \_contiguous\_buffer, 816
  - \_element\_size, 817
  - \_flags, 817
  - \_length, 817
  - \_maximum, 816
  - \_token1, 817
  - \_token2, 817
  - ~FooSeq, 807
  - copy\_no\_alloc, 809
  - ensure\_length, 812
  - FooSeq, 807
  - from\_array, 809
  - get\_contiguous\_buffer, 816
  - has\_ownership, 816
  - length, 811, 812
  - loan\_contiguous, 814
  - max\_alloc\_size, 817
  - maximum, 813
  - operator=, 808
  - operator[], 811
  - to\_array, 810
  - unloan, 815
- FooTypeSupport, 818
  - copy\_data, 820
  - create\_data, 819
  - delete\_data, 819
  - deserialize\_data\_from\_cdr\_buffer, 823
  - finalize\_data, 819
  - initialize\_data, 818
  - register\_type, 820
  - serialize\_data\_to\_cdr\_buffer, 823
  - serialize\_data\_to\_cdr\_buffer\_ex, 822
  - unregister\_type, 821
- from\_array
  - FooSeq, 809
- General Utilities and Compliance Configuration, 139
- generate\_uuid
  - OSAPI\_SystemI, 842
- get\_conditions
  - DDSWaitSet, 770
- get\_contiguous\_buffer
  - FooSeq, 816
- get\_current\_time
  - DDSDomainParticipant, 705
- get\_default\_datareader\_qos
  - DDSSubscriber, 751
- get\_default\_datawriter\_qos
  - DDSPublisher, 740
- get\_default\_flowcontroller\_property
  - DDSDomainParticipant, 706
- get\_default\_participant\_qos
  - DDSDomainParticipantFactory, 716
- get\_default\_publisher\_qos
  - DDSDomainParticipant, 695
- get\_default\_subscriber\_qos
  - DDSDomainParticipant, 696
- get\_default\_topic\_qos
  - DDSDomainParticipant, 697
- get\_discovered\_participant\_data
  - DDSDomainParticipant, 705
- get\_discovered\_participants
  - DDSDomainParticipant, 705
- get\_enabled\_statuses
  - DDSStatusCondition, 747
- get\_hostname
  - OSAPI\_SystemI, 842
- get\_inconsistent\_topic\_status
  - DDSTopic, 762
- get\_instance
  - DDSDomainParticipantFactory, 711
- get\_instance\_handle
  - DDSEntity, 728
- get\_instance\_replaced\_missed\_status
  - DDSDataReader, 662
- get\_interface
  - DPDEDDiscoveryFactory, 776
  - DPSEDDiscoveryFactory, 779
  - RHSMHistoryFactory, 848
  - SHMEMInterfaceFactory, 852
  - UDPIInterfaceFactory, 859
  - WHSMHistoryFactory, 860
- get\_interface\_list
  - NETIO\_DGRAM\_Interfacel, 828
- get\_listener
  - DDSDataReader, 650
  - DDSDataWriter, 672
  - DDSDomainParticipant, 704
  - DDSPublisher, 744
  - DDSSubscriber, 756
  - DDSTopic, 762
- get\_liveliness\_changed\_status
  - DDSDataReader, 659
- get\_liveliness\_lost\_status
  - DDSDataWriter, 679
- get\_loan
  - FooDataWriter, 802
- get\_matched\_publication\_data
  - DDSDataReader, 660
- get\_matched\_publications
  - DDSDataReader, 660
- get\_matched\_subscription\_data
  - DDSDataReader, 660

- DDSDataWriter, [679](#)
- get\_matched\_subscriptions
  - DDSDataWriter, [679](#)
- get\_name
  - DDSFlowController, [730](#)
  - DDSTopic, [763](#)
  - DDSTopicDescription, [765](#)
- get\_offered\_deadline\_missed\_status
  - DDSDataWriter, [680](#)
- get\_offered\_incompatible\_qos\_status
  - DDSDataWriter, [680](#)
- get\_participant
  - DDSFlowController, [731](#)
  - DDSPublisher, [742](#)
  - DDSSubscriber, [755](#)
  - DDSTopic, [763](#)
  - DDSTopicDescription, [765](#)
- get\_property
  - DDSFlowController, [730](#)
- get\_publication\_matched\_status
  - DDSDataWriter, [678](#)
- get\_publisher
  - DDSDataWriter, [670](#)
- get\_qos
  - DDSDataReader, [649](#)
  - DDSDataWriter, [670](#)
  - DDSDomainParticipant, [694](#)
  - DDSDomainParticipantFactory, [716](#)
  - DDSPublisher, [743](#)
  - DDSSubscriber, [755](#)
  - DDSTopic, [761](#)
- get\_registry
  - DDSDomainParticipantFactory, [717](#)
- get\_requested\_deadline\_missed\_status
  - DDSDataReader, [661](#)
- get\_requested\_incompatible\_qos\_status
  - DDSDataReader, [661](#)
- get\_route\_table
  - NETIO\_DGRAM\_Interface, [830](#)
  - ZCOPY\_NotifUserInterface, [864](#)
- get\_sample\_lost\_status
  - DDSDataReader, [661](#)
- get\_sample\_rejected\_status
  - DDSDataReader, [660](#)
- get\_status\_changes
  - DDSEntity, [728](#)
- get\_statuscondition
  - DDSEntity, [728](#)
- get\_subscriber
  - DDSDataReader, [648](#)
- get\_subscription\_matched\_status
  - DDSDataReader, [659](#)
- get\_ticktime
  - OSAPI\_SystemI, [842](#)
- get\_time
  - OSAPI\_SystemI, [841](#)
- get\_timer\_resolution
  - OSAPI\_SystemI, [841](#)
- get\_topic
  - DDSDataWriter, [670](#)
- get\_topicdescription
  - DDSDataReader, [648](#)
- get\_type\_name
  - DDSTopic, [763](#)
  - DDSTopicDescription, [765](#)
- GROUP\_DATA, [130](#)
- group\_data
  - DDS\_PublicationBuiltinTopicData, [593](#)
  - DDS\_PublisherQos, [596](#)
  - DDS\_SubscriberQos, [624](#)
  - DDS\_SubscriptionBuiltinTopicData, [629](#)
- GUID Support, [85](#)
  - DDS\_GUID\_t, [85](#)
  - DDS\_GUID\_UNKNOWN, [85](#)
- handle
  - DDS\_WriteParams\_t, [644](#)
- has\_ownership
  - FooSeq, [816](#)
- heartbeat\_period
  - DDS\_RtpsReliableWriterProtocol\_t, [609](#)
- heartbeats\_per\_max\_samples
  - DDS\_RtpsReliableWriterProtocol\_t, [609](#)
- high
  - DDS\_SequenceNumber\_t, [623](#)
- HISTORY, [109](#)
  - DDS\_HistoryQosPolicyKind, [110](#)
  - DDS\_KEEP\_ALL\_HISTORY\_QOS, [110](#)
  - DDS\_KEEP\_LAST\_HISTORY\_QOS, [110](#)
- history
  - DDS\_DataReaderQos, [506](#)
  - DDS\_DataWriterQos, [516](#)
- hostname
  - OSAPI\_SystemProperty, [843](#)
- if\_table
  - UDP\_InterfaceFactoryProperty, [854](#)
- ifname
  - NETIO\_DGRAM\_InterfaceTableEntry, [834](#)
  - UDP\_InterfaceTableEntry, [858](#)
- inactive\_count
  - DDS\_ReliableReaderActivityChangedStatus, [601](#)
- inactive\_count\_change
  - DDS\_ReliableReaderActivityChangedStatus, [602](#)
- Infrastructure Module, [79](#)
  - DDS\_DATAREADER\_ENTITY\_KIND, [80](#)
  - DDS\_DATAWRITER\_ENTITY\_KIND, [80](#)
  - DDS\_EntityKind\_t, [80](#)
  - DDS\_PARTICIPANT\_ENTITY\_KIND, [80](#)

- DDS\_PUBLISHER\_ENTITY\_KIND, 80
- DDS\_SUBSCRIBER\_ENTITY\_KIND, 80
- DDS\_TOPIC\_ENTITY\_KIND, 80
- DDS\_UNKNOWN\_ENTITY\_KIND, 80
- DDS\_WRITEPARAMS\_DEFAULT, 80
- initial\_participant\_announcement\_period
  - DPDE\_DiscoveryPluginProperty, 772
  - DPSE\_DiscoveryPluginProperty, 777
- initial\_participant\_announcements
  - DPDE\_DiscoveryPluginProperty, 772
  - DPSE\_DiscoveryPluginProperty, 777
- initial\_peers
  - DDS\_DiscoveryQosPolicy, 526
- initialize\_data
  - FooTypeSupport, 818
- initialize\_writer\_loaned\_sample
  - DDS\_DataWriterResourceLimitsQosPolicy, 519
- Instance States, 76
  - DDS\_ALIVE\_INSTANCE\_STATE, 78
  - DDS\_ANY\_INSTANCE\_STATE, 78
  - DDS\_InstanceStateKind, 77
  - DDS\_InstanceStateMask, 77
  - DDS\_NOT\_ALIVE\_DISPOSED\_INSTANCE\_STATE, 78
  - DDS\_NOT\_ALIVE\_INSTANCE\_STATE, 78
  - DDS\_NOT\_ALIVE\_NO\_WRITERS\_INSTANCE\_STATE, 78
- instance\_handle
  - DDS\_ReliableSampleUnacknowledgedStatus, 603
  - DDS\_SampleInfo, 617
- instance\_replacement
  - DDS\_DataReaderResourceLimitsQosPolicy, 510
- instance\_state
  - DDS\_SampleInfo, 617
- intf\_addr
  - ZCOPY\_NotifMechanismProperty, 862
- is\_data\_consistent
  - FooDataReader, 791
- is\_data\_consistent\_untyped
  - DDSDDataReader, 657
- is\_default\_interface
  - UDP\_InterfaceFactoryProperty, 855
- key
  - DDS\_ParticipantBuiltinTopicData, 575
  - DDS\_PublicationBuiltinTopicData, 591
  - DDS\_SubscriptionBuiltinTopicData, 626
- kind
  - DDS\_DestinationOrderQosPolicy, 524
  - DDS\_DurabilityQosPolicy, 541
  - DDS\_HistoryQosPolicy, 556
  - DDS\_LivelinessQosPolicy, 564
  - DDS\_Locator\_t, 565
  - DDS\_OwnershipQosPolicy, 572
  - DDS\_PublishModeQosPolicy, 598
  - DDS\_ReliabilityQosPolicy, 600
  - NETIO\_Address, 825
- last\_instance\_handle
  - DDS\_DataReaderInstanceReplacedStatus, 503
  - DDS\_OfferedDeadlineMissedStatus, 566
  - DDS\_ReliableReaderActivityChangedStatus, 602
  - DDS\_RequestedDeadlineMissedStatus, 604
  - DDS\_SampleRejectedStatus, 621
- last\_policy\_id
  - DDS\_OfferedIncompatibleQosStatus, 568
  - DDS\_RequestedIncompatibleQosStatus, 605
- last\_publication\_handle
  - DDS\_LivelinessChangedStatus, 560
  - DDS\_SubscriptionMatchedStatus, 630
- last\_reason
  - DDS\_SampleRejectedStatus, 621
- last\_replacement\_instance
  - DDS\_DataReaderInstanceReplacedStatus, 503
- last\_subscription\_handle
  - DDS\_PublicationMatchedStatus, 595
- LATENCY\_BUDGET, 105
- latency\_budget
  - DDS\_PublicationBuiltinTopicData, 592
  - DDS\_SubscriptionBuiltinTopicData, 627
- lease\_duration
  - DDS\_LivelinessQosPolicy, 564
- length
  - FooSeq, 811, 812
- Lightweight Security API, 226
  - DDS\_PSK\_DEFAULT\_PLUGIN\_NAME, 227
  - DDS\_PskServiceFactoryProperty\_INITIALIZER, 226
- LIVELINESS, 107
  - DDS\_AUTOMATIC\_LIVELINESS\_QOS, 108
  - DDS\_LivelinessQosPolicyKind, 107
  - DDS\_MANUAL\_BY\_PARTICIPANT\_LIVELINESS\_QOS, 108
  - DDS\_MANUAL\_BY\_TOPIC\_LIVELINESS\_QOS, 108
- liveliness
  - DDS\_DataReaderQos, 506
  - DDS\_DataWriterQos, 515
  - DDS\_PublicationBuiltinTopicData, 592
  - DDS\_SubscriptionBuiltinTopicData, 627
- liveliness\_lease\_duration
  - DDS\_ParticipantBuiltinTopicData, 576
- loan\_contiguous
  - FooSeq, 814
- local\_publisher\_allocation, 229
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 534
- local\_reader\_allocation, 230

- DDS\_DomainParticipantResourceLimitsQosPolicy, 534
- local\_subscriber\_allocation, 230
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 534
- local\_topic\_allocation, 229
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 534
- local\_type\_allocation, 229
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 534
- local\_writer\_allocation, 230
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 534
- locator\_kind
  - NETIO\_DGRAM\_InterfaceTableEntry, 833
- Log API, 219
  - OSAPI\_Log\_clear, 223
  - OSAPI\_Log\_get\_display\_handler, 222
  - OSAPI\_Log\_get\_last\_error\_code, 224
  - OSAPI\_Log\_get\_log\_handler, 221
  - OSAPI\_Log\_get\_property, 223
  - OSAPI\_Log\_set\_display\_handler, 222
  - OSAPI\_Log\_set\_log\_handler, 221
  - OSAPI\_Log\_set\_property, 223
  - OSAPI\_Log\_set\_verbosity, 222
  - OSAPI\_LOG\_VERBOSITY\_DEBUG, 221
  - OSAPI\_LOG\_VERBOSITY\_ERROR, 221
  - OSAPI\_LOG\_VERBOSITY\_SILENT, 221
  - OSAPI\_LOG\_VERBOSITY\_WARNING, 221
  - OSAPI\_Log\_write\_buffer\_T, 220
  - OSAPI\_LogHandler\_T, 220
  - OSAPI\_LogVerbosity\_T, 221
- Log Codes, 245
- log\_detail
  - OSAPI\_LogProperty, 839
- lookup\_datareader
  - DDSSubscriber, 752
- lookup\_datareader\_by\_name
  - DDSDomainParticipant, 701
  - DDSSubscriber, 753
- lookup\_datawriter
  - DDSPublisher, 741
- lookup\_datawriter\_by\_name
  - DDSDomainParticipant, 700
  - DDSPublisher, 742
- lookup\_flowcontroller
  - DDSDomainParticipant, 709
- lookup\_instance
  - FooDataReader, 791
- lookup\_instance\_untyped
  - DDSDataReader, 657
- lookup\_participant
  - DDSDomainParticipantFactory, 714
- lookup\_participant\_by\_name
  - DDSDomainParticipantFactory, 715
- lookup\_property
  - DDSPROPERTYQosPolicyHelper, 736
- lookup\_publisher\_by\_name
  - DDSDomainParticipant, 699
- lookup\_subscriber\_by\_name
  - DDSDomainParticipant, 700
- lookup\_topicdescription
  - DDSDomainParticipant, 699
- lost\_samples
  - DDS\_DataReaderInstanceReplacedStatus, 503
- low
  - DDS\_SequenceNumber\_t, 623
- major
  - DDS\_ProductVersion, 583
  - DDS\_ProductVersion\_t, 585
  - DDS\_ProtocolVersion, 588
- majorRev
  - NDDS\_Type\_PluginVersion, 824
- mask
  - NETIO\_Netmask, 836
- matching\_reader\_writer\_pair\_allocation
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 535
- matching\_writer\_reader\_pair\_allocation, 230
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 535
- max\_alloc\_size
  - FooSeq, 817
- max\_blocking\_time
  - DDS\_ReliabilityQosPolicy, 600
- max\_buffer\_size, 241
  - OSAPI\_LogProperty, 839
- max\_components, 228
  - DDS\_SystemResourceLimitsQosPolicy, 632
- max\_destination\_ports, 232
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 536
- max\_fragmented\_samples, 238
  - DDS\_DataReaderResourceLimitsQosPolicy, 511
- max\_fragmented\_samples\_per\_remote\_writer, 238
  - DDS\_DataReaderResourceLimitsQosPolicy, 511
- max\_heartbeat\_retries
  - DDS\_RtpsReliableWriterProtocol\_t, 610
- max\_instances, 235, 240
  - DDS\_ResourceLimitsQosPolicy, 607
- max\_locators\_per\_discovered\_participant, 243, 244
  - DPDE\_DiscoveryPluginProperty, 773
  - DPSE\_DiscoveryPluginProperty, 778
- max\_message\_size, 241
  - UDP\_InterfaceFactoryProperty, 854
- max\_outstanding\_reads, 238

- DDS\_DataReaderResourceLimitsQosPolicy, 510
- max\_participant\_locators, 242, 243
  - DPDE\_DiscoveryPluginProperty, 773
  - DPSE\_DiscoveryPluginProperty, 778
- max\_participants, 228
  - DDS\_SystemResourceLimitsQosPolicy, 631
- max\_partition\_cumulative\_characters
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 539
- max\_partition\_string\_allocation
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 540
- max\_partition\_string\_size
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 539
- max\_partitions
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 539
- max\_receive\_buffer\_size
  - UDP\_InterfaceFactoryProperty, 854
- max\_receive\_ports, 233
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 536
  - ZCOPY\_NotifMechanismProperty, 862
- max\_remote\_reader\_filters
  - DDS\_DataWriterResourceLimitsQosPolicy, 520
- max\_remote\_readers, 241
  - DDS\_DataWriterResourceLimitsQosPolicy, 519
- max\_remote\_writers, 237
  - DDS\_DataReaderResourceLimitsQosPolicy, 509
- max\_remote\_writers\_per\_instance, 235
  - DDS\_DataReaderResourceLimitsQosPolicy, 509
- max\_routes
  - ZCOPY\_NotifMechanismProperty, 862
- max\_routes\_per\_reader, 240
  - DDS\_DataWriterResourceLimitsQosPolicy, 519
- max\_routes\_per\_writer, 237
  - DDS\_DataReaderResourceLimitsQosPolicy, 510
- max\_samples, 236, 239
  - DDS\_ResourceLimitsQosPolicy, 607
- max\_samples\_per\_builtin\_endpoint\_reader, 244
  - DPDE\_DiscoveryPluginProperty, 773
- max\_samples\_per\_instance, 236, 240
  - DDS\_ResourceLimitsQosPolicy, 607
- max\_samples\_per\_notif
  - ZCOPY\_NotifInterfaceFactoryProperty, 861
- max\_samples\_per\_remote\_builtin\_endpoint\_writer
  - DPDE\_DiscoveryPluginProperty, 774
- max\_samples\_per\_remote\_writer, 237
  - DDS\_DataReaderResourceLimitsQosPolicy, 510
- max\_send\_buffer\_size
  - UDP\_InterfaceFactoryProperty, 854
- max\_send\_message\_size
  - UDP\_InterfaceFactoryProperty, 854
- max\_send\_window
  - DDS\_RtpsReliableWriterProtocol\_t, 610
- max\_timers
  - OSAPI\_SystemProperty, 843
- max\_tokens
  - DDS\_FlowControllerTokenBucketProperty\_t, 551
- max\_user\_blocking\_threads
  - OSAPI\_SystemProperty, 843
- maximum
  - FooSeq, 813
- message\_size\_max
  - NETIO\_SHMEMInterfaceFactoryProperty, 838
- metattraffic\_multicast\_locators
  - DDS\_ParticipantBuiltinTopicData, 576
- metattraffic\_unicast\_locators
  - DDS\_ParticipantBuiltinTopicData, 576
- minor
  - DDS\_ProductVersion, 583
  - DDS\_ProductVersion\_t, 585
  - DDS\_ProtocolVersion, 588
- minorRev
  - NDDS\_Type\_PluginVersion, 824
- mode
  - OSAPI\_SharedMemorySegmentHandle, 840
  - OSAPI\_SharedMemorySegmentHeader, 840
- mtu
  - NETIO\_DGRAM\_InterfaceTableEntry, 833
- multicast\_group
  - NETIO\_DGRAM\_InterfaceTableEntry, 834
- multicast\_interface
  - UDP\_InterfaceFactoryProperty, 855
- multicast\_locator
  - DDS\_SubscriptionBuiltinTopicData, 628
- multicast\_loopback\_disabled
  - UDP\_InterfaceFactoryProperty, 855
- multicast\_ttl
  - UDP\_InterfaceFactoryProperty, 854
- nack\_period
  - DDS\_RtpsReliableReaderProtocol\_t, 608
- name
  - DDS\_DiscoveryComponent, 525
  - DDS\_EntityNameQosPolicy, 544
  - DDS\_FilterQosPolicy, 545
  - DDS\_PartitionQosPolicy, 579
  - DDS\_Property\_t, 586
- nanosec
  - DDS\_Duration\_t, 541
  - DDS\_Time\_t, 633
  - OSAPI\_SystemTime, 845
- narrow
  - DDSTopic, 760
  - FooDataReader, 782
  - FooDataWriter, 794

## nat

- UDP\_InterfaceFactoryProperty, [854](#)
- NDDS\_Discovery\_Property, [824](#)
- NDDS\_Type\_PluginVersion, [824](#)
  - majorRev, [824](#)
  - minorRev, [824](#)
  - Type Support, [134](#)
- NDDS\_TYPEPLUGIN\_NO\_KEY
  - Type Support, [135](#)
- NDDS\_TYPEPLUGIN\_USER\_KEY
  - Type Support, [135](#)
- NDDS\_TypePluginKeyKind
  - Type Support, [135](#)
- NETIO, [279](#)
  - NETIO\_LOG\_AR\_ADD\_INVALID\_INTERFACE\_EC, [287](#)
  - NETIO\_LOG\_AR\_ADDRESS\_RESOLVE\_FAILED\_EC, [288](#)
  - NETIO\_LOG\_AR\_ADDRESS\_STRING\_EXCEEDED\_EC, [288](#)
  - NETIO\_LOG\_AR\_ALLOC\_FAILED\_EC, [287](#)
  - NETIO\_LOG\_AR\_CONTEXT\_UNINITIALIZED\_EC, [288](#)
  - NETIO\_LOG\_AR\_INVALID\_TOKEN\_EC, [288](#)
  - NETIO\_LOG\_AR\_PORT\_RESOLVE\_FAILED\_EC, [288](#)
  - NETIO\_LOG\_BIND\_CREATE\_RTABLE\_EC, [286](#)
  - NETIO\_LOG\_BIND\_DELETE\_RTABLE\_EC, [286](#)
  - NETIO\_LOG\_BIND\_EXTERNAL\_FAILED\_EC, [286](#)
  - NETIO\_LOG\_BIND\_NEW\_EC, [285](#)
  - NETIO\_LOG\_BIND\_NEW\_RTABLE\_EC, [286](#)
  - NETIO\_LOG\_BIND\_SET\_LENGTH\_EC, [286](#)
  - NETIO\_LOG\_BIND\_SET\_MAX\_EC, [286](#)
  - NETIO\_LOG\_GETHOST\_BYNAME\_EC, [285](#)
  - NETIO\_LOG\_ILLEGAL\_ROUTE\_OPERATION\_EC, [288](#)
  - NETIO\_LOG\_INVALID\_ADDRESS\_INDEX\_EC, [288](#)
  - NETIO\_LOG\_LOOP\_BINDX\_DB\_EC, [290](#)
  - NETIO\_LOG\_LOOP\_CREATE\_TABLE\_EC, [289](#)
  - NETIO\_LOG\_LOOP\_CURSOR\_ERROR\_EC, [291](#)
  - NETIO\_LOG\_LOOP\_DELETE\_TABLE\_EC, [289](#)
  - NETIO\_LOG\_LOOP\_FWD\_EC, [290](#)
  - NETIO\_LOG\_LOOP\_INITIALIZE\_FAILED\_EC, [290](#)
  - NETIO\_LOG\_LOOP\_INVALID\_COMPONENT\_EC, [291](#)
  - NETIO\_LOG\_LOOP\_INVALID\_FACTORY\_EC, [291](#)
  - NETIO\_LOG\_LOOP\_INVALID\_PROPERTY\_EC, [290](#)
  - NETIO\_LOG\_LOOP\_SELECT\_TABLE\_EC, [290](#)
  - NETIO\_LOG\_LOOP\_SET\_LENGTH\_EC, [290](#)
  - NETIO\_LOG\_LOOP\_UNBINDX\_DB\_EC, [290](#)
  - NETIO\_LOG\_PACKET\_INITIALIZE\_EC, [289](#)
  - NETIO\_LOG\_PACKET\_SET\_HEAD\_EC, [289](#)
  - NETIO\_LOG\_PACKET\_SET\_TAIL\_EC, [289](#)
  - NETIO\_LOG\_ROUTE\_RTABLE\_ADD\_EC, [287](#)
  - NETIO\_LOG\_ROUTE\_RTABLE\_ALLOC\_EC, [287](#)
  - NETIO\_LOG\_ROUTE\_RTABLE\_CREATE\_EC, [287](#)
  - NETIO\_LOG\_ROUTE\_RTABLE\_DELETE\_EC, [287](#)
  - NETIO\_LOG\_ROUTE\_RTABLE\_UPDATE\_EC, [287](#)
  - NETIO\_LOG\_SET\_NAME\_EC, [289](#)
  - NETIO\_LOG\_UNBIND\_EXTERNAL\_FAILED\_EC, [286](#)
  - NETIO\_SHMEM\_LOG\_ADDRESS\_IN\_USE\_EC, [302](#)
  - NETIO\_SHMEM\_LOG\_BINDX\_DB\_EC, [300](#)
  - NETIO\_SHMEM\_LOG\_CHECK\_SYSTEM\_SETTINGS\_EC, [302](#)
  - NETIO\_SHMEM\_LOG\_CREATE\_ATTACH\_INFINITE\_EC, [303](#)
  - NETIO\_SHMEM\_LOG\_CREATE\_TABLE\_EC, [300](#)
  - NETIO\_SHMEM\_LOG\_CURSOR\_ERROR\_EC, [301](#)
  - NETIO\_SHMEM\_LOG\_DELETE\_TABLE\_EC, [300](#)
  - NETIO\_SHMEM\_LOG\_FAILED\_TAKE\_SIGN\_SEM\_EC, [303](#)
  - NETIO\_SHMEM\_LOG\_FAILED\_TO\_ALLOCATE\_STRUCT\_EC, [302](#)
  - NETIO\_SHMEM\_LOG\_FAILED\_TO\_ATTACH\_CONCURRENT\_Q\_EC, [304](#)
  - NETIO\_SHMEM\_LOG\_FAILED\_TO\_GET\_TIMESTAMP\_EC, [304](#)
  - NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_CONCURRENT\_Q\_EC, [304](#)
  - NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_MUTEX\_EC, [303](#)
  - NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_SEGMENT\_ADDR\_EC, [303](#)
  - NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_SIG\_SEM\_EC, [303](#)
  - NETIO\_SHMEM\_LOG\_FAILED\_TO\_INITIALIZE\_EC, [303](#)
  - NETIO\_SHMEM\_LOG\_FWD\_EC, [300](#)
  - NETIO\_SHMEM\_LOG\_IGNORE\_TRANSPORT\_COMPATIBILITY\_EC, [305](#)
  - NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_CONCURRENT\_Q\_EC, [304](#)
  - NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_COOKIE\_EC, [301](#)
  - NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_HEADER\_EC, [302](#)
  - NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_HEADER\_SIZE\_EC, [302](#)
  - NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_VERSION\_EC, [301](#)
  - NETIO\_SHMEM\_LOG\_INCONSISTENT\_MESSAGE\_SIZE\_EC, [304](#)
  - NETIO\_SHMEM\_LOG\_INITIALIZE\_FAILED\_EC, [301](#)
  - NETIO\_SHMEM\_LOG\_INVALID\_COMPONENT\_EC,



- 301
- NETIO\_SHMEM\_LOG\_INVALID\_FACTORY\_EC, 301
- NETIO\_SHMEM\_LOG\_INVALID\_PROPERTY\_EC, 301
- NETIO\_SHMEM\_LOG\_NONFATAL\_SIG\_SEM\_RC\_EC, 304
- NETIO\_SHMEM\_LOG\_PACKET\_FWD\_EC, 305
- NETIO\_SHMEM\_LOG\_PACKET\_HEAD\_EC, 305
- NETIO\_SHMEM\_LOG\_PACKET\_INIT\_EC, 304
- NETIO\_SHMEM\_LOG\_QUEUE\_FULL\_EC, 305
- NETIO\_SHMEM\_LOG\_RECORD\_EC, 305
- NETIO\_SHMEM\_LOG\_ROUTE\_DELETE\_FAILED\_EC, 305
- NETIO\_SHMEM\_LOG\_ROUTE\_LOOKUP\_FAILED\_EC, 305
- NETIO\_SHMEM\_LOG\_SELECT\_TABLE\_EC, 300
- NETIO\_SHMEM\_LOG\_SET\_LENGTH\_EC, 300
- NETIO\_SHMEM\_LOG\_SHMEM\_MUTEX\_LOCK\_FAILED\_EC, 302
- NETIO\_SHMEM\_LOG\_SHMEM\_MUTEX\_UNLOCK\_FAILED\_EC, 302
- NETIO\_SHMEM\_LOG\_SIG\_SEM\_SIGNAL\_FAILED\_EC, 303
- NETIO\_SHMEM\_LOG\_UNBINDX\_DB\_EC, 300
- NETIO\_SHMEM\_LOG\_UNSUPPORTED\_TRANSPORT\_COMPATIBILITY\_EC, 306
- UDP\_LOG\_ALLOC\_EC, 296
- UDP\_LOG\_CURSOR\_ERROR\_EC, 297
- UDP\_LOG\_DROPGRP\_EC, 296
- UDP\_LOG\_FINALIZE\_EC, 293
- UDP\_LOG\_GET\_BUF\_ALLOC\_EC, 295
- UDP\_LOG\_GET\_BUF\_SIZE\_EC, 295
- UDP\_LOG\_GET\_IFLIST\_EC, 295
- UDP\_LOG\_GET\_LENGTH\_EC, 296
- UDP\_LOG\_GET\_NAMEINFO\_EC, 295
- UDP\_LOG\_GETHOSTNAME\_EC, 291
- UDP\_LOG\_INCONSISTENT\_MAX\_MESSAGE\_SIZE\_EC, 297
- UDP\_LOG\_INITIALIZE\_FAILED\_EC, 297
- UDP\_LOG\_LOADLIB\_EC, 294
- UDP\_LOG\_MULTICAST\_ENABLE\_EC, 295
- UDP\_LOG\_MULTICAST\_NOT\_ENABLED\_EC, 298
- UDP\_LOG\_NAT\_DELETE\_EC, 293
- UDP\_LOG\_PACKET\_FWD\_EC, 292
- UDP\_LOG\_PACKET\_HEAD\_EC, 292
- UDP\_LOG\_PACKET\_INIT\_EC, 292
- UDP\_LOG\_PORT\_ADD\_EC, 294
- UDP\_LOG\_PORT\_ALLOC\_EC, 293
- UDP\_LOG\_PORT\_ENTRY\_EC, 293
- UDP\_LOG\_PORT\_NOT\_FOUND\_EC, 297
- UDP\_LOG\_PORT\_THREAD\_EC, 294
- UDP\_LOG\_PROPERTY\_FINALIZE\_EC, 296
- UDP\_LOG\_RECORD\_EC, 296
- UDP\_LOG\_RECV\_ERROR\_EC, 298
- UDP\_LOG\_SEND\_ERROR\_EC, 297
- UDP\_LOG\_SEND\_PING\_EC, 296
- UDP\_LOG\_SET\_LENGTH\_EC, 296
- UDP\_LOG\_SOCKET\_ADDGRP\_EC, 293
- UDP\_LOG\_SOCKET\_BIND\_EC, 293
- UDP\_LOG\_SOCKET\_CLOSE\_EC, 294
- UDP\_LOG\_SOCKET\_CREATE\_EC, 292
- UDP\_LOG\_SOCKET\_IFFLAGS\_EC, 294
- UDP\_LOG\_SOCKET\_IFLIST\_EC, 294
- UDP\_LOG\_SOCKET\_INIT\_EC, 291
- UDP\_LOG\_SOCKET\_REUSE\_PORT\_EC, 293
- UDP\_LOG\_SOCKET\_RXBUF\_EC, 294
- UDP\_LOG\_SOCKET\_SET\_MCAST\_EC, 291
- UDP\_LOG\_SOCKET\_SET\_MCASTIF\_EC, 291
- UDP\_LOG\_SOCKET\_SETFD\_EC, 292
- UDP\_LOG\_SOCKET\_SNDBUF\_EC, 292
- UDP\_LOG\_SOCKET\_TTL\_EC, 292
- UDP\_LOG\_TRANSPORT\_NO\_INTERFACES\_EC, 298
- UDP\_LOG\_UDP\_CREATE\_INDEX\_EC, 295
- UDP\_LOG\_UDP\_CREATE\_TABLE\_EC, 295
- UDP\_LOG\_WINSOCK\_INCOMPATIBLE\_EC, 297
- UDP\_LOG\_WSA\_STARTUP\_EC, 297
- UDP\_TRANSFORM\_LOG\_ADD\_EC, 299
- UDP\_TRANSFORM\_LOG\_ALLOC\_EC, 298
- UDP\_TRANSFORM\_LOG\_CREATE\_EC, 299
- UDP\_TRANSFORM\_LOG\_DUPLICATE\_EC, 299
- UDP\_TRANSFORM\_LOG\_FACTORY\_NOT\_FOUND\_EC, 298
- UDP\_TRANSFORM\_LOG\_FIND\_EC, 299
- UDP\_TRANSFORM\_LOG\_INVALID\_FACTORY\_REFERENCE\_EC, 298
- UDP\_TRANSFORM\_LOG\_INVALID\_TUDP\_LOCATOR\_KIND\_EC, 299
- UDP\_TRANSFORM\_LOG\_INVALID\_UDP\_MODE\_EC, 299
- UDP\_TRANSFORM\_LOG\_TRANSFORM\_EC, 299
- NETIO Address API, 149
  - NETIO\_Address\_INITIALIZER, 150
  - NETIO\_ADDRESS\_KIND\_UDPv4, 150
  - NETIO\_ADDRESS\_KIND\_UDPv6, 150
  - NETIO\_ADDRESS\_RTPS\_RESERVED\_HIGH, 150
  - NETIO\_ADDRESS\_RTPS\_RESERVED\_LOW, 150
  - NETIO\_Netmask\_INITIALIZER, 150
- NETIO API, 146
- NETIO DGRAM API, 178
  - NETIO\_ADDRESS\_UDPv4\_ANY\_MULTICAST\_ROUTE, 184
  - NETIO\_ADDRESS\_UDPv4\_ANY\_UNICAST\_ROUTE, 184
  - NETIO\_ADDRESS\_UDPv6\_ANY\_MULTICAST\_ROUTE, 184

- NETIO\_ADDRESS\_UDPv6\_ANY\_UNICAST\_ROUTE, 184
- NETIO\_DGRAM\_Interface\_createFunc, 182
- NETIO\_DGRAM\_INTERFACE\_MAX\_ADDRESSES, 180
- NETIO\_DGRAM\_INTERFACE\_MAX\_INTERFACES, 180
- NETIO\_DGRAM\_INTERFACE\_SHARED\_PORT\_FLAG, 180
- NETIO\_DGRAM\_InterfaceFactory\_register, 183
- NETIO\_DGRAM\_InterfaceI, 183
- NETIO\_DGRAM\_InterfaceMultiCastGroup\_INITIALIZER, 180
- NETIO\_DGRAM\_InterfaceMultiCastGroup\_Invalid, 181
- NETIO\_DGRAM\_InterfaceMultiCastGroup\_UDPv4, 181
- NETIO\_DGRAM\_InterfaceMultiCastGroup\_UDPv6, 181
- NETIO\_DGRAM\_InterfaceTableEntry\_INITIALIZER, 181
- NETIO\_DGRAM\_InterfaceTableEntrySeq\_INITIALIZER, 182
- NETIO\_DGRAMUserInterface\_bind\_addressFunc, 182
- NETIO\_DGRAMUserInterface\_deleteFunc, 183
- NETIO\_DGRAMUserInterface\_get\_interface\_listFunc, 182
- NETIO\_DGRAMUserInterface\_resolve\_addressFunc, 182
- NETIO Interface API, 147
  - NETIO\_Interface\_receiveFunc, 147
  - NETIO\_Interface\_reserve\_addressFunc, 147
  - NETIO\_Interface\_T, 147
- NETIO Packet API, 146
  - NETIO\_Packet\_T, 146
- NETIO Shared Memory API, 159
- NETIO\_Address, 825
  - kind, 825
  - port, 825
  - value, 825
- NETIO\_Address\_INITIALIZER
  - NETIO Address API, 150
- NETIO\_ADDRESS\_KIND\_UDPv4
  - NETIO Address API, 150
- NETIO\_ADDRESS\_KIND\_UDPv6
  - NETIO Address API, 150
- NETIO\_ADDRESS\_RTPS\_RESERVED\_HIGH
  - NETIO Address API, 150
- NETIO\_ADDRESS\_RTPS\_RESERVED\_LOW
  - NETIO Address API, 150
- NETIO\_ADDRESS\_UDPv4\_ANY\_MULTICAST\_ROUTE
  - NETIO DGRAM API, 184
- NETIO\_ADDRESS\_UDPv4\_ANY\_UNICAST\_ROUTE
  - NETIO DGRAM API, 184
- NETIO\_ADDRESS\_UDPv6\_ANY\_MULTICAST\_ROUTE
  - NETIO DGRAM API, 184
- NETIO\_ADDRESS\_UDPv6\_ANY\_UNICAST\_ROUTE
  - NETIO DGRAM API, 184
- NETIO\_AddressSeq, 826
- NETIO\_AddressUInt32, 826
  - value, 826
- NETIO\_AddressValue, 826
  - address, 827
  - as\_uint32, 827
- NETIO\_DEFAULT\_NOTIF\_NAME
  - Notification Interface, 189
- NETIO\_DGRAM\_Interface\_createFunc
  - NETIO DGRAM API, 182
- NETIO\_DGRAM\_INTERFACE\_MAX\_ADDRESSES
  - NETIO DGRAM API, 180
- NETIO\_DGRAM\_INTERFACE\_MAX\_INTERFACES
  - NETIO DGRAM API, 180
- NETIO\_DGRAM\_INTERFACE\_SHARED\_PORT\_FLAG
  - NETIO DGRAM API, 180
- NETIO\_DGRAM\_InterfaceFactory\_register
  - NETIO DGRAM API, 183
- NETIO\_DGRAM\_InterfaceI, 827
  - bind\_address, 830
  - create\_instance, 828
  - delete\_instance, 828
  - get\_interface\_list, 828
  - get\_route\_table, 830
  - NETIO DGRAM API, 183
  - release\_address, 829
  - resolve\_address, 829
  - send, 829
- NETIO\_DGRAM\_InterfaceMultiCastGroup, 831
  - address\_high, 831
  - address\_low, 831
  - netmask, 831
- NETIO\_DGRAM\_InterfaceMultiCastGroup\_INITIALIZER
  - NETIO DGRAM API, 180
- NETIO\_DGRAM\_InterfaceMultiCastGroup\_Invalid
  - NETIO DGRAM API, 181
- NETIO\_DGRAM\_InterfaceMultiCastGroup\_UDPv4
  - NETIO DGRAM API, 181
- NETIO\_DGRAM\_InterfaceMultiCastGroup\_UDPv6
  - NETIO DGRAM API, 181
- NETIO\_DGRAM\_InterfaceRouteEntry, 832
  - address, 832
  - netmask, 832
- NETIO\_DGRAM\_InterfaceTableEntry, 833
  - address, 834
  - flags, 833
  - ifname, 834
  - locator\_kind, 833
  - mtu, 833



- multicast\_group, [834](#)
- NETIO\_DGRAM\_InterfaceTableEntry\_INITIALIZER
  - NETIO DGRAM API, [181](#)
- NETIO\_DGRAM\_InterfaceTableEntrySeq, [834](#)
- NETIO\_DGRAM\_InterfaceTableEntrySeq\_INITIALIZER
  - NETIO DGRAM API, [182](#)
- NETIO\_DGRAMUserInterface\_bind\_addressFunc
  - NETIO DGRAM API, [182](#)
- NETIO\_DGRAMUserInterface\_deleteFunc
  - NETIO DGRAM API, [183](#)
- NETIO\_DGRAMUserInterface\_get\_interface\_listFunc
  - NETIO DGRAM API, [182](#)
- NETIO\_DGRAMUserInterface\_resolve\_addressFunc
  - NETIO DGRAM API, [182](#)
- NETIO\_Interface, [835](#)
- NETIO\_Interface\_receiveFunc
  - NETIO Interface API, [147](#)
- NETIO\_Interface\_reserve\_addressFunc
  - NETIO Interface API, [147](#)
- NETIO\_Interface\_T
  - NETIO Interface API, [147](#)
- NETIO\_InterfaceI, [835](#)
- netio\_log.h, [911](#)
  - UDP\_TRANSFORM\_LOG\_KIND\_DST\_INDEX, [917](#)
  - UDP\_TRANSFORM\_LOG\_KIND\_ENTRY, [916](#)
  - UDP\_TRANSFORM\_LOG\_KIND\_FACTORY, [916](#)
  - UDP\_TRANSFORM\_LOG\_KIND\_FACTORY\_INDEX, [917](#)
  - UDP\_TRANSFORM\_LOG\_KIND\_RULE, [916](#)
  - UDP\_TRANSFORM\_LOG\_KIND\_SRC\_INDEX, [917](#)
  - UDP\_TRANSFORM\_LOG\_KIND\_TRANSFORM\_DESTINATION, [917](#)
  - UDP\_TRANSFORM\_LOG\_KIND\_TRANSFORM\_NETWORK\_INDEX, [917](#)
  - UDP\_TRANSFORM\_LOG\_KIND\_TRANSFORM\_SOURCE, [917](#)
- NETIO\_LOG\_AR\_ADD\_INVALID\_INTERFACE\_EC
  - NETIO, [287](#)
- NETIO\_LOG\_AR\_ADDRESS\_RESOLVE\_FAILED\_EC
  - NETIO, [288](#)
- NETIO\_LOG\_AR\_ADDRESS\_STRING\_EXCEEDED\_EC
  - NETIO, [288](#)
- NETIO\_LOG\_AR\_ALLOC\_FAILED\_EC
  - NETIO, [287](#)
- NETIO\_LOG\_AR\_CONTEXT\_UNINITIALIZED\_EC
  - NETIO, [288](#)
- NETIO\_LOG\_AR\_INVALID\_TOKEN\_EC
  - NETIO, [288](#)
- NETIO\_LOG\_AR\_PORT\_RESOLVE\_FAILED\_EC
  - NETIO, [288](#)
- NETIO\_LOG\_BASE
  - osapi\_log.h, [930](#)
- NETIO\_LOG\_BIND\_CREATE\_RTABLE\_EC
  - NETIO, [286](#)
- NETIO\_LOG\_BIND\_DELETE\_RTABLE\_EC
  - NETIO, [286](#)
- NETIO\_LOG\_BIND\_EXTERNAL\_FAILED\_EC
  - NETIO, [286](#)
- NETIO\_LOG\_BIND\_NEW\_EC
  - NETIO, [285](#)
- NETIO\_LOG\_BIND\_NEW\_RTABLE\_EC
  - NETIO, [286](#)
- NETIO\_LOG\_BIND\_SET\_LENGTH\_EC
  - NETIO, [286](#)
- NETIO\_LOG\_BIND\_SET\_MAX\_EC
  - NETIO, [286](#)
- NETIO\_LOG\_GETHOST\_BYNAME\_EC
  - NETIO, [285](#)
- NETIO\_LOG\_ILLEGAL\_ROUTE\_OPERATION\_EC
  - NETIO, [288](#)
- NETIO\_LOG\_INVALID\_ADDRESS\_INDEX\_EC
  - NETIO, [288](#)
- NETIO\_LOG\_LOOP\_BINDX\_DB\_EC
  - NETIO, [290](#)
- NETIO\_LOG\_LOOP\_CREATE\_TABLE\_EC
  - NETIO, [289](#)
- NETIO\_LOG\_LOOP\_CURSOR\_ERROR\_EC
  - NETIO, [291](#)
- NETIO\_LOG\_LOOP\_DELETE\_TABLE\_EC
  - NETIO, [289](#)
- NETIO\_LOG\_LOOP\_FWD\_EC
  - NETIO, [290](#)
- NETIO\_LOG\_LOOP\_INITIALIZE\_FAILED\_EC
  - NETIO, [290](#)
- NETIO\_LOG\_LOOP\_INVALID\_COMPONENT\_EC
  - NETIO, [291](#)
- NETIO\_LOG\_LOOP\_INVALID\_FACTORY\_EC
  - NETIO, [291](#)
- NETIO\_LOG\_LOOP\_INVALID\_PROPERTY\_EC
  - NETIO, [290](#)
- NETIO\_LOG\_LOOP\_SELECT\_TABLE\_EC
  - NETIO, [290](#)
- NETIO\_LOG\_LOOP\_SET\_LENGTH\_EC
  - NETIO, [290](#)
- NETIO\_LOG\_LOOP\_UNBINDX\_DB\_EC
  - NETIO, [290](#)
- NETIO\_LOG\_PACKET\_INITIALIZE\_EC
  - NETIO, [289](#)
- NETIO\_LOG\_PACKET\_SET\_HEAD\_EC
  - NETIO, [289](#)
- NETIO\_LOG\_PACKET\_SET\_TAIL\_EC
  - NETIO, [289](#)
- NETIO\_LOG\_ROUTE\_RTABLE\_ADD\_EC
  - NETIO, [287](#)
- NETIO\_LOG\_ROUTE\_RTABLE\_ALLOC\_EC
  - NETIO, [287](#)
- NETIO\_LOG\_ROUTE\_RTABLE\_CREATE\_EC
  - NETIO, [287](#)

- NETIO\_LOG\_ROUTE\_RTABLE\_DELETE\_EC  
NETIO, [287](#)
- NETIO\_LOG\_ROUTE\_RTABLE\_UPDATE\_EC  
NETIO, [287](#)
- NETIO\_LOG\_SET\_NAME\_EC  
NETIO, [289](#)
- NETIO\_LOG\_UNBIND\_EXTERNAL\_FAILED\_EC  
NETIO, [286](#)
- NETIO\_Netmask, [835](#)  
bits, [836](#)  
mask, [836](#)
- NETIO\_Netmask\_INITIALIZER  
NETIO Address API, [150](#)
- NETIO\_NetmaskSeq, [836](#)
- NETIO\_Packet, [836](#)
- NETIO\_Packet\_T  
NETIO Packet API, [146](#)
- NETIO\_SharedMemoryMutex\_attach  
Shared Memory Mutex API, [161](#)
- NETIO\_SharedMemoryMutex\_create  
Shared Memory Mutex API, [161](#)
- NETIO\_SharedMemoryMutex\_create\_or\_attach  
Shared Memory Mutex API, [162](#)
- NETIO\_SharedMemoryMutex\_delete  
Shared Memory Mutex API, [165](#)
- NETIO\_SharedMemoryMutex\_detach  
Shared Memory Mutex API, [165](#)
- NETIO\_SharedMemoryMutex\_lock  
Shared Memory Mutex API, [164](#)
- NETIO\_SharedMemoryMutex\_unlock  
Shared Memory Mutex API, [163](#)
- NETIO\_SharedMemorySegment\_attach  
Shared Memory Segment API, [169](#)
- NETIO\_SharedMemorySegment\_create  
Shared Memory Segment API, [167](#)
- NETIO\_SharedMemorySegment\_create\_or\_attach  
Shared Memory Segment API, [167](#)
- NETIO\_SharedMemorySegment\_delete  
Shared Memory Segment API, [168](#)
- NETIO\_SharedMemorySegment\_detach  
Shared Memory Segment API, [169](#)
- NETIO\_SharedMemorySegment\_get\_address  
Shared Memory Segment API, [170](#)
- NETIO\_SharedMemorySegment\_get\_max\_size  
Shared Memory Segment API, [171](#)
- NETIO\_SharedMemorySegment\_get\_size  
Shared Memory Segment API, [170](#)
- NETIO\_SharedMemorySegmentHeader, [837](#)
- NETIO\_SharedMemorySignalingSemaphore\_attach  
Shared Memory Signaling Semaphore, [173](#)
- NETIO\_SharedMemorySignalingSemaphore\_create  
Shared Memory Signaling Semaphore, [172](#)
- NETIO\_SharedMemorySignalingSemaphore\_create\_or\_attach  
Shared Memory Signaling Semaphore, [174](#)
- NETIO\_SharedMemorySignalingSemaphore\_delete  
Shared Memory Signaling Semaphore, [177](#)
- NETIO\_SharedMemorySignalingSemaphore\_detach  
Shared Memory Signaling Semaphore, [176](#)
- NETIO\_SharedMemorySignalingSemaphore\_signal  
Shared Memory Signaling Semaphore, [175](#)
- NETIO\_SharedMemorySignalingSemaphore\_wait  
Shared Memory Signaling Semaphore, [176](#)
- netio\_shmem\_log.h, [918](#)
- NETIO\_SHMEM\_LOG\_ADDRESS\_IN\_USE\_EC  
NETIO, [302](#)
- NETIO\_SHMEM\_LOG\_BINDX\_DB\_EC  
NETIO, [300](#)
- NETIO\_SHMEM\_LOG\_CHECK\_SYSTEM\_SETTINGS\_EC  
NETIO, [302](#)
- NETIO\_SHMEM\_LOG\_CQ\_INCOMPATIBLE\_VERSION\_EC  
REDA, [270](#)
- NETIO\_SHMEM\_LOG\_CQ\_INCONSISTENT\_STATE\_EC  
REDA, [270](#)
- NETIO\_SHMEM\_LOG\_CQ\_INT\_REPRESENTATION\_EC  
REDA, [270](#)
- NETIO\_SHMEM\_LOG\_CQ\_INVALID\_SIGNATURE\_EC  
REDA, [270](#)
- NETIO\_SHMEM\_LOG\_CQ\_UNLINKED\_EC  
REDA, [270](#)
- NETIO\_SHMEM\_LOG\_CREATE\_ATTACH\_INFINITE\_EC  
NETIO, [303](#)
- NETIO\_SHMEM\_LOG\_CREATE\_TABLE\_EC  
NETIO, [300](#)
- NETIO\_SHMEM\_LOG\_CURSOR\_ERROR\_EC  
NETIO, [301](#)
- NETIO\_SHMEM\_LOG\_DELETE\_TABLE\_EC  
NETIO, [300](#)
- NETIO\_SHMEM\_LOG\_FAILED\_TAKE\_SIGN\_SEM\_EC  
NETIO, [303](#)
- NETIO\_SHMEM\_LOG\_FAILED\_TO\_ALLOCATE\_STRUCT\_EC  
NETIO, [302](#)
- NETIO\_SHMEM\_LOG\_FAILED\_TO\_ATTACH\_CONCURRENT\_Q\_EC  
NETIO, [304](#)
- NETIO\_SHMEM\_LOG\_FAILED\_TO\_GET\_TIMESTAMP\_EC  
NETIO, [304](#)
- NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_CONCURRENT\_Q\_EC  
NETIO, [304](#)
- NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_MUTEX\_EC  
NETIO, [303](#)
- NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_SEGMENT\_ADDR\_EC  
NETIO, [303](#)
- NETIO\_SHMEM\_LOG\_FAILED\_TO\_INIT\_SIG\_SEM\_EC  
NETIO, [303](#)
- NETIO\_SHMEM\_LOG\_FAILED\_TO\_INITIALIZE\_EC  
NETIO, [303](#)
- NETIO\_SHMEM\_LOG\_FWD\_EC  
NETIO, [300](#)
- NETIO\_SHMEM\_LOG\_IGNORE\_TRANSPORT\_COMPATIBILITY\_EC

NETIO, 305  
 NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_CONCURRENT\_Q\_EC  
   NETIO, 304  
 NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_COOKIE\_EC  
   NETIO, 301  
 NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_HEADER\_EC  
   NETIO, 302  
 NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_HEADER\_SIZE\_EC  
   NETIO, 302  
 NETIO\_SHMEM\_LOG\_INCOMPATIBLE\_VERSION\_EC  
   NETIO, 301  
 NETIO\_SHMEM\_LOG\_INCONSISTENT\_MESSAGE\_SIZE\_EC  
   NETIO, 304  
 NETIO\_SHMEM\_LOG\_INITIALIZE\_FAILED\_EC  
   NETIO, 301  
 NETIO\_SHMEM\_LOG\_INVALID\_COMPONENT\_EC  
   NETIO, 301  
 NETIO\_SHMEM\_LOG\_INVALID\_FACTORY\_EC  
   NETIO, 301  
 NETIO\_SHMEM\_LOG\_INVALID\_PROPERTY\_EC  
   NETIO, 301  
 NETIO\_SHMEM\_LOG\_NONFATAL\_SIG\_SEM\_RC\_EC  
   NETIO, 304  
 NETIO\_SHMEM\_LOG\_PACKET\_FWD\_EC  
   NETIO, 305  
 NETIO\_SHMEM\_LOG\_PACKET\_HEAD\_EC  
   NETIO, 305  
 NETIO\_SHMEM\_LOG\_PACKET\_INIT\_EC  
   NETIO, 304  
 NETIO\_SHMEM\_LOG\_QUEUE\_FULL\_EC  
   NETIO, 305  
 NETIO\_SHMEM\_LOG\_RECORD\_EC  
   NETIO, 305  
 NETIO\_SHMEM\_LOG\_ROUTE\_DELETE\_FAILED\_EC  
   NETIO, 305  
 NETIO\_SHMEM\_LOG\_ROUTE\_LOOKUP\_FAILED\_EC  
   NETIO, 305  
 NETIO\_SHMEM\_LOG\_SELECT\_TABLE\_EC  
   NETIO, 300  
 NETIO\_SHMEM\_LOG\_SET\_LENGTH\_EC  
   NETIO, 300  
 NETIO\_SHMEM\_LOG\_SHMEM\_MUTEX\_LOCK\_FAILED\_EC  
   NETIO, 302  
 NETIO\_SHMEM\_LOG\_SHMEM\_MUTEX\_UNLOCK\_FAILED\_EC  
   NETIO, 302  
 NETIO\_SHMEM\_LOG\_SIG\_SEM\_SIGNAL\_FAILED\_EC  
   NETIO, 303  
 NETIO\_SHMEM\_LOG\_UNBINDX\_DB\_EC  
   NETIO, 300  
 NETIO\_SHMEM\_LOG\_UNSUPPORTED\_TRANSPORT\_COMPAT\_EC  
   NETIO, 306  
 NETIO\_SHMEMInterfaceFactoryProperty, 837  
   message\_size\_max, 838  
   pro\_minimum\_compatibility\_version, 838  
   receive\_buffer\_size, 837  
   received\_message\_count\_max, 837  
   recv\_thread\_property, 838  
   netio\_zcopy\_log.h, 920  
   NETIO\_ZCOPY\_LOG\_BASE  
   osapi\_log.h, 932  
   netio\_zcopy\_notif\_interface.h, 920  
   ZCOPY\_NotifInterface\_receive, 921  
   ZCOPY\_NotifInterfaceFactory\_register, 921  
   ZCOPY\_NotifInterfaceFactory\_unregister, 922  
   netio\_zcopy\_whsq\_log.h, 922  
   WHSQ\_LOG\_HISTORY\_OBJECT, 923  
   netmask  
     NETIO\_DGRAM\_InterfaceMultiCastGroup, 831  
     NETIO\_DGRAM\_InterfaceRouteEntry, 832  
     UDP\_InterfaceTableEntry, 858  
   not\_alive\_count  
     DDS\_LivelinessChangedStatus, 559  
   not\_alive\_count\_change  
     DDS\_LivelinessChangedStatus, 560  
   Notification Interface, 188  
     NETIO\_DEFAULT\_NOTIF\_NAME, 189  
   notify\_recv\_port  
     ZCOPY\_NotifUserInterfaceI, 867  
  
   on\_before\_sample\_commit  
     DDSDDataReaderListener, 665  
     DDSDomainParticipantListener, 721  
   on\_before\_sample\_deserialize  
     DDSDDataReaderListener, 665  
     DDSDomainParticipantListener, 721  
   on\_data\_available  
     DDSDDataReaderListener, 664  
     DDSDomainParticipantListener, 719  
   on\_data\_on\_readers  
     DDSDomainParticipantListener, 722  
     DDSSubscriberListener, 758  
   on\_inconsistent\_topic  
     DDSDomainParticipantListener, 724  
     DDSTopicListener, 767  
   on\_instance\_replaced  
     DDSDDataReaderListener, 665  
     DDSDomainParticipantListener, 720  
   on\_liveliness\_changed  
     DDSDDataReaderListener, 664  
     DDSDomainParticipantListener, 720  
   on\_liveliness\_lost  
     DDSDDataWriterListener, 682  
     DDSDomainParticipantListener, 722  
   on\_offered\_deadline\_missed  
     DDSDDataWriterListener, 682  
     DDSDomainParticipantListener, 722  
   on\_offered\_incompatible\_qos  
     DDSDDataWriterListener, 682

- DDSDomainParticipantListener, [723](#)
- on\_publication\_matched
  - DDSDataWriterListener, [683](#)
  - DDSDomainParticipantListener, [723](#)
- on\_reliable\_reader\_activity\_changed
  - DDSDataWriterListener, [683](#)
  - DDSDomainParticipantListener, [724](#)
- on\_reliable\_sample\_unacknowledged
  - DDSDataWriterListener, [684](#)
- on\_requested\_deadline\_missed
  - DDSDataReaderListener, [664](#)
  - DDSDomainParticipantListener, [719](#)
- on\_requested\_incompatible\_qos
  - DDSDataReaderListener, [664](#)
  - DDSDomainParticipantListener, [720](#)
- on\_sample\_lost
  - DDSDataReaderListener, [665](#)
  - DDSDomainParticipantListener, [720](#)
- on\_sample\_rejected
  - DDSDataReaderListener, [664](#)
  - DDSDomainParticipantListener, [720](#)
- on\_sample\_removed
  - DDSDataWriterListener, [684](#)
- on\_subscription\_matched
  - DDSDataReaderListener, [664](#)
  - DDSDomainParticipantListener, [720](#)
- operator=
  - FooSeq, [808](#)
- operator[]
  - FooSeq, [811](#)
- options
  - OSAPI\_ThreadProperty, [846](#)
- ordered\_access
  - DDS\_PresentationQosPolicy, [582](#)
- OS API, [189](#)
- OSAPI, [241](#), [245](#)
  - OSAPI\_LOG\_CLOSE\_EC, [259](#)
  - OSAPI\_LOG\_EXHAUSTED\_POSIX\_SEM\_LIMIT\_EC, [260](#)
  - OSAPI\_LOG\_GET\_NEXT\_OBJECT\_ID\_EC, [250](#)
  - OSAPI\_LOG\_HEAP\_INTERNAL\_ALLOCATE\_EC, [255](#)
  - OSAPI\_LOG\_LAST\_RECORDED\_ERROR\_EC, [256](#)
  - OSAPI\_LOG\_MUTEX\_DELETE\_EC, [255](#)
  - OSAPI\_LOG\_MUTEX\_GIVE\_EC, [255](#)
  - OSAPI\_LOG\_MUTEX\_INIT\_EC, [255](#)
  - OSAPI\_LOG\_MUTEX\_NEW\_EC, [255](#)
  - OSAPI\_LOG\_MUTEX\_TAKE\_EC, [255](#)
  - OSAPI\_LOG\_PTHREAD\_COND\_DESTROY\_EC, [263](#)
  - OSAPI\_LOG\_PTHREAD\_COND\_INIT\_EC, [263](#)
  - OSAPI\_LOG\_PTHREAD\_COND\_SIGNAL\_EC, [263](#)
  - OSAPI\_LOG\_PTHREAD\_COND\_WAIT\_EC, [263](#)
  - OSAPI\_LOG\_PTHREAD\_CONDATTR\_DESTROY\_EC, [263](#)
  - OSAPI\_LOG\_PTHREAD\_CONDATTR\_INIT\_EC, [262](#)
  - OSAPI\_LOG\_PTHREAD\_CONDATTR\_SETPSHARED\_EC, [263](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEX\_CONSISTENT\_EC, [262](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEX\_DESTROY\_EC, [262](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEX\_INIT\_EC, [262](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEX\_LOCK\_EC, [262](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEX\_UNLOCK\_EC, [262](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_DESTROY\_EC, [262](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_INIT\_EC, [261](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETPRIOCEILING\_EC, [264](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETPROTOCOL\_EC, [264](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETPSHARED\_EC, [261](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETROBUST\_EC, [261](#)
  - OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETTYPE\_EC, [263](#)
  - OSAPI\_LOG\_SD\_OPEN\_EC, [257](#)
  - OSAPI\_LOG\_SD\_UNMAP\_EC, [256](#)
  - OSAPI\_LOG\_SEGMENT\_DETACH\_FAILED\_EC, [257](#)
  - OSAPI\_LOG\_SEGMENT\_KEY\_ALREADY\_EXISTS\_EC, [257](#)
  - OSAPI\_LOG\_SEGMENT\_KEY\_DOESNT\_EXIST\_EC, [257](#)
  - OSAPI\_LOG\_SEM\_GIVE\_EC, [257](#)
  - OSAPI\_LOG\_SEM\_INFO\_GET\_EC, [256](#)
  - OSAPI\_LOG\_SEM\_OPEN\_EC, [256](#)
  - OSAPI\_LOG\_SEM\_TAKE\_EC, [257](#)
  - OSAPI\_LOG\_SEMAPHORE\_DELETE\_EC, [254](#)
  - OSAPI\_LOG\_SEMAPHORE\_GIVE\_EC, [254](#)
  - OSAPI\_LOG\_SEMAPHORE\_NEW\_EC, [254](#)
  - OSAPI\_LOG\_SEMAPHORE\_NEW\_INIT\_EC, [254](#)
  - OSAPI\_LOG\_SEMAPHORE\_TAKE\_EC, [255](#)
  - OSAPI\_LOG\_SET\_THREAD\_NAME\_EC, [256](#)
  - OSAPI\_LOG\_SHMEM\_ATTACH\_FAILED\_EC, [258](#)
  - OSAPI\_LOG\_SHMEM\_FSTAT\_EC, [261](#)
  - OSAPI\_LOG\_SHMEM\_FTRUNCATE\_EC, [259](#)
  - OSAPI\_LOG\_SHMEM\_GIVE\_FAILED\_EC, [258](#)
  - OSAPI\_LOG\_SHMEM\_INIT\_DESCRIPTOR\_EC, [259](#)
  - OSAPI\_LOG\_SHMEM\_MMAP\_EC, [259](#)
  - OSAPI\_LOG\_SHMEM\_MUNMAP\_EC, [259](#)

- OSAPI\_LOG\_SHMEM\_MUTEX\_INIT\_TIMEOUT\_EC, [261](#)
- OSAPI\_LOG\_SHMEM\_POSIX\_GIVE\_EC, [260](#)
- OSAPI\_LOG\_SHMEM\_SEGMENT\_FILE\_MAP\_FAILED\_EC, [258](#)
- OSAPI\_LOG\_SHMEM\_SEM\_DELETE\_FAILED\_EC, [258](#)
- OSAPI\_LOG\_SHMEM\_SEM\_MUTEX\_CREATED\_EC, [258](#)
- OSAPI\_LOG\_SHMEM\_SEM\_UNLINK\_EC, [260](#)
- OSAPI\_LOG\_SHMEM\_SEM\_WAIT\_EC, [260](#)
- OSAPI\_LOG\_SHMEM\_SET\_DESCRIPTOR\_EC, [258](#)
- OSAPI\_LOG\_SHMEM\_SHM\_OPEN\_EC, [259](#)
- OSAPI\_LOG\_SHMEM\_SIZE\_EXCEEDED\_EC, [260](#)
- OSAPI\_LOG\_SHMEM\_TAKE\_FAILED\_EC, [258](#)
- OSAPI\_LOG\_SHMEM\_UNLINK\_EC, [259](#)
- OSAPI\_LOG\_SYSTEM\_GET\_TIME\_EC, [256](#)
- OSAPI\_LOG\_SYSTEM\_SET\_PROPERTY\_EC, [250](#)
- OSAPI\_LOG\_SYSTEM\_TIMER\_START\_EC, [250](#)
- OSAPI\_LOG\_SYSTEM\_TIMER\_STOP\_EC, [250](#)
- OSAPI\_LOG\_THREAD\_CREATE\_EC, [251](#)
- OSAPI\_LOG\_THREAD\_DESTROY\_EC, [251](#)
- OSAPI\_LOG\_THREAD\_DESTROY\_NO\_START\_EC, [251](#)
- OSAPI\_LOG\_THREAD\_DESTROY\_NO\_WAKEUP\_EC, [252](#)
- OSAPI\_LOG\_THREAD\_EXEC\_CREATE\_EC, [251](#)
- OSAPI\_LOG\_THREAD\_EXEC\_START\_EC, [251](#)
- OSAPI\_LOG\_THREAD\_GET\_MAX\_PRIORITY\_EC, [261](#)
- OSAPI\_LOG\_THREAD\_GET\_POLICY\_EC, [252](#)
- OSAPI\_LOG\_THREAD\_INIT\_EC, [252](#)
- OSAPI\_LOG\_THREAD\_JOIN\_EC, [261](#)
- OSAPI\_LOG\_THREAD\_NEW\_EC, [250](#)
- OSAPI\_LOG\_THREAD\_POLICY\_DIFFER\_EC, [252](#)
- OSAPI\_LOG\_THREAD\_PRIORITY\_MAP\_EC, [252](#)
- OSAPI\_LOG\_THREAD\_SCHEDPARAM\_EC, [252](#)
- OSAPI\_LOG\_THREAD\_SEM\_EC, [251](#)
- OSAPI\_LOG\_THREAD\_SEMPOOL\_EXCEEDED\_EC, [260](#)
- OSAPI\_LOG\_THREAD\_SEMPOOL\_INVALID\_SEM\_EC, [260](#)
- OSAPI\_LOG\_THREAD\_START\_EC, [251](#)
- OSAPI\_LOG\_TIMER\_DELETE\_EC, [252](#)
- OSAPI\_LOG\_TIMER\_DELETE\_MUTEX\_EC, [254](#)
- OSAPI\_LOG\_TIMER\_DELETE\_STOP\_TIMER\_EC, [254](#)
- OSAPI\_LOG\_TIMER\_GET\_USER\_DATA\_EPOCH\_EC, [253](#)
- OSAPI\_LOG\_TIMER\_MUTEX\_EC, [254](#)
- OSAPI\_LOG\_TIMER\_NEW\_EC, [253](#)
- OSAPI\_LOG\_TIMER\_NEW\_ENTRY\_EC, [253](#)
- OSAPI\_LOG\_TIMER\_NEW\_MUTEX\_EC, [253](#)
- OSAPI\_LOG\_TIMER\_NEW\_START\_TIMER\_EC, [253](#)
- OSAPI\_LOG\_TIMER\_NEW\_WHEEL\_EC, [253](#)
- OSAPI\_LOG\_TIMER\_TICK\_MUTEX\_EC, [253](#)
- OSAPI\_LOG\_UNKNOWN\_SEMAPHORE\_TYPE\_EC, [257](#)
- OSAPI Heap, [190](#)
  - OSAPI\_ALIGNMENT\_DEFAULT, [190](#)
  - OSAPI\_Heap\_align\_size\_up, [190](#)
  - OSAPI\_Heap\_allocate, [191](#)
  - OSAPI\_Heap\_allocate\_buffer, [192](#)
  - OSAPI\_Heap\_free, [192](#)
  - OSAPI\_Heap\_free\_buffer, [192](#)
  - OSAPI\_Heap\_is\_address\_aligned, [190](#)
  - OSAPI\_Heap\_realloc, [191](#)
- OSAPI Memory API, [198](#)
- OSAPI Mutex, [193](#)
  - OSAPI\_Mutex\_delete, [193](#)
  - OSAPI\_Mutex\_give, [194](#)
  - OSAPI\_Mutex\_new, [194](#)
  - OSAPI\_Mutex\_T, [193](#)
  - OSAPI\_Mutex\_take, [194](#)
- OSAPI Process, [195](#)
  - OSAPI\_Process\_getpid, [195](#)
  - OSAPI\_Process\_is\_alive, [195](#)
- OSAPI Semaphore, [196](#)
  - OSAPI\_Semaphore\_delete, [197](#)
  - OSAPI\_SEMAPHORE\_RESULT\_ERROR, [197](#)
  - OSAPI\_SEMAPHORE\_RESULT\_OK, [197](#)
  - OSAPI\_SEMAPHORE\_RESULT\_TIMEOUT, [197](#)
  - OSAPI\_Semaphore\_T, [197](#)
  - OSAPI\_SEMAPHORE\_TIMEOUT\_INFINITE, [197](#)
- OSAPI String API, [198](#)
- OSAPI System API, [198](#)
  - OSAPI\_System\_generate\_uuid\_T, [201](#)
  - OSAPI\_System\_get\_hostname\_T, [200](#)
  - OSAPI\_System\_get\_ticktime\_T, [201](#)
  - OSAPI\_System\_get\_time\_T, [200](#)
  - OSAPI\_System\_get\_timer\_resolution\_T, [199](#)
  - OSAPI\_System\_initialize\_T, [200](#)
  - OSAPI\_SYSTEM\_MAX\_HOSTNAME, [199](#)
  - OSAPI\_System\_start\_timer\_T, [199](#)
  - OSAPI\_SystemI, [201](#)
- OSAPI Thread API, [202](#)
  - OSAPI\_THREAD\_DEFAULT\_OPTIONS, [204](#)
  - OSAPI\_THREAD\_FLOATING\_POINT, [204](#)
  - OSAPI\_THREAD\_PRIORITY\_ABOVE\_NORMAL, [203](#)
  - OSAPI\_THREAD\_PRIORITY\_BELOW\_NORMAL, [203](#)
  - OSAPI\_THREAD\_PRIORITY\_HIGH, [203](#)
  - OSAPI\_THREAD\_PRIORITY\_INHERIT, [203](#)
  - OSAPI\_THREAD\_PRIORITY\_LOW, [203](#)
  - OSAPI\_THREAD\_PRIORITY\_NORMAL, [203](#)

- OSAPI\_THREAD\_PROPERTY\_DEFAULT, [204](#)
- OSAPI\_THREAD\_REALTIME\_PRIORITY, [204](#)
- OSAPI\_THREAD\_STDIO, [204](#)
- OSAPI\_THREAD\_USE\_OSDEFAULT\_STACKSIZE, [204](#)
- OSAPI\_ThreadOptions, [205](#)
- OSAPI\_ThreadProperty\_INITIALIZER, [204](#)
- OSAPI Time, [205](#)
  - OSAPI\_SystemTime, [205](#)
- OSAPI Timer, [205](#)
- OSAPI Types, [206](#)
  - RTI\_BOOL, [208](#)
  - RTI\_FALSE, [208](#)
  - RTI\_INT16, [207](#)
  - RTI\_INT32, [207](#)
  - RTI\_INT64, [207](#)
  - RTI\_INT8, [206](#)
  - RTI\_SIZE\_T, [208](#)
  - RTI\_TRUE, [208](#)
  - RTI\_UINT16, [207](#)
  - RTI\_UINT32, [207](#)
  - RTI\_UINT64, [208](#)
  - RTI\_UINT8, [206](#)
- OSAPI\_ALIGNMENT\_DEFAULT
  - OSAPI Heap, [190](#)
- OSAPI\_Heap\_align\_size\_up
  - OSAPI Heap, [190](#)
- OSAPI\_Heap\_allocate
  - OSAPI Heap, [191](#)
- OSAPI\_Heap\_allocate\_buffer
  - OSAPI Heap, [192](#)
- OSAPI\_Heap\_free
  - OSAPI Heap, [192](#)
- OSAPI\_Heap\_free\_buffer
  - OSAPI Heap, [192](#)
- OSAPI\_Heap\_is\_address\_aligned
  - OSAPI Heap, [190](#)
- OSAPI\_Heap\_realloc
  - OSAPI Heap, [191](#)
- osapi\_log.h, [923](#)
  - APPGEN\_LOG\_BASE, [932](#)
  - CDR\_LOG\_BASE, [931](#)
  - DB\_LOG\_BASE, [930](#)
  - DDS\_FILTER\_LOG\_BASE, [932](#)
  - DDSC\_LOG\_BASE, [931](#)
  - DPDE\_LOG\_BASE, [932](#)
  - DPSE\_LOG\_BASE, [931](#)
  - NETIO\_LOG\_BASE, [930](#)
  - NETIO\_ZCOPY\_LOG\_BASE, [932](#)
  - OSAPI\_LOG\_BASE, [930](#)
  - OSAPI\_LOG\_INVALID\_TIMER\_RES\_EC, [932](#)
  - OSAPI\_LOG\_MATH\_OFV\_EC, [932](#)
  - REDA\_LOG\_BASE, [930](#)
  - RHSM\_LOG\_BASE, [931](#)
  - RT\_LOG\_BASE, [930](#)
  - RTPS\_LOG\_BASE, [931](#)
  - SDM\_LOG\_BASE, [930](#)
  - SEC\_CORE\_LOG\_BASE, [932](#)
  - SHMEM\_LOG\_BASE, [930](#)
  - UDP\_LOG\_BASE, [930](#)
  - WHSM\_LOG\_BASE, [931](#)
  - XCDR\_LOG\_BASE, [931](#)
- OSAPI\_LOG\_BASE
  - osapi\_log.h, [930](#)
- OSAPI\_Log\_clear
  - Log API, [223](#)
- OSAPI\_LOG\_CLOSE\_EC
  - OSAPI, [259](#)
- OSAPI\_LOG\_EXHAUSTED\_POSIX\_SEM\_LIMIT\_EC
  - OSAPI, [260](#)
- OSAPI\_Log\_get\_display\_handler
  - Log API, [222](#)
- OSAPI\_Log\_get\_last\_error\_code
  - Log API, [224](#)
- OSAPI\_Log\_get\_log\_handler
  - Log API, [221](#)
- OSAPI\_LOG\_GET\_NEXT\_OBJECT\_ID\_EC
  - OSAPI, [250](#)
- OSAPI\_Log\_get\_property
  - Log API, [223](#)
- OSAPI\_LOG\_HEAP\_INTERNAL\_ALLOCATE\_EC
  - OSAPI, [255](#)
- OSAPI\_LOG\_INVALID\_TIMER\_RES\_EC
  - osapi\_log.h, [932](#)
- OSAPI\_LOG\_LAST\_RECORDED\_ERROR\_EC
  - OSAPI, [256](#)
- OSAPI\_LOG\_MATH\_OFV\_EC
  - osapi\_log.h, [932](#)
- OSAPI\_LOG\_MUTEX\_DELETE\_EC
  - OSAPI, [255](#)
- OSAPI\_LOG\_MUTEX\_GIVE\_EC
  - OSAPI, [255](#)
- OSAPI\_LOG\_MUTEX\_INIT\_EC
  - OSAPI, [255](#)
- OSAPI\_LOG\_MUTEX\_NEW\_EC
  - OSAPI, [255](#)
- OSAPI\_LOG\_MUTEX\_TAKE\_EC
  - OSAPI, [255](#)
- OSAPI\_LOG\_PTHREAD\_COND\_DESTROY\_EC
  - OSAPI, [263](#)
- OSAPI\_LOG\_PTHREAD\_COND\_INIT\_EC
  - OSAPI, [263](#)
- OSAPI\_LOG\_PTHREAD\_COND\_SIGNAL\_EC
  - OSAPI, [263](#)
- OSAPI\_LOG\_PTHREAD\_COND\_WAIT\_EC
  - OSAPI, [263](#)
- OSAPI\_LOG\_PTHREAD\_CONDATTR\_DESTROY\_EC
  - OSAPI, [263](#)



- OSAPI\_LOG\_PTHREAD\_CONDATTR\_INIT\_EC  
OSAPI, [262](#)
- OSAPI\_LOG\_PTHREAD\_CONDATTR\_SETPSHARED\_EC  
OSAPI, [263](#)
- OSAPI\_LOG\_PTHREAD\_MUTEX\_CONSISTENT\_EC  
OSAPI, [262](#)
- OSAPI\_LOG\_PTHREAD\_MUTEX\_DESTROY\_EC  
OSAPI, [262](#)
- OSAPI\_LOG\_PTHREAD\_MUTEX\_INIT\_EC  
OSAPI, [262](#)
- OSAPI\_LOG\_PTHREAD\_MUTEX\_LOCK\_EC  
OSAPI, [262](#)
- OSAPI\_LOG\_PTHREAD\_MUTEX\_UNLOCK\_EC  
OSAPI, [262](#)
- OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_DESTROY\_EC  
OSAPI, [262](#)
- OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_INIT\_EC  
OSAPI, [261](#)
- OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETPRIOCEILING\_EC  
OSAPI, [264](#)
- OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETPROTOCOL\_EC  
OSAPI, [264](#)
- OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETPSHARED\_EC  
OSAPI, [261](#)
- OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETROBUST\_EC  
OSAPI, [261](#)
- OSAPI\_LOG\_PTHREAD\_MUTEXATTR\_SETTYPE\_EC  
OSAPI, [263](#)
- OSAPI\_LOG\_SD\_OPEN\_EC  
OSAPI, [257](#)
- OSAPI\_LOG\_SD\_UNMAP\_EC  
OSAPI, [256](#)
- OSAPI\_LOG\_SEGMENT\_DETACH\_FAILED\_EC  
OSAPI, [257](#)
- OSAPI\_LOG\_SEGMENT\_KEY\_ALREADY\_EXISTS\_EC  
OSAPI, [257](#)
- OSAPI\_LOG\_SEGMENT\_KEY\_DOESNT\_EXIST\_EC  
OSAPI, [257](#)
- OSAPI\_LOG\_SEM\_GIVE\_EC  
OSAPI, [257](#)
- OSAPI\_LOG\_SEM\_INFO\_GET\_EC  
OSAPI, [256](#)
- OSAPI\_LOG\_SEM\_OPEN\_EC  
OSAPI, [256](#)
- OSAPI\_LOG\_SEM\_TAKE\_EC  
OSAPI, [257](#)
- OSAPI\_LOG\_SEMAPHORE\_DELETE\_EC  
OSAPI, [254](#)
- OSAPI\_LOG\_SEMAPHORE\_GIVE\_EC  
OSAPI, [254](#)
- OSAPI\_LOG\_SEMAPHORE\_NEW\_EC  
OSAPI, [254](#)
- OSAPI\_LOG\_SEMAPHORE\_NEW\_INIT\_EC  
OSAPI, [254](#)
- OSAPI\_LOG\_SEMAPHORE\_TAKE\_EC  
OSAPI, [255](#)
- OSAPI\_Log\_set\_display\_handler  
Log API, [222](#)
- OSAPI\_Log\_set\_log\_handler  
Log API, [221](#)
- OSAPI\_Log\_set\_property  
Log API, [223](#)
- OSAPI\_LOG\_SET\_THREAD\_NAME\_EC  
OSAPI, [256](#)
- OSAPI\_Log\_set\_verbosity  
Log API, [222](#)
- OSAPI\_LOG\_SHMEM\_ATTACH\_FAILED\_EC  
OSAPI, [258](#)
- OSAPI\_LOG\_SHMEM\_FSTAT\_EC  
OSAPI, [261](#)
- OSAPI\_LOG\_SHMEM\_FTRUNCATE\_EC  
OSAPI, [259](#)
- OSAPI\_LOG\_SHMEM\_GIVE\_FAILED\_EC  
OSAPI, [258](#)
- OSAPI\_LOG\_SHMEM\_INIT\_DESCRIPTOR\_EC  
OSAPI, [259](#)
- OSAPI\_LOG\_SHMEM\_MMAP\_EC  
OSAPI, [259](#)
- OSAPI\_LOG\_SHMEM\_MUNMAP\_EC  
OSAPI, [259](#)
- OSAPI\_LOG\_SHMEM\_MUTEX\_INIT\_TIMEOUT\_EC  
OSAPI, [261](#)
- OSAPI\_LOG\_SHMEM\_POSIX\_GIVE\_EC  
OSAPI, [260](#)
- OSAPI\_LOG\_SHMEM\_SEGMENT\_FILE\_MAP\_FAILED\_EC  
OSAPI, [258](#)
- OSAPI\_LOG\_SHMEM\_SEM\_DELETE\_FAILED\_EC  
OSAPI, [258](#)
- OSAPI\_LOG\_SHMEM\_SEM\_MUTEX\_CREATED\_EC  
OSAPI, [258](#)
- OSAPI\_LOG\_SHMEM\_SEM\_UNLINK\_EC  
OSAPI, [260](#)
- OSAPI\_LOG\_SHMEM\_SEM\_WAIT\_EC  
OSAPI, [260](#)
- OSAPI\_LOG\_SHMEM\_SET\_DESCRIPTOR\_EC  
OSAPI, [258](#)
- OSAPI\_LOG\_SHMEM\_SHM\_OPEN\_EC  
OSAPI, [259](#)
- OSAPI\_LOG\_SHMEM\_SIZE\_EXCEEDED\_EC  
OSAPI, [260](#)
- OSAPI\_LOG\_SHMEM\_TAKE\_FAILED\_EC  
OSAPI, [258](#)
- OSAPI\_LOG\_SHMEM\_UNLINK\_EC  
OSAPI, [259](#)
- OSAPI\_LOG\_SYSTEM\_GET\_TIME\_EC  
OSAPI, [256](#)
- OSAPI\_LOG\_SYSTEM\_SET\_PROPERTY\_EC  
OSAPI, [250](#)

- OSAPI\_LOG\_SYSTEM\_TIMER\_START\_EC
  - OSAPI, [250](#)
- OSAPI\_LOG\_SYSTEM\_TIMER\_STOP\_EC
  - OSAPI, [250](#)
- OSAPI\_LOG\_THREAD\_CREATE\_EC
  - OSAPI, [251](#)
- OSAPI\_LOG\_THREAD\_DESTROY\_EC
  - OSAPI, [251](#)
- OSAPI\_LOG\_THREAD\_DESTROY\_NO\_START\_EC
  - OSAPI, [251](#)
- OSAPI\_LOG\_THREAD\_DESTROY\_NO\_WAKEUP\_EC
  - OSAPI, [252](#)
- OSAPI\_LOG\_THREAD\_EXEC\_CREATE\_EC
  - OSAPI, [251](#)
- OSAPI\_LOG\_THREAD\_EXEC\_START\_EC
  - OSAPI, [251](#)
- OSAPI\_LOG\_THREAD\_GET\_MAX\_PRIORITY\_EC
  - OSAPI, [261](#)
- OSAPI\_LOG\_THREAD\_GET\_POLICY\_EC
  - OSAPI, [252](#)
- OSAPI\_LOG\_THREAD\_INIT\_EC
  - OSAPI, [252](#)
- OSAPI\_LOG\_THREAD\_JOIN\_EC
  - OSAPI, [261](#)
- OSAPI\_LOG\_THREAD\_NEW\_EC
  - OSAPI, [250](#)
- OSAPI\_LOG\_THREAD\_POLICY\_DIFFER\_EC
  - OSAPI, [252](#)
- OSAPI\_LOG\_THREAD\_PRIORITY\_MAP\_EC
  - OSAPI, [252](#)
- OSAPI\_LOG\_THREAD\_SCHEDPARAM\_EC
  - OSAPI, [252](#)
- OSAPI\_LOG\_THREAD\_SEM\_EC
  - OSAPI, [251](#)
- OSAPI\_LOG\_THREAD\_SEMPOOL\_EXCEEDED\_EC
  - OSAPI, [260](#)
- OSAPI\_LOG\_THREAD\_SEMPOOL\_INVALID\_SEM\_EC
  - OSAPI, [260](#)
- OSAPI\_LOG\_THREAD\_START\_EC
  - OSAPI, [251](#)
- OSAPI\_LOG\_TIMER\_DELETE\_EC
  - OSAPI, [252](#)
- OSAPI\_LOG\_TIMER\_DELETE\_MUTEX\_EC
  - OSAPI, [254](#)
- OSAPI\_LOG\_TIMER\_DELETE\_STOP\_TIMER\_EC
  - OSAPI, [254](#)
- OSAPI\_LOG\_TIMER\_GET\_USER\_DATA\_EPOCH\_EC
  - OSAPI, [253](#)
- OSAPI\_LOG\_TIMER\_MUTEX\_EC
  - OSAPI, [254](#)
- OSAPI\_LOG\_TIMER\_NEW\_EC
  - OSAPI, [253](#)
- OSAPI\_LOG\_TIMER\_NEW\_ENTRY\_EC
  - OSAPI, [253](#)
- OSAPI\_LOG\_TIMER\_NEW\_MUTEX\_EC
  - OSAPI, [253](#)
- OSAPI\_LOG\_TIMER\_NEW\_START\_TIMER\_EC
  - OSAPI, [253](#)
- OSAPI\_LOG\_TIMER\_NEW\_WHEEL\_EC
  - OSAPI, [253](#)
- OSAPI\_LOG\_TIMER\_TICK\_MUTEX\_EC
  - OSAPI, [253](#)
- OSAPI\_LOG\_UNKNOWN\_SEMAPHORE\_TYPE\_EC
  - OSAPI, [257](#)
- OSAPI\_LOG\_VERBOSITY\_DEBUG
  - Log API, [221](#)
- OSAPI\_LOG\_VERBOSITY\_ERROR
  - Log API, [221](#)
- OSAPI\_LOG\_VERBOSITY\_SILENT
  - Log API, [221](#)
- OSAPI\_LOG\_VERBOSITY\_WARNING
  - Log API, [221](#)
- OSAPI\_Log\_write\_buffer\_T
  - Log API, [220](#)
- OSAPI\_LogHandler\_T
  - Log API, [220](#)
- OSAPI\_LogProperty, [838](#)
  - log\_detail, [839](#)
  - max\_buffer\_size, [839](#)
  - write\_buffer, [839](#)
- OSAPI\_LogVerbosity\_T
  - Log API, [221](#)
- OSAPI\_Mutex\_delete
  - OSAPI Mutex, [193](#)
- OSAPI\_Mutex\_give
  - OSAPI Mutex, [194](#)
- OSAPI\_Mutex\_new
  - OSAPI Mutex, [194](#)
- OSAPI\_Mutex\_T
  - OSAPI Mutex, [193](#)
- OSAPI\_Mutex\_take
  - OSAPI Mutex, [194](#)
- osapi\_process.h, [933](#)
- OSAPI\_Process\_getpid
  - OSAPI Process, [195](#)
- OSAPI\_Process\_is\_alive
  - OSAPI Process, [195](#)
- osapi\_semaphore.h, [933](#)
  - OSAPI\_Semaphore\_give, [935](#)
  - OSAPI\_Semaphore\_new, [934](#)
  - OSAPI\_Semaphore\_take, [934](#)
- OSAPI\_Semaphore\_delete
  - OSAPI Semaphore, [197](#)
- OSAPI\_Semaphore\_give
  - osapi\_semaphore.h, [935](#)
- OSAPI\_Semaphore\_new
  - osapi\_semaphore.h, [934](#)
- OSAPI\_SEMAPHORE\_RESULT\_ERROR



- OSAPI Semaphore, [197](#)
- OSAPI\_SEMAPHORE\_RESULT\_OK
  - OSAPI Semaphore, [197](#)
- OSAPI\_SEMAPHORE\_RESULT\_TIMEOUT
  - OSAPI Semaphore, [197](#)
- OSAPI\_Semaphore\_T
  - OSAPI Semaphore, [197](#)
- OSAPI\_Semaphore\_take
  - osapi\_semaphore.h, [934](#)
- OSAPI\_SEMAPHORE\_TIMEOUT\_INFINITE
  - OSAPI Semaphore, [197](#)
- OSAPI\_SharedMemory\_is\_robust\_mutex\_supported
  - OSAPI\_SharedMemorySegment, [219](#)
- OSAPI\_SharedMemoryMonitor, [209](#)
  - OSAPI\_SharedMemoryMonitor\_acquire, [210](#)
  - OSAPI\_SharedMemoryMonitor\_finalize, [210](#)
  - OSAPI\_SharedMemoryMonitor\_get\_size, [209](#)
  - OSAPI\_SharedMemoryMonitor\_initialize, [209](#)
  - OSAPI\_SharedMemoryMonitor\_mark\_consistent, [212](#)
  - OSAPI\_SharedMemoryMonitor\_release, [211](#)
  - OSAPI\_SharedMemoryMonitor\_signal, [211](#)
  - OSAPI\_SharedMemoryMonitor\_wait, [211](#)
- OSAPI\_SharedMemoryMonitor\_acquire
  - OSAPI\_SharedMemoryMonitor, [210](#)
- OSAPI\_SharedMemoryMonitor\_finalize
  - OSAPI\_SharedMemoryMonitor, [210](#)
- OSAPI\_SharedMemoryMonitor\_get\_size
  - OSAPI\_SharedMemoryMonitor, [209](#)
- OSAPI\_SharedMemoryMonitor\_initialize
  - OSAPI\_SharedMemoryMonitor, [209](#)
- OSAPI\_SharedMemoryMonitor\_mark\_consistent
  - OSAPI\_SharedMemoryMonitor, [212](#)
- OSAPI\_SharedMemoryMonitor\_release
  - OSAPI\_SharedMemoryMonitor, [211](#)
- OSAPI\_SharedMemoryMonitor\_signal
  - OSAPI\_SharedMemoryMonitor, [211](#)
- OSAPI\_SharedMemoryMonitor\_wait
  - OSAPI\_SharedMemoryMonitor, [211](#)
- OSAPI\_SharedMemorySegment, [212](#)
  - OSAPI\_SharedMemory\_is\_robust\_mutex\_supported, [219](#)
  - OSAPI\_SharedMemorySegment\_attach\_impl, [217](#)
  - OSAPI\_SharedMemorySegment\_create\_impl, [216](#)
  - OSAPI\_SharedMemorySegment\_delete\_impl, [216](#)
  - OSAPI\_SharedMemorySegment\_detach\_impl, [217](#)
  - OSAPI\_SharedMemorySegment\_get\_max\_size, [215](#)
  - OSAPI\_SharedMemorySegment\_is\_owner\_alive, [215](#)
  - OSAPI\_SharedMemorySegment\_lock\_impl, [217](#)
  - OSAPI\_SharedMemorySegment\_mark\_consistent\_impl, [218](#)
  - OSAPI\_SharedMemorySegment\_unlock\_impl, [218](#)
- OSAPI\_SharedMemorySegmentHandle\_get\_size, [215](#)
- OSAPI\_SharedMemorySegmentHeader\_get\_size, [215](#)
- OSAPI\_SHMEM\_ATTACH\_STATUS, [214](#)
- OSAPI\_SHMEM\_ATTACH\_STATUS\_OK, [214](#)
- OSAPI\_SHMEM\_ATTACH\_STATUS\_SEGMENT\_NOT\_FOUND, [214](#)
- OSAPI\_SHMEM\_MODE, [214](#)
- OSAPI\_SHMEM\_MODE\_LOCKABLE, [214](#)
- OSAPI\_SHMEM\_MODE\_READ, [214](#)
- OSAPI\_SHMEM\_MODE\_ROBUST, [214](#)
- OSAPI\_SHMEM\_MODE\_UNKNOWN, [214](#)
- OSAPI\_SHMEM\_MODE\_WRITE, [214](#)
- OSAPI\_SHMEM\_STATUS, [214](#)
- OSAPI\_SHMEM\_STATUS\_OK, [214](#)
- OSAPI\_SHMEM\_STATUS\_OWNER\_DEAD, [214](#)
- OSAPI\_SharedMemorySegment\_attach\_impl
  - OSAPI\_SharedMemorySegment, [217](#)
- OSAPI\_SharedMemorySegment\_create\_impl
  - OSAPI\_SharedMemorySegment, [216](#)
- OSAPI\_SharedMemorySegment\_delete\_impl
  - OSAPI\_SharedMemorySegment, [216](#)
- OSAPI\_SharedMemorySegment\_detach\_impl
  - OSAPI\_SharedMemorySegment, [217](#)
- OSAPI\_SharedMemorySegment\_get\_max\_size
  - OSAPI\_SharedMemorySegment, [215](#)
- OSAPI\_SharedMemorySegment\_is\_owner\_alive
  - OSAPI\_SharedMemorySegment, [215](#)
- OSAPI\_SharedMemorySegment\_lock\_impl
  - OSAPI\_SharedMemorySegment, [217](#)
- OSAPI\_SharedMemorySegment\_mark\_consistent\_impl
  - OSAPI\_SharedMemorySegment, [218](#)
- OSAPI\_SharedMemorySegment\_unlock\_impl
  - OSAPI\_SharedMemorySegment, [218](#)
- OSAPI\_SharedMemorySegmentHandle, [839](#)
  - mode, [840](#)
  - ptr\_header, [839](#)
  - ptr\_user\_data, [839](#)
- OSAPI\_SharedMemorySegmentHandle\_get\_size
  - OSAPI\_SharedMemorySegment, [215](#)
- OSAPI\_SharedMemorySegmentHeader, [840](#)
  - mode, [840](#)
  - size, [840](#)
- OSAPI\_SharedMemorySegmentHeader\_get\_size
  - OSAPI\_SharedMemorySegment, [215](#)
- OSAPI\_SHMEM\_ATTACH\_STATUS
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_ATTACH\_STATUS\_OK
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_ATTACH\_STATUS\_SEGMENT\_NOT\_FOUND
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_MODE
  - OSAPI\_SharedMemorySegment, [214](#)

- OSAPI\_SHMEM\_MODE\_LOCKABLE
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_MODE\_READ
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_MODE\_ROBUST
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_MODE\_UNKNOWN
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_MODE\_WRITE
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_STATUS
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_STATUS\_OK
  - OSAPI\_SharedMemorySegment, [214](#)
- OSAPI\_SHMEM\_STATUS\_OWNER\_DEAD
  - OSAPI\_SharedMemorySegment, [214](#)
- osapi\_stdio.h, [935](#)
- OSAPI\_System\_generate\_uuid\_T
  - OSAPI System API, [201](#)
- OSAPI\_System\_get\_hostname\_T
  - OSAPI System API, [200](#)
- OSAPI\_System\_get\_ticktime\_T
  - OSAPI System API, [201](#)
- OSAPI\_System\_get\_time\_T
  - OSAPI System API, [200](#)
- OSAPI\_System\_get\_timer\_resolution\_T
  - OSAPI System API, [199](#)
- OSAPI\_System\_initialize\_T
  - OSAPI System API, [200](#)
- OSAPI\_SYSTEM\_MAX\_HOSTNAME
  - OSAPI System API, [199](#)
- OSAPI\_System\_start\_timer\_T
  - OSAPI System API, [199](#)
- OSAPI\_SystemI, [841](#)
  - generate\_uuid, [842](#)
  - get\_hostname, [842](#)
  - get\_ticktime, [842](#)
  - get\_time, [841](#)
  - get\_timer\_resolution, [841](#)
  - OSAPI System API, [201](#)
  - start\_timer, [841](#)
- OSAPI\_SystemProperty, [842](#)
  - hostname, [843](#)
  - max\_timers, [843](#)
  - max\_user\_blocking\_threads, [843](#)
  - timer\_property, [843](#)
- OSAPI\_SystemTime, [844](#)
  - nanosec, [845](#)
  - OSAPI Time, [205](#)
  - sec, [845](#)
- osapi\_task.h, [936](#)
- OSAPI\_THREAD\_DEFAULT\_OPTIONS
  - OSAPI Thread API, [204](#)
- OSAPI\_THREAD\_FLOATING\_POINT
  - OSAPI Thread API, [204](#)
- OSAPI\_THREAD\_PRIORITY\_ABOVE\_NORMAL
  - OSAPI Thread API, [203](#)
- OSAPI\_THREAD\_PRIORITY\_BELOW\_NORMAL
  - OSAPI Thread API, [203](#)
- OSAPI\_THREAD\_PRIORITY\_HIGH
  - OSAPI Thread API, [203](#)
- OSAPI\_THREAD\_PRIORITY\_INHERIT
  - OSAPI Thread API, [203](#)
- OSAPI\_THREAD\_PRIORITY\_LOW
  - OSAPI Thread API, [203](#)
- OSAPI\_THREAD\_PRIORITY\_NORMAL
  - OSAPI Thread API, [203](#)
- OSAPI\_THREAD\_PROPERTY\_DEFAULT
  - OSAPI Thread API, [204](#)
- OSAPI\_THREAD\_REALTIME\_PRIORITY
  - OSAPI Thread API, [204](#)
- OSAPI\_THREAD\_STDIO
  - OSAPI Thread API, [204](#)
- OSAPI\_THREAD\_USE\_OSDEFAULT\_STACKSIZE
  - OSAPI Thread API, [204](#)
- OSAPI\_ThreadOptions
  - OSAPI Thread API, [205](#)
- OSAPI\_ThreadProperty, [845](#)
  - options, [846](#)
  - priority, [845](#)
  - stack\_size, [845](#)
- OSAPI\_ThreadProperty\_INITIALIZER
  - OSAPI Thread API, [204](#)
- osapi\_time.h, [936](#)
- OSAPI\_TimerProperty, [846](#)
  - thread, [846](#)
- OWNERSHIP, [105](#)
  - DDS\_EXCLUSIVE\_OWNERSHIP\_QOS, [106](#)
  - DDS\_OwnershipQosPolicyKind, [106](#)
  - DDS\_SHARED\_OWNERSHIP\_QOS, [106](#)
- ownership
  - DDS\_DataReaderQos, [506](#)
  - DDS\_DataWriterQos, [516](#)
  - DDS\_PublicationBuiltinTopicData, [592](#)
  - DDS\_SubscriptionBuiltinTopicData, [627](#)
- OWNERSHIP\_STRENGTH, [106](#)
- ownership\_strength
  - DDS\_DataWriterQos, [516](#)
  - DDS\_PublicationBuiltinTopicData, [592](#)
- Participant Built-in Topic, [39](#)
  - DDS\_PARTICIPANT\_TOPIC\_NAME, [40](#)
  - DDS\_ParticipantBuiltinTopicData, [40, 41](#)
  - DDS\_ParticipantBuiltinTopicData\_finalize, [41](#)
  - DDS\_ParticipantBuiltinTopicData\_is\_equal, [41](#)
- participant\_id
  - DDS\_WireProtocolQosPolicy, [640](#)
- participant\_id\_gain

- DDS\_RtpsWellKnownPorts\_t, 613
- participant\_key
  - DDS\_PublicationBuiltinTopicData, 591
  - DDS\_SubscriptionBuiltinTopicData, 626
- participant\_liveliness\_assert\_period
  - DPDE\_DiscoveryPluginProperty, 772
  - DPSE\_DiscoveryPluginProperty, 777
- participant\_liveliness\_lease\_duration
  - DPDE\_DiscoveryPluginProperty, 772
  - DPSE\_DiscoveryPluginProperty, 777
- participant\_message\_reader\_reliability\_kind
  - DPDE\_DiscoveryPluginProperty, 775
  - DPSE\_DiscoveryPluginProperty, 778
- participant\_name
  - DDS\_DomainParticipantQos, 530
  - DDS\_ParticipantBuiltinTopicData, 575
- participant\_user\_data\_max\_count
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 537
- participant\_user\_data\_max\_length
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 536
- PARTITION, 129
- partition
  - DDS\_PublicationBuiltinTopicData, 593
  - DDS\_PublisherQos, 596
  - DDS\_SubscriberQos, 624
  - DDS\_SubscriptionBuiltinTopicData, 628
- period
  - DDS\_DeadlineQosPolicy, 523
  - DDS\_FlowControllerTokenBucketProperty\_t, 552
- policies
  - DDS\_OfferedIncompatibleQosStatus, 568
  - DDS\_RequestedIncompatibleQosStatus, 605
- port
  - DDS\_Locator\_t, 565
  - NETIO\_Address, 825
- port\_base
  - DDS\_RtpsWellKnownPorts\_t, 612
- PRESENTATION, 115
  - DDS\_GROUP\_PRESENTATION\_QOS, 116
  - DDS\_INSTANCE\_PRESENTATION\_QOS, 116
  - DDS\_PresentationQosPolicyAccessScopeKind, 116
  - DDS\_TOPIC\_PRESENTATION\_QOS, 116
- presentation
  - DDS\_SubscriptionBuiltinTopicData, 628
- priority
  - DDS\_PublishModeQosPolicy, 598
  - OSAPI\_ThreadProperty, 845
- pro\_minimum\_compatibility\_version
  - NETIO\_SHMEMInterfaceFactoryProperty, 838
- product\_version
  - DDS\_ParticipantBuiltinTopicData, 576
- propagate
  - DDS\_Property\_t, 586
- PROPERTY, 129
- property
  - DDS\_DataReaderQos, 508
  - DDS\_DataWriterQos, 517
  - DDS\_DomainParticipantQos, 531
  - DDS\_ParticipantBuiltinTopicData, 576
- Property Reference, 139
  - DDS\_XTYPES\_COMPLIANCE\_MASK\_PROPERTY, 140
- protocol
  - DDS\_DataReaderQos, 507
  - DDS\_DataWriterQos, 517
  - DDS\_DomainParticipantQos, 530
- provider\_name
  - DDS\_SecTransform\_Configuration, 622
- PSK\_ErrListener, 846
  - callback, 847
  - user\_ptr, 847
- psl\_config
  - DDS\_PskServiceFactoryProperty, 589
- psl\_get\_interface\_func
  - DDS\_PskServiceFactoryProperty, 589
- ptr\_header
  - OSAPI\_SharedMemorySegmentHandle, 839
- ptr\_user\_data
  - OSAPI\_SharedMemorySegmentHandle, 839
- Publication Built-in Topic, 42
  - DDS\_PUBLICATION\_TOPIC\_NAME, 42
  - DDS\_PublicationBuiltinTopicData, 42, 43
  - DDS\_PublicationBuiltinTopicData\_finalize, 43
  - DDS\_PublicationBuiltinTopicData\_is\_equal, 43
- Publication Module, 57
- publication\_handle
  - DDS\_DataReaderInstanceReplacedStatus, 503
  - DDS\_SampleInfo, 617
- publication\_name
  - DDS\_DataWriterQos, 518
- publication\_sequence\_number
  - DDS\_SampleInfo, 618
- PUBLISH\_MODE, 127
  - DDS\_ASYNCHRONOUS\_PUBLISH\_MODE\_QOS, 129
  - DDS\_AUTOMATIC\_PUBLISH\_MODE\_QOS, 129
  - DDS\_PUBLICATION\_PRIORITY\_AUTOMATIC, 128
  - DDS\_PUBLICATION\_PRIORITY\_UNDEFINED, 128
  - DDS\_PublishModeQosPolicyKind, 128
  - DDS\_SYNCHRONOUS\_PUBLISH\_MODE\_QOS, 128
- publish\_mode
  - DDS\_DataWriterQos, 517
- Publisher, 63
  - DDS\_DATAWRITER\_QOS\_DEFAULT, 65
  - DDS\_Publisher, 64

publisher\_group\_data\_max\_count  
     DDS\_DomainParticipantResourceLimitsQosPolicy, [537](#)  
 publisher\_group\_data\_max\_length  
     DDS\_DomainParticipantResourceLimitsQosPolicy, [537](#)  
 publisher\_name  
     DDS\_PublisherQos, [596](#)  
 QoS Policies, [97](#)  
     DDS\_DEADLINE\_QOS\_POLICY\_ID, [101](#)  
     DDS\_ENTITYFACTORY\_QOS\_POLICY\_ID, [101](#)  
     DDS\_HISTORY\_QOS\_POLICY\_ID, [101](#)  
     DDS\_INVALID\_QOS\_POLICY\_ID, [101](#)  
     DDS\_LIVELINESS\_QOS\_POLICY\_ID, [101](#)  
     DDS\_OWNERSHIP\_QOS\_POLICY\_ID, [101](#)  
     DDS\_OWNERSHIPSTRENGTH\_QOS\_POLICY\_ID, [101](#)  
     DDS\_QosPolicyId\_t, [101](#)  
     DDS\_RELIABILITY\_QOS\_POLICY\_ID, [101](#)  
 read  
     FooDataReader, [784](#)  
 read\_instance  
     FooDataReader, [788](#)  
 read\_instance\_untyped  
     DDSDDataReader, [654](#)  
 read\_next\_sample  
     FooDataReader, [783](#)  
 read\_next\_sample\_untyped  
     DDSDDataReader, [656](#)  
 read\_untyped  
     DDSDDataReader, [650](#)  
 reader\_resource\_limits  
     DDS\_DataReaderQos, [507](#)  
 reader\_user\_data\_max\_count  
     DDS\_DomainParticipantResourceLimitsQosPolicy, [539](#)  
 reader\_user\_data\_max\_length  
     DDS\_DomainParticipantResourceLimitsQosPolicy, [538](#)  
 reason  
     DDS\_SampleLostStatus, [620](#)  
 receive\_buffer\_size  
     NETIO\_SHMEMInterfaceFactoryProperty, [837](#)  
 received\_message\_count\_max  
     NETIO\_SHMEMInterfaceFactoryProperty, [837](#)  
 reception\_timestamp  
     DDS\_SampleInfo, [618](#)  
 recv\_thread  
     UDP\_InterfaceFactoryProperty, [855](#)  
 recv\_thread\_property  
     NETIO\_SHMEMInterfaceFactoryProperty, [838](#)  
 REDA, [264](#)

NETIO\_SHMEM\_LOG\_CQ\_INCOMPATIBLE\_VERSION\_EC, [270](#)  
 NETIO\_SHMEM\_LOG\_CQ\_INCONSISTENT\_STATE\_EC, [270](#)  
 NETIO\_SHMEM\_LOG\_CQ\_INT\_REPRESENTATION\_EC, [270](#)  
 NETIO\_SHMEM\_LOG\_CQ\_INVALID\_SIGNATURE\_EC, [270](#)  
 NETIO\_SHMEM\_LOG\_CQ\_UNLINKED\_EC, [270](#)  
 REDA\_LOG\_BUFFER\_OUT\_OF\_MEMORY\_EC, [269](#)  
 REDA\_LOG\_BUFFERPOOL\_BUFFER\_INITIALIZATION\_FAILED\_EC, [266](#)  
 REDA\_LOG\_BUFFERPOOL\_DOUBLE\_FREE\_EC, [267](#)  
 REDA\_LOG\_BUFFERPOOL\_NOT\_EMPTY\_EC, [267](#)  
 REDA\_LOG\_BUFFERPOOL\_NULL\_POINTER\_EC, [267](#)  
 REDA\_LOG\_BUFFERPOOL\_OUT\_OF\_RESOURCES\_EC, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_ALLOC\_EC, [269](#)  
 REDA\_LOG\_HEAP\_MGR\_GET\_BUFFER\_EC, [269](#)  
 REDA\_LOG\_HEAP\_MGR\_INDEXER, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_LISTNODE, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_MUTEX, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_MUTEX\_FAILURE\_EC, [269](#)  
 REDA\_LOG\_HEAP\_MGR\_OBJECT\_MEMORY, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_SAMPLE\_BUFFERPOOL, [266](#)  
 REDA\_LOG\_INDEX\_ENTRY\_EXISTS\_EC, [269](#)  
 REDA\_LOG\_INDEX\_FULL\_EC, [269](#)  
 REDA\_LOG\_INVALID\_LIST\_NODE\_EC, [269](#)  
 REDA\_LOG\_MEMPOOL\_INCONSISTENT\_PROPERTIES\_EC, [267](#)  
 REDA\_LOG\_MEMPOOL\_OUT\_OF\_RESOURCES\_EC, [267](#)  
 REDA\_LOG\_MEMPOOL\_POOL\_ALLOCATION\_EC, [267](#)  
 REDA\_LOG\_SEQUENCE\_ALLOC\_FAILED\_EC, [268](#)  
 REDA\_LOG\_SEQUENCE\_COPY\_FAILED\_EC, [267](#)  
 REDA\_LOG\_SEQUENCE\_INDEX\_OUT\_OF\_BOUNDS\_EC, [268](#)  
 REDA\_LOG\_SEQUENCE\_INVALID\_LENGTH\_EC, [268](#)  
 REDA\_LOG\_SEQUENCE\_INVALID\_OPERATION\_EC, [268](#)  
 REDA\_LOG\_SEQUENCE\_REALLOCATION\_EC, [268](#)  
 REDA\_LOG\_SEQUENCE\_SET\_MAX\_FAILED\_EC, [268](#)  
 REDA\_LOG\_STRING\_ALLOC\_FAILED\_EC, [268](#)

reda\_log.h, [937](#)  
 REDA\_LOG\_BASE  
     osapi\_log.h, [930](#)  
 REDA\_LOG\_BUFFER\_OUT\_OF\_MEMORY\_EC  
     REDA, [269](#)  
 REDA\_LOG\_BUFFERPOOL\_BUFFER\_INITIALIZATION\_FAILED\_EC  
     REDA, [266](#)  
 REDA\_LOG\_BUFFERPOOL\_DOUBLE\_FREE\_EC  
     REDA, [267](#)  
 REDA\_LOG\_BUFFERPOOL\_NOT\_EMPTY\_EC  
     REDA, [267](#)  
 REDA\_LOG\_BUFFERPOOL\_NULL\_POINTER\_EC  
     REDA, [267](#)  
 REDA\_LOG\_BUFFERPOOL\_OUT\_OF\_RESOURCES\_EC  
     REDA, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_ALLOC\_EC  
     REDA, [269](#)  
 REDA\_LOG\_HEAP\_MGR\_GET\_BUFFER\_EC  
     REDA, [269](#)  
 REDA\_LOG\_HEAP\_MGR\_INDEXER  
     REDA, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_LISTNODE  
     REDA, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_MUTEX  
     REDA, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_MUTEX\_FAILURE\_EC  
     REDA, [269](#)  
 REDA\_LOG\_HEAP\_MGR\_OBJECT\_MEMORY  
     REDA, [266](#)  
 REDA\_LOG\_HEAP\_MGR\_SAMPLE\_BUFFERPOOL  
     REDA, [266](#)  
 REDA\_LOG\_INDEX\_ENTRY\_EXISTS\_EC  
     REDA, [269](#)  
 REDA\_LOG\_INDEX\_FULL\_EC  
     REDA, [269](#)  
 REDA\_LOG\_INVALID\_LIST\_NODE\_EC  
     REDA, [269](#)  
 REDA\_LOG\_MEMPOOL\_INCONSISTENT\_PROPERTIES\_EC  
     REDA, [267](#)  
 REDA\_LOG\_MEMPOOL\_OUT\_OF\_RESOURCES\_EC  
     REDA, [267](#)  
 REDA\_LOG\_MEMPOOL\_POOL\_ALLOCATION\_EC  
     REDA, [267](#)  
 REDA\_LOG\_SEQUENCE\_ALLOC\_FAILED\_EC  
     REDA, [268](#)  
 REDA\_LOG\_SEQUENCE\_COPY\_FAILED\_EC  
     REDA, [267](#)  
 REDA\_LOG\_SEQUENCE\_INDEX\_OUT\_OF\_BOUNDS\_EC  
     REDA, [268](#)  
 REDA\_LOG\_SEQUENCE\_INVALID\_LENGTH\_EC  
     REDA, [268](#)  
 REDA\_LOG\_SEQUENCE\_INVALID\_OPERATION\_EC  
     REDA, [268](#)  
 REDA\_LOG\_SEQUENCE\_REALLOCATION\_EC  
     REDA, [268](#)  
 REDA\_LOG\_SEQUENCE\_SET\_MAX\_FAILED\_EC  
     REDA, [268](#)  
 REDA\_LOG\_STRING\_ALLOC\_FAILED\_EC  
     REDA, [268](#)  
 register\_instance  
     FooDataWriter, [795](#)  
 register\_instance\_untyped  
     DDSDDataWriter, [672](#)  
 register\_instance\_w\_timestamp  
     FooDataWriter, [796](#)  
 register\_instance\_w\_timestamp\_untyped  
     DDSDDataWriter, [673](#)  
 register\_type  
     DDSDomainParticipant, [702](#)  
     FooTypeSupport, [820](#)  
 related\_topic\_name  
     DDS\_ContentFilterProperty, [499](#)  
 release  
     DDS\_ProductVersion, [584](#)  
     DDS\_ProductVersion\_t, [585](#)  
 release\_address  
     NETIO\_DGRAM\_InterfaceI, [829](#)  
     ZCOPY\_NotifUserInterfaceI, [865](#)  
 RELIABILITY, [108](#)  
     DDS\_BEST\_EFFORT\_RELIABILITY\_QOS, [109](#)  
     DDS\_ReliabilityQosPolicyKind, [109](#)  
     DDS\_RELIABLE\_RELIABILITY\_QOS, [109](#)  
 reliability  
     DDS\_DataReaderQos, [506](#)  
     DDS\_DataWriterQos, [516](#)  
     DDS\_PublicationBuiltinTopicData, [592](#)  
     DDS\_SubscriptionBuiltinTopicData, [627](#)  
 remote\_participant\_allocation, [231](#)  
     DDS\_DomainParticipantResourceLimitsQosPolicy, [535](#)  
 remote\_reader\_allocation, [232](#)  
     DDS\_DomainParticipantResourceLimitsQosPolicy, [535](#)  
 remote\_writer\_allocation, [232](#)  
     DDS\_DomainParticipantResourceLimitsQosPolicy, [535](#)  
 RemoteParticipant\_assert  
     DPSEDiscoveryPlugin, [780](#)  
 RemotePublication\_assert  
     DPSEDiscoveryPlugin, [780](#)  
 RemoteSubscription\_assert  
     DPSEDiscoveryPlugin, [780](#)  
 remove\_property  
     DDSPropertyQosPolicyHelper, [737](#)  
 representation  
     DDS\_DataReaderQos, [507](#)

- DDS\_DataWriterQos, [517](#)
  - DDS\_PublicationBuiltinTopicData, [593](#)
  - DDS\_SubscriptionBuiltinTopicData, [628](#)
  - require\_crc
    - DDS\_ChecksumProperty, [497](#)
    - DDS\_ChecksumProperty\_t, [498](#)
    - DDS\_WireProtocolQosPolicy, [642](#)
  - reserve\_address
    - ZCOPY\_NotifUserInterfacel, [865](#)
  - resolve\_address
    - NETIO\_DGRAM\_Interfacel, [829](#)
    - ZCOPY\_NotifUserInterfacel, [864](#)
  - Resource Limits, [227](#)
  - RESOURCE\_LIMITS, [114](#)
    - DDS\_LENGTH\_AUTO, [115](#)
    - DDS\_LENGTH\_UNLIMITED, [115](#)
    - DDS\_SIZE\_AUTO, [115](#)
  - resource\_limits
    - DDS\_DataReaderQos, [506](#)
    - DDS\_DataWriterQos, [516](#)
    - DDS\_DomainParticipantFactoryQos, [529](#)
    - DDS\_DomainParticipantQos, [530](#)
    - DDS\_FilterQosPolicy, [545](#)
  - Return Codes, [86](#)
    - DDS\_RETCODE\_ALREADY\_DELETED, [88](#)
    - DDS\_RETCODE\_BAD\_PARAMETER, [88](#)
    - DDS\_RETCODE\_ERROR, [88](#)
    - DDS\_RETCODE\_ILLEGAL\_OPERATION, [88](#)
    - DDS\_RETCODE\_IMMUTABLE\_POLICY, [88](#)
    - DDS\_RETCODE\_INCONSISTENT\_POLICY, [88](#)
    - DDS\_RETCODE\_NO\_DATA, [88](#)
    - DDS\_RETCODE\_NOT\_ALLOWED\_BY\_SEC, [88](#)
    - DDS\_RETCODE\_NOT\_ENABLED, [88](#)
    - DDS\_RETCODE\_OK, [88](#)
    - DDS\_RETCODE\_OUT\_OF\_RESOURCES, [88](#)
    - DDS\_RETCODE\_PRECONDITION\_NOT\_MET, [88](#)
    - DDS\_RETCODE\_TIMEOUT, [88](#)
    - DDS\_RETCODE\_UNSUPPORTED, [88](#)
    - DDS\_ReturnCode\_t, [87](#)
  - return\_loan
    - FooDataReader, [782](#)
  - return\_loan\_untyped
    - DDSDataReader, [658](#)
  - revision
    - DDS\_ProductVersion, [584](#)
    - DDS\_ProductVersion\_t, [585](#)
  - RH, [450](#)
    - RHSM\_LOG\_DESTINATION\_BY\_SOURCE\_NOT\_SUPPORTED\_EC, [455](#)
    - RHSM\_LOG\_ENTRY\_RESERVATION\_FAILED\_EC, [453](#)
    - RHSM\_LOG\_FAILED\_TO\_REMOVE\_OLDEST\_EC, [453](#)
  - RHSM\_LOG\_GET\_RECEPTION\_TIMESTAMP\_EC, [453](#)
  - RHSM\_LOG\_GET\_TIME\_EC, [454](#)
  - RHSM\_LOG\_HISTORY\_UPDATE\_EC, [454](#)
  - RHSM\_LOG\_INFO\_ARRAY\_EC, [452](#)
  - RHSM\_LOG\_INVALID\_INSTANCE\_REPLACEMENT\_EC, [453](#)
  - RHSM\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC, [452](#)
  - RHSM\_LOG\_MAX\_SAMPLE\_TOO\_SMALL\_EC, [452](#)
  - RHSM\_LOG\_NO\_PROPERTY\_EC, [455](#)
  - RHSM\_LOG\_NO\_PROPERTY\_QOS\_EC, [451](#)
  - RHSM\_LOG\_OBJECT\_ALLOCATE\_EC, [454](#)
  - RHSM\_LOG\_OBJECT\_DELETE\_EC, [454](#)
  - RHSM\_LOG\_OBJECT\_INDEX\_EC, [454](#)
  - RHSM\_LOG\_OBJECT\_INVALID\_EC, [454](#)
  - RHSM\_LOG\_OUTSTANDING\_SAMPLE\_EC, [451](#)
  - RHSM\_LOG\_RECOMMIT\_FAILURE\_EC, [455](#)
  - RHSM\_LOG\_RETURN\_SAMPLE\_EC, [454](#)
  - RHSM\_LOG\_RW\_PRUNE\_FAILED\_EC, [453](#)
  - RHSM\_LOG\_SAMPLE\_INFO\_POOL\_EC, [452](#)
  - RHSM\_LOG\_SAMPLE\_POOL\_EC, [452](#)
  - RHSM\_LOG\_SAMPLE\_POOL\_EMPTY\_EC, [453](#)
  - RHSM\_LOG\_SAMPLE\_PTR\_ARRAY\_EC, [452](#)
  - RHSM\_LOG\_TBF\_NOT\_SUPPORTED\_EC, [452](#)
- rh\_sm\_log.h, [938](#)
  - RHSM\_LOG\_HISTORY\_OBJECT, [940](#)
  - RHSM\_LOG\_INSTANCEHANDLE\_OBJECT, [942](#)
  - RHSM\_LOG\_KEY\_OBJECT, [941](#)
  - RHSM\_LOG\_KEYINDEXPOOL\_OBJECT, [941](#)
  - RHSM\_LOG\_KEYPOOL\_OBJECT, [941](#)
  - RHSM\_LOG\_OWNER\_OBJECT, [941](#)
  - RHSM\_LOG\_REMOVELIST\_OBJECT, [940](#)
  - RHSM\_LOG\_RW\_OBJECT, [941](#)
  - RHSM\_LOG\_RWINDEX\_OBJECT, [940](#)
  - RHSM\_LOG\_RWPOOL\_OBJECT, [940](#)
  - RHSM\_LOG\_SAMPLEINFO\_OBJECT, [942](#)
  - RHSM\_LOG\_SAMPLEPOOL\_OBJECT, [941](#)
  - RHSM\_LOG\_SAMPLEPTPOOL\_OBJECT, [941](#)
- RHSM Reader History API, [145](#)
- RHSM\_LOG\_BASE
  - osapi\_log.h, [931](#)
- RHSM\_LOG\_DESTINATION\_BY\_SOURCE\_NOT\_SUPPORTED\_EC
  - RH, [455](#)
- RHSM\_LOG\_ENTRY\_RESERVATION\_FAILED\_EC
  - RH, [453](#)
- RHSM\_LOG\_FAILED\_TO\_REMOVE\_OLDEST\_EC
  - RH, [453](#)
- RHSM\_LOG\_GET\_RECEPTION\_TIMESTAMP\_EC
  - RH, [453](#)
- RHSM\_LOG\_GET\_TIME\_EC
  - RH, [454](#)
- RHSM\_LOG\_HISTORY\_OBJECT



- rh\_sm\_log.h, [940](#)
- RHSM\_LOG\_HISTORY\_UPDATE\_EC
  - RH, [454](#)
- RHSM\_LOG\_INFO\_ARRAY\_EC
  - RH, [452](#)
- RHSM\_LOG\_INSTANCEHANDLE\_OBJECT
  - rh\_sm\_log.h, [942](#)
- RHSM\_LOG\_INVALID\_INSTANCE\_REPLACEMENT\_EC
  - RH, [453](#)
- RHSM\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC
  - RH, [452](#)
- RHSM\_LOG\_KEY\_OBJECT
  - rh\_sm\_log.h, [941](#)
- RHSM\_LOG\_KEYINDEXPOOL\_OBJECT
  - rh\_sm\_log.h, [941](#)
- RHSM\_LOG\_KEYPOOL\_OBJECT
  - rh\_sm\_log.h, [941](#)
- RHSM\_LOG\_MAX\_SAMPLE\_TOO\_SMALL\_EC
  - RH, [452](#)
- RHSM\_LOG\_NO\_PROPERTY\_EC
  - RH, [455](#)
- RHSM\_LOG\_NO\_PROPERTY\_QOS\_EC
  - RH, [451](#)
- RHSM\_LOG\_OBJECT\_ALLOCATE\_EC
  - RH, [454](#)
- RHSM\_LOG\_OBJECT\_DELETE\_EC
  - RH, [454](#)
- RHSM\_LOG\_OBJECT\_INDEX\_EC
  - RH, [454](#)
- RHSM\_LOG\_OBJECT\_INVALID\_EC
  - RH, [454](#)
- RHSM\_LOG\_OUTSTANDING\_SAMPLE\_EC
  - RH, [451](#)
- RHSM\_LOG\_OWNER\_OBJECT
  - rh\_sm\_log.h, [941](#)
- RHSM\_LOG\_RECOMMIT\_FAILURE\_EC
  - RH, [455](#)
- RHSM\_LOG\_REMOVE\_LIST\_OBJECT
  - rh\_sm\_log.h, [940](#)
- RHSM\_LOG\_RETURN\_SAMPLE\_EC
  - RH, [454](#)
- RHSM\_LOG\_RW\_OBJECT
  - rh\_sm\_log.h, [941](#)
- RHSM\_LOG\_RW\_PRUNE\_FAILED\_EC
  - RH, [453](#)
- RHSM\_LOG\_RWINDEX\_OBJECT
  - rh\_sm\_log.h, [940](#)
- RHSM\_LOG\_RWPOOL\_OBJECT
  - rh\_sm\_log.h, [940](#)
- RHSM\_LOG\_SAMPLE\_INFO\_POOL\_EC
  - RH, [452](#)
- RHSM\_LOG\_SAMPLE\_POOL\_EC
  - RH, [452](#)
- RHSM\_LOG\_SAMPLE\_POOL\_EMPTY\_EC
  - RH, [453](#)
- RHSM\_LOG\_SAMPLE\_PTR\_ARRAY\_EC
  - RH, [452](#)
- RHSM\_LOG\_SAMPLEINFO\_OBJECT
  - rh\_sm\_log.h, [942](#)
- RHSM\_LOG\_SAMPLEPOOL\_OBJECT
  - rh\_sm\_log.h, [941](#)
- RHSM\_LOG\_SAMPLEPTPOOL\_OBJECT
  - rh\_sm\_log.h, [941](#)
- RHSM\_LOG\_TBF\_NOT\_SUPPORTED\_EC
  - RH, [452](#)
- RHSMHistoryFactory, [847](#)
  - get\_interface, [848](#)
- RT, [276](#)
  - RT\_LOG\_REGISTRY\_EXISTS\_EC, [277](#)
  - RT\_LOG\_REGISTRY\_FINALIZE\_EC, [277](#)
  - RT\_LOG\_REGISTRY\_INCONSISTENT\_CID\_EC, [278](#)
  - RT\_LOG\_REGISTRY\_INIT\_FACTORY\_EC, [277](#)
  - RT\_LOG\_REGISTRY\_INIT\_FAILURE\_EC, [277](#)
  - RT\_LOG\_REGISTRY\_NAME\_TOO\_LONG\_EC, [277](#)
  - RT\_LOG\_REGISTRY\_NOT\_INITIALIZED\_EC, [278](#)
  - RT\_LOG\_REGISTRY\_REGISTER\_EC, [277](#)
  - RT\_LOG\_SET\_IMMUTABLE\_PROPERTY\_EC, [277](#)
- RT Run-Time API, [145](#)
- rt\_log.h, [942](#)
- RT\_LOG\_BASE
  - osapi\_log.h, [930](#)
- RT\_LOG\_REGISTRY\_EXISTS\_EC
  - RT, [277](#)
- RT\_LOG\_REGISTRY\_FINALIZE\_EC
  - RT, [277](#)
- RT\_LOG\_REGISTRY\_INCONSISTENT\_CID\_EC
  - RT, [278](#)
- RT\_LOG\_REGISTRY\_INIT\_FACTORY\_EC
  - RT, [277](#)
- RT\_LOG\_REGISTRY\_INIT\_FAILURE\_EC
  - RT, [277](#)
- RT\_LOG\_REGISTRY\_NAME\_TOO\_LONG\_EC
  - RT, [277](#)
- RT\_LOG\_REGISTRY\_NOT\_INITIALIZED\_EC
  - RT, [278](#)
- RT\_LOG\_REGISTRY\_REGISTER\_EC
  - RT, [277](#)
- RT\_LOG\_SET\_IMMUTABLE\_PROPERTY\_EC
  - RT, [277](#)
- RTI Connex Micro C++ API, [1](#)
- RTI\_BOOL
  - OSAPI Types, [208](#)
- RTI\_FALSE
  - OSAPI Types, [208](#)
- RTI\_INT16
  - OSAPI Types, [207](#)
- RTI\_INT32

- OSAPI Types, [207](#)
- RTI\_INT64
  - OSAPI Types, [207](#)
- RTI\_INT8
  - OSAPI Types, [206](#)
- RTI\_SIZE\_T
  - OSAPI Types, [208](#)
- RTI\_TRUE
  - OSAPI Types, [208](#)
- RTI\_UINT16
  - OSAPI Types, [207](#)
- RTI\_UINT32
  - OSAPI Types, [207](#)
- RTI\_UINT64
  - OSAPI Types, [208](#)
- RTI\_UINT8
  - OSAPI Types, [206](#)
- RTPS, [306](#)
  - RTPS\_LOG\_ACK\_EC, [319](#)
  - RTPS\_LOG\_ADD\_FILTERED\_ROUTE\_EC, [336](#)
  - RTPS\_LOG\_ALLOCATE\_EC, [313](#)
  - RTPS\_LOG\_APPLY\_CONTENT\_FILTER\_EC, [336](#)
  - RTPS\_LOG\_ASSERT\_PEER\_EC, [318](#)
  - RTPS\_LOG\_ASSERT\_TRANSPORT\_EC, [318](#)
  - RTPS\_LOG\_BAD\_PARAMETER\_EC, [317](#)
  - RTPS\_LOG\_BITCOUNT\_OUT\_OF\_BOUNDS\_EC, [322](#)
  - RTPS\_LOG\_BITMAP\_SET\_BIT\_EC, [324](#)
  - RTPS\_LOG\_BITMAP\_SHIFT\_EC, [327](#)
  - RTPS\_LOG\_BUFFERPOOL\_DELETE\_EC, [326](#)
  - RTPS\_LOG\_CHECKSUM\_CHECKSUM\_CALCULATION\_FAILED\_EC, [333](#)
  - RTPS\_LOG\_CHECKSUM\_CHECKSUM\_DESERIALIZE\_ERROR\_EC, [335](#)
  - RTPS\_LOG\_CHECKSUM\_CHECKSUM\_ERROR\_EC, [335](#)
  - RTPS\_LOG\_CHECKSUM\_ILLEGAL\_OVERRIDE\_CHECKSUM\_EC, [333](#)
  - RTPS\_LOG\_CHECKSUM\_ILLEGAL\_OVERRIDE\_CHECKSUM\_EC, [333](#)
  - RTPS\_LOG\_CHECKSUM\_ILLEGAL\_OVERRIDE\_CHECKSUM\_EC, [333](#)
  - RTPS\_LOG\_CHECKSUM\_INCONSISTENT\_LENGTH\_EC, [335](#)
  - RTPS\_LOG\_CHECKSUM\_INVALID\_CRC32\_FLAGS\_EC, [334](#)
  - RTPS\_LOG\_CHECKSUM\_INVALID\_OVERRIDE\_BUILTIN1\_EC, [333](#)
  - RTPS\_LOG\_CHECKSUM\_INVALID\_OVERRIDE\_BUILTIN3\_EC, [333](#)
  - RTPS\_LOG\_CHECKSUM\_INVALID\_OVERRIDE\_BUILTIN6\_EC, [333](#)
  - RTPS\_LOG\_CHECKSUM\_LENGTH\_DESERIALIZE\_ERROR\_EC, [335](#)
  - RTPS\_LOG\_CHECKSUM\_MISSING\_CHECKSUM\_LENGTH\_EC, [334](#)
  - RTPS\_LOG\_CHECKSUM\_TOO\_MANY\_SG\_BUFFERS\_EC, [335](#)
  - RTPS\_LOG\_CHECKSUM\_UNSUPPORTED\_EC, [335](#)
  - RTPS\_LOG\_CREATE\_BIND\_TABLE\_EC, [325](#)
  - RTPS\_LOG\_CREATE\_BUFFER\_POOL\_EC, [328](#)
  - RTPS\_LOG\_CREATE\_EVENT\_TIMER\_EC, [322](#)
  - RTPS\_LOG\_CREATE\_HB\_EVENT\_EC, [322](#)
  - RTPS\_LOG\_CREATE\_PEERS\_INDEX\_EC, [329](#)
  - RTPS\_LOG\_CREATE\_ROUTE\_TABLE\_EC, [313](#)
  - RTPS\_LOG\_DATA\_ALREADY\_RECEIVED\_EC, [320](#)
  - RTPS\_LOG\_DATA\_OUT\_OF\_RANGE\_EC, [320](#)
  - RTPS\_LOG\_DB\_CREATE\_BIND\_ENTRY\_EC, [315](#)
  - RTPS\_LOG\_DB\_CREATE\_BIND\_PEER\_INDEX\_EC, [314](#)
  - RTPS\_LOG\_DB\_CREATE\_DIRECT\_INDEX\_EC, [329](#)
  - RTPS\_LOG\_DB\_CREATE\_EXT\_BIND\_ENTRY\_EC, [324](#)
  - RTPS\_LOG\_DB\_CREATE\_GROUP\_INDEX\_EC, [329](#)
  - RTPS\_LOG\_DB\_CREATE\_ROUTE\_ENTRY\_EC, [314](#)
  - RTPS\_LOG\_DB\_CREATE\_ROUTE\_PEER\_INDEX\_EC, [313](#)
  - RTPS\_LOG\_DB\_CREATE\_ROUTE\_XPORT\_INDEX\_EC, [313](#)
  - RTPS\_LOG\_DB\_CREATE\_SELECTED\_GROUP\_INDEX\_EC, [313](#)
  - RTPS\_LOG\_DB\_DELETE\_EXTERNAL\_BIND\_ENTRY\_EC, [315](#)
  - RTPS\_LOG\_DB\_DELETE\_ROUTE\_ENTRY\_EC, [315](#)
  - RTPS\_LOG\_DB\_INSERT\_BIND\_ENTRY\_EC, [315](#)
  - RTPS\_LOG\_DB\_INSERT\_ROUTE\_ENTRY\_EC, [315](#)
  - RTPS\_LOG\_DB\_LOCK\_EC, [325](#)
  - RTPS\_LOG\_DB\_REMOVE\_BIND\_ENTRY\_EC, [315](#)
  - RTPS\_LOG\_DB\_REMOVE\_EXTERNAL\_BIND\_ENTRY\_EC, [316](#)
  - RTPS\_LOG\_DB\_SELECT\_ALL\_EC, [314](#)
  - RTPS\_LOG\_DB\_SELECT\_MATCH\_EC, [314](#)
  - RTPS\_LOG\_DB\_SELECT\_RANGE\_EC, [314](#)
  - RTPS\_LOG\_DB\_UNLOCK\_EC, [325](#)
  - RTPS\_LOG\_DELETE\_BUFFER\_POOL\_EC, [328](#)
  - RTPS\_LOG\_DELETE\_FILTERED\_ROUTE\_EC, [336](#)
  - RTPS\_LOG\_DELETE\_INDEX\_EC, [324](#)
  - RTPS\_LOG\_DELETE\_INDEXER\_EC, [329](#)
  - RTPS\_LOG\_DELETE\_RECORD\_EC, [336](#)
  - RTPS\_LOG\_DELETE\_TABLE\_EC, [324](#)
  - RTPS\_LOG\_DESERIALIZE\_APP\_ID\_EC, [323](#)
  - RTPS\_LOG\_DESERIALIZE\_HOST\_ID\_EC, [323](#)



RTPS\_LOG\_DESERIALIZE\_INSTANCE\_ID\_EC, [323](#)  
 RTPS\_LOG\_DESERIALIZE\_OBJECT\_ID\_EC, [323](#)  
 RTPS\_LOG\_DIRECT\_SEND\_EC, [326](#)  
 RTPS\_LOG\_EXCEEDED\_LIMIT\_PEERS\_EC, [318](#)  
 RTPS\_LOG\_EXCEEDED\_LIMIT\_TRANSPORTS\_EC, [318](#)  
 RTPS\_LOG\_FINALIZE\_INTERFACE\_EC, [327](#)  
 RTPS\_LOG\_FIND\_EXTERNAL\_INTERFACE\_EC, [318](#)  
 RTPS\_LOG\_FIRST\_SN\_GREATER\_LAST\_SN\_EC, [322](#)  
 RTPS\_LOG\_FORWARD\_UPSTREAM\_EC, [316](#)  
 RTPS\_LOG\_FULL\_SEND\_WINDOW\_EC, [317](#)  
 RTPS\_LOG\_FULLY\_ACKED\_READER\_EC, [320](#)  
 RTPS\_LOG\_FUNC\_UNSUPPORTED\_EC, [317](#)  
 RTPS\_LOG\_GET\_PEER\_BUFFER\_EC, [329](#)  
 RTPS\_LOG\_GET\_WINDOW\_BUFFER\_EC, [325](#)  
 RTPS\_LOG\_INCONSISTENT\_HDR\_EXT\_LENGTH\_ERROR\_EC, [334](#)  
 RTPS\_LOG\_INDEX\_ADD\_ENTRY\_EC, [314](#)  
 RTPS\_LOG\_INDEXER\_REMOVE\_ENTRY\_EC, [327](#)  
 RTPS\_LOG\_INITIALIZE\_INTERFACE\_EC, [313](#)  
 RTPS\_LOG\_INSERT\_WINDOW\_FULL\_EC, [326](#)  
 RTPS\_LOG\_INTERFACE\_MISMATCH\_EC, [317](#)  
 RTPS\_LOG\_INTERFACE\_NOT\_ENABLED\_EC, [320](#)  
 RTPS\_LOG\_INTF\_MODE\_UNDEF\_EC, [328](#)  
 RTPS\_LOG\_INVALID\_HDR\_EXT\_PID\_ERROR\_EC, [334](#)  
 RTPS\_LOG\_INVALID\_HDR\_EXT\_PID\_LIST\_EC, [334](#)  
 RTPS\_LOG\_INVALID\_PACKET\_EC, [321](#)  
 RTPS\_LOG\_LOCK\_EC, [332](#)  
 RTPS\_LOG\_LOOKUP\_EXT\_BIND\_ENTRY\_EC, [328](#)  
 RTPS\_LOG\_LOOKUP\_PEER\_EC, [327](#)  
 RTPS\_LOG\_MESSAGE\_EXCEED\_LENGTH\_ERROR\_EC, [334](#)  
 RTPS\_LOG\_NEGATIVE\_RESERVATION\_COUNT\_EC, [330](#)  
 RTPS\_LOG\_NETIO\_PACKET\_INSUFFICIENT\_BUFFER\_EC, [329](#)  
 RTPS\_LOG\_NETIO\_PACKET\_SET\_HEAD\_EC, [320](#)  
 RTPS\_LOG\_NETIO\_PACKET\_SET\_TAIL\_EC, [317](#)  
 RTPS\_LOG\_NONEXISTENT\_BIND\_EC, [319](#)  
 RTPS\_LOG\_NONEXISTENT\_EXTERNAL\_BIND\_EC, [319](#)  
 RTPS\_LOG\_NONEXISTENT\_ROUTE\_EC, [318](#)  
 RTPS\_LOG\_NOT\_ENABLED\_EC, [317](#)  
 RTPS\_LOG\_PACKET\_INITIALIZE\_EC, [326](#)  
 RTPS\_LOG\_PROCESS\_ACKNACK\_EC, [321](#)  
 RTPS\_LOG\_PROCESS\_DATA\_BATCH\_EC, [330](#)  
 RTPS\_LOG\_PROCESS\_DATA\_EC, [321](#)  
 RTPS\_LOG\_PROCESS\_GAP\_EC, [321](#)  
 RTPS\_LOG\_PROCESS\_HEARTBEAT\_BATCH\_EC, [330](#)  
 RTPS\_LOG\_PROCESS\_HEARTBEAT\_EC, [321](#)  
 RTPS\_LOG\_READER\_SEND\_ACKNACK\_EC, [327](#)  
 RTPS\_LOG\_READER\_UNSUPPORTED\_EC, [317](#)  
 RTPS\_LOG\_RECEIVE\_EC, [316](#)  
 RTPS\_LOG\_REQUEST\_EC, [319](#)  
 RTPS\_LOG\_RESTORE\_TRUST\_LOAN\_BUFFER\_EC, [330](#)  
 RTPS\_LOG\_RETURN\_LOAN\_EC, [319](#)  
 RTPS\_LOG\_ROUTE\_PACKET\_EC, [316](#)  
 RTPS\_LOG\_SCHEDULE\_PACKET\_EC, [332](#)  
 RTPS\_LOG\_SEND\_EC, [316](#)  
 RTPS\_LOG\_SEND\_HEARTBEAT\_EC, [320](#)  
 RTPS\_LOG\_SEND\_LIVELINESS\_EC, [335](#)  
 RTPS\_LOG\_SEQ\_FINALIZE\_EC, [332](#)  
 RTPS\_LOG\_SEQ\_INIT\_EC, [328](#)  
 RTPS\_LOG\_SEQ\_LEN\_EC, [328](#)  
 RTPS\_LOG\_SEQ\_MAX\_EC, [328](#)  
 RTPS\_LOG\_SEQ\_NUM\_OUT\_OF\_RANGE\_EC, [322](#)  
 RTPS\_LOG\_SERIALIZE\_APP\_ID\_EC, [323](#)  
 RTPS\_LOG\_SERIALIZE\_HOST\_ID\_EC, [322](#)  
 RTPS\_LOG\_SERIALIZE\_INSTANCE\_ID\_EC, [323](#)  
 RTPS\_LOG\_SERIALIZE\_OBJECT\_ID\_EC, [323](#)  
 RTPS\_LOG\_SET\_GAP\_EC, [325](#)  
 RTPS\_LOG\_SHIFT\_BITMAP\_EC, [320](#)  
 RTPS\_LOG\_SHIFT\_SN\_OUT\_OF\_RANGE\_EC, [322](#)  
 RTPS\_LOG\_STALE\_ACK\_EPOCH\_EC, [319](#)  
 RTPS\_LOG\_STALE\_HB\_EPOCH\_EC, [319](#)  
 RTPS\_LOG\_STATUS\_CHANGE\_EC, [325](#)  
 RTPS\_LOG\_TIMER\_DELETE\_TIMEOUT\_EC, [326](#)  
 RTPS\_LOG\_TRANSFORM\_RTPS\_EC, [330](#)  
 RTPS\_LOG\_TRUST\_BIND\_FAILED\_EC, [331](#)  
 RTPS\_LOG\_TRUST\_INCONSISTENT\_INTERCEPTOR\_HANDLE\_EC, [331](#)  
 RTPS\_LOG\_TRUST\_INTERCEPTOR\_HANDLE\_LIST\_FULL\_EC, [331](#)  
 RTPS\_LOG\_TRUST\_INVALID\_SUBMSG\_EC, [332](#)  
 RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_CREATE\_FAILED\_EC, [331](#)  
 RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_INSERT\_FAILED\_EC, [331](#)  
 RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_LOOKUP\_FAILED\_EC, [330](#)  
 RTPS\_LOG\_TRUST\_SUBMSG\_CONVERSION\_FAILED\_EC, [331](#)  
 RTPS\_LOG\_TRUST\_UNEXPECTED\_READER\_STATE\_EC, [332](#)  
 RTPS\_LOG\_UNKNOWN\_SUBMESSAGE\_EC, [321](#)  
 RTPS\_LOG\_UNLOCK\_EC, [332](#)  
 RTPS\_LOG\_UNSUPPORTED\_INFO\_REPLY\_EC, [324](#)

- RTPS\_LOG\_UNSUPPORTED\_INFO\_REPLY\_IP4\_EC, RTPS\_ChecksumProperty, [849](#)
- [324](#)
- RTPS\_LOG\_UNSUPPORTED\_INFO\_SRC\_EC, [324](#)
- RTPS\_LOG\_UNSUPPORTED\_INTERFACE\_EC, [321](#)
- RTPS\_LOG\_UNSUPPORTED\_MANDATORY\_HDR\_EXT\_ID\_EC, [334](#)
- RTPS\_LOG\_UPDATE\_RELIABLE\_PEERS\_FILTER\_EC, [336](#)
- RTPS\_LOG\_WINDOW\_ADVANCE\_EC, [327](#)
- RTPS\_LOG\_WINDOW\_ADVANCE\_UNACKED\_EC, [327](#)
- RTPS\_LOG\_WINDOW\_INSERT\_EC, [326](#)
- RTPS\_LOG\_WINDOW\_POOL\_ALLOC\_EC, [325](#)
- RTPS\_LOG\_WRITER\_REQUEST\_SENT\_HISTORY\_EC, [326](#)
- RTPS Transport API, [154](#)
  - allow\_builtin\_override, [158](#)
  - builtin\_checksum128\_class, [158](#)
  - builtin\_checksum32\_class, [157](#)
  - builtin\_checksum64\_class, [157](#)
  - checksum, [158](#)
  - checksum128, [157](#)
  - checksum32, [157](#)
  - checksum64, [157](#)
  - checksum\_calculate, [157](#)
  - checksum\_tx\_mode, [158](#)
  - class\_id, [157](#)
  - context, [157](#)
  - RTPS\_Checksum\_T, [155](#)
  - RTPS\_CHECKSUM\_TXMODE\_OMG, [156](#)
  - RTPS\_CHECKSUM\_TXMODE\_RTICRC32, [156](#)
  - RTPS\_ChecksumCalculate\_T, [155](#)
  - RTPS\_ChecksumClass\_T, [156](#)
  - RTPS\_ChecksumClassId\_T, [155](#)
  - RTPS\_ChecksumTxMode\_T, [156](#)
  - RTPS\_INTERFACE\_FACTORY\_DEFAULT, [158](#)
  - RTPS\_InterfaceFactoryProperty\_T, [156](#)
- rtps\_app\_id
  - DDS\_WireProtocolQosPolicy, [641](#)
- RTPS\_Checksum, [848](#)
- RTPS\_Checksum\_T
  - RTPS Transport API, [155](#)
- RTPS\_CHECKSUM\_TXMODE\_OMG
  - RTPS Transport API, [156](#)
- RTPS\_CHECKSUM\_TXMODE\_RTICRC32
  - RTPS Transport API, [156](#)
- RTPS\_ChecksumCalculate\_T
  - RTPS Transport API, [155](#)
- RTPS\_ChecksumClass, [848](#)
- RTPS\_ChecksumClass\_T
  - RTPS Transport API, [156](#)
- RTPS\_ChecksumClassId\_T
  - RTPS Transport API, [155](#)
- RTPS\_ChecksumProperty, [849](#)
- RTPS\_ChecksumTxMode\_T
  - RTPS Transport API, [156](#)
- rtps\_config.h, [943](#)
- rtps\_dll.h, [943](#)
- rtps\_instance\_id
  - DDS\_WireProtocolQosPolicy, [640](#)
  - DDS\_WireProtocolQosPolicy, [641](#)
- RTPS\_INTERFACE\_FACTORY\_DEFAULT
  - RTPS Transport API, [158](#)
- RTPS\_InterfaceFactoryProperty, [849](#)
- RTPS\_InterfaceFactoryProperty\_T
  - RTPS Transport API, [156](#)
- rtps\_log.h, [943](#)
- RTPS\_LOG\_DB\_REMOVE\_ROUTE\_ENTRY\_EC, [951](#)
- RTPS\_LOG\_ACK\_EC
  - RTPS, [319](#)
- RTPS\_LOG\_ADD\_FILTERED\_ROUTE\_EC
  - RTPS, [336](#)
- RTPS\_LOG\_ALLOCATE\_EC
  - RTPS, [313](#)
- RTPS\_LOG\_APPLY\_CONTENT\_FILTER\_EC
  - RTPS, [336](#)
- RTPS\_LOG\_ASSERT\_PEER\_EC
  - RTPS, [318](#)
- RTPS\_LOG\_ASSERT\_TRANSPORT\_EC
  - RTPS, [318](#)
- RTPS\_LOG\_BAD\_PARAMETER\_EC
  - RTPS, [317](#)
- RTPS\_LOG\_BASE
  - osapi\_log.h, [931](#)
- RTPS\_LOG\_BITCOUNT\_OUT\_OF\_BOUNDS\_EC
  - RTPS, [322](#)
- RTPS\_LOG\_BITMAP\_SET\_BIT\_EC
  - RTPS, [324](#)
- RTPS\_LOG\_BITMAP\_SHIFT\_EC
  - RTPS, [327](#)
- RTPS\_LOG\_BUFFERPOOL\_DELETE\_EC
  - RTPS, [326](#)
- RTPS\_LOG\_CHECKSUM\_CHECKSUM\_CALCULATION\_FAILED\_EC
  - RTPS, [333](#)
- RTPS\_LOG\_CHECKSUM\_CHECKSUM\_DESERIALIZE\_ERROR\_EC
  - RTPS, [335](#)
- RTPS\_LOG\_CHECKSUM\_CHECKSUM\_ERROR\_EC
  - RTPS, [335](#)
- RTPS\_LOG\_CHECKSUM\_ILLEGAL\_OVERRIDE\_CHECKSUM128\_EC
  - RTPS, [333](#)
- RTPS\_LOG\_CHECKSUM\_ILLEGAL\_OVERRIDE\_CHECKSUM32\_EC
  - RTPS, [333](#)
- RTPS\_LOG\_CHECKSUM\_ILLEGAL\_OVERRIDE\_CHECKSUM64\_EC
  - RTPS, [333](#)
- RTPS\_LOG\_CHECKSUM\_INCONSISTENT\_LENGTH\_EC

RTPS, [335](#)  
 RTPS\_LOG\_CHECKSUM\_INVALID\_CRC32\_FLAGS\_EC RTPS, [334](#)  
 RTPS\_LOG\_CHECKSUM\_INVALID\_OVERRIDE\_BUILTIN128\_EC RTPS, [333](#)  
 RTPS\_LOG\_CHECKSUM\_INVALID\_OVERRIDE\_BUILTIN32\_EC RTPS, [333](#)  
 RTPS\_LOG\_CHECKSUM\_INVALID\_OVERRIDE\_BUILTIN64\_EC RTPS, [333](#)  
 RTPS\_LOG\_CHECKSUM\_LENGTH\_DESERIALIZE\_ERROR\_EC RTPS, [335](#)  
 RTPS\_LOG\_CHECKSUM\_MISSING\_CHECKSUM\_LENGTH\_EC RTPS, [334](#)  
 RTPS\_LOG\_CHECKSUM\_TOO\_MANY\_SG\_BUFFERS\_EC RTPS, [335](#)  
 RTPS\_LOG\_CHECKSUM\_UNSUPPORTED\_EC RTPS, [335](#)  
 RTPS\_LOG\_CREATE\_BIND\_TABLE\_EC RTPS, [325](#)  
 RTPS\_LOG\_CREATE\_BUFFER\_POOL\_EC RTPS, [328](#)  
 RTPS\_LOG\_CREATE\_EVENT\_TIMER\_EC RTPS, [322](#)  
 RTPS\_LOG\_CREATE\_HB\_EVENT\_EC RTPS, [322](#)  
 RTPS\_LOG\_CREATE\_PEERS\_INDEX\_EC RTPS, [329](#)  
 RTPS\_LOG\_CREATE\_ROUTE\_TABLE\_EC RTPS, [313](#)  
 RTPS\_LOG\_DATA\_ALREADY\_RECEIVED\_EC RTPS, [320](#)  
 RTPS\_LOG\_DATA\_OUT\_OF\_RANGE\_EC RTPS, [320](#)  
 RTPS\_LOG\_DB\_CREATE\_BIND\_ENTRY\_EC RTPS, [315](#)  
 RTPS\_LOG\_DB\_CREATE\_BIND\_PEER\_INDEX\_EC RTPS, [314](#)  
 RTPS\_LOG\_DB\_CREATE\_DIRECT\_INDEX\_EC RTPS, [329](#)  
 RTPS\_LOG\_DB\_CREATE\_EXT\_BIND\_ENTRY\_EC RTPS, [324](#)  
 RTPS\_LOG\_DB\_CREATE\_GROUP\_INDEX\_EC RTPS, [329](#)  
 RTPS\_LOG\_DB\_CREATE\_ROUTE\_ENTRY\_EC RTPS, [314](#)  
 RTPS\_LOG\_DB\_CREATE\_ROUTE\_PEER\_INDEX\_EC RTPS, [313](#)  
 RTPS\_LOG\_DB\_CREATE\_ROUTE\_XPORT\_INDEX\_EC RTPS, [313](#)  
 RTPS\_LOG\_DB\_CREATE\_SELECTED\_GROUP\_INDEX\_EC RTPS, [332](#)  
 RTPS\_LOG\_DB\_DELETE\_EXTERNAL\_BIND\_ENTRY\_EC RTPS, [316](#)  
 RTPS\_LOG\_DB\_DELETE\_ROUTE\_ENTRY\_EC RTPS, [315](#)  
 RTPS\_LOG\_DB\_INSERT\_BIND\_ENTRY\_EC RTPS, [315](#)  
 RTPS\_LOG\_DB\_INSERT\_ROUTE\_ENTRY\_EC RTPS, [315](#)  
 RTPS\_LOG\_DB\_LOCK\_EC RTPS, [325](#)  
 RTPS\_LOG\_DB\_REMOVE\_BIND\_ENTRY\_EC RTPS, [315](#)  
 RTPS\_LOG\_DB\_REMOVE\_EXTERNAL\_BIND\_ENTRY\_EC RTPS, [316](#)  
 RTPS\_LOG\_DB\_REMOVE\_ROUTE\_ENTRY\_EC RTPS, [316](#)  
 rtps\_log.h, [951](#)  
 RTPS\_LOG\_DB\_SELECT\_ALL\_EC RTPS, [314](#)  
 RTPS\_LOG\_DB\_SELECT\_MATCH\_EC RTPS, [314](#)  
 RTPS\_LOG\_DB\_SELECT\_RANGE\_EC RTPS, [314](#)  
 RTPS\_LOG\_DB\_UNLOCK\_EC RTPS, [325](#)  
 RTPS\_LOG\_DELETE\_BUFFER\_POOL\_EC RTPS, [328](#)  
 RTPS\_LOG\_DELETE\_FILTERED\_ROUTE\_EC RTPS, [336](#)  
 RTPS\_LOG\_DELETE\_INDEX\_EC RTPS, [324](#)  
 RTPS\_LOG\_DELETE\_INDEXER\_EC RTPS, [329](#)  
 RTPS\_LOG\_DELETE\_RECORD\_EC RTPS, [336](#)  
 RTPS\_LOG\_DELETE\_TABLE\_EC RTPS, [324](#)  
 RTPS\_LOG\_DESERIALIZE\_APP\_ID\_EC RTPS, [323](#)  
 RTPS\_LOG\_DESERIALIZE\_HOST\_ID\_EC RTPS, [323](#)  
 RTPS\_LOG\_DESERIALIZE\_INSTANCE\_ID\_EC RTPS, [323](#)  
 RTPS\_LOG\_DESERIALIZE\_OBJECT\_ID\_EC RTPS, [323](#)  
 RTPS\_LOG\_DIRECT\_SEND\_EC RTPS, [326](#)  
 RTPS\_LOG\_EXCEEDED\_LIMIT\_PEERS\_EC RTPS, [318](#)  
 RTPS\_LOG\_EXCEEDED\_LIMIT\_TRANSPORTS\_EC RTPS, [318](#)  
 RTPS\_LOG\_FINALIZE\_INTERFACE\_EC RTPS, [327](#)  
 RTPS\_LOG\_FIND\_EXTERNAL\_INTERFACE\_EC RTPS, [318](#)  
 RTPS\_LOG\_FIRST\_SN\_GREATER\_LAST\_SN\_EC RTPS, [322](#)  
 RTPS\_LOG\_FORWARD\_UPSTREAM\_EC

RTPS, [316](#)  
 RTPS\_LOG\_FULL\_SEND\_WINDOW\_EC  
     RTPS, [317](#)  
 RTPS\_LOG\_FULLY\_ACKED\_READER\_EC  
     RTPS, [320](#)  
 RTPS\_LOG\_FUNC\_UNSUPPORTED\_EC  
     RTPS, [317](#)  
 RTPS\_LOG\_GET\_PEER\_BUFFER\_EC  
     RTPS, [329](#)  
 RTPS\_LOG\_GET\_WINDOW\_BUFFER\_EC  
     RTPS, [325](#)  
 RTPS\_LOG\_INCONSISTENT\_HDR\_EXT\_LENGTH\_ERROR\_EC  
     RTPS, [334](#)  
 RTPS\_LOG\_INDEX\_ADD\_ENTRY\_EC  
     RTPS, [314](#)  
 RTPS\_LOG\_INDEXER\_REMOVE\_ENTRY\_EC  
     RTPS, [327](#)  
 RTPS\_LOG\_INITIALIZE\_INTERFACE\_EC  
     RTPS, [313](#)  
 RTPS\_LOG\_INSERT\_WINDOW\_FULL\_EC  
     RTPS, [326](#)  
 RTPS\_LOG\_INTERFACE\_MISMATCH\_EC  
     RTPS, [317](#)  
 RTPS\_LOG\_INTERFACE\_NOT\_ENABLED\_EC  
     RTPS, [320](#)  
 RTPS\_LOG\_INTF\_MODE\_UNDEF\_EC  
     RTPS, [328](#)  
 RTPS\_LOG\_INVALID\_HDR\_EXT\_PID\_ERROR\_EC  
     RTPS, [334](#)  
 RTPS\_LOG\_INVALID\_HDR\_EXT\_PID\_LIST\_EC  
     RTPS, [334](#)  
 RTPS\_LOG\_INVALID\_PACKET\_EC  
     RTPS, [321](#)  
 RTPS\_LOG\_LOCK\_EC  
     RTPS, [332](#)  
 RTPS\_LOG\_LOOKUP\_EXT\_BIND\_ENTRY\_EC  
     RTPS, [328](#)  
 RTPS\_LOG\_LOOKUP\_PEER\_EC  
     RTPS, [327](#)  
 RTPS\_LOG\_MESSAGE\_EXCEED\_LENGTH\_ERROR\_EC  
     RTPS, [334](#)  
 RTPS\_LOG\_NEGATIVE\_RESERVATION\_COUNT\_EC  
     RTPS, [330](#)  
 RTPS\_LOG\_NETIO\_PACKET\_INSUFFICIENT\_BUFFER\_EC  
     RTPS, [329](#)  
 RTPS\_LOG\_NETIO\_PACKET\_SET\_HEAD\_EC  
     RTPS, [320](#)  
 RTPS\_LOG\_NETIO\_PACKET\_SET\_TAIL\_EC  
     RTPS, [317](#)  
 RTPS\_LOG\_NONEXISTENT\_BIND\_EC  
     RTPS, [319](#)  
 RTPS\_LOG\_NONEXISTENT\_EXTERNAL\_BIND\_EC  
     RTPS, [319](#)  
 RTPS\_LOG\_NONEXISTENT\_ROUTE\_EC  
     RTPS, [318](#)  
 RTPS\_LOG\_NOT\_ENABLED\_EC  
     RTPS, [317](#)  
 RTPS\_LOG\_PACKET\_INITIALIZE\_EC  
     RTPS, [326](#)  
 RTPS\_LOG\_PROCESS\_ACKNACK\_EC  
     RTPS, [321](#)  
 RTPS\_LOG\_PROCESS\_DATA\_BATCH\_EC  
     RTPS, [330](#)  
 RTPS\_LOG\_PROCESS\_DATA\_EC  
     RTPS, [321](#)  
 RTPS\_LOG\_PROCESS\_GAP\_EC  
     RTPS, [321](#)  
 RTPS\_LOG\_PROCESS\_HEARTBEAT\_BATCH\_EC  
     RTPS, [330](#)  
 RTPS\_LOG\_PROCESS\_HEARTBEAT\_EC  
     RTPS, [321](#)  
 RTPS\_LOG\_READER\_SEND\_ACKNACK\_EC  
     RTPS, [327](#)  
 RTPS\_LOG\_READER\_UNSUPPORTED\_EC  
     RTPS, [317](#)  
 RTPS\_LOG\_RECEIVE\_EC  
     RTPS, [316](#)  
 RTPS\_LOG\_REQUEST\_EC  
     RTPS, [319](#)  
 RTPS\_LOG\_RESTORE\_TRUST\_LOAN\_BUFFER\_EC  
     RTPS, [330](#)  
 RTPS\_LOG\_RETURN\_LOAN\_EC  
     RTPS, [319](#)  
 RTPS\_LOG\_ROUTE\_PACKET\_EC  
     RTPS, [316](#)  
 RTPS\_LOG\_SCHEDULE\_PACKET\_EC  
     RTPS, [332](#)  
 RTPS\_LOG\_SEND\_EC  
     RTPS, [316](#)  
 RTPS\_LOG\_SEND\_HEARTBEAT\_EC  
     RTPS, [320](#)  
 RTPS\_LOG\_SEND\_LIVELINESS\_EC  
     RTPS, [335](#)  
 RTPS\_LOG\_SEQ\_FINALIZE\_EC  
     RTPS, [332](#)  
 RTPS\_LOG\_SEQ\_INIT\_EC  
     RTPS, [328](#)  
 RTPS\_LOG\_SEQ\_LEN\_EC  
     RTPS, [328](#)  
 RTPS\_LOG\_SEQ\_MAX\_EC  
     RTPS, [328](#)  
 RTPS\_LOG\_SEQ\_NUM\_OUT\_OF\_RANGE\_EC  
     RTPS, [322](#)  
 RTPS\_LOG\_SERIALIZE\_APP\_ID\_EC  
     RTPS, [323](#)  
 RTPS\_LOG\_SERIALIZE\_HOST\_ID\_EC  
     RTPS, [322](#)  
 RTPS\_LOG\_SERIALIZE\_INSTANCE\_ID\_EC

- RTPS, [323](#)
- RTPS\_LOG\_SERIALIZE\_OBJECT\_ID\_EC
  - RTPS, [323](#)
- RTPS\_LOG\_SET\_GAP\_EC
  - RTPS, [325](#)
- RTPS\_LOG\_SHIFT\_BITMAP\_EC
  - RTPS, [320](#)
- RTPS\_LOG\_SHIFT\_SN\_OUT\_OF\_RANGE\_EC
  - RTPS, [322](#)
- RTPS\_LOG\_STALE\_ACK\_EPOCH\_EC
  - RTPS, [319](#)
- RTPS\_LOG\_STALE\_HB\_EPOCH\_EC
  - RTPS, [319](#)
- RTPS\_LOG\_STATUS\_CHANGE\_EC
  - RTPS, [325](#)
- RTPS\_LOG\_TIMER\_DELETE\_TIMEOUT\_EC
  - RTPS, [326](#)
- RTPS\_LOG\_TRANSFORM\_RTPS\_EC
  - RTPS, [330](#)
- RTPS\_LOG\_TRUST\_BIND\_FAILED\_EC
  - RTPS, [331](#)
- RTPS\_LOG\_TRUST\_INCONSISTENT\_INTERCEPTOR\_HANDLE\_EC
  - RTPS, [331](#)
- RTPS\_LOG\_TRUST\_INTERCEPTOR\_HANDLE\_LIST\_FULL\_EC
  - RTPS, [331](#)
- RTPS\_LOG\_TRUST\_INVALID\_SUBMSG\_EC
  - RTPS, [332](#)
- RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_CHANGE\_EC
  - RTPS, [331](#)
- RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_INSERT\_FAILED\_EC
  - RTPS, [331](#)
- RTPS\_LOG\_TRUST\_REMOTE\_PARTICIPANT\_HANDLE\_LOOK\_UP\_FAILED\_EC
  - RTPS, [330](#)
- RTPS\_LOG\_TRUST\_SUBMSG\_CONVERSION\_FAILED\_EC
  - RTPS, [331](#)
- RTPS\_LOG\_TRUST\_UNEXPECTED\_READER\_STATE\_EC
  - RTPS, [332](#)
- RTPS\_LOG\_UNKNOWN\_SUBMESSAGE\_EC
  - RTPS, [321](#)
- RTPS\_LOG\_UNLOCK\_EC
  - RTPS, [332](#)
- RTPS\_LOG\_UNSUPPORTED\_INFO\_REPLY\_EC
  - RTPS, [324](#)
- RTPS\_LOG\_UNSUPPORTED\_INFO\_REPLY\_IP4\_EC
  - RTPS, [324](#)
- RTPS\_LOG\_UNSUPPORTED\_INFO\_SRC\_EC
  - RTPS, [324](#)
- RTPS\_LOG\_UNSUPPORTED\_INTERFACE\_EC
  - RTPS, [321](#)
- RTPS\_LOG\_UNSUPPORTED\_MANDTORY\_HDR\_EXT\_PID\_EC
  - RTPS, [334](#)
- RTPS\_LOG\_UPDATE\_RELIABLE\_PEERS\_FILTER\_EC
  - RTPS, [336](#)
- RTPS\_LOG\_WINDOW\_ADVANCE\_EC
  - RTPS, [327](#)
- RTPS\_LOG\_WINDOW\_ADVANCE\_UNACKED\_EC
  - RTPS, [327](#)
- RTPS\_LOG\_WINDOW\_INSERT\_EC
  - RTPS, [326](#)
- RTPS\_LOG\_WINDOW\_POOL\_ALLOC\_EC
  - RTPS, [325](#)
- RTPS\_LOG\_WRITER\_REQUEST\_SENT\_HISTORY\_EC
  - RTPS, [326](#)
- rtps\_object\_id
  - DDS\_DataReaderProtocolQosPolicy, [504](#)
  - DDS\_DataWriterProtocolQosPolicy, [513](#)
- rtps\_protocol\_version
  - DDS\_ParticipantBuiltinTopicData, [575](#)
- rtps\_reliable\_reader
  - DDS\_DataReaderProtocolQosPolicy, [504](#)
- rtps\_reliable\_writer
  - DDS\_DataWriterProtocolQosPolicy, [513](#)
- rtps\_rtps\_impl.h, [951](#)
- rtps\_vendor\_id
  - DDS\_ParticipantBuiltinTopicData, [575](#)
- rtcp\_level
  - DDS\_Unknown\_ports
  - DDS\_WireProtocolQosPolicy, [641](#)
- Registry, [850](#)
- register\_component, [850](#)
- unregister, [851](#)
- Sample States, [73](#)
  - DDS\_ANY\_SAMPLE\_STATE, [74](#)
  - DDS\_NOT\_READ\_SAMPLE\_STATE, [74](#)
  - DDS\_READ\_SAMPLE\_STATE, [74](#)
  - DDS\_SampleStateKind, [74](#)
  - DDS\_SampleStateMask, [74](#)
- sample\_info
  - DDS\_SampleLostStatus, [620](#)
- sample\_state
  - DDS\_SampleInfo, [617](#)
- scheduling\_policy
  - DDS\_FlowControllerProperty\_t, [550](#)
- SDM\_LOG\_BASE
  - osapi\_log.h, [930](#)
- sec
  - DDS\_Duration\_t, [541](#)
  - DDS\_Time\_t, [633](#)
  - OSAPI\_SystemTime, [845](#)
- SEC\_CORE\_LOG\_BASE
  - osapi\_log.h, [932](#)
- send
  - NETIO\_DGRAM\_InterfaceI, [829](#)
  - ZCOPY\_NotifUserInterfaceI, [867](#)
- Sequence Number Support, [86](#)
  - DDS\_SequenceNumber\_compare, [86](#)
- Sequence Support, [133](#)
- sequence\_number

- DDS\_ReliableSampleUnacknowledgedStatus, 603
- DDS\_SampleIdentity\_t, 615
- serialize\_data\_to\_cdr\_buffer
  - FooTypeSupport, 823
- serialize\_data\_to\_cdr\_buffer\_ex
  - FooTypeSupport, 822
- serialize\_on\_write
  - DDS\_DataWriterProtocolQosPolicy, 514
- service\_name
  - DDS\_PskServiceFactoryProperty, 589
- set\_default\_datareader\_qos
  - DDSSubscriber, 751
- set\_default\_datawriter\_qos
  - DDSPublisher, 741
- set\_default\_flowcontroller\_property
  - DDSDomainParticipant, 707
- set\_default\_participant\_qos
  - DDSDomainParticipantFactory, 715
- set\_default\_publisher\_qos
  - DDSDomainParticipant, 695
- set\_default\_subscriber\_qos
  - DDSDomainParticipant, 697
- set\_default\_topic\_qos
  - DDSDomainParticipant, 698
- set\_enabled\_statuses
  - DDSStatusCondition, 747
- set\_listener
  - DDSDataReader, 649
  - DDSDataWriter, 671
  - DDSDomainParticipant, 704
  - DDSPublisher, 744
  - DDSSubscriber, 756
  - DDSTopic, 762
- set\_name
  - ENTITY\_NAME, 127
- set\_property
  - DDSFlowController, 729
- set\_qos
  - DDSDataReader, 648
  - DDSDataWriter, 671
  - DDSDomainParticipant, 694
  - DDSDomainParticipantFactory, 717
  - DDSPublisher, 743
  - DDSSubscriber, 755
  - DDSTopic, 761
- set\_trigger\_value
  - DDSGuardCondition, 732
- Shared Memory Mutex API, 160
  - NETIO\_SharedMemoryMutex\_attach, 161
  - NETIO\_SharedMemoryMutex\_create, 161
  - NETIO\_SharedMemoryMutex\_create\_or\_attach, 162
  - NETIO\_SharedMemoryMutex\_delete, 165
  - NETIO\_SharedMemoryMutex\_detach, 165
  - NETIO\_SharedMemoryMutex\_lock, 164
  - NETIO\_SharedMemoryMutex\_unlock, 163
- Shared Memory Segment API, 166
  - NETIO\_SharedMemorySegment\_attach, 169
  - NETIO\_SharedMemorySegment\_create, 167
  - NETIO\_SharedMemorySegment\_create\_or\_attach, 167
  - NETIO\_SharedMemorySegment\_delete, 168
  - NETIO\_SharedMemorySegment\_detach, 169
  - NETIO\_SharedMemorySegment\_get\_address, 170
  - NETIO\_SharedMemorySegment\_get\_max\_size, 171
  - NETIO\_SharedMemorySegment\_get\_size, 170
- Shared Memory Signaling Semaphore, 172
  - NETIO\_SharedMemorySignalingSemaphore\_attach, 173
  - NETIO\_SharedMemorySignalingSemaphore\_create, 172
  - NETIO\_SharedMemorySignalingSemaphore\_create\_or\_attach, 174
  - NETIO\_SharedMemorySignalingSemaphore\_delete, 177
  - NETIO\_SharedMemorySignalingSemaphore\_detach, 176
  - NETIO\_SharedMemorySignalingSemaphore\_signal, 175
  - NETIO\_SharedMemorySignalingSemaphore\_wait, 176
- Shared Memory Transport API, 158
- SHMEM\_LOG\_BASE
  - osapi\_log.h, 930
- shmem\_ref\_settings
  - DDS\_DataWriterTransferModeQosPolicy, 522
- shmem\_ref\_transfer\_mode\_attached\_segment\_allocation
  - DDS\_DataReaderResourceLimitsQosPolicy, 511
- shmem\_ref\_transfer\_mode\_max\_segments
  - DDS\_DomainParticipantResourceLimitsQosPolicy, 536
- SHMEMInterfaceFactory, 851
  - get\_interface, 852
- size
  - OSAPI\_SharedMemorySegmentHeader, 840
- source\_rules
  - UDP\_InterfaceFactoryProperty, 856
- source\_timestamp
  - DDS\_SampleInfo, 618
  - DDS\_WriteParams\_t, 644
- source\_timestamp\_tolerance
  - DDS\_DestinationOrderQosPolicy, 524
- sql\_predicate\_max\_count\_per\_expression
  - DDS\_FilterResourceLimits, 548
- stack\_size
  - OSAPI\_ThreadProperty, 845
- start\_timer
  - OSAPI\_SystemI, 841
- Status Kinds, 88



- DDS\_DATA\_AVAILABLE\_STATUS, [94](#)
- DDS\_DATA\_ON\_READERS\_STATUS, [94](#)
- DDS\_DATA\_WRITER\_SAMPLE\_REMOVED\_STATUS, [97](#)
- DDS\_INCONSISTENT\_TOPIC\_STATUS, [92](#)
- DDS\_INSTANCE\_REPLACED\_STATUS, [96](#)
- DDS\_LIVELINESS\_CHANGED\_STATUS, [95](#)
- DDS\_LIVELINESS\_LOST\_STATUS, [95](#)
- DDS\_OFFERED\_DEADLINE\_MISSED\_STATUS, [92](#)
- DDS\_OFFERED\_INCOMPATIBLE\_QOS\_STATUS, [93](#)
- DDS\_PUBLICATION\_MATCHED\_STATUS, [96](#)
- DDS\_RELIABLE\_READER\_ACTIVITY\_CHANGED\_STATUS, [97](#)
- DDS\_REQUESTED\_DEADLINE\_MISSED\_STATUS, [93](#)
- DDS\_REQUESTED\_INCOMPATIBLE\_QOS\_STATUS, [93](#)
- DDS\_SAMPLE\_LOST\_STATUS, [94](#)
- DDS\_SAMPLE\_REJECTED\_STATUS, [94](#)
- DDS\_STATUS\_MASK\_ALL, [91](#)
- DDS\_STATUS\_MASK\_NONE, [91](#)
- DDS\_StatusKind, [92](#)
- DDS\_StatusMask, [91](#)
- DDS\_SUBSCRIPTION\_MATCHED\_STATUS, [96](#)
- String Support, [135](#)
  - DDS\_String\_alloc, [137](#)
  - DDS\_String\_cmp, [138](#)
  - DDS\_String\_dup, [137](#)
  - DDS\_String\_free, [137](#)
  - DDS\_String\_length, [138](#)
  - DDS\_String\_ncmp, [138](#)
- Subscriber, [68](#)
  - DDS\_DATAREADER\_QOS\_DEFAULT, [69](#)
  - DDS\_DATAREADER\_QOS\_USE\_TOPIC\_QOS, [69](#)
- subscriber\_group\_data\_max\_count
  - DDS\_DomainParticipantResourceLimitsQosPolicy, [538](#)
- subscriber\_group\_data\_max\_length
  - DDS\_DomainParticipantResourceLimitsQosPolicy, [538](#)
- subscriber\_name
  - DDS\_SubscriberQos, [624](#)
- Subscription Built-in Topic, [44](#)
  - DDS\_SUBSCRIPTION\_TOPIC\_NAME, [44](#)
  - DDS\_SubscriptionBuiltinTopicData, [44](#), [45](#)
  - DDS\_SubscriptionBuiltinTopicData\_finalize, [45](#)
  - DDS\_SubscriptionBuiltinTopicData\_is\_equal, [45](#)
- Subscription Module, [67](#)
- subscription\_name
  - DDS\_DataReaderQos, [508](#)
- SYSTEM\_RESOURCE\_LIMITS, [120](#)
- DATA\_REPRESENTATION, [113](#), [114](#)
- take
  - FooDataReader, [785](#)
- take\_instance
  - FooDataReader, [790](#)
- take\_instance\_untyped
  - DDSDDataReader, [655](#)
- take\_next\_sample
  - FooDataReader, [783](#)
- take\_next\_sample\_untyped
  - DDSDDataReader, [656](#)
- take\_untyped
  - DDSDDataReader, [651](#)
- thread
  - OSAPI\_TimerProperty, [846](#)
- thread\_prop
  - ZCOPY\_NotifMechanismProperty, [862](#)
- Time Support, [81](#)
  - DDS\_Duration\_equal, [82](#)
  - DDS\_DURATION\_INFINITE, [84](#)
  - DDS\_DURATION\_INFINITE\_NSEC, [84](#)
  - DDS\_DURATION\_INFINITE\_SEC, [83](#)
  - DDS\_Duration\_is\_infinite, [82](#)
  - DDS\_Duration\_is\_zero, [83](#)
  - DDS\_DURATION\_ZERO, [84](#)
  - DDS\_DURATION\_ZERO\_NSEC, [84](#)
  - DDS\_DURATION\_ZERO\_SEC, [84](#)
  - DDS\_Time\_greater\_than, [82](#)
  - DDS\_TIME\_INVALID, [83](#)
  - DDS\_TIME\_INVALID\_NSEC, [83](#)
  - DDS\_TIME\_INVALID\_SEC, [83](#)
  - DDS\_Time\_is\_zero, [82](#)
  - DDS\_TIME\_ZERO, [82](#)
- timer\_property
  - OSAPI\_SystemProperty, [843](#)
- to\_array
  - FooSeq, [810](#)
- token\_bucket
  - DDS\_FlowControllerProperty\_t, [550](#)
- tokens\_added\_per\_period
  - DDS\_FlowControllerTokenBucketProperty\_t, [551](#)
- tokens\_leaked\_per\_period
  - DDS\_FlowControllerTokenBucketProperty\_t, [552](#)
- Topic, [46](#), [52](#)
- TOPIC\_DATA, [131](#)
- topic\_data
  - DDS\_PublicationBuiltinTopicData, [593](#)
  - DDS\_SubscriptionBuiltinTopicData, [629](#)
  - DDS\_TopicQos, [634](#)
- topic\_data\_max\_count
  - DDS\_DomainParticipantResourceLimitsQosPolicy, [537](#)
- topic\_data\_max\_length

## T

- DDS\_DomainParticipantResourceLimitsQosPolicy, 537
- topic\_name
  - DDS\_PublicationBuiltinTopicData, 591
  - DDS\_SubscriptionBuiltinTopicData, 626
- total\_count
  - DDS\_DataReaderInstanceReplacedStatus, 503
  - DDS\_LivelinessLostStatus, 561
  - DDS\_OfferedDeadlineMissedStatus, 566
  - DDS\_OfferedIncompatibleQosStatus, 568
  - DDS\_PublicationMatchedStatus, 594
  - DDS\_RequestedDeadlineMissedStatus, 604
  - DDS\_RequestedIncompatibleQosStatus, 605
  - DDS\_SampleLostStatus, 620
  - DDS\_SampleRejectedStatus, 621
  - DDS\_SubscriptionMatchedStatus, 630
- total\_count\_change
  - DDS\_DataReaderInstanceReplacedStatus, 503
  - DDS\_LivelinessLostStatus, 561
  - DDS\_OfferedDeadlineMissedStatus, 566
  - DDS\_OfferedIncompatibleQosStatus, 568
  - DDS\_PublicationMatchedStatus, 594
  - DDS\_RequestedDeadlineMissedStatus, 604
  - DDS\_RequestedIncompatibleQosStatus, 605
  - DDS\_SampleLostStatus, 620
  - DDS\_SampleRejectedStatus, 621
  - DDS\_SubscriptionMatchedStatus, 630
- transfer\_mode
  - DATA\_WRITER\_TRANSFER\_MODE, 132
- transform\_locator\_kind
  - UDP\_InterfaceFactoryProperty, 857
- transform\_udp\_mode
  - UDP\_InterfaceFactoryProperty, 856
- TRANSPORT, 125
- transport
  - DDS\_DataReaderQos, 507
  - DDS\_DataWriterQos, 517
- transports
  - DDS\_DomainParticipantQos, 531
- trigger\_flow
  - DDSFlowController, 730
- trust
  - DDS\_DomainParticipantQos, 531
- Type Support, 134
  - DDS\_InstanceId\_t, 134
  - DDS\_TypePluginI, 135
  - NDDS\_Type\_PluginVersion, 134
  - NDDS\_TYPEPLUGIN\_NO\_KEY, 135
  - NDDS\_TYPEPLUGIN\_USER\_KEY, 135
  - NDDS\_TypePluginKeyKind, 135
- type\_name
  - DDS\_PublicationBuiltinTopicData, 591
  - DDS\_SubscriptionBuiltinTopicData, 626
- UDP Transform API, 154
- UDP Transport, 241
- UDP Transport API, 151
  - add\_entry, 153
  - UDP\_INTERFACE\_INTERFACE\_MULTICAST\_FLAG, 152
  - UDP\_INTERFACE\_INTERFACE\_UP\_FLAG, 152
  - UDP\_INTERFACE\_MAX\_IFNAME, 152
  - UDP\_INTERFACE\_MAX\_NETMASK\_BITS, 152
  - UDP\_InterfaceFactoryProperty\_finalize, 153
  - UDP\_InterfaceFactoryProperty\_initialize, 152
- UDP\_INTERFACE\_INTERFACE\_MULTICAST\_FLAG
  - UDP Transport API, 152
- UDP\_INTERFACE\_INTERFACE\_UP\_FLAG
  - UDP Transport API, 152
- UDP\_INTERFACE\_MAX\_IFNAME
  - UDP Transport API, 152
- UDP\_INTERFACE\_MAX\_NETMASK\_BITS
  - UDP Transport API, 152
- UDP\_InterfaceFactoryProperty, 852
  - allow\_interface, 853
  - deny\_interface, 853
  - destination\_rules, 856
  - disable\_auto\_interface\_config, 855
  - disable\_multicast\_bind, 856
  - disable\_multicast\_interface\_select, 856
  - enable\_interface\_bind, 855
  - if\_table, 854
  - is\_default\_interface, 855
  - max\_message\_size, 854
  - max\_receive\_buffer\_size, 854
  - max\_send\_buffer\_size, 854
  - max\_send\_message\_size, 854
  - multicast\_interface, 855
  - multicast\_loopback\_disabled, 855
  - multicast\_ttl, 854
  - nat, 854
  - recv\_thread, 855
  - source\_rules, 856
  - transform\_locator\_kind, 857
  - transform\_udp\_mode, 856
- UDP\_InterfaceFactoryProperty\_finalize
  - UDP Transport API, 153
- UDP\_InterfaceFactoryProperty\_initialize
  - UDP Transport API, 152
- UDP\_InterfaceTableEntry, 857
  - address, 857
  - flags, 857
  - ifname, 858
  - netmask, 858
- UDP\_InterfaceTableEntrySeq, 858
- UDP\_LOG\_ALLOC\_EC
  - NETIO, 296
- UDP\_LOG\_BASE



- osapi\_log.h, [930](#)
- UDP\_LOG\_CURSOR\_ERROR\_EC
  - NETIO, [297](#)
- UDP\_LOG\_DROPGRP\_EC
  - NETIO, [296](#)
- UDP\_LOG\_FINALIZE\_EC
  - NETIO, [293](#)
- UDP\_LOG\_GET\_BUF\_ALLOC\_EC
  - NETIO, [295](#)
- UDP\_LOG\_GET\_BUF\_SIZE\_EC
  - NETIO, [295](#)
- UDP\_LOG\_GET\_IFLIST\_EC
  - NETIO, [295](#)
- UDP\_LOG\_GET\_LENGTH\_EC
  - NETIO, [296](#)
- UDP\_LOG\_GET\_NAMEINFO\_EC
  - NETIO, [295](#)
- UDP\_LOG\_GETHOSTNAME\_EC
  - NETIO, [291](#)
- UDP\_LOG\_INCONSISTENT\_MAX\_MESSAGE\_SIZE\_EC
  - NETIO, [297](#)
- UDP\_LOG\_INITIALIZE\_FAILED\_EC
  - NETIO, [297](#)
- UDP\_LOG\_LOADLIB\_EC
  - NETIO, [294](#)
- UDP\_LOG\_MULTICAST\_ENABLE\_EC
  - NETIO, [295](#)
- UDP\_LOG\_MULTICAST\_NOT\_ENABLED\_EC
  - NETIO, [298](#)
- UDP\_LOG\_NAT\_DELETE\_EC
  - NETIO, [293](#)
- UDP\_LOG\_PACKET\_FWD\_EC
  - NETIO, [292](#)
- UDP\_LOG\_PACKET\_HEAD\_EC
  - NETIO, [292](#)
- UDP\_LOG\_PACKET\_INIT\_EC
  - NETIO, [292](#)
- UDP\_LOG\_PORT\_ADD\_EC
  - NETIO, [294](#)
- UDP\_LOG\_PORT\_ALLOC\_EC
  - NETIO, [293](#)
- UDP\_LOG\_PORT\_ENTRY\_EC
  - NETIO, [293](#)
- UDP\_LOG\_PORT\_NOT\_FOUND\_EC
  - NETIO, [297](#)
- UDP\_LOG\_PORT\_THREAD\_EC
  - NETIO, [294](#)
- UDP\_LOG\_PROPERTY\_FINALIZE\_EC
  - NETIO, [296](#)
- UDP\_LOG\_RECORD\_EC
  - NETIO, [296](#)
- UDP\_LOG\_RECV\_ERROR\_EC
  - NETIO, [298](#)
- UDP\_LOG\_SEND\_ERROR\_EC
  - NETIO, [297](#)
- UDP\_LOG\_SEND\_PING\_EC
  - NETIO, [296](#)
- UDP\_LOG\_SET\_LENGTH\_EC
  - NETIO, [296](#)
- UDP\_LOG\_SOCKET\_ADDGRP\_EC
  - NETIO, [293](#)
- UDP\_LOG\_SOCKET\_BIND\_EC
  - NETIO, [293](#)
- UDP\_LOG\_SOCKET\_CLOSE\_EC
  - NETIO, [294](#)
- UDP\_LOG\_SOCKET\_CREATE\_EC
  - NETIO, [292](#)
- UDP\_LOG\_SOCKET\_IFFLAGS\_EC
  - NETIO, [294](#)
- UDP\_LOG\_SOCKET\_IFLIST\_EC
  - NETIO, [294](#)
- UDP\_LOG\_SOCKET\_INIT\_EC
  - NETIO, [291](#)
- UDP\_LOG\_SOCKET\_REUSE\_PORT\_EC
  - NETIO, [293](#)
- UDP\_LOG\_SOCKET\_RXBUF\_EC
  - NETIO, [294](#)
- UDP\_LOG\_SOCKET\_SET\_MCAST\_EC
  - NETIO, [291](#)
- UDP\_LOG\_SOCKET\_SET\_MCASTIF\_EC
  - NETIO, [291](#)
- UDP\_LOG\_SOCKET\_SETFD\_EC
  - NETIO, [292](#)
- UDP\_LOG\_SOCKET\_SNDBUF\_EC
  - NETIO, [292](#)
- UDP\_LOG\_SOCKET\_TTL\_EC
  - NETIO, [292](#)
- UDP\_LOG\_TRANSPORT\_NO\_INTERFACES\_EC
  - NETIO, [298](#)
- UDP\_LOG\_UDP\_CREATE\_INDEX\_EC
  - NETIO, [295](#)
- UDP\_LOG\_UDP\_CREATE\_TABLE\_EC
  - NETIO, [295](#)
- UDP\_LOG\_WINSOCK\_INCOMPATIBLE\_EC
  - NETIO, [297](#)
- UDP\_LOG\_WSA\_STARTUP\_EC
  - NETIO, [297](#)
- UDP\_NatEntry, [858](#)
- UDP\_NatEntrySeq, [859](#)
- UDP\_TRANSFORM\_LOG\_ADD\_EC
  - NETIO, [299](#)
- UDP\_TRANSFORM\_LOG\_ALLOC\_EC
  - NETIO, [298](#)
- UDP\_TRANSFORM\_LOG\_CREATE\_EC
  - NETIO, [299](#)
- UDP\_TRANSFORM\_LOG\_DUPLICATE\_EC
  - NETIO, [299](#)
- UDP\_TRANSFORM\_LOG\_FACTORY\_NOT\_FOUND\_EC

- NETIO, [298](#)
- UDP\_TRANSFORM\_LOG\_FIND\_EC
  - NETIO, [299](#)
- UDP\_TRANSFORM\_LOG\_INVALID\_FACTORY\_REFERENCE\_EC
  - NETIO, [298](#)
- UDP\_TRANSFORM\_LOG\_INVALID\_TUDP\_LOCATOR\_KIND\_EC
  - NETIO, [299](#)
- UDP\_TRANSFORM\_LOG\_INVALID\_UDP\_MODE\_EC
  - NETIO, [299](#)
- UDP\_TRANSFORM\_LOG\_KIND\_DST\_INDEX
  - netio\_log.h, [917](#)
- UDP\_TRANSFORM\_LOG\_KIND\_ENTRY
  - netio\_log.h, [916](#)
- UDP\_TRANSFORM\_LOG\_KIND\_FACTORY
  - netio\_log.h, [916](#)
- UDP\_TRANSFORM\_LOG\_KIND\_FACTORY\_INDEX
  - netio\_log.h, [917](#)
- UDP\_TRANSFORM\_LOG\_KIND\_RULE
  - netio\_log.h, [916](#)
- UDP\_TRANSFORM\_LOG\_KIND\_SRC\_INDEX
  - netio\_log.h, [917](#)
- UDP\_TRANSFORM\_LOG\_KIND\_TRANSFORM\_DESTINATION
  - netio\_log.h, [917](#)
- UDP\_TRANSFORM\_LOG\_KIND\_TRANSFORM\_NETMASK\_INDEX
  - netio\_log.h, [917](#)
- UDP\_TRANSFORM\_LOG\_KIND\_TRANSFORM\_SOURCE
  - netio\_log.h, [917](#)
- UDP\_TRANSFORM\_LOG\_TRANSFORM\_EC
  - NETIO, [299](#)
- UDPInterfaceFactory, [859](#)
  - get\_interface, [859](#)
- UDPInterfaceTable, [860](#)
- unacknowledged\_count
  - DDS\_ReliableSampleUnacknowledgedStatus, [603](#)
- unbind
  - ZCOPY\_NotifUserInterfaceI, [866](#)
- unicast\_locator
  - DDS\_PublicationBuiltinTopicData, [593](#)
  - DDS\_SubscriptionBuiltinTopicData, [628](#)
- unloan
  - FooSeq, [815](#)
- unregister
  - RTRegistry, [851](#)
- unregister\_instance
  - FooDataWriter, [797](#)
- unregister\_instance\_untyped
  - DDSDDataWriter, [673](#)
- unregister\_instance\_w\_timestamp
  - FooDataWriter, [799](#)
- unregister\_instance\_w\_timestamp\_untyped
  - DDSDDataWriter, [675](#)
- unregister\_type
  - DDSDomainParticipant, [703](#)
  - FooTypeSupport, [821](#)
- User Data Type Support, [54](#)
  - DDS\_DATAREADER\_CPP, [55](#)
  - DDS\_DATAWRITER\_CPP, [55](#)
  - DDS\_HANDLE\_NIL, [57](#)
  - DDS\_InstanceHandle\_equals, [56](#)
  - DDS\_InstanceHandle\_is\_nil, [56](#)
  - DDS\_InstanceHandle\_t, [56](#)
  - DDS\_TYPESUPPORT\_CPP, [55](#)
- USER\_DATA, [130](#)
- user\_data
  - DDS\_DataReaderQos, [507](#)
  - DDS\_DataWriterQos, [517](#)
  - DDS\_DomainParticipantQos, [531](#)
  - DDS\_ParticipantBuiltinTopicData, [576](#)
  - DDS\_PublicationBuiltinTopicData, [593](#)
  - DDS\_SubscriptionBuiltinTopicData, [628](#)
- user\_intf
  - ZCOPY\_NotifInterfaceFactoryProperty, [861](#)
- user\_multicast\_port\_offset
  - DDS\_RtpsWellKnownPorts\_t, [614](#)
- user\_property
  - ZCOPY\_NotifInterfaceFactoryProperty, [861](#)
- user\_ptr
  - DDS\_ErrListener, [847](#)
- USER\_TRAFFIC, [104](#)
- user\_traffic
  - DDS\_DomainParticipantQos, [531](#)
- user\_unicast\_port\_offset
  - DDS\_RtpsWellKnownPorts\_t, [614](#)
- valid\_data
  - DDS\_SampleInfo, [618](#)
- value
  - DDS\_BuiltinTopicKey\_t, [496](#)
  - DDS\_GroupDataQosPolicy, [554](#)
  - DDS\_GUID\_t, [555](#)
  - DDS\_OwnershipStrengthQosPolicy, [573](#)
  - DDS\_Property\_t, [586](#)
  - DDS\_PropertyQosPolicy, [587](#)
  - DDS\_TopicDataQosPolicy, [634](#)
  - DDS\_UserDataQosPolicy, [637](#)
  - NETIO\_Address, [825](#)
  - NETIO\_AddressUInt32, [826](#)
- vendorId
  - DDS\_VendorId, [638](#)
  - DDS\_VendorId\_t, [639](#)
- View States, [75](#)
  - DDS\_ANY\_VIEW\_STATE, [76](#)
  - DDS\_NEW\_VIEW\_STATE, [76](#)
  - DDS\_NOT\_NEW\_VIEW\_STATE, [76](#)
  - DDS\_ViewStateKind, [76](#)
  - DDS\_ViewStateMask, [75](#)
- view\_state
  - DDS\_SampleInfo, [617](#)

wait  
 DDSWaitSet, 769

WH, 455  
 WHSM\_LOG\_INVALID\_INDEX\_OBJECT\_EC, 457  
 WHSM\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC, 456  
 WHSM\_LOG\_MAX\_SAMPLES\_TOO\_SMALL\_EC, 456  
 WHSM\_LOG\_NO\_PROPERTY\_EC, 457  
 WHSM\_LOG\_OBJECT\_ALLOCATE\_EC, 456  
 WHSM\_LOG\_OBJECT\_DELETE\_EC, 456  
 WHSM\_LOG\_OBJECT\_INDEX\_EC, 456  
 WHSM\_LOG\_UNLIMITED\_HISTORY\_NOT\_SUPPORTED\_EC, 456

wh\_sm\_log.h, 951  
 WHSM\_LOG\_HISTORY\_OBJECT, 952  
 WHSM\_LOG\_HISTORYINDEX\_OBJECT, 953  
 WHSM\_LOG\_KEY\_OBJECT, 952  
 WHSM\_LOG\_KEYINDEX\_OBJECT, 953  
 WHSM\_LOG\_KEYPOOL\_OBJECT, 952  
 WHSM\_LOG\_SAMPLE\_OBJECT, 952  
 WHSM\_LOG\_SAMPLEINDEX\_OBJECT, 953  
 WHSM\_LOG\_SAMPLEPOOL\_OBJECT, 953

WHSM Writer History API, 145

WHSM\_LOG\_BASE  
 osapi\_log.h, 931

WHSM\_LOG\_HISTORY\_OBJECT  
 wh\_sm\_log.h, 952

WHSM\_LOG\_HISTORYINDEX\_OBJECT  
 wh\_sm\_log.h, 953

WHSM\_LOG\_INVALID\_INDEX\_OBJECT\_EC  
 WH, 457

WHSM\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC  
 WH, 456

WHSM\_LOG\_KEY\_OBJECT  
 wh\_sm\_log.h, 952

WHSM\_LOG\_KEYINDEX\_OBJECT  
 wh\_sm\_log.h, 953

WHSM\_LOG\_KEYPOOL\_OBJECT  
 wh\_sm\_log.h, 952

WHSM\_LOG\_MAX\_SAMPLES\_TOO\_SMALL\_EC  
 WH, 456

WHSM\_LOG\_NO\_PROPERTY\_EC  
 WH, 457

WHSM\_LOG\_OBJECT\_ALLOCATE\_EC  
 WH, 456

WHSM\_LOG\_OBJECT\_DELETE\_EC  
 WH, 456

WHSM\_LOG\_OBJECT\_INDEX\_EC  
 WH, 456

WHSM\_LOG\_SAMPLE\_OBJECT  
 wh\_sm\_log.h, 952

WHSM\_LOG\_SAMPLEINDEX\_OBJECT  
 wh\_sm\_log.h, 953

WHSM\_LOG\_SAMPLEPOOL\_OBJECT  
 wh\_sm\_log.h, 953

WHSM\_LOG\_UNLIMITED\_HISTORY\_NOT\_SUPPORTED\_EC  
 WH, 456

WHSM HistoryFactory, 860  
 get\_interface, 860

WHSQ, 448  
 WHSQ\_LOG\_COMMIT\_FAILED\_EC, 450  
 WHSQ\_LOG\_CONFIGURATION\_EC, 450  
 WHSQ\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC, 449  
 WHSQ\_LOG\_MAX\_SAMPLES\_TOO\_SMALL\_EC, 449  
 WHSQ\_LOG\_NO\_DDS\_FACTORY\_EC, 450  
 WHSQ\_LOG\_OBJECT\_ALLOCATE\_EC, 449  
 WHSQ\_LOG\_OBJECT\_DELETE\_EC, 449  
 WHSQ\_LOG\_PURGE\_FAILED\_EC, 449  
 WHSQ\_LOG\_UNLIMITED\_HISTORY\_NOT\_SUPPORTED\_EC, 449  
 WHSQ\_LOG\_WH\_CREATE\_FAILED\_EC, 449

WHSQ\_LOG\_COMMIT\_FAILED\_EC  
 WHSQ, 450

WHSQ\_LOG\_CONFIGURATION\_EC  
 WHSQ, 450

WHSQ\_LOG\_HISTORY\_OBJECT  
 netio\_zcopy\_whsq\_log.h, 923

WHSQ\_LOG\_KEEP\_ALL\_HISTORY\_NOT\_SUPPORTED\_EC  
 WHSQ, 449

WHSQ\_LOG\_MAX\_SAMPLES\_TOO\_SMALL\_EC  
 WHSQ, 449

WHSQ\_LOG\_NO\_DDS\_FACTORY\_EC  
 WHSQ, 450

WHSQ\_LOG\_OBJECT\_ALLOCATE\_EC  
 WHSQ, 449

WHSQ\_LOG\_OBJECT\_DELETE\_EC  
 WHSQ, 449

WHSQ\_LOG\_PURGE\_FAILED\_EC  
 WHSQ, 449

WHSQ\_LOG\_UNLIMITED\_HISTORY\_NOT\_SUPPORTED\_EC  
 WHSQ, 449

WHSQ\_LOG\_WH\_CREATE\_FAILED\_EC  
 WHSQ, 449

WIRE\_PROTOCOL, 121  
 DDS\_CHECKSUM\_AUTO, 122  
 DDS\_CHECKSUM\_BUILTIN128, 122  
 DDS\_CHECKSUM\_BUILTIN32, 122  
 DDS\_CHECKSUM\_BUILTIN64, 122  
 DDS\_CHECKSUM\_NONE, 122  
 DDS\_ChecksumKind\_t, 123  
 DDS\_ChecksumKindMask\_t, 123  
 DDS\_INTEROPERABLE\_RTPS\_WELL\_KNOWN\_PORTS, 124  
 DDS\_RTI\_BACKWARDS\_COMPATIBLE\_RTPS\_WELL\_KNOWN\_PORTS, 124

- write
  - FooDataWriter, [794](#)
- write\_buffer
  - OSAPI\_LogProperty, [839](#)
- write\_untyped
  - DDSDataWriter, [669](#)
- write\_w\_timestamp
  - FooDataWriter, [801](#)
- write\_w\_timestamp\_untyped
  - DDSDataWriter, [678](#)
- writer\_guid
  - DDS\_SampleIdentity\_t, [615](#)
- writer\_loaned\_sample\_allocation
  - DDS\_DataWriterResourceLimitsQosPolicy, [519](#)
- writer\_resource\_limits
  - DDS\_DataWriterQos, [517](#)
- writer\_user\_data\_max\_count
  - DDS\_DomainParticipantResourceLimitsQosPolicy, [538](#)
- writer\_user\_data\_max\_length
  - DDS\_DomainParticipantResourceLimitsQosPolicy, [538](#)
- XCDR\_LOG\_BASE
  - osapi\_log.h, [931](#)
- ZCOPY\_NotifInterface\_receive
  - netio\_zcopy\_notif\_interface.h, [921](#)
- ZCOPY\_NotifInterfaceFactory\_register
  - netio\_zcopy\_notif\_interface.h, [921](#)
- ZCOPY\_NotifInterfaceFactory\_unregister
  - netio\_zcopy\_notif\_interface.h, [922](#)
- ZCOPY\_NotifInterfaceFactoryProperty, [861](#)
  - max\_samples\_per\_notif, [861](#)
  - user\_intf, [861](#)
  - user\_property, [861](#)
- ZCOPY\_NotifInterfaceFactoryProperty\_INITIALIZER
  - Zero Copy v2 API, [186](#)
- ZCOPY\_NotifMechanismProperty, [862](#)
  - intf\_addr, [862](#)
  - max\_receive\_ports, [862](#)
  - max\_routes, [862](#)
  - thread\_prop, [862](#)
- ZCOPY\_NotifUserInterface\_add\_routeFunc
  - Zero Copy v2 API, [187](#)
- ZCOPY\_NotifUserInterface\_bindFunc
  - Zero Copy v2 API, [187](#)
- ZCOPY\_NotifUserInterface\_createFunc
  - Zero Copy v2 API, [186](#)
- ZCOPY\_NotifUserInterface\_delete\_routeFunc
  - Zero Copy v2 API, [187](#)
- ZCOPY\_NotifUserInterface\_deleteFunc
  - Zero Copy v2 API, [186](#)
- ZCOPY\_NotifUserInterface\_notify\_portFunc
  - Zero Copy v2 API, [188](#)
- ZCOPY\_NotifUserInterface\_release\_addressFunc
  - Zero Copy v2 API, [187](#)
- ZCOPY\_NotifUserInterface\_reserve\_addressFunc
  - Zero Copy v2 API, [186](#)
- ZCOPY\_NotifUserInterface\_sendFunc
  - Zero Copy v2 API, [187](#)
- ZCOPY\_NotifUserInterface\_unbindFunc
  - Zero Copy v2 API, [188](#)
- ZCOPY\_NotifUserInterfaceI, [863](#)
  - add\_route, [865](#)
  - bind, [866](#)
  - create\_instance, [863](#)
  - delete\_instance, [864](#)
  - delete\_route, [866](#)
  - get\_route\_table, [864](#)
  - notify\_rcv\_port, [867](#)
  - release\_address, [865](#)
  - reserve\_address, [865](#)
  - resolve\_address, [864](#)
  - send, [867](#)
  - unbind, [866](#)
  - Zero Copy v2 API, [188](#)
- Zero Copy Transfer Over Shared Memory, [53](#)
- Zero Copy v2 API, [185](#)
  - ZCOPY\_NotifInterfaceFactoryProperty\_INITIALIZER, [186](#)
  - ZCOPY\_NotifUserInterface\_add\_routeFunc, [187](#)
  - ZCOPY\_NotifUserInterface\_bindFunc, [187](#)
  - ZCOPY\_NotifUserInterface\_createFunc, [186](#)
  - ZCOPY\_NotifUserInterface\_delete\_routeFunc, [187](#)
  - ZCOPY\_NotifUserInterface\_deleteFunc, [186](#)
  - ZCOPY\_NotifUserInterface\_notify\_portFunc, [188](#)
  - ZCOPY\_NotifUserInterface\_release\_addressFunc, [187](#)
  - ZCOPY\_NotifUserInterface\_reserve\_addressFunc, [186](#)
  - ZCOPY\_NotifUserInterface\_sendFunc, [187](#)
  - ZCOPY\_NotifUserInterface\_unbindFunc, [188](#)
  - ZCOPY\_NotifUserInterfaceI, [188](#)